# Inhaltsverzeichnis


.eslintrc.js....18
apps/storage/Dockerfile....18
apps/storage/package.json....18
apps/storage/src/index.js....19
apps/web/.eslintrc.js....20
apps/web/b.js....20
apps/web/done.json....21
apps/web/next-env.d.ts....21
apps/web/next.config.js....22
apps/web/nodemon.json....22
apps/web/package.json....22
apps/web/public/prism.css....25
apps/web/README.md....25
apps/web/rollup.config.mjs....26
apps/web/scripts/collect-info.js....26
apps/web/src/global/getS3.ts....28
apps/web/src/global/i18n/onlineTranslateV3.ts....28
apps/web/src/global/md/defaultProps.ts....29
apps/web/src/global/PopupManager/AlertPopupComponent.tsx....31
apps/web/src/global/PopupManager/ConfirmPopupComponent.tsx....33
apps/web/src/global/PopupManager/PromptPopupComponent.tsx....34
apps/web/src/global/PopupManager/SelectPopupComponent.tsx....36
apps/web/src/global/PopupManager.tsx....38
apps/web/src/global/rce/language/LanguageTemplates.ts....43
apps/web/src/global/server/isLoggedIn.ts....43
apps/web/src/global/server/rce.ts....44
apps/web/src/global/theme.ts....46
apps/web/src/pages/404.tsx....49
apps/web/src/pages/500.tsx....49
apps/web/src/pages/account/reset/[token].tsx....50
apps/web/src/pages/account/settings.tsx....53
apps/web/src/pages/admin/analytics/index.tsx....61
apps/web/src/pages/admin/analytics/[id].tsx....71
apps/web/src/pages/admin/blog.tsx....80
apps/web/src/pages/admin/certificates.tsx....86
apps/web/src/pages/admin/challenges.tsx....89
apps/web/src/pages/admin/contacts.tsx....94
apps/web/src/pages/admin/contents/courses/index.tsx....97
apps/web/src/pages/admin/contents/courses/new.tsx....104
apps/web/src/pages/admin/contents/courses/[id].tsx....108

apps/web/src/pages/admin/contents/new.tsx ..... 114
apps/web/src/pages/admin/discovery.tsx ..... 120
apps/web/src/pages/admin/editor.tsx ..... 122
apps/web/src/pages/admin/errors.tsx ..... 124
apps/web/src/pages/admin/flows.tsx ..... 128
apps/web/src/pages/admin/ide.tsx ..... 130
apps/web/src/pages/admin/index.tsx ..... 133
apps/web/src/pages/admin/kids.tsx ..... 140
apps/web/src/pages/admin/newsletter.tsx ..... 144
apps/web/src/pages/admin/orgs/index.tsx ..... 146
apps/web/src/pages/admin/profiles/index.tsx ..... 148
apps/web/src/pages/admin/project-ideas/index.tsx ..... 151
apps/web/src/pages/admin/roads.tsx ..... 159
apps/web/src/pages/admin/shorts.tsx ..... 163
apps/web/src/pages/admin/snippets.tsx ..... 166
apps/web/src/pages/admin/testimonials.tsx ..... 171
apps/web/src/pages/admin/todos.tsx ..... 181
apps/web/src/pages/admin/users/index.tsx ..... 184
apps/web/src/pages/api/account/action.ts ..... 190
apps/web/src/pages/api/account/check-reset-token/[token].ts ..... 191
apps/web/src/pages/api/account/check-username/[name].ts ..... 192
apps/web/src/pages/api/account/delete.tsx ..... 193
apps/web/src/pages/api/account/login.tsx ..... 195
apps/web/src/pages/api/account/register.tsx ..... 197
apps/web/src/pages/api/account/reset-confirm.ts ..... 199
apps/web/src/pages/api/account/reset.tsx ..... 200
apps/web/src/pages/api/account/try.ts ..... 202
apps/web/src/pages/api/account/update-2fa.ts ..... 203
apps/web/src/pages/api/account/update.tsx ..... 205
apps/web/src/pages/api/account/verify.ts ..... 208
apps/web/src/pages/api/account/[username]/picture.ts ..... 210
apps/web/src/pages/api/admin/account/create.ts ..... 212
apps/web/src/pages/api/admin/account/delete.ts ..... 214
apps/web/src/pages/api/admin/account/jwt.ts ..... 215
apps/web/src/pages/api/admin/account/list.ts ..... 217
apps/web/src/pages/api/admin/account/update.ts ..... 218
apps/web/src/pages/api/admin/analytics/delete.ts ..... 220
apps/web/src/pages/api/admin/analytics/summary.ts ..... 221
apps/web/src/pages/api/admin/analytics/user-summary.ts ..... 225
apps/web/src/pages/api/admin/analytics/views.ts ..... 227
apps/web/src/pages/api/admin/blog/index.ts ..... 229
apps/web/src/pages/api/admin/certificates/delete.ts ..... 232

apps/web/src/pages/api/admin/certificates/list.ts ..... 233
apps/web/src/pages/api/admin/challenges/index.ts ..... 234
apps/web/src/pages/api/admin/contact/messages.ts ..... 236
apps/web/src/pages/api/admin/contact/user.tsx ..... 238
apps/web/src/pages/api/admin/contents/index.ts ..... 240
apps/web/src/pages/api/admin/course/create.ts ..... 244
apps/web/src/pages/api/admin/course/delete.ts ..... 245
apps/web/src/pages/api/admin/course/edit.ts ..... 247
apps/web/src/pages/api/admin/course/list.ts ..... 249
apps/web/src/pages/api/admin/discovery/delete.ts ..... 250
apps/web/src/pages/api/admin/editor/index.ts ..... 251
apps/web/src/pages/api/admin/errors.ts ..... 253
apps/web/src/pages/api/admin/flows/index.ts ..... 254
apps/web/src/pages/api/admin/forum_profiles/delete.ts ..... 255
apps/web/src/pages/api/admin/forum_profiles/list.ts ..... 257
apps/web/src/pages/api/admin/forum_profiles/update.ts ..... 258
apps/web/src/pages/api/admin/ide/index.ts ..... 260
apps/web/src/pages/api/admin/kids/index.ts ..... 261
apps/web/src/pages/api/admin/newsletter/index.tsx ..... 262
apps/web/src/pages/api/admin/newsletter/members.ts ..... 264
apps/web/src/pages/api/admin/orgs/index.ts ..... 265
apps/web/src/pages/api/admin/overview.ts ..... 267
apps/web/src/pages/api/admin/project-ideas/index.ts ..... 269
apps/web/src/pages/api/admin/roads/index.ts ..... 271
apps/web/src/pages/api/admin/shorturls/index.ts ..... 272
apps/web/src/pages/api/admin/snippets/index.ts ..... 274
apps/web/src/pages/api/admin/testimonials/index.ts ..... 276
apps/web/src/pages/api/admin/text/create.ts ..... 278
apps/web/src/pages/api/admin/text/list.ts ..... 279
apps/web/src/pages/api/admin/text/[tid].ts ..... 281
apps/web/src/pages/api/admin/todos/index.ts ..... 283
apps/web/src/pages/api/admin/video/create.ts ..... 284
apps/web/src/pages/api/admin/video/list.ts ..... 286
apps/web/src/pages/api/admin/video/youtube/[vid].ts ..... 287
apps/web/src/pages/api/analytics/index.ts ..... 289
apps/web/src/pages/api/ask/posts/index.ts ..... 291
apps/web/src/pages/api/ask/profiles/[username].ts ..... 296
apps/web/src/pages/api/blog/index.ts ..... 299
apps/web/src/pages/api/blog/[id].ts ..... 300
apps/web/src/pages/api/cert.ts ..... 300
apps/web/src/pages/api/certificates/my.ts ..... 302
apps/web/src/pages/api/challenges/check.ts ..... 303

apps/web/src/pages/api/challenges/completed.ts ..... 305
apps/web/src/pages/api/challenges/index.ts ..... 305
apps/web/src/pages/api/community/feed.ts ..... 306
apps/web/src/pages/api/community/has-profile/[id].ts ..... 308
apps/web/src/pages/api/community/post/create.ts ..... 309
apps/web/src/pages/api/community/post/[id]/delete.ts ..... 311
apps/web/src/pages/api/community/post/[id]/like.ts ..... 313
apps/web/src/pages/api/community/post/[id].ts ..... 314
apps/web/src/pages/api/community/profile/create.ts ..... 315
apps/web/src/pages/api/community/profile/update.ts ..... 317
apps/web/src/pages/api/community/profile/[name]/avatar.ts ..... 318
apps/web/src/pages/api/community/profile/[name]/followers.ts ..... 319
apps/web/src/pages/api/community/profile/[name]/is-following/[othername].ts ..... 320
apps/web/src/pages/api/community/profile/[name]/posts.ts ..... 321
apps/web/src/pages/api/community/profile/[name]/toggle-follow.ts ..... 323
apps/web/src/pages/api/community/profile/[name].ts ..... 324
apps/web/src/pages/api/community/search.ts ..... 325
apps/web/src/pages/api/contact.ts ..... 326
apps/web/src/pages/api/course/enrollments.ts ..... 328
apps/web/src/pages/api/course/list-courses.ts ..... 329
apps/web/src/pages/api/course/[course]/certificate.ts ..... 331
apps/web/src/pages/api/course/[course]/enrollment.ts ..... 334
apps/web/src/pages/api/course/[course]/index.ts ..... 338
apps/web/src/pages/api/course/[course]/progress.ts ..... 340
apps/web/src/pages/api/course/[course]/[section].ts ..... 342
apps/web/src/pages/api/creator/certificate.ts ..... 345
apps/web/src/pages/api/creator/content.ts ..... 348
apps/web/src/pages/api/creator/course.ts ..... 353
apps/web/src/pages/api/creator/top.ts ..... 357
apps/web/src/pages/api/discovery/comment.ts ..... 357
apps/web/src/pages/api/discovery/index.ts ..... 359
apps/web/src/pages/api/discovery/star.ts ..... 360
apps/web/src/pages/api/editor/projects.ts ..... 361
apps/web/src/pages/api/editor/published-domain.ts ..... 362
apps/web/src/pages/api/forum/chats/index.ts ..... 363
apps/web/src/pages/api/forum/chats/[id]/messages.ts ..... 366
apps/web/src/pages/api/forum/embeds.ts ..... 368
apps/web/src/pages/api/forum/post/all.ts ..... 369
apps/web/src/pages/api/forum/post/index.ts ..... 370
apps/web/src/pages/api/forum/post/[id]/comments.ts ..... 373
apps/web/src/pages/api/forum/post/[id]/delete.ts ..... 374
apps/web/src/pages/api/forum/post/[id]/index.ts ..... 376

apps/web/src/pages/api/forum/post/[id]/like.ts ..... 376
apps/web/src/pages/api/forum/profiles.ts ..... 378
apps/web/src/pages/api/forum/[username]/posts.ts ..... 379
apps/web/src/pages/api/forum/[username]/profile.ts ..... 380
apps/web/src/pages/api/ide/projects/contributors/add.ts ..... 383
apps/web/src/pages/api/ide/projects/contributors/list.ts ..... 385
apps/web/src/pages/api/ide/projects/contributors/permissions.ts ..... 388
apps/web/src/pages/api/ide/projects/contributors/remove.ts ..... 390
apps/web/src/pages/api/ide/projects/create.ts ..... 392
apps/web/src/pages/api/ide/projects/delete.ts ..... 393
apps/web/src/pages/api/ide/projects/duplicate.ts ..... 395
apps/web/src/pages/api/ide/projects/list.ts ..... 396
apps/web/src/pages/api/ide/projects/update/name.ts ..... 399
apps/web/src/pages/api/ide/projects/[id]/discovery.ts ..... 400
apps/web/src/pages/api/ide/projects/[id]/file/delete.ts ..... 402
apps/web/src/pages/api/ide/projects/[id]/file/save.ts ..... 404
apps/web/src/pages/api/ide/projects/[id]/index.ts ..... 406
apps/web/src/pages/api/ide/projects/[id]/permissions.ts ..... 407
apps/web/src/pages/api/ide/projects/[id]/preview/[[...path]].ts ..... 409
apps/web/src/pages/api/ide/projects/[id]/publish.ts ..... 412
apps/web/src/pages/api/ide/projects/[id]/template/has.ts ..... 414
apps/web/src/pages/api/ide/projects/[id]/template/publish.ts ..... 416
apps/web/src/pages/api/ide/projects/[id]/template/remove.ts ..... 418
apps/web/src/pages/api/ide/projects/[id]/template/update.ts ..... 419
apps/web/src/pages/api/ide/projects/[id]/toggle-pin.ts ..... 421
apps/web/src/pages/api/ide/templates/list.ts ..... 423
apps/web/src/pages/api/ide/templates/[id]/index.ts ..... 423
apps/web/src/pages/api/ide/templates/[id]/rating.ts ..... 424
apps/web/src/pages/api/ide/templates/[id]/use.ts ..... 425
apps/web/src/pages/api/kids/projects.ts ..... 427
apps/web/src/pages/api/newsletter/subscribe.ts ..... 430
apps/web/src/pages/api/newsletter/subscribed.ts ..... 432
apps/web/src/pages/api/newsletter/unsubscribe.ts ..... 433
apps/web/src/pages/api/orgs/create.ts ..... 434
apps/web/src/pages/api/orgs/delete.ts ..... 436
apps/web/src/pages/api/orgs/my.ts ..... 437
apps/web/src/pages/api/orgs/[name]/blog.ts ..... 439
apps/web/src/pages/api/orgs/[name]/index.ts ..... 440
apps/web/src/pages/api/orgs/[name]/members.ts ..... 442
apps/web/src/pages/api/orgs/[name]/templates.ts ..... 445
apps/web/src/pages/api/orgs/[name]/update.ts ..... 449
apps/web/src/pages/api/orgs/[name]/use-template.ts ..... 451

apps/web/src/pages/api/payments/cancelSubscription.ts ..... 454
apps/web/src/pages/api/payments/createCheckoutSession.ts ..... 455
apps/web/src/pages/api/payments/subscription_callback.ts ..... 457
apps/web/src/pages/api/project-ideas.ts ..... 459
apps/web/src/pages/api/projects/create.ts ..... 459
apps/web/src/pages/api/projects/my.ts ..... 460
apps/web/src/pages/api/projects/[id]/download.ts ..... 461
apps/web/src/pages/api/projects/[id]/files/[[...path]].ts ..... 463
apps/web/src/pages/api/projects/[id]/flows.ts ..... 466
apps/web/src/pages/api/projects/[id]/gitops/branch.ts ..... 468
apps/web/src/pages/api/projects/[id]/gitops/commit.ts ..... 470
apps/web/src/pages/api/projects/[id]/gitops/commits.ts ..... 471
apps/web/src/pages/api/projects/[id]/gitops/tree.ts ..... 473
apps/web/src/pages/api/projects/[id]/i18n/getTranslation.ts ..... 476
apps/web/src/pages/api/projects/[id]/i18n/index.ts ..... 477
apps/web/src/pages/api/projects/[id]/index.ts ..... 481
apps/web/src/pages/api/projects/[id]/issues/index.ts ..... 482
apps/web/src/pages/api/projects/[id]/static/files.ts ..... 485
apps/web/src/pages/api/projects/[id]/static/upload.ts ..... 487
apps/web/src/pages/api/projects/[id]/uml/index.ts ..... 488
apps/web/src/pages/api/projects/[id]/wiki/pages.ts ..... 490
apps/web/src/pages/api/rce/go.ts ..... 493
apps/web/src/pages/api/rce/java.ts ..... 494
apps/web/src/pages/api/rce/javascript.ts ..... 495
apps/web/src/pages/api/rce/python.ts ..... 495
apps/web/src/pages/api/rce/typescript.ts ..... 496
apps/web/src/pages/api/roadmap/create.ts ..... 496
apps/web/src/pages/api/roadmap/my.ts ..... 498
apps/web/src/pages/api/roadmap/[id]/delete.ts ..... 499
apps/web/src/pages/api/roadmap/[id]/index.ts ..... 501
apps/web/src/pages/api/roadmap/[id]/update.ts ..... 502
apps/web/src/pages/api/s3/[project_id]/[name].ts ..... 505
apps/web/src/pages/api/snippets/index.ts ..... 506
apps/web/src/pages/api/sso/discord.ts ..... 509
apps/web/src/pages/api/sso/github.ts ..... 512
apps/web/src/pages/api/sso/gitlab.ts ..... 514
apps/web/src/pages/api/sso/request.ts ..... 516
apps/web/src/pages/api/stats.ts ..... 517
apps/web/src/pages/api/testimonials/index.ts ..... 518
apps/web/src/pages/api/todo/lists/create.ts ..... 519
apps/web/src/pages/api/todo/lists/delete.ts ..... 520
apps/web/src/pages/api/todo/lists/[id]/tasks/create.ts ..... 522

apps/web/src/pages/api/todo/lists/[id]/tasks/delete.ts ..... 524
apps/web/src/pages/api/todo/lists/[id]/tasks/toggle-finish.ts ..... 525
apps/web/src/pages/api/todo/lists/[id]/tasks/update.ts ..... 527
apps/web/src/pages/api/todo/lists/[id].ts ..... 529
apps/web/src/pages/api/todo/my-lists.ts ..... 530
apps/web/src/pages/api/todo/projects/index.ts ..... 531
apps/web/src/pages/api/user/[name].ts ..... 534
apps/web/src/pages/api/users/[id].ts ..... 535
apps/web/src/pages/ask/index.tsx ..... 536
apps/web/src/pages/ask/t/[id].tsx ..... 538
apps/web/src/pages/ask/u/[username].tsx ..... 540
apps/web/src/pages/blog/index.tsx ..... 544
apps/web/src/pages/blog/[id].tsx ..... 546
apps/web/src/pages/certificates/index.tsx ..... 551
apps/web/src/pages/certificates/[id].tsx ..... 555
apps/web/src/pages/challenges/index.tsx ..... 562
apps/web/src/pages/challenges/[id].tsx ..... 565
apps/web/src/pages/codeup/about.tsx ..... 569
apps/web/src/pages/codeup/contact.tsx ..... 579
apps/web/src/pages/community/index.tsx ..... 582
apps/web/src/pages/community/post/[id].tsx ..... 585
apps/web/src/pages/community/profile/create.tsx ..... 586
apps/web/src/pages/community/profile/[name].tsx ..... 589
apps/web/src/pages/community/search.tsx ..... 600
apps/web/src/pages/course/index.tsx ..... 603
apps/web/src/pages/course/[course]/index.tsx ..... 607
apps/web/src/pages/course/[course]/[section].tsx ..... 612
apps/web/src/pages/courses/index.tsx ..... 616
apps/web/src/pages/creator/index.tsx ..... 619
apps/web/src/pages/creator/old-landing.tsx ..... 622
apps/web/src/pages/creator/studio/certificate/[courseid].tsx ..... 625
apps/web/src/pages/creator/studio/create/content/[course].tsx ..... 646
apps/web/src/pages/creator/studio/create/course.tsx ..... 653
apps/web/src/pages/creator/studio/edit/[contentId]/info.tsx ..... 657
apps/web/src/pages/creator/studio/edit/[contentId].tsx ..... 661
apps/web/src/pages/creator/studio/index.tsx ..... 664
apps/web/src/pages/creator/studio/[id].tsx ..... 669
apps/web/src/pages/dashboard/index.tsx ..... 676
apps/web/src/pages/discovery/index.tsx ..... 681
apps/web/src/pages/editor/index.tsx ..... 684
apps/web/src/pages/editor/[project_id]/contributors.tsx ..... 693
apps/web/src/pages/editor/[project_id]/deployment.tsx ..... 694

apps/web/src/pages/editor/[project_id]/discovery.tsx ..... 698
apps/web/src/pages/editor/[project_id]/export.tsx ..... 703
apps/web/src/pages/editor/[project_id]/index.tsx ..... 708
apps/web/src/pages/editor/[project_id]/pages/[page_id].tsx ..... 713
apps/web/src/pages/editor/[project_id]/serverless/index.tsx ..... 718
apps/web/src/pages/editor/[project_id]/serverless/[page_id].tsx ..... 722
apps/web/src/pages/editor/[project_id]/staticassets.tsx ..... 726
apps/web/src/pages/error.tsx ..... 731
apps/web/src/pages/forum/index.tsx ..... 731
apps/web/src/pages/forum/messages/index.tsx ..... 736
apps/web/src/pages/forum/messages/[id].tsx ..... 741
apps/web/src/pages/forum/post/[id].tsx ..... 741
apps/web/src/pages/forum/[username].tsx ..... 744
apps/web/src/pages/ide/index.tsx ..... 759
apps/web/src/pages/ide/project/[id].tsx ..... 761
apps/web/src/pages/ideas/index.tsx ..... 777
apps/web/src/pages/index.tsx ..... 781
apps/web/src/pages/kids/index.tsx ..... 785
apps/web/src/pages/kids/playground.tsx ..... 791
apps/web/src/pages/kids/[id].tsx ..... 793
apps/web/src/pages/legal/imprint.tsx ..... 796
apps/web/src/pages/legal/privacy.tsx ..... 798
apps/web/src/pages/old-landing-v2.tsx ..... 811
apps/web/src/pages/old-landing-v3.tsx ..... 816
apps/web/src/pages/old-landing.tsx ..... 819
apps/web/src/pages/orgs/index.tsx ..... 822
apps/web/src/pages/orgs/[name]/dashboard.tsx ..... 828
apps/web/src/pages/orgs/[name]/manage.tsx ..... 838
apps/web/src/pages/payment/success/thank_you.tsx ..... 846
apps/web/src/pages/planning/index.tsx ..... 846
apps/web/src/pages/planning/[id]/index.tsx ..... 851
apps/web/src/pages/pricing.tsx ..... 859
apps/web/src/pages/projects/index.tsx ..... 867
apps/web/src/pages/projects/[id]/editor/professional.tsx ..... 871
apps/web/src/pages/projects/[id]/flows/[flow_id].tsx ..... 876
apps/web/src/pages/projects/[id]/flows.tsx ..... 885
apps/web/src/pages/projects/[id]/i18n.tsx ..... 889
apps/web/src/pages/projects/[id]/index.tsx ..... 897
apps/web/src/pages/projects/[id]/issues.tsx ..... 897
apps/web/src/pages/projects/[id]/planning/overview.tsx ..... 905
apps/web/src/pages/projects/[id]/static.tsx ..... 908
apps/web/src/pages/projects/[id]/uml/[uml_id].tsx ..... 911

apps/web/src/pages/projects/[id]/uml.tsx ..... 914
apps/web/src/pages/projects/[id]/wiki/[pageId].tsx ..... 919
apps/web/src/pages/projects/[id]/wiki.tsx ..... 922
apps/web/src/pages/redirect.tsx ..... 925
apps/web/src/pages/roadmap/index.tsx ..... 926
apps/web/src/pages/roadmap/[id].tsx ..... 930
apps/web/src/pages/showcase.tsx ..... 936
apps/web/src/pages/snippets/my.tsx ..... 939
apps/web/src/pages/sso/callback.tsx ..... 947
apps/web/src/pages/sso/error.tsx ..... 947
apps/web/src/pages/test-analytics.tsx ..... 948
apps/web/src/pages/test-datatable.tsx ..... 948
apps/web/src/pages/test-grid-layout.tsx ..... 951
apps/web/src/pages/test-gridlayout-v2.tsx ..... 952
apps/web/src/pages/test-img.tsx ..... 954
apps/web/src/pages/test-player.tsx ..... 955
apps/web/src/pages/todo/index.tsx ..... 955
apps/web/src/pages/try.tsx ..... 960
apps/web/src/pages/user-cert/[id].tsx ..... 961
apps/web/src/pages/_app.tsx ..... 965
apps/web/src/pages/_document.tsx ..... 967
apps/web/src/server/index.ts ..... 968
apps/web/src/server/socket/channels/Authenticate.ts ..... 983
apps/web/src/server/socket/channels/Channel.ts ..... 984
apps/web/src/server/socket/channels/EditorProjectChangeSetting.ts ..... 984
apps/web/src/server/socket/channels/EditorProjectContributorAddChannel.ts ..... 986
apps/web/src/server/socket/channels/EditorProjectContributorChannel.ts ..... 988
apps/web/src/server/socket/channels/EditorProjectContributorDeleteChannel.ts ..... 990
apps/web/src/server/socket/channels/EditorProjectContributorPermissionChannel.ts ..... 992
apps/web/src/server/socket/channels/EditorProjectCreate.ts ..... 994
apps/web/src/server/socket/channels/EditorProjectDelete.ts ..... 996
apps/web/src/server/socket/channels/EditorProjectFunctionContent.ts ..... 997
apps/web/src/server/socket/channels/EditorProjectFunctionContentChange.ts ..... 999
apps/web/src/server/socket/channels/EditorProjectFunctionCreate.ts ..... 1000
apps/web/src/server/socket/channels/EditorProjectFunctionDelete.ts ..... 1002
apps/web/src/server/socket/channels/EditorProjectFunctionRename.ts ..... 1004
apps/web/src/server/socket/channels/EditorProjectFunctions.ts ..... 1006
apps/web/src/server/socket/channels/EditorProjectInfo.ts ..... 1007
apps/web/src/server/socket/channels/EditorProjectList.ts ..... 1008
apps/web/src/server/socket/channels/EditorProjectPageContent.ts ..... 1009
apps/web/src/server/socket/channels/EditorProjectPageContentChange.ts ..... 1011
apps/web/src/server/socket/channels/EditorProjectPageCreate.ts ..... 1014

apps/web/src/server/socket/channels/EditorProjectPageDelete.ts ..... 1016
apps/web/src/server/socket/channels/EditorProjectPageRename.ts ..... 1017
apps/web/src/server/socket/channels/EditorProjectPages.ts ..... 1019
apps/web/src/server/socket/channels/EditorProjectRename.ts ..... 1020
apps/web/src/server/socket/channels/EditorProjectStaticAssetDeleteChannel.ts ..... 1022
apps/web/src/server/socket/channels/EditorProjectStaticAssetsChannel.ts ..... 1023
apps/web/src/server/socket/channels/EditorProjectStaticAssetUploadChannel.ts ..... 1025
apps/web/src/server/socket/index.ts ..... 1026
apps/web/src/server/socket/SocketRegistry.ts ..... 1030
apps/web/src/special-components/codeblock.module.css ..... 1031
apps/web/src/special-components/CodeBlock.tsx ..... 1031
apps/web/src/special-components/MDRenderer.tsx ..... 1032
apps/web/src/styles/globals.scss ..... 1033
apps/web/src/styles/tiptap.css ..... 1037
apps/web/src/types/IAskPost.ts ..... 1039
apps/web/src/types/IAskProfile.ts ..... 1039
apps/web/src/types/IIssue.ts ..... 1039
apps/web/src/types/IProject.ts ..... 1040
apps/web/src/types/IUMLDiagram.ts ..... 1040
apps/web/src/types/IWikiPage.ts ..... 1040
apps/web/tsconfig.json ..... 1041
apps/web/tsconfig.server.json ..... 1041
captain-definition ..... 1041
captain-definition-codeup ..... 1041
CODE_OF_CONDUCT.md ..... 1042
docker-compose.yml ..... 1044
Dockerfile ..... 1044
images.docker/codeup-java-rce/Dockerfile ..... 1044
images.docker/codeup-tsx-rce/Dockerfile ..... 1044
package.json ..... 1044
packages/components/admin/AdminContentsLayout.tsx ..... 1045
packages/components/admin/AdminLayout.tsx ..... 1047
packages/components/admin/AdminProfileViewer.tsx ..... 1051
packages/components/admin/AdminQuizMaker.tsx ..... 1057
packages/components/admin/AdminUserContact.tsx ..... 1060
packages/components/admin/AdminUserCreator.tsx ..... 1062
packages/components/admin/AdminUserViewer.tsx ..... 1064
packages/components/admin/AuthenticateLoadingIndicator.tsx ..... 1075
packages/components/ask/AskLayout.tsx ..... 1075
packages/components/ask/AskProfileCertificateDisplay.tsx ..... 1078
packages/components/ask/AskProfileProjectDisplay.tsx ..... 1079
packages/components/ask/EditProfileModal.tsx ..... 1080

packages/components/ask/FTSPopup.tsx ..... 1084
packages/components/ask/PostCardV3.tsx ..... 1086
packages/components/codeup/CodeUpModal.tsx ..... 1091
packages/components/community/CommunityNavigationBar.tsx ..... 1092
packages/components/community/index.ts ..... 1094
packages/components/community/post/index.ts ..... 1094
packages/components/community/post/PostButton.tsx ..... 1095
packages/components/community/post/PostCard.tsx ..... 1096
packages/components/community/post/PostMediaUploader.tsx ..... 1101
packages/components/community/post/StarterKit.ts ..... 1103
packages/components/community/post/TipTap.tsx ..... 1105
packages/components/community/profile/index.ts ..... 1113
packages/components/community/profile/ProfileAvatar.tsx ..... 1113
packages/components/community/profile/ProfileAvatarModifier.tsx ..... 1114
packages/components/community/profile/ProfileBadges.tsx ..... 1116
packages/components/community/profile/ProfileBanner.tsx ..... 1117
packages/components/community/profile/ProfileBannerModifier.tsx ..... 1118
packages/components/community/profile/ProfileDataDisplay.tsx ..... 1120
packages/components/community/profile/ProfileMenuButton.tsx ..... 1122
packages/components/community/profile/ProfileNotFound.tsx ..... 1124
packages/components/course/CourseNavigationBar.tsx ..... 1125
packages/components/course/CourseSections.tsx ..... 1127
packages/components/course/CourseTitle.tsx ..... 1129
packages/components/course/HTMLCourseSections.tsx ..... 1129
packages/components/course/index.ts ..... 1130
packages/components/course/ModuleCard.tsx ..... 1131
packages/components/course/ModuleCardRow.tsx ..... 1132
packages/components/course/Quiz.tsx ..... 1132
packages/components/creator/studio/CourseEditor.tsx ..... 1138
packages/components/creator/studio/CreatorStudioLayout.tsx ..... 1142
packages/components/editor/CollaboratorManager.tsx ..... 1143
packages/components/editor/CollaboratorManagerBody.tsx ..... 1144
packages/components/editor/EditorPageSidebar.tsx ..... 1153
packages/components/editor/EditorPagesSidebar.tsx ..... 1156
packages/components/editor/EditorPreviewModal.tsx ..... 1158
packages/components/editor/EditorProjectLayout.tsx ..... 1160
packages/components/editor/PageResourceManager.tsx ..... 1161
packages/components/editorv2/EditorFileExplorer.tsx ..... 1165
packages/components/editorv2/EditorFileExplorerIcon.tsx ..... 1169
packages/components/editorv2/EditorProjectLayout.tsx ..... 1171
packages/components/editorv2/uml/UMLEditor.tsx ..... 1173
packages/components/editorv2/uml/UMLEREditor.tsx ..... 1176

packages/components/editorv2/uml/UMLImplEditor.tsx ..... 1183
packages/components/editorv2/uml/UMLPreview.tsx ..... 1192
packages/components/editorv2/uml/UMLSeqEditor.tsx ..... 1193
packages/components/emails/DeleteUser.tsx ..... 1196
packages/components/emails/Login.tsx ..... 1197
packages/components/emails/Newsletter.tsx ..... 1198
packages/components/emails/PrivateMessage.tsx ..... 1199
packages/components/emails/Register.tsx ..... 1200
packages/components/emails/Reset.tsx ..... 1202
packages/components/emails/UpdateEmail.tsx ..... 1204
packages/components/explore/CourseCard.tsx ..... 1205
packages/components/extra/DataTable.tsx ..... 1206
packages/components/forum/ChatCardV2.tsx ..... 1210
packages/components/forum/ChatMessageV2.tsx ..... 1212
packages/components/forum/ChatViewV2.tsx ..... 1212
packages/components/forum/DeferredPostCardV2.tsx ..... 1218
packages/components/forum/ForumNavigationBar.tsx ..... 1219
packages/components/forum/PostCardV2.tsx ..... 1221
packages/components/global/AppShell.tsx ..... 1227
packages/components/global/CHeading.tsx ..... 1229
packages/components/global/CommandPalette.tsx ..... 1230
packages/components/global/DividerWithText.tsx ..... 1241
packages/components/global/DividerWithTextV2.tsx ..... 1241
packages/components/global/ErrorComponent.tsx ..... 1242
packages/components/global/Footer.tsx ..... 1243
packages/components/global/GridLayout.tsx ..... 1247
packages/components/global/GridLayoutV2.tsx ..... 1250
packages/components/global/index.ts ..... 1253
packages/components/global/MenuButtons.tsx ..... 1254
packages/components/global/NavigationBar.tsx ..... 1260
packages/components/global/NavigationBarMenu.tsx ..... 1270
packages/components/global/Newsletter.tsx ..... 1273
packages/components/global/Sidebar.tsx ..... 1275
packages/components/global/UserAuthAction.tsx ..... 1277
packages/components/global/UserAuthLogin.tsx ..... 1278
packages/components/global/UserAuthRegister.tsx ..... 1282
packages/components/global/UserAuthReset.tsx ..... 1287
packages/components/global/UserAuthSSO.tsx ..... 1289
packages/components/global/VideoPlayer.tsx ..... 1290
packages/components/icons/InCodeIcon.tsx ..... 1291
packages/components/ide/ContributorManager.tsx ..... 1292
packages/components/ide/CreateProjectMenu.tsx ..... 1302

packages/components/ide/FileManager.tsx ..... 1310
packages/components/ide/index.ts ..... 1325
packages/components/ide/KeyboardShortcuts.tsx ..... 1325
packages/components/ide/OpenProjectMenu.tsx ..... 1327
packages/components/ide/PreviewModal.tsx ..... 1346
packages/components/ide/PublishManager.tsx ..... 1348
packages/components/ide/SnippetViewer.tsx ..... 1356
packages/components/ide/TemplateImporter.tsx ..... 1359
packages/components/ide/TemplatePicker.tsx ..... 1362
packages/components/ide/TemplatePublisher.tsx ..... 1367
packages/components/landingpage/BrandStats.tsx ..... 1369
packages/components/landingpage/Milestones.tsx ..... 1371
packages/components/landingpage/OverviewSection.tsx ..... 1374
packages/components/landingpage/SuperText.tsx ..... 1376
packages/components/landingpage/Testimonials.tsx ..... 1377
packages/components/landingpagev2/BrandStats.tsx ..... 1380
packages/components/landingpagev2/Milestones.tsx ..... 1382
packages/components/landingpagev2/Testimonials.tsx ..... 1385
packages/components/orgs/OrgAdminLayout.tsx ..... 1388
packages/components/package.json ..... 1389
packages/components/projectflows/IssueNode.tsx ..... 1390
packages/components/roadmap/CourseSelectorNode.tsx ..... 1391
packages/components/roadmap/EditorProjectSelectorNode.tsx ..... 1395
packages/components/roadmap/FlowSelectorNode.tsx ..... 1399
packages/components/roadmap/ProjectIdeaSelectorNode.tsx ..... 1404
packages/components/roadmap/ToDoSelectorNode.tsx ..... 1406
packages/components/todo/ListItem.tsx ..... 1411
packages/components/todo/ListViewer.tsx ..... 1415
packages/components/tsconfig.json ..... 1418
packages/eslint-config-custom/index.js ..... 1418
packages/eslint-config-custom/package.json ..... 1419
packages/hooks/hooks/useAnalytics.ts ..... 1419
packages/hooks/hooks/useCourseProgress.ts ..... 1419
packages/hooks/hooks/useCourseRedirect.ts ..... 1421
packages/hooks/hooks/useHasProfile.ts ..... 1421
packages/hooks/hooks/useHover.ts ..... 1422
packages/hooks/hooks/useKeyPress.ts ..... 1423
packages/hooks/hooks/useLoggedIn.ts ..... 1424
packages/hooks/hooks/useUserInfo.ts ..... 1425
packages/hooks/hooks/useUserLoggedIn.ts ..... 1426
packages/hooks/hooks/useUserLoggedInRedirect.ts ..... 1426
packages/hooks/index.ts ..... 1427

packages/hooks/package.json ..... 1427
packages/hooks/tsconfig.json ..... 1428
packages/i18n/package.json ..... 1428
packages/i18n/src/index.ts ..... 1428
packages/i18n/tsconfig.json ..... 1429
packages/incode-ast-generator/package.json ..... 1430
packages/incode-ast-generator/src/AbstractSyntaxTreeGenerator.ts ..... 1430
packages/incode-ast-generator/src/index.ts ..... 1432
packages/incode-ast-generator/src/types/AbstractSyntaxTree.ts ..... 1432
packages/incode-ast-generator/src/util/PreCompiler.ts ..... 1433
packages/incode-ast-generator/tsconfig.json ..... 1433
packages/incode-blockly/package.json ..... 1433
packages/incode-blockly/src/blockly/BlocklyCompiler.ts ..... 1434
packages/incode-blockly/src/blockly/blocks/AddToElementBlock.ts ..... 1436
packages/incode-blockly/src/blockly/blocks/AddToScreenBlock.ts ..... 1436
packages/incode-blockly/src/blockly/blocks/AskBlock.ts ..... 1437
packages/incode-blockly/src/blockly/blocks/CallMethodBlock.ts ..... 1437
packages/incode-blockly/src/blockly/blocks/CreateMethodBlock.ts ..... 1438
packages/incode-blockly/src/blockly/blocks/CreateVarBlock.ts ..... 1439
packages/incode-blockly/src/blockly/blocks/CreateVarTypeBlock.ts ..... 1439
packages/incode-blockly/src/blockly/blocks/ElseBlock.ts ..... 1440
packages/incode-blockly/src/blockly/blocks/ElseIfNumberBlock.ts ..... 1440
packages/incode-blockly/src/blockly/blocks/ElseIfTextBlock.ts ..... 1441
packages/incode-blockly/src/blockly/blocks/FunctionBlock.ts ..... 1442
packages/incode-blockly/src/blockly/blocks/IfEventBlock.ts ..... 1443
packages/incode-blockly/src/blockly/blocks/IfNumberBlock.ts ..... 1444
packages/incode-blockly/src/blockly/blocks/IfTextBlock.ts ..... 1445
packages/incode-blockly/src/blockly/blocks/LogBlock.ts ..... 1446
packages/incode-blockly/src/blockly/blocks/RepeatWhile.ts ..... 1446
packages/incode-blockly/src/blockly/blocks/RepeatWhileNumber.ts ..... 1447
packages/incode-blockly/src/blockly/blocks/RepeatXTimesBlock.ts ..... 1448
packages/incode-blockly/src/blockly/blocks/SetVarBorderStyle.ts ..... 1449
packages/incode-blockly/src/blockly/blocks/SetVarColorBlock.ts ..... 1450
packages/incode-blockly/src/blockly/blocks/SetVarDecorationPropsBlock.ts ..... 1451
packages/incode-blockly/src/blockly/blocks/SetVarFontWeight.ts ..... 1451
packages/incode-blockly/src/blockly/blocks/SetVarPositionBlock.ts ..... 1452
packages/incode-blockly/src/blockly/blocks/SetVarPropertyBlock.ts ..... 1453
packages/incode-blockly/src/blockly/blocks/SetVarTextAlign.ts ..... 1454
packages/incode-blockly/src/blockly/blocks/StartBlock.ts ..... 1455
packages/incode-blockly/src/blockly/Blocks.ts ..... 1455
packages/incode-blockly/src/blockly/theme/BlocklyThemeDark.ts ..... 1456
packages/incode-blockly/src/BlocklyEditor.tsx ..... 1457

packages/incode-blockly/tsconfig.json ..... 1458
packages/incode-compiler/jest.config.js ..... 1459
packages/incode-compiler/package.json ..... 1459
packages/incode-compiler/src/Blockly.ts ..... 1459
packages/incode-compiler/src/index.ts ..... 1461
packages/incode-compiler/src/module/AbstractSyntaxTreeGenerator.ts ..... 1461
packages/incode-compiler/src/module/AliasManager.ts ..... 1463
packages/incode-compiler/src/module/CodeGenerator.ts ..... 1465
packages/incode-compiler/src/module/commands/AddCommand.ts ..... 1466
packages/incode-compiler/src/module/commands/AskCommand.ts ..... 1466
packages/incode-compiler/src/module/commands/CallCommand.ts ..... 1467
packages/incode-compiler/src/module/commands/CommandExecutor.ts ..... 1467
packages/incode-compiler/src/module/commands/CommentCommand.ts ..... 1468
packages/incode-compiler/src/module/commands/CreateCommand.ts ..... 1469
packages/incode-compiler/src/module/commands/ElseCommand.ts ..... 1469
packages/incode-compiler/src/module/commands/IfCommand.ts ..... 1470
packages/incode-compiler/src/module/commands/InCodeCommand.ts ..... 1471
packages/incode-compiler/src/module/commands/JSCommand.ts ..... 1471
packages/incode-compiler/src/module/commands/PrintCommand.ts ..... 1471
packages/incode-compiler/src/module/commands/RemoveCommand.ts ..... 1472
packages/incode-compiler/src/module/commands/RepeatCommand.ts ..... 1472
packages/incode-compiler/src/module/commands/SetCommand.ts ..... 1473
packages/incode-compiler/src/module/commands/WaitCommand.ts ..... 1483
packages/incode-compiler/src/module/Compiler.ts ..... 1483
packages/incode-compiler/src/module/Error.ts ..... 1484
packages/incode-compiler/src/module/__tests__/AbstractSyntaxTreeGenerator.test.ts ..... 1484
packages/incode-compiler/src/module/__tests__/CodeGenerator.test.ts ..... 1485
packages/incode-compiler/src/types/AbstractSyntaxTree.ts ..... 1486
packages/incode-compiler/src/types/JSONObject.ts ..... 1486
packages/incode-compiler/src/util/PreCompiler.ts ..... 1487
packages/incode-compiler/src/util/__tests__/PreCompiler.test.ts ..... 1487
packages/incode-compiler/tsconfig.json ..... 1487
packages/rest/package.json ..... 1487
packages/rest/src/api/Account.ts ..... 1488
packages/rest/src/api/Admin.ts ..... 1490
packages/rest/src/api/Analytics.ts ..... 1504
packages/rest/src/api/Ask.ts ..... 1505
packages/rest/src/api/Blog.ts ..... 1507
packages/rest/src/api/Certificates.ts ..... 1508
packages/rest/src/api/CodeChallenges.ts ..... 1509
packages/rest/src/api/Community.ts ..... 1510
packages/rest/src/api/Course.ts ..... 1513

packages/rest/src/api/Creator.ts ..... 1515
packages/rest/src/api/Discovery.ts ..... 1518
packages/rest/src/api/Editor.ts ..... 1519
packages/rest/src/api/Forum.ts ..... 1520
packages/rest/src/api/IDE.ts ..... 1523
packages/rest/src/api/Kids.ts ..... 1529
packages/rest/src/api/Landingpage.ts ..... 1531
packages/rest/src/api/Newsletter.ts ..... 1531
packages/rest/src/api/Orgs.ts ..... 1532
packages/rest/src/api/Payments.ts ..... 1536
packages/rest/src/api/ProjectIdeas.ts ..... 1536
packages/rest/src/api/Projects.ts ..... 1536
packages/rest/src/api/RCE.ts ..... 1546
packages/rest/src/api/Roadmap.ts ..... 1547
packages/rest/src/api/Snippets.ts ..... 1548
packages/rest/src/api/ToDo.ts ..... 1550
packages/rest/src/index.ts ..... 1553
packages/rest/src/makeRequest.ts ..... 1563
packages/rest/src/types/IResponse.ts ..... 1564
packages/rest/tsconfig.json ..... 1565
packages/serverless-runtime/Dockerfile ..... 1565
packages/serverless-runtime/index.js ..... 1565
packages/serverless-runtime/package.json ..... 1566
packages/tsconfig/base.json ..... 1566
packages/tsconfig/nextjs.json ..... 1567
packages/tsconfig/package.json ..... 1567
packages/tsconfig/react-library.json ..... 1567
packages/tsconfig/README.md ..... 1568
packages/util/package.json ..... 1568
packages/util/src/AccountManager.ts ..... 1568
packages/util/src/blockly/blockly/BlocklyCompiler.ts ..... 1570
packages/util/src/blockly/blockly/Blocks.ts ..... 1572
packages/util/src/blockly/blockly/theme/BlocklyThemeDark.ts ..... 1581
packages/util/src/blockly/BlocklyEditor.tsx ..... 1582
packages/util/src/blockly/toolbox.ts ..... 1582
packages/util/src/checkPassword.ts ..... 1584
packages/util/src/courses/Course.ts ..... 1584
packages/util/src/courses/CourseDB.ts ..... 1585
packages/util/src/generateJWT.ts ..... 1585
packages/util/src/generateMail.ts ..... 1586
packages/util/src/grapejs/plugin-flexbox.ts ..... 1587
packages/util/src/models/AnalyticsModel.ts ..... 1590

packages/util/src/models/AskPostModel.ts 1591
packages/util/src/models/AskProfileModel.ts 1592
packages/util/src/models/BlogModel.ts 1593
packages/util/src/models/Certificate.ts 1593
packages/util/src/models/CodeChallengeModel.ts 1595
packages/util/src/models/CodeSnippet.ts 1595
packages/util/src/models/CodeUpKidsProjectModel.ts 1596
packages/util/src/models/CommunityPostModel.ts 1597
packages/util/src/models/CommunityProfileModel.ts 1598
packages/util/src/models/CompletedUserChallenges.ts 1599
packages/util/src/models/ContactModel.ts 1599
packages/util/src/models/ContentModel.ts 1600
packages/util/src/models/CourseEnrollment.ts 1601
packages/util/src/models/CourseModel.ts 1602
packages/util/src/models/CourseProgressModel.ts 1603
packages/util/src/models/CourseSectionProgressModel.ts 1604
packages/util/src/models/CustomerChoiceModel.ts 1604
packages/util/src/models/EditorProjectFunctionModel.ts 1605
packages/util/src/models/EditorProjectKVStorageModel.ts 1605
packages/util/src/models/EditorProjectModel.ts 1606
packages/util/src/models/EditorProjectPageModel.ts 1607
packages/util/src/models/ErrorModel.ts 1608
packages/util/src/models/ForumChatModel.ts 1609
packages/util/src/models/ForumPostModel.ts 1610
packages/util/src/models/ForumProfileModel.ts 1610
packages/util/src/models/IDEProjectModel.ts 1611
packages/util/src/models/IDETemplateModel.ts 1613
packages/util/src/models/NewsletterModel.ts 1614
packages/util/src/models/ObtainedCertificateModel.ts 1614
packages/util/src/models/OrgBlogModel.ts 1615
packages/util/src/models/OrgModel.ts 1616
packages/util/src/models/OrgProjectTemplateModel.ts 1617
packages/util/src/models/ProjectFlowModel.ts 1617
packages/util/src/models/ProjectI18nModel.ts 1618
packages/util/src/models/ProjectIdeaModel.ts 1618
packages/util/src/models/ProjectIssueModel.ts 1619
packages/util/src/models/ProjectModel.ts 1620
packages/util/src/models/ProjectUMLModel.ts 1621
packages/util/src/models/ProjectWikiPageModel.ts 1621
packages/util/src/models/RoadmapModel.ts 1622
packages/util/src/models/ShortURLModel.ts 1623
packages/util/src/models/TextContentModel.ts 1623

packages/util/src/models/TodoListModel.ts ..... 1624
packages/util/src/models/ToDoProjectModel.ts ..... 1625
packages/util/src/models/UserModel.ts ..... 1626
packages/util/src/models/VideoModel.ts ..... 1627
packages/util/src/monaco/uml.ts ..... 1629
packages/util/src/mongo.ts ..... 1632
packages/util/src/Networking.ts ..... 1632
packages/util/src/pathsToStructure.ts ..... 1633
packages/util/src/sendMail.ts ..... 1634
packages/util/src/types/CourseProgress.ts ..... 1635
packages/util/src/updateForumBadges.ts ..... 1635
packages/util/src/userworkflow/redirect.ts ..... 1636
packages/util/tsconfig.json ..... 1637
pullImages.sh ..... 1637
qodana.yaml ..... 1638
README.md ..... 1638
rest-api.md ..... 1639
SECURITY.md ..... 1641
sloc.js ..... 1642
toDocx.js ..... 1644
turbo.json ..... 1646

## .eslintrc.js

```
module.exports = {
  root: true,
  // This tells ESLint to load the config from the package `eslint-config-custom`
  extends: ["custom"],
  settings: {
    next: {
      rootDir: ["apps/*/"],
    },
  },
};
```

## apps/storage/Dockerfile

```
FROM node:16

WORKDIR /codeup-storage-server

COPY . .

RUN npm install

EXPOSE 3000

ENTRYPOINT ["node", "src/index.js"]
```

## apps/storage/package.json

```
{
  "name": "@codeup/storage",
  "version": "1.0.0",
```

```
  "description": "Storage Server",
  "main": "src/index.js",
  "repository": "https://github.com/MCTzOCK/codeup",
  "author": "Ben Siebert",
  "license": "GPL-3.0-only",
  "private": false,
  "scripts": {
    "docker-build": "docker build . -t mctzock/codeup-storage"
  },
  "dependencies": {
    "body-parser": "^1.20.0",
    "cors": "^2.8.5",
    "express": "^4.18.1",
    "express-fileupload": "^1.4.0"
  }
}
```

## apps/storage/src/index.js

```
/**
 * src/index.js
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 05.07.2022
 *
 */

const fs = require('fs');
const path = require('path')
const os = require('os');
const express = require('express');
const cors = require('cors');
const fileUpload = require('express-fileupload');
const bodyParser = require("body-parser");

if(!fs.existsSync(path.join(os.homedir(), ".codeup-storage"))) {
  fs.mkdirSync(path.join(os.homedir(), ".codeup-storage"));
}

const app = express();

app.use(fileUpload());

app.use("/file", express.static(path.join(os.homedir(), ".codeup-storage")));

app.post("/upload", (req, res) => {
  let uploadFile, uploadPath;

  if(!req.files || Object.keys(req.files).length === 0 || !req.files.upload) {
    res.status(400).json({
      error: "No files were uploaded."
    }).end();
    return;
  }

  uploadFile = req.files.upload;

  let uploadName = "";
  let f = false;
```

```
  while(!f) {
    uploadName = randomString(64);
    if(!fs.existsSync(path.join(os.homedir(), ".codeup-storage", uploadName, uploadFile.name))) {
      f = true;
    }
  }

  fs.mkdirSync(path.join(os.homedir(), ".codeup-storage", uploadName));

  uploadPath = path.join(os.homedir(), ".codeup-storage", uploadName, uploadFile.name);

  uploadFile.mv(uploadPath, (err) => {
    if(err) return res.status(500).json({error: err});
    res.status(200).json({
      success: true,
      url: "https://storage.codeup.space/file/" + uploadName + "/" + uploadFile.name
    })
  })
})

app.use(cors());

app.listen(3000, "0.0.0.0", () => {
  console.log("Listening on port 3000")
});

function randomString(length) {
  let result = "";
  const characters =
    "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789";
  const charactersLength = characters.length;
  for (let i = 0; i < length; i++) {
    result += characters.charAt(Math.floor(Math.random() * charactersLength));
  }
  return result;
}
```

## apps/web/.eslintrc.js

```
module.exports = {
  root: true,
  extends: ["custom"],
};
```

## apps/web/b.js

```
/**
 * apps/web/b.js
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2024 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 09.05.2024
 *
 */

const fs = require("fs");
const path = require("path");

const a = fs
  .readdirSync(path.join(__dirname, "src"), { recursive: true })
  .filter((f) => f.endsWith(".tsx"));
```

```
let b = {};

a.map((i) => {
  b[i] = false;
});

fs.writeFileSync(path.join(__dirname, "done.json"), JSON.stringify(b, null, 2));
```

## apps/web/done.json


```
{
  "pages\\admin\\analytics\\[id].tsx": false,
  "pages\\admin\\analytics\\index.tsx": false,
  "pages\\admin\\blog.tsx": false,
  "pages\\admin\\certificates.tsx": false,
  "pages\\admin\\contacts.tsx": false,
  "pages\\admin\\contents\\courses\\[id].tsx": false,
  "pages\\admin\\contents\\courses\\index.tsx": false,
  "pages\\admin\\contents\\courses\\new.tsx": false,
  "pages\\admin\\contents\\new.tsx": false,
  "pages\\admin\\discovery.tsx": false,
  "pages\\admin\\editor.tsx": false,
  "pages\\admin\\errors.tsx": false,
  "pages\\admin\\flows.tsx": false,
  "pages\\admin\\ide.tsx": false,
  "pages\\admin\\index.tsx": false,
  "pages\\admin\\kids.tsx": false,
  "pages\\admin\\newsletter.tsx": false,
  "pages\\admin\\orgs\\index.tsx": false,
  "pages\\admin\\profiles\\index.tsx": false,
  "pages\\admin\\project-ideas\\index.tsx": false,
  "pages\\admin\\roads.tsx": false,
  "pages\\admin\\shorts.tsx": false,
  "pages\\admin\\snippets.tsx": false,
  "pages\\admin\\testimonials.tsx": false,
  "pages\\admin\\todos.tsx": false,
  "pages\\admin\\users\\index.tsx": false,
  "pages\\ask\\index.tsx": false,
  "pages\\ask\\t\\[id].tsx": false,
  "pages\\ask\\u\\[username].tsx": false,
  "pages\\community\\index.tsx": false,
  "pages\\community\\post\\[id].tsx": false,
  "pages\\community\\profile\\[name].tsx": false,
  "pages\\community\\profile\\create.tsx": false,
  "pages\\community\\search.tsx": false,
  "pages\\courses\\index.tsx": false,
  "pages\\forum\\[username].tsx": false,
  "pages\\forum\\index.tsx": false,
  "pages\\forum\\messages\\[id].tsx": false,
  "pages\\forum\\messages\\index.tsx": false,
  "pages\\forum\\post\\[id].tsx": false,
  "pages\\showcase.tsx": false,
  "pages\\todo\\index.tsx": false
}
```


## apps/web/next-env.d.ts

```
/// <reference types="next" />
/// <reference types="next/image-types/global" />

// NOTE: This file should not be edited
// see https://nextjs.org/docs/basic-features/typescript for more information.
```

## apps/web/next.config.js

```
const path = require("path");
const withTM = require("next-transpile-modules")([
  "@codeup/components",
  "@codeup/util",
  "@codeup/hooks",
  "@codeup/rest",
  "octokit",
  "@webcontainer/api",
]);

const withMDX = require("@next/mdx")({
  extension: /\.mdx?$/,
  options: {
    remarkPlugins: [],
    rehypePlugins: [],
    providerImportSource: "@mdx-js/react",
  },
});

module.exports = withTM(
  withMDX({
    reactStrictMode: true,
    pageExtensions: ["ts", "tsx", "js", "jsx", "md", "mdx"],
    sassOptions: {
      includePaths: [path.join(__dirname, "src", "styles")],
    },
  })
);
```

## apps/web/nodemon.json

```
{
  "watch": ["src/server/**/*"],
  "exec": "cross-env NODE_ENV=development ts-node --project tsconfig.server.json
src/server/index.ts",
  "ext": "js ts"
}
```

## apps/web/package.json

```
{
  "name": "@codeup/web",
  "version": "2.0.0",
  "private": true,
  "scripts": {
    "dev": "nodemon",
    "build": "yarn build:collect-info && yarn build:next",
    "build:collect-info": "node scripts/collect-info.js",
    "build:server": "tsc --project tsconfig.server.json",
    "bundle:server": "rollup server-build/index.js -o server-build/bundle.mjs -c rollup.config.mjs",
    "build:next": "next build",
    "next:start": "node server-build/index.js",
    "next:lint": "next lint",
    "ts-start": "cross-env NODE_ENV=development ts-node --project tsconfig.server.json
src/server/index.ts",
    "ts-start-prod": "cross-env NODE_ENV=production ts-node --project tsconfig.server.json
src/server/index.ts"
  },
  "dependencies": {
    "@animxyz/react": "^0.6.7",
    "@brainstack/diagram": "^1.0.16",
```

"@chakra-ui/card": "^2.1.1",
"@chakra-ui/react": "^2.1.2",
"@choc-ui/chakra-autocomplete": "^5.1.4",
"@codeup/components": "*",
"@codeup/hooks": "*",
"@codeup/incode-ast-generator": "*",
"@codeup/incode-blockly": "*",
"@codeup/incode-compiler": "*",
"@codeup/util": "*",
"@dicebear/avatars": "^4.10.5",
"@dicebear/avatars-bottts-sprites": "^4.10.5",
"@dicebear/avatars-male-sprites": "^4.10.5",
"@emotion/react": "^11",
"@emotion/styled": "^11",
"@hello-pangea/dnd": "^16.0.1",
"@incodelang/compiler": "^2.0.9",
"@mdx-js/loader": "^2.1.2",
"@mdx-js/react": "^2.1.2",
"@monaco-editor/react": "^4.4.5",
"@next/mdx": "^12.2.5",
"@react-email/render": "^0.0.7",
"@remotion/cli": "^3.0.25",
"@remotion/player": "^3.0.25",
"@tiptap/extension-code": "^2.0.0-beta.28",
"@tiptap/extension-color": "^2.0.0-beta.12",
"@tiptap/extension-dropcursor": "^2.0.0-beta.27",
"@tiptap/extension-font-family": "^2.0.0-beta.24",
"@tiptap/extension-highlight": "^2.0.0-beta.35",
"@tiptap/extension-image": "^2.0.0-beta.30",
"@tiptap/extension-link": "^2.0.0-beta.41",
"@tiptap/extension-text-align": "^2.0.0-beta.31",
"@tiptap/extension-text-style": "^2.0.0-beta.26",
"@tiptap/react": "^2.0.0-beta.112",
"@tiptap/starter-kit": "^2.0.0-beta.188",
"@tsparticles/react": "^3.0.0",
"@types/archiver": "^6.0.2",
"@types/byte-size": "^8.1.0",
"@types/dockerode": "^3.3.14",
"@types/dom-to-image": "^2.6.4",
"@types/formidable": "^2.0.5",
"@types/jsonwebtoken": "^8.5.8",
"@types/mime-types": "^2.1.1",
"@types/nodemailer": "^6.4.4",
"@types/prismjs": "^1.26.0",
"@types/project-name-generator": "^2.1.1",
"@types/qrcode": "^1.5.0",
"@types/ua-parser-js": "^0.7.36",
"@vercel/og": "^0.0.20",
"@webcontainer/api": "^1.1.9",
"archiver": "^7.0.1",
"blockly": "^9.3.1",
"bson": "^4.6.5",
"byte-size": "^8.1.0",
"canvg": "^4.0.1",
"chakra-ui-contextmenu": "^1.0.5",
"chakra-ui-steps": "^1.8.0",
"chart.js": "4.2.0",
"cross-env": "^7.0.3",
"dockerode": "^3.3.4",
"dom-to-image": "^2.6.0",

```
    "dotenv": "^16.0.3",
    "easymde": "^2.18.0",
    "file-loader": "^6.2.0",
    "formidable": "^2.1.1",
    "formik": "^2.2.9",
    "framer-motion": "^6.5.1",
    "grapesjs": "^0.20.1",
    "grapesjs-react": "^4.0.1",
    "html2canvas": "^1.4.1",
    "jsonwebtoken": "^8.5.1",
    "jspdf": "^2.5.1",
    "jszip": "^3.10.1",
    "markdown-to-jsx": "^7.1.8",
    "mime-types": "^2.1.35",
    "minio": "^8.0.0",
    "monaco-editor": "^0.33.0",
    "mongoose": "^6.3.5",
    "next": "^12.3.0",
    "next-seo": "^5.4.0",
    "nextjs-google-analytics": "^2.2.1",
    "nodemailer": "^6.7.5",
    "octokit": "^3.1.2",
    "open-graph-scraper": "^5.0.2",
    "otplib": "^12.0.1",
    "plyr-react": "^5.1.0",
    "posthog-js": "^1.54.0",
    "posthog-node": "^3.1.1",
    "project-name-generator": "^2.1.9",
    "puppeteer": "^19.3.0",
    "qrcode": "^1.5.1",
    "react": "^18.2.0",
    "react-chartjs-2": "5.1.0",
    "react-dom": "^18.2.0",
    "react-hook-form": "^7.31.3",
    "react-icons": "^5.2.1",
    "react-infinite-scroll-component": "^6.1.0",
    "react-markdown": "^8.0.3",
    "react-resizable-panels": "^2.0.19",
    "react-simplemde-editor": "^5.2.0",
    "react-terminal-ui": "^1.3.0",
    "react-typed": "^1.2.0",
    "react-xarrows": "^2.0.2",
    "reactflow": "^11.3.0",
    "remotion": "^3.0.25",
    "sass": "^1.54.0",
    "simple-git": "^3.24.0",
    "socket.io": "^4.5.4",
    "socket.io-client": "^4.5.4",
    "stripe": "^12.9.0",
    "ua-parser-js": "^1.0.32",
    "uuidv4": "^6.2.13"
  },
  "devDependencies": {
    "@rollup/plugin-commonjs": "^23.0.4",
    "@rollup/plugin-json": "^5.0.2",
    "@rollup/plugin-node-resolve": "^15.0.1",
    "@rollup/plugin-replace": "^5.0.1",
    "@types/node": "17.0.40",
    "@types/react": "^18.0.14",
    "@types/react-dom": "^18.0.5",
```

```
    "@vitalets/google-translate-api": "^8.0.0",
    "chalk": "4",
    "eslint": "8.17.0",
    "eslint-config-custom": "*",
    "eslint-config-next": "12.1.6",
    "next-transpile-modules": "9.0.0",
    "nodemon": "^2.0.20",
    "prettier": "^2.7.1",
    "rollup": "^3.7.0",
    "rollup-plugin-polyfill-node": "^0.11.0",
    "sloc": "^0.2.1",
    "sort-json": "^2.0.1",
    "ts-node": "^10.9.1",
    "tsconfig": "*",
    "typescript": "4.7.3"
  }
}
```

## apps/web/public/prism.css

```
/* PrismJS 1.29.0
https://prismjs.com/download.html#themes=prism-okaidia&languages=markup+css+clike+javascript */
code[class*=language-],pre[class*=language-]{color:#f8f8f2;background:0 0;text-shadow:0 1px rgba(0,0,0,.3);font-family:Consolas,Monaco,'Andale Mono','Ubuntu Mono',monospace;font-size:1em;text-align:left;white-space:pre;word-spacing:normal;word-break:normal;word-wrap:normal;line-height:1.5;-moz-tab-size:4;-o-tab-size:4;tab-size:4;-webkit-hyphens:none;-moz-hyphens:none;-ms-hyphens:none;hyphens:none}pre[class*=language-]{padding:1em;margin:.5em 0;overflow:auto;border-radius:.3em}:not(pre)>code[class*=language-],pre[class*=language-]{background:#272822}:not(pre)>code[class*=language-]{padding:.1em;border-radius:.3em;white-space:normal}.token.cdata,.token.comment,.token.doctype,.token.prolog{color:#8292a2}.token.punctuation{color:#f8f8f2}.token.namespace{opacity:.7}.token.constant,.token.deleted,.token.property,.token.symbol,.token.tag{color:#f92672}.token.boolean,.token.number{color:#ae81ff}.token.attr-name,.token.builtin,.token.char,.token.inserted,.token.selector,.token.string{color:#a6e22e}.language-css .token.string,.style
.token.string,.token.entity,.token.operator,.token.url,.token.variable{color:#f8f8f2}.token.atrule,.token.attr-value,.token.class-
name,.token.function{color:#e6db74}.token.keyword{color:#66d9ef}.token.important,.token.regex{color:#fd971f}.token.bold,.token.important{font-weight:700}.token.italic{font-style:italic}.token.entity{cursor:help}
```

## apps/web/README.md

```
## Getting Started

First, run the development server:

```bash
yarn dev
```

Open [http://localhost:3000](http://localhost:3000) with your browser to see the result.

You can start editing the page by modifying `pages/index.js`. The page auto-updates as you edit the file.

[API routes](https://nextjs.org/docs/api-routes/introduction) can be accessed on [http://localhost:3000/api/hello](http://localhost:3000/api/hello). This endpoint can be edited in `pages/api/hello.js`.

The `pages/api` directory is mapped to `/api/*`. Files in this directory are treated as [API routes](https://nextjs.org/docs/api-routes/introduction) instead of React pages.
```

## Learn More

To learn more about Next.js, take a look at the following resources:

- [Next.js Documentation](https://nextjs.org/docs) - learn about Next.js features and API.
- [Learn Next.js](https://nextjs.org/learn) - an interactive Next.js tutorial.

You can check out [the Next.js GitHub repository](https://github.com/vercel/next.js/) - your feedback and contributions are welcome!

## Deploy on Vercel

The easiest way to deploy your Next.js app is to use the [Vercel Platform](https://vercel.com/new?utm_source=github.com&utm_medium=referral&utm_campaign=turborepo-readme) from the creators of Next.js.

Check out our [Next.js deployment documentation](https://nextjs.org/docs/deployment) for more details.

## apps/web/rollup.config.mjs

```
import * as path from 'path';
import json from "@rollup/plugin-json";
import commonjs from "@rollup/plugin-commonjs";
import {nodeResolve} from "@rollup/plugin-node-resolve";
import nodePolyfills from "rollup-plugin-polyfill-node";

export default {
  input: "server-build/index.js",
  output: {
    name: "server",
    file: "server-build/bundle.js",
    format: "iife"
  },
  plugins: [json(), commonjs(), nodeResolve({
    browser: false,
    exportConditions: ["default", "module", "import", "require"]
  }),
    nodePolyfills()
  ],
};
```

## apps/web/scripts/collect-info.js

```
/**
 * apps/web/build/collect-package-info.js
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 05.12.2022
 *
 */

console.log("> Collecting package info...");

const fs = require("fs");
const path = require("path");
const os = require("os");
const { execSync } = require("child_process");

const INFO = {
```

```
  git: {
    commits: 0,
    commit: "",
    branch: "",
    lastCommitMessage: "",
  },
  packages: {
    count: 0,
    packages: [],
  },
  app: {
    totalLines: 0,
  },
};

const root = path.join(__dirname, "..");

const getGitInfo = () => {
  const cwd = process.cwd();

  const gitCommit = execSync("git rev-parse --short HEAD", { cwd }).toString();
  const gitCommits = execSync("git rev-list --count HEAD", { cwd }).toString();
  const gitBranch = execSync("git rev-parse --abbrev-ref HEAD", {
    cwd,
  }).toString();
  const gitLastCommitMessage = execSync("git log -1 --pretty=%B", {
    cwd,
  })
    .toString()
    .replace(/\n/g, "");

  INFO.git.commits = parseInt(gitCommits.replace("\n", ""));
  INFO.git.commit = gitCommit.replace("\n", "");
  INFO.git.branch = gitBranch.replace("\n", "");
  INFO.git.lastCommitMessage = gitLastCommitMessage.replace("\n", "");
};

const getPackageInfo = () => {
  const npmPackages = execSync("npm list --depth=0 --json", {
    cwd: root,
  }).toString();

  const getPackage = (pkg) => {
    let packages = [];
    console.log(pkg);
    for (let i in pkg.dependencies) {
      packages.push({
        name: i,
        version: pkg.dependencies[i].version,
      });
      if (pkg.dependencies[i].dependencies) {
        packages = packages.concat(getPackage(pkg.dependencies[i]));
      }
    }
    return packages;
  };

  let packages = getPackage(JSON.parse(npmPackages));

  // remove duplicates
```

```
  packages = packages.filter(
    (v, i, a) => a.findIndex((t) => t.name === v.name) === i
  );

  for (const p of packages) {
    INFO.packages.packages.push({
      name: p.name,
      version: p.version,
    });

    INFO.packages.count++;
  }
};

const getAppInfo = () => {
  const lines = execSync("node sloc --codeup-stats-only-lines-raw", {
    cwd: path.join(__dirname, "..", "..", ".."),
  });

  INFO.app.totalLines = parseInt(lines.toString().replace("\n", ""));
};

getGitInfo();
getPackageInfo();
getAppInfo();

fs.writeFileSync(
  path.join(__dirname, "..", ".codeup_app_stats.json"),
  JSON.stringify(INFO, null, 2)
);

console.log("> Collecting package info... Done!");
```

## apps/web/src/global/getS3.ts

```
/**
 * apps/web/src/global/getS3.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2024 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 30.05.2024
 *
 */

import * as minio from "minio";

export function getS3() {
  return new minio.Client({
    endPoint: process.env.MINIO_ENDPOINT as string,
    useSSL: process.env.MINIO_USE_SSL === "true",
    port: parseInt(process.env.MINIO_PORT as string),
    accessKey: process.env.MINIO_ACCESS_KEY as string,
    secretKey: process.env.MINIO_SECRET_KEY as string,
  });
}
```

## apps/web/src/global/i18n/onlineTranslateV3.ts

```
/**
 * apps/web/src/global/i18n/onlineTranslateV3.ts
 *
```

```
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2024 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 05.06.2024
 *
 */

export async function translateOnlineV3(options: {
  text: string;
  to: string;
  from?: string;
}) {
  const body = {
    text: options.text,
    to: options.to,
    from: options.from || "de",
  };

  const res = await fetch("https://translate-cache.ben-siebert.com/translate", {
    method: "POST",
    body: JSON.stringify(body),
    headers: {
      "Content-Type": "application/json",
      Accept: "application/json",
    },
  });

  const j = await res.json();

  return j.text;
}
```

## apps/web/src/global/md/defaultProps.ts

```
/**
 * apps/web/src/global/md/defaultProps.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2024 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 31.05.2024
 *
 */
import {
  Code,
  Heading,
  Link,
  ListItem,
  OrderedList,
  Table,
  Tbody,
  Td,
  Text,
  Th,
  Thead,
  Tr,
  UnorderedList,
} from "@chakra-ui/react";
import { CHeading } from "@codeup/components/dist/global/CHeading";

export const defaultMdOpts = {
```

```
overrides: {
  h1: {
    component: CHeading,
    props: {
      size: "2xl",
    },
  },
  h2: {
    component: CHeading,
    props: {
      size: "xl",
    },
  },
  h3: {
    component: CHeading,
    props: {
      size: "lg",
    },
  },
  h4: {
    component: CHeading,
    props: {
      size: "md",
    },
  },
  h5: {
    component: CHeading,
    props: {
      size: "md",
    },
  },
  h6: {
    component: CHeading,
    props: {
      size: "md",
    },
  },
  a: {
    component: Link,
    props: {
      fontSize: "lg",
      color: "brand.500",
    },
  },
  ul: {
    component: UnorderedList,
    props: {
      fontSize: "lg",
    },
  },
  ol: {
    component: OrderedList,
    props: {
      fontSize: "lg",
    },
  },
  li: {
    component: ListItem,
    props: {
      fontSize: "lg",
```

```
      },
    },
    table: {
      component: Table,
      props: {
        fontSize: "lg",
      },
    },
    thead: {
      component: Thead,
      props: {
        fontSize: "lg",
      },
    },
    tbody: {
      component: Tbody,
      props: {
        fontSize: "lg",
      },
    },
    tr: {
      component: Tr,
      props: {
        fontSize: "lg",
      },
    },
    th: {
      component: Th,
      props: {
        fontSize: "lg",
      },
    },
    td: {
      component: Td,
      props: {
        fontSize: "lg",
      },
    },
    p: {
      component: Text,
      props: {
        fontSize: "lg",
      },
    },
    code: {
      component: Code,
      props: {
        fontSize: "lg",
      },
    },
  },
};
```

## apps/web/src/global/PopupManager/AlertPopupComponent.tsx

```
/**
 * apps/web/src/global/PopupManager/AlertPopupComponent.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
```

```
 * Created At: 20.01.2023
 *
 */

import * as React from "react";
import {
  AlertDialog,
  AlertDialogBody,
  AlertDialogContent,
  AlertDialogFooter,
  AlertDialogHeader,
  AlertDialogOverlay,
  Button,
  useDisclosure,
} from "@chakra-ui/react";

export default function AlertPopupComponent(props: {
  title: string | undefined;
  description: string | React.ReactNode;
  customSize?: string;
  onClose: () => void;
  inputType?: "INPUT" | "TEXTAREA";
}) {
  const { isOpen, onOpen, onClose } = useDisclosure({
    defaultIsOpen: true,
  });

  const whatEverRef = React.useRef();

  return (
    <>
      <AlertDialog
        isOpen={isOpen}
        onClose={() => {
          onClose();
          props.onClose();
        }}
        isCentered
        // @ts-ignore
        leastDestructiveRef={whatEverRef}
        size={props.customSize ? props.customSize : "md"}
      >
        <AlertDialogOverlay bg="blackAlpha.300" backdropFilter="blur(10px)">
          <AlertDialogContent bg={"black"}>
            <AlertDialogHeader fontSize="lg" fontWeight="bold">
              {props.title ? props.title : "Meldung"}
            </AlertDialogHeader>

            <AlertDialogBody>{props.description}</AlertDialogBody>

            <AlertDialogFooter>
              <Button
                variant="brand"
                onClick={() => {
                  onClose();
                  props.onClose();
                }}
                ml={3}
              >
                Schließen
```

```
            </Button>
          </AlertDialogFooter>
        </AlertDialogContent>
      </AlertDialogOverlay>
    </AlertDialog>
  </>
 );
}
```

## apps/web/src/global/PopupManager/ConfirmPopupComponent.tsx

```
/**
 * apps/web/src/global/PopupManager/ConfirmPopupComponent.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 20.01.2023
 *
 */

import * as React from "react";
import {
  AlertDialog,
  AlertDialogBody,
  AlertDialogContent,
  AlertDialogFooter,
  AlertDialogHeader,
  AlertDialogOverlay,
  Button,
  useDisclosure,
} from "@chakra-ui/react";

export default function ConfirmPopupComponent(props: {
  title: string;
  question: string;
  onClose: () => void;
  approve: () => void;
  deny: () => void;
}) {
  const { isOpen, onOpen, onClose } = useDisclosure({
    defaultIsOpen: true,
  });

  const whatEverRef = React.useRef();

  return (
    <>
      <AlertDialog
        isOpen={isOpen}
        onClose={() => {
          onClose();
          props.onClose();
        }}
        isCentered
        // @ts-ignore
        leastDestructiveRef={whatEverRef}
      >
        <AlertDialogOverlay bg="blackAlpha.300" backdropFilter="blur(10px)">
          <AlertDialogContent bg={"black"}>
            <AlertDialogHeader fontSize="lg" fontWeight="bold">
```

```
            {props.title}
          </AlertDialogHeader>

          <AlertDialogBody>{props.question}</AlertDialogBody>

          <AlertDialogFooter>
            <Button
              onClick={() => {
                onClose();
                props.deny();
                props.onClose();
              }}
              // @ts-ignore
              ref={whatEverRef}
            >
              Nein
            </Button>
            <Button
              variant="brand"
              onClick={() => {
                onClose();
                props.approve();
                props.onClose();
              }}
              ml={3}
            >
              Ja
            </Button>
          </AlertDialogFooter>
        </AlertDialogContent>
      </AlertDialogOverlay>
    </AlertDialog>
  </>
 );
}
```

## apps/web/src/global/PopupManager/PromptPopupComponent.tsx

```
/**
 * apps/web/src/global/PopupManager/PromptPopupComponent.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 20.01.2023
 *
 */

import * as React from "react";
import { useId, useRef } from "react";
import {
  AlertDialog,
  AlertDialogBody,
  AlertDialogContent,
  AlertDialogFooter,
  AlertDialogHeader,
  AlertDialogOverlay,
  Button,
  FormControl,
  FormHelperText,
  FormLabel,
```

```
  Input,
  InputGroup,
  InputLeftElement,
  Textarea,
  useDisclosure,
} from "@chakra-ui/react";
import { FaPen } from "react-icons/fa";

export default function PromptPopupComponent(props: {
  title: string;
  helperText: string;
  callback: (i: string) => void;
  onClose: () => void;
  inputType?: "INPUT" | "TEXTAREA";
}) {
  const { isOpen, onOpen, onClose } = useDisclosure({
    defaultIsOpen: true,
  });

  const whatEverRef = useRef();

  const id = useId();

  return (
    <>
      <AlertDialog
        isOpen={isOpen}
        onClose={() => {
          onClose();
          props.onClose();
        }}
        isCentered
        // @ts-ignore
        leastDestructiveRef={whatEverRef}
      >
        <AlertDialogOverlay bg="blackAlpha.300" backdropFilter="blur(10px)">
          <AlertDialogContent bg={"black"}>
            <AlertDialogHeader fontSize="lg" fontWeight="bold">
              {props.title}
            </AlertDialogHeader>

            <AlertDialogBody>
              <FormControl isRequired>
                <FormLabel>Eingabe</FormLabel>
                {props.inputType === "INPUT" ? (
                  <InputGroup>
                    <InputLeftElement>
                      <FaPen />
                    </InputLeftElement>
                    <Input id={"input-" + id} placeholder={"Eingabe"} />
                  </InputGroup>
                ) : (
                  <Textarea id={"input-" + id} placeholder={"Eingabe"} />
                )}
                <FormHelperText>{props.helperText}</FormHelperText>
              </FormControl>
            </AlertDialogBody>

            <AlertDialogFooter>
              <Button
```

```
              onClick={() => {
                onClose();
                props.onClose();
              }}
              // @ts-ignore
              ref={whatEverRef}
            >
              Abbrechen
            </Button>
            <Button
              variant={"brand"}
              onClick={() => {
                const data = document.getElementById(
                  "input-" + id
                ) as HTMLInputElement;

                props.callback(data.value);
                onClose();
                props.onClose();
              }}
              ml={3}
            >
              Bestätigen
            </Button>
          </AlertDialogFooter>
        </AlertDialogContent>
      </AlertDialogOverlay>
    </AlertDialog>
  </>
 );
}
```

## apps/web/src/global/PopupManager/SelectPopupComponent.tsx

```
/**
 * apps/web/src/global/PopupManager/SelectPopupComponent.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 21.01.2023
 *
 */

import * as React from "react";
import {
  AlertDialog,
  AlertDialogBody,
  AlertDialogContent,
  AlertDialogFooter,
  AlertDialogHeader,
  AlertDialogOverlay,
  Button,
  FormControl,
  FormHelperText,
  FormLabel,
  Input,
  InputGroup,
  InputLeftElement,
  Select,
  useDisclosure,
```

```
} from "@chakra-ui/react";
import { FaPen } from "react-icons/fa";
import { useId, useRef } from "react";

export default function SelectPopupComponent(props: {
  title: string;
  helperText: string;
  choices: string[];
  callback: (i: string) => void;
  onClose: () => void;
}) {
  const { isOpen, onOpen, onClose } = useDisclosure({
    defaultIsOpen: true,
  });

  const whatEverRef = useRef();

  const id = useId();

  return (
    <>
      <AlertDialog
        isOpen={isOpen}
        onClose={() => {
          onClose();
          props.onClose();
        }}
        isCentered
        // @ts-ignore
        leastDestructiveRef={whatEverRef}
      >
        <AlertDialogOverlay bg="blackAlpha.300" backdropFilter="blur(10px)">
          <AlertDialogContent bg={"black"}>
            <AlertDialogHeader fontSize="lg" fontWeight="bold">
              {props.title}
            </AlertDialogHeader>

            <AlertDialogBody>
              <FormControl isRequired>
                <FormLabel>Eingabe</FormLabel>
                <Select id={"input-" + id} placeholder="Wähle eine Option">
                  {props.choices.map((c, i) => {
                    return <option key={i}>{c}</option>;
                  })}
                </Select>
                <FormHelperText>{props.helperText}</FormHelperText>
              </FormControl>
            </AlertDialogBody>

            <AlertDialogFooter>
              <Button
                onClick={() => {
                  onClose();
                  props.onClose();
                }}
                // @ts-ignore
                ref={whatEverRef}
              >
                Abbrechen
              </Button>
```

```
            <Button
              variant={"brand"}
              onClick={() => {
                const data = document.getElementById(
                  "input-" + id
                ) as HTMLSelectElement;

                props.callback(data.options[data.selectedIndex].value);
                onClose();
                props.onClose();
              }}
              ml={3}
            >
              Bestätigen
            </Button>
          </AlertDialogFooter>
        </AlertDialogContent>
      </AlertDialogOverlay>
    </AlertDialog>
  </>
 );
}
```

## apps/web/src/global/PopupManager.tsx

```
/**
 * apps/web/src/global/PopupManager.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 20.01.2023
 *
 */
import {
  ChakraProvider,
  FormControl,
  FormHelperText,
  Input,
  InputGroup,
  InputLeftElement,
} from "@chakra-ui/react";
import ReactDOM from "react-dom";
import PromptPopupComponent from "./PopupManager/PromptPopupComponent";
import theme from "./theme";
import ConfirmPopupComponent from "./PopupManager/ConfirmPopupComponent";
import AlertPopupComponent from "./PopupManager/AlertPopupComponent";
import SelectPopupComponent from "./PopupManager/SelectPopupComponent";
import * as React from "react";
import { ReactNode } from "react";
import { FaFile } from "react-icons/fa";

export default class PopupManager {
  public static prompt(options: {
    title: string;
    helperText: string;
    callback: (input: string) => void;
    inputType?: "INPUT" | "TEXTAREA";
  }): void {
    ReactDOM.render(
      <>
```

```
      <ChakraProvider theme={theme}>
        <PromptPopupComponent
          title={options.title}
          helperText={options.helperText}
          callback={options.callback}
          inputType={options.inputType || "INPUT"}
          onClose={() => {
            ReactDOM.unmountComponentAtNode(
              document.querySelector(
                "#__codeup_manual_mount_point"
              ) as HTMLElement
            );
          }}
        />
      </ChakraProvider>
    </>,
    document.querySelector("#__codeup_manual_mount_point") as HTMLElement
  );
}

public static confirm(options: {
  title: string;
  question: string;
  approve: () => void;
  deny: () => void;
}): void {
  ReactDOM.render(
    <>
      <ChakraProvider theme={theme}>
        <ConfirmPopupComponent
          title={options.title}
          question={options.question}
          onClose={() => {
            ReactDOM.unmountComponentAtNode(
              document.querySelector(
                "#__codeup_manual_mount_point"
              ) as HTMLElement
            );
          }}
          approve={options.approve}
          deny={options.deny}
        />
      </ChakraProvider>
    </>,
    document.querySelector("#__codeup_manual_mount_point") as HTMLElement
  );
}

public static alert(options: {
  title?: string | undefined;
  description: string | ReactNode;
  callback?: () => void;
  customSize?: string;
}): void {
  ReactDOM.render(
    <>
      <ChakraProvider theme={theme}>
        <AlertPopupComponent
          customSize={options.customSize}
          title={options.title}
```

```
          description={options.description}
          onClose={() => {
            ReactDOM.unmountComponentAtNode(
              document.querySelector(
                "#__codeup_manual_mount_point"
              ) as HTMLElement
            );
            if (options.callback) options.callback();
          }}
        />
      </ChakraProvider>
    </>,
    document.querySelector("#__codeup_manual_mount_point") as HTMLElement
  );
}

public static select(options: {
  title: string;
  helperText: string;
  choices: string[];
  callback: (input: string) => void;
}) {
  ReactDOM.render(
    <>
      <ChakraProvider theme={theme}>
        <SelectPopupComponent
          title={options.title}
          helperText={options.helperText}
          callback={options.callback}
          choices={options.choices}
          onClose={() => {
            ReactDOM.unmountComponentAtNode(
              document.querySelector(
                "#__codeup_manual_mount_point"
              ) as HTMLElement
            );
          }}
        />
      </ChakraProvider>
    </>,
    document.querySelector("#__codeup_manual_mount_point") as HTMLElement
  );
}

public static uploadFile(options: {
  title: string;
  helperText: string;
  callback: (input: FileList) => void;
}): void {
  ReactDOM.render(
    <>
      <ChakraProvider theme={theme}>
        <AlertPopupComponent
          title={options.title}
          description={
            <>
              <FormControl>
                <InputGroup>
                  <InputLeftElement>
                    <FaFile />
```

```
              </InputLeftElement>
              <Input
                type="file"
                onChange={(e) => {
                  options.callback(e.target.files as FileList);
                  ReactDOM.unmountComponentAtNode(
                    document.querySelector(
                      "#__codeup_manual_mount_point"
                    ) as HTMLElement
                  );
                }}
              />
            </InputGroup>
            <FormHelperText>{options.helperText}</FormHelperText>
          </FormControl>
        </>
      }
      onClose={() => {
        ReactDOM.unmountComponentAtNode(
          document.querySelector(
            "#__codeup_manual_mount_point"
          ) as HTMLElement
        );
      }}
    />
  </ChakraProvider>
 </>,
 document.querySelector("#__codeup_manual_mount_point") as HTMLElement
 );
}

public static promptAsync(options: {
 title: string;
 helperText: string;
 inputType?: "INPUT" | "TEXTAREA";
}): Promise<string> {
 return new Promise((resolve) => {
  this.prompt({
   title: options.title,
   helperText: options.helperText,
   inputType: options.inputType || "INPUT",
   callback: (input: string) => {
    resolve(input);
   },
  });
 });
}

public static confirmAsync(options: {
 title: string;
 question: string;
}): Promise<boolean> {
 return new Promise((resolve) => {
  this.confirm({
   title: options.title,
   question: options.question,
   approve: () => {
    resolve(true);
   },
   deny: () => {
```

```
        resolve(false);
      },
    });
  });
}

public static selectAsync(options: {
  title: string;
  helperText: string;
  choices: string[];
}): Promise<string> {
  return new Promise((resolve) => {
    this.select({
      title: options.title,
      helperText: options.helperText,
      choices: options.choices,
      callback: (input: string) => {
        resolve(input);
      },
    });
  });
}

public static alertAsync(options: {
  title?: string;
  description: string | ReactNode;
  customSize?: string;
}): Promise<void> {
  return new Promise((resolve) => {
    this.alert({
      title: options.title,
      description: options.description,
      customSize: options.customSize,
      callback: () => {
        resolve();
      },
    });
  });
}

public static uploadFileAsync(options: {
  title: string;
  helperText: string;
}): Promise<FileList> {
  return new Promise((resolve) => {
    this.uploadFile({
      title: options.title,
      helperText: options.helperText,
      callback: (input: FileList) => {
        resolve(input);
      },
    });
  });
}

public static removeCurrentPopup(): void {
  ReactDOM.unmountComponentAtNode(
    document.querySelector("#__codeup_manual_mount_point") as HTMLElement
  );
}
```

```
}
```

## apps/web/src/global/rce/language/LanguageTemplates.ts

```
/**
 * apps/web/src/global/rce/language/LanguageTemplates.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2024 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 18.05.2024
 *
 */

export const LanguageTemplates: {
  [key: string]: string;
} = {
  typescript: "const x: string = 'Hello World';",
  javascript: "const x = 'Hello World';",
  python: "x = 'Hello World'",
  java: 'public class Main {\n\tpublic static void main(String[] args) {\n\t\tString x = "Hello
World";\n\t}\n}',
  go: 'package main\n\nimport "fmt"\n\nfunc main() {\n\tx := "Hello World"\n}',
};
```

## apps/web/src/global/server/isLoggedIn.ts

```
/**
 * apps/web/src/global/server/isLoggedIn.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2024 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 17.05.2024
 *
 */
import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/dist/mongo";
import UserModel from "@codeup/util/dist/models/UserModel";
import jwt from "jsonwebtoken";

export async function isLoggedIn(
  req: NextApiRequest,
  res: NextApiResponse
): Promise<{
  auth: boolean;
  user?: any;
}> {
  if (!req.headers["x-jwt"]) {
    res.status(401).send({
      error: "Not authenticated",
    });
    return {
      auth: false,
    };
  }

  const token = req.headers["x-jwt"] as string;

  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET as string);
```

```
    if (!decoded) {
      res.status(401).send({
        error: "Not authenticated",
      });
      return {
        auth: false,
      };
    }

    const user = await UserModel.findById((decoded as any).id);

    if (!user || !user.active) {
      res.status(401).send({
        error: "Not authenticated",
      });
      return {
        auth: false,
      };
    }

    return {
      auth: true,
      user,
    };
  } catch (e: any) {
    console.log(e);
    res.status(401).send({
      error: "Not authenticated",
    });
    return {
      auth: false,
    };
  }
}
```

## apps/web/src/global/server/rce.ts

```
/**
 * apps/web/src/global/server/rce/rce.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2024 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 18.05.2024
 *
 */
import { NextApiRequest, NextApiResponse } from "next";
import Docker from "dockerode";
import { isLoggedIn } from "./isLoggedIn";
import { createHash } from "crypto";
import fs from "fs";
import path from "path";
import { tmpdir } from "node:os";

const docker = new Docker();

export async function rce(
  req: NextApiRequest,
  res: NextApiResponse,
  options: {
    timeout: number;
```

```
    image: string;
    env: string[];
    cmd?: string[];
    entrypoint?: string[];
    code: string;
    tmpfs: {
      [key: string]: string;
    };
  }
) {
  if (req.method !== "POST") {
    res.status(405).json({
      status: 405,
      error: "Method not allowed",
    });
    return;
  }

  const { auth, user } = await isLoggedIn(req, res);

  if (!auth) return;

  try {
    const tempFile = `codeup-rce-${new Date().getTime()}-${
      user._id
    }-${createHash("sha256")
      .update(options.code)
      .digest("hex")
      .substring(0, 10)}`;

    fs.mkdirSync(path.join(tmpdir(), tempFile), { recursive: true });

    for (const key of Object.keys(options.tmpfs)) {
      fs.writeFileSync(path.join(tmpdir(), tempFile, key), options.tmpfs[key]);
    }

    const container = await docker.createContainer({
      name: `codeup-rce-${new Date().getTime()}-${user._id}-${createHash(
        "sha256"
      )
        .update(options.code)
        .digest("hex")
        .substring(0, 10)}`,
      Image: options.image,
      Cmd: options.cmd,
      Entrypoint: options.entrypoint,
      HostConfig: {
        Mounts: [
          {
            Type: "bind",
            Source: path.join(tmpdir(), tempFile),
            Target: "/usr/src/app",
            ReadOnly: false,
          },
        ],
      },
      Tty: true,
      Env: options.env,
      NetworkDisabled: true,
    });
```

```
    await container.start();

    container.attach(
      { stream: true, stdout: true, stderr: true },
      (err, stream) => {
        if (err) {
          res.status(500).json({
            status: 500,
            error: err.message,
          });
          return;
        }

        if (!stream) {
          res.status(500).json({
            status: 500,
            error: "No stream",
          });
          return;
        }
        res.setHeader("Content-Type", "text/plain");
        stream.pipe(res);
      }
    );

    setTimeout(async () => {
      try {
        await container.kill();
      } catch (e) {}
      try {
        await container.remove();
        fs.rmdirSync(path.join(tmpdir(), tempFile), { recursive: true });
      } catch (e) {}
    }, 1000 * options.timeout);
  } catch (e: any) {
    console.log(e);
    res.status(500).json({
      status: 500,
      error: e.message,
    });
  }
}
```

## apps/web/src/global/theme.ts

```
/**
 * apps/web/src/global/theme.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 20.01.2023
 *
 */

import { extendTheme } from "@chakra-ui/react";
import { StepsStyleConfig as Steps } from "chakra-ui-steps";

const theme = extendTheme({
  config: {
```

```
  useSystemColorMode: false,
  initialColorMode: "dark",
},
colors: {
  brand: {
    "100": "#FEFCD7",
    "200": "#F7F4B0",
    "300": "#F0EC89",
    "400": "#E9E462",
    "500": "#F7DE1F",
    "600": "#D8C31A",
    "700": "#B9A916",
    "800": "#9B8B11",
    "900": "#7C6E0C",
  },
  gray: {
    "100": "#C1C2C5",
    "200": "#A6A7AB",
    "300": "#909296",
    "400": "#5c5f66",
    "500": "#373A40",
    "600": "#2C2E33",
    "700": "#25262b",
    "800": "#1A1B1E",
    "900": "#141517",
  },
  red: {
    "900": "#ffe3e3",
    "800": "#ffc9c9",
    "700": "#ffa8a8",
    "600": "#ff8787",
    "500": "#ff6b6b",
    "400": "#fa5252",
    "300": "#f03e3e",
    "200": "#e03131",
    "100": "#c92a2a",
  },
  green: {
    "900": "#ebfbee",
    "800": "#d3f9d8",
    "700": "#b2f2bb",
    "600": "#8ce99a",
    "500": "#69db7c",
    "400": "#51cf66",
    "300": "#40c057",
    "200": "#37b24d",
    "100": "#2f9e44",
  },
  blue: {
    "900": "#d0ebff",
    "800": "#a5d8ff",
    "700": "#74c0fc",
    "600": "#4dabf7",
    "500": "#339af0",
    "400": "#228be6",
    "300": "#1c7ed6",
    "200": "#1971c2",
    "100": "#1864ab",
  },
  cyan: {
```

```
    "900": "#c5f6fa",
    "800": "#99e9f2",
    "700": "#66d9e8",
    "600": "#3bc9db",
    "500": "#22b8cf",
    "400": "#15aabf",
    "300": "#1098ad",
    "200": "#0c8599",
    "100": "#0b7285",
  },
  teal: {
    "100": "#e6fcf5",
    "200": "#c3fae8",
    "300": "#96f2d7",
    "400": "#63e6be",
    "500": "#38d9a9",
    "600": "#20c997",
    "700": "#12b886",
    "800": "#0ca678",
    "900": "#099268",
  },
  swhite: {
    "100": "#FFFFFF",
    "200": "#F2F2F2",
    "300": "#E5E5E5",
    "400": "#D9D9D9",
    "500": "#CCCCCC",
    "600": "#BFBFBF",
    "700": "#B3B3B3",
    "800": "#A6A6A6",
    "900": "#999999",
  },
  //black: "#101113",
  black: "#101010",
},
components: {
  Steps,
  Button: {
    variants: {
      brand: {
        bg: "brand.500",
        color: "black",
        _hover: {
          bg: "brand.600",
        },
      },
    },
  },
  Input: {
    defaultProps: {
      focusBorderColor: "brand.500",
    },
  },
  Textarea: {
    defaultProps: {
      focusBorderColor: "brand.500",
    },
  },
},
styles: {
```

```
    global: (props: any) => ({
      body: {
        bg: "black",
      },
    }),
  },
});

export default theme;
```

## apps/web/src/pages/404.tsx

```
/**
 * src/pages/404.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 24.06.2022
 *
 */

import * as React from "react";
import { Button, Flex, Heading } from "@chakra-ui/react";
import Link from "next/link";
import Image from "next/image";
import ErrorComponent from "@codeup/components/global/ErrorComponent";
import Head from "next/head";

export default function NotFound() {
  return (
    <>
      <Head>
        <title>Seite nicht gefunden - CodeUp</title>
      </Head>
      <ErrorComponent
        description={"Die angeforderte Seite konnte nicht gefunden werden."}
        title={"Seite nicht gefunden"}
        image={"/_static/vectors/error/404_v2.svg"}
      />
    </>
  );
}
```

## apps/web/src/pages/500.tsx

```
/**
 * src/pages/505.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 24.06.2022
 *
 */

import * as React from "react";
import { Flex, Heading } from "@chakra-ui/react";
import Image from "next/image";
import ErrorComponent from "@codeup/components/global/ErrorComponent";
import Head from "next/head";
```

```
export default function NotFound() {
  return (
    <>
      <Head>
        <title>Interner Server Fehler - CodeUp</title>
      </Head>
      <ErrorComponent
        description={
          "Die Seite kann aufgrund eines Fehlers nicht angezeigt werden."
        }
        title={"Interner Server Fehler"}
        image={"/_static/vectors/error/500_v2.svg"}
      />
    </>
  );
}
```

## apps/web/src/pages/account/reset/[token].tsx

```
/**
 * src/pages/account/reset-confirm.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 06.06.2022
 *
 */

import * as React from "react";
import { useEffect } from "react";
import {
  Alert,
  AlertDescription,
  AlertIcon,
  Box,
  Button,
  Flex,
  FormControl,
  FormErrorMessage,
  FormLabel,
  Heading,
  Input,
  InputGroup,
  InputRightElement,
  Text,
  useColorModeValue,
} from "@chakra-ui/react";
import { AiFillEye, AiFillEyeInvisible } from "react-icons/ai";
import { useRouter } from "next/router";
import { default as redirect } from "@codeup/util/src/userworkflow/redirect";
import REST from "@codeup/rest";
import Head from "next/head";
import { CHeading } from "@codeup/components/dist/global/CHeading";

export default function Token() {
  const router = useRouter();

  const { token } = router.query;

  const [loading, setLoading] = React.useState(false);
```

```
const [alertVisible, setAlertVisible] = React.useState(false);
const [alertMessage, setAlertMessage] = React.useState("");
const [alertType, setAlertType] = React.useState<
  "error" | "info" | "success" | "warning"
>("error");

const [passwordVisible, setPasswordVisible] = React.useState(false);
const [passwordConfirmVisible, setPasswordConfirmVisible] =
  React.useState(false);

const [passwordInvalid, setPasswordInvalid] = React.useState(false);

const [password, setPassword] = React.useState("");

const [disabled, setDisabled] = React.useState(false);

useEffect(() => {
  if (token === undefined) {
    return;
  }

  REST.Account.checkResetToken(token as string).then((res) => {
    if (res.status !== 200) {
      setAlertVisible(true);
      setAlertMessage(res.payload.error);
      setAlertType("error");
      setDisabled(true);
    }
  });
}, [token]);

useEffect(() => {
  redirect({
    allowAnonymous: true,
    redirectTo: "/dashboard",
    redirectToLogin: false,
    forceAnonymous: true,
  });
}, []);

return (
  <>
    <Head>
      <title>Passwort zutücksetzen - CodeUp</title>
    </Head>
    {alertVisible ? (
      <Alert status={alertType} variant={"top-accent"}>
        <AlertIcon />
        <AlertDescription>{alertMessage}</AlertDescription>
      </Alert>
    ) : null}
    <Flex
      alignItems={"center"}
      justifyContent={"center"}
      minW={"100%"}
      minH={"100vh"}
      padding={"1.2rem"}
      bg={"#000"}
    >
      <Box
```

```
  maxW={["100%", "50%"]}
  minW={["100%", "50%"]}
  bgColor={useColorModeValue("gray.400", "gray.900")}
  padding={"4rem"}
  rounded={"12px"}
  shadow={"xl"}
>
  <Flex direction={"column"} gap={"2rem"}>
    <Flex
      alignItems={"center"}
      justifyContent={"center"}
      direction={"column"}
    >
      <CHeading size={"xl"}>Passwort zurücksetzen</CHeading>
      <Text textAlign={"center"} mt={".4rem"}>
        Gib ein neues Passwort ein.
      </Text>
    </Flex>
    <form
      onSubmit={async (e) => {
        e.preventDefault();
        setLoading(true);

        const res = await REST.Account.resetConfirm({
          resetToken: token as string,
          // @ts-ignore
          password: e.target.elements.password.value,
        });

        setLoading(false);
        if (res.status === 200) {
          setAlertVisible(true);
          setAlertMessage(res.payload.message);
          setAlertType("success");
          setDisabled(true);
        } else {
          setAlertVisible(true);
          setAlertMessage(res.payload.error);
          setAlertType("error");
        }
      }}
    >
      <FormControl isInvalid={passwordInvalid}>
        <FormLabel htmlFor={"password"}>Passwort</FormLabel>
        <InputGroup>
          <Input
            id={"password"}
            placeholder={"Passwort"}
            disabled={disabled || loading}
            type={passwordVisible ? "text" : "password"}
            onChange={(e) => {
              if (
                e.target.value.match(
                  /^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@$!%*?&])[A-Za-z\d@$!%*?&]{8,}$/
                )
              ) {
                setPasswordInvalid(false);
                setPassword(e.target.value);
              } else {
                setPasswordInvalid(true);
```

```
                }
              }}
            />
            <InputRightElement
              cursor={"pointer"}
              onClick={() => {
                setPasswordVisible(!passwordVisible);
              }}
            >
              {passwordVisible ? <AiFillEyeInvisible /> : <AiFillEye />}
            </InputRightElement>
          </InputGroup>
          <FormErrorMessage>
            Bitte gib ein Passwort ein, welches aus mindestens 8
            Buchstaben, einen Großbuchstaben, einen Kleinbuchstaben eine
            Ziffer und ein Sonderzeichen besteht.
          </FormErrorMessage>
        </FormControl>
        <Button
          variant={"brand"}
          type={"submit"}
          isLoading={loading}
          disabled={disabled || loading}
          w={"100%"}
          mt={"2rem"}
        >
          Passwort zurücksetzen
        </Button>
      </form>
    </Flex>
  </Box>
 </Flex>
 </>
 );
}
```

## apps/web/src/pages/account/settings.tsx

```
/**
 * src/pages/settings.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 06.06.2022
 *
 */

import {
  Box,
  Button,
  Flex,
  FormControl,
  FormErrorMessage,
  FormLabel,
  Heading,
  Input,
  Link,
  Popover,
  PopoverArrow,
  PopoverBody,
```

```
  PopoverCloseButton,
  PopoverContent,
  PopoverHeader,
  PopoverTrigger,
  SimpleGrid,
  Spinner,
  Stack,
  Text,
  useColorModeValue,
  useDisclosure,
  useToast,
} from "@chakra-ui/react";
import { useLoggedIn } from "@codeup/hooks";
import REST from "@codeup/rest";
import { default as redirect } from "@codeup/util/src/userworkflow/redirect";
import { useRouter } from "next/router";
import * as React from "react";
import { useEffect } from "react";
import { BsFillHouseDoorFill } from "react-icons/bs";
import {
  FaCrown,
  FaInfo,
  FaLock,
  FaPen,
  FaTimes,
  FaTrash,
  FaUnlock,
  FaUserCircle,
} from "react-icons/fa";
import Head from "next/head";
import CodeUpModal from "@codeup/components/dist/codeup/CodeUpModal";
import GridLayout from "@codeup/components/global/GridLayout";
import * as qr from "qrcode";
import PopupManager from "../../global/PopupManager";
import { IoWarning } from "react-icons/io5";
import AccountManager from "@codeup/util/dist/AccountManager";
import { CHeading } from "@codeup/components/dist/global/CHeading";

export default function Settings() {
  const router = useRouter();
  const { userInfo, loggedIn, loaded } = useLoggedIn();

  useEffect(() => {
    if (loaded && !loggedIn) {
      window.dispatchEvent(new CustomEvent("codeup::open_login"));
      router.push("/#login");
    }
  }, [loaded, userInfo, loggedIn]);

  const [currentComponent, setCurrentComponent] =
    React.useState<JSX.Element | null>(
      <>
        <Spinner size="xl" />
      </>
    );

  const [currentSelected, setCurrentSelected] = React.useState<string>("home");

  const [twoFactorURL, setTwoFactorURL] = React.useState<string | null>(null);
```

```
const {
  onOpen: on2FASuccessOpen,
  isOpen: is2FASuccessOpen,
  onClose: on2FASuccessClose,
} = useDisclosure();

return (
  <>
    <Head>
      <title>Einstellungen - CodeUp</title>
    </Head>

    <Box bg={"#000"} h={"fit-content"} minH={"100vh"}>
      <Flex pt={6} pb={6} w={"100%"} justifyContent={"center"}>
        <CHeading size={"xl"}>Konto Einstellungen</CHeading>
      </Flex>

      <Flex alignItems={"center"} justifyContent={"center"}>
        <Box
          maxW={["90%", "90%", "60%"]}
          bg={"gray.900"}
          p={4}
          rounded={"xl"}
          minW={["100px", "200px", "400px"]}
        >
          <Stack gap={2}>
            <FormControl>
              <FormLabel>Vorname</FormLabel>
              <Input
                id={"sett.firstname"}
                placeholder={"Vorname"}
                defaultValue={userInfo.firstName}
              />
            </FormControl>
            <Button
              variant={"brand"}
              onClick={async () => {
                const newName = (
                  document.getElementById(
                    "sett.firstname"
                  ) as HTMLInputElement
                ).value;

                if (!newName) return;

                const res = await REST.Account.update({
                  keyName: "firstName",
                  keyValue: newName,
                  token: localStorage.getItem("token") as string,
                });

                if (res.payload.success) {
                  localStorage.setItem("token", res.payload.newToken);
                  PopupManager.alert({
                    title: "Erfolgreich",
                    description: "Dein Vorname wurde erfolgreich geändert!",
                  });
                } else {
                  PopupManager.alert({
                    title: "Fehler",
```

```
          description: res.payload.message,
        });
      }
    }}
  >
    Speichern
  </Button>
</Stack>
<Stack gap={2} mt={2}>
  <FormControl>
    <FormLabel>Nachname</FormLabel>
    <Input
      id={"sett.lastname"}
      placeholder={"Nachname"}
      defaultValue={userInfo.lastName}
    />
  </FormControl>
  <Button
    variant={"brand"}
    onClick={async () => {
      const newName = (
        document.getElementById("sett.lastname") as HTMLInputElement
      ).value;

      if (!newName) return;

      const res = await REST.Account.update({
        keyName: "lastName",
        keyValue: newName,
        token: localStorage.getItem("token") as string,
      });

      if (res.payload.success) {
        localStorage.setItem("token", res.payload.newToken);
        PopupManager.alert({
          title: "Erfolgreich",
          description: "Dein Nachname wurde erfolgreich geändert!",
        });
      } else {
        PopupManager.alert({
          title: "Fehler",
          description: res.payload.message,
        });
      }
    }}
  >
    Speichern
  </Button>
</Stack>
<Stack gap={2} mt={2}>
  <FormControl>
    <FormLabel>E-Mail</FormLabel>
    <Input
      id={"sett.email"}
      placeholder={"E-Mail"}
      defaultValue={userInfo.email}
    />
  </FormControl>
  <Button
    variant={"brand"}
```

```
            onClick={async () => {
              const newMail = (
                document.getElementById("sett.lastname") as HTMLInputElement
              ).value;
              if (
                !newMail.match(
                  /^(([^<>()\[\]\\.,;:\s@"]+(\.[^<>()\[\]\\.,;:\s@"]+)*)|(".+"))@((\[[0-9]{1,3}\.[0-9]{1,3}\.[0-
9]{1,3}\.[0-9]{1,3}\])|(([a-zA-Z\-0-9]+\.)+[a-zA-Z]{2,}))$/
                )
              ) {
                PopupManager.alert({
                  title: "Fehler",
                  description: "Die E-Mail Adresse ist ungültig!",
                });
                return;
              }

              const res = await REST.Account.update({
                keyName: "email",
                keyValue: newMail,
                token: localStorage.getItem("token") as string,
              });

              if (res.payload.success) {
                localStorage.setItem("token", res.payload.newToken);
                PopupManager.alert({
                  title: "Erfolgreich",
                  description:
                    "Überprüfe dein Postfach, um deine neue E-Mail Adresse zu bestätigen!",
                });
              } else {
                PopupManager.alert({
                  title: "Fehler",
                  description: res.payload.message,
                });
              }
            }}
          >
            Speichern
          </Button>
        </Stack>
        <Stack gap={2} mt={2}>
          <FormControl>
            <FormLabel>Zwei Faktor Authentifizierung</FormLabel>
            <Button
              w={"100%"}
              variant={"brand"}
              onClick={async () => {
                // @ts-ignore
                if (!userInfo.twoFactorEnabled) {
                  const res = await REST.Account.update2FA({
                    token: localStorage.getItem("token") as string,
                    enabled: true,
                  });

                  if (res.status === 200) {
                    setTwoFactorURL(
                      (await qr.toCanvas(res.payload.url)).toDataURL()
                    );
```

```
            on2FASuccessOpen();

            localStorage.setItem("token", res.payload.newToken);
          } else {
            PopupManager.alert({
              title: "Fehler",
              description:
                "Zwei Faktor Authentifizierung fehlgeschlagen: " +
                res.payload.error,
            });
          }
        } else {
          const code = await PopupManager.promptAsync({
            title: "Zwei Faktor Authentifizierung deaktivieren",
            helperText:
              "Bitte gebe den Code aus deiner Authentifizierungs App ein",
          });

          const res = await REST.Account.update2FA({
            token: localStorage.getItem("token") as string,
            enabled: false,
            code: code,
          });

          if (res.status === 200) {
            localStorage.setItem("token", res.payload.newToken);
            PopupManager.alert({
              title: "Erfolgreich",
              description:
                "Zwei Faktor Authentifizierung erfolgreich deaktiviert!",
            });
          } else {
            PopupManager.alert({
              title: "Fehler",
              description:
                "Zwei Faktor Authentifizierung fehlgeschlagen: " +
                res.payload.error,
            });
          }
        }
      }}
    >
      {
        // @ts-ignore
        userInfo.twoFactorEnabled ? "Deaktivieren" : "Aktivieren"
      }
    </Button>
  </FormControl>
</Stack>
<Stack gap={2} mt={2}>
  <FormControl>
    <FormLabel>Passwort</FormLabel>
    <Button
      w={"100%"}
      variant={"brand"}
      onClick={async () => {
        const newPassword = await PopupManager.promptAsync({
          title: "Passwort ändern",
          helperText: "Bitte gebe dein neues Passwort ein",
        });
```

```
      if (!newPassword) return;

      const newPasswordRepeat = await PopupManager.promptAsync({
        title: "Passwort ändern",
        helperText: "Bitte wiederhole dein neues Passwort",
      });

      if (!newPasswordRepeat) return;

      if (newPassword !== newPasswordRepeat) {
        PopupManager.alert({
          title: "Fehler",
          description: "Die Passwörter stimmen nicht überein!",
        });
        return;
      }

      const res = await REST.Account.update({
        keyName: "password",
        keyValue: newPassword,
        token: AccountManager.getActiveToken(),
      });

      if (res.status === 200) {
        PopupManager.alert({
          title: "Erfolgreich",
          description: "Passwort erfolgreich geändert!",
        });
        return;
      }

      PopupManager.alert({
        title: "Fehler",
        description:
          "Passwort konnte nicht geändert werden: " +
          res.payload.error,
      });
    }}
  >
    Passwort ändern
  </Button>
 </FormControl>
</Stack>
<Stack gap={2} mt={2}>
 <FormControl>
  <FormLabel>
   CodeUp PRO (
   {userInfo.subscription === "PRO" ? "Aktiv" : "Inaktiv"})
  </FormLabel>
  <Button
   w={"100%"}
   variant={"brand"}
   onClick={async () => {
    if (userInfo.subscription === "PRO") {
     if (
      !(await PopupManager.confirmAsync({
       title: "Kündigen",
       question:
        "Willst du CodeUp PRO wirklich kündigen und all deine Vorteile verlieren?",
```

```
                  }))
                ) {
                  return;
                }

                const res = await REST.Payments.cancelSubscription(
                  AccountManager.getActiveToken()
                );
                if (res.status === 200) {
                  await PopupManager.alertAsync({
                    title: "Erfolgreich",
                    description: "CodeUp PRO erfolgreich gekündigt!",
                  });
                  AccountManager.overrideActiveToken(res.payload.jwt);
                } else {
                  PopupManager.alert({
                    title: "Fehler",
                    description:
                      "Es ist ein Fehler aufgetreten. Bitte kontaktiere den Support mit dieser
Fehlermeldung: " +
                      res.payload.error,
                  });
                }
              } else {
                const res = await REST.Payments.createCheckoutLink(
                  AccountManager.getActiveToken()
                );

                if (res.status === 200) {
                  window.location.href = res.payload.url;
                } else {
                  PopupManager.alert({
                    title: "Fehler",
                    description:
                      "Es ist ein Fehler aufgetreten. Bitte versuche es später erneut: " +
                      res.payload.error,
                  });
                }
              }
            }}
          >
            {
              // @ts-ignore
              userInfo.subscription === "PRO" ? "Kündigen" : "Aktivieren"
            }
          </Button>
        </FormControl>
      </Stack>
      <Stack gap={2} mt={2}>
        <FormControl>
          <FormLabel>Konto löschen</FormLabel>
          <Text>
            Diese Aktion kann unter keinen Umständen rückgängig gemacht
            werden!
          </Text>
          <Button
            w={"100%"}
            colorScheme={"red"}
            onClick={async () => {
              if (
```

```
                !(await PopupManager.confirmAsync({
                  title: "Konto löschen",
                  question:
                    "Bist du dir sicher, dass du dein Konto löschen möchtest? Diese Aktion kann nicht
rückgängig gemacht werden!",
                }))
              )
                return;

              const res = await REST.Account.deleteAccount(
                localStorage.getItem("token") as string
              );

              localStorage.removeItem("token");
              document.cookie = "jwt=;path=/";

              await router.push("/");
            }}
          >
            KONTO LÖSCHEN
          </Button>
        </FormControl>
      </Stack>
    </Box>
  </Flex>
</Box>

<CodeUpModal
  onOpen={on2FASuccessOpen}
  onClose={on2FASuccessClose}
  isOpen={is2FASuccessOpen}
  title={"2-Faktor-Authentifizierung"}
>
  <Text mb={"2rem"} fontSize={"lg"}>
    Scanne den folgenden QR-Code mit deiner Authentifizierungs-App, um die
    2-Faktor-Authentifizierung zu aktivieren. Wenn du noch keine
    Authentifizierungs-App hast, empfehlen wir dir{" "}
    <Link href={"https://authy.com/"} isExternal color={"blue.400"}>
      Authy
    </Link>
  </Text>
  <Flex alignItems={"center"} w={"100%"} justifyContent={"center"}>
    <img src={twoFactorURL as string} />
  </Flex>
</CodeUpModal>
</>
);
}
```

## apps/web/src/pages/admin/analytics/index.tsx

```
/**
 * apps/web/src/pages/admin/analytics/index.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 25.11.2022
 *
 */
```

```
import * as React from "react";
import { useState } from "react";
import REST from "@codeup/rest";
import {
  Avatar,
  Box,
  Button,
  Flex,
  FormControl,
  FormLabel,
  Grid,
  GridItem,
  Heading,
  HStack,
  Select,
  Stack,
  Table,
  TableContainer,
  Tbody,
  Td,
  Text,
  Th,
  Thead,
  Tr,
  useToast,
} from "@chakra-ui/react";
import AdminLayout from "@codeup/components/admin/AdminLayout";
import {
  ArcElement,
  BarElement,
  CategoryScale,
  Chart as ChartJS,
  Legend,
  LinearScale,
  LineElement,
  PointElement,
  Title,
  Tooltip,
} from "chart.js";
import { Bar, Pie } from "react-chartjs-2";
import { useRouter } from "next/router";
import Head from "next/head";
import SuperText from "@codeup/components/landingpage/SuperText";
import { CHeading } from "@codeup/components/dist/global/CHeading";

export default function Index() {
  ChartJS.register(
    ArcElement,
    Tooltip,
    Legend,
    CategoryScale,
    LinearScale,
    BarElement,
    PointElement,
    LineElement,
    Title
  );
  const [summary, setSummary] = React.useState<{
    browsers: {
      // BROWSER: Percentage
```

```
    [key: string]: number;
  };
  os: {
    // OS: Percentage
    [key: string]: number;
  };
  paths: {
    // PATH: Percentage
    [key: string]: number;
  };
  // Logged in views percentage
  authenticatedViewPercentage: number;
  // Not logged in views percentage
  anonymousViewPercentage: number;
  // Total views
  totalViews: number;
  todayActiveUsers: {
    username: string;
    views: string;
    userId: string;
  }[];
}>();

const toast = useToast();
const router = useRouter();

const [periodOffset, setPeriodOffset] = useState(7);

const [period, setPeriod] = React.useState<{
  start: string;
  end: string;
}>({
  start: new Date(new Date().getTime() - 30 * 24 * 60 * 60 * 1000)
    .toISOString()
    .split("T")[0],
  end: new Date(new Date().getTime() + 24 * 60 * 60 * 1000)
    .toISOString()
    .split("T")[0],
});

const [views, setViews] = useState<{
  [key: string]: number;
}>();

React.useEffect(() => {
  REST.Analytics.getSummary(localStorage.getItem("token") as string).then(
    (res) => {
      if (res.status === 200) {
        setSummary(res.payload.data);
      } else {
        toast({
          title: "Error",
          description:
            "An error occurred while loading the analytics summary",
          status: "error",
          duration: 5000,
          isClosable: true,
        });
      }
    }
```

```
  );
}, []);

React.useEffect(() => {
  const startDate = new Date(
    new Date().getTime() - periodOffset * 24 * 60 * 60 * 1000
  )
    .toISOString()
    .split("T")[0];
  const endDate = new Date(new Date().getTime() + 24 * 60 * 60 * 1000)
    .toISOString()
    .split("T")[0];

  setPeriod({
    start: startDate,
    end: endDate,
  });

  REST.Analytics.getViews({
    token: localStorage.getItem("token") as string,
    periodStart: startDate,
    periodEnd: endDate,
  }).then((res) => {
    if (res.status === 200) {
      // extract every date from period
      const dates = [];

      const nStartDate = new Date(startDate);
      const nEndDate = new Date(endDate);

      while (nStartDate.getTime() <= nEndDate.getTime()) {
        dates.push(nStartDate.toISOString().split("T")[0]);
        nStartDate.setDate(nStartDate.getDate() + 1);
      }

      const _views: {
        [key: string]: number;
      } = {};

      for (let i = 0; i < dates.length; i++) {
        _views[dates[i]] = res.payload.data[i] ? res.payload.data[i] : 0;
      }

      setViews(_views);
      console.log(_views);
    } else {
      toast({
        title: "Error",
        description: res.payload.error,
        status: "error",
        duration: 5000,
        isClosable: true,
      });
    }
  });
}, [toast, periodOffset]);

return (
  <>
    <Head>
```

```
  <title>Analytics - Admin - CodeUp</title>
</Head>
<AdminLayout>
  <SuperText>Analytics</SuperText>
  <Grid
    templateColumns={[
      "repeat(1, 1fr)",
      "repeat(2, 1fr)",
      "repeat(3, 1fr)",
    ]}
    maxW={"100%"}
    overflow={"hidden"}
    gap={[0, "4rem"]}
    p={[0, "2rem"]}
  >
    <GridItem>
      <Box bg={"gray.900"} rounded={["none", "lg"]} h={"100%"} p={6}>
        <CHeading size={"lg"}>Browsers (%)</CHeading>
        <Stack w={"100%"} h={"35vh"} alignItems={"center"}>
          <Pie
            data={{
              labels: Object.keys(summary?.browsers || {}),
              datasets: [
                {
                  label: "Browser",
                  data: Object.values(summary?.browsers || {}),
                  backgroundColor: [
                    "rgba(255, 99, 132, 0.2)",
                    "rgba(54, 162, 235, 0.2)",
                    "rgba(255, 206, 86, 0.2)",
                    "rgba(75, 192, 192, 0.2)",
                    "rgba(153, 102, 255, 0.2)",
                    "rgba(255, 159, 64, 0.2)",
                  ],
                  borderColor: [
                    "rgba(255, 99, 132, 1)",
                    "rgba(54, 162, 235, 1)",
                    "rgba(255, 206, 86, 1)",
                    "rgba(75, 192, 192, 1)",
                    "rgba(153, 102, 255, 1)",
                    "rgba(255, 159, 64, 1)",
                  ],
                  borderWidth: 1,
                },
              ],
            }}
          />
        </Stack>
      </Box>
    </GridItem>
    <GridItem>
      <Box bg={"gray.900"} rounded={["none", "lg"]} h={"100%"} p={6}>
        <CHeading size={"lg"}>Operating Systems (%)</CHeading>
        <Stack w={"100%"} h={"35vh"} alignItems={"center"}>
          <Pie
            data={{
              labels: Object.keys(summary?.os || {}),
              datasets: [
                {
                  label: "OS",
```

```
            data: Object.values(summary?.os || {}),
            backgroundColor: [
              "rgba(255, 99, 132, 0.2)",
              "rgba(54, 162, 235, 0.2)",
              "rgba(255, 206, 86, 0.2)",
              "rgba(75, 192, 192, 0.2)",
              "rgba(153, 102, 255, 0.2)",
              "rgba(255, 159, 64, 0.2)",
            ],
            borderColor: [
              "rgba(255, 99, 132, 1)",
              "rgba(54, 162, 235, 1)",
              "rgba(255, 206, 86, 1)",
              "rgba(75, 192, 192, 1)",
              "rgba(153, 102, 255, 1)",
              "rgba(255, 159, 64, 1)",
            ],
            borderWidth: 1,
          },
        ],
      }}
    />
  </Stack>
 </Box>
</GridItem>
<GridItem>
 <Box bg={"gray.900"} rounded={["none", "lg"]} h={"100%"} p={6}>
  <CHeading size={"lg"}>Views (%)</CHeading>
  <Text>Total Views: {summary?.totalViews}</Text>
  <Stack w={"100%"} h={"35vh"} alignItems={"center"}>
   <Pie
    data={{
     labels: ["Logged in", "Not logged in"],
     datasets: [
      {
       label: "Views",
       data: [
        summary?.authenticatedViewPercentage,
        summary?.anonymousViewPercentage,
       ],
       backgroundColor: [
        "rgba(255, 99, 132, 0.2)",
        "rgba(54, 162, 235, 0.2)",
       ],
       borderColor: [
        "rgba(255, 99, 132, 1)",
        "rgba(54, 162, 235, 1)",
       ],
       borderWidth: 1,
      },
     ],
    }}
   />
  </Stack>
 </Box>
</GridItem>
<GridItem>
 <Box bg={"gray.900"} rounded={["none", "lg"]} h={"100%"} p={6}>
  <CHeading size={"lg"}>Paths (%)</CHeading>
  <Stack w={"100%"} h={"40vh"} alignItems={"center"}>
```

```
      <Pie
        data={{
          labels: pathsToLabels(summary?.paths),
          datasets: [
            {
              label: "Views",
              data: pathsToData(summary?.paths),
              backgroundColor: [
                "rgba(255, 99, 132, 0.2)",
                "rgba(54, 162, 235, 0.2)",
                "rgba(255, 206, 86, 0.2)",
                "rgba(75, 192, 192, 0.2)",
                "rgba(153, 102, 255, 0.2)",
                "rgba(255, 159, 64, 0.2)",
              ],
              borderColor: [
                "rgba(255, 99, 132, 1)",
                "rgba(54, 162, 235, 1)",
                "rgba(255, 206, 86, 1)",
                "rgba(75, 192, 192, 1)",
                "rgba(153, 102, 255, 1)",
                "rgba(255, 159, 64, 1)",
              ],
              borderWidth: 1,
            },
          ],
        }}
      />
    </Stack>
  </Box>
</GridItem>
<GridItem>
  <Box bg={"gray.900"} rounded={["none", "lg"]} h={"100%"} p={6}>
    <HStack gap={6}>
      <CHeading size={"lg"}>Aufrufe (Historie)</CHeading>
      <Select
        value={periodOffset}
        onChange={(e) => {
          // @ts-ignore
          setPeriodOffset(e.target.value as number);
        }}
      >
        <option value={0}>Heute</option>
        <option value={7}>Eine Woche</option>
        <option value={14}>2 Wochen</option>
        <option value={30}>Ein Monat</option>
        <option value={90}>3 Monate</option>
        <option value={180}>6 Monate</option>
        <option value={365}>Ein Jahr</option>
      </Select>
    </HStack>
    <Stack w={"100%"} h={"40vh"} alignItems={"center"}>
      <Bar
        data={{
          // extract every date from period.start to period.end
          labels: Array.from(
            {
              length:
                (new Date(period.end).getTime() -
                  new Date(period.start).getTime()) /
```

```
            (1000 * 60 * 60 * 24),
          },
          (_, i) => {
            const date = new Date(
              new Date(period.start).getTime() +
                i * 1000 * 60 * 60 * 24
            );
            return `${date.getDate()}.${
              date.getMonth() + 1
            }.${date.getFullYear()}`;
          }
        ),
        datasets: [
          {
            label: "Aufrufe",
            data: Array.from(
              {
                length:
                  (new Date(period.end).getTime() -
                    new Date(period.start).getTime()) /
                  (1000 * 60 * 60 * 24),
              },
              (_, i) => {
                const date = new Date(
                  new Date(period.start).getTime() +
                    i * 1000 * 60 * 60 * 24
                );

                // "YEAR-MONTH-DAY"
                const dateString = `${date.getFullYear()}-${
                  date.getMonth() + 1
                }-${date.getDate()}`;

                const parts = dateString.split("-");

                if (parts[0].length === 1) {
                  parts[0] = "0" + parts[0];
                }

                if (parts[1].length === 1) {
                  parts[1] = "0" + parts[1];
                }

                if (parts[2].length === 1) {
                  parts[2] = "0" + parts[2];
                }

                return views ? views[parts.join("-")] : 0;
              }
            ),
            backgroundColor: "rgba(75, 192, 192, 1)",
          },
        ],
      }}
    />
  </Stack>
 </Box>
</GridItem>
<GridItem>
 <Box bg={"gray.900"} rounded={["none", "lg"]} h={"100%"} p={6}>
```

```
    <CHeading size={"lg"}>Aktive Nutzer (24h)</CHeading>
    <Stack
      w={"100%"}
      h={"40vh"}
      alignItems={"center"}
      overflow={"auto"}
    >
      <TableContainer mt={6} w={"100%"}>
        <Table>
          <Thead>
            <Th>Profilbild</Th>
            <Th>Benutername</Th>
            <Th>Aufrufe</Th>
          </Thead>
          <Tbody>
            {summary?.todayActiveUsers.map((user) => (
              <Tr
                key={user.userId}
                _hover={{
                  backgroundColor: "gray.700",
                  cursor: "pointer",
                }}
                onClick={() => {
                  router.push(`/admin/analytics/${user.userId}`);
                }}
              >
                <Td>
                  <Avatar
                    size={"lg"}
                    src={`/api/account/${user.username}/picture`}
                  />
                </Td>
                <Td>@{user.username}</Td>
                <Td>{user.views}</Td>
              </Tr>
            ))}
          </Tbody>
        </Table>
      </TableContainer>
    </Stack>
  </Box>
</GridItem>
<GridItem>
  <Box bg={"gray.900"} rounded={["none", "lg"]} h={"100%"} p={6}>
    <CHeading size={"lg"}>Daten löschen</CHeading>
    <Text>
      Du kannst die Analylse Daten für einen bestimmten Zeitraum
      löschen. Diese können allerdings nicht wiederhergestellt werden.
    </Text>

    <form
      onSubmit={async (e) => {
        e.preventDefault();

        const deleteDataPeriodInput = (
          (e.target as HTMLFormElement).elements.namedItem(
            "delete-data-period"
          ) as HTMLInputElement
        ).value as unknown as number;
```

```
              REST.Analytics.deleteAnalytics({
                token: localStorage.getItem("token") as string,
                periodStart: new Date(
                  new Date().getTime() -
                    deleteDataPeriodInput * 24 * 60 * 60 * 1000
                )
                  .toISOString()
                  .split("T")[0],
                periodEnd: new Date(
                  new Date().getTime() + 24 * 60 * 60 * 1000
                )
                  .toISOString()
                  .split("T")[0],
              }).then((res) => {
                router.reload();
              });
            }}
          >
            <FormControl id={"delete-period"}>
              <FormLabel>Zeitraum</FormLabel>
              <Select name={"delete-data-period"} defaultValue={7}>
                <option value={0}>Heute</option>
                <option value={7}>Eine Woche</option>
                <option value={14}>2 Wochen</option>
                <option value={30}>Ein Monat</option>
                <option value={90}>3 Monate</option>
                <option value={180}>6 Monate</option>
                <option value={365}>Ein Jahr</option>
                <option value={420000}>Alles</option>
              </Select>
            </FormControl>
            <Button mt={"1rem"} colorScheme={"red"} type={"submit"}>
              Analytics löschen
            </Button>
          </form>
        </Box>
      </GridItem>
    </Grid>
  </AdminLayout>
 </>
 );
}

function pathsToLabels(_paths: any) {
 return Object.keys(_paths || {})
  .sort((a, b) => {
   return (_paths as any)[b] - (_paths as any)[a];
  })
  .slice(0, 5)
  .concat(["Other"]);
}

function pathsToData(_paths: any) {
 return Object.values(
  Object.fromEntries(
   Object.entries(_paths || {})
    // @ts-ignore
    .sort((a, b) => b[1] - a[1])
    .slice(0, 5)
    .concat([
```

```
          [
            "Other",
            Object.keys(_paths || {})
              .sort((a, b) => {
                return (_paths as any)[b] - (_paths as any)[a];
              })
              .slice(5, Object.keys(_paths || {}).length)
              .map((e) => (_paths as any)[e])
              .reduce((a, b) => a + b, 0),
          ],
        ])
    )
  );
}
```

## apps/web/src/pages/admin/analytics/[id].tsx

```
/**
 * apps/web/src/pages/admin/analytics/[id].tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 07.04.2023
 *
 */

import * as React from "react";
import AdminLayout from "@codeup/components/dist/admin/AdminLayout";
import SuperText from "@codeup/components/dist/landingpage/SuperText";
import { useRouter } from "next/router";
import REST from "@codeup/rest/src";
import AccountManager from "@codeup/util/dist/AccountManager";
import AuthenticateLoadingIndicator from
"@codeup/components/dist/admin/AuthenticateLoadingIndicator";
import {
  Box,
  Heading,
  Flex,
  Text,
  chakra,
  Grid,
  GridItem,
  Accordion,
  AccordionPanel,
  AccordionItem,
  AccordionButton,
  AccordionIcon,
  Stack,
  TableContainer,
  Table,
  Thead,
  Tr,
  Th,
  Tbody,
  Td,
  InputGroup,
  InputLeftElement,
  Input,
  TableCaption,
} from "@chakra-ui/react";
```

```
import { FaSearch } from "react-icons/fa";
import {
  ArcElement,
  BarElement,
  CategoryScale,
  Chart as ChartJS,
  Legend,
  LinearScale,
  LineElement,
  PointElement,
  Title,
  Tooltip,
} from "chart.js";
import { Bar, Pie } from "react-chartjs-2";
import { CHeading } from "@codeup/components/dist/global/CHeading";

export default function UserViewer() {
  const [user, setUser] = React.useState<{
    id: string;
    username: string;
    firstName: string;
    lastName: string;
    profilePicture: string;
  }>();

  ChartJS.register(
    ArcElement,
    Tooltip,
    Legend,
    CategoryScale,
    LinearScale,
    BarElement,
    PointElement,
    LineElement,
    Title
  );

  const [fullAnalytics, setFullAnalytics] = React.useState<
    {
      _id: string;
      loggedIn: boolean;
      userId: string;
      path: string;
      createdAt: string;
      browser: string;
      os: string;
      __v: number;
    }[]
  >([]);

  const [calculatedOverview, setCalculatedOverview] = React.useState<{
    first: {
      view: string;
      page: string;
      browser: string;
      os: string;
    };
    last: {
      view: string;
      page: string;
```

```
    browser: string;
    os: string;
  };
  totalViews: number;
  averageViewsPerDay: number;
}>();

const [summary, setSummary] = React.useState<{
  browsers: {
    // BROWSER: Percentage
    [key: string]: number;
  };
  os: {
    // OS: Percentage
    [key: string]: number;
  };
  paths: {
    // PATH: Percentage
    [key: string]: number;
  };
  // Logged in views percentage
  authenticatedViewPercentage: number;
  // Not logged in views percentage
  anonymousViewPercentage: number;
  // Total views
  totalViews: number;
}>();

const router = useRouter();
const [filteredAnalytics, setFilteredAnalytics] = React.useState<
  {
    _id: string;
    loggedIn: boolean;
    userId: string;
    path: string;
    createdAt: string;
    browser: string;
    os: string;
    __v: number;
  }[]
>([]);

const [loaded, setLoaded] = React.useState(false);

React.useEffect(() => {
  if (router && router.query && router.query.id) {
    REST.Account.getUserById(router.query.id as string).then(async (res) => {
      setUser(res.payload.user);

      const fullAnalyticsRes = await REST.Admin.getUserAnalytics({
        token: AccountManager.getActiveToken(),
        userId: router.query.id as string,
        detailed: true,
      });

      if (fullAnalyticsRes.status === 200) {
        setFullAnalytics(fullAnalyticsRes.payload.analytics);
        setFilteredAnalytics(fullAnalyticsRes.payload.analytics);
      }
```

```
      const calculatedOverviewRes = await REST.Admin.getUserAnalytics({
        token: AccountManager.getActiveToken(),
        userId: router.query.id as string,
        detailed: false,
      });

      if (calculatedOverviewRes.status === 200) {
        setCalculatedOverview(calculatedOverviewRes.payload as any);
      }

      const useSummaryRes = await REST.Analytics.getSummary(
        AccountManager.getActiveToken(),
        router.query.id as string
      );

      if (useSummaryRes.status === 200) {
        setSummary(useSummaryRes.payload.data);
      }

      setLoaded(true);
    });
  }
}, [router]);

return (
  <>
    <AdminLayout>
      <SuperText>Analytics</SuperText>
      {!loaded || !user ? (
        <>
          <AuthenticateLoadingIndicator
            text={"Informationen werden gesammelt..."}
          />
        </>
      ) : (
        <>
          <Box w={"100%"}>
            <Box m={4} p={4}>
              <Flex justifyContent={"center"} mt={4}>
                <chakra.img
                  src={user.profilePicture}
                  alt={"Profilbild"}
                  borderRadius={"full"}
                  w={"100px"}
                  h={"100px"}
                />
              </Flex>
              <Flex
                justifyContent={"center"}
                mt={4}
                direction={"column"}
                alignItems={"center"}
              >
                <Heading>@{user.username}</Heading>
                <Text>
                  {user.firstName} {user.lastName}
                </Text>
              </Flex>
              {calculatedOverview && filteredAnalytics ? (
                <>
```

```
<Flex
  w={"100%"}
  alignItems={"center"}
  justifyContent={"center"}
>
  <Grid
    templateColumns={"repeat(2, 1fr)"}
    gap={6}
    mt={4}
    p={4}
    bg={"gray.900"}
    borderRadius={"md"}
    w={"50%"}
  >
    <GridItem>
      <Heading size={"md"}>Erster Besuch</Heading>
      <Text>
        Am 
        {new Date(calculatedOverview.first.view)
          .toLocaleString()
          .replace(", ", " um ") + " Uhr"}
      </Text>
      <Text>
        <code>{calculatedOverview.first.page}</code>{" "}
        auf 
        {calculatedOverview.first.browser} mit 
        {calculatedOverview.first.os}
      </Text>
    </GridItem>
    <GridItem>
      <Heading size={"md"}>Letzter Besuch</Heading>
      <Text>
        Am 
        {new Date(calculatedOverview.last.view)
          .toLocaleString()
          .replace(", ", " um ") + " Uhr"}
      </Text>
      <Text>
        <code>{calculatedOverview.last.page}</code>{" "}
        auf 
        {calculatedOverview.last.browser} mit 
        {calculatedOverview.last.os}
      </Text>
    </GridItem>
    <GridItem>
      <Heading size={"md"}>Seitenaufrufe</Heading>
      <Text>{calculatedOverview.totalViews}</Text>
    </GridItem>
    <GridItem>
      <Heading size={"md"}>
        &#216; Seitenaufrufe / Tag
      </Heading>
      <Text>
        {Math.round(calculatedOverview.averageViewsPerDay)}
      </Text>
    </GridItem>
  </Grid>
</Flex>
<Grid
  templateColumns={["repeat(1, 1fr)", "repeat(2, 1fr)"]}
```

```
  p={"2rem"}
  gap={"4rem"}
>
  <GridItem>
    <Box
      bg={"gray.900"}
      rounded={["none", "lg"]}
      h={"100%"}
      p={6}
    >
      <CHeading size={"lg"}>Browser (%)</CHeading>
      <Stack w={"100%"} h={"35vh"} alignItems={"center"}>
        <Pie
          data={{
            labels: Object.keys(summary?.browsers || {}),
            datasets: [
              {
                label: "Browser",
                data: Object.values(
                  summary?.browsers || {}
                ),
                backgroundColor: [
                  "rgba(255, 99, 132, 0.2)",
                  "rgba(54, 162, 235, 0.2)",
                  "rgba(255, 206, 86, 0.2)",
                  "rgba(75, 192, 192, 0.2)",
                  "rgba(153, 102, 255, 0.2)",
                  "rgba(255, 159, 64, 0.2)",
                ],
                borderColor: [
                  "rgba(255, 99, 132, 1)",
                  "rgba(54, 162, 235, 1)",
                  "rgba(255, 206, 86, 1)",
                  "rgba(75, 192, 192, 1)",
                  "rgba(153, 102, 255, 1)",
                  "rgba(255, 159, 64, 1)",
                ],
                borderWidth: 1,
              },
            ],
          }}
        />
      </Stack>
    </Box>
  </GridItem>
  <GridItem>
    <Box
      bg={"gray.900"}
      rounded={["none", "lg"]}
      h={"100%"}
      p={6}
    >
      <CHeading size={"lg"}>Betriebssysteme (%)</CHeading>
      <Stack w={"100%"} h={"35vh"} alignItems={"center"}>
        <Pie
          data={{
            labels: Object.keys(summary?.os || {}),
            datasets: [
              {
                label: "OS",
```

```
              data: Object.values(summary?.os || {}),
              backgroundColor: [
                "rgba(255, 99, 132, 0.2)",
                "rgba(54, 162, 235, 0.2)",
                "rgba(255, 206, 86, 0.2)",
                "rgba(75, 192, 192, 0.2)",
                "rgba(153, 102, 255, 0.2)",
                "rgba(255, 159, 64, 0.2)",
              ],
              borderColor: [
                "rgba(255, 99, 132, 1)",
                "rgba(54, 162, 235, 1)",
                "rgba(255, 206, 86, 1)",
                "rgba(75, 192, 192, 1)",
                "rgba(153, 102, 255, 1)",
                "rgba(255, 159, 64, 1)",
              ],
              borderWidth: 1,
            },
          ],
        }}
      />
    </Stack>
  </Box>
</GridItem>
<GridItem>
  <Box
    bg={"gray.900"}
    rounded={["none", "lg"]}
    h={"100%"}
    p={6}
  >
    <CHeading size={"lg"}>Seiten (%)</CHeading>
    <Stack w={"100%"} h={"35vh"} alignItems={"center"}>
      <Pie
        data={{
          labels: pathsToLabels(summary?.paths),
          datasets: [
            {
              label: "Views",
              data: pathsToData(summary?.paths),
              backgroundColor: [
                "rgba(255, 99, 132, 0.2)",
                "rgba(54, 162, 235, 0.2)",
                "rgba(255, 206, 86, 0.2)",
                "rgba(75, 192, 192, 0.2)",
                "rgba(153, 102, 255, 0.2)",
                "rgba(255, 159, 64, 0.2)",
              ],
              borderColor: [
                "rgba(255, 99, 132, 1)",
                "rgba(54, 162, 235, 1)",
                "rgba(255, 206, 86, 1)",
                "rgba(75, 192, 192, 1)",
                "rgba(153, 102, 255, 1)",
                "rgba(255, 159, 64, 1)",
              ],
              borderWidth: 1,
            },
          ],
```

```
        }}
      />
    </Stack>
  </Box>
</GridItem>
<GridItem>
  <Box
    bg={"gray.900"}
    rounded={["none", "lg"]}
    h={"100%"}
    p={6}
  >
    <Accordion allowToggle mt={8} mb={4}>
      <AccordionItem>
        <AccordionButton>
          <Box flex="1" textAlign="left">
            Alle Seitenaufrufe
          </Box>
          <AccordionIcon />
        </AccordionButton>
        <AccordionPanel>
          <InputGroup>
            <InputLeftElement>
              <FaSearch />
            </InputLeftElement>
            <Input
              placeholder={"Suche"}
              onChange={(e) => {
                const filter = e.target.value;

                setFilteredAnalytics(
                  fullAnalytics.filter((a) => {
                    if (filter === "") return true;
                    return (
                      a.path
                        .toLowerCase()
                        .includes(filter.toLowerCase()) ||
                      a.browser
                        .toLowerCase()
                        .includes(filter.toLowerCase()) ||
                      a.os
                        .toLowerCase()
                        .includes(filter.toLowerCase())
                    );
                  })
                );
              }}
            />
          </InputGroup>
          <TableContainer>
            <Table>
              <Thead>
                <Tr>
                  <Th>Zeitpunkt</Th>
                  <Th>Seite</Th>
                  <Th>Browser</Th>
                  <Th>Betriebssystem</Th>
                </Tr>
              </Thead>
              <Tbody>
```

```
                                {filteredAnalytics
                                  .sort((a, b) => {
                                    return (
                                      new Date(b.createdAt).getTime() -
                                      new Date(a.createdAt).getTime()
                                    );
                                  })
                                  .map((analytics) => {
                                    return (
                                      <>
                                        <Tr>
                                          <Td>
                                            {new Date(analytics.createdAt)
                                              .toLocaleString()
                                              .replace(", ", " um ") +
                                              " Uhr"}
                                          </Td>
                                          <Td>
                                            <code>{analytics.path}</code>
                                          </Td>
                                          <Td>{analytics.browser}</Td>
                                          <Td>{analytics.os}</Td>
                                        </Tr>
                                      </>
                                    );
                                  })}
                              </Tbody>
                              <TableCaption>
                                {filteredAnalytics.length} Einträge
                              </TableCaption>
                            </Table>
                          </TableContainer>
                        </AccordionPanel>
                      </AccordionItem>
                    </Accordion>
                  </Box>
                </GridItem>
              </Grid>
            </>
          ) : null}
        </Box>
      </Box>
    </>
  )}
</AdminLayout>
</>
);
}

function pathsToLabels(_paths: any) {
  return Object.keys(_paths || {})
    .sort((a, b) => {
      return (_paths as any)[b] - (_paths as any)[a];
    })
    .slice(0, 5)
    .concat(["Other"]);
}

function pathsToData(_paths: any) {
  return Object.values(
```

```
    Object.fromEntries(
      Object.entries(_paths || {})
        // @ts-ignore
        .sort((a, b) => b[1] - a[1])
        .slice(0, 5)
        .concat([
          [
            "Other",
            Object.keys(_paths || {})
              .sort((a, b) => {
                return (_paths as any)[b] - (_paths as any)[a];
              })
              .slice(5, Object.keys(_paths || {}).length)
              .map((e) => (_paths as any)[e])
              .reduce((a, b) => a + b, 0),
          ],
        ])
    )
  );
}
```

## apps/web/src/pages/admin/blog.tsx

```
/**
 * apps/web/src/pages/admin/blog.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 27.12.2022
 *
 */

import * as React from "react";
import AdminLayout from "@codeup/components/dist/admin/AdminLayout";
import {
  Flex,
  Grid,
  GridItem,
  Heading,
  Text,
  useDisclosure,
  useToast,
  chakra,
  Stack,
  FormControl,
  FormLabel,
  Input,
  Tag,
  TagLabel,
  TagLeftIcon,
  TagRightIcon,
  TagCloseButton,
  InputGroup,
  InputRightElement,
  HStack,
  ButtonGroup,
  Button,
  Box,
} from "@chakra-ui/react";
import REST from "@codeup/rest";
```

```
import ErrorComponent from "@codeup/components/dist/global/ErrorComponent";
import {
  FaClock,
  FaEdit,
  FaFile,
  FaPlus,
  FaTags,
  FaTrash,
  FaUser,
} from "react-icons/fa";
import CodeUpModal from "@codeup/components/dist/codeup/CodeUpModal";
import Editor from "@monaco-editor/react";
import SuperText from "@codeup/components/dist/landingpage/SuperText";
import GridLayoutV2 from "@codeup/components/dist/global/GridLayoutV2";

export default function Blog() {
  const [blogs, setBlogs] = React.useState<
    {
      _id: string;
      title: string;
      content: string;
      tags: string[];
      published_at: string;
      authorUsername: string;
    }[]
  >([]);

  const [filteredBlogs, setFilteredBlogs] = React.useState<typeof blogs>([]);

  const toast = useToast();
  const [error, setError] = React.useState(false);
  const [query, setQuery] = React.useState("");

  React.useEffect(() => {
    reload();
  }, []);

  const reload = () => {
    REST.Admin.getBlogs({
      token: localStorage.getItem("token") as string,
    }).then((res) => {
      if (res.status === 200) {
        setBlogs(res.payload.blogs);
        setFilteredBlogs(
          res.payload.blogs.filter(
            (b: any) => b.title.includes(query) || b.content.includes(query)
          )
        );
      } else {
        setError(true);
      }
    });
  };

  React.useEffect(() => {
    let q = query;
    q = q.trim();
    setFilteredBlogs(
      blogs.filter(
        (b: any) =>
```

```
        b.title.toLowerCase().includes(q.toLowerCase()) ||
        b.content.toLowerCase().includes(q.toLowerCase()) ||
        b.tags.join(" ").toLowerCase().includes(q.toLowerCase())
    )
  );
}, [query]);

const { isOpen, onOpen, onClose } = useDisclosure();

const [currentBlog, setCurrentBlog] = React.useState<{
  _id: string;
  title: string;
  content: string;
  tags: string[];
  published_at: string;
  authorUsername: string;
} | null>(null);

return (
  <>
    <AdminLayout>
      {error ? (
        <ErrorComponent description={"Es ist ein Fehler aufgetreten!"} />
      ) : (
        <>
          <CodeUpModal
            onOpen={onOpen}
            onClose={onClose}
            isOpen={isOpen}
            title={currentBlog?.title || "Neuer Blog"}
            noCloseButton
          >
            <Stack spacing={4}>
              <FormControl>
                <FormLabel>Titel</FormLabel>
                <Input
                  type={"text"}
                  placeholder={"Titel des Blogs"}
                  value={currentBlog?.title}
                  onChange={(e) => {
                    if (currentBlog) {
                      setCurrentBlog({
                        ...currentBlog,
                        title: e.target.value as string,
                      });
                    }
                  }}
                />
              </FormControl>
              <FormControl>
                <FormLabel>Tags</FormLabel>
                <HStack mb={4} mt={4} overflow={"scroll"} w={"fit-content"}>
                  {currentBlog?.tags.map((tag) => {
                    return (
                      <Tag
                        size={"md"}
                        key={tag}
                        borderRadius={"full"}
                        variant={"solid"}
                        colorScheme={"brand"}
```

```
        >
          <TagLabel>{tag.toUpperCase()}</TagLabel>
          <TagCloseButton
            onClick={() => {
              setCurrentBlog({
                ...currentBlog,
                tags: currentBlog.tags.filter((t) => t !== tag),
              });
            }}
          />
        </Tag>
      );
    })}
  </HStack>
  <InputGroup>
    <Input
      type={"text"}
      placeholder={"Tag hinzufügen"}
      onKeyDown={(e) => {
        if (e.key === "Enter") {
          if (currentBlog) {
            if (
              !currentBlog.tags.includes(
                e.currentTarget.value as string
              )
            ) {
              setCurrentBlog({
                ...currentBlog,
                tags: [
                  ...currentBlog.tags,
                  e.currentTarget.value,
                ],
              });

              e.currentTarget.value = "";
            }
          }
        }
      }}
    />
  </InputGroup>
</FormControl>
<FormControl data-color-mode={"dark"}>
  <FormLabel>Inhalt</FormLabel>
  <Editor
    height={"50vh"}
    language={"markdown"}
    theme={"vs-dark"}
    value={currentBlog?.content}
    onChange={(value) => {
      if (currentBlog) {
        setCurrentBlog({
          ...currentBlog,
          content: value as string,
        });
      }
    }}
  />
</FormControl>
<ButtonGroup
```

```
  display={"flex"}
  justifyContent={"flex-end"}
  w={"100%"}
>
  <Button
    leftIcon={<FaTrash />}
    colorScheme={"red"}
    onClick={() => {
      REST.Admin.deleteBlog({
        token: localStorage.getItem("token") as string,
        id: currentBlog?._id as string,
      }).then((res) => {
        if (res.status === 200) {
          toast({
            title: "Blog gelöscht",
            description: "Der Blog wurde erfolgreich gelöscht",
            status: "success",
            duration: 1500,
            isClosable: false,
            position: "bottom-right",
          });
          reload();
          onClose();
        } else {
          toast({
            title: "Fehler",
            description: res.payload.error,
            status: "error",
            duration: 1500,
            isClosable: false,
            position: "bottom-right",
          });
        }
      });
    }}
  >
    Löschen
  </Button>
  <Button
    colorScheme={"red"}
    onClick={() => {
      onClose();
    }}
  >
    Abbrechen
  </Button>
  <Button
    variant={"brand"}
    onClick={() => {
      REST.Admin.updateBlog({
        token: localStorage.getItem("token") as string,
        id: currentBlog?._id as string,
        update: {
          title: currentBlog?.title as string,
          content: currentBlog?.content as string,
          tags: currentBlog?.tags as string[],
        },
      }).then((res) => {
        if (res.status === 200) {
          reload();
```

```
            onClose();
            setCurrentBlog(null);
          } else {
            toast({
              title: "Fehler",
              description: res.payload.error,
              status: "error",
              duration: 1500,
              isClosable: false,
              position: "bottom-right",
            });
          }
        });
      }}
    >
      Speichern
    </Button>
  </ButtonGroup>
 </Stack>
</CodeUpModal>

<GridLayoutV2
 pageTitle={"Blog"}
 search={{
  show: true,
  fields: true,
 }}
 topButtons={[
  {
   text: "Neu",
   leftIcon: <FaPlus />,
   variant: "brand",
   onClick: async () => {
    REST.Admin.createBlog({
     token: localStorage.getItem("token") as string,
     title: "Neuer Blog",
     content: "Inhalt",
     tags: [],
    }).then((res) => {
     reload();
     setCurrentBlog(res.payload.blog);
     onOpen();
    });
   },
  },
 ]}
 entries={blogs.map((blog) => {
  return {
   title: blog.title,
   buttonRows: [
    [
     {
      text: "Bearbeiten",
      variant: "brand",
      leftIcon: <FaEdit />,
      onClick: () => {
       setCurrentBlog(blog);
       onOpen();
      },
     },
```

```
                ],
              ],
              fields: [
                {
                  text: "Autor: @" + blog.authorUsername,
                  icon: <FaUser />,
                },
                {
                  text: new Date(blog.published_at).toLocaleString(),
                  icon: <FaClock />,
                },
                {
                  text: blog.tags.join(", "),
                  icon: <FaTags />,
                },
                {
                  text: blog.content.substring(0, 50),
                  icon: <FaFile />,
                },
              ],
            };
          })}
        />
      </>
    )}
  </AdminLayout>
 </>
 );
}
```

## apps/web/src/pages/admin/certificates.tsx

```
/**
 * apps/web/src/pages/admin/certificates.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 03.12.2022
 *
 */

import AdminLayout from "@codeup/components/admin/AdminLayout";
import {
  Avatar,
  Box,
  Button,
  Flex,
  Heading,
  IconButton,
  useDisclosure,
} from "@chakra-ui/react";
import router from "next/router";
import { FaHome, FaPlus, FaTrash, FaTrashAlt } from "react-icons/fa";
import REST from "@codeup/rest";
import { useEffect, useState } from "react";
import DataTable from "@codeup/components/extra/DataTable";
import AdminProfileViewer from "@codeup/components/admin/AdminProfileViewer";
import Head from "next/head";
import * as React from "react";
import SuperText from "@codeup/components/dist/landingpage/SuperText";
```

```
import { CHeading } from "@codeup/components/dist/global/CHeading";

export default function Certificates() {
  const [certificates, setCertificates] = useState<any[]>([]);

  const [loaded, setLoaded] = useState<boolean>(true);

  const [error, setError] = useState<string>("");

  const { isOpen, onOpen, onClose } = useDisclosure();

  const reload = () => {
    REST.Admin.getAllObtainedCertificates(
      localStorage.getItem("token") as string
    ).then((res) => {
      if (res.status === 200) {
        setCertificates(res.payload.certificates);
        setLoaded(true);
      } else {
        setError("Profile konnten nicht geladen werden.");
        setLoaded(true);
      }
    });
  };
  useEffect(() => {
    reload();
  }, []);

  const certToRow = (cert: any) => {
    return [
      (e: any) => {
        if (e.target.tagName === "TD") {
          router.push("/user-cert/" + cert._id);
        }
      },
      <Avatar size={"md"} src={"/api/account/" + cert.username + "/picture"} />,
      cert.username,
      cert.name,
      cert.course,
      cert.percentage + "%",
      new Date(cert.date).toLocaleString(),
      <>
        <IconButton
          colorScheme={"red"}
          aria-label={"delete"}
          variant={"ghost"}
          icon={<FaTrash />}
          onClick={() => {
            REST.Admin.deleteObtainedCertificate({
              token: localStorage.getItem("token") as string,
              id: cert._id,
            }).then(() => {
              reload();
            });
          }}
        />
      </>,
    ];
  };
```

```
return (
  <>
    <Head>
      <title>Zertifikate - Admin - CodeUp</title>
    </Head>
    <AdminLayout>
      <Box textAlign={"center"}>
        <CHeading size={"xl"}>Zertifikate</CHeading>
      </Box>
      <Flex w={"100%"} minH={"100vh"} h="fit-content" direction={"column"}>
        <Flex
          w={"100%"}
          h={"100%"}
          direction={"column"}
          gap={"6rem"}
          alignItems={"center"}
          mt={4}
          p={"2rem"}
        >
          <Box w={"100%"}>
            <DataTable
              columns={[
                "Profilbild",
                "Benutzername",
                "Anzeigename",
                "Kurs",
                "Erfolg",
                "Datum",
                "Aktion",
              ]}
              data={certificates.map(certToRow)}
              filter={(q: string, f: string, s: string) => {
                let x = certificates.filter((certificate: any) => {
                  return (
                    certificate.username
                      .toLowerCase()
                      .includes(q.toLowerCase()) ||
                    certificate.name
                      .toLowerCase()
                      .includes(q.toLowerCase()) ||
                    certificate.course.toLowerCase().includes(q.toLowerCase())
                  );
                });

                switch (f.toLowerCase()) {
                  case "alle":
                    break;
                  default:
                    break;
                }

                switch (s.toLowerCase()) {
                  case "alphabetisch":
                    x = x.sort((a: any, b: any) => {
                      if (
                        a.username.toLowerCase() < b.username.toLowerCase()
                      ) {
                        return -1;
                      } else if (
                        a.username.toLowerCase() > b.username.toLowerCase()
```

```
                    ) {
                      return 1;
                    } else {
                      return 0;
                    }
                  });
                  break;
                case "neuste zuerst":
                  x = x.sort((a: any, b: any) => {
                    return (
                      new Date(b.date).getTime() -
                      new Date(a.date).getTime()
                    );
                  });
                  break;
                case "älteste zuerst":
                  x = x.sort((a: any, b: any) => {
                    return (
                      new Date(a.date).getTime() -
                      new Date(b.date).getTime()
                    );
                  });
                  break;
                default:
                  break;
              }

              return x.map(certToRow);
            }}
            filterOptions={["Alle"]}
            sortOptions={[
              "Alphabetisch",
              "Neuste zuerst",
              "Älteste zuerst",
            ]}
            reload={reload}
          />
        </Box>
      </Flex>
    </Flex>
  </AdminLayout>
 </>
 );
}
```

## apps/web/src/pages/admin/challenges.tsx

```
/**
 * apps/web/src/pages/admin/challenges.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2024 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 18.05.2024
 *
 */

import * as React from "react";

import AdminLayout from "@codeup/components/admin/AdminLayout";
import { CHeading } from "@codeup/components/dist/global/CHeading";
```

```
import {
  Box,
  Button,
  Flex,
  FormControl,
  FormLabel,
  IconButton,
  Input,
  Select,
  Stack,
  Table,
  TableContainer,
  Tbody,
  Td,
  Textarea,
  Th,
  Thead,
  Tr,
  useDisclosure,
} from "@chakra-ui/react";
import REST from "@codeup/rest/src";
import AccountManager from "@codeup/util/dist/AccountManager";
import { FaPen, FaPlus, FaTrash } from "react-icons/fa";
import CodeUpModal from "@codeup/components/dist/codeup/CodeUpModal";
import PopupManager from "../../global/PopupManager";
import DataTable from "@codeup/components/dist/extra/DataTable";

export default function ChallengesPage() {
  const [challenges, setChallenges] = React.useState<
    {
      createdAt: string;
      _id: string;
      title: string;
      difficulty: number;
      task: string;
      expectedOutput: string;
    }[]
  >([]);

  const [currentChallenge, setCurrentChallenge] = React.useState<{
    createdAt: string;
    _id: string;
    title: string;
    difficulty: number;
    task: string;
    expectedOutput: string;
  } | null>(null);

  const reload = async () => {
    const res = await REST.Challenges.getChallenges(
      AccountManager.getActiveToken()
    );
    setChallenges(res.payload.challenges);
  };

  React.useEffect(() => {
    reload();
  }, []);

  const { isOpen, onOpen, onClose } = useDisclosure();
```

```
const [query, setQuery] = React.useState<string>("");

return (
  <AdminLayout>
    <Flex p={4} justifyContent={"space-between"} alignItems={"center"}>
      <Box w={"100%"} textAlign={"center"}>
        <CHeading size={"xl"}>Challenges</CHeading>
      </Box>
      <Button
        variant={"brand"}
        leftIcon={<FaPlus />}
        onClick={() => {
          onOpen();
        }}
      >
        Neu
      </Button>
    </Flex>
    <Box p={4} maxW={"99vw"} overflow={"auto"}>
      <Input
        placeholder={"Suche..."}
        mb={4}
        onChange={(e) => {
          setQuery(e.target.value);
        }}
      />
      <TableContainer>
        <Table variant="simple">
          <Thead>
            <Th>Erstellt</Th>
            <Th>Titel</Th>
            <Th>Schwierigkeit</Th>
            <Th>Aufgabe</Th>
            <Th>Erwartete Ausgabe</Th>
            <Th>Aktionen</Th>
          </Thead>
          <Tbody>
            {challenges
              .filter((c) => c.title.includes(query))
              .map((c) => {
                return (
                  <>
                    <Tr>
                      <Td>{new Date(c.createdAt).toLocaleString()}</Td>
                      <Td>{c.title}</Td>
                      <Td>
                        {c.difficulty === 1
                          ? "Leicht"
                          : c.difficulty === 2
                          ? "Mittel"
                          : "Schwer"}
                      </Td>
                      <Td>{c.task.substring(0, 100)}...</Td>
                      <Td>{c.expectedOutput}</Td>
                      <Td display={"flex"} gap={2}>
                        <IconButton
                          aria-label={"Edit"}
                          variant={"brand"}
                          icon={<FaPen />}
```

```
                onClick={() => {
                  setCurrentChallenge(c);
                  onOpen();
                }}
              />
              <IconButton
                aria-label={"Edit"}
                colorScheme={"red"}
                icon={<FaTrash />}
                onClick={async () => {
                  if (
                    !(await PopupManager.confirmAsync({
                      title: "Löschen",
                      question:
                        "Möchtest du diese Challenge wirklich löschen?",
                    }))
                  )
                    return;

                  const res = await REST.Admin.deleteChallenge({
                    token: AccountManager.getActiveToken(),
                    id: c._id,
                  });

                  if (res.status === 200) {
                    reload();
                  } else {
                    PopupManager.alertAsync({
                      title: "Fehler",
                      description: res.payload.error,
                    });
                  }
                }}
              />
            </Td>
          </Tr>
        </>
      );
    })}
  </Tbody>
 </Table>
</TableContainer>
</Box>
<CodeUpModal
 onOpen={onOpen}
 onClose={() => {
  onClose();
  setCurrentChallenge(null);
 }}
 isOpen={isOpen}
 title={currentChallenge ? "Bearbeiten" : "Neue Challenge"}
>
 <form
  onSubmit={async (e) => {
   e.preventDefault();
   const data = new FormData(e.target as HTMLFormElement);

   if (currentChallenge) {
    const res = await REST.Admin.updateChallenge({
     id: currentChallenge._id,
```

```
        token: AccountManager.getActiveToken(),
        title: data.get("title") as string,
        task: data.get("task") as string,
        difficulty: parseInt(data.get("difficulty") as string),
        expectedOutput: data.get("expectedOutput") as string,
      });

      if (res.status === 200) {
        onClose();
        reload();
      } else {
        PopupManager.alertAsync({
          title: "Fehler",
          description: res.payload.error,
        });
      }
    } else {
      const res = await REST.Admin.createChallenge({
        token: AccountManager.getActiveToken(),
        title: data.get("title") as string,
        task: data.get("task") as string,
        difficulty: parseInt(data.get("difficulty") as string),
        expectedOutput: data.get("expectedOutput") as string,
      });

      if (res.status === 200) {
        onClose();
        reload();
      } else {
        PopupManager.alertAsync({
          title: "Fehler",
          description: res.payload.error,
        });
      }
    }
  }}
>
  <Stack spacing={4}>
    <FormControl isRequired>
      <FormLabel>Titel</FormLabel>
      <Input
        placeholder={"Titel"}
        name={"title"}
        defaultValue={currentChallenge?.title}
      />
    </FormControl>
    <FormControl isRequired>
      <FormLabel>Aufgabe (Markdown)</FormLabel>
      <Textarea
        placeholder={"Aufgabe"}
        name={"task"}
        minH={"200px"}
        defaultValue={currentChallenge?.task}
      />
    </FormControl>
    <FormControl isRequired>
      <FormLabel>Schwierigkeit</FormLabel>
      <Select
        placeholder={"Schwierigkeit"}
        name={"difficulty"}
```

```
              defaultValue={currentChallenge?.difficulty}
            >
              <option value={1}>Leicht</option>
              <option value={2}>Mittel</option>
              <option value={3}>Schwer</option>
            </Select>
          </FormControl>
          <FormControl isRequired>
            <FormLabel>Erwartete Ausgabe</FormLabel>
            <Textarea
              placeholder={"Erwartete Ausgabe"}
              name={"expectedOutput"}
              minH={"200px"}
              defaultValue={currentChallenge?.expectedOutput}
            />
          </FormControl>
          <Button type={"submit"} variant={"brand"}>
            Speichern
          </Button>
        </Stack>
      </form>
    </CodeUpModal>
  </AdminLayout>
 );
}
```

## apps/web/src/pages/admin/contacts.tsx

```
/**
 * apps/web/src/pages/admin/contacts.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 26.11.2022
 *
 */

import * as React from "react";
import AdminLayout from "@codeup/components/admin/AdminLayout";
import REST from "@codeup/rest";
import {
  Avatar,
  Box,
  Button,
  ButtonGroup,
  Flex,
  FormControl,
  FormLabel,
  Heading,
  IconButton,
  Textarea,
  useDisclosure,
  useToast,
} from "@chakra-ui/react";
import DataTable from "@codeup/components/extra/DataTable";
import { FaEnvelope, FaTrash } from "react-icons/fa";
import { FiMail } from "react-icons/fi";
import CodeUpModal from "@codeup/components/codeup/CodeUpModal";
import Head from "next/head";
import SuperText from "@codeup/components/dist/landingpage/SuperText";
```

```
import AdminUserContact from "@codeup/components/admin/AdminUserContact";
import { CHeading } from "@codeup/components/dist/global/CHeading";

export default function Contacts() {
  const [messages, setMessages] = React.useState<
    {
      _id: string;
      message: string;
      title: string;
      username: string;
      category: string;
      createdAt: string;
    }[]
  >([]);

  const toast = useToast();

  const reload = () => {
    REST.Admin.getContactMessages(localStorage.getItem("token") as string).then(
      (res) => {
        if (res.status === 200) {
          setMessages(res.payload.messages);
        } else {
          toast({
            title: "Fehler",
            description: "Es ist ein Fehler aufgetreten: " + res.payload.error,
            status: "error",
            duration: 1500,
            isClosable: true,
            position: "bottom-right",
          });
        }
      }
    );
  };

  React.useEffect(() => {
    reload();
  }, []);

  const { onOpen, isOpen, onClose } = useDisclosure();
  const [selected, setSelected] = React.useState<string>("");

  const messageToRow = (msg: {
    _id: string;
    message: string;
    title: string;
    username: string;
    category: string;
    createdAt: string;
  }) => {
    return [
      () => {},
      <Avatar size={"md"} src={"/api/account/" + msg.username + "/picture"} />,
      msg.username,
      msg.title,
      msg.category,
      msg.message,
      new Date(msg.createdAt).toLocaleString(),
      <>
```

```
      <ButtonGroup>
        <IconButton
          aria-label={"delete"}
          icon={<FaTrash />}
          colorScheme={"red"}
          onClick={() => {
            REST.Admin.deleteContactMessage({
              token: localStorage.getItem("token") as string,
              id: msg._id,
            }).then((res) => {
              if (res.status === 200) {
                toast({
                  title: "Erfolgreich",
                  description: "Nachricht gelöscht",
                  status: "success",
                  duration: 1500,
                  isClosable: true,
                  position: "bottom-right",
                });
                reload();
              } else {
                toast({
                  title: "Fehler",
                  description:
                    "Es ist ein Fehler aufgetreten: " + res.payload.error,
                  status: "error",
                  duration: 1500,
                  isClosable: true,
                  position: "bottom-right",
                });
              }
            });
          }}
        />
        <IconButton
          aria-label={"delete"}
          variant={"brand"}
          onClick={() => {
            setSelected(msg.username);
            onOpen();
          }}
          icon={<FaEnvelope />}
        />
      </ButtonGroup>
    </>,
  ];
};

return (
  <>
    <Head>
      <title>Nachrichten - Admin - CodeUp</title>
    </Head>
    <AdminLayout>
      <Box textAlign={"center"}>
        <CHeading>Nachrichten</CHeading>
      </Box>
      <Flex justifyContent={"center"} w={"100%"} pt={6}>
        <Box maxW={"75%"} w={"100%"}>
          <DataTable
```

```
            columns={[
              "Profilbild",
              "Benutzer",
              "Titel",
              "Kategorie",
              "Nachricht",
              "Datum",
              "Aktion",
            ]}
            data={messages.map(messageToRow)}
            filter={(q: string, f: string, s: string) => {
              let x = messages;

              if (s === "Neuste Zuerst") {
                x = x.sort((a, b) => {
                  return (
                    new Date(b.createdAt).getTime() -
                    new Date(a.createdAt).getTime()
                  );
                });
              } else if (s === "Älteste Zuerst") {
                x = x.sort((a, b) => {
                  return (
                    new Date(a.createdAt).getTime() -
                    new Date(b.createdAt).getTime()
                  );
                });
              }

              x = x.filter((m) => {
                return (
                  m.username.toLowerCase().includes(q.toLowerCase()) ||
                  m.title.toLowerCase().includes(q.toLowerCase()) ||
                  m.category.toLowerCase().includes(q.toLowerCase()) ||
                  m.message.toLowerCase().includes(q.toLowerCase())
                );
              });

              return x.map(messageToRow);
            }}
            filterOptions={["Alle"]}
            sortOptions={["Neuste Zuerst", "Älteste Zuerst"]}
            reload={reload}
          />
        </Box>
      </Flex>
      <AdminUserContact
        onOpen={onOpen}
        isOpen={isOpen}
        onClose={onClose}
        title={"Nachricht an " + selected}
        username={selected}
      />
    </AdminLayout>
  </>
 );
}
```

## apps/web/src/pages/admin/contents/courses/index.tsx

```
/**
```

```
 * apps/web/src/pages/admin/contents/courses/index.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 30.10.2022
 *
 */

import * as React from "react";
import AdminLayout from "@codeup/components/admin/AdminLayout";
import AdminContentsLayout from "@codeup/components/admin/AdminContentsLayout";
import { useEffect } from "react";
import { useRouter } from "next/router";
import REST from "@codeup/rest";
import {
  Badge,
  Box,
  Button,
  ButtonGroup,
  Checkbox,
  Flex,
  FormControl,
  Grid,
  GridItem,
  Heading,
  Icon,
  Image,
  Input,
  InputGroup,
  InputLeftAddon,
  Select,
  Text,
  Textarea,
} from "@chakra-ui/react";
import {
  FaBookReader,
  FaCheckCircle,
  FaPen,
  FaPlus,
  FaRegTimesCircle,
  FaTimesCircle,
  FaTrash,
} from "react-icons/fa";
import Head from "next/head";
import SuperText from "@codeup/components/dist/landingpage/SuperText";

export default function Index() {
  const [courses, setCourses] = React.useState<
    {
      _id: string;
      name: string;
      friendlyName: string;
      description: string;
      splashImage: string;
      available: boolean;
    }[]
  >([]);

  const router = useRouter();
```

```
useEffect(() => {
  refresh();
}, []);

const refresh = () => {
  REST.Admin.listCourses(localStorage.getItem("token") as string).then(
    (res) => {
      if (res.status === 200) {
        setCourses(res.payload.courses);
      }
    }
  );
};

const [edit, setEdit] = React.useState<string>("");

return (
  <>
    <Head>
      <title>Kurse - Admin - CodeUp</title>
    </Head>
    <AdminLayout>
      <AdminContentsLayout>
        <Box mt={8}></Box>
        <SuperText>Kurse</SuperText>
        <Grid templateColumns="repeat(3, 1fr)" gap={6}>
          {courses.map((course) => (
            <>
              <GridItem m={4}>
                <Box
                  bg={"gray.900"}
                  color={"white"}
                  rounded={"lg"}
                  p={6}
                  h={"100%"}
                >
                  <Flex
                    justifyContent={"space-between"}
                    direction={"column"}
                    gap={2}
                  >
                    {edit === course.friendlyName ? (
                      <>
                        <form
                          onSubmit={(e) => {
                            e.preventDefault();

                            const data = new FormData(
                              e.target as HTMLFormElement
                            );

                            const name = data.get("name") as string;
                            const friendlyName = data.get(
                              "friendlyName"
                            ) as string;
                            const available = data.get("available") as any;
                            const description = data.get(
                              "description"
                            ) as string;
```

```
    const splashImage = data.get(
      "splashImage"
    ) as File;

    const FR = new FileReader();

    FR.readAsDataURL(splashImage);

    FR.onload = async () => {
      const res = await REST.Admin.updateCourse({
        token: localStorage.getItem(
          "token"
        ) as string,
        // @ts-ignore
        id: course._id,
        update: {
          name: name === "" ? course.name : name,
          friendlyName:
            friendlyName === ""
              ? course.friendlyName
              : friendlyName,
          // @ts-ignore
          available: available === "public",
          description:
            description === ""
              ? course.description
              : description,
          // @ts-ignore
          splashImage:
            FR.result ===
            "data:application/octet-stream; base64,"
              ? course.splashImage
              : FR.result,
        },
      });

      refresh();
      setEdit("");
    };
  }}
>
  <Image
    src={course.splashImage}
    alt={course.name}
    shadow={"xl"}
    w={"100%"}
    h={"150px"}
    cursor={"pointer"}
    _hover={{
      filter: "brightness(0.5)",
    }}
    transition={"filter 0.25s ease"}
    onClick={() => {
      document
        .getElementById(
          `edit_${course.friendlyName}-splashImage`
        )
        ?.click();
    }}
    id={
```

```
    "edit_" +
    course.friendlyName +
    "-splashImage_Image"
  }
/>
<input
  type={"file"}
  accept="image/svg+xml"
  name={"splashImage"}
  id={
    "edit_" + course.friendlyName + "-splashImage"
  }
  style={{
    display: "none",
  }}
  onChange={(e) => {
    const file = e.target.files?.item(0);
    if (file) {
      const reader = new FileReader();
      reader.readAsDataURL(file);
      reader.onload = () => {
        if (reader.result) {
          document
            .getElementById(
              "edit_" +
                course.friendlyName +
                "-splashImage_Image"
            )!
            .setAttribute(
              "src",
              reader.result as string
            );
        }
      };
    }
  }}
/>
<Input
  placeholder={"Name"}
  name={"name"}
  defaultValue={course.name}
  mt={"2rem"}
/>
<InputGroup mt={"2rem"}>
  <InputLeftAddon>/course/</InputLeftAddon>
  <Input
    placeholder={"URL Freundlicher Name"}
    name={"friendlyName"}
    defaultValue={course.friendlyName}
  />
</InputGroup>
<Select
  name={"available"}
  mt={"2rem"}
  defaultValue={
    course.available ? "public" : "private"
  }
>
  <option value={"public"}>Öffentlich</option>
  <option value={"private"}>Privat</option>
```

```
      </Select>
      <Textarea
        placeholder={"Beschreibung"}
        mt={"2rem"}
        name={"description"}
        defaultValue={course.description}
      />
      <ButtonGroup mt={"2rem"} w={"100%"}>
        <Button
          colorScheme={"red"}
          leftIcon={<FaTimesCircle />}
          w={"100%"}
          onClick={() => setEdit("")}
        >
          Abbrechen
        </Button>

        <Button
          colorScheme={"green"}
          leftIcon={<FaCheckCircle />}
          w={"100%"}
          type={"submit"}
        >
          Speichern
        </Button>
      </ButtonGroup>
    </form>
  </>
) : (
  <>
    <Image
      src={course.splashImage}
      alt={course.name}
      shadow={"xl"}
      w={"100%"}
      h={"150px"}
    />
    <Heading
      size={"lg"}
      color={"white"}
      fontWeight={"bold"}
      textAlign={"left"}
      mt={4}
    >
      {course.name}
    </Heading>
    <Flex direction={"row"} gap={4}>
      <Heading
        size={"sm"}
        color={"gray.400"}
        fontWeight={"normal"}
        textAlign={"left"}
      >
        /course/{course.friendlyName}
      </Heading>
      <Heading
        size={"sm"}
        color={"gray.400"}
        fontWeight={"normal"}
        textAlign={"left"}
```

```
    display={"flex"}
    flexDirection={"row"}
    gap={2}
  >
    by
    <Text color={"green.400"}>
      @
      {
        // @ts-ignore
        course.creator
      }
    </Text>
  </Heading>
</Flex>
<Heading
  size={"md"}
  color={"gray.400"}
  fontWeight={"normal"}
  textAlign={"left"}
>
  {course.available ? (
    <>
      <Badge colorScheme={"green"} fontSize={"md"}>
        Öffentlich
      </Badge>
    </>
  ) : (
    <>
      <Badge colorScheme={"red"} fontSize={"md"}>
        Privat
      </Badge>
    </>
  )}
</Heading>
<Heading
  size={"md"}
  color={"gray.400"}
  fontWeight={"normal"}
  textAlign={"left"}
>
  {course.description}
</Heading>
<ButtonGroup mt={6}>
  <Button
    colorScheme={"red"}
    leftIcon={<FaTrash />}
    w={"100%"}
    onClick={async () => {
      await REST.Admin.deleteCourse({
        token: localStorage.getItem(
          "token"
        ) as string,
        friendlyName: course.friendlyName,
      });

      refresh();
    }}
  >
    Löschen
  </Button>
```

```
                    <Button
                      colorScheme={"blue"}
                      leftIcon={<FaPen />}
                      w={"100%"}
                      onClick={() => setEdit(course.friendlyName)}
                    >
                      Bearbeiten
                    </Button>
                  </ButtonGroup>
                  <ButtonGroup mt={6}>
                    <Button
                      colorScheme={"green"}
                      leftIcon={<FaBookReader />}
                      w={"100%"}
                      onClick={() => {
                        router.push(
                          // @ts-ignore
                          "/admin/contents/courses/" + course._id
                        );
                      }}
                    >
                      Inhalte anzeigen
                    </Button>
                  </ButtonGroup>
                </>
              )}
            </Flex>
          </Box>
        </GridItem>
      </>
    ))}
  </Grid>
 </AdminContentsLayout>
</AdminLayout>
</>
);
}
```

## apps/web/src/pages/admin/contents/courses/new.tsx

```
/**
 * apps/web/src/pages/admin/contents/courses/new.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 30.10.2022
 *
 */

import * as React from "react";
import { useEffect, useState } from "react";
import AdminLayout from "@codeup/components/admin/AdminLayout";
import AdminContentsLayout from "@codeup/components/admin/AdminContentsLayout";
import {
  Box,
  Button,
  Flex,
  FormControl,
  FormLabel,
  Grid,
```

```
  GridItem,
  Heading,
  Image,
  Input,
  InputGroup,
  InputLeftAddon,
  Textarea,
} from "@chakra-ui/react";
import { FaBookOpen, FaGlobe } from "react-icons/fa";
import REST from "@codeup/rest";
import { useRouter } from "next/router";
import dynamic from "next/dynamic";
import "easymde/dist/easymde.min.css";
import AdminQuizMaker from "@codeup/components/admin/AdminQuizMaker";
import { Step, Steps, useSteps } from "chakra-ui-steps";
import Head from "next/head";
import SuperText from "@codeup/components/dist/landingpage/SuperText";

const SimpleMdeReact = dynamic(import("react-simplemde-editor"), {
  ssr: false,
});

export default function NewComponent() {
  const { nextStep, prevStep, reset, activeStep } = useSteps({
    initialStep: 0,
  });
  const steps = [
    { label: "Name" },
    { label: "Beschreibung" },
    { label: "Vorschaubild" },
  ];

  const router = useRouter();

  const [quizQuestions, setQuizQuestions] = useState<
    {
      question: string;
      answers: {
        answer: string;
        correct: boolean;
      }[];
    }[]
  >([]);

  const [title, setTitle] = useState<string>("");
  const [friendlyName, setFriendlyName] = useState<string>("");
  const [splashImage, setSplashImage] = useState<File>();
  const [description, setDescription] = useState<string>("");

  return (
    <>
      <Head>
        <title>Neuer Kurs - Admin - CodeUp</title>
      </Head>
      <AdminLayout>
        <AdminContentsLayout>
          <Flex flexDir="column" width="100%" p={"5rem"}>
            <Box mt={8}></Box>
            <SuperText>Kurse</SuperText>
            <Steps size={"lg"} activeStep={activeStep}>
```

```
{steps.map(({ label }, index) => (
  <Step label={label} key={label} mb={"2rem"}>
    {index === 0 ? (
      <>
        <FormControl isRequired>
          <FormLabel>Titel</FormLabel>
          <InputGroup>
            <InputLeftAddon>
              <FaBookOpen />
            </InputLeftAddon>
            <Input
              type={"text"}
              name={"title"}
              id={"new_video_title"}
              placeholder={"Titel"}
              required
              onChange={(e) => setTitle(e.target.value)}
            />
          </InputGroup>
        </FormControl>
        <FormControl mt={"2rem"} isRequired>
          <FormLabel>URL Freundlicher Name</FormLabel>
          <InputGroup>
            <InputLeftAddon>
              <FaGlobe />
            </InputLeftAddon>
            <Input
              type={"text"}
              name={"friendlyName"}
              id={"new_video_friendlyName"}
              placeholder={"URL Freundlicher Name"}
              required
              onChange={(e) => setFriendlyName(e.target.value)}
            />
          </InputGroup>
        </FormControl>
      </>
    ) : index === 1 ? (
      <>
        <FormControl mt={"2rem"} isRequired>
          <FormLabel>Beschreibung</FormLabel>
          <Textarea
            name={"description"}
            placeholder={"Beschreibung"}
            required
            onChange={(e) => setDescription(e.target.value)}
          />
        </FormControl>
      </>
    ) : index === 2 ? (
      <>
        <FormControl mt={"2rem"} isRequired>
          <FormLabel>Vorschaubild</FormLabel>
          <Input
            required
            type={"file"}
            accept="image/svg+xml"
            name={"splashImage"}
            onChange={(e) => {
              // @ts-ignore
```

```
              setSplashImage(e.target.files[0] as File);
            }}
          />
        </FormControl>
      </>
    ) : index === 3 ? (
      <>
        <FormControl mt={"2rem"} isRequired>
          <FormLabel>Beschreibung</FormLabel>
          <SimpleMdeReact
            value={description}
            onChange={(e) => setDescription(e)}
          />
        </FormControl>
      </>
    ) : index === 4 ? (
      <>
        <AdminQuizMaker
          quizQuestions={quizQuestions}
          setQuizQuestions={setQuizQuestions}
        />
      </>
    ) : (
      <></>
    )}
  </Step>
 ))}
</Steps>
{activeStep === steps.length ? (
  <Flex px={4} py={4} width="100%" flexDirection="column">
    <Heading fontSize="xl" textAlign="center">
      Woohoo! All steps completed!
    </Heading>
    <Button mx="auto" mt={6} size="sm" onClick={reset}>
      Zurücksetzen
    </Button>
  </Flex>
) : (
  <Flex width="100%" justify="flex-end" mt={"2rem"}>
    <Button
      isDisabled={activeStep === 0}
      mr={4}
      onClick={prevStep}
      size="lg"
      variant="ghost"
    >
      Vorheriger
    </Button>
    <Button
      size="lg"
      onClick={async () => {
        if (activeStep !== steps.length - 1) {
          switch (activeStep) {
            case 0:
              if (title === "" || friendlyName === "") return;
              break;
            case 1:
              if (description === "") return;
              break;
            case 2:
```

```
                    if (!splashImage) return;
                    break;
                  default:
                    break;
                }
                nextStep();
                return;
              }

              const base64 = await new Promise<string>((resolve) => {
                if (splashImage) {
                  const reader = new FileReader();
                  reader.readAsDataURL(splashImage);
                  reader.onload = () => resolve(reader.result as string);
                  reader.onerror = (error) => console.log(error);
                } else {
                  resolve("");
                }
              });

              await REST.Admin.createCourse({
                token: localStorage.getItem("token") as string,
                name: title,
                friendlyName,
                splashImage: base64,
                description,
              });

              router.push("/admin/contents/courses");
            }}
          >
            {activeStep === steps.length - 1
              ? "Fertigstellen"
              : "Nächster"}
          </Button>
        </Flex>
      )}
    </Flex>
  </AdminContentsLayout>
 </AdminLayout>
 </>
 );
}
```

## apps/web/src/pages/admin/contents/courses/[id].tsx

```
/**
 * apps/web/src/pages/admin/contents/courses/[id].tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 11.11.2022
 *
 */

import * as React from "react";
import { useRouter } from "next/router";
import REST from "@codeup/rest";
import AdminLayout from "@codeup/components/admin/AdminLayout";
import AdminContentsLayout from "@codeup/components/admin/AdminContentsLayout";
```

```
import {
  Badge,
  Box,
  Button,
  ButtonGroup,
  Flex,
  Grid,
  Heading,
  IconButton,
  Stack,
  Text,
  useToast,
} from "@chakra-ui/react";
import {
  FaEye,
  FaFile,
  FaTrash,
  FaYoutube,
  FaYoutubeSquare,
} from "react-icons/fa";
import { DragDropContext, Draggable, Droppable } from "@hello-pangea/dnd";
import Head from "next/head";

export default function Course() {
  const router = useRouter();

  const { id } = router.query;

  const [courseName, setCourseName] = React.useState<string[]>([
    "Inhalte Laden...",
  ]);

  const [contentPositions, setContentPositions] = React.useState<string[]>([]);

  const [contents, setContents] = React.useState<
    {
      _id: string;
      displayName: string;
      course: string;
      url: string;
      createdAt: string;
      type: "TEXT" | "VIDEO";
      textContent: string;
      quiz: {
        question: string;
        answers: {
          answer: string;
          correct: boolean;
        };
      }[];
      videoUrl?: string;
    }[]
  >([]);

  React.useEffect(() => {
    if (id && id !== "") {
      REST.Admin.listCourses(localStorage.getItem("token") as string).then(
        (res) => {
          if (res.status === 200) {
            const course = res.payload.courses.filter(
```

```
            (c: any) => c._id === id
          )[0];
          setCourseName([course.name, course.friendlyName]);
          setContentPositions(course.contentPositions);
        } else {
          console.error(res);
          alert("Fehler beim Laden der Inhalte. Details siehe Konsole.");
        }
      }
    );
  }
}, [id]);

React.useEffect(() => {
  reloadContents();
}, [contentPositions]);

const reloadContents = () => {
  REST.Admin.getContents({
    token: localStorage.getItem("token") as string,
    course: id as string,
  }).then((res) => {
    if (res.status === 200) {
      setContents(res.payload.contents);

      const sortedContents = contentPositions.map((id) => {
        return res.payload.contents.filter((c: any) => c._id === id)[0];
      });

      const missingContents = res.payload.contents.filter(
        (c: any) => !contentPositions.includes(c._id)
      );

      let x: any = [];

      if (sortedContents && missingContents) {
        x = [...sortedContents, ...missingContents];
      } else if (sortedContents) {
        x = [...sortedContents];
      } else if (missingContents) {
        x = [...missingContents];
      }

      x = x.filter((c: any) => c !== undefined);

      setItems(x);
    } else {
      console.error(res);
      alert("Fehler beim Laden der Inhalte. Details siehe Konsole.");
    }
  });
};

const toast = useToast();

const [items, setItems] = React.useState<any[]>([]);

return (
  <>
    <Head>
```

```
  <title>Kursbetrachter - Admin - CodeUp</title>
</Head>
<AdminLayout>
  <AdminContentsLayout>
    <Box p={"2rem"}>
      <Heading size={"lg"}>{courseName[0]}</Heading>
    </Box>{" "}
    <Flex
      alignItems={"center"}
      p={6}
      w={"100%"}
      justifyContent={"center"}
    >
      <Box maxW={["100%", "50%"]} w={"100%"} id={"_tmp.up"}>
        <DragDropContext
          onDragEnd={(result) => {
            if (!result.destination) return;

            const items0 = Array.from(items);

            const [reorderedItem] = items0.splice(result.source.index, 1);
            items0.splice(result.destination.index, 0, reorderedItem);

            setItems(items0);

            REST.Admin.updateCourse({
              token: localStorage.getItem("token") as string,
              id: id as string,
              update: {
                // @ts-ignore
                contentPositions: items0.map((i) => i._id),
              },
            });
          }}
        >
          <Droppable droppableId={"contents"}>
            {(provided, snapshopt) => (
              <>
                <Box
                  ref={provided.innerRef}
                  {...provided.droppableProps}
                  p={"2rem"}
                >
                  {items.map((content, index) => (
                    <>
                      {content !== undefined ? (
                        <>
                          <Draggable
                            key={content._id}
                            draggableId={content._id}
                            index={index}
                          >
                            {(provided, snapshot) => (
                              <>
                                <Box
                                  ref={provided.innerRef}
                                  {...provided.draggableProps}
                                  {...provided.dragHandleProps}
                                  bg={"gray.900"}
                                  color={"white"}
```

```
  rounded={"lg"}
  p={6}
  mt={6}
  borderWidth="1px"
  borderRadius="lg"
  _hover={{
    borderColor: "blue.500",
  }}
  cursor={"pointer"}
>
  <Flex
    justifyContent={"space-between"}
    direction={"column"}
    gap={2}
  >
    <Flex
      direction={"row"}
      w={"100%"}
      gap={8}
    >
      <Flex
        direction={"column"}
        flex="50%"
        gap={4}
      >
        <Heading size={"md"}>
          {content.displayName}
        </Heading>
        <Text
          size={"sm"}
          color={"gray.400"}
        >
          /course/{courseName[1]}/
          {content.url}
        </Text>
      </Flex>
      <Box flex={"50%"}>
        <Stack>
          <Button
            w={"100%"}
            leftIcon={<FaFile />}
            onClick={() => {
              const blob = new Blob(
                [content.textContent],
                {
                  type: "text/plain",
                }
              );

              const url =
                URL.createObjectURL(blob);

              window.open(url, "_blank");
            }}
          >
            Text Anzeigen
          </Button>
          {content.type === "VIDEO" ? (
            <>
              <Button
```

```
      leftIcon={<FaYoutube />}
      w={"100%"}
      onClick={() => {
        window.open(
          content.videoUrl,
          "_blank"
        );
      }}
    >
      Video Anzeigen
    </Button>
  </>
) : null}
<Button
  w={"100%"}
  colorScheme={"blue"}
  leftIcon={<FaEye />}
  onClick={() => {
    router.push(
      `/course/${courseName[1]}/${content.url}`
    );
  }}
>
  Benutzeransicht
</Button>
<Button
  w={"100%"}
  leftIcon={<FaTrash />}
  colorScheme={"red"}
  onClick={async () => {
    if (
      !confirm(
        "Willst du den Inhalt wirklich löschen?"
      )
    ) {
      return;
    }

    const res =
      await REST.Creator.deleteContent(
        {
          token:
            localStorage.getItem(
              "token"
            ) as string,
          cid: content._id,
        }
      );

    if (res.status === 200) {
      reloadContents();
      toast({
        title:
          "Inhalt gelöscht",
        description:
          "Der Inhalt wurde erfolgreich gelöscht.",
        status: "success",
        duration: 1500,
        isClosable: true,
        position:
```

```
                                        "bottom-right",
                                      });
                                    } else {
                                      console.error(res);
                                      toast({
                                        title:
                                          "Fehler beim Löschen",
                                        description:
                                          "Der Inhalt konnte nicht gelöscht werden. Details siehe Konsole.",
                                        status: "error",
                                        duration: 1500,
                                        isClosable: true,
                                        position:
                                          "bottom-right",
                                      });
                                    }
                                  }}
                                >
                                  Löschen
                                </Button>
                              </Stack>
                            </Box>
                          </Flex>
                          <Text
                            fontSize={"sm"}
                            color={"gray.400"}
                          >{`Erstellt am: ${new Date(
                            content.createdAt
                          )
                            .toLocaleString()
                            .replace(",", " um")}`}</Text>
                        </Flex>
                      </Box>
                    </>
                  )}
                </Draggable>
              </>
            ) : (
              <></>
            )}
          </>
        ))}
      </Box>
    </>
  )}
</Droppable>
</DragDropContext>
</Box>
</Flex>
</AdminContentsLayout>
</AdminLayout>
</>
);
}
```

## apps/web/src/pages/admin/contents/new.tsx

```
/**
 * apps/web/src/pages/admin/contents/new.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
```

```
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 06.11.2022
 *
 */

import AdminContentsLayout from "@codeup/components/admin/AdminContentsLayout";
import AdminLayout from "@codeup/components/admin/AdminLayout";
import REST from "@codeup/rest";
import { useRouter } from "next/router";
import React, { useEffect, useState } from "react";
import { FaBookOpen, FaGlobe, FaYoutube } from "react-icons/fa";
import {
  Box,
  Button,
  Flex,
  FormControl,
  FormLabel,
  Grid,
  GridItem,
  Heading,
  Image,
  Input,
  InputGroup,
  InputLeftAddon,
} from "@chakra-ui/react";
import dynamic from "next/dynamic";
import "easymde/dist/easymde.min.css";
import AdminQuizMaker from "@codeup/components/admin/AdminQuizMaker";
import { Step, Steps, useSteps } from "chakra-ui-steps";
import Builder from "../../../special-components/Builder";
import Head from "next/head";
import SuperText from "@codeup/components/dist/landingpage/SuperText";

const SimpleMdeReact = dynamic(import("react-simplemde-editor"), {
  ssr: false,
});

export default function NewContents() {
  const router = useRouter();

  const [courses, setCourses] = React.useState([]);

  useEffect(() => {
    REST.Admin.listCourses(localStorage.getItem("token") as string).then(
      (res) => {
        if (res.status === 200) {
          setCourses(res.payload.courses);
        }
      }
    );
  }, []);

  const [displayName, setDisplayName] = React.useState("");
  const [course, setCourse] = React.useState("");
  const [url, setURL] = React.useState("");
  const [textContent, setTextContent] = React.useState("");
  const [quizQuestions, setQuizQuestions] = React.useState<
    {
      question: string;
```

```
    answers: {
      answer: string;
      correct: boolean;
    }[];
  }[]
>([]);
const [videoURL, setVideoURL] = React.useState("");

const [steps, setSteps] = useState([
  { label: "Name" },
  { label: "Kurs" },
  { label: "Inhalt" },
  { label: "Quiz" },
]);

const { nextStep, prevStep, reset, activeStep } = useSteps({
  initialStep: 0,
});

return (
  <>
    <Head>
      <title>Neuer Inhalt - Admin - CodeUp</title>
    </Head>
    <AdminLayout>
      <AdminContentsLayout>
        <>
          <Flex flexDir="column" width="100%" p={"5rem"}>
            <Box mt={8}></Box>
            <SuperText>Kurse</SuperText>
            <Steps size={"lg"} activeStep={activeStep}>
              {steps.map(({ label }, index) => (
                <Step label={label} key={label} mb={"2rem"}>
                  {index === 0 ? (
                    <>
                      <FormControl isRequired>
                        <FormLabel>Titel</FormLabel>
                        <InputGroup>
                          <InputLeftAddon>
                            <FaBookOpen />
                          </InputLeftAddon>
                          <Input
                            type={"text"}
                            name={"title"}
                            id={"new_content_title"}
                            placeholder={"Titel"}
                            required
                            onChange={(e) => setDisplayName(e.target.value)}
                          />
                        </InputGroup>
                      </FormControl>
                      <FormControl mt={"2rem"} isRequired>
                        <FormLabel>URL Freundlicher Name</FormLabel>
                        <InputGroup>
                          <InputLeftAddon>
                            <FaGlobe />
                          </InputLeftAddon>
                          <Input
                            type={"text"}
                            name={"friendlyName"}
```

```
          id={"new_content_friendlyName"}
          placeholder={"URL Freundlicher Name"}
          required
          onChange={(e) => setURL(e.target.value)}
        />
      </InputGroup>
    </FormControl>
  </>
) : index === 1 ? (
  <>
    <Grid templateColumns="repeat(3, 1fr)" gap={6}>
      {courses.map((c: any) => {
        return (
          <>
            <GridItem m={4}>
              <Box
                bg={
                  course === c._id ? "gray.700" : "gray.900"
                }
                transition={"background 0.5s ease-in-out"}
                color={"white"}
                rounded={"lg"}
                p={6}
                h={"100%"}
                onClick={() => {
                  if (course === c._id) {
                    setCourse("");
                  } else {
                    setCourse(c._id);
                  }
                }}
                cursor={"pointer"}
              >
                <Flex
                  justifyContent={"space-between"}
                  direction={"column"}
                  gap={2}
                >
                  <Image
                    src={c.splashImage}
                    alt={c.name}
                    shadow={"xl"}
                    w={"100%"}
                    h={"150px"}
                  />
                  <Heading
                    size={"lg"}
                    color={"white"}
                    fontWeight={"bold"}
                    textAlign={"left"}
                    mt={4}
                  >
                    {c.name}
                  </Heading>
                  <Heading
                    size={"sm"}
                    color={"gray.400"}
                    fontWeight={"normal"}
                    textAlign={"left"}
                  >
```

```
                  /course/{c.friendlyName}
                </Heading>
                <Heading
                  size={"md"}
                  color={"gray.400"}
                  fontWeight={"normal"}
                  textAlign={"left"}
                >
                  {c.description}
                </Heading>
              </Flex>
            </Box>
          </GridItem>
        </>
      );
    })}
  </Grid>
 </>
) : index === 2 ? (
 <>
  <Builder custom_event_id={"admin_event_id"} />
 </>
) : index === 3 ? (
 <>
  <AdminQuizMaker
   quizQuestions={quizQuestions}
   setQuizQuestions={setQuizQuestions}
  />
 </>
) : (
 <></>
)}
</Step>
))}
</Steps>
{activeStep === steps.length ? (
 <Flex px={4} py={4} width="100%" flexDirection="column">
  <Heading fontSize="xl" textAlign="center">
   Woohoo! All steps completed!
  </Heading>
  <Button mx="auto" mt={6} size="sm" onClick={reset}>
   Zurücksetzen
  </Button>
 </Flex>
) : (
 <Flex width="100%" justify="flex-end" mt={"2rem"}>
  <Button
   mr={4}
   onClick={() => {
    if (activeStep === 0) return;
    prevStep();
   }}
   size="lg"
   variant="ghost"
  >
   Vorheriger
  </Button>
  <Button
   size="lg"
   onClick={async () => {
```

```
  if (activeStep !== steps.length - 1) {
    switch (activeStep) {
      case 0:
        if (displayName === "" || url === "") {
          return;
        }
        nextStep();
        break;
      case 1:
        if (course === "") return;
        nextStep();

        break;
      case 2:
        window.dispatchEvent(
          new CustomEvent(
            "codeup::webbuilder::export_admin_event_id"
          )
        );

        const mergedCode =
          // @ts-ignore
          window.codeup.webBuilderHTMLExport;

        setTextContent(mergedCode);

        // @ts-ignore
        delete window.codeup.webBuilderHTMLExport;

        nextStep();
        break;
      case 3:
        if (quizQuestions.length === 0) return;

        nextStep();
        break;
      default:
        break;
    }
  } else {
    const res = await REST.Admin.createContent({
      token: localStorage.getItem("token") as string,
      course,
      displayName,
      quiz: quizQuestions,
      textContent,
      type: "TEXT",
      url,
    });

    if (res.status === 200) {
      router.push("/admin/contents/courses/" + course);
    } else {
      alert(
        "Fehler beim Erstellen des Inhalts. Deteils siehe Konsole."
      );
      console.log(res.payload);
    }
  }
}}
```

```
                >
                  {activeStep === steps.length - 1
                    ? "Fertigstellen"
                    : "Nächster"}
                </Button>
              </Flex>
            )}
          </Flex>
        </>
      </AdminContentsLayout>
    </AdminLayout>
  </>
 );
}
```

## apps/web/src/pages/admin/discovery.tsx

```
/**
 * apps/web/src/pages/admin/discovery.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 13.04.2023
 *
 */

import * as React from "react";
import REST from "@codeup/rest/src";
import { useRouter } from "next/router";
import AdminLayout from "@codeup/components/admin/AdminLayout";
import { Box } from "@chakra-ui/react";
import SuperText from "@codeup/components/dist/landingpage/SuperText";
import { useLoggedIn } from "@codeup/hooks";
import { FaInfo, FaLink, FaStar, FaTrash, FaUser } from "react-icons/fa";
import GridLayout from "@codeup/components/dist/global/GridLayout";
import PopupManager from "../../global/PopupManager";
import AccountManager from "@codeup/util/dist/AccountManager";
import GridLayoutV2 from "@codeup/components/dist/global/GridLayoutV2";

export default function Discovery() {
  const { userInfo, loggedIn, loaded } = useLoggedIn();

  const [projects, setProjects] = React.useState<
    {
      name: string;
      description: string;
      author: string;
      domain: string;
      id: string;
      stars: string[];
    }[]
  >([]);

  const [error, setError] = React.useState(false);

  React.useEffect(() => {
    REST.Discovery.getDiscoveryProjects().then((res) => {
      if (res.status === 200) {
        setProjects(res.payload.projects);
      } else {
```

```
      setError(true);
    }
  });
}, []);

const router = useRouter();

return (
  <>
    <AdminLayout>
      <GridLayoutV2
        pageTitle={"Discovery"}
        search={{
          show: true,
          fields: true,
        }}
        topButtons={[]}
        entries={projects.map((p) => {
          return {
            title: p.name,
            fields: [
              {
                icon: <FaUser />,
                text: "Benutzer: @" + p.author,
              },
              {
                icon: <FaStar />,
                text: "Sterne: " + p.stars.length,
              },
              {
                icon: <FaLink />,
                text: "Domain: " + p.domain,
              },
              {
                icon: <FaInfo />,
                text:
                  "Beschreibung: " + p.description.substring(0, 100) + "...",
              },
            ],
            buttonRows: [
              [
                {
                  text: "Löschen",
                  leftIcon: <FaTrash />,
                  colorScheme: "red",
                  onClick: async () => {
                    if (
                      !(await PopupManager.confirmAsync({
                        title: "Entfernen",
                        question:
                          "Möchtest du das Projekt wirklich entfernen?",
                      }))
                    )
                      return;

                    const res = await REST.Admin.removeFromDiscovery({
                      token: AccountManager.getActiveToken(),
                      id: p.id,
                    });
```

```
                if (res.status === 200) {
                  await PopupManager.alertAsync({
                    title: "Entfernt",
                    description: "Das Projekt wurde entfernt!",
                  });

                  setProjects(projects.filter((pr) => pr.id !== p.id));
                } else {
                  await PopupManager.alertAsync({
                    title: "Fehler",
                    description:
                      "Ein Fehler ist aufgetreten: " + res.payload.error,
                  });
                }
              },
              disabled: false,
            },
          ],
        ],
      };
    })}
  />
 </AdminLayout>
 </>
 );
}
```

## apps/web/src/pages/admin/editor.tsx

```
/**
 * apps/web/src/pages/admin/ide.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 19.04.2023
 *
 */

import * as React from "react";
import { useRouter } from "next/router";
import REST from "@codeup/rest/src";
import AccountManager from "@codeup/util/dist/AccountManager";
import PopupManager from "../../global/PopupManager";
import AdminLayout from "@codeup/components/admin/AdminLayout";
import AuthenticateLoadingIndicator from
"@codeup/components/dist/admin/AuthenticateLoadingIndicator";
import ErrorComponent from "@codeup/components/dist/global/ErrorComponent";
import {
  FaEye,
  FaFile,
  FaPen,
  FaPlus,
  FaRocket,
  FaTrash,
  FaUser,
  FaUsers,
} from "react-icons/fa";
import GridLayout from "@codeup/components/dist/global/GridLayout";
import { Box } from "@chakra-ui/react";
import SuperText from "@codeup/components/dist/landingpage/SuperText";
```

```
import GridLayoutV2 from "@codeup/components/dist/global/GridLayoutV2";

export default function Kids() {
  const router = useRouter();
  const [error, setError] = React.useState<boolean>(false);
  const [loading, setLoading] = React.useState<boolean>(true);
  const [projects, setProjects] = React.useState<
    {
      _id: string;
      owner: string;
      name: string;
      discovery: boolean;
      discoveryDescription: string;
      discoveryLikes: string[];
      discoveryComments: any[];
      customDomain: string;
      contributors: string[];
      updated_at: string;
      created_at: string;
      __v: string;
    }[]
  >([]);

  const reload = () => {
    REST.Admin.getEditorProjects(AccountManager.getActiveToken()).then(
      (res) => {
        if (res.status !== 200) {
          setError(true);
          setLoading(false);
          return;
        }

        setLoading(false);
        setProjects(res.payload.projects);
      }
    );
  };

  React.useEffect(() => {
    reload();
  }, []);
  return (
    <>
      <AdminLayout>
        {loading ? (
          <>
            <AuthenticateLoadingIndicator text={"Lade Projekte..."} />
          </>
        ) : (
          <>
            {error ? (
              <>
                <ErrorComponent
                  description={"Es ist ein Fehler aufgetreten."}
                />
              </>
            ) : (
              <>
                <GridLayoutV2
                  pageTitle={"Projekte"}
```

```
                search={{
                  show: true,
                  fields: true,
                }}
                topButtons={[]}
                entries={projects.map((p) => ({
                  title: p.name,
                  fields: [
                    {
                      text: "Besitzer: " + p.owner,
                      icon: <FaUser />,
                    },
                    {
                      text:
                        "Veröffentlicht: " +
                        (p.customDomain.startsWith("disabled:")
                          ? "Nein"
                          : p.customDomain),
                      icon: <FaRocket />,
                    },
                    {
                      text: "Mitwirkende: " + p.contributors.length,
                      icon: <FaUsers />,
                    },
                  ],
                  buttonRows: [
                    [
                      {
                        text: "Öffnen",
                        onClick: () => {
                          router.push(`/editor/${p._id}`);
                        },
                        leftIcon: <FaEye />,
                        variant: "brand",
                        disabled: false,
                      },
                    ],
                  ],
                }))}
              />
            </>
          )}
        </>
      )}
    </AdminLayout>
  </>
 );
}
```

## apps/web/src/pages/admin/errors.tsx

```
/**
 * apps/web/src/pages/admin/errors.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 26.04.23
 *
 */
```

```
import React from "react";
import AdminLayout from "@codeup/components/dist/admin/AdminLayout";
import REST from "@codeup/rest/src";
import AuthenticateLoadingIndicator from
"@codeup/components/dist/admin/AuthenticateLoadingIndicator";
import ErrorComponent from "@codeup/components/dist/global/ErrorComponent";
import {
  Box,
  Accordion,
  AccordionItem,
  AccordionButton,
  AccordionPanel,
  AccordionIcon,
  Flex,
  InputGroup,
  InputLeftElement,
  Input,
  Select,
  IconButton,
  MenuButton,
  MenuList,
  MenuItem,
  Menu,
} from "@chakra-ui/react";
import SuperText from "@codeup/components/dist/landingpage/SuperText";
import AccountManager from "@codeup/util/dist/AccountManager";
import { FaCheck, FaFilter, FaSearch } from "react-icons/fa";
import { FiRotateCw } from "react-icons/fi";
import { CHeading } from "@codeup/components/dist/global/CHeading";

export default function Errors() {
  const [errors, setErrors] = React.useState<
    {
      name: string;
      message: string;
      stack: string;
      createdAt: string;
      _id: string;
    }[]
  >([]);
  const [errorNames, setErrorNames] = React.useState<string[]>([]);
  const [selectedErrorName, setSelectedErrorName] =
    React.useState<string>("_all");
  const [query, setQuery] = React.useState<string>("");

  const [loading, setLoading] = React.useState(true);
  const [error, setError] = React.useState(false);

  React.useEffect(() => {
    reload();
  }, []);

  const reload = () => {
    REST.Admin.getErrors(AccountManager.getActiveToken()).then((res) => {
      if (res.status === 200) {
        setErrors(res.payload as any);
        let errNs: string[] = [
          ...new Set(
            (res.payload as any).map((e: any) => {
              return e.name;
```

```
          })
        ),
      ] as string[];
      setErrorNames(errNs);
      setLoading(false);
    } else {
      setError(true);
      setLoading(false);
    }
  });
};

return (
  <>
    <AdminLayout>
      {loading ? (
        <AuthenticateLoadingIndicator text={"Lade Fehler..."} />
      ) : (
        <>
          {error ? (
            <>
              <ErrorComponent
                description={
                  "Beim Laden der Fehler ist ein Fehler aufgetreten."
                }
              />
            </>
          ) : (
            <>
              <Box textAlign={"center"}>
                <CHeading size={"xl"}>Fehler</CHeading>
              </Box>
              <Box p={6}>
                <Flex w={"100%"} gap={4} mb={4}>
                  <InputGroup>
                    <InputLeftElement>
                      <FaSearch />
                    </InputLeftElement>
                    <Input
                      placeholder={"Suchen"}
                      value={query}
                      onChange={(e) => {
                        setQuery(e.target.value);
                      }}
                    />
                  </InputGroup>
                  <Menu>
                    <IconButton
                      aria-label={"refresh"}
                      icon={<FiRotateCw />}
                      onClick={async () => {
                        setLoading(true);
                        reload();
                      }}
                    />
                    <MenuButton
                      as={IconButton}
                      icon={<FaFilter />}
                      aria-label={"Filter"}
                    />
```

```
    <MenuList>
      <MenuItem
        key={"_all"}
        onClick={() => {
          setSelectedErrorName("_all");
        }}
        icon={
          selectedErrorName === "_all" ? <FaCheck /> : <></>
        }
      >
        Alle
      </MenuItem>
      {errorNames.map((option) => (
        <MenuItem
          key={option}
          onClick={() => {
            setSelectedErrorName(option);
          }}
          icon={
            selectedErrorName === option ? <FaCheck /> : <></>
          }
        >
          {option}
        </MenuItem>
      ))}
    </MenuList>
  </Menu>
</Flex>
<Accordion allowMultiple allowToggle>
  {errors
    .filter((e) => {
      return (
        e.name.toLowerCase().includes(query.toLowerCase()) ||
        e.message
          .toLowerCase()
          .includes(query.toLowerCase()) ||
        e.stack.toLowerCase().includes(query.toLowerCase()) ||
        new Date(e.createdAt)
          .toLocaleString()
          .toLowerCase()
          .includes(query.toLowerCase())
      );
    })
    .filter((e) => {
      return (
        selectedErrorName === "_all" ||
        e.name === selectedErrorName
      );
    })
    .map((e) => {
      return (
        <>
          <AccordionItem>
            <h2>
              <AccordionButton>
                <Box as="span" flex="1" textAlign="left">
                  {new Date(e.createdAt).toLocaleString()}
                   - 
                  {e.name}
                   - 
```

```
                        {e.message}
                      </Box>
                      <AccordionIcon />
                    </AccordionButton>
                  </h2>
                  <AccordionPanel pb={4}>
                    <pre>{e.stack}</pre>
                  </AccordionPanel>
                </AccordionItem>
              </>
            );
          })}
        </Accordion>
      </Box>
    </>
  )}
 </>
)}
</AdminLayout>
</>
);
}
```

## apps/web/src/pages/admin/flows.tsx

```
/**
 * apps/web/src/pages/admin/flows.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 13.04.2023
 *
 */

import * as React from "react";
import AdminLayout from "@codeup/components/admin/AdminLayout";
import REST from "@codeup/rest/src";
import AuthenticateLoadingIndicator from
"@codeup/components/dist/admin/AuthenticateLoadingIndicator";
import ErrorComponent from "@codeup/components/dist/global/ErrorComponent";
import { Box, useToast } from "@chakra-ui/react";
import SuperText from "@codeup/components/dist/landingpage/SuperText";
import { FaEye, FaPlus, FaTrash, FaUser } from "react-icons/fa";
import PopupManager from "../../global/PopupManager";
import GridLayout from "@codeup/components/dist/global/GridLayout";
import { useRouter } from "next/router";
import GridLayoutV2 from "@codeup/components/dist/global/GridLayoutV2";

export default function Flows() {
  const [flows, setFlows] = React.useState<any[]>([]);
  const [error, setError] = React.useState<boolean>(false);
  const [loading, setLoading] = React.useState<boolean>(true);

  const reload = () => {
    REST.Admin.getFlows(localStorage.getItem("token") as string).then((res) => {
      if (res.status === 200) {
        setFlows(res.payload.flows);
      } else {
        setError(true);
      }
```

```
    setLoading(false);
  });
};

React.useEffect(() => {
  reload();
}, []);

const router = useRouter();
const toast = useToast();

return (
  <>
    <AdminLayout>
      {loading ? (
        <AuthenticateLoadingIndicator text={"Lade Flows..."} />
      ) : (
        <>
          {error ? (
            <>
              <ErrorComponent
                description={
                  "Beim Laden der Flows ist ein Fehler aufgetreten."
                }
              />
            </>
          ) : (
            <>
              <GridLayoutV2
                pageTitle={"Flows"}
                search={{
                  show: true,
                  fields: true,
                }}
                topButtons={[]}
                entries={flows.map((p) => {
                  return {
                    title: p.name,
                    fields: [
                      {
                        text: "Besitzer: " + p.owner.username,
                        icon: <FaUser />,
                      },
                    ],
                    buttonRows: [
                      [
                        {
                          text: "Öffnen",
                          leftIcon: <FaEye />,
                          variant: "brand",
                          onClick: () => {
                            router.push("/planning/" + p._id);
                          },
                        },
                        {
                          text: "Löschen",
                          leftIcon: <FaTrash />,
                          colorScheme: "red",
                          onClick: async () => {
                            REST.ToDo.deleteV2Project({
```

```
                                token: localStorage.getItem("token") as string,
                                id: p._id,
                              }).then((res) => {
                                if (res.status === 200) {
                                  toast({
                                    position: "bottom-right",
                                    status: "success",
                                    title: "Erfolgreich",
                                    description: "Das Projekt wurde gelöscht",
                                    isClosable: true,
                                    duration: 1500,
                                  });
                                  reload();
                                } else {
                                  toast({
                                    position: "bottom-right",
                                    status: "error",
                                    title: "Fehler",
                                    description: res.payload.error,
                                    isClosable: true,
                                    duration: 1500,
                                  });
                                }
                              });
                            },
                          },
                        ],
                      ],
                    };
                  })}
                />
              </>
            )}
          </>
        )}
      </AdminLayout>
    </>
  );
}
```

## apps/web/src/pages/admin/ide.tsx

```
/**
 * apps/web/src/pages/admin/ide.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 19.04.2023
 *
 */

import * as React from "react";
import { useRouter } from "next/router";
import REST from "@codeup/rest/src";
import AccountManager from "@codeup/util/dist/AccountManager";
import PopupManager from "../../global/PopupManager";
import AdminLayout from "@codeup/components/admin/AdminLayout";
import AuthenticateLoadingIndicator from
"@codeup/components/dist/admin/AuthenticateLoadingIndicator";
import ErrorComponent from "@codeup/components/dist/global/ErrorComponent";
```

```
import {
  FaEye,
  FaFile,
  FaPen,
  FaPlus,
  FaRocket,
  FaTrash,
  FaUser,
  FaUsers,
} from "react-icons/fa";
import GridLayout from "@codeup/components/dist/global/GridLayout";
import { Box } from "@chakra-ui/react";
import SuperText from "@codeup/components/dist/landingpage/SuperText";
import GridLayoutV2 from "@codeup/components/dist/global/GridLayoutV2";

export default function Kids() {
  const router = useRouter();
  const [error, setError] = React.useState<boolean>(false);
  const [loading, setLoading] = React.useState<boolean>(true);
  const [projects, setProjects] = React.useState<
    {
      _id: string;
      owner: string;
      name: string;
      pinned: string[];
      published: boolean;
      publishedCustomName: string;
      discovery: boolean;
      discoveryDescription: string;
      contributors: any[];
      files: any[];
      updated_at: string;
      created_at: string;
      __v: string;
    }[]
  >([]);

  const reload = () => {
    REST.Admin.getIDEProjects(AccountManager.getActiveToken()).then((res) => {
      if (res.status !== 200) {
        setError(true);
        setLoading(false);
        return;
      }

      setLoading(false);
      setProjects(res.payload.projects);
    });
  };

  React.useEffect(() => {
    reload();
  }, []);
  return (
    <>
      <AdminLayout>
        {loading ? (
          <>
            <AuthenticateLoadingIndicator text={"Lade Projekte..."} />
          </>
```

```
) : (
  <>
    {error ? (
      <>
        <ErrorComponent
          description={"Es ist ein Fehler aufgetreten."}
        />
      </>
    ) : (
      <>
        <GridLayoutV2
          pageTitle={"Legacy Projekte"}
          search={{
            show: true,
            fields: true,
          }}
          topButtons={[]}
          entries={projects.map((p) => ({
            title: p.name,
            fields: [
              {
                text: "Besitzer: " + p.owner,
                icon: <FaUser />,
              },
              {
                text:
                  "Veröffentlicht: " + (p.published ? "Ja" : "Nein"),
                icon: <FaRocket />,
              },
              {
                text: "Mitwirkende: " + p.contributors.length,
                icon: <FaUsers />,
              },
              {
                text: "Dateien: " + p.files.length,
                icon: <FaFile />,
              },
            ],
            buttonRows: [
              [
                {
                  text: "Öffnen",
                  onClick: () => {
                    router.push(`/ide/project/${p._id}`);
                  },
                  leftIcon: <FaEye />,
                  variant: "brand",
                  disabled: false,
                },

                {
                  text: "Löschen",
                  onClick: async () => {
                    if (
                      !(await PopupManager.confirmAsync({
                        title: "Projekt löschen",
                        question:
                          "Bist du dir sicher, dass du das Projekt löschen möchtest?",
                      }))
                    )
```

```
                  return;

                const res = await REST.IDE.deleteProject({
                  token: AccountManager.getActiveToken(),
                  projectId: p._id,
                });

                if (res.status !== 200) {
                  PopupManager.alertAsync({
                    title: "Fehler",
                    description:
                      "Es ist ein Fehler aufgetreten: " +
                      res.payload.error,
                  });
                  return;
                }

                reload();
              },
              leftIcon: <FaTrash />,
              colorScheme: "red",
              disabled: false,
            },
          ],
        ],
      }))}
    />
  </>
 )}
 </>
 )}
 </AdminLayout>
 </>
 );
}
```

## apps/web/src/pages/admin/index.tsx

```
/**
 * apps/web/src/pages/admin/index.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 01.04.2023
 *
 */

import * as React from "react";
import { ReactElement, useState } from "react";
import { useLoggedIn } from "@codeup/hooks";
import AdminLayout from "@codeup/components/admin/AdminLayout";
import Head from "next/head";
import REST from "@codeup/rest/dist";
import AccountManager from "@codeup/util/dist/AccountManager";
import {
  Box,
  Flex,
  Grid,
  Heading,
  HStack,
```

```
  Stack,
  Text,
  useToast,
} from "@chakra-ui/react";
import SuperText from "@codeup/components/dist/landingpage/SuperText";
import {
  ArcElement,
  BarElement,
  CategoryScale,
  Chart as ChartJS,
  Legend,
  LinearScale,
  LineElement,
  PointElement,
  Title,
  Tooltip,
} from "chart.js";
import {
  FaBook,
  FaBookmark,
  FaBookOpen,
  FaCertificate,
  FaChartBar,
  FaCheck,
  FaChild,
  FaCode,
  FaComment,
  FaCross,
  FaLaptopCode,
  FaLink,
  FaNewspaper,
  FaPeopleArrows,
  FaProjectDiagram,
  FaRocket,
  FaSitemap,
  FaUser,
} from "react-icons/fa";
import { BiError } from "react-icons/bi";
import { CHeading } from "@codeup/components/dist/global/CHeading";

export default function Index() {
  ChartJS.register(
    ArcElement,
    Tooltip,
    Legend,
    CategoryScale,
    LinearScale,
    BarElement,
    PointElement,
    LineElement,
    Title
  );
  const { userInfo, loggedIn, loaded } = useLoggedIn();

  const [views, setViews] = useState<number>(0);

  const [stats, setStats] = React.useState<{
    users: {
      count: number;
      admins: number;
```

```
    editors: number;
    newUsers: number;
  };
  orgs: {
    count: number;
    newOrgs: number;
  };
  projects: {
    count: number;
    newProjects: number;
    projectPages: number;
    publishedProjects: number;
    projectFunctions: number;
  };
  forum: {
    profiles: number;
    newProfiles: number;
    posts: number;
    newPosts: number;
  };
  todo: {
    count: number;
  };
  flowCharts: {
    count: number;
  };
  newsletterMembers: number;
  certificates: number;
  kids: number;
  contactMessages: number;
  ideProjects: number;
  roadmaps: number;
  snippets: number;
  courses: number;
  lessons: number;
  shorts: number;
  errors: number;
}>();
React.useEffect(() => {
  REST.Admin.getOverview(AccountManager.getActiveToken()).then((res) => {
    if (res.status === 200) {
      setStats(res.payload as any);
    } else {
      toast({
        title: "Fehler",
        description: "Die Statistiken konnten nicht geladen werden.",
        status: "error",
        duration: 5000,
        isClosable: true,
      });
    }
  });
  REST.Analytics.getSummary(AccountManager.getActiveToken()).then((res) => {
    if (res.status === 200) {
      setViews(res.payload.data.totalViews);
    } else {
      toast({
        title: "Fehler",
        description: "Die Statistiken konnten nicht geladen werden.",
        status: "error",
```

```
        duration: 5000,
        isClosable: true,
      });
    }
  });
}, []);

const toast = useToast();

return (
  <>
    <Head>
      <title>Admin - CodeUp</title>
    </Head>
    <AdminLayout>
      <Flex w={"100%"} minH={"100vh"} h={"fit-content"} direction={"column"}>
        <SuperText>Dashboard</SuperText>
        <Stack>
          <Box p={8} pb={0}>
            <CHeading size={"lg"}>Generell</CHeading>
          </Box>
          <Grid
            templateColumns={[
              "repeat(1, 1fr)",
              "repeat(2, 1fr)",
              "repeat(3, 1fr)",
            ]}
            gap={8}
            p={8}
            maxW={"100%"}
          >
            <DashCard
              icon={<FaChartBar />}
              title={"Aufrufe"}
              number={views!.toString() as string}
              color={"orange.400"}
            />
            <DashCard
              icon={<FaComment />}
              title={"kontaktanfragen"}
              number={stats?.contactMessages!.toString() as string}
              color={"orange.400"}
            />
            <DashCard
              icon={<FaLink />}
              title={"shorts"}
              number={stats?.shorts!.toString() as string}
              color={"orange.400"}
            />
            <DashCard
              icon={<BiError />}
              title={"Fehler"}
              number={stats?.errors!.toString() as string}
              color={"orange.400"}
            />
          </Grid>
          <Box p={8} pb={0}>
            <CHeading size={"lg"}>Benutzer</CHeading>
          </Box>
          <Grid
```

```
  templateColumns={[
    "repeat(1, 1fr)",
    "repeat(2, 1fr)",
    "repeat(3, 1fr)",
  ]}
  gap={8}
  p={8}
  maxW={"100%"}
>
  <DashCard
    icon={<FaUser />}
    title={"benutzer"}
    number={stats?.users.count!.toString() as string}
    subtitle={`${stats?.users.newUsers} neue Benutzer`}
    color={"green.300"}
  />
  <DashCard
    icon={<FaNewspaper />}
    title={"newsletter"}
    number={stats?.newsletterMembers!.toString() as string}
    color={"green.300"}
  />
  <DashCard
    icon={<FaSitemap />}
    title={"organisationen"}
    number={stats?.orgs.count!.toString() as string}
    subtitle={`${stats?.orgs.newOrgs} neue Organisationen`}
    color={"green.300"} // purple.300
  />
</Grid>

<Box p={8} pb={0}>
  <CHeading size={"lg"}>Entwicklung</CHeading>
</Box>

<Grid
  templateColumns={[
    "repeat(1, 1fr)",
    "repeat(2, 1fr)",
    "repeat(3, 1fr)",
  ]}
  gap={8}
  p={8}
  maxW={"100%"}
>
  <DashCard
    icon={<FaCode />}
    title={"projekte"}
    number={stats?.projects.count!.toString() as string}
    subtitle={`${stats?.projects.newProjects} neue Projekte`}
    color={"red.300"}
  />
  <DashCard
    icon={<FaCode />}
    title={"projektseiten"}
    number={stats?.projects.projectPages!.toString() as string}
    subtitle={`${stats?.projects.projectFunctions} Funktionen`}
    color={"red.300"}
  />
  <DashCard
```

```
    icon={<FaRocket />}
    title={"roadmaps"}
    number={stats?.roadmaps!.toString() as string}
    color={"red.300"}
  />
  <DashCard
    icon={<FaLaptopCode />}
    title={"snippets"}
    number={stats?.snippets!.toString() as string}
    color={"red.300"}
  />
  <DashCard
    icon={<FaChild />}
    title={"kids"}
    number={stats?.kids!.toString() as string}
    color={"red.300"}
  />
  <DashCard
    icon={<FaCode />}
    title={"legacy projekte"}
    number={stats?.ideProjects!.toString() as string}
    color={"red.300"}
  />
</Grid>

<Box p={8} pb={0}>
  <CHeading size={"lg"}>Austausch</CHeading>
</Box>

<Grid
  templateColumns={[
    "repeat(1, 1fr)",
    "repeat(2, 1fr)",
    "repeat(3, 1fr)",
  ]}
  gap={8}
  p={8}
  maxW={"100%"}
>
  <DashCard
    icon={<FaPeopleArrows />}
    title={"profile"}
    number={stats?.forum.profiles!.toString() as string}
    subtitle={`${stats?.forum.newProfiles} neue Profile`}
    color={"pink.400"}
  />
  <DashCard
    icon={<FaComment />}
    title={"posts"}
    number={stats?.forum.posts!.toString() as string}
    subtitle={`${stats?.forum.newPosts} neue Posts`}
    color={"pink.400"}
  />
</Grid>

<Box p={8} pb={0}>
  <CHeading size={"lg"}>Planung</CHeading>
</Box>

<Grid
```

```
          templateColumns={[
            "repeat(1, 1fr)",
            "repeat(2, 1fr)",
            "repeat(3, 1fr)",
          ]}
          gap={8}
          p={8}
          maxW={"100%"}
        >
          <DashCard
            icon={<FaCheck />}
            title={"todos"}
            number={stats?.todo.count!.toString() as string}
            color={"blue.400"}
          />
          <DashCard
            icon={<FaProjectDiagram />}
            title={"flows"}
            number={stats?.flowCharts.count!.toString() as string}
            color={"blue.400"}
          />
        </Grid>

        <Box p={8} pb={0}>
          <CHeading size={"lg"}>Lernen</CHeading>
        </Box>

        <Grid
          templateColumns={[
            "repeat(1, 1fr)",
            "repeat(2, 1fr)",
            "repeat(3, 1fr)",
          ]}
          gap={8}
          p={8}
          maxW={"100%"}
        >
          <DashCard
            icon={<FaBook />}
            title={"kurse"}
            number={stats?.courses!.toString() as string}
            color={"teal.600"}
          />
          <DashCard
            icon={<FaBookmark />}
            title={"lektionen"}
            number={stats?.lessons!.toString() as string}
            color={"teal.600"}
          />
          <DashCard
            icon={<FaCertificate />}
            title={"zertifikate"}
            number={stats?.certificates!.toString() as string}
            color={"teal.600"}
          />
        </Grid>
      </Stack>
    </Flex>
  </AdminLayout>
</>
```

```
  );
}

const DashCard = (props: {
  icon: ReactElement;
  title: string;
  number: string;
  subtitle?: string;
  color: string;
}) => {
  return (
    <>
      <Box
        w={"100%"}
        boxShadow={"xl"}
        backgroundColor={"gray.900"}
        rounded={"xl"}
        maxW={"600px"}
        minW={"fit-content"}
      >
        <HStack>
          <Heading size={"2xl"} p={8} color={"brand.500"}>
            {props.icon}
          </Heading>
          <Box pl={8} pt={4} pb={4} pr={4}>
            <Text
              fontSize={"md"}
              textAlign={"left"}
              color={"gray.200"}
              fontWeight={"extrabold"}
            >
              {props.title.toUpperCase()}
            </Text>
            <Heading size={"xl"} color={"brand.500"}>
              {props.number}
            </Heading>

            {props.subtitle && (
              <>
                <Text
                  fontSize={"md"}
                  textAlign={"left"}
                  color={"gray.200"}
                  fontWeight={"extrabold"}
                >
                  {props.subtitle}
                </Text>
              </>
            )}
          </Box>
        </HStack>
      </Box>
    </>
  );
};
```

## apps/web/src/pages/admin/kids.tsx

```
/**
 * apps/web/src/pages/admin/kids.tsx
 *
```

```
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 13.04.2023
 *
 */

import * as React from "react";
import { useRouter } from "next/router";
import REST from "@codeup/rest/src";
import AccountManager from "@codeup/util/dist/AccountManager";
import PopupManager from "../../global/PopupManager";
import AdminLayout from "@codeup/components/admin/AdminLayout";
import AuthenticateLoadingIndicator from
"@codeup/components/dist/admin/AuthenticateLoadingIndicator";
import ErrorComponent from "@codeup/components/dist/global/ErrorComponent";
import { FaEye, FaPen, FaPlus, FaTrash, FaUser } from "react-icons/fa";
import GridLayout from "@codeup/components/dist/global/GridLayout";
import { Box } from "@chakra-ui/react";
import SuperText from "@codeup/components/dist/landingpage/SuperText";
import GridLayoutV2 from "@codeup/components/dist/global/GridLayoutV2";

export default function Kids() {
  const router = useRouter();
  const [error, setError] = React.useState<boolean>(false);
  const [loading, setLoading] = React.useState<boolean>(true);
  const [projects, setProjects] = React.useState<
    {
      _id: string;
      name: string;
      code: string;
      owner: {
        username: string;
      };
    }[]
  >([]);

  const reload = () => {
    REST.Admin.getKidsProjects(AccountManager.getActiveToken()).then((res) => {
      if (res.status !== 200) {
        setError(true);
        setLoading(false);
        return;
      }

      setLoading(false);
      setProjects(res.payload.projects);
    });
  };

  React.useEffect(() => {
    reload();
  }, []);
  return (
    <>
      <AdminLayout>
        {loading ? (
          <>
            <AuthenticateLoadingIndicator text={"Lade Projekte..."} />
          </>
```

```
) : (
  <>
    {error ? (
      <>
        <ErrorComponent
          description={"Es ist ein Fehler aufgetreten."}
        />
      </>
    ) : (
      <>
        <GridLayoutV2
          pageTitle={"Kids Projekte"}
          search={{
            show: true,
            fields: true,
          }}
          topButtons={[]}
          entries={projects.map((p) => ({
            title: p.name,
            fields: [
              {
                text: "Besitzer: " + p.owner.username,
                icon: <FaUser />,
              },
            ],
            buttonRows: [
              [
                {
                  text: "Öffnen",
                  onClick: () => {
                    router.push(`/kids/${p._id}`);
                  },
                  leftIcon: <FaEye />,
                  variant: "brand",
                  disabled: false,
                },
                {
                  text: "Umbenennen",
                  onClick: async () => {
                    const newName = await PopupManager.promptAsync({
                      title: "Projekt umbenennen",
                      inputType: "INPUT",
                      helperText:
                        "Gib einen neuen Namen für dein Projekt ein",
                    });

                    if (!newName) return;

                    const res = await REST.Kids.updateProject({
                      token: AccountManager.getActiveToken(),
                      name: newName,
                      id: p._id,
                    });

                    if (res.status !== 200) {
                      PopupManager.alertAsync({
                        title: "Fehler",
                        description:
                          "Es ist ein Fehler aufgetreten: " +
                          res.payload.error,
```

```
                      });
                      return;
                    }

                    reload();

                    PopupManager.alertAsync({
                      title: "Projekt umbenannt",
                      description:
                        "Dein Projekt wurde erfolgreich umbenannt.",
                    });
                  },
                  leftIcon: <FaPen />,
                  variant: "brand",
                  disabled: false,
                },
                {
                  text: "Löschen",
                  onClick: async () => {
                    if (
                      !(await PopupManager.confirmAsync({
                        title: "Projekt löschen",
                        question:
                          "Bist du dir sicher, dass du das Projekt löschen möchtest?",
                      }))
                    )
                      return;

                    const res = await REST.Kids.deleteProject({
                      token: AccountManager.getActiveToken(),
                      id: p._id,
                    });

                    if (res.status !== 200) {
                      PopupManager.alertAsync({
                        title: "Fehler",
                        description:
                          "Es ist ein Fehler aufgetreten: " +
                          res.payload.error,
                      });
                      return;
                    }

                    reload();
                  },
                  leftIcon: <FaTrash />,
                  colorScheme: "red",
                  disabled: false,
                },
              ],
            ],
          }))}
        />
      </>
    )}
  </>
  </AdminLayout>
);
```

```
}
```

## apps/web/src/pages/admin/newsletter.tsx

```
/**
 * apps/web/src/pages/admin/newsletter.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 13.04.2023
 *
 */

import * as React from "react";
import AdminLayout from "@codeup/components/admin/AdminLayout";
import {
  Box,
  Button,
  ButtonGroup,
  Divider,
  Flex,
  Heading,
  Input,
  InputGroup,
  Text,
  Textarea,
} from "@chakra-ui/react";
import SuperText from "@codeup/components/dist/landingpage/SuperText";
import REST from "@codeup/rest/src";
import AccountManager from "@codeup/util/dist/AccountManager";
import ErrorComponent from "@codeup/components/dist/global/ErrorComponent";
import PopupManager from "../../global/PopupManager";

export default function Newsletter() {
  const [newsletterMembers, setNewsletterMembers] = React.useState<number>(0);
  const [error, setError] = React.useState<boolean>(false);

  React.useEffect(() => {
    REST.Admin.getNewsletterMembers({
      token: AccountManager.getActiveToken(),
      count: true,
    }).then((res) => {
      if (res.status === 200) {
        setNewsletterMembers(res.payload.count);
      } else {
        setError(true);
      }
    });
  }, []);

  return (
    <>
      <AdminLayout>
        <Box mt={8}></Box>
        <SuperText>Newsletter</SuperText>
        {error ? (
          <>
            <ErrorComponent description={"Ein Fehler ist aufgetreten!"} />
          </>
        ) : (
```

```
<>
  <Text textAlign={"center"} fontSize={"lg"} mt={4}>
    Es sind {newsletterMembers} Mitglieder in der Newsletterliste
  </Text>
  <Flex
    w={"100%"}
    justifyContent={"center"}
    alignItems={"center"}
    mt={8}
  >
    <Box
      bg={"black"}
      rounded={"xl"}
      w={["100%", "50%"]}
      boxShadow={"xl"}
      p={4}
    >
      <Input placeholder={"Titel"} id={"newsletter-title"} />
      <Textarea
        w={"100%"}
        resize={"vertical"}
        placeholder={"Inhalt des Newsletters"}
        mt={4}
        minH={"45vh"}
        id={"newsletter-content"}
      />
      <Button
        w={"100%"}
        variant={"brand"}
        mt={4}
        onClick={async () => {
          const title = (
            document.getElementById(
              "newsletter-title"
            ) as HTMLInputElement
          ).value;
          const content = (
            document.getElementById(
              "newsletter-content"
            ) as HTMLInputElement
          ).value;

          const res = await REST.Admin.sendNewsletter({
            token: AccountManager.getActiveToken(),
            content,
            title,
          });

          if (res.status === 200) {
            PopupManager.alert({
              title: "Erfolgreich",
              description:
                "Der Newsletter wurde erfolgreich versendet!",
            });
          } else {
            PopupManager.alert({
              title: "Fehler",
              description:
                "Es ist ein Fehler aufgetreten: " + res.payload.error,
            });
```

```
                }
              }}
            >
              Newsletter versenden
            </Button>
          </Box>
        </Flex>
      </>
    )}
  </AdminLayout>
 </>
 );
}
```

## apps/web/src/pages/admin/orgs/index.tsx

```
/**
 * apps/web/src/pages/admin/orgs/index.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 13.04.2023
 *
 */

import * as React from "react";
import AdminLayout from "@codeup/components/dist/admin/AdminLayout";
import AuthenticateLoadingIndicator from
"@codeup/components/dist/admin/AuthenticateLoadingIndicator";
import REST from "@codeup/rest/src";
import AccountManager from "@codeup/util/dist/AccountManager";
import ErrorComponent from "@codeup/components/dist/global/ErrorComponent";
import { Box } from "@chakra-ui/react";
import SuperText from "@codeup/components/dist/landingpage/SuperText";
import GridLayout from "@codeup/components/dist/global/GridLayout";
import {
  FaClock,
  FaEye,
  FaInfo,
  FaMoneyBill,
  FaUser,
  FaUsers,
} from "react-icons/fa";
import { useRouter } from "next/router";
import GridLayoutV2 from "@codeup/components/dist/global/GridLayoutV2";

export default function Index() {
  const router = useRouter();
  const [orgs, setOrgs] = React.useState<
    {
      _id: string;
      owner: {
        username: string;
      };
      name: string;
      members: string[];
      icon: string;
      subscription: string;
      createdAt: string;
      __v: number;
```

```
  }[]
>([]);

const [loading, setLoading] = React.useState(true);
const [error, setError] = React.useState(false);

React.useEffect(() => {
  REST.Admin.getOrgs(AccountManager.getActiveToken()).then((res) => {
    if (res.status === 200) {
      setOrgs(res.payload.orgs);
      setLoading(false);
    } else {
      setError(true);
      setLoading(false);
    }
  });
}, []);

return (
  <>
    <AdminLayout>
      {loading ? (
        <>
          <AuthenticateLoadingIndicator text={"Laden..."} />
        </>
      ) : (
        <>
          {error ? (
            <>
              <ErrorComponent
                description={"Es ist ein Fehler aufgetreten!"}
                title={"Fehler"}
              />
            </>
          ) : (
            <>
              <GridLayoutV2
                pageTitle={"Organisationen"}
                search={{
                  show: true,
                  fields: true,
                }}
                topButtons={[]}
                entries={orgs.map((org) => {
                  return {
                    title: org.name,
                    buttonRows: [
                      [
                        {
                          text: "Anzeigen",
                          leftIcon: <FaEye />,
                          disabled: false,
                          variant: "brand",
                          onClick: () => {
                            router.push("/orgs/" + org.name + "/manage");
                          },
                        },
                      ],
                    ],
                    fields: [
```

```
                    {
                      text: "ID: " + org._id,
                      icon: <FaInfo />,
                    },
                    {
                      text: "Mitglieder: " + org.members.length,
                      icon: <FaUsers />,
                    },
                    {
                      text:
                        "Erstellt am: " +
                        new Date(org.createdAt).toLocaleString(),
                      icon: <FaClock />,
                    },
                    {
                      text: "Abonnement: " + org.subscription,
                      icon: <FaMoneyBill />,
                    },
                    {
                      text: "Besitzer: " + org.owner.username,
                      icon: <FaUser />,
                    },
                  ],
                };
              })}
            />
          </>
        )}
      </>
    )}
  </AdminLayout>
 </>
 );
}
```

## apps/web/src/pages/admin/profiles/index.tsx

```
/**
 * apps/web/src/pages/admin/profiles/index.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 08.10.22
 *
 */
import AdminLayout from "@codeup/components/admin/AdminLayout";
import {
  Avatar,
  Box,
  Button,
  Flex,
  Heading,
  useDisclosure,
} from "@chakra-ui/react";
import router from "next/router";
import { FaHome, FaPlus } from "react-icons/fa";
import REST from "@codeup/rest";
import { useEffect, useState } from "react";
import DataTable from "@codeup/components/extra/DataTable";
import AdminProfileViewer from "@codeup/components/admin/AdminProfileViewer";
```

```
import Head from "next/head";
import * as React from "react";
import SuperText from "@codeup/components/dist/landingpage/SuperText";
import { CHeading } from "@codeup/components/dist/global/CHeading";

export default function AdminProfiles() {
  const [profiles, setProfiles] = useState<any[]>([]);

  const [loaded, setLoaded] = useState<boolean>(true);

  const [error, setError] = useState<string>("");

  const { isOpen, onOpen, onClose } = useDisclosure();

  const [currentProfile, setCurrentProfile] = useState<any>(null);

  const reload = () => {
    REST.Admin.listForumProfiles(localStorage.getItem("token") as string).then(
      (res) => {
        if (res.status === 200) {
          setProfiles(res.payload.profiles);
          setLoaded(true);
        } else {
          setError("Profile konnten nicht geladen werden.");
          setLoaded(true);
        }
      }
    );
  };
  useEffect(() => {
    reload();
  }, []);

  const profileToRow = (profile: any) => {
    return [
      () => {
        setCurrentProfile(profile);
        onOpen();
      },
      <Avatar
        size={"md"}
        src={"/api/account/" + profile.username + "/picture"}
      />,
      profile.username,
      profile.displayName,
      profile.verified ? "Ja" : "Nein",
      profile.bio,
      profile.website,
      profile.location,
      new Date(profile.createdAt).toLocaleString(),
    ];
  };

  return (
    <>
      <Head>
        <title>Profile - Admin - CodeUp</title>
      </Head>
      <AdminLayout>
        <Flex w={"100%"} minH={"100vh"} h="fit-content" direction={"column"}>
```

```
<Box textAlign={"center"}>
  <CHeading size={"xl"}>Profile</CHeading>
</Box>

<Flex
  w={"100%"}
  h={"100%"}
  direction={"column"}
  gap={"6rem"}
  alignItems={"center"}
  mt={4}
  p={"2rem"}
>
  <Box w={"100%"}>
    <DataTable
      columns={[
        "Profilbild",
        "Benutzername",
        "Anzeigename",
        "Verifiziert",
        "Biography",
        "Webseite",
        "Ort",
        "Erstellt",
      ]}
      data={profiles.map(profileToRow)}
      filter={(q: string, f: string, s: string) => {
        let x = profiles.filter((profile: any) => {
          return (
            profile.username
              .toLowerCase()
              .includes(q.toLowerCase()) ||
            profile.displayName
              .toLowerCase()
              .includes(q.toLowerCase()) ||
            profile.bio.toLowerCase().includes(q.toLowerCase()) ||
            profile.website.toLowerCase().includes(q.toLowerCase()) ||
            profile.location.toLowerCase().includes(q.toLowerCase())
          );
        });

        switch (f.toLowerCase()) {
          case "alle":
            break;
          case "verifizierte":
            x = x.filter((profile: any) => profile.verified);
            break;
          default:
            break;
        }

        switch (s.toLowerCase()) {
          case "alphabetisch":
            x = x.sort((a: any, b: any) => {
              if (
                a.username.toLowerCase() < b.username.toLowerCase()
              ) {
                return -1;
              } else if (
                a.username.toLowerCase() > b.username.toLowerCase()
```

```
                    ) {
                      return 1;
                    } else {
                      return 0;
                    }
                  });
                  break;
                case "neuste zuerst":
                  x = x.sort((a: any, b: any) => {
                    return (
                      new Date(b.createdAt).getTime() -
                      new Date(a.createdAt).getTime()
                    );
                  });
                  break;
                case "älteste zuerst":
                  x = x.sort((a: any, b: any) => {
                    return (
                      new Date(a.createdAt).getTime() -
                      new Date(b.createdAt).getTime()
                    );
                  });
                  break;
                default:
                  break;
              }

              return x.map(profileToRow);
            }}
            filterOptions={["Alle", "Verifizierte"]}
            sortOptions={[
              "Alphabetisch",
              "Neuste zuerst",
              "Älteste zuerst",
            ]}
            reload={reload}
          />
        </Box>
      </Flex>
    </Flex>
    <AdminProfileViewer
      onOpen={onOpen}
      onClose={onClose}
      isOpen={isOpen}
      profile={currentProfile}
      setCurrentProfile={setCurrentProfile}
      setProfiles={setProfiles}
      profiles={profiles}
    />
  </AdminLayout>
 </>
 );
}
```

## apps/web/src/pages/admin/project-ideas/index.tsx

```
/**
 * apps/web/src/pages/admin/project-ideas/index.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
```

```
 * License: Project License
 * Created At: 03.12.2022
 *
 */

import * as React from "react";
import { useEffect } from "react";
import { useLoggedIn } from "@codeup/hooks";
import AdminLayout from "@codeup/components/admin/AdminLayout";
import REST from "@codeup/rest";
import {
  Badge,
  Box,
  Button,
  ButtonGroup,
  Flex,
  FormControl,
  FormLabel,
  Grid,
  Heading,
  Input,
  Popover,
  PopoverArrow,
  PopoverBody,
  PopoverCloseButton,
  PopoverContent,
  PopoverTrigger,
  Portal,
  Select,
  Stack,
  Text,
  Textarea,
  useToast,
} from "@chakra-ui/react";
import { FaCheck, FaInfo, FaPen, FaPlus, FaTrash } from "react-icons/fa";
import { Card, CardBody, CardFooter } from "@chakra-ui/card";
import Head from "next/head";
import SuperText from "@codeup/components/dist/landingpage/SuperText";
import GridLayoutV2 from "@codeup/components/dist/global/GridLayoutV2";
import PopupManager from "../../../global/PopupManager";

export default function Editor() {
  const { userInfo, loggedIn, loaded } = useLoggedIn();

  const [projectIdeas, setProjectIdeas] = React.useState<
    {
      _id: string;
      title: string;
      description: string;
      exampleDeployedURL?: string;
      exampleSourceURL?: string;
      level: string;
    }[]
  >([]);

  const [filteredProjectIdeas, setFilteredProjectIdeas] = React.useState<
    {
      _id: string;
      title: string;
      description: string;
```

```
    exampleDeployedURL?: string;
    exampleSourceURL?: string;
    level: string;
  }[]
>([]);

const toast = useToast();

const reload = () => {
  REST.Admin.getProjectIdeasAdmin(
    localStorage.getItem("token") as string
  ).then((res) => {
    if (res.status === 200) {
      setProjectIdeas(res.payload.ideas);
      setFilteredProjectIdeas(res.payload.ideas);
    } else {
      toast({
        title: "Fehler",
        description: "Ein Fehler ist aufgetreten: " + res.payload.error,
        status: "error",
        duration: 1500,
        isClosable: true,
        position: "bottom-right",
      });
    }
  });
};

useEffect(() => {
  if (loaded && loggedIn && userInfo?.role === "admin") {
    reload();
  }
}, [userInfo, loggedIn, loaded]);

const [query, setQuery] = React.useState<string>("");

useEffect(() => {
  setFilteredProjectIdeas(
    projectIdeas.filter((idea) =>
      idea.title.toLowerCase().includes(query.toLowerCase())
    )
  );
}, [query]);

return (
  <>
    <Head>
      <title>Projektideen - Admin - CodeUp</title>
    </Head>
    <AdminLayout>
      <GridLayoutV2
        pageTitle={"Projektideen"}
        search={{
          show: true,
          fields: true,
        }}
        topButtons={[
          {
            text: "Neu",
            leftIcon: <FaPlus />,
```

```
variant: "brand",
onClick: async () => {
  await PopupManager.alertAsync({
    title: "Neue Projektidee",
    description: (
      <>
        <form
          onSubmit={(e) => {
            e.preventDefault();

            const data = new FormData(
              e.target as HTMLFormElement
            );

            const title = data.get("title") as string;
            const description = data.get("description") as string;
            const exampleDeployedURL = data.get(
              "exampleDeployedURL"
            ) as string;
            const exampleSourceURL = data.get(
              "exampleSourceURL"
            ) as string;
            const level = data.get("level") as string;

            REST.Admin.createProjectIdea({
              token: localStorage.getItem("token") as string,
              title,
              description,
              exampleDeployedURL,
              exampleSourceURL,
              level,
            }).then((res) => {
              PopupManager.removeCurrentPopup();
              if (res.status === 200) {
                toast({
                  title: "Erfolgreich",
                  description:
                    "Die Projekt Idee wurde erfolgreich erstellt!",
                  status: "success",
                  duration: 1500,
                  isClosable: true,
                  position: "bottom-right",
                });
                reload();
              } else {
                toast({
                  title: "Fehler",
                  description:
                    "Ein Fehler ist aufgetreten: " +
                    res.payload.error,
                  status: "error",
                  duration: 1500,
                  isClosable: true,
                  position: "bottom-right",
                });
              }
            });
          }}
        >
          <FormControl isRequired>
```

```
              <FormLabel>Titel</FormLabel>
              <Input placeholder={"Titel"} name={"title"} />
            </FormControl>
            <FormControl isRequired mt={4}>
              <FormLabel>Beschreibung</FormLabel>
              <Textarea
                placeholder={"Beschreibung"}
                name={"description"}
              />
            </FormControl>
            <FormControl isRequired mt={4}>
              <FormLabel>Level</FormLabel>
              <Select name={"level"}>
                <option value={"easy"}>Einfach</option>
                <option value={"medium"}>Mittel</option>
                <option value={"hard"}>Schwer</option>
              </Select>
            </FormControl>
            <FormControl mt={4}>
              <FormLabel>Beispiel/Vorschau</FormLabel>
              <Input
                placeholder={"https://codeup.space"}
                name={"exampleDeployedURL"}
                type={"url"}
              />
            </FormControl>
            <FormControl mt={4}>
              <FormLabel>Beispiel/Code</FormLabel>
              <Input
                placeholder={"https://github.com/MCTzOCK/codeup"}
                name={"exampleSourceURL"}
                type={"url"}
              />
            </FormControl>
            <ButtonGroup mt={4} w={"100%"}>
              <Button
                w={"100%"}
                leftIcon={<FaPlus />}
                type={"submit"}
                variant={"brand"}
              >
                Erstellen
              </Button>
            </ButtonGroup>
          </form>
        </>
      ),
    });
  },
 },
]}
entries={projectIdeas.map((idea) => {
 return {
  title: idea.title,
  fields: [
   {
    text: "ID: " + idea._id,
    icon: <FaInfo />,
   },
   {
```

```
    text:
      "Level: " +
      (idea.level === "easy"
        ? "Einfach"
        : idea.level === "medium"
        ? "Mittel"
        : "Schwer"),
    icon: <FaInfo />,
  },
  {
    text: idea.description,
    icon: <FaInfo />,
  },
  {
    text: "Deployment URL: " + idea.exampleDeployedURL || "K/A",
    icon: <FaInfo />,
  },
  {
    text: "Quellcode-URL: " + idea.exampleSourceURL || "K/A",
    icon: <FaInfo />,
  },
],
buttonRows: [
  [
    {
      text: "Bearbeiten",
      leftIcon: <FaPen />,
      variant: "brand",
      onClick: async () => {
        await PopupManager.alertAsync({
          title: "Bearbeiten",
          description: (
            <>
              <form
                onSubmit={async (e) => {
                  e.preventDefault();

                  const data = new FormData(
                    e.target as HTMLFormElement
                  );

                  const title = data.get("title") as string;
                  const description = data.get(
                    "description"
                  ) as string;
                  const exampleDeployedURL = data.get(
                    "exampleDeployedURL"
                  ) as string;
                  const exampleSourceURL = data.get(
                    "exampleSourceURL"
                  ) as string;
                  const level = data.get("level") as string;

                  const res0 = await REST.Admin.deleteProjectIdea(
                    {
                      token: localStorage.getItem(
                        "token"
                      ) as string,
                      id: idea._id,
                    }
```

```
    );

    if (res0.status === 200) {
      const res1 =
        await REST.Admin.createProjectIdea({
          token: localStorage.getItem(
            "token"
          ) as string,
          title,
          description,
          exampleDeployedURL,
          exampleSourceURL,
          level,
        });

      PopupManager.removeCurrentPopup();

      if (res1.status === 200) {
        toast({
          title: "Erfolgreich",
          description:
            "Die Projekt Idee wurde erfolgreich gespeichert!",
          status: "success",
          duration: 1500,
          isClosable: true,
          position: "bottom-right",
        });
        reload();
      } else {
        toast({
          title: "Fehler",
          description:
            "Ein Fehler ist aufgetreten: " +
            res1.payload.error,
          status: "error",
          duration: 1500,
          isClosable: true,
          position: "bottom-right",
        });
      }
    } else {
      toast({
        title: "Fehler",
        description:
          "Ein Fehler ist aufgetreten: " +
          res0.payload.error,
        status: "error",
        duration: 1500,
        isClosable: true,
        position: "bottom-right",
      });
    }
  }}
>
  <FormControl isRequired>
    <FormLabel>Titel</FormLabel>
    <Input
      placeholder={"Titel"}
      name={"title"}
      defaultValue={idea.title}
```

```
  />
</FormControl>
<FormControl isRequired mt={4}>
  <FormLabel>Beschreibung</FormLabel>
  <Textarea
    defaultValue={idea.description}
    placeholder={"Beschreibung"}
    name={"description"}
  />
</FormControl>
<FormControl isRequired mt={4}>
  <FormLabel>Level</FormLabel>
  <Select name={"level"}>
    <option
      value={"easy"}
      selected={idea.level === "easy"}
    >
      Einfach
    </option>
    <option
      value={"medium"}
      selected={idea.level === "medium"}
    >
      Mittel
    </option>
    <option
      value={"hard"}
      selected={idea.level === "hard"}
    >
      Schwer
    </option>
  </Select>
</FormControl>
<FormControl mt={4}>
  <FormLabel>Beispiel/Vorschau</FormLabel>
  <Input
    placeholder={"https://codeup.space"}
    name={"exampleDeployedURL"}
    type={"url"}
    defaultValue={idea.exampleDeployedURL}
  />
</FormControl>
<FormControl mt={4}>
  <FormLabel>Beispiel/Code</FormLabel>
  <Input
    placeholder={
      "https://github.com/MCTzOCK/codeup"
    }
    name={"exampleSourceURL"}
    type={"url"}
    defaultValue={idea.exampleSourceURL}
  />
</FormControl>
<ButtonGroup mt={4} w={"100%"}>
  <Button
    w={"100%"}
    leftIcon={<FaCheck />}
    type={"submit"}
    variant={"brand"}
  >
```

```
                        Abschließen
                      </Button>
                    </ButtonGroup>
                  </form>
                </>
              ),
            });
          },
        },
        {
          text: "Löschen",
          leftIcon: <FaTrash />,
          colorScheme: "red",
          onClick: async () => {
            REST.Admin.deleteProjectIdea({
              token: localStorage.getItem("token") as string,
              id: idea._id,
            }).then((res) => {
              if (res.status === 200) {
                toast({
                  title: "Erfolgreich",
                  description:
                    "Die Projekt Idee wurde erfolgreich gelöscht!",
                  status: "success",
                  duration: 1500,
                  isClosable: true,
                  position: "bottom-right",
                });
                reload();
              } else {
                toast({
                  title: "Fehler",
                  description:
                    "Ein Fehler ist aufgetreten: " +
                    res.payload.error,
                  status: "error",
                  duration: 1500,
                  isClosable: true,
                  position: "bottom-right",
                });
              }
            });
          },
        },
      ],
    ],
  };
})}
/>
</AdminLayout>
</>
);
}
```

## apps/web/src/pages/admin/roads.tsx

```
/**
 * apps/web/src/pages/admin/roads.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
```

```
 * License: Project License
 * Created At: 13.04.2023
 *
 */

import * as React from "react";
import { useRouter } from "next/router";
import REST from "@codeup/rest/src";
import AccountManager from "@codeup/util/dist/AccountManager";
import PopupManager from "../../global/PopupManager";
import AdminLayout from "@codeup/components/admin/AdminLayout";
import AuthenticateLoadingIndicator from
"@codeup/components/dist/admin/AuthenticateLoadingIndicator";
import ErrorComponent from "@codeup/components/dist/global/ErrorComponent";
import {
  FaBookOpen,
  FaCheck,
  FaCode,
  FaEye,
  FaLightbulb,
  FaPen,
  FaPlus,
  FaProjectDiagram,
  FaTrash,
  FaUser,
} from "react-icons/fa";
import GridLayout from "@codeup/components/dist/global/GridLayout";
import { Box } from "@chakra-ui/react";
import SuperText from "@codeup/components/dist/landingpage/SuperText";
import GridLayoutV2 from "@codeup/components/dist/global/GridLayoutV2";

export default function Kids() {
  const router = useRouter();
  const [error, setError] = React.useState<boolean>(false);
  const [loading, setLoading] = React.useState<boolean>(true);
  const [roadmaps, setRoadmaps] = React.useState<
    {
      owner: {
        username: string;
      };
      _id: string;
      name: string;
      projectIdea: string;
      linkedProject: string;
      linkedCourses: string[];
      linkedFlows: string[];
      linkedTodos: string[];
    }[]
  >([]);

  const reload = () => {
    REST.Admin.getRoads(AccountManager.getActiveToken()).then((res) => {
      if (res.status !== 200) {
        setError(true);
        setLoading(false);
        return;
      } else {
        setLoading(false);
        setRoadmaps(res.payload.roads);
      }
```

```
  });
};

React.useEffect(() => {
  reload();
}, []);
return (
  <>
    <AdminLayout>
      {loading ? (
        <>
          <AuthenticateLoadingIndicator text={"Lade Roads..."} />
        </>
      ) : (
        <>
          {error ? (
            <>
              <ErrorComponent
                description={"Es ist ein Fehler aufgetreten."}
              />
            </>
          ) : (
            <>
              <GridLayoutV2
                pageTitle={"Roadmaps"}
                search={{
                  show: true,
                  fields: true,
                }}
                topButtons={[]}
                entries={roadmaps.map((r) => {
                  return {
                    fields: [
                      {
                        icon: <FaUser />,
                        text: "Besitzer: " + r.owner.username,
                      },
                      {
                        icon: <FaLightbulb />,
                        text:
                          "Idee Verlinkt: " + (r.projectIdea ? "Ja" : "Nein"),
                      },
                      {
                        icon: <FaCode />,
                        text:
                          "Projekt Verlinkt: " +
                          (r.linkedProject ? "Ja" : "Nein"),
                      },
                      {
                        icon: <FaBookOpen />,
                        text: "Kurse Verlinkt: " + r.linkedCourses.length,
                      },
                      {
                        icon: <FaProjectDiagram />,
                        text: "Flows Verlinkt: " + r.linkedFlows.length,
                      },
                      {
                        icon: <FaCheck />,
                        text: "Todos Verlinkt: " + r.linkedTodos.length,
                      },
```

```
],
title: r.name,
buttonRows: [
 [
  {
   text: "Öffnen",
   leftIcon: <FaEye />,
   disabled: false,
   variant: "brand",
   onClick: () => {
    router.push("/roadmap/" + r._id);
   },
  },
  {
   text: "Löschen",
   leftIcon: <FaTrash />,
   disabled: false,
   colorScheme: "red",
   onClick: async () => {
    if (
     !(await PopupManager.confirmAsync({
      title: "Löschen",
      question:
       "Willst du die Roadmap wirklich löschen?",
     }))
    ) {
     return;
    }

    const res = await REST.Roadmap.deleteRoad({
     token: AccountManager.getActiveToken(),
     id: r._id,
    });

    if (res.status !== 200) {
     PopupManager.alert({
      title: "Fehler",
      description:
       "Es ist ein Fehler aufgetreten: " +
       res.payload.error,
     });
     return;
    }

    reload();
   },
  },
 ],
 [
  {
   text: "Umbenennen",
   leftIcon: <FaPen />,
   variant: "brand",
   disabled: false,
   onClick: async () => {
    const name = await PopupManager.promptAsync({
     title: "Umbenennen",
     helperText:
      "Gib einen neuen Namen für die Roadmap ein",
     inputType: "INPUT",
```

```
                        });

                        if (!name) return;

                        const res = await REST.Roadmap.updateRoad({
                          token: AccountManager.getActiveToken(),
                          update: {
                            name,
                          },
                          id: r._id,
                        });

                        if (res.status !== 200) {
                          PopupManager.alert({
                            title: "Fehler",
                            description:
                              "Es ist ein Fehler aufgetreten: " +
                              res.payload.error,
                          });
                          return;
                        }

                        reload();
                      },
                    },
                  ],
                ],
              };
            })}
          />
        </>
      )}
    </>
  )}
</AdminLayout>
  </>
 );
}
```

## apps/web/src/pages/admin/shorts.tsx

```
/**
 * apps/web/src/pages/admin/shorts.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 19.04.2023
 *
 */

import * as React from "react";
import AdminLayout from "@codeup/components/dist/admin/AdminLayout";
import REST from "@codeup/rest/dist";
import AccountManager from "@codeup/util/dist/AccountManager";
import AuthenticateLoadingIndicator from
"@codeup/components/dist/admin/AuthenticateLoadingIndicator";
import ErrorComponent from "@codeup/components/dist/global/ErrorComponent";
import { Box } from "@chakra-ui/react";
import SuperText from "@codeup/components/dist/landingpage/SuperText";
import GridLayout from "@codeup/components/dist/global/GridLayout";
```

```
import { FaLink, FaPlus, FaTrash } from "react-icons/fa";
import PopupManager from "../../global/PopupManager";
import GridLayoutV2 from "@codeup/components/dist/global/GridLayoutV2";

export default function Shorts() {
  const [loading, setLoading] = React.useState(true);
  const [error, setError] = React.useState<boolean>(false);

  const [shorts, setShorts] = React.useState<
    {
      createdAt: string;
      short: string;
      destination: string;
      _id: string;
    }[]
  >([]);

  React.useEffect(() => {
    reload();
  }, []);

  const reload = () => {
    REST.Admin.getShorts(AccountManager.getActiveToken()).then((res) => {
      setLoading(false);
      if (res.status === 200) {
        setShorts(res.payload as any);
      } else {
        setError(true);
      }
    });
  };

  return (
    <>
      <AdminLayout>
        {loading ? (
          <AuthenticateLoadingIndicator text={"Lade Shorts..."} />
        ) : (
          <>
            {error ? (
              <>
                <ErrorComponent
                  description={
                    "Beim Laden der Shorts ist ein Fehler aufgetreten."
                  }
                />
              </>
            ) : (
              <>
                <GridLayoutV2
                  pageTitle={"Shorts"}
                  search={{
                    show: true,
                    fields: true,
                  }}
                  topButtons={[
                    {
                      text: "Neu",
                      leftIcon: <FaPlus />,
                      variant: "brand",
```

```
                onClick: async () => {
                  const short = await PopupManager.promptAsync({
                    title: "Short erstellen",
                    inputType: "INPUT",
                    helperText: "Die Kurz-URL",
                  });

                  if (!short) return;

                  const destination = await PopupManager.promptAsync({
                    title: "Short erstellen",
                    inputType: "INPUT",
                    helperText: "Die Ziel-URL",
                  });

                  if (!destination) return;

                  const res = await REST.Admin.createShort({
                    token: AccountManager.getActiveToken(),
                    short,
                    dest: destination,
                  });

                  if (res.status === 200) {
                    reload();
                  } else {
                    PopupManager.alert({
                      title: "Fehler",
                      description:
                        "Beim Erstellen des Shorts ist ein Fehler aufgetreten: " +
                        res.payload.error,
                    });
                  }
                },
              },
            ]}
            entries={shorts.map((s) => ({
              title: `https://codeup.space/${s.short}`,
              fields: [
                {
                  icon: <FaLink />,
                  text: s.destination,
                },
              ],
              buttonRows: [
                [
                  {
                    text: "Löschen",
                    leftIcon: <FaTrash />,
                    disabled: false,
                    colorScheme: "red",
                    onClick: async () => {
                      if (
                        !(await PopupManager.confirmAsync({
                          title: "Short löschen",
                          question: `Möchtest du den Short https://codeup.space/${s.short} wirklich
löschen?`,
                        }))
                      )
                        return;
```

```
                        const res = await REST.Admin.deleteShort({
                          token: AccountManager.getActiveToken(),
                          short: s._id,
                        });

                        if (res.status === 200) {
                          reload();
                        } else {
                          PopupManager.alert({
                            title: "Fehler",
                            description:
                              "Beim Löschen des Shorts ist ein Fehler aufgetreten: " +
                              res.payload.error,
                          });
                        }
                      },
                    },
                  ],
                ],
              }))}
            />
          </>
        )}
      </>
    )}
  </AdminLayout>
 </>
 );
}
```

## apps/web/src/pages/admin/snippets.tsx

```
/**
 * apps/web/src/pages/admin/snippets.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 13.04.2023
 *
 */

import * as React from "react";
import AdminLayout from "@codeup/components/dist/admin/AdminLayout";
import {
  Box,
  Button,
  ButtonGroup,
  FormControl,
  FormLabel,
  Heading,
  Input,
  Stack,
  useDisclosure,
  useToast,
} from "@chakra-ui/react";
import REST from "@codeup/rest/src";
import AuthenticateLoadingIndicator from
"@codeup/components/dist/admin/AuthenticateLoadingIndicator";
import ErrorComponent from "@codeup/components/dist/global/ErrorComponent";
```

```
import CodeUpModal from "@codeup/components/dist/codeup/CodeUpModal";
import Editor from "@monaco-editor/react";
import {
  FaClock,
  FaCode,
  FaCopy,
  FaFile,
  FaPen,
  FaPlus,
  FaTrash,
  FaUser,
} from "react-icons/fa";
import GridLayout from "@codeup/components/dist/global/GridLayout";
import SuperText from "@codeup/components/dist/landingpage/SuperText";
import GridLayoutV2 from "@codeup/components/dist/global/GridLayoutV2";

export default function Snippets() {
  const [snippets, setSnippets] = React.useState<
    {
      _id: string;
      title: string;
      code: string;
      language: string;
      createdAt: string;
      username: string;
    }[]
  >([]);

  const [loading, setLoading] = React.useState(true);
  const [error, setError] = React.useState(false);
  const { isOpen, onOpen, onClose } = useDisclosure();

  React.useEffect(() => {
    reload();
  }, []);

  const reload = () => {
    REST.Admin.getSnippets(localStorage.getItem("token") as string).then(
      (res) => {
        setLoading(false);
        if (res.status === 200) {
          setSnippets(res.payload.snippets);
        } else {
          setError(true);
        }
      }
    );
  };
  const toast = useToast();

  const [currentSnippet, setCurrentSnippet] = React.useState<{
    _id: string;
    title: string;
    code: string;
    language: string;
    createdAt: string;
    username: string;
  } | null>(null);

  return (
```

```
<>
  <AdminLayout>
    {loading ? (
      <>
        <AuthenticateLoadingIndicator text={"Lade Snippets..."} />
      </>
    ) : (
      <>
        {error ? (
          <>
            <ErrorComponent
              description={
                "Beim Laden der Snippets ist ein Fehler aufgetreten."
              }
            />
          </>
        ) : (
          <>
            <CodeUpModal
              onOpen={onOpen}
              onClose={onClose}
              isOpen={isOpen}
              title={currentSnippet?.title || "Neues Snippet"}
              noCloseButton
            >
              <Stack spacing={4}>
                <FormControl>
                  <FormLabel>Titel</FormLabel>
                  <Input
                    type={"text"}
                    placeholder={"Titel des Snippets"}
                    value={currentSnippet?.title}
                    onChange={(e) => {
                      if (currentSnippet) {
                        setCurrentSnippet({
                          ...currentSnippet,
                          title: e.target.value as string,
                        });
                      }
                    }}
                  />
                </FormControl>
                <FormControl>
                  <FormLabel>Sprache</FormLabel>
                  <Input
                    type={"text"}
                    placeholder={"Sprache des Snippets"}
                    value={currentSnippet?.language}
                    onChange={(e) => {
                      if (currentSnippet) {
                        setCurrentSnippet({
                          ...currentSnippet,
                          language: e.target.value as string,
                        });
                      }
                    }}
                  />
                </FormControl>
                <FormControl data-color-mode={"dark"}>
                  <FormLabel>Code</FormLabel>
```

```
  <Editor
    height={"50vh"}
    language={currentSnippet?.language || "javascript"}
    theme={"vs-dark"}
    value={currentSnippet?.code}
    onChange={(value) => {
      if (currentSnippet) {
        setCurrentSnippet({
          ...currentSnippet,
          code: value as string,
        });
      }
    }}
  />
</FormControl>
<ButtonGroup
  display={"flex"}
  justifyContent={"flex-end"}
  w={"100%"}
>
  <Button
    leftIcon={<FaTrash />}
    colorScheme={"red"}
    onClick={() => {
      REST.Snippets.deleteSnippet({
        token: localStorage.getItem("token") as string,
        id: currentSnippet?._id as string,
      }).then((res) => {
        if (res.status === 200) {
          toast({
            title: "Snippet gelöscht",
            description:
              "Das Snippet wurde erfolgreich gelöscht",
            status: "success",
            duration: 1500,
            isClosable: false,
            position: "bottom-right",
          });
          reload();
          onClose();
        } else {
          toast({
            title: "Fehler",
            description: res.payload.error,
            status: "error",
            duration: 1500,
            isClosable: false,
            position: "bottom-right",
          });
        }
      });
    }}
  >
    Löschen
  </Button>
  <Button
    colorScheme={"red"}
    onClick={() => {
      onClose();
    }}
```

```
      >
        Abbrechen
      </Button>
      <Button
        variant={"brand"}
        onClick={() => {
          REST.Snippets.updateSnippet({
            token: localStorage.getItem("token") as string,
            id: currentSnippet?._id as string,
            title: currentSnippet?.title as string,
            code: currentSnippet?.code as string,
            language: currentSnippet?.language as string,
          }).then((res) => {
            if (res.status === 200) {
              reload();
              onClose();
              setCurrentSnippet(null);
            } else {
              toast({
                title: "Fehler",
                description: res.payload.error,
                status: "error",
                duration: 1500,
                isClosable: false,
                position: "bottom-right",
              });
            }
          });
        }}
      >
        Speichern
      </Button>
    </ButtonGroup>
  </Stack>
</CodeUpModal>
<GridLayoutV2
  pageTitle={"Code Snippets"}
  search={{
    show: true,
    fields: true,
  }}
  topButtons={[]}
  entries={snippets.map((snippet) => {
    return {
      title: snippet.title,
      fields: [
        {
          icon: <FaUser />,
          text: "Besitzer: @" + snippet.username,
        },
        {
          icon: <FaCode />,
          text: snippet.language,
        },
        {
          icon: <FaClock />,
          text: new Date(snippet.createdAt).toLocaleString(),
        },
        {
          icon: <FaFile />,
```

```
                    text: snippet.code.substring(0, 50) + "...",
                  },
                ],
                buttonRows: [
                  [
                    {
                      leftIcon: <FaPen />,
                      text: "Bearbeiten",
                      variant: "brand",
                      onClick: async () => {
                        setCurrentSnippet(snippet);
                        onOpen();
                      },
                    },
                    {
                      leftIcon: <FaCopy />,
                      text: "Kopieren",
                      variant: "brand",
                      onClick: async () => {
                        navigator.clipboard.writeText(snippet.code);
                        toast({
                          title: "Code kopiert",
                          description:
                            "Der Code wurde erfolgreich kopiert",
                          status: "success",
                          duration: 1500,
                          isClosable: false,
                          position: "bottom-right",
                        });
                      },
                    },
                  ],
                ],
              };
            })}
          />
        </>
      )}
    </>
  )}
  </AdminLayout>
 </>
 );
}
```

## apps/web/src/pages/admin/testimonials.tsx

```
/**
 * apps/web/src/pages/admin/testimonials.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 12.04.2023
 *
 */

import * as React from "react";
import Head from "next/head";
import AdminLayout from "@codeup/components/dist/admin/AdminLayout";
import SuperText from "@codeup/components/dist/landingpage/SuperText";
```

```
import {
  Avatar,
  Box,
  Button,
  ButtonGroup,
  SimpleGrid,
  Stack,
  StackDivider,
  Text,
} from "@chakra-ui/react";
import REST from "@codeup/rest/dist";
import { FaPen, FaPlus, FaTrash } from "react-icons/fa";
import PopupManager from "../../global/PopupManager";
import AccountManager from "@codeup/util/dist/AccountManager";
import { CHeading } from "@codeup/components/dist/global/CHeading";

export default function Testimonials() {
  const [testimonials, setTestimonials] = React.useState<
    {
      _id: string;
      name: string;
      position: string;
      company: string;
      image: string;
      content: string;
    }[]
  >([]);

  React.useEffect(() => {
    REST.Landingpage.getTestimonials().then((res) => {
      if (res.status === 200) {
        setTestimonials(res.payload as any);
      } else {
        alert(res.payload.error);
      }
    });
  }, []);

  return (
    <>
      <Head>
        <title>Meinungen - Admin - CodeUp</title>
      </Head>
      <AdminLayout>
        <Box textAlign={"center"}>
          <CHeading size={"xl"}>Meinungen</CHeading>
        </Box>
        <SimpleGrid
          columns={{ base: 1, md: 2 }}
          placeItems="center"
          spacing={1}
          mt={12}
          mb={4}
        >
          <Stack
            key={"-1"}
            direction={{ base: "column", sm: "row" }}
            spacing={2}
            mb={5}
            justify="center"
```

```
>
  <Stack
    maxW="330px"
    minW={"300px"}
    boxShadow="lg"
    rounded="md"
    p={6}
    pos="relative"
    bg={"gray.900"}
    _after={{
      content: `""`,
      w: "0",
      h: "0",
      borderColor: `transparent ${"#141517"} transparent`,
      borderStyle: "solid",
      borderWidth: "10px 0 10px 10px",
      position: "absolute",
      top: { base: "unset", sm: "45%" },
      right: { base: "unset", sm: "-10px" },
      left: { base: "48%", sm: "unset" },
      bottom: { base: "-15px", sm: "unset" },
      transform: { base: "rotate(90deg)", sm: "unset" },
      display: "block",
    }}
  >
    <Text fontWeight="medium" fontSize="md">
      Neue Meinung hinzufügen
    </Text>
    <StackDivider />
    <Button
      variant={"brand"}
      leftIcon={<FaPlus />}
      onClick={async () => {
        const name = await PopupManager.promptAsync({
          title: "Name",
          helperText: "Name des Kunden",
        });

        if (!name) return;

        const position = await PopupManager.promptAsync({
          title: "Position",
          helperText: "Position des Kunden",
        });

        if (!position) return;

        const company = await PopupManager.promptAsync({
          title: "Firma",
          helperText: "Firma des Kunden",
        });

        if (!company) return;

        const content = await PopupManager.promptAsync({
          title: "Meinung",
          helperText: "Meinung des Kunden",
          inputType: "TEXTAREA",
        });
```

```
if (!content) return;

const imageFL = await PopupManager.uploadFileAsync({
  title: "Bild",
  helperText: "Bild des Kunden",
});

if (!imageFL || !imageFL.item(0)) return;

const form = document.createElement("form");

const input = document.createElement("input");
input.type = "file";
input.name = "file";
input.files = imageFL;

form.appendChild(input);

const formData = new FormData(form);

const r = await fetch("/media/upload", {
  method: "POST",
  body: formData,
});

if (r.status === 200) {
  const res = await r.json();

  const url = res.data.url;

  const res2 = await REST.Admin.createTestimonial({
    token: AccountManager.getActiveToken(),
    company,
    content,
    image: url,
    name,
    position,
  });

  if (res2.status === 200) {
    PopupManager.alert({
      title: "Erfolgreich",
      description:
        "Die Meinung wurde erfolgreich hinzugefügt!",
    });

    setTestimonials([...testimonials, res2.payload as any]);
  } else {
    PopupManager.alert({
      title: "Fehler",
      description:
        "Beim hinzufügen der Meinung ist ein Fehler aufgetreten: " +
        res2.payload.error,
    });
  }
} else {
  PopupManager.alert({
    title: "Fehler",
    description:
      "Beim hochladen des Bildes ist ein Fehler aufgetreten: " +
```

```
              r.statusText,
            });
          }
        }}
      >
        Hinzufügen
      </Button>
    </Stack>
    <Stack
      direction="column"
      spacing={2}
      p={2}
      justify="flex-end"
      alignItems="center"
    >
      <Avatar
        size="xl"
        showBorder={true}
        borderColor="brand.500"
        name="avatar"
        src={"https://avatars.githubusercontent.com/u/53553315"}
      />
      <Box textAlign="center">
        <Text fontWeight="bold" fontSize="lg">
          Neue Meinung
        </Text>
      </Box>
    </Stack>
  </Stack>
  {testimonials.map((obj, index) => (
    <>
      <Stack
        key={index}
        direction={{ base: "column", sm: "row" }}
        spacing={2}
        mb={5}
        justify="center"
      >
        <Stack
          maxW="330px"
          minW={"300px"}
          boxShadow="lg"
          rounded="md"
          p={6}
          pos="relative"
          bg={"gray.900"}
          _after={{
            content: `""`,
            w: "0",
            h: "0",
            borderColor: `transparent ${"#141517"} transparent`,
            borderStyle: "solid",
            borderWidth: "10px 0 10px 10px",
            position: "absolute",
            top: { base: "unset", sm: "45%" },
            right: { base: "unset", sm: "-10px" },
            left: { base: "48%", sm: "unset" },
            bottom: { base: "-15px", sm: "unset" },
            transform: { base: "rotate(90deg)", sm: "unset" },
            display: "block",
```

```
  }}
>
  <Text fontWeight="medium" fontSize="sm">
    {obj.content}
  </Text>
  <ButtonGroup>
    <Button
      colorScheme={"red"}
      leftIcon={<FaTrash />}
      onClick={async () => {
        if (
          !(await PopupManager.confirmAsync({
            title: "Löschen",
            question:
              "Soll die Meinung wirklich gelöscht werden?",
          }))
        )
          return;

        const res = await REST.Admin.deleteTestimonial({
          token: AccountManager.getActiveToken(),
          id: obj._id,
        });

        if (res.status === 200) {
          PopupManager.alert({
            title: "Erfolgreich",
            description:
              "Die Meinung wurde erfolgreich gelöscht!",
          });

          const newTestimonials = [...testimonials];
          newTestimonials.splice(index, 1);

          setTestimonials(newTestimonials);
          return;
        }

        PopupManager.alert({
          title: "Fehler",
          description:
            "Beim löschen der Meinung ist ein Fehler aufgetreten: " +
            res.payload.error,
        });
      }}
    >
      Löschen
    </Button>
    <Button
      variant={"brand"}
      leftIcon={<FaPen />}
      onClick={async () => {
        const selected = await PopupManager.selectAsync({
          title: "Bearbeiten",
          helperText: "Wähle aus was du bearbeiten willst",
          choices: [
            "Name",
            "Position",
            "Firma",
            "Meinung",
```

```
    "Bild",
  ],
});

if (!selected) return;

switch (selected) {
  case "Name": {
    const name = await PopupManager.promptAsync({
      title: "Name",
      helperText: "Name des Kunden",
    });

    if (!name) break;

    const res = await REST.Admin.updateTestimonial({
      token: AccountManager.getActiveToken(),
      id: obj._id,
      update: {
        name,
      },
    });

    if (res.status === 200) {
      PopupManager.alert({
        title: "Erfolgreich",
        description:
          "Der Name wurde erfolgreich geändert!",
      });

      const newTestimonials = [...testimonials];
      newTestimonials[index].name = name;

      setTestimonials(newTestimonials);

      break;
    } else {
      PopupManager.alert({
        title: "Fehler",
        description:
          "Beim ändern des Namens ist ein Fehler aufgetreten: " +
          res.payload.error,
      });
    }
    break;
  }
  case "Position": {
    const position = await PopupManager.promptAsync({
      title: "Position",
      helperText: "Position des Kunden",
    });

    if (!position) break;

    const res = await REST.Admin.updateTestimonial({
      token: AccountManager.getActiveToken(),
      id: obj._id,
      update: {
        position,
      },
```

```
    });

    if (res.status === 200) {
      PopupManager.alert({
        title: "Erfolgreich",
        description:
          "Die Position wurde erfolgreich geändert!",
      });

      const newTestimonials = [...testimonials];
      newTestimonials[index].position = position;

      setTestimonials(newTestimonials);

      break;
    } else {
      PopupManager.alert({
        title: "Fehler",
        description:
          "Beim ändern der Position ist ein Fehler aufgetreten: " +
          res.payload.error,
      });
    }
    break;
  }
  case "Firma": {
    const company = await PopupManager.promptAsync({
      title: "Firma",
      helperText: "Firma des Kunden",
    });

    if (!company) break;

    const res = await REST.Admin.updateTestimonial({
      token: AccountManager.getActiveToken(),
      id: obj._id,
      update: {
        company,
      },
    });

    if (res.status === 200) {
      PopupManager.alert({
        title: "Erfolgreich",
        description:
          "Die Firma wurde erfolgreich geändert!",
      });

      const newTestimonials = [...testimonials];
      newTestimonials[index].company = company;

      setTestimonials(newTestimonials);
      break;
    } else {
      PopupManager.alert({
        title: "Fehler",
        description:
          "Beim ändern der Firma ist ein Fehler aufgetreten: " +
          res.payload.error,
      });
```

```
  }
  break;
}
case "Meinung": {
  const content = await PopupManager.promptAsync({
    title: "Meinung",
    helperText: "Meinung des Kunden",
    inputType: "TEXTAREA",
  });

  if (!content) break;

  const res = await REST.Admin.updateTestimonial({
    token: AccountManager.getActiveToken(),
    id: obj._id,
    update: {
      content,
    },
  });

  if (res.status === 200) {
    PopupManager.alert({
      title: "Erfolgreich",
      description:
        "Die Meinung wurde erfolgreich geändert!",
    });

    const newTestimonials = [...testimonials];
    newTestimonials[index].content = content;

    setTestimonials(newTestimonials);
    break;
  } else {
    PopupManager.alert({
      title: "Fehler",
      description:
        "Beim ändern der Meinung ist ein Fehler aufgetreten: " +
        res.payload.error,
    });
  }
  break;
}
case "Bild": {
  const imageFL = await PopupManager.uploadFileAsync({
    title: "Bild",
    helperText: "Bild des Kunden",
  });

  if (!imageFL || !imageFL.item(0)) break;

  const form = document.createElement("form");

  const input = document.createElement("input");
  input.type = "file";
  input.name = "file";
  input.files = imageFL;

  form.appendChild(input);

  const formData = new FormData(form);
```

```
          const r = await fetch("/media/upload", {
            method: "POST",
            body: formData,
          });

          if (r.status === 200) {
            const res = await r.json();

            const url = res.data.url;

            const res2 = await REST.Admin.updateTestimonial({
              token: AccountManager.getActiveToken(),
              id: obj._id,
              update: {
                image: url,
              },
            });

            if (res2.status === 200) {
              PopupManager.alert({
                title: "Erfolgreich",
                description:
                  "Die Meinung wurde erfolgreich geändert!",
              });

              const newTestimonials = [...testimonials];
              newTestimonials[index].image = url;

              setTestimonials(newTestimonials);
            } else {
              PopupManager.alert({
                title: "Fehler",
                description:
                  "Beim Ändern der Meinung ist ein Fehler aufgetreten: " +
                  res2.payload.error,
              });
            }
          } else {
            PopupManager.alert({
              title: "Fehler",
              description:
                "Beim hochladen des Bildes ist ein Fehler aufgetreten: " +
                r.statusText,
            });
          }
        }
        default: {
          PopupManager.alert({
            title: "Fehler",
            description:
              "Diese Funktion ist noch nicht verfügbar",
          });
        }
      }
    }}
  >
    Bearbeiten
  </Button>
</ButtonGroup>
```

```
            </Stack>
            <Stack
              direction="column"
              spacing={2}
              p={2}
              justify="flex-end"
              alignItems="center"
            >
              <Avatar
                size="xl"
                showBorder={true}
                borderColor="brand.500"
                name="avatar"
                src={obj.image}
              />
              <Box textAlign="center">
                <Text fontWeight="bold" fontSize="lg">
                  {obj.name}
                </Text>
                <Text fontWeight="medium" fontSize="xs" color="white">
                  {obj.position}, {obj.company}
                </Text>
              </Box>
            </Stack>
          </Stack>
        </>
      ))}
    </SimpleGrid>
  </AdminLayout>
 </>
 );
}
```

## apps/web/src/pages/admin/todos.tsx

```
/**
 * apps/web/src/pages/admin/todos.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 13.04.2023
 *
 */

import * as React from "react";
import AdminLayout from "@codeup/components/dist/admin/AdminLayout";
import { useRouter } from "next/router";
import { Box, useDisclosure, useToast } from "@chakra-ui/react";
import { useEffect } from "react";
import REST from "@codeup/rest/src";
import AuthenticateLoadingIndicator from
"@codeup/components/dist/admin/AuthenticateLoadingIndicator";
import ErrorComponent from "@codeup/components/dist/global/ErrorComponent";
import SuperText from "@codeup/components/dist/landingpage/SuperText";
import { FaCheck, FaEye, FaPlus, FaTrash, FaUser } from "react-icons/fa";
import PopupManager from "../../global/PopupManager";
import AccountManager from "@codeup/util/dist/AccountManager";
import GridLayout from "@codeup/components/dist/global/GridLayout";
import ListViewer from "@codeup/components/dist/todo/ListViewer";
import GridLayoutV2 from "@codeup/components/dist/global/GridLayoutV2";
```

```
export default function Todos() {
  const [error, setError] = React.useState<boolean>(false);
  const [loading, setLoading] = React.useState<boolean>(false);

  const router = useRouter();
  const toast = useToast();

  const [lists, setLists] = React.useState<
    {
      _id: string;
      name: string;
      owner: string;
      tasks: {
        name: string;
        finished: boolean;
        description: string;
        uuid: string;
      }[];
    }[]
  >([]);

  const [currentList, setCurrentList] = React.useState<{
    _id: string;
    name: string;
    owner: string;
    tasks: {
      name: string;
      finished: boolean;
      description: string;
      uuid: string;
    }[];
  } | null>(null);

  useEffect(() => {
    reloadLists();
  }, []);

  const { isOpen, onOpen, onClose } = useDisclosure();

  async function reloadLists() {
    const res = await REST.Admin.getTodos(
      localStorage.getItem("token") as string
    );

    setLoading(false);
    if (res.status === 200) {
      setLists(res.payload.lists);

      if (currentList) {
        setCurrentList(
          res.payload.lists.find((e: any) => e._id === currentList._id)
        );
      }
    } else {
      setError(true);
    }
  }

  return (
```

```
<>
  <AdminLayout>
    {loading ? (
      <AuthenticateLoadingIndicator text={"Lade Listen..."} />
    ) : (
      <>
        {error ? (
          <>
            <ErrorComponent description={"Fehler beim Laden der Listen."} />
          </>
        ) : (
          <>
            <GridLayoutV2
              pageTitle={"Todos"}
              search={{
                show: true,
                fields: true,
              }}
              topButtons={[]}
              entries={lists.map((list) => {
                return {
                  title: list.name,
                  fields: [
                    {
                      icon: <FaUser />,
                      text: "Besitzer: " + (list.owner as any).username,
                    },
                    {
                      icon: <FaCheck />,
                      text:
                        "Erledigt: " +
                        list.tasks.filter((task) => task.finished).length +
                        "/" +
                        list.tasks.length,
                    },
                  ],
                  buttonRows: [
                    [
                      {
                        text: "Öffnen",
                        leftIcon: <FaEye />,
                        variant: "brand",
                        onClick: () => {
                          setCurrentList(list);
                          onOpen();
                        },
                      },
                      {
                        text: "Löschen",
                        leftIcon: <FaTrash />,
                        colorScheme: "red",
                        onClick: async () => {
                          if (
                            await PopupManager.confirmAsync({
                              title: "Liste löschen",
                              question:
                                "Willst du die Liste wirklich löschen?",
                            })
                          ) {
                            const res = await REST.ToDo.deleteList({
```

```
                                token: AccountManager.getActiveToken(),
                                listId: list._id,
                              });

                              if (res.status === 200) {
                                reloadLists();
                                return;
                              }

                              PopupManager.alert({
                                title: "Fehler",
                                description:
                                  "Die Liste konnte nicht gelöscht werden: " +
                                  res.payload.error,
                              });
                            }
                          },
                        },
                      ],
                    ],
                  };
                })}
              />
            </>
          )}
        </>
      )}
      <ListViewer
        isOpen={isOpen}
        onClose={onClose}
        currentList={currentList}
        setCurrentList={setCurrentList}
        reloadLists={reloadLists}
      />
    </AdminLayout>
  </>
 );
}
```

## apps/web/src/pages/admin/users/index.tsx

```
/**
 * apps/web/src/pages/admin/users/index.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 07.10.22
 *
 */

import { useLoggedIn } from "@codeup/hooks";
import router from "next/router";
import { useEffect, useState } from "react";
import {
  FaCheck,
  FaEdit,
  FaEllipsisH,
  FaEllipsisV,
  FaEye,
  FaFilter,
```

```
  FaHome,
  FaPlus,
  FaSort,
  FaTrash,
  FaUser,
} from "react-icons/fa";
import {
  Alert,
  AlertDescription,
  AlertIcon,
  AlertTitle,
  Avatar,
  Box,
  Button,
  ButtonGroup,
  Center,
  CircularProgress,
  Flex,
  Grid,
  Heading,
  HStack,
  Icon,
  IconButton,
  Input,
  InputGroup,
  InputLeftElement,
  Menu,
  MenuButton,
  MenuItem,
  MenuList,
  Progress,
  Spinner,
  Table,
  TableCaption,
  TableContainer,
  Tbody,
  Td,
  Text,
  Tfoot,
  Th,
  Thead,
  Tr,
  useDisclosure,
  useToast,
} from "@chakra-ui/react";
import AuthenticateLoadingIndicator from
"@codeup/components/admin/AuthenticateLoadingIndicator";
import ErrorComponent from "@codeup/components/global/ErrorComponent";
import AdminLayout from "@codeup/components/admin/AdminLayout";
import REST from "@codeup/rest";
import { FiMoreHorizontal, FiRotateCw } from "react-icons/fi";
import AdminUserViewer from "@codeup/components/admin/AdminUserViewer";
import AdminUserCreator from "@codeup/components/admin/AdminUserCreator";
import DataTable from "@codeup/components/extra/DataTable";
import Head from "next/head";
import * as React from "react";
import SuperText from "@codeup/components/dist/landingpage/SuperText";

type User = {
  id: string;
```

```
  username: string;
  email: string;
  password: string;
  firstName: string;
  lastName: string;
  twoFactorEnabled: boolean;
  twoFactorCode: string;
  active: boolean;
  activationToken: string;
  deletionToken: string;
  resetToken: string;
  emailChangeCode: string;
  newEmail: string;
  profilePicture: string;
  createdAt: string;
  sso: string;
  role: string;
  __v: number;
};

export default function UserManagement() {
  const [users, setUsers] = useState<User[]>([]);

  const [loaded, setLoaded] = useState<boolean>(true);

  const [error, setError] = useState<string>("");

  const { isOpen, onOpen, onClose } = useDisclosure();
  const {
    isOpen: isOpenViewer,
    onOpen: onOpenViewer,
    onClose: onCloseViewer,
  } = useDisclosure();

  const [currentUser, setCurrentUser] = useState<any>(null);

  const reload = () => {
    REST.Admin.listAccounts(localStorage.getItem("token") as string).then(
      (res) => {
        if (res.status === 200) {
          setUsers(res.payload.accounts);
          setLoaded(true);
        } else {
          setError("Benutzer konnten nicht geladen werden.");
          setLoaded(true);
        }
      }
    );
  };
  useEffect(() => {
    reload();
  }, []);

  const userToRow = (user: any) => {
    return [
      () => {
        setCurrentUser(user);
        onOpen();
      },
      <Avatar size={"md"} src={"/api/account/" + user.username + "/picture"} />,
```

```
    <>
      <Text>
        {user.firstName} {user.lastName}
      </Text>
      <Text>{user.username}</Text>
    </>,
    user.email,
    user.role,
    user.active ? "Ja" : "Nein",
    user.twoFactor ? "Ja" : "Nein",
    new Date(user.createdAt).toLocaleString(),
  ];
};

return (
  <>
    <Head>
      <title>Benutzer - Admin - CodeUp</title>
    </Head>
    {loaded && error !== "" ? (
      <>
        <ErrorComponent description={error} />
      </>
    ) : null}
    {loaded && error === "" ? (
      <>
        <AdminLayout>
          <>
            <Flex maxW={"100vw"}>
              <Box flex={["1%", "10%"]}></Box>
              <Box flex={["99%", "80%"]}>
                <Flex
                  w={"100%"}
                  maxW={"100vw"}
                  minH={"100vh"}
                  h="fit-content"
                  direction={"column"}
                >
                  <Flex
                    direction={"row"}
                    justify={"center"}
                    alignItems={"center"}
                    p={4}
                  >
                    <Box flex={"100%"}>
                      <SuperText>Benutzer</SuperText>
                    </Box>
                    <Button
                      onClick={() => {
                        onOpenViewer();
                      }}
                      variant={"brand"}
                      leftIcon={<FaPlus />}
                    >
                      Neu
                    </Button>
                  </Flex>
                  <Flex
                    w={"100%"}
                    h={"100%"}
```

```
  direction={"column"}
  gap={"6rem"}
  alignItems={"center"}
  mt={4}
>
  <Box w={"100%"} p={2}>
    <DataTable
      columns={[
        "Profilbild",
        "Name",
        "E-Mail",
        "Rolle",
        "Aktiv",
        "2FA",
        "Erstellt",
      ]}
      data={users.map(userToRow)}
      filter={(q: string, f: string, s: string) => {
        let x = users.filter((user: any) => {
          return (
            user.username
              .toLowerCase()
              .includes(q.toLowerCase()) ||
            user.email
              .toLowerCase()
              .includes(q.toLowerCase()) ||
            user._id
              .toLowerCase()
              .includes(q.toLowerCase()) ||
            (user.firstName + " " + user.lastName)
              .toLowerCase()
              .includes(q.toLowerCase())
          );
        });

        switch (f.toLowerCase()) {
          case "alle":
            break;
          case "aktive":
            x = x.filter((user: any) => user.active);
            break;
          case "inaktive":
            x = x.filter((user: any) => !user.active);
            break;
          default:
            break;
        }

        switch (s.toLowerCase()) {
          case "alphabetisch":
            x = x.sort((a: any, b: any) => {
              if (
                a.username.toLowerCase() <
                b.username.toLowerCase()
              ) {
                return -1;
              } else if (
                a.username.toLowerCase() >
                b.username.toLowerCase()
              ) {
```

```
                    return 1;
                  } else {
                    return 0;
                  }
                });
                break;
              case "neuste zuerst":
                x = x.sort((a: any, b: any) => {
                  return (
                    new Date(b.createdAt).getTime() -
                    new Date(a.createdAt).getTime()
                  );
                });
                break;
              case "älteste zuerst":
                x = x.sort((a: any, b: any) => {
                  return (
                    new Date(a.createdAt).getTime() -
                    new Date(b.createdAt).getTime()
                  );
                });
                break;
              default:
                break;
            }

            return x.map(userToRow);
          }}
          filterOptions={["Alle", "Aktive", "Inaktive"]}
          sortOptions={[
            "Alphabetisch",
            "Neuste zuerst",
            "Älteste zuerst",
          ]}
          reload={reload}
        />
      </Box>
    </Flex>
  </Flex>
</Box>
<Box flex={["1%", "10%"]}></Box>
</Flex>
</>
<AdminUserViewer
  onOpen={onOpen}
  onClose={onClose}
  isOpen={isOpen}
  user={(currentUser as User) || null}
  users={users}
  setUsers={setUsers}
  setCurrentUser={setCurrentUser}
/>
<AdminUserCreator
  onOpen={onOpenViewer}
  onClose={onCloseViewer}
  isOpen={isOpenViewer}
  setUsers={setUsers}
  users={users}
/>
</AdminLayout>
```

```
        </>
      ) : null}
    </>
  );
}
```

## apps/web/src/pages/api/account/action.ts

```
/**
 * src/pages/api/account/operation.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 05.06.2022
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/src/mongo";
import UserModel from "@codeup/util/src/models/UserModel";
import generateJWT from "@codeup/util/src/generateJWT";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "GET") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  if (!req.query.operation) {
    res.status(400).json({
      error: "Missing operation",
    });
    return;
  }

  if (!req.query.redirect) {
    res.status(400).json({
      error: "Missing redirect",
    });
    return;
  }

  if (req.query.operation === "activate") {
    if (!req.query.token) {
      res.status(400).json({
        error: "Missing token",
      });
      return;
    }

    await mongoConnect();

    const user = await UserModel.findOne({
      activationToken: req.query.token,
    });
```

```
    if (!user) {
      res.status(400).json({
        error: "Invalid token",
      });
      return;
    }

    user.activationToken = null;
    user.active = true;

    await user.save();

    res.redirect(req.query.redirect as string);
  }

  if (req.query.operation === "changeEmail") {
    if (!req.query.token) {
      res.status(400).json({
        error: "Missing token",
      });
      return;
    }

    await mongoConnect();

    const user = await UserModel.findOne({
      emailChangeCode: req.query.token,
    });

    if (!user) {
      res.status(400).json({
        error: "Invalid token",
      });
      return;
    }

    if (!user.newEmail) {
      res.status(400).json({
        error: "Invalid token",
      });
      return;
    }

    user.emailChangeCode = null;
    user.email = user.newEmail;
    user.newEmail = null;

    await user.save();

    res.redirect(
      (req.query.redirect as string) + "?newToken=" + generateJWT(user)
    );
  }
}
```

## apps/web/src/pages/api/account/check-reset-token/[token].ts

```
/**
 * apps/web/src/pages/api/account/check-reset-token/[token].ts
 *
```

```
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 20.07.2022
 *
 */
import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/src/mongo";
import UserModel from "@codeup/util/src/models/UserModel";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "GET") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  const { token } = req.query;

  if (!token) {
    res.status(400).json({
      error: "Missing token",
    });
    return;
  }

  await mongoConnect();

  const user = await UserModel.findOne({
    resetToken: token,
  });

  if (!user) {
    res.status(400).json({
      error: "Invalid token",
    });
    return;
  }

  res.status(200).json({
    valid: true,
  });
}
```

## apps/web/src/pages/api/account/check-username/[name].ts

```
/**
 * src/pages/api/account/check-username/[name].ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 18.06.2022
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
```

```
import mongoConnect from "@codeup/util/src/mongo";
import UserModel from "@codeup/util/src/models/UserModel";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "GET") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  const { name } = req.query;

  if (!name) {
    res.status(400).json({
      error: "Missing name",
    });
    return;
  }

  await mongoConnect();

  const user = await UserModel.findOne({
    username: name as string,
  });

  if (!user) {
    res.status(200).json({
      error: "Username available",
    });
    return;
  }

  res.status(400).json({
    error: "Username taken",
  });
}
```

## apps/web/src/pages/api/account/delete.tsx

```
/**
 * apps/web/src/pages/api/account/delete.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 15.07.2022
 *
 */
import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/src/mongo";
import sendMail, { sendMailV2 } from "@codeup/util/src/sendMail";
import generateMail from "@codeup/util/src/generateMail";
import UserModel from "@codeup/util/src/models/UserModel";
import * as crypto from "crypto";
import checkPassword from "@codeup/util/src/checkPassword";
import jwt from "jsonwebtoken";
import DeleteUserEmail from "@codeup/components/emails/DeleteUser";
```

```
import { render } from "@react-email/render";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "POST") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  if (!req.headers["x-jwt"]) {
    res.status(400).json({
      error: "Auth-Header is missing",
    });
    return;
  }

  // @ts-ignore
  jwt.verify(
    req.headers["x-jwt"] as string,
    process.env.JWT_SECRET as string,
    async (err, decoded) => {
      if (err) {
        res.status(400).json({
          error: "Invalid token",
          details: err,
        });
        return;
      }

      await mongoConnect();

      const user = await UserModel.findById((decoded as any).id);

      if (!user) {
        res.status(400).json({
          error: "Invalid token",
        });
        return;
      }

      await sendMailV2({
        to: user.email,
        subject: "Konto gelöscht",
        text: "Dein Konto wurde erfolgreich gelöscht!",
        html: render(<DeleteUserEmail firstName={user.firstName} />)
      })
      await user.remove();

      // DELETE: CommunityProfile, CommunityPosts, TodoLists with no contributors, IDEProjects with
no contributors
      // DELETE: CourseProgress

      res.status(200).json({
        success: true,
        message: "Deleted.",
      });
```

```
    }
  );
}
```

## apps/web/src/pages/api/account/login.tsx

```
/**
 * src/pages/api/account/Login.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 05.06.2022
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/src/mongo";
import UserModel from "@codeup/util/src/models/UserModel";
import * as crypto from "crypto";
import generateJWT from "@codeup/util/src/generateJWT";
import { default as sm, sendMailV2 } from "@codeup/util/src/sendMail";
import { authenticator } from "otplib";
import { render } from "@react-email/render";
import LoginEmail from "@codeup/components/emails/Login";
import Stripe from "stripe";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "POST") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  if (!req.body || !req.body.password) {
    res.status(400).json({
      error: "Missing required fields",
    });
    return;
  }

  if (!req.body.email && !req.body.username) {
    res.status(400).json({
      error: "Missing either E-Mail oder Username",
    });
    return;
  }

  try {
    await mongoConnect();

    let user: any | null = null;

    if (req.body.email) {
      user = await UserModel.findOne({ email: req.body.email });
    }
```

```
if (req.body.username) {
  user = await UserModel.findOne({ username: req.body.username });
}

if (!user) {
  res.status(400).json({
    error: "Benutzer nicht gefunden",
  });
  return;
}

if (
  crypto.createHash("sha256").update(req.body.password).digest("hex") !==
  user.password
) {
  res.status(400).json({
    error: "Falsches Passwort",
  });
  return;
}

if (!user.active) {
  res.status(400).json({
    error:
      "Konto nicht aktiviert. Überprüfen Sie Ihre E-Mail für einen Aktivierungslink.",
  });
  return;
}

const stripe = new Stripe(process.env.STRIPE_SECRET_KEY as string, {
  apiVersion: "2022-11-15"
});

if(user.stripeSubscriptionId && user.stripeSubscriptionId !== "") {
  const subscription = await stripe.subscriptions.retrieve(user.stripeSubscriptionId);
  console.log(subscription);
  if(subscription.status !== "active") {
    user.subscription = "FREE";
    user.stripeSubscriptionId = "";
    await user.save();
  }
}

if (!user.twoFactorEnabled) {
  await sendMail(user.email, user.firstName);
  res.status(200).json({
    error: null,
    token: generateJWT(user),
  });
  return;
} else {
  if (req.body.code) {
    if (!authenticator.check(req.body.code, user.twoFactorSecret)) {
      res.status(400).json({
        error: "Falscher Code",
      });
      return;
    }

    user.twoFactorCode = null;
```

```
        await user.save();

        await sendMail(user.email, user.firstName);

        res.status(200).json({
          success: true,
          token: generateJWT(user),
        });
        return;
      } else {
        res.status(200).json({
          error: null,
          _2fa: true,
        });
        return;
      }
    }
  } catch (e) {
    res.status(500).json({
      error: "Internal Server Error",
      // @ts-ignore
      message: e.message,
      // @ts-ignore
      stack: e.stack,
    });
    return;
  }
}

const sendMail = async (email: string, firstName: string) => {
  await sendMailV2({
    to: email,
    subject: "Angemeldet",
    html: await render(<LoginEmail firstName={firstName} />),
    text: "",
  });
};
```

## apps/web/src/pages/api/account/register.tsx

```
/**
 * src/pages/api/account/register.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 05.06.2022
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/src/mongo";
import UserModel from "@codeup/util/src/models/UserModel";
import * as crypto from "crypto";
import { uuid } from "uuidv4";
import sendMail, { sendMailV2 } from "@codeup/util/src/sendMail";
import checkPassword from "@codeup/util/src/checkPassword";
import { createAvatar } from "@dicebear/avatars";
import * as style from "@dicebear/avatars-bottts-sprites";
import { render } from "@react-email/render";
import RegisterEmail from "@codeup/components/emails/Register"
```

```
export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "POST") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  if (
    !req.body ||
    !req.body.email ||
    !req.body.firstName ||
    !req.body.lastName ||
    !req.body.password ||
    !req.body.username
  ) {
    res.status(400).json({
      error: "Missing required fields",
    });
    return;
  }

  if (
    !req.body.email.match(
      /^(([^<>()\[\]\\.,;:\s@"]+(\.[^<>()\[\]\\.,;:\s@"]+)*)|(".+"))@((\[[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}\.[0-
9]{1,3}\])|(([a-zA-Z\-0-9]+\.)+[a-zA-Z]{2,}))$/
    ) ||
    !(req.body.firstName.length > 0) ||
    !(req.body.lastName.length > 0) ||
    !checkPassword(req.body.password) ||
    !/^(?=.{4,20}$)(?![_.])(?!.*[_.]{2})[a-zA-Z0-9._]+(?<![_.])$/.test(
      req.body.username
    )
  ) {
    if (
      !/^(?=.{4,20}$)(?![_.])(?!.*[_.]{2})[a-zA-Z0-9._]+(?<![_.])$/.test(
        req.body.username
      )
    ) {
      res.status(400).json({
        error:
          "Benutzername muss zwischen 4 und 20 Zeichen lang sein und darf nur Buchstaben, Zahlen
und Unterstriche enthalten.",
      });
      return;
    }
    res.status(400).json({
      error: "Invalid fields",
    });
    return;
  }

  await mongoConnect();

  try {
    const acToken = uuid();
```

```
    const svg = createAvatar(style, {
      seed: req.body.username,
      colorful: true,
    });

    const user = await UserModel.create({
      username: req.body.username,
      email: req.body.email,
      firstName: req.body.firstName,
      lastName: req.body.lastName,
      password: crypto
        .createHash("sha256")
        .update(req.body.password)
        .digest("hex"),
      activationToken: acToken,
      profilePicture: svg,
    });

    await user.save();

    await sendMailV2({
      to: req.body.email,
      subject: "Account aktivieren",
      html: render(<RegisterEmail firstName={req.body.firstName}
link={`https://${req.headers.host}/api/account/action?operation=activate&token=${acToken}&redirect=
https://${req.headers.host}#login`} />),
      text: ""
    })

    res.status(200).json({
      success: true,
      message: "Überprüfe deine E-Mails für einen Aktivierungslink",
    });
  } catch (e: any) {
    res.status(500).json({
      error: e.message,
    });
    return;
  }
}
```

## apps/web/src/pages/api/account/reset-confirm.ts

```
/**
 * src/pages/api/account/reset-confirm.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 06.06.2022
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/src/mongo";
import UserModel from "@codeup/util/src/models/UserModel";
import * as crypto from "crypto";
import checkPassword from "@codeup/util/src/checkPassword";

export default async function handleRequest(
```

```
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "POST") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  if (!req.body || !req.body.token || !req.body.password) {
    res.status(400).json({
      error: "Missing required fields",
    });
    return;
  }

  const token = req.body.token;

  await mongoConnect();

  const user = await UserModel.findOne({
    resetToken: token,
  });

  if (!user) {
    res.status(400).json({
      error: "Invalid token",
    });
    return;
  }

  if (!checkPassword(req.body.password)) {
    res.status(400).json({
      error:
        "Password must be at least 8 characters long and contain at least one uppercase letter, one
lowercase letter, one number and one special character",
    });
    return;
  }

  user.password = crypto
    .createHash("sha256")
    .update(req.body.password)
    .digest("hex");

  user.resetToken = null;

  await user.save();

  res.status(200).json({
    success: true,
    message: "Password changed successfully",
  });
}
```

## apps/web/src/pages/api/account/reset.tsx

```
/**
 * src/pages/api/account/reset.ts
 *
```

```
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 06.06.2022
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import UserModel from "@codeup/util/src/models/UserModel";
import mongoConnect from "@codeup/util/src/mongo";
import { uuid } from "uuidv4";
import sendMail, { sendMailV2 } from "@codeup/util/src/sendMail";
import { render } from "@react-email/render";
import ResetEmail from "@codeup/components/emails/Reset";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "POST") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  if (!req.body || !req.body.email) {
    res.status(400).json({
      error: "Missing parameters",
    });
    return;
  }

  await mongoConnect();

  const user = await UserModel.findOne({ email: req.body.email });

  if (!user) {
    res.status(404).json({
      error: "User not found",
    });
    return;
  }

  if (user.resetToken !== null && user.resetToken !== "") {
    res.status(400).json({
      error: "Reset E-Mail already sent",
    });
    return;
  }

  const token = uuid();

  user.resetToken = token;

  await user.save();

  await sendMailV2({
    to: user.email,
    subject: "Passwort zurücksetzen",
```

```
    html: render(<ResetEmail firstName={user.firstName}
link={`http://${req.headers.host}/account/reset/${token}`} />),
    text: ""
  });

  res.status(200).json({
    success: true,
    message: "Reset E-Mail sent",
  });
}
```

## apps/web/src/pages/api/account/try.ts

```
/**
 * apps/web/src/pages/api/account/try.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 13.12.2022
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/src/mongo";
import UserModel from "@codeup/util/src/models/UserModel";
import { createAvatar } from "@dicebear/avatars";
import * as style from "@dicebear/avatars-bottts-sprites";
import crypto from "crypto";
import generateJWT from "@codeup/util/dist/generateJWT";

export default async function handler(
  req: NextApiRequest,
  res: NextApiResponse
) {
  try {
    await mongoConnect();

    const svg = createAvatar(style, {
      seed: req.body.username,
      colorful: true,
    });

    const userIdentifier = randomString(10);

    const user = await UserModel.create({
      username: `test.${userIdentifier}`,
      email: `test.${userIdentifier}@codeup.space`,
      password: crypto
        .createHash("sha256")
        .update(randomStringWithLetters(64))
        .digest("hex"),
      firstName: "Test",
      lastName: "Benutzer",
      createdAt: Date.now(),
      twoFactorEnabled: false,
      twoFactorCode: "",
      active: true,
      deletionToken: "",
      resetToken: "",
      emailChangeCode: "",
```

```
      newEmail: "",
      profilePicture: svg,
      role: "temp",
    });

    await user.save();

    const token = generateJWT(user);

    res.status(200).json({
      token,
    });
  } catch (e: any) {
    if (e.message) {
      res.status(500).json({ error: e.message });
    } else {
      res.status(500).json({ error: "An unknown error occured." });
    }
  }
}

function randomString(l: number): string {
  let s = "";
  while (s.length < l) {
    s += Math.random().toString(36).substr(2);
  }
  return s.substr(0, l);
}

function randomStringWithLetters(l: number): string {
  let x = "abcdefghijklmnopqrstuvwxyz0123456789";
  return randomString(l)
    .split("")
    .map((c) => {
      if (Math.random() > 0.5) {
        return c.toUpperCase();
      } else {
        return c;
      }
    })
    .join("");
}
```

## apps/web/src/pages/api/account/update-2fa.ts

```
/**
 * apps/web/src/pages/api/account/update-2fa.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 25.12.2022
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import jwt from "jsonwebtoken";
import UserModel from "@codeup/util/src/models/UserModel";
import mongoConnect from "@codeup/util/src/mongo";
import { authenticator } from "otplib";
import generateJWT from "@codeup/util/src/generateJWT";
```

```
export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "POST") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  if (!req.body || !req.headers["x-jwt"]) {
    res.status(400).json({
      error: "Missing parameters",
    });
    return;
  }

  const token = req.headers["x-jwt"] as string;

  if (jwt.verify(token, process.env.JWT_SECRET as string)) {
    const decoded = jwt.decode(token) as any;

    await mongoConnect();

    const user = await UserModel.findOne({
      _id: decoded.id,
    });

    if (!user) {
      res.status(404).json({
        error: "User not found",
      });
      return;
    }

    if (req.body.enable) {
      if (user.twoFactorEnabled) {
        res.status(400).json({
          error: "2FA already enabled",
        });
        return;
      }

      const secret = authenticator.generateSecret();
      const url = authenticator.keyuri(user.username, "CodeUp", secret);

      user.twoFactorEnabled = true;

      user.twoFactorSecret = secret;

      await user.save();

      res.status(200).json({
        url,
        newToken: generateJWT(user),
      });
    } else if (req.body.disable) {
      if (!user.twoFactorEnabled) {
```

```
        res.status(400).json({
          error: "2FA already disabled",
        });
        return;
      }

      if (!req.body.code) {
        res.status(400).json({
          error: "Cannot change 2FA without code",
        });
        return;
      }

      const code = req.body.code as string;

      if (!authenticator.check(code, user.twoFactorSecret)) {
        res.status(400).json({
          error: "Invalid code",
        });
        return;
      }

      user.twoFactorEnabled = false;
      user.twoFactorSecret = null;

      await user.save();

      res.status(200).json({
        newToken: generateJWT(user),
      });
      return;
    }

    res.status(400).json({
      error: "Invalid request",
    });
  } else {
    res.status(401).json({
      error: "Invalid token",
    });
  }
}
```

## apps/web/src/pages/api/account/update.tsx

```
/**
 * src/pages/api/account/update.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 06.06.2022
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import jwt from "jsonwebtoken";
import UserModel from "@codeup/util/src/models/UserModel";
import mongoConnect from "@codeup/util/src/mongo";
import checkPassword from "@codeup/util/src/checkPassword";
import generateJWT from "@codeup/util/src/generateJWT";
```

```
import { uuid } from "uuidv4";
import sendMail, { sendMailV2 } from "@codeup/util/src/sendMail";
import * as crypto from "crypto";
import { render } from "@react-email/render";
import UpdateEmail from "@codeup/components/emails/UpdateEmail";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "POST") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  if (
    !req.body ||
    !req.headers["x-jwt"] ||
    !req.body.keyName ||
    !req.body.keyValue
  ) {
    res.status(400).json({
      error: "Missing parameters",
    });
    return;
  }

  const token = req.headers["x-jwt"] as string;

  if (jwt.verify(token, process.env.JWT_SECRET as string)) {
    const decoded = jwt.decode(token) as any;

    await mongoConnect();

    const user = await UserModel.findOne({
      _id: decoded.id,
    });

    if (!user) {
      res.status(404).json({
        error: "User not found",
      });
      return;
    }

    let allowedKeyNames = ["firstName", "lastName", "email", "2fa", "password"];

    if (!allowedKeyNames.includes(req.body.keyName)) {
      res.status(400).json({
        error: "Invalid key name",
      });
      return;
    }

    let ret = true;

    switch (req.body.keyName) {
      case "firstName":
```

```
        user.firstName = req.body.keyValue;
        break;
      case "lastName":
        user.lastName = req.body.keyValue;
        break;
      case "email":
        ret = false;
        if (
          !req.body.keyValue.match(
            /^(([^<>()\[\]\\.,;:\s@"]+(\.[^<>()\[\]\\.,;:\s@"]+)*)|(".+"))@((\[[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}\])|(([a-zA-Z\-0-9]+\.)+[a-zA-Z]{2,}))$/
          )
        ) {
          res.status(400).json({
            error: "Invalid email",
          });
        } else {
          if (user.emailChangeCode !== "" && user.emailChangeCode !== null) {
            res.status(400).json({
              error: "Email change code already sent",
            });
          }
          if (
            await UserModel.findOne({
              email: req.body.email,
            })
          ) {
            res.status(400).json({
              error: "Email already in use",
            });
          } else {
            user.emailChangeCode = uuid();
            user.newEmail = req.body.keyValue;

            await sendMailV2({
              to: user.email,
              subject: "E-Mail Adresse ändern",
              text: "",
              html: render(<UpdateEmail firstName={user.firstName} link={`https://${req.headers.host}/api/account/action?operation=changeEmail&token=${user.emailChangeCode}&redirect=${encodeURIComponent(`https://${req.headers.host}/account/settings?component=account&newToken=${generateJWT(user)}`)}`} />)
            })

            res.status(200).json({
              success: true,
              message: "Please check your email for a verification code",
            });
          }
        }
        break;
      case "2fa":
        user.twoFactorEnabled = req.body.keyValue === "true";
        break;
      case "password":
        const password = req.body.keyValue;

        if (!checkPassword(password)) {
          res.status(400).json({
            error:
```

```
            "Das Passwort muss aus mindestens 8 Buchstaben, einen Großbuchstaben, einen Kleinbuchstaben eine Ziffer und ein Sonderzeichen bestehen.",
          });
          return;
        }

        user.password = crypto
          .createHash("sha256")
          .update(password)
          .digest("hex");
        break;
    }

    await user.save();

    if (ret) {
      res.status(200).json({
        success: true,
        newToken: generateJWT(user),
      });
    }
  } else {
    res.status(401).json({
      error: "Invalid token",
    });
  }
}
```

## apps/web/src/pages/api/account/verify.ts

```
/**
 * src/pages/api/account/verify.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 18.09.2022
 *
 */

import UserModel from "@codeup/util/src/models/UserModel";
import mongoConnect from "@codeup/util/src/mongo";
import generateJWT from "@codeup/util/src/generateJWT";
import { decode, verify } from "jsonwebtoken";
import { NextApiRequest, NextApiResponse } from "next";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  const headers = req.headers;

  if (headers["x-jwt"]) {
    const token = headers["x-jwt"];

    try {
      if (verify(token as string, process.env.JWT_SECRET as string)) {
        const decoded = verify(
          token as string,
          process.env.JWT_SECRET as string
        ) as any;
```

```
      await mongoConnect();

      const user = await UserModel.findOne({
        username: decoded.username as string,
      });

      if (!user || !user.active) {
        res.status(401).json({
          error: "Unauthorized",
        });
        return;
      }

      // detect diff between user and decoded

      const diffKeys = [
        "username",
        "email",
        "admin",
        "firstName",
        "lastName",
        "twoFactorEnabled",
        "subscription",
      ];

      let dU = decoded as any;

      dU.twoFactorEnabled = dU._2fa;
      delete dU._2fa;

      const diff = diffKeys.filter((key) => {
        return user[key] !== dU[key];
      });

      if (diff.length > 0) {
        const newToken = generateJWT(user);
        res.status(200).json({
          token: newToken,
          verified: true,
        });
        return;
      }

      res.status(200).json({
        verified: true,
        data: decoded,
      });
    } else {
      res.status(401).json({
        verified: false,
      });
    }
  } catch (e) {
    res.status(401).json({
      verified: false,
    });
  }
} else {
  res.status(403).json({
```

```
      loggedIn: false,
    });
  }
}
```

## apps/web/src/pages/api/account/[username]/picture.ts

```
/**
 * apps/web/src/pages/api/account/[username]/picture.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 18.09.2022
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import { decode, verify } from "jsonwebtoken";
import mongoConnect from "@codeup/util/src/mongo";
import UserModel from "@codeup/util/src/models/UserModel";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  try {
    await mongoConnect();

    const user = await UserModel.findOne({ username: req.query.username });

    if (user) {
      res.setHeader("Content-Type", "image/svg+xml");
      res.status(200).send(user.profilePicture);
    } else {
      throw new Error("User not found");
    }
  } catch (e) {
    res.setHeader("Content-Type", "image/svg+xml");
    res
      .status(200)
      .send(
        `<svg xmlns:dc="http://purl.org/dc/elements/1.1/" xmlns:cc="http://creativecommons.org/ns#"
xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#" xmlns:svg="http://www.w3.org/2000/svg"
xmlns="http://www.w3.org/2000/svg" viewBox="0 0 180
180"><metadata><rdf:RDF><cc:Work><dc:format>image/svg+xml</dc:format><dc:type
rdf:resource="http://purl.org/dc/dcmitype/StillImage"/><dc:title>Bottts</dc:title><dc:creator><cc:Agent
><dc:title>Pablo
Stanley</dc:title></cc:Agent></dc:creator><dc:source>https://bottts.com/</dc:source><cc:license
rdf:resource="https://bottts.com/"/><dc:contributor><cc:Agent><dc:title>Florian
Körner</dc:title></cc:Agent></dc:contributor></cc:Work></rdf:RDF></metadata><mask
id="avatarsRadiusMask"><rect width="180" height="180" rx="0" ry="0" x="0" y="0"
fill="#fff"/></mask><g mask="url(#avatarsRadiusMask)"><g transform="translate(0, 66)"><path fill-
rule="evenodd" clip-rule="evenodd" d="M14.9809 20.9141C14 22.8393 14 25.3595 14 30.4V45.6C14
50.6405 14 53.1607 14.9809 55.0859C15.8438 56.7794 17.2206 58.1562 18.9141 59.0191C20.8393
60 23.3595 60 28.4 60H35.6C40.6405 60 43.1607 60 45.0859 59.0191C46.7794 58.1562 48.1562
56.7794 49.0191 55.0859C50 53.1607 50 50.6405 50 45.6V30.4C50 25.3595 50 22.8393 49.0191
20.9141C48.1562 19.2206 46.7794 17.8438 45.0859 16.9809C43.1607 16 40.6405 16 35.6
16H28.4C23.3595 16 20.8393 16 18.9141 16.9809C17.2206 17.8438 15.8438 19.2206 14.9809
20.9141ZM130.981 20.9141C130 22.8393 130 25.3595 130 30.4V45.6C130 50.6405 130 53.1607
130.981 55.0859C131.844 56.7794 133.221 58.1562 134.914 59.0191C136.839 60 139.36 60 144.4
```


60H151.6C156.64 60 159.161 60 161.086 59.0191C162.779 58.1562 164.156 56.7794 165.019 55.0859C166 53.1607 166 50.6405 166 45.6V30.4C166 25.3595 166 22.8393 165.019 20.9141C164.156 19.2206 162.779 17.8438 161.086 16.9809C159.161 16 156.64 16 151.6 16H144.4C139.36 16 136.839 16 134.914 16.9809C133.221 17.8438 131.844 19.2206 130.981 20.9141Z" fill="#0076DE"/><mask id="sidesSquareMask0" mask-type="alpha" maskUnits="userSpaceOnUse" x="14" y="16" width="152" height="44"><path fill-rule="evenodd" clip-rule="evenodd" d="M14.9809 20.9141C14 22.8393 14 25.3595 14 30.4V45.6C14 50.6405 14 53.1607 14.9809 55.0859C15.8438 56.7794 17.2206 58.1562 18.9141 59.0191C20.8393 60 23.3595 60 28.4 60H35.6C40.6405 60 43.1607 60 45.0859 59.0191C46.7794 58.1562 48.1562 56.7794 49.0191 55.0859C50 53.1607 50 50.6405 50 45.6V30.4C50 25.3595 50 22.8393 49.0191 20.9141C48.1562 19.2206 46.7794 17.8438 45.0859 16.9809C43.1607 16 40.6405 16 35.6 16H28.4C23.3595 16 20.8393 16 18.9141 16.9809C17.2206 17.8438 15.8438 19.2206 14.9809 20.9141ZM130.981 20.9141C130 22.8393 130 25.3595 130 30.4V45.6C130 50.6405 130 53.1607 130.981 55.0859C131.844 56.7794 133.221 58.1562 134.914 59.0191C136.839 60 139.36 60 144.4 60H151.6C156.64 60 159.161 60 161.086 59.0191C162.779 58.1562 164.156 56.7794 165.019 55.0859C166 53.1607 166 50.6405 166 45.6V30.4C166 25.3595 166 22.8393 165.019 20.9141C164.156 19.2206 162.779 17.8438 161.086 16.9809C159.161 16 156.64 16 151.6 16H144.4C139.36 16 136.839 16 134.914 16.9809C133.221 17.8438 131.844 19.2206 130.981 20.9141Z" fill="white"/></mask><g mask="url(#sidesSquareMask0)"><rect width="180" height="76" fill="#D4E157"/><rect y="38" width="180" height="38" fill="black" fill-opacity="0.1"/></g></g><g transform="translate(41, 0)"><path fill-rule="evenodd" clip-rule="evenodd" d="M52 5H48V36H40C38.8954 36 38 36.8954 38 38V52H62V38C62 36.8954 61.1046 36 60 36H52V5Z" fill="#E1E6E8"/><mask id="topAntennaMask0" mask-type="alpha" maskUnits="userSpaceOnUse" x="38" y="5" width="24" height="47"><path fill-rule="evenodd" clip-rule="evenodd" d="M52 5H48V36H40C38.8954 36 38 36.8954 38 38V52H62V38C62 36.8954 61.1046 36 60 36H52V5Z" fill="white"/></mask><g mask="url(#topAntennaMask0)"><rect width="100" height="52" fill="#D4E157"/><rect x="38" y="36" width="24" height="16" fill="white" fill-opacity="0.2"/></g><path fill-rule="evenodd" clip-rule="evenodd" d="M50 16C54.4183 16 58 12.4183 58 8C58 3.58172 54.4183 0 50 0C45.5817 0 42 3.58172 42 8C42 12.4183 45.5817 16 50 16Z" fill="#FFE65C"/><path fill-rule="evenodd" clip-rule="evenodd" d="M53 8C54.6569 8 56 6.65685 56 5C56 3.34315 54.6569 2 53 2C51.3431 2 50 3.34315 50 5C50 6.65685 51.3431 8 53 8Z" fill="white"/></g><g transform="translate(25, 44)"><path fill-rule="evenodd" clip-rule="evenodd" d="M0 18C0 8.05888 8.05888 0 18 0H112C121.941 0 130 8.05888 130 18V45.1483C130 49.6831 129.229 54.1848 127.72 58.4611L110.239 107.991C107.699 115.187 100.896 120 93.2647 120H36.7353C29.1036 120 22.3014 115.187 19.7614 107.991L2.28038 58.4611C0.771117 54.1848 0 49.6831 0 45.1483L0 18Z" fill="#0076DE"/><mask id="faceSquare03Mask0" mask-type="alpha" maskUnits="userSpaceOnUse" x="0" y="0" width="130" height="120"><path fill-rule="evenodd" clip-rule="evenodd" d="M0 18C0 8.05888 8.05888 0 18 0H112C121.941 0 130 8.05888 130 18V45.1483C130 49.6831 129.229 54.1848 127.72 58.4611L110.239 107.991C107.699 115.187 100.896 120 93.2647 120H36.7353C29.1036 120 22.3014 115.187 19.7614 107.991L2.28038 58.4611C0.771117 54.1848 0 49.6831 0 45.1483L0 18Z" fill="white"/></mask><g mask="url(#faceSquare03Mask0)"><rect x="-2" y="-2" width="134" height="124" fill="#C0CA33"/><path fill-rule="evenodd" clip-rule="evenodd" d="M0.880859 128.092C5.24177 97.4016 13.9384 123.534 34.866 118.124C44.1119 115.734 36.6414 105.296 43.5373 100.772C49.3622 96.9503 57.3002 100.029 63.9726 97.5579C73.6018 93.9913 73.4753 86.7354 82.4474 85.651C90.9322 84.6255 99.8208 88.005 108.378 86.7354C124.529 84.3395 117.907 52.7111 130.018 47.1252C142.129 41.5393 162.068 51.1858 164.678 67C167.222 82.4167 150.22 139.78 150.22 139.78H0.880859" fill="black" fill-opacity="0.2"/><path fill-rule="evenodd" clip-rule="evenodd" d="M45.9268 128C83.6043 128 87.5268 113.25 106.065 106.493C124.603 99.7354 140.846 117.214 147.078 100.233C153.31 83.2528 153.276 136.181 153.276 136.181H45.9268L45.9268 128Z" fill="black" fill-opacity="0.1"/><path fill-rule="evenodd" clip-rule="evenodd" d="M7.99964 10.8673C19.7328 9.74652 15.9616 22.8061 20.597 22.8061C22.4461 22.8061 24.6249 14.3489 27.9829 14.3489C30.4376 14.3489 29.9749 19.8425 35.3046 19.2617C40.3073 18.7166 39.375 15.2687 41.9023 15.2687C47.345 15.2687 45.2869 35.4636 49.6437 35.4636C54.0005 35.4636 55.3734 20.668 57.2911 14.8164C59.7177 7.4119 74.2632 3.26163 63.8919 0.320129H7.71289" fill="black" fill-opacity="0.4"/></g></g><g transform="translate(52, 124)"><path fill-rule="evenodd" clip-rule="evenodd" d="M18 10.2222C18 21.7845 24.4741 28 38 28C51.5182 28 58 21.6615 58 10.2222C58 9.49622 57.1739 8 55 8C39.2707 8 29.1917 8 21 8C18.949 8 18 9.38479 18 10.2222Z" fill="black" fill-opacity="0.8"/><mask id="mouthSmilie02Mask0" mask-type="alpha" maskUnits="userSpaceOnUse" x="18" y="8" width="40" height="20"><path fill-rule="evenodd" clip-rule="evenodd" d="M18 10.2222C18 21.7845

```
24.4741 28 38 28C51.5182 28 58 21.6615 58 10.2222C58 9.49622 57.1739 8 55 8C39.2707 8
29.1917 8 21 8C18.949 8 18 9.38479 18 10.2222Z" fill="white"/></mask><g
mask="url(#mouthSmilie02Mask0)"><rect x="30" y="2" width="16" height="14" rx="2"
fill="white"/></g></g><g transform="translate(38, 76)"><path fill-rule="evenodd" clip-rule="evenodd"
d="M25 27.2L30.5 32.7C31 33.1 31.7 33.1 32.1 32.7L33.7 31.1C34.1 30.6 34.1 29.9 33.7 29.5L28.2
24L33.7 18.5C34.1 18 34.1 17.3 33.7 16.9L32.1 15.3C31.6 14.9 30.9 14.9 30.5 15.3L25 20.8L19.5
15.3C19 14.9 18.3 14.9 17.9 15.3L16.3 16.9C15.9 17.3 15.9 18 16.3 18.5L21.8 24L16.3 29.5C15.9
30 15.9 30.7 16.3 31.1L17.9 32.7C18.4 33.1 19.1 33.1 19.5 32.7L25 27.2Z" fill="black" fill-
opacity="0.8"/><path fill-rule="evenodd" clip-rule="evenodd" d="M79 27.2L84.5 32.7C85 33.1 85.7
33.1 86.1 32.7L87.7 31.1C88.1 30.6 88.1 29.9 87.7 29.5L82.2 24L87.7 18.5C88.1 18 88.1 17.3 87.7
16.9L86.1 15.3C85.6 14.9 84.9 14.9 84.5 15.3L79 20.8L73.5 15.3C73 14.9 72.3 14.9 71.9 15.3L70.3
16.9C69.9 17.3 69.9 18 70.3 18.5L75.8 24L70.3 29.5C69.9 30 69.9 30.7 70.3 31.1L71.9 32.7C72.4
33.1 73.1 33.1 73.5 32.7L79 27.2Z" fill="black" fill-opacity="0.8"/></g></g></svg>`
      );
  }
}
```

## apps/web/src/pages/api/admin/account/create.ts

```
/**
 * apps/web/src/pages/api/admin/account/create.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 08.10.22
 *
 */

import mongoConnect from "@codeup/util/src/mongo";
import { NextApiRequest, NextApiResponse } from "next";
import { verify } from "jsonwebtoken";
import UserModel from "@codeup/util/src/models/UserModel";
import { createHash } from "crypto";
import { createAvatar } from "@dicebear/avatars";
import * as style from "@dicebear/avatars-bottts-sprites";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "POST") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  try {
    await mongoConnect();

    if (!req.headers["x-jwt"]) {
      res.status(401).json({
        error: "Unauthorized",
      });
      return;
    }

    const { username, email, firstName, lastName, active, password, role } =
      req.body;
```

```
if (
  !username ||
  !email ||
  !firstName ||
  !lastName ||
  !active ||
  !password ||
  !role
) {
  res.status(400).json({
    error: "Bad request",
  });
  return;
}

if (
  !verify(req.headers["x-jwt"] as string, process.env.JWT_SECRET as string)
) {
  res.status(401).json({
    error: "Unauthorized",
  });
  return;
}

const decoded = verify(
  req.headers["x-jwt"] as string,
  process.env.JWT_SECRET as string
) as any;

const adminuser = await UserModel.findOne({ username: decoded.username });

if (adminuser.role !== "admin") {
  res.status(401).json({
    error: "Unauthorized",
  });
  return;
}

const user = await UserModel.findOne({ username });

if (user) {
  res.status(400).json({
    error: "User already exists",
  });
  return;
}

const svg = createAvatar(style, {
  seed: username,
  colorful: true,
});

const newUser = new UserModel({
  username,
  email,
  firstName,
  lastName,
  active,
  role,
  password: createHash("sha256").update(password).digest("hex"),
```

```
      profilePicture: svg,
    });

    await newUser.save();

    res.status(200).json({
      success: true,
      user: newUser,
    });
  } catch (e) {
    res.status(500).json({
      error: "Internal Server Error",
      // @ts-ignore
      message: e.message,
    });
  }
}
```

## apps/web/src/pages/api/admin/account/delete.ts

```
/**
 * apps/web/src/pages/api/admin/account/delete.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 07.10.22
 *
 */
import mongoConnect from "@codeup/util/src/mongo";
import { NextApiRequest, NextApiResponse } from "next";
import { verify } from "jsonwebtoken";
import UserModel from "@codeup/util/src/models/UserModel";
import { createHash } from "crypto";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "POST") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  try {
    await mongoConnect();

    if (!req.headers["x-jwt"]) {
      res.status(401).json({
        error: "Unauthorized",
      });
      return;
    }

    const { username } = req.body;

    if (!username) {
      res.status(400).json({
        error: "Bad request",
```

```
      });
      return;
    }

    if (
      !verify(req.headers["x-jwt"] as string, process.env.JWT_SECRET as string)
    ) {
      res.status(401).json({
        error: "Unauthorized",
      });
      return;
    }

    const decoded = verify(
      req.headers["x-jwt"] as string,
      process.env.JWT_SECRET as string
    ) as any;

    const adminuser = await UserModel.findOne({ username: decoded.username });

    if (adminuser.role !== "admin") {
      res.status(401).json({
        error: "Unauthorized",
      });
      return;
    }

    const user = await UserModel.findOne({ username });

    if (!user) {
      res.status(404).json({
        error: "User not found",
      });
      return;
    }

    await user.remove();

    res.status(200).json({
      success: true,
    });
  } catch (e) {
    res.status(500).json({
      error: "Internal Server Error",
    });
  }
}
```

## apps/web/src/pages/api/admin/account/jwt.ts

```
/**
 * apps/web/src/pages/api/admin/account/jwt.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 25.01.23
 *
 */
import mongoConnect from "@codeup/util/src/mongo";
import { NextApiRequest, NextApiResponse } from "next";
```

```
import { verify } from "jsonwebtoken";
import UserModel from "@codeup/util/src/models/UserModel";
import { createHash } from "crypto";
import generateJWT from "@codeup/util/dist/generateJWT";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "POST") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  try {
    await mongoConnect();

    if (!req.headers["x-jwt"]) {
      res.status(401).json({
        error: "Unauthorized",
      });
      return;
    }

    const { username } = req.body;

    if (!username) {
      res.status(400).json({
        error: "Bad request",
      });
      return;
    }

    if (
      !verify(req.headers["x-jwt"] as string, process.env.JWT_SECRET as string)
    ) {
      res.status(401).json({
        error: "Unauthorized",
      });
      return;
    }

    const decoded = verify(
      req.headers["x-jwt"] as string,
      process.env.JWT_SECRET as string
    ) as any;

    const adminuser = await UserModel.findOne({ username: decoded.username });

    if (adminuser.role !== "admin") {
      res.status(401).json({
        error: "Unauthorized",
      });
      return;
    }

    const user = await UserModel.findOne({ username });
```

```
    if (!user) {
      res.status(404).json({
        error: "User not found",
      });
      return;
    }

    const jwt = generateJWT(user);

    res.status(200).json({
      jwt,
      success: true
    });
  } catch (e) {
    res.status(500).json({
      error: "Internal Server Error",
    });
  }
}
```

## apps/web/src/pages/api/admin/account/list.ts

```
/**
 * apps/web/src/pages/api/admin/account/list.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 07.10.22
 *
 */
import { NextApiRequest, NextApiResponse } from "next";
import { verify } from "jsonwebtoken";
import mongoConnect from "@codeup/util/src/mongo";
import UserModel from "@codeup/util/src/models/UserModel";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "GET") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  if (!req.headers["x-jwt"]) {
    res.status(401).json({
      error: "Unauthorized",
    });
    return;
  }

  try {
    const jwt = req.headers["x-jwt"];
    if (!verify(jwt as string, process.env.JWT_SECRET as string)) {
      res.status(401).json({
        error: "Unauthorized",
      });
      return;
```

```
    }

    const decoded = verify(
      jwt as string,
      process.env.JWT_SECRET as string
    ) as any;

    const adminuser = await UserModel.findOne({ username: decoded.username });

    if (adminuser.role !== "admin") {
      res.status(401).json({
        error: "Unauthorized",
      });
      return;
    }

    await mongoConnect();

    const accounts = await UserModel.find();

    res.status(200).json({
      accounts,
    });
  } catch (e) {
    res.status(500).json({
      error: "Internal Server Error",
    });
  }
}
```

## apps/web/src/pages/api/admin/account/update.ts

```
/**
 * apps/web/src/pages/api/admin/account/update.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 07.10.22
 *
 */
import mongoConnect from "@codeup/util/src/mongo";
import { NextApiRequest, NextApiResponse } from "next";
import { verify } from "jsonwebtoken";
import UserModel from "@codeup/util/src/models/UserModel";
import { createHash } from "crypto";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "POST") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  try {
    await mongoConnect();
```

```
if (!req.headers["x-jwt"]) {
  res.status(401).json({
    error: "Unauthorized",
  });
  return;
}

const { username, update } = req.body;

if (!username || !update) {
  res.status(400).json({
    error: "Bad request",
  });
  return;
}

if (
  !verify(req.headers["x-jwt"] as string, process.env.JWT_SECRET as string)
) {
  res.status(401).json({
    error: "Unauthorized",
  });
  return;
}

const decoded = verify(
  req.headers["x-jwt"] as string,
  process.env.JWT_SECRET as string
) as any;

const adminuser = await UserModel.findOne({ username: decoded.username });

if (adminuser.role !== "admin") {
  res.status(401).json({
    error: "Unauthorized",
  });
  return;
}

const user = await UserModel.findOne({ username });

if (!user) {
  res.status(404).json({
    error: "User not found",
  });
  return;
}

const allowedKeys = [
  "username",
  "email",
  "password",
  "firstName",
  "lastName",
  "twoFactorEnabled",
  "active",
  "role",
];

for (const key in update) {
```

```
      if (allowedKeys.includes(key)) {
        let v = update[key];

        if (key === "password") {
          v = createHash("sha256").update(v).digest("hex");
        }
        user[key] = v;
      }
    }

    await user.save();

    res.status(200).json({
      success: true,
      user: user,
    });
  } catch (e) {
    res.status(500).json({
      error: "Internal Server Error",
    });
  }
}
```

## apps/web/src/pages/api/admin/analytics/delete.ts

```
/**
 * apps/web/src/pages/api/admin/analytics/views.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 25.11.2022
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import { verify } from "jsonwebtoken";
import mongoConnect from "@codeup/util/src/mongo";
import UserModel from "@codeup/util/src/models/UserModel";
import AnalyticsModel from "@codeup/util/src/models/AnalyticsModel";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "POST") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  if (!req.headers["x-jwt"]) {
    res.status(401).json({
      error: "Unauthorized",
    });
    return;
  }

  try {
    const jwt = req.headers["x-jwt"];
```

```
    if (!verify(jwt as string, process.env.JWT_SECRET as string)) {
      res.status(401).json({
        error: "Unauthorized",
      });
      return;
    }

    const decoded = verify(
      jwt as string,
      process.env.JWT_SECRET as string
    ) as any;

    const adminuser = await UserModel.findOne({ username: decoded.username });

    if (adminuser.role !== "admin") {
      res.status(401).json({
        error: "Unauthorized",
      });
      return;
    }

    await mongoConnect();

    let period = {
      start: req.query.periodStart as string,
      end: req.query.periodEnd as string,
    };

    await AnalyticsModel.remove({
      createdAt: {
        $gte: new Date(period.start),
        $lte: new Date(period.end),
      },
    });

    res.status(200).json({
      success: true,
    });
  } catch (e) {
    console.log(e);

    res.status(500).json({
      error: "Internal Server Error",
    });
  }
}
```

## apps/web/src/pages/api/admin/analytics/summary.ts

```
/**
 * apps/web/src/pages/api/admin/analytics/summary.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 25.11.2022
 *
 */
import { NextApiRequest, NextApiResponse } from "next";
import { verify } from "jsonwebtoken";
import mongoConnect from "@codeup/util/src/mongo";
```

```
import UserModel from "@codeup/util/src/models/UserModel";
import AnalyticsModel from "@codeup/util/src/models/AnalyticsModel";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "GET") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  if (!req.headers["x-jwt"]) {
    res.status(401).json({
      error: "Unauthorized",
    });
    return;
  }

  try {
    const jwt = req.headers["x-jwt"];
    if (!verify(jwt as string, process.env.JWT_SECRET as string)) {
      res.status(401).json({
        error: "Unauthorized",
      });
      return;
    }

    const decoded = verify(
      jwt as string,
      process.env.JWT_SECRET as string
    ) as any;

    const adminuser = await UserModel.findOne({ username: decoded.username });

    if (adminuser.role !== "admin") {
      res.status(401).json({
        error: "Unauthorized",
      });
      return;
    }

    await mongoConnect();

    const data: {
      browsers: {
        // BROWSER: Percentage
        [key: string]: number;
      };
      os: {
        // OS: Percentage
        [key: string]: number;
      };
      paths: {
        // PATH: Percentage
        [key: string]: number;
      };
      // Logged in views percentage
```

```
  authenticatedViewPercentage: number;
  // Not logged in views percentage
  anonymousViewPercentage: number;
  // Total views
  totalViews: number;
  todayActiveUsers: {
    username: string;
    userId: string;
    views: number;
  }[];
} = {
  browsers: {},
  paths: {},
  os: {},
  authenticatedViewPercentage: 0,
  anonymousViewPercentage: 0,
  totalViews: 0,
  todayActiveUsers: [],
};

let findObject: any = {};

if (req.query.userId) {
  findObject.userId = req.query.userId;
}

const analytics = await AnalyticsModel.find(findObject);

analytics.forEach((a) => {
  data.totalViews++;

  if (a.browser) {
    if (data.browsers[a.browser]) {
      data.browsers[a.browser]++;
    } else {
      data.browsers[a.browser] = 1;
    }
  }

  if (a.os) {
    if (data.os[a.os]) {
      data.os[a.os]++;
    } else {
      data.os[a.os] = 1;
    }
  }

  if (a.path) {
    if (data.paths[a.path]) {
      data.paths[a.path]++;
    } else {
      data.paths[a.path] = 1;
    }
  }

  if (a.loggedIn) {
    data.authenticatedViewPercentage++;
  } else {
    data.anonymousViewPercentage++;
  }
```

```
  });

  data.authenticatedViewPercentage =
    (data.authenticatedViewPercentage / data.totalViews) * 100;
  data.anonymousViewPercentage =
    (data.anonymousViewPercentage / data.totalViews) * 100;

  for (const b in data.browsers) {
    data.browsers[b] = Math.round((data.browsers[b] / data.totalViews) * 100);
  }

  for (const o in data.os) {
    data.os[o] = Math.round((data.os[o] / data.totalViews) * 100);
  }

  for (const p in data.paths) {
    data.paths[p] = Math.round((data.paths[p] / data.totalViews) * 100);
  }

  const todayViews = await AnalyticsModel.aggregate([
    {
      $match: {
        createdAt: {
          $gte: new Date(new Date().getTime() - 24 * 60 * 60 * 1000),
        },
      },
    },
  ]);

  const todayActiveUsers: {
    username: string;
    views: number;
    userId: string;
  }[] = [];

  for (const v of todayViews) {
    const user = await UserModel.findOne({ _id: v.userId });

    if (user) {
      if (todayActiveUsers.find((u) => u.username === user.username)) {
        todayActiveUsers.find((u) => u.username === user.username)!.views++;
      } else {
        todayActiveUsers.push({
          username: user.username,
          views: 1,
          userId: user._id,
        });
      }
    }
  }

  data.todayActiveUsers = todayActiveUsers;

  res.status(200).json({ data });
} catch (e) {
  console.log(e);

  res.status(500).json({
    error: "Internal Server Error",
  });
```

```
  }
}
```

## apps/web/src/pages/api/admin/analytics/user-summary.ts

```
/**
 * apps/web/src/pages/api/admin/analytics/user-summary.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 07.04.2023
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import { verify } from "jsonwebtoken";
import mongoConnect from "@codeup/util/src/mongo";
import UserModel from "@codeup/util/src/models/UserModel";
import AnalyticsModel from "@codeup/util/src/models/AnalyticsModel";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "GET") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  if (!req.headers["x-jwt"]) {
    res.status(401).json({
      error: "Unauthorized",
    });
    return;
  }

  if (!req.query.userId) {
    res.status(400).json({
      error: "Bad Request",
    });
    return;
  }

  try {
    const jwt = req.headers["x-jwt"];
    if (!verify(jwt as string, process.env.JWT_SECRET as string)) {
      res.status(401).json({
        error: "Unauthorized",
      });
      return;
    }

    const decoded = verify(
      jwt as string,
      process.env.JWT_SECRET as string
    ) as any;

    await mongoConnect();
```

```
const adminuser = await UserModel.findOne({ username: decoded.username });

if (!adminuser) {
  res.status(401).json({
    error: "Unauthorized",
  });
  return;
}

if (adminuser.role !== "admin") {
  res.status(401).json({
    error: "Unauthorized",
  });
  return;
}

const user = await UserModel.findById(req.query.userId as string);

if (!user) {
  res.status(400).json({
    error: "Bad Request",
  });
  return;
}
const analytics = await AnalyticsModel.find({
  userId: user._id,
});

if (req.query.detailed === "true") {
  res.status(200).json({
    analytics,
  });
} else {
  if (analytics.length === 0) {
    res.status(200).json({
      first: {
        view: "2023-04-06T23:14:59.367Z",
        page: "",
        browser: "",
        os: "",
      },
      last: {
        view: "2023-04-07T23:14:59.368Z",
        page: "",
        browser: "",
        os: "",
      },
      totalViews: 0,
      averageViewsPerDay: 0,
    });
    return;
  }

  const firstView = analytics.sort((a, b) => a.createdAt - b.createdAt)[0];

  const lastView = analytics.sort((a, b) => b.createdAt - a.createdAt)[0];

  res.status(200).json({
    first: {
```

```
          view: firstView.createdAt,
          page: firstView.path,
          browser: firstView.browser,
          os: firstView.os,
        },
        last: {
          view: lastView.createdAt,
          page: lastView.path,
          browser: lastView.browser,
          os: lastView.os,
        },
        totalViews: analytics.length,
        averageViewsPerDay:
          (analytics.length / (new Date().getTime() - firstView.createdAt)) *
          1000 *
          60 *
          60 *
          24,
      });
    }
  } catch (e) {
    console.log(e);

    res.status(500).json({
      error: "Internal Server Error",
    });
  }
}
```

## apps/web/src/pages/api/admin/analytics/views.ts

```
/**
 * apps/web/src/pages/api/admin/analytics/views.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 25.11.2022
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import { verify } from "jsonwebtoken";
import mongoConnect from "@codeup/util/src/mongo";
import UserModel from "@codeup/util/src/models/UserModel";
import AnalyticsModel from "@codeup/util/src/models/AnalyticsModel";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "GET") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  if (!req.headers["x-jwt"]) {
    res.status(401).json({
      error: "Unauthorized",
```

```
  });
  return;
}

try {
  const jwt = req.headers["x-jwt"];
  if (!verify(jwt as string, process.env.JWT_SECRET as string)) {
    res.status(401).json({
      error: "Unauthorized",
    });
    return;
  }

  const decoded = verify(
    jwt as string,
    process.env.JWT_SECRET as string
  ) as any;

  await mongoConnect();

  const adminuser = await UserModel.findOne({ username: decoded.username });

  if (adminuser.role !== "admin") {
    res.status(401).json({
      error: "Unauthorized",
    });
    return;
  }

  const data: number[] = [];

  const defaultPeriod = {
    start: new Date(new Date().getTime() - 7 * 24 * 60 * 60 * 1000)
      .toISOString()
      .split("T")[0],
    end: new Date(new Date().getTime() + 24 * 60 * 60 * 1000)
      .toISOString()
      .split("T")[0],
  };
  let period = defaultPeriod;

  if (req.query.periodStart && req.query.periodEnd) {
    period = {
      start: req.query.periodStart as string,
      end: req.query.periodEnd as string,
    };
  }

  const analytics = await AnalyticsModel.find({
    createdAt: {
      $gte: new Date(period.start),
      $lte: new Date(period.end),
    },
  });

  const days = Math.round(
    (new Date(period.end).getTime() - new Date(period.start).getTime()) /
      (1000 * 60 * 60 * 24)
  );
```

```
    for (let i = 0; i < days; i++) {
      data.push(0);

      analytics.forEach((a) => {
        if (
          new Date(a.createdAt).getDate() ===
            new Date(
              new Date(period.start).getTime() + i * 24 * 60 * 60 * 1000
            ).getDate() &&
          new Date(a.createdAt).getMonth() ===
            new Date(
              new Date(period.start).getTime() + i * 24 * 60 * 60 * 1000
            ).getMonth() &&
          new Date(a.createdAt).getFullYear() ===
            new Date(
              new Date(period.start).getTime() + i * 24 * 60 * 60 * 1000
            ).getFullYear()
        ) {
          data[i]++;
        }
      });
    }

    res.status(200).json({ data });
  } catch (e) {
    console.log(e);

    res.status(500).json({
      error: "Internal Server Error",
    });
  }
}
```

## apps/web/src/pages/api/admin/blog/index.ts

```
/**
 * apps/web/src/pages/api/admin/blog/index.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 27.12.2022
 *
 */
import { NextApiRequest, NextApiResponse } from "next";
import { verify } from "jsonwebtoken";
import mongoConnect from "@codeup/util/src/mongo";
import UserModel from "@codeup/util/src/models/UserModel";
import BlogModel from "@codeup/util/src/models/BlogModel";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (!req.headers["x-jwt"]) {
    res.status(401).json({
      error: "Unauthorized",
    });
    return;
  }
```

```
try {
  const jwt = req.headers["x-jwt"];
  if (!verify(jwt as string, process.env.JWT_SECRET as string)) {
    res.status(401).json({
      error: "Unauthorized",
    });
    return;
  }

  const decoded = verify(
    jwt as string,
    process.env.JWT_SECRET as string
  ) as any;

  const adminuser = await UserModel.findOne({ username: decoded.username });

  if (adminuser.role !== "admin") {
    res.status(401).json({
      error: "Unauthorized",
    });
    return;
  }

  await mongoConnect();

  if (req.method === "GET") {
    const blogs = await BlogModel.find({});

    res.status(200).json({
      blogs,
    });
    return;
  } else if (req.method === "POST") {
    const { title, content, tags } = req.body;

    if (!title || !content || !tags) {
      res.status(400).json({
        error: "Bad Request",
      });
      return;
    }

    const blog = await BlogModel.create({
      title,
      content,
      tags,
      authorUsername: adminuser.username,
      published_at: new Date(),
      comments: [],
    });

    res.status(200).json({
      blog,
    });
    return;
  } else if (req.method === "DELETE") {
    const { id } = req.query;

    if (!id) {
      res.status(400).json({
```

```
        error: "Bad Request",
      });
      return;
    }

    const blog = await BlogModel.findById(id);

    if (!blog) {
      res.status(404).json({
        error: "Not Found",
      });
      return;
    }

    await blog.remove();

    res.status(200).json({
      success: true,
    });
    return;
  } else if (req.method === "PUT") {
    const { id, update } = req.body;

    if (!id || !update) {
      res.status(400).json({
        error: "Bad Request",
      });
      return;
    }

    const blog = await BlogModel.findById(id);
    if (!blog) {
      res.status(404).json({
        error: "Not Found",
      });
      return;
    }

    let failedKeys = [];

    for (const key in update) {
      if (["tags", "title", "content"].includes(key.toLowerCase())) {
        blog[key] = update[key];
      } else {
        failedKeys.push(key);
      }
    }

    await blog.save();

    res.status(200).json({
      success: true,
      failedKeys,
    });
  }
} catch (e) {
  res.status(500).json({
    error: "Internal Server Error",
  });
}
```

```
}
```

## apps/web/src/pages/api/admin/certificates/delete.ts

```
/**
 * apps/web/src/pages/api/admin/certificates/list.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 03.12.2022
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import { verify } from "jsonwebtoken";
import mongoConnect from "@codeup/util/src/mongo";
import UserModel from "@codeup/util/src/models/UserModel";
import ObtainedCertificateModel from "@codeup/util/src/models/ObtainedCertificateModel";
import CourseModel from "@codeup/util/src/models/CourseModel";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "DELETE") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  if (!req.headers["x-jwt"]) {
    res.status(401).json({
      error: "Unauthorized",
    });
    return;
  }

  if (!req.query.id) {
    res.status(400).json({
      error: "Missing id",
    });
    return;
  }

  try {
    const jwt = req.headers["x-jwt"];
    if (!verify(jwt as string, process.env.JWT_SECRET as string)) {
      res.status(401).json({
        error: "Unauthorized",
      });
      return;
    }

    const decoded = verify(
      jwt as string,
      process.env.JWT_SECRET as string
    ) as any;

    const adminuser = await UserModel.findOne({ username: decoded.username });
```

```
    if (adminuser.role !== "admin") {
      res.status(401).json({
        error: "Unauthorized",
      });
      return;
    }

    await mongoConnect();

    const cert = await ObtainedCertificateModel.findById(req.query.id);

    if (!cert) {
      res.status(404).json({
        error: "Certificate not found",
      });
      return;
    }

    await cert.remove();

    res.status(200).json({
      success: true,
    });
  } catch (e) {
    res.status(500).json({
      error: "Internal Server Error",
    });
  }
}
```

## apps/web/src/pages/api/admin/certificates/list.ts

```
/**
 * apps/web/src/pages/api/admin/certificates/list.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 03.12.2022
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import { verify } from "jsonwebtoken";
import mongoConnect from "@codeup/util/src/mongo";
import UserModel from "@codeup/util/src/models/UserModel";
import ObtainedCertificateModel from "@codeup/util/src/models/ObtainedCertificateModel";
import CourseModel from "@codeup/util/src/models/CourseModel";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "GET") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }
```

```
  if (!req.headers["x-jwt"]) {
    res.status(401).json({
      error: "Unauthorized",
    });
    return;
  }

  try {
    const jwt = req.headers["x-jwt"];
    if (!verify(jwt as string, process.env.JWT_SECRET as string)) {
      res.status(401).json({
        error: "Unauthorized",
      });
      return;
    }

    const decoded = verify(
      jwt as string,
      process.env.JWT_SECRET as string
    ) as any;

    const adminuser = await UserModel.findOne({ username: decoded.username });

    if (adminuser.role !== "admin") {
      res.status(401).json({
        error: "Unauthorized",
      });
      return;
    }

    await mongoConnect();

    const certs = await ObtainedCertificateModel.find({});

    const c = certs.map((c) => c.toObject());

    for (const cert of c) {
      const course = await CourseModel.findOne({ _id: cert.course });
      cert.course = course.name;
    }

    res.status(200).json({
      certificates: c,
    });
  } catch (e) {
    res.status(500).json({
      error: "Internal Server Error",
    });
  }
}
```

## apps/web/src/pages/api/admin/challenges/index.ts

```
/**
 * apps/web/src/pages/api/admin/challenges/index.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2024 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 18.05.2024
 *
```

```
 */

import { NextApiRequest, NextApiResponse } from "next";
import { isLoggedIn } from "../../../../global/server/isLoggedIn";
import CodeChallengeModel from "@codeup/util/dist/models/CodeChallengeModel";

export default async function (req: NextApiRequest, res: NextApiResponse) {
  try {
    const { auth, user } = await isLoggedIn(req, res);

    if (!auth) return;

    if (user.role !== "admin") {
      res.status(403).json({
        status: 403,
        error: "You are not allowed to access this route.",
      });
      return;
    }

    if (req.method === "POST") {
      const { title, task, difficulty, expectedOutput } = req.body;

      if (!title || !task || !difficulty || !expectedOutput) {
        res.status(400).json({
          status: 400,
          error: "Please provide all required fields.",
        });
        return;
      }

      const challenge = await CodeChallengeModel.create({
        title,
        task,
        difficulty,
        expectedOutput,
      });

      res.status(200).json({
        status: 200,
        data: challenge,
      });
    } else if (req.method === "DELETE") {
      if (!req.query.id) {
        res.status(400).json({
          status: 400,
          error: "Please provide a challenge id.",
        });
        return;
      }

      const challenge = await CodeChallengeModel.findById(req.query.id);

      if (!challenge) {
        res.status(404).json({
          status: 404,
          error: "Challenge not found.",
        });
        return;
      }
```

```
      await challenge.deleteOne();

      res.status(200).json({
        status: 200,
      });
    } else if (req.method === "PUT") {
      const { title, task, difficulty, expectedOutput, id } = req.body;

      if (!title || !task || !difficulty || !expectedOutput || !id) {
        res.status(400).json({
          status: 400,
          error: "Please provide all required fields.",
        });
        return;
      }

      const challenge = await CodeChallengeModel.findById(id);

      if (!challenge) {
        res.status(404).json({
          status: 404,
          error: "Challenge not found.",
        });
        return;
      }

      challenge.title = title;
      challenge.task = task;
      challenge.difficulty = difficulty;
      challenge.expectedOutput = expectedOutput;

      await challenge.save();

      res.status(200).json({
        status: 200,
        data: challenge,
      });
    } else {
      res.status(405).json({
        status: 405,
        error: "Method not allowed",
      });
    }
  } catch (e: any) {
    res.status(500).json({
      status: 500,
      error: e.message,
    });
  }
}
```

## apps/web/src/pages/api/admin/contact/messages.ts

```
/**
 * apps/web/src/pages/api/admin/contact/messages.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 26.11.2022
```

```
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import ContactModel from "@codeup/util/src/models/ContactModel";
import UserModel from "@codeup/util/src/models/UserModel";
import mongoConnect from "@codeup/util/src/mongo";
import { verify } from "jsonwebtoken";
import CourseModel from "@codeup/util/src/models/CourseModel";

export default async function (req: NextApiRequest, res: NextApiResponse) {
  if (!["GET", "DELETE"].includes(req.method as string)) {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  try {
    const jwt = req.headers["x-jwt"];
    if (!verify(jwt as string, process.env.JWT_SECRET as string)) {
      res.status(401).json({
        error: "Unauthorized",
      });
      return;
    }

    const decoded = verify(
      jwt as string,
      process.env.JWT_SECRET as string
    ) as any;

    const adminuser = await UserModel.findOne({ username: decoded.username });

    if (adminuser.role !== "admin") {
      res.status(401).json({
        error: "Unauthorized",
      });
      return;
    }

    await mongoConnect();

    if (req.method === "GET") {
      const messages = await ContactModel.find({});

      if (req.query.count) {
        res.status(200).json({
          count: messages.length,
        });
        return;
      }

      res.status(200).json({ messages });
    } else if (req.method === "DELETE") {
      const id = req.query.id as string;

      if (!id) {
        res.status(400).json({
          error: "Bad Request",
```

```
        });
        return;
      }

      const message = await ContactModel.findById(id);

      if (!message) {
        res.status(404).json({
          error: "Not Found",
        });
        return;
      }

      await message.remove();

      res.status(200).json({
        success: true,
      });
    }
  } catch (e) {
    // @ts-ignore
    if (e.message) {
      res.status(500).json({
        // @ts-ignore
        error: "Internal Server Error:" + e.message,
        success: false,
      });
    } else {
      res.status(500).json({
        error: "Internal Server Error: Unknown Error",
        success: false,
      });
    }
  }
}
```

## apps/web/src/pages/api/admin/contact/user.tsx

```
/**
 * apps/web/src/pages/api/admin/contact/user.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 26.11.2022
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import ContactModel from "@codeup/util/src/models/ContactModel";
import UserModel from "@codeup/util/src/models/UserModel";
import mongoConnect from "@codeup/util/src/mongo";
import { verify } from "jsonwebtoken";
import CourseModel from "@codeup/util/src/models/CourseModel";
import sendMail, { sendMailV2 } from "@codeup/util/src/sendMail";
import { render } from "@react-email/render";
import PrivateMessageEmail from "@codeup/components/emails/PrivateMessage";

export default async function (req: NextApiRequest, res: NextApiResponse) {
  if (req.method !== "POST") {
    res.status(405).json({
```

```
    error: "Method not allowed",
  });
  return;
}

if (!req.body.username || !req.body.message) {
  res.status(400).json({
    error: "Bad Request",
  });
  return;
}

try {
  const jwt = req.headers["x-jwt"];
  if (!verify(jwt as string, process.env.JWT_SECRET as string)) {
    res.status(401).json({
      error: "Unauthorized",
    });
    return;
  }

  const decoded = verify(
    jwt as string,
    process.env.JWT_SECRET as string
  ) as any;

  const adminuser = await UserModel.findOne({ username: decoded.username });

  if (adminuser.role !== "admin") {
    res.status(401).json({
      error: "Unauthorized",
    });
    return;
  }

  await mongoConnect();

  const user = await UserModel.findOne({ username: req.body.username });

  if (!user) {
    res.status(404).json({
      error: "User not found",
    });
    return;
  }

  await sendMailV2({
    to: user.email,
    subject: "Nachricht von CodeUp",
    text: req.body.message,
    html: render(<PrivateMessageEmail firstName={user.firstName} content={req.body.message} />)
  });


  res.status(200).json({
    success: true,
  });
} catch (e) {
  // @ts-ignore
  if (e.message) {
```

```
      res.status(500).json({
        // @ts-ignore
        error: "Internal Server Error:" + e.message,
        success: false,
      });
    } else {
      res.status(500).json({
        error: "Internal Server Error: Unknown Error",
        success: false,
      });
    }
  }
}
```

## apps/web/src/pages/api/admin/contents/index.ts

```
/**
 * apps/web/src/pages/api/admin/contents/index.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 06.11.2022
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import ContentModel from "@codeup/util/src/models/ContentModel";
import UserModel from "@codeup/util/src/models/UserModel";
import mongoConnect from "@codeup/util/src/mongo";
import { verify } from "jsonwebtoken";
import CourseModel from "@codeup/util/src/models/CourseModel";

export default async function (req: NextApiRequest, res: NextApiResponse) {
  try {
    const jwt = req.headers["x-jwt"];
    if (!verify(jwt as string, process.env.JWT_SECRET as string)) {
      res.status(401).json({
        error: "Unauthorized",
      });
      return;
    }

    const decoded = verify(
      jwt as string,
      process.env.JWT_SECRET as string
    ) as any;

    const adminuser = await UserModel.findOne({ username: decoded.username });

    if (adminuser.role !== "admin") {
      res.status(401).json({
        error: "Unauthorized",
      });
      return;
    }

    await mongoConnect();

    if (req.method === "GET") {
      if (req.query.cid) {
```

```
    const content = await ContentModel.findById(req.query.id as string);

    if (!content) {
      res.status(404).json({
        error: "Not Found",
      });
      return;
    }

    res.status(200).json({
      content,
    });
    return;
  }

  if (req.query.search) {
    const contents = await ContentModel.find({
      $text: { $search: req.query.search as string },
    });

    if (!contents) {
      res.status(404).json({
        error: "Not Found",
      });
      return;
    }

    res.status(200).json({
      contents,
    });
    return;
  }

  if (req.query.type) {
    const contents = await ContentModel.find({
      type: req.query.type as string,
    });

    if (!contents) {
      res.status(404).json({
        error: "Not Found",
      });
      return;
    }

    res.status(200).json({
      contents,
    });
    return;
  }

  if (req.query.course) {
    const contents = await ContentModel.find({
      course: req.query.course as string,
    });

    if (!contents) {
      res.status(404).json({
        error: "Not Found",
      });
```

```
      return;
    }

    res.status(200).json({
      contents,
    });
    return;
  }

  const contents = await ContentModel.find({});
  res.status(200).json({
    contents,
    success: true,
  });
  return;
}

if (req.method === "POST") {
  const { displayName, course, url, type, textContent, quiz, videoUrl } =
    req.body;

  if (!displayName || !course || !url || !type) {
    res.status(400).json({
      error: "Bad Request",
      success: false,
    });
    return;
  }

  if (!textContent && !videoUrl) {
    res.status(400).json({
      error: "Bad Request",
      success: false,
    });
    return;
  }

  const courseModel = await CourseModel.findById(course);

  if (!courseModel) {
    res.status(404).json({
      error: "Not Found",
      success: false,
    });
    return;
  }

  const content = await ContentModel.create({
    displayName,
    course,
    url,
    type,
    textContent: textContent || "",
    quiz,
    creator: adminuser.username,
    videoUrl: videoUrl ? videoUrl : undefined,
  });

  courseModel.contentPositions.push(content._id);
```

```
    await courseModel.save();

    res.status(200).json({
      content,
      success: true,
    });
  }

  if (req.method === "DELETE") {
    const { id } = req.body;

    if (!id) {
      res.status(400).json({
        error: "Bad Request",
        success: false,
      });
      return;
    }

    const content = await ContentModel.findById(id);

    if (!content) {
      res.status(404).json({
        error: "Not Found",
        success: false,
      });
      return;
    }

    const course = await CourseModel.findById(content.course);

    if (!course) {
      res.status(404).json({
        error: "Not Found",
        success: false,
      });
      return;
    }

    course.contentPositions = course.contentPositions.filter(
      (c: string) => c !== req.body.id
    );

    await course.save();

    await content.remove();

    res.status(200).json({
      success: true,
    });
  }
} catch (e) {
  // @ts-ignore
  if (e.message) {
    res.status(500).json({
      // @ts-ignore
      error: "Internal Server Error:" + e.message,
      success: false,
    });
  } else {
```

```
      res.status(500).json({
        error: "Internal Server Error: Unknown Error",
        success: false,
      });
    }
  }
}
```

## apps/web/src/pages/api/admin/course/create.ts

```
/**
 * apps/web/src/pages/api/admin/course/create.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 30.10.2022
 *
 */
import mongoConnect from "@codeup/util/src/mongo";
import { NextApiRequest, NextApiResponse } from "next";
import { verify } from "jsonwebtoken";
import UserModel from "@codeup/util/src/models/UserModel";
import CourseModel from "@codeup/util/src/models/CourseModel";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "POST") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  try {
    await mongoConnect();

    if (!req.headers["x-jwt"]) {
      res.status(401).json({
        error: "Unauthorized",
      });
      return;
    }

    const { name, friendlyName, description, splashImage } = req.body;

    if (!name || !friendlyName || !description || !splashImage) {
      res.status(400).json({
        error: "Bad Request",
      });
      return;
    }

    if (
      !verify(req.headers["x-jwt"] as string, process.env.JWT_SECRET as string)
    ) {
      res.status(401).json({
        error: "Unauthorized",
      });
```

```
      return;
    }

    const decoded = verify(
      req.headers["x-jwt"] as string,
      process.env.JWT_SECRET as string
    ) as any;

    const adminuser = await UserModel.findOne({ username: decoded.username });

    if (adminuser.role !== "admin") {
      res.status(401).json({
        error: "Unauthorized",
      });
      return;
    }

    const existingCourse = await CourseModel.findOne({
      friendlyName: friendlyName,
    });

    if (existingCourse) {
      res.status(400).json({
        error: "Course already exists",
      });
      return;
    }

    const course = await CourseModel.create({
      name: name,
      friendlyName: friendlyName,
      description: description,
      splashImage: splashImage,
      available: false,
      creator: adminuser.username,
    });

    res.status(200).json({
      course: course,
    });
  } catch (e) {
    res.status(500).json({
      error: "Internal Server Error",
      // @ts-ignore
      message: e.message,
    });
  }
}
```

## apps/web/src/pages/api/admin/course/delete.ts

```
/**
 * apps/web/src/pages/api/admin/course/delete.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 30.10.2022
 *
 */
import mongoConnect from "@codeup/util/src/mongo";
```

```
import { NextApiRequest, NextApiResponse } from "next";
import { verify } from "jsonwebtoken";
import UserModel from "@codeup/util/src/models/UserModel";
import CourseModel from "@codeup/util/src/models/CourseModel";
import ContentModel from "@codeup/util/src/models/ContentModel";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "POST") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  try {
    await mongoConnect();

    if (!req.headers["x-jwt"]) {
      res.status(401).json({
        error: "Unauthorized",
      });
      return;
    }

    const { friendlyName } = req.body;

    if (!friendlyName) {
      res.status(400).json({
        error: "Bad request",
      });
      return;
    }

    if (
      !verify(req.headers["x-jwt"] as string, process.env.JWT_SECRET as string)
    ) {
      res.status(401).json({
        error: "Unauthorized",
      });
      return;
    }

    const decoded = verify(
      req.headers["x-jwt"] as string,
      process.env.JWT_SECRET as string
    ) as any;

    const adminuser = await UserModel.findOne({ username: decoded.username });

    if (adminuser.role !== "admin") {
      res.status(401).json({
        error: "Unauthorized",
      });
      return;
    }

    const course = await CourseModel.findOne({ friendlyName: friendlyName });
```

```
    if (!course) {
      res.status(404).json({
        error: "Course not found",
      });
      return;
    }

    const contents = await ContentModel.find({ course: course._id });

    for (const content of contents) {
      content.remove();
    }

    await course.remove();

    res.status(200).json({
      success: true,
    });
  } catch (e) {
    res.status(500).json({
      error: "Internal Server Error",
    });
  }
}
```

## apps/web/src/pages/api/admin/course/edit.ts

```
/**
 * apps/web/src/pages/api/admin/course/edit.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 30.10.2022
 *
 */
import mongoConnect from "@codeup/util/src/mongo";
import { NextApiRequest, NextApiResponse } from "next";
import { verify } from "jsonwebtoken";
import UserModel from "@codeup/util/src/models/UserModel";
import CourseModel from "@codeup/util/src/models/CourseModel";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "POST") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  try {
    await mongoConnect();

    if (!req.headers["x-jwt"]) {
      res.status(401).json({
        error: "Unauthorized",
      });
```

```
    return;
  }

  const { update, id } = req.body;

  if (!update || !id) {
    res.status(400).json({
      error: "Bad Request",
    });
    return;
  }

  if (
    !verify(req.headers["x-jwt"] as string, process.env.JWT_SECRET as string)
  ) {
    res.status(401).json({
      error: "Unauthorized",
    });
    return;
  }

  const decoded = verify(
    req.headers["x-jwt"] as string,
    process.env.JWT_SECRET as string
  ) as any;

  const adminuser = await UserModel.findOne({ username: decoded.username });

  if (adminuser.role !== "admin") {
    res.status(401).json({
      error: "Unauthorized",
    });
    return;
  }

  const existingCourse = await CourseModel.findById(req.body.id);

  if (!existingCourse) {
    res.status(400).json({
      error: "Course does not exists",
    });
    return;
  }

  for (const key in req.body.update) {
    existingCourse[key] = req.body.update[key];
  }

  await existingCourse.save();

  res.status(200).json({
    existingCourse,
  });
} catch (e) {
  res.status(500).json({
    error: "Internal Server Error",
    // @ts-ignore
    message: e.message,
  });
}
```

```
}
```

## apps/web/src/pages/api/admin/course/list.ts

```
/**
 * apps/web/src/pages/api/admin/course/list.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 30.10.2022
 *
 */
import { NextApiRequest, NextApiResponse } from "next";
import { verify } from "jsonwebtoken";
import mongoConnect from "@codeup/util/src/mongo";
import UserModel from "@codeup/util/src/models/UserModel";
import CourseModel from "@codeup/util/src/models/CourseModel";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "GET") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  if (!req.headers["x-jwt"]) {
    res.status(401).json({
      error: "Unauthorized",
    });
    return;
  }

  try {
    const jwt = req.headers["x-jwt"];
    if (!verify(jwt as string, process.env.JWT_SECRET as string)) {
      res.status(401).json({
        error: "Unauthorized",
      });
      return;
    }

    const decoded = verify(
      jwt as string,
      process.env.JWT_SECRET as string
    ) as any;

    const adminuser = await UserModel.findOne({ username: decoded.username });

    if (adminuser.role !== "admin") {
      res.status(401).json({
        error: "Unauthorized",
      });
      return;
    }

    await mongoConnect();
```

```
    const courses = await CourseModel.find();

    if (req.query.list) {
      res.status(200).json({
        courses: courses.map((c) => {
          return {
            name: c.name,
            url: c.friendlyName,
            id: c._id,
          };
        }),
      });
      return;
    }

    res.status(200).json({
      courses,
    });
  } catch (e) {
    res.status(500).json({
      error: "Internal Server Error",
    });
  }
}
```

## apps/web/src/pages/api/admin/discovery/delete.ts

```
/**
 * apps/web/src/pages/api/admin/discovery/delete.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 13.04.2023
 *
 */
import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/src/mongo";
import jwt, { JwtPayload, VerifyErrors } from "jsonwebtoken";
import UserModel from "@codeup/util/src/models/UserModel";
import EditorProjectModel from "@codeup/util/src/models/EditorProjectModel";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "DELETE") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  if (!req.headers["x-jwt"]) {
    res.status(400).json({
      error: "Missing Auth-Header (x-jwt)",
    });
    return;
  }
```

```
  if (!req.query || !req.query.id) {
    res.status(400).json({
      error: "Missing parameters",
    });
    return;
  }

  jwt.verify(
    req.headers["x-jwt"] as string,
    process.env.JWT_SECRET as string,
    async (err: VerifyErrors | null, decoded: JwtPayload | string | any) => {
      if (err) {
        res.status(400).json({
          error: "Invalid token",
        });
        return;
      }

      await mongoConnect();

      const user = await UserModel.findById((decoded as any).id);

      if (!user) {
        res.status(400).json({
          error: "Invalid token",
        });
        return;
      }

      if (user.role !== "admin") {
        res.status(403).json({
          error: "You are not allowed to do this",
        });
        return;
      }

      const project = await EditorProjectModel.findById(req.query.id);

      if (!project) {
        res.status(400).json({
          error: "Project not found",
        });
        return;
      }

      project.discovery = false;
      project.discoveryDescription = "";

      await project.save();

      res.status(200).json({
        success: true,
      });
    }
  );
}
```

## apps/web/src/pages/api/admin/editor/index.ts

```
/**
 * apps/web/src/pages/api/admin/editor/index.ts
```

```
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 20.04.2023
 *
 */
import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/src/mongo";
import jwt, { JwtPayload, VerifyErrors } from "jsonwebtoken";
import UserModel from "@codeup/util/src/models/UserModel";
import EditorProjectModel from "@codeup/util/dist/models/EditorProjectModel";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "GET") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  if (!req.headers["x-jwt"]) {
    res.status(400).json({
      error: "Missing Auth-Header (x-jwt)",
    });
    return;
  }

  jwt.verify(
    req.headers["x-jwt"] as string,
    process.env.JWT_SECRET as string,
    async (err: VerifyErrors | null, decoded: JwtPayload | string | any) => {
      if (err) {
        res.status(400).json({
          error: "Invalid token",
        });
        return;
      }

      await mongoConnect();

      const user = await UserModel.findById((decoded as any).id);

      if (!user) {
        res.status(400).json({
          error: "Invalid token",
        });
        return;
      }

      if (user.role !== "admin") {
        res.status(403).json({
          error: "You are not allowed to do this",
        });
        return;
      }
```

```
    const projects = await EditorProjectModel.find({});

    for (const p of projects) {
      await p.populate("owner");
      if (p.owner === null) {
        p.owner = {
          username: "Deleted User",
        };
      }
    }

    res.status(200).json({
      success: true,
      projects,
    });
  }
 );
}
```

## apps/web/src/pages/api/admin/errors.ts

```
/**
 * apps/web/src/pages/api/admin/errors.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 26.04.2023
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import jwt from "jsonwebtoken";
import mongoConnect from "@codeup/util/dist/mongo";
import UserModel from "@codeup/util/dist/models/UserModel";
import ErrorModel from "@codeup/util/dist/models/ErrorModel";

export default async function (req: NextApiRequest, res: NextApiResponse) {
  if (req.method !== "GET") {
    res.status(405).send({
      error: "Method not allowed",
    });
    return;
  }

  if (!req.headers["x-jwt"]) {
    res.status(401).send({
      error: "Unauthorized",
    });
    return;
  }

  const token = req.headers["x-jwt"] as string;

  try {
    await mongoConnect();

    const decoded = jwt.verify(token, process.env.JWT_SECRET as string) as any;

    const user = await UserModel.findById(decoded.id);
```

```
    if (!user || user.role !== "admin") {
      res.status(401).send({
        error: "Unauthorized",
      });
      return;
    }

    res.status(200).send(await ErrorModel.find({}).sort({ createdAt: -1 }));
  } catch (e: any) {
    res.status(500).send({
      error: e.message,
    });
  }
}
```

## apps/web/src/pages/api/admin/flows/index.ts

```
/**
 * apps/web/src/pages/api/admin/flows/index.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 13.04.2023
 *
 */
import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/src/mongo";
import jwt, { JwtPayload, VerifyErrors } from "jsonwebtoken";
import UserModel from "@codeup/util/src/models/UserModel";
import ToDoProjectModel from "@codeup/util/src/models/ToDoProjectModel";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "GET") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  if (!req.headers["x-jwt"]) {
    res.status(400).json({
      error: "Missing Auth-Header (x-jwt)",
    });
    return;
  }

  jwt.verify(
    req.headers["x-jwt"] as string,
    process.env.JWT_SECRET as string,
    async (err: VerifyErrors | null, decoded: JwtPayload | string | any) => {
      if (err) {
        res.status(400).json({
          error: "Invalid token",
        });
        return;
      }
```

```
      await mongoConnect();

      const user = await UserModel.findById((decoded as any).id);

      if (!user) {
        res.status(400).json({
          error: "Invalid token",
        });
        return;
      }

      if (user.role !== "admin") {
        res.status(403).json({
          error: "You are not allowed to do this",
        });
        return;
      }

      const flows = await ToDoProjectModel.find({});

      for (const f of flows) {
        await f.populate("owner");
        if (f.owner === null) {
          f.owner = {
            username: "Deleted User",
          };
        }
      }

      res.status(200).json({
        success: true,
        flows: flows,
      });
    }
  );
}
```

## apps/web/src/pages/api/admin/forum_profiles/delete.ts

```
/**
 * apps/web/src/pages/api/admin/forum_profiles/delete.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 08.10.22
 *
 */
import mongoConnect from "@codeup/util/src/mongo";
import { NextApiRequest, NextApiResponse } from "next";
import { verify } from "jsonwebtoken";
import UserModel from "@codeup/util/src/models/UserModel";
import ForumProfileModel from "@codeup/util/src/models/ForumProfileModel";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "POST") {
    res.status(405).json({
      error: "Method not allowed",
```

```
  });
  return;
}

try {
  await mongoConnect();

  if (!req.headers["x-jwt"]) {
    res.status(401).json({
      error: "Unauthorized",
    });
    return;
  }

  const { username } = req.body;

  if (!username) {
    res.status(400).json({
      error: "Bad request",
    });
    return;
  }

  if (
    !verify(req.headers["x-jwt"] as string, process.env.JWT_SECRET as string)
  ) {
    res.status(401).json({
      error: "Unauthorized",
    });
    return;
  }

  const decoded = verify(
    req.headers["x-jwt"] as string,
    process.env.JWT_SECRET as string
  ) as any;

  const adminuser = await UserModel.findOne({ username: decoded.username });

  if (adminuser.role !== "admin") {
    res.status(401).json({
      error: "Unauthorized",
    });
    return;
  }

  const profile = await ForumProfileModel.findOne({ username });

  if (!profile) {
    res.status(404).json({
      error: "Profile not found",
    });
    return;
  }

  await profile.remove();

  res.status(200).json({
    success: true,
  });
```

```
  } catch (e) {
    res.status(500).json({
      error: "Internal Server Error",
    });
  }
}
```

## apps/web/src/pages/api/admin/forum_profiles/list.ts

```
/**
 * apps/web/src/pages/api/admin/forum_profiles/list.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 08.10.22
 *
 */
import { NextApiRequest, NextApiResponse } from "next";
import { verify } from "jsonwebtoken";
import mongoConnect from "@codeup/util/src/mongo";
import UserModel from "@codeup/util/src/models/UserModel";
import ForumProfileModel from "@codeup/util/src/models/ForumProfileModel";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "GET") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  if (!req.headers["x-jwt"]) {
    res.status(401).json({
      error: "Unauthorized",
    });
    return;
  }

  try {
    const jwt = req.headers["x-jwt"];
    if (!verify(jwt as string, process.env.JWT_SECRET as string)) {
      res.status(401).json({
        error: "Unauthorized",
      });
      return;
    }

    const decoded = verify(
      jwt as string,
      process.env.JWT_SECRET as string
    ) as any;

    const adminuser = await UserModel.findOne({ username: decoded.username });

    if (adminuser.role !== "admin") {
      res.status(401).json({
        error: "Unauthorized",
```

```
      });
      return;
    }

    await mongoConnect();

    const profiles = await ForumProfileModel.find();

    res.status(200).json({
      profiles,
    });
  } catch (e) {
    res.status(500).json({
      error: "Internal Server Error",
    });
  }
}
```

## apps/web/src/pages/api/admin/forum_profiles/update.ts

```
/**
 * apps/web/src/pages/api/admin/forum_profiles/update.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 10.10.22
 *
 */
import mongoConnect from "@codeup/util/src/mongo";
import { NextApiRequest, NextApiResponse } from "next";
import { verify } from "jsonwebtoken";
import UserModel from "@codeup/util/src/models/UserModel";
import ForumProfileModel from "@codeup/util/src/models/ForumProfileModel";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "POST") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  try {
    await mongoConnect();

    if (!req.headers["x-jwt"]) {
      res.status(401).json({
        error: "Unauthorized",
      });
      return;
    }

    const { username, update } = req.body;

    if (!username || !update) {
      res.status(400).json({
        error: "Bad request",
```

```
      });
      return;
    }

    if (
      !verify(req.headers["x-jwt"] as string, process.env.JWT_SECRET as string)
    ) {
      res.status(401).json({
        error: "Unauthorized",
      });
      return;
    }

    const decoded = verify(
      req.headers["x-jwt"] as string,
      process.env.JWT_SECRET as string
    ) as any;

    const adminuser = await UserModel.findOne({ username: decoded.username });

    if (adminuser.role !== "admin") {
      res.status(401).json({
        error: "Unauthorized",
      });
      return;
    }

    const profile = await ForumProfileModel.findOne({ username });

    if (!profile) {
      res.status(404).json({
        error: "User not found",
      });
      return;
    }

    const allowedKeys = [
      "displayName",
      "verified",
      "bio",
      "website",
      "location",
    ];

    for (const key in update) {
      if (allowedKeys.includes(key)) {
        profile[key] = update[key];
      }
    }

    await profile.save();

    res.status(200).json({
      success: true,
      profile: profile,
    });
  } catch (e) {
    res.status(500).json({
      error: "Internal Server Error",
      // @ts-ignore
```

```
      message: e.message,
    });
  }
}
```

## apps/web/src/pages/api/admin/ide/index.ts

```
/**
 * apps/web/src/pages/api/admin/ide/index.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 19.04.2023
 *
 */
import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/src/mongo";
import jwt, { JwtPayload, VerifyErrors } from "jsonwebtoken";
import UserModel from "@codeup/util/src/models/UserModel";
import CodeUpKidsProjectModel from "@codeup/util/dist/models/CodeUpKidsProjectModel";
import IDEProjectModel from "@codeup/util/dist/models/IDEProjectModel";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "GET") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  if (!req.headers["x-jwt"]) {
    res.status(400).json({
      error: "Missing Auth-Header (x-jwt)",
    });
    return;
  }

  jwt.verify(
    req.headers["x-jwt"] as string,
    process.env.JWT_SECRET as string,
    async (err: VerifyErrors | null, decoded: JwtPayload | string | any) => {
      if (err) {
        res.status(400).json({
          error: "Invalid token",
        });
        return;
      }

      await mongoConnect();

      const user = await UserModel.findById((decoded as any).id);

      if (!user) {
        res.status(400).json({
          error: "Invalid token",
        });
        return;
```

```
    }

    if (user.role !== "admin") {
      res.status(403).json({
        error: "You are not allowed to do this",
      });
      return;
    }

    const projects = await IDEProjectModel.find({});

    for (const p of projects) {
      await p.populate("owner");
      if (p.owner === null) {
        p.owner = {
          username: "Deleted User",
        };
      }
    }

    res.status(200).json({
      success: true,
      projects,
    });
  }
 );
}
```

## apps/web/src/pages/api/admin/kids/index.ts

```
/**
 * apps/web/src/pages/api/admin/kids/index.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 13.04.2023
 *
 */
import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/src/mongo";
import jwt, { JwtPayload, VerifyErrors } from "jsonwebtoken";
import UserModel from "@codeup/util/src/models/UserModel";
import CodeUpKidsProjectModel from "@codeup/util/dist/models/CodeUpKidsProjectModel";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "GET") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  if (!req.headers["x-jwt"]) {
    res.status(400).json({
      error: "Missing Auth-Header (x-jwt)",
    });
    return;
```

```
  }

  jwt.verify(
    req.headers["x-jwt"] as string,
    process.env.JWT_SECRET as string,
    async (err: VerifyErrors | null, decoded: JwtPayload | string | any) => {
      if (err) {
        res.status(400).json({
          error: "Invalid token",
        });
        return;
      }

      await mongoConnect();

      const user = await UserModel.findById((decoded as any).id);

      if (!user) {
        res.status(400).json({
          error: "Invalid token",
        });
        return;
      }

      if (user.role !== "admin") {
        res.status(403).json({
          error: "You are not allowed to do this",
        });
        return;
      }

      const projects = await CodeUpKidsProjectModel.find({});

      for (const p of projects) {
        await p.populate("owner");
        if (p.owner === null) {
          p.owner = {
            username: "Deleted User",
          };
        }
      }

      res.status(200).json({
        success: true,
        projects,
      });
    }
  );
}
```

## apps/web/src/pages/api/admin/newsletter/index.tsx

```
/**
 * apps/web/src/pages/api/admin/newsletter/index.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 13.04.2023
 *
 */
```

```
import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/src/mongo";
import jwt, { JwtPayload, VerifyErrors } from "jsonwebtoken";
import UserModel from "@codeup/util/src/models/UserModel";
import NewsletterModel from "@codeup/util/src/models/NewsletterModel";
import sendMail from "@codeup/util/dist/sendMail";
import { sendMailV2 } from "@codeup/util/src/sendMail";
import { render } from "@react-email/render";
import NewsletterEmail from "@codeup/components/emails/Newsletter";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "POST") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  if (!req.headers["x-jwt"]) {
    res.status(400).json({
      error: "Missing Auth-Header (x-jwt)",
    });
    return;
  }

  if (!req.body || !req.body.title || !req.body.content) {
    res.status(400).json({
      error: "Missing parameters",
    });
    return;
  }

  jwt.verify(
    req.headers["x-jwt"] as string,
    process.env.JWT_SECRET as string,
    async (err: VerifyErrors | null, decoded: JwtPayload | string | any) => {
      if (err) {
        res.status(400).json({
          error: "Invalid token",
        });
        return;
      }

      await mongoConnect();

      const user = await UserModel.findById((decoded as any).id);

      if (!user) {
        res.status(400).json({
          error: "Invalid token",
        });
        return;
      }

      if (user.role !== "admin") {
        res.status(403).json({
          error: "You are not allowed to do this",
```

```
      });
      return;
    }

    const newsletterMembers = await NewsletterModel.find({});

    for (const member of newsletterMembers) {
      const user = await UserModel.findById(member.user);

      if (!user) {
        continue;
      }

      if(user.email.endsWith("users.codeup.space")) continue;

      await sendMailV2({
        to: user.email,
        subject: req.body.title,
        text: req.body.content,
        html: render(<NewsletterEmail firstName={user.firstName} content={req.body.content} />)
      })
    }

    res.status(200).json({
      success: true,
    });
  }
 );
}
```

## apps/web/src/pages/api/admin/newsletter/members.ts

```
/**
 * apps/web/src/pages/api/admin/newsletter/members.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 13.04.2023
 *
 */
import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/src/mongo";
import jwt, { JwtPayload, VerifyErrors } from "jsonwebtoken";
import UserModel from "@codeup/util/src/models/UserModel";
import NewsletterModel from "@codeup/util/src/models/NewsletterModel";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "GET") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  if (!req.headers["x-jwt"]) {
    res.status(400).json({
      error: "Missing Auth-Header (x-jwt)",
```

```
    });
    return;
  }

  jwt.verify(
    req.headers["x-jwt"] as string,
    process.env.JWT_SECRET as string,
    async (err: VerifyErrors | null, decoded: JwtPayload | string | any) => {
      if (err) {
        res.status(400).json({
          error: "Invalid token",
        });
        return;
      }

      await mongoConnect();

      const user = await UserModel.findById((decoded as any).id);

      if (!user) {
        res.status(400).json({
          error: "Invalid token",
        });
        return;
      }

      if (user.role !== "admin") {
        res.status(403).json({
          error: "You are not allowed to do this",
        });
        return;
      }

      if (req.query.count) {
        res.status(200).json({
          count: await NewsletterModel.countDocuments(),
        });
        return;
      }

      const newsletterMembers = await NewsletterModel.find({});

      res.status(200).json({
        members: newsletterMembers.map((m) => m.user),
      });
    }
  );
}
```

## apps/web/src/pages/api/admin/orgs/index.ts

```
/**
 * apps/web/src/pages/api/admin/orgs/index.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 13.04.2023
 *
 */
import { NextApiRequest, NextApiResponse } from "next";
```

```
import mongoConnect from "@codeup/util/src/mongo";
import jwt, { JwtPayload, VerifyErrors } from "jsonwebtoken";
import UserModel from "@codeup/util/src/models/UserModel";
import OrgModel from "@codeup/util/src/models/OrgModel";
import mongoose from "mongoose";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "GET") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  if (!req.headers["x-jwt"]) {
    res.status(400).json({
      error: "Missing Auth-Header (x-jwt)",
    });
    return;
  }

  jwt.verify(
    req.headers["x-jwt"] as string,
    process.env.JWT_SECRET as string,
    async (err: VerifyErrors | null, decoded: JwtPayload | string | any) => {
      if (err) {
        res.status(400).json({
          error: "Invalid token",
        });
        return;
      }

      await mongoConnect();

      const user = await UserModel.findById((decoded as any).id);

      if (!user) {
        res.status(400).json({
          error: "Invalid token",
        });
        return;
      }

      if (user.role !== "admin") {
        res.status(403).json({
          error: "You are not allowed to do this",
        });
        return;
      }

      const orgs = await OrgModel.find();

      for (const o of orgs) {
        await o.populate("owner");
        if (o.owner === null) {
          o.owner = {
            username: "Deleted User",
```

```
        };
      }
    }

    res.status(200).json({
      success: true,
      orgs,
    });
  }
 );
}
```

## apps/web/src/pages/api/admin/overview.ts

```
/**
 * apps/web/src/pages/api/admin/overview.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 01.04.2023
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import jwt from "jsonwebtoken";
import mongoConnect from "@codeup/util/dist/mongo";
import UserModel from "@codeup/util/dist/models/UserModel";
import OrgModel from "@codeup/util/dist/models/OrgModel";
import ErrorModel from "@codeup/util/dist/models/ErrorModel";
import EditorProjectModel from "@codeup/util/dist/models/EditorProjectModel";
import EditorProjectPageModel from "@codeup/util/dist/models/EditorProjectPageModel";
import ForumProfileModel from "@codeup/util/dist/models/ForumProfileModel";
import ForumPostModel from "@codeup/util/dist/models/ForumPostModel";
import TodoListModel from "@codeup/util/dist/models/TodoListModel";
import ToDoProjectModel from "@codeup/util/dist/models/ToDoProjectModel";
import NewsletterModel from "@codeup/util/dist/models/NewsletterModel";
import ObtainedCertificateModel from "@codeup/util/dist/models/ObtainedCertificateModel";
import CodeUpKidsProjectModel from "@codeup/util/dist/models/CodeUpKidsProjectModel";
import ContactModel from "@codeup/util/dist/models/ContactModel";
import EditorProjectFunctionModel from "@codeup/util/dist/models/EditorProjectFunctionModel";
import IDEProjectModel from "@codeup/util/dist/models/IDEProjectModel";
import RoadmapModel from "@codeup/util/dist/models/RoadmapModel";
import CodeSnippet from "@codeup/util/dist/models/CodeSnippet";
import CourseModel from "@codeup/util/dist/models/CourseModel";
import ContentModel from "@codeup/util/dist/models/ContentModel";
import ShortURLModel from "@codeup/util/dist/models/ShortURLModel";

export default async function (req: NextApiRequest, res: NextApiResponse) {
  if (req.method !== "GET") {
    res.status(405).send({
      error: "Method not allowed",
    });
    return;
  }

  if (!req.headers["x-jwt"]) {
    res.status(401).send({
      error: "Unauthorized",
    });
    return;
```

```
}

const token = req.headers["x-jwt"] as string;

try {
  await mongoConnect();

  const decoded = jwt.verify(token, process.env.JWT_SECRET as string) as any;

  const user = await UserModel.findById(decoded.id);

  if (!user || user.role !== "admin") {
    res.status(401).send({
      error: "Unauthorized",
    });
    return;
  }

  const stats = {
    users: {
      count: await UserModel.countDocuments(),
      admins: await UserModel.countDocuments({ role: "admin" }),
      editors: await UserModel.countDocuments({ role: "editor" }),
      newUsers: await UserModel.countDocuments({
        createdAt: {
          $gte: new Date(new Date().getTime() - 1000 * 60 * 60 * 24),
        },
      }),
    },
    orgs: {
      count: await OrgModel.countDocuments(),
      newOrgs: await OrgModel.countDocuments({
        createdAt: {
          $gte: new Date(new Date().getTime() - 1000 * 60 * 60 * 24),
        },
      }),
    },
    projects: {
      count: await EditorProjectModel.countDocuments(),
      newProjects: await EditorProjectModel.countDocuments({
        created_at: {
          $gte: new Date(new Date().getTime() - 1000 * 60 * 60 * 24),
        },
      }),
      projectPages: await EditorProjectPageModel.countDocuments(),
      publishedProjects: await EditorProjectModel.countDocuments({
        customDomain: {
          $not: /disabled:/,
        },
      }),
      projectFunctions: await EditorProjectFunctionModel.countDocuments(),
    },
    forum: {
      profiles: await ForumProfileModel.countDocuments(),
      newProfiles: await ForumProfileModel.countDocuments({
        createdAt: {
          $gte: new Date(new Date().getTime() - 1000 * 60 * 60 * 24),
        },
      }),
      posts: await ForumPostModel.countDocuments(),
```

```
      newPosts: await ForumPostModel.countDocuments({
        createdAt: {
          $gte: new Date(new Date().getTime() - 1000 * 60 * 60 * 24),
        },
      }),
    },
    todo: {
      count: await TodoListModel.countDocuments(),
      newTodos: await TodoListModel.countDocuments({
        createdAt: {
          $gte: new Date(new Date().getTime() - 1000 * 60 * 60 * 24),
        },
      }),
    },
    flowCharts: {
      count: await ToDoProjectModel.countDocuments(),
      newFlowCharts: await ToDoProjectModel.countDocuments({
        createdAt: {
          $gte: new Date(new Date().getTime() - 1000 * 60 * 60 * 24),
        },
      }),
    },
    newsletterMembers: await NewsletterModel.countDocuments(),
    // from here
    certificates: await ObtainedCertificateModel.countDocuments(),
    kids: await CodeUpKidsProjectModel.countDocuments(),
    contactMessages: await ContactModel.countDocuments(),
    ideProjects: await IDEProjectModel.countDocuments(),
    roadmaps: await RoadmapModel.countDocuments(),
    snippets: await CodeSnippet.countDocuments(),
    courses: await CourseModel.countDocuments(),
    lessons: await ContentModel.countDocuments(),
    shorts: await ShortURLModel.countDocuments(),
    errors: await ErrorModel.countDocuments(),
  };

  res.status(200).send(stats);
 } catch (e: any) {
  res.status(500).send({
    error: e.message,
  });
 }
}
```

## apps/web/src/pages/api/admin/project-ideas/index.ts

```
/**
 * apps/web/src/pages/api/admin/project-ideas/index.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 03.12.2022
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import ProjectIdeaModel from "@codeup/util/src/models/ProjectIdeaModel";
import UserModel from "@codeup/util/src/models/UserModel";
import mongoConnect from "@codeup/util/src/mongo";
import { verify } from "jsonwebtoken";
```

```
export default async function (req: NextApiRequest, res: NextApiResponse) {
  try {
    const jwt = req.headers["x-jwt"];
    if (!verify(jwt as string, process.env.JWT_SECRET as string)) {
      res.status(401).json({
        error: "Unauthorized",
      });
      return;
    }

    const decoded = verify(
      jwt as string,
      process.env.JWT_SECRET as string
    ) as any;

    const adminuser = await UserModel.findOne({ username: decoded.username });

    if (adminuser.role !== "admin") {
      res.status(401).json({
        error: "Unauthorized",
      });
      return;
    }

    await mongoConnect();

    if (req.method === "GET") {
      const ideas = await ProjectIdeaModel.find({});
      res.status(200).json({ ideas });
      return;
    } else if (req.method == "DELETE") {
      const idea = await ProjectIdeaModel.findById(req.query.id);

      if (!idea) {
        res.status(404).json({
          error: "Not Found",
        });

        return;
      }

      await idea.remove();

      res.status(200).json({
        message: "Idea deleted",
      });
    } else if (req.method == "POST") {
      if (
        !req.body ||
        !req.body.title ||
        !req.body.description ||
        !req.body.level
      ) {
        res.status(400).json({
          error: "Bad Request",
        });
        return;
      }
```

```
      const idea = await ProjectIdeaModel.create({
        title: req.body.title,
        description: req.body.description,
        exampleDeployedURL: req.body.exampleDeployedURL || "",
        exampleSourceURL: req.body.exampleSourceURL || "",
        level: req.body.level,
      });

      res.status(200).json({
        idea,
      });
    }
  } catch (e) {
    // @ts-ignore
    if (e.message) {
      res.status(500).json({
        // @ts-ignore
        error: "Internal Server Error:" + e.message,
        success: false,
      });
    } else {
      res.status(500).json({
        error: "Internal Server Error: Unknown Error",
        success: false,
      });
    }
  }
}
```

## apps/web/src/pages/api/admin/roads/index.ts

```
/**
 * apps/web/src/pages/api/admin/roads/index.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 13.04.2023
 *
 */
import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/src/mongo";
import jwt, { JwtPayload, VerifyErrors } from "jsonwebtoken";
import UserModel from "@codeup/util/src/models/UserModel";
import RoadmapModel from "@codeup/util/dist/models/RoadmapModel";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "GET") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  if (!req.headers["x-jwt"]) {
    res.status(400).json({
      error: "Missing Auth-Header (x-jwt)",
    });
```

```
    return;
  }

  jwt.verify(
    req.headers["x-jwt"] as string,
    process.env.JWT_SECRET as string,
    async (err: VerifyErrors | null, decoded: JwtPayload | string | any) => {
      if (err) {
        res.status(400).json({
          error: "Invalid token",
        });
        return;
      }

      await mongoConnect();

      const user = await UserModel.findById((decoded as any).id);

      if (!user) {
        res.status(400).json({
          error: "Invalid token",
        });
        return;
      }

      if (user.role !== "admin") {
        res.status(403).json({
          error: "You are not allowed to do this",
        });
        return;
      }

      const roads = await RoadmapModel.find({});

      for (const r of roads) {
        await r.populate("owner");
        if (r.owner === null) {
          r.owner = {
            username: "Deleted User",
          };
        }
      }

      res.status(200).json({
        success: true,
        roads,
      });
    }
  );
}
```

## apps/web/src/pages/api/admin/shorturls/index.ts

```
/**
 * apps/web/src/pages/api/admin/shorturls/index.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 14.04.2023
 *
```

```
 */

import { NextApiRequest, NextApiResponse } from "next";
import jwt from "jsonwebtoken";
import ShortURLModel from "@codeup/util/src/models/ShortURLModel";
import UserModel from "@codeup/util/src/models/UserModel";
import mongoConnect from "@codeup/util/dist/mongo";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (!req.headers["x-jwt"]) {
    res.status(401).json({
      error: "Unauthorized",
    });
    return;
  }

  try {
    const decoded = jwt.verify(
      req.headers["x-jwt"] as string,
      process.env.JWT_SECRET as string
    );

    if (!decoded) {
      res.status(401).json({
        error: "Unauthorized",
      });
      return;
    }

    await mongoConnect();

    const user = await UserModel.findOne({
      _id: (decoded as any).id,
    });

    if (!user || user.role !== "admin") {
      res.status(401).json({
        error: "Unauthorized",
      });
      return;
    }

    if (req.method === "GET") {
      const shortUrls = await ShortURLModel.find({});
      res.status(200).json(shortUrls);
    } else if (req.method === "POST") {
      if (!req.body || !req.body.short || !req.body.destination) {
        res.status(400).json({
          error: "Bad Request",
        });
        return;
      }

      const existing = await ShortURLModel.findOne({
        short: req.body.short,
      });
```

```
      if (existing) {
        res.status(400).json({
          error: "Short already exists",
        });
        return;
      }

      const shortUrl = await ShortURLModel.create({
        short: req.body.short,
        destination: req.body.destination,
      });

      res.status(200).json(shortUrl);
    } else if (req.method === "DELETE") {
      if (!req.query.short) {
        res.status(400).json({
          error: "Bad Request",
        });
        return;
      }

      const existing = await ShortURLModel.findById(req.query.short);

      if (!existing) {
        res.status(400).json({
          error: "Short doesn't exists",
        });
        return;
      }

      await existing.remove();

      res.status(200).json({
        success: true,
      });
    } else {
      res.status(405).json({
        error: "Method not allowed",
      });
    }
  } catch (e: any) {
    res.status(500).json({
      error: e.message,
    });
  }
}
```

## apps/web/src/pages/api/admin/snippets/index.ts

```
/**
 * apps/web/src/pages/api/admin/snippets/index.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 13.04.2023
 *
 */
import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/src/mongo";
import jwt, { JwtPayload, VerifyErrors } from "jsonwebtoken";
```

```
import UserModel from "@codeup/util/src/models/UserModel";
import CodeSnippet from "@codeup/util/dist/models/CodeSnippet";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "GET") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  if (!req.headers["x-jwt"]) {
    res.status(400).json({
      error: "Missing Auth-Header (x-jwt)",
    });
    return;
  }

  jwt.verify(
    req.headers["x-jwt"] as string,
    process.env.JWT_SECRET as string,
    async (err: VerifyErrors | null, decoded: JwtPayload | string | any) => {
      if (err) {
        res.status(400).json({
          error: "Invalid token",
        });
        return;
      }

      await mongoConnect();

      const user = await UserModel.findById((decoded as any).id);

      if (!user) {
        res.status(400).json({
          error: "Invalid token",
        });
        return;
      }

      if (user.role !== "admin") {
        res.status(403).json({
          error: "You are not allowed to do this",
        });
        return;
      }

      const snippets = await CodeSnippet.find({});

      res.status(200).json({
        success: true,
        snippets,
      });
    }
  );
}
```

## apps/web/src/pages/api/admin/testimonials/index.ts

```
/**
 * apps/web/src/pages/api/admin/testimonials/index.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 12.04.2023
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/dist/mongo";
import jwt from "jsonwebtoken";
import UserModel from "@codeup/util/dist/models/UserModel";
import CustomerChoiceModel from "@codeup/util/dist/models/CustomerChoiceModel";

export default async (req: NextApiRequest, res: NextApiResponse) => {
  try {
    await mongoConnect();

    const token = req.headers["x-jwt"] as string;

    if (!token) {
      res.status(401).json({ error: "Not authorized" });
      return;
    }

    const decoded = jwt.verify(token, process.env.JWT_SECRET as string);

    if (!decoded) {
      res.status(401).json({ error: "Not authorized" });
      return;
    }

    const user = await UserModel.findById((decoded as any).id);

    if (!user || user.role !== "admin") {
      res.status(401).json({ error: "Not authorized" });
      return;
    }

    if (req.method === "GET") {
      const testimonials = await CustomerChoiceModel.find();

      res.status(200).json(testimonials);
      return;
    }

    if (req.method === "POST") {
      if (
        !req.body ||
        !req.body.name ||
        !req.body.position ||
        !req.body.company ||
        !req.body.company ||
        !req.body.image
      ) {
        res.status(400).json({ error: "Missing data" });
        return;
```

```
  }

  const testimonial = await CustomerChoiceModel.create({
    name: req.body.name,
    position: req.body.position,
    company: req.body.company,
    content: req.body.content,
    image: req.body.image,
  });

  res.status(200).json(testimonial);
  return;
}

if (req.method === "DELETE") {
  if (!req.query || !req.query.id) {
    res.status(400).json({ error: "Missing data" });
    return;
  }

  const testimonial = await CustomerChoiceModel.findById(req.query.id);

  if (!testimonial) {
    res.status(404).json({ error: "Not found" });
    return;
  }

  await testimonial.remove();

  res.status(200).json({ success: true });
  return;
}

if (req.method === "PUT") {
  if (!req.query || !req.query.id) {
    res.status(400).json({ error: "Missing data" });
    return;
  }

  if (!req.body || !req.body.update) {
    res.status(400).json({ error: "Missing data" });
    return;
  }

  const testimonial = await CustomerChoiceModel.findById(req.query.id);

  if (!testimonial) {
    res.status(404).json({ error: "Not found" });
    return;
  }

  for (const key in req.body.update) {
    if (key === "_id" || key === "__v") return;
    if (testimonial[key]) {
      testimonial[key] = req.body.update[key];
    }
  }

  await testimonial.save();
```

```
      res.status(200).json(testimonial);
      return;
    }

    res.status(404).json({ error: "Not found" });
  } catch (e: any) {
    res.status(500).json({ error: e.message });
  }
};
```

## apps/web/src/pages/api/admin/text/create.ts

```
/**
 * apps/web/src/pages/api/admin/text/create.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 04.11.2022
 *
 */
import mongoConnect from "@codeup/util/src/mongo";
import { NextApiRequest, NextApiResponse } from "next";
import { verify } from "jsonwebtoken";
import UserModel from "@codeup/util/src/models/UserModel";
import TextContentModel from "@codeup/util/src/models/TextContentModel";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "POST") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  try {
    await mongoConnect();

    if (!req.headers["x-jwt"]) {
      res.status(401).json({
        error: "Unauthorized",
      });
      return;
    }

    const { title, friendlyName, course, content, quiz } = req.body;

    if (!title || !friendlyName || !course || !content || !quiz) {
      res.status(400).json({
        error: "Bad Request",
      });
      return;
    }

    if (
      !verify(req.headers["x-jwt"] as string, process.env.JWT_SECRET as string)
    ) {
      res.status(401).json({
```

```
        error: "Unauthorized",
      });
      return;
    }

    const decoded = verify(
      req.headers["x-jwt"] as string,
      process.env.JWT_SECRET as string
    ) as any;

    const adminuser = await UserModel.findOne({ username: decoded.username });

    if (adminuser.role !== "admin") {
      res.status(401).json({
        error: "Unauthorized",
      });
      return;
    }

    const existingText = await TextContentModel.findOne({
      friendlyName: friendlyName,
    });

    if (existingText) {
      res.status(400).json({
        error: "Course already exists",
      });
      return;
    }

    const text = await TextContentModel.create({
      name: title,
      friendlyName,
      course,
      content,
      quiz,
    });

    res.status(200).json({
      success: true,
      text,
    });
  } catch (e) {
    res.status(500).json({
      error: "Internal Server Error",
      // @ts-ignore
      message: e.message,
    });
  }
}
```

## apps/web/src/pages/api/admin/text/list.ts

```
/**
 * apps/web/src/pages/api/admin/text/list.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 04.11.2022
 *
```

```
 */
import { NextApiRequest, NextApiResponse } from "next";
import { verify } from "jsonwebtoken";
import mongoConnect from "@codeup/util/src/mongo";
import UserModel from "@codeup/util/src/models/UserModel";
import TextContentModel from "@codeup/util/src/models/TextContentModel";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "GET") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  if (!req.headers["x-jwt"]) {
    res.status(401).json({
      error: "Unauthorized",
    });
    return;
  }

  try {
    const jwt = req.headers["x-jwt"];
    if (!verify(jwt as string, process.env.JWT_SECRET as string)) {
      res.status(401).json({
        error: "Unauthorized",
      });
      return;
    }

    const decoded = verify(
      jwt as string,
      process.env.JWT_SECRET as string
    ) as any;

    const adminuser = await UserModel.findOne({ username: decoded.username });

    if (adminuser.role !== "admin") {
      res.status(401).json({
        error: "Unauthorized",
      });
      return;
    }

    await mongoConnect();

    const texts = await TextContentModel.find();

    res.status(200).json({
      texts,
    });
  } catch (e) {
    res.status(500).json({
      error: "Internal Server Error",
    });
  }
```

```
}
```

## apps/web/src/pages/api/admin/text/[tid].ts

```
/**
 * apps/web/src/pages/api/admin/course/[tid].ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 04.11.2022
 *
 */
import { NextApiRequest, NextApiResponse } from "next";
import { verify } from "jsonwebtoken";
import mongoConnect from "@codeup/util/src/mongo";
import UserModel from "@codeup/util/src/models/UserModel";
import TextContentModel from "@codeup/util/src/models/TextContentModel";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (
    req.method !== "GET" &&
    req.method !== "POST" &&
    req.method !== "DELETE"
  ) {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  if (!req.headers["x-jwt"]) {
    res.status(401).json({
      error: "Unauthorized",
    });
    return;
  }

  const { tid } = req.query;

  if (!tid) {
    res.status(400).json({
      error: "Bad Request",
    });
    return;
  }

  try {
    const jwt = req.headers["x-jwt"];
    if (!verify(jwt as string, process.env.JWT_SECRET as string)) {
      res.status(401).json({
        error: "Unauthorized",
      });
      return;
    }

    const decoded = verify(
      jwt as string,
```

```
      process.env.JWT_SECRET as string
    ) as any;

    const adminuser = await UserModel.findOne({ username: decoded.username });

    if (adminuser.role !== "admin") {
      res.status(401).json({
        error: "Unauthorized",
      });
      return;
    }

    await mongoConnect();

    const text = await TextContentModel.findById(tid);

    if (!text) {
      res.status(404).json({
        error: "Not Found",
      });
      return;
    }

    if (req.method === "GET") {
      res.status(200).json({
        text,
      });
      return;
    }

    if (req.method === "POST") {
      if (!req.body.update) {
        res.status(400).json({
          error: "Bad Request",
        });
        return;
      }

      for (const key of Object.keys(req.body.update)) {
        text[key] = req.body.update[key];
      }

      await text.save();

      res.status(200).json({
        text,
      });
      return;
    }

    if (req.method === "DELETE") {
      await text.delete();

      res.status(200).json({
        deleted: true,
        success: true,
      });
      return;
    }
  } catch (e) {
```

```
    res.status(500).json({
      error: "Internal Server Error",
    });
  }
}
```

## apps/web/src/pages/api/admin/todos/index.ts

```
/**
 * apps/web/src/pages/api/admin/todo/index.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 13.04.2023
 *
 */
import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/src/mongo";
import jwt, { JwtPayload, VerifyErrors } from "jsonwebtoken";
import UserModel from "@codeup/util/src/models/UserModel";
import TodoListModel from "@codeup/util/src/models/TodoListModel";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "GET") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  if (!req.headers["x-jwt"]) {
    res.status(400).json({
      error: "Missing Auth-Header (x-jwt)",
    });
    return;
  }

  jwt.verify(
    req.headers["x-jwt"] as string,
    process.env.JWT_SECRET as string,
    async (err: VerifyErrors | null, decoded: JwtPayload | string | any) => {
      if (err) {
        res.status(400).json({
          error: "Invalid token",
        });
        return;
      }

      await mongoConnect();

      const user = await UserModel.findById((decoded as any).id);

      if (!user) {
        res.status(400).json({
          error: "Invalid token",
        });
        return;
```

```
    }

    if (user.role !== "admin") {
      res.status(403).json({
        error: "You are not allowed to do this",
      });
      return;
    }

    const lists = await TodoListModel.find({});

    for (const l of lists) {
      await l.populate("owner");
      if (l.owner === null) {
        l.owner = {
          username: "Deleted User",
        };
      }
    }

    res.status(200).json({
      success: true,
      lists,
    });
  }
 );
}
```

## apps/web/src/pages/api/admin/video/create.ts

```
/**
 * apps/web/src/pages/api/admin/video/create.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 31.10.2022
 *
 */
import mongoConnect from "@codeup/util/src/mongo";
import { NextApiRequest, NextApiResponse } from "next";
import { verify } from "jsonwebtoken";
import UserModel from "@codeup/util/src/models/UserModel";
import VideoModel from "@codeup/util/src/models/VideoModel";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "POST") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  try {
    await mongoConnect();

    if (!req.headers["x-jwt"]) {
      res.status(401).json({
```

```
    error: "Unauthorized",
  });
  return;
}

const { title, friendlyName, youtubeURL, course, transcript, quiz } =
  req.body;

if (
  !title ||
  !friendlyName ||
  !youtubeURL ||
  !course ||
  !transcript ||
  !quiz
) {
  res.status(400).json({
    error: "Bad Request",
  });
  return;
}

if (
  !verify(req.headers["x-jwt"] as string, process.env.JWT_SECRET as string)
) {
  res.status(401).json({
    error: "Unauthorized",
  });
  return;
}

const decoded = verify(
  req.headers["x-jwt"] as string,
  process.env.JWT_SECRET as string
) as any;

const adminuser = await UserModel.findOne({ username: decoded.username });

if (adminuser.role !== "admin") {
  res.status(401).json({
    error: "Unauthorized",
  });
  return;
}

const existingVideo = await VideoModel.findOne({
  friendlyName: friendlyName,
});

if (existingVideo) {
  res.status(400).json({
    error: "Course already exists",
  });
  return;
}

const video = await VideoModel.create({
  name: title,
  friendlyName: friendlyName,
  youtube: youtubeURL,
```

```
      course: course,
      transcript: transcript,
      quiz: quiz,
    });

    res.status(200).json({
      success: true,
      video: video,
    });
  } catch (e) {
    res.status(500).json({
      error: "Internal Server Error",
      // @ts-ignore
      message: e.message,
    });
  }
}
```

## apps/web/src/pages/api/admin/video/list.ts

```
/**
 * apps/web/src/pages/api/admin/course/list.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 30.10.2022
 *
 */
import { NextApiRequest, NextApiResponse } from "next";
import { verify } from "jsonwebtoken";
import mongoConnect from "@codeup/util/src/mongo";
import UserModel from "@codeup/util/src/models/UserModel";
import VideoModel from "@codeup/util/src/models/VideoModel";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "GET") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  if (!req.headers["x-jwt"]) {
    res.status(401).json({
      error: "Unauthorized",
    });
    return;
  }

  try {
    const jwt = req.headers["x-jwt"];
    if (!verify(jwt as string, process.env.JWT_SECRET as string)) {
      res.status(401).json({
        error: "Unauthorized",
      });
      return;
    }
```

```
    const decoded = verify(
      jwt as string,
      process.env.JWT_SECRET as string
    ) as any;

    const adminuser = await UserModel.findOne({ username: decoded.username });

    if (adminuser.role !== "admin") {
      res.status(401).json({
        error: "Unauthorized",
      });
      return;
    }

    await mongoConnect();

    const videos = await VideoModel.find();

    res.status(200).json({
      videos,
    });
  } catch (e) {
    res.status(500).json({
      error: "Internal Server Error",
    });
  }
}
```

## apps/web/src/pages/api/admin/video/youtube/[vid].ts

```
/**
 * apps/web/src/pages/api/admin/course/youtube/[vid].ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 02.11.2022
 *
 */
import { NextApiRequest, NextApiResponse } from "next";
import { verify } from "jsonwebtoken";
import mongoConnect from "@codeup/util/src/mongo";
import UserModel from "@codeup/util/src/models/UserModel";
import VideoModel from "@codeup/util/src/models/VideoModel";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (
    req.method !== "GET" &&
    req.method !== "POST" &&
    req.method !== "DELETE"
  ) {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }
```

```
if (!req.headers["x-jwt"]) {
  res.status(401).json({
    error: "Unauthorized",
  });
  return;
}

const { vid } = req.query;

if (!vid) {
  res.status(400).json({
    error: "Bad Request",
  });
  return;
}

try {
  const jwt = req.headers["x-jwt"];
  if (!verify(jwt as string, process.env.JWT_SECRET as string)) {
    res.status(401).json({
      error: "Unauthorized",
    });
    return;
  }

  const decoded = verify(
    jwt as string,
    process.env.JWT_SECRET as string
  ) as any;

  const adminuser = await UserModel.findOne({ username: decoded.username });

  if (adminuser.role !== "admin") {
    res.status(401).json({
      error: "Unauthorized",
    });
    return;
  }

  await mongoConnect();

  const video = await VideoModel.findOne({
    youtube: "https://www.youtube.com/watch?v=" + vid,
  });

  if (!video) {
    res.status(404).json({
      error: "Not Found",
    });
    return;
  }

  if (req.method === "GET") {
    res.status(200).json({
      video,
    });
    return;
  }

  if (req.method === "POST") {
```

```
      if (!req.body.update) {
        res.status(400).json({
          error: "Bad Request",
        });
        return;
      }

      for (const key of Object.keys(req.body.update)) {
        video[key] = req.body.update[key];
      }

      await video.save();

      res.status(200).json({
        video,
      });
      return;
    }

    if (req.method === "DELETE") {
      await video.delete();

      res.status(200).json({
        deleted: true,
        success: true,
      });
      return;
    }
  } catch (e) {
    res.status(500).json({
      error: "Internal Server Error",
    });
  }
}
```

## apps/web/src/pages/api/analytics/index.ts

```
/**
 * apps/web/src/pages/api/analytics/index.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 25.11.2022
 *
 */
import { NextApiRequest, NextApiResponse } from "next";
import UAParser from "ua-parser-js";
import { verify } from "jsonwebtoken";
import AnalyticsModel from "@codeup/util/src/models/AnalyticsModel";
import mongoConnect from "@codeup/util/src/mongo";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "POST") {
    res.status(405).json({ error: "Method not allowed" });
    return;
  }
```

```
if (
  !req.headers["referer"] ||
  !req.headers["user-agent"] ||
  !req.headers["host"]
) {
  res.status(400).json({ error: "Bad request" });
  return;
}

const parser = new UAParser(req.headers["user-agent"] as string);

const browser = parser.getBrowser().name;
const os = parser.getOS().name + " " + parser.getOS().version;

const token = req.headers["x-jwt"];

let loggedIn = false;
let userId = null;

if (token) {
  const decoded = verify(token as string, process.env.JWT_SECRET as string);
  if (decoded) {
    loggedIn = true;
    userId = (decoded as any)["id"] as any;

    if ((decoded as any)["role"] === "admin") {
      res.status(403).json({ error: "Analytics are disabled for Admins." });
      return;
    }
  } else {
    loggedIn = false;
    userId = null;
  }
}

const path = (req.headers["referer"] as string)
  .replace("https://", "")
  .replace("http://", "")
  .replace(req.headers["host"] as string, "");

try {
  await mongoConnect();

  const analytics = await AnalyticsModel.create({
    browser,
    os,
    path,
    loggedIn,
    userId,
    createdAt: new Date().toISOString(),
  });

  res.status(200).json({
    success: true,
  });
} catch (e) {
  // @ts-ignore
  if (e.message) {
    // @ts-ignore
    res.status(500).json({ error: e.message });
```

```
    } else {
      // @ts-ignore
      res.status(500).json({ error: "Unknown Internal server error" });
    }
  }
}
```

## apps/web/src/pages/api/ask/posts/index.ts

```
/**
 * apps/web/src/pages/api/ask/posts/index.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 23.06.23
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/src/mongo";
import { decode, verify } from "jsonwebtoken";
import UserModel from "@codeup/util/dist/models/UserModel";
import AskPostModel from "@codeup/util/dist/models/AskPostModel";
import ogs from "open-graph-scraper";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (!["GET", "DELETE", "POST"].includes(req.method as string)) {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  try {
    await mongoConnect();

    let user: any;

    if (req.method !== "GET") {
      const jwt = req.headers["x-jwt"];

      if (!jwt) {
        res.status(401).json({
          error: "Unauthorized",
        });
        return;
      }

      const decoded = verify(
        jwt as string,
        process.env.JWT_SECRET as string
      ) as {
        id: string;
      };

      if (!decoded) {
        res.status(401).json({
```

```
      error: "Unauthorized",
    });
    return;
  }

  user = await UserModel.findById(decoded.id);

  if (!user || !user.active) {
    res.status(401).json({
      error: "Unauthorized",
    });
    return;
  }
}

if (req.method === "GET") {
  let posts: any[] = [];

  if (req.query.id) {
    const post = await AskPostModel.findById(req.query.id);

    if (!post) {
      res.status(404).json({
        error: "Not Found",
      });
      return;
    }

    posts.push(post);
  } else {
    if (req.query.parent) {
      const post = await AskPostModel.find({
        parent: req.query.parent,
      });

      posts.push(...post);
    } else if (req.query.user) {
      const post = await AskPostModel.find({
        username: req.query.user,
      });

      posts.push(...post);
    } else if (req.query.query) {
      const post = await AskPostModel.find({
        content: { $regex: req.query.query as string, $options: "gi" },
      });

      posts.push(...post);
    } else {
      const post = await AskPostModel.find({
        parent: null,
      });

      posts.push(...post);
    }
  }

  posts = posts.sort((a, b) => {
    if (a.createdAt > b.createdAt) {
      return -1;
```

```
    } else if (a.createdAt < b.createdAt) {
      return 1;
    } else {
      return 0;
    }
  });

  const PAGE_SIZE = 10;

  let page = 1;

  if (req.query.page) {
    page = parseInt(req.query.page as string);
  }

  let pages = Math.ceil(posts.length / PAGE_SIZE);

  if (posts.length > PAGE_SIZE) {
    posts = posts.slice((page - 1) * PAGE_SIZE, page * PAGE_SIZE);
  }

  res.status(200).json({
    pages,
    posts,
  });
} else if (req.method === "DELETE") {
  if (!req.query.id) {
    res.status(400).json({
      error: "Bad Request",
    });
    return;
  }

  const post = await AskPostModel.findById(req.query.id);

  if (!post) {
    res.status(404).json({
      error: "Not Found",
    });
    return;
  }

  if (post.username !== user.username) {
    res.status(403).json({
      error: "Forbidden",
    });
    return;
  }

  const children = await AskPostModel.find({
    parent: post._id,
  });

  for (const child of children) {
    await child.remove();
  }

  await post.remove();

  res.status(200).json({
```

```
        success: true,
      });
    } else if (req.method === "POST") {
      const { content, parent } = req.body;

      if (!content) {
        res.status(400).json({
          error: "Bad Request",
        });
        return;
      }

      let parentId: string | null = null;

      if (parent) {
        const parentPost = await AskPostModel.findById(parent);

        if (!parentPost || parentPost.parent !== null) {
          res.status(400).json({
            error: "Bad Request",
          });
          return;
        }

        parentId = parentPost._id;
      }

      const post = await AskPostModel.create({
        user: user._id,
        username: user.username,
        content: content,
        parent: parentId,
        embeds: await getEmbeds(content),
      });

      res.status(200).json({
        success: true,
        post: post,
      });
    }
  } catch (e: any) {
    res.status(500).json({ error: e.message });
  }
}

async function getEmbeds(content: string): Promise<any[]> {
  const embeds: {
    type: "file" | "image" | "video" | "audio" | "link";
    url?: string;
    title?: string;
    description?: string;
    image?: string;
    site_name?: string;
  }[] = [];

  const links = content.match(/(https?:\/\/[^\s]+)/g) || [];

  for (const link of links) {
    if (link.match(/(https?:\/\/[^\s]+)*\.codeup\.space/g)) {
      let name = "";
```

```
    if (link.includes("https://")) {
      name = link.replace("https://", "").split(".")[0];
    } else {
      name = link.replace("http://", "").split(".")[0];
    }

    embeds.push({
      type: "link",
      url: link,
      title: "CodeUp Projekt " + name,
      description:
        "Dies ist ein Link, der zu einem Projekt führt, welches mit CodeUp veröffentlicht wurde.",
      image: "https://codeup.space/codeup-splash-github.jpg",
      site_name: name,
    });
  } else {
    try {
      const headRes = await fetch(link, {
        method: "HEAD",
      });

      const contentType = (
        (headRes.headers.get("Content-Type") as string) || ""
      )
        .toLowerCase()
        .split(";")[0];

      if (contentType === "text/html") {
        const o = await ogs({ url: link });
        if (!o.error) {
          embeds.push({
            title: (o.result as any).ogTitle as string,
            description: (o.result as any).ogDescription as string,
            image: (o.result as any).ogImage.url as string,
            url: (o.result as any).ogUrl as string,
            site_name: (o.result as any).ogSiteName as string,
            type: "link",
          });
        }
      } else if (contentType.startsWith("image/")) {
        embeds.push({
          url: link,
          type: "image",
        });
      } else if (contentType.startsWith("video/")) {
        embeds.push({
          url: link,
          type: "video",
        });
      } else {
        embeds.push({
          url: link,
          type: "file",
        });
      }
    } catch (e) {
      console.log(e);
    }
  }
}
```

```
  return embeds;
}
```

## apps/web/src/pages/api/ask/profiles/[username].ts

```
/**
 * apps/web/src/pages/api/ask/profiles/[username].ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 26.06.23
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/dist/mongo";
import UserModel from "@codeup/util/dist/models/UserModel";
import AskProfileModel from "@codeup/util/dist/models/AskProfileModel";
import { verify } from "jsonwebtoken";
import EditorProjectModel from "@codeup/util/dist/models/EditorProjectModel";
import ObtainedCertificateModel from "@codeup/util/dist/models/ObtainedCertificateModel";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (!["GET", "POST"].includes(req.method as string)) {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  try {
    await mongoConnect();

    if (req.method === "GET") {
      const username = req.query.username as string;

      const profile = await getProfileByUsername(username);

      if (!profile) {
        res.status(404).json({
          error: "Profile not found",
        });
        return;
      }

      res.status(200).json({
        profile,
      });
    } else if (req.method === "POST") {
      const username = req.query.username as string;

      const profile = await getProfileByUsername(username);

      if (!profile) {
        res.status(404).json({
          error: "Profile not found",
```

```
  });
  return;
}

const token = req.headers["x-jwt"] as string;

if (!token) {
  res.status(401).json({
    error: "Unauthorized",
  });
  return;
}

const decoded = verify(token, process.env.JWT_SECRET as string) as any;

if (!decoded) {
  res.status(401).json({
    error: "Unauthorized",
  });
  return;
}

const user = await UserModel.findOne({
  username: decoded.username,
});

if (!user) {
  res.status(401).json({
    error: "Unauthorized",
  });
  return;
}

if (user.username !== profile.username) {
  res.status(401).json({
    error: "Unauthorized",
  });
  return;
}

const { update } = req.body;

if (!update) {
  res.status(400).json({
    error: "Bad Request",
  });
  return;
}

const updatableKeys = [
  "banner",
  "website",
  "bio",
  "certificates",
  "projects",
];

for (const key of updatableKeys) {
  if (update[key]) {
    if (["banner", "website"].includes(key)) {
```

```
      if (key === "website" && update[key] === "codeup.remove()") {
        profile[key] = "";
        continue;
      }

      if (update[key].match(/(https?:\/\/[^\s]+)/g)) {
        profile[key] = update[key];
      } else {
        res.status(400).json({
          error: "Banner or Website is not a valid URL",
        });
        return;
      }
    } else if (key === "bio") {
      if (update[key] === "codeup.remove()") {
        profile[key] = "";
        continue;
      }

      if (update[key].length > 0 && update[key].length < 200) {
        profile[key] = update[key];
      } else {
        res.status(400).json({
          error: "Bio is too long",
        });
        return;
      }
    } else if (key === "projects") {
      const projects = update[key] as any[];
      for (const p of projects) {
        const project = await EditorProjectModel.findById(p);
        if (!project || !user._id.equals(project.owner)) {
          res.status(400).json({
            error: "Project not found",
          });
          return;
        }
      }

      profile[key] = projects;
    } else if (key === "certificates") {
      const certificates = update[key] as any[];

      for (const c of certificates) {
        const certificate = await ObtainedCertificateModel.findById(c);
        if (!certificate || user.username !== certificate.username) {
          res.status(400).json({
            error: "Certificate not found",
          });
          return;
        }
      }

      profile[key] = certificates;
    }
  }
}

await profile.save();
```

```
      res.status(200).json({
        profile,
      });
    }
  } catch (e: any) {
    res.status(500).json({
      error: e.message,
    });
  }
}

const getProfileByUsername = async (
  username: string
): Promise<typeof AskProfileModel | any | null> => {
  const user = await UserModel.findOne({
    username,
  });

  if (!user) {
    return null;
  }

  let profile = await AskProfileModel.findOne({
    user: user._id,
  });

  if (!profile) {
    profile = await AskProfileModel.create({
      user: user._id,
      username: username,
      banner: "/_static/images/software-developer-6521720_1920.jpg",
      badges: [],
      website: "",
      bio: "",
      certificates: [],
      projects: [],
    });
  }

  return profile;
};
```

## apps/web/src/pages/api/blog/index.ts

```
/**
 * apps/web/src/pages/api/blog/index.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 27.12.2022
 *
 */
import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/src/mongo";
import BlogModel from "@codeup/util/src/models/BlogModel";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
```

```
  try {
    await mongoConnect();

    const blogs = await BlogModel.find({});

    res.status(200).json({ blogs });
  } catch (e) {
    res.status(500).json({
      error: e,
    });
  }
}
```

## apps/web/src/pages/api/blog/[id].ts

```
/**
 * apps/web/src/pages/api/blog/[id].ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 27.12.2022
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/src/mongo";
import BlogModel from "@codeup/util/src/models/BlogModel";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  try {
    const id = req.query.id as string;

    await mongoConnect();

    const blog = await BlogModel.findById(id);

    res.status(200).json({ blog });
  } catch (e) {
    res.status(500).json({
      error: e,
    });
  }
}
```

## apps/web/src/pages/api/cert.ts

```
/**
 * apps/web/src/pages/api/cert.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 27.11.2022
 *
 */

import ObtainedCertificateModel from "@codeup/util/src/models/ObtainedCertificateModel";
import CourseModel from "@codeup/util/src/models/CourseModel";
```

```
import mongoConnect from "@codeup/util/src/mongo";
import { NextApiRequest, NextApiResponse } from "next";

export default async function (req: NextApiRequest, res: NextApiResponse) {
  if (req.method !== "GET") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  if (!req.query.id) {
    res.status(400).json({
      error: "Missing id",
    });
    return;
  }

  try {
    await mongoConnect();

    const obtainedCertificate = await ObtainedCertificateModel.findOne({
      _id: req.query.id,
    });

    if (!obtainedCertificate) {
      res.status(404).json({
        error: "Certificate not found",
      });
      return;
    }

    const course = await CourseModel.findOne({
      _id: obtainedCertificate.course,
    });

    if (!course) {
      res.status(404).json({
        error: "Course not found",
      });
      return;
    }

    if (req.query.export) {
      if (!obtainedCertificate.screenshot) {
        res.redirect(
          "/error?error=Das Zertifikat ist noch nicht für den Export bereit!"
        );
      } else {
        const decoded = obtainedCertificate.screenshot
          .toString()
          .replace("", "");
        const imageResp = new Buffer(decoded, "base64");
        res.writeHead(200, {
          "Content-Type": "image/png",
          "Content-Length": imageResp.length,
        });

        res.end(imageResp);
      }
```

```
    } else {
      res.status(200).json({
        obtainedCertificate,
        course,
      });
    }
  } catch (e) {
    // @ts-ignore
    if (e.message) {
      // @ts-ignore
      res.status(500).json({ error: "Internal Server Error: " + e.message });
    } else {
      res.status(500).json({ error: "Unknown internal server error" });
    }
  }
}
```

## apps/web/src/pages/api/certificates/my.ts

```
/**
 * apps/web/src/pages/api/certificates/my.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 03.12.2022
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import jwt from "jsonwebtoken";
import UserModel from "@codeup/util/src/models/UserModel";
import ObtainedCertificateModel from "@codeup/util/src/models/ObtainedCertificateModel";
import CourseModel from "@codeup/util/src/models/CourseModel";
import mongoConnect from "@codeup/util/src/mongo";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "GET") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  try {
    const token = req.headers["x-jwt"] as string;

    if (jwt.verify(token, process.env.JWT_SECRET as string)) {
      const decoded = jwt.decode(token) as any;

      await mongoConnect();

      const user = await UserModel.findOne({
        _id: decoded.id,
      });

      if (!user) {
        res.status(404).json({
```

```
          error: "User not found",
        });
        return;
      }

      const certs = await ObtainedCertificateModel.find({
        username: user.username,
      });

      const c = certs.map((c) => c.toObject());

      for (const cert of c) {
        const course = await CourseModel.findOne({ _id: cert.course });
        cert.course = course.name;
        cert.courseObj = course;
        delete cert.screenshot;
      }

      res.status(200).json({
        certs: c,
      });
    } else {
      res.status(401).json({
        error: "Invalid token",
      });
    }
  } catch (e) {
    res.status(500).json({
      error: e,
    });
  }
}
```

## apps/web/src/pages/api/challenges/check.ts

```
/**
 * apps/web/src/pages/api/challenges/check.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2024 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 18.05.2024
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import { isLoggedIn } from "../../../global/server/isLoggedIn";
import CodeChallengeModel from "@codeup/util/dist/models/CodeChallengeModel";
import CompletedUserChallenges from "@codeup/util/dist/models/CompletedUserChallenges";

export default async function (req: NextApiRequest, res: NextApiResponse) {
  try {
    const { auth, user } = await isLoggedIn(req, res);

    if (!auth) return;

    if (req.method !== "POST") {
      res.status(405).json({
        status: 405,
        error: "Method Not Allowed",
      });
```

```
  return;
}

const { challengeId, code, language } = req.body;

if (!challengeId || !code || !language) {
  res.status(400).json({
    status: 400,
    error: "Bad Request",
  });
  return;
}

const challenge = await CodeChallengeModel.findById(challengeId);

if (!challenge) {
  res.status(404).json({
    status: 404,
    error: "Not Found",
  });
  return;
}

const rx = await fetch(process.env.URL + "/api/rce/" + language, {
  method: "POST",
  headers: {
    "Content-Type": "application/json",
    "X-JWT": req.headers["x-jwt"] as string,
  },
  body: JSON.stringify({
    code,
  }),
});

let data = await rx.text();

data = data.trim();
data = data.replace(/\r/g, "");
let o = challenge.expectedOutput.trim().replace(/\r/g, "");

if (data === o) {
  res.status(200).json({
    status: 200,
    message: "Correct",
  });

  const mod = await CompletedUserChallenges.findOne({
    user: user._id,
  });

  if (mod) {
    mod.challenges.push(challengeId);
    await mod.markModified("challenges");
    await mod.save();
  } else {
    await CompletedUserChallenges.create({
      user: user._id,
      challenges: [challengeId],
    });
  }
```

```
    } else {
      res.status(400).json({
        status: 400,
        message: "Incorrect",
      });
    }
  } catch (e: any) {
    res.status(500).json({
      status: 500,
      error: e.message,
    });
  }
}
```

## apps/web/src/pages/api/challenges/completed.ts

```
/**
 * apps/web/src/pages/api/challenges/completed.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2024 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 18.05.2024
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import { isLoggedIn } from "../../../global/server/isLoggedIn";
import CompletedUserChallenges from "@codeup/util/dist/models/CompletedUserChallenges";

export default async function (req: NextApiRequest, res: NextApiResponse) {
  try {
    const { auth, user } = await isLoggedIn(req, res);

    if (!auth) return;

    const completedChallenges = await CompletedUserChallenges.findOne({
      user: user._id,
    });

    res.status(200).json({
      status: 200,
      data: completedChallenges,
    });
  } catch (e: any) {
    res.status(500).json({
      status: 500,
      error: e.message,
    });
  }
}
```

## apps/web/src/pages/api/challenges/index.ts

```
/**
 * apps/web/src/pages/api/challenges/index.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2024 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 18.05.2024
 *
```

```
 */

import { NextApiRequest, NextApiResponse } from "next";
import { isLoggedIn } from "../../../global/server/isLoggedIn";
import CodeChallengeModel from "@codeup/util/dist/models/CodeChallengeModel";

export default async function (req: NextApiRequest, res: NextApiResponse) {
  try {
    const { auth, user } = await isLoggedIn(req, res);

    if (!auth) return;

    if (req.method !== "GET") {
      res.status(405).json({
        status: 405,
        error: "Method not allowed.",
      });
      return;
    }

    const { id } = req.query;

    if (id) {
      const challenge = await CodeChallengeModel.findById(id);

      if (!challenge) {
        res.status(404).json({
          status: 404,
          error: "Challenge not found.",
        });
        return;
      }

      res.status(200).json({
        status: 200,
        challenge,
      });
      return;
    }

    const challenges = await CodeChallengeModel.find({});

    res.status(200).json({
      status: 200,
      challenges,
    });
  } catch (e: any) {
    res.status(500).json({
      status: 500,
      error: e.message,
    });
  }
}
```

## apps/web/src/pages/api/community/feed.ts

```
/**
 * src/pages/api/community/feed.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
```

```
 * License: Project License
 * Created At: 20.06.2022
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/src/mongo";
import CommunityPostModel from "@codeup/util/src/models/CommunityPostModel";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "GET") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  await mongoConnect();

  // in the future the feed will possibly be user specific
  // in the meantime, we'll just return the latest posts

  const { from, to, idsOnly, noReplies } = req.query;

  const posts = await CommunityPostModel.find({
    parent: noReplies ? null : { $ne: null },
  }).sort({
    createdAt: -1,
  });

  if (from && to) {
    let f2u = parseInt(from as string);
    let t2u = parseInt(to as string);

    if (f2u > t2u) {
      f2u = t2u;
      t2u = f2u;
    }

    if (f2u < 0) {
      f2u = 0;
    }

    if (t2u > posts.length) {
      t2u = posts.length;
    }

    try {
      let r = posts.slice(f2u, t2u);

      if (idsOnly) {
        r = r.map((p) => p._id);
      }

      res.status(200).json({
        posts: r,
        success: true,
```

```
      });
      return;
    } catch (e) {
      res.status(400).json({
        error: "Invalid range",
      });
      return;
    }
  } else {
    res.status(400).json({
      error: "Invalid range",
    });
    return;
  }
}
```

## apps/web/src/pages/api/community/has-profile/[id].ts

```
/**
 * src/pages/api/community/has-profile/[name].ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 12.06.2022
 *
 */
import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/src/mongo";
import CommunityProfileModel from "@codeup/util/src/models/CommunityProfileModel";
import mongoose from "mongoose";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "GET") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  const { id } = req.query;

  if (!id) {
    res.status(400).json({
      error: "Missing ID",
    });
    return;
  }

  await mongoConnect();

  try {
    const profile = await CommunityProfileModel.findOne({
      user: new mongoose.Types.ObjectId(id as string),
    });

    if (!profile) {
      res.status(404).json({
```

```
        error: "Not Found",
        hasProfile: false,
      });
      return;
    }

    res.status(200).json({
      hasProfile: true,
    });
  } catch (e) {
    res.status(404).json({
      error: "Not Found",
      hasProfile: false,
    });
    return;
  }
}
```

## apps/web/src/pages/api/community/post/create.ts

```
/**
 * src/pages/api/community/post/create.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 19.06.2022
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/src/mongo";
import jwt from "jsonwebtoken";
import UserModel from "@codeup/util/src/models/UserModel";
import CommunityProfileModel from "@codeup/util/src/models/CommunityProfileModel";
import CommunityPostModel from "@codeup/util/src/models/CommunityPostModel";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "POST") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  if (!req.body || !req.headers["x-jwt"] || !req.body.content) {
    res.status(400).json({
      error: "Missing required fields",
    });
    return;
  }

  jwt.verify(
    req.headers["x-jwt"] as string,
    // @ts-ignore
    process.env.JWT_SECRET as string,
    async (err, decoded) => {
      if (err) {
```

```
    res.status(400).json({
      error: "Invalid token",
    });
    return;
  } else {
    await mongoConnect();
    const user = await UserModel.findOne({
      _id: (decoded as any).id as string,
    });

    if (!user) {
      res.status(400).json({
        error: "User not found",
      });
      return;
    }

    const profile = await CommunityProfileModel.findOne({
      username: user.username,
    });

    if (!profile) {
      res.status(400).json({
        error: "Profile not found",
      });
    }

    let content = req.body.content;

    let SCRIPT_REGEX =
      /<script\b[^<]*(?:(?!<\/script>)<[^<]*)*<\/script>/gi;
    while (SCRIPT_REGEX.test(content)) {
      content = content.replace(SCRIPT_REGEX, "");
    }

    if (req.body.parent) {
      const parent = await CommunityPostModel.findOne({
        _id: req.body.parent,
      });

      if (!parent) {
        res.status(400).json({
          error: "Parent post not found",
        });
        return;
      }

      if (parent.parent !== null) {
        res.status(400).json({
          error: "Parent post is not a root post",
        });
        return;
      }

      const post = await CommunityPostModel.create({
        user: user._id,
        username: user.username,
        content: content,
        likes: [],
        comments: [],
```

```
            createdAt: Date.now(),
            parent: parent._id,
          });

          await post.save();

          parent.comments.push(post._id);

          await parent.save();

          res.status(200).json({
            post: post._id,
            success: true,
          });
        } else {
          const post = await CommunityPostModel.create({
            user: user._id,
            username: user.username,
            content: content,
            likes: [],
            comments: [],
            createdAt: Date.now(),
          });

          await post.save();

          res.status(200).json({
            post: post._id,
            success: true,
          });
        }
      }
    }
  );
}
```

## apps/web/src/pages/api/community/post/[id]/delete.ts

```
/**
 * src/pages/api/community/post/delete.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 19.06.2022
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/src/mongo";
import jwt from "jsonwebtoken";
import UserModel from "@codeup/util/src/models/UserModel";
import CommunityPostModel from "@codeup/util/src/models/CommunityPostModel";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "POST") {
    res.status(405).json({
      error: "Method not allowed",
```

```
  });
  return;
}

if (!req.headers["x-jwt"]) {
  res.status(400).json({
    error: "Missing token",
  });
  return;
}

const { id } = req.query;

jwt.verify(
  req.headers["x-jwt"] as string,
  // @ts-ignore
  process.env.JWT_SECRET,
  async (err, decoded) => {
    await mongoConnect();

    const user = await UserModel.findById((decoded as any).id);

    if (!user) {
      res.status(400).json({
        error: "Invalid token",
      });
      return;
    }

    try {
      const post = await CommunityPostModel.findById(id);

      if (!post) {
        res.status(404).json({
          error: "Not Found",
        });
        return;
      }

      if (post.username !== user.username) {
        res.status(403).json({
          error: "Forbidden",
        });
        return;
      }

      if (post.parent) {
        const parent = await CommunityPostModel.findById(post.parent);
        parent.comments.splice(parent.comments.indexOf(post._id), 1);
        await parent.save();
      } else {
        const comments = await CommunityPostModel.find({
          parent: id,
        });
        for (const comment of comments) {
          await comment.remove();
        }
      }

      await post.remove();
```

```
        res.status(200).json({
          success: true,
        });
      } catch (e) {
        res.status(500).json({
          // @ts-ignore
          error: e.message,
        });
      }
    }
  );
}
```

## apps/web/src/pages/api/community/post/[id]/like.ts

```
/**
 * src/pages/api/community/post/[id]/like.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 19.06.2022
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/src/mongo";
import jwt from "jsonwebtoken";
import UserModel from "@codeup/util/src/models/UserModel";
import CommunityPostModel from "@codeup/util/src/models/CommunityPostModel";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "POST") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  if (!req.headers["x-jwt"]) {
    res.status(400).json({
      error: "Missing required fields",
    });
    return;
  }

  const { id } = req.query;

  jwt.verify(
    req.headers["x-jwt"] as string,
    // @ts-ignore
    process.env.JWT_SECRET,
    async (err, decoded) => {
      await mongoConnect();

      const user = await UserModel.findById((decoded as any).id);
```

```
      if (!user) {
        res.status(400).json({
          error: "Invalid token",
        });
        return;
      }

      try {
        const post = await CommunityPostModel.findById(id);

        if (!post) {
          res.status(404).json({
            error: "Not Found",
          });
          return;
        }

        if (post.likes.includes(user._id)) {
          post.likes.splice(post.likes.indexOf(user._id), 1);
        } else {
          post.likes.push(user._id);
        }

        await post.save();

        res.status(200).json({
          success: true,
          post: post,
        });
      } catch (e) {
        res.status(500).json({
          // @ts-ignore
          error: e.message,
        });
      }
    }
  );
}
```

## apps/web/src/pages/api/community/post/[id].ts

```
/**
 * src/pages/api/community/post/[id].ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 19.06.2022
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/src/mongo";
import CommunityPostModel from "@codeup/util/src/models/CommunityPostModel";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "GET") {
    res.status(405).json({
```

```
      error: "Method not allowed",
    });
    return;
  }

  const { id } = req.query;
  if (!id) {
    res.status(400).json({
      error: "Missing ID",
    });
    return;
  }

  await mongoConnect();

  try {
    const post = await CommunityPostModel.findById(id);
    if (!post) {
      res.status(404).json({
        error: "Not Found",
      });
      return;
    }

    res.status(200).json({
      post,
      success: true,
    });
  } catch (e) {
    res.status(500).json({
      // @ts-ignore
      error: e.message,
    });
  }
}
```

## apps/web/src/pages/api/community/profile/create.ts

```
/**
 * src/pages/api/community/profile/create.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 13.06.2022
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/src/mongo";
import jwt from "jsonwebtoken";
import UserModel from "@codeup/util/src/models/UserModel";
import CommunityProfileModel from "@codeup/util/src/models/CommunityProfileModel";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "POST") {
    res.status(405).json({
      error: "Method not allowed",
```

```
  });
  return;
}

if (
  !req.body ||
  !req.headers["x-jwt"] ||
  !req.body.displayName ||
  !req.body.bio
) {
  res.status(400).json({
    error: "Missing required fields",
  });
  return;
}

jwt.verify(
  req.headers["x-jwt"] as string,
  // @ts-ignore
  process.env.JWT_SECRET,
  async (err, decoded) => {
    if (err) {
      res.status(400).json({
        error: "Invalid token",
      });
      return;
    } else {
      await mongoConnect();
      const user = await UserModel.findOne({
        _id: (decoded as any).id as string,
      });

      if (!user) {
        res.status(400).json({
          error: "User not found",
        });
        return;
      }

      const oldProfile = await CommunityProfileModel.findOne({
        user: user._id,
      });

      if (oldProfile) {
        res.status(400).json({
          error: "Profile already exists",
        });
        return;
      }

      const profile = await CommunityProfileModel.create({
        user: user._id,
        displayName: req.body.displayName,
        bio: req.body.bio,
        username: user.username,
      });

      res.status(200).json({
        profile: profile,
        success: true,
```

```
        });
        return;
      }
    }
  );
}
```

## apps/web/src/pages/api/community/profile/update.ts

```
/**
 * src/pages/api/community/profile/update.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 18.06.2022
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/src/mongo";
import jwt from "jsonwebtoken";
import UserModel from "@codeup/util/src/models/UserModel";
import CommunityProfileModel from "@codeup/util/src/models/CommunityProfileModel";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "POST") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  if (!req.body || !req.headers["x-jwt"] || !req.body.update) {
    res.status(400).json({
      error: "Missing required fields",
    });
    return;
  }

  jwt.verify(
    req.headers["x-jwt"] as string,
    // @ts-ignore
    process.env.JWT_SECRET,
    async (err, decoded) => {
      if (err) {
        res.status(400).json({
          error: "Invalid token",
        });
        return;
      } else {
        await mongoConnect();
        const user = await UserModel.findOne({
          _id: (decoded as any).id as string,
        });

        if (!user) {
          res.status(400).json({
```

```
          error: "User not found",
        });
        return;
      }

      const profile = await CommunityProfileModel.findOne({
        user: user._id,
      });

      if (profile) {
        const allowedUpdates = [
          "profilePicture",
          "profileBanner",
          "displayName",
          "bio",
          "website",
          "location",
          "birthday",
        ];

        Object.keys(req.body.update).forEach((key) => {
          if (allowedUpdates.includes(key)) {
            profile[key] = req.body.update[key];
          }
        });

        await profile.save();

        res.status(200).json({
          success: true,
        });
      } else {
        res.status(400).json({
          error: "Profile not found",
        });
        return;
      }
    }
  }
  );
}
```

## apps/web/src/pages/api/community/profile/[name]/avatar.ts

```
/**
 * src/pages/api/community/profile/[name]/avatar.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 19.06.2022
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/src/mongo";
import CommunityProfileModel from "@codeup/util/src/models/CommunityProfileModel";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
```

```
) {
  if (req.method !== "GET") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  const { name } = req.query;

  await mongoConnect();

  const profile = await CommunityProfileModel.findOne({
    username: name,
  });

  if (!profile) {
    res.status(404).json({
      error: "Not Found",
    });
    return;
  }

  res.redirect(profile.profilePicture);
}
```

## apps/web/src/pages/api/community/profile/[name]/followers.ts

```
/**
 * src/pages/api/community/profile/followers.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 16.06.2022
 *
 */
import { NextApiRequest, NextApiResponse } from "next";
import { default as mongoConnect } from "@codeup/util/src/mongo";
import { default as CommunityProfileModel } from "@codeup/util/src/models/CommunityProfileModel";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "GET") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  const { name, count, from, to } = req.query;

  if (!name) {
    res.status(400).json({
      error: "Missing Name",
    });
    return;
  }
```

```
  await mongoConnect();

  const profile = await CommunityProfileModel.findOne({
    username: name,
  });

  if (!profile) {
    res.status(404).json({
      error: "Not Found",
    });
    return;
  }

  if (count) {
    res.status(200).json({
      count: profile.followers.length,
      success: true,
    });
    return;
  }

  if (from && to) {
    res.status(200).json({
      followers: profile.followers.slice(from, to),
      success: true,
    });
    return;
  }

  res.status(200).json({
    followers: profile.followers,
    success: true,
  });
}
```

## apps/web/src/pages/api/community/profile/[name]/is-following/[othername].ts

```
/**
 * src/pages/api/community/profile/[name]/is-following/[othername].ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 18.06.2022
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/src/mongo";
import CommunityProfileModel from "@codeup/util/src/models/CommunityProfileModel";
import UserModel from "@codeup/util/src/models/UserModel";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "GET") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
```

```
  }

  const { name, othername } = req.query;

  await mongoConnect();

  const profile = await CommunityProfileModel.findOne({ username: name });

  if (!profile) {
    res.status(404).json({
      error: "Invalid Profile",
    });
    return;
  }

  const user = await UserModel.findOne({ username: othername });

  if (!user) {
    res.status(404).json({
      error: "Invalid User",
    });
    return;
  }

  const isFollowing = profile.followers.includes(user.username.toString());

  res.status(200).json({
    isFollowing,
    success: true,
  });
}
```

## apps/web/src/pages/api/community/profile/[name]/posts.ts

```
/**
 * src/pages/api/community/profile/[name]/posts.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 19.06.2022
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/src/mongo";
import CommunityProfileModel from "@codeup/util/src/models/CommunityProfileModel";
import CommunityPostModel from "@codeup/util/src/models/CommunityPostModel";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "GET") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  const { name, from, to, count } = req.query;
```

```
await mongoConnect();

const profile = await CommunityProfileModel.findOne({
  username: name,
});

if (!profile) {
  res.status(404).json({
    error: "Not Found",
  });
  return;
}

const posts = await CommunityPostModel.find({
  username: name,
});

if (!posts) {
  res.status(200).json({
    posts: [],
    success: true,
  });
  return;
}

if (count) {
  res.status(200).json({
    postCount: posts.length,
    success: true,
  });
  return;
}

if (from && to) {
  let f2u = parseInt(from as string);
  let t2u = parseInt(to as string);

  if (f2u > t2u) {
    f2u = t2u;
    t2u = f2u;
  }

  if (f2u < 0) {
    f2u = 0;
  }

  if (t2u > posts.length) {
    t2u = posts.length;
  }

  try {
    res.status(200).json({
      posts: posts.reverse().slice(f2u, t2u),
      success: true,
    });
    return;
  } catch (e) {
    res.status(400).json({
      error: "Invalid range",
```

```
      });
      return;
    }
  }

  res.status(200).json({
    posts: posts.reverse(),
    success: true,
  });
}
```

## apps/web/src/pages/api/community/profile/[name]/toggle-follow.ts

```
/**
 * src/pages/api/community/profile/toggle-follow.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 18.06.2022
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/src/mongo";
import jwt from "jsonwebtoken";
import UserModel from "@codeup/util/src/models/UserModel";
import CommunityProfileModel from "@codeup/util/src/models/CommunityProfileModel";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "POST") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  if (!req.headers["x-jwt"]) {
    res.status(400).json({
      error: "Missing required fields",
    });
    return;
  }

  const { name } = req.query;

  jwt.verify(
    req.headers["x-jwt"] as string,
    // @ts-ignore
    process.env.JWT_SECRET,
    async (err, decoded) => {
      await mongoConnect();

      const user = await UserModel.findById((decoded as any).id);

      if (!user) {
        res.status(400).json({
          error: "Invalid token",
```

```
      });
      return;
    }

    const profile = await CommunityProfileModel.findOne({
      username: name,
    });

    if (!profile) {
      res.status(400).json({
        error: "Profile not found",
      });
      return;
    }

    if (profile.followers.includes(user.username)) {
      profile.followers.splice(profile.followers.indexOf(user.username), 1);
    } else {
      profile.followers.push(user.username);
    }

    await profile.save();

    res.status(200).json({
      success: true,
      following: profile.followers.includes(user.username),
    });
  }
 );
}
```

## apps/web/src/pages/api/community/profile/[name].ts

```
/**
 * src/pages/api/community/profile/[id].ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 13.06.2022
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/src/mongo";
import CommunityProfileModel from "@codeup/util/src/models/CommunityProfileModel";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "GET") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  const { name, small } = req.query;

  if (!name) {
```

```
    res.status(400).json({
      error: "Missing Name",
    });
    return;
  }

  await mongoConnect();

  try {
    const profile = await CommunityProfileModel.findOne({
      username: name,
    });

    if (!profile) {
      res.status(404).json({
        error: "Not Found",
      });
      return;
    }

    if (!small) {
      res.status(200).json({
        profile,
        success: true,
      });
      return;
    }

    res.status(200).json({
      profile: {
        displayName: profile.displayName,
        bio: profile.bio,
        avatar: profile.avatar,
        username: profile.username,
      },
      success: true,
    });
    return;
  } catch (e) {
    res.status(404).json({
      error: "Not Found",
    });
    return;
  }
}
```

## apps/web/src/pages/api/community/search.ts

```
/**
 * src/pages/api/community/search.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 21.06.2022
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/src/mongo";
import CommunityPostModel from "@codeup/util/src/models/CommunityPostModel";
```

```
import CommunityProfileModel from "@codeup/util/src/models/CommunityProfileModel";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "GET") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  const { term } = req.query;

  if (!term) {
    res.status(400).json({
      error: "Missing term",
    });
    return;
  }

  await mongoConnect();

  const posts = await CommunityPostModel.find({
    content: {
      $regex: term,
      $options: "i",
    },
  });

  const profiles = await CommunityProfileModel.find({
    displayName: {
      $regex: term,
      $options: "i",
    },
    username: {
      $regex: term,
      $options: "i",
    },
  });

  res.status(200).json({
    posts: posts,
    profiles: profiles,
  });
}
```

## apps/web/src/pages/api/contact.ts

```
/**
 * apps/web/src/pages/api/contact.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 04.10.22
 *
 */
import { NextApiRequest, NextApiResponse } from "next";
import { verify } from "jsonwebtoken";
```

```
import mongoConnect from "@codeup/util/src/mongo";
import UserModel from "@codeup/util/src/models/UserModel";
import ContactModel from "@codeup/util/src/models/ContactModel";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "POST") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  if (
    !req.headers["x-jwt"] ||
    !req.body ||
    !req.body.message ||
    !req.body.title ||
    !req.body.category
  ) {
    res.status(400).json({
      error: "Bad Request",
    });
    return;
  }

  const token = req.headers["x-jwt"];

  try {
    await mongoConnect();

    if (!verify(token as string, process.env.JWT_SECRET as string)) {
      res.status(401).json({
        error: "Unauthorized",
      });
      return;
    }

    const decoded = verify(
      token as string,
      process.env.JWT_SECRET as string
    ) as any;

    const user = await UserModel.findOne({ _id: decoded.id });

    if (!user) {
      res.status(401).json({
        error: "Unauthorized",
      });
      return;
    }

    const contact = await ContactModel.create({
      username: user.username,
      message: req.body.message,
      title: req.body.title,
      category: req.body.category,
    });
```

```
    res.status(200).json({
      success: true,
      message_id: contact._id,
    });
  } catch (e) {
    res.status(500).json({
      error: "Internal Server Error",
    });
    return;
  }
}
```

## apps/web/src/pages/api/course/enrollments.ts

```
/**
 * apps/web/src/pages/api/course/enrollments.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2024 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 16.05.2024
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/dist/mongo";
import jwt from "jsonwebtoken";
import UserModel from "@codeup/util/dist/models/UserModel";
import CourseEnrollment from "@codeup/util/src/models/CourseEnrollment";

export default async function (req: NextApiRequest, res: NextApiResponse) {
  try {
    await mongoConnect();
    if (req.method !== "GET") {
      res.status(405).json({
        error: "Method not allowed",
      });
      return;
    }

    if (!req.headers["x-jwt"]) {
      res.status(404).json({
        error: "Unauthorized",
      });
      return;
    }

    jwt.verify(
      req.headers["x-jwt"] as string,
      // @ts-ignore
      process.env.JWT_SECRET as string,
      async (err, decoded) => {
        const user = await UserModel.findOne({ _id: (decoded as any).id });

        if (!user) {
          res.status(401).json({
            error: "Unauthorized",
          });
          return;
        }
```

```
        const enrollments = await CourseEnrollment.find({
          user: user._id,
        });

        for (const e of enrollments) {
          await e.populate("course");
        }

        res.status(200).json({
          enrollments,
        });
      }
    );
  } catch (e: any) {
    res.status(500).json({
      status: 500,
      error: e.message,
    });
  }
}
```

## apps/web/src/pages/api/course/list-courses.ts

```
/**
 * apps/web/src/pages/api/course/list-courses.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 13.11.2022
 *
 */

import { NextApiRequest, NextApiResponse } from "next";

import mongoConnect from "@codeup/util/src/mongo";

import CourseModel from "@codeup/util/src/models/CourseModel";
import * as jwt from "jsonwebtoken";
import UserModel from "@codeup/util/dist/models/UserModel";
import OrgModel from "@codeup/util/dist/models/OrgModel";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  try {
    await mongoConnect();

    if (!req.headers["x-jwt"]) {
      const courses = await CourseModel.find({ available: "public" });

      res.status(200).json({
        courses,
      });
      return;
    }

    jwt.verify(
      req.headers["x-jwt"] as string,
```

```
      // @ts-ignore
      process.env.JWT_SECRET as string,
      async (err, decoded) => {
        if (err) throw err;

        const user = await UserModel.findOne({ _id: (decoded as any).id });

        if (!user) {
          res.status(401).json({
            error: "Unauthorized",
          });
          return;
        }

        if (user.role === "admin") {
          const courses = await CourseModel.find({});
          res.status(200).json({
            courses,
          });
          return;
        }

        const orgs: any[] = [];

        orgs.push(...(await OrgModel.find({ owner: user._id })));

        orgs.push(
          ...(await OrgModel.find({
            members: {
              $in: [user._id.toString()],
            },
          }))
        );

        const courses: any[] = [];

        for (const o of orgs) {
          courses.push(
            await CourseModel.find({
              available: "org:" + o._id.toString(),
            })
          );
        }

        courses.push(
          ...(await CourseModel.find({
            available: "public",
          }))
        );

        res.status(200).json({
          courses: courses.flat(),
        });
      }
    );
  } catch (e) {
    res.status(500).json({ error: e });
  }
}
```

## apps/web/src/pages/api/course/[course]/certificate.ts

```
/**
 * apps/web/src/pages/api/course/[course]/certificate.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 27.11.2022
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/src/mongo";
import CourseModel from "@codeup/util/src/models/CourseModel";
import Certificate from "@codeup/util/src/models/Certificate";
import UserModel from "@codeup/util/src/models/UserModel";
import ObtainedCertificateModel from "@codeup/util/src/models/ObtainedCertificateModel";
import { verify } from "jsonwebtoken";
import puppeteer from "puppeteer";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  const { course } = req.query;

  try {
    await mongoConnect();

    const token = req.headers["x-jwt"];

    if (!token) {
      res.status(401).json({ error: "Unauthorized" });
      return;
    }

    if (!verify(token as string, process.env.JWT_SECRET as string)) {
      res.status(401).json({
        error: "Unauthorized",
      });
      return;
    }

    const decoded = verify(
      token as string,
      process.env.JWT_SECRET as string
    ) as any;

    const user = await UserModel.findOne({ username: decoded.username });

    if (!user) {
      res.status(401).json({
        error: "Unauthorized",
      });
      return;
    }

    let courseObject = await CourseModel.findById(course);

    if (!courseObject) {
```

```
  courseObject = await CourseModel.findOne({
    friendlyName: course,
  });
}

if (!courseObject) {
  res.status(404).json({
    error: "Course not found",
  });
}

let certificate = await Certificate.findOne({
  course: courseObject._id,
});

if (!certificate) {
  res.status(404).json({
    error: "Certificate not found",
  });
  return;
}

if (req.method === "GET") {
  if (req.query.onlyLookup && req.query.onlyLookup === "true") {
    res.status(200).json({
      certificate: certificate._id,
    });
    return;
  }

  let certificateObject = certificate.toObject();

  for (const step of certificateObject.steps) {
    if (step.step_type === "MULTIPLE_CHOICE") {
      for (const answer of step.multiple_choice.answers) {
        delete answer.correct;
      }
    } else if (step.step_type === "TEXT") {
      for (const a of step.text_parts) {
        if (a.type === "INPUT") {
          delete a.value;
        }
      }
    } else if (step.step_type === "CODE_CHECK") {
      delete step.code_check.correct;
    } else if (step.step_type === "CODE_COMPARE") {
      delete step.code_compare.code_a.correct;
      delete step.code_compare.code_b.correct;
    }
  }

  delete certificateObject._id;
  delete certificateObject.__v;
  delete certificateObject.minPercentage;

  res.status(200).json({ certificate: certificateObject });
} else if (req.method === "POST") {
  const { answers } = req.body;

  if (!answers) {
```

```
  res.status(400).json({
    error: "Bad Request",
  });
  return;
}

let correct = 0;
let total = 0;

for (const step of certificate.steps) {
  const index = certificate.steps.indexOf(step);

  const answer = answers.find((a: any) => a.step === index);

  if (!answer) {
    res.status(400).json({
      error: "Bad Request",
    });
    return;
  }

  if (step.step_type === "MULTIPLE_CHOICE") {
    total++;
    const selectedAnswerIndex = answer.answer;

    if (step.multiple_choice.answers[selectedAnswerIndex].correct) {
      correct++;
    }
  } else if (step.step_type === "TEXT") {
    for (const textPart of step.text_parts) {
      if (textPart.type === "INPUT") {
        total++;
        if (textPart.value === answer[textPart.id]) {
          correct++;
        }
      }
    }
  } else if (step.step_type === "CODE_CHECK") {
    total++;
    if (step.code_check.correct && answer.answer === true) {
      correct++;
    }
  } else if (step.step_type === "CODE_COMPARE") {
    total++;
    if (step.code_compare.code_a.correct && answer.answer === 0) {
      correct++;
    } else if (step.code_compare.code_b.correct && answer.answer === 1) {
      correct++;
    }
  }
}

const percentage = (correct / total) * 100;

const alreadyObtained = await ObtainedCertificateModel.findOne({
  username: user.username,
  course: courseObject._id,
});

if (percentage >= certificate.minPercentage) {
```

```
        let cert_id = "";

        if (alreadyObtained) {
          alreadyObtained.date = new Date();
          alreadyObtained.percentage = percentage;
          await alreadyObtained.save();

          cert_id = alreadyObtained._id;
        } else {
          const obtainedCertificate = await ObtainedCertificateModel.create({
            username: user.username,
            course: courseObject._id,
            name: user.firstName + " " + user.lastName,
            date: new Date(),
            percentage: percentage,
          });

          cert_id = obtainedCertificate._id;
        }

        res.status(200).json({
          success: true,
          obtainedCertificate: true,
          obtainedCertificateId: cert_id,
        });
      } else {
        if (alreadyObtained) {
          await alreadyObtained.remove();
        }

        res.status(200).json({
          success: true,
          obtainedCertificate: false,
          percentage,
        });
      }
    } else {
      res.status(405).json({
        error: "Method not allowed",
      });
    }
  } catch (e) {
    console.log(e);
    // @ts-ignore
    if (e.message) {
      // @ts-ignore
      res.status(500).json({ error: "Internal Server Error: " + e.message });
    } else {
      res.status(500).json({ error: "Unknown internal server error" });
    }
  }
}
```

## apps/web/src/pages/api/course/[course]/enrollment.ts

```
/**
 * apps/web/src/pages/api/course/[course]/enrollment.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2024 Ben Siebert. All rights reserved.
 * License: Project License
```

```
 * Created At: 16.05.2024
 *
 */

import CourseModel from "@codeup/util/src/models/CourseModel";
import CourseEnrollmentModel from "@codeup/util/src/models/CourseEnrollment";
import mongoConnect from "@codeup/util/src/mongo";
import { NextApiRequest, NextApiResponse } from "next";
import jwt from "jsonwebtoken";
import UserModel from "@codeup/util/dist/models/UserModel";
import OrgModel from "@codeup/util/dist/models/OrgModel";

export default async function (req: NextApiRequest, res: NextApiResponse) {
  const { course: courseQuery } = req.query;

  try {
    await mongoConnect();

    if (req.method === "GET") {
      let course = await CourseModel.findOne({
        friendlyName: courseQuery,
      });

      if (!course) {
        course = await CourseModel.findById(courseQuery);

        if (!course) {
          res.status(404).json({
            error: "Course not found",
          });
          return;
        }
      }

      if (!req.headers["x-jwt"]) {
        if (course.available !== "public") {
          res.status(404).json({
            error: "Course not found",
          });
          return;
        }
      }

      jwt.verify(
        req.headers["x-jwt"] as string,
        // @ts-ignore
        process.env.JWT_SECRET as string,
        async (err, decoded) => {
          const user = await UserModel.findOne({ _id: (decoded as any).id });
          if (!user) {
            res.status(401).json({
              error: "Course not found",
            });
            return;
          }

          if (!(await hasAccessToCourse(course, user))) {
            res.status(401).json({
              error: "Course not found",
            });
```

```
        return;
      }

      const enrollment = await CourseEnrollmentModel.findOne({
        course: course._id,
        user: user._id,
      });

      if (!enrollment) {
        res.status(404).json({
          error: "Not enrolled",
        });
        return;
      }

      res.status(200).json({
        success: true,
        enrolled: true,
      });
    }
  );
} else if (req.method === "POST") {
  jwt.verify(
    req.headers["x-jwt"] as string,
    // @ts-ignore
    process.env.JWT_SECRET as string,
    async (err, decoded) => {
      const user = await UserModel.findOne({ _id: (decoded as any).id });
      if (!user) {
        res.status(401).json({
          error: "Course not found",
        });
        return;
      }

      let course = await CourseModel.findOne({
        friendlyName: courseQuery,
      });

      if (!course) {
        course = await CourseModel.findById(courseQuery);

        if (!course) {
          res.status(404).json({
            error: "Course not found",
          });
          return;
        }
      }

      if (!(await hasAccessToCourse(course, user))) {
        res.status(401).json({
          error: "Course not found",
        });
        return;
      }

      const enrollment = await CourseEnrollmentModel.findOne({
        course: course._id,
        user: user._id,
```

```
        });

        if (enrollment) {
          await enrollment.deleteOne();

          res.status(200).json({
            success: true,
            enrolled: false,
          });
          return;
        }

        const enrollmentMod = await CourseEnrollmentModel.create({
          user: user._id,
          course: course._id,
        });

        res.status(200).json({
          success: true,
          enrolled: true,
        });
      }
    );
  } else {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }
 } catch (e) {
  res.status(500).json({
    error: "Internal Server Error",
  });
 }
}

async function hasAccessToCourse(course: any, user: any): Promise<boolean> {
 if (course.available === "public" || user.role === "admin") {
  return true;
 }

 if (course.available === "private") {
  return course.creator === user.username;
 }

 if (course.available.startsWith("org:")) {
  const org = await OrgModel.findOne({
    _id: course.available.split(":")[1],
  });

  if (org.owner.toString() === user._id.toString()) {
    return true;
  }

  if (org.members.includes(user._id)) {
    return true;
  }
 }

 return false;
```

```
}
```

## apps/web/src/pages/api/course/[course]/index.ts

```
/**
 * apps/web/src/pages/api/course/[course]/index.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 13.11.2022
 *
 */

import CourseModel from "@codeup/util/src/models/CourseModel";

import mongoConnect from "@codeup/util/src/mongo";

import { NextApiRequest, NextApiResponse } from "next";
import jwt from "jsonwebtoken";
import UserModel from "@codeup/util/dist/models/UserModel";
import OrgModel from "@codeup/util/dist/models/OrgModel";

export default async function (req: NextApiRequest, res: NextApiResponse) {
  const { course: courseQuery } = req.query;

  if (req.method !== "GET") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  try {
    await mongoConnect();

    let course = await CourseModel.findOne({
      friendlyName: courseQuery,
    });

    if (!course) {
      course = await CourseModel.findById(courseQuery);

      if (!course) {
        res.status(404).json({
          error: "Course not found",
        });
        return;
      }
    }

    if (!req.headers["x-jwt"]) {
      if (course.available !== "public") {
        res.status(404).json({
          error: "Course not found",
        });
        return;
      }

      res.status(200).json({
        course,
```

```
  });
  return;
}

jwt.verify(
  req.headers["x-jwt"] as string,
  // @ts-ignore
  process.env.JWT_SECRET as string,
  async (err, decoded) => {
    const user = await UserModel.findOne({ _id: (decoded as any).id });

    if (user.role === "admin" || course.available === "public") {
      res.status(200).json({
        course,
      });
      return;
    }

    if (course.available === "private") {
      if (course.creator === user.username) {
        res.status(200).json({
          course,
        });
      } else {
        res.status(404).json({
          error: "Course not found",
        });
      }
      return;
    }

    if (course.available.startsWith("org:")) {
      const org = await OrgModel.findOne({
        _id: course.available.split(":")[1],
      });

      if (org.owner.toString() === user._id.toString()) {
        res.status(200).json({
          course,
        });
        return;
      }

      if (org.members.includes(user._id)) {
        res.status(200).json({
          course,
        });
        return;
      }

      res.status(404).json({
        error: "Course not found",
      });
      return;
    }

    res.status(404).json({
      error: "Course not found",
    });
    return;
```

```
      }
    );
  } catch (e) {
    res.status(500).json({
      // @ts-ignore
      error: e.message,
    });
  }
}
```

## apps/web/src/pages/api/course/[course]/progress.ts

```
/**
 * apps/web/src/pages/api/course/[course]/progress.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2024 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 16.05.2024
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/dist/mongo";
import CourseModel from "@codeup/util/dist/models/CourseModel";
import OrgModel from "@codeup/util/dist/models/OrgModel";
import jwt from "jsonwebtoken";
import UserModel from "@codeup/util/dist/models/UserModel";
import CourseSectionProgressModel from "@codeup/util/dist/models/CourseSectionProgressModel";

export default async function (req: NextApiRequest, res: NextApiResponse) {
  try {
    const { course: courseQuery } = req.query;
    await mongoConnect();

    if (!["POST", "GET"].includes(req.method as string)) {
      res.status(405).json({
        error: "Method not allowed",
      });
      return;
    }

    if (!req.headers["x-jwt"]) {
      res.status(404).json({
        error: "Unauthorized",
      });
      return;
    }

    jwt.verify(
      req.headers["x-jwt"] as string,
      // @ts-ignore
      process.env.JWT_SECRET as string,
      async (err, decoded) => {
        const user = await UserModel.findOne({ _id: (decoded as any).id });

        if (req.method === "GET") {
          let course = await CourseModel.findOne({
            friendlyName: courseQuery,
          });
```

```
  if (!course) {
    course = await CourseModel.findById(courseQuery);

    if (!course) {
      res.status(404).json({
        error: "Course not found",
      });
      return;
    }
  }

  if (!(await hasAccessToCourse(course, user))) {
    res.status(403).json({
      error: "Forbidden",
    });
    return;
  }

  const progress = await CourseSectionProgressModel.find({
    user: user._id,
    course: course._id,
  });

  res.status(200).json({
    status: 200,
    progress: [
      ...progress.map((p) => {
        return p.section;
      }),
    ],
  });
  return;
}

if (req.method === "POST") {
  let course = await CourseModel.findOne({
    friendlyName: courseQuery,
  });

  if (!course) {
    course = await CourseModel.findById(courseQuery);

    if (!course) {
      res.status(404).json({
        error: "Course not found",
      });
      return;
    }
  }

  if (!(await hasAccessToCourse(course, user))) {
    res.status(403).json({
      error: "Forbidden",
    });
    return;
  }

  const { section } = req.body;

  const progress = await CourseSectionProgressModel.findOne({
```

```
            user: user._id,
            course: course._id,
            section,
          });

          if (!progress) {
            await CourseSectionProgressModel.create({
              user: user._id,
              course: course._id,
              section,
            });
          }

          res.status(200).json({
            status: 200,
            message: "Progress saved",
          });
          return;
        }
      }
    );
  } catch (e: any) {
    res.status(500).json({
      status: 500,
      error: e.message,
    });
  }
}

async function hasAccessToCourse(course: any, user: any): Promise<boolean> {
  if (course.available === "public" || user.role === "admin") {
    return true;
  }

  if (course.available === "private") {
    return course.creator === user.username;
  }

  if (course.available.startsWith("org:")) {
    const org = await OrgModel.findOne({
      _id: course.available.split(":")[1],
    });

    if (org.owner.toString() === user._id.toString()) {
      return true;
    }

    if (org.members.includes(user._id)) {
      return true;
    }
  }

  return false;
}
```

## apps/web/src/pages/api/course/[course]/[section].ts

```
/**
 * apps/web/src/pages/api/course/[course]/[section].ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
```

```
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 05.11.2022
 *
 */

import CourseModel from "@codeup/util/src/models/CourseModel";
import ContentModel from "@codeup/util/src/models/ContentModel";
import mongoConnect from "@codeup/util/src/mongo";
import { NextApiRequest, NextApiResponse } from "next";
import jwt from "jsonwebtoken";
import UserModel from "@codeup/util/dist/models/UserModel";
import OrgModel from "@codeup/util/dist/models/OrgModel";

export default async function (req: NextApiRequest, res: NextApiResponse) {
  const { course: courseQuery, section } = req.query;

  if (req.method !== "GET") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  try {
    await mongoConnect();

    let course = await CourseModel.findOne({
      friendlyName: courseQuery,
    });

    if (!course) {
      course = await CourseModel.findById(courseQuery);

      if (!course) {
        res.status(404).json({
          error: "Course not found",
        });
        return;
      }
    }

    let content = await ContentModel.findOne({
      course: course._id,
      url: section,
    });

    if (!content) {
      content = await ContentModel.findById(section);

      if (!content) {
        res.status(404).json({
          error: "Course not found",
        });
        return;
      }
    }

    if (!req.headers["x-jwt"]) {
      if (course.available !== "public") {
```

```
    res.status(404).json({
      error: "Course not found",
    });
    return;
  }

  sendResponse(req, res, content);
  return;
}

jwt.verify(
  req.headers["x-jwt"] as string,
  // @ts-ignore
  process.env.JWT_SECRET as string,
  async (err, decoded) => {
    const user = await UserModel.findOne({ _id: (decoded as any).id });

    if (user.role === "admin" || course.available === "public") {
      sendResponse(req, res, content);
      return;
    }

    if (course.available === "private") {
      if (course.creator === user.username) {
        sendResponse(req, res, content);
      } else {
        res.status(404).json({
          error: "Course not found",
        });
      }
      return;
    }

    if (course.available.startsWith("org:")) {
      const org = await OrgModel.findOne({
        _id: course.available.split(":")[1],
      });

      if (org.owner.toString() === user._id.toString()) {
        sendResponse(req, res, content);
        return;
      }

      if (org.members.includes(user._id)) {
        sendResponse(req, res, content);
        return;
      }

      res.status(404).json({
        error: "Course not found",
      });
      return;
    }

    res.status(404).json({
      error: "Course not found",
    });
    return;
  }
);
```

```
  } catch (e) {
    res.status(500).json({
      error: "Internal Server Error",
    });
  }
}

function sendResponse(req: any, res: any, content: any) {
  if (req.query.raw) {
    res.status(200).end(content.textContent);
    return;
  }
  res.status(200).json({
    content: content,
    success: true,
  });
}
```

## apps/web/src/pages/api/creator/certificate.ts

```
/**
 * apps/web/src/pages/api/creator/certificates.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 26.11.2022
 *
 */

import { NextApiRequest, NextApiResponse } from "next";

import {
  CertificateBuilder,
  MultipleChoiceBuilder,
} from "@codeup/util/src/certificates/CertificateBuilder";

import CertificateModel from "@codeup/util/src/models/Certificate";
import CourseModel from "@codeup/util/src/models/CourseModel";
import UserModel from "@codeup/util/src/models/UserModel";
import mongoConnect from "@codeup/util/src/mongo";
import { verify } from "jsonwebtoken";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (!["POST", "GET", "DELETE", "PATCH"].includes(req.method as string)) {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  const token = req.headers["x-jwt"];

  if (!token) {
    res.status(401).json({ error: "Unauthorized" });
    return;
  }
```

```
try {
  if (!verify(token as string, process.env.JWT_SECRET as string)) {
    res.status(401).json({
      error: "Unauthorized",
    });
    return;
  }

  const decoded = verify(
    token as string,
    process.env.JWT_SECRET as string
  ) as any;

  await mongoConnect();

  const editorUser = await UserModel.findOne({ username: decoded.username });

  if (!editorUser) {
    res.status(401).json({
      error: "Unauthorized",
    });
    return;
  }

  /* -- REMOVE CREATOR LOCK -- */

  /*
  if (!["editor", "admin"].includes(editorUser.role)) {
    res.status(401).json({
      error: "Unauthorized",
    });
    return;
  }
  */

  if (req.method === "GET") {
    const { courseId } = req.query;

    if (!courseId) {
      res.status(400).json({
        error: "Bad Request",
      });
      return;
    }

    const course = await CourseModel.findById(courseId);

    if (!course) {
      res.status(404).json({
        error: "Not Found",
      });
      return;
    }

    const certificate = await CertificateModel.findOne({ course: courseId });

    if (!certificate) {
      res.status(404).json({
        error: "Not Found",
      });
```

```
    return;
  }

  res.status(200).json({ certificate });
} else if (req.method === "POST") {
  if (req.body.create) {
    const { courseId } = req.body;

    if (!courseId) {
      res.status(400).json({
        error: "Bad Request",
      });
      return;
    }

    const course = await CourseModel.findById(courseId);

    if (!course) {
      res.status(404).json({
        error: "Not Found",
      });
      return;
    }

    const exisiting = await CertificateModel.findOne({
      course: courseId,
    });

    if (exisiting) {
      res.status(200).json({ certificate: exisiting });
      return;
    }

    const cert = await CertificateModel.create({
      course: courseId,
      minPercentage: 75,
      steps: [],
    });

    res.status(200).json({ certificate: cert });

    return;
  }

  const { courseId, update } = req.body;

  if (!courseId || !update) {
    res.status(400).json({
      error: "Bad Request",
    });
    return;
  }

  const course = await CourseModel.findById(courseId);

  if (!course) {
    res.status(404).json({
      error: "Not Found",
    });
    return;
```

```
      }

      const certificate = await CertificateModel.findOne({ course: courseId });

      if (!certificate) {
        res.status(400).json({
          error: "Bad Request",
        });
        return;
      }

      if (update.minPercentage) {
        if (update.minPercentage < 0 || update.minPercentage > 100) {
          res.status(400).json({
            error: "Bad Request",
          });
          return;
        }
        certificate.minPercentage = update.minPercentage;
      }

      if (update.steps) {
        certificate.steps = update.steps;
      }

      await certificate.save();

      res.status(200).json({ certificate });
    }
  } catch (e) {
    // @ts-ignore
    if (e.message) {
      // @ts-ignore
      res.status(500).json({ error: e.message });
    } else {
      res
        .status(500)
        .json({ error: "Internal Server Error with no specific message" });
    }
  }
}
```

## apps/web/src/pages/api/creator/content.ts

```
/**
 * apps/web/src/pages/api/creator/content.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 18.11.2022
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import UserModel from "@codeup/util/src/models/UserModel";
import ContentModel from "@codeup/util/src/models/ContentModel";
import CourseModel from "@codeup/util/src/models/CourseModel";
import mongoConnect from "@codeup/util/src/mongo";
import { verify } from "jsonwebtoken";
```

```
export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (!["POST", "GET", "DELETE", "PATCH"].includes(req.method as string)) {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  const token = req.headers["x-jwt"];

  if (!token) {
    res.status(401).json({ error: "Unauthorized" });
    return;
  }

  try {
    if (!verify(token as string, process.env.JWT_SECRET as string)) {
      res.status(401).json({
        error: "Unauthorized",
      });
      return;
    }

    const decoded = verify(
      token as string,
      process.env.JWT_SECRET as string
    ) as any;

    await mongoConnect();

    const editorUser = await UserModel.findOne({ username: decoded.username });

    if (!editorUser) {
      res.status(401).json({
        error: "Unauthorized",
      });
      return;
    }
    /* -- REMOVE CREATOR LOCK -- */

    /*
    if (!["editor", "admin"].includes(editorUser.role)) {
      res.status(401).json({
        error: "Unauthorized",
      });
      return;
    }
    */

    if (req.method === "GET") {
      if (req.query.course) {
        const contents = await ContentModel.find({
          creator: editorUser.username,
          course: req.query.course,
        });

        res.status(200).json({
```

```
      contents,
    });
  } else {
    if (req.query.contentId) {
      const content = await ContentModel.findById(req.query.contentId);

      if (content) {
        res.status(200).json({
          content,
        });
        return;
      }

      res.status(404).json({
        error: "Content not found",
      });
      return;
    }

    const contents = await ContentModel.find({
      creator: editorUser.username,
    });

    res.status(200).json({
      contents,
    });
  }
} else if (req.method === "POST") {
  const { displayName, course, url, type, textContent, quiz, videoUrl } =
    req.body;

  if (!displayName || !course || !url || !type) {
    res.status(400).json({
      error: "Bad Request",
      success: false,
    });
    return;
  }

  if (!textContent && !videoUrl) {
    res.status(400).json({
      error: "Bad Request",
      success: false,
    });
    return;
  }

  const courseModel = await CourseModel.findById(course);

  if (!courseModel) {
    res.status(404).json({
      error: "Not Found",
      success: false,
    });
    return;
  }

  if (courseModel.creator !== editorUser.username) {
    res.status(401).json({
      error: "Unauthorized",
```

```
      success: false,
    });
    return;
  }

  const content = await ContentModel.create({
    displayName,
    course,
    url,
    type,
    textContent,
    quiz,
    videoUrl: videoUrl ? videoUrl : undefined,
    creator: editorUser.username,
  });

  courseModel.contentPositions.push(content._id);

  await courseModel.save();

  res.status(200).json({
    content,
    success: true,
  });
} else if (req.method === "DELETE") {
  const { id } = req.body;

  if (!id) {
    res.status(400).json({
      error: "Bad Request",
      success: false,
    });
    return;
  }

  const content = await ContentModel.findById(id);

  if (!content) {
    res.status(404).json({
      error: "Not Found",
      success: false,
    });
    return;
  }

  if (content.creator !== editorUser.username) {
    res.status(401).json({
      error: "Unauthorized",
      success: false,
    });
    return;
  }

  const course = await CourseModel.findById(content.course);

  if (!course) {
    res.status(404).json({
      error: "Not Found",
      success: false,
    });
```

```
    return;
  }

  if (course.creator !== editorUser.username) {
    res.status(401).json({
      error: "Unauthorized",
      success: false,
    });
    return;
  }

  course.contentPositions = course.contentPositions.filter(
    (c: string) => c !== req.body.id
  );

  await course.save();

  await content.remove();

  res.status(200).json({
    success: true,
  });
} else if (req.method === "PATCH") {
  const { id, update } = req.body;

  if (!id || !update) {
    res.status(400).json({
      error: "Bad Request",
      success: false,
    });
    return;
  }

  const content = await ContentModel.findById(id);

  if (!content) {
    res.status(404).json({
      error: "Not Found",
      success: false,
    });
    return;
  }

  if (content.creator !== editorUser.username) {
    res.status(401).json({
      error: "Unauthorized",
      success: false,
    });
    return;
  }

  for (const key in update) {
    content[key] = update[key];
  }

  await content.save();

  res.status(200).json({
    success: true,
    content,
```

```
      });
    }
  } catch (e) {
    // @ts-ignore
    if (e.message) {
      // @ts-ignore
      res.status(500).json({ error: e.message });
    } else {
      res
        .status(500)
        .json({ error: "Internal Server Error with no specific message" });
    }
  }
}
```

## apps/web/src/pages/api/creator/course.ts

```
/**
 * apps/web/src/pages/api/creator/course.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 18.11.2022
 *
 */

import UserModel from "@codeup/util/src/models/UserModel";
import ContentModel from "@codeup/util/src/models/ContentModel";
import CourseModel from "@codeup/util/src/models/CourseModel";
import mongoConnect from "@codeup/util/src/mongo";
import { NextApiRequest, NextApiResponse } from "next";
import { verify } from "jsonwebtoken";
import OrgModel from "@codeup/util/dist/models/OrgModel";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (!["GET", "POST", "DELETE", "PATCH"].includes(req.method as string)) {
    res.status(405).json({ error: "Method not allowed" });
    return;
  }

  const token = req.headers["x-jwt"];

  if (!token) {
    res.status(401).json({ error: "Unauthorized" });
    return;
  }

  try {
    if (!verify(token as string, process.env.JWT_SECRET as string)) {
      res.status(401).json({
        error: "Unauthorized",
      });
      return;
    }

    const decoded = verify(
      token as string,
```

```
  process.env.JWT_SECRET as string
) as any;

await mongoConnect();

const editorUser = await UserModel.findOne({ username: decoded.username });

if (!editorUser) {
  res.status(401).json({
    error: "Unauthorized",
  });
  return;
}

/* -- REMOVE CREATOR LOCK -- */
/*
if (!["editor", "admin"].includes(editorUser.role)) {
  res.status(401).json({
    error: "Unauthorized",
  });
  return;
}
*/

if (req.method === "GET") {
  const courses = await CourseModel.find({
    creator: editorUser.username,
  });

  res.status(200).json({ courses });
  return;
} else if (req.method === "POST") {
  const { name, friendlyName, description, splashImage } = req.body;

  if (!name || !friendlyName || !description || !splashImage) {
    res.status(400).json({
      error: "Bad Request",
    });
    return;
  }

  const existingCourse = await CourseModel.findOne({
    friendlyName: friendlyName,
  });

  if (existingCourse) {
    res.status(400).json({
      error: "URL freundlicher Name bereits vergeben",
    });
    return;
  }

  const course = await CourseModel.create({
    name: name,
    friendlyName: friendlyName,
    description: description,
    splashImage: splashImage,
    available: "private",
    creator: editorUser.username,
  });
```

```
  res.status(200).json({
    course: course,
  });
} else if (req.method === "DELETE") {
  const { id } = req.body;

  if (!id) {
    res.status(400).json({
      error: "Bad Request",
    });
    return;
  }

  const course = await CourseModel.findOne({ _id: id });

  if (!course) {
    res.status(400).json({
      error: "Bad Request",
    });
    return;
  }

  if (course.creator !== editorUser.username) {
    res.status(401).json({
      error: "Unauthorized",
    });
    return;
  }

  const contents = await ContentModel.find({ course: course._id });

  for (const content of contents) {
    content.remove();
  }

  await course.remove();

  res.status(200).json({
    message: "OK",
  });
} else if (req.method === "PATCH") {
  const { id } = req.body;

  if (!id) {
    res.status(400).json({
      error: "Bad Request",
    });
    return;
  }

  const course = await CourseModel.findOne({ _id: id });

  if (!course) {
    res.status(400).json({
      error: "Bad Request",
    });
  }

  if (course.creator !== editorUser.username) {
```

```
        res.status(401).json({
          error: "Unauthorized",
        });
        return;
      }

      const disallowed = ["_id", "createdAt", "creator"];

      const { update } = req.body;

      for (const key in update) {
        if (key === "available") {
          const permissionLevel = !["editor", "admin"].includes(editorUser.role)
            ? 0
            : 1;

          if (update[key] === "public" && permissionLevel === 0) {
            continue;
          }

          if (update[key].startsWith("org:")) {
            const org = await OrgModel.findById(
              update[key].replace("org:", "")
            );

            if (!org) {
              continue;
            }

            if (org.owner.toString() !== editorUser._id.toString()) {
              continue;
            }
          }
        }

        if (!disallowed.includes(key)) {
          course[key] = update[key];
        }
      }

      await course.save();

      res.status(200).json({
        message: "OK",
        course,
      });
    }
  } catch (e) {
    // @ts-ignore
    if (e.message) {
      // @ts-ignore
      res.status(500).json({ error: e.message });
    } else {
      res
        .status(500)
        .json({ error: "Internal Server Error with no specific message" });
    }
  }
}
```

## apps/web/src/pages/api/creator/top.ts

```
/**
 * apps/web/src/pages/api/creator/top.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 19.11.2022
 *
 */

import mongoConnect from "@codeup/util/src/mongo";
import { NextApiRequest, NextApiResponse } from "next";
import CourseModel from "@codeup/util/src/models/CourseModel";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  try {
    await mongoConnect();

    const creators = await CourseModel.aggregate([
      {
        $match: {
          available: "public",
        },
      },
      {
        $group: {
          _id: "$creator",
          count: { $sum: 1 },
        },
      },
      { $sort: { count: -1 } },
      { $limit: 5 },
    ]);

    res.status(200).json({ creators });
  } catch (e) {
    // @ts-ignore
    if (e.message) {
      // @ts-ignore
      res.status(500).json({ error: e.message });
    } else {
      res.status(500).json({ error: "An unknown error occurred" });
    }
  }
}
```

## apps/web/src/pages/api/discovery/comment.ts

```
/**
 * apps/web/src/pages/api/discovery/comment.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 25.03.2023
 *
 */
```

```
import { NextApiRequest, NextApiResponse } from "next";
import IDEProjectModel from "@codeup/util/src/models/IDEProjectModel";
import UserModel from "@codeup/util/dist/models/UserModel";
import EditorProjectModel from "@codeup/util/dist/models/EditorProjectModel";
import mongoConnect from "@codeup/util/dist/mongo";
import jwt from "jsonwebtoken";

export default async (req: NextApiRequest, res: NextApiResponse) => {
  if (req.method !== "POST") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  if (!req.headers["x-jwt"]) {
    res.status(401).json({
      error: "Unauthorized",
    });
    return;
  }

  if (!req.body || !req.body.projectId || !req.body.comment) {
    res.status(400).json({
      error: "Bad Request",
    });
    return;
  }

  try {
    await mongoConnect();

    jwt.verify(
      req.headers["x-jwt"] as string,
      // @ts-ignore
      process.env.JWT_SECRET,
      async (err: any, decoded: any) => {
        if (err) throw err;

        const user = await UserModel.findById((decoded as any).id);

        if (!user) {
          res.status(401).json({
            error: "Unauthorized",
          });
          return;
        }

        const project = await EditorProjectModel.findOne({
          _id: req.body.projectId,
        });

        if (!project) {
          res.status(404).json({
            error: "Not Found",
          });
          return;
        }
```

```
        if (!project.discovery) {
          res.status(403).json({
            error: "Forbidden",
          });
          return;
        }

        project.discoveryComments.push({
          username: user.username,
          comment: req.body.comment,
          created_at: new Date(),
        });

        await project.save();

        res.status(200).json({
          success: true,
        });
      }
    );
  } catch (e: any) {
    res.status(500).json({ error: e.message, success: false });
  }
};
```

## apps/web/src/pages/api/discovery/index.ts

```
/**
 * apps/web/src/pages/api/discovery/index.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 25.03.2023
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import IDEProjectModel from "@codeup/util/src/models/IDEProjectModel";
import UserModel from "@codeup/util/dist/models/UserModel";
import EditorProjectModel from "@codeup/util/dist/models/EditorProjectModel";

export default async (req: NextApiRequest, res: NextApiResponse) => {
  if (req.method === "GET") {
    const projects = await EditorProjectModel.find({
      discovery: true,
    });

    let p: any[] = [];

    for (const project of projects) {
      const user = await UserModel.findById(project.owner);

      p.push({
        name: project.name,
        description: project.discoveryDescription,
        author: user.username,
        domain: project.customDomain,
        id: project._id,
        stars: project.discoveryLikes,
      });
```

```
    }

    p = p.sort((a, b) => {
      return b.stars - a.stars;
    });

    res.status(200).json({ projects: p });
  } else {
    res.status(404).json({ error: "Not Found" });
  }
};
```

## apps/web/src/pages/api/discovery/star.ts

```
/**
 * apps/web/src/pages/api/discovery/star.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 25.03.2023
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import IDEProjectModel from "@codeup/util/src/models/IDEProjectModel";
import UserModel from "@codeup/util/dist/models/UserModel";
import EditorProjectModel from "@codeup/util/dist/models/EditorProjectModel";
import mongoConnect from "@codeup/util/dist/mongo";
import jwt from "jsonwebtoken";

export default async (req: NextApiRequest, res: NextApiResponse) => {
  if (req.method !== "POST") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  if (!req.headers["x-jwt"]) {
    res.status(401).json({
      error: "Unauthorized",
    });
    return;
  }

  if (!req.body || !req.body.projectId) {
    res.status(400).json({
      error: "Bad Request",
    });
    return;
  }

  try {
    await mongoConnect();

    jwt.verify(
      req.headers["x-jwt"] as string,
      // @ts-ignore
      process.env.JWT_SECRET,
      async (err: any, decoded: any) => {
```

```
        if (err) throw err;

        const user = await UserModel.findById((decoded as any).id);

        if (!user) {
          res.status(401).json({
            error: "Unauthorized",
          });
          return;
        }

        const project = await EditorProjectModel.findOne({
          _id: req.body.projectId,
        });

        if (!project) {
          res.status(404).json({
            error: "Not Found",
          });
          return;
        }

        if (!project.discovery) {
          res.status(403).json({
            error: "Forbidden",
          });
          return;
        }

        if (project.discoveryLikes.includes(user._id.toString())) {
          project.discoveryLikes = project.discoveryLikes.filter(
            (id: any) => id.toString() !== user._id.toString()
          );
        } else {
          project.discoveryLikes.push(user._id);
        }

        await project.save();

        res.status(200).json({
          success: true,
          stars: project.discoveryLikes,
        });
      }
    );
  } catch (e: any) {
    res.status(500).json({ error: e.message, success: false });
  }
};
```

## apps/web/src/pages/api/editor/projects.ts

```
/**
 * apps/web/src/pages/api/editor/projects.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 18.03.2023
 *
 */
```

```
import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/dist/mongo";
import EditorProjectModel from "@codeup/util/dist/models/EditorProjectModel";
import UserModel from "@codeup/util/dist/models/UserModel";
import jwt from "jsonwebtoken";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (!req.headers["x-jwt"]) {
    res.status(401).json({ error: "Not authorized", success: false });
    return;
  }

  try {
    const decoded = jwt.verify(
      req.headers["x-jwt"] as string,
      process.env.JWT_SECRET as string
    );

    if (!decoded) {
      res.status(401).json({ error: "Not authorized", success: false });
      return;
    }

    await mongoConnect();

    const user = await UserModel.findOne({ _id: (decoded as any).id });

    if (!user) {
      res.status(401).json({ error: "Not authorized", success: false });
      return;
    }

    const projects = await EditorProjectModel.find({ owner: user._id });

    res.status(200).json({ success: true, projects });
  } catch (e: any) {
    res.status(500).json({ error: e.message, success: false });
  }
}
```

## apps/web/src/pages/api/editor/published-domain.ts

```
/**
 * apps/web/src/pages/api/editor/published-domain.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 02.07.2023
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/dist/mongo";
import EditorProjectModel from "@codeup/util/dist/models/EditorProjectModel";
import UserModel from "@codeup/util/dist/models/UserModel";
import jwt from "jsonwebtoken";
```

```
export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (!req.query.id) {
    res.status(400).json({ error: "No id given", success: false });
    return;
  }

  try {
    await mongoConnect();

    const project = await EditorProjectModel.findById(req.query.id as string);

    if (!project || project.customDomain.startsWith("disabled:")) {
      res.status(404).json({ error: "Not found", success: false });
      return;
    }

    res.status(200).json({
      success: true,
      project: {
        name: project.name,
        domain: project.customDomain,
      },
    });
  } catch (e: any) {
    res.status(500).json({ error: e.message, success: false });
  }
}
```

## apps/web/src/pages/api/forum/chats/index.ts

```
/**
 * apps/web/src/pages/api/forum/chats/index.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 24.09.2022
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import { verify } from "jsonwebtoken";
import ForumProfileModel from "@codeup/util/src/models/ForumProfileModel";
import ForumChatModel from "@codeup/util/src/models/ForumChatModel";
import UserModel from "@codeup/util/src/models/UserModel";
import mongoConnect from "@codeup/util/src/mongo";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (
    req.method !== "GET" &&
    req.method !== "POST" &&
    req.method !== "DELETE"
  ) {
    res.status(405).json({
```

```
    error: "Method not allowed",
  });
  return;
}

if (!req.headers["x-jwt"]) {
  res.status(401).json({
    error: "Unauthorized",
  });
  return;
}

const token = req.headers["x-jwt"];

if (!verify(token as string, process.env.JWT_SECRET as string)) {
  res.status(401).json({
    error: "Unauthorized",
  });
  return;
}

const username = (
  verify(token as string, process.env.JWT_SECRET as string) as any
).username;

try {
  await mongoConnect();

  if (req.method === "GET") {
    const chats = await ForumChatModel.find({
      users: {
        $in: [username],
      },
    });
    const filteredChats: any[] = [];

    for (const chat of chats) {
      let users = [];
      for (const username0 of chat.users) {
        const profile = await ForumProfileModel.findOne({
          username: username0,
        });

        users.push({
          username: username0,
          displayName: profile.displayName,
          verified: profile.verified,
        });
      }
      filteredChats.push({
        id: chat._id,
        users,
      });
    }

    res.status(200).json({
      chats: filteredChats,
    });

    return;
```

```
} else if (req.method === "POST") {
  if (!req.body || !req.body.user) {
    res.status(400).json({
      error: "Bad Request",
    });
    return;
  }

  if (req.body.user === username) {
    res.status(400).json({
      error: "Bad Request",
    });
    return;
  }

  const user = await UserModel.findOne({
    username: req.body.user,
  });

  if (!user) {
    res.status(400).json({
      error: "Bad Request",
    });
    return;
  }

  const chat = await ForumChatModel.findOne({
    users: {
      $all: [username, req.body.user],
    },
  });

  if (chat) {
    res.status(200).json({
      chat,
    });
    return;
  }

  const newChat = await ForumChatModel.create({
    users: [username, req.body.user],
  });

  await newChat.save();

  res.status(200).json({
    chat: newChat,
  });
} else if (req.method === "DELETE") {
  if (!req.body || !req.body.chat) {
    res.status(400).json({
      error: "Bad Request",
    });
    return;
  }

  const chat = await ForumChatModel.findOne({
    _id: req.body.chat,
  });
```

```
      if (!chat) {
        res.status(400).json({
          error: "Bad Request",
        });
        return;
      }

      if (!chat.users.includes(username)) {
        res.status(400).json({
          error: "Bad Request",
        });
        return;
      }

      await ForumChatModel.deleteOne({
        _id: req.body.chat,
      });

      res.status(200).json({
        success: true,
      });
    }
  } catch (e) {
    res.status(500).json({
      // @ts-ignore
      error: e.message,
    });
  }
}
```

## apps/web/src/pages/api/forum/chats/[id]/messages.ts

```
/**
 * apps/web/src/pages/api/forum/chats/[id]/messages.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 24.09.2022
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import { verify } from "jsonwebtoken";
import ForumProfileModel from "@codeup/util/src/models/ForumProfileModel";
import ForumChatModel from "@codeup/util/src/models/ForumChatModel";
import UserModel from "@codeup/util/src/models/UserModel";
import mongoConnect from "@codeup/util/src/mongo";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "GET" && req.method !== "POST") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  if (!req.headers["x-jwt"]) {
```

```
  res.status(401).json({
    error: "Unauthorized",
  });
  return;
}

const token = req.headers["x-jwt"];

const id = req.query.id as string;

if (!verify(token as string, process.env.JWT_SECRET as string)) {
  res.status(401).json({
    error: "Unauthorized",
  });
  return;
}

const username = (
  verify(token as string, process.env.JWT_SECRET as string) as any
).username;

try {
  await mongoConnect();

  if (req.method === "GET") {
    const chat = await ForumChatModel.findOne({
      _id: id,
    });

    if (!chat) {
      res.status(404).json({
        error: "Not found",
      });
      return;
    }

    if (!chat.users.includes(username)) {
      res.status(403).json({
        error: "Forbidden",
      });
      return;
    }

    res.status(200).json({
      messages: chat.messages,
    });

    return;
  } else if (req.method === "POST") {
    if (!req.body || !req.body.content) {
      res.status(400).json({
        error: "Bad Request",
      });
      return;
    }

    const chat = await ForumChatModel.findOne({
      _id: id,
    });
```

```
      if (!chat) {
        res.status(404).json({
          error: "Not found",
        });
        return;
      }

      if (!chat.users.includes(username)) {
        res.status(403).json({
          error: "Forbidden",
        });
        return;
      }

      chat.messages.push({
        content: req.body.content,
        createdAt: new Date(),
        username: username,
      });

      await chat.save();

      res.status(200).json({
        messages: chat.messages,
      });
    }
  } catch (e) {
    res.status(500).json({
      // @ts-ignore
      error: e.message,
    });
  }
}
```

## apps/web/src/pages/api/forum/embeds.ts

```
/**
 * apps/web/src/pages/api/forum/embeds.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 30.12.2022
 *
 */
import { NextApiRequest, NextApiResponse } from "next";
import ogs from "open-graph-scraper";

export default async function handler(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (!req.body) {
    res.status(400).send("No body provided");
    return;
  }

  const links = req.body.links;

  if (!links) {
    res.status(400).send("No links provided");
```

```
    return;
  }

  const embeds: {
    type: "file" | "image" | "video" | "audio" | "link";
    url?: string;
    title?: string;
    description?: string;
    image?: string;
    site_name?: string;
  }[] = [];

  for (const link of links) {
    const headRes = await fetch(link, {
      method: "HEAD",
    });

    const contentType = ((headRes.headers.get("Content-Type") as string) || "")
      .toLowerCase()
      .split(";")[0];

    if (contentType === "text/html") {
      const o = await ogs({ url: link });
      if (!o.error) {
        embeds.push({
          title: (o.result as any).ogTitle as string,
          description: (o.result as any).ogDescription as string,
          image: (o.result as any).ogImage.url as string,
          url: (o.result as any).ogUrl as string,
          site_name: (o.result as any).ogSiteName as string,
          type: "link",
        });
      }
    } else if (contentType.startsWith("image/")) {
      embeds.push({
        url: link,
        type: "image",
      });
    } else if (contentType.startsWith("video/")) {
      embeds.push({
        url: link,
        type: "video",
      });
    } else {
      embeds.push({
        url: link,
        type: "file",
      });
    }
  }

  res.status(200).send(embeds);
}
```

## apps/web/src/pages/api/forum/post/all.ts

```
/**
 * apps/web/src/pages/api/forum/post/all.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
```

```
 * License: Project License
 * Created At: 24.09.2022
 *
 */
import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/src/mongo";
import ForumPostModel from "@codeup/util/src/models/ForumPostModel";

export default async function handler(
  req: NextApiRequest,
  res: NextApiResponse
) {
  const { id } = req.query;

  try {
    await mongoConnect();

    const posts = await ForumPostModel.find();

    res.status(200).json({
      posts,
    });
  } catch (e) {
    res.status(500).json({
      error: e,
    });
  }
}
```

## apps/web/src/pages/api/forum/post/index.ts

```
/**
 * apps/web/src/pages/api/forum/post/index.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 24.09.2022
 *
 */
import mongoConnect from "@codeup/util/src/mongo";
import { NextApiRequest, NextApiResponse } from "next";
import { decode, verify } from "jsonwebtoken";
import UserModel from "@codeup/util/src/models/UserModel";
import ForumPostModel from "@codeup/util/src/models/ForumPostModel";
import ogs from "open-graph-scraper";
import updateForumBadges from "@codeup/util/src/updateForumBadges";

export default async function handler(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "POST") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  if (!req.headers["x-jwt"]) {
    res.status(401).json({
```

```
    error: "Unauthorized",
  });
  return;
}

if (!req.body || !req.body.content) {
  res.status(400).json({
    error: "Bad request",
  });
  return;
}

try {
  await mongoConnect();

  if (
    !verify(req.headers["x-jwt"] as string, process.env.JWT_SECRET as string)
  ) {
    res.status(401).json({
      error: "Unauthorized",
    });
    return;
  }

  const decoded = decode(req.headers["x-jwt"] as string);

  const user = await UserModel.findOne({
    // @ts-ignore
    username: decoded.username,
  });

  if (!user) {
    res.status(401).json({
      error: "Unauthorized",
    });
    return;
  }

  if (req.body.parent) {
    const parent = await ForumPostModel.findById(req.body.parent);
    if (!parent) {
      res.status(400).json({
        error: "Bad request",
      });
      return;
    }
    const post = await ForumPostModel.create({
      user: user._id,
      username: user.username,
      content: req.body.content,
      parent: parent._id,
      embeds: await getEmbeds(req.body.content),
    });

    await post.save();

    res.status(200).json({
      post,
    });
    await updateForumBadges(user.username);
```

```
      return;
    }

    const post = await ForumPostModel.create({
      user: user._id,
      username: user.username,
      content: req.body.content,
      embeds: await getEmbeds(req.body.content),
    });

    await post.save();

    res.status(200).json({
      post,
    });

    await updateForumBadges(user.username);
  } catch (e) {
    res.status(500).json({
      error: e,
    });
  }
}

async function getEmbeds(content: string): Promise<any[]> {
  const embeds: {
    type: "file" | "image" | "video" | "audio" | "link";
    url?: string;
    title?: string;
    description?: string;
    image?: string;
    site_name?: string;
  }[] = [];

  const links = content.match(/(https?:\/\/[^\s]+)/g) || [];

  for (const link of links) {
    if (link.includes("https://codeup.space/project/")) {
      const name = link.split("https://codeup.space/project/")[1].split("/")[0];

      embeds.push({
        type: "link",
        url: link,
        title: "CodeUp Projekt " + name,
        description:
          "Dies ist ein Link, der zu einem Projekt führt, welches auf CodeUp veröffentlicht wurde.",
        image:

"https://camo.githubusercontent.com/457388f99649600f589dcb47b8dce5996d51153bc7e2b095d111
a0ba08e7567a/68747470733a2f2f7374617469632e636f6e74656e742e62656e2d736965626572742e
64652f6170692f7075626c69632f646c2f74494d4732635a543f696e6c696e653d74727565",
        site_name: name,
      });
    } else {
      try {
        const headRes = await fetch(link, {
          method: "HEAD",
        });

        const contentType = (
```

```
          (headRes.headers.get("Content-Type") as string) || ""
        )
          .toLowerCase()
          .split(";")[0];

        if (contentType === "text/html") {
          const o = await ogs({ url: link });
          if (!o.error) {
            embeds.push({
              title: (o.result as any).ogTitle as string,
              description: (o.result as any).ogDescription as string,
              image: (o.result as any).ogImage.url as string,
              url: (o.result as any).ogUrl as string,
              site_name: (o.result as any).ogSiteName as string,
              type: "link",
            });
          }
        } else if (contentType.startsWith("image/")) {
          embeds.push({
            url: link,
            type: "image",
          });
        } else if (contentType.startsWith("video/")) {
          embeds.push({
            url: link,
            type: "video",
          });
        } else {
          embeds.push({
            url: link,
            type: "file",
          });
        }
      } catch (e) {
        console.log(e);
      }
    }
  }

  return embeds;
}
```

## apps/web/src/pages/api/forum/post/[id]/comments.ts

```
/**
 * apps/web/src/pages/api/forum/post/[id]/comments.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 24.09.2022
 *
 */
import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/src/mongo";
import ForumPostModel from "@codeup/util/src/models/ForumPostModel";

export default async function handler(
  req: NextApiRequest,
  res: NextApiResponse
) {
```

```
  const { id } = req.query;

  try {
    await mongoConnect();

    const post = await ForumPostModel.findOne({
      _id: id,
    });

    if (!post) {
      res.status(404).json({
        error: "Not found",
      });
      return;
    }

    const comments = await ForumPostModel.find({
      parent: post._id,
    });

    const sortedComments = comments.sort((a, b) => {
      return new Date(b.createdAt).getTime() - new Date(a.createdAt).getTime();
    });

    res.status(200).json({
      post,
      comments: sortedComments,
    });
  } catch (e) {
    res.status(500).json({
      error: e,
    });
  }
}
```

## apps/web/src/pages/api/forum/post/[id]/delete.ts

```
/**
 * apps/web/src/pages/api/forum/post/[id]/delete.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 24.09.2022
 *
 */
import mongoConnect from "@codeup/util/src/mongo";
import { NextApiRequest, NextApiResponse } from "next";
import { decode, verify } from "jsonwebtoken";
import UserModel from "@codeup/util/src/models/UserModel";
import ForumPostModel from "@codeup/util/src/models/ForumPostModel";

export default async function handler(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "POST") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
```

```
}

if (!req.headers["x-jwt"]) {
  res.status(401).json({
    error: "Unauthorized",
  });
  return;
}

try {
  await mongoConnect();

  if (
    !verify(req.headers["x-jwt"] as string, process.env.JWT_SECRET as string)
  ) {
    res.status(401).json({
      error: "Unauthorized",
    });
    return;
  }

  const decoded = decode(req.headers["x-jwt"] as string);

  const user = await UserModel.findOne({
    // @ts-ignore
    username: decoded.username,
  });

  if (!user) {
    res.status(401).json({
      error: "Unauthorized",
    });
    return;
  }

  const post = await ForumPostModel.findOne({
    _id: req.query.id,
  });

  if (!post) {
    res.status(400).json({
      error: "Post not found!",
    });
    return;
  }

  if (post.user.toString() !== user._id.toString() && user.role !== "admin") {
    res.status(401).json({
      error: "Unauthorized",
    });
    return;
  }

  const children = await ForumPostModel.find({
    parent: post._id,
  });

  for (const child of children) {
    await child.delete();
  }
```

```
    await post.delete();

    res.status(200).json({
      deleted: true,
    });
  } catch (e) {
    res.status(500).json({
      error: e,
    });
  }
}
```

## apps/web/src/pages/api/forum/post/[id]/index.ts

```
/**
 * apps/web/src/pages/api/forum/post/[id]/index.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 24.09.2022
 *
 */
import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/src/mongo";
import ForumPostModel from "@codeup/util/src/models/ForumPostModel";

export default async function handler(
  req: NextApiRequest,
  res: NextApiResponse
) {
  const { id } = req.query;

  try {
    await mongoConnect();

    const post = await ForumPostModel.findOne({
      _id: id,
    });

    if (!post) {
      res.status(404).json({
        error: "Not found",
      });
      return;
    }

    res.status(200).json({
      post,
    });
  } catch (e) {
    res.status(500).json({
      error: e,
    });
  }
}
```

## apps/web/src/pages/api/forum/post/[id]/like.ts

```
/**
 * apps/web/src/pages/api/forum/post/[id]/like.ts
```

```
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 24.09.2022
 *
 */
import mongoConnect from "@codeup/util/src/mongo";
import { NextApiRequest, NextApiResponse } from "next";
import { decode, verify } from "jsonwebtoken";
import UserModel from "@codeup/util/src/models/UserModel";
import ForumPostModel from "@codeup/util/src/models/ForumPostModel";

export default async function handler(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "POST") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  if (!req.headers["x-jwt"]) {
    res.status(401).json({
      error: "Unauthorized",
    });
    return;
  }

  try {
    await mongoConnect();

    if (
      !verify(req.headers["x-jwt"] as string, process.env.JWT_SECRET as string)
    ) {
      res.status(401).json({
        error: "Unauthorized",
      });
      return;
    }

    const decoded = decode(req.headers["x-jwt"] as string);

    const user = await UserModel.findOne({
      // @ts-ignore
      username: decoded.username,
    });

    if (!user) {
      res.status(401).json({
        error: "Unauthorized",
      });
      return;
    }

    const post = await ForumPostModel.findOne({
      _id: req.query.id,
    });
```

```
    if (!post) {
      res.status(400).json({
        error: "Post not found!",
      });
      return;
    }

    if (post.likes.includes(user._id)) {
      post.likes.splice(post.likes.indexOf(user._id), 1);
    } else {
      post.likes.push(user._id);
    }

    await post.save();

    res.status(200).json({
      post,
    });
  } catch (e) {
    res.status(500).json({
      error: e,
    });
  }
}
```

## apps/web/src/pages/api/forum/profiles.ts

```
/**
 * apps/web/src/pages/api/forum/profiles.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 24.09.2022
 *
 */
import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/src/mongo";
import ForumProfileModel from "@codeup/util/src/models/ForumProfileModel";
import UserModel from "@codeup/util/src/models/UserModel";

export default async function handler(
  req: NextApiRequest,
  res: NextApiResponse
) {
  try {
    await mongoConnect();

    const profiles = await ForumProfileModel.find();

    let sortedProfiles = profiles.sort((a, b) => {
      if (a.verified && !b.verified) {
        return -1;
      } else if (!a.verified && b.verified) {
        return 1;
      } else {
        return a.username.localeCompare(b.username);
      }
    });
```

```
    let pfps = [];

    for (const profile of sortedProfiles) {
      const user = await UserModel.findOne({
        _id: profile.user,
      });

      if (!user) continue;

      pfps.push({
        ...profile.toJSON(),
        username: user.username,
      });
    }

    res.status(200).json({
      profiles: pfps,
    });
  } catch (e) {
    console.log(e);
    // @ts-ignore
    if (e.message) {
      // @ts-ignore
      res.status(500).json({ error: e.message });
    } else {
      res
        .status(500)
        .json({ error: "Internal Server Error with no specific message" });
    }
  }
}
```

## apps/web/src/pages/api/forum/[username]/posts.ts

```
/**
 * apps/web/src/pages/api/forum/[username]/posts.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 24.09.2022
 *
 */
import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/src/mongo";
import ForumPostModel from "@codeup/util/src/models/ForumPostModel";

export default async function handler(
  req: NextApiRequest,
  res: NextApiResponse
) {
  const { username } = req.query;

  try {
    await mongoConnect();

    const posts = await ForumPostModel.find({
      username: username,
    });

    const sortedPosts = posts.sort((a, b) => {
```

```
      return new Date(b.createdAt).getTime() - new Date(a.createdAt).getTime();
    });

    res.status(200).json({
      posts: sortedPosts,
    });
  } catch (e) {
    res.status(500).json({
      error: e,
    });
  }
}
```

## apps/web/src/pages/api/forum/[username]/profile.ts

```
/**
 * apps/web/src/pages/api/forum/[username]/profile.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 23.09.2022
 *
 */
import { NextApiRequest, NextApiResponse } from "next";
import ForumProfileModel from "@codeup/util/src/models/ForumProfileModel";
import ForumPostModel from "@codeup/util/src/models/ForumPostModel";
import UserModel from "@codeup/util/src/models/UserModel";
import mongoConnect from "@codeup/util/src/mongo";
import { decode, verify } from "jsonwebtoken";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  const username = req.query.username as string;

  try {
    await mongoConnect();

    if (req.method === "GET") {
      const user = await UserModel.findOne({ username: username });

      if (!user) {
        res.status(404).json({
          error: "User not found",
        });
        return;
      }

      const profile = await ForumProfileModel.findOne({ user: user._id });

      if (!profile) {
        res.status(404).json({
          error: "Profile not found",
        });
        return;
      }

      res.status(200).json({
        profile: {
```

```
      ...(profile.toJSON() as any),
      username: user.username,
      profilePicture: user.profilePicture,
    },
  });
} else if (req.method === "POST") {
  if (!req.headers["x-jwt"]) {
    res.status(401).json({
      error: "Unauthorized",
    });
    return;
  }

  const token = req.headers["x-jwt"] as string;

  if (!verify(token, process.env.JWT_SECRET as string)) {
    res.status(401).json({
      error: "Unauthorized",
    });
    return;
  }

  const decoded = decode(token) as any;

  const user = await UserModel.findOne({ username: username });

  if (!user) {
    res.status(404).json({
      error: "User not found",
    });
    return;
  }

  const profile = await ForumProfileModel.findOne({ user: user._id });

  if (profile) {
    if (decoded.id !== user._id.toString()) {
      res.status(401).json({
        error: "Unauthorized",
      });
      return;
    }

    const updatableKeys = ["certificates"];

    if (!process.env.DISABLE_PAYMENTS && user.subscription === "PRO") {
      updatableKeys.push("bio");
      updatableKeys.push("website");
      updatableKeys.push("displayName");
      updatableKeys.push("location");
    }

    for (const key of updatableKeys) {
      if (req.body[key]) {
        profile[key] = req.body[key];
      }
    }

    await profile.save();
```

```
    res.status(200).json({
      profile: {
        ...(profile.toJSON() as any),
        username: user.username,
        profilePicture: user.profilePicture,
      },
    });
  } else {
    const newProfile = await ForumProfileModel.create({
      user: user._id,
      username: user.username,
      displayName: user.firstName + " " + user.lastName,
      verified: false,
      bio: "CodeUp Benutzer",
      website: "",
      location: "",
      birthday: "2007-03-20",
      createdAt: Date.now(),
    });

    res.status(200).json({
      profile: newProfile,
    });
  }
} else if (req.method === "DELETE") {
  if (!req.headers["x-jwt"]) {
    res.status(401).json({
      error: "Unauthorized",
    });
    return;
  }

  const token = req.headers["x-jwt"] as string;

  if (!verify(token, process.env.JWT_SECRET as string)) {
    res.status(401).json({
      error: "Unauthorized",
    });
    return;
  }

  const decoded = decode(token) as any;

  const user = await UserModel.findOne({ username: username });

  if (!user) {
    res.status(404).json({
      error: "User not found",
    });
    return;
  }

  const profile = await ForumProfileModel.findOne({ user: user._id });

  if (!profile) {
    res.status(404).json({
      error: "Profile not found",
    });
    return;
  }
```

```
      if (decoded.id !== user._id.toString()) {
        res.status(401).json({
          error: "Unauthorized",
        });
        return;
      }

      const posts = await ForumPostModel.find({ user: user._id });

      for (const post of posts) {
        const children = await ForumPostModel.find({ parent: post._id });

        for (const child of children) {
          await child.delete();
        }

        await post.delete();
      }

      await profile.remove();

      res.status(200).json({
        deleted: true,
      });
    } else {
      res.status(405).json({
        error: "Method not allowed",
      });
    }
  } catch (e) {
    res.status(500).json({
      error: e,
    });
    return;
  }
}
```

## apps/web/src/pages/api/ide/projects/contributors/add.ts

```
/**
 * apps/web/src/pages/api/ide/projects/contributors/add.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 07.07.2022
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/src/mongo";
import jwt, { JwtPayload, VerifyErrors } from "jsonwebtoken";
import UserModel from "@codeup/util/src/models/UserModel";
import IDEProjectModel from "@codeup/util/src/models/IDEProjectModel";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "POST") {
```

```
  res.status(405).json({
    error: "Method not allowed",
  });
  return;
}

if (
  !req.body ||
  !req.headers["x-jwt"] ||
  !req.body.project ||
  !req.body.contributor
) {
  res.status(400).json({
    error: "Missing required fields",
  });
  return;
}

jwt.verify(
  req.headers["x-jwt"] as string,
  process.env.JWT_SECRET as string,
  async (err: VerifyErrors | null, decoded: JwtPayload | string | any) => {
    await mongoConnect();

    try {
      const user = await UserModel.findById((decoded as any).id);

      if (!user) {
        res.status(400).json({
          error: "Invalid Token",
        });
        return;
      }

      const IDEProject = await IDEProjectModel.findById(req.body.project);

      if (!IDEProject) {
        res.status(400).json({
          error: "Project not found",
        });
        return;
      }

      if (
        !IDEProject.owner.equals(user._id) &&
        !(
          IDEProject.contributors.find((c: any) => c.user.equals(user._id)) &&
          IDEProject.contributors
            .find((c: any) => c.user.equals(user._id))
            .permissions.includes("MANAGE-CONTRIBUTORS")
        ) &&
        user.role !== "admin"
      ) {
        res.status(403).json({
          error: "You do not have permissions to add contributors",
        });
        return;
      }

      const contributor = await UserModel.findOne({
```

```
          email: req.body.contributor,
        });

        if (!contributor) {
          res.status(400).json({
            error: "Contributor not found",
          });
          return;
        }

        if (
          IDEProject.contributors.find((c: any) =>
            c.user.equals(contributor._id)
          ) ||
          IDEProject.owner.equals(contributor._id)
        ) {
          res.status(400).json({
            error: "Contributor already added",
          });
          return;
        }

        IDEProject.contributors.push({
          user: contributor._id,
          permissions: ["READ", "WRITE", "DUPLICATE"],
        });

        IDEProject.updated_at = Date.now();

        await IDEProject.save();

        res.status(200).json({
          success: true,
          contributor: {
            email: contributor.email,
            id: contributor._id,
            permissions: ["READ", "WRITE", "DUPLICATE"],
            username: contributor.username,
            firstName: contributor.firstName,
            lastName: contributor.lastName,
          },
        });
      } catch (e: any) {
        res.status(500).json({
          error: e.message,
        });
        return;
      }
    }
  );
}
```

## apps/web/src/pages/api/ide/projects/contributors/list.ts

```
/**
 * apps/web/src/pages/api/ide/projects/contributors/list.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 07.07.2022
```

```
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/src/mongo";
import jwt, { JwtPayload, VerifyErrors } from "jsonwebtoken";
import UserModel from "@codeup/util/src/models/UserModel";
import IDEProjectModel from "@codeup/util/src/models/IDEProjectModel";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "POST") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  if (!req.body || !req.headers["x-jwt"] || !req.body.project) {
    res.status(400).json({
      error: "Missing required fields",
    });
    return;
  }

  jwt.verify(
    req.headers["x-jwt"] as string,
    process.env.JWT_SECRET as string,
    async (err: VerifyErrors | null, decoded: JwtPayload | string | any) => {
      await mongoConnect();

      try {
        const user = await UserModel.findById((decoded as any).id);

        if (!user) {
          res.status(400).json({
            error: "Invalid Token",
          });
          return;
        }

        const IDEProject = await IDEProjectModel.findById(req.body.project);

        if (!IDEProject) {
          res.status(400).json({
            error: "Project not found",
          });
          return;
        }

        if (
          !IDEProject.owner.equals(user._id) &&
          !IDEProject.contributors.find((c: any) => c.user.equals(user._id)) &&
          user.role !== "admin"
        ) {
          res.status(400).json({
            error:
              "You do not have permissions to list the contributors of project " +
```

```
          IDEProject._id,
        });
        return;
      }

      let c: {
        email: string;
        id: string;
        permissions: string[];
        username: string;
        firstName: string;
        lastName: string;
      }[] = [];
      let owner: {
        email: string;
        id: string;
        permissions: string[];
        username: string;
        firstName: string;
        lastName: string;
      };

      for (const contributor of IDEProject.contributors) {
        const contributor0 = await UserModel.findById(contributor.user);
        c = [
          ...c,
          {
            email: contributor0.email,
            id: contributor.user,
            permissions: contributor.permissions,
            username: contributor0.username,
            firstName: contributor0.firstName,
            lastName: contributor0.lastName,
          },
        ];
      }

      const owner0 = await UserModel.findById(IDEProject.owner);
      owner = {
        email: owner0.email,
        id: owner0._id,
        permissions: ["OWNER"],
        username: owner0.username,
        firstName: owner0.firstName,
        lastName: owner0.lastName,
      };

      res.status(200).json({
        success: true,
        owner: owner,
        contributors: c,
      });
    } catch (e: any) {
      res.status(500).json({
        error: e.message,
      });
      return;
    }
  }
);
```

```
}
```

## apps/web/src/pages/api/ide/projects/contributors/permissions.ts

```
/**
 * apps/web/src/pages/api/ide/projects/contributors/add.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 07.07.2022
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/src/mongo";
import jwt, { JwtPayload, VerifyErrors } from "jsonwebtoken";
import UserModel from "@codeup/util/src/models/UserModel";
import IDEProjectModel from "@codeup/util/src/models/IDEProjectModel";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "POST") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  if (
    !req.body ||
    !req.headers["x-jwt"] ||
    !req.body.project ||
    !req.body.contributor ||
    !req.body.permissions
  ) {
    res.status(400).json({
      error: "Missing required fields",
    });
    return;
  }

  jwt.verify(
    req.headers["x-jwt"] as string,
    process.env.JWT_SECRET as string,
    async (err: VerifyErrors | null, decoded: JwtPayload | string | any) => {
      await mongoConnect();

      try {
        const user = await UserModel.findById((decoded as any).id);

        if (!user) {
          res.status(400).json({
            error: "Invalid Token",
          });
          return;
        }

        const IDEProject = await IDEProjectModel.findById(req.body.project);
```

```
if (!IDEProject) {
  res.status(400).json({
    error: "Project not found",
  });
  return;
}

if (
  !IDEProject.owner.equals(user._id) &&
  !(
    IDEProject.contributors.find((c: any) => c.user.equals(user._id)) &&
    IDEProject.contributors
      .find((c: any) => c.user.equals(user._id))
      .permissions.includes("MANAGE-CONTRIBUTORS")
  ) &&
  user.role !== "admin"
) {
  res.status(403).json({
    error: "You do not have permissions to modify contributors",
  });
  return;
}

const contributor = await UserModel.findOne({
  email: req.body.contributor,
});

if (!contributor) {
  res.status(400).json({
    error: "Contributor not found",
  });
  return;
}

if (
  !IDEProject.contributors.find((c: any) =>
    c.user.equals(contributor._id)
  )
) {
  res.status(400).json({
    error: "Contributor not added yet",
  });
  return;
}

if (
  IDEProject.contributors
    .find((c: any) => c.user.equals(contributor._id))
    .user.equals(user._id)
) {
  res.status(400).json({
    error: "You can not update yourself.",
  });
  return;
}

IDEProject.contributors.find((c: any) =>
  c.user.equals(contributor._id)
).permissions = req.body.permissions;
```

```
        await IDEProject.save();

        res.status(200).json({
          success: true,
          contributor: {
            email: contributor.email,
            id: contributor._id,
            permissions: req.body.permissions,
            username: contributor.username,
            firstName: contributor.firstName,
            lastName: contributor.lastName,
          },
        });
      } catch (e: any) {
        res.status(500).json({
          error: e.message,
        });
        return;
      }
    }
  );
}
```

## apps/web/src/pages/api/ide/projects/contributors/remove.ts

```
/**
 * apps/web/src/pages/api/ide/projects/contributors/add.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 07.07.2022
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/src/mongo";
import jwt, { JwtPayload, VerifyErrors } from "jsonwebtoken";
import UserModel from "@codeup/util/src/models/UserModel";
import IDEProjectModel from "@codeup/util/src/models/IDEProjectModel";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "POST") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  if (
    !req.body ||
    !req.headers["x-jwt"] ||
    !req.body.project ||
    !req.body.contributor
  ) {
    res.status(400).json({
      error: "Missing required fields",
```

```
  });
  return;
}

jwt.verify(
  req.headers["x-jwt"] as string,
  process.env.JWT_SECRET as string,
  async (err: VerifyErrors | null, decoded: JwtPayload | string | any) => {
    await mongoConnect();

    try {
      const user = await UserModel.findById((decoded as any).id);

      if (!user) {
        res.status(400).json({
          error: "Invalid Token",
        });
        return;
      }

      const IDEProject = await IDEProjectModel.findById(req.body.project);

      if (!IDEProject) {
        res.status(400).json({
          error: "Project not found",
        });
        return;
      }

      if (
        !IDEProject.owner.equals(user._id) &&
        !(
          IDEProject.contributors.find((c: any) => c.user.equals(user._id)) &&
          IDEProject.contributors
            .find((c: any) => c.user.equals(user._id))
            .permissions.includes("MANAGE-CONTRIBUTORS")
        ) &&
        user.role !== "admin"
      ) {
        res.status(403).json({
          error: "You do not have permissions to add contributors",
        });
        return;
      }

      const contributor = await UserModel.findOne({
        email: req.body.contributor,
      });

      if (!contributor) {
        res.status(400).json({
          error: "Contributor not found",
        });
        return;
      }

      if (
        !IDEProject.contributors.find((c: any) =>
          c.user.equals(contributor._id)
        )
```

```
        ) {
          res.status(400).json({
            error: "Contributor not added yet",
          });
          return;
        }

        IDEProject.contributors = IDEProject.contributors.filter(
          (c: any) => !c.user.equals(contributor._id)
        );

        IDEProject.updated_at = Date.now();
        await IDEProject.save();

        res.status(200).json({
          success: true,
        });
      } catch (e: any) {
        res.status(500).json({
          error: e.message,
        });
        return;
      }
    }
  );
}
```

## apps/web/src/pages/api/ide/projects/create.ts

```
/**
 * apps/web/src/pages/api/ide/projects/create.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 06.07.2022
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/src/mongo";
import jwt, { JwtPayload, VerifyErrors } from "jsonwebtoken";
import UserModel from "@codeup/util/src/models/UserModel";
import IDEProjectModel from "@codeup/util/src/models/IDEProjectModel";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "POST") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  if (!req.body || !req.headers["x-jwt"] || !req.body.name) {
    res.status(400).json({
      error: "Missing required fields",
    });
    return;
```

```
  }

  jwt.verify(
    req.headers["x-jwt"] as string,
    process.env.JWT_SECRET as string,
    async (err: VerifyErrors | null, decoded: JwtPayload | string | any) => {
      await mongoConnect();

      try {
        const user = await UserModel.findById((decoded as any).id);

        if (!user) {
          res.status(400).json({
            error: "User not found",
          });
          return;
        }

        const IDEProject = await IDEProjectModel.create({
          owner: user._id,
          name: req.body.name,
          published: false,
          publishedCustomName: "",
        });

        res.status(200).json({
          success: true,
          project: IDEProject,
        });
      } catch (e: any) {
        res.status(500).json({
          error: e.message,
        });
      }
    }
  );
}
```

## apps/web/src/pages/api/ide/projects/delete.ts

```
/**
 * apps/web/src/pages/api/ide/projects/create.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 06.07.2022
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/src/mongo";
import jwt, { JwtPayload, VerifyErrors } from "jsonwebtoken";
import UserModel from "@codeup/util/src/models/UserModel";
import IDEProjectModel from "@codeup/util/src/models/IDEProjectModel";
import IDETemplateModel from "@codeup/util/src/models/IDETemplateModel";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
```

```
if (req.method !== "POST") {
  res.status(405).json({
    error: "Method not allowed",
  });
  return;
}

if (!req.body || !req.headers["x-jwt"] || !req.body.id) {
  res.status(400).json({
    error: "Missing required fields",
  });
  return;
}

jwt.verify(
  req.headers["x-jwt"] as string,
  process.env.JWT_SECRET as string,
  async (err: VerifyErrors | null, decoded: JwtPayload | string | any) => {
    await mongoConnect();

    try {
      const user = await UserModel.findById((decoded as any).id);

      if (!user) {
        res.status(400).json({
          error: "User not found",
        });
        return;
      }

      const IDEProject = await IDEProjectModel.findById(req.body.id);

      if (!IDEProject) {
        res.status(400).json({
          error: "Project not found",
        });
        return;
      }

      if (
        !IDEProject.owner.equals(user._id) &&
        !(
          IDEProject.contributors.find((c: any) => c.user.equals(user._id)) &&
          IDEProject.contributors
            .find((c: any) => c.user.equals(user._id))
            .permissions.includes("DELETE")
        ) &&
        user.role !== "admin"
      ) {
        res.status(403).json({
          error: "You do not have permissions to delete this project",
        });
        return;
      }

      const tpl = await IDETemplateModel.findOne({
        project: IDEProject._id,
      });

      if (tpl) {
```

```
        await tpl.remove();
      }

      await IDEProject.remove();

      res.status(200).json({
        success: true,
        message: "Project removed.",
      });
    } catch (e: any) {
      res.status(500).json({
        error: e.message,
      });
    }
  }
 );
}
```

## apps/web/src/pages/api/ide/projects/duplicate.ts

```
/**
 * apps/web/src/pages/api/ide/projects/update/name.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 07.07.2022
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/src/mongo";
import jwt, { JwtPayload, VerifyErrors } from "jsonwebtoken";
import UserModel from "@codeup/util/src/models/UserModel";
import IDEProjectModel from "@codeup/util/src/models/IDEProjectModel";
import { uuid } from "uuidv4";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "POST") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  if (!req.body || !req.headers["x-jwt"] || !req.body.id || !req.body.name) {
    res.status(400).json({
      error: "Missing required fields",
    });
    return;
  }

  jwt.verify(
    req.headers["x-jwt"] as string,
    process.env.JWT_SECRET as string,
    async (err: VerifyErrors | null, decoded: JwtPayload | string | any) => {
      await mongoConnect();
```

```
      try {
        const user = await UserModel.findById((decoded as any).id);

        if (!user) {
          res.status(400).json({
            error: "User not found",
          });
          return;
        }

        const IDEProject = await IDEProjectModel.findById(req.body.id);

        if (!IDEProject) {
          res.status(400).json({
            error: "Project not found",
          });
          return;
        }

        if (
          !IDEProject.owner.equals(user._id) &&
          !(
            IDEProject.contributors.find((c: any) => c.user.equals(user._id)) &&
            IDEProject.contributors
              .find((c: any) => c.user.equals(user._id))
              .permissions.includes("DUPLICATE")
          )
        ) {
          res.status(403).json({
            error: "You do not have permissions to duplicate this project",
          });
          return;
        }

        const DupIDEProject = await IDEProjectModel.create({
          owner: user._id,
          name: req.body.name,
          files: IDEProject.files,
          contributors: [],
        });

        res.status(200).json({
          success: true,
          project: DupIDEProject,
        });
      } catch (e: any) {
        res.status(500).json({
          error: e.message,
        });
      }
    }
  );
}
```

## apps/web/src/pages/api/ide/projects/list.ts

```
/**
 * apps/web/src/pages/api/ide/projects/list.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
```

```
 * License: Project License
 * Created At: 06.07.2022
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/src/mongo";
import jwt, { JwtPayload, VerifyErrors } from "jsonwebtoken";
import UserModel from "@codeup/util/src/models/UserModel";
import IDEProjectModel from "@codeup/util/src/models/IDEProjectModel";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "POST") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  if (!req.body || !req.headers["x-jwt"] || !req.body.type) {
    res.status(400).json({
      error: "Missing required fields",
    });
    return;
  }

  if (!["OWNER", "CONTRIBUTOR", "ALL"].includes(req.body.type.toUpperCase())) {
    res.status(400).json({
      error: "Field type does not match Type ['OWNER' | 'CONTRIBUTOR' | 'ALL']",
    });
    return;
  }

  jwt.verify(
    req.headers["x-jwt"] as string,
    process.env.JWT_SECRET as string,
    async (err: VerifyErrors | null, decoded: JwtPayload | string | any) => {
      await mongoConnect();

      try {
        const user = await UserModel.findById((decoded as any).id);

        if (!user) {
          res.status(400).json({
            error: "User not found",
          });
          return;
        }

        let projects: any[] = [];

        switch (req.body.type.toUpperCase()) {
          case "OWNER":
            projects = await IDEProjectModel.find({
              owner: (decoded as any).id,
            });
            break;
```

```
    case "CONTRIBUTOR":
      projects = await IDEProjectModel.find({
        contributors: (decoded as any).id,
      });
      break;
    case "ALL":
      projects = [
        ...(await IDEProjectModel.find({
          contributors: {
            $elemMatch: {
              user: (decoded as any).id,
            },
          },
        })),
        ...(await IDEProjectModel.find({
          owner: (decoded as any).id,
        })),
      ];
      break;
  }

  if (req.body.minimal) {
    let t: any[] = [];
    projects.map((project) => {
      t.push({
        name: project.name,
        id: project._id,
        updated_at: project.updated_at,
        pinned: project.pinned,
        stats:
          req.body.withStats === true ? getStats(project) : undefined,
      });
    });
    res.status(200).json({
      success: true,
      projects: t,
    });
    return;
  }

  let ret = projects;

  if (req.body.withStats === true) {
    let x: typeof projects = [];
    projects.map((project) => {
      x.push({
        ...project._doc,
        stats: getStats(project),
      });
    });
    ret = x;
  }

  res.status(200).json({
    success: true,
    projects: ret,
  });
} catch (e: any) {
  res.status(500).json({
    error: e.message,
```

```
        });
      }
    }
  );
}

function getStats(project: any) {
  return {
    total_files: project.files.length,
    total_contributors: project.contributors.length,
    total_size: new TextEncoder().encode(JSON.stringify(project)).length,
  };
}
```

## apps/web/src/pages/api/ide/projects/update/name.ts

```
/**
 * apps/web/src/pages/api/ide/projects/update/name.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 07.07.2022
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/src/mongo";
import jwt, { JwtPayload, VerifyErrors } from "jsonwebtoken";
import UserModel from "@codeup/util/src/models/UserModel";
import IDEProjectModel from "@codeup/util/src/models/IDEProjectModel";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "POST") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  if (!req.body || !req.headers["x-jwt"] || !req.body.id || !req.body.name) {
    res.status(400).json({
      error: "Missing required fields",
    });
    return;
  }

  jwt.verify(
    req.headers["x-jwt"] as string,
    process.env.JWT_SECRET as string,
    async (err: VerifyErrors | null, decoded: JwtPayload | string | any) => {
      await mongoConnect();

      try {
        const user = await UserModel.findById((decoded as any).id);

        if (!user) {
          res.status(400).json({
```

```
            error: "User not found",
          });
          return;
        }

        const IDEProject = await IDEProjectModel.findById(req.body.id);

        if (!IDEProject) {
          res.status(400).json({
            error: "Project not found",
          });
          return;
        }

        if (
          !IDEProject.owner.equals(user._id) &&
          !(
            IDEProject.contributors.find((c: any) => c.user.equals(user._id)) &&
            IDEProject.contributors
              .find((c: any) => c.user.equals(user._id))
              .permissions.includes("RENAME")
          ) &&
          user.role !== "admin"
        ) {
          res.status(403).json({
            error: "You do not have permissions to update this project",
          });
          return;
        }

        await IDEProject.update({
          name: req.body.name,
          updated_at: Date.now(),
        });

        res.status(200).json({
          success: true,
          message: "Project Name updated.",
        });
      } catch (e: any) {
        res.status(500).json({
          error: e.message,
        });
      }
    }
  );
}
```

## apps/web/src/pages/api/ide/projects/[id]/discovery.ts

```
/**
 * apps/web/src/pages/api/ide/projects/[id]/discovery.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 23.12.2022
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
```

```
import mongoConnect from "@codeup/util/src/mongo";
import jwt, { JwtPayload, VerifyErrors } from "jsonwebtoken";
import UserModel from "@codeup/util/src/models/UserModel";
import IDEProjectModel from "@codeup/util/src/models/IDEProjectModel";
import IDETemplateModel from "@codeup/util/src/models/IDETemplateModel";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  const id = req.query.id as string;

  if (!req.body || !req.headers["x-jwt"]) {
    res.status(400).json({
      error: "Missing required fields",
    });
    return;
  }

  jwt.verify(
    req.headers["x-jwt"] as string,
    process.env.JWT_SECRET as string,
    async (err: VerifyErrors | null, decoded: JwtPayload | string | any) => {
      await mongoConnect();

      try {
        const user = await UserModel.findById((decoded as any).id);

        if (!user) {
          res.status(400).json({
            error: "User not found",
          });
          return;
        }

        const IDEProject = await IDEProjectModel.findById(id);

        if (!IDEProject) {
          res.status(400).json({
            error: "Project not found",
          });
          return;
        }

        if (
          !IDEProject.owner.equals(user._id) &&
          !IDEProject.contributors.find(
            (c: any) => c.user.toString() === user._id.toString()
          ) &&
          user.role !== "admin"
        ) {
          res.status(403).json({
            error: "You do not have permissions to delete this project",
          });
          return;
        }

        if (req.method === "POST") {
          if (!req.body) {
            res.status(400).json({
```

```
            error: "Missing required fields",
          });
          return;
        }

        if (!IDEProject.published) {
          res.status(400).json({
            error:
              "The project has to be published to be eligible for discovery",
          });
          return;
        }

        if (req.body.enableDiscovery) {
          if (req.body.discoveryDescription) {
            IDEProject.discovery = true;
            IDEProject.discoveryDescription = req.body.discoveryDescription;
          } else {
            res.status(400).json({
              error: "Missing required fields",
            });
            return;
          }
        } else if (req.body.disableDiscovery) {
          IDEProject.discovery = false;
          IDEProject.discoveryDescription = "";
        }

        await IDEProject.save();

        res.status(200).json({
          success: true,
          discovery: IDEProject.discovery,
        });
      } else {
        res.status(405).json({
          error: "Method not allowed",
        });
        return;
      }
    } catch (e: any) {
      res.status(500).json({
        error: e.message,
      });
    }
  }
 );
}
```

## apps/web/src/pages/api/ide/projects/[id]/file/delete.ts

```
/**
 * apps/web/src/pages/api/ide/projects/[id]/file/save.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 20.07.2022
 *
 */
```

```
import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/src/mongo";
import jwt, { JwtPayload, VerifyErrors } from "jsonwebtoken";
import UserModel from "@codeup/util/src/models/UserModel";
import IDEProjectModel from "@codeup/util/src/models/IDEProjectModel";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "POST") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  if (!req.body || !req.headers["x-jwt"] || !req.query.id || !req.body.path) {
    res.status(400).json({
      error: "Missing required fields",
    });
    return;
  }

  jwt.verify(
    req.headers["x-jwt"] as string,
    process.env.JWT_SECRET as string,
    async (err: VerifyErrors | null, decoded: JwtPayload | string | any) => {
      await mongoConnect();

      try {
        const user = await UserModel.findById((decoded as any).id);

        if (!user) {
          res.status(400).json({
            error: "Invalid token",
          });
          return;
        }

        const IDEProject = await IDEProjectModel.findById(req.query.id);

        if (!IDEProject) {
          res.status(400).json({
            error: "Project not found",
          });
          return;
        }

        if (
          !IDEProject.owner.equals(user._id) &&
          !(
            IDEProject.contributors.find((c: any) => c.user.equals(user._id)) &&
            IDEProject.contributors
              .find((c: any) => c.user.equals(user._id))
              .permissions.includes("WRITE")
          ) &&
          user.role !== "admin"
        ) {
          res.status(400).json({
```

```
            error: "Project not found",
          });
          return;
        }

        if (IDEProject.files.find((f: any) => f.path === req.body.path)) {
          IDEProject.files = IDEProject.files.filter(
            (f: any) => f.path !== req.body.path
          );
        }

        IDEProject.updated_at = Date.now();

        await IDEProject.save();

        res.status(200).json({
          success: true,
        });
      } catch (e: any) {
        res.status(500).json({
          error: e.message,
        });
      }
    }
  );
}
```

## apps/web/src/pages/api/ide/projects/[id]/file/save.ts

```
/**
 * apps/web/src/pages/api/ide/projects/[id]/file/save.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 20.07.2022
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/src/mongo";
import jwt, { JwtPayload, VerifyErrors } from "jsonwebtoken";
import UserModel from "@codeup/util/src/models/UserModel";
import IDEProjectModel from "@codeup/util/src/models/IDEProjectModel";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "POST") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  if (
    !req.body ||
    !req.headers["x-jwt"] ||
    !req.query.id ||
    !req.body.path ||
```

```
  !req.body.content
) {
  res.status(400).json({
    error: "Missing required fields",
  });
  return;
}

jwt.verify(
  req.headers["x-jwt"] as string,
  process.env.JWT_SECRET as string,
  async (err: VerifyErrors | null, decoded: JwtPayload | string | any) => {
    await mongoConnect();

    try {
      const user = await UserModel.findById((decoded as any).id);

      if (!user) {
        res.status(400).json({
          error: "Invalid token",
        });
        return;
      }

      const IDEProject = await IDEProjectModel.findById(req.query.id);

      if (!IDEProject) {
        res.status(400).json({
          error: "Project not found",
        });
        return;
      }

      if (
        !IDEProject.owner.equals(user._id) &&
        !(
          IDEProject.contributors.find((c: any) => c.user.equals(user._id)) &&
          IDEProject.contributors
            .find((c: any) => c.user.equals(user._id))
            .permissions.includes("WRITE")
        ) &&
        user.role !== "admin"
      ) {
        res.status(400).json({
          error: "Project not found",
        });
        return;
      }

      if (IDEProject.files.find((f: any) => f.path === req.body.path)) {
        IDEProject.files.find((f: any) => f.path === req.body.path).content =
          req.body.content;
        await IDEProject.save();
        res.status(200).json({
          success: true,
        });
        return;
      }

      IDEProject.files.push({
```

```
          path: req.body.path,
          content: req.body.content,
        });

        IDEProject.updated_at = Date.now();

        await IDEProject.save();

        res.status(200).json({
          success: true,
        });
      } catch (e: any) {
        res.status(500).json({
          error: e.message,
        });
      }
    }
  );
}
```

## apps/web/src/pages/api/ide/projects/[id]/index.ts

```
/**
 * apps/web/src/pages/api/ide/projects/[id]/index.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 10.07.2022
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/src/mongo";
import jwt, { JwtPayload, VerifyErrors } from "jsonwebtoken";
import UserModel from "@codeup/util/src/models/UserModel";
import IDEProjectModel from "@codeup/util/src/models/IDEProjectModel";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "GET") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  if (!req.headers["x-jwt"] || !req.query.id) {
    res.status(400).json({
      error: "Missing required fields",
    });
    return;
  }

  jwt.verify(
    req.headers["x-jwt"] as string,
    process.env.JWT_SECRET as string,
    async (err: VerifyErrors | null, decoded: JwtPayload | string | any) => {
      await mongoConnect();
```

```
      try {
        const user = await UserModel.findById((decoded as any).id);

        if (!user) {
          res.status(400).json({
            error: "Invalid token",
          });
          return;
        }

        const IDEProject = await IDEProjectModel.findById(req.query.id);

        if (!IDEProject) {
          res.status(400).json({
            error: "Project not found",
          });
          return;
        }

        if (
          !IDEProject.owner.equals(user._id) &&
          !(
            IDEProject.contributors.find((c: any) => c.user.equals(user._id)) &&
            IDEProject.contributors
              .find((c: any) => c.user.equals(user._id))
              .permissions.includes("READ")
          ) &&
          !IDEProject.published &&
          user.role !== "admin"
        ) {
          res.status(400).json({
            error: "Project not found",
          });
          return;
        }

        res.status(200).json({
          success: true,
          project: IDEProject,
        });
      } catch (e: any) {
        res.status(500).json({
          error: e.message,
        });
      }
    }
  );
}
```

## apps/web/src/pages/api/ide/projects/[id]/permissions.ts

```
/**
 * apps/web/src/pages/api/ide/projects/permissions.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 07.07.2022
 *
 */
```

```
import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/src/mongo";
import jwt, { JwtPayload, VerifyErrors } from "jsonwebtoken";
import UserModel from "@codeup/util/src/models/UserModel";
import IDEProjectModel from "@codeup/util/src/models/IDEProjectModel";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "GET") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  if (!req.headers["x-jwt"] || !req.query.id) {
    res.status(400).json({
      error: "Missing required fields",
    });
    return;
  }

  jwt.verify(
    req.headers["x-jwt"] as string,
    process.env.JWT_SECRET as string,
    async (err: VerifyErrors | null, decoded: JwtPayload | string | any) => {
      await mongoConnect();

      try {
        const user = await UserModel.findById((decoded as any).id);

        if (!user) {
          res.status(400).json({
            error: "User not found",
          });
          return;
        }

        const IDEProject = await IDEProjectModel.findById(req.query.id);

        if (!IDEProject) {
          res.status(400).json({
            error: "Project not found",
          });
          return;
        }

        if (IDEProject.owner.equals(user._id) || user.role === "admin") {
          res.status(200).json({
            success: true,
            permissions: ["OWNER"],
          });
          return;
        }

        if (IDEProject.contributors.find((c: any) => c.user.equals(user._id))) {
          res.status(200).json({
```

```
            success: true,
            permissions: [
              ...IDEProject.contributors.find((c: any) =>
                c.user.equals(user._id)
              ).permissions,
            ],
          });
          return;
        }

        res.status(200).json({
          success: true,
          permissions: [],
        });
        return;
      } catch (e: any) {
        res.status(500).json({
          error: e.message,
        });
      }
    }
  );
}
```

## apps/web/src/pages/api/ide/projects/[id]/preview/[[...path]].ts

```
/**
 * apps/web/src/pages/api/ide/projects/[id]/preview/[[...path]].tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 17.07.2022
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/src/mongo";
import jwt, { JwtPayload, VerifyErrors } from "jsonwebtoken";
import UserModel from "@codeup/util/src/models/UserModel";
import IDEProjectModel from "@codeup/util/src/models/IDEProjectModel";
import * as ts from "typescript";
import * as sass from "sass";
import { lookup as mimeLookup } from "mime-types";
import * as InCode from "@codeup/incode-compiler/dist";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  let path = (req.query.path as string[])?.join("/") || "/index.html";

  if (!path.startsWith("/")) {
    path = "/" + path;
  }
  if (req.method !== "GET") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }
```

```
if (!req.query.id || !req.cookies.jwt) {
  res.status(400).json({
    error: "Missing required fields",
  });
  return;
}

jwt.verify(
  req.cookies.jwt as string,
  process.env.JWT_SECRET as string,
  async (err: VerifyErrors | null, decoded: JwtPayload | string | any) => {
    await mongoConnect();

    let mimeType;
    try {
      const user = await UserModel.findById((decoded as any).id);

      if (!user) {
        res.status(400).json({
          error: "Invalid token",
        });
        return;
      }

      const IDEProject = await IDEProjectModel.findById(req.query.id);

      if (!IDEProject) {
        res.status(400).json({
          error: "Project not found",
        });
        return;
      }

      if (
        !IDEProject.owner.equals(user._id) &&
        !(
          IDEProject.contributors.find((c: any) => c.user.equals(user._id)) &&
          IDEProject.contributors
            .find((c: any) => c.user.equals(user._id))
            .permissions.includes("READ")
        ) &&
        user.role !== "admin"
      ) {
        res.status(400).json({
          error: "Project not found",
        });
        return;
      }

      const file = IDEProject.files.find(
        (f: any) => f.path.toLowerCase() === path.toLowerCase()
      );

      if (!file) {
        res.status(404).write("Not Found");
        res.end();
        return;
      }
```

```
        let ret = file.content;

        if (file.path.toLowerCase().endsWith(".ts")) {
          ret = ts.transpileModule(ret, {
            compilerOptions: {
              module: ts.ModuleKind.ESNext,
            },
          }).outputText;
        }

        if (
          file.path.toLowerCase().endsWith(".scss") ||
          file.path.toLowerCase().endsWith(".sass")
        ) {
          ret = sass.compileString(file.content, {
            syntax: file.path.toLowerCase().endsWith(".scss")
              ? "scss"
              : "indented",
          }).css;
        }

        if (file.path.toLowerCase().endsWith(".ic")) {
          ret = `<html><head><title>${
            file.path
          }</title></head><body><script defer>${InCode.Compiler.compile(
            ret
          )}</script></body></html>`;
        }

        if (
          ret.match(
            /^\s*data:([a-z]+\/[a-z]+(;[a-z\-]+\=[a-z\-]+)?)?(;base64)?,[a-z0-9\!\$\&\'\,\(\)\*\+\,\;\=\-
\.\_\~\:\@\/\?\%\s]*\s*$/i
          )
        ) {
          res.redirect(ret);
          return;
        }

        const mimeTypes: { [key: string]: string } = {
          js: "application/javascript",
          css: "text/css",
          html: "text/html",
          json: "application/json",
          png: "image/png",
          jpg: "image/jpeg",
          jpeg: "image/jpeg",
          gif: "image/gif",
          svg: "image/svg+xml",
          ic: "text/html",
        };
        const replaceTypes: { [key: string]: string } = {
          "text/x-scss": "text/css",
          "text/x-sass": "text/css",
          "text/x-typescript": "application/javascript",
        };

        let mimeType = "";

        if (mimeTypes[file.path.split(".").pop() as string]) {
```

```
        mimeType = mimeTypes[file.path.split(".").pop() as string];
      } else {
        mimeType = mimeLookup(file.path) as string;
      }

      if (!mimeType || !Object.values(mimeTypes).includes(mimeType)) {
        if (mimeType && Object.keys(replaceTypes).includes(mimeType)) {
          mimeType = replaceTypes[mimeType];
        } else {
          mimeType = "text/plain";
        }
      }

      res.setHeader("Content-Type", mimeType);

      res.status(200).write(ret);

      res.end();
    } catch (e: any) {
      res.status(500).json({
        error: e.message,
      });
    }
  }
 );
}
```

## apps/web/src/pages/api/ide/projects/[id]/publish.ts

```
/**
 * apps/web/src/pages/api/ide/projects/[id]/publish.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 06.12.2022
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/src/mongo";
import jwt, { JwtPayload, VerifyErrors } from "jsonwebtoken";
import UserModel from "@codeup/util/src/models/UserModel";
import IDEProjectModel from "@codeup/util/src/models/IDEProjectModel";
import IDETemplateModel from "@codeup/util/src/models/IDETemplateModel";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  const id = req.query.id as string;

  if (!req.body || !req.headers["x-jwt"]) {
    res.status(400).json({
      error: "Missing required fields",
    });
    return;
  }

  jwt.verify(
    req.headers["x-jwt"] as string,
```

```
process.env.JWT_SECRET as string,
async (err: VerifyErrors | null, decoded: JwtPayload | string | any) => {
  await mongoConnect();

  try {
    const user = await UserModel.findById((decoded as any).id);

    if (!user) {
      res.status(400).json({
        error: "User not found",
      });
      return;
    }

    const IDEProject = await IDEProjectModel.findById(id);

    if (!IDEProject) {
      res.status(400).json({
        error: "Project not found",
      });
      return;
    }

    if (
      !IDEProject.owner.equals(user._id) &&
      !IDEProject.contributors.find(
        (c: any) => c.user.toString() === user._id.toString()
      ) &&
      user.role !== "admin"
    ) {
      res.status(403).json({
        error: "You do not have permissions to delete this project",
      });
      return;
    }

    if (req.method === "POST") {
      if (!req.body || !req.body.publishState) {
        res.status(400).json({
          error: "Missing required fields2",
        });
        return;
      }

      const publishState = req.body.publishState;

      if (publishState === "public" && !req.body.customName) {
        res.status(400).json({
          error: "Missing required fields1",
        });
        return;
      }

      if (publishState === "public") {
        const customName = req.body.customName;

        const existingProject = await IDEProjectModel.findOne({
          publishedCustomName: customName,
        });
```

```
            if (existingProject) {
              res.status(700).json({
                error: "This custom name is already taken",
              });
              return;
            }

            IDEProject.published = true;
            IDEProject.publishedCustomName = customName;

            await IDEProject.save();

            res.status(200).json({
              message: "Project published",
            });
            return;
          } else {
            IDEProject.published = false;
            IDEProject.publishedCustomName = "";

            IDEProject.discovery = false;
            IDEProject.discoveryDescription = "";

            await IDEProject.save();

            res.status(200).json({
              message: "Project unpublished",
              project: IDEProject,
            });

            return;
          }
        } else {
          res.status(405).json({
            error: "Method not allowed",
          });
          return;
        }
      } catch (e: any) {
        res.status(500).json({
          error: e.message,
        });
      }
    }
  );
}
```

## apps/web/src/pages/api/ide/projects/[id]/template/has.ts

```
/**
 * apps/web/src/pages/api/ide/projects/[id]/template/has.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 24.07.2022
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/src/mongo";
```

```
import jwt, { JwtPayload, VerifyErrors } from "jsonwebtoken";
import UserModel from "@codeup/util/src/models/UserModel";
import IDEProjectModel from "@codeup/util/src/models/IDEProjectModel";
import IDETemplateModel from "@codeup/util/src/models/IDETemplateModel";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "GET") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  if (!req.headers["x-jwt"]) {
    res.status(400).json({
      error: "Missing required fields",
    });
    return;
  }

  jwt.verify(
    req.headers["x-jwt"] as string,
    process.env.JWT_SECRET as string,
    async (err: VerifyErrors | null, decoded: JwtPayload | string | any) => {
      await mongoConnect();

      try {
        const user = await UserModel.findById((decoded as any).id);

        if (!user) {
          res.status(400).json({
            error: "Invalid token",
          });
          return;
        }

        const IDEProject = await IDEProjectModel.findById(req.query.id);

        if (!IDEProject) {
          res.status(400).json({
            error: "Project not found",
          });
          return;
        }

        if (!IDEProject.owner.equals(user._id) && user.role !== "admin") {
          res.status(400).json({
            error: "Project not found",
          });
          return;
        }

        const tpl = await IDETemplateModel.findOne({
          project: req.query.id,
        });

        if (!tpl) {
```

```
          res.status(200).json({
            success: true,
            hasTemplate: false,
          });
          return;
        }

        res.status(200).json({
          success: true,
          hasTemplate: true,
        });
      } catch (e: any) {
        res.status(500).json({
          error: e.message,
        });
      }
    }
  );
}
```

## apps/web/src/pages/api/ide/projects/[id]/template/publish.ts

```
/**
 * apps/web/src/pages/api/ide/projects/[id]/template/publish.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 24.07.2022
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/src/mongo";
import jwt, { JwtPayload, VerifyErrors } from "jsonwebtoken";
import UserModel from "@codeup/util/src/models/UserModel";
import IDEProjectModel from "@codeup/util/src/models/IDEProjectModel";
import IDETemplateModel from "@codeup/util/src/models/IDETemplateModel";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "POST") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  if (
    !req.body ||
    !req.headers["x-jwt"] ||
    !req.query.id ||
    !req.body.displayName ||
    !req.body.description
  ) {
    res.status(400).json({
      error: "Missing required fields",
    });
    return;
```

```
}

jwt.verify(
  req.headers["x-jwt"] as string,
  process.env.JWT_SECRET as string,
  async (err: VerifyErrors | null, decoded: JwtPayload | string | any) => {
    await mongoConnect();

    try {
      const user = await UserModel.findById((decoded as any).id);

      if (!user) {
        res.status(400).json({
          error: "Invalid token",
        });
        return;
      }

      const IDEProject = await IDEProjectModel.findById(req.query.id);

      if (!IDEProject) {
        res.status(400).json({
          error: "Project not found",
        });
        return;
      }

      if (!IDEProject.owner.equals(user._id) && user.role !== "admin") {
        res.status(400).json({
          error: "Project not found",
        });
        return;
      }

      const tpl = await IDETemplateModel.findOne({
        project: IDEProject._id,
      });

      if (tpl) {
        res.status(400).json({
          error: "Template already exists for Project " + IDEProject._id,
        });
        return;
      }

      IDEProject.updated_at = Date.now();
      await IDEProject.save();

      const template = await IDETemplateModel.create({
        owner: user._id,
        displayName: req.body.displayName,
        description: req.body.description,
        splashImage: req.body.splashImage,
        project: IDEProject._id,
        authorDisplayName: user.username,
      });

      await template.save();

      res.status(200).json({
```

```
          success: true,
          template: template,
        });
      } catch (e: any) {
        res.status(500).json({
          error: e.message,
        });
      }
    }
  );
}
```

## apps/web/src/pages/api/ide/projects/[id]/template/remove.ts

```
/**
 * apps/web/src/pages/api/ide/projects/[id]/template/publish.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 24.07.2022
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/src/mongo";
import jwt, { JwtPayload, VerifyErrors } from "jsonwebtoken";
import UserModel from "@codeup/util/src/models/UserModel";
import IDEProjectModel from "@codeup/util/src/models/IDEProjectModel";
import IDETemplateModel from "@codeup/util/src/models/IDETemplateModel";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "POST") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  if (!req.headers["x-jwt"] || !req.query.id) {
    res.status(400).json({
      error: "Missing required fields",
    });
    return;
  }

  jwt.verify(
    req.headers["x-jwt"] as string,
    process.env.JWT_SECRET as string,
    async (err: VerifyErrors | null, decoded: JwtPayload | string | any) => {
      await mongoConnect();

      try {
        const user = await UserModel.findById((decoded as any).id);

        if (!user) {
          res.status(400).json({
            error: "Invalid token",
```

```
        });
        return;
      }

      const IDEProject = await IDEProjectModel.findById(req.query.id);

      if (!IDEProject) {
        res.status(400).json({
          error: "Project not found",
        });
        return;
      }

      if (!IDEProject.owner.equals(user._id) && user.role !== "admin") {
        res.status(400).json({
          error: "Project not found",
        });
        return;
      }

      const tpl = await IDETemplateModel.findOne({
        project: IDEProject._id,
      });

      if (!tpl) {
        res.status(400).json({
          error: "Template not published yet.",
        });
        return;
      }

      IDEProject.updated_at = Date.now();
      await IDEProject.save();

      await tpl.remove();

      res.status(200).json({
        success: true,
      });
    } catch (e: any) {
      res.status(500).json({
        error: e.message,
      });
    }
  }
);
}
```

## apps/web/src/pages/api/ide/projects/[id]/template/update.ts

```
/**
 * apps/web/src/pages/api/ide/projects/[id]/template/update.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 24.07.2022
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
```

```
import mongoConnect from "@codeup/util/src/mongo";
import jwt, { JwtPayload, VerifyErrors } from "jsonwebtoken";
import UserModel from "@codeup/util/src/models/UserModel";
import IDEProjectModel from "@codeup/util/src/models/IDEProjectModel";
import IDETemplateModel from "@codeup/util/src/models/IDETemplateModel";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "POST") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  if (!req.headers["x-jwt"] || !req.query.id || !req.body || !req.body.update) {
    res.status(400).json({
      error: "Missing required fields",
    });
    return;
  }

  jwt.verify(
    req.headers["x-jwt"] as string,
    process.env.JWT_SECRET as string,
    async (err: VerifyErrors | null, decoded: JwtPayload | string | any) => {
      await mongoConnect();

      try {
        const user = await UserModel.findById((decoded as any).id);

        if (!user) {
          res.status(400).json({
            error: "Invalid token",
          });
          return;
        }

        const IDEProject = await IDEProjectModel.findById(req.query.id);

        if (!IDEProject) {
          res.status(400).json({
            error: "Project not found",
          });
          return;
        }

        if (!IDEProject.owner.equals(user._id) && user.role !== "admin") {
          res.status(400).json({
            error: "Project not found",
          });
          return;
        }

        const tpl = await IDETemplateModel.findOne({
          project: IDEProject._id,
        });
```

```
        if (!tpl) {
          res.status(400).json({
            error: "Template not published yet.",
          });
          return;
        }

        const allowedKeys = ["displayName", "description"];

        for (const key of Object.keys(req.body.update)) {
          if (allowedKeys.includes(key)) {
            tpl[key] = req.body.update[key];
          }
        }

        IDEProject.updated_at = Date.now();
        await IDEProject.save();

        await tpl.save();

        res.status(200).json({
          success: true,
          template: tpl,
        });
      } catch (e: any) {
        res.status(500).json({
          error: e.message,
        });
      }
    }
  );
}
```

## apps/web/src/pages/api/ide/projects/[id]/toggle-pin.ts

```
/**
 * apps/web/src/pages/api/ide/projects/[id]/toggle-pin.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 05.08.2022
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/src/mongo";
import jwt, { JwtPayload, VerifyErrors } from "jsonwebtoken";
import UserModel from "@codeup/util/src/models/UserModel";
import IDEProjectModel from "@codeup/util/src/models/IDEProjectModel";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "POST") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }
```

```
if (!req.headers["x-jwt"] || !req.query.id) {
  res.status(400).json({
    error: "Missing required fields",
  });
  return;
}

jwt.verify(
  req.headers["x-jwt"] as string,
  process.env.JWT_SECRET as string,
  async (err: VerifyErrors | null, decoded: JwtPayload | string | any) => {
    await mongoConnect();

    try {
      const user = await UserModel.findById((decoded as any).id);

      if (!user) {
        res.status(400).json({
          error: "Invalid token",
        });
        return;
      }

      const IDEProject = await IDEProjectModel.findById(req.query.id);

      if (!IDEProject) {
        res.status(400).json({
          error: "Project not found",
        });
        return;
      }

      if (
        !IDEProject.owner.equals(user._id) &&
        !(
          IDEProject.contributors.find((c: any) => c.user.equals(user._id)) &&
          IDEProject.contributors
            .find((c: any) => c.user.equals(user._id))
            .permissions.includes("READ")
        ) &&
        user.role !== "admin"
      ) {
        res.status(400).json({
          error: "Project not found",
        });
        return;
      }

      if (IDEProject.pinned.includes(user._id)) {
        IDEProject.pinned = IDEProject.pinned.filter(
          (x: any) => !x.equals(user._id)
        );
      } else {
        IDEProject.pinned.push(user._id);
      }

      await IDEProject.save();

      res.status(200).json({
```

```
          success: true,
          project: IDEProject,
        });
      } catch (e: any) {
        res.status(500).json({
          error: e.message,
        });
      }
    }
  );
}
```

## apps/web/src/pages/api/ide/templates/list.ts

```
/**
 * apps/web/src/pages/api/ide/templates/all.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 24.07.2022
 *
 */
import IDETemplateModel from "@codeup/util/src/models/IDETemplateModel";
import { NextApiRequest, NextApiResponse } from "next";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  const t = await IDETemplateModel.find({});

  res.status(200).json({
    success: true,
    templates: t,
  });
}
```

## apps/web/src/pages/api/ide/templates/[id]/index.ts

```
/**
 * apps/web/src/pages/api/ide/templates/[id]/index.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 24.07.2022
 *
 */
import IDETemplateModel from "@codeup/util/src/models/IDETemplateModel";
import mongoConnect from "@codeup/util/src/mongo";
import { NextApiRequest, NextApiResponse } from "next";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "GET") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
```

```
  }

  try {
    await mongoConnect();

    const tpl = await IDETemplateModel.findById(req.query.id);

    if (!tpl) {
      res.status(404).json({
        error: "Could not find template " + req.query.id,
      });
      return;
    }

    res.status(200).json({
      success: true,
      template: tpl,
    });
  } catch (e: any) {
    res.status(500).json({
      error: e.message,
    });
  }
}
```

## apps/web/src/pages/api/ide/templates/[id]/rating.ts

```
/**
 * apps/web/src/pages/api/ide/templates/[id]/rating.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 24.07.2022
 *
 */
import { NextApiRequest, NextApiResponse } from "next";
import jwt, { JwtPayload, VerifyErrors } from "jsonwebtoken";
import mongoConnect from "@codeup/util/src/mongo";
import UserModel from "@codeup/util/src/models/UserModel";
import IDETemplateModel from "@codeup/util/src/models/IDETemplateModel";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "POST") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  if (!req.headers["x-jwt"] || !req.body || !req.body.rating) {
    res.status(400).json({
      error: "Missing required fields",
    });
    return;
  }

  if (req.body.rating > 5 || req.body.rating < 1) {
```

```
    res.status(400).json({
      error: "Rating must be between 1 and 5",
    });
    return;
  }

  jwt.verify(
    req.headers["x-jwt"] as string,
    process.env.JWT_SECRET as string,
    async (err: VerifyErrors | null, decoded: JwtPayload | string | any) => {
      await mongoConnect();

      try {
        const user = await UserModel.findById((decoded as any).id);

        if (!user) {
          res.status(400).json({
            error: "Invalid Token",
          });
          return;
        }

        const tpl = await IDETemplateModel.findById(req.query.id);

        if (!tpl) {
          res.status(404).json({
            error: "Template not found",
          });
          return;
        }

        if (tpl.ratings.find((r: any) => r.user.equals(user._id))) {
          res.status(400).json({
            error: "You have already rated this template",
          });
          return;
        }

        tpl.ratings.push({
          user: user._id,
          rating: req.body.rating,
        });

        await tpl.save();

        res.status(200).json({
          success: true,
        });
      } catch (e: any) {
        res.status(500).json({
          error: e.message,
        });
      }
    }
  );
}
```

## apps/web/src/pages/api/ide/templates/[id]/use.ts

```
/**
 * apps/web/src/pages/api/ide/templates/[id]/useTemplate.ts
```

```
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 24.07.2022
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import jwt, { JwtPayload, VerifyErrors } from "jsonwebtoken";
import mongoConnect from "@codeup/util/src/mongo";
import UserModel from "@codeup/util/src/models/UserModel";
import IDETemplateModel from "@codeup/util/src/models/IDETemplateModel";
import IDEProjectModel from "@codeup/util/src/models/IDEProjectModel";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "POST") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  if (!req.headers["x-jwt"]) {
    res.status(400).json({
      error: "Missing required fields",
    });
    return;
  }

  jwt.verify(
    req.headers["x-jwt"] as string,
    process.env.JWT_SECRET as string,
    async (err: VerifyErrors | null, decoded: JwtPayload | string | any) => {
      await mongoConnect();

      try {
        const user = await UserModel.findById((decoded as any).id);

        if (!user) {
          res.status(400).json({
            error: "Invalid Token",
          });
          return;
        }

        const tpl = await IDETemplateModel.findById(req.query.id);

        if (!tpl) {
          res.status(404).json({
            error: "Template not found",
          });
          return;
        }

        const IDEProject = await IDEProjectModel.findById(tpl.project);
```

```
        tpl.uses++;

        await tpl.save();

        res.status(200).json({
          success: true,
          files: IDEProject.files,
        });
      } catch (e: any) {
        res.status(500).json({
          error: e.message,
        });
      }
    }
  );
}
```

## apps/web/src/pages/api/kids/projects.ts

```
/**
 * apps/web/src/pages/api/kids/projects.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 02.04.2023
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/dist/mongo";
import CodeUpKidsProjectModel from "@codeup/util/src/models/CodeUpKidsProjectModel";
import UserModel from "@codeup/util/dist/models/UserModel";
import jwt from "jsonwebtoken";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (!req.headers["x-jwt"]) {
    res.status(401).json({
      error: "Unauthorized",
    });
    return;
  }

  try {
    await mongoConnect();

    const decoded = jwt.verify(
      req.headers["x-jwt"] as string,
      process.env.JWT_SECRET as string
    );

    const user = await UserModel.findById((decoded as any).id);

    if (!user) {
      res.status(401).json({
        error: "Unauthorized",
      });
      return;
```

```
}

if (req.method === "GET") {
  if (!req.query.id) {
    const projects = await CodeUpKidsProjectModel.find({
      owner: user._id,
    });

    res.status(200).json({
      projects,
    });
    return;
  }

  const project = await CodeUpKidsProjectModel.findById(req.query.id);

  if (!project) {
    res.status(404).json({
      error: "Not Found",
    });
    return;
  }

  if (
    project.owner.toString() !== user._id.toString() &&
    user.role !== "admin"
  ) {
    res.status(401).json({
      error: "Unauthorized",
    });
    return;
  }

  res.status(200).json({
    project,
  });
  return;
}

if (req.method === "POST") {
  if (!req.body || !req.body.name) {
    res.status(400).json({
      error: "Bad Request",
    });
    return;
  }

  if (user.subscription !== "PRO" && user.role !== "admin") {
    let allowed = true;

    const projects = await CodeUpKidsProjectModel.find({
      owner: user._id,
    });

    if (projects.length >= 2) {
      allowed = false;
    }

    if (process.env.DISABLE_PAYMENTS) {
      allowed = true;
```

```
    }

    if (!allowed) {
      res.status(403).json({
        error:
          "Du musst CodeUp PRO abonnieren, um mehr als 2 Kids-Projekte zu erstellen.",
      });
      return;
    }
  }

  const project = await CodeUpKidsProjectModel.create({
    owner: user._id,
    name: req.body.name,
    code:
      req.body.code ||
      '<xml><block type="start" id="|(^9%DCME)E4UEoWv~G]" x="134" y="70"></block></xml>',
  });

  res.status(200).json({
    project,
    success: true,
  });
  return;
}

if (req.method === "PUT") {
  if (!req.body || !req.body.id || !req.body.update) {
    res.status(400).json({
      error: "Bad Request",
    });
    return;
  }

  const project = await CodeUpKidsProjectModel.findById(req.body.id);

  if (!project) {
    res.status(404).json({
      error: "Not Found",
    });
    return;
  }

  if (
    project.owner.toString() !== user._id.toString() &&
    user.role !== "admin"
  ) {
    res.status(401).json({
      error: "Unauthorized",
    });
    return;
  }

  if (req.body.update.name) {
    project.name = req.body.update.name;
  }

  if (req.body.update.code) {
    project.code = req.body.update.code;
  }
```

```
      await project.save();

      res.status(200).json({
        project,
        success: true,
      });
      return;
    }

    if (req.method === "DELETE") {
      if (!req.body || !req.body.id) {
        res.status(400).json({
          error: "Bad Request",
        });
        return;
      }

      const project = await CodeUpKidsProjectModel.findById(req.body.id);

      if (!project) {
        res.status(404).json({
          error: "Not Found",
        });
        return;
      }

      if (
        project.owner.toString() !== user._id.toString() &&
        user.role !== "admin"
      ) {
        res.status(401).json({
          error: "Unauthorized",
        });
        return;
      }

      await project.delete();

      res.status(200).json({
        success: true,
      });
      return;
    }

    res.status(405).json({
      error: "Method Not Allowed",
    });
  } catch (e: any) {
    res.status(500).json({
      error: e.message,
    });
  }
}
```

## apps/web/src/pages/api/newsletter/subscribe.ts

```
/**
 * apps/web/src/pages/api/newsletter/subscribe.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
```

```
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 13.04.2023
 *
 */
import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/src/mongo";
import jwt, { JwtPayload, VerifyErrors } from "jsonwebtoken";
import UserModel from "@codeup/util/src/models/UserModel";
import NewsletterModel from "@codeup/util/src/models/NewsletterModel";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "GET") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  if (!req.headers["x-jwt"]) {
    res.status(400).json({
      error: "Missing Auth-Header (x-jwt)",
    });
    return;
  }

  jwt.verify(
    req.headers["x-jwt"] as string,
    process.env.JWT_SECRET as string,
    async (err: VerifyErrors | null, decoded: JwtPayload | string | any) => {
      if (err) {
        res.status(400).json({
          error: "Invalid token",
        });
        return;
      }

      await mongoConnect();

      const user = await UserModel.findById((decoded as any).id);

      if (!user) {
        res.status(400).json({
          error: "Invalid token",
        });
        return;
      }

      const newsletterExists = await NewsletterModel.findOne({
        user: user._id,
      });

      if (newsletterExists) {
        res.status(400).json({
          error: "You are already subscribed to the newsletter",
        });
        return;
```

```
      }

      const newsletter = await NewsletterModel.create({
        user: user._id,
      });

      await newsletter.save();

      res.status(200).json({
        success: true,
      });
    }
  );
}
```

## apps/web/src/pages/api/newsletter/subscribed.ts

```
/**
 * apps/web/src/pages/api/newsletter/subscribed.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 13.04.2023
 *
 */
import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/src/mongo";
import jwt, { JwtPayload, VerifyErrors } from "jsonwebtoken";
import UserModel from "@codeup/util/src/models/UserModel";
import NewsletterModel from "@codeup/util/src/models/NewsletterModel";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "GET") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  if (!req.headers["x-jwt"]) {
    res.status(400).json({
      error: "Missing Auth-Header (x-jwt)",
    });
    return;
  }

  jwt.verify(
    req.headers["x-jwt"] as string,
    process.env.JWT_SECRET as string,
    async (err: VerifyErrors | null, decoded: JwtPayload | string | any) => {
      if (err) {
        res.status(400).json({
          error: "Invalid token",
        });
        return;
      }
```

```
      await mongoConnect();

      const user = await UserModel.findById((decoded as any).id);

      if (!user) {
        res.status(400).json({
          error: "Invalid token",
        });
        return;
      }

      const newsletterExists = await NewsletterModel.findOne({
        user: user._id,
      });

      if (newsletterExists) {
        res.status(200).json({
          success: true,
          newsletter: true,
          message: "You are subscribed to the newsletter",
        });
        return;
      }

      res.status(200).json({
        success: true,
        newsletter: false,
        message: "You are not subscribed to the newsletter",
      });
    }
  );
}
```

## apps/web/src/pages/api/newsletter/unsubscribe.ts

```
/**
 * apps/web/src/pages/api/newsletter/unsubscribe.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 13.04.2023
 *
 */
import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/src/mongo";
import jwt, { JwtPayload, VerifyErrors } from "jsonwebtoken";
import UserModel from "@codeup/util/src/models/UserModel";
import NewsletterModel from "@codeup/util/src/models/NewsletterModel";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "GET") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }
```

```
  if (!req.headers["x-jwt"]) {
    res.status(400).json({
      error: "Missing Auth-Header (x-jwt)",
    });
    return;
  }

  jwt.verify(
    req.headers["x-jwt"] as string,
    process.env.JWT_SECRET as string,
    async (err: VerifyErrors | null, decoded: JwtPayload | string | any) => {
      if (err) {
        res.status(400).json({
          error: "Invalid token",
        });
        return;
      }

      await mongoConnect();

      const user = await UserModel.findById((decoded as any).id);

      if (!user) {
        res.status(400).json({
          error: "Invalid token",
        });
        return;
      }

      const newsletter = await NewsletterModel.findOne({
        user: user._id,
      });

      if (!newsletter) {
        res.status(400).json({
          error: "You are not subscribed to the newsletter",
        });
        return;
      }

      await newsletter.remove();

      res.status(200).json({
        success: true,
      });
    }
  );
}
```

## apps/web/src/pages/api/orgs/create.ts

```
/**
 * apps/web/src/pages/api/orgs/create.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 04.03.23
 *
 */
import { NextApiRequest, NextApiResponse } from "next";
```

```
import mongoConnect from "@codeup/util/src/mongo";
import jwt from "jsonwebtoken";
import UserModel from "@codeup/util/src/models/UserModel";
import OrgModel from "@codeup/util/src/models/OrgModel";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "POST") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  if (!req.body || !req.headers["x-jwt"] || !req.body.name) {
    res.status(400).json({
      error: "Missing required fields",
    });
    return;
  }

  try {
    jwt.verify(
      req.headers["x-jwt"] as string,
      // @ts-ignore
      process.env.JWT_SECRET as string,
      async (err, decoded) => {
        if (err) {
          res.status(400).json({
            error: "Invalid token",
          });
          return;
        } else {
          await mongoConnect();
          const user = await UserModel.findOne({
            _id: (decoded as any).id as string,
          });

          if (!user) {
            res.status(400).json({
              error: "User not found",
            });
            return;
          }

          try {
            const org = await OrgModel.create({
              owner: user._id,
              name: req.body.name,
              members: [],
              icon: req.body.icon,
              subscription: "FREE",
            });

            res.status(200).json({
              success: true,
              org,
            });
```

```
          } catch (e: any) {
            res.status(400).json({
              success: false,
              error: e.toString(),
            });
          }
        }
      }
    );
  } catch (e) {
    res.status(500).json({
      error: "Invalid token",
    });
  }
}
```

## apps/web/src/pages/api/orgs/delete.ts

```
/**
 * apps/web/src/pages/api/orgs/delete.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 04.03.23
 *
 */
import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/src/mongo";
import jwt from "jsonwebtoken";
import UserModel from "@codeup/util/src/models/UserModel";
import OrgModel from "@codeup/util/src/models/OrgModel";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "POST") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  if (!req.body || !req.headers["x-jwt"] || !req.body.name) {
    res.status(400).json({
      error: "Missing required fields",
    });
    return;
  }

  try {
    jwt.verify(
      req.headers["x-jwt"] as string,
      // @ts-ignore
      process.env.JWT_SECRET as string,
      async (err, decoded) => {
        if (err) {
          res.status(400).json({
            error: "Invalid token",
          });
```

```
          return;
        } else {
          await mongoConnect();
          const user = await UserModel.findOne({
            _id: (decoded as any).id as string,
          });

          if (!user) {
            res.status(400).json({
              error: "User not found",
            });
            return;
          }

          try {
            const org = await OrgModel.findOne({
              name: req.body.name,
            });

            if (
              (org && org.owner.toString() === user._id.toString()) ||
              user.role === "admin"
            )
              org.remove();

            res.status(200).json({
              success: true,
            });
          } catch (e) {
            res.status(400).json({
              success: false,
              error: e,
            });
          }
        }
      }
    );
  } catch (e) {
    res.status(500).json({
      error: "Invalid token",
    });
  }
}
```

## apps/web/src/pages/api/orgs/my.ts

```
/**
 * apps/web/src/pages/api/orgs/my.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 04.03.23
 *
 */
import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/src/mongo";
import jwt, { JwtPayload, VerifyErrors } from "jsonwebtoken";
import UserModel from "@codeup/util/src/models/UserModel";
import OrgModel from "@codeup/util/src/models/OrgModel";
import mongoose from "mongoose";
```

```
export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "GET") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  if (!req.headers["x-jwt"]) {
    res.status(400).json({
      error: "Missing Auth-Header (x-jwt)",
    });
    return;
  }

  jwt.verify(
    req.headers["x-jwt"] as string,
    process.env.JWT_SECRET as string,
    async (err: VerifyErrors | null, decoded: JwtPayload | string | any) => {
      if (err) {
        res.status(400).json({
          error: "Invalid token",
        });
        return;
      }

      await mongoConnect();

      const user = await UserModel.findById((decoded as any).id);

      if (!user) {
        res.status(400).json({
          error: "Invalid token",
        });
        return;
      }

      const ownerOrgs = await OrgModel.find({
        owner: user._id,
      });

      /*
       * OrgModel Structure:
       * ...
       * members: string[]
       * ...
       */

      // a variable containing all orgs where the user is a member
      const memberOrgs = await OrgModel.find({
        members: {
          $in: [user._id.toString()],
        },
      });

      res.status(200).json({
```

```
        success: true,
        orgs: [...ownerOrgs, ...memberOrgs],
      });
    }
  );
}
```

## apps/web/src/pages/api/orgs/[name]/blog.ts

```
/**
 * apps/web/src/pages/api/orgs/[name]/blog.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 19.03.2023
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/dist/mongo";
import UserModel from "@codeup/util/dist/models/UserModel";
import OrgModel from "@codeup/util/dist/models/OrgModel";
import OrgBlogModel from "@codeup/util/src/models/OrgBlogModel";
import jwt from "jsonwebtoken";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (!req.headers["x-jwt"]) {
    res.status(400).json({
      error: "Missing token",
    });
    return;
  }

  try {
    jwt.verify(
      req.headers["x-jwt"] as string,
      // @ts-ignore
      process.env.JWT_SECRET as string,
      async (err, decoded) => {
        if (err) throw new Error("Invalid token");

        await mongoConnect();

        const user = await UserModel.findOne({
          _id: (decoded as any).id as string,
        });

        if (!user) throw new Error("User not found");

        const name = req.query.name;

        const org = await OrgModel.findOne({
          name: name,
        });

        if (!org) throw new Error("Org not found");
```

```
        let permissionLevel = 0;

        if (org.owner.toString() === user._id.toString()) {
          permissionLevel = 2;
        } else if (org.members.includes(user._id.toString())) {
          permissionLevel = 1;
        } else if (user.role === "admin") {
          permissionLevel = 2;
        }

        if (permissionLevel < 1) throw new Error("Permission denied");

        if (req.method === "POST") {
          if (permissionLevel < 2) throw new Error("Permission denied");

          if (!req.body.title || !req.body.content) {
            res.status(400).json({
              error: "Missing data",
              success: false,
            });
            return;
          }

          const blog = await OrgBlogModel.create({
            title: req.body.title,
            content: req.body.content,
            org: org._id,
            createdAt: Date.now(),
          });

          res.status(200).json({
            blog: blog,
            success: true,
          });
        } else if (req.method === "GET") {
          const blogs = await OrgBlogModel.find({
            org: org._id,
          });

          res.status(200).json({
            blogs: blogs,
            success: true,
          });
        }
      }
    );
  } catch (e: any) {
    res.status(500).json({
      error: e.message,
      success: false,
    });
  }
}
```

## apps/web/src/pages/api/orgs/[name]/index.ts

```
/**
 * apps/web/src/pages/api/orgs/[name]/index.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
```

```
 * License: Project License
 * Created At: 07.03.23
 *
 */
import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/src/mongo";
import jwt from "jsonwebtoken";
import UserModel from "@codeup/util/src/models/UserModel";
import OrgModel from "@codeup/util/src/models/OrgModel";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "GET") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  if (!req.headers["x-jwt"]) {
    res.status(400).json({
      error: "Missing token",
    });
    return;
  }

  try {
    jwt.verify(
      req.headers["x-jwt"] as string,
      // @ts-ignore
      process.env.JWT_SECRET as string,
      async (err, decoded) => {
        if (err) {
          res.status(400).json({
            error: "Invalid token",
          });
          return;
        } else {
          await mongoConnect();
          const user = await UserModel.findOne({
            _id: (decoded as any).id as string,
          });

          if (!user) {
            res.status(400).json({
              error: "User not found",
            });
            return;
          }

          const name = req.query.name;

          try {
            let org = await OrgModel.findOne({
              owner: user._id,
              name: name,
            });
```

```
            if (!org) {
              org = await OrgModel.findOne({
                name: name,
                members: {
                  $in: [user._id.toString()],
                },
              });
              if (!org) {
                if (user.role !== "admin") {
                  res.status(404).json({
                    success: false,
                    error: "Organisation not found",
                  });
                  return;
                }
                org = await OrgModel.findOne({
                  name: name,
                });

                if (!org) {
                  res.status(404).json({
                    success: false,
                    error: "Organisation not found",
                  });
                  return;
                }
              }
            }

            res.status(200).json({
              success: true,
              org,
            });
          } catch (e) {
            res.status(400).json({
              success: false,
              error: e,
            });
          }
        }
      }
    );
  } catch (e) {
    res.status(500).json({
      error: "Invalid token",
    });
  }
}
```

## apps/web/src/pages/api/orgs/[name]/members.ts

```
/**
 * apps/web/src/pages/api/orgs/[name]/members.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 10.03.2023
 *
 */
```

```
import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/src/mongo";
import jwt from "jsonwebtoken";
import UserModel from "@codeup/util/src/models/UserModel";
import OrgModel from "@codeup/util/src/models/OrgModel";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "POST") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  /*
   * Body Structure:
   * {
   *  action: "ADD" | "REMOVE",
   *  userId: string
   * }
   */

  if (
    !req.headers["x-jwt"] ||
    !req.body ||
    !req.body.action ||
    !req.body.userId
  ) {
    res.status(400).json({
      error: "Missing token",
    });
    return;
  }

  try {
    jwt.verify(
      req.headers["x-jwt"] as string,
      // @ts-ignore
      process.env.JWT_SECRET as string,
      async (err, decoded) => {
        if (err) {
          res.status(400).json({
            error: "Invalid token",
          });
          return;
        } else {
          await mongoConnect();
          const user = await UserModel.findOne({
            _id: (decoded as any).id as string,
          });

          if (!user) {
            res.status(400).json({
              error: "User not found",
            });
            return;
          }
```

```
const name = req.query.name;

try {
  let org = await OrgModel.findOne({
    name: name,
  });

  if (!org) {
    res.status(400).json({
      success: false,
      error: "Organisation not found",
    });
    return;
  }

  if (req.body.action === "ADD") {
    if (
      org.owner.toString() !== user._id.toString() &&
      user.role !== "admin"
    ) {
      res.status(400).json({
        success: false,
        error: "Not allowed to add members",
      });
      return;
    }
    if (
      org.members.find(
        (m: any) => m.toString() === req.body.userId
      ) ||
      org.owner.toString() === req.body.userId
    ) {
      res.status(400).json({
        success: false,
        error: "User already in organisation",
      });
      return;
    }

    const existingUser = await UserModel.findOne({
      _id: req.body.userId,
    });

    if (!existingUser) {
      res.status(400).json({
        success: false,
        error: "User not found",
      });
      return;
    }

    org.members.push(req.body.userId);
  } else if (req.body.action === "REMOVE") {
    if (
      org.owner.toString() !== user._id.toString() &&
      user._id.toString() !== req.body.userId
    ) {
      res.status(400).json({
        success: false,
```

```
              error: "Not allowed to remove members",
            });
            return;
          }
          const index = org.members.findIndex(
            (m: any) => m.toString() === req.body.userId
          );
          if (index === -1 || org.owner.toString() === req.body.userId) {
            res.status(400).json({
              success: false,
              error: "User not in organisation",
            });
            return;
          }
          org.members.splice(index, 1);
        }

        await org.save();

        res.status(200).json({
          success: true,
        });
      } catch (e) {
        res.status(400).json({
          success: false,
          error: e,
        });
      }
    }
  }
 );
 } catch (e) {
  res.status(500).json({
    error: "Invalid token",
  });
 }
}
```

## apps/web/src/pages/api/orgs/[name]/templates.ts

```
/**
 * apps/web/src/pages/api/orgs/[name]/templates.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 18.03.2023
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/src/mongo";
import jwt from "jsonwebtoken";
import UserModel from "@codeup/util/src/models/UserModel";
import OrgModel from "@codeup/util/src/models/OrgModel";
import OrgProjectTemplateModel from "@codeup/util/src/models/OrgProjectTemplateModel";
import EditorProjectModel from "@codeup/util/dist/models/EditorProjectModel";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
```

```
) {
  if (req.method === "POST") {
    if (
      !req.headers["x-jwt"] ||
      !req.body ||
      !req.body.action ||
      !req.body.projectId
    ) {
      res.status(400).json({
        error: "Missing token",
      });
      return;
    }

    try {
      jwt.verify(
        req.headers["x-jwt"] as string,
        // @ts-ignore
        process.env.JWT_SECRET as string,
        async (err, decoded) => {
          if (err) {
            res.status(400).json({
              error: "Invalid token",
            });
            return;
          } else {
            await mongoConnect();
            const user = await UserModel.findOne({
              _id: (decoded as any).id as string,
            });

            if (!user) {
              res.status(400).json({
                error: "User not found",
              });
              return;
            }

            const name = req.query.name;

            try {
              let org = await OrgModel.findOne({
                name: name,
              });

              if (!org) {
                res.status(400).json({
                  success: false,
                  error: "Organisation not found",
                });
                return;
              }

              if (
                org.owner.toString() !== user._id.toString() &&
                user.role !== "admin"
              ) {
                res.status(400).json({
                  success: false,
                  error: "You are not the owner of this organisation",
```

```
    });
    return;
  }

  if (req.body.action === "ADD") {
    const project = await EditorProjectModel.findById(
      req.body.projectId
    );

    if (
      !project ||
      project.owner.toString() !== user._id.toString()
    ) {
      res.status(400).json({
        success: false,
        error: "Project not found",
      });
      return;
    }

    const currentOrgTemplate =
      await OrgProjectTemplateModel.findOne({
        org: org._id,
        projectId: project._id,
      });

    if (currentOrgTemplate) {
      res.status(400).json({
        success: false,
        error: "Project is already a template",
      });
      return;
    }

    const template = await OrgProjectTemplateModel.create({
      org: org._id,
      projectId: project._id,
      name: project.name,
    });

    res.status(200).json({
      success: true,
      template: template,
    });

    return;
  } else if (req.body.action === "REMOVE") {
    const template = await OrgProjectTemplateModel.findById(
      req.body.projectId
    );

    if (
      !template ||
      template.org.toString() !== org._id.toString()
    ) {
      res.status(400).json({
        success: false,
        error: "Template not found",
      });
      return;
```

```
            }

            await template.remove();

            res.status(200).json({
              success: true,
            });
            return;
          }

          res.status(400).json({
            success: false,
            error: "Action must be type of ADD or REMOVE",
          });
        } catch (e: any) {
          res.status(400).json({
            success: false,
            error: e.message,
          });
        }
      }
    }
  );
 } catch (e) {
  res.status(500).json({
    error: "Invalid token",
  });
 }
} else if (req.method === "GET") {
 if (!req.headers["x-jwt"]) {
  res.status(400).json({
    error: "Missing token",
  });
  return;
 }

 try {
  await mongoConnect();

  const decoded = jwt.verify(
    req.headers["x-jwt"] as string,
    // @ts-ignore
    process.env.JWT_SECRET as string
  );

  if (!decoded) {
    res.status(400).json({
      error: "Invalid token",
    });
    return;
  }

  const user = await UserModel.findOne({
    _id: (decoded as any).id as string,
  });

  if (!user) {
    res.status(400).json({
      error: "User not found",
    });
```

```
      return;
    }

    const org = await OrgModel.findOne({
      name: req.query.name,
    });

    if (!org) {
      res.status(400).json({
        error: "Organisation not found",
      });
      return;
    }

    if (!org.members.includes(user._id) && !org.owner === user._id) {
      res.status(400).json({
        error: "Organisation not found",
      });
      return;
    }

    const templates = await OrgProjectTemplateModel.find({
      org: org._id,
    });

    res.status(200).json({
      success: true,
      templates: templates,
    });
  } catch (e) {
    res.status(500).json({
      error: "Invalid token",
    });
  }
 } else {
  res.status(405).json({
    error: "Method not allowed",
  });
 }
}
```

## apps/web/src/pages/api/orgs/[name]/update.ts

```
/**
 * apps/web/src/pages/api/orgs/[name]/update.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 10.03.2023
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/src/mongo";
import jwt from "jsonwebtoken";
import UserModel from "@codeup/util/src/models/UserModel";
import OrgModel from "@codeup/util/src/models/OrgModel";

export default async function handleRequest(
  req: NextApiRequest,
```

```
  res: NextApiResponse
) {
  if (req.method !== "POST") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  if (!req.headers["x-jwt"] || !req.body || !req.body.update) {
    res.status(400).json({
      error: "Missing token",
    });
    return;
  }

  try {
    jwt.verify(
      req.headers["x-jwt"] as string,
      // @ts-ignore
      process.env.JWT_SECRET as string,
      async (err, decoded) => {
        if (err) {
          res.status(400).json({
            error: "Invalid token",
          });
          return;
        } else {
          await mongoConnect();
          const user = await UserModel.findOne({
            _id: (decoded as any).id as string,
          });

          if (!user) {
            res.status(400).json({
              error: "User not found",
            });
            return;
          }

          const name = req.query.name;

          try {
            let org = await OrgModel.findOne({
              name: name,
            });

            if (
              org.owner.toString() !== user._id.toString() &&
              user.role !== "admin"
            ) {
              res.status(400).json({
                success: false,
                error: "You are not allowed to do this",
              });
              return;
            }

            const allowedKeys = ["name", "icon"];
```

```
            for (const key in req.body.update) {
              if (allowedKeys.includes(key)) {
                if (key === "name") {
                  const exists = await OrgModel.findOne({
                    name: req.body.update[key],
                  });

                  if (exists) {
                    res.status(400).json({
                      success: false,
                      error: "Name already exists",
                    });
                    return;
                  }

                  org.name = req.body.update[key];
                } else {
                  org[key] = req.body.update[key];
                }
              }
            }

            await org.save();

            res.status(200).json({
              success: true,
              org,
            });
          } catch (e) {
            res.status(400).json({
              success: false,
              error: e,
            });
          }
        }
      }
    );
  } catch (e) {
    res.status(500).json({
      error: "Invalid token",
    });
  }
}
```

## apps/web/src/pages/api/orgs/[name]/use-template.ts

```
/**
 * apps/web/src/pages/api/orgs/[name]/use-template.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 18.03.2023
 *
 */
import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/src/mongo";
import jwt from "jsonwebtoken";
import UserModel from "@codeup/util/src/models/UserModel";
import OrgModel from "@codeup/util/src/models/OrgModel";
import OrgProjectTemplateModel from "@codeup/util/src/models/OrgProjectTemplateModel";
```

```
import EditorProjectModel from "@codeup/util/dist/models/EditorProjectModel";
import EditorProjectPageModel from "@codeup/util/dist/models/EditorProjectPageModel";
import EditorProjectFunctionModel from "@codeup/util/dist/models/EditorProjectFunctionModel";
import { v4 } from "uuid";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method === "GET") {
    if (!req.headers["x-jwt"]) {
      res.status(400).json({
        error: "Missing token",
      });
      return;
    }

    if (!req.query.templateId || !req.query.newProjectName) {
      res.status(400).json({
        error: "Missing templateId or newProjectName",
      });
      return;
    }

    const templateId = req.query.templateId as string;
    const newProjectName = req.query.newProjectName as string;

    try {
      await mongoConnect();

      jwt.verify(
        req.headers["x-jwt"] as string,
        // @ts-ignore
        process.env.JWT_SECRET as string,
        async (err, decoded) => {
          if (!decoded) {
            res.status(400).json({
              error: "Invalid token",
            });
            return;
          }

          const user = await UserModel.findOne({
            _id: (decoded as any).id as string,
          });

          if (!user) {
            res.status(400).json({
              error: "User not found",
            });
            return;
          }

          const template = await OrgProjectTemplateModel.findOne({
            _id: templateId,
          });

          if (!template) {
            res.status(400).json({
              error: "Template not found",
```

```
  });
  return;
}

const org = await OrgModel.findOne({
  _id: template.org,
});

if (!org) {
  res.status(400).json({
    error: "Org not found",
  });
  return;
}

if (
  !org.members.includes(user._id) &&
  org.owner.toString() !== user._id.toString()
) {
  res.status(400).json({
    error: "User is not a member of the org",
  });
  return;
}

const newProject = await EditorProjectModel.create({
  name: newProjectName,
  owner: user._id,
  customDomain: "disabled:" + v4(),
});

const pages = await EditorProjectPageModel.find({
  project: template.projectId,
});

for (const page of pages) {
  await EditorProjectPageModel.create({
    project: newProject._id,
    type: page.type,
    mainContent: page.mainContent,
    resources: page.resources,
    url: page.url,
  });
}

const functions = await EditorProjectFunctionModel.find({
  project: template.projectId,
});

for (const func of functions) {
  await EditorProjectFunctionModel.create({
    path: func.path,
    code: func.code,
    project: newProject._id,
  });
}

res.status(200).json({
  success: true,
  project: newProject,
```

```
          });
        }
      );
    } catch (e) {
      res.status(500).json({
        error: "Invalid token",
      });
    }
  } else {
    res.status(405).json({
      error: "Method not allowed",
    });
  }
}
```

## apps/web/src/pages/api/payments/cancelSubscription.ts

```
/**
 * apps/web/src/pages/api/payments/cancelSubscription.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 18.06.23
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/src/mongo";
import UserModel from "@codeup/util/dist/models/UserModel";
import { decode, verify } from "jsonwebtoken";
import Stripe from "stripe";
import generateJWT from "@codeup/util/dist/generateJWT";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  try {
    const jwt = req.headers["x-jwt"] as string;

    if (!jwt) {
      res.status(401).json({
        error: "Unauthorized",
      });
      return;
    }

    const decoded = verify(jwt, process.env.JWT_SECRET as string) as any;

    if (!decoded) {
      res.status(401).json({
        error: "Unauthorized",
      });
      return;
    }

    const user = await UserModel.findById(decoded.id);

    if (!user) {
      res.status(401).json({
```

```
        error: "Unauthorized",
      });
      return;
    }

    if (user.subscription !== "PRO") {
      res.status(403).json({
        error: "Forbidden",
      });
      return;
    }

    if (!user.stripeSubscriptionId || user.stripeSubscriptionId.length === 0) {
      res.status(400).json({
        error: "Bad Request",
      });
      return;
    }

    const stripe = new Stripe(process.env.STRIPE_SECRET_KEY as string, {
      apiVersion: "2022-11-15",
    });

    const subscription = await stripe.subscriptions.retrieve(
      user.stripeSubscriptionId
    );

    if (!subscription) {
      res.status(400).json({
        error: "Bad Request",
      });
      return;
    }

    await stripe.subscriptions.cancel(user.stripeSubscriptionId);

    user.subscription = "FREE";
    user.stripeSubscriptionId = "";
    await user.save();

    res.status(200).json({
      success: true,
      jwt: generateJWT(user),
    });
  } catch (e: any) {
    res.status(500).json({
      error: e.message,
    });
  }
}
```

## apps/web/src/pages/api/payments/createCheckoutSession.ts

```
/**
 * apps/web/src/pages/api/payments/createCheckoutSession.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 16.06.2023
 *
```

```
 */

import { NextApiRequest, NextApiResponse } from "next";
import Stripe from "stripe";
import { decode } from "jsonwebtoken";
import UserModel from "@codeup/util/dist/models/UserModel";
import mongoConnect from "@codeup/util/dist/mongo";

export default async function handler(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (process.env.DISABLE_PAYMENTS) {
    res.status(403).json({
      error: "Payments are disabled",
    });
    return;
  }

  try {
    await mongoConnect();

    if (req.method !== "POST") {
      res.status(405).json({
        error: "Method not allowed",
      });
      return;
    }

    const jwt = req.headers["x-jwt"] as string;

    if (!jwt) {
      res.status(401).json({
        error: "Unauthorized",
      });
      return;
    }

    const decoded = (await decode(jwt)) as any;

    if (!decoded) {
      res.status(401).json({
        error: "Unauthorized",
      });
      return;
    }

    const user = await UserModel.findById(decoded["id"] as string);

    if (!user) {
      res.status(401).json({
        error: "Unauthorized",
      });
      return;
    }

    if (!user.active || user.role === "admin" || user.subscription === "PRO") {
      res.status(403).json({
        error: "Forbidden",
      });
```

```
      return;
    }

    const stripe = new Stripe(process.env.STRIPE_SECRET_KEY as string, {
      apiVersion: "2022-11-15",
    });

    let customerId: string = "";

    if (!user.stripeCustomerId || user.stripeCustomerId.length === 0) {
      customerId = (
        await stripe.customers.create({
          email: user.email,
        })
      ).id;
    } else {
      customerId = user.stripeCustomerId;
    }

    user.stripeCustomerId = customerId;
    await user.save();

    const session = await stripe.checkout.sessions.create({
      success_url:
`${req.headers.origin}/api/payments/subscription_callback?session_id={CHECKOUT_SESSION_ID}`,
      mode: "subscription",
      customer: customerId,
      line_items: [
        {
          price: process.env.STRIPE_CODEUP_PRO_PRICE_ID as string,
          quantity: 1,
        },
      ],
      allow_promotion_codes: true,
    });

    res.status(200).json({
      url: session.url as string,
    });
  } catch (e: any) {
    res.status(500).json({
      error: e.message,
    });
  }
}
```

## apps/web/src/pages/api/payments/subscription_callback.ts

```
/**
 * apps/web/src/pages/api/payments/subscription_callback.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 16.06.2023
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import Stripe from "stripe";
import { decode } from "jsonwebtoken";
```

```
import UserModel from "@codeup/util/dist/models/UserModel";
import mongoConnect from "@codeup/util/dist/mongo";

export default async function handler(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (process.env.DISABLE_PAYMENTS) {
    res.status(403).json({
      error: "Payments are disabled",
    });
    return;
  }
  try {
    await mongoConnect();

    if (req.method !== "GET") {
      res.status(405).json({
        error: "Method not allowed",
      });
      return;
    }

    const { session_id } = req.query;

    if (!session_id) {
      res.redirect(
        "/error?error=Keine Session ID übermittelt. Bitte kontaktiere den Support."
      );
      return;
    }

    const stripe = new Stripe(process.env.STRIPE_SECRET_KEY as string, {
      apiVersion: "2022-11-15",
    });

    const session = await stripe.checkout.sessions.retrieve(
      session_id as string
    );

    if (!session) {
      res.redirect(
        "/error?error=Die Session ID ist ungültig. Bitte kontaktiere den Support."
      );
      return;
    }

    console.log(session.customer);

    const user = await UserModel.findOne({
      stripeCustomerId: session.customer,
    });

    if (!user) {
      res.redirect(
        "/error?error=Der Benutzer konnte nicht gefunden werden. Bitte kontaktiere den Support."
      );
      return;
    }
```

```
    if (session.payment_status === "paid") {
      user.subscription = "PRO";
      user.stripeSubscriptionId = session.subscription;
      await user.save();
      res.redirect("/payment/success/thank_you");
    } else {
      res.redirect(
        "/error?error=Die Zahlung ist fehlgeschlagen. Bitte kontaktiere den Support."
      );
    }
  } catch (e: any) {
    res.status(500).json({
      error: e.message,
    });
  }
}
```

## apps/web/src/pages/api/project-ideas.ts

```
/**
 * apps/web/src/pages/api/project-ideas.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 03.12.2022
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import ProjectIdeaModel from "@codeup/util/src/models/ProjectIdeaModel";

export default async (req: NextApiRequest, res: NextApiResponse) => {
  if (req.method === "GET") {
    const projectIdeas = await ProjectIdeaModel.find({});
    res.status(200).json({ ideas: projectIdeas });
  } else {
    res.status(404).json({ error: "Not Found" });
  }
};
```

## apps/web/src/pages/api/projects/create.ts

```
/**
 * apps/web/src/pages/api/projects/create.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2024 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 21.05.2024
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import { isLoggedIn } from "../../../global/server/isLoggedIn";
import ProjectModel from "@codeup/util/dist/models/ProjectModel";
import simpleGit from "simple-git";
import * as fs from "fs";
import * as path from "path";
import { getS3 } from "../../../global/getS3";

export default async function (req: NextApiRequest, res: NextApiResponse) {
```

```
  try {
    const { auth, user } = await isLoggedIn(req, res);
    if (!auth) return;

    const { name, projectType, modifiers } = req.body;
    if (!name || !projectType || !modifiers) {
      return res.status(400).json({
        status: 400,
        error: "Missing data",
      });
    }

    const project = await ProjectModel.create({
      name,
      projectType,
      modifiers,
      users: [user._id],
    });

    const GIT_HOME = process.env.GIT_DATA_HOME as string;

    fs.mkdirSync(path.join(GIT_HOME, project._id.toString()), {
      recursive: true,
    });

    const git = simpleGit({
      baseDir: path.join(GIT_HOME, project._id.toString()),
    });

    await git.clone(
      "https://github.com/CodeUpProjects/template-" + projectType + ".git",
      "."
    );

    const client = getS3();

    await client.makeBucket(
      "codeup-project-" + project._id.toString(),
      "eu-central-1"
    );
    res.status(200).json({
      status: 200,
      data: project,
      s3_bucket: "codeup-project-" + project._id.toString(),
    });
  } catch (e: any) {
    console.log(e);
    res.status(500).json({
      status: 500,
      error: e.message,
    });
  }
}
```

## apps/web/src/pages/api/projects/my.ts

```
/**
 * apps/web/src/pages/api/projects/my.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2024 Ben Siebert. All rights reserved.
```

```
 * License: Project License
 * Created At: 22.05.2024
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import { isLoggedIn } from "../../../global/server/isLoggedIn";
import ProjectModel from "@codeup/util/dist/models/ProjectModel";

export default async function (req: NextApiRequest, res: NextApiResponse) {
  try {
    const { user, auth } = await isLoggedIn(req, res);

    if (!auth) return;

    const projects = await ProjectModel.find({
      users: {
        $in: [user._id],
      },
    });

    res.status(200).json({
      status: 200,
      data: projects,
    });
  } catch (e: any) {
    res.status(500).json({
      status: 500,
      error: e.message,
    });
  }
}
```

## apps/web/src/pages/api/projects/[id]/download.ts

```
/**
 * apps/web/src/pages/api/projects/[id]/download.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2024 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 26.05.2024
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import jwt, { JwtPayload, VerifyErrors } from "jsonwebtoken";
import mongoConnect from "@codeup/util/dist/mongo";
import UserModel from "@codeup/util/dist/models/UserModel";
import ProjectModel from "@codeup/util/dist/models/ProjectModel";
import * as fs from "fs";
import * as path from "path";
import { tmpdir } from "os";
import archiver from "archiver";

export default async function (req: NextApiRequest, res: NextApiResponse) {
  try {
    await mongoConnect();
    const { token, id } = req.query;

    if (!id || !token) {
```

```
  res.redirect("/error?error=Invalid Request");
  return;
}

jwt.verify(
  token as string,
  process.env.JWT_SECRET as string,
  async (err: VerifyErrors | null, decoded: JwtPayload | string | any) => {
    if (err) {
      res.redirect("/error?error=Invalid Token");
      return;
    }

    const user = await UserModel.findById((decoded as any).id);

    if (!user) {
      res.redirect("/error?error=Invalid Token");
      return;
    }

    const project = await ProjectModel.findById(id);

    if (!project) {
      res.redirect("/error?error=Project not found");
      return;
    }

    if (
      !project.users.includes(user._id) &&
      !project.modifiers.includes("dl-public")
    ) {
      res.redirect(
        "/error?error=You are not allowed to download this project"
      );
      return;
    }

    if (
      !fs.existsSync(
        path.join(process.env.GIT_DATA_HOME as string, id as string)
      )
    ) {
      res.redirect("/error?error=Project not found");
    }

    const a = archiver("zip", {
      zlib: { level: 9 },
    });

    const name = `${id}-${Math.random().toString(36).substring(7)}.zip`;

    const output = fs.createWriteStream(path.join(tmpdir(), name));

    a.pipe(output);

    a.directory(
      path.join(process.env.GIT_DATA_HOME as string, id as string),
      false
    );
```

```
        a.on("end", () => {
          res.writeHead(200, {
            "Content-Type": "application/zip",
            "Content-Disposition": `attachment; filename=${id}.zip`,
          });

          const stream = fs.createReadStream(path.join(tmpdir(), name));

          stream.on("data", (data) => {
            res.write(data);
          });

          stream.on("end", () => {
            res.end();
            fs.unlinkSync(path.join(tmpdir(), name));
          });

          stream.on("error", () => {
            res.redirect(
              "/error?error=An error occurred while downloading the project"
            );
          });
        });

        a.finalize();
      }
    );
  } catch (e: any) {
    res.status(500).json({
      status: 500,
      error: e.message,
    });
  }
}
```

## apps/web/src/pages/api/projects/[id]/files/[[...path]].ts

```
/**
 * apps/web/src/pages/api/projects/[id]/files/[[...path]].ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2024 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 29.05.2024
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/dist/mongo";
import { isLoggedIn } from "../../../../../global/server/isLoggedIn";
import ProjectModel from "@codeup/util/dist/models/ProjectModel";
import fs from "fs";
import path from "path";
import simpleGit from "simple-git";

export default async function (req: NextApiRequest, res: NextApiResponse) {
  try {
    await mongoConnect();

    if (!req.query.id) {
      return res.status(400).json({
```

```
    status: 400,
    error: "Missing data",
  });
}

const { auth, user } = await isLoggedIn(req, res);

if (!auth) return;

const project = await ProjectModel.findById(req.query.id as string);

if (!project) {
  return res.status(404).json({
    status: 404,
    error: "Project not found",
  });
}

if (!project.users.includes(user._id)) {
  return res.status(403).json({
    status: 403,
    error: "You are not allowed to access this project",
  });
}

if (
  !fs.existsSync(
    path.join(process.env.GIT_DATA_HOME as string, project._id.toString())
  )
) {
  return res.status(404).json({
    status: 404,
    error: "Project not found",
  });
}

const file = req.query.path as string[];

const baseDir = path.resolve(
  process.env.GIT_DATA_HOME as string,
  project._id.toString()
);

const relativePath = path.join(baseDir, file.join("/"));

if (!path.resolve(relativePath).startsWith(baseDir)) {
  res.status(403).json({
    status: 403,
    error: "You are not allowed to access this file",
  });
  return;
}
if (req.method === "GET") {
  const filePath = path.resolve(baseDir, relativePath);

  if (!fs.existsSync(filePath)) {
    res.status(404).json({
      status: 404,
      error: "File not found",
    });
```

```
    return;
  }

  const content = fs.readFileSync(filePath, "utf-8");

  res.status(200).json({
    status: 200,
    data: content,
  });
} else if (req.method === "POST") {
  const filePath = path.resolve(baseDir, relativePath);

  if (req.body.folder) {
    fs.mkdirSync(filePath, { recursive: true });
    res.status(200).json({
      status: 200,
      data: "Folder created",
    });

    return;
  }

  const content = req.body.content;

  fs.mkdirSync(path.dirname(filePath), { recursive: true });

  fs.writeFileSync(filePath, content);

  res.status(200).json({
    status: 200,
    data: "File saved",
  });
} else if (req.method === "DELETE") {
  const filePath = path.resolve(baseDir, relativePath);

  if (!filePath.startsWith(baseDir)) {
    res.status(403).json({
      status: 403,
      error: "You are not allowed to access this file",
    });
    return;
  }

  if (!fs.existsSync(filePath)) {
    res.status(404).json({
      status: 404,
      error: "File not found",
    });
    return;
  }

  if (fs.lstatSync(filePath).isDirectory()) {
    fs.rmdirSync(filePath, { recursive: true });
  } else {
    fs.unlinkSync(filePath);
  }

  res.status(200).json({
    status: 200,
    data: "File deleted",
```

```
      });
    } else if (req.method === "PUT") {
      const filePath = path.resolve(baseDir, relativePath);
      const newFilePath = path.resolve(
        baseDir,
        path.join(path.dirname(relativePath), req.body.newName)
      );

      if (!fs.existsSync(filePath))
        return res.status(404).json({
          status: 404,
          error: "File not found",
        });

      if (fs.existsSync(newFilePath))
        return res.status(409).json({
          status: 409,
          error: "File already exists",
        });

      if (!newFilePath.startsWith(baseDir))
        return res.status(403).json({
          status: 403,
          error: "You are not allowed to access this file",
        });

      fs.renameSync(filePath, newFilePath);

      res.status(200).json({
        status: 200,
        data: "File renamed",
      });
    }
  } catch (e: any) {
    res.status(500).json({
      status: 500,
      error: e.message,
    });
  }
}
```

## apps/web/src/pages/api/projects/[id]/flows.ts

```
/**
 * apps/web/src/pages/api/projects/[id]/flows.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2024 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 01.06.2024
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/dist/mongo";
import { isLoggedIn } from "../../../../global/server/isLoggedIn";
import ProjectModel from "@codeup/util/dist/models/ProjectModel";
import ProjectFlowModel from "@codeup/util/dist/models/ProjectFlowModel";

export default async function (req: NextApiRequest, res: NextApiResponse) {
  try {
```

```
await mongoConnect();

const { auth, user } = await isLoggedIn(req, res);

if (!auth) return;

const project = await ProjectModel.findById(req.query.id);

if (!project) {
  res.status(404).json({
    status: 404,
    error: "Project not found",
  });
  return;
}

if (!project.users.includes(user._id)) {
  res.status(403).json({
    status: 400,
    error: "You are not allowed to view this project",
  });
  return;
}

if (req.method === "GET") {
  if (req.query.flowId) {
    const flow = await ProjectFlowModel.findById(
      req.query.flowId as string
    );

    if (flow.project.toString() !== project._id.toString()) {
      res.status(403).json({
        status: 403,
        error: "You are not allowed to view this flow",
      });
      return;
    }

    res.status(200).json({
      status: 200,
      flow,
    });
    return;
  }

  const flows = await ProjectFlowModel.find({
    project: project._id,
  });

  res.status(200).json({
    status: 200,
    flows,
  });
  return;
} else if (req.method === "POST") {
  const flow = await ProjectFlowModel.create({
    project: project._id,
    saved: "",
    name: req.body.name,
  });
```

```
      res.status(200).json({
        status: 200,
        flow,
      });
      return;
    } else if (req.method === "PUT") {
      const flow = await ProjectFlowModel.findById(req.query.flowId as string);

      if (flow.project.toString() !== project._id.toString()) {
        res.status(403).json({
          status: 403,
          error: "You are not allowed to edit this flow",
        });
        return;
      }

      flow.saved = req.body.saved;
      flow.name = req.body.name;

      await flow.save();

      res.status(200).json({
        status: 200,
        flow,
      });
    } else if (req.method === "DELETE") {
      const flow = await ProjectFlowModel.findById(req.query.flowId as string);

      if (flow.project.toString() !== project._id.toString()) {
        res.status(403).json({
          status: 403,
          error: "You are not allowed to delete this flow",
        });
        return;
      }

      await flow.delete();

      res.status(200).json({
        status: 200,
        flow,
      });
    }
  } catch (e: any) {
    res.status(500).json({
      status: 500,
      error: e.message,
    });
  }
}
```

## apps/web/src/pages/api/projects/[id]/gitops/branch.ts

```
/**
 * apps/web/src/pages/api/projects/[id]/gitops/branch.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2024 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 29.05.2024
```

```
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/dist/mongo";
import { isLoggedIn } from "../../../../../global/server/isLoggedIn";
import ProjectModel from "@codeup/util/dist/models/ProjectModel";
import * as fs from "fs";
import * as path from "path";
import simpleGit from "simple-git";

export default async function (req: NextApiRequest, res: NextApiResponse) {
  try {
    await mongoConnect();

    if (!req.query.id) {
      return res.status(400).json({
        status: 400,
        error: "Missing data",
      });
    }

    const { auth, user } = await isLoggedIn(req, res);

    if (!auth) return;

    const project = await ProjectModel.findById(req.query.id as string);

    if (!project) {
      return res.status(404).json({
        status: 404,
        error: "Project not found",
      });
    }

    if (!project.users.includes(user._id)) {
      return res.status(403).json({
        status: 403,
        error: "You are not allowed to access this project",
      });
    }

    if (
      !fs.existsSync(
        path.join(process.env.GIT_DATA_HOME as string, project._id.toString())
      )
    ) {
      return res.status(404).json({
        status: 404,
        error: "Project not found",
      });
    }

    const GIT_HOME = process.env.GIT_DATA_HOME as string;

    const git = simpleGit({
      baseDir: path.join(GIT_HOME, project._id.toString()),
    });

    if (req.method === "GET") {
```

```
      res.status(200).json({
        status: 200,
        data: await git.branch(),
      });
    } else if (req.method === "POST") {
      const { branch } = req.body;

      if (!(await git.branch()).all.find((b) => b === branch)) {
        await git.branch([branch]);
      }
      await git.checkout(branch);

      res.status(200).json({
        status: 200,
        data: await git.branch(),
      });
    }
  } catch (e: any) {
    res.status(500).json({
      status: 500,
      error: e.message,
    });
  }
}
```

## apps/web/src/pages/api/projects/[id]/gitops/commit.ts

```
/**
 * apps/web/src/pages/api/projects/[id]/gitops/commit.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2024 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 29.05.2024
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/dist/mongo";
import { isLoggedIn } from "../../../../../global/server/isLoggedIn";
import ProjectModel from "@codeup/util/dist/models/ProjectModel";
import * as fs from "fs";
import * as path from "path";
import simpleGit from "simple-git";

export default async function (req: NextApiRequest, res: NextApiResponse) {
  try {
    await mongoConnect();

    if (req.method !== "POST") {
      return res.status(405).json({
        status: 405,
        error: "Method not allowed",
      });
    }

    if (!req.query.id) {
      return res.status(400).json({
        status: 400,
        error: "Missing data",
      });
```

```
    }

    const { auth, user } = await isLoggedIn(req, res);

    if (!auth) return;

    const project = await ProjectModel.findById(req.query.id as string);

    if (!project) {
      return res.status(404).json({
        status: 404,
        error: "Project not found",
      });
    }

    if (!project.users.includes(user._id)) {
      return res.status(403).json({
        status: 403,
        error: "You are not allowed to access this project",
      });
    }

    if (
      !fs.existsSync(
        path.join(process.env.GIT_DATA_HOME as string, project._id.toString())
      )
    ) {
      return res.status(404).json({
        status: 404,
        error: "Project not found",
      });
    }

    const GIT_HOME = process.env.GIT_DATA_HOME as string;

    const git = simpleGit({
      baseDir: path.join(GIT_HOME, project._id.toString()),
    });

    git.addConfig("user.email", user.email);
    git.addConfig("user.name", user.username);

    const { message } = req.body;

    await git.add(".");
    await git.commit(message);

    res.status(200).json({
      status: 200,
      data: "Commit done",
    });
  } catch (e: any) {
    res.status(500).json({
      status: 500,
      error: e.message,
    });
  }
}
```

apps/web/src/pages/api/projects/[id]/gitops/commits.ts

```
/**
 * apps/web/src/pages/api/projects/[id]/gitops/commits.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2024 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 29.05.2024
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/dist/mongo";
import { isLoggedIn } from "../../../../../global/server/isLoggedIn";
import ProjectModel from "@codeup/util/dist/models/ProjectModel";
import * as fs from "fs";
import * as path from "path";
import simpleGit from "simple-git";

export default async function (req: NextApiRequest, res: NextApiResponse) {
  try {
    await mongoConnect();

    if (req.method !== "GET") {
      return res.status(405).json({
        status: 405,
        error: "Method not allowed",
      });
    }

    if (!req.query.id) {
      return res.status(400).json({
        status: 400,
        error: "Missing data",
      });
    }

    const { auth, user } = await isLoggedIn(req, res);

    if (!auth) return;

    const project = await ProjectModel.findById(req.query.id as string);

    if (!project) {
      return res.status(404).json({
        status: 404,
        error: "Project not found",
      });
    }

    if (!project.users.includes(user._id)) {
      return res.status(403).json({
        status: 403,
        error: "You are not allowed to access this project",
      });
    }

    if (
      !fs.existsSync(
        path.join(process.env.GIT_DATA_HOME as string, project._id.toString())
      )
```

```
    ) {
      return res.status(404).json({
        status: 404,
        error: "Project not found",
      });
    }

    const GIT_HOME = process.env.GIT_DATA_HOME as string;

    const git = simpleGit({
      baseDir: path.join(GIT_HOME, project._id.toString()),
    });

    const x = await git.log();
    const all_commits = x.all;
    const total_commits = x.total;

    res.status(200).json({
      status: 200,
      data: {
        all: all_commits,
        total: total_commits,
      },
    });
  } catch (e: any) {
    res.status(500).json({
      status: 500,
      error: e.message,
    });
  }
}
```

## apps/web/src/pages/api/projects/[id]/gitops/tree.ts

```
/**
 * apps/web/src/pages/api/projects/[id]/gitops/tree.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2024 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 29.05.2024
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/dist/mongo";
import { isLoggedIn } from "../../../../../global/server/isLoggedIn";
import ProjectModel from "@codeup/util/dist/models/ProjectModel";
import * as fs from "fs";
import * as path from "path";

export default async function (req: NextApiRequest, res: NextApiResponse) {
  try {
    await mongoConnect();

    if (req.method !== "GET") {
      return res.status(405).json({
        status: 405,
        error: "Method not allowed",
      });
    }
```

```
if (!req.query.id) {
  return res.status(400).json({
    status: 400,
    error: "Missing data",
  });
}

const { auth, user } = await isLoggedIn(req, res);

if (!auth) return;

const project = await ProjectModel.findById(req.query.id as string);

if (!project) {
  return res.status(404).json({
    status: 404,
    error: "Project not found",
  });
}

if (!project.users.includes(user._id)) {
  return res.status(403).json({
    status: 403,
    error: "You are not allowed to access this project",
  });
}

if (
  !fs.existsSync(
    path.join(process.env.GIT_DATA_HOME as string, project._id.toString())
  )
) {
  return res.status(404).json({
    status: 404,
    error: "Project not found",
  });
}

type entry = {
  name: string;
  type: string;
  children: entry[];
  content?: any;
};

let tree: entry[] = [];

const walk = (dir: string, parent: entry) => {
  const files = fs.readdirSync(dir);
  const children: entry[] = [];
  for (const file of files) {
    //if (file === ".git") continue;
    const stat = fs.statSync(path.join(dir, file));
    if (stat.isDirectory()) {
      const entry: entry = {
        name: file,
        type: "directory",
        children: [],
      };
```

```
      walk(path.join(dir, file), entry);
      children.push(entry);
    } else {
      let content = "";
      if (req.query.withContent) {
        content = fs.readFileSync(path.join(dir, file), "utf-8");
      }
      children.push({
        name: file,
        type: "file",
        children: [],
        content,
      });
    }
  }

  parent.children = children;
};

const parent: entry = {
  name: project.name,
  type: "directory",
  children: tree,
};
walk(
  path.join(process.env.GIT_DATA_HOME as string, project._id.toString()),
  parent
);

for (const c of parent.children) {
  tree.push(c);
}

let x: {
  [key: string]: any;
} = {};

const createWCStructure = (
  e: entry
): {
  [key: string]: any;
} => {
  let result: any = {};

  for (const c of e.children) {
    if (c.type === "directory") {
      result[c.name] = {
        directory: createWCStructure(c),
      };
    } else {
      result[c.name] = {
        file: {
          contents: c.content,
        },
      };
    }
  }

  return result;
};
```

```
    if (req.query.withContent && req.query.withContent === "webcontainer") {
      x = createWCStructure(parent);
    }

    res.status(200).json({
      status: 200,
      tree,
      data: x,
    });
  } catch (e: any) {
    res.status(500).json({
      status: 500,
      error: e.message,
    });
  }
}
```

## apps/web/src/pages/api/projects/[id]/i18n/getTranslation.ts

```
/**
 * apps/web/src/pages/api/projects/[id]/i18n/getTranslation.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2024 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 05.06.2024
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/dist/mongo";
import ProjectModel from "@codeup/util/dist/models/ProjectModel";
import ProjectI18nModel from "@codeup/util/dist/models/ProjectI18nModel";

export default async function (req: NextApiRequest, res: NextApiResponse) {
  try {
    await mongoConnect();

    const project = await ProjectModel.findById(req.query.id);

    if (!project) {
      res.status(404).json({
        status: 404,
        error: "Project not found",
      });
      return;
    }

    const i18n = await ProjectI18nModel.findOne({ project: project._id });

    if (!i18n) {
      res.status(404).json({
        status: 404,
        error: "No i18n found for this project",
      });
      return;
    }
    const { key, language } = req.query;

    if (!key || !language) {
      res.status(400).json({
```

```
        status: 400,
        error: "You need to provide a key and a language",
      });
      return;
    }

    if (!i18n.translations[language as string]) {
      res.status(200).json({
        status: 200,
        translation: "codeup.i18n.languageNotFound",
      });
      return;
    }

    if (!i18n.translations[language as string][key as string]) {
      res.status(200).json({
        status: 200,
        translation: key,
      });
      return;
    }

    res.status(200).json({
      status: 200,
      translation: i18n.translations[language as string][key as string],
    });
  } catch (e: any) {
    res.status(500).json({
      status: 500,
      translation: "codeup.i18n.error",
      error: e.message,
    });
  }
}
```

## apps/web/src/pages/api/projects/[id]/i18n/index.ts

```
/**
 * apps/web/src/pages/api/projects/[id]/i18n/index.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2024 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 02.06.2024
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/dist/mongo";
import { isLoggedIn } from "../../../../../global/server/isLoggedIn";
import ProjectModel from "@codeup/util/dist/models/ProjectModel";
import ProjectI18nModel from "@codeup/util/dist/models/ProjectI18nModel";

export default async function (req: NextApiRequest, res: NextApiResponse) {
  try {
    await mongoConnect();

    const { auth, user } = await isLoggedIn(req, res);

    if (!auth) return;
```

```
const project = await ProjectModel.findById(req.query.id);

if (!project) {
  res.status(404).json({
    status: 404,
    error: "Project not found",
  });
  return;
}

if (!project.users.includes(user._id)) {
  res.status(403).json({
    status: 400,
    error: "You are not allowed to view this project",
  });
  return;
}

if (req.method === "GET") {
  let i18n = await ProjectI18nModel.findOne({ project: project._id });

  if (!i18n) {
    i18n = await ProjectI18nModel.create({
      project: project._id,
      defaultLanguage: "de",
      languages: ["de", "en"],
      translations: {
        de: {
          "product.name": project.name,
        },
      },
    });
  }

  res.status(200).json({
    status: 200,
    i18n,
  });
  return;
} else if (req.method === "POST") {
  const i18n = await ProjectI18nModel.findOne({ project: project._id });

  if (!i18n) {
    res.status(404).json({
      status: 404,
      error: "I18n not found",
    });
    return;
  }

  const { action } = req.body;

  if (action === "addLanguage") {
    const { language } = req.body;

    if (i18n.languages.includes(language)) {
      res.status(400).json({
        status: 400,
        error: "Language already exists",
      });
```

```
    return;
  }

  i18n.languages.push(language);

  await i18n.save();

  res.status(200).json({
    status: 200,
    i18n,
  });
} else if (action === "removeLanguage") {
  const { language } = req.body;

  if (!i18n.languages.includes(language)) {
    res.status(400).json({
      status: 400,
      error: "Language not found",
    });
    return;
  }

  if (i18n.languages.length === 1) {
    res.status(400).json({
      status: 400,
      error: "You can't remove the last language",
    });
    return;
  }

  i18n.languages = i18n.languages.filter((l: string) => l !== language);

  if (i18n.defaultLanguage === language) {
    i18n.defaultLanguage = i18n.languages[0];
  }

  await i18n.save();

  res.status(200).json({
    status: 200,
    i18n,
  });
} else if (action === "setDefaultLanguage") {
  const { language } = req.body;

  if (!i18n.languages.includes(language)) {
    i18n.languages.push(language);
  }

  i18n.defaultLanguage = language;

  await i18n.save();

  res.status(200).json({
    status: 200,
    i18n,
  });
} else if (action === "setKey") {
  const { key, value, language } = req.body;
```

```
      if (!i18n.languages.includes(language)) {
        res.status(400).json({
          status: 400,
          error: "Language not found",
        });
        return;
      }

      if (!i18n.translations[language]) {
        i18n.translations[language] = {};
      }

      i18n.translations[language][key] = value;

      await i18n.markModified("translations");
      await i18n.save();

      res.status(200).json({
        status: 200,
        i18n,
      });
    } else if (action === "deleteKey") {
      const { key, language } = req.body;

      if (!i18n.languages.includes(language)) {
        res.status(400).json({
          status: 400,
          error: "Language not found",
        });
        return;
      }

      if (!i18n.translations[language]) {
        res.status(400).json({
          status: 400,
          error: "Key not found",
        });
        return;
      }

      delete i18n.translations[language][key];

      await i18n.markModified("translations");
      await i18n.save();

      res.status(200).json({
        status: 200,
        i18n,
      });
    }

    return;
  } else {
    res.status(405).json({
      status: 405,
      error: "Method not allowed",
    });
    return;
  }
} catch (e: any) {
```

```
    res.status(500).json({
      status: 500,
      error: e.message,
    });
  }
}
```

## apps/web/src/pages/api/projects/[id]/index.ts

```
/**
 * apps/web/src/pages/api/projects/[id]/index.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2024 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 30.05.2024
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import jwt, { JwtPayload, VerifyErrors } from "jsonwebtoken";
import mongoConnect from "@codeup/util/dist/mongo";
import UserModel from "@codeup/util/dist/models/UserModel";
import ProjectModel from "@codeup/util/dist/models/ProjectModel";
import * as fs from "fs";
import * as path from "path";
import { tmpdir } from "os";
import archiver from "archiver";
import { isLoggedIn } from "../../../../global/server/isLoggedIn";

export default async function (req: NextApiRequest, res: NextApiResponse) {
  try {
    await mongoConnect();
    const { id } = req.query;

    const { auth, user } = await isLoggedIn(req, res);
    if (!auth) return;

    if (!id) {
      res.status(400).json({
        status: 400,
        error: "Invalid Request",
      });
      return;
    }

    const project = await ProjectModel.findById(id);

    if (!project) {
      res.status(404).json({
        status: 404,
        error: "Project not found",
      });
      return;
    }

    if (!project.users.includes(user._id)) {
      res.status(403).json({
        status: 400,
        error: "You are not allowed to view this project",
      });
```

```
      return;
    }

    res.status(200).json({
      status: 200,
      project,
    });
  } catch (e: any) {
    res.status(500).json({
      status: 500,
      error: e.message,
    });
  }
}
```

## apps/web/src/pages/api/projects/[id]/issues/index.ts

```
/**
 * apps/web/src/pages/api/projects/[id]/issues/index.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2024 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 31.05.2024
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/dist/mongo";
import { isLoggedIn } from "../../../../../global/server/isLoggedIn";
import ProjectModel from "@codeup/util/dist/models/ProjectModel";
import ProjectIssueModel from "@codeup/util/dist/models/ProjectIssueModel";

export default async function (req: NextApiRequest, res: NextApiResponse) {
  try {
    await mongoConnect();

    const { auth, user } = await isLoggedIn(req, res);

    const project = await ProjectModel.findById(req.query.id as string);

    if (!project) {
      res.status(404).json({
        status: 404,
        error: "Project not found",
      });
      return;
    }

    if (!project.users.includes(user._id.toString())) {
      res.status(403).json({
        status: 403,
        error: "You are not allowed to view this project",
      });
      return;
    }

    if (req.method === "GET") {
      if (req.query.issueId) {
        const issue = await ProjectIssueModel.findById(
          req.query.issueId as string
```

```
    );

    if (!issue) {
      res.status(404).json({
        status: 404,
        error: "Issue not found",
      });
      return;
    }

    if (issue.project.toString() !== project._id.toString()) {
      res.status(403).json({
        status: 403,
        error: "You are not allowed to view this issue",
      });
      return;
    }

    res.status(200).json({
      status: 200,
      issue,
    });

    return;
  }

  const issues = await ProjectIssueModel.find({
    project: project._id,
  });

  res.status(200).json({
    status: 200,
    issues,
  });
  return;
} else if (req.method === "POST") {
  const { title, description, priority, status } = req.body;

  if (!title || !description || !priority || !status) {
    res.status(400).json({
      status: 400,
      error: "Missing required fields",
    });
    return;
  }

  const newIssue = await ProjectIssueModel.create({
    title,
    description,
    priority,
    status,
    project: project._id,
  });

  await newIssue.save();

  res.status(200).json({
    status: 200,
    issue: newIssue,
  });
```

```
    return;
  } else if (req.method === "PUT") {
    const { title, description, priority, status } = req.body;

    if (!title || !description || !priority || !status) {
      res.status(400).json({
        status: 400,
        error: "Missing required fields",
      });
      return;
    }

    const issue = await ProjectIssueModel.findById(
      req.query.issueId as string
    );

    if (!issue) {
      res.status(404).json({
        status: 404,
        error: "Issue not found",
      });
      return;
    }

    if (issue.project.toString() !== project._id.toString()) {
      res.status(403).json({
        status: 403,
        error: "You are not allowed to edit this issue",
      });
      return;
    }

    issue.title = title;
    issue.description = description;
    issue.priority = priority;
    issue.status = status;

    await issue.save();

    res.status(200).json({
      status: 200,
      issue,
    });
    return;
  } else if (req.method === "DELETE") {
    const issue = await ProjectIssueModel.findById(
      req.query.issueId as string
    );

    if (!issue) {
      res.status(404).json({
        status: 404,
        error: "Issue not found",
      });
      return;
    }

    if (issue.project.toString() !== project._id.toString()) {
      res.status(403).json({
        status: 403,
```

```
          error: "You are not allowed to delete this issue",
        });
        return;
      }

      await issue.remove();

      res.status(200).json({
        status: 200,
      });
      return;
    }

    res.status(405).json({
      status: 405,
      error: "Method not allowed",
    });
  } catch (e: any) {
    res.status(500).json({
      status: 500,
      error: e.message,
    });
  }
}
```

## apps/web/src/pages/api/projects/[id]/static/files.ts

```
/**
 * apps/web/src/pages/api/projects/[id]/static/files.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2024 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 31.05.2024
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/dist/mongo";
import ProjectModel from "@codeup/util/dist/models/ProjectModel";
import { isLoggedIn } from "../../../../../global/server/isLoggedIn";
import { getS3 } from "../../../../../global/getS3";

export default async function (req: NextApiRequest, res: NextApiResponse) {
  try {
    await mongoConnect();
    const { id } = req.query;

    const { auth, user } = await isLoggedIn(req, res);
    if (!auth) return;

    if (!id) {
      res.status(400).json({
        status: 400,
        error: "Invalid Request",
      });
      return;
    }

    const project = await ProjectModel.findById(id);
```

```
    if (!project) {
      res.status(404).json({
        status: 404,
        error: "Project not found",
      });
      return;
    }

    if (!project.users.includes(user._id)) {
      res.status(403).json({
        status: 400,
        error: "You are not allowed to view this project",
      });
      return;
    }

    const client = getS3();

    if (req.method === "GET") {
      const files = client.listObjectsV2(
        "codeup-project-" + project._id,
        "",
        true
      );

      let filesArray = [];

      for await (const file of files) {
        filesArray.push(file);
      }

      res.status(200).json({
        status: 200,
        filesArray,
      });
    } else if (req.method === "DELETE") {
      const { file } = req.body;

      if (!file) {
        res.status(400).json({
          status: 400,
          error: "Invalid Request",
        });
        return;
      }

      await client.removeObjects("codeup-project-" + project._id, [file]);

      res.status(200).json({
        status: 200,
        message: "File deleted",
      });
    }
  } catch (e: any) {
    res.status(500).json({
      status: 500,
      error: e.message,
    });
  }
}
```

## apps/web/src/pages/api/projects/[id]/static/upload.ts

```
/**
 * apps/web/src/pages/api/projects/[id]/static/upload.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2024 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 31.05.2024
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/dist/mongo";
import { isLoggedIn } from "../../../../../global/server/isLoggedIn";
import fs from "fs";
import { randomBytes } from "crypto";
import ProjectModel from "@codeup/util/dist/models/ProjectModel";
import { tmpdir } from "node:os";
import { getS3 } from "../../../../../global/getS3";

export default async function (req: NextApiRequest, res: NextApiResponse) {
  try {
    await mongoConnect();
    const { id } = req.query;

    const { auth, user } = await isLoggedIn(req, res);

    if (!auth) return;

    if (!id) {
      res.status(400).json({
        status: 400,
        error: "Invalid Request",
      });
      return;
    }

    const project = await ProjectModel.findById(id);

    if (!project) {
      res.status(404).json({
        status: 404,
        error: "Project not found",
      });
      return;
    }

    if (!project.users.includes(user._id)) {
      res.status(403).json({
        status: 400,
        error: "You are not allowed to view this project",
      });
      return;
    }

    const { content, name } = req.body;

    const nameX = tmpdir() + "/" + randomBytes(16).toString("hex");
```

```
    fs.writeFileSync(nameX, content, "base64");

    const client = getS3();

    await client.fPutObject(
      `codeup-project-${project._id.toString()}`,
      name,
      nameX
    );

    res.status(200).json({
      status: 200,
    });
  } catch (e: any) {
    res.status(500).json({
      status: 500,
      error: e.message,
    });
  }
}

export const config = {
  api: {
    bodyParser: {
      sizeLimit: "10mb",
    },
  },
};
```

## apps/web/src/pages/api/projects/[id]/uml/index.ts

```
/**
 * apps/web/src/pages/api/projects/[id]/uml/index.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2024 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 08.06.2024
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/dist/mongo";
import { isLoggedIn } from "../../../../../global/server/isLoggedIn";
import ProjectModel from "@codeup/util/dist/models/ProjectModel";
import ProjectUMLModel from "@codeup/util/dist/models/ProjectUMLModel";

export default async function (req: NextApiRequest, res: NextApiResponse) {
  try {
    await mongoConnect();

    const { auth, user } = await isLoggedIn(req, res);

    if (!auth) return;

    const project = await ProjectModel.findById(req.query.id);

    if (!project) {
      res.status(404).json({
        status: 404,
        error: "Project not found",
```

```
  });
  return;
}

if (!project.users.includes(user._id)) {
  res.status(403).json({
    status: 400,
    error: "You are not allowed to view this project",
  });
  return;
}

if (req.method === "GET") {
  if (req.query.umlId) {
    const uml = await ProjectUMLModel.findById(req.query.umlId as string);

    if (uml.project.toString() !== project._id.toString()) {
      res.status(403).json({
        status: 403,
        error: "You are not allowed to view this UML",
      });
      return;
    }

    res.status(200).json({
      status: 200,
      uml,
    });
    return;
  }

  const umls = await ProjectUMLModel.find({ project: project._id });

  res.status(200).json({
    status: 200,
    umls,
  });
  return;
} else if (req.method === "POST") {
  if (req.query.umlId) {
    const uml = await ProjectUMLModel.findById(req.query.umlId as string);

    if (uml.project.toString() !== project._id.toString()) {
      res.status(403).json({
        status: 403,
        error: "You are not allowed to view this UML",
      });
      return;
    }

    const { data } = req.body;

    uml.data = data;

    await uml.save();

    res.status(200).json({
      status: 200,
      uml: uml,
    });
```

```
      return;
    }

    const { name, data, type } = req.body;

    const uml = await ProjectUMLModel.create({
      project: project._id,
      name,
      data,
      type,
    });

    res.status(200).json({
      status: 200,
      uml,
    });
    return;
  } else if (req.method === "DELETE") {
    const uml = await ProjectUMLModel.findById(req.query.umlId as string);

    if (uml.project.toString() !== project._id.toString()) {
      res.status(403).json({
        status: 403,
        error: "You are not allowed to delete this UML",
      });
      return;
    }

    await uml.delete();

    res.status(200).json({
      status: 200,
      uml,
    });
    return;
  }

  res.status(405).json({
    status: 405,
    error: "Method not allowed",
  });
 } catch (e: any) {
  res.status(500).json({
    status: 500,
    error: e.message,
  });
 }
}
```

## apps/web/src/pages/api/projects/[id]/wiki/pages.ts

```
/**
 * apps/web/src/pages/api/projects/[id]/wiki/pages.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2024 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 31.05.2024
 *
 */
```

```
import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/dist/mongo";
import { isLoggedIn } from "../../../../../global/server/isLoggedIn";
import ProjectModel from "@codeup/util/dist/models/ProjectModel";
import ProjectWikiPageModel from "@codeup/util/dist/models/ProjectWikiPageModel";

export default async function (req: NextApiRequest, res: NextApiResponse) {
  try {
    await mongoConnect();

    const { auth, user } = await isLoggedIn(req, res);

    if (!auth) return;

    const project = await ProjectModel.findById(req.query.id as string);

    if (!project) {
      res.status(404).json({
        status: 404,
        error: "Project not found",
      });
      return;
    }

    if (!project.users.includes(user._id.toString())) {
      res.status(403).json({
        status: 403,
        error: "You are not allowed to view this project",
      });
      return;
    }

    if (req.method === "GET") {
      if (req.query.pageId) {
        const page = await ProjectWikiPageModel.findById(
          req.query.pageId as string
        );
        if (!page) {
          res.status(404).json({
            status: 404,
            error: "Page not found",
          });
          return;
        }

        if (page.project.toString() !== project._id.toString()) {
          res.status(403).json({
            status: 403,
            error: "You are not allowed to view this page",
          });
          return;
        }

        res.status(200).json({
          status: 200,
          page,
        });
        return;
      }
```

```
  const pages = await ProjectWikiPageModel.find({ project: project._id });

  res.status(200).json({
    status: 200,
    pages,
  });
} else if (req.method === "POST") {
  const { title, content } = req.body;

  if (!title || !content) {
    res.status(400).json({
      status: 400,
      error: "Invalid Request",
    });
    return;
  }

  if (req.body.page_id) {
    const page = await ProjectWikiPageModel.findById(
      req.body.page_id as string
    );

    if (!page) {
      res.status(404).json({
        status: 404,
        error: "Page not found",
      });
      return;
    }

    if (page.project.toString() !== project._id.toString()) {
      res.status(403).json({
        status: 403,
        error: "You are not allowed to edit this page",
      });
      return;
    }

    page.title = title;
    page.content = content;

    await page.save();

    res.status(200).json({
      status: 200,
      page,
    });
    return;
  }

  const page = await ProjectWikiPageModel.create({
    project: project._id,
    title,
    content,
  });

  res.status(200).json({
    status: 200,
    page,
  });
```

```
    } else if (req.method === "DELETE") {
      const { page_id } = req.body;

      if (!page_id) {
        res.status(400).json({
          status: 400,
          error: "Invalid Request",
        });
        return;
      }

      const page = await ProjectWikiPageModel.findById(page_id);

      if (!page) {
        res.status(404).json({
          status: 404,
          error: "Page not found",
        });
        return;
      }

      if (page.project.toString() !== project._id.toString()) {
        res.status(403).json({
          status: 403,
          error: "You are not allowed to delete this page",
        });
        return;
      }

      await page.remove();

      res.status(200).json({
        status: 200,
      });
    }
  } catch (e: any) {
    res.status(500).json({
      status: 500,
      error: e.message,
    });
  }
}
```

## apps/web/src/pages/api/rce/go.ts

```
/**
 * apps/web/src/pages/api/rce/go.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2024 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 18.05.2024
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import { rce } from "../../../global/server/rce";

export default async function (req: NextApiRequest, res: NextApiResponse) {
  const { code } = req.body;
```

```
  if (!code) {
    res.status(400).json({
      status: 400,
      error: "No code provided",
    });
    return;
  }

  await rce(req, res, {
    timeout: 7.5,
    code,
    tmpfs: {
      "main.go": code,
    },
    cmd: ["go", "run", "/usr/src/app/main.go"],
    env: ["NO_COLOR=true"],
    image: "rce-golang:codeup",
  });
}
```

## apps/web/src/pages/api/rce/java.ts

```
/**
 * apps/web/src/pages/api/rce/java.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2024 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 18.05.2024
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import { rce } from "../../../global/server/rce";

export default async function (req: NextApiRequest, res: NextApiResponse) {
  const { code } = req.body;

  if (!code) {
    res.status(400).json({
      status: 400,
      error: "No code provided",
    });
    return;
  }
  let extractedClassName = code.match(/public class (\w+)/)?.[1];

  const tmpfs: any = {
    "r.sh": `#!/bin/bash\njavac ${extractedClassName}.java && java ${extractedClassName}`,
  };

  tmpfs[`${extractedClassName}.java`] = code;

  await rce(req, res, {
    timeout: 5,
    code,
    entrypoint: ["/bin/sh"],
    cmd: ["/usr/src/app/r.sh"],
    env: [],
    image: "rce-java:codeup",
    tmpfs: tmpfs,
```

```
  });
}
```

## apps/web/src/pages/api/rce/javascript.ts

```
/**
 * apps/web/src/pages/api/rce/javascript.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2024 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 17.05.2024
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import { rce } from "../../../global/server/rce";
export default async function (req: NextApiRequest, res: NextApiResponse) {
  const { code } = req.body;

  if (!code) {
    res.status(400).json({
      status: 400,
      error: "No code provided",
    });
    return;
  }
  await rce(req, res, {
    timeout: 2.5,
    cmd: ["node", "/usr/src/app/index.js"],
    env: ["NO_COLOR=true"],
    image: "rce-node:codeup",
    code,
    tmpfs: {
      "index.js": code,
    },
  });
}
```

## apps/web/src/pages/api/rce/python.ts

```
/**
 * apps/web/src/pages/api/rce/python.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2024 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 18.05.2024
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import { rce } from "../../../global/server/rce";

export default async function (req: NextApiRequest, res: NextApiResponse) {
  const { code } = req.body;

  if (!code) {
    res.status(400).json({
      status: 400,
      error: "No code provided",
    });
```

```
    return;
  }

  await rce(req, res, {
    timeout: 2.5,
    code,
    cmd: ["python", "/usr/src/app/main.py"],
    env: ["NO_COLOR=true"],
    image: "rce-python:codeup",
    tmpfs: {
      "main.py": code,
    },
  });
}
```

## apps/web/src/pages/api/rce/typescript.ts

```
/**
 * apps/web/src/pages/api/rce/typescript.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2024 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 17.05.2024
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import Docker from "dockerode";
import { rce } from "../../../global/server/rce";

export default async function (req: NextApiRequest, res: NextApiResponse) {
  const { code } = req.body;

  if (!code) {
    res.status(400).json({
      status: 400,
      error: "No code provided",
    });
    return;
  }

  await rce(req, res, {
    timeout: 2.5,
    code,
    cmd: ["tsx", "/usr/src/app/index.ts"],
    env: ["NO_COLOR=true"],
    image: "rce-tsx:codeup",
    tmpfs: {
      "index.ts": code,
    },
  });
}
```

## apps/web/src/pages/api/roadmap/create.ts

```
/**
 * apps/web/src/pages/api/roadmap/create.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
```

```
 * Created At: 07.04.2023
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/dist/mongo";
import jwt from "jsonwebtoken";
import RoadmapModel from "@codeup/util/dist/models/RoadmapModel";
import UserModel from "@codeup/util/dist/models/UserModel";
import EditorProjectModel from "@codeup/util/dist/models/EditorProjectModel";
import SocketRegistry from "../../../server/socket/SocketRegistry";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "POST") {
    res.status(405).send({
      error: "Method not allowed",
    });
    return;
  }

  if (!req.headers["x-jwt"]) {
    res.status(401).send({
      error: "Not authenticated",
    });
    return;
  }

  if (!req.body || !req.body.name) {
    res.status(400).send({
      error: "Bad request",
    });
    return;
  }

  const token = req.headers["x-jwt"] as string;

  try {
    await mongoConnect();

    const decoded = jwt.verify(token, process.env.JWT_SECRET as string);

    if (!decoded) {
      res.status(401).send({
        error: "Not authenticated",
      });
      return;
    }

    const user = await UserModel.findById((decoded as any).id);

    if (!user) {
      res.status(401).send({
        error: "Not authenticated",
      });
      return;
    }
```

```
    let allowedToCreate = true;

    if (user.subscription !== "PRO" && user.role !== "admin") {
      const maps = await RoadmapModel.find({
        owner: user._id,
      });
      if (maps.length >= 1) {
        allowedToCreate = false;
      }
      if (process.env.DISABLE_PAYMENTS) {
        allowedToCreate = true;
      }
    }

    if (!allowedToCreate) {
      res.status(403).send({
        error:
          "Du musst CodeUp PRO abonnieren, um mehr als 1 Roadmap zu erstellen.",
      });
      return;
    }

    const roadmap = await RoadmapModel.create({
      owner: user._id,
      name: req.body.name,
    });

    res.status(200).send({
      roadmap: roadmap,
      success: true,
    });
  } catch (e: any) {
    res.status(500).send({
      error: e.message as string,
    });
    return;
  }
}
```

## apps/web/src/pages/api/roadmap/my.ts

```
/**
 * apps/web/src/pages/api/roadmap/my.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 07.04.2023
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/dist/mongo";
import jwt from "jsonwebtoken";
import RoadmapModel from "@codeup/util/dist/models/RoadmapModel";
import UserModel from "@codeup/util/dist/models/UserModel";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
```

```
  if (req.method !== "GET") {
    res.status(405).send({
      error: "Method not allowed",
    });
    return;
  }

  if (!req.headers["x-jwt"]) {
    res.status(401).send({
      error: "Not authenticated",
    });
    return;
  }

  const token = req.headers["x-jwt"] as string;

  try {
    await mongoConnect();

    const decoded = jwt.verify(token, process.env.JWT_SECRET as string);

    if (!decoded) {
      res.status(401).send({
        error: "Not authenticated",
      });
      return;
    }

    const user = await UserModel.findById((decoded as any).id);

    if (!user) {
      res.status(401).send({
        error: "Not authenticated",
      });
      return;
    }

    const roadmaps = await RoadmapModel.find({
      owner: user._id,
    });

    res.status(200).send({
      roadmaps,
      success: true,
    });
  } catch (e: any) {
    res.status(500).send({
      error: e.message as string,
    });
    return;
  }
}
```

## apps/web/src/pages/api/roadmap/[id]/delete.ts

```
/**
 * apps/web/src/pages/api/roadmap/[id]/delete.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
```

```
 * Created At: 07.04.2023
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/dist/mongo";
import jwt from "jsonwebtoken";
import RoadmapModel from "@codeup/util/dist/models/RoadmapModel";
import UserModel from "@codeup/util/dist/models/UserModel";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "DELETE") {
    res.status(405).send({
      error: "Method not allowed",
    });
    return;
  }

  if (!req.headers["x-jwt"]) {
    res.status(401).send({
      error: "Not authenticated",
    });
    return;
  }

  const token = req.headers["x-jwt"] as string;

  try {
    await mongoConnect();

    const decoded = jwt.verify(token, process.env.JWT_SECRET as string);

    if (!decoded) {
      res.status(401).send({
        error: "Not authenticated",
      });
      return;
    }

    const user = await UserModel.findById((decoded as any).id);

    if (!user) {
      res.status(401).send({
        error: "Not authenticated",
      });
      return;
    }

    const roadmap = await RoadmapModel.findById(req.query.id);

    if (!roadmap) {
      res.status(404).send({
        error: "Roadmap not found",
      });
      return;
    }
```

```
    if (
      roadmap.owner.toString() !== user._id.toString() &&
      user.role !== "admin"
    ) {
      res.status(403).send({
        error: "Not allowed",
      });
      return;
    }

    await roadmap.remove();

    res.status(200).send({
      success: true,
    });
  } catch (e: any) {
    res.status(500).send({
      error: e.message as string,
    });
    return;
  }
}
```

## apps/web/src/pages/api/roadmap/[id]/index.ts

```
/**
 * apps/web/src/pages/api/roadmap/[id]/index.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 07.04.2023
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/dist/mongo";
import jwt from "jsonwebtoken";
import RoadmapModel from "@codeup/util/dist/models/RoadmapModel";
import UserModel from "@codeup/util/dist/models/UserModel";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "GET") {
    res.status(405).send({
      error: "Method not allowed",
    });
    return;
  }

  if (!req.headers["x-jwt"]) {
    res.status(401).send({
      error: "Not authenticated",
    });
    return;
  }

  const token = req.headers["x-jwt"] as string;
```

```
  try {
    await mongoConnect();

    const decoded = jwt.verify(token, process.env.JWT_SECRET as string);

    if (!decoded) {
      res.status(401).send({
        error: "Not authenticated",
      });
      return;
    }

    const user = await UserModel.findById((decoded as any).id);

    if (!user) {
      res.status(401).send({
        error: "Not authenticated",
      });
      return;
    }

    const roadmap = await RoadmapModel.findById(req.query.id as string);

    if (!roadmap) {
      res.status(404).send({
        error: "Roadmap not found",
      });
      return;
    }

    if (roadmap.owner !== user._id && user.role !== "admin") {
      res.status(401).send({
        error: "Not authorized",
      });
      return;
    }

    res.status(200).send({
      roadmap,
      success: true,
    });
  } catch (e: any) {
    res.status(500).send({
      error: e.message as string,
    });
    return;
  }
}
```

## apps/web/src/pages/api/roadmap/[id]/update.ts

```
/**
 * apps/web/src/pages/api/roadmap/[id]/update.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 07.04.2023
 *
 */
```

```
import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/dist/mongo";
import jwt from "jsonwebtoken";
import RoadmapModel from "@codeup/util/dist/models/RoadmapModel";
import UserModel from "@codeup/util/dist/models/UserModel";
import ToDoProjectModel from "@codeup/util/dist/models/ToDoProjectModel";
import TodoListModel from "@codeup/util/dist/models/TodoListModel";
import EditorProjectModel from "@codeup/util/dist/models/EditorProjectModel";
import ProjectIdeaModel from "@codeup/util/dist/models/ProjectIdeaModel";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "POST") {
    res.status(405).send({
      error: "Method not allowed",
    });
    return;
  }

  if (!req.headers["x-jwt"]) {
    res.status(401).send({
      error: "Not authenticated",
    });
    return;
  }

  const token = req.headers["x-jwt"] as string;

  if (!req.body || !req.body.update) {
    res.status(400).send({
      error: "Bad request",
    });
    return;
  }

  try {
    await mongoConnect();

    const decoded = jwt.verify(token, process.env.JWT_SECRET as string);

    if (!decoded) {
      res.status(401).send({
        error: "Not authenticated",
      });
      return;
    }

    const user = await UserModel.findById((decoded as any).id);

    if (!user) {
      res.status(401).send({
        error: "Not authenticated",
      });
      return;
    }

    const roadmap = await RoadmapModel.findById(req.query.id);
```

```
if (!roadmap) {
  res.status(404).send({
    error: "Roadmap not found",
  });
  return;
}

if (
  roadmap.owner.toString() !== user._id.toString() &&
  user.role !== "admin"
) {
  res.status(403).send({
    error: "Not allowed",
  });
  return;
}

const update = req.body.update;

if (update.linkedCourses) {
  roadmap.linkedCourses = update.linkedCourses;
}

if (update.linkedFlows) {
  for (const flow of update.linkedFlows) {
    if (!roadmap.linkedFlows.includes(flow)) {
      const flowX = await ToDoProjectModel.findById(flow);
      if (!flowX || flowX.owner.toString() !== user._id.toString()) {
        res.status(403).send({
          error: "Not allowed",
        });
        return;
      }

      roadmap.linkedFlows.push(flow);
    }
  }
}

if (update.linkedTodos) {
  for (const todo of update.linkedTodos) {
    const todoX = await TodoListModel.findById(todo);
    if (!todoX || todoX.owner.toString() !== user._id.toString()) {
      res.status(403).send({
        error: "Not allowed",
      });
      return;
    }
  }

  roadmap.linkedTodos = update.linkedTodos;
}

if (update.linkedProject) {
  const project = await EditorProjectModel.findById(update.linkedProject);
  if (!project || project.owner.toString() !== user._id.toString()) {
    res.status(403).send({
      error: "Not allowed",
    });
    return;
```

```
      }

      roadmap.linkedProject = update.linkedProject;
    }

    if (update.name) {
      roadmap.name = update.name;
    }

    if (update.projectIdea) {
      roadmap.projectIdea = update.projectIdea;
    }

    await roadmap.save();

    res.status(200).send({
      success: true,
      roadmap,
    });
  } catch (e: any) {
    res.status(500).send({
      error: e.message as string,
    });
    return;
  }
}
```

## apps/web/src/pages/api/s3/[project_id]/[name].ts

```
/**
 * apps/web/src/pages/api/s3/[project_id]/[name].ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2024 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 31.05.2024
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/dist/mongo";
import { getS3 } from "../../../../global/getS3";

export default async function (req: NextApiRequest, res: NextApiResponse) {
  try {
    await mongoConnect();

    const { project_id, name } = req.query;

    if (!project_id || !name) {
      res.status(400).json({
        status: 400,
        error: "Invalid Request",
      });
      return;
    }

    const client = getS3();

    const url = await client.presignedGetObject(
      "codeup-project-" + project_id,
```

```
      name as string,
      60 * 60 * 24
    );

    res.redirect(url);
  } catch (e: any) {
    res.status(500).json({
      status: 500,
      error: e.message,
    });
  }
}
```

## apps/web/src/pages/api/snippets/index.ts

```
/**
 * apps/web/src/pages/api/snippets/index.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 30.12.2022
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/src/mongo";
import jwt, { JwtPayload, VerifyErrors } from "jsonwebtoken";
import UserModel from "@codeup/util/src/models/UserModel";
import CodeSnippet from "@codeup/util/src/models/CodeSnippet";
import CodeUpKidsProjectModel from "@codeup/util/dist/models/CodeUpKidsProjectModel";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (!req.headers["x-jwt"]) {
    res.status(400).json({
      error: "Missing Auth-Header (x-jwt)",
    });
    return;
  }

  jwt.verify(
    req.headers["x-jwt"] as string,
    process.env.JWT_SECRET as string,
    async (err: VerifyErrors | null, decoded: JwtPayload | string | any) => {
      if (err) {
        res.status(400).json({
          error: "Invalid token",
        });
        return;
      }

      await mongoConnect();

      const user = await UserModel.findById((decoded as any).id);

      if (!user) {
        res.status(400).json({
          error: "Invalid token",
```

```
  });
  return;
}

if (req.method === "GET") {
  if (req.query.id) {
    const snippet = await CodeSnippet.findById(req.query.id);

    if (!snippet) {
      res.status(404).json({
        error: "Snippet not found",
      });
      return;
    }

    if (snippet.username !== user.username) {
      res.status(403).json({
        error: "You are not allowed to access this snippet",
      });
      return;
    }

    res.status(200).json({ snippet });
  } else {
    const snippets = await CodeSnippet.find({
      username: user.username,
    });

    res.status(200).json({ snippets });
  }
} else if (req.method === "POST") {
  const code = req.body.code;
  const language = req.body.language;
  const title = req.body.title;

  if (!code || !language || !title) {
    res.status(400).json({
      error: "Missing parameters",
    });
    return;
  }

  if (user.subscription !== "PRO" && user.role !== "admin") {
    let allowed = true;

    const snippets = await CodeSnippet.find({
      username: user.username,
    });

    if (snippets.length >= 3) {
      allowed = false;
    }

    if (process.env.DISABLE_PAYMENTS) {
      allowed = true;
    }

    if (!allowed) {
      res.status(403).json({
        error:
```

```
          "Du musst CodeUp PRO abonnieren, um mehr als 3 Code-Snippets zu erstellen.",
        });
        return;
      }
    }

    const snippet = await CodeSnippet.create({
      username: user.username,
      code,
      title,
      language,
      createdAt: new Date(),
    });

    res.status(200).json({ snippet });
  } else if (req.method === "DELETE") {
    if (req.query.id) {
      const snippet = await CodeSnippet.findById(req.query.id);

      if (!snippet) {
        res.status(404).json({
          error: "Snippet not found",
        });
        return;
      }

      if (snippet.username !== user.username && user.role !== "admin") {
        res.status(403).json({
          error: "You are not allowed to access this snippet",
        });
        return;
      }

      await snippet.remove();

      res.status(200).json({ success: true });
    } else {
      res.status(400).json({
        error: "Missing parameters",
      });
      return;
    }
  } else if (req.method === "PUT") {
    if (req.query.id) {
      const snippet = await CodeSnippet.findById(req.query.id);

      if (!snippet) {
        res.status(404).json({
          error: "Snippet not found",
        });
        return;
      }

      if (snippet.username !== user.username && user.role !== "admin") {
        res.status(403).json({
          error: "You are not allowed to access this snippet",
        });
        return;
      }
```

```
        const code = req.body.code;
        const language = req.body.language;
        const title = req.body.title;

        if (!code || !language || !title) {
          res.status(400).json({
            error: "Missing parameters",
          });
          return;
        }

        snippet.code = code;
        snippet.language = language;
        snippet.title = title;

        await snippet.save();

        res.status(200).json({ snippet });
      } else {
        res.status(400).json({
          error: "Missing parameters",
        });
        return;
      }
    }
  }
  );
}
```

## apps/web/src/pages/api/sso/discord.ts

```
/**
 * apps/web/src/pages/api/sso/discord.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 02.10.2022
 *
 */

import generateJWT from '@codeup/util/src/generateJWT';
import UserModel from '@codeup/util/src/models/UserModel';
import { NextApiRequest, NextApiResponse } from 'next';
import { createAvatar } from '@dicebear/avatars';
import * as style from '@dicebear/avatars-bottts-sprites';
import { createHash } from 'crypto';
import { default as mongoConnect } from '@codeup/util/src/mongo';

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== 'GET') {
    res.status(405).json({
      error: 'Method not allowed',
    });
    return;
  }

  if (!req.query.code) {
```

```
  res.status(400).json({
    error: 'Missing required fields',
  });
  return;
}

const code = req.query.code;
var details: { [key: string]: any } = {
  client_id: process.env.DISCORD_CLIENT_ID,
  client_secret: process.env.DISCORD_CLIENT_SECRET,
  code: code,
  grant_type: 'authorization_code',
  redirect_uri: process.env.DISCORD_REDIRECT_URI,
  scope: 'identify',
};

var formBody: any = [];
for (var property in details) {
  var encodedKey = encodeURIComponent(property);
  var encodedValue = encodeURIComponent(details[property]);
  formBody.push(encodedKey + '=' + encodedValue);
}
formBody = formBody.join('&');

const tokenRequest = await fetch('https://discord.com/api/v10/oauth2/token', {
  headers: {
    'Content-Type': 'application/x-www-form-urlencoded',
  },
  method: 'POST',
  body: formBody,
});

const token = await tokenRequest.json();

if (token.error) {
  res.status(400).json({
    error: token.error,
  });
  return;
}

const accessToken = token.access_token;

const userRequest = await fetch('https://discord.com/api/v10/users/@me', {
  headers: {
    Authorization: `Bearer ${accessToken}`,
  },
});

const user = await userRequest.json();

let username = user.username;
const userId = user.id;
const primaryEmail = user.email;
const name = "CodeUp Benutzer";

await mongoConnect();


const existingUser = await UserModel.findOne({
```

```
    sso: "discord:" + userId,
  });

  if (existingUser) {
    res.redirect("/sso/callback?token=" + generateJWT(existingUser));
  } else {
    const existingUsername = await UserModel.findOne({
      username: username,
    });

    if (existingUsername) {
      username = "discord:" + userId;
    }

    const existingEmail = await UserModel.findOne({
      email: primaryEmail,
    });

    if (existingEmail) {
      res.redirect("/sso/error?error=E-Mail wird bereits verwendet.");
      return;
    }

    const svg = createAvatar(style, {
      seed: userId,
      colorful: true,
    });

    const newUser = await UserModel.create({
      username,
      email: primaryEmail,
      firstName: name.split(" ")[0],
      lastName: name.split(" ")[name.split(" ").length - 1],
      password: createHash("sha256").update(randomString0(128)).digest("hex"),
      twoFactorEnabled: false,
      active: true,
      profilePicture: svg,
      sso: "discord:" + userId,
    });

    await newUser.save();

    const token0 = generateJWT(newUser);

    res.redirect("/sso/callback?token=" + token0);


  }


}

function randomString0(size: number) {
  const chars =
    '0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ!$%&{([)]}?+*~#-
_.:,;<|>/';
  let result = '';
  for (let i = size; i > 0; --i)
    result += chars[Math.floor(Math.random() * chars.length)];
  return result;
```

```
}
```

## apps/web/src/pages/api/sso/github.ts

```
/**
 * apps/web/src/pages/api/sso/github.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 22.09.2022
 *
 */

import generateJWT from "@codeup/util/src/generateJWT";
import UserModel from "@codeup/util/src/models/UserModel";
import { NextApiRequest, NextApiResponse } from "next";
import { createAvatar } from "@dicebear/avatars";
import * as style from "@dicebear/avatars-bottts-sprites";
import { createHash } from "crypto";
import { default as mongoConnect } from "@codeup/util/src/mongo";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "GET") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  if (!req.query.code) {
    res.status(400).json({
      error: "Missing required fields",
    });
    return;
  }

  const code = req.query.code;

  const response = await fetch("https://github.com/login/oauth/access_token", {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
      Accept: "application/json",
    },
    body: JSON.stringify({
      client_id: process.env.GITHUB_CLIENT_ID,
      client_secret: process.env.GITHUB_CLIENT_SECRET,
      code: code,
      redirection_uri: process.env.GITHUB_REDIRECT_URI,
    }),
  });

  if (response.status !== 200) {
    res.redirect("/sso/error?error=Zugriff verweigert.");
    return;
  }
```

```
const j = await response.json();

const accessToken = j.access_token;

const userResponse = await fetch("https://api.github.com/user", {
  headers: {
    Authorization: `Bearer ${accessToken}`,
  },
});

if (userResponse.status !== 200) {
  res.redirect("/sso/error?error=Benutzer konnte nicht abgerufen werden.");
  return;
}

const user = await userResponse.json();

const userId = user.id;

await mongoConnect();

const existingUser = await UserModel.findOne({
  sso: "github:" + userId,
});

if (existingUser) {
  res.redirect("/sso/callback?token=" + generateJWT(existingUser));
} else {
  const emailResponse = await fetch("https://api.github.com/user/emails", {
    headers: {
      Authorization: `Bearer ${accessToken}`,
    },
  });

  if (emailResponse.status !== 200) {
    res.redirect("/sso/error?error=E-Mails konnten nicht abgerufen werden.");
    return;
  }

  const emails = await emailResponse.json();

  let username = user.login;
  const name = user.name;
  const primaryEmail = emails.find((e: any) => e.primary).email;

  const existingUsername = await UserModel.findOne({
    username: username,
  });

  if (existingUsername) {
    username = "github:" + userId;
  }

  const existingEmail = await UserModel.findOne({
    email: primaryEmail,
  });

  if (existingEmail) {
    res.redirect("/sso/error?error=E-Mail wird bereits verwendet.");
    return;
```

```
    }

    const svg = createAvatar(style, {
      seed: userId,
      colorful: true,
    });

    const newUser = await UserModel.create({
      username,
      email: primaryEmail,
      firstName: name.split(" ")[0],
      lastName: name.split(" ")[name.split(" ").length - 1],
      password: createHash("sha256").update(randomString0(128)).digest("hex"),
      twoFactorEnabled: false,
      active: true,
      profilePicture: svg,
      sso: "github:" + userId,
    });

    await newUser.save();

    const token = generateJWT(newUser);

    res.redirect("/sso/callback?token=" + token);
  }
}

function randomString0(size: number) {
  const chars =
    "0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ!$%&{([)]}?+*~#-
_.:,;<|>/";
  let result = "";
  for (let i = size; i > 0; --i)
    result += chars[Math.floor(Math.random() * chars.length)];
  return result;
}
```

## apps/web/src/pages/api/sso/gitlab.ts

```
/**
 * apps/web/src/pages/api/sso/gitlab.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 22.09.2022
 *
 */

import generateJWT from "@codeup/util/src/generateJWT";
import UserModel from "@codeup/util/src/models/UserModel";
import { NextApiRequest, NextApiResponse } from "next";
import { createAvatar } from "@dicebear/avatars";
import * as style from "@dicebear/avatars-bottts-sprites";
import { createHash } from "crypto";
import mongoConnect from "@codeup/util/src/mongo";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
```

```
if (req.method !== "GET") {
  res.status(405).json({
    error: "Method not allowed",
  });
  return;
}

if (!req.query.code) {
  res.status(400).json({
    error: "Missing required fields",
  });
  return;
}

const res0 = await fetch(`https://gitlab.com/oauth/token`, {
  method: "POST",
  headers: {
    "Content-Type": "application/json",
    Accept: "application/json",
  },
  body: JSON.stringify({
    client_id: process.env.GITLAB_CLIENT_ID,
    code: req.query.code,
    grant_type: "authorization_code",
    redirect_uri: process.env.GITLAB_REDIRECT_URI,
    code_verifier: req.cookies["code_verifier"],
  }),
});

if (res0.status !== 200) {
  res.redirect("/sso/error?error=Zugriff verweigert.");
  return;
}

const j0 = await res0.json();

const accessToken = j0.access_token;

const res1 = await fetch(`https://gitlab.com/api/v4/user`, {
  headers: {
    Authorization: `Bearer ${accessToken}`,
  },
});

if (res1.status !== 200) {
  res.redirect("/sso/error?error=Zugriff verweigert.");
  return;
}

const user = await res1.json();

const userId = user.id;

await mongoConnect();

const existingUser = await UserModel.findOne({
  sso: "gitlab:" + userId,
});

if (existingUser) {
```

```
    res.redirect("/sso/login?token=" + generateJWT(existingUser));
    return;
  } else {
    let username = user.username;
    const name = user.name;
    const primaryEmail = user.email;

    const existingUsername = await UserModel.findOne({
      username: username,
    });

    if (existingUsername) {
      username = "gitlab:" + userId;
    }

    const existingEmail = await UserModel.findOne({
      email: primaryEmail,
    });

    if (existingEmail) {
      res.redirect("/sso/error?error=E-Mail wird bereits verwendet.");
      return;
    }

    const svg = createAvatar(style, {
      seed: userId,
      colorful: true,
    });

    const newUser = await UserModel.create({
      username,
      email: primaryEmail,
      firstName: name.split(" ")[0],
      lastName: name.split(" ")[name.split(" ").length - 1],
      password: createHash("sha256").update(randomString0(128)).digest("hex"),
      twoFactorEnabled: false,
      active: true,
      profilePicture: svg,
      sso: "gitlab:" + userId,
    });

    await newUser.save();

    const token = generateJWT(newUser);

    res.redirect("/sso/callback?token=" + token);
  }
}

function randomString0(size: number) {
  const chars =
    "0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ!$%&{([)]}?+*~#-
_.:,;<|>/";
  let result = "";
  for (let i = size; i > 0; --i)
    result += chars[Math.floor(Math.random() * chars.length)];
  return result;
}
```

## apps/web/src/pages/api/sso/request.ts

```
/**
 * apps/web/src/pages/api/sso/request.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 22.09.2022
 *
 */

import { NextApiRequest, NextApiResponse } from "next";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "GET") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  if (!req.query.type) {
    res.status(400).json({
      error: "Missing required fields",
    });
    return;
  }

  switch (req.query.type) {
    case "github":
      res.redirect(

`https://github.com/login/oauth/authorize?client_id=${process.env.GITHUB_CLIENT_ID}&redirect_uri
=${process.env.GITHUB_REDIRECT_URI}&scope=user&allow_signup=false`
      );
      break;
    case "gitlab":
      res.redirect(

`https://gitlab.com/oauth/authorize?client_id=${process.env.GITLAB_CLIENT_ID}&redirect_uri=${proc
ess.env.GITLAB_REDIRECT_URI}&response_type=code`
      );
      break;
    case "discord":
      res.redirect(

`https://discord.com/api/oauth2/authorize?client_id=${process.env.DISCORD_CLIENT_ID}&redirect_
uri=${process.env.DISCORD_REDIRECT_URI}&response_type=code&scope=email%20identify`
      );
      break;
    default:
      res.redirect("/sso/error?error=Unbekannter Anbieter");
  }
}
```

## apps/web/src/pages/api/stats.ts

```
/**
 * apps/web/src/pages/api/stats.ts
```

```
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 12.03.2023
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/dist/mongo";
import UserModel from "@codeup/util/dist/models/UserModel";
import EditorProjectModel from "@codeup/util/dist/models/EditorProjectModel";
import ContentModel from "@codeup/util/dist/models/ContentModel";
import OrgModel from "@codeup/util/dist/models/OrgModel";
import CodeUpKidsProjectModel from "@codeup/util/dist/models/CodeUpKidsProjectModel";
import IDEProjectModel from "@codeup/util/dist/models/IDEProjectModel";

export default async (req: NextApiRequest, res: NextApiResponse) => {
  try {
    await mongoConnect();

    const users = await UserModel.countDocuments();
    const projects =
      (await EditorProjectModel.countDocuments()) +
      (await CodeUpKidsProjectModel.countDocuments()) +
      (await IDEProjectModel.countDocuments());
    const lessons = await ContentModel.countDocuments();
    const orgs = await OrgModel.countDocuments();

    res.status(200).json({
      users,
      projects,
      lessons,
      orgs,
    });
  } catch (e) {
    res.status(500).json({
      users: 0,
      projects: 0,
      lessons: 0,
      orgs: 0,
    });
  }
};
```

## apps/web/src/pages/api/testimonials/index.ts

```
/**
 * apps/web/src/pages/api/testimonials/index.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 12.04.2023
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/dist/mongo";
import CustomerChoiceModel from "@codeup/util/dist/models/CustomerChoiceModel";
```

```
export default async (req: NextApiRequest, res: NextApiResponse) => {
  try {
    await mongoConnect();

    const testimonials = await CustomerChoiceModel.find();

    res.status(200).json(testimonials);
  } catch (e: any) {
    res.status(500).json({ error: e.message });
  }
};
```

## apps/web/src/pages/api/todo/lists/create.ts

```
/**
 * apps/web/src/pages/api/todo/lists/create.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 16.07.2022
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/src/mongo";
import jwt, { JwtPayload, VerifyErrors } from "jsonwebtoken";
import UserModel from "@codeup/util/src/models/UserModel";
import IDEProjectModel from "@codeup/util/src/models/IDEProjectModel";
import TodoListModel from "@codeup/util/src/models/TodoListModel";
import ToDoProjectModel from "@codeup/util/dist/models/ToDoProjectModel";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "POST") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  if (!req.body || !req.headers["x-jwt"] || !req.body.name) {
    res.status(400).json({
      error: "Missing required fields",
    });
    return;
  }

  jwt.verify(
    req.headers["x-jwt"] as string,
    process.env.JWT_SECRET as string,
    async (err: VerifyErrors | null, decoded: JwtPayload | string | any) => {
      if (err) {
        res.status(400).json({
          error: "Invalid token",
        });
        return;
      }
```

```
      await mongoConnect();

      const user = await UserModel.findById((decoded as any).id);

      if (!user) {
        res.status(400).json({
          error: "Invalid token",
        });
        return;
      }

      if (user.subscription !== "PRO" && user.role !== "admin") {
        let allowed = true;

        const lists = await TodoListModel.find({
          owner: user._id,
        });

        if (lists.length >= 3) {
          allowed = false;
        }

        if (process.env.DISABLE_PAYMENTS) {
          allowed = true;
        }

        if (!allowed) {
          res.status(403).json({
            error:
              "Du musst CodeUp PRO abonnieren, um mehr als 3 ToDo-Listen zu erstellen.",
          });
          return;
        }
      }

      const list = await TodoListModel.create({
        owner: user._id,
        name: req.body.name,
      });

      await list.save();

      res.status(200).json({
        list: list._id,
        success: true,
      });
    }
  );
}
```

## apps/web/src/pages/api/todo/lists/delete.ts

```
/**
 * apps/web/src/pages/api/todo/lists/delete.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 16.07.2022
 *
 */
```

```
import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/src/mongo";
import jwt, { JwtPayload, VerifyErrors } from "jsonwebtoken";
import UserModel from "@codeup/util/src/models/UserModel";
import IDEProjectModel from "@codeup/util/src/models/IDEProjectModel";
import TodoListModel from "@codeup/util/src/models/TodoListModel";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "POST") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  if (!req.body || !req.headers["x-jwt"] || !req.body.list) {
    res.status(400).json({
      error: "Missing required fields",
    });
    return;
  }

  jwt.verify(
    req.headers["x-jwt"] as string,
    process.env.JWT_SECRET as string,
    async (err: VerifyErrors | null, decoded: JwtPayload | string | any) => {
      if (err) {
        res.status(400).json({
          error: "Invalid token",
        });
        return;
      }

      await mongoConnect();

      const user = await UserModel.findById((decoded as any).id);

      if (!user) {
        res.status(400).json({
          error: "Invalid token",
        });
        return;
      }

      const list = await TodoListModel.findById(req.body.list);

      if (!list) {
        res.status(400).json({
          error: "Invalid list",
        });
        return;
      }

      if (!list.owner.equals(user._id) && user.role !== "admin") {
        res.status(400).json({
          error: "Invalid list",
```

```
        });
        return;
      }

      await list.remove();

      res.status(200).json({
        success: true,
      });
    }
  );
}
```

## apps/web/src/pages/api/todo/lists/[id]/tasks/create.ts

```
/**
 * apps/web/src/pages/api/todo/lists/[id]/tasks/create.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 16.07.2022
 *
 */
import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/src/mongo";
import jwt, { JwtPayload, VerifyErrors } from "jsonwebtoken";
import UserModel from "@codeup/util/src/models/UserModel";
import IDEProjectModel from "@codeup/util/src/models/IDEProjectModel";
import TodoListModel from "@codeup/util/src/models/TodoListModel";
import { uuid } from "uuidv4";
import ToDoProjectModel from "@codeup/util/dist/models/ToDoProjectModel";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "POST") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  if (!req.body || !req.headers["x-jwt"] || !req.query.id || !req.body.task) {
    res.status(400).json({
      error: "Missing required fields",
    });
    return;
  }

  jwt.verify(
    req.headers["x-jwt"] as string,
    process.env.JWT_SECRET as string,
    async (err: VerifyErrors | null, decoded: JwtPayload | string | any) => {
      if (err) {
        res.status(400).json({
          error: "Invalid token",
        });
        return;
      }
```

```
await mongoConnect();

const user = await UserModel.findById((decoded as any).id);

if (!user) {
  res.status(400).json({
    error: "Invalid token",
  });
  return;
}

const list = await TodoListModel.findById(req.query.id);

if (!list) {
  res.status(400).json({
    error: "Invalid list",
  });
  return;
}

if (!list.owner.equals(user._id) && user.role !== "admin") {
  res.status(400).json({
    error: "Invalid list",
  });
  return;
}

if (user.subscription !== "PRO" && user.role !== "admin") {
  let allowed = true;

  if (list.tasks.length >= 10) {
    allowed = false;
  }

  if (process.env.DISABLE_PAYMENTS) {
    allowed = true;
  }

  if (!allowed) {
    res.status(403).json({
      error:
        "Du musst CodeUp PRO abonnieren, um mehr als 10 Einträge in einer Liste zu erstellen.",
    });
    return;
  }
}

let tId = uuid();

list.tasks.push({
  finished: false,
  name: req.body.task,
  description: "",
  uuid: tId,
});

await list.save();

res.status(200).json({
```

```
        success: true,
        task: tId,
      });
    }
  );
}
```

## apps/web/src/pages/api/todo/lists/[id]/tasks/delete.ts

```
/**
 * apps/web/src/pages/api/todo/lists/[id]/tasks/delete.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 16.07.2022
 *
 */
import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/src/mongo";
import jwt, { JwtPayload, VerifyErrors } from "jsonwebtoken";
import UserModel from "@codeup/util/src/models/UserModel";
import IDEProjectModel from "@codeup/util/src/models/IDEProjectModel";
import TodoListModel from "@codeup/util/src/models/TodoListModel";
import { uuid } from "uuidv4";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "POST") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  if (!req.body || !req.headers["x-jwt"] || !req.query.id || !req.body.task) {
    res.status(400).json({
      error: "Missing required fields",
    });
    return;
  }

  jwt.verify(
    req.headers["x-jwt"] as string,
    process.env.JWT_SECRET as string,
    async (err: VerifyErrors | null, decoded: JwtPayload | string | any) => {
      if (err) {
        res.status(400).json({
          error: "Invalid token",
        });
        return;
      }

      await mongoConnect();

      const user = await UserModel.findById((decoded as any).id);

      if (!user) {
        res.status(400).json({
```

```
        error: "Invalid token",
      });
      return;
    }

    const list = await TodoListModel.findById(req.query.id);

    if (!list) {
      res.status(400).json({
        error: "Invalid list",
      });
      return;
    }

    if (!list.owner.equals(user._id) && user.role !== "admin") {
      res.status(400).json({
        error: "Invalid list",
      });
      return;
    }

    const task = list.tasks.find((t: any) => t.uuid === req.body.task);

    if (!task) {
      res.status(400).json({
        error: "Invalid Task",
      });
      return;
    }

    list.tasks = list.tasks.filter((t: any) => t.uuid !== req.body.task);

    await list.save();

    res.status(200).json({
      success: true,
    });
  }
 );
}
```

## apps/web/src/pages/api/todo/lists/[id]/tasks/toggle-finish.ts

```
/**
 * apps/web/src/pages/api/todo/lists/[id]/tasks/toggle-finish.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 05.08.2022
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import { JwtPayload, verify, VerifyErrors } from "jsonwebtoken";
import mongoConnect from "@codeup/util/src/mongo";
import UserModel from "@codeup/util/src/models/UserModel";
import TodoListModel from "@codeup/util/src/models/TodoListModel";

export default async function handleRequest(
  req: NextApiRequest,
```

```
  res: NextApiResponse
) {
  if (req.method !== "POST") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  if (!req.body || !req.headers["x-jwt"] || !req.query.id || !req.body.task) {
    res.status(400).json({
      error: "Missing required fields",
    });
    return;
  }

  verify(
    req.headers["x-jwt"] as string,
    process.env.JWT_SECRET as string,
    async (err: VerifyErrors | null, decoded: JwtPayload | string | any) => {
      if (err) {
        res.status(400).json({
          error: "Invalid token",
        });
        return;
      }

      await mongoConnect();

      const user = await UserModel.findById((decoded as any).id);

      if (!user) {
        res.status(400).json({
          error: "Invalid token",
        });
        return;
      }

      const list = await TodoListModel.findById(req.query.id);

      if (!list) {
        res.status(400).json({
          error: "Invalid list",
        });
        return;
      }

      if (!list.owner.equals(user._id) && user.role !== "admin") {
        res.status(400).json({
          error: "Invalid list",
        });
        return;
      }

      if (
        ![null, undefined].includes(
          list.tasks.find((t: any) => t.uuid === req.body.task)
        )
      ) {
        list.tasks.find((t: any) => t.uuid === req.body.task).finished =
```

```
        !list.tasks.find((t: any) => t.uuid === req.body.task).finished;
      await list.save();
    } else {
      res.status(400).json({
        error: "Invalid Item",
      });
      return;
    }

    await list.save();

    res.status(200).json({
      success: true,
      finished: list.tasks.find((t: any) => t.uuid === req.body.task)
        .finished,
    });
  }
 );
}
```

## apps/web/src/pages/api/todo/lists/[id]/tasks/update.ts

```
/**
 * apps/web/src/pages/api/todo/lists/[id]/tasks/update.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 26.03.2023
 *
 */
import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/src/mongo";
import jwt, { JwtPayload, VerifyErrors } from "jsonwebtoken";
import UserModel from "@codeup/util/src/models/UserModel";
import IDEProjectModel from "@codeup/util/src/models/IDEProjectModel";
import TodoListModel from "@codeup/util/src/models/TodoListModel";
import { uuid } from "uuidv4";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "POST") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  if (
    !req.body ||
    !req.headers["x-jwt"] ||
    !req.query.id ||
    !req.body.task ||
    !req.body.update
  ) {
    res.status(400).json({
      error: "Missing required fields",
    });
    return;
```

```
}

jwt.verify(
  req.headers["x-jwt"] as string,
  process.env.JWT_SECRET as string,
  async (err: VerifyErrors | null, decoded: JwtPayload | string | any) => {
    if (err) {
      res.status(400).json({
        error: "Invalid token",
      });
      return;
    }

    await mongoConnect();

    const user = await UserModel.findById((decoded as any).id);

    if (!user) {
      res.status(400).json({
        error: "Invalid token",
      });
      return;
    }

    const list = await TodoListModel.findById(req.query.id);

    if (!list) {
      res.status(400).json({
        error: "Invalid list",
      });
      return;
    }

    if (!list.owner.equals(user._id) && user.role !== "admin") {
      res.status(400).json({
        error: "Invalid list",
      });
      return;
    }

    const task = list.tasks.find((t: any) => t.uuid === req.body.task);

    if (!task) {
      res.status(400).json({
        error: "Invalid Task",
      });
      return;
    }

    const updatable = ["name", "description", "finished"];

    for (const key in req.body.update) {
      if (updatable.includes(key)) {
        task[key] = req.body.update[key];
      }
    }

    await list.save();

    res.status(200).json({
```

```
        success: true,
      });
    }
  );
}
```

## apps/web/src/pages/api/todo/lists/[id].ts

```
/**
 * apps/web/src/pages/api/todo/lists/[id].ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 16.07.2022
 *
 */
import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/src/mongo";
import jwt, { JwtPayload, VerifyErrors } from "jsonwebtoken";
import UserModel from "@codeup/util/src/models/UserModel";
import IDEProjectModel from "@codeup/util/src/models/IDEProjectModel";
import TodoListModel from "@codeup/util/src/models/TodoListModel";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "GET") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  if (!req.headers["x-jwt"] || !req.query.id) {
    res.status(400).json({
      error: "Missing required fields",
    });
    return;
  }

  jwt.verify(
    req.headers["x-jwt"] as string,
    process.env.JWT_SECRET as string,
    async (err: VerifyErrors | null, decoded: JwtPayload | string | any) => {
      if (err) {
        res.status(400).json({
          error: "Invalid token",
        });
        return;
      }

      await mongoConnect();

      const user = await UserModel.findById((decoded as any).id);

      if (!user) {
        res.status(400).json({
          error: "Invalid token",
        });
```

```
      return;
    }

    const list = await TodoListModel.findById(req.query.id);

    if (!list) {
      res.status(400).json({
        error: "Invalid list",
      });
      return;
    }

    if (!list.owner.equals(user._id) && user.role !== "admin") {
      res.status(400).json({
        error: "Invalid list",
      });
      return;
    }

    res.status(200).json({
      success: true,
      list: list,
    });
  }
 );
}
```

## apps/web/src/pages/api/todo/my-lists.ts

```
/**
 * apps/web/src/pages/api/todo/my-lists.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 16.07.2022
 *
 */
import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/src/mongo";
import jwt, { JwtPayload, VerifyErrors } from "jsonwebtoken";
import UserModel from "@codeup/util/src/models/UserModel";
import IDEProjectModel from "@codeup/util/src/models/IDEProjectModel";
import TodoListModel from "@codeup/util/src/models/TodoListModel";
import { uuid } from "uuidv4";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "GET") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  if (!req.headers["x-jwt"]) {
    res.status(400).json({
      error: "Missing Auth-Header (x-jwt)",
    });
```

```
    return;
  }

  jwt.verify(
    req.headers["x-jwt"] as string,
    process.env.JWT_SECRET as string,
    async (err: VerifyErrors | null, decoded: JwtPayload | string | any) => {
      if (err) {
        res.status(400).json({
          error: "Invalid token",
        });
        return;
      }

      await mongoConnect();

      const user = await UserModel.findById((decoded as any).id);

      if (!user) {
        res.status(400).json({
          error: "Invalid token",
        });
        return;
      }

      const lists = await TodoListModel.find({
        owner: user._id,
      });

      res.status(200).json({
        success: true,
        lists: lists,
      });
    }
  );
}
```

## apps/web/src/pages/api/todo/projects/index.ts

```
/**
 * apps/web/src/pages/api/todo/projects/index.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 04.12.2022
 *
 */
import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/src/mongo";
import jwt, { JwtPayload, VerifyErrors } from "jsonwebtoken";
import UserModel from "@codeup/util/src/models/UserModel";
import ToDoProjectModel from "@codeup/util/src/models/ToDoProjectModel";
import CodeSnippet from "@codeup/util/dist/models/CodeSnippet";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (!["GET", "POST", "DELETE"].includes(req.method as string)) {
    res.status(405).json({
```

```
    error: "Method not allowed",
  });
  return;
}

if (!req.headers["x-jwt"]) {
  res.status(400).json({
    error: "Missing Auth-Header (x-jwt)",
  });
  return;
}
try {
  jwt.verify(
    req.headers["x-jwt"] as string,
    process.env.JWT_SECRET as string,
    async (err: VerifyErrors | null, decoded: JwtPayload | string | any) => {
      if (err) {
        res.status(400).json({
          error: "Invalid token",
        });
        return;
      }

      await mongoConnect();

      const user = await UserModel.findById((decoded as any).id);

      if (!user) {
        res.status(400).json({
          error: "Invalid token",
        });
        return;
      }

      if (req.method === "GET") {
        const ret = [];

        if (req.query.id) {
          const project = await ToDoProjectModel.findById(req.query.id);

          if (
            project.owner.toString() !== user._id.toString() &&
            user.role !== "admin"
          ) {
            res.status(400).json({
              error: "Invalid token",
            });
            return;
          }

          ret.push(project);
        } else {
          const projects = await ToDoProjectModel.find({
            owner: user._id,
          });

          ret.push(...projects);
        }

        res.status(200).json({
```

```
    projects: ret,
  });
} else if (req.method === "POST") {
  if (!req.query.id) {
    if (!req.query.name) {
      res.status(400).json({
        error: "Missing id",
      });

      return;
    }

    if (user.subscription !== "PRO" && user.role !== "admin") {
      let allowed = true;

      const projects = await ToDoProjectModel.find({
        owner: user._id,
      });

      if (projects.length >= 2) {
        allowed = false;
      }
      if (process.env.DISABLE_PAYMENTS) {
        allowed = true;
      }

      if (!allowed) {
        res.status(403).json({
          error:
            "Du musst CodeUp PRO abonnieren, um mehr als 2 Flows zu erstellen.",
        });
        return;
      }
    }

    const newProject = await ToDoProjectModel.create({
      owner: user._id,
      name: req.query.name,
      saved: "",
    });

    res.status(200).json({
      project: newProject,
    });

    return;
  }

  const project = await ToDoProjectModel.findById(req.query.id);

  if (project) {
    if (
      project.owner.toString() !== user._id.toString() &&
      user.role !== "admin"
    ) {
      res.status(400).json({
        error: "Invalid token",
      });
      return;
    }
```

```
            if (!req.body.saved) {
              res.status(400).json({
                error: "Bad Request",
              });
              return;
            }

            project.saved = req.body.saved;

            await project.save();

            res.status(200).json({
              project: project,
            });
          } else {
            res.status(400).json({
              error: "Invalid token",
            });
          }
        } else if (req.method === "DELETE") {
          if (!req.query.id) {
            res.status(400).json({
              error: "Missing id",
            });
            return;
          }

          const project = await ToDoProjectModel.findById(req.query.id);

          if (
            project.owner.toString() !== user._id.toString() &&
            user.role !== "admin"
          ) {
            res.status(400).json({
              error: "Invalid token",
            });
            return;
          }

          await project.remove();

          res.status(200).json({
            success: true,
          });
        }
      }
    );
  } catch (e) {
    res.status(500).json({
      error: "Internal Server Error",
    });
    return;
  }
}
```

## apps/web/src/pages/api/user/[name].ts

```
/**
 * apps/web/src/pages/api/user/[name].ts
 *
```

```
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 11.03.23
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
import mongoConnect from "@codeup/util/dist/mongo";
import UserModel from "@codeup/util/dist/models/UserModel";

export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  try {
    await mongoConnect();

    const user = await UserModel.findOne({
      username: req.query.name,
    });

    if (!user) {
      res.status(400).json({
        error: "User not found",
        success: false,
      });
      return;
    }

    res.status(200).json({
      success: true,
      user: {
        id: user._id,
        username: user.username,
        profilePicture: "/api/account/" + user.username + "/picture",
        firstName: user.firstName,
        lastName: user.lastName,
      },
    });
  } catch (e: any) {
    res.status(500).json({
      error: e,
      success: false,
    });
  }
}
```

## apps/web/src/pages/api/users/[id].ts

```
/**
 * apps/web/src/pages/api/users/[id].ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 10.03.2023
 *
 */

import { NextApiRequest, NextApiResponse } from "next";
```

```
import mongoConnect from "@codeup/util/src/mongo";
import UserModel from "@codeup/util/dist/models/UserModel";


export default async function handleRequest(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== "GET") {
    res.status(405).json({
      error: "Method not allowed",
    });
    return;
  }

  await mongoConnect();

  const id = req.query.id;

  if (!id) {
    res.status(400).json({
      error: "Missing id",
    });
    return;
  }

  const user = await UserModel.findOne({
    _id: id,
  });

  if (!user) {
    res.status(400).json({
      error: "User not found",
    });
    return;
  }

  res.status(200).json({
    user: {
      id: user._id,
      username: user.username,
      firstName: user.firstName,
      lastName: user.lastName,
      profilePicture: "/api/account/" + user.username + "/picture"
    }
  });

}
```

## apps/web/src/pages/ask/index.tsx

```
/**
 * apps/web/src/pages/ask/index.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 24.06.23
 *
 */
import { useLoggedIn } from "@codeup/hooks";
import React, { useState } from "react";
```

```
import { IAskPost } from "../../types/IAskPost";
import REST from "@codeup/rest/dist";
import { Spinner, Stack } from "@chakra-ui/react";
import PostCardV3 from "@codeup/components/ask/PostCardV3";
import InfiniteScroll from "react-infinite-scroll-component";
import { useRouter } from "next/router";
import AskLayout from "@codeup/components/ask/AskLayout";

export default function Ask() {
  const { userInfo, loggedIn } = useLoggedIn();

  const [posts, setPosts] = useState<IAskPost[]>([]);
  const [page, setPage] = useState<number>(0);
  const [maxPage, setMaxPage] = useState<number>(0);

  const loadNext = () => {
    REST.Ask.getPosts(page + 1).then((res) => {
      setPosts([...posts, ...res.payload.posts]);
      setPage(page + 1);
      setMaxPage(res.payload.pages);
    });
  };
  React.useEffect(loadNext, []);
  const router = useRouter();

  return (
    <>
      <AskLayout>
        <Stack
          flex={["100%", "100%", "100%", "90%"]}
          w={"100%"}
          roundedTopRight={"xl"}
          roundedBottomRight={"xl"}
          maxH={"100vh"}
          overflow={"auto"}
          id={"scrollableDiv"}
          bg={"black"}
        >
          <InfiniteScroll
            next={loadNext}
            hasMore={page < maxPage && posts.length > 0}
            loader={<Spinner />}
            dataLength={posts.length}
            scrollableTarget="scrollableDiv"
          >
            {posts.map((p) => {
              return (
                <>
                  <PostCardV3
                    post={p}
                    loggedIn={loggedIn}
                    owner={
                      p.username === userInfo.username ||
                      userInfo.role === "admin"
                    }
                    setPosts={setPosts}
                    onClick={() => {
                      router.push("/ask/t/" + p._id);
                    }}
                  />
```

```
            </>
          );
        })}
      </InfiniteScroll>
    </Stack>
  </AskLayout>
 </>
 );
}
```

## apps/web/src/pages/ask/t/[id].tsx

```
/**
 * apps/web/src/pages/api/ask/posts/t/[id].tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 24.06.23
 *
 */
import React, { useState } from "react";
import { IAskPost } from "../../../types/IAskPost";
import AskLayout from "@codeup/components/ask/AskLayout";
import PostCardV3 from "@codeup/components/ask/PostCardV3";
import { Heading, HStack, Spinner, Stack, Text } from "@chakra-ui/react";
import REST from "@codeup/rest/dist";
import { useRouter } from "next/router";
import { FaTimes } from "react-icons/fa";
import { useLoggedIn } from "@codeup/hooks";
import InfiniteScroll from "react-infinite-scroll-component";
import { CHeading } from "@codeup/components/dist/global/CHeading";

export default function ThreadViewer() {
  const [notFound, setNotFound] = React.useState(false);

  const [post, setPost] = React.useState<IAskPost>();
  const [comments, setComments] = React.useState<IAskPost[]>([]);
  const [page, setPage] = React.useState<number>(0);
  const [maxPage, setMaxPage] = useState<number>(0);

  const { loggedIn, userInfo } = useLoggedIn();
  const router = useRouter();

  React.useEffect(() => {
    if (router.query && router.query.id) {
      REST.Ask.getPost(router.query.id as string).then((res) => {
        if (res.status === 200) {
          setPost(res.payload.posts[0]);
          loadNextComments();
        } else {
          setNotFound(true);
        }
      });
    }
  }, [router]);

  const loadNextComments = () => {
    REST.Ask.getPostsByParent(router.query.id as string).then((res) => {
      setComments([...comments, ...res.payload.posts]);
      setPage(page + 1);
```

```
    setMaxPage(res.payload.pages);
  });
};

return (
  <>
    <AskLayout>
      <Stack
        flex={["100%", "100%", "100%", "90%"]}
        w={"100%"}
        roundedTopRight={"xl"}
        roundedBottomRight={"xl"}
        maxH={"100vh"}
        overflow={"auto"}
        bgColor={"black"}
        id={"scrollableDiv"}
      >
        {notFound ? (
          <>
            <HStack
              color={"red.400"}
              alignItems={"center"}
              w={"100%"}
              justifyContent={"center"}
              fontSize={"2xl"}
            >
              <FaTimes />
              <Text>Dieser Beitrag konnte nicht gefunden werden</Text>
            </HStack>
          </>
        ) : (
          <>
            {post && (
              <PostCardV3
                post={post}
                setPosts={() => {}}
                loggedIn={loggedIn}
                owner={post.username === userInfo.username}
              />
            )}
          </>
        )}
        <CHeading size={"xl"}>Kommentare</CHeading>
        <InfiniteScroll
          next={loadNextComments}
          hasMore={page < maxPage && comments.length > 0}
          loader={<Spinner />}
          dataLength={comments.length}
          scrollableTarget="scrollableDiv"
        >
          {comments.map((p) => {
            return (
              <>
                <PostCardV3
                  post={p}
                  loggedIn={loggedIn}
                  owner={
                    p.username === userInfo.username ||
                    userInfo.role === "admin"
                  }
```

```
                setPosts={setComments}
              />
            </>
          );
        })}
      </InfiniteScroll>
    </Stack>
  </AskLayout>
 </>
 );
}
```

## apps/web/src/pages/ask/u/[username].tsx

```
/**
 * apps/web/src/pages/ask/u/[username].tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 26.06.23
 *
 */
import { useRouter } from "next/router";
import React, { Component, useState } from "react";
import { IAskProfile } from "../../../types/IAskProfile";
import ErrorComponent from "@codeup/components/dist/global/ErrorComponent";
import AuthenticateLoadingIndicator from
"@codeup/components/dist/admin/AuthenticateLoadingIndicator";
import AskLayout from "@codeup/components/ask/AskLayout";
import REST from "@codeup/rest/src";
import { ProfileAvatar } from "@codeup/components/dist/community/profile";
import AskProfileCertificateDisplay from "@codeup/components/ask/AskProfileCertificateDisplay";
import AskProfileProjectDisplay from "@codeup/components/ask/AskProfileProjectDisplay";
import {
  Box,
  Button,
  chakra,
  Flex,
  Heading,
  IconButton,
  Image,
  Link,
  Spinner,
  Stack,
  Text,
  useDisclosure,
} from "@chakra-ui/react";
import { HiBadgeCheck } from "react-icons/hi";
import { FaGlobe, FaPen } from "react-icons/fa";
import { useLoggedIn } from "@codeup/hooks";
import { IAskPost } from "../../../types/IAskPost";
import InfiniteScroll from "react-infinite-scroll-component";
import PostCardV3 from "@codeup/components/dist/ask/PostCardV3";
import EditProfileModal from "@codeup/components/ask/EditProfileModal";
import { CHeading } from "@codeup/components/dist/global/CHeading";

export default function UserProfileViewer() {
  const router = useRouter();

  const [profile, setProfile] = React.useState<IAskProfile>();
```

```
const [notFound, setNotFound] = React.useState<boolean>(false);

React.useEffect(() => {
  if (router.query && router.query.username) {
    const uName = (router.query.username as string).split("@")[1];

    REST.Ask.getProfile(uName).then((res) => {
      if (res.status === 200) {
        setProfile(res.payload.profile);
      } else {
        setNotFound(true);
      }
    });
  }
}, [router, router.query]);

const { userInfo, loggedIn } = useLoggedIn();

const [posts, setPosts] = useState<IAskPost[]>([]);
const [page, setPage] = useState<number>(0);
const [maxPage, setMaxPage] = useState<number>(0);

const loadNext = () => {
  if (!router || !router.query || !router.query.username) return;
  REST.Ask.getPostsByUser(
    (router.query.username as string).split("@")[1],
    page + 1
  ).then((res) => {
    let newPosts = [...posts, ...res.payload.posts];
    // remove duplicates
    newPosts = newPosts.filter(
      (post, index, self) =>
        index === self.findIndex((t) => t._id === post._id)
    );
    setPosts(newPosts);
    setPage(page + 1);
    setMaxPage(res.payload.pages);
  });
};
React.useEffect(loadNext, [router, router.query]);

const { onOpen, onClose, isOpen } = useDisclosure();

if (notFound) {
  return (
    <>
      <ErrorComponent
        description={"Das Profil konnte nicht gefunden werden!"}
      />
    </>
  );
}

if (!profile) {
  return (
    <>
      <AuthenticateLoadingIndicator text={"Lade Profil..."} />
    </>
  );
}
```

```
return (
  <>
    <AskLayout>
      <Box
        w={"100%"}
        flex={["100%", "100%", "100%", "90%"]}
        roundedTopRight={"xl"}
        roundedBottomRight={"xl"}
        bgColor={"black"}
        minH={"100vh"}
        h={"fit-content"}
      >
        <Image
          src={profile.banner}
          w={"100%"}
          h={[312, 624]}
          maxHeight={[250, 450]}
          maxW={"100vw"}
          objectFit={"cover"}
          borderRadius={"12px"}
          borderTopRadius={0}
          alt={"Banner"}
          shadow={"lg"}
        />
        <Flex flexDirection={["column", "row"]} flex={"100%"} gap={"1.2rem"}>
          <ProfileAvatar
            url={"/api/account/" + profile.username + "/picture"}
          />
          <Flex
            justifyContent={"center"}
            alignItems={"center"}
            flexDirection={"row"}
            flex={"20%"}
            mt={"1rem"}
            ml={[0, "-128px"]}
            gap={4}
          >
            <CHeading size={"xl"}>@{profile.username}</CHeading>
            <Heading
              size={"lg"}
              display={"flex"}
              alignItems={"center"}
              gap={"1.2rem"}
            >
              {profile.badges.includes("verified") ? (
                <>
                  <chakra.span color={"brand.500"}>
                    <HiBadgeCheck />
                  </chakra.span>
                </>
              ) : null}
            </Heading>

            {userInfo.username === profile.username && (
              <IconButton
                aria-label={"Bearbeiten"}
                variant={"ghost"}
                icon={<FaPen />}
                colorScheme={"blue"}
```

```
                onClick={onOpen}
              />
            )}
          </Flex>
        </Flex>
        <Flex
          direction={["column", "row"]}
          p={8}
          gap={6}
          alignItems={"center"}
        >
          {profile.bio && profile.bio.length > 0 ? (
            <Text mt={4} flex={["100%", "60%"]}>
              {profile.bio}
            </Text>
          ) : null}
          {profile.website.match(/(https?:\/\/[^\s]+)/g) ? (
            <Stack>
              <Button
                as={Link}
                href={profile.website}
                variant={"ghost"}
                leftIcon={<FaGlobe />}
                isExternal={true}
                target={"_blankw"}
              >
                Webseite
              </Button>
            </Stack>
          ) : null}
        </Flex>
        <Box pt={4} pb={4}>
          <CHeading size={"xl"}>Zertifikate</CHeading>
        </Box>
        <AskProfileCertificateDisplay certificates={profile.certificates} />
        <Box pt={4} pb={4}>
          <CHeading size={"xl"}>Projekte</CHeading>
        </Box>
        <AskProfileProjectDisplay projects={profile.projects} />
        <Box pt={4} pb={4}>
          <CHeading size={"xl"}>Beiträge</CHeading>
        </Box>

        <Stack w={"100%"} roundedBottomRight={"xl"} overflow={"scroll"}>
          {posts.map((post) => {
            return <PostCardV3 post={post} setPosts={setPosts} />;
          })}
          <Button onClick={loadNext}>Mehr laden</Button>
        </Stack>
      </Box>
    </AskLayout>
    <EditProfileModal
      onOpen={onOpen}
      onClose={onClose}
      isOpen={isOpen}
      profile={profile}
    />
  </>
 );
}
```

## apps/web/src/pages/blog/index.tsx

```
/**
 * apps/web/src/pages/blog/index.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 27.12.2022
 *
 */

import * as React from "react";
import {
  Box,
  chakra,
  Flex,
  Grid,
  GridItem,
  Heading,
  HStack,
  Input,
  Link,
  Tag,
  TagLabel,
  Text,
  useToast,
} from "@chakra-ui/react";
import REST from "@codeup/rest";
import ErrorComponent from "@codeup/components/dist/global/ErrorComponent";
import { FaPlus } from "react-icons/fa";
import { useRouter } from "next/router";
import { CHeading } from "@codeup/components/dist/global/CHeading";

export default function Blogs() {
  const [blogs, setBlogs] = React.useState<
    {
      _id: string;
      title: string;
      content: string;
      tags: string[];
      published_at: string;
      authorUsername: string;
    }[]
  >([]);

  const [filteredBlogs, setFilteredBlogs] = React.useState<typeof blogs>([]);

  const toast = useToast();
  const [error, setError] = React.useState(false);
  const [query, setQuery] = React.useState("");

  React.useEffect(() => {
    REST.Blog.getBlogsB().then((res) => {
      if (res.status === 200) {
        setBlogs(res.payload.blogs);
        setFilteredBlogs(res.payload.blogs);
      } else {
        setError(true);
      }
```

```
  });
}, []);

React.useEffect(() => {
  let q = query;
  q = q.trim();
  setFilteredBlogs(
    blogs.filter(
      (b: any) =>
        b.title.toLowerCase().includes(q.toLowerCase()) ||
        b.content.toLowerCase().includes(q.toLowerCase()) ||
        b.tags.join(" ").toLowerCase().includes(q.toLowerCase())
    )
  );
}, [query]);

const router = useRouter();

return (
  <>
    {error ? (
      <>
        <ErrorComponent description={"Blogs konnten nicht geladen werden."} />
      </>
    ) : (
      <Box bg={"#000"} h={"fit-content"} minH={"100vh"}>
        <Box p={6} textAlign={"center"}>
          <CHeading size={"4xl"}>Blog</CHeading>
        </Box>
        <Flex w={"100%"} alignItems={"center"} justifyContent={"center"}>
          <Input
            w={["75%", "25%"]}
            placeholder={"Suche"}
            onChange={(e) => {
              setQuery(e.currentTarget.value);
            }}
          />
        </Flex>
        <Grid
          templateColumns={[
            "repeat(1, 1fr)",
            "repeat(2, 1fr)",
            "repeat(3, 1fr)",
          ]}
          gap={"2rem"}
          p={8}
        >
          {filteredBlogs.map((blog) => (
            <>
              <GridItem
                key={blog._id}
                bg={"black"}
                p={"1rem"}
                borderRadius={"0.5rem"}
                as={Link}
                href={"/blog/" + blog._id}
                cursor={"pointer"}
                _hover={{
                  textDecoration: "none",
                }}
```

```
              onClick={() => {
                router.push("/blog/" + blog._id);
              }}
            >
              <Flex alignItems={"center"}>
                <CHeading size={"lg"}>{blog.title}</CHeading>
                <Text ml={"auto"}>
                  {new Date(blog.published_at).toLocaleString()}
                </Text>
              </Flex>

              <Text>
                von
                <chakra.span color={"blue.300"} ml={2}>
                  @{blog.authorUsername}
                </chakra.span>
              </Text>
              <HStack mt={4} mb={4} overflow={"scroll"} w={"fit-content"}>
                {blog.tags.map((tag) => {
                  return (
                    <>
                      <Tag size={"md"} key={tag} borderRadius={"full"}>
                        <TagLabel>{tag.toUpperCase()}</TagLabel>
                      </Tag>
                    </>
                  );
                })}
              </HStack>
              <p>{blog.content.substring(0, 200)} ...</p>
            </GridItem>
          </>
        ))}
      </Grid>
    </Box>
  )}
 </>
 );
}
```

## apps/web/src/pages/blog/[id].tsx

```
/**
 * apps/web/src/pages/blog/[id].tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 27.12.2022
 *
 */

import * as React from "react";
import { ReactNode, useEffect } from "react";
import {
  Box,
  CircularProgress,
  Flex,
  Heading,
  Link,
  ListItem,
  OrderedList,
```

```
  Table,
  Tbody,
  Tr,
  Text,
  Thead,
  UnorderedList,
  Td,
  Th,
  Code,
  Divider,
} from "@chakra-ui/react";
import ErrorComponent from "@codeup/components/dist/global/ErrorComponent";
import { useRouter } from "next/router";
import REST from "@codeup/rest";
import Markdown from "markdown-to-jsx";
import { CHeading } from "@codeup/components/dist/global/CHeading";

export default function BlogViewer() {
  const [status, setStatus] = React.useState<"loading" | "error" | "success">(
    "loading"
  );
  const [blog, setBlog] = React.useState<{
    _id: string;
    title: string;
    content: string;
    tags: string[];
    published_at: string;
    authorUsername: string;
  } | null>(null);

  const markdownToListOfElements = (markdown: string) => {
    const elements: ReactNode[] = [];

    // recognize all headings
    const headings = markdown.match(/(#+)(.*)/g);
    if (headings) {
      headings.forEach((heading) => {
        const level = heading.match(/#+/g)?.[0].length;
        const content = heading.replace(/#+/g, "").trim();
        elements.push(
          <Heading
            mb={4}
            mt={4}
            fontSize={
              level === 1
                ? "2xl"
                : level === 2
                ? "xl"
                : level === 3
                ? "lg"
                : level === 4
                ? "md"
                : level === 5
                ? "sm"
                : "xs"
            }
          >
            {content}
          </Heading>
        );
```

```
    });
  }

  // recognize bold, italic and strikethrough
  const bold = markdown.match(/(\*\*|__)(.*?)\1/g);
  if (bold) {
    bold.forEach((bold) => {
      const content = bold.replace(/(\*\*|__)/g, "").trim();
      elements.push(<b>{content}</b>);
    });
  }

  const italic = markdown.match(/(\*|_)(.*?)\1/g);
  if (italic) {
    italic.forEach((italic) => {
      const content = italic.replace(/(\*|_)/g, "").trim();
      elements.push(<i>{content}</i>);
    });
  }

  return elements;
};

const router = useRouter();

useEffect(() => {
  const id = router.query.id;

  if (id) {
    REST.Blog.getBlog(id as string).then((res) => {
      if (res.status === 200) {
        setBlog(res.payload.blog);
        setStatus("success");
      } else {
        setStatus("error");
      }
    });
  }
}, [router]);

return (
  <>
    {status === "loading" ? (
      <>
        <Flex
          w={"100%"}
          h={"100vh"}
          justifyContent={"center"}
          alignItems={"center"}
          direction={"column"}
          bg={"#000"}
        >
          <CircularProgress size={"80px"} isIndeterminate color="red.300" />
          <Heading size={"lg"} mt={4}>
            Laden...
          </Heading>
        </Flex>
      </>
    ) : (
      <>
```

```
{status === "error" ? (
  <>
    <ErrorComponent description={"Ein Fehler ist aufgetreten!"} />
  </>
) : (
  <>
    <Box
      w={"100%"}
      minH={"100vh"}
      h={"fit-content"}
      display={"flex"}
      flexDirection={"column"}
      alignItems={"center"}
      bg={"#000"}
    >
      <Box
        w={"100%"}
        mt={8}
        mb={8}
        maxW={["100%", "70%"]}
        h={"fit-content"}
        flexDirection={"column"}
        alignItems={"center"}
        justifyContent={"center"}
        p={4}
        bg={"black"}
        borderRadius={"md"}
        boxShadow={"2xl"}
      >
        <Box mb={6} textAlign={"center"}>
          <CHeading size={"2xl"}>{blog?.title}</CHeading>
        </Box>
        <Divider />
        <Markdown
          options={{
            overrides: {
              h1: {
                component: Heading,
                props: {
                  fontSize: "6xl",
                  mb: 4,
                  mt: 4,
                  color: "brand.500",
                  fontFamily: "Jetbrains Mono ExtraBold",
                  letterSpacing: -2,
                },
              },
              h2: {
                component: Heading,
                props: {
                  fontSize: "4xl",
                  mb: 4,
                  mt: 4,
                  color: "brand.500",
                  fontFamily: "Jetbrains Mono ExtraBold",
                },
              },
              h3: {
                component: Heading,
                props: {
```

```
    fontSize: "2xl",
    mb: 4,
    mt: 4,
  },
},
h4: {
  component: Heading,
  props: {
    fontSize: "xl",
    mb: 4,
    mt: 4,
  },
},
h5: {
  component: Heading,
  props: {
    fontSize: "lg",
    mb: 4,
    mt: 4,
  },
},
h6: {
  component: Heading,
  props: {
    fontSize: "md",
    mb: 4,
    mt: 4,
  },
},
a: {
  component: Link,
  props: {
    fontSize: "xl",
    color: "brand.500",
  },
},
ul: {
  component: UnorderedList,
  props: {
    fontSize: "xl",
  },
},
ol: {
  component: OrderedList,
  props: {
    fontSize: "xl",
  },
},
li: {
  component: ListItem,
  props: {
    fontSize: "xl",
  },
},
table: {
  component: Table,
  props: {
    fontSize: "xl",
  },
},
```

```
                    thead: {
                      component: Thead,
                      props: {
                        fontSize: "xl",
                      },
                    },
                    tbody: {
                      component: Tbody,
                      props: {
                        fontSize: "xl",
                      },
                    },
                    tr: {
                      component: Tr,
                      props: {
                        fontSize: "xl",
                      },
                    },
                    th: {
                      component: Th,
                      props: {
                        fontSize: "xl",
                      },
                    },
                    td: {
                      component: Td,
                      props: {
                        fontSize: "xl",
                      },
                    },
                    p: {
                      component: Text,
                      props: {
                        fontSize: "xl",
                      },
                    },
                    code: {
                      component: Code,
                      props: {
                        fontSize: "xl",
                      },
                    },
                  },
                }}
              >
                {blog?.content as string}
              </Markdown>
            </Box>
          </Box>
        </>
      )}
    </>
  )}
 </>
 );
}
```

## apps/web/src/pages/certificates/index.tsx

```
/**
 * apps/web/src/pages/certificates/index.tsx
```

```
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 27.11.2022
 *
 */

import * as React from "react";
import REST from "@codeup/rest";
import {
  Box,
  chakra,
  Flex,
  Grid,
  GridItem,
  Heading,
  Image,
  Input,
  InputGroup,
  InputLeftElement,
  Text,
  useToast,
} from "@chakra-ui/react";
import { FaSearch } from "react-icons/fa";
import { useRouter } from "next/router";
import { useEffect } from "react";
import { useLoggedIn } from "@codeup/hooks";
import Head from "next/head";
import AccountManager from "@codeup/util/dist/AccountManager";
import { CHeading } from "@codeup/components/dist/global/CHeading";

export default function Index() {
  const [certificates, setCertificates] = React.useState<string[]>([]);
  const [courses, setCourses] = React.useState<any[]>([]);

  const toast = useToast();

  const { userInfo, loggedIn, loaded } = useLoggedIn();
  useEffect(() => {
    if (loaded && !loggedIn) {
      router.push("/");
      window.dispatchEvent(new CustomEvent("codeup::open_login"));
    }
  }, [userInfo, loggedIn, loaded]);

  React.useEffect(() => {
    REST.Course.listCourses({
      token: AccountManager.getActiveToken(),
    }).then(async (res) => {
      if (res.status === 200) {
        const courses = [];
        const certs = [];

        for (const course of res.payload.courses) {
          const r1 = await REST.Certificates.getCertificate({
            token: localStorage.getItem("token") as string,
            onlyLookup: true,
            course: course._id,
          });
```

```
        if (r1.status === 200) {
          courses.push(course);
          certs.push(r1.payload);
        }
      }

      // @ts-ignore
      setCertificates(certs);
      setCourses(courses);
    } else {
      toast({
        title: "Fehler",
        description: "Es ist ein Fehler aufgetreten: " + res.payload.error,
        status: "error",
        duration: 1500,
        isClosable: true,
        position: "bottom-left",
      });
    }
  });
}, []);

const router = useRouter();

return (
  <>
    <Head>
      <title>Zertifikatübersicht - CodeUp</title>
    </Head>
    <Flex
      direction="column"
      w={"100%"}
      gap={"2rem"}
      pt={"2rem"}
      h={"fit-content"}
      minH={"100vh"}
      alignItems={"center"}
      bg={"#000"}
    >
      <Flex
        w={"100%"}
        h={"fit-content"}
        direction={"column"}
        alignItems={"center"}
      >
        <Box textAlign={"center"}>
          <CHeading size={"3xl"}>Zertifikate</CHeading>
        </Box>
        <Text fontSize={["xl", "2xl"]}>
          Aktuell kannst du in folgenden Kursen ein Zertifikat erhalten:
        </Text>
      </Flex>
      <Grid
        templateColumns={
          courses.length > 0
            ? ["repeat(1, 1fr)", "repeat(3, 1fr)"]
            : "repeat(1, 1fr)"
        }
        gap={6}
```

```
  w={"100%"}
  justifyContent="center"
  alignItems={"center"}
  h={"fit-content"}
>
  {courses.map((c) => (
    <>
      <GridItem m={8}>
        <Box
          bg={"gray.900"}
          color={"white"}
          rounded={"lg"}
          p={6}
          h={"100%"}
          cursor={"pointer"}
          _hover={{
            shadow: "2xl",
          }}
          onClick={() => {
            router.push("/certificates/" + c._id);
          }}
        >
          <Flex
            justifyContent={"space-between"}
            direction={"column"}
            gap={2}
          >
            <Image
              src={c.splashImage}
              alt={c.name}
              w={"100%"}
              h={"150px"}
            />
            <CHeading size={"xl"}>{c.name}</CHeading>
            <Text fontSize={"xl"} color={"gray.300"} textAlign={"left"}>
              von 
              <chakra.span color={"brand.400"}>
                @{c.creator}
              </chakra.span>
            </Text>
            <Heading
              size={"lg"}
              color={"gray.300"}
              fontWeight={"normal"}
              textAlign={"left"}
            >
              {c.description}
            </Heading>
          </Flex>
        </Box>
      </GridItem>
    </>
  ))}
</Grid>
{courses.length === 0 ? (
  <>
    <Heading
      size={"md"}
      color={"gray.400"}
      fontWeight={"normal"}
```

```
            textAlign={"center"}
          >
            Keine Kurse gefunden
          </Heading>
        </>
      ) : null}
    </Flex>
  </>
 );
}
```

## apps/web/src/pages/certificates/[id].tsx

```
/**
 * apps/web/src/pages/certificates/[id].tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 27.11.2022
 *
 */

import * as React from "react";
import { useRouter } from "next/router";
import { useEffect } from "react";
import REST from "@codeup/rest";
import {
  Box,
  Flex,
  Heading,
  Text,
  useToast,
  chakra,
  ButtonGroup,
  Button,
  useDisclosure,
  Stack,
  Input,
  Image,
} from "@chakra-ui/react";
import CodeUpModal from "@codeup/components/codeup/CodeUpModal";
import { useLoggedIn } from "@codeup/hooks";
import Head from "next/head";
import { CHeading } from "@codeup/components/dist/global/CHeading";

export default function CertViewer() {
  const router = useRouter();

  const { userInfo, loggedIn, loaded } = useLoggedIn();

  const { id } = router.query;

  const [steps, setSteps] = React.useState<any[]>([]);
  const toast = useToast();

  useEffect(() => {
    if (id && loggedIn && loaded) {
      REST.Certificates.getCertificate({
        token: localStorage.getItem("token") as string,
        onlyLookup: false,
```

```
      course: id as string,
    }).then((res) => {
      if (res.status === 200) {
        setSteps(res.payload.certificate.steps);
      } else {
        toast({
          title: "Fehler",
          description: "Das Zertifikat konnte nicht geladen werden.",
          status: "error",
          duration: 1500,
          isClosable: true,
          position: "bottom-right",
        });
      }
    });
  }
}, [id, loggedIn, loaded]);

useEffect(() => {
  if (loaded && !loggedIn) {
    router.push("/");
    window.dispatchEvent(new CustomEvent("codeup::open_login"));
  }
}, [userInfo, loggedIn, loaded]);

const { isOpen, onOpen, onClose } = useDisclosure();

const [currentStep, setCurrentStep] = React.useState(0);

const [answers, setAnswers] = React.useState<any[]>([]);

const [finishButtonLoading, setFinishButtonLoading] = React.useState(false);

React.useEffect(() => {
  console.log(answers);
}, [answers]);

return (
  <>
    <Head>
      <title>Zertifikat beantragen - CodeUp</title>
    </Head>
    <Flex
      w={"100%"}
      h={"fit-content"}
      minH={"100vh"}
      alignItems={"center"}
      justifyContent={"center"}
      direction={"column"}
      bg={"#000"}
    >
      <Box maxW={["90%", "30%"]}>
        <Box textAlign={"center"}>
          <CHeading size={"2xl"}>Zertifikat</CHeading>
        </Box>
        <Text fontSize="2xl" mt={8} textAlign={"center"}>
          Fülle die folgenden Schritte aus, um dein Zertifikat zu erhalten.
          <br />
          <chakra.span color={"brand.500"} fontWeight={"bold"}>
            ACHTUNG:
```

```
      </chakra.span>
       Du kannst die Schritte nicht mehr bearbeiten, wenn du sie
      abgeschickt hast! Ein Zertifikat kannst du allerdings jederzeit
      erneut beantragen, um deine Punktzahl zu verbessern.
    </Text>
    <ButtonGroup
      mt={10}
      alignItems={"center"}
      justifyContent={"center"}
      w={"100%"}
    >
      <Button
        size={"lg"}
        variant={"brand"}
        onClick={() => {
          onOpen();
          setCurrentStep(0);
          setAnswers([]);
        }}
      >
        Zertifikat beantragen
      </Button>
    </ButtonGroup>
  </Box>
</Flex>

<CodeUpModal
  onOpen={onOpen}
  onClose={onClose}
  isOpen={isOpen}
  noCloseButton
  blackBackground={true}
  title={
    "Zertifikat beantragen (Schritt " +
    (currentStep === steps.length
      ? steps.length + "/" + steps.length + ")"
      : currentStep + 1 + "/" + steps.length + ")")
  }
  customSize={["full", "5xl"]}
>
  <>
    {steps[currentStep] && (
      <>
        {steps[currentStep].step_type === "MULTIPLE_CHOICE" && (
          <>
            <Heading as="h1" size="2xl" mt={8} textAlign={"center"}>
              {steps[currentStep].multiple_choice.question}
            </Heading>
            <Stack mt={8}>
              {steps[currentStep].multiple_choice.answers.map(
                (a: any, i: any) => {
                  return (
                    <>
                      <Button
                        variant={"solid"}
                        onClick={() => {
                          setAnswers([
                            ...answers,
                            {
                              step: currentStep,
```

```
                  answer: i,
                },
              ]);
              setCurrentStep(currentStep + 1);
            }}
            rounded={"none"}
          >
            {a.answer}
          </Button>
        </>
      );
    }
  )}
  </Stack>
 </>
)}
{steps[currentStep].step_type === "TEXT" && (
 <>
  <Heading size={"2xl"} textAlign={"center"} mt={8}>
   {steps[currentStep].step_title}
  </Heading>
  <form
   onSubmit={(e) => {
    e.preventDefault();
    const form = e.target as HTMLFormElement;

    const data = new FormData(form);

    setAnswers([
     ...answers,
     {
      step: currentStep,
      ...Object.fromEntries(data),
     },
    ]);

    setCurrentStep(currentStep + 1);
   }}
  >
   {steps[currentStep].text_parts.map((t: any) => (
    <>
     {t.type === "TEXT" && (
      <>
       <Text mt={8} fontSize={"xl"}>
        {t.value}
       </Text>
      </>
     )}
     {t.type === "INPUT" && (
      <>
       <Input mt={8} name={t.id} />
      </>
     )}
    </>
   ))}
   <ButtonGroup w={"100%"} mt={8}>
    <Button w={"100%"} colorScheme={"blue"} type={"submit"}>
     Weiter
    </Button>
   </ButtonGroup>
```

```
    </form>
  </>
)}

{steps[currentStep].step_type === "CODE_CHECK" && (
  <>
    <Heading size={"2xl"} textAlign={"center"} mt={8}>
      {steps[currentStep].step_title}
    </Heading>
    <Flex
      direction={"column"}
      w={"100%"}
      h={"fit-content"}
      paddingInline={10}
      paddingBlock={5}
      gap={40}
    >
      <Box flex={"100%"}>
        <Stack>
          <Heading>Ist der folgende Code richtig?</Heading>
          <Image
            src={steps[currentStep].code_check?.screenshot}
            alt="Code Screenshot"
          />
        </Stack>
      </Box>
      <Box flex={"100%"}>
        <form>
          <ButtonGroup w={"100%"}>
            <Button
              w={"100%"}
              colorScheme={"red"}
              onClick={() => {
                setAnswers([
                  ...answers,
                  {
                    step: currentStep,
                    answer: false,
                  },
                ]);
                setCurrentStep(currentStep + 1);
              }}
            >
              Falsch
            </Button>
            <Button
              w={"100%"}
              colorScheme={"green"}
              onClick={() => {
                setAnswers([
                  ...answers,
                  {
                    step: currentStep,
                    answer: true,
                  },
                ]);
                setCurrentStep(currentStep + 1);
              }}
            >
              Richtig
```

```
          </Button>
        </ButtonGroup>
      </form>
    </Box>
  </Flex>
 </>
)}
{steps[currentStep].step_type === "CODE_COMPARE" && (
 <>
  <Heading size={"2xl"} textAlign={"center"} mt={8}>
   {steps[currentStep].step_title}
  </Heading>
  <Flex
   direction={"column"}
   w={"100%"}
   h={"fit-content"}
   paddingInline={10}
   paddingBlock={5}
   gap={40}
  >
   <Box flex={"100%"}>
    <Stack>
     <Heading>
      Welcher der Codes bringt die gewünschte Ausgabe?
     </Heading>
     <Image
      src={steps[currentStep].code_compare?.output}
      alt="Output Image"
     />
    </Stack>
    <Stack>
     <Heading>Code 1</Heading>
     <Image
      src={
       steps[currentStep].code_compare?.code_a.screenshot
      }
      alt="Code 1 Screenshot"
     />
     <Heading>Code 2</Heading>
     <Image
      src={
       steps[currentStep].code_compare?.code_b.screenshot
      }
      alt="Code 2 Screenshot"
     />
    </Stack>
   </Box>
   <Box flex={"100%"}>
    <form>
     <ButtonGroup w={"100%"}>
      <Button
       w={"100%"}
       colorScheme={"red"}
       onClick={() => {
        setAnswers([
         ...answers,
         {
          step: currentStep,
          answer: 0,
         },
```

```
                ]);
                setCurrentStep(currentStep + 1);
              }}
            >
              Code 1
            </Button>
            <Button
              w={"100%"}
              colorScheme={"green"}
              onClick={() => {
                setAnswers([
                  ...answers,
                  {
                    step: currentStep,
                    answer: 1,
                  },
                ]);
                setCurrentStep(currentStep + 1);
              }}
            >
              Code 2
            </Button>
          </ButtonGroup>
        </form>
      </Box>
    </Flex>
  </>
 )}
</>
)}
{currentStep === steps.length && (
 <>
  <Box textAlign={"center"} pt={4}>
   <CHeading size={"2xl"}>Zertifikat beantragen</CHeading>
  </Box>
  <Text fontSize="2xl" mt={8} textAlign={"center"}>
   Du hast alle Schritte abgeschlossen. Möchtest du das Zertifikat
   beantragen?
  </Text>
  <ButtonGroup
   mt={10}
   alignItems={"center"}
   justifyContent={"center"}
   w={"100%"}
  >
   <Button
    size={"lg"}
    variant={"brand"}
    isLoading={finishButtonLoading}
    onClick={async () => {
     setFinishButtonLoading(true);

     const res = await REST.Certificates.submitCertificate({
      token: localStorage.getItem("token") as string,
      course: id as string,
      answers,
     });

     if (res.status === 200) {
      if (res.payload.obtainedCertificate) {
```

```
                    router.push(
                      "/user-cert/" + res.payload.obtainedCertificateId
                    );
                  } else {
                    router.push(
                      "/error?error=Du hast nicht genug Punkte für das Zertifikat erreicht."
                    );
                  }
                } else {
                  setFinishButtonLoading(false);
                  toast({
                    title: "Fehler",
                    description:
                      "Beim Abschicken ist ein Fehler aufgetreten: " +
                      res.payload.error,
                    status: "error",
                    duration: 1500,
                    isClosable: true,
                    position: "bottom-right",
                  });
                }
              }}
            >
              Zertifikat beantragen
            </Button>
          </ButtonGroup>
        </>
      )}
    </>
  </CodeUpModal>
 </>
);
}
```

## apps/web/src/pages/challenges/index.tsx

```
/**
 * apps/web/src/pages/challenges/index.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2024 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 18.05.2024
 *
 */

import * as React from "react";
import {
  Box,
  Button,
  Table,
  TableContainer,
  Tbody,
  Td,
  Th,
  Thead,
  Tr,
  chakra,
} from "@chakra-ui/react";
import { CHeading } from "@codeup/components/dist/global/CHeading";
import { useEffect } from "react";
```

```
import redirect from "@codeup/util/dist/userworkflow/redirect";
import REST from "@codeup/rest";
import AccountManager from "@codeup/util/dist/AccountManager";
import { FaCheckCircle, FaTimesCircle } from "react-icons/fa";
import { useRouter } from "next/router";

export default function Index() {
  const [challenges, setChallenges] = React.useState<
    {
      _id: string;
      title: string;
      difficulty: number;
      task: string;
    }[]
  >([]);
  const [completed, setCompleted] = React.useState<string[]>([]);

  useEffect(() => {
    redirect({
      forceAnonymous: false,
      allowAnonymous: false,
      redirectToLogin: true,
      redirectTo: "/challenges",
    });
  });

  useEffect(() => {
    reload();
  }, []);

  const reload = async () => {
    const r0 = await REST.Challenges.getChallenges(
      AccountManager.getActiveToken()
    );

    if (r0.status !== 200) {
      return;
    }

    setChallenges(r0.payload.challenges);

    const r1 = await REST.Challenges.getCompletedChallenges(
      AccountManager.getActiveToken()
    );

    if (r1.status !== 200) {
      return;
    }

    setCompleted(r1.payload.data.challenges);
  };

  const router = useRouter();

  const [showCompleted, setShowCompleted] = React.useState<boolean>(false);

  return (
    <>
      <Box w={"100%"} h={"fit-content"} minH={"100vh"} bg={"#000"} p={4}>
        <Box textAlign={"center"}>
```

```
  <CHeading size={"xl"}>Challenges</CHeading>
  <Button
    mt={3}
    variant={"brand"}
    onClick={() => {
      setShowCompleted(!showCompleted);
    }}
  >
    Abgeschlossene {showCompleted ? "verstecken" : "anzeigen"}
  </Button>
</Box>
<Box p={4}>
  <TableContainer>
    <Table variant={"simple"}>
      <Thead>
        <Th>Abgeschlossen</Th>
        <Th>Titel</Th>
        <Th>Schwierigkeit</Th>
        <Th>Aufgabe</Th>
      </Thead>
      <Tbody>
        {challenges
          .filter((c) => showCompleted || !completed.includes(c._id))
          .map((c) => {
            return (
              <>
                <Tr
                  _hover={{ bg: "gray.700" }}
                  cursor={"pointer"}
                  onClick={() => {
                    router.push("/challenges/" + c._id);
                  }}
                >
                  <Td>
                    {completed.includes(c._id) ? (
                      <chakra.span color={"brand.500"}>
                        <FaCheckCircle />
                      </chakra.span>
                    ) : (
                      <chakra.span color={"red.500"}>
                        <FaTimesCircle />
                      </chakra.span>
                    )}
                  </Td>
                  <Td>{c.title}</Td>
                  <Td
                    color={
                      c.difficulty === 1
                        ? "green.500"
                        : c.difficulty === 2
                        ? "orange.500"
                        : "red.500"
                    }
                  >
                    {c.difficulty === 1
                      ? "Leicht"
                      : c.difficulty === 2
                      ? "Mittel"
                      : "Schwer"}
                  </Td>
```

```
                    <Td>{c.task.substring(0, 100)}...</Td>
                  </Tr>
                </>
              );
            })}
          </Tbody>
        </Table>
      </TableContainer>
    </Box>
  </Box>
 </>
);
}
```

## apps/web/src/pages/challenges/[id].tsx

```
/**
 * apps/web/src/pages/challenges/[id].tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2024 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 18.05.2024
 *
 */

import * as React from "react";
import { useEffect, useRef, useState } from "react";
import redirect from "@codeup/util/dist/userworkflow/redirect";
import {
  Box,
  Button,
  Flex,
  Select,
  chakra,
  Text,
  useDisclosure,
} from "@chakra-ui/react";
import REST from "@codeup/rest/dist";
import AccountManager from "@codeup/util/dist/AccountManager";
import { useRouter } from "next/router";
import AuthenticateLoadingIndicator from
"@codeup/components/dist/admin/AuthenticateLoadingIndicator";
import { CHeading } from "@codeup/components/dist/global/CHeading";
import Editor, { useMonaco } from "@monaco-editor/react";
import { editor } from "monaco-editor";
import IStandaloneCodeEditor = editor.IStandaloneCodeEditor;
import { LanguageTemplates } from "../../global/rce/language/LanguageTemplates";
import { FaCheck, FaPlay } from "react-icons/fa";
import { IResponse } from "@codeup/rest/dist/types/IResponse";
import CodeUpModal from "@codeup/components/dist/codeup/CodeUpModal";
import PopupManager from "../../global/PopupManager";

export default function ChallengeViewer() {
  const [challenge, setChallenge] = React.useState<{
    _id: string;
    title: string;
    difficulty: number;
    task: string;
  } | null>(null);
  useEffect(() => {
```

```
  redirect({
    forceAnonymous: false,
    allowAnonymous: false,
    redirectToLogin: true,
    redirectTo: "/challenges",
  });
});

const router = useRouter();

useEffect(() => {
  if (!router.query.id) return;

  REST.Challenges.getChallenges(
    AccountManager.getActiveToken(),
    router.query.id as string
  ).then((r) => {
    if (r.status === 200) {
      setChallenge(r.payload.challenge);
    } else {
      router.push("/error?error=Die Challenge konnte nicht geladen werden");
    }
  });
}, [router.query.id]);

const monaco = useMonaco();
const [monacoTheme, setMonacoTheme] = useState("vs-dark");
const [monacoLanguage, setMonacoLanguage] = useState("javascript");

const editorRef = useRef<IStandaloneCodeEditor>();

React.useEffect(() => {
  if (monaco && monaco.editor) {
    monaco.editor.defineTheme("codeup-dark", {
      base: "vs-dark",
      inherit: true,
      rules: [],
      colors: {
        "editor.background": "#141517",
      },
    });

    setMonacoTheme("codeup-dark");
  }
}, [monaco]);
React.useEffect(() => {
  if (editorRef && editorRef.current) {
    monaco?.editor.setModelLanguage(
      editorRef.current.getModel() as editor.ITextModel,
      monacoLanguage
    );
    editorRef.current.setValue(
      LanguageTemplates[monacoLanguage as any] as any
    );
  }
}, [monacoLanguage]);

const [resultLoading, setResultLoading] = useState<boolean>(false);
const [result, setResult] = useState<string | null>(null);
```

```
const { isOpen, onOpen, onClose } = useDisclosure();

return (
  <>
    <Box w={"100%"} h={"fit-content"} minH={"100vh"} bg={"#000"} p={4}>
      {challenge ? (
        <>
          <Box bg={"gray.900"} rounded={"xl"} p={4} maxW={"500px"}>
            <CHeading size={"lg"}>Aufgabe</CHeading>
            <Text>{challenge.task}</Text>
          </Box>
          <Box
            mt={4}
            w={"100%"}
            h={"fit-content"}
            minH={"80vh"}
            bg={"gray.900"}
            rounded={"xl"}
            p={4}
          >
            <Flex alignItems={"center"} gap={4} mb={4}>
              <Select
                onChange={(e) => setMonacoLanguage(e.target.value)}
                w={"fit-content"}
                value={monacoLanguage}
              >
                {Object.keys(LanguageTemplates).map((lang) => {
                  return (
                    <option key={lang} value={lang}>
                      {lang}
                    </option>
                  );
                })}
              </Select>
              <Button
                variant={"brand"}
                leftIcon={<FaPlay />}
                onClick={async () => {
                  setResultLoading(true);
                  onOpen();
                  let res: IResponse | null = null;

                  switch (monacoLanguage) {
                    case "javascript":
                      res = await REST.Rce.rceJavaScript(
                        editorRef.current!.getValue(),
                        AccountManager.getActiveToken()
                      );
                      break;
                    case "typescript":
                      res = await REST.Rce.rceTypeScript(
                        editorRef.current!.getValue(),
                        AccountManager.getActiveToken()
                      );
                      break;
                    case "go":
                      res = await REST.Rce.rceGo(
                        editorRef.current!.getValue(),
                        AccountManager.getActiveToken()
                      );
```

```
                  break;
                case "python":
                  res = await REST.Rce.rcePython(
                    editorRef.current!.getValue(),
                    AccountManager.getActiveToken()
                  );
                  break;
                case "java":
                  res = await REST.Rce.rceJava(
                    editorRef.current!.getValue(),
                    AccountManager.getActiveToken()
                  );
                  break;
                default:
                  setResultLoading(false);
                  return;
              }

              if (res) {
                setResult(res.payload as any);
                setResultLoading(false);
              }
            }}
          >
            Ausführen
          </Button>
          <Button
            variant={"brand"}
            leftIcon={<FaCheck />}
            onClick={async () => {
              if (
                !(await PopupManager.confirmAsync({
                  title: "Challenge abschließen",
                  question:
                    "Möchtest du deine Lösung wirklich überprüfen?",
                }))
              )
                return;

              const res = await REST.Challenges.finalCheck(
                AccountManager.getActiveToken(),
                challenge._id,
                editorRef.current!.getValue(),
                monacoLanguage
              );

              if (res.status === 200) {
                await PopupManager.alertAsync({
                  title: "Challenge abgeschlossen",
                  description:
                    "Deine Lösung wurde erfolgreich überprüft und du hast die Challenge
abgeschlossen.",
                });
                router.push("/challenges");
              } else {
                await PopupManager.alertAsync({
                  title: "Fehler",
                  description: "Deine Lösung war leider nicht korrekt!",
                });
              }
```

```
              }}
            >
              Abschicken
            </Button>
          </Flex>
          <Editor
            defaultValue={LanguageTemplates[monacoLanguage as any] as any}
            defaultLanguage={monacoLanguage}
            onMount={(e) => {
              editorRef.current = e;
            }}
            onChange={(value, ev) => {}}
            theme={monacoTheme}
            options={{
              insertSpaces: false,
              wrappingIndent: "indent",
              fontSize: 22,
            }}
            height={"80vh"}
          />
        </Box>
      </>
    ) : (
      <>
        <AuthenticateLoadingIndicator text={"Lade Challenge"} />
      </>
    )}
  </Box>
  <CodeUpModal
    onOpen={onOpen}
    onClose={onClose}
    isOpen={isOpen}
    title={"Ausgabe"}
  >
    {resultLoading ? (
      <AuthenticateLoadingIndicator text={"Lade Ausgabe"} />
    ) : (
      <>
        {result &&
          result!.split("\n").map((line, i) => {
            return (
              <Flex key={i} w={"100%"} gap={2} alignItems={"center"}>
                <chakra.span color={"gray.400"}>{i + 1}</chakra.span>
                <Text>{line}</Text>
              </Flex>
            );
          })}
      </>
    )}
  </CodeUpModal>
 </>
 );
}
```

## apps/web/src/pages/codeup/about.tsx

```
/**
 * src/pages/about.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
```

```
 * License: Project License
 * Created At: 11.09.2022
 *
 */

import { Box, Flex, Heading, Link, SimpleGrid, Text } from "@chakra-ui/react";
import { useEffect, useState } from "react";
import Head from "next/head";
import * as React from "react";
import { CHeading } from "@codeup/components/dist/global/CHeading";

export default function About() {
  const [appInfo, setAppInfo] = useState<{
    git: {
      commits: number;
      commit: string;
      branch: string;
      lastCommitMessage: string;
    };
    app: {
      totalLines: number;
    };
    packages: {
      count: number;
      packages: {
        name: string;
        version: string;
      }[];
    };
  } | null>(null);

  const [local, setLocal] = useState<boolean>(false);

  const [showingPackages, setShowingPackages] = useState<number>(12);

  useEffect(() => {
    if (
      window &&
      window.location &&
      window.location.toString().match(/:([0-9]{4})/) &&
      !window.location.toString().match(/disableLocal/)
    ) {
      setLocal(true);
    } else {
      const disableLocal = window.location.toString().match(/disableLocal/);
      const fetchData = window.location.toString().match(/forceFetch/);

      if (disableLocal && !fetchData) {
        // garbage data

        setAppInfo({
          git: {
            commits: 42,
            commit: "edoCneb",
            branch: "features/42",
            lastCommitMessage: "Added 42",
          },
          app: {
            totalLines: 42000,
          },
```

```
packages: {
  count: 100,
  packages: [
    {
      name: "@chakra-ui/react",
      version: "1.6.10",
    },
    {
      name: "react",
      version: "17.0.2",
    },
    {
      name: "react-dom",
      version: "17.0.2",
    },
    {
      name: "next",
      version: "11.0.1",
    },
    {
      name: "chakra-ui",
      version: "1.6.7",
    },
    {
      name: "typescript",
      version: "4.3.5",
    },
    {
      name: "react-icons",
      version: "4.2.0",
    },
    {
      name: "react-markdown",
      version: "6.0.2",
    },
    {
      name: "react-syntax-highlighter",
      version: "15.4.5",
    },
    {
      name: "react-typist",
      version: "2.0.5",
    },
    {
      name: "react-use",
      version: "17.2.4",
    },
    {
      name: "react-use-gesture",
      version: "9.1.3",
    },
    {
      name: "react-use-measure",
      version: "3.0.1",
    },
    {
      name: "react-useportal",
      version: "1.0.0",
    },
    {
```

```
            name: "react-useportal-hook",
            version: "1.0.0",
          },
          {
            name: "react-useportal-hook",
            version: "1.0.0",
          },
        ],
      },
    });
  } else {
    fetch("/_internal/api/app/stats").then((res) => {
      res.json().then((data) => {
        setAppInfo(data);
      });
    });
  }
 }
}, []);

return (
  <>
    <Head>
      <title>Über CodeUp</title>
    </Head>
    <Box bg={"#000"} pb={"4rem"} pt={"4rem"}>
      <Flex
        w={"100%"}
        gap="2rem"
        direction={"column"}
        textAlign={"center"}
        paddingInline={["1rem", "30%"]}
      >
        <CHeading size={"2xl"}>Über CodeUp</CHeading>

        <Heading size={"md"} fontFamily={"monospace"} color={"gray.200"}>
          CodeUp ist eine Platform, die dir mittels <b>interaktiven Kursen</b>
          , einer <b>fortschrittlichen Entwicklungsumgebung</b>, einer 
          <b>Community</b> aus Gleichgesinnten und erfahrenen Entwicklern
          hilft, dich in der Welt der Programmierung zurechtzufinden. CodeUp
          wurde von Ben Siebert für Jugend forscht 2023 entwickelt. Im Mai
          2024 wurde das Projekt wieder aufgenommen und wird vorraussichtlich
          auch an Jugend forscht 2025 teilnehmen.
        </Heading>

        <CHeading size={"xl"}>Statistiken</CHeading>

        {local && (
          <Heading size={"md"} fontFamily={"monospace"} color={"gray.200"}>
            Du siehst diese Seite lokal. Daher sind keine Statistiken
            verfügbar.
          </Heading>
        )}

        {appInfo && (
          <SimpleGrid columns={2} spacing={10}>
            <Heading size={"md"} fontFamily={"monospace"} color={"gray.200"}>
              Commits: {appInfo.git.commits}
            </Heading>
            <Heading size={"md"} fontFamily={"monospace"} color={"gray.200"}>
```

```
              Branch: {appInfo.git.branch}
            </Heading>
            <Heading size={"md"} fontFamily={"monospace"} color={"gray.200"}>
              Commit: {appInfo.git.commit}
            </Heading>
            <Heading size={"md"} fontFamily={"monospace"} color={"gray.200"}>
              Letzte Änderung: {appInfo.git.lastCommitMessage}
            </Heading>
            <Heading size={"md"} fontFamily={"monospace"} color={"gray.200"}>
              Zeilen: {appInfo.app.totalLines}
            </Heading>
            <Heading size={"md"} fontFamily={"monospace"} color={"gray.200"}>
              Verwendete Pakete: {appInfo.packages.count}
            </Heading>
          </SimpleGrid>
        )}
      </Flex>
      <Flex
        mt="4rem"
        w={"100%"}
        gap="2rem"
        direction={"column"}
        textAlign={"center"}
        paddingInline={[0, "30%"]}
      >
        <Heading size={"xl"} fontFamily={"monospace"} color={"gray.200"}>
          Verwendete Technologien
        </Heading>

        <SimpleGrid columns={[2, 4]} spacing="40px">
          <TechListItem
            name={"Next.js"}
            icon={"/_static/images/nextjs.png"}
          />
          <TechListItem
            name={"React"}
            icon={
              "https://upload.wikimedia.org/wikipedia/commons/thumb/a/a7/React-icon.svg/512px-React-
icon.svg.png?20220125121207"
            }
          />
          <TechListItem
            name={"Chakra UI"}
            icon={
              "https://avatars.githubusercontent.com/u/54212428?s=280&v=4"
            }
          />
          <TechListItem
            name={"JWT"}
            icon={"https://jwt.io/img/pic_logo.svg"}
          />
          <TechListItem
            name={"MongoDB"}
            icon={
              "https://colmenaenlanube.es/wp-content/uploads/2021/04/mongodb-compass.png"
            }
          />
          <TechListItem
            name={"Monaco Editor"}
            icon={
```

```
            "https://code.visualstudio.com/assets/images/code-stable.png"
          }
        />
        <TechListItem
          name={"nodemailer"}
          icon={
            "https://repository-images.githubusercontent.com/1272424/d1995000-0ab7-11ea-8ed3-
04a082c36b0d"
          }
        />
        <TechListItem
          name={"TypeScript"}
          icon={

"https://upload.wikimedia.org/wikipedia/commons/thumb/4/4c/Typescript_logo_2020.svg/512px-
Typescript_logo_2020.svg.png"
          }
        />
        <TechListItem
          name={"ESLint"}
          icon={
            "https://upload.wikimedia.org/wikipedia/commons/thumb/e/e3/ESLint_logo.svg/1200px-
ESLint_logo.svg.png"
          }
        />
        <TechListItem
          name={"Prettier"}
          icon={

"
EUaKzQzRlBWs7T7v0fqXl7Gk8YvQkz/xUgTGSUfMDkrPUdUrq/+wUf/x0gAGjMPJzTXpUNZursAJzE
AHjNOoaNYTje3j0D0YWEAIygTFyQ7O0yZdZ3NmM0VHyoiND1FjZAjQUgPJy5/ZIVgUWvRV1gAITM
AFjP0ukYYJS/IVFaffz14ZDpxPEI7d3swX2OVRklSNTyrTE8nS1GqgKy8jb7lsUXhXFxKl5o4b3QuWV8
zOTUqNDVpXDlGRDY7Mjq2jkDKnkKDbzxpO0HfrUW7UlRIRVkpMj9BMjpQSjcyMDheOT+jSk2EQ0e
mhT90X39USmGUeD1uYDpa0JZDAAAKWklEQVR4nO2dCVfaShTHai8aGgEEVuxQFgeWHdZRM
QKiqhVAb//l3nQmlmyzZ1sxb77Oz09PacQ8s9MZrtbLIYgCIIgCIIgCIIgCPLRaXz7JzK+Nf6APqV7kY2
M4jknsbC+5oX1gozA84t4pHS/kd9W1j55ZV0BKywU09EqTNNWXPcscC4R3ISX2WgFxuNFoxELPgR
+WoM2YuM4aoHxbCAKP8EVbkStMJg2BCuMlSIeaOLpS+M9VCJ5DxcvYqRDTbpIp4uC97EU/Br+as
XiRjoq4tlj9tY8t6LEZLFgvqT5+nX+56vgb8BHRH//kTUNgiAIEgy6M+aPaqERor6nUxce+M/2HrdD4rHH
/5LihpQ+5XbU2XFkpXryRD97dTCop5JhkEq2c33mxgWrHalN/ktnZ8WNnTP62VldTYSFqrb7VKBoxSq
xLn0YuepbcJv5/VHtcTM8gQuNOdJRxetV8N4ic9gRKjwzFA7rYQpMqPVtY7gBLL6hjZg5FApcGb0rrM
1CFZhIhKJQ/1kVKjwx2vCtHW4v3bw2FIp7qcSLeCLQt1Ml11Ju1BAlqokhuSlFqFBiMP3x3FlxHE13Vjr
VUzLra9e59nzMC4d6+7VHZ33B9n8Nfogx76ex25MzR55fnjL0s5py8JoLh5vv/Ri7rFHW3ZCb8mN6xg
3+s1ptKyRqYS7bEARBEN80vvnDdMirf4GgO95DI+hD44ZyfFT0xRFvua6U8wAmY0Zjo9B1vFzBbsaX
snKXin7taxvZLr2ncb61KmZvtdW8JxIb5xcb3OWY+3NYg0us2pQjn/p+USLXKwP0/aZJll4Fs4WPtqKjjR
HcisFYudOG1VO/b4IVrk7eG7HRNfeiC8OI6nyaEbWVO/vPu8IpXOBq/sv7PVi60QZR6Ly9iNjKbTx0fQ
p5C98pE4Xme8gGqTAQK3eaDDW7+T2owNbU6KXnpjclfSR2SJEYao43fFu50xf0gVZaUIlleg98N01n
S+R/nF7ENYn5Qrn0aeWOZ49KzOWm+daemNVmZUy/VDi+iDOXO2dur2DbiIT4vwiJZ/wr8R4F8LY+
R5Mb5jvU/0FvtZJwSyyGpZ/IAiCIMtEdIZ2F3cBmBOBR7Zz9ZSY+qDPfgnoVsgdvz+97K8429OtBvb9
Q0cvP5lj/YXJDGRQUutD2opgn0J2/Xjmbk63sQs901Y0bYXhxjXtegA1mKl3j4ZEsfHL+rQzh7IC5/w0JF
pW4OC9hXYAt1zXZzXZJvxE93GZfXmBxMBuc5oBbURtCBaYSLx6UWh000zVg8L9ABT24Zbr+tCDQ
vLCZEYeFD47KoS/iFe5JFCgukmcQSSCXcj7op96eA9/kPs0P1T4BnE+1CQglutE4m6bMUKDBdJHn
XmR1de5pfZZ00OVsXJryjC3KWYwu+Km/HVITBa/Fc+cPu9LMHp+4AzQTBSY5BY/Vtv6LGarJr6QCH
dzusjAjiAIgvxRdIhdOojtqKbVZAjOSWo6KYuZTBVOY8HW+mw2RnMzs3Y9nH2HMzvocRoV+osFqU
Pvcbm5CjE0tPK7VCI0rI71IdTeNusyzoqJ9oBZJyrr7LJNxjAjYZbO0ycH31yQW9F6sm7wqnp35fiLYIky
ZulWxWhEiQhessupvUI3MYxGsue2nAutgRVWwAKp0dbTDvhfDy7U6uCz4zMF+3lLKTTa0IvCz3deF
G75VyjTS6e+eunMQy8lpwreT6JisQnYt4AxS3sYaWJXd0m5VlSTA9pQ5ocqMZiOJ02YWbrMmG35sd
ulBZkb0bZzbVUiFCrRvnlkJkRurJHcAu9OK2Km9/yXlAIE7mXRev0DCd62rzSnX5Q1c0dmRFieiEQEQ
RBEFudTWG3rX2c+82fEgWV9Czz9ia687HcM9g9jzLynvQ3adUcGB0ywWSN2fLERDBeX3P3Z2dOl
NsD6zyprE9oZ0QD8q1zKxQCuqqkBifkMNClaukgFOHpfwlc1TyPe6LVzYnxXG6YES+PkjFymG2QqJu
```

qQG4AHrX5rjuWuGo1YE27o1DtiUQw261uWJMvy78luE8ttKNwSW/jrhkIl2JxoQXqy//1t+Pe/h/+DsdTH
fDj8GPNhzPuaht+hLu+aBkEQBAmEBgTTd3SIg5P5jDUKp247SpfHYi7P+S+d3h6K+clle9PHbofr013+
B+Bn6SIK3QtIdB4fg/fjpApxSu+MDmm2N/0+33Kxp7fylZDytVnCjB05ol86A/pP7nQOyXfGTffYy71WhQ
oUGn7gEiWSzmeNLM7WLZezRLKS1yei4NK9POmoAeZrk4p0PnrfmGaeoQJXVjqGu/0XsSm2RZIQC
AXKxHLDBcaNAOQMtJMuIAoBlthQFJrjqF3YODYUAhL1GewbvfRLWdhLmxK9FOyLYY6jdqNIE7eN
wApfyA+NhVHezEgjVCgxmJaOsnHxZDGfLYolMuvrD2cd0GzB5gbVp03X2aI5YaO7Bfna5LbAl90jMd1
LhVnW6Mrhy4mYl1P2d/TdipvjHG9GV9wc52St3KBFm3nZBvNFNy3FXLwgw122IQiCIL5hh3B+yNYg
EVEy/JEcnroyyTcN8hX2v7T+DSCqTYbB6yMnMYqqZPouv6vJM0un7/XAc+om6sMtcn0fxYIk3EvHps
ROexPSgm8i65oX1BQ1WEVSDcmanItsYWrhZGBXc86ezRKAzWs2ft6GQoCV25PCO2fvdBngCs1t2
CJt6MH1HKLQxTs9DlWWBGt7xNlZm4XxHiZSbyQzQzRVyUwJ1vaadJvWG0i6ngNQUzkPEeFWpKq
S3TMJ1lpl9uS5N7tTAWlPJFA3h6zzdlRVyZytB5J+2ei8jSAIgnxgYAlHrQDyjIaFlD5g0lgbQ0OZPYf3tci
UX9PIbPLhiX/NsGtYH0XEPSFhmtkVWNddJZZJd4m0BRfA9/iW/aEMLXKoE7XAkGK5rXwEhTJJ1C0
0icKIX0OZDaICzxphhZ6u+juR8AB8pNF3F55KnvS12OjuiIcaqapkCqyoiIXylO8MgGwugSHpyQ4rDCM
sFYMgCIL81TRsypKZPKDsSld+mKKT9lXJumxpMC3W/35jLRy6zV0n0hlfZpPvUJWMLQ2m9V7bNpn
W6u0ZTTYGTdIeEDK1ZJ2rktFWHCZsbVBq4oBGWEapb0EQVcmMSmPatZO5Wx2QRlze3ZOLn/fG
e6UxbdvR3E1iuZd4B+ziq5+lCp3asP0BFMYcq5LR0mC9nIPC5A3ppct7EuUYM8OWBuvbFohQk0wO
88hPEyXmi8KxTVUyvjSY0s+1kyk+m1oq2c6xNT2W+ER43lHPrZT4S/Qety1c8wlUl/dUf4HVa90cbog
WeQRBkP87wHoaVv70jUN5eO5I1AqjVF+emKtEvKaRmfNvqx5q+PyCzUe0xOvSHyOvAhc5pchllndvl
RF4bkOV9NOoBcL3hzb52iTokIpTy6tQv636ULhP2nB5z2liT9JVCSk7NFA78rM2iXxtD16q9v3mmZku
orZyy+VrG1W9cHbLXcdjkFb4kV0x72satHIjCIIgCIIgCILI8x/INPTVzXECQQAAAABJRU5ErkJggg=
="
          }
        />
        <TechListItem
          name={"react-icons"}
          icon={

"
VEX////pHmPoAFjoAFnoAFvoAFbpGmHpE1/nAFPnAFLoDV3/+/3+8/f+9/r//P372eP97vP3u8z60t7tVo
T1q8D84envapH96e/4w9LqJ2nvb5T5y9juY4zuXor2scT+7/TsRXnyiafrM2/wd5rzmbL0obnykKzrPnXxg
qL0nLXqLmzxfqD2rsLsTX773OXsRHk23n+eAAAWYklEQVR4nO1d6XaqOhQuISWIWkVEcazz1KLv/3
ZXrNkJIUAStb1nrX5/zll1yibZ85C3tz/84Q9/+MMf/vC9+/+MMf/vCHPzyO1d6XaqOhQuISSIWkVEcazz1KLv/3
ZXrNkJIUAStb1nrX5/zll1yibZ85C3tz/84Q9/+MMf/vCHPzyO1d6XaqOhQuISSIWkVEcazz1KLv/3
ZXrNkJIUAStb1nrX5/zll1yibZ85C3tz/84Q9/+MMf/vCHPzyO4Xw3W+wXs8O889tLeQW6x73vBpQQQg
Ps7Xcfv72gJ6M3bWDicBDsLd9/e1FPRCdyqSMDr3q/va5nobX2ivRl+4i2v72052DuByr6Mvi7317cE9C
aoTL6rkD/Pok9p3QDv0ns//YKH8SxIQpQh1AXeR4KhD82Jr+9xocwzp1QikbrsNcdJrsR5kSPWr+9Sns
0IyzS510mzfsr7zsftjEY/OoiH0FrL7AgQVHuOPYokOglv7XCB9E9CUqQUlmixA4jkTi/sr6H8SESiKJh4
Q09j72KN7+wvofREggkSElCn5FIgn9Q2LRXnEBKSuzPKPiHNzHiQiYYdUveNGTeBnHaP7q6J2DK1
QROy52kHXub+6+Z4AeXEzir8ALbTJ6S0c8t7hmY+5zAaeU7N2wTvX/KduNqoI7Atyaz6n7VsGk2698j
4mPEDbKo7s2H4DcVxsdks56m+9Vqf0kHh20S630sBT0RrGrffGYci8IHV2uMeLNqIEyz0FgWHaMBvr
o9l11YtE0k7MCdoCeNfYnuz4PWbvdzkUQ4FxkD4wPh/TKpUl4TxN+ss+nz+yYS8pPH9GPqKyNHjMp
gEJbxZpszYUPLZWgxaYp+0MOYKyJ/EpGuP1aL9xnYMuio92uD+48Fn08koRobX3E+C6DotCvy5Bw
Uhbb4n983kdDnklGOrVdGlLyTGA2kjeyCW0tGupZmt3H/iK8pqx/FvEAgKd9S6p1yQn4Gx9s/a//i6v797
vzpxKgw9AR6CMXIDUj2D8KBklCCTlswPOcgR00s6d2dden6FQQVMBPcOkTH20487HaHw3jytbx4H
qZFMok7upPTBAVTb8sISO7PhSxeQZCMCdjMBEUFK6MzX6+uVBZpxP1Me6yZHCW0zCNUYQh+8
E84iWBxEVwSi463aVAwBoh7mr/14Ix6ZhzFGNH7gVQUWImEVgi2Yf/iy1xJvOhidUavmN4fK/56bPU6
WLIDU+euxbsRkk4rVxSByRm9YnvXiD+h85nLTce1b22GaYnpY5xrYaKGpnarNkDM5IyvVUjQGTQUe
SXzdQ7vP0tO5ks2RMjs/Hq/7hvDAy7QiGr9Kxntu01BiPFHTcHYMNBPWw7XMo3YPGy2uDMHerndx
oSakSMTj6WzihamxTJj9rsvD0exZ4nNnmU8wDkSScMwdc2CNWU6+Glo3r1X4pgW82zkk7o32kamL
vCrk/rtu0ojJ8Po2vtINnKop+n/3pDcJRx9tUJsM19UV5QybF2nAHTRF4zM3qMzwx82RRvZUdgubOFtu
Vg7Pni++6Rkb7pkQ9juYR+2MGfJEU/X4esylf/q7EWbxZ8N+RBcDdxZ52I82NEUOCyQ4Vms2gTNu1l
qWP/BJOHNck5GouYgrp78Z35Xw2rd+nhnjho1sZ64IG1kSqb9KdYIEVRvw1/B/BL/1T4w83+RfhhJ4M
Lg8P2HyUjUjnilYT6wh+Qb+l3GGN+XZmS1sY0nmO18eyCGsyitl6l75uW/2jAF68nAeA4hAyhIzi9RqBJ
/WfkNrThm4QE3jOOPV1YO9+9SwsS2YI8/7zUNU7GgzU0l0dUexr1wu/ucXUa44Xt8y13P8/0GHl3Gu
83XJH56sobljQw0b485zXK0c+eKJ3X0ffyarU7YP8z2I+oiFwdZWbujwq3YHSPs7GfL+dnQhqzCEHx87
Y8wh8tpyCw0IeJJDcJe/3M28rwrYWV0lVDqkegQPquwHzJdunG9mOUAFAnOdiqYq8RFGWmalEl00
sD1V5veM1QJxPV0RQ2TTWrndVOTpTMiEyNnXZmb1cKRiZqaEgoGXg+j4Nzedv08Am8/QhE5PKhP
mBtDqN7DglSMVGfQ6h1T3ytGqR4nMvCjh0IdH2xPNCMmTFWQEVdizd4xclAxu1GxbCL8t1YKUbR4
JB/OKiOCaiV9R4fJXgg/nLdTosxQFYiiQYCzqnbfx84J/ry47B33qiEzXULLvob6qX1jGPMT9IKzY1AVm
TBvTg7Ur6eOBC7ynVU0PmzDSefbTDgxu/RbxbfjXrJdjqORd0tfqmhc28ockP46+qLF0jN08NZOxiOkT
qJKOB3nk247b5ox5vDzaq/Z6s2XaXbkC1+MR7bsyDhL55iCV0E/pwEq37zriRQrvhXfXOkftjrbMfHljB6
x7bZhKWeC69+7gB8t1eVZhtyj0WEgmKmoWKVR7+PHxxRJRCq+RwcdOKa1AuucjwMXQTGi+/G8k

9mVO4FELC+txeI0lUX7w+0qn9HDkZXFyliivh5mV0FhtnX+7Djh/ka/wV/FUtSQcX9tCGwya4g0yt+jB6h
pdWtCGb0yzXU7mLOjLKoSwS2Wnj4rSNVIzMVjcR+RTQg5ZhRWhtiHmxVSV58EiKRHpZE8oWUkM
q9Ny1g8iwa9VbsNC9aQUekpDyOZ6+8rxP5omZQ6OmfBnwouwgssqs8iPTXo84drVeQfltiaDMMNlVP
432fTxdN+tV0sdpkEe742yMxotl0MT2DxYi3jKw+IDir5fjJQFp5erf7pvD7m0L5wY5xyEpem2bU2b+fwL
DZxV14/3195RX/h6tSgdaIXUmnvRRLZX8fGmdn3PTsNFinntw8m1yW//WOHXcX2Ibo2YXdhF4PLnd
Mj8zhti50kqwIOMKjFIqX4s1gKdXuGodkxEbsug+hmnr4zUxEbxNohl4AsAnJQGcU3sTPw1Ga1eQG6
SCK+fZpFCoxyz6yAwy75DwG0+6d7UWlHvU2l1kwgMbNKWsykMaqnYSfNKvkPxukt/NKbFbQDN2d8
m6Ct2Pc14CYNMWIpFr7GJvl0QAScmJxnhbp9ikaMcyy7JISD6o6hUEmj1kwAa2PQC0fIgPCEE8j7dz
XLZgkkizTzgwUIJKLl1iJdwivgLbszo5JAEsF0GQu9WybST8S72IHJYhiuUWMQcxEsazZ7ylkWV8tle+
M7ZoTYFy43F5xExtRmFbRsE8yeC8e4uInUW7DaX5YztDEoGPaFX/BMpFaLPRjP8hgl8iZSIILiAe/+k
YxmSyaxwplRgDkIZp/iWEr+O/Uj4QhBzPGh+pePUZ5EM/sLet40ExB5JE6eQOrNcixyMQjIVeAjH78y6
rc4M+PZpg+lPZDcdxLlQ8xtpkuKBlMz2c3SxWwrnt54Gy3S2S4pnqZOjkQjpuZ5S3OTIymMzJE1MfBoI
fi3cVAWjKfYixiNcZRF+SkNkFNUXBOxBcnEwAR1ZX5I38eKzjU/7x59slJiKRzXPXHvit5rTHfcIiLuqVB
MEvKub8czKDVhwlw/l8vQO6lChFIjWkkrVmuVkxxu2n5rpbmqRboqHKktl9kGqhucdGPvcJPjQALHNR
DPaVxi0Ayk001XvZOkEBRB2E94BsTVzSiBmHF8M9epFYlKkKALxE4dTzAc5mpdUTSDSDHgqDhU
3EAkRM8/5NWegRkX9nLVWjTYCp1sBPPzxcwdSVesdTK+iraYQUmIsRxT+CWzpsww13nopdkRhO
EjosxibCjFjbQKLYqtsO8n4XNYJ4a9g3NtFkpcClLNofQuRPq8H41pKwiKuzmx0RU/Xw5fPojD3Of8em
9sDkxoZlJNRaGHIhDcQrveXQ58scaavBg767UPe3I4LczXiOM6acP7/Y00RTMVlATxBeOCn1NWorx
mbJgPwfc0KZRXtcuzL3GqC/c6PODnGbi+3ZXwM/iUY9+Q959/qwyqZsNYk0JZNqSyQqkUjx27MW9d
R0xYjaX45xioRxlz8v42SXvrUShXzIGbxxdQYZ4Khizd63tNXcGLIY0Cq7/z1smsWwjmWMihv1lJ4COH
QuSqw9iKE1rOXj0esyUGnUviDiqncnFVnjEJGKWypC74zCoU8lngavLHTGjJ7iRCBbmvn0kQx44Fe2
VEgCugK5OkajZ8E6zhchTTWQOIs3GnO1BHFeeCiYQMvMKUC5nSqVxcGrhLeHPhYST1GrH45Nm
5u7pOC/gVTxVcWgoWiYbaBAiNdeWNA23O4NByr0guruuKM3DBiwe51WjydJJQFA9oTgUm8A185Y
2gPytSzJNC65bS81xVCxtafCpgxWcvcU4uOEWxqM9MaoUSXgJSXdawkcWIMuQwDKpIpE7RxV3nr
PgDHAJJZYRiqWPDYAeHXPpW7WCGqeQ5qEMOw1E5iVQxXPBdsuJ5gFFsCW6K3VTEN4kAcwm
C65IiTakDD6sDQDnjoXYHeanQvQ+JG0bCOT2vRJMSmySc+dEL6oMB5/zM+DJPrlvCi3SkUkTAhuzr
eFADzulO9KMpNonLxMCEdKVhASWiWVYe2xRrEYT375W5cJgRBbKDz5lq3B7JeSUKuWBl1BgF4Q
O9sWM5aVOReFWRGKzUyf5TwXyAOuybgdde5moj0MAogs/3RJd3hdRtVWzzPZVJDFI1gWf2zASu5
vFFL5zkmxkbhplCiB6UGEkK8IxYdRNmlFf9uCxRDM0P4hu41Ma5SBalhiUJcxhFoz2V6+qFcOflUnVg
ZiKJpQSCLM+lcT/UNY9uappFgy1UGoElCIVamMraTrncQok2lAbnBGRx2lh9h58CE6vAv1g0W61CB+
yIuOXfD1aaVHE9LSgcm7s+mLlkFnIcizKk2s5bf5PoVqTMmKspZ34mkolIXL2qzBzeodjZqJB4n2MRV
Hly1tk6UVVOUJ3/GEouCkFWl7XwA2LSfdKW1IBfKb6vDp1X9QzAcwoEGRKPG/kzSu0qgqBUw6yYY
ijLAK8yXbQjlW7OVjbZMvrkbjA6tSxEYPaM2QOSGaRuNm61BZLKJltnXCw+1iyMLgKGwJl03oOhLKB
hPf4XRu3ddUVyUY6FNVshoAVVW0bFoRCfFrIw1hOOISyZdfa1+ydV8bhjW2txdePYBAwjU5YFxoIdD
28S2+m4LL5KP9/iA5Grj3l2VjtvmgOz4A2nmCzgbMeCyag3L1hGE2qtP1PFjMKUJ7Ktyh8hi2o0bhL8fN
R6m3CHmLg2JPKcUyHxSNAl5EU7muPxZEB8xESZDqHXpJ1r8rE6qOuycAfxVrevC+0LSjJYDWCOc
/MNEx4eIuYHtVnSNUU96PIFA8pKnIK2DQwUKrQnfee1QvcBidpXDM+6frMb8WcFdldgM36Xz0IwsEu
/pCmcQrkPMSJxchgpCCTYH+dYDjqwTMeF3gBJgmrzOQeoymXmqFjR1NDMlbSSNVI1DF/1hXyrJX
h4Ns1Nb188bKcdI2eVwbwq90sw43TGlrbCqaPU7AHa94tinW+DTQ0p19mVHo4IGBzJAybCbSS1Y+
Y7xwi7qpoUgr21UqZDG51VwbyQ0MR7PS+YaShP4JZQCLj75VZyN1xTxRTpjLyr9OyXbRG00S30Ce
MYczOa6g0sZb+Xq6wOhVi4+jQ0e5uUljSzE+SsKxQ6NDd5Nlq/LV6q5a40BAX7ufzMsVx/b0Eyd7azij
ESZB9WFsC2mNa0Uhj5ImSCRvM6Cw5G/Of/PBFs1ODCv6TVO87UY9z5ptdZCnANjd39Hvki5CuNiu
scBHywrnkivTARhP93ZqHZTXZp/YCT+i5e0NuW1wrEJLcEglFU0epa3jXfE3iROsl2vPJQ+cgkCo6RR
u060/q2gz9bF8m2oLiRbkska6e8xWwoZBeJW3KNwu1FivDgE/rG6uP0EJpv2A4zWRaqXCnGq7FqhF
jFHNX3pFDapCIvQKPBVbQws19nimiTKVz72y8mTrH5NSu2d2af/UluN5nhLWZy4l54/Bzsg9pqIRLgx
mrXy55bC+b26OSSmJlhGc24Yaec50RuNzw16GK8O84nvTgesjgUnU6+trvDNJuX9z0MqGb/sgdGBl/
MfGYZp++5PXWA7p/ggTlf3cLUcWF1GaUuQghzwXjlpYp5efIX4NsYCeHQQ3emlvBoGY6uKkE8bigbt
R9DdnuLv99M8ls10Z8Pc8OskAm3Q2s38ixH4pUSuJpuekWBxYLRun3NX8XQuC2SaVB3lZUR1KcQ
Ovt0x5MPoSLlCeNbu/2ZrzfOSo2MZ4VPBydFgOUTQt268ZPVUxgR0EwOC081XKuOtivPeU702c/d4
1ysx4W8r34R81qVC38M3WQTjXB2w5NGxWhwVSqIntJ1P4wzpmuLRYTFmmpQFfpNv3NVPcPjaA4
n288L/VZ4V9VABeWQ9doFOJuY18D7wW4bnnMHbil2v0l18VA+YNArAdbiSy63fj9Pwu1uPR7M0hRG
Iu8H48/d8SuZlBiwolPsEF8MbpSNkqwEO9ivnvYNhRj1iumcq99zUy4E+ShJg9gS2/eXT4pmmQYN1Zv
vggtg3jWEloykxqAYAXsNWOxKq785d4ki8abfbidElkqKN9VgacuXX2ECQX2tmEmS864Dkm0jn/1tlLve
arPHgzC8Z6Ob864JmrV4rkXLqwAAe7z6ChOQ2roG4i5feYehUMZwqXB4jKa7WEB7uiEgf4cACFijhN
cbPzz6RZSWAN880P5Ie+YpzCLT4OePUfgG3R8Gn/lSNCiYDrVJjETcI7Cal9KN5FiXcRHQ/KckDc/M
mFXS9ZHUJhoZVuL9mLawvh7tQ9pG4kdGmXkWX7WdJqQPsC2MJzOFUj8KdacGyaTRc13gCjDfzm
IsTVuuiKP+THe90Af/8ltouEwzD84eizV/1FuV3tCeA6uOevl9V1eVz2rt9qZXwQyVscrabNcNUIn1Axex
Qj+L8dXEZV3Q2Xi7OvEB3bPWVZAGgKpUw5wzv6+bFlKK2XjXScUT68MMR8uhXkaY2YntD34x2+a
cFke6UOQMSvKWzQM8HKuiGlOwkiFDq4S3kK2uDJysvOLN3oGLFodEjlK0N7zxieCXS9I34RZ0o6Ei
vOetca93joo03qj0vPRwDHvxFedeuNx7goBynxctrUBXavnUQkfRZ9+bleQPbokuF2P3drOF+ILpvZp2e

```
LcJsIM5Q6jAax2zbJfJkIiHMDbPdPHhSlILWeu40s52EY2M/3OQa5/XAm/sVfivyUwv20Veb5Ey8NEtm
ozIe1AIVRlC3X7aqL1WCI/sei6sAFaNXtf/kEdNSyvKu9sIV6T0rmbPy91CEdBQp5UHavNxJ5XLbCeH
PVVet3A1Bj5ffuF6DlCzqXVHPB8WHNQ+keFks/D973uebrjqDQ9PDSe+Pw4w9HUSD7z3mzh6NuW
wl2wP68EsjQbjwzaxv7fqAaz16wy4SUnwD4qKR8FrReqsb06g41tXbP0GIE9aoxLHwuSbn/AKngeeiK8
s+FkLZeSvvjH9yYBiiioH4128xMLmio1fBRia5aPjukJGhi5+wDV/LjowboKU+KSJOAHO7Bbz/wdgpGqJ
YSOOHaNGl5j/XxBXNhR19kJoVD1Z6P8P7vIVep2aB9FTCIqzn/8NtHnjIcobb1+5kTm4ZPDOP4BQ0
OYwfP2t9UXzM48eqMz+dQiju4NgEJ5bw0l/lu84J/+YJSOhLfZXBK7nFwo2Kf6BGPwrEavuJxPgXv5R
GcMxqYoF/uMn9I5EfYdlRp97+hXX9emIibJJg2DywPU6/y98TIvbSDA9/LNKUIEwf18nCRA+/kSG6Cc
RzpCb9QdltfreVTP+9npegeH8kC72i2h6rMrk/uEPf/jDH/7whz/8QRv/AYrJPa2NE9G+AAAAAElFTkSu
QmCC"
          }
        />
        <TechListItem
          name={"GitHub"}
          icon={"https://avatars.githubusercontent.com/u/9919?s=200&v=4"}
        />
        <TechListItem
          name={"Webstorm"}
          icon={
            "https://upload.wikimedia.org/wikipedia/commons/thumb/c/c0/WebStorm_Icon.svg/1200px-
WebStorm_Icon.svg.png"
          }
        />
        <TechListItem
          name={"GitKraken"}
          icon={
            "https://yt3.ggpht.com/ytc/AMLnZu_1xh8LDkCMpLKwHAvxDhIROsQp-
_KBRR1pVrdc=s900-c-k-c0x00ffffff-no-rj"
          }
        />
        <TechListItem
          name={"Undraw"}
          icon={"https://undraw.co/apple-touch-icon.png"}
        />
        <TechListItem
          name={"Git"}
          icon={
            "https://git-scm.com/images/logos/downloads/Git-Icon-1788C.png"
          }
        />
        <TechListItem
          name={"Turborepo"}
          icon={
            "https://seeklogo.com/images/T/turborepo-logo-D9CF2C830E-seeklogo.com.png"
          }
        />
        <TechListItem
          name={"Yarn"}
          icon={
            "https://avatars.githubusercontent.com/u/22247014?s=200&v=4"
          }
        />
        <TechListItem
          name={"Docker"}
          icon={
            "https://www.docker.com/wp-content/uploads/2022/03/vertical-logo-monochromatic.png"
          }
        />

        <TechListItem
```

```
    name={"InCode"}
    icon={"/_static/vectors/tech/incode.svg"}
  />
  <TechListItem
    name={"DiceBear"}
    icon={"https://avatars.githubusercontent.com/u/7983162?s=200&v=4"}
  />
  <TechListItem
    name={"Copilot"}
    icon={
      "https://github.githubassets.com/images/modules/site/copilot/copilot.webp"
    }
  />
  <TechListItem
    name={"GrapesJS"}
    icon={"https://grapesjs.com/img/grapesjs-logo-cl.png"}
  />
</SimpleGrid>

<Box mt={4}></Box>
<CHeading>Alle Pakete</CHeading>

{appInfo && (
  <Heading size={"md"} fontFamily={"monospace"} color={"gray.200"}>
    Insgesamt verwenden wir {appInfo.packages.count} Pakete
  </Heading>
)}

{local && (
  <Heading size={"md"} fontFamily={"monospace"} color={"gray.200"}>
    Du siehst diese Seite lokal. Daher werden die Pakete nicht
    angezeigt.
  </Heading>
)}

<SimpleGrid columns={[1, 1, 2, 3]} spacing={10} mt={10}>
  {appInfo ? (
    <>
      {appInfo.packages.packages
        .sort((a, b) => a.name.localeCompare(b.name))
        .slice(0, showingPackages)
        .map((p) => {
          return (
            <>
              <PackageListItem name={p.name} version={p.version} />
            </>
          );
        })}
    </>
  ) : null}
</SimpleGrid>
{appInfo && appInfo.packages.packages.length > showingPackages ? (
  <>
    <Link
      color={"blue.400"}
      onClick={() => {
        setShowingPackages(showingPackages + 12);
      }}
    >
      Weitere Pakete anzeigen...
```

```
            </Link>
          </>
        ) : null}
      </Flex>
    </Box>
  </>
 );
}

function TechListItem(props: { name: string; icon: string }) {
 return (
  <Flex
   gap="1rem"
   w={"100%"}
   direction={"column"}
   alignItems="center"
   justifyContent={"center"}
   textAlign="center"
  >
   {/* eslint-disable-next-line @next/next/no-img-element */}
   <img src={props.icon} width={"64"} height={"64"} alt={""} />
   <Heading size={"md"} fontFamily={"monospace"} color={"gray.200"}>
    {props.name}
   </Heading>
  </Flex>
 );
}

function PackageListItem(props: { name: string; version: string }) {
 return (
  <>
   <Box p={5} shadow={"md"} rounded={"md"} bg={"black"} color={"gray.200"}>
    <Heading size={"md"} fontFamily={"monospace"} color={"brand.500"}>
     {props.name}
    </Heading>
    <Text mt={2} fontSize={"sm"}>
     Version: {props.version}
    </Text>
    <Link
     href={"https://www.npmjs.com/package/" + props.name}
     isExternal
     color={"brand.500"}
     display={props.name.startsWith("@codeup/") ? "none" : "block"}
    >
     npmjs.com
    </Link>
   </Box>
  </>
 );
}
```

## apps/web/src/pages/codeup/contact.tsx

```
/**
 * apps/web/src/pages/codeup/contact.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 04.10.22
 *
```

```
 */

import * as React from "react";

import {
  Button,
  Flex,
  FormControl,
  FormLabel,
  Heading,
  Input,
  Link,
  Select,
  Stack,
  Text,
  Textarea,
  useToast,
} from "@chakra-ui/react";
import { useLoggedIn } from "@codeup/hooks";
import makeRequest from "@codeup/rest/src/makeRequest";
import DividerWithText from "@codeup/components/dist/global/DividerWithText";
import { useRouter } from "next/router";
import Head from "next/head";
import { CHeading } from "@codeup/components/dist/global/CHeading";

export default function Contact() {
  const [disableButton, setDisableButton] = React.useState(false);

  const { loggedIn } = useLoggedIn();

  const router = useRouter();

  const toast = useToast();

  return (
    <>
      <Head>
        <title>Kontakt - CodeUp</title>
      </Head>
      <Flex
        align="center"
        justify="center"
        w={"100%"}
        h={"fit-content"}
        minH={"100vh"}
        bg={"#000"}
      >
        <Stack
          spacing={8}
          w="100%"
          maxW="800px"
          p={6}
          bgColor={["transparent", "black"]}
          rounded="xl"
          shadow={["none", "xl"]}
        >
          <CHeading size={"2xl"}>Kontakt</CHeading>

          <form
            onSubmit={async (e) => {
```

```
            e.preventDefault();

            if (!loggedIn) {
              window.dispatchEvent(new CustomEvent("codeup::open_login"));
              return;
            }

            const data = new FormData(e.target as HTMLFormElement);

            const category = data.get("category") as string;
            const title = data.get("title") as string;
            const message = data.get("message") as string;

            const r = await makeRequest({
              token: localStorage.getItem("token") as string,
              method: "POST",
              path: "/api/contact",
              body: {
                message,
                category,
                title,
              },
            });

            if (r.status === 200) {
              toast({
                title: "Nachricht gesendet",
                description: "Wir werden uns schnellstmöglich bei dir melden",
                status: "success",
                duration: 5000,
                isClosable: true,
                position: "top",
              });
              setDisableButton(true);
            } else {
              toast({
                title: "Fehler",
                description:
                  "Es ist ein Fehler aufgetreten, versuche es später eneut oder wende dich per E-Mail
unter hello@ben-siebert.de an uns",
                status: "error",
                duration: 5000,
                isClosable: true,
                position: "top",
              });
            }
          }}
        >
          <Stack spacing={4}>
            <FormControl id="category" isRequired>
              <FormLabel>Kategorie</FormLabel>
              <Select name={"category"}>
                <option value="sonstiges">Sonstiges</option>
                <option value="fehler">Fehler melden</option>
                <option value="feature">Funktion beantragen</option>
                <option
                  value="creator"
                  selected={router.query.creator !== undefined}
                >
                  Creator Bewerbung
```

```
              </option>
            </Select>
          </FormControl>
          <FormControl id="title" isRequired>
            <FormLabel>Titel</FormLabel>
            <Input name={"title"} placeholder={"Titel"} />
          </FormControl>
          <FormControl id="message" isRequired>
            <FormLabel>Nachricht</FormLabel>
            <Textarea name={"message"} placeholder={"Nachricht ..."} />
          </FormControl>
          <Button type="submit" variant={"brand"} disabled={disableButton}>
            {loggedIn ? "Absenden" : "Anmelden zum Senden"}
          </Button>
        </Stack>
      </form>
      <DividerWithText text={"ODER"} />
      <Text textAlign={"center"}>
        Schreibe uns eine E-Mail an 
        <Link color={"brand.500"} href="mailto:hello@ben-siebert.de">
          hello@ben-siebert.de
        </Link>
      </Text>
    </Stack>
  </Flex>
 </>
 );
}
```

## apps/web/src/pages/community/index.tsx

```
/**
 * src/pages/community/index.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 12.06.2022
 *
 */

import * as React from "react";
import { useEffect, useState } from "react";
import { useHasProfile } from "@codeup/hooks";
import { default as redirect } from "@codeup/util/src/userworkflow/redirect";
import { AiFillPlusSquare } from "react-icons/ai";
import { Box, Flex, Heading, Text } from "@chakra-ui/react";
import { useRouter } from "next/router";
import Link from "next/link";
import { PostCard } from "@codeup/components/community/post";
import REST from "@codeup/rest";
import Head from "next/head";

export default function CommunityHome() {
  useEffect(() => {
    redirect({
      allowAnonymous: false,
      redirectTo: "/community",
      redirectToLogin: true,
      forceAnonymous: false,
    });
```

```
  }, []);

  const hasProfile = useHasProfile();

  return (
    <>
      <Head>
        <title>(Veraltet) Community - CodeUp</title>
      </Head>
      {hasProfile.tried && hasProfile.hasProfile ? (
        <>
          <CommunityFeed />
        </>
      ) : (
        <>
          <ProfileCreatorHero />
        </>
      )}
    </>
  );
}

function ProfileCreatorHero() {
  return (
    <Link href={"/community/profile/create"}>
      <Flex
        alignItems={"center"}
        justifyContent={"center"}
        minW={"100%"}
        minH={"99vh"}
        cursor={"pointer"}
      >
        <Heading
          color={"green.300"}
          fontSize={"10rem"}
          textAlign={"center"}
          display={"flex"}
          justifyContent={"center"}
          alignItems={"center"}
          flexDirection={"column"}
          as={"div"}
        >
          <AiFillPlusSquare />
          <Heading color={"white"} mt={"1.2rem"}>
            Erstelle dein Profile
          </Heading>
          <Text
            color={"white"}
            fontSize={"1.2rem"}
            textDecoration={"none"}
            fontWeight={"normal"}
            mt={"1.2rem"}
          >
            Erstelle dein Profile um deine Fähigkeiten zu teilen und Rückmeldung
            von anderen zu erhalten!
          </Text>
        </Heading>
      </Flex>
    </Link>
  );
```

```
}

function CommunityFeed() {
  const [postIds, setPostIds] = useState<string[]>([]);
  const [currentRange, setCurrentRange] = useState<{
    from: number;
    to: number;
  }>({
    from: 0,
    to: 5,
  });

  useEffect(() => {
    let origPosts = postIds;

    REST.Community.getFeed({
      from: currentRange.from,
      to: currentRange.to,
      idsOnly: true,
      noReplies: true,
    }).then((res) => {
      setPostIds(res.payload.posts);
    });
    // eslint-disable-next-line react-hooks/exhaustive-deps
  }, [currentRange]);

  useEffect(() => {
    function handleScroll() {
      if (window.innerHeight + window.scrollY >= document.body.offsetHeight) {
        setCurrentRange({
          from: currentRange.to,
          to: currentRange.to + 5,
        });
      }
    }

    window.addEventListener("scroll", handleScroll);
    return () => window.removeEventListener("scroll", handleScroll);
  }, [currentRange]);

  const router = useRouter();

  return (
    <Box p={["1.2rem", "2rem"]}>
      {postIds.map((postId) => {
        return (
          <>
            <Box
              cursor="pointer"
              onClick={() => {
                router.push("/community/post/" + postId);
              }}
            >
              <PostCard id={postId} />
            </Box>
          </>
        );
      })}
    </Box>
  );
```

```
}
```

## apps/web/src/pages/community/post/[id].tsx

```
/**
 * src/pages/community/post/[id].tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 19.06.2022
 *
 */

import * as React from "react";
import { useEffect, useState } from "react";
import { useRouter } from "next/router";
import { Box, Flex, Heading, useToast } from "@chakra-ui/react";
import { useHasProfile } from "@codeup/hooks";
import { PostCard } from "@codeup/components/community/post";
import REST from "@codeup/rest";
import Head from "next/head";

export default function PostView() {
  const router = useRouter();

  const { id } = router.query;

  const [replies, setReplies] = useState<string[]>([]);

  const toast = useToast();

  const hasProfile = useHasProfile();

  useEffect(() => {
    if (hasProfile.tried && !hasProfile.hasProfile) {
      router.push("/community");
    }
  }, [hasProfile, router]);

  useEffect(() => {
    if (id) {
      if (typeof id === "string") {
        REST.Community.getPost(id).then((res) => {
          if (res.payload.success) {
            if (res.payload.post.parent !== null) {
              router.push("/community/post/" + res.payload.post.parent);
              return;
            }
            setReplies(res.payload.post.comments.reverse());
          } else {
            toast({
              title: "Fehler",
              description: res.payload.error,
              status: "error",
              duration: 9000,
              position: "top",
            });
          }
        });
      }
```

```
    }
  }, [router, id, toast]);

  return (
    <>
      <Head>
        <title>(Veraltet) Community Beitrag - CodeUp</title>
      </Head>
      <Flex
        justifyContent={"center"}
        alignItems={"center"}
        direction={"column"}
        width={"100%"}
        paddingInline={".4rem"}
      >
        <Box
          maxWidth={["100%", "40%"]}
          minHeight={[312, 624]}
          minWidth={["100%", "40%"]}
          mt={"2rem"}
        >
          <PostCard id={id as string} />
          <Heading size={"lg"} mt={"2rem"}>
            Kommentare
          </Heading>
          {replies.map((reply) => {
            return (
              // eslint-disable-next-line react/jsx-key
              <Box mt={"2rem"}>
                <PostCard id={reply} />
              </Box>
            );
          })}
        </Box>
      </Flex>
    </>
  );
}
```

## apps/web/src/pages/community/profile/create.tsx

```
/**
 * src/pages/community/profile/create.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 13.06.2022
 *
 */

import * as React from "react";
import { useEffect, useState } from "react";
import {
  Box,
  Button,
  Flex,
  FormControl,
  FormErrorMessage,
  FormLabel,
  Heading,
```

```
  Input,
  InputGroup,
  Textarea,
  useToast,
} from "@chakra-ui/react";
import { default as redirect } from "@codeup/util/src/userworkflow/redirect";
import { useHasProfile } from "@codeup/hooks";
import REST from "@codeup/rest";
import Head from "next/head";

export default function Create() {
  const hasProfile = useHasProfile();

  useEffect(() => {
    if (hasProfile.tried && hasProfile.hasProfile) {
      redirect({
        allowAnonymous: false,
        redirectTo: "/community",
        redirectToLogin: false,
        forceAnonymous: false,
      });
    }
  }, [hasProfile]);

  const [displayNameInvalid, setDisplayNameInvalid] = useState(false);
  const [bioInvalid, setBioInvalid] = useState(false);

  const toast = useToast({
    position: "top",
  });

  return (
    <>
      <Head>
        <title>(Veraltet) Community Profil Erstellen - CodeUp</title>
      </Head>
      <Flex justifyContent={"center"} minW={"100%"} minH={"99vh"}>
        <Box
          mt={"2rem"}
          background={"gray.900"}
          p={"2rem"}
          rounded={"xl"}
          h={"fit-content"}
        >
          <Heading
            color={"green.300"}
            fontSize={"10rem"}
            textAlign={"center"}
            display={"flex"}
            flexDirection={"column"}
            as={"div"}
          >
            <Heading color={"white"}>Profil Erstellen</Heading>
          </Heading>
          <form
            onSubmit={(e) => {
              e.preventDefault();
              if (displayNameInvalid || bioInvalid) {
                return;
              }
```

```
    // @ts-ignore
    const displayName = e.target.elements.displayName.value;
    // @ts-ignore
    const bio = e.target.elements.bio.value;

    REST.Community.createProfile({
      displayName,
      biography: bio,
      token: localStorage.getItem("token") as string,
    }).then((res) => {
      if (res.payload.success) {
        toast({
          title: "Erfolg",
          description: "Dein Profil wurde erstellt.",
          status: "success",
          duration: 3000,
          onCloseComplete: () => {
            window.location.href = "/community";
          },
        });
      }
    });
  }}
>
  <FormControl isInvalid={displayNameInvalid} mt={"2rem"} isRequired>
    <FormLabel htmlFor={"displayName"}>Anzeigename</FormLabel>
    <InputGroup>
      <Input
        name={"displayName"}
        placeholder={"Display Name"}
        onChange={(e) => {
          // @ts-ignore
          if (e.target.value.length > 0) {
            setDisplayNameInvalid(false);
          } else {
            setDisplayNameInvalid(true);
          }
        }}
      />
    </InputGroup>
    <FormErrorMessage>Anzeigename wird benötigt.</FormErrorMessage>
  </FormControl>
  <FormControl isInvalid={bioInvalid} mt={"2rem"} isRequired>
    <FormLabel htmlFor={"bio"}>Biographie</FormLabel>
    <InputGroup>
      <Textarea
        name={"bio"}
        placeholder={"Biography"}
        onChange={(e) => {
          // @ts-ignore
          if (e.target.value.length > 0) {
            setBioInvalid(false);
          } else {
            setBioInvalid(true);
          }
        }}
      />
    </InputGroup>
    <FormErrorMessage>Biographie wird benötigt.</FormErrorMessage>
  </FormControl>
```

```
            <Box mt={"2rem"} textAlign={"center"}>
              <Button type={"submit"} colorScheme={"whatsapp"}>
                Erstellen
              </Button>
            </Box>
          </form>
        </Box>
      </Flex>
    </>
  );
}
```

## apps/web/src/pages/community/profile/[name].tsx

```
/**
 * src/pages/community/profile/[id].ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 13.06.2022
 *
 */
import { PostCard } from "@codeup/components/community/post";
import { useRouter } from "next/router";
import React, { useEffect, useState } from "react";
import { default as redirect } from "@codeup/util/src/userworkflow/redirect";
import {
  Avatar,
  Box,
  Button,
  ButtonGroup,
  Drawer,
  DrawerBody,
  DrawerCloseButton,
  DrawerContent,
  DrawerFooter,
  DrawerHeader,
  DrawerOverlay,
  Flex,
  FormControl,
  FormErrorMessage,
  FormLabel,
  Heading,
  Image,
  Input,
  InputGroup,
  InputLeftAddon,
  Text,
  Textarea,
  useDisclosure,
  useToast,
} from "@chakra-ui/react";

import {
  ProfileAvatar,
  ProfileAvatarModifier,
  ProfileBadges,
  ProfileBanner,
  ProfileBannerModifier,
  ProfileDataDisplay,
```

```
  ProfileMenuButton,
  ProfileNotFound,
} from "@codeup/components/community/profile";

import {
  FaBirthdayCake,
  FaCheckCircle,
  FaGlobe,
  FaInfo,
  FaMapMarkerAlt,
} from "react-icons/fa";
import { useHasProfile, useLoggedIn } from "@codeup/hooks";
import REST from "@codeup/rest";
import Head from "next/head";

//
// we need to disable SSR until I finally know
// why this page is throwing a Hydration Error
//
// Future me, forgive me.
// ~ Ben 24.06.2022
// Please rewrite the whole component
// and split it into more subcomponents
// to clean up the f*cking mess of JSX code
//
// nvm, this error only affects development builds
// I am really sorry, future me, that you have to deal
// with this in developing mode.
// please forgive me. You can safely delete the SSR disabling code
// btw, it never worked :(
//
const SSR = typeof window === "undefined";

export default function ProfileView() {
  const router = useRouter();

  const [profile, setProfile] = useState({
    _id: "",
    user: "",
    profilePicture: "",
    profileBanner: "",
    username: "",
    displayName: "",
    verified: false,
    specialStatus: "",
    bio: "",
    location: "",
    birthday: "",
    sexuality: "",
    gender: "",
    followers: [],
    createdAt: new Date(),
    website: "",
    __v: 0,
  });

  const name = router.query.name;

  const [notFound, setNotFound] = useState(false);
  const {
```

```
  isOpen: isEditOpen,
  onOpen: onEditOpen,
  onClose: onEditClose,
} = useDisclosure();

const hasProfile = useHasProfile();

const [displayNameValid, setDisplayNameValid] = useState(true);
const [bioValid, setBioValid] = useState(true);
const [locationValid, setLocationValid] = useState(true);
const [birthdayValid, setBirthdayValid] = useState(true);
const [websiteValid, setWebsiteValid] = useState(true);

const toast = useToast();

const {
  isOpen: isAvatarEditOpen,
  onOpen: onAvatarEditOpen,
  onClose: onAvatarEditClose,
} = useDisclosure();
const {
  isOpen: isBannerEditOpen,
  onOpen: onBannerEditOpen,
  onClose: onBannerEditClose,
} = useDisclosure();

const [currentRange, setCurrentRange] = useState({
  from: 0,
  to: 5,
});

const [currentPosts, setCurrentPosts] = useState([]);
const [maxPosts, setMaxPosts] = useState(0);

const { loggedIn, userInfo } = useLoggedIn();

useEffect(() => {
  redirect({
    allowAnonymous: false,
    redirectTo: window.location.href,
    redirectToLogin: true,
    forceAnonymous: false,
  });
}, []);

useEffect(() => {
  REST.Community.getProfile(name as string).then((res) => {
    if (res.status === 200) {
      setProfile(res.payload.profile);
      setNotFound(false);
    } else {
      setNotFound(true);
    }
  });
}, [name, userInfo]);

useEffect(() => {
  if (hasProfile.tried && !hasProfile.hasProfile) {
    router.push("/community");
  }
```

```
}, [hasProfile, router]);

useEffect(() => {
  if (name !== undefined) {
    REST.Community.getProfilePosts({
      name: name as string,
      from: currentRange.from,
      to: currentRange.to,
    }).then((res) => {
      if (res.status === 200) {
        setCurrentPosts(res.payload.posts);
      } else {
        toast({
          title: "Fehler",
          description: "Beiträge konnten nicht geladen werden",
          status: "error",
          duration: 6000,
          isClosable: false,
        });
      }
    });
  }
}, [currentRange, name, toast]);

useEffect(() => {
  if (name !== undefined) {
    REST.Community.getProfilePosts({
      name: name as string,
      count: true,
    }).then((res) => {
      if (res.status === 200) {
        setMaxPosts(res.payload.postCount);
      } else {
        toast({
          title: "Fehler",
          description: "Beiträge konnten nicht geladen werden",
          status: "error",
          duration: 6000,
          isClosable: false,
        });
      }
    });
  }
}, [name, toast]);

return (
  <>
    <Head>
      <title>(Veraltet) Community Profil - CodeUp</title>
    </Head>
    {SSR ? null : (
      <>
        {notFound ? (
          <ProfileNotFound />
        ) : (
          <Profile
            profile={profile}
            editProfile={{
              isEditOpen,
              onEditOpen,
```

```
      onEditClose,
    }}
    range={{
      currentRange,
      setCurrentRange,
    }}
    currentPosts={currentPosts}
    maxPosts={maxPosts}
    key={"p-ro-file-000"}
  />
)}
<ProfileAvatarModifier
  isOpen={isAvatarEditOpen}
  onOpen={onAvatarEditOpen}
  onClose={onAvatarEditClose}
/>
<ProfileBannerModifier
  isOpen={isBannerEditOpen}
  onOpen={onBannerEditOpen}
  onClose={onBannerEditClose}
/>
<Drawer
  isOpen={isEditOpen}
  placement="left"
  onClose={onEditClose}
  size={["full", "lg"]}
>
  <DrawerOverlay />
  <DrawerContent overflow={"auto"}>
    <form
      onSubmit={(e) => {
        e.preventDefault();

        setDisplayNameValid(true);
        setBioValid(true);
        setLocationValid(true);
        setBirthdayValid(true);
        setWebsiteValid(true);

        const displayName = (
          document.getElementById("displayName") as HTMLInputElement
        ).value;
        const bio = (
          document.getElementById("biography") as HTMLTextAreaElement
        ).value;
        const location = (
          document.getElementById("location") as HTMLInputElement
        ).value;
        const birthday = (
          document.getElementById("birthday") as HTMLInputElement
        ).value;
        const website = (
          document.getElementById("website") as HTMLInputElement
        ).value;

        if (!(displayName.length > 0 && displayName.length < 51)) {
          setDisplayNameValid(false);
          return;
        }
```

```
            if (!(bio.length > 0 && bio.length < 201)) {
              setBioValid(false);
              return;
            }

            if (website.length > 0) {
              if (
                !/https?:\/\/(www\.)?[-a-zA-Z0-9@:%._\+~#=]{1,256}\.[a-zA-Z0-9()]{1,6}\b([-a-zA-Z0-
9()@:%_\+.~#?&//=]*)/.test(
                  website
                )
              ) {
                setWebsiteValid(false);
                return;
              }
            }

            if (!(location.length < 51)) {
              setLocationValid(false);
              return;
            }

            if (!/^\d{4}-\d{2}-\d{2}$/.test(birthday)) {
              setBirthdayValid(false);
              return;
            }

            REST.Community.updateProfile({
              token: localStorage.getItem("token") as string,
              update: {
                displayName,
                bio,
                location,
                birthday,
                website,
              },
            }).then((res) => {
              if (res.status === 200) {
                toast({
                  title: "Profil aktualisiert",
                  description:
                    "Dein Profil wurde erfolgreich aktualisiert.",
                  status: "success",
                  position: "top",
                  duration: 3000,
                  onCloseComplete: () => {
                    window.location.reload();
                  },
                });
              } else {
                toast({
                  title: "Fehler",
                  description:
                    "Dein Profil konnte nicht aktualisiert werden.",
                  status: "error",
                });
              }
            });
          }}
        >
```

```
<DrawerCloseButton />
<DrawerHeader>Profil bearbeiten</DrawerHeader>

<DrawerBody>
  <Flex flexDirection="column" gap={"2rem"}>
    <Box textAlign={"center"}>
      <Avatar
        src={"/api/account/" + userInfo.username + "/picture"}
        w={128}
        h={128}
        borderColor={"#FFF"}
        borderWidth={"4px"}
        cursor={"pointer"}
        _hover={{
          filter: "brightness(0.7)",
        }}
        onClick={onAvatarEditOpen}
      />
    </Box>
    <Box textAlign={"center"}>
      <Image
        alt={"Profil Banner"}
        src={profile.profileBanner}
        w={"100%"}
        borderColor={"#FFF"}
        borderWidth={"4px"}
        cursor={"pointer"}
        _hover={{
          filter: "brightness(0.7)",
        }}
        onClick={onBannerEditOpen}
      />
    </Box>
    <FormControl isInvalid={!displayNameValid}>
      <FormLabel>Anzeigename</FormLabel>
      <Input
        id={"displayName"}
        placeholder={"Display Name"}
        defaultValue={profile.displayName}
      />
      <FormErrorMessage>
        Anzeigename wird benötigt.
      </FormErrorMessage>
    </FormControl>
    <FormControl isInvalid={!bioValid}>
      <FormLabel>Biographie</FormLabel>
      <Textarea
        id={"biography"}
        placeholder={"Biographie"}
        defaultValue={profile.bio}
        maxLength={256}
      />
      <FormErrorMessage>
        Biographie wird benötigt.
      </FormErrorMessage>
    </FormControl>
    <FormControl isInvalid={!websiteValid}>
      <FormLabel>Website</FormLabel>
      <InputGroup>
        <InputLeftAddon>
```

```
              <FaGlobe />
            </InputLeftAddon>
            <Input
              id={"website"}
              placeholder={"Website"}
              defaultValue={profile.website}
            />
          </InputGroup>
          <FormErrorMessage>
            Website wird benötigt.
          </FormErrorMessage>
        </FormControl>
        <FormControl isInvalid={!locationValid}>
          <FormLabel>Standort</FormLabel>
          <InputGroup>
            <InputLeftAddon>
              <FaMapMarkerAlt />
            </InputLeftAddon>
            <Input
              id={"location"}
              placeholder={"Standort"}
              defaultValue={profile.location}
            />
          </InputGroup>
          <FormErrorMessage>
            Standort wird benötigt.
          </FormErrorMessage>
        </FormControl>
        <FormControl isInvalid={!birthdayValid}>
          <FormLabel>Birthday</FormLabel>
          <InputGroup>
            <InputLeftAddon>
              <FaBirthdayCake />
            </InputLeftAddon>
            <Input
              id={"birthday"}
              type={"date"}
              placeholder={"Geburtstag"}
              defaultValue={profile.birthday}
            />
          </InputGroup>
          <FormErrorMessage>
            Geburtstag wird benötigt.
          </FormErrorMessage>
        </FormControl>
      </Flex>
    </DrawerBody>

    <DrawerFooter>
      <Button variant="outline" mr={3} onClick={onEditClose}>
        Abbrechen
      </Button>
      <Button colorScheme="blue" type={"submit"}>
        Speichern
      </Button>
    </DrawerFooter>
  </form>
 </DrawerContent>
</Drawer>
</>
```

```
      )}
    </>
  );
}

function Profile(props: {
  profile: {
    _id: string;
    user: string;
    username: string;
    profilePicture: string;
    profileBanner: string;
    displayName: string;
    verified: boolean;
    specialStatus: string;
    bio: string;
    location: string;
    birthday: string;
    sexuality: string;
    gender: string;
    followers: string[];
    createdAt: Date;
    website: string;
    __v: number;
  };
  editProfile: {
    onEditOpen: () => void;
    onEditClose: () => void;
    isEditOpen: boolean;
  };
  range: {
    currentRange: { from: number; to: number };
    setCurrentRange: ({ from, to }: { from: number; to: number }) => void;
  };
  currentPosts: {
    _id: string;
    user: string;
    username: string;
    content: string;
    createdAt: string;
    likes: string[];
    comments: string[];
    parent: string | null;
  }[];
  maxPosts: number;
}) {
  return (
    <>
      <Flex
        justifyContent={"center"}
        alignItems={"center"}
        direction={"column"}
      >
        <Box
          maxWidth={1640}
          minHeight={[312, 624]}
          minWidth={{ base: "fit-content", xl: 1640 }}
        >
          <ProfileBanner url={props.profile.profileBanner} />
          <Flex flexDirection={["column", "row"]} flex={"100%"} gap={"1.2rem"}>
```

```
  <ProfileAvatar
    url={"/api/account/" + props.profile.username + "/picture"}
  />
  <Flex
    justifyContent={"center"}
    alignItems={"center"}
    flexDirection={"column"}
    flex={"20%"}
    mt={"1rem"}
    ml={[0, "-128px"]}
  >
    <Heading as={"h1"} fontSize={["3xl", "4xl"]}>
      {props.profile.displayName}
    </Heading>
    <Text color={"gray.500"}>@{props.profile.username}</Text>
    <ProfileDataDisplay
      profile={{
        username: props.profile.username,
        displayName: props.profile.displayName,
      }}
    />
    <ProfileBadges
      verified={props.profile.verified}
      specialStatus={props.profile.specialStatus}
    />
  </Flex>
  <ProfileMenuButton
    profile={{
      website: props.profile.website,
      username: props.profile.username,
    }}
    editProfile={props.editProfile}
  />
</Flex>
<Box w={"100%"} alignItems={"center"} justifyContent={"center"}>
  <Flex
    direction={["column", "row"]}
    mt={"4rem"}
    gap={"2rem"}
    maxW={"100%"}
    p={"2rem"}
  >
    <Box flex={["100%", "40%"]}>
      <Box
        bg={"gray.900"}
        paddingInline={"2rem"}
        paddingBlock={"1.2rem"}
        borderRadius={"12px"}
        w={"100%"}
      >
        <Heading as={"h2"} fontSize={["2xl", "3xl"]}>
          Über
        </Heading>
        <Text
          textAlign={"left"}
          fontFamily={"Comic Sans MS, Comic Sans"}
        >
          &amp;quot;{props.profile.bio}&amp;quot;
        </Text>
        <Heading as={"h2"} fontSize={["2xl", "3xl"]} mt={"2rem"}>
```

```
      Informationen
    </Heading>
    <Box textAlign={"left"} color={"gray.400"}>
      {props.profile.verified ? (
        <Box
          display={"flex"}
          alignItems={"center"}
          gap={".4rem"}
          mt={".3rem"}
          color={"green.400"}
        >
          <FaCheckCircle />
           Offizielles Konto
        </Box>
      ) : null}
      {props.profile.specialStatus.length > 0 ? (
        <Box
          display={"flex"}
          alignItems={"center"}
          gap={".4rem"}
          mt={".3rem"}
          color={"yellow.400"}
        >
          <FaInfo /> {props.profile.specialStatus}
        </Box>
      ) : null}
      {props.profile.location.length > 0 ? (
        <Box
          display={"flex"}
          alignItems={"center"}
          gap={".4rem"}
          mt={".3rem"}
        >
          <FaMapMarkerAlt /> {props.profile.location}
        </Box>
      ) : null}
      {props.profile.birthday.length > 0 ? (
        <Box
          display={"flex"}
          alignItems={"center"}
          gap={".4rem"}
          mt={".3rem"}
        >
          <FaBirthdayCake />
           
          {new Date(props.profile.birthday).toLocaleDateString()}
        </Box>
      ) : null}
    </Box>
  </Box>
</Box>
<Box flex={["100%", "60%"]}>
  <Heading mb={"4rem"} textAlign={"center"}>
    Beiträge
  </Heading>
  {props.currentPosts.map((post) => {
    // eslint-disable-next-line react/jsx-key
    return <PostCard id={post._id} />;
  })}
```

```
              <ButtonGroup mt={"4"} w={"100%"}>
                <Button
                  flex={"100%"}
                  disabled={props.range.currentRange.from === 0}
                  onClick={() => {
                    props.range.setCurrentRange({
                      from: props.range.currentRange.from - 5,
                      to: props.range.currentRange.to - 5,
                    });
                  }}
                >
                  Vorherige
                </Button>
                <Button
                  flex={"100%"}
                  disabled={!(props.range.currentRange.to < props.maxPosts)}
                  onClick={() => {
                    props.range.setCurrentRange({
                      from: props.range.currentRange.from + 5,
                      to: props.range.currentRange.to + 5,
                    });
                  }}
                >
                  Nächste
                </Button>
              </ButtonGroup>
            </Box>
          </Flex>
        </Box>
      </Box>
    </Flex>
  </>
 );
}
```

## apps/web/src/pages/community/search.tsx

```
/**
 * src/pages/community/search.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 21.06.2022
 *
 */

import * as React from "react";
import { useEffect, useState } from "react";
import {
  Avatar,
  Box,
  Divider,
  Flex,
  Heading,
  Input,
  InputGroup,
  InputRightAddon,
  Text,
  useToast,
} from "@chakra-ui/react";
```

```
import { FaSearch } from "react-icons/fa";
import { useRouter } from "next/router";
import { PostCard } from "@codeup/components/community/post";
import REST from "@codeup/rest";
import Head from "next/head";

export default function Search() {
  const [term, setTerm] = useState("");

  useEffect(() => {
    if (new URLSearchParams(window.location.search).has("term")) {
      setTerm(
        new URLSearchParams(window.location.search).get("term") as string
      );
    }
  }, []);

  const router = useRouter();

  const [results, setResults] = useState<{ posts: []; profiles: [] }>({
    posts: [],
    profiles: [],
  });
  const toast = useToast();

  useEffect(() => {
    if (term.length > 2) {
      REST.Community.search(term).then((res) => {
        setResults({
          posts: res.payload.posts,
          profiles: res.payload.posts,
        });
      });
    }
  }, [term, toast]);

  return (
    <>
      <Head>
        <title>(Veraltet) Community Suche - CodeUp</title>
      </Head>
      <Flex
        alignItems={"center"}
        justifyContent={"center"}
        w={"100%"}
        mt={"2rem"}
      >
        <InputGroup maxW={["100%", "75%"]}>
          <Input
            onChange={(e) => {
              router.push(`/community/search?term=${e.target.value}`);
              setTerm(e.target.value);
            }}
            type={"text"}
            defaultValue={term}
            placeholder={"Suchen..."}
          />
          <InputRightAddon cursor={"pointer"} onClick={() => {}}>
            <FaSearch />
          </InputRightAddon>
```

```
  </InputGroup>
</Flex>
{(term.length > 2 && results.posts.length > 0) ||
results.profiles.length > 0 ? (
  <Flex w={"100%"} direction={["column", "row"]} gap={"2rem"} p={"2rem"}>
    <Box flex={"100%"} mt={"2rem"}>
      <Heading textAlign={"center"}>Beiträge</Heading>
      {
        // @ts-ignore
        results.posts
          ? results.posts.map((post: any) => {
              return (
                <>
                  <Box
                    cursor="pointer"
                    onClick={() => {
                      router.push("/community/post/" + post._id);
                    }}
                  >
                    <PostCard id={post._id} />
                  </Box>
                </>
              );
            })
          : null
      }
    </Box>
    <Box flex={"100%"} mt={"2rem"}>
      <Heading textAlign={"center"}>Profile</Heading>
      {
        // @ts-ignore
        results.profiles
          ? results.profiles.map((profile: any) => {
              return (
                <>
                  <Flex
                    direction={"row"}
                    mb={"1.2rem"}
                    cursor={"pointer"}
                    onClick={() => {
                      router.push(
                        "/community/profile/" + profile.username
                      );
                    }}
                  >
                    <Avatar
                      src={profile.profilePicture}
                      size={"lg"}
                      borderColor={"#FFF"}
                      borderWidth={"2px"}
                    />
                    <Flex direction={"column"} ml={"1.2rem"}>
                      <Heading as={"h1"} fontSize={["2xl", "3xl"]}>
                        {profile.displayName}
                      </Heading>
                      <Text color={"gray.500"}>@{profile.username}</Text>
                    </Flex>
                  </Flex>
                  <Divider />
                </>
```

```
                  );
                })
              : null
          }
        </Box>
      </Flex>
    ) : null}
  </>
 );
}
```

## apps/web/src/pages/course/index.tsx

```
/**
 * apps/web/src/pages/course/index.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 13.11.2022
 *
 */

import * as React from "react";
import REST from "@codeup/rest";
import {
  useToast,
  Box,
  Button,
  ButtonGroup,
  chakra,
  Flex,
  Heading,
  Image,
  Text,
  Grid,
  GridItem,
  Link,
  Input,
  InputLeftElement,
  InputGroup,
} from "@chakra-ui/react";
import { useRouter } from "next/router";
import { FaSearch } from "react-icons/fa";
import Head from "next/head";
import AccountManager from "@codeup/util/dist/AccountManager";
import AuthenticateLoadingIndicator from
"@codeup/components/dist/admin/AuthenticateLoadingIndicator";
import { CHeading } from "@codeup/components/dist/global/CHeading";

export default function Index() {
  const [courses, setCourses] = React.useState<
    {
      _id: string;
      name: string;
      friendlyName: string;
      description: string;
      splashImage: string;
      available: boolean;
      contentPositions: string[];
      createdAt: string;
```

```
    creator: string;
    __v: number;
  }[]
>([]);

const [filteredCourses, setFilteredCourses] = React.useState<
  {
    _id: string;
    name: string;
    friendlyName: string;
    description: string;
    splashImage: string;
    available: boolean;
    contentPositions: string[];
    createdAt: string;
    creator: string;
    __v: number;
  }[]
>([]);

const [coursesLoading, setCoursesLoading] = React.useState<boolean>(true);

const toast = useToast();
const router = useRouter();

React.useEffect(() => {
  REST.Course.listCourses({
    token: AccountManager.getActiveToken(),
  }).then((res) => {
    if (res.status === 200) {
      setCourses(res.payload.courses);
      setFilteredCourses(res.payload.courses);
      setCoursesLoading(false);
    } else {
      toast({
        title: "Fehler",
        description: "Es ist ein Fehler aufgetreten",
        status: "error",
        duration: 1500,
        isClosable: true,
        position: "bottom-right",
      });
    }
  });
}, [toast]);

return (
  <>
    <Head>
      <title>Kursübersicht - CodeUp</title>
    </Head>
    <Flex
      direction="column"
      w={"100%"}
      h={"fit-content"}
      minH={"100vh"}
      bg={"#000"}
    >
      <Flex
        w={"100%"}
```

```
  h={"fit-content"}
  direction={"column"}
  alignItems={"center"}
  mt={4}
>
  <Box textAlign={"center"}>
    <CHeading size={"3xl"}>Kurse</CHeading>
  </Box>
  <Text fontSize={["xl", "2xl"]} textAlign={["center", "initial"]}>
    Aktuell werden 
    <chakra.b color={"brand.500"}>
      {courses.length} Kurs
      {courses.length !== 1 ? "e" : ""}
    </chakra.b>
     mit insgesamt 
    <chakra.b color={"brand.500"}>
      {courses
        .map((course) => course.contentPositions.length)
        .reduce((a, b) => a + b, 0)}
       Lektion
      {courses
        .map((course) => course.contentPositions.length)
        .reduce((a, b) => a + b, 0) !== 1
        ? "en"
        : ""}
    </chakra.b>
     angeboten.
  </Text>
  <InputGroup
    w={["75%", "50%"]}
    mt={"2rem"}
    maxW={["75%", "50%"]}
    boxShadow={"xl"}
  >
    <InputLeftElement pointerEvents="none">
      <FaSearch />
    </InputLeftElement>
    <Input
      type="text"
      placeholder="Kurse durchsuchen"
      fontSize={"xl"}
      onChange={(e) => {
        setFilteredCourses(
          courses.filter(
            (course) =>
              course.name
                .toLowerCase()
                .includes(e.target.value.toLowerCase()) ||
              course.friendlyName
                .toLowerCase()
                .includes(e.target.value.toLowerCase()) ||
              course.description
                .toLowerCase()
                .includes(e.target.value.toLowerCase())
          )
        );
      }}
    />
  </InputGroup>
</Flex>
```

```
<Grid
  templateColumns={[
    "repeat(1, 1fr)",
    "repeat(2, 1fr)",
    "repeat(3, 1fr)",
    "repeat(4, 1fr)",
    "repeat(5, 1fr)",
  ]}
  p={4}
  w={"100%"}
  justifyContent="center"
  alignItems={"center"}
  h={"fit-content"}
>
  {filteredCourses.map((c) => (
    <>
      <GridItem h={"100%"} p={4}>
        <Box
          bg={"gray.900"}
          color={"white"}
          rounded={"xl"}
          p={6}
          h={"100%"}
          cursor={"pointer"}
          _hover={{
            shadow: "2xl",
          }}
          onClick={() => {
            router.push(`/course/${c.friendlyName}`);
          }}
        >
          <Flex
            justifyContent={"space-between"}
            direction={"column"}
            gap={2}
          >
            <Image
              src={c.splashImage}
              alt={c.name}
              w={"100%"}
              h={"150px"}
            />
            <CHeading size={"xl"}>{c.name}</CHeading>
            <Text fontSize={"xl"} color={"gray.300"} textAlign={"left"}>
              von 
              <chakra.span color={"brand.400"}>
                @{c.creator}
              </chakra.span>
            </Text>
            <Heading
              size={"lg"}
              color={"gray.300"}
              fontWeight={"normal"}
              textAlign={"left"}
            >
              {c.description.substring(0, 100)}
              ...
            </Heading>
            <Text fontSize={"xl"} color={"gray.300"} textAlign={"left"}>
              {c.contentPositions.length} Lektion
```

```
                {c.contentPositions.length !== 1 ? "en" : ""}
              </Text>
            </Flex>
          </Box>
        </GridItem>
      </>
    ))}
  </Grid>
  {filteredCourses.length === 0 && !coursesLoading ? (
    <>
      <Heading
        size={"md"}
        color={"gray.400"}
        fontWeight={"normal"}
        textAlign={"center"}
      >
        Keine Kurse gefunden
      </Heading>
    </>
  ) : null}
  {coursesLoading && (
    <>
      <AuthenticateLoadingIndicator text={"Lade Kurse..."} />
    </>
  )}
 </Flex>
 </>
 );
}
```

## apps/web/src/pages/course/[course]/index.tsx

```
/**
 * apps/web/src/pages/course/[course]/index.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 13.11.2022
 *
 */

import * as React from "react";
import REST from "@codeup/rest";
import {
  useToast,
  Box,
  Button,
  ButtonGroup,
  chakra,
  Flex,
  Heading,
  Image,
  Text,
  Grid,
  GridItem,
  Link,
  Input,
  Badge,
  Stack,
  StackDivider,
```

```
  Divider,
  Progress,
} from "@chakra-ui/react";
import { useRouter } from "next/router";
import Head from "next/head";
import AccountManager from "@codeup/util/dist/AccountManager";
import Milestones from "@codeup/components/landingpage/Milestones";
import AuthenticateLoadingIndicator from
"@codeup/components/dist/admin/AuthenticateLoadingIndicator";
import { CHeading } from "@codeup/components/dist/global/CHeading";
import { useEffect, useState } from "react";
import redirect from "@codeup/util/dist/userworkflow/redirect";
import { FaCheck } from "react-icons/fa";

export default function Index() {
  const [course, setCourse] = React.useState<{
    _id: string;
    name: string;
    friendlyName: string;
    description: string;
    splashImage: string;
    available: boolean;
    contentPositions: string[];
    createdAt: string;
    creator: string;
    __v: number;
  }>();

  const [contents, setContents] = React.useState<
    {
      _id: string;
      displayName: string;
      url: string;
      course: string;
      createdAt: string;
      type: "VIDEO" | "TEXT";
      textContent: string;
      quiz: any[];
      videoUrl: string;
    }[]
  >([]);
  const [contentsLoading, setContentsLoading] = React.useState<boolean>(true);
  const [progress, setProgress] = React.useState<string[]>([]);

  const [isEnrolled, setEnrolled] = useState<boolean>(false);

  const router = useRouter();

  const { course: courseName } = router.query;
  const toast = useToast();

  useEffect(() => {
    redirect({
      forceAnonymous: false,
      allowAnonymous: false,
      redirectToLogin: true,
      redirectTo: "/course/" + courseName,
    });
  });
```

```
React.useEffect(() => {
  if (courseName) {
    REST.Course.getCourse({
      course: courseName as string,
      token: AccountManager.getActiveToken(),
    }).then(async (res) => {
      if (res.status === 200) {
        setCourse(res.payload.course);

        let contents0 = [];
        for (const contentId of res.payload.course.contentPositions) {
          const res0 = await REST.Course.getSection({
            course: courseName as string,
            section: contentId,
            token: AccountManager.getActiveToken(),
          });

          if (res0.status === 200) {
            contents0.push(res0.payload.content);
          }
        }

        setContents(contents0);
        setContentsLoading(false);
      } else {
        toast({
          title: "Error",
          description: "Der Kurs konnte nicht geladen werden",
          status: "error",
          duration: 5000,
          isClosable: true,
        });
      }
    });

    REST.Course.getEnrollment({
      token: AccountManager.getActiveToken(),
      course: courseName as string,
    }).then(async (res) => {
      let enrolled = res.payload.enrolled ? true : false;
      setEnrolled(enrolled);

      if (enrolled) {
        const rx = await REST.Course.getCourseProgress({
          token: AccountManager.getActiveToken(),
          course: courseName as string,
        });

        if (rx.status === 200) {
          setProgress(rx.payload.progress);
        }
      }
    });
  }
}, [courseName]);

return (
  <>
    <Head>
      <title>{course ? course.name : "Kursübersicht"} - CodeUp</title>
```

```
</Head>
{course ? (
  <>
    <Flex
      direction={"column"}
      pt={"2rem"}
      alignItems={"center"}
      w={"100%"}
      minH={"100vh"}
      h={"fit-content"}
      bg={"#000"}
    >
      <Box
        w={["100%", "75%"]}
        backgroundColor={"black"}
        p={4}
        rounded={"xl"}
        boxShadow={"xl"}
      >
        <CHeading size={"2xl"}>
          {course.name}
          <chakra.sub pl={4} fontSize={"2xl"}>
            von @{course.creator}
          </chakra.sub>
        </CHeading>
        <Flex direction={["column", "row"]} w={"100%"} mt={6}>
          <Box flex={"100%"}>
            <Image
              src={course.splashImage}
              alt={course.name}
              w={["100%", "75%"]}
              maxH={"300px"}
              objectFit="cover"
              borderRadius="lg"
            />
            <Text mt={4} fontSize={"lg"}>
              <CHeading size={"lg"}>Beschreibung</CHeading>
              {course.description}
            </Text>
            <Text mt={4} fontSize={"lg"}>
              <CHeading size={"lg"}>Status</CHeading>
              {isEnrolled ? (
                <>
                  <Badge colorScheme="green">Eingeschrieben</Badge>
                  <br />
                  Du kannst deinen Fortschritt nachverfolgen. <br />
                  Eine Lektion wird als abgeschlossen markiert, wenn du
                  alle Quizfragen richtig beantwortet hast.
                </>
              ) : (
                <>
                  <Badge colorScheme="red">Nicht eingeschrieben</Badge>
                  <br />
                  Du kannst deinen Fortschritt nicht nachverfolgen.
                  <br />
                  <Button
                    variant={"brand"}
                    mt={2}
                    onClick={async () => {
                      await REST.Course.changeEnrollment({
```

```
                  token: AccountManager.getActiveToken(),
                  course: courseName as string,
                });

                setEnrolled(true);
              }}
            >
              Einschreiben
            </Button>
          </>
        )}
      </Text>
      {isEnrolled && (
        <>
          <Text mt={4} fontSize={"lg"}>
            <CHeading size={"lg"}>Fortschritt</CHeading>
            <br />
            <Progress
              value={progress.length}
              max={contents.length}
              colorScheme={"brand"}
            />
          </Text>
        </>
      )}
    </Box>
    <Box flex={"100%"}>
      <Milestones
        milestones={contents.map((content) => {
          return {
            title: content.displayName,
            lwdIcon: progress.includes(content._id) ? (
              <FaCheck />
            ) : null,
            date: new Date(content.createdAt).toLocaleDateString(),
            description:
              "Diese Lektion enthält " +
              content.quiz.length +
              " Quizfragen.",
            cursor: "pointer",
            onClick: () => {
              router.push(
                "/course/" + course.friendlyName + "/" + content._id
              );
            },
          };
        })}
        title={"Lektionen"}
      />
      {contentsLoading && (
        <>
          <AuthenticateLoadingIndicator text={"Lade Inhalte..."} />
        </>
      )}
    </Box>
  </Flex>
 </Box>
</Flex>
</>
) : null}
```

```
    </>
  );
}
```

## apps/web/src/pages/course/[course]/[section].tsx

```
/**
 * apps/web/src/pages/course/[course]/[section].tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 05.11.2022
 *
 */

import * as React from "react";
import { useRouter } from "next/router";
import { useEffect, useState } from "react";
import {
  Box,
  Button,
  ButtonGroup,
  CircularProgress,
  Flex,
  Heading,
  Link,
  ListItem,
  OrderedList,
  Text,
  UnorderedList,
} from "@chakra-ui/react";
import ErrorComponent from "@codeup/components/global/ErrorComponent";
import REST from "@codeup/rest";
import ReactMarkdown from "react-markdown";
import CodeBlock from "../../../special-components/CodeBlock";
import MDRenderer from "../../../special-components/MDRenderer";
import VideoPlayer from "@codeup/components/global/VideoPlayer";
import { Quiz } from "@codeup/components/course";
import Head from "next/head";
import AccountManager from "@codeup/util/dist/AccountManager";
import { FaBackward, FaForward } from "react-icons/fa";
import PopupManager from "../../../global/PopupManager";

export default function SectionViewer() {
  const router = useRouter();

  const [courseQuery, setCourseQuery] = useState<string>("");
  const [sectionQuery, setSectionQuery] = useState<string>("");

  useEffect(() => {
    if (router.query.course) {
      setCourseQuery(router.query.course as string);
    }
    if (router.query.section) {
      setSectionQuery(router.query.section as string);
    }
  }, [router.query.course, router.query.section]);

  const [loading, setLoading] = useState<boolean>(true);
```

```
const [notFound, setNotFound] = useState<boolean>(false);
const [content, setContent] = useState<{
  _id: string;
  displayName: string;
  course: string;
  url: string;
  type: "TEXT" | "VIDEO";
  textContent: string;
  quiz: {
    question: string;
    answers: {
      answer: string;
      correct: boolean;
    }[];
  }[];
  creator: string;
  createdAt: string;
  __v: number;
}>();

const [contentPositions, setContentPositions] = useState<string[]>([]);

useEffect(() => {
  if (courseQuery === "" || sectionQuery === "") return;
  REST.Course.getSection({
    course: courseQuery,
    section: sectionQuery,
    token: AccountManager.getActiveToken(),
  }).then((res) => {
    if (res.status !== 200) {
      setNotFound(true);
    } else {
      setContent(res.payload.content);
      REST.Course.getCourse({
        course: courseQuery,
        token: AccountManager.getActiveToken(),
      }).then((res0) => {
        if (res0.status === 200) {
          setContentPositions(res0.payload.course.contentPositions);
        } else {
          setNotFound(true);
        }
        setLoading(false);
      });
    }
  });
}, [courseQuery, sectionQuery]);

const [imgId, setImgId] = useState(Math.floor(Math.random() * 12) + 1);

useEffect(() => {
  // router.reload on key ENTER press
  const handleKeyDown = (event: KeyboardEvent) => {
    if (event.key === "r") {
      setImgId(Math.floor(Math.random() * 12) + 1);
    }
  };
  window.addEventListener("keydown", handleKeyDown);
  return () => window.removeEventListener("keydown", handleKeyDown);
}, []);
```

```
return (
  <>
    <Head>
      <title>{content ? content.displayName : "Inhalt"} - CodeUp</title>
    </Head>
    {loading ? (
      <>
        <Flex
          w={"100%"}
          h={"100vh"}
          justifyContent={"center"}
          alignItems={"center"}
          direction={"column"}
          bg={"#000"}
        >
          <CircularProgress size={"80px"} isIndeterminate color="brand.500" />
          <Heading size={"lg"} mt={4}>
            Lade Inhalte...
          </Heading>
        </Flex>
      </>
    ) : null}
    {notFound ? (
      <>
        <ErrorComponent
          description={"Die angeforderte Seite konnte nicht gefunden werden!"}
          title={"Inhalte konnten nicht geladen werden!"}
        />
      </>
    ) : null}
    {!loading && !notFound && content ? (
      <>
        <Box
          w={"100%"}
          minH={"100vh"}
          h={"fit-content"}
          display={"flex"}
          flexDirection={"column"}
          alignItems={"center"}
          bgPosition={"center"}
          bgRepeat={"no-repeat"}
          bgSize={"cover"}
          bgImage={`url(/_static/images/courses/desk_backgrounds/${imgId}.jpg)`}
        >
          <Box
            w={"100%"}
            mt={8}
            mb={8}
            maxW={["100%", "40%"]}
            h={"fit-content"}
            flexDirection={"column"}
            alignItems={"center"}
            justifyContent={"center"}
            p={4}
            bg={"#000000"}
            borderRadius={"md"}
            boxShadow={"2xl"}
          >
            <Heading size={"lg"} textAlign={"center"} mb={6}>
```

```
  {content.displayName}
</Heading>
{
  // @ts-ignore
  content.videoUrl !== "" && (
    <>
      <VideoPlayer
        url={
          // @ts-ignore
          content.videoUrl
        }
      />
    </>
  )
}
{/* <MDRenderer content={content.textContent} /> */}
{content.textContent &&
content.textContent !== "__codeup.__no_text_content" ? (
  <iframe
    src={
      "/api/course/" +
      courseQuery +
      "/" +
      sectionQuery +
      "?raw=true"
    }
    style={{
      width: "100%",
      minHeight: "50vh",
      height: "fit-content",
      border: "none",
      maxWidth: "100%",
    }}
    onLoad={(e) => {
      const iframe = e.target as HTMLIFrameElement;

      if (iframe && iframe.contentDocument) {
        iframe.contentDocument.body.style.maxWidth = "100%";
        iframe.contentDocument.body.style.overflowX = "hidden";
        iframe.contentDocument.body.style.margin = "0";
        iframe.contentDocument.body.style.padding = "0";
      }
    }}
  />
) : null}
{content.quiz && content.quiz.length > 0 && (
  <Quiz
    title={content.displayName}
    questions={content.quiz}
    successCB={async () => {
      await REST.Course.updateCourseProgress({
        course: courseQuery,
        section: sectionQuery,
        token: AccountManager.getActiveToken(),
      });
      PopupManager.alertAsync({
        title: "Abgeschlossen!",
        description:
          "Du hast diese Lektion erfolgreich abgeschlossen!",
      });
```

```
              }}
            />
          )}
          <ButtonGroup w={"100%"} mt={"2rem"}>
            <Button
              w={"100%"}
              variant={"brand"}
              leftIcon={<FaBackward />}
              onClick={() => {
                router.push(
                  "/course/" +
                    courseQuery +
                    "/" +
                    contentPositions[
                      contentPositions.indexOf(sectionQuery) - 1
                    ]
                );
              }}
              disabled={contentPositions.indexOf(content._id) === 0}
            >
              Zurück
            </Button>
            <Button
              w={"100%"}
              variant={"brand"}
              rightIcon={<FaForward />}
              onClick={() => {
                router.push(
                  "/course/" +
                    courseQuery +
                    "/" +
                    contentPositions[
                      contentPositions.indexOf(sectionQuery) + 1
                    ]
                );
              }}
              disabled={
                contentPositions.indexOf(content._id) ===
                contentPositions.length - 1
              }
            >
              Weiter
            </Button>
          </ButtonGroup>
        </Box>
      </Box>
    </>
  ) : null}
 </>
);
}
```

## apps/web/src/pages/courses/index.tsx

```
/**
 * apps/web/src/pages/courses/index.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 19.09.2022
```

```
 *
 */

import * as React from "react";
import { useEffect } from "react";
import {
  Box,
  Button,
  ButtonGroup,
  chakra,
  Flex,
  Heading,
} from "@chakra-ui/react";
import Link from "next/link";
import HTMLCourseSections from "@codeup/components/course/HTMLCourseSections";
import Head from "next/head";

export default function Index() {
  interface x {
    name: string;
    image: string;
    description: string;
    url: string;
  }

  const courses: x[] = [
    {
      name: "HTML Basics",
      image: "/_static/images/courses/splashes/html-basics.svg",
      description:
        "Mit HTML werden Webseiten erstellt. In diesem Kurs lernst du die Grundlagen von HTML.",
      url: "/course/html-basics/basics",
    },
  ];

  useEffect(() => {
    courses.forEach((course) => {
      window.dispatchEvent(
        new CustomEvent(
          "codeup::courses:toggle_s_button?" +
            course.name.toLowerCase().replace(" ", "_")
        )
      );
    });
  }, []);

  return (
    <>
      <Head>
        <title>(Old) Kurse - CodeUp</title>
      </Head>
      <Flex
        direction="column"
        w={"100%"}
        justifyContent="center"
        gap={"2rem"}
        alignItems={"center"}
        mt={"2rem"}
      >
        {courses.map((course) => {
```

```
return (
  <>
    <Box
      bg="white"
      p={[0, "2rem"]}
      _dark={{
        bg: "gray.900",
      }}
      display={["initial", "flex"]}
      w={["100%", "75%"]}
      shadow={"xl"}
      rounded={"lg"}
    >
      <Box w={"100%"}>
        <Box
          h={{
            base: 64,
            lg: "full",
          }}
          rounded={{
            lg: "lg",
          }}
          bgSize="cover"
          style={{
            backgroundImage: "url('" + course.image + "')",
            backgroundPosition: "center",
            backgroundSize: "cover",
          }}
        />
      </Box>

      <Box py={12} px={6} w={"100%"}>
        <Heading fontSize={"3xl"} fontWeight="bold">
          {course.name}
        </Heading>
        <chakra.p mt={4} color="gray">
          {course.description}
        </chakra.p>

        <Box mt={8}>
          <ButtonGroup>
            <Link href={course.url}>
              <Button colorScheme="whatsapp" size="lg">
                Kurs öffnen
              </Button>
            </Link>
            <Button
              colorScheme="linkedin"
              size="lg"
              onClick={() => {
                window.dispatchEvent(
                  new CustomEvent(
                    "codeup::courses:open_sections?" +
                      course.name.toLowerCase().replace(" ", "_")
                  )
                );
              }}
            >
              Lektionen
            </Button>
```

```
                </ButtonGroup>
              </Box>
            </Box>
          </Box>
        </>
      );
    })}
  </Flex>
  <HTMLCourseSections />
 </>
 );
}
```

## apps/web/src/pages/creator/index.tsx

```
/**
 * apps/web/src/pages/creator/index.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2024 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 10.05.2024
 *
 */

import * as React from "react";
import {
 Button,
 ButtonGroup,
 Flex,
 Heading,
 Image,
 Text,
 Box,
 Grid,
 Avatar,
} from "@chakra-ui/react";
import { useRouter } from "next/router";
import { useEffect } from "react";
import REST from "@codeup/rest/dist";
import { CHeading } from "@codeup/components/dist/global/CHeading";

export default function Index() {
 const router = useRouter();
 const [topCreators, setTopCreators] = React.useState<
  {
   _id: string;
   count: number;
  }[]
 >([]);

 useEffect(() => {
  REST.Creator.getTopCreators().then((res) => {
   if (res.status === 200) {
    setTopCreators(res.payload.creators);
   }
  });
 }, []);

 return (
  <>
```

```
<Box bg={"#000"}>
  <Flex w={"100%"} minH={"100vh"} h={"fit-content"} bg={"#000"}>
    <Flex
      w={"100%"}
      h={"fit-content"}
      p={[5, 10, 20]}
      direction={["column", "column", "row"]}
      alignItems={"center"}
      minH={"100vh"}
      zIndex={2}
    >
      <Flex
        flex={["100%", "100%", "100%"]}
        alignItems={"center"}
        w={"100%"}
        justifyContent={"center"}
      >
        <Flex gap={2} pt={10} direction={"column"}>
          <Heading
            as={"h3"}
            fontSize={["2xl", "3xl", "4xl"]}
            color={"gray.300"}
            fontWeight={1000}
            letterSpacing={-2}
          >
            CodeUp for Creators
          </Heading>
          <Heading
            as={"h1"}
            fontSize={["4xl", "5xl", "8xl"]}
            fontWeight={900}
            letterSpacing={-2}
            fontFamily={"Jetbrains Mono ExtraBold"}
            color={"brand.500"}
          >
            Creator Platform
          </Heading>
          <Text fontWeight={100} fontSize={["xl", "2xl", "4xl"]}>
            Erstelle interaktive Videokurse.
            <br />
            Teile dein Wissen mit der Community.
            <br />
            Baue eigene Zertifikate für deine Kurse.
            <br />
            Nutze die besten Werkzeuge.
            <br />
            Erhalte Unterstützung von CodeUp.
          </Text>
          <ButtonGroup spacing={4} mt={2}>
            <Button
              rounded={"md"}
              variant={"brand"}
              fontSize={"xl"}
              size={"lg"}
              color={"black"}
              onClick={() => {
                router.push("/codeup/contact?creator=true");
              }}
            >
              Bewerben
```

```
          </Button>
          <Button
            rounded={"md"}
            fontSize={"xl"}
            size={"lg"}
            onClick={() => {
              router.push("/creator/studio");
            }}
          >
            Studio
          </Button>
        </ButtonGroup>
      </Flex>
    </Flex>
    <Flex
      alignItems={"center"}
      justifyContent={"center"}
      w={"100%"}
      h={"100%"}
    >
      <Image
        src={"/_static/images/showcase/framed_course.png"}
        alt={"Software Developer"}
        w={"100%"}
        h={"100%"}
        objectFit={"cover"}
        objectPosition={"center"}
        borderRadius={"lg"}
        maxW={["400px", "500px", "1000px"]}
        shadow={"xl"}
      />
    </Flex>
  </Flex>
</Flex>
<Box h={"fit-content"}>
  <Box textAlign={"center"}>
    <CHeading>Creator Network</CHeading>
  </Box>
  <Flex alignItems={"center"} justifyContent={"center"}>
    <Grid
      templateColumns={["repeat(1, 1fr)", "repeat(2, 1fr)"]}
      maxW={["100%", "60%", "40%"]}
    >
      {topCreators.map((creator) => (
        <Box
          key={creator._id}
          w={"100%"}
          h={"fit-content"}
          p={"2rem"}
          display={"flex"}
          flexDirection={"column"}
          alignItems={"center"}
          justifyContent={"center"}
        >
          <Avatar
            src={`/api/account/${creator._id}/picture`}
            size={"2xl"}
            bgColor={"transparent"}
            name={creator._id}
            objectFit={"cover"}
```

```
                objectPosition={"center"}
                borderRadius={"lg"}
                maxW={["400px", "500px", "1000px"]}
                shadow={"xl"}
              />
              <Text fontWeight={"bold"} fontSize={"xl"} color={"brand.500"}>
                @{creator._id}
              </Text>
              <Text fontWeight={"bold"} fontSize={"xl"} color={"brand.500"}>
                {creator.count} Kurse
              </Text>
            </Box>
          ))}
        </Grid>
      </Flex>
    </Box>
  </Box>
 </>
 );
}
```

## apps/web/src/pages/creator/old-landing.tsx

```
/**
 * apps/web/src/pages/creator/index.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 18.11.2022
 *
 */

import * as React from "react";
import { useEffect } from "react";
import {
  Avatar,
  Box,
  Button,
  ButtonGroup,
  Flex,
  Heading,
  Text,
} from "@chakra-ui/react";
import { useRouter } from "next/router";
import REST from "@codeup/rest";
import Head from "next/head";

export default function OldLanding() {
  const router = useRouter();

  const [topCreators, setTopCreators] = React.useState<
    {
      _id: string;
      count: number;
    }[]
  >([]);

  useEffect(() => {
    REST.Creator.getTopCreators().then((res) => {
      if (res.status === 200) {
```

```
      setTopCreators(res.payload.creators);
    }
  });
}, []);

return (
  <>
    <Head>
      <title>Creator - CodeUp</title>
    </Head>
    <Flex
      w={"100%"}
      minH={"100vh"}
      h={"fit-content"}
      direction={"column"}
      alignItems={"center"}
      justifyContent={"center"}
      bgPosition={"center"}
      bgRepeat={"no-repeat"}
      bgSize={"cover"}
      bgImage={"url(/_static/images/creator/worker.jpg)"}
    >
      <Heading
        mt={"4rem"}
        fontSize={["3xl", "6xl"]}
        color={"white"}
        textAlign={"center"}
        fontWeight={"bold"}
        fontFamily={"Seymour One"}
        maxW={["75%", "50%"]}
        textShadow={"0px 10px 50px #000000"}
      >
        Werde ein Creator auf CodeUp
      </Heading>

      <Text
        mt={"2rem"}
        fontSize={["xl", "2xl"]}
        color={"white"}
        textAlign={"center"}
        fontFamily={"Seymour One"}
        maxW={["75%", "50%"]}
        textShadow={"0px 10px 50px #000000"}
      >
        Du bist ein Experte in einem bestimmten Bereich? Dann bewirb dich
        jetzt als Creator auf CodeUp! Wir suchen immer wieder neue Creator,
        die hochwertige Inhalte für unsere Community erstellen.
      </Text>

      <ButtonGroup mt={"2rem"} spacing={"2rem"}>
        <Button
          colorScheme={"red"}
          size={"lg"}
          fontSize={"xl"}
          fontWeight={"bold"}
          fontFamily={"Seymour One"}
          textTransform={"uppercase"}
          onClick={() => {
            router.push("/codeup/contact?creator=true");
          }}
```

```
    >
      Bewerben
    </Button>
    <Button
      colorScheme={"facebook"}
      size={"lg"}
      fontSize={"xl"}
      fontWeight={"bold"}
      fontFamily={"Seymour One"}
      textTransform={"uppercase"}
      onClick={() => {
        router.push("/creator/studio");
      }}
    >
      Studio
    </Button>
  </ButtonGroup>
</Flex>
<Box w={"100%"} h={"fit-content"} p={"4rem"}>
  <Heading
    w={"100%"}
    fontSize={["3xl", "6xl"]}
    textAlign={"center"}
    fontWeight={"bold"}
    fontFamily={"Seymour One"}
  >
    Top Creator
  </Heading>
  <Flex
    w={"100%"}
    h={"100%"}
    direction={["column", "row"]}
    alignItems={"center"}
    justifyContent={"center"}
    mt={4}
  >
    {topCreators.map((creator) => (
      <Box
        key={creator._id}
        w={"100%"}
        h={"fit-content"}
        p={"2rem"}
        display={"flex"}
        flexDirection={"column"}
        alignItems={"center"}
        justifyContent={"center"}
      >
        <Avatar
          size={"2xl"}
          name={creator._id}
          bgColor={"transparent"}
          src={`/api/account/${creator._id}/picture`}
        />
        <Heading
          mt={2}
          fontSize={["xl", "2xl"]}
          textAlign={"center"}
          fontWeight={"bold"}
          fontFamily={"Seymour One"}
        >
```

```
              @{creator._id} <br />
              <Text
                fontSize={["md", "xl"]}
                textAlign={"center"}
                fontWeight={"normal"}
                fontFamily={"Seymour One"}
              >
                {creator.count} Kurs(e)
              </Text>
            </Heading>
          </Box>
        ))}
      </Flex>
    </Box>
  </>
 );
}
```

## apps/web/src/pages/creator/studio/certificate/[courseid].tsx

```
/**
 * apps/web/src/pages/creator/studio/certificate/[id].tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 26.11.2022
 *
 */

import * as React from "react";
import { useRouter } from "next/router";
import CreatorStudioLayout from "@codeup/components/creator/studio/CreatorStudioLayout";
import {
  Heading,
  useToast,
  Slider,
  SliderTrack,
  SliderFilledTrack,
  SliderThumb,
  SliderMark,
  Box,
  Flex,
  Stack,
  FormControl,
  FormLabel,
  Popover,
  PopoverTrigger,
  PopoverContent,
  PopoverHeader,
  PopoverBody,
  PopoverFooter,
  PopoverArrow,
  PopoverCloseButton,
  PopoverAnchor,
  Select,
  Button,
  Input,
  Badge,
  IconButton,
  InputGroup,
```

```
  Grid,
  Switch,
  Image,
  InputLeftElement,
  ButtonGroup,
  useDisclosure,
} from "@chakra-ui/react";
import REST from "@codeup/rest";
import { FaArrowsAlt, FaCamera, FaCode, FaPlus, FaTrash } from "react-icons/fa";
import * as Builders from "@codeup/util/src/certificates/CertificateBuilder";
import { DragDropContext, Draggable, Droppable } from "@hello-pangea/dnd";
import Editor, { Monaco } from "@monaco-editor/react";
import html2canvas from "html2canvas";
import CodeUpModal from "@codeup/components/codeup/CodeUpModal";
import { editor } from "monaco-editor";
import IStandaloneCodeEditor = editor.IStandaloneCodeEditor;
import Head from "next/head";
import AccountManager from "@codeup/util/dist/AccountManager";
import { CHeading } from "@codeup/components/dist/global/CHeading";

export default function CertificateEditor() {
  const router = useRouter();

  const [course, setCourse] = React.useState<{
    _id: string;
    name: string;
    friendlyName: string;
    description: string;
    splashImage: string;
    available: boolean;
    contentPositions: string[];
    createdAt: string;
    creator: string;
    __v: number;
  }>();

  const toast = useToast();

  const { courseid } = router.query;
  React.useEffect(() => {
    if (courseid) {
      REST.Course.getCourse({
        course: courseid as string,
        token: AccountManager.getActiveToken(),
      }).then(async (res) => {
        if (res.status === 200) {
          setCourse(res.payload.course);
        } else {
          toast({
            title: "Error",
            description:
              "Der Kurs konnte nicht geladen werden. Denke daran ihn zuerst zu veröffentlichen.",
            status: "error",
            duration: 5000,
            isClosable: true,
          });
        }
      });
    }
  }, [courseid]);
```

```
const [certificate, setCertificate] = React.useState<any>(null);
const [minPercentage, setMinPercentage] = React.useState(75);
const [steps, setSteps] = React.useState<
  {
    step_type: "MULTIPLE_CHOICE" | "TEXT" | "CODE_COMPARE" | "CODE_CHECK";
    step_title: string;
    multiple_choice?: {
      question: string;
      answers: {
        answer: string;
        correct: boolean;
      }[];
    };
    text_parts?: {
      type: "TEXT" | "INPUT";
      value: string;
      id: number;
    }[];
    code_compare?: {
      code_a: {
        correct: boolean;
        screenshot: string;
      };
      code_b: {
        correct: boolean;
        screenshot: string;
      };
      output: string;
    };
    code_check?: {
      screenshot: string;
      correct: boolean;
    };
  }[]
>([]);

const labelStyles = {
  mt: "2",
  ml: "-2.5",
  fontSize: "sm",
};

const [hidePlusButton, setHidePlusButton] = React.useState(false);

React.useEffect(() => {
  if (courseid && course) {
    REST.Creator.updateCertificate({
      token: localStorage.getItem("token") as string,
      courseId: courseid as string,
      update: {
        minPercentage: minPercentage,
      },
    });
  }
}, [minPercentage]);

React.useEffect(() => {
  if (courseid && steps.length > 0 && course) {
    REST.Creator.updateCertificate({
```

```
      token: localStorage.getItem("token") as string,
      courseId: courseid as string,
      update: {
        steps: steps,
      },
    });
  }
}, [steps]);

React.useEffect(() => {
  if (courseid) {
    REST.Creator.getCertificate({
      courseId: courseid as string,
      token: localStorage.getItem("token") as string,
    }).then(async (res) => {
      if (res.status === 200) {
        setCertificate(res.payload.certificate);
        setMinPercentage(res.payload.certificate.minPercentage);
        setSteps(res.payload.certificate.steps);
      } else if (res.status !== 200) {
        REST.Creator.createCertificate({
          courseId: courseid as string,
          token: localStorage.getItem("token") as string,
        }).then(async (res0) => {
          if (res0.status === 200) {
            setCertificate(res0.payload.certificate);
            setMinPercentage(res0.payload.certificate.minPercentage);
            setSteps(res0.payload.certificate.steps);
          } else {
            toast({
              title: "Error",
              description: "Das Zertifikat konnte nicht erstellt werden.",
              status: "error",
              duration: 5000,
              isClosable: true,
            });
          }
        });
      }
    });
  }
}, [courseid]);

const { isOpen, onOpen, onClose } = useDisclosure();
const [monacoHook, setMonacoHook] = React.useState<Monaco | null>(null);
const [monacoEditorHook, setMonacoEditorHook] =
  React.useState<IStandaloneCodeEditor | null>(null);
const [monacoEditorLanguage, setMonacoEditorLanguage] = React.useState<
  string | null
>(null);
const [monacoEditorContent, setMonacoEditorContent] =
  React.useState<string>("");

return (
  <>
    <Head>
      <title>Zerifikat Editor - Creator Studio - CodeUp</title>
    </Head>
    <CreatorStudioLayout>
      <Box textAlign={"center"} pt={6}>
```

```
  <CHeading size={"xl"}>Zertifikatseditor</CHeading>
</Box>

<Stack w={"100%"} alignItems={"center"} justifyContent={"center"}>
  <Box w={["100%", "100%", "75%"]}>
    <Flex
      w={"100%"}
      alignItems={"center"}
      justifyContent={"center"}
      direction={"column"}
    >
      <FormControl maxW={"50%"} mt={8}>
        <FormLabel>Mindestpunktzahl (in %)</FormLabel>
        <Box pt={6} pb={2}>
          <Slider
            aria-label="slider-ex-6"
            value={minPercentage}
            colorScheme={"brand"}
            onChange={(val) => setMinPercentage(val)}
          >
            <SliderMark value={25} {...labelStyles}>
              25%
            </SliderMark>
            <SliderMark value={50} {...labelStyles}>
              50%
            </SliderMark>
            <SliderMark value={75} {...labelStyles}>
              75%
            </SliderMark>
            <SliderMark
              value={minPercentage}
              textAlign="center"
              bg="brand.600"
              color="white"
              mt="-10"
              ml="-5"
              w="12"
            >
              {minPercentage}%
            </SliderMark>
            <SliderTrack>
              <SliderFilledTrack />
            </SliderTrack>
            <SliderThumb />
          </Slider>
        </Box>
      </FormControl>

      <Box w={"100%"}>
        <Heading size={"lg"} textAlign={"center"} mt={8}>
          Schritte
        </Heading>

        {steps.length === 0 ? (
          <Box
            mt={8}
            bgColor={"gray.900"}
            w={"100%"}
            minH={"10vh"}
            rounded={"xl"}
```

```
  >
    <Flex
      w={"100%"}
      h={"10vh"}
      alignItems={"center"}
      justifyContent={"center"}
      direction={"column"}
    >
      <Heading
        size={"md"}
        textAlign={"center"}
        color={"gray.200"}
      >
        Es wurden noch keine Schritte hinzugefügt.
      </Heading>
    </Flex>
  </Box>
) : (
  <>
    <DragDropContext
      onDragStart={() => setHidePlusButton(true)}
      onDragEnd={(result) => {
        setHidePlusButton(false);

        if (!result.destination) return;

        const items = Array.from(steps);

        const [reorderedItem] = items.splice(
          result.source.index,
          1
        );

        items.splice(
          result.destination.index,
          0,
          reorderedItem
        );

        setSteps(items);
      }}
    >
      <Droppable droppableId={"steps"}>
        {(provided, snapshot) => (
          <>
            <Box
              ref={provided.innerRef}
              {...provided.droppableProps}
              p={"2rem"}
            >
              {steps.map((step, index) => (
                <>
                  <Draggable
                    key={step.step_title}
                    draggableId={step.step_title}
                    index={index}
                  >
                    {(provided, snapshot) => (
                      <>
                        <Box
```

```
  ref={provided.innerRef}
  {...provided.draggableProps}
  {...provided.dragHandleProps}
  mt={8}
  bgColor={"gray.900"}
  w={"100%"}
  minH={"10vh"}
  rounded={"xl"}
  p={"1.2rem"}
>
  <Badge
    color={
      step.step_type ===
      "MULTIPLE_CHOICE"
        ? "green.400"
        : step.step_type === "TEXT"
        ? "blue.400"
        : step.step_type ===
          "CODE_COMPARE"
        ? "purple.400"
        : "yellow.400"
    }
    ml={2}
    mt={2}
  >
    {step.step_type ===
    "MULTIPLE_CHOICE"
      ? "Multiple Choice"
      : step.step_type === "TEXT"
      ? "Text"
      : step.step_type ===
        "CODE_COMPARE"
      ? "Code Vergleich"
      : "Code Check"}
  </Badge>
  <Flex
    w={"100%"}
    alignItems={"center"}
    justifyContent={"space-between"}
    direction={"row"}
  >
    <Heading
      size={"md"}
      textAlign={"center"}
      color={"gray.200"}
      ml={2}
    >
      {step.step_title}
    </Heading>
    <IconButton
      aria-label="Delete Step"
      icon={<FaTrash />}
      colorScheme="red"
      onClick={() => {
        setSteps(
          steps.filter(
            (s) =>
              s.step_title !==
              step.step_title
          )
```

```
      );
    }}
  />
</Flex>
{step.step_type ===
"MULTIPLE_CHOICE" ? (
  <>
    <Box
      display={
        hidePlusButton
          ? "none"
          : "inherit"
      }
    >
      <FormControl>
        <FormLabel>Frage</FormLabel>
        <Input
          defaultValue={
            step.multiple_choice
              ?.question || ""
          }
          onChange={(e) => {
            // @ts-ignore
            setSteps(
              // @ts-ignore
              steps.map((s) =>
                s.step_title ===
                step.step_title
                  ? {
                      ...s,
                      multiple_choice:
                        {
                          ...s.multiple_choice,
                          question:
                            e.target
                              .value,
                        },
                    }
                  : s
              )
            );
          }}
        />
      </FormControl>
      <Grid
        templateColumns={
          "repeat(2, 1fr)"
        }
        gap={6}
        mt={4}
      >
        {(
          step.multiple_choice
            ?.answers || []
        ).map((answer, index) => (
          <>
            <FormControl>
              <FormLabel>
                Antwort {index + 1}
              </FormLabel>
```

```
<Input
  defaultValue={
    answer.answer
  }
  onChange={(e) => {
    setSteps(
      // @ts-ignore
      steps.map((s) =>
        s.step_title ===
        step.step_title
          ? {
              ...s,
              multiple_choice:
                {
                  ...s.multiple_choice,
                  answers:
                    s.multiple_choice?.answers.map(
                      (
                        a,
                        i
                      ) =>
                        i ===
                        index
                          ? {
                              ...a,
                              answer:
                                e
                                  .target
                                  .value,
                            }
                          : a
                    ),
                },
            }
          : s
      )
    );
  }}
/>
</FormControl>
<FormControl>
  <FormLabel>
    Ist die Antwort
    richtig?
  </FormLabel>
  <Switch
    isChecked={
      answer.correct
    }
    colorScheme={"brand"}
    onChange={() => {
      setSteps(
        // @ts-ignore
        steps.map((s) =>
          s.step_title ===
          step.step_title
            ? {
                ...s,
                multiple_choice:
                  {
```

```
                                ...s.multiple_choice,
                                answers:
                                  s.multiple_choice?.answers.map(
                                    (
                                      a,
                                      i
                                    ) =>
                                      i ===
                                      index
                                        ? {
                                            ...a,
                                            correct:
                                              !a.correct,
                                          }
                                        : a
                                  ),
                              },
                            }
                          : s
                      )
                    );
                  }}
                />
              </FormControl>
            </>
          ))}
        </Grid>
      </Box>
    </>
  ) : null}
  {step.step_type === "TEXT" && (
    <>
      <Box
        display={
          hidePlusButton
            ? "none"
            : "inherit"
        }
      >
        <Grid
          templateColumns={
            "repeat(3, 1fr)"
          }
          gap={6}
          mt={4}
        >
          {step.text_parts?.map((t) => (
            <>
              <FormControl mt={1}>
                <FormLabel>
                  {t.type === "INPUT"
                    ? "Eingabefeld"
                    : "Text"}
                </FormLabel>
                <Input
                  defaultValue={t.value}
                  onChange={(e) => {
                    // use indexes instead of id
                    setSteps(
                      steps.map((s) =>
```

```
            s.step_title ===
            step.step_title
              ? {
                  ...s,
                  text_parts:
                    s.text_parts?.map(
                      (
                        t2
                      ) =>
                        t2.id ===
                        t.id
                          ? {
                              ...t2,
                              value:
                                e
                                  .target
                                  .value,
                            }
                          : t2
                    ),
                }
              : s
          )
        );
      }}
    />
  </FormControl>
  <FormControl mt={1}>
    <FormLabel>
      Eingabefeld
    </FormLabel>
    <Switch
      colorScheme={"brand"}
      isChecked={
        t.type === "INPUT"
      }
      onChange={() => {
        setSteps(
          steps.map((s) =>
            s.step_title ===
            step.step_title
              ? {
                  ...s,
                  text_parts:
                    s.text_parts?.map(
                      (
                        t2
                      ) =>
                        t2.value ===
                        t.value
                          ? {
                              ...t2,
                              type:
                                t2.type ===
                                "INPUT"
                                  ? "TEXT"
                                  : "INPUT",
                            }
                          : t2
                    ),
```

```
                  }
                : s
            )
          );
        }}
      />
    </FormControl>
    <FormControl mt={1}>
      <FormLabel>
        Aktion
      </FormLabel>
      <Button
        colorScheme="red"
        onClick={() => {
          setSteps(
            steps.map((s) =>
              s.step_title ===
              step.step_title
                ? {
                    ...s,
                    text_parts:
                      s.text_parts?.filter(
                        (
                          t2
                        ) =>
                          t2.value !==
                          t.value
                      ),
                  }
                : s
            )
          );
        }}
      >
        Löschen
      </Button>
    </FormControl>
  </>
))}
<Button
  variant={"brand"}
  onClick={() => {
    setSteps(
      steps.map((s) =>
        s.step_title ===
        step.step_title
          ? {
              ...s,
              text_parts:
                s.text_parts?.concat(
                  {
                    type: "TEXT",
                    value:
                      "Text " +
                      randomInt(
                        1000,
                        9999
                      ),
                    id: randomInt(
                      1000,
```

```
                        9999
                      ),
                    }
                  ),
                }
              : s
          )
        );
      }}
    >
      Hinzufügen
    </Button>
  </Grid>
 </Box>
</>
)}
{step.step_type ===
 "CODE_COMPARE" && (
 <>
  <Flex
   direction={"row"}
   w={"100%"}
   h={"fit-content"}
   paddingInline={10}
   paddingBlock={5}
   gap={40}
  >
   <Box flex={"100%"}>
    <Image
     src={
      step.code_compare?.output
     }
     alt="Output"
     _hover={{
      opacity: 0.5,
     }}
     onClick={() => {
      // upload an image
      const input =
       document.createElement(
        "input"
       );
      input.type = "file";
      input.accept = "image/*";
      input.onchange = (e) => {
       const base64EncodedImage =
        (
         (e.target as any)
          .files as any
        )[0];
       const reader =
        new FileReader();
       reader.readAsDataURL(
        base64EncodedImage
       );
       reader.onloadend =
        () => {
         setSteps(
          // @ts-ignore
          steps.map((s) =>
```

```
            s.step_title ===
            step.step_title
              ? {
                  ...s,
                  code_compare:
                    {
                      ...s.code_compare,
                      output:
                        reader.result,
                    },
                }
              : s
          )
        );
      };
    };
    input.click();
  }}
/>
<Image
  src={
    step.code_compare?.code_a
      .screenshot
  }
  alt="Code A"
  _hover={{
    opacity: 0.5,
  }}
  onClick={() => {
    // @ts-ignore
    window._temp_screenShot_callback =
      (data: string) => {
        // @ts-ignore
        delete window._temp_screenShot_callback;
        console.log(2);
        setSteps(
          // @ts-ignore
          steps.map((s) =>
            s.step_title ===
            step.step_title
              ? {
                  ...s,
                  code_compare:
                    {
                      ...s.code_compare,
                      code_a:
                        {
                          ...s
                            .code_compare
                            ?.code_a,
                          screenshot:
                            data,
                        },
                    },
                }
              : s
          )
        );
      };
    onOpen();
```

```
    setMonacoEditorContent(
      ""
    );
  }}
/>
<Image
  src={
    step.code_compare?.code_b
      .screenshot
  }
  alt="Code B"
  _hover={{
    opacity: 0.5,
  }}
  onClick={() => {
    // @ts-ignore
    window._temp_screenShot_callback =
      (data: string) => {
        // @ts-ignore
        delete window._temp_screenShot_callback;
        console.log(2);
        setSteps(
          // @ts-ignore
          steps.map((s) =>
            s.step_title ===
            step.step_title
              ? {
                  ...s,
                  code_compare:
                    {
                      ...s.code_compare,
                      code_b:
                        {
                          ...s
                            .code_compare
                            ?.code_b,
                          screenshot:
                            data,
                        },
                    },
                }
              : s
          )
        );
      };
    onOpen();
    setMonacoEditorContent(
      ""
    );
  }}
/>
<ButtonGroup
  mt={2}
  w={"100%"}
>
  <Button
    variant={"brand"}
    w={"100%"}
    maxW={"600px"}
    onClick={() => {
```

```
                setSteps(
                  // @ts-ignore
                  steps.map((s) =>
                    s.step_title ===
                    step.step_title
                      ? {
                          ...s,
                          code_compare:
                            {
                              ...s.code_compare,
                              code_a: {
                                ...s
                                  .code_compare
                                  ?.code_a,
                                correct:
                                  !s
                                    .code_compare
                                    ?.code_a
                                    .correct,
                              },
                              code_b: {
                                ...s
                                  .code_compare
                                  ?.code_b,
                                correct:
                                  !s
                                    .code_compare
                                    ?.code_b
                                    .correct,
                              },
                            }
                        : s
                    )
                  );
                }}
              >
                Richtig: 
                {step.code_compare?.code_a
                  .correct
                  ? "A"
                  : "B"}
              </Button>
            </ButtonGroup>
          </Box>
        </Flex>
      </>
    )}
    {step.step_type === "CODE_CHECK" && (
      <>
        <Flex
          direction={"row"}
          w={"100%"}
          h={"fit-content"}
          paddingInline={10}
          paddingBlock={5}
          gap={40}
        >
          <Box flex={"100%"}>
            <Image
```

```
  src={
    step.code_check
      ?.screenshot
  }
  alt="Code"
  _hover={{
    opacity: 0.5,
  }}
  onClick={() => {
    // @ts-ignore
    window._temp_screenShot_callback =
      (data: string) => {
        // @ts-ignore
        delete window._temp_screenShot_callback;
        console.log(2);
        setSteps(
          // @ts-ignore
          steps.map((s) =>
            s.step_title ===
            step.step_title
              ? {
                  ...s,
                  code_check:
                    {
                      correct:
                        s
                          .code_check
                          ?.correct,
                      screenshot:
                        data,
                    },
                }
              : s
          )
        );
      };
    onOpen();
    setMonacoEditorContent(
      ""
    );
  }}
/>
<ButtonGroup
  mt={2}
  w={"100%"}
>
  <Button
    colorScheme={
      step.code_check?.correct
        ? "green"
        : "red"
    }
    w={"100%"}
    maxW={"600px"}
    onClick={() => {
      setSteps(
        // @ts-ignore
        steps.map((s) =>
          s.step_title ===
          step.step_title
```

```
                                    ? {
                                        ...s,
                                        code_check: {
                                          screenshot:
                                            s
                                              .code_check
                                              ?.screenshot,
                                          correct:
                                            !s
                                              .code_check
                                              ?.correct,
                                        },
                                      }
                                    : s
                                )
                              );
                            }}
                          >
                            {step.code_check?.correct
                              ? "Richtig"
                              : "Falsch"}
                          </Button>
                        </ButtonGroup>
                      </Box>
                    </Flex>
                  </>
                )}
              </Box>
            </>
          )}
        </Draggable>
      </>
    ))}
  </Box>
</>
)}

<CodeUpModal
  onOpen={onOpen}
  onClose={onClose}
  isOpen={isOpen}
  title={"Code Block Erstellen"}
  customSize={["6xl"]}
>
  <Box w={"100%"} minH={"40vh"} h={"fit-content"}>
    {!monacoEditorLanguage && (
      <InputGroup w={"100%"} overflow={"auto"} mb={"2rem"}>
        <InputLeftElement>
          <FaCode />
        </InputLeftElement>
        <Input
          placeholder={"Programmiersprache eingeben..."}
          w={"100%"}
          id={"codeup-code-block-language"}
        />
        <Button
```

```
      ml={2}
      variant={"brand"}
      boxShadow={"2xl"}
      onClick={() => {
        // remove monaco editor if exists
        if (monacoEditorHook) {
          monacoEditorHook.dispose();
        }

        // set language
        setMonacoEditorLanguage(
          (
            document.getElementById(
              "codeup-code-block-language"
            ) as HTMLInputElement
          )?.value || ""
        );
      }}
    >
      Erstellen
    </Button>
  </InputGroup>
)}
{monacoEditorLanguage && (
  <>
    <Box id={"codeup_code_image_creator_monaco"}>
      <Editor
        defaultLanguage={monacoEditorLanguage}
        defaultValue={monacoEditorContent}
        onMount={(editor, monaco) => {
          if (editor && monaco) {
            setMonacoHook(monaco);
            setMonacoEditorHook(editor);
          }
        }}
        onChange={(value, ev) => {
          setMonacoEditorContent(value || "");
        }}
        theme={"vs-dark"}
        height={"40vh"}
        options={{
          fontSize: 20,
          minimap: {
            enabled: false,
          },
          scrollbar: {
            scrollByPage: true,
          },
        }}
      />
    </Box>
  </>
)}
{monacoEditorLanguage && (
  <ButtonGroup w={"100%"} mt={"2rem"}>
    <Button
      leftIcon={<FaArrowsAlt />}
      w={"100%"}
      colorScheme={"red"}
      onClick={() => {
```

```
            setMonacoEditorLanguage(null);
          }}
        >
          Sprache wechseln
        </Button>
        <Button
          variant={"brand"}
          w={"100%"}
          leftIcon={<FaCamera />}
          onClick={async () => {
            const i = await html2canvas(
              document.getElementsByClassName(
                "monaco-editor"
              )[0] as HTMLElement
            );

            // @ts-ignore
            window._temp_screenShot_callback(
              i.toDataURL("image/png")
            );

            onClose();
          }}
        >
          Screenshots erstellen
        </Button>
      </ButtonGroup>
    )}
  </Box>
</CodeUpModal>

<Popover>
  <PopoverTrigger>
    <Flex
      w={"100%"}
      justifyContent={"center"}
      alignItems={"center"}
    >
      <Box
        mt={8}
        mb={8}
        bgColor={"gray.900"}
        w={"100%"}
        maxW={"600px"}
        minH={"10vh"}
        rounded={"xl"}
        cursor={"pointer"}
        display={hidePlusButton ? "none" : "inherit"}
      >
        <Flex
          w={"100%"}
          h={"10vh"}
          alignItems={"center"}
          justifyContent={"center"}
          direction={"column"}
          color={"brand.500"}
        >
          <FaPlus size={64} />
        </Flex>
      </Box>
```

```
  </Flex>
</PopoverTrigger>
<PopoverContent bgColor={"black"}>
  <PopoverArrow bgColor={"black"} />
  <PopoverCloseButton />
  <PopoverHeader>Neuer Schritt</PopoverHeader>
  <PopoverBody>
    <form
      onSubmit={(e) => {
        e.preventDefault();

        const form = e.target as HTMLFormElement;

        const data = new FormData(form);

        const title = data.get("step_title") as string;
        const type = data.get("new_step_step_type") as
          | "MULTIPLE_CHOICE"
          | "TEXT"
          | "CODE_COMPARE"
          | "CODE_CHECK";

        // IF title EXISTS return
        if (steps.find((step) => step.step_title === title))
          return;

        let step = {};

        switch (type) {
          case "MULTIPLE_CHOICE":
            step = new Builders.MultipleChoiceBuilder()
              .setTitle(title)
              .setQuestion("Frage")
              .addAnswer("Antwort 1", true)
              .addAnswer("Antwort 2", false)
              .addAnswer("Antwort 3", false)
              .addAnswer("Antwort 4", false)
              .build();
            break;
          case "TEXT":
            step = new Builders.TextBuilder()
              .setTitle(title)
              .addText("Text " + randomInt(1000, 9999))
              .addInput("Eingabe " + randomInt(1000, 9999))
              .build();

            // @ts-ignore
            step.text_parts = step.text_parts?.map((t) => ({
              ...t,
              id: randomInt(1000, 9999),
            }));
            break;
          case "CODE_COMPARE":
            step = new Builders.CodeCompareBuilder()
              .setTitle(title)
              .setCodeA("Code A", true)
              .setCodeB("Code B", false)
              .setOutput("Ausgabe")
              .build();
            break;
```

```
                    case "CODE_CHECK":
                      step = new Builders.CodeCheckBuilder()
                        .setTitle(title)
                        .setCodeA("Code A", true)
                        .setCodeB("Code B", false)
                        .build();
                      break;
                    default:
                      console.log(type);
                      break;
                  }

                  // @ts-ignore
                  setSteps((prev) => [...prev, step]);

                  console.log(step);
                 }}
                >
                  <Stack spacing={4}>
                    <FormControl isRequired>
                      <FormLabel>Titel</FormLabel>
                      <Input name={"step_title"} placeholder={"Titel"} />
                    </FormControl>
                    <FormControl isRequired>
                      <FormLabel>Typ</FormLabel>
                      <Select name={"new_step_step_type"}>
                        <option value="MULTIPLE_CHOICE">
                          Multiple Choice
                        </option>
                        <option value="TEXT">Text</option>
                        <option value="CODE_COMPARE">
                          Code Vergleich
                        </option>
                        <option value="CODE_CHECK">Code Check</option>
                      </Select>
                    </FormControl>
                    <Button variant={"brand"} type={"submit"}>
                      Hinzufügen
                    </Button>
                  </Stack>
                </form>
              </PopoverBody>
            </PopoverContent>
          </Popover>
        </Box>
      </Flex>
    </Box>
  </Stack>
 </CreatorStudioLayout>
 </>
 );
}

function randomInt(min: number, max: number) {
  return Math.floor(Math.random() * (max - min + 1)) + min;
}
```

## apps/web/src/pages/creator/studio/create/content/[course].tsx

```
/**
 * apps/web/src/pages/creator/studio/create/content/[course].tsx
```

```
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 18.11.2022
 *
 */

import REST from "@codeup/rest";
import { useRouter } from "next/router";
import React, { useEffect, useState } from "react";
import { FaBookOpen, FaGlobe } from "react-icons/fa";
import {
  Box,
  Button,
  CircularProgress,
  Flex,
  FormControl,
  FormLabel,
  Heading,
  Input,
  InputGroup,
  InputLeftAddon,
  useToast,
} from "@chakra-ui/react";
import dynamic from "next/dynamic";
import "easymde/dist/easymde.min.css";
import AdminQuizMaker from "@codeup/components/admin/AdminQuizMaker";
import { Step, Steps, useSteps } from "chakra-ui-steps";
import CreatorStudioLayout from "@codeup/components/creator/studio/CreatorStudioLayout";
import ErrorComponent from "@codeup/components/global/ErrorComponent";
import Builder from "../../../../../special-components/Builder";
import Head from "next/head";
import { CHeading } from "@codeup/components/dist/global/CHeading";

export default function NewContents() {
  const router = useRouter();

  const [displayName, setDisplayName] = React.useState("");
  const [course, setCourse] = React.useState("");
  const [url, setURL] = React.useState("");
  const [textContent, setTextContent] = React.useState("");
  const [videoURL, setVideoURL] = React.useState("");
  const [quizQuestions, setQuizQuestions] = React.useState<
    {
      question: string;
      answers: {
        answer: string;
        correct: boolean;
      }[];
    }[]
  >([]);

  const [steps, setSteps] = useState([
    { label: "Name" },
    { label: "Inhalt" },
    { label: "Quiz" },
    { label: "Video" },
  ]);
```

```
const { nextStep, prevStep, reset, activeStep } = useSteps({
  initialStep: 0,
});

const [loaded, setLoaded] = useState(false);
const [error, setError] = useState(false);
const toast = useToast();

const [custom_event_id, setCustomEventId] = React.useState<string>("");

useEffect(() => {
  if (router.query.course) {
    REST.Creator.getCourses(localStorage.getItem("token") as string).then(
      (res) => {
        if (res.status === 200) {
          if (
            res.payload.courses.find(
              (c: { _id: string }) => c._id === router.query.course
            )
          ) {
            setCourse(router.query.course as string);
            setCustomEventId(randomString());
            setLoaded(true);
          } else {
            setError(true);
          }
        }
      }
    );
  }
}, [router]);

return (
  <>
    <Head>
      <title>Creator Studio - CodeUp</title>
    </Head>
    <CreatorStudioLayout>
      <Box textAlign={"center"} pt={6}>
        <CHeading size={"xl"}>Neuer Inhalt</CHeading>
      </Box>
      <>
        {loaded && !error ? (
          <>
            <Flex flexDir="column" width="100%" p={"5rem"}>
              <Steps
                size={"lg"}
                activeStep={activeStep}
                colorScheme={"brand"}
              >
                {steps.map(({ label }, index) => (
                  <Step label={label} key={label} mb={"2rem"}>
                    {index === 0 ? (
                      <>
                        <FormControl isRequired>
                          <FormLabel>Titel</FormLabel>
                          <InputGroup>
                            <InputLeftAddon>
                              <FaBookOpen />
                            </InputLeftAddon>
```

```
          <Input
            type={"text"}
            name={"title"}
            id={"new_content_title"}
            placeholder={"Titel"}
            required
            onChange={(e) => setDisplayName(e.target.value)}
          />
        </InputGroup>
      </FormControl>
      <FormControl mt={"2rem"} isRequired>
        <FormLabel>URL Freundlicher Name</FormLabel>
        <InputGroup>
          <InputLeftAddon>
            <FaGlobe />
          </InputLeftAddon>
          <Input
            type={"text"}
            name={"friendlyName"}
            id={"new_content_friendlyName"}
            placeholder={"URL Freundlicher Name"}
            required
            onChange={(e) => setURL(e.target.value)}
          />
        </InputGroup>
      </FormControl>
    </>
  ) : index === 1 ? (
    <>
      <Builder custom_event_id={custom_event_id} />
    </>
  ) : index === 2 ? (
    <>
      <AdminQuizMaker
        quizQuestions={quizQuestions}
        setQuizQuestions={setQuizQuestions}
      />
    </>
  ) : index === 3 ? (
    <>
      <FormControl isRequired>
        <FormLabel>Video URL</FormLabel>
        <InputGroup>
          <InputLeftAddon>
            youtube.com/watch?v=
          </InputLeftAddon>
          <Input
            type={"text"}
            name={"video"}
            id={"new_content_video"}
            placeholder={"Video URL"}
            required
            onChange={(e) => {
              setVideoURL(
                "https://youtube.com/watch?v=" +
                  e.target.value
              );
            }}
          />
        </InputGroup>
```

```
          </FormControl>
        </>
      ) : (
        <></>
      )}
    </Step>
  ))}
</Steps>
{activeStep === steps.length ? (
  <Flex px={4} py={4} width="100%" flexDirection="column">
    <Heading fontSize="xl" textAlign="center">
      Woohoo! All steps completed!
    </Heading>
    <Button mx="auto" mt={6} size="sm" onClick={reset}>
      Zurücksetzen
    </Button>
  </Flex>
) : (
  <Flex width="100%" justify="flex-end" mt={"2rem"}>
    {activeStep === 1 || activeStep === 3 ? (
      <>
        <Button
          mr={4}
          size="lg"
          onClick={() => {
            if (activeStep === 1) {
              setTextContent("__codeup.__no_text_content");
              nextStep();
            } else if (activeStep === 3) {
              if (
                textContent === "__codeup.__no_text_content"
              ) {
                alert(
                  "Du musst mindestens eine der folgenden Sachen ausfüllen: Text, Video"
                );
                return;
              } else {
                (
                  document.querySelector(
                    ".new_content_submit"
                  ) as HTMLButtonElement
                ).click();
              }
            }
          }}
          colorScheme={"red"}
        >
          Überspringen
        </Button>
      </>
    ) : null}
    <Button
      mr={4}
      onClick={() => {
        if (activeStep === 0) return;
        prevStep();
      }}
      size="lg"
    >
      Vorheriger
```

```
</Button>
<Button
  size="lg"
  id={"new_content_submit"}
  variant={"brand"}
  onClick={async () => {
    if (activeStep !== steps.length - 1) {
      switch (activeStep) {
        case 0:
          if (displayName === "" || url === "") {
            return;
          }
          nextStep();
          break;
        case 1:
          window.dispatchEvent(
            new CustomEvent(
              "codeup::webbuilder::export_" +
                custom_event_id
            )
          );

          const mergedCode =
            // @ts-ignore
            window.codeup.webBuilderHTMLExport;

          setTextContent(mergedCode);

          // @ts-ignore
          delete window.codeup.webBuilderHTMLExport;

          nextStep();
          break;
        case 2:
          let anyAnswerEmpty = false;

          for (const q of quizQuestions) {
            for (const a of q.answers) {
              if (a.answer === "") {
                anyAnswerEmpty = true;
              }
            }
          }

          if (anyAnswerEmpty) return;

          nextStep();
          break;
        default:
          break;
      }
    } else {
      const res = await REST.Creator.createContent({
        token: localStorage.getItem("token") as string,
        course: course,
        displayName,
        quiz: quizQuestions,
        textContent:
          textContent !== "__codeup.__no_text_content"
            ? textContent
```

```
                    : "",
                  type: "TEXT",
                  videoUrl: videoURL || "",
                  url,
                });

                if (res.status === 200) {
                  router.push("/creator/studio/" + course);
                } else {
                  console.log(res.payload);
                  toast({
                    title: "Fehler",
                    description:
                      "Fehler beim Erstellen: " + res.payload.error,
                    status: "error",
                    duration: 1500,
                    isClosable: true,
                  });
                }
              }
            }}
          >
            {activeStep === steps.length - 1
              ? "Fertigstellen"
              : "Nächster"}
          </Button>
        </Flex>
      )}
    </Flex>
  </>
) : (
  <>
    {!loaded && !error && (
      <Flex
        w={"100%"}
        h={"100vh"}
        justifyContent={"center"}
        alignItems={"center"}
        direction={"column"}
      >
        <CircularProgress
          size={"80px"}
          isIndeterminate
          color="purple.300"
        />
        <Heading size={"lg"} mt={4}>
          Lade Kurs...
        </Heading>
      </Flex>
    )}
  </>
)}
{error && (
  <>
    <ErrorComponent description={"Fehler beim Laden der Kurse."} />
  </>
)}
</>
</CreatorStudioLayout>
</>
```

```
  );
}

function randomString(length: number = 32) {
  let result = "";
  const characters =
    "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789";
  const charactersLength = characters.length;
  for (let i = 0; i < length; i++)
    result += characters.charAt(Math.floor(Math.random() * charactersLength));
  return result;
}
```

## apps/web/src/pages/creator/studio/create/course.tsx

```
/**
 * apps/web/src/pages/admin/contents/courses/new.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 30.10.2022
 *
 */

import * as React from "react";
import { useEffect, useState } from "react";
import AdminLayout from "@codeup/components/admin/AdminLayout";
import AdminContentsLayout from "@codeup/components/admin/AdminContentsLayout";
import {
  Box,
  Button,
  Flex,
  FormControl,
  FormLabel,
  Grid,
  GridItem,
  Heading,
  Image,
  Input,
  InputGroup,
  InputLeftAddon,
  Textarea,
  useToast,
} from "@chakra-ui/react";
import { FaBookOpen, FaGlobe } from "react-icons/fa";
import REST from "@codeup/rest";
import { useRouter } from "next/router";
import dynamic from "next/dynamic";
import "easymde/dist/easymde.min.css";
import AdminQuizMaker from "@codeup/components/admin/AdminQuizMaker";
import { Step, Steps, useSteps } from "chakra-ui-steps";
import CreatorStudioLayout from "@codeup/components/creator/studio/CreatorStudioLayout";
import Head from "next/head";

const SimpleMdeReact = dynamic(import("react-simplemde-editor"), {
  ssr: false,
});

export default function NewComponent() {
  const { nextStep, prevStep, reset, activeStep } = useSteps({
```

```
  initialStep: 0,
});
const steps = [
  { label: "Name" },
  { label: "Beschreibung" },
  { label: "Vorschaubild" },
];

const router = useRouter();

const [quizQuestions, setQuizQuestions] = useState<
  {
    question: string;
    answers: {
      answer: string;
      correct: boolean;
    }[];
  }[]
>([]);

const [title, setTitle] = useState<string>("");
const [friendlyName, setFriendlyName] = useState<string>("");
const [splashImage, setSplashImage] = useState<File>();
const [description, setDescription] = useState<string>("");

const toast = useToast();

return (
  <>
    <Head>
      <title>Neuer Kurs - Creator Studio - CodeUp</title>
    </Head>
    <CreatorStudioLayout>
      <Flex flexDir="column" width="100%" p={"5rem"}>
        <Heading size="lg" mb={"2rem"} textAlign={"left"}>
          Neuer Kurs
        </Heading>
        <Steps size={"lg"} activeStep={activeStep} colorScheme={"brand"}>
          {steps.map(({ label }, index) => (
            <Step label={label} key={label} mb={"2rem"}>
              {index === 0 ? (
                <>
                  <FormControl isRequired>
                    <FormLabel>Titel</FormLabel>
                    <InputGroup>
                      <InputLeftAddon>
                        <FaBookOpen />
                      </InputLeftAddon>
                      <Input
                        type={"text"}
                        name={"title"}
                        id={"new_video_title"}
                        placeholder={"Titel"}
                        defaultValue={title}
                        required
                        onChange={(e) => setTitle(e.target.value)}
                      />
                    </InputGroup>
                  </FormControl>
                  <FormControl mt={"2rem"} isRequired>
```

```
      <FormLabel>URL Freundlicher Name</FormLabel>
      <InputGroup>
        <InputLeftAddon>
          <FaGlobe />
        </InputLeftAddon>
        <Input
          type={"text"}
          name={"friendlyName"}
          id={"new_video_friendlyName"}
          placeholder={"URL Freundlicher Name"}
          defaultValue={friendlyName}
          required
          onChange={(e) =>
            setFriendlyName(
              e.target.value
                .toLowerCase()
                .replace(" ", "-")
                .replace("ä", "ae")
                .replace("ö", "oe")
                .replace("ü", "ue")
                .replace("ß", "ss")
            )
          }
        />
      </InputGroup>
    </FormControl>
  </>
) : index === 1 ? (
  <>
    <FormControl mt={"2rem"} isRequired>
      <FormLabel>Beschreibung</FormLabel>
      <Textarea
        name={"description"}
        placeholder={"Beschreibung"}
        defaultValue={description}
        required
        onChange={(e) => setDescription(e.target.value)}
      />
    </FormControl>
  </>
) : index === 2 ? (
  <>
    <FormControl mt={"2rem"} isRequired>
      <FormLabel>Vorschaubild</FormLabel>
      <Input
        required
        type={"file"}
        name={"splashImage"}
        onChange={(e) => {
          // @ts-ignore
          setSplashImage(e.target.files[0] as File);
        }}
        // only svg images
        accept={"image/svg+xml"}
      />
    </FormControl>
  </>
) : (
  <></>
)}
```

```
    </Step>
  ))}
</Steps>
{activeStep === steps.length ? (
  <Flex px={4} py={4} width="100%" flexDirection="column">
    <Heading fontSize="xl" textAlign="center">
      Woohoo! All steps completed!
    </Heading>
    <Button mx="auto" mt={6} size="sm" onClick={reset}>
      Zurücksetzen
    </Button>
  </Flex>
) : (
  <Flex width="100%" justify="flex-end" mt={"2rem"}>
    <Button
      colorScheme={"red"}
      mr={4}
      onClick={() => {
        if (activeStep === 0) {
          router.push("/creator/studio");
        } else {
          prevStep();
        }
      }}
      size="lg"
    >
      {activeStep === 0 ? "Abbrechen" : "Vorheriger"}
    </Button>
    <Button
      size="lg"
      variant={"brand"}
      onClick={async () => {
        if (activeStep !== steps.length - 1) {
          switch (activeStep) {
            case 0:
              if (title === "" || friendlyName === "") return;
              break;
            case 1:
              if (description === "") return;
              break;
            case 2:
              if (!splashImage) return;
              break;
            default:
              break;
          }
          nextStep();
          return;
        }

        if (!splashImage) {
          return;
        }

        const base64 = await new Promise<string>((resolve) => {
          const reader = new FileReader();
          reader.readAsDataURL(splashImage);
          reader.onload = () => resolve(reader.result as string);
          reader.onerror = (error) => console.log(error);
        });
```

```
                const res = await REST.Creator.createCourse({
                  token: localStorage.getItem("token") as string,
                  name: title,
                  friendlyName,
                  splashImage: base64,
                  description,
                });

                if (res.status === 200) {
                  router.push("/creator/studio");
                } else {
                  toast({
                    title: "Fehler",
                    description:
                      "Ein Fehler ist aufgetreten: " + res.payload.error,
                    status: "error",
                    duration: 1500,
                    isClosable: true,
                  });
                }
              }}
            >
              {activeStep === steps.length - 1 ? "Fertigstellen" : "Nächster"}
            </Button>
          </Flex>
        )}
      </Flex>
    </CreatorStudioLayout>
  </>
 );
}
```

## apps/web/src/pages/creator/studio/edit/[contentId]/info.tsx

```
/**
 * apps/web/src/pages/creator/studio/edit/[contentId]/info.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 22.11.2022
 *
 */

import * as React from "react";
import { useRouter } from "next/router";
import {
  Box,
  Button,
  ButtonGroup,
  Flex,
  FormControl,
  FormLabel,
  Heading,
  Input,
  InputGroup,
  InputLeftElement,
  Stack,
  useToast,
} from "@chakra-ui/react";
```

```
import { useEffect } from "react";
import REST from "@codeup/rest";
import CreatorStudioLayout from "@codeup/components/creator/studio/CreatorStudioLayout";
import { FaPen } from "react-icons/fa";
import AdminQuizMaker from "@codeup/components/admin/AdminQuizMaker";
import Head from "next/head";
import { CHeading } from "@codeup/components/dist/global/CHeading";

export default function Info() {
  const router = useRouter();

  const { contentId } = router.query;

  const [content, setContent] = React.useState<{
    _id: string;
    displayName: string;
    course: string;
    url: string;
    textContent: string;
    videoUrl: string;
    quiz: {
      question: string;
      answers: {
        answer: string;
        correct: boolean;
      }[];
    };
  }>();

  const toast = useToast();

  const [quiz, setQuiz] = React.useState<
    {
      question: string;
      answers: {
        answer: string;
        correct: boolean;
      }[];
    }[]
  >([]);

  useEffect(() => {
    if (contentId) {
      REST.Creator.getContents({
        token: localStorage.getItem("token") as string,
        contentId: contentId as string,
      }).then((res) => {
        if (res.status === 200) {
          setContent(res.payload.content);
          setQuiz(res.payload.content.quiz);
        } else {
          toast({
            title: "Fehler",
            description: "Fehler beim Laden: " + res.payload.error,
            status: "error",
            duration: 1500,
            isClosable: true,
            position: "bottom-left",
          });
        }
```

```
    });
  }
}, [contentId]);

return (
  <>
    <Head>
      <title>Inhalt Editor - Creator Studio - CodeUp</title>
    </Head>
    <CreatorStudioLayout>
      <Box textAlign={"center"} pt={"2rem"}>
        <CHeading size={"xl"}>Bearbeiten</CHeading>
      </Box>
      {content ? (
        <>
          <Flex
            flexDir="column"
            width="100%"
            p={"5rem"}
            minH={"100vh"}
            alignItems={"center"}
          >
            <Flex flexDir="column" width={["100", "35%"]}>
              <Stack spacing={4} width={"100%"}>
                <FormControl id="displayName">
                  <FormLabel>Anzeigename</FormLabel>
                  <InputGroup>
                    <InputLeftElement>
                      <FaPen />
                    </InputLeftElement>
                    <Input
                      name={"displayName"}
                      defaultValue={content.displayName}
                      placeholder={"Name"}
                    />
                  </InputGroup>
                </FormControl>
                <FormControl id="url">
                  <FormLabel>URL</FormLabel>
                  <InputGroup>
                    <InputLeftElement>
                      <FaPen />
                    </InputLeftElement>
                    <Input
                      name={"url"}
                      defaultValue={content.url}
                      placeholder={"URL"}
                    />
                  </InputGroup>
                </FormControl>
                <FormControl id="url">
                  <FormLabel>Video-URL</FormLabel>
                  <InputGroup>
                    <InputLeftElement>
                      <FaPen />
                    </InputLeftElement>
                    <Input
                      name={"video-url"}
                      defaultValue={content.videoUrl}
                      placeholder={"Video-URL"}
```

```
        />
      </InputGroup>
    </FormControl>
    <AdminQuizMaker
      quizQuestions={quiz}
      setQuizQuestions={setQuiz}
    />
  </Stack>
</Flex>
<ButtonGroup
  mt={"2rem"}
  w={"100%"}
  alignItems={"center"}
  justifyContent={"center"}
>
  <Button
    size={"lg"}
    colorScheme={"red"}
    onClick={() => {
      router.back();
    }}
  >
    Abbrechen
  </Button>
  <Button
    size={"lg"}
    variant={"brand"}
    onClick={async () => {
      const displayName = document.querySelector(
        "input[name=displayName]"
      ) as HTMLInputElement;
      const url = document.querySelector(
        "input[name=url]"
      ) as HTMLInputElement;
      const videoUrl = document.querySelector(
        "input[name=video-url]"
      ) as HTMLInputElement;

      const res = await REST.Creator.updateContent({
        token: localStorage.getItem("token") as string,
        cid: content._id,
        update: {
          displayName: displayName.value || content.displayName,
          url: url.value || content.url,
          quiz: quiz || content.quiz,
          videoUrl: videoUrl.value || content.videoUrl,
        },
      });

      if (res.status === 200) {
        router.back();
      } else {
        toast({
          title: "Fehler",
          description:
            "Fehler beim Speichern: " + res.payload.error,
          status: "error",
          duration: 1500,
          isClosable: true,
          position: "bottom-left",
```

```
                  });
                }
              }}
            >
              Speichern
            </Button>
          </ButtonGroup>
        </Flex>
      </>
    ) : null}
  </CreatorStudioLayout>
 </>
 );
}
```

## apps/web/src/pages/creator/studio/edit/[contentId].tsx

```
/**
 * apps/web/src/pages/creator/studio/edit/[contentId].tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 22.11.2022
 *
 */

import * as React from "react";

import { useRouter } from "next/router";
import { useEffect } from "react";
import REST from "@codeup/rest";
import CreatorStudioLayout from "@codeup/components/creator/studio/CreatorStudioLayout";
import {
  Box,
  Button,
  ButtonGroup,
  Flex,
  Heading,
  useToast,
} from "@chakra-ui/react";
import Builder from "../../../../special-components/Builder";
import Head from "next/head";
import { CHeading } from "@codeup/components/dist/global/CHeading";

export default function ContentEditor() {
  const router = useRouter();

  const { contentId } = router.query;

  const [content, setContent] = React.useState<{
    _id: string;
    displayName: string;
    course: string;
    url: string;
    textContent: string;
    quiz: {
      question: string;
      answers: {
        answer: string;
        correct: boolean;
```

```
    }[];
  };
}>();

const toast = useToast();

const [custom_event_id, setCustomEventId] = React.useState<string>("");

useEffect(() => {
  if (contentId) {
    REST.Creator.getContents({
      token: localStorage.getItem("token") as string,
      contentId: contentId as string,
    }).then((res) => {
      if (res.status === 200) {
        setContent(res.payload.content);
        setCustomEventId(randomString());
      } else {
        toast({
          title: "Fehler",
          description: "Fehler beim Laden: " + res.payload.error,
          status: "error",
          duration: 1500,
          isClosable: true,
          position: "bottom-left",
        });
      }
    });
  }
}, [router, contentId]);

return (
  <>
    <Head>
      <title>Inhalt Editor - Creator Studio - CodeUp</title>
    </Head>
    <CreatorStudioLayout>
      <Box textAlign={"center"} pt={"2rem"}>
        <CHeading>Inhalt bearbeiten</CHeading>
      </Box>
      {content ? (
        <>
          <Flex flexDir="column" width="100%" p={"5rem"} minH={"100vh"}>
            <Builder
              content={content.textContent}
              custom_event_id={custom_event_id}
            />
            <ButtonGroup
              mt={"2rem"}
              w={"100%"}
              alignItems={"center"}
              justifyContent={"center"}
            >
              <Button
                size={"lg"}
                colorScheme={"red"}
                onClick={async () => {
                  const res = await REST.Creator.updateContent({
                    token: localStorage.getItem("token") as string,
                    cid: content._id,
```

```
      update: {
        textContent: "__codeup.__no_text_content",
      },
    });
    if (res.status === 200) {
      router.back();
    } else {
      toast({
        title: "Fehler",
        description:
          "Fehler beim Speichern: " + res.payload.error,
        status: "error",
        duration: 1500,
        isClosable: true,
        position: "bottom-left",
      });
    }
  }}
>
  Inhalt Löschen
</Button>
<Button
  size={"lg"}
  colorScheme={"red"}
  onClick={() => {
    router.back();
  }}
>
  Abbrechen
</Button>
<Button
  size={"lg"}
  variant={"brand"}
  onClick={async () => {
    window.dispatchEvent(
      new CustomEvent(
        "codeup::webbuilder::export_" + custom_event_id
      )
    );

    const mergedCode =
      // @ts-ignore
      window.codeup.webBuilderHTMLExport;

    // @ts-ignore
    delete window.codeup.webBuilderHTMLExport;

    const res = await REST.Creator.updateContent({
      token: localStorage.getItem("token") as string,
      cid: content._id,
      update: {
        textContent: mergedCode,
      },
    });

    if (res.status === 200) {
      router.back();
    } else {
      toast({
        title: "Fehler",
```

```
                    description:
                      "Fehler beim Speichern: " + res.payload.error,
                    status: "error",
                    duration: 1500,
                    isClosable: true,
                    position: "bottom-left",
                  });
                }
              }}
            >
              Speichern
            </Button>
          </ButtonGroup>
        </Flex>
      </>
    ) : null}
  </CreatorStudioLayout>
 </>
);
}

function randomString(length: number = 32) {
 let result = "";
 const characters =
  "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789";
 const charactersLength = characters.length;
 for (let i = 0; i < length; i++)
  result += characters.charAt(Math.floor(Math.random() * charactersLength));
 return result;
}
```

## apps/web/src/pages/creator/studio/index.tsx

```
/**
 * apps/web/src/pages/creator/studio/index.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 18.11.2022
 *
 */

import * as React from "react";
import { useEffect } from "react";
import CreatorStudioLayout from "@codeup/components/creator/studio/CreatorStudioLayout";
import {
 Badge,
 Box,
 Button,
 ButtonGroup,
 Flex,
 Grid,
 GridItem,
 Heading,
 Image,
 Input,
 InputGroup,
 InputLeftElement,
 useDisclosure,
 useToast,
```

```
} from "@chakra-ui/react";
import {
  FaCertificate,
  FaEye,
  FaEyeSlash,
  FaLock,
  FaPen,
  FaPlus,
  FaSearch,
  FaTrash,
} from "react-icons/fa";
import REST from "@codeup/rest";
import { useRouter } from "next/router";
import CourseEditor from "@codeup/components/creator/studio/CourseEditor";
import Head from "next/head";
import { CHeading } from "@codeup/components/dist/global/CHeading";
import GridLayoutV2 from "@codeup/components/global/GridLayoutV2";
import PopupManager from "../../../global/PopupManager";

export default function Index() {
  const [query, setQuery] = React.useState("");

  const {
    onOpen: onCourseEditorOpen,
    onClose: onCourseEditorClose,
    isOpen: isCourseEditorOpen,
  } = useDisclosure();

  const [courses, setCourses] = React.useState<
    {
      __v: number;
      _id: string;
      available: string;
      contentPositions: string[];
      createdAt: string;
      creator: string;
      description: string;
      friendlyName: string;
      name: string;
      splashImage: string;
    }[]
  >([]);

  const [currentCourse, setCurrentCourse] = React.useState<{
    __v: number;
    _id: string;
    available: string;
    contentPositions: string[];
    createdAt: string;
    creator: string;
    description: string;
    friendlyName: string;
    name: string;
    splashImage: string;
  }>();

  useEffect(() => {
    reload();
  }, []);
```

```
const reload = () => {
  REST.Creator.getCourses(localStorage.getItem("token") as string).then(
    (res) => {
      if (res.status === 200) {
        setCourses(res.payload.courses);
      } else {
        toast({
          title: "Fehler",
          description:
            "Kurse konnten nicht geladen werden: " + res.payload.error,
          status: "error",
          duration: 1500,
          isClosable: true,
        });
      }
    }
  );
};

const router = useRouter();
const toast = useToast();

return (
  <>
    <Head>
      <title>Creator Studio - CodeUp</title>
    </Head>
    <CreatorStudioLayout>
      <Flex
        w={"100%"}
        h={"fit-content"}
        direction={"column"}
        bg={"#000"}
        minH={"100vh"}
      >
        <GridLayoutV2
          pageTitle={"Creator Studio"}
          search={{
            show: true,
            fields: true,
          }}
          topButtons={[
            {
              text: "Neu",
              leftIcon: <FaPlus />,
              variant: "brand",
              onClick: async () => {
                router.push("/creator/studio/create/course");
              },
            },
          ]}
          entries={courses.map((course) => {
            return {
              title: course.name,
              image: course.splashImage,
              fields: [
                {
                  icon:
                    course.available === "public" ? (
                      <FaEye />
```

```
      ) : course.available === "private" ? (
        <FaEyeSlash />
      ) : (
        <FaLock />
      ),
    text:
      course.available === "public"
        ? "Öffentlich"
        : course.available === "private"
        ? "Privat"
        : "Organisation",
  },
],
buttonRows: [
  [
    {
      text: "Bearbeiten",
      variant: "brand",
      leftIcon: <FaPen />,
      onClick: async () => {
        setCurrentCourse(course);
        onCourseEditorOpen();
      },
    },
    {
      text: "Löschen",
      colorScheme: "red",
      leftIcon: <FaTrash />,
      onClick: async () => {
        if (
          await PopupManager.confirmAsync({
            title: "Kurs löschen",
            question: "Willst du diesen Kurs wirklich löschen?",
          })
        ) {
          const res = await REST.Creator.deleteCourse({
            token: localStorage.getItem("token") as string,
            id: course._id,
          });

          if (res.status === 200) {
            toast({
              title: "Kurs gelöscht",
              description:
                "Der Kurs wurde erfolgreich gelöscht",
              status: "success",
              duration: 1500,
              isClosable: true,
            });

            setCourses(
              courses.filter((c) => c._id !== course._id)
            );
          } else {
            toast({
              title: "Fehler",
              description:
                "Löschen fehlgeschlagen: " + res.payload.error,
              status: "error",
              duration: 1500,
```

```
                    isClosable: true,
                  });
                }
              }
            },
          },
        ],
        [
          {
            text: "Lektionen",
            variant: "brand",
            leftIcon: <FaEye />,
            onClick: () => {
              router.push("/creator/studio/" + course._id);
            },
          },
          {
            text: "Zertifikat",
            variant: "brand",
            leftIcon: <FaCertificate />,
            onClick: () => {
              router.push(
                "/creator/studio/certificate/" + course._id
              );
            },
          },
        ],
      ],
    };
  })}
/>
{courses &&
  courses.length !== 0 &&
  courses.filter((c) =>
    c.name.toLowerCase().includes(query.toLowerCase())
  ).length === 0 && (
    <>
      <Heading
        size={"md"}
        color={"gray.400"}
        fontWeight={"normal"}
        textAlign={"center"}
      >
        Keine Kurse gefunden
      </Heading>
    </>
  )}
{courses && courses.length === 0 && (
  <>
    <Heading
      size={"md"}
      color={"gray.400"}
      fontWeight={"normal"}
      textAlign={"center"}
    >
      Du hast noch keine Kurse erstellt
    </Heading>
  </>
)}
</Flex>
```

```
      </CreatorStudioLayout>
      <CourseEditor
        course={currentCourse ? currentCourse : null}
        reload={reload}
        onOpen={onCourseEditorOpen}
        onClose={onCourseEditorClose}
        isOpen={isCourseEditorOpen}
      />
    </>
  );
}
```

## apps/web/src/pages/creator/studio/[id].tsx

```
/**
 * apps/web/src/pages/creator/studio/[id].tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 18.11.2022
 *
 */

import * as React from "react";
import { useRouter } from "next/router";
import REST from "@codeup/rest";
import {
  Badge,
  Box,
  Button,
  ButtonGroup,
  Flex,
  Heading,
  HStack,
  IconButton,
  Stack,
  Text,
  useToast,
} from "@chakra-ui/react";
import AdminLayout from "@codeup/components/admin/AdminLayout";
import AdminContentsLayout from "@codeup/components/admin/AdminContentsLayout";
import { DragDropContext, Draggable, Droppable } from "@hello-pangea/dnd";
import {
  FaEye,
  FaFile,
  FaHome,
  FaPen,
  FaTrash,
  FaYoutube,
} from "react-icons/fa";
import CreatorStudioLayout from "@codeup/components/creator/studio/CreatorStudioLayout";
import course from "../../api/course/[course]";
import ErrorComponent from "@codeup/components/global/ErrorComponent";
import Head from "next/head";

export default function CourseViewer() {
  const router = useRouter();

  const { id } = router.query;
```

```
const [courseName, setCourseName] = React.useState<string[]>([
  "Inhalte Laden...",
]);

const [_rawCourseObject, _setRawCourseObject] = React.useState<any>(null);

const [contentPositions, setContentPositions] = React.useState<string[]>([]);

const [contents, setContents] = React.useState<
  {
    _id: string;
    displayName: string;
    course: string;
    url: string;
    createdAt: string;
    type: "TEXT" | "VIDEO";
    textContent: string;
    quiz: {
      question: string;
      answers: {
        answer: string;
        correct: boolean;
      };
    }[];
    videoUrl?: string;
  }[]
>([]);

React.useEffect(() => {
  if (id && id !== "") {
    REST.Creator.getCourses(localStorage.getItem("token") as string).then(
      (res) => {
        if (res.status === 200) {
          const course = res.payload.courses.filter(
            (c: any) => c._id === id
          )[0];
          setCourseName([course.name, course.friendlyName]);
          setContentPositions(course.contentPositions);
          _setRawCourseObject(course);
        } else {
          console.error(res);
          alert("Fehler beim Laden der Inhalte. Details siehe Konsole.");
        }
      }
    );
  }
}, [id]);

React.useEffect(() => {
  reloadContents();
}, [contentPositions]);

const reloadContents = () => {
  REST.Creator.getContents({
    token: localStorage.getItem("token") as string,
    courseId: id as string,
  }).then((res) => {
    if (res.status === 200) {
      setContents(res.payload.contents);
```

```
      const sortedContents = contentPositions.map((id) => {
        return res.payload.contents.filter((c: any) => c._id === id)[0];
      });

      const missingContents = res.payload.contents.filter(
        (c: any) => !contentPositions.includes(c._id)
      );

      let x: any = [];

      if (sortedContents && missingContents) {
        x = [...sortedContents, ...missingContents];
      } else if (sortedContents) {
        x = [...sortedContents];
      } else if (missingContents) {
        x = [...missingContents];
      }

      x = x.filter((c: any) => c !== undefined);

      setItems(x);
    } else {
      console.error(res);
      toast({
        title: "Fehler beim Laden der Inhalte",
        description: "Details siehe Konsole",
        status: "error",
        duration: 1500,
        isClosable: true,
      });
    }
  });
};

const toast = useToast();

const [items, setItems] = React.useState<any[]>([]);

return (
  <>
    <Head>
      <title>Kurs Editor - Creator Studio - CodeUp</title>
    </Head>
    <CreatorStudioLayout>
      <Flex p={"2rem"} gap={6} alignItems={"center"}>
        <IconButton
          aria-label={"Zurück"}
          onClick={() => router.push("/creator/studio")}
          icon={<FaHome />}
        />

        <Heading size={"lg"}>{courseName[0]}</Heading>

        <Button
          variant={"brand"}
          ml={2}
          onClick={() => {
            router.push("/creator/studio/create/content/" + id);
          }}
        >
```

```
    Neu
  </Button>
</Flex>
{contents.length === 0 ? (
  <ErrorComponent
    description={
      "Keine Inhalte vorhanden. Erstelle Inhalte, damit sie hier angezeigt werden."
    }
    title={"Keine Inhalte vorhanden"}
  />
) : null}
<Flex alignItems={"center"} p={6} w={"100%"} justifyContent={"center"}>
  <Box maxW={["100%", "50%"]} w={"100%"} id={"_tmp.up"}>
    <DragDropContext
      onDragEnd={(result) => {
        if (!result.destination) return;

        const items0 = Array.from(items);

        const [reorderedItem] = items0.splice(result.source.index, 1);
        items0.splice(result.destination.index, 0, reorderedItem);

        setItems(items0);

        REST.Creator.updateCourse({
          token: localStorage.getItem("token") as string,
          id: id as string,
          update: {
            // @ts-ignore
            contentPositions: items0.map((i) => i._id),
          },
        });
      }}
    >
      <Droppable droppableId={"contents"}>
        {(provided, snapshopt) => (
          <>
            <Box
              ref={provided.innerRef}
              {...provided.droppableProps}
              p={"2rem"}
            >
              {items.map((content, index) => (
                <>
                  {content !== undefined ? (
                    <>
                      <Draggable
                        key={content._id}
                        draggableId={content._id}
                        index={index}
                      >
                        {(provided, snapshot) => (
                          <>
                            <Box
                              ref={provided.innerRef}
                              {...provided.draggableProps}
                              {...provided.dragHandleProps}
                              bg={"gray.900"}
                              color={"white"}
                              rounded={"lg"}
```

```
  p={6}
  mt={6}
  borderWidth="1px"
  borderRadius="lg"
  _hover={{
    borderColor: "brand.500",
  }}
  cursor={"pointer"}
>
  <Flex
    justifyContent={"space-between"}
    direction={"column"}
    gap={2}
  >
    <Flex
      direction={"row"}
      w={"100%"}
      gap={8}
    >
      <Flex
        direction={"column"}
        flex="50%"
        gap={4}
      >
        <Heading size={"md"}>
          {content.displayName}
        </Heading>
        <Text
          size={"sm"}
          color={"gray.400"}
        >
          /course/{courseName[1]}/
          {content.url}
        </Text>
      </Flex>
      <Box flex={"50%"}>
        <Stack>
          <Button
            w={"100%"}
            variant={"brand"}
            leftIcon={<FaEye />}
            onClick={() => {
              if (
                !_rawCourseObject.available
              ) {
                if (
                  confirm(
                    "Kurs veröffentlichen, um Benutzeransicht zu sehen?"
                  )
                ) {
                  REST.Creator.updateCourse(
                    {
                      token:
                        localStorage.getItem(
                          "token"
                        ) as string,
                      update: {
                        available: true,
                      },
                      id: _rawCourseObject._id,
```

```
          }
        ).then((res) => {
          if (
            res.status === 200
          ) {
            router.push(
              `/course/${courseName[1]}/${content.url}`
            );
          } else {
            toast({
              title:
                "Fehler beim Veröffentlichen",
              description:
                "Veröffentlichen fehlgeschlagen: " +
                res.payload.error,
              status: "error",
              duration: 1500,
              isClosable: true,
            });
          }
        });
      } else {
        return;
      }
    }

    router.push(
      `/course/${courseName[1]}/${content.url}`
    );
  }}
>
  Benutzeransicht
</Button>
<HStack>
  <Button
    w={"100%"}
    leftIcon={<FaPen />}
    variant={"brand"}
    onClick={() => {
      router.push(
        `/creator/studio/edit/${content._id}/info`
      );
    }}
  >
    Bearbeiten
  </Button>
  <Button
    w={"100%"}
    leftIcon={<FaPen />}
    variant={"brand"}
    onClick={() => {
      router.push(
        `/creator/studio/edit/${content._id}`
      );
    }}
  >
    Inhalt Bearbeiten
  </Button>
</HStack>
<Button
```

```
        w={"100%"}
        leftIcon={<FaTrash />}
        colorScheme={"red"}
        onClick={async () => {
          if (
            !confirm(
              "Willst du den Inhalt wirklich löschen?"
            )
          ) {
            return;
          }

          const res =
            await REST.Creator.deleteContent(
              {
                token:
                  localStorage.getItem(
                    "token"
                  ) as string,
                cid: content._id,
              }
            );

          if (res.status === 200) {
            reloadContents();
            toast({
              title: "Inhalt gelöscht",
              description:
                "Der Inhalt wurde erfolgreich gelöscht.",
              status: "success",
              duration: 1500,
              isClosable: true,
              position: "bottom-right",
            });
          } else {
            console.error(res);
            toast({
              title:
                "Fehler beim Löschen",
              description:
                "Der Inhalt konnte nicht gelöscht werden. Details siehe Konsole.",
              status: "error",
              duration: 1500,
              isClosable: true,
              position: "bottom-right",
            });
          }
        }}
      >
        Löschen
      </Button>
    </Stack>
  </Box>
</Flex>
<Text
  fontSize={"sm"}
  color={"gray.400"}
>{`Erstellt am: ${new Date(
  content.createdAt
)
```

```
                                .toLocaleString()
                                .replace(",", " um")}`}</Text>
                            </Flex>
                          </Box>
                        </>
                      )}
                    </Draggable>
                  </>
                ) : (
                  <></>
                )}
              </>
            ))}
          </Box>
        </>
      )}
    </Droppable>
  </DragDropContext>
</Box>
</Flex>
</CreatorStudioLayout>
</>
);
}
```

## apps/web/src/pages/dashboard/index.tsx

```
/**
 * apps/web/src/pages/dashboard/index.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 04.08.2022
 *
 */

import * as React from "react";
import {
  Box,
  Flex,
  Grid,
  Heading,
  Spinner,
  Text,
  Tooltip,
} from "@chakra-ui/react";
import { useHover, useLoggedIn } from "@codeup/hooks";
import { NextRouter, useRouter } from "next/router";
import { ReactNode, useEffect, useState } from "react";
import REST from "@codeup/rest";
import redirect from "@codeup/util/src/userworkflow/redirect";
import Head from "next/head";
import {
  FaCertificate,
  FaCheck,
  FaCode,
  FaEye,
  FaInfo,
  FaPlus,
  FaProjectDiagram,
```

```
} from "react-icons/fa";
import byteSize from "byte-size";
import GridLayout from "@codeup/components/global/GridLayout";
import AccountManager from "@codeup/util/dist/AccountManager";
import { CHeading } from "@codeup/components/dist/global/CHeading";
import GridLayoutV2 from "@codeup/components/dist/global/GridLayoutV2";

export default function Dashboard() {
  const { userInfo, loggedIn, loaded } = useLoggedIn();

  const [projectEntries, setProjectEntries] = useState<
    { isNewItem?: boolean; title: string; description?: string; href: string }[]
  >([]);
  const [certificates, setCertificates] = useState<
    {
      isNewItem?: boolean;
      title: string;
      description?: string;
      href: string;
    }[]
  >([]);

  const [todoEntries, setTodoEntries] = useState<
    { isNewItem?: boolean; title: string; description?: string; href: string }[]
  >([]);

  const [projectPlanningEntries, setProjectPlanningEntries] = useState<
    { isNewItem?: boolean; title: string; description?: string; href: string }[]
  >([]);

  const router = useRouter();

  useEffect(() => {
    if (loaded && !loggedIn) {
      window.dispatchEvent(new CustomEvent("codeup::open_login"));
      router.push("/#login");
    }
  }, [loaded, userInfo, loggedIn]);

  useEffect(() => {
    if (loaded && loggedIn) {
      REST.Editor.getProjectsV2(AccountManager.getActiveToken()).then((res) => {
        let x = [];
        for (const project of res.payload.projects) {
          x.push({
            title: project.name,
            href: "/editor/" + project._id,
            description:
              "Erstellt: " + new Date(project.created_at).toLocaleString(),
          });
        }

        if (x.length === 0) {
          x.push({
            isNewItem: true,
            title: "Keine Projekte vorhanden",
            href: "/editor",
            description: "Klicke, um ein neues Projekt zu erstellen",
          });
        }
```

```
  setProjectEntries(x as any);
});
REST.ToDo.getMyLists(localStorage.getItem("token") as string).then(
  (res) => {
    let x = [];
    for (const list of res.payload.lists) {
      x.push({
        title: list.name,
        href: "/todo",
        description:
          list.tasks.length > 0
            ? "Noch " + list.tasks.length + " Aufgaben"
            : "Alle Aufgaben erledigt",
      });
    }

    if (x.length === 0) {
      x.push({
        isNewItem: true,
        title: "Keine Listen vorhanden",
        href: "/todo",
        description: "Klicke, um eine neue Liste zu erstellen",
      });
    }

    setTodoEntries(x as any);
  }
);

REST.Certificates.getObtainedCertificates(
  localStorage.getItem("token") as string
).then((res) => {
  if (res.status !== 200) {
    setCertificates([
      {
        isNewItem: true,
        title: "Keine Zertifikate erhalten",
        href: "/certificates",
        description: "Klicke, um Zertifikate zu erhalten",
      },
    ]);
    return;
  }
  let x = [];
  for (const cert of res.payload.certs) {
    x.push({
      title: cert.course,
      href: "/user-cert/" + cert._id,
      description:
        "Bestanden am " +
        new Date(cert.date).toLocaleString().replace(",", " um") +
        " mit " +
        cert.percentage +
        "%",
    });
  }

  if (x.length === 0) {
    x.push({
```

```
          isNewItem: true,
          title: "Keine Zertifikate erhalten",
          href: "/certificates",
          description: "Klicke, um Zertifikate zu erhalten",
        });
      }

      setCertificates(x as any);
    });

    REST.ToDo.getV2Projects(localStorage.getItem("token") as string).then(
      (res) => {
        let x = [];
        for (const project of res.payload.projects) {
          x.push({
            title: project.name,
            href: "/planning/" + project._id,
            description:
              "Gespeicherte Daten: " +
              byteSize(project.saved.length).toString(),
          });
        }

        if (x.length === 0) {
          x.push({
            isNewItem: true,
            title: "Keine Projekte vorhanden",
            href: "/planning",
            description: "Klicke, um ein neues Projekt zu erstellen",
          });
        }

        setProjectPlanningEntries(x as any);
      }
    );
  }
}, [loaded, loggedIn]);

return (
  <>
    <Head>
      <title>Dashboard - CodeUp</title>
    </Head>
    <Flex
      direction={"column"}
      pt={"2rem"}
      bg={"#000"}
      h={"fit-content"}
      minH={"100vh"}
    >
      <Box textAlign={"center"}>
        <CHeading>Willkommen, {userInfo.firstName}!</CHeading>
      </Box>
      <Box padding={[".4rem", "2rem"]}>
        <DSection
          title={"Projekte"}
          entries={projectEntries}
          icon={<FaCode />}
          router={router}
        />
```

```
          <DSection
            title={"Projektplanung"}
            entries={projectPlanningEntries}
            icon={<FaProjectDiagram />}
            router={router}
          />
          <DSection
            title={"Todo-Listen"}
            entries={todoEntries}
            icon={<FaCheck />}
            router={router}
          />
          <DSection
            title={"Zertifikate"}
            entries={certificates}
            icon={<FaCertificate />}
            router={router}
          />
        </Box>
      </Flex>
    </>
  );
}

function DSection(props: {
  title: string;
  entries: {
    title: string;
    description?: string;
    href: string;
    isNewItem?: boolean;
  }[];
  icon: any;
  router: NextRouter;
}) {
  return (
    <>
      <GridLayoutV2
        pageTitle={props.title}
        search={{
          show: false,
          fields: false,
        }}
        topButtons={[]}
        entries={props.entries
          .filter((x) => x.isNewItem !== true)
          .map((y) => {
            return {
              title: y.title,
              fields: [
                {
                  icon: (<FaInfo />) as ReactNode,
                  text: y.description as string,
                },
              ],
              buttonRows: [
                [
                  {
                    icon: (<FaEye />) as ReactNode,
                    onClick: () => {
```

```
                  props.router.push(y.href);
                },
                text: "Ansehen",
                variant: "brand",
              },
            ],
          ],
        };
      })}
    />
  </>
 );
}
```

## apps/web/src/pages/discovery/index.tsx

```
/**
 * apps/web/src/pages/discovery/index.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 23.12.2022
 *
 */

import * as React from "react";
import REST from "@codeup/rest/src";
import ErrorComponent from "@codeup/components/dist/global/ErrorComponent";
import { Box, Heading } from "@chakra-ui/react";
import { useLoggedIn } from "@codeup/hooks";
import { FaComment, FaEye, FaInfo, FaStar, FaUser } from "react-icons/fa";
import { useRouter } from "next/router";
import GridLayout from "@codeup/components/global/GridLayout";
import AccountManager from "@codeup/util/dist/AccountManager";
import PopupManager from "../../global/PopupManager";
import Head from "next/head";
import { CHeading } from "@codeup/components/dist/global/CHeading";
import GridLayoutV2 from "@codeup/components/dist/global/GridLayoutV2";

export default function Discovery() {
  const { userInfo, loggedIn, loaded } = useLoggedIn();

  const [projects, setProjects] = React.useState<
    {
      name: string;
      description: string;
      author: string;
      domain: string;
      id: string;
      stars: string[];
    }[]
  >([]);

  const [error, setError] = React.useState(false);

  React.useEffect(() => {
    REST.Discovery.getDiscoveryProjects().then((res) => {
      if (res.status === 200) {
        setProjects(res.payload.projects);
      } else {
```

```
      setError(true);
    }
  });
}, []);

const router = useRouter();

return (
  <>
    <Head>
      <title>Discovery - CodeUp</title>
    </Head>
    {error ? (
      <>
        <ErrorComponent
          description={"Ein Fehler ist aufgetreten!"}
          title={"Fehler"}
        />
      </>
    ) : (
      <Box bg={"#000"} minH={"100vh"} h={"fit-content"}>
        <GridLayoutV2
          pageTitle={"Discovery"}
          search={{
            show: true,
            fields: true,
          }}
          topButtons={[]}
          entries={projects.map((p) => {
            return {
              title: p.name,
              fields: [
                {
                  icon: <FaUser />,
                  text: "Autor: @" + p.author,
                },
                {
                  icon: <FaStar />,
                  text: "Sterne: " + p.stars.length,
                },
                {
                  icon: <FaInfo />,
                  text:
                    "Beschreibung: " +
                    p.description.substring(0, 100) +
                    "...",
                },
              ],
              buttonRows: [
                [
                  {
                    text: "Ansehen",
                    leftIcon: <FaEye />,
                    variant: "brand",
                    onClick: () => {
                      window.open("http://" + p.domain, "_blank");
                    },
                  },
                  {
                    text: p.stars.includes(userInfo.id)
```

```
      ? "Stern entfernen"
      : "Stern hinzufügen",
    leftIcon: <FaStar />,
    variant: "brand",
    disabled: !loggedIn,
    onClick: async () => {
      const res = await REST.Discovery.starProject({
        token: AccountManager.getActiveToken(),
        projectId: p.id,
      });

      if (res.status === 200) {
        setProjects(
          projects.map((pr) => {
            if (pr.id === p.id) {
              return {
                ...pr,
                stars: res.payload.stars,
              };
            } else {
              return pr;
            }
          })
        );
        PopupManager.alert({
          title: "Erfolgreich",
          description:
            "Deine Aktion wurde erfolgreich ausgeführt!",
        });
      } else {
        PopupManager.alert({
          title: "Fehler",
          description:
            "Ein Fehler ist aufgetreten: " +
            res.payload.error,
        });
      }
    },
  },
],
[
  {
    text: "Kommentieren",
    leftIcon: <FaComment />,
    variant: "brand",
    onClick: async () => {
      const comment = await PopupManager.promptAsync({
        title: "Kommentar",
        helperText:
          "Gib deinen Kommentar ein. Achte dabei auf deine Wortwahl!",
        inputType: "TEXTAREA",
      });

      if (!comment) return;

      const res = await REST.Discovery.commentProject({
        token: AccountManager.getActiveToken(),
        projectId: p.id,
        comment: comment,
      });
```

```
                    if (res.status === 200) {
                      PopupManager.alert({
                        title: "Erfolgreich",
                        description:
                          "Dein Kommentar wurde erfolgreich hinzugefügt!",
                      });
                      return;
                    }

                    PopupManager.alert({
                      title: "Fehler",
                      description:
                        "Ein Fehler ist aufgetreten: " + res.payload.error,
                    });
                  },
                  disabled: !loggedIn,
                },
              ],
            ],
          };
        })}
      />
    </Box>
  )}
 </>
 );
}
```

## apps/web/src/pages/editor/index.tsx

```
/**
 * apps/web/src/pages/editor/index.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 05.01.2023
 *
 */

import * as React from "react";
import { useLoggedIn } from "@codeup/hooks";
import { useRouter } from "next/router";
import {
  Box,
  CircularProgress,
  Flex,
  Heading,
  useDisclosure,
} from "@chakra-ui/react";
import { io, Socket } from "socket.io-client";
import {
  FaClock,
  FaEye,
  FaFileUpload,
  FaPen,
  FaPlus,
  FaTrash,
  FaUsers,
} from "react-icons/fa";
```

```
import CollaboratorManager from "@codeup/components/editor/CollaboratorManager";
import PopupManager from "../../global/PopupManager";
import JSZip from "jszip";
import GridLayout from "@codeup/components/global/GridLayout";
import Head from "next/head";
import GridLayoutV2 from "@codeup/components/dist/global/GridLayoutV2";

export default function Editor() {
  const { loggedIn, userInfo, loaded } = useLoggedIn();
  const router = useRouter();

  const { isOpen, onOpen, onClose } = useDisclosure();
  const {
    isOpen: isCollabOpen,
    onOpen: onCollabOpen,
    onClose: onCollabClose,
  } = useDisclosure();

  const [reactSocket, setReactSocket] = React.useState<Socket>();
  let socket: Socket | null = null;

  const [projects, setProjects] = React.useState<any[]>([]);

  React.useEffect(() => {
    if (loaded) {
      if (!loggedIn) {
        router.push("/#login");
        window.dispatchEvent(new CustomEvent("codeup::open_login"));
      } else {
        socket = io();
        setReactSocket(socket);
        registerSocket();
      }
    }
  }, [loggedIn, userInfo, loaded, router]);

  const registerSocket = () => {
    if (socket) {
      socket.on("authenticate", (data: any) => {
        console.debug(data);
        if (data.authenticated) {
          console.debug("Authenticated");

          if (socket) {
            socket.on("editor:projects:list", (data: any) => {
              console.debug(data);
              if (data.success) {
                setProjects(data.projects);
              }
            });

            socket.emit("editor:projects:list");
          }
        } else {
          router.push("/#login");
          window.dispatchEvent(new CustomEvent("codeup::open_login"));
        }
      });

      socket.emit("authenticate", localStorage.getItem("token") as string);
```

```
    // @ts-ignore
    window.__backup_socket = socket;
  }
};

const [editProjectPropID, setEditProjectPropID] = React.useState<
  string | null
>(null);

return (
  <>
    <Head>
      <title>Editor - CodeUp</title>
    </Head>
    {loggedIn ? (
      <Box minH={"100vh"} h={"fit-content"} bg={"#000"}>
        <GridLayoutV2
          pageTitle={"Projekte"}
          search={{
            show: true,
            fields: true,
          }}
          topButtons={[
            {
              text: "Neu",
              leftIcon: <FaPlus />,
              variant: "brand",
              onClick: async () => {
                const name = await PopupManager.promptAsync({
                  title: "Neues Projekt",
                  helperText:
                    "Der Name sollte das Projekt bestmöglich beschreiben, damit du es später wieder
findest.",
                });

                if (name) {
                  reactSocket?.emit("editor:projects:create", {
                    name: name,
                  });

                  reactSocket?.on("editor:projects:create", (data: any) => {
                    if (data.success) {
                      reactSocket?.emit("editor:projects:list");
                    } else {
                      PopupManager.alert({
                        description: data.error as string,
                      });
                    }
                  });
                }
              },
            },
            {
              text: "Importieren",
              leftIcon: <FaFileUpload />,
              variant: "brand",
              onClick: async () => {
                const name = await PopupManager.promptAsync({
                  title: "Neues Projekt",
```

```
                helperText:
                  "Der Name sollte das Projekt bestmöglich beschreiben, damit du es später wieder
findest.",
              });

              if (name) {
                const files = await PopupManager.uploadFileAsync({
                  title: "Projekt importieren",
                  helperText:
                    "Wähle eine ZIP-Datei aus, die du importieren möchtest.",
                });

                if (files.length > 0) {
                  reactSocket?.emit("editor:projects:create", {
                    name: name,
                  });

                  reactSocket?.on(
                    "editor:projects:create",
                    (projectData: any) => {
                      if (projectData.success) {
                        const file = files[0];
                        const reader = new FileReader();
                        reader.readAsArrayBuffer(file);

                        reader.onload = () => {
                          const buffer = Buffer.from(
                            reader.result as ArrayBuffer
                          );
                          const zip = new JSZip();

                          zip.loadAsync(buffer).then(async (zip) => {
                            if (
                              zip.files["pages/"] &&
                              zip.files["pages/"].dir
                            ) {
                              const corrupted = [];
                              let pageDir = Object.keys(zip.files).filter(
                                (file) => file.startsWith("pages/")
                              );
                              let funcDir = Object.keys(zip.files).filter(
                                (file) => file.startsWith("functions/")
                              );

                              // only get top level directories
                              let pdT = [];

                              for (const page of pageDir) {
                                if (
                                  page.endsWith("/") &&
                                  page !== "pages/"
                                ) {
                                  pdT.push(page);
                                }
                              }

                              pageDir = pdT;

                              for (const page of pageDir) {
                                let indexType = "html";
```

```
let indexContent = "";
let pageCorrupted = false;

if (zip.files[`${page}index.html`]) {
  indexContent = await zip.files[
    `${page}index.html`
  ].async("string");
} else {
  if (zip.files[`${page}index.ic`]) {
    indexType = "incode";
    indexContent = await zip.files[
      `${page}index.ic`
    ].async("string");
  } else {
    corrupted.push(page);
    pageCorrupted = true;
  }
}

if (!pageCorrupted) {
  reactSocket?.emit(
    "editor:projects:pages:create",
    {
      id: projectData.project._id as string,
      url: page
        .split("/")
        [
          page.split("/").length - 2
        ].replace("_", "/"),
      type:
        indexType === "html"
          ? "HTML"
          : "InCode",
      content: indexContent,
    }
  );

  await new Promise((resolve) => {
    reactSocket?.on(
      "editor:projects:pages:create",
      async (data: any) => {
        if (data.success) {
          if (
            // @ts-ignore
            !window.__codeup__temp__editor__resource_imports
          ) {
            // @ts-ignore
            window.__codeup__temp__editor__resource_imports =
              [];
          }

          let resourceFiles = Object.keys(
            zip.files
          ).filter((file) =>
            file.startsWith(page)
          );

          resourceFiles =
            resourceFiles.filter(
              (file) =>
```

```
        !file.endsWith(
          "index.html"
        ) &&
        !file.endsWith("index.ic")
    );

    const resources: any[] = [];

    for (const resource of resourceFiles) {
      const resourceContent =
        await zip.files[
          resource
        ].async("string");
      resources.push({
        content: resourceContent,
        name: resource
          .split("/")
          [
            resource.split("/")
              .length - 1
          ].split(".")[0],
        type: resource
          .split("/")
          [
            resource.split("/")
              .length - 1
          ].split(".")[1],
      });
    }

    for (const resource of resources) {
      if (
        // @ts-ignore
        window.__codeup__temp__editor__resource_imports.indexOf(
          page + "/" + resource.name
        ) === -1
      ) {
        // @ts-ignore
        window.__codeup__temp__editor__resource_imports.push(
          page + "/" + resource.name
        );
      } else {
        return;
      }
      reactSocket?.emit(
        "editor:projects:page:content:set",
        {
          projectId:
            projectData.project._id,
          pageId: data.page._id,
          resource: resource.name,
          resourceType: resource.type,
          content: resource.content,
        }
      );
    }

    resolve(true);
  }
}
```

```
      );
    });
  }
}

let functions = zip.files["functions/"].dir
  ? Object.keys(zip.files)
      .filter((file) =>
        file.startsWith("functions/")
      )
      .filter((file) => file !== "functions/")
  : [];

functions = functions.map((f) => {
  return f.split("/")[1];
});

for (const func of functions) {
  let funcContent = "";

  if (zip.files[`functions/${func}`]) {
    funcContent = await zip.files[
      `functions/${func}`
    ].async("string");
  } else {
    corrupted.push(func);
  }

  reactSocket?.emit(
    "editor:projects:functions:create",
    {
      id: projectData.project._id as string,
      url: func
        .replace(/_/g, "/")
        .replace(/\/api/g, "")
        .replace(/\.js/g, ""),
    }
  );

  await new Promise((resolve) => {
    reactSocket?.on(
      "editor:projects:functions:create",
      (data: any) => {
        if (data.success) {
          reactSocket?.emit(
            "editor:projects:functions:content:set",
            {
              projectId:
                projectData.project._id,
              pageId: data.function._id,
              code: funcContent,
            }
          );

          resolve(true);
        }
      }
    );
  });
}
```

```
                    reactSocket?.emit("editor:projects:list");
                  }
                });
              };
            } else {
              PopupManager.alert({
                description: projectData.error as string,
              });
            }
          }
        );
      }
    }
  },
 ]}
 entries={projects.map((project) => {
  return {
    title: project.name,
    fields: [
     {
      icon: <FaClock />,
      text: new Date(project.updated_at).toLocaleString(),
     },
    ],
    buttonRows: [
     [
      {
       text: "Löschen",
       onClick: async () => {
        if (
         await PopupManager.confirmAsync({
          title: "Projekt löschen",
          question:
           "Willst du das Projekt wirklich unwiederuflich löschen? Dies kann nicht rückgängig
gemacht werden.",
         })
        ) {
         reactSocket?.on(
          "editor:projects:delete",
          (data: any) => {
           if (data.success) {
            setProjects(
             projects.filter((p) => p._id !== project._id)
            );
           } else {
            PopupManager.alert({
             description: data.error as string,
            });
           }
          }
         );

         reactSocket?.emit("editor:projects:delete", {
          id: project._id,
         });
        }
       },
       colorScheme: "red",
```

```
                leftIcon: <FaTrash />,
              },
              {
                text: "Öffnen",
                onClick: () => {
                  router.push(`/editor/${project._id}`);
                },
                variant: "brand",
                leftIcon: <FaEye />,
              },
            ],
            [
              {
                text: "Umbenennen",
                onClick: async () => {
                  const newName = await PopupManager.promptAsync({
                    title: "Projekt umbenennen",
                    helperText:
                      "Der Name sollte das Projekt bestmöglich beschreiben, damit du es später wieder
findest.",
                  });

                  if (newName) {
                    reactSocket?.on(
                      "editor:projects:rename",
                      (data: any) => {
                        if (data.success) {
                          reactSocket?.emit("editor:projects:list");
                        } else {
                          PopupManager.alert({
                            description: data.error as string,
                          });
                        }
                      }
                    );

                    reactSocket?.emit("editor:projects:rename", {
                      id: project._id,
                      name: newName,
                    });
                  }
                },
                variant: "brand",
                leftIcon: <FaPen />,
              },
              {
                text: "Mitwirkende",
                onClick: () => {
                  setEditProjectPropID(project._id);
                  onCollabOpen();
                },
                variant: "brand",
                leftIcon: <FaUsers />,
              },
            ],
          ],
        };
      })}
    />
  </Box>
```

```
      ) : (
        <>
          <Flex
            w={"100%"}
            h={"100vh"}
            justifyContent={"center"}
            alignItems={"center"}
            direction={"column"}
            bg={"#000"}
          >
            <CircularProgress size={"80px"} isIndeterminate color="green.300" />
            <Heading size={"lg"} mt={4}>
              Anmelden ...
            </Heading>
          </Flex>
        </>
      )}

      <CollaboratorManager
        isOpen={isCollabOpen}
        onClose={onCollabClose}
        onOpen={onCollabOpen}
        projectId={editProjectPropID || ""}
        socket={reactSocket as Socket}
      />
    </>
  );
}
```

## apps/web/src/pages/editor/[project_id]/contributors.tsx

```
/**
 * apps/web/src/pages/editor/[project_id]/contributors.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 21.01.2023
 *
 */

import * as React from "react";
import EditorProjectLayout from "@codeup/components/editor/EditorProjectLayout";
import CollaboratorManagerBody from "@codeup/components/editor/CollaboratorManagerBody";
import { Box, Heading } from "@chakra-ui/react";
import { useRouter } from "next/router";
import { io, Socket } from "socket.io-client";
import PopupManager from "../../../global/PopupManager";
import { useLoggedIn } from "@codeup/hooks";
import Head from "next/head";
import { CHeading } from "@codeup/components/dist/global/CHeading";

export default function Contributors() {
  const router = useRouter();
  const { loggedIn, userInfo, loaded } = useLoggedIn();
  const [reactSocket, setReactSocket] = React.useState<Socket | null>(null);
  let socket: Socket | null = null;

  React.useEffect(() => {
    if (loaded) {
      if (!loggedIn) {
```

```
        router.push("/#login");
        window.dispatchEvent(new CustomEvent("codeup::open_login"));
      } else {
        if (router.query.project_id) {
          socket = io();
          setReactSocket(socket);
          registerSocket();
        }
      }
    }
  }, [loggedIn, userInfo, loaded, router]);

  const registerSocket = () => {
    if (socket) {
      socket.on("authenticate", (data: any) => {
        if (!data.authenticated) {
          router.push("/#login");
          window.dispatchEvent(new CustomEvent("codeup::open_login"));
        }
      });

      socket.emit("authenticate", localStorage.getItem("token") as string);

      // @ts-ignore
      window.__backup_socket = socket;
    }
  };
  return (
    <>
      <Head>
        <title>Mitwirkende - Editor - CodeUp</title>
      </Head>

      <EditorProjectLayout>
        <Box textAlign={"center"} pt={6}>
          <CHeading size={"xl"}>Mitwirkende</CHeading>
        </Box>
        {reactSocket ? (
          <>
            <CollaboratorManagerBody
              projectId={(router.query.project_id as string) || "0"}
              socket={reactSocket as Socket}
            />
          </>
        ) : (
          <></>
        )}
      </EditorProjectLayout>
    </>
  );
}
```

## apps/web/src/pages/editor/[project_id]/deployment.tsx

```
/**
 * apps/web/src/pages/editor/[project_id]/deployment.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 27.01.23
```

```
 *
 */
import * as React from "react";
import EditorProjectLayout from "@codeup/components/editor/EditorProjectLayout";
import CollaboratorManagerBody from "@codeup/components/editor/CollaboratorManagerBody";
import {
  Box,
  Button,
  Flex,
  FormControl,
  Heading,
  Input,
  InputGroup,
  InputRightAddon,
  Stack,
  StackDivider,
  Text,
} from "@chakra-ui/react";
import { useRouter } from "next/router";
import { io, Socket } from "socket.io-client";
import PopupManager from "../../../global/PopupManager";
import { useLoggedIn } from "@codeup/hooks";
import { useId } from "react";
import { FaExternalLinkAlt } from "react-icons/fa";
import { v4 } from "uuid";
import { CHeading } from "@codeup/components/dist/global/CHeading";

export default function Contributors() {
  const router = useRouter();
  const { loggedIn, userInfo, loaded } = useLoggedIn();
  const [reactSocket, setReactSocket] = React.useState<Socket | null>(null);
  let socket: Socket | null = null;
  const [project, setProject] = React.useState<any>({});
  const [customDomain, setCustomDomain] = React.useState<string>("");

  React.useEffect(() => {
    if (loaded) {
      if (!loggedIn) {
        router.push("/#login");
        window.dispatchEvent(new CustomEvent("codeup::open_login"));
      } else {
        if (router.query.project_id) {
          socket = io();
          setReactSocket(socket);
          registerSocket();
        }
      }
    }
  }, [loggedIn, userInfo, loaded, router]);

  const registerSocket = () => {
    if (socket) {
      socket.on("authenticate", (data: any) => {
        if (!data.authenticated) {
          router.push("/#login");
          window.dispatchEvent(new CustomEvent("codeup::open_login"));
        } else {
          socket?.emit("editor:projects:info", { id: router.query.project_id });
        }
      });
```

```
    socket.on("editor:projects:info", (data: any) => {
      if (data.success) {
        setProject(data.project);
        if (!data.project.customDomain.startsWith("disabled:")) {
          setCustomDomain(data.project.customDomain.split(".")[0]);
        }
      } else {
        PopupManager.alert({
          title: "Fehler",
          description: data.error,
        });
      }
    });

    socket.emit("authenticate", localStorage.getItem("token") as string);

    // @ts-ignore
    window.__backup_socket = socket;
  }
};

const inputId = useId();

return (
  <>
    <EditorProjectLayout>
      <Box textAlign={"center"} pt={6}>
        <CHeading>Deployment</CHeading>
      </Box>
      {reactSocket ? (
        <>
          <Flex w={"100%"} alignItems={"center"} justifyContent={"center"}>
            <Box w={"50%"} mt={4}>
              <Stack>
                <Text>
                  Wenn du möchtest, dass andere mit einem Link auf dein
                  gesamtes Projekt zugreifen können, kannst du es hier
                  veröffentlichen. Du kannst dir eine eigene Domain aussuchen.
                  Zum Beispiel kann dein Projekt so unter
                  https://mein-projekt.codeup.space erreichbar sein. Gib
                  einfach deine Wunsch-Domain in das Feld unten ein und wir
                  schauen, ob diese verfügbar ist. Wenn du die Freigabe
                  entfernen möchtest, kannst du das Feld einfach leer lassen.
                  Nach einer Änderung musst du auf den Knopf
                  &amp;quot;Speichern&amp;quot; klicken.
                </Text>
                <StackDivider />
                <FormControl>
                  <InputGroup>
                    <Input
                      id={inputId}
                      placeholder={
                        "Sub-Domain Name (ohne https://, Beispiel: mein-projekt)"
                      }
                      defaultValue={customDomain}
                    />
                    <InputRightAddon>.codeup.space</InputRightAddon>
                  </InputGroup>
                </FormControl>
                <Button
```

```
  onClick={() => {
    const newDomain = (
      document.getElementById(inputId) as HTMLInputElement
    ).value;

    let d =
      newDomain +
      "." +
      (window.location.hostname.includes(":")
        ? window.location.hostname.split(":")[0]
        : window.location.hostname);

    if (newDomain === "") d = "disabled:" + v4();

    if (reactSocket) {
      reactSocket.on(
        "editor:projects:settings",
        (data: any) => {
          if (data.success) {
            if (newDomain === "") {
              PopupManager.alert({
                title: "Erfolgreich",
                description:
                  "Deine App ist nun nicht mehr öffentlich erreichbar.",
              });
              return;
            }

            PopupManager.alert({
              title: "Erfolgreich",
              description:
                "Deine App ist nun unter http://" +
                d +
                " erreichbar.",
            });
          } else {
            PopupManager.alert({
              title: "Fehler",
              description: data.error,
            });
          }
        }
      );

      reactSocket.emit("editor:projects:settings", {
        id: router.query.project_id,
        update: {
          customDomain: d,
        },
      });
    }
  }}
>
  Speichern
</Button>
{customDomain !== "" ? (
  <Button
    leftIcon={<FaExternalLinkAlt />}
    variant={"brand"}
    onClick={() => {
```

```
                    window.open(
                      `http://${customDomain}.codeup.space`,
                      "_blank"
                    );
                  }}
                >
                  Deploment Öffnen
                </Button>
              ) : null}
            </Stack>
          </Box>
        </Flex>
      </>
    ) : (
      <></>
    )}
  </EditorProjectLayout>
 </>
);
}
```

## apps/web/src/pages/editor/[project_id]/discovery.tsx

```
/**
 * apps/web/src/pages/editor/[project_id]/discovery.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 25.03.2023
 *
 */

import * as React from "react";
import { useRouter } from "next/router";
import { useLoggedIn } from "@codeup/hooks";
import { io, Socket } from "socket.io-client";
import PopupManager from "../../../global/PopupManager";
import EditorProjectLayout from "@codeup/components/editor/EditorProjectLayout";
import AuthenticateLoadingIndicator from
"@codeup/components/dist/admin/AuthenticateLoadingIndicator";
import {
  Accordion,
  AccordionButton,
  AccordionIcon,
  AccordionItem,
  AccordionPanel,
  Box,
  Button,
  Flex,
  FormControl,
  FormLabel,
  Heading,
  Link,
  Stack,
  StackDivider,
  Text,
} from "@chakra-ui/react";
import { FaCheck, FaExclamationTriangle } from "react-icons/fa";
import Head from "next/head";
import { CHeading } from "@codeup/components/dist/global/CHeading";
```

```
export default function Discovery() {
  const router = useRouter();
  const { loggedIn, userInfo, loaded } = useLoggedIn();
  const [reactSocket, setReactSocket] = React.useState<Socket | null>(null);
  let socket: Socket | null = null;
  const [project, setProject] = React.useState<any>({});

  React.useEffect(() => {
    if (loaded) {
      if (!loggedIn) {
        router.push("/#login");
        window.dispatchEvent(new CustomEvent("codeup::open_login"));
      } else {
        if (router.query.project_id) {
          socket = io();
          setReactSocket(socket);
          registerSocket();
        }
      }
    }
  }, [loggedIn, userInfo, loaded, router]);

  const registerSocket = () => {
    if (socket) {
      socket.on("authenticate", (data: any) => {
        if (!data.authenticated) {
          router.push("/#login");
          window.dispatchEvent(new CustomEvent("codeup::open_login"));
        } else {
          socket?.emit("editor:projects:info", { id: router.query.project_id });
        }
      });

      socket.on("editor:projects:info", (data: any) => {
        if (data.success) {
          setProject(data.project);
        } else {
          PopupManager.alert({
            title: "Fehler",
            description: data.error,
          });
        }
      });

      socket.emit("authenticate", localStorage.getItem("token") as string);

      // @ts-ignore
      window.__backup_socket = socket;
    }
  };

  return (
    <>
      <Head>
        <title>Discovery - Editor - CodeUp</title>
      </Head>
      <EditorProjectLayout>
        {reactSocket ? (
          <>
```

```
<Box textAlign={"center"} pt={6}>
  <CHeading size={"xl"}>Discovery</CHeading>
</Box>
<Flex w={"100%"} alignItems={"center"} justifyContent={"center"}>
  <Box w={"50%"} mt={4}>
    {project && project.customDomain && (
      <>
        <Stack>
          <Text fontSize={"lg"}>
            Wenn du deine App mit einer eigenen Domain
            (*.codeup.space) verbunden hast, kannst du diese im
            Discovery anzeigen. Hierdurch können andere Benutzer
            deine App finden und verwenden.
          </Text>
          <Flex fontSize={"lg"}>
            {project.customDomain.startsWith("disabled:") ? (
              <Flex color={"red.300"} alignItems={"center"} gap={4}>
                <FaExclamationTriangle /> Du hast deine App noch
                nicht mit einer Domain verbunden. Dies kannst du
                <Text
                  color={"brand.500"}
                  cursor={"pointer"}
                  _hover={{
                    textDecoration: "underline",
                  }}
                  onClick={() => {
                    router.push(
                      "/editor/" + project._id + "/deployment"
                    );
                  }}
                >
                  Hier
                </Text>
                nachholen.
              </Flex>
            ) : (
              <>
                <Stack>
                  <Flex
                    color={"brand.500"}
                    alignItems={"center"}
                    gap={4}
                  >
                    <FaCheck /> Du hast deine App mit einer Domain
                    verbunden. Diese lautet: {project.customDomain}
                  </Flex>
                  {project.discovery ? (
                    <>
                      <Flex
                        color={"brand.500"}
                        alignItems={"center"}
                        gap={4}
                      >
                        <FaCheck /> Deine App ist im Discovery
                        verfügbar
                      </Flex>
                    </>
                  ) : (
                    <>
                      <Flex
```

```
      color={"red.300"}
      alignItems={"center"}
      gap={4}
    >
      <FaExclamationTriangle /> Deine App ist
      nicht im Discovery verfügbar
    </Flex>
  </>
)}
<StackDivider />
<Button
  variant={"brand"}
  onClick={async () => {
    if (project.discovery) {
      reactSocket?.emit(
        "editor:projects:settings",
        {
          id: project._id,
          update: {
            discovery: false,
            discoveryDescription: "",
          },
        }
      );

      reactSocket?.on(
        "editor:projects:settings",
        (data: any) => {
          if (data.success) {
            setProject({
              ...project,
              discovery: false,
              discoveryDescription: "",
            });
            PopupManager.alert({
              title: "Erfolg",
              description:
                "Deine App wurde erfolgreich aus dem Discovery entfernt.",
            });
          } else {
            PopupManager.alert({
              title: "Fehler",
              description:
                "Deine App konnte nicht aus dem Discovery entfernt werden: " +
                data.error,
            });
          }
        }
      );
    } else {
      const description =
        await PopupManager.promptAsync({
          title: "Beschreibung",
          helperText:
            "Bitte gib eine Beschreibung für deine App an",
          inputType: "TEXTAREA",
        });

      if (!description) return;
      if (description.length > 1000)
```

```
        return PopupManager.alert({
          title: "Fehler",
          description:
            "Die Beschreibung darf nicht länger als 1000 Zeichen sein.",
        });

      if (description.length < 100)
        return PopupManager.alert({
          title: "Fehler",
          description:
            "Die Beschreibung muss mindestens 100 Zeichen lang sein.",
        });

      reactSocket?.emit(
        "editor:projects:settings",
        {
          id: project._id,
          update: {
            discovery: true,
            discoveryDescription: description,
          },
        }
      );

      reactSocket?.on(
        "editor:projects:settings",
        (data: any) => {
          if (data.success) {
            setProject({
              ...project,
              discovery: true,
              discoveryDescription: description,
            });
            PopupManager.alert({
              title: "Erfolg",
              description:
                "Deine App wurde erfolgreich im Discovery veröffentlicht.",
            });
          } else {
            PopupManager.alert({
              title: "Fehler",
              description:
                "Deine App konnte nicht im Discovery veröffentlicht werden: " +
                data.error,
            });
          }
        }
      );
    }
  }}
>
  {project.discovery
    ? "Discovery Deaktivieren"
    : "Discovery Aktivieren"}
</Button>
<StackDivider />
<Heading size={"xl"}>Kommentare</Heading>
{project.discoveryComments.length === 0 && (
  <>
    <Text color={"red.300"}>
```

```
                    Es gibt noch keine Kommentare.
                  </Text>
                 </>
                )}

                <Accordion allowToggle allowMultiple>
                 {project.discoveryComments
                   .sort(
                     (a: any, b: any) =>
                       new Date(b.created_at).getTime() -
                       new Date(a.created_at).getTime()
                   )
                   .map((comment: any) => {
                     return (
                       <>
                         <AccordionItem>
                           <h2>
                             <AccordionButton>
                               <Box
                                 as={"span"}
                                 flex={1}
                                 textAlign={"left"}
                               >
                                 {new Date(
                                   comment.created_at
                                 ).toLocaleString() +
                                   " - " +
                                   comment.username}
                               </Box>
                               <AccordionIcon />
                             </AccordionButton>
                             <AccordionPanel pb={4}>
                              {comment.comment}
                             </AccordionPanel>
                           </h2>
                         </AccordionItem>
                       </>
                     );
                   })}
                 </Accordion>
               </Stack>
             </>
            )}
           </Flex>
          </Stack>
         </>
        )}
       </Box>
      </Flex>
     </>
    ) : (
     <AuthenticateLoadingIndicator />
    )}
   </EditorProjectLayout>
  </>
 );
}
```

apps/web/src/pages/editor/[project_id]/export.tsx

```
/**
```

```
 * apps/web/src/pages/editor/[project_id]/export.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 19.02.2023
 *
 */

import * as React from "react";
import { useId } from "react";
import { useRouter } from "next/router";
import { useLoggedIn } from "@codeup/hooks";
import { io, Socket } from "socket.io-client";
import PopupManager from "../../../global/PopupManager";
import EditorProjectLayout from "@codeup/components/dist/editor/EditorProjectLayout";
import {
  Box,
  Button,
  Flex,
  Heading,
  HStack,
  Stack,
  StackDivider,
  Text,
} from "@chakra-ui/react";
import JSZip from "jszip";
import Networking from "@codeup/util/dist/Networking";
import AuthenticateLoadingIndicator from
"@codeup/components/dist/admin/AuthenticateLoadingIndicator";
import Head from "next/head";
import { FaCrown } from "react-icons/fa";
import { CHeading } from "@codeup/components/dist/global/CHeading";

export default function Export() {
  const router = useRouter();
  const { loggedIn, userInfo, loaded } = useLoggedIn();
  const [reactSocket, setReactSocket] = React.useState<Socket | null>(null);
  let socket: Socket | null = null;
  const [project, setProject] = React.useState<any>({});

  React.useEffect(() => {
    if (loaded) {
      if (!loggedIn) {
        router.push("/#login");
        window.dispatchEvent(new CustomEvent("codeup::open_login"));
      } else {
        if (router.query.project_id) {
          socket = io();
          setReactSocket(socket);
          registerSocket();
        }
      }
    }
  }, [loggedIn, userInfo, loaded, router]);

  const registerSocket = () => {
    if (socket) {
      socket.on("authenticate", (data: any) => {
        if (!data.authenticated) {
```

```
        router.push("/#login");
        window.dispatchEvent(new CustomEvent("codeup::open_login"));
      } else {
        socket?.emit("editor:projects:info", { id: router.query.project_id });
      }
    });

    socket.on("editor:projects:info", (data: any) => {
      if (data.success) {
        setProject(data.project);
      } else {
        PopupManager.alert({
          title: "Fehler",
          description: data.error,
        });
      }
    });

    socket.emit("authenticate", localStorage.getItem("token") as string);

    // @ts-ignore
    window.__backup_socket = socket;
  }
};

return (
  <>
    <Head>
      <title>Export - Editor - CodeUp</title>
    </Head>
    <EditorProjectLayout>
      <Box textAlign={"center"} pt={6}>
        <CHeading size={"xl"}>Exportieren</CHeading>
      </Box>
      {reactSocket ? (
        <>
          <Flex w={"100%"} alignItems={"center"} justifyContent={"center"}>
            <Box w={"50%"} mt={4}>
              <Stack>
                <HStack color={"brand.500"}>
                  <FaCrown />
                  <Text>
                    Exportiere dein Projekt mit <b>CodeUp PRO</b>
                  </Text>
                </HStack>
                <Text fontSize={"lg"}>
                  Du kannst deine App exportieren und auf deinem eigenen
                  Server hosten. Klicke dazu auf die Schaltfläche unten. Der
                  Prozess wird einige Minuten in Anspruch nehmen und die
                  ZIP-Datei wird, nach Abschluss des Exports, automatisch
                  heruntergeladen.
                </Text>
                <StackDivider />
                <Button
                  variant={"brand"}
                  onClick={async () => {
                    if (userInfo.subscription !== "PRO") {
                      await PopupManager.alertAsync({
                        title: "CodeUp PRO",
                        description:
```

```
        "Du kannst dein Projekt nur exportieren, wenn du eine aktive CodeUp PRO Lizenz hast!",
      });
      return;
    }

    let pages: any[] = [];
    let staticAssets: any[] = [];
    let functions: any[] = [];

    reactSocket.on("editor:projects:pages", (data: any) => {
      if (data.success) {
        let realPages: any[] = [];
        for (const p of data.pages) {
          reactSocket.on(
            "editor:projects:page:content",
            (data: any) => {
              if (data.success) {
                realPages.push(data.page);
              }
            }
          );
          reactSocket.emit("editor:projects:page:content", {
            projectId: router.query.project_id,
            pageId: p._id,
          });
        }
        pages = realPages;

        reactSocket.on(
          "editor:projects:static:assets",
          (data: any) => {
            if (data.success) {
              staticAssets = data.files;

              reactSocket.on(
                "editor:projects:functions",
                async (data: any) => {
                  if (data.success) {
                    functions = data.functions;

                    const zip = new JSZip();

                    zip.file(
                      "project.json",
                      JSON.stringify(project, null, 2)
                    );
                    const pagesFolder = zip.folder(
                      "pages"
                    ) as JSZip;
                    const staticFolder = zip.folder(
                      "static"
                    ) as JSZip;
                    const functionsFolder = zip.folder(
                      "functions"
                    ) as JSZip;

                    for (let page of pages) {
                      let url = page.url.replace(/\//g, "_");
```

```
    const pageFolder = pagesFolder.folder(
      url
    ) as JSZip;
    if (page.type === "HTML") {
      pageFolder!.file(
        "index.html",
        page.mainContent
      );
    } else {
      pageFolder!.file(
        "index.ic",
        page.mainContent
      );
    }

    for (let resource of page.resources) {
      pageFolder!.file(
        resource.name + "." + resource.type,
        resource.content
      );
    }
  }

  for (let asset of staticAssets) {
    staticFolder!.file(
      asset.filename.replace("/", "_") +
        ".json",
      JSON.stringify(
        {
          originalName: asset.filename,
          downloadUrl:
            "https://codeup.space/editor-static/" +
            asset.filename,
        },
        null,
        2
      )
    );
  }

  for (let func of functions) {
    functionsFolder!.file(
      func.path.replace(/\/g, "_") + ".js",
      func.code
    );
  }

  // download zip
  const blob = await zip.generateAsync({
    type: "blob",
  });
  const url = URL.createObjectURL(blob);
  const a = document.createElement("a");
  a.href = url;
  a.download = `${project.name}.zip`;
  a.click();
  URL.revokeObjectURL(url);
} else {
  PopupManager.alert({
    title: "Fehler",
```

```
                                description: data.error,
                              });
                            }
                          }
                        );
                        reactSocket.emit("editor:projects:functions", {
                          id: router.query.project_id,
                        });
                      } else {
                        PopupManager.alert({
                          title: "Fehler",
                          description: data.error,
                        });
                      }
                    }
                  );
                  reactSocket.emit("editor:projects:static:assets", {
                    id: router.query.project_id,
                  });
                } else {
                  PopupManager.alert({
                    title: "Fehler",
                    description: data.error,
                  });
                }
              });
              reactSocket.emit("editor:projects:pages", {
                id: router.query.project_id,
              });
            }}
          >
            Exportieren
          </Button>
        </Stack>
      </Box>
    </Flex>
  </>
) : (
  <>
    <AuthenticateLoadingIndicator />
  </>
)}
</EditorProjectLayout>
</>
);
}
```

## apps/web/src/pages/editor/[project_id]/index.tsx

```
/**
 * apps/web/src/pages/editor/[project_id]/index.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 20.01.2023
 *
 */

import * as React from "react";
import { io, Socket } from "socket.io-client";
```

```
import { useRouter } from "next/router";
import { useLoggedIn } from "@codeup/hooks";
import EditorProjectLayout from "@codeup/components/editor/EditorProjectLayout";
import { Heading } from "@chakra-ui/react";
import {
  FaCode,
  FaEdit,
  FaHtml5,
  FaLink,
  FaPlus,
  FaTrash,
} from "react-icons/fa";
import PopupManager from "../../../global/PopupManager";
import GridLayout from "@codeup/components/global/GridLayout";
import Head from "next/head";
import GridLayoutV2 from "@codeup/components/dist/global/GridLayoutV2";

export default function ProjectViewer() {
  const router = useRouter();

  const { loggedIn, loaded, userInfo } = useLoggedIn();

  const [reactSocket, setReactSocket] = React.useState<Socket>();
  let socket: Socket | null = null;

  const [pages, setPages] = React.useState<
    {
      _id: string;
      url: string;
      type: string;
    }[]
  >([]);

  React.useEffect(() => {
    if (loaded) {
      if (!loggedIn) {
        router.push("/#login");
        window.dispatchEvent(new CustomEvent("codeup::open_login"));
      } else {
        if (router.query.project_id) {
          socket = io();
          setReactSocket(socket);
          registerSocket();
        }
      }
    }
  }, [loggedIn, userInfo, loaded, router]);

  const registerSocket = () => {
    if (socket) {
      socket.on("authenticate", (data: any) => {
        if (data.authenticated) {
          if (socket) {
            socket.on("editor:projects:pages", (data: any) => {
              if (data.success) {
                setPages(data.pages);
              } else {
                PopupManager.alert({
                  title: "Fehler",
                  description: data.error,
```

```
            });
          }
        });

        socket.emit("editor:projects:pages", {
          id: router.query.project_id,
        });
      }
    } else {
      router.push("/#login");
      window.dispatchEvent(new CustomEvent("codeup::open_login"));
    }
  });

  socket.emit("authenticate", localStorage.getItem("token") as string);

  // @ts-ignore
  window.__backup_socket = socket;
 }
};

 return (
  <>
   <Head>
    <title>Seiten - Editor - CodeUp</title>
   </Head>
   <EditorProjectLayout>
    <GridLayoutV2
     pageTitle={"Seiten"}
     search={{
      show: true,
      fields: true,
     }}
     topButtons={[
      {
       text: "Neu",
       leftIcon: <FaPlus />,
       variant: "brand",
       onClick: async () => {
        const url = await PopupManager.promptAsync({
         title: "Neue Seite",
         helperText:
          "Gib die URL der neuen Seite ein. (z.B. about). Leerlassen um die Startseite zu
erstellen.",
        });

        const type = await PopupManager.selectAsync({
         title: "Seitentyp",
         helperText: "Wähle den Seitentyp aus.",
         choices: ["HTML", "InCode"],
        });

        if (type) {
         if (reactSocket) {
          reactSocket.emit("editor:projects:pages:create", {
           id: router.query.project_id,
           url: "/" + url,
           type: type,
          });
```

```
          reactSocket.on(
            "editor:projects:pages:create",
            (data: any) => {
              if (data.success) {
                setPages([
                  ...pages,
                  {
                    _id: data.page._id,
                    url: data.page.url,
                    type: data.page.type,
                  },
                ]);
              } else {
                PopupManager.alert({
                  title: "Fehler",
                  description: data.error as string,
                });
              }
            }
          );
        }
      }
    },
  },
]}
entries={pages.map((page) => {
  return {
    title: page.url === "/" ? "Hauptseite (/)" : page.url,
    fields: [
      {
        text: page.type === "HTML" ? "HTML" : "InCode",
        icon: page.type === "HTML" ? <FaHtml5 /> : <FaCode />,
      },
    ],
    buttonRows: [
      [
        {
          text: "Löschen",
          leftIcon: <FaTrash />,
          colorScheme: "red",
          onClick: async () => {
            if (
              await PopupManager.confirmAsync({
                title: "Seite löschen",
                question: "Möchtest du diese Seite wirklich löschen?",
              })
            ) {
              if (reactSocket) {
                reactSocket.on(
                  "editor:projects:pages:delete",
                  (data: any) => {
                    if (data.success) {
                      setPages(
                        pages.filter((p) => p._id !== page._id)
                      );
                    } else {
                      PopupManager.alert({
                        title: "Fehler",
                        description: data.error as string,
                      });
```

```
          }
        }
      );

      reactSocket.emit("editor:projects:pages:delete", {
        projectId: router.query.project_id,
        pageId: page._id,
      });
    }
  }
 },
},
{
 text: "URL Ändern",
 leftIcon: <FaLink />,
 variant: "brand",
 onClick: async () => {
  let newUrl = await PopupManager.promptAsync({
   title: "Seite bearbeiten",
   helperText: "Gib die neue URL der Seite ein.",
  });

  if (newUrl) {
   newUrl = ["/", newUrl].join("");

   if (reactSocket) {
    reactSocket.on(
     "editor:projects:pages:rename",
     (data: any) => {
      if (data.success) {
       setPages(
        pages.map((p) => {
         if (p._id === page._id) {
          return {
           ...p,
           url: newUrl,
          };
         } else {
          return p;
         }
        })
       );
      } else {
       PopupManager.alert({
        title: "Fehler",
        description: data.error as string,
       });
      }
     }
    );

    reactSocket.emit("editor:projects:pages:rename", {
     projectId: router.query.project_id,
     pageId: page._id,
     newUrl,
    });
   }
  }
 },
},
```

```
            ],
            [
              {
                text: "Bearbeiten",
                leftIcon: <FaEdit />,
                variant: "brand",
                onClick: async () => {
                  router.push(
                    `/editor/${router.query.project_id}/pages/${page._id}`
                  );
                },
              },
            ],
          ],
        };
      })}
    />
  </EditorProjectLayout>
 </>
 );
}
```

## apps/web/src/pages/editor/[project_id]/pages/[page_id].tsx

```
/**
 * apps/web/src/pages/editor/[project_id]/pages/[page_id].tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 21.01.2023
 *
 */

import * as React from "react";
import { useRouter } from "next/router";
import { useLoggedIn } from "@codeup/hooks";
import { io, Socket } from "socket.io-client";
import PopupManager from "../../../../global/PopupManager";
import AuthenticateLoadingIndicator from
"@codeup/components/admin/AuthenticateLoadingIndicator";
import EditorPageSidebar from "@codeup/components/editor/EditorPageSidebar";
import { Box, Flex, Heading, Text } from "@chakra-ui/react";
import Editor, { useMonaco } from "@monaco-editor/react";
import { useRef, useState } from "react";
import { editor, Position } from "monaco-editor";
import IStandaloneCodeEditor = editor.IStandaloneCodeEditor;
import Head from "next/head";
import { CHeading } from "@codeup/components/dist/global/CHeading";

export default function PageEditor() {
  const router = useRouter();

  const { loggedIn, loaded, userInfo } = useLoggedIn();

  const [reactSocket, setReactSocket] = React.useState<Socket>();
  let socket: Socket | null = null;
  const [currentResource, setCurrentResource] = React.useState<string>(
    "codeup.main_resource"
  );
```

```
const [page, setPage] = React.useState<{
  _id: string;
  url: string;
  type: string;
  mainContent: string;
  resources: {
    name: string;
    type: string;
    content: string;
  }[];
}>();

React.useEffect(() => {
  if (loaded) {
    if (!loggedIn) {
      router.push("/#login");
      window.dispatchEvent(new CustomEvent("codeup::open_login"));
    } else {
      if (router.query.project_id) {
        socket = io();
        // @ts-ignore
        window.__editor_current_resource = "codeup.main_resource";
        setReactSocket(socket);
        registerSocket();
      }
    }
  }
}, [loggedIn, userInfo, loaded, router]);

const registerSocket = () => {
  if (socket) {
    socket.on("authenticate", (data: any) => {
      if (data.authenticated) {
        if (socket) {
          socket.on("editor:projects:page:content", (data: any) => {
            if (data.success) {
              if (data.user && data.user === userInfo?.username) return;

              if (page !== data.page) {
                if (!page) setPage(data.page);

                if (editorRef) {
                  if (editorRef.current) {
                    const oldPos = editorRef.current.getPosition();
                    if (
                      // @ts-ignore
                      window.__editor_current_resource ===
                      "codeup.main_resource"
                    ) {
                      if (
                        data.page.mainContent === editorRef.current.getValue()
                      )
                        return;
                      editorRef.current.setValue(data.page.mainContent);
                      // @ts-ignore
                      window.__editor_do_not_push = true;
                    }

                    editorRef.current.setPosition(oldPos as Position);
                  }
```

```
            }
          }
        } else {
          PopupManager.alertAsync({
            title: "Fehler",
            description: data.error,
          }).then(() => {
            router.push("/editor");
          });
        }
      });

      socket.on("editor:projects:page:content:set", (data: any) => {
        if (!data.success) {
          PopupManager.alert({
            title: "Speichern fehlgeschlagen",
            description: data.error,
          });
        }
      });

      socket.emit("editor:projects:page:content", {
        projectId: router.query.project_id,
        pageId: router.query.page_id,
      });
    }
  } else {
    router.push("/#login");
    window.dispatchEvent(new CustomEvent("codeup::open_login"));
  }
});

socket.emit("authenticate", localStorage.getItem("token") as string);

// @ts-ignore
window.__backup_socket = socket;
}
};

const monaco = useMonaco();
const [monacoTheme, setMonacoTheme] = useState("vs-dark");
const [monacoLanguage, setMonacoLanguage] = useState("html");

const editorRef = useRef<IStandaloneCodeEditor>();

React.useEffect(() => {
  if (monaco && monaco.editor) {
    monaco.editor.defineTheme("codeup-dark", {
      base: "vs-dark",
      inherit: true,
      rules: [],
      colors: {
        "editor.background": "#101113",
      },
    });

    setMonacoTheme("codeup-dark");
  }
}, [monaco]);
```

```
React.useEffect(() => {
  if (editorRef && editorRef.current && page) {
    if (currentResource !== "codeup.main_resource") {
      editorRef.current.setValue(
        page.resources.find((x) => x.name == currentResource)?.content || ""
      );
    } else {
      editorRef.current.setValue(page.mainContent);
    }

    setMonacoLanguage(
      currentResource === "codeup.main_resource"
        ? page.type.toLowerCase() === "html"
          ? "html"
          : "plain"
        : (page.resources
            .find((r) => r.name === currentResource)
            ?.type.toLowerCase() as string)
    );
  }
}, [currentResource]);

React.useEffect(() => {
  if (editorRef && editorRef.current) {
    monaco?.editor.setModelLanguage(
      editorRef.current.getModel() as editor.ITextModel,
      monacoLanguage
    );
  }
}, [monacoLanguage]);

if (page) {
  return (
    <>
      <Head>
        <title>Seite - Editor - CodeUp</title>
      </Head>
      <Flex
        w={"100%"}
        minH={"100vh"}
        maxW={"100%"}
        direction={"row"}
        bg={"#000"}
      >
        <Box flex={["30%", "20%", "10%"]} minH={"90vh"} rounded={"xl"}>
          <EditorPageSidebar
            page={page}
            projectId={router.query.project_id as string}
            socket={reactSocket as Socket}
            setPage={setPage}
            setCurrentResource={setCurrentResource}
          />
        </Box>
        <Box
          flex={["70%", "80%", "90%"]}
          minH={"90vh"}
          m={"2rem"}
          boxShadow={"2xl"}
          rounded={"xl"}
        >
```

```
<Flex direction={"column"} minH={"90vh"} w={"100%"} rounded={"xl"}>
  <Box bg={"black"} p={4}>
    <CHeading size={"xl"}>
      {currentResource === "codeup.main_resource"
        ? "Hauptwebseite (" + page.url + ")"
        : "Resource (" + currentResource + ")"}
    </CHeading>
  </Box>

  <Editor
    defaultLanguage={monacoLanguage}
    defaultValue={
      currentResource === "codeup.main_resource"
        ? page.mainContent
        : (
            page.resources.find(
              (r) => r.name === currentResource
            ) as any
          ).content
    }
    onMount={(e) => {
      editorRef.current = e;
    }}
    onChange={(value, ev) => {
      if (reactSocket) {
        // @ts-ignore
        if (window.__editor_do_not_push) {
          // @ts-ignore
          window.__editor_do_not_push = false;
          return;
        }
        reactSocket.emit("editor:projects:page:content:set", {
          projectId: router.query.project_id,
          pageId: router.query.page_id,
          content: value,
          resource: currentResource,
        });
        setPage({
          ...page,
          mainContent:
            currentResource === "codeup.main_resource"
              ? (value as string)
              : (page.mainContent as string),
          resources: page.resources.map((r) => {
            if (r.name === currentResource) {
              return {
                ...r,
                content: value as string,
              };
            } else {
              return r;
            }
          }),
        });
      }
    }}
    theme={monacoTheme}
    options={{
      insertSpaces: false,
      wrappingIndent: "indent",
```

```
                fontSize: 22,
              }}
              height={"90vh"}
            />
          </Flex>
        </Box>
      </Flex>
    </>
  );
 } else {
  return <AuthenticateLoadingIndicator />;
 }
}
```

## apps/web/src/pages/editor/[project_id]/serverless/index.tsx

```
/**
 * apps/web/src/pages/editor/[project_id]/functions/index.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 20.01.2023
 *
 */

import * as React from "react";
import { io, Socket } from "socket.io-client";
import { useRouter } from "next/router";
import { useLoggedIn } from "@codeup/hooks";
import EditorProjectLayout from "@codeup/components/dist/editor/EditorProjectLayout";
import GridLayout from "@codeup/components/global/GridLayout";
import {
  Button,
  Flex,
  FormControl,
  Grid,
  GridItem,
  Heading,
  HStack,
  Input,
  InputGroup,
  InputLeftElement,
  Stack,
  Text,
} from "@chakra-ui/react";
import {
  FaCode,
  FaEdit,
  FaHtml5,
  FaLink,
  FaPlus,
  FaSearch,
  FaTrash,
} from "react-icons/fa";
import PopupManager from "../../../../global/PopupManager";
import Head from "next/head";
import GridLayoutV2 from "@codeup/components/dist/global/GridLayoutV2";

export default function ProjectfunctionsViewer() {
  const router = useRouter();
```

```
const { loggedIn, loaded, userInfo } = useLoggedIn();

const [reactSocket, setReactSocket] = React.useState<Socket>();
let socket: Socket | null = null;

const [functions, setFunctions] = React.useState<
  {
    _id: string;
    path: string;
  }[]
>([]);

const [query, setQuery] = React.useState<string>("");

React.useEffect(() => {
  if (loaded) {
    if (!loggedIn) {
      router.push("/#login");
      window.dispatchEvent(new CustomEvent("codeup::open_login"));
    } else {
      if (router.query.project_id) {
        socket = io();
        setReactSocket(socket);
        registerSocket();
      }
    }
  }
}, [loggedIn, userInfo, loaded, router]);

const registerSocket = () => {
  if (socket) {
    socket.on("authenticate", (data: any) => {
      if (data.authenticated) {
        if (socket) {
          socket.on("editor:projects:functions", (data: any) => {
            if (data.success) {
              setFunctions(data.functions);
            } else {
              PopupManager.alert({
                title: "Fehler",
                description: data.error,
              });
            }
          });

          socket.emit("editor:projects:functions", {
            id: router.query.project_id,
          });
        }
      } else {
        router.push("/#login");
        window.dispatchEvent(new CustomEvent("codeup::open_login"));
      }
    });

    socket.emit("authenticate", localStorage.getItem("token") as string);

    // @ts-ignore
    window.__backup_socket = socket;
```

```
  }
};

return (
  <>
    <Head>
      <title>Serverless - Editor - CodeUp</title>
    </Head>
    <EditorProjectLayout>
      <GridLayoutV2
        pageTitle={"Serverless"}
        search={{
          show: true,
          fields: true,
        }}
        topButtons={[
          {
            text: "Neu",
            leftIcon: <FaPlus />,
            variant: "brand",
            onClick: async () => {
              const url = await PopupManager.promptAsync({
                title: "Neue Funktion",
                helperText:
                  "Gib die URL der Funktion an. (z.B. account/login)",
              });

              if (reactSocket) {
                reactSocket.emit("editor:projects:functions:create", {
                  id: router.query.project_id,
                  url: "/" + url,
                });

                reactSocket.on(
                  "editor:projects:functions:create",
                  (data: any) => {
                    if (data.success) {
                      setFunctions([
                        ...functions,
                        {
                          _id: data.function._id,
                          path: data.function.path,
                        },
                      ]);
                    } else {
                      PopupManager.alert({
                        title: "Fehler",
                        description: data.error as string,
                      });
                    }
                  }
                );
              }
            },
          },
        ]}
        entries={functions.map((func) => {
          return {
            title: func.path,
            fields: [
```

```
  {
    text: "",
    icon: null,
  },
],
buttonRows: [
  [
    {
      colorScheme: "red",
      leftIcon: <FaTrash />,
      text: "Löschen",
      onClick: async () => {
        if (
          await PopupManager.confirmAsync({
            title: "Seite löschen",
            question: "Möchtest du diese Seite wirklich löschen?",
          })
        ) {
          if (reactSocket) {
            reactSocket.on(
              "editor:projects:functions:delete",
              (data: any) => {
                if (data.success) {
                  setFunctions(
                    functions.filter((p) => p._id !== func._id)
                  );
                } else {
                  PopupManager.alert({
                    title: "Fehler",
                    description: data.error as string,
                  });
                }
              }
            );

            reactSocket.emit("editor:projects:functions:delete", {
              projectId: router.query.project_id,
              pageId: func._id,
            });
          }
        }
      },
    },
  ],
  [
    {
      variant: "brand",
      leftIcon: <FaLink />,
      text: "URL Ändern",
      onClick: async () => {
        let newUrl = await PopupManager.promptAsync({
          title: "Seite bearbeiten",
          helperText: "Gib die neue URL der Seite ein.",
        });

        if (newUrl) {
          newUrl = ["/api/", newUrl].join("");

          if (reactSocket) {
            reactSocket.on(
```

```
                    "editor:projects:functions:rename",
                    (data: any) => {
                      if (data.success) {
                        setFunctions(
                          functions.map((p) => {
                            if (p._id === func._id) {
                              return {
                                ...p,
                                path: newUrl,
                              };
                            } else {
                              return p;
                            }
                          })
                        );
                      } else {
                        PopupManager.alert({
                          title: "Fehler",
                          description: data.error as string,
                        });
                      }
                    }
                  );

                  reactSocket.emit("editor:projects:functions:rename", {
                    projectId: router.query.project_id,
                    pageId: func._id,
                    newUrl,
                  });
                }
              }
            },
          },
          {
            text: "Bearbeiten",
            variant: "brand",
            leftIcon: <FaEdit />,
            onClick: () => {
              router.push(
                `/editor/${router.query.project_id}/serverless/${func._id}`
              );
            },
          },
        ],
      ],
    };
  })}
/>
</EditorProjectLayout>
</>
);
}
```

## apps/web/src/pages/editor/[project_id]/serverless/[page_id].tsx

```
/**
 * apps/web/src/pages/editor/[project_id]/serverless/[page_id].tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
```

```
 * Created At: 19.02.2023
 *
 */

import * as React from "react";
import { useRef, useState } from "react";
import { useRouter } from "next/router";
import { useLoggedIn } from "@codeup/hooks";
import { io, Socket } from "socket.io-client";
import PopupManager from "../../../../global/PopupManager";
import AuthenticateLoadingIndicator from
"@codeup/components/admin/AuthenticateLoadingIndicator";
import {
  Box,
  Flex,
  IconButton,
  Tooltip,
  useDisclosure,
} from "@chakra-ui/react";
import Editor, { useMonaco } from "@monaco-editor/react";
import { editor } from "monaco-editor";
import IStandaloneCodeEditor = editor.IStandaloneCodeEditor;
import { FaCode, FaEye, FaFileAlt, FaHome, FaLaptopCode } from "react-icons/fa";
import PageResourceManager from "@codeup/components/dist/editor/PageResourceManager";
import { SnippetViewer } from "@codeup/components/dist/ide";
import EditorPreviewModal from "@codeup/components/dist/editor/EditorPreviewModal";
import Head from "next/head";

export default function FunctionEditor() {
  const router = useRouter();

  const { loggedIn, loaded, userInfo } = useLoggedIn();

  const [reactSocket, setReactSocket] = React.useState<Socket>();
  let socket: Socket | null = null;

  const [func, setFunc] = React.useState<{
    _id: string;
    path: string;
    code: string;
  }>();

  React.useEffect(() => {
    if (loaded) {
      if (!loggedIn) {
        router.push("/#login");
        window.dispatchEvent(new CustomEvent("codeup::open_login"));
      } else {
        if (router.query.project_id) {
          socket = io();
          setReactSocket(socket);
          registerSocket();
        }
      }
    }
  }, [loggedIn, userInfo, loaded, router]);

  const registerSocket = () => {
    if (socket) {
      socket.on("authenticate", (data: any) => {
```

```
      if (data.authenticated) {
        if (socket) {
          socket.on("editor:projects:functions:content", (data: any) => {
            if (data.success) {
              setFunc(data.function);
            } else {
              PopupManager.alertAsync({
                title: "Fehler",
                description: data.error,
              }).then(() => {
                router.push("/editor");
              });
            }
          });

          socket.on("editor:projects:functions:content:set", (data: any) => {
            if (!data.success) {
              PopupManager.alert({
                title: "Speichern fehlgeschlagen",
                description: data.error,
              });
            }
          });

          socket.emit("editor:projects:functions:content", {
            projectId: router.query.project_id,
            pageId: router.query.page_id,
          });
        }
      } else {
        router.push("/#login");
        window.dispatchEvent(new CustomEvent("codeup::open_login"));
      }
    });

    socket.emit("authenticate", localStorage.getItem("token") as string);

    // @ts-ignore
    window.__backup_socket = socket;
  }
};

const monaco = useMonaco();
const [monacoTheme, setMonacoTheme] = useState("vs-dark");

const editorRef = useRef<IStandaloneCodeEditor>();

React.useEffect(() => {
  if (monaco && monaco.editor) {
    monaco.editor.defineTheme("codeup-dark", {
      base: "vs-dark",
      inherit: true,
      rules: [],
      colors: {
        "editor.background": "#101113",
      },
    });

    setMonacoTheme("codeup-dark");
  }
```

```
}, [monaco]);

const {
  isOpen: isSVOpen,
  onOpen: onSVOpen,
  onClose: onSVClose,
} = useDisclosure();

if (func) {
  return (
    <>
      <Head>
        <title>Funktion - Editor - CodeUp</title>
      </Head>
      <Flex
        w={"100%"}
        minH={"100vh"}
        maxW={"100%"}
        direction={"row"}
        bg={"#000"}
      >
        <Box flex={["30%", "20%", "10%"]} minH={"90vh"}>
          <Box p={"2rem"} w={"100%"} h={"100%"}>
            <Box
              backgroundColor={"black"}
              h={"100%"}
              boxShadow={"2xl"}
              rounded={"xl"}
            >
              <Flex direction={"column"} h={"100%"}>
                <Tooltip label={"Snippets"} placement={"right"} hasArrow>
                  <IconButton
                    aria-label={""}
                    variant="ghost"
                    size={"lg"}
                    fontSize={"2xl"}
                    p={"2rem"}
                    icon={<FaLaptopCode />}
                    rounded={"none"}
                    onClick={onSVOpen}
                  />
                </Tooltip>

                <Tooltip label={"Übersicht"} placement={"right"} hasArrow>
                  <IconButton
                    aria-label={""}
                    variant="ghost"
                    size={"lg"}
                    fontSize={"2xl"}
                    p={"2rem"}
                    icon={<FaHome />}
                    rounded={"none"}
                    onClick={() => {
                      router.push(
                        "/editor/" + router.query.project_id + "/serverless"
                      );
                    }}
                  />
                </Tooltip>
              </Flex>
```

```
            </Box>
          </Box>
          <SnippetViewer isOpen={isSVOpen} onClose={onSVClose} />
        </Box>
        <Box
          flex={["70%", "80%", "90%"]}
          minH={"90vh"}
          m={"2rem"}
          boxShadow={"2xl"}
        >
          <Flex direction={"column"} minH={"90vh"} w={"100%"} rounded={"xl"}>
            <Editor
              defaultLanguage={"javascript"}
              defaultValue={func.code}
              onMount={(e) => {
                editorRef.current = e;
              }}
              onChange={(value, ev) => {
                if (reactSocket) {
                  reactSocket.emit("editor:projects:functions:content:set", {
                    projectId: router.query.project_id,
                    pageId: router.query.page_id,
                    code: value,
                  });
                  setFunc({
                    ...func,
                    code: value as string,
                  });
                }
              }}
              theme={monacoTheme}
              options={{
                insertSpaces: false,
                wrappingIndent: "indent",
                fontSize: 22,
              }}
              height={"90vh"}
            />
          </Flex>
        </Box>
      </Flex>
    </>
  );
 } else {
  return <AuthenticateLoadingIndicator />;
 }
}
```

## apps/web/src/pages/editor/[project_id]/staticassets.tsx

```
/**
 * apps/web/src/pages/editor/[project_id]/staticassets.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 28.01.23
 *
 */

import * as React from "react";
```

```
import EditorProjectLayout from "@codeup/components/editor/EditorProjectLayout";
import CollaboratorManagerBody from "@codeup/components/editor/CollaboratorManagerBody";
import {
  Box,
  Button,
  ButtonGroup,
  Heading,
  IconButton,
  Table,
  TableContainer,
  Tbody,
  Td,
  Th,
  Thead,
  Tr,
} from "@chakra-ui/react";
import { useRouter } from "next/router";
import { io, Socket } from "socket.io-client";
import PopupManager from "../../../global/PopupManager";
import { useLoggedIn } from "@codeup/hooks";
import REST from "@codeup/rest/dist";
import byteSize from "byte-size";
import {
  FaExternalLinkAlt,
  FaExternalLinkSquareAlt,
  FaLink,
  FaPlus,
  FaTrash,
} from "react-icons/fa";
import Head from "next/head";
import { CHeading } from "@codeup/components/dist/global/CHeading";

export default function Contributors() {
  const router = useRouter();
  const { loggedIn, userInfo, loaded } = useLoggedIn();
  const [assets, setAssets] = React.useState<any[]>([]);
  const [reactSocket, setReactSocket] = React.useState<Socket | null>(null);
  let socket: Socket | null = null;

  React.useEffect(() => {
    if (loaded) {
      if (!loggedIn) {
        router.push("/#login");
        window.dispatchEvent(new CustomEvent("codeup::open_login"));
      } else {
        if (router.query.project_id) {
          socket = io();
          setReactSocket(socket);
          registerSocket();
        }
      }
    }
  }, [loggedIn, userInfo, loaded, router]);

  const registerSocket = () => {
    if (socket) {
      socket.on("authenticate", (data: any) => {
        if (!data.authenticated) {
          router.push("/#login");
          window.dispatchEvent(new CustomEvent("codeup::open_login"));
```

```
      } else {
        socket?.emit("editor:projects:static:assets", {
          id: router.query.project_id,
        });
      }
    });

    socket.on("editor:projects:static:assets", (data: any) => {
      if (data.success) {
        setAssets(data.files);
      } else {
        PopupManager.alert({
          title: "Fehler",
          description: data.error,
        });
      }
    });

    socket.on("editor:projects:static:assets:upload", (data: any) => {
      if (data.success) {
        socket?.emit("editor:projects:static:assets", {
          id: router.query.project_id,
        });
      } else {
        PopupManager.alert({
          title: "Fehler",
          description: data.error,
        });
      }
    });

    socket.on("editor:projects:static:assets:delete", (data: any) => {
      if (data.success) {
        socket?.emit("editor:projects:static:assets", {
          id: router.query.project_id,
        });
      } else {
        PopupManager.alert({
          title: "Fehler",
          description: data.error,
        });
      }
    });

    socket.emit("authenticate", localStorage.getItem("token") as string);

    // @ts-ignore
    window.__backup_socket = socket;
  }
};
return (
  <>
    <Head>
      <title>Statische Dateien - Editor - CodeUp</title>
    </Head>
    <EditorProjectLayout>
      <Box textAlign={"center"} pt={6}>
        <CHeading size={"xl"}>Statische Dateien</CHeading>
      </Box>
      {reactSocket ? (
```

```
<>
  <TableContainer mt={"2rem"} p={6}>
    <Table>
      <Thead>
        <Tr>
          <Th>Name</Th>
          <Th>Hochladedatum</Th>
          <Th>Größe</Th>
          <Th>Aktion</Th>
        </Tr>
      </Thead>
      <Tbody>
        <Tr>
          <Td>
            <b>Neue Datei hochladen</b>
          </Td>
          <Td />
          <Td />
          <Td>
            <Button
              variant={"brand"}
              leftIcon={<FaPlus />}
              onClick={async () => {
                const input = document.createElement("input");
                input.type = "file";
                input.onchange = async (e: Event) => {
                  // @ts-ignore
                  if (e.target.files) {
                    const binaryData =
                      // @ts-ignore
                      await e.target.files[0]!.arrayBuffer();

                    const binary = new Uint8Array(binaryData);

                    reactSocket?.emit(
                      "editor:projects:static:assets:upload",
                      {
                        id: router.query.project_id,
                        // @ts-ignore
                        fileName: e.target.files[0].name,
                        fileContent: binary,
                      }
                    );
                  }
                };

                input.click();
              }}
            >
              Hochladen
            </Button>
          </Td>
        </Tr>
        {assets.map((asset) => (
          <Tr>
            <Td>{asset.filename.split("/")[1]}</Td>
            <Td>{new Date(asset.uploadDate).toLocaleString()}</Td>
            <Td>{byteSize(asset.length).toString()}</Td>
            <Td>
              <ButtonGroup isAttached>
```

```
<IconButton
  aria-label={"Löschen"}
  colorScheme={"red"}
  icon={<FaTrash />}
  onClick={async () => {
    if (
      await PopupManager.confirmAsync({
        title: "Löschen",
        question:
          "Möchtest du diese Datei wirklich löschen?",
      })
    ) {
      reactSocket?.emit(
        "editor:projects:static:assets:delete",
        {
          projectId: router.query.project_id,
          assetName: asset.filename.split("/")[1],
        }
      );
    }
  }}
/>
<IconButton
  aria-label={"Kopieren"}
  variant={"brand"}
  icon={<FaLink />}
  onClick={() => {
    PopupManager.alert({
      title: "Kopiert",
      description:
        "Link: " +
        window.location.origin +
        "/editor-static/" +
        asset.filename,
    });

    navigator.clipboard
      .writeText(
        window.location.origin +
          "/editor-static/" +
          asset.filename
      )
      .catch((e) => {
        PopupManager.alert({
          title: "Fehler",
          description: e.toString(),
        });
      });
  }}
/>
<IconButton
  aria-label={"Kopieren"}
  colorScheme={"blue"}
  icon={<FaExternalLinkAlt />}
  onClick={() => {
    window.open(
      window.location.origin +
        "/editor-static/" +
        asset.filename,
      "_blank"
```

```
                        );
                      }}
                    />
                  </ButtonGroup>
                </Td>
              </Tr>
            ))}
          </Tbody>
        </Table>
      </TableContainer>
    </>
  ) : (
    <></>
  )}
 </EditorProjectLayout>
 </>
 );
}
```

## apps/web/src/pages/error.tsx

```
/**
 * apps/web/src/pages/error.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 06.10.2022
 *
 */

import * as React from "react";
import { useRouter } from "next/router";
import ErrorComponent from "@codeup/components/global/ErrorComponent";
import Head from "next/head";

export default function Error() {
 const router = useRouter();

 return (
  <>
   <Head>
    <title>Fehler - CodeUp</title>
   </Head>
   <ErrorComponent
    description={router.query.error as string}
    title={"Fehler"}
    image={"/_static/vectors/error/500_v2.svg"}
   />
  </>
 );
}
```

## apps/web/src/pages/forum/index.tsx

```
/**
 * apps/web/src/pages/forum/index.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 20.09.2022
```

```
 *
 */

import * as React from "react";
import { useEffect } from "react";
import { useLoggedIn } from "@codeup/hooks";
import { useRouter } from "next/router";
import REST from "@codeup/rest";
import {
  Avatar,
  Box,
  Button,
  ButtonGroup,
  chakra,
  Flex,
  Heading,
  Input,
  Text,
  useToast,
} from "@chakra-ui/react";
import { FaCheckCircle } from "react-icons/fa";
import DeferredPostCardV2 from "@codeup/components/forum/DeferredPostCardV2";
import Head from "next/head";
import { CHeading } from "@codeup/components/dist/global/CHeading";

export default function Index() {
  const { loggedIn, userInfo, loaded } = useLoggedIn();

  const [hasProfile, setHasProfile] = React.useState(false);
  const [profiles, setProfiles] = React.useState<
    {
      _id: string;
      user: string;
      displayName: string;
      verified: boolean;
      bio: string;
      website: string;
      location: string;
      createdAt: string;
      __v: number;
      username: string;
    }[]
  >([]);

  const [posts, setPosts] = React.useState<any[]>([]);

  const [profileQuery, setProfileQuery] = React.useState("");
  const [postQuery, setPostQuery] = React.useState<string>("");

  const router = useRouter();

  useEffect(() => {
    if (loaded) {
      if (!loggedIn) {
        router.push("/");
        window.dispatchEvent(new CustomEvent("codeup::open_login"));
      }
    }
  }, [loaded, loggedIn]);
```

```
useEffect(() => {
  if (userInfo && userInfo.username) {
    REST.Forum.getProfileV2(userInfo.username).then((res) => {
      if (res.status === 200) {
        setHasProfile(true);
      }
    });
  }
}, [loaded, userInfo]);

useEffect(() => {
  if (hasProfile) {
    REST.Forum.getProfilesV2().then((res) => {
      if (res.status === 200) {
        setProfiles(res.payload.profiles);
      }
    });
    REST.Forum.getAllPostsV2().then((res) => {
      if (res.status === 200) {
        setPosts(res.payload.posts);
      }
    });
  }
}, [hasProfile]);

const toast = useToast();

return (
  <>
    <Head>
      <title>Forum - CodeUp</title>
    </Head>
    {!hasProfile ? (
      <>
        <Flex w={"100%"} justifyContent={"center"} p={4}>
          <Box
            w={"100%"}
            maxW={"600px"}
            p={4}
            bg={"black"}
            borderRadius={"md"}
            boxShadow={"md"}
          >
            <CHeading size={"lg"}>Profil erstellen</CHeading>
            <Text mt={4}>
              Du hast noch kein Profil erstellt. Erstelle jetzt dein Profil
              und werde Teil der Community!
            </Text>
            <ButtonGroup w={"100%"} justifyContent={"flex-end"}>
              <Button
                mt={4}
                variant={"brand"}
                onClick={async () => {
                  const r = await REST.Forum.createProfileV2({
                    username: userInfo.username,
                    token: localStorage.getItem("token") as string,
                  });

                  if (r.status === 200) {
                    setHasProfile(true);
```

```
              toast({
                title: "Profil erstellt",
                description: "Dein Profil wurde erfolgreich erstellt!",
                duration: 5000,
                isClosable: true,
                position: "top",
                status: "success",
              });
            } else {
              toast({
                title: "Fehler",
                description:
                  "Dein Profil konnte nicht erstellt werden!",
                duration: 5000,
                isClosable: true,
                position: "top",
                status: "error",
              });
            }
          }}
        >
          Profil erstellen
        </Button>
      </ButtonGroup>
    </Box>
  </Flex>
 </>
) : (
 <>
  <Flex w={"100%"} direction={["column", "row"]} gap={"2rem"} p={4}>
   <Flex flex={"100%"} justifyContent={"center"} mt={[0, "2rem"]}>
    <Box
     w={"100%"}
     maxW={"600px"}
     p={4}
     bg={"black"}
     borderRadius={"md"}
     boxShadow={"md"}
    >
     <CHeading size={"lg"}>Öffentliche Profile</CHeading>
     <Text mt={4}>
      Hier findest du alle öffentlichen Profile der Community.
     </Text>
     <Input
      mt={4}
      placeholder={"Profil suchen..."}
      onChange={(e) => {
       setProfileQuery(e.target.value);
      }}
     />
     <Flex
      w={"100%"}
      flexDirection={"column"}
      justifyContent={"center"}
      alignItems={"center"}
     >
      {profiles.map((profile) => (
       <>
        {profile.displayName
```

```
          .toLowerCase()
          .includes(profileQuery.toLowerCase()) ||
        profile.username
          .toLowerCase()
          .includes(profileQuery.toLowerCase()) ? (
          <>
            <Flex
              w={"100%"}
              p={4}
              bg={"gray.800"}
              borderRadius={"md"}
              mt={4}
              justifyContent={"space-between"}
              alignItems={"center"}
              _hover={{
                bg: "gray.700",
              }}
              cursor={"pointer"}
              onClick={() => {
                router.push(`/forum/${profile.username}`);
              }}
            >
              <Flex direction={"row"}>
                <Avatar
                  size={"lg"}
                  src={
                    "/api/account/" +
                    profile.username +
                    "/picture"
                  }
                />
                <Flex flexDirection={"column"} ml={4}>
                  <Heading
                    size={"md"}
                    display={"flex"}
                    alignItems={"center"}
                    gap={"1.2rem"}
                  >
                    {profile.displayName}
                    {profile.verified ? (
                      <>
                        <chakra.span color={"brand.500"}>
                          <FaCheckCircle />
                        </chakra.span>
                      </>
                    ) : null}
                  </Heading>
                  <Text color={"gray"}>@{profile.username}</Text>
                  <Text>{profile.bio}</Text>
                </Flex>
              </Flex>
            </Flex>
          </>
        ) : (
          <></>
        )}
      </>
    ))}
  </Flex>
</Box>
```

```
          </Flex>
          <Flex flex={"100%"} justifyContent={"center"} mt={[0, "2rem"]}>
            <Box
              w={"100%"}
              maxW={"600px"}
              p={4}
              bg={"black"}
              borderRadius={"md"}
              boxShadow={"md"}
            >
              <CHeading size={"lg"}>Beiträge</CHeading>
              <Text mt={4}>Hier findest du alle Beiträge.</Text>
              <Input
                mt={4}
                placeholder={"Beiträge suchen..."}
                onChange={(e) => {
                  setPostQuery(e.target.value);
                }}
              />
              <Flex w={"100%"} flexDirection={"column"}>
                {posts.map((post) => {
                  return (
                    <>
                      {post.content
                        .toLowerCase()
                        .includes(postQuery.toLowerCase()) ? (
                        <DeferredPostCardV2
                          post={post}
                          debug={true}
                          reloadPosts={() => {}}
                        />
                      ) : null}
                    </>
                  );
                })}
              </Flex>
            </Box>
          </Flex>
        </Flex>
      </>
    )}
  </>
 );
}
```

## apps/web/src/pages/forum/messages/index.tsx

```
/**
 * apps/web/src/pages/forum/messages/index.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 24.09.2022
 *
 */

import * as React from "react";
import {
  Avatar,
  Badge,
```

```
  Box,
  Button,
  ButtonGroup,
  chakra,
  Divider,
  Flex,
  Heading,
  Spinner,
  Text,
  useMediaQuery,
  Popover,
  PopoverTrigger,
  PopoverContent,
  PopoverHeader,
  PopoverBody,
  PopoverFooter,
  PopoverArrow,
  PopoverCloseButton,
  PopoverAnchor,
  InputGroup,
  InputLeftAddon,
  Input,
  useToast,
} from "@chakra-ui/react";
import { useEffect, useState } from "react";
import { FaCheckCircle, FaPlus } from "react-icons/fa";
import { useRouter } from "next/router";
import ChatCardV2 from "@codeup/components/forum/ChatCardV2";
import REST from "@codeup/rest";
import { useLoggedIn, useUserInfo } from "@codeup/hooks";
import ChatViewV2 from "@codeup/components/forum/ChatViewV2";
import Head from "next/head";
import { CHeading } from "@codeup/components/dist/global/CHeading";

export default function MessageView() {
  const [chats, setChats] = useState<
    {
      id: string;
      users: {
        username: string;
        displayName: string;
        verified: boolean;
      }[];
    }[]
  >([]);

  const { userInfo, loggedIn, loaded: loadedUserInfo } = useLoggedIn();

  const [loaded, setLoaded] = useState(false);

  const [activeChat, setActiveChat] = useState<string>("");

  const router = useRouter();

  const reloadChats = () => {
    REST.Forum.getChatListV2({
      token: localStorage.getItem("token") as string,
    }).then((r) => {
      if (r.status === 200) {
        setChats(r.payload.chats);
```

```
      setLoaded(true);
    }
  });
};

useEffect(reloadChats, []);

useEffect(() => {
  const s = setInterval(() => {
    reloadChats();
  }, 1000);

  return () => {
    clearInterval(s);
  };
}, []);

const toast = useToast();

useEffect(() => {
  // activeChat should be set to the query parameter "chat" if it is not empty

  if (router.query.chat && router.query.chat !== "") {
    setActiveChat(router.query.chat as string);
  }
}, [router]);

return (
  <>
    <Head>
      <title>Direktnachrichten - CodeUp</title>
    </Head>
    <Flex direction={"row"} w={"100%"} h={"80vh"}>
      <Box flex={["100%", "25%"]} shadow={"xl"}>
        <Flex
          direction={"column"}
          w={"100%"}
          h={"fit-content"}
          maxH={"80vh"}
          overflow={"auto"}
        >
          <Box textAlign={"center"} mt={4}>
            <CHeading size={"lg"}>Chats</CHeading>
          </Box>
          <Divider mt={4} />
          {!loaded ? (
            <Flex
              w={"100%"}
              alignItems={"center"}
              justifyContent={"center"}
              mt={4}
            >
              <Spinner size={"xl"} />
            </Flex>
          ) : null}

          {loaded ? (
            <Flex
              w={"100%"}
              alignItems={"center"}
```

```
  justifyContent={"center"}
  mt={4}
  direction={"column"}
>
  <Popover>
    {({ isOpen, onClose }) => (
      <>
        <PopoverTrigger>
          <Button
            variant={"brand"}
            leftIcon={<FaPlus />}
            w={"80%"}
          >
            Neuer Chat
          </Button>
        </PopoverTrigger>
        <PopoverContent bg={"black"}>
          <PopoverArrow bg={"black"} />
          <PopoverCloseButton />
          <PopoverHeader>Neuer Chat</PopoverHeader>
          <PopoverBody>
            <InputGroup>
              <InputLeftAddon>@</InputLeftAddon>
              <Input
                id={"forum_msg-new-chat"}
                placeholder={"Benutzername"}
              />
            </InputGroup>
          </PopoverBody>
          <PopoverFooter>
            <Button
              variant={"brand"}
              id={"forum_msg-new-chat-btn"}
              onClick={async () => {
                const username = document.getElementById(
                  "forum_msg-new-chat"
                ) as HTMLInputElement;

                const r = await REST.Forum.createChatV2({
                  token: localStorage.getItem("token") as string,
                  user: username.value,
                });

                if (r.status === 200) {
                  reloadChats();
                  setActiveChat(r.payload.chat._id);
                } else {
                  toast({
                    title: "Fehler",
                    description:
                      "Der Chat konnte nicht erstellt werden.",
                    status: "error",
                    duration: 5000,
                    isClosable: true,
                    position: "top",
                  });
                }
                username.value = "";
                onClose();
              }}
```

```
                >
                  Erstellen
                </Button>
              </PopoverFooter>
            </PopoverContent>
          </>
        )}
      </Popover>
      <Divider mt={4} />
    </Flex>
  ) : null}

  {chats
    ? chats.map((chat) => (
        <>
          <ChatCardV2
            chat={{
              id: chat.id,
              displayName:
                chat.users[0].username === userInfo.username
                  ? chat.users[1].displayName
                  : chat.users[0].displayName,
              username:
                chat.users[0].username === userInfo.username
                  ? chat.users[1].username
                  : chat.users[0].username,
              verified:
                chat.users[0].username === userInfo.username
                  ? chat.users[1].verified
                  : chat.users[0].verified,
            }}
            setActiveChat={setActiveChat}
            reloadChats={reloadChats}
          />
          <Divider />
        </>
      ))
    : null}
  </Flex>
</Box>
<Box flex={[0, "75%"]} display={["none", "initial"]}>
  <Flex
    direction={"column"}
    w={"100%"}
    minH={"100%"}
    maxH={"100%"}
    overflow={"auto"}
  >
    <Box pt={4} textAlign={"center"}>
      <CHeading size={"lg"}>Chat</CHeading>
    </Box>
    <Divider mt={4} />
    {activeChat ? (
      <>
        <ChatViewV2 chatId={activeChat} setActiveChat={setActiveChat} />
      </>
    ) : (
      <Flex
        w={"100%"}
        alignItems={"center"}
```

```
              justifyContent={"center"}
              mt={4}
            >
              <Text>Wähle einen Chat aus</Text>
            </Flex>
          )}
        </Flex>
      </Box>
    </Flex>
  </>
 );
}
```

## apps/web/src/pages/forum/messages/[id].tsx

```
/**
 * apps/web/src/pages/forum/messages/[test].tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 24.09.2022
 *
 */

import * as React from "react";
import { useRouter } from "next/router";
import ChatViewV2 from "@codeup/components/forum/ChatViewV2";
import { useEffect } from "react";
import Head from "next/head";

export default function MessageView() {
  const router = useRouter();

  const { id } = router.query;

  useEffect(() => {
    console.log(id);
  }, [id]);

  return (
    <>
      <Head>
        <title>Direktnachrichten - CodeUp</title>
      </Head>
      {id ? (
        <ChatViewV2 chatId={id as string} setActiveChat={() => {}} />
      ) : null}
    </>
  );
}
```

## apps/web/src/pages/forum/post/[id].tsx

```
/**
 * apps/web/src/pages/forum/post/[id].tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 24.09.2022
 *
```

```
 */

import * as React from "react";
import { useRouter } from "next/router";
import { useEffect, useState } from "react";
import REST from "@codeup/rest";
import {
  Box,
  Button,
  Divider,
  Flex,
  Heading,
  Spinner,
  Text,
} from "@chakra-ui/react";
import PostCardV2 from "@codeup/components/forum/PostCardV2";
import { useLoggedIn } from "@codeup/hooks";
import DeferredPostCardV2 from "@codeup/components/forum/DeferredPostCardV2";
import Head from "next/head";
import { CHeading } from "@codeup/components/dist/global/CHeading";

export default function PostView() {
  const router = useRouter();

  const { id } = router.query;

  const [post, setPost] = useState<any>(null);
  const [comments, setComments] = useState<any[]>([]);

  const reloadPosts = () => {
    if (id) {
      REST.Forum.getPostV2(id as string).then(async (res) => {
        if (res.status === 200) {
          setPost(res.payload.post);

          const r = await REST.Forum.getCommentsV2(id as string);

          if (r.status === 200) {
            setComments(r.payload.comments);
          } else {
            router.push("/forum");
          }
        } else {
          router.push("/forum");
        }
      });
    }
  };

  useEffect(reloadPosts, [id, router]);

  return (
    <>
      <Head>
        <title>Forum Beitrag - CodeUp</title>
      </Head>
      {!post ? (
        <Flex
          alignItems={"center"}
          justifyContent={"center"}
```

```
    w={"100%"}
    h={"100vh"}
  >
    <Spinner size={"xl"} />
  </Flex>
) : null}
{post ? (
  <>
    <Flex p={4} w={"100%"} justifyContent={"center"}>
      <Box
        w={"100%"}
        maxW={"600px"}
        p={4}
        borderRadius={"md"}
        boxShadow={"md"}
        bg={"black"}
        position={"relative"}
      >
        <DeferredPostCardV2 post={post} reloadPosts={reloadPosts} />
        <Divider mb={4} />
        <CHeading size={"lg"}>Kommentare</CHeading>

        {post.parent !== null ? (
          <>
            <Text mt={4} color={"gray.400"}>
              Dieser Beitrag ist ein Kommentar auf einen anderen Beitrag.
              Deshalb kann unter diesem Beitrag nicht kommentiert werden!
            </Text>
            <Divider mt={4} />
            <Button
              mt={8}
              colorScheme={"green"}
              onClick={() => {
                router.push("/forum/post/" + post.parent);
              }}
            >
              Original Beitrag aufrufen
            </Button>
          </>
        ) : null}
        {post.parent === null ? (
          <>
            {comments.map((comment) => {
              return (
                <DeferredPostCardV2
                  post={comment}
                  key={comment._id}
                  reloadPosts={reloadPosts}
                />
              );
            })}
            {comments.length === 0 ? (
              <Text>Keine Kommentare vorhanden</Text>
            ) : null}
          </>
        ) : null}
      </Box>
    </Flex>
  </>
) : null}
```

```
    </>
  );
}
```

## apps/web/src/pages/forum/[username].tsx

```
/**
 * apps/web/src/pages/forum/[username].tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 23.09.2022
 *
 */

import * as React from "react";
import { useEffect, useState } from "react";
import { useRouter } from "next/router";
import {
  Avatar,
  Box,
  Button,
  ButtonGroup,
  chakra,
  Divider,
  Flex,
  Grid,
  GridItem,
  Heading,
  HStack,
  IconButton,
  Image,
  Input,
  InputGroup,
  InputLeftAddon,
  Link,
  ListItem,
  Menu,
  MenuButton,
  MenuItem,
  MenuList,
  Popover,
  PopoverArrow,
  PopoverBody,
  PopoverContent,
  PopoverFooter,
  PopoverHeader,
  PopoverTrigger,
  Spinner,
  Stack,
  Text,
  Textarea,
  Tooltip,
  UnorderedList,
  useDisclosure,
  useToast,
} from "@chakra-ui/react";
import REST from "@codeup/rest";
import {
  FaBirthdayCake,
```

```
  FaCheckCircle,
  FaCrown,
  FaEnvelope,
  FaFile,
  FaGlobe,
  FaImage,
  FaMapMarkerAlt,
  FaTimesCircle,
  FaVideo,
} from "react-icons/fa";
import { useLoggedIn } from "@codeup/hooks";
import { FiMoreHorizontal } from "react-icons/fi";
import PostCardV2 from "@codeup/components/forum/PostCardV2";
import Head from "next/head";
import PopupManager from "../../global/PopupManager";
import AccountManager from "@codeup/util/dist/AccountManager";

export default function Profile() {
  const router = useRouter();

  const { loggedIn, loaded, userInfo } = useLoggedIn();

  const {
    isOpen: isDeleteProfileOpen,
    onOpen: onDeleteProfileOpen,
    onClose: onDeleteProfileClose,
  } = useDisclosure();
  const whatEverRef = React.useRef();

  const [profile, setProfile] = useState<{
    _id: string;
    user: string;
    displayName: string;
    verified: boolean;
    bio: string;
    website: string;
    location: string;
    createdAt: string;
    __v: number;
    username: string;
    profilePicture: string;
    certificates: {
      _id: string;
      course: {
        _id: string;
        name: string;
        friendlyName: string;
        splashImage: string;
      };
      percentage: number;
      date: string;
    }[];
  } | null>(null);

  const [certificates, setCertificates] = useState<
    {
      _id: string;
      percentage: number;
      date: string;
      course: {
```

```
      _id: string;
      name: string;
      friendlyName: string;
      splashImage: string;
    };
  }[]
>([]);

const [editMode, setEditMode] = useState<boolean>(false);
const [saving, setSaving] = useState<boolean>(false);
const [posts, setPosts] = useState<any[]>([]);

const toast = useToast();

const reloadProfile = () => {
  REST.Forum.getProfileV2(router.query.username as string).then(
    async (res) => {
      if (res.status === 200) {
        const certificates: any[] = [];

        for (const certificate of (res.payload as any).profile.certificates) {
          const r = await REST.Certificates.getObtainedCertificate(
            certificate
          );

          if (r.status === 200) {
            let x = r.payload;
            certificates.push({
              _id: x.obtainedCertificate._id,
              percentage: x.obtainedCertificate.percentage,
              date: x.obtainedCertificate.date,
              course: {
                _id: x.course._id,
                name: x.course.name,
                friendlyName: x.course.friendlyName,
                splashImage: x.course.splashImage,
              },
            });
          }
        }

        let pf = {
          ...(res.payload as any).profile,
          certificates,
        };

        setProfile(pf);
      } else {
        router.push("/404");
      }
    }
  );

  REST.Certificates.getObtainedCertificates(
    AccountManager.getActiveToken()
  ).then((res) => {
    if (res.status === 200) {
      setCertificates(
        res.payload.certs.map((c: any) => {
          return {
```

```
            ...c,
            course: c.courseObj,
          };
        })
      );
    }
  });
};

useEffect(() => {
  if (router.query.username) {
    reloadProfile();
  }
}, [router]);

const reloadPosts = () => {
  if (profile && profile.username) {
    REST.Forum.getProfilePostsV2({
      username: profile.username,
    }).then((res) => {
      if (res.status === 200) {
        setPosts(res.payload.posts);
      } else {
        toast({
          title: "Fehler",
          description: "Beim Laden der Beiträge ist ein Fehler aufgetreten.",
          status: "error",
          duration: 9000,
          isClosable: true,
        });
      }
    });
  }
};

useEffect(reloadPosts, [profile]);

function upload(options: {
  prefix: string;
  mimeType: string;
  fileNameAsSuffix: boolean;
}) {
  const form = document.createElement("form");
  form.method = "POST";
  form.action = "/media/upload";
  const a = document.createElement("input");
  a.type = "file";
  a.accept = options.mimeType;
  a.name = "file";

  form.appendChild(a);

  a.click();

  a.onchange = async () => {
    const formData = new FormData(form);
    const r = await fetch("/media/upload", {
      method: "POST",
      body: formData,
    });
```

```
    if (r.status === 200) {
      const json = await r.json();

      const url = await json.data.url;

      const r0 = await REST.Forum.createPostV2({
        token: localStorage.getItem("token") as string,
        content: location.origin + url,
      });

      if (r0.status === 200) {
        reloadPosts();
      } else {
        toast({
          title: "Fehler",
          description: "Beim Hochladen ist ein Fehler aufgetreten.",
          status: "error",
          duration: 1500,
          isClosable: true,
          position: "top",
        });
      }
    } else {
      toast({
        title: "Fehler",
        description: "Beim Hochladen ist ein Fehler aufgetreten.",
        status: "error",
        duration: 1500,
        isClosable: true,
        position: "top",
      });
    }
  };
}

return (
  <>
    <Head>
      <title>@{router.query.username || "user"} - CodeUp</title>
    </Head>
    {!profile ? (
      <Flex
        alignItems={"center"}
        justifyContent={"center"}
        w={"100%"}
        h={"100vh"}
      >
        <Spinner size={"xl"} />
      </Flex>
    ) : null}
    {profile ? (
      <>
        <Flex p={4} w={"100%"} justifyContent={"center"}>
          <Box
            w={"100%"}
            maxW={"600px"}
            p={4}
            borderRadius={"md"}
            boxShadow={"md"}
```

```
  bg={"black"}
  position={"relative"}
>
  {userInfo.username !== profile.username ? (
    <>
      <Box
        position={"absolute"}
        top={"2rem"}
        right={"1rem"}
        zIndex={editMode ? -1 : 1}
      >
        <IconButton
          aria-label={"Nachricht schreiben"}
          icon={<FaEnvelope />}
          onClick={async () => {
            const r = await REST.Forum.createChatV2({
              user: profile.username,
              token: localStorage.getItem("token") as string,
            });

            if (r.status === 200) {
              router.push(
                "/forum/messages?chat=" + r.payload.chat._id
              );
            } else {
              toast({
                title: "Fehler",
                description:
                  "Beim Erstellen des Chats ist ein Fehler aufgetreten.",
                status: "error",
                duration: 9000,
                isClosable: true,
              });
            }
          }}
        />
      </Box>
    </>
  ) : null}
  {loggedIn && userInfo.username === profile.username ? (
    <>
      <Box
        position={"absolute"}
        top={"2rem"}
        right={"1rem"}
        zIndex={editMode ? -1 : 1}
      >
        <Menu>
          <MenuButton>
            <IconButton
              aria-label={"Edit"}
              icon={<FiMoreHorizontal />}
              variant={"ghost"}
              size={"lg"}
            />
          </MenuButton>
          <MenuList>
            <MenuItem
              onClick={() => {
                setEditMode(true);
```

```
                  }}
                >
                  Bearbeiten
                </MenuItem>
                <MenuItem
                  onClick={async () => {
                    if (
                      await PopupManager.confirmAsync({
                        title: "Profil löschen",
                        question:
                          "Bist du dir sicher, dass du dein Profil löschen möchtest? Dies kann nicht
rückgängig gemacht werden.",
                      })
                    )
                      if (profile) {
                        await REST.Forum.deleteProfileV2({
                          username: profile.username,
                          token: localStorage.getItem("token") || "",
                        });
                      }

                    onDeleteProfileClose();
                    router.push("/forum");
                  }}
                >
                  Löschen
                </MenuItem>
              </MenuList>
            </Menu>
          </Box>
        </>
      ) : null}

      <Flex alignItems={"center"}>
        <Avatar
          src={"/api/account/" + profile.username + "/picture"}
          size={"xl"}
        />
        <Box ml={4}>
          {editMode &&
          userInfo.subscription !== "PRO" &&
          userInfo.role !== "admin" ? (
            <>
              <HStack color={"orange.400"}>
                <FaCrown />
                <Text>
                  Personalisiere dein Profil mit <b>CodeUp PRO</b>
                </Text>
              </HStack>
            </>
          ) : null}
          {editMode ? (
            <>
              <Input
                type={"text"}
                id={"edit-display_name"}
                placeholder={"Anzeigename"}
                defaultValue={profile.displayName}
                onChange={(e) => {}}
                disabled={userInfo.subscription !== "PRO"}
```

```
        />
      </>
    ) : (
      <>
        <Heading
          size={"lg"}
          display={"flex"}
          alignItems={"center"}
          gap={"1.2rem"}
        >
          {profile.displayName}
          {profile.verified ? (
            <>
              <chakra.span color={"brand.500"}>
                <FaCheckCircle />
              </chakra.span>
            </>
          ) : null}
        </Heading>
      </>
    )}
    <Text color={"gray"}>@{profile.username}</Text>
  </Box>
</Flex>
<Box mt={4}>
  {!editMode ? (
    <>
      <HStack>
        {
          // @ts-ignore
          profile.badges.map((b: string) => (
            <>
              <Tooltip label={"Der Benuter ist ein " + b}>
                <Image
                  src={
                    "/_static/vectors/forum/badges/" + b + ".svg"
                  }
                  alt={b}
                  w={8}
                  h={8}
                />
              </Tooltip>
            </>
          ))
        }
      </HStack>
    </>
  ) : null}
</Box>
<Box mt={4}>
  <UnorderedList>
    {editMode ? (
      <>
        <InputGroup mt={4}>
          <InputLeftAddon>
            <FaGlobe />
          </InputLeftAddon>
          <Input
            type={"text"}
            id={"edit-website"}
```

```
          placeholder={"Webseite"}
          defaultValue={profile.website}
          disabled={userInfo.subscription !== "PRO"}
        />
      </InputGroup>
    </>
  ) : null}
  {profile.website && !editMode ? (
    <>
      <Link
        href={profile.website}
        target={"_blank"}
        mt={"1rem"}
      >
        <ListItem
          display={"flex"}
          gap={"1.2rem"}
          alignItems={"center"}
        >
          <FaGlobe />
          Webseite
        </ListItem>
      </Link>
    </>
  ) : null}
  {editMode ? (
    <>
      <InputGroup mt={4}>
        <InputLeftAddon>
          <FaMapMarkerAlt />
        </InputLeftAddon>
        <Input
          type={"text"}
          id={"edit-location"}
          placeholder={"Ort"}
          defaultValue={profile.location}
          disabled={userInfo.subscription !== "PRO"}
        />
      </InputGroup>
    </>
  ) : null}
  {profile.location && !editMode ? (
    <>
      <ListItem
        display={"flex"}
        gap={"1.2rem"}
        mt={"1rem"}
        alignItems={"center"}
      >
        <FaMapMarkerAlt />
        {profile.location}
      </ListItem>
    </>
  ) : null}

  <ListItem
    display={"flex"}
    gap={"1.2rem"}
    mt={"1rem"}
    alignItems={"center"}
```

```
    >
      <FaBirthdayCake />
      Mitglied seit: 
      {new Date(profile.createdAt).toLocaleDateString()}
    </ListItem>
  </UnorderedList>
</Box>
<Box mt={4}>
  {editMode ? (
    <>
      <Textarea
        disabled={userInfo.subscription !== "PRO"}
        defaultValue={profile.bio}
        id={"edit-bio"}
      />
    </>
  ) : (
    <Text>{profile.bio}</Text>
  )}
</Box>
<Box mt={4}>
  {editMode ? (
    <>
      <Grid
        templateColumns={[
          "repeat(1, 1fr)",
          "repeat(2, 1fr)",
          "repeat(3, 1fr)",
        ]}
        gap={6}
        mt={"2rem"}
      >
        {certificates.map((cert) => (
          <>
            <GridItem>
              <Stack
                alignItems={"center"}
                cursor={"pointer"}
                onClick={async () => {
                  let newCerts: typeof profile.certificates =
                    profile.certificates;

                  if (newCerts.find((c) => c._id === cert._id)) {
                    newCerts = newCerts.filter(
                      (c) => c._id !== cert._id
                    );
                  } else {
                    newCerts.push({
                      _id: cert._id,
                      course: cert.course,
                      percentage: cert.percentage,
                      date: cert.date,
                    });
                  }

                  const res = await REST.Forum.modifyProfileV2({
                    token: AccountManager.getActiveToken(),
                    update: {
                      certificates: newCerts.map(
                        (c: any) => c._id
```

```
                ),
              },
              username: profile.username,
            });
            if (res.status === 200) {
              reloadProfile();
              toast({
                title: "Erfolgreich",
                position: "top-right",
                description:
                  "Deine Änderungen wurden gespeichert",
                status: "success",
                duration: 1500,
                variant: "solid",
              });
            } else {
              toast({
                title: "Fehler",
                position: "top-right",
                description:
                  "Deine Änderungen konnten nicht gespeichert werden",
                status: "error",
                duration: 1500,
                variant: "solid",
              });
            }
          }}
        >
          <Image
            src={cert.course.splashImage}
            w={32}
            h={32}
            alt={cert.course.name}
          />
          <Text>{cert.course.name}</Text>
          <Text>
            Aktuell angezeigt: 
            {profile.certificates.find(
              (c) => c._id === cert._id
            )
              ? "Ja"
              : "Nein"}
          </Text>
        </Stack>
      </GridItem>
    </>
  ))}
</Grid>
</>
) : (
<>
  <Heading size={"md"}>Zertifikate</Heading>
  <Grid
    templateColumns={[
      "repeat(1, 1fr)",
      "repeat(2, 1fr)",
      "repeat(3, 1fr)",
    ]}
    gap={6}
    mt={"2rem"}
```

```
      >
        {profile.certificates.map((cert) => {
          console.log(cert);
          return (
            <>
              <GridItem>
                <Stack
                  alignItems={"center"}
                  cursor={"pointer"}
                  onClick={() => {
                    router.push("/user-cert/" + cert._id);
                  }}
                >
                  <Image
                    src={cert.course.splashImage}
                    w={32}
                    h={32}
                    alt={cert.course.name}
                  />
                </Stack>
              </GridItem>
            </>
          );
        })}
      </Grid>
    </>
  )}
</Box>
{editMode ? (
  <Box mt={4}>
    <ButtonGroup w={"100%"} justifyContent={"flex-end"}>
      <Button
        colorScheme={"gray"}
        onClick={() => {
          setEditMode(false);
        }}
      >
        Abbrechen
      </Button>
      <Button
        variant={"brand"}
        isLoading={saving}
        onClick={async () => {
          setSaving(true);

          const displayName = document.getElementById(
            "edit-display_name"
          ) as HTMLInputElement;

          const website = document.getElementById(
            "edit-website"
          ) as HTMLInputElement;

          const location = document.getElementById(
            "edit-location"
          ) as HTMLInputElement;

          const bio = document.getElementById(
            "edit-bio"
          ) as HTMLInputElement;
```

```
          const data = {
            displayName:
              displayName.value === ""
                ? "CodeUp Benutzer"
                : displayName.value,
            website:
              website.value === ""
                ? "https://codeup.space"
                : website.value,
            location:
              location.value === "" ? "CodeUp" : location.value,
            bio: bio.value === "" ? "CodeUp Benutzer" : bio.value,
          };

          const r = await REST.Forum.modifyProfileV2({
            token: localStorage.getItem("token") as string,
            username: profile.username,
            update: {
              displayName:
                data.displayName !== profile.displayName
                  ? data.displayName
                  : undefined,
              website:
                data.website !== profile.website
                  ? data.website
                  : undefined,
              location:
                data.location !== profile.location
                  ? data.location
                  : undefined,
              bio:
                data.bio !== profile.bio ? data.bio : undefined,
            },
          });

          if (r.status === 200) {
            reloadProfile();
            setSaving(false);
            setEditMode(false);
          } else {
            toast({
              title: "Fehler",
              description: r.payload.message,
              status: "error",
              duration: 5000,
              isClosable: true,
              position: "top",
            });
          }
        }}
      >
        Speichern
      </Button>
    </ButtonGroup>
  </Box>
) : null}
<Box mt={8}>
  <Flex w={"100%"} alignItems={"center"}>
    <Heading size={"lg"} flex={"100%"}>
```

```
  Beiträge
</Heading>
{profile.username === userInfo.username ? (
  <>
    <Box flex={"100%"} textAlign={"right"}>
      <Stack>
        <Popover>
          <PopoverTrigger>
            <Button variant={"brand"}>Beitrag</Button>
          </PopoverTrigger>
          <PopoverContent bg={"black"} textAlign={"left"}>
            <PopoverArrow bg={"black"} />
            <PopoverHeader>Neuer Beitrag</PopoverHeader>
            <PopoverBody>
              <Textarea
                id={"new-post_text"}
                placeholder={"Inhalt"}
              />
            </PopoverBody>
            <PopoverFooter>
              <Button
                variant={"brand"}
                onClick={async () => {
                  const content = document.getElementById(
                    "new-post_text"
                  ) as HTMLInputElement;

                  const r = await REST.Forum.createPostV2({
                    token: localStorage.getItem(
                      "token"
                    ) as string,
                    content: content.value,
                  });

                  if (r.status === 200) {
                    reloadPosts();
                    content.value = "";
                  } else {
                    toast({
                      title: "Fehler",
                      description:
                        "Der Beitrag konnte nicht veröffentlicht werden",
                      status: "error",
                      duration: 5000,
                      isClosable: true,
                      position: "top",
                    });
                  }
                }}
              >
                Veröffentlichen
              </Button>
            </PopoverFooter>
          </PopoverContent>
        </Popover>
        <Menu>
          <MenuButton as={Button} variant={"brand"}>
            Medium
          </MenuButton>
          <MenuList>
```

```
            <MenuItem
              icon={<FaImage />}
              onClick={async () => {
                upload({
                  fileNameAsSuffix: false,
                  mimeType: "image/*",
                  prefix: "image",
                });
              }}
            >
              Bild
            </MenuItem>
            <MenuItem
              icon={<FaVideo />}
              onClick={async () => {
                upload({
                  fileNameAsSuffix: false,
                  mimeType: "video/*",
                  prefix: "video",
                });
              }}
            >
              Video
            </MenuItem>
            <MenuItem
              icon={<FaFile />}
              onClick={async () => {
                upload({
                  fileNameAsSuffix: true,
                  mimeType: "*",
                  prefix: "file",
                });
              }}
            >
              Datei
            </MenuItem>
          </MenuList>
        </Menu>
      </Stack>
    </Box>
  </>
 ) : null}
</Flex>
<Box mt={4}>
 {posts.length === 0 ? (
  <Text>Keine Beiträge vorhanden</Text>
 ) : (
  <>
   {posts.map((post) => {
    return (
     <>
      <PostCardV2
       post={post}
       profile={profile}
       postOwner={userInfo.username === profile.username}
       reloadPosts={reloadPosts}
       userId={userInfo.id}
      />
      <Divider />
     </>
```

```
                    );
                  })}
                </>
              )}
            </Box>
          </Box>
        </Box>
      </Flex>
    </>
  ) : null}
 </>
 );
}
```

## apps/web/src/pages/ide/index.tsx

```
/**
 * apps/web/src/pages/ide/index.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 06.07.2022
 *
 */
import * as React from "react";
import { useEffect } from "react";
import redirect from "@codeup/util/src/userworkflow/redirect";
import {
  Box,
  Button,
  Flex,
  Heading,
  Spinner,
  useColorModeValue,
} from "@chakra-ui/react";
import { useRouter } from "next/router";
import { FaPlus, FaRegFolderOpen } from "react-icons/fa";
import CreateProjectMenu from "@codeup/components/ide/CreateProjectMenu";
import OpenProjectMenu from "@codeup/components/ide/OpenProjectMenu";
import Head from "next/head";
import { CHeading } from "@codeup/components/dist/global/CHeading";

export default function Index() {
  useEffect(() => {
    redirect({
      forceAnonymous: false,
      allowAnonymous: false,
      redirectToLogin: true,
      redirectTo: "/ide",
    });
  });

  const [currentComponent, setCurrentComponent] =
    React.useState<JSX.Element | null>(
      <>
        <Spinner size="xl" />
      </>
    );

  const [currentSelected, setCurrentSelected] =
```

```
  React.useState<string>("create");
const router = useRouter();

useEffect(() => {
  const query = new URLSearchParams(window.location.search);
  if (query.has("component")) {
    switch (query.get("component")) {
      case "create":
        setCurrentComponent(<CreateProjectMenu />);
        setCurrentSelected("create");
        break;
      default:
        setCurrentComponent(<OpenProjectMenu />);
        setCurrentSelected("open");
        break;
    }
  } else {
    router.push("/ide?component=create");
  }
}, [router]);

return (
  <>
    <Head>
      <title>Editor - CodeUp</title>
    </Head>
    <Flex direction={["column", "row"]} minW={"100%"} minH={"100vh"}>
      <Flex
        direction={"column"}
        flex={"15%"}
        minH={"100%"}
        bgColor={"gray.900"}
        p={"1.2rem"}
      >
        <Box textAlign={"center"}>
          <CHeading size={"xl"}>Projekte</CHeading>
        </Box>
        <Button
          m={"1.2rem"}
          justifyContent={"flex-start"}
          variant={currentSelected === "create" ? "brand" : "ghost"}
          leftIcon={<FaPlus />}
          onClick={() => {
            router.push("/ide?component=create");
          }}
        >
          Neu
        </Button>
        <Button
          marginInline={"1.2rem"}
          justifyContent={"flex-start"}
          variant={currentSelected === "open" ? "brand" : "ghost"}
          leftIcon={<FaRegFolderOpen />}
          onClick={() => {
            setCurrentSelected("account");
            router.push("/ide?component=open");
          }}
        >
          Meine Projekte
        </Button>
```

```
        </Flex>
        <Flex direction={"column"} flex={"85%"} minH={"100%"} bg={"#000"}>
          {currentComponent}
        </Flex>
      </Flex>
    </>
  );
}
```

## apps/web/src/pages/ide/project/[id].tsx

```
/**
 * apps/web/src/pages/ide/project/[id].tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 08.07.2022
 *
 */

import * as React from "react";
import { useEffect, useRef, useState } from "react";
import { useRouter } from "next/router";
import redirect from "@codeup/util/src/userworkflow/redirect";
import {
  Alert,
  AlertDescription,
  AlertIcon,
  AlertTitle,
  Box,
  Button,
  ButtonGroup,
  Center,
  Drawer,
  DrawerBody,
  DrawerCloseButton,
  DrawerContent,
  DrawerFooter,
  DrawerHeader,
  DrawerOverlay,
  Flex,
  FormControl,
  FormLabel,
  Heading,
  IconButton,
  Input,
  Menu,
  MenuButton,
  MenuItem,
  MenuList,
  NumberDecrementStepper,
  NumberIncrementStepper,
  NumberInput,
  NumberInputField,
  NumberInputStepper,
  Popover,
  PopoverArrow,
  PopoverBody,
  PopoverCloseButton,
  PopoverContent,
```

```
  PopoverHeader,
  PopoverTrigger,
  Stack,
  Text,
  Tooltip,
  useDisclosure,
  useToast,
} from "@chakra-ui/react";
import {
  FaCloudUploadAlt,
  FaCodeBranch,
  FaCogs,
  FaEye,
  FaFile,
  FaFileCode,
  FaHome,
  FaKeyboard,
  FaUserFriends,
} from "react-icons/fa";
import Editor, { useMonaco } from "@monaco-editor/react";
import { FiEdit, FiFilePlus, FiTrash2 } from "react-icons/fi";
import REST from "@codeup/rest";
import { BiGitRepoForked, BiRename } from "react-icons/bi";
import {
  ContributorManager,
  KeyboardShortcuts,
  PreviewModal,
  SnippetViewer,
} from "@codeup/components/ide";
import { useKeyPress, useLoggedIn } from "@codeup/hooks";
import {
  SiCss3,
  SiHtml5,
  SiJavascript,
  SiMarkdown,
  SiSass,
  SiTypescript,
} from "react-icons/si";
import FileManager from "@codeup/components/ide/FileManager";
import PublishManager from "@codeup/components/ide/PublishManager";
import Head from "next/head";

export default function Index() {
  const router = useRouter();
  const toast = useToast();
  const [firstLoad, setFirstLoad] = useState(true);

  const LANGUAGES: {
    [key: string]: {
      icon: any;
      language: string;
    };
  } = {
    md: {
      language: "markdown",
      icon: <SiMarkdown />,
    },
    js: {
      language: "javascript",
      icon: <SiJavascript />,
```

```
  },
  html: {
    language: "html",
    icon: <SiHtml5 />,
  },
  css: {
    language: "css",
    icon: <SiCss3 />,
  },
  ts: {
    language: "typescript",
    icon: <SiTypescript />,
  },
  scss: {
    language: "scss",
    icon: <SiSass />,
  },
  sass: {
    language: "sass",
    icon: <SiSass />,
  },
  ic: {
    language: "codeup-incode",
    icon: <FaHome />,
  },
  _default: {
    language: "plain",
    icon: <FaFile />,
  },
};

const [editorOptions, setEditorOptions] = useState<{
  fontSize: number;
}>({
  fontSize: 22,
});

useEffect(() => {
  if (localStorage.getItem("monaco.props") !== null) {
    setEditorOptions(
      JSON.parse(localStorage.getItem("monaco.props") as string)
    );
  } else {
    localStorage.setItem("monaco.props", JSON.stringify(editorOptions));
  }
}, []);

const [currentFile, setCurrentFile] = useState<{
  path: string;
  content: string;
} | null>(null);

const {
  isOpen: isFileMenuOpen,
  onOpen: onFileMenuOpen,
  onClose: onFileMenuClose,
} = useDisclosure();

const {
  isOpen: isContributorOpen,
```

```
  onOpen: onContributorOpen,
  onClose: onContributorClose,
} = useDisclosure();

const {
  isOpen: isPublishOpen,
  onOpen: onPublishOpen,
  onClose: onPublishClose,
} = useDisclosure();

const [errorAlert, setErrorAlert] = useState<{
  show: boolean;
  error: string;
}>({
  show: false,
  error: "",
});

const [project, setProject] = useState<{
  _id: string;
  owner: string;
  name: string;
  contributors: {
    user: string;
    permissions: string[];
  }[];
  files: {
    path: string;
    content: string;
  }[];
  published: boolean;
  publishedCustomName: string;
} | null>(null);

const { userInfo, loggedIn, loaded } = useLoggedIn();

useEffect(() => {
  if (!loggedIn && loaded) {
    router.push("/");
    window.dispatchEvent(new CustomEvent("codeup::open_login"));
  }
}, [router, userInfo, loggedIn, loaded]);

useEffect(() => {
  if (router.query.id) {
    REST.IDE.getProject({
      token: localStorage.getItem("token") as string,
      projectId: router.query.id as string,
    }).then((res) => {
      if (res.status === 200) {
        setProject(res.payload.project);
      } else {
        setErrorAlert({
          error: res.payload.error,
          show: true,
        });
      }
    });
  }
}, [router]);
```

```
const editorRef = useRef(null);
const monaco = useMonaco();
const [monacoTheme, setMonacoTheme] = useState("vs-dark");

useEffect(() => {
  if (monaco && monaco.editor && project && project.files) {
    monaco.editor.defineTheme("codeup-dark", {
      base: "vs-dark",
      inherit: true,
      rules: [],
      colors: {
        "editor.background": "#141517",
      },
    });
    monaco.languages.typescript.javascriptDefaults.setCompilerOptions({
      target: monaco.languages.typescript.ScriptTarget.ESNext,
      allowNonTsExtensions: true,
    });

    monaco.languages.register({ id: "codeup-incode" });

    monaco.languages.setMonarchTokensProvider("codeup-incode", {
      ignoreCase: true,
      tokenizer: {
        root: [
          [/erstelle/, "command"],
          [/addiere/, "command"],
          [/subtrahiere/, "command"],
          [/multipliziere/, "command"],
          [/dividiere/, "command"],
          [/füge/, "command"],
          [/entferne/, "command"],
          [/gib/, "command"],
          [/frage/, "command"],
          [/funktion/, "command"],
          [/rufe/, "command"],
          [/setze/, "command"],
          [/wiederhole/, "command"],
          [/wenn/, "command"],
          [/sonst/, "command"],
          [/stopp/, "command"],
          [/funktion/, "command"],

          [/#([a-f0-9]{3}|[a-f0-9]{6})/, "value"],

          [/\/\/.*/, "comment"],
          [/#.*/, "comment"],
          [/!.*/, "comment"],

          [/".*"/, "data"],

          [/\d+/, "data"],

          [/in/, "secondary"],
          [/der/, "secondary"],
          [/die/, "secondary"],
          [/das/, "secondary"],
          [/den/, "secondary"],
          [/dem/, "secondary"],
```

```
[/auf/, "secondary"],
[/aus/, "secondary"],
[/als/, "secondary"],
[/mit/, "secondary"],
[/zum/, "secondary"],
[/zu/, "secondary"],
[/von/, "secondary"],
[/durch/, "secondary"],
[/hinzu/, "secondary"],
[/von/, "secondary"],
[/vom/, "secondary"],
[/speicher/, "secondary"],
[/Antwort/, "secondary"],
[/ist/, "secondary"],
[/wird/, "secondary"],

[/und/, "condition-joiner"],
[/oder/, "condition-joiner"],

[/gleich/, "operator"],
[/ungleich/, "operator"],
[/kleiner/, "operator"],
[/kleiner-gleich/, "operator"],
[/größer/, "operator"],
[/größer-gleich/, "operator"],

[/unterstrichen/, "value"],
[/überstrichen/, "value"],
[/durchgestrichen/, "value"],
[/unterstrichen-überstrichen/, "value"],
[/links/, "value"],
[/zentriert/, "value"],
[/rechts/, "value"],
[/normal/, "value"],
[/fett/, "value"],
[/schwächer/, "value"],
[/leichter/, "value"],
[/absolut/, "value"],
[/relativ/, "value"],
[/fest/, "value"],
[/keine/, "value"],
[/durchgezogen/, "value"],
[/gestrichen/, "value"],
[/gepunktet/, "value"],
[/doppelt/, "value"],
[/unsichtbar/, "value"],
[/sichtbar/, "value"],

[/solange/, "type"],
[/taste/, "type"],
[/variable/, "type"],
[/dialogbox/, "type"],
[/bildschirm/, "type"],
[/wert/, "type"],
[/farbe/, "type"],
[/hintergrundfarbe/, "type"],
[/textdekoration/, "type"],
[/textausrichtung/, "type"],
[/text/, "type"],
[/schriftbreite/, "type"],
```

[/schriftart/, "type"],
[/abstand-oben/, "type"],
[/abstand-unten/, "type"],
[/abstand-rechts/, "type"],
[/abstand-links/, "type"],
[/abstand/, "type"],
[/innerer-abstand-oben/, "type"],
[/innerer-abstand-unten/, "type"],
[/innerer-abstand-rechts/, "type"],
[/innerer-abstand-links/, "type"],
[/innerer-abstand/, "type"],
[/schriftgröße/, "type"],
[/breite/, "type"],
[/höhe/, "type"],
[/quelle/, "type"],
[/position/, "type"],
[/umrandungsbreite-oben/, "type"],
[/umrandungsbreite-unten/, "type"],
[/umrandungsbreite-links/, "type"],
[/umrandungsbreite-rechts/, "type"],
[/umrandungsbreite/, "type"],
[/umrandungsstil-oben/, "type"],
[/umrandungsstil-unten/, "type"],
[/umrandungsstil-links/, "type"],
[/umrandungsstil-rechts/, "type"],
[/umrandungsstil/, "type"],
[/umrandungsradius/, "type"],
[/umrandungsradius-oben-links/, "type"],
[/umrandungsradius-oben-rechts/, "type"],
[/umrandungsradius-unten-links/, "type"],
[/umrandungsradius-unten-rechts/, "type"],
[/umrandungsfarbe/, "type"],
[/umrandungsfarbe-oben/, "type"],
[/umrandungsfarbe-rechts/, "type"],
[/umrandungsfarbe-links/, "type"],
[/umrandungsfarbe-unten/, "type"],
[/sichtbarkeit/, "type"],

[/knopf/, "type"],
[/textfeld/, "type"],
[/absatz/, "type"],
[/text/, "type"],
[/bild/, "type"],
[/überschrift/, "type"],
[/überschrift2/, "type"],
[/überschrift3/, "type"],
[/überschrift4/, "type"],
[/überschrift5/, "type"],
[/überschrift6/, "type"],
[/tabelle/, "type"],
[/zeile/, "type"],
[/spalte/, "type"],
[/zeilenumbruch/, "type"],
[/video/, "type"],
[/audio/, "type"],
[/fenster/, "type"],
[/container/, "type"],
[/listenpunkt/, "type"],
[/liste/, "type"],
[/ungeordnete-liste/, "type"],

```
      [/geordnete-liste/, "type"],
      [/link/, "type"],

      [/gedrückt/, "event"],
      [/berührt/, "event"],
      [/verlassen/, "event"],
    ],
  },
});

monaco.editor.defineTheme("codeup-dark-incode", {
  base: "vs-dark",
  inherit: true,
  rules: [
    {
      token: "command",
      foreground: "#ffa500",
      fontStyle: "bold",
    },
    {
      token: "comment",
      foreground: "#808080",
    },
    {
      token: "data",
      foreground: "#FF00FF",
    },
    {
      token: "secondary",
      foreground: "#545454",
    },
    {
      token: "type",
      foreground: "#38d920",
    },
    {
      token: "operator",
      foreground: "#d92020",
      fontStyle: "italic",
    },
    {
      token: "condition-joiner",
      foreground: "#2086d9",
    },
    {
      token: "value",
      foreground: "#d9b420",
    },
    {
      token: "event",
      foreground: "#20d992",
    },
  ],
  colors: {
    "editor.background": "#141517",
  },
});

setMonacoTheme("codeup-dark-incode");
```

```
    if (
      project.files.find((f) => f.path.toLowerCase() === "/readme.md") &&
      firstLoad
    ) {
      setFirstLoad(false);
      setCurrentFile({
        path: (
          project.files.find(
            (f) => f.path.toLowerCase() === "/readme.md"
          ) as any
        ).path,
        content: (
          project.files.find(
            (f) => f.path.toLowerCase() === "/readme.md"
          ) as any
        ).content,
      });
    }
  }
}, [monaco, project]);

const saveKeyPress = useKeyPress("s", true);
const openKeyPress = useKeyPress("o", true);

useEffect(() => {
  if (saveKeyPress && currentFile) {
    save();
  }
}, [saveKeyPress, currentFile]);

useEffect(() => {
  if (openKeyPress && !isFileMenuOpen) {
    onFileMenuOpen();
  }
}, [openKeyPress]);

function handleEditorDidMount(editor: any, monaco: any) {
  editorRef.current = editor;
}

const [permissions, setPermissions] = useState<string[]>([]);

useEffect(() => {
  if (project && project._id) {
    REST.IDE.getProjectPermissions({
      token: localStorage.getItem("token") as string,
      projectId: project._id,
    }).then((res) => {
      setPermissions(res.payload.permissions);
    });
  }
}, [project]);

const {
  isOpen: isPreviewOpen,
  onOpen: onPreviewOpen,
  onClose: onPreviewClose,
} = useDisclosure();

const {
```

```
  isOpen: isKSOpen,
  onOpen: onKSOpen,
  onClose: onKSClose,
} = useDisclosure();

const {
  isOpen: isCSOpen,
  onOpen: onCSOpen,
  onClose: onCSClose,
} = useDisclosure();

const [settingsOpen, setSettingsOpen] = useState<boolean>(false);

const save = async () => {
  const res = await REST.IDE.saveFile({
    token: localStorage.getItem("token") as string,
    path: currentFile?.path as string,
    content:
      // @ts-ignore
      editorRef?.current?.getValue() as string,
    projectId: project?._id as string,
  });
  if (res.status === 200) {
    let f = project?.files;

    if (f && f.find((p) => p.path === (currentFile?.path as string))) {
      (
        f.find((p) => p.path === (currentFile?.path as string)) as {
          content: string;
          path: string;
        }
      ).content =
        // @ts-ignore
        editorRef?.current?.getValue();
    }

    if (project && f) {
      setProject({
        _id: project?._id,
        contributors: project?.contributors,
        owner: project?.owner,
        name: project?.name,
        files: f,
        published: project?.published,
        publishedCustomName: project?.publishedCustomName,
      });
    }

    toast({
      status: "success",
      variant: "solid",
      position: "top",
      title: "Erfolgreich",
      description: "Die Datei wurde erfolgreich gespeichert",
      isClosable: true,
      duration: 4000,
    });
  } else {
    toast({
      status: "error",
```

```
      variant: "solid",
      position: "top",
      title: "Fehler",
      description: res.payload.error,
      isClosable: true,
      duration: 4000,
    });
  }
};

return (
  <>
    <Head>
      <title>
        {currentFile
          ? currentFile.path.split("/")[
              currentFile.path.split("/").length - 1
            ]
          : "Neue Datei"}
         - Editor - CodeUp
      </title>
    </Head>
    {errorAlert.show ? (
      <Alert status="error" variant={"top-accent"}>
        <AlertIcon />
        <AlertTitle>Fehler!</AlertTitle>
        <AlertDescription>{errorAlert.error}</AlertDescription>
      </Alert>
    ) : (
      <>
        <Flex
          direction={"row"}
          w={"100%"}
          minH={"100vh"}
          id={"editor-wrapper-flex"}
          bg={"#000"}
        >
          <Box
            flex={"5%"}
            minH={"90vh"}
            bgColor={"gray.900"}
            w={"100%"}
            margin={"2rem"}
            rounded={"xl"}
            shadow={"2xl"}
          >
            <Flex direction={"column"} h={"100%"}>
              <Tooltip label={"Dateien"} placement={"right"} hasArrow>
                <IconButton
                  aria-label={""}
                  variant="ghost"
                  size={"lg"}
                  p={"2rem"}
                  icon={<FaFile />}
                  rounded={"none"}
                  roundedTopLeft={"xl"}
                  roundedTopRight={"xl"}
                  onClick={onFileMenuOpen}
                />
              </Tooltip>
```

```
{!permissions.includes("WRITE") &&
!permissions.includes("OWNER") ? (
  <Tooltip label={"Fork"} placement={"right"} hasArrow>
    <IconButton
      aria-label={""}
      variant="ghost"
      size={"lg"}
      p={"2rem"}
      icon={<BiGitRepoForked />}
      rounded={"none"}
      onClick={async () => {
        const projectTemplateRequest =
          await REST.IDE.getProject({
            token: localStorage.getItem("token") as string,
            projectId: project?._id as string,
          });

        if (projectTemplateRequest.status === 200) {
          const newProjectRequest =
            await REST.IDE.createProject({
              token: localStorage.getItem("token") as string,
              name:
                "Fork von " +
                projectTemplateRequest.payload.project.name,
            });

          if (newProjectRequest.status === 200) {
            for (const file of projectTemplateRequest.payload
              .project.files) {
              await REST.IDE.saveFile({
                token: localStorage.getItem("token") as string,
                projectId:
                  newProjectRequest.payload.project._id,
                path: file.path,
                content: file.content,
              });

              router.push(
                "/ide/project/" +
                  newProjectRequest.payload.project._id
              );
            }
          } else {
            toast({
              status: "error",
              variant: "solid",
              position: "top",
              title: "Fehler",
              description: projectTemplateRequest.payload.error,
              isClosable: true,
              duration: 4000,
            });
          }
        } else {
          toast({
            status: "error",
            variant: "solid",
            position: "top",
            title: "Fehler",
            description: projectTemplateRequest.payload.error,
```

```
            isClosable: true,
            duration: 4000,
          });
        }
      }}
    />
  </Tooltip>
) : null}
{permissions.includes("WRITE") ||
permissions.includes("OWNER") ? (
  <Tooltip label={"Code Snippets"} placement={"right"} hasArrow>
    <IconButton
      aria-label={""}
      variant="ghost"
      size={"lg"}
      p={"2rem"}
      icon={<FaFileCode />}
      rounded={"none"}
      onClick={async () => {
        onCSOpen();
      }}
    />
  </Tooltip>
) : null}
{permissions.includes("MANAGE-CONTRIBUTORS") ||
permissions.includes("OWNER") ? (
  <Tooltip label={"Mitwirkende"} placement={"right"} hasArrow>
    <IconButton
      aria-label={""}
      variant="ghost"
      size={"lg"}
      p={"2rem"}
      icon={<FaUserFriends />}
      rounded={"none"}
      onClick={() => {
        onContributorOpen();
      }}
    />
  </Tooltip>
) : null}
{permissions.includes("PUBLISH") ||
permissions.includes("OWNER") ? (
  <Tooltip
    label={"Veröffentlichen"}
    placement={"right"}
    hasArrow
  >
    <IconButton
      aria-label={""}
      variant="ghost"
      size={"lg"}
      p={"2rem"}
      icon={<FaCloudUploadAlt />}
      rounded={"none"}
      onClick={() => {
        onPublishOpen();
      }}
    />
  </Tooltip>
) : null}
```

```
{currentFile &&
!settingsOpen &&
(currentFile.path.toLowerCase().endsWith(".html") ||
  currentFile.path.toLowerCase().endsWith(".ic")) ? (
  <Tooltip label={"Vorschau"} placement={"right"} hasArrow>
    <IconButton
      aria-label={""}
      variant="ghost"
      size={"lg"}
      p={"2rem"}
      icon={<FaEye />}
      rounded={"none"}
      onClick={async () => {
        await save();
        document.cookie =
          (("jwt=" + localStorage.getItem("token")) as string) +
          "; path=/";
        onPreviewOpen();
      }}
    />
  </Tooltip>
) : null}
<Tooltip label={"Tastenkürzel"} placement={"right"} hasArrow>
  <IconButton
    aria-label={""}
    variant="ghost"
    size={"lg"}
    p={"2rem"}
    icon={<FaKeyboard />}
    rounded={"none"}
    onClick={() => {
      onKSOpen();
    }}
  />
</Tooltip>
<Tooltip label={"Einstellungen"} placement={"right"} hasArrow>
  <IconButton
    aria-label={""}
    variant="ghost"
    size={"lg"}
    p={"2rem"}
    icon={<FaCogs />}
    rounded={"none"}
    onClick={() => {
      if (currentFile) {
        REST.IDE.saveFile({
          token: localStorage.getItem("token") as string,
          projectId: project?._id as string,
          path: currentFile?.path,
          // @ts-ignore
          content: editorRef?.current?.getValue(),
        }).then(async () => {
          const r = await REST.IDE.getProject({
            token: localStorage.getItem("token") as string,
            projectId: project?._id as string,
          });

          setProject(r.payload.project);
        });
      }
```

```
          setSettingsOpen(!settingsOpen);
        }}
      />
    </Tooltip>
    <Tooltip label={"Hautpmenü"} placement={"right"} hasArrow>
      <IconButton
        aria-label={""}
        variant="ghost"
        size={"lg"}
        p={"2rem"}
        icon={<FaHome />}
        rounded={"none"}
        onClick={() => {
          router.push("/ide?component=open");
        }}
      />
    </Tooltip>
  </Flex>
</Box>
<Box
  flex={"95%"}
  minH={"90vh"}
  bgColor={"gray.900"}
  w={"100%"}
  maxW={"100%"}
  margin={"2rem"}
  rounded={"xl"}
  shadow={"2xl"}
>
  <Flex
    direction={"column"}
    w={"100%"}
    h={"100%"}
    p={".6rem"}
    gap={".4rem"}
  >
    {settingsOpen ? (
      <>
        <Heading size={"lg"} textAlign={"center"}>
          Einstellungen
        </Heading>
        <Box maxH={"85vh"} h={"100%"} overflow={"auto"}>
          <Flex direction={"column"}>
            <Box padding={"2rem"} maxW={["100%", "25%"]}>
              <Heading size={"md"}>Schriftgröße</Heading>
              <NumberInput
                defaultValue={editorOptions.fontSize}
                mt={"1.2rem"}
                onChange={(s, n) => {
                  let x = editorOptions;

                  x.fontSize = n;

                  setEditorOptions(x);

                  localStorage.setItem(
                    "monaco.props",
                    JSON.stringify(x)
                  );
                }}
```

```
          >
            <NumberInputField />
            <NumberInputStepper>
              <NumberIncrementStepper />
              <NumberDecrementStepper />
            </NumberInputStepper>
          </NumberInput>
        </Box>
      </Flex>
    </Box>
  </>
) : null}
{!settingsOpen ? (
  <>
    {currentFile ? (
      <>
        <Box flex={"5%"}>
          <ButtonGroup>
            <Button
              variant={"ghost"}
              onClick={async () => {
                save();
              }}
            >
              Speichern
            </Button>
          </ButtonGroup>
        </Box>
        <Box flex={"95%"}>
          <Editor
            defaultLanguage={
              LANGUAGES[
                currentFile?.path.split(".").pop() as string
              ]
                ? LANGUAGES[
                    currentFile?.path.split(".").pop() as string
                  ].language
                : LANGUAGES["_default"].language
            }
            defaultValue={currentFile?.content}
            onMount={handleEditorDidMount}
            theme={monacoTheme}
            options={{
              insertSpaces: false,
              wrappingIndent: "indent",
              fontSize: editorOptions.fontSize,
              readOnly:
                !permissions.includes("WRITE") &&
                !permissions.includes("OWNER"),
            }}
          />
        </Box>
      </>
    ) : (
      <>
        <Center>
          <Text
            color="blue.500"
            onClick={() => onFileMenuOpen()}
            cursor="pointer"
```

```
                    >
                      Öffne eine Datei
                    </Text>
                    , damit du diese bearbeiten/lesen kannst.
                  </Center>
                </>
              )}
            </>
          ) : null}
        </Flex>
      </Box>
    </Flex>
    <FileManager
      onFileMenuClose={onFileMenuClose}
      isFileMenuOpen={isFileMenuOpen}
      permissions={permissions}
      project={project}
      setProject={setProject}
      currentFile={currentFile}
      setCurrentFile={setCurrentFile}
      setSettingsOpen={setSettingsOpen}
      editorRef={editorRef}
      monaco={monaco}
    />
    <PreviewModal
      project={project?._id}
      path={currentFile?.path}
      isOpen={isPreviewOpen}
      onOpen={onPreviewOpen}
      onClose={onPreviewClose}
    />
    <ContributorManager
      isOpen={isContributorOpen}
      onOpen={onContributorOpen}
      onClose={onContributorClose}
      projectId={project?._id}
    />
    <KeyboardShortcuts
      isOpen={isKSOpen}
      onOpen={onKSOpen}
      onClose={onKSClose}
    />
    <PublishManager
      projectId={project?._id}
      isOpen={isPublishOpen}
      onClose={onPublishClose}
      onOpen={onPublishOpen}
    />
    <SnippetViewer isOpen={isCSOpen} onClose={onCSClose} />
  </>
 )}
</>
 );
}
```

## apps/web/src/pages/ideas/index.tsx

```
/**
 * apps/web/src/pages/ideas/index.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
```

```
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 03.12.2022
 *
 */

import * as React from "react";
import { useEffect } from "react";
import { useLoggedIn } from "@codeup/hooks";
import AdminLayout from "@codeup/components/admin/AdminLayout";
import REST from "@codeup/rest";
import {
  Badge,
  Box,
  Button,
  ButtonGroup,
  Flex,
  FormControl,
  FormLabel,
  Grid,
  Heading,
  Input,
  Popover,
  PopoverArrow,
  PopoverBody,
  PopoverCloseButton,
  PopoverContent,
  PopoverTrigger,
  Portal,
  Stack,
  Text,
  Textarea,
  useToast,
} from "@chakra-ui/react";
import { FaPlus } from "react-icons/fa";
import { Card, CardBody, CardFooter } from "@chakra-ui/card";
import { useRouter } from "next/router";
import Head from "next/head";
import { CHeading } from "@codeup/components/dist/global/CHeading";

export default function Viewer() {
  const [projectIdeas, setProjectIdeas] = React.useState<
    {
      _id: string;
      title: string;
      description: string;
      exampleDeployedURL?: string;
      exampleSourceURL?: string;
      level: string;
    }[]
  >([]);

  const [filteredProjectIdeas, setFilteredProjectIdeas] = React.useState<
    {
      _id: string;
      title: string;
      description: string;
      exampleDeployedURL?: string;
      exampleSourceURL?: string;
      level: string;
```

```
  }[]
>([]);

const toast = useToast();
const router = useRouter();

const reload = () => {
  REST.Ideas.getProjectIdeas().then((res) => {
    if (res.status === 200) {
      setProjectIdeas(res.payload.ideas);
      setFilteredProjectIdeas(res.payload.ideas);
    } else {
      toast({
        title: "Fehler",
        description: "Ein Fehler ist aufgetreten: " + res.payload.error,
        status: "error",
        duration: 1500,
        isClosable: true,
        position: "bottom-right",
      });
    }
  });
};

useEffect(() => {
  reload();
}, []);

const [query, setQuery] = React.useState<string>("");

useEffect(() => {
  setFilteredProjectIdeas(
    projectIdeas.filter((idea) =>
      idea.title.toLowerCase().includes(query.toLowerCase())
    )
  );
}, [query]);

return (
  <>
    <Head>
      <title>Projektideen - CodeUp</title>
    </Head>
    <Box bg={"#000"} h={"fit-content"} minH={"100vh"}>
      <Flex
        w={"100%"}
        pt={"2rem"}
        alignItems={"center"}
        justifyContent={"center"}
        direction={"column"}
      >
        <Box textAlign={"center"} pt={4}>
          <CHeading size={"2xl"}>Projektideen</CHeading>
        </Box>
        <Input
          placeholder={"Suche nach Projektideen"}
          value={query}
          onChange={(e) => {
            setQuery(e.target.value);
          }}
```

```
    w={"50%"}
    mt={"2rem"}
   />
  </Flex>
  <Grid
   templateColumns={
    filteredProjectIdeas.length > 0
     ? [
        "repeat(1, 1fr)",
        "repeat(2, 1fr)",
        "repeat(3, 1fr)",
        "repeat(4, 1fr)",
        "repeat(5, 1fr)",
       ]
     : "repeat(1, 1fr)"
   }
   gap={6}
   p={10}
  >
   {filteredProjectIdeas.map((idea) => (
    <>
     <Card
      maxW="sm"
      h={"100%"}
      bgColor={"gray.900"}
      rounded={"xl"}
      boxShadow={"xl"}
      p={5}
      id={"_card_" + idea._id}
     >
      <CardBody>
       <Stack mt={2} spacing="3">
        <Heading size="md">{idea.title}</Heading>
        <Badge
         w={"100%"}
         fontSize={"lg"}
         colorScheme={
          idea.level === "easy"
           ? "green"
           : idea.level === "medium"
           ? "yellow"
           : "red"
         }
        >
         {idea.level === "easy"
          ? "Einfach"
          : idea.level === "medium"
          ? "Mittel"
          : "Schwer"}
        </Badge>
        <Text>{idea.description}</Text>
       </Stack>
      </CardBody>
      <CardFooter mt={4}>
       <ButtonGroup spacing="2">
        {idea.exampleDeployedURL &&
        idea.exampleDeployedURL !== "" ? (
         <>
          <Button
           colorScheme="blue"
```

```
                    variant={"solid"}
                    onClick={() => {
                      window.open(idea.exampleDeployedURL, "_blank");
                    }}
                  >
                    Beispiel
                  </Button>
                </>
              ) : null}
              {idea.exampleSourceURL && idea.exampleSourceURL !== "" ? (
                <>
                  <Button
                    variant="ghost"
                    colorScheme="blue"
                    onClick={() => {
                      window.open(idea.exampleSourceURL, "_blank");
                    }}
                  >
                    Source Code
                  </Button>
                </>
              ) : null}
            </ButtonGroup>
          </CardFooter>
        </Card>
      </>
    ))}
    {filteredProjectIdeas.length === 0 ? (
      <>
        <Text
          textAlign={"center"}
          fontSize={"2xl"}
          color={"gray.500"}
          mt={4}
        >
          Keine Projekt Ideen gefunden!
        </Text>
      </>
    ) : null}
  </Grid>
 </Box>
 </>
 );
}
```

## apps/web/src/pages/index.tsx

```
"use client";

/**
 * apps/web/src/pages/index.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 16.12.2022
 *
 */

import * as React from "react";
```

```
import {
  Badge,
  Box,
  Button,
  ButtonGroup,
  Flex,
  Heading,
  Image,
  Stack,
  Text,
} from "@chakra-ui/react";
import { motion, useScroll } from "framer-motion";
import Typed from "react-typed";
import { useLoggedIn } from "@codeup/hooks";
import { useRouter } from "next/router";
import Head from "next/head";
import OverviewSection from "@codeup/components/landingpage/OverviewSection";
import BrandStats from "@codeup/components/landingpagev2/BrandStats";
import Testimonials from "@codeup/components/landingpagev2/Testimonials";
import Milestones from "@codeup/components/landingpagev2/Milestones";

export default function Home() {
  const { loggedIn } = useLoggedIn();
  const router = useRouter();

  const { scrollYProgress } = useScroll();

  return (
    <>
      <Head>
        <title>CodeUp - Einfach Programmieren lernen</title>
      </Head>
      <Flex
        w={"100%"}
        minH={"100vh"}
        h={"fit-content"}
        bg={"transparent"}
        overflow={"hidden"}
      >
        <img
          // src={"/_static/images/software-developer-6521720_1920.jpg"}
          src={"/_static/images/sf-developer.jpg"}
          alt={""}
          className={"index-header-image"}
        />
        <Flex
          w={"100%"}
          h={"fit-content"}
          p={[5, 10, 20]}
          direction={["column", "column", "row"]}
          alignItems={"center"}
          minH={"100vh"}
          overflow={"hidden"}
        >
          <Flex
            flex={["100%", "100%", "100%"]}
            alignItems={"center"}
            w={"100%"}
            justifyContent={"center"}
            zIndex={2}
```

```
        >
          <Flex gap={2} pt={10} direction={"column"}>
            <Heading
              as={"h3"}
              fontSize={["2xl", "3xl", "4xl"]}
              color={"gray.300"}
              fontWeight={1000}
              letterSpacing={-2}
            >
              CodeUp
            </Heading>
            <Heading
              as={"h1"}
              fontSize={["4xl", "5xl", "8xl"]}
              fontWeight={900}
              letterSpacing={-2}
              fontFamily={"Jetbrains Mono ExtraBold"}
              color={"brand.500"}
            >
              Programmieren lernen
            </Heading>
            <Text
              fontWeight={[400, 400, 100]}
              fontSize={["xl", "2xl", "4xl"]}
              maxW={["100%", "100%", "60%"]}
            >
              Von der ersten Zeile Code bis zum eigenen Projekt. Wir
              unterstützen dich auf deinem Weg mit allem was du brauchst.
            </Text>
            <ButtonGroup spacing={4} mt={2}>
              <Button
                rounded={"md"}
                variant={"brand"}
                fontSize={"xl"}
                size={"lg"}
                color={"black"}
                onClick={() => {
                  if (loggedIn) {
                    router.push("/dashboard");
                  } else {
                    window.dispatchEvent(
                      new CustomEvent("codeup::open_login")
                    );
                  }
                }}
              >
                {loggedIn ? "Dashboard" : "Jetzt starten"}
              </Button>
            </ButtonGroup>
          </Flex>
        </Flex>
      </Flex>
    </Flex>

    <Box
      bg={
        "linear-gradient(to bottom, rgba(0, 0, 0, 0), rgba(0, 0, 0, 0.05), rgba(0, 0, 0, 0.1), rgba(0, 0, 0,
0.15), rgba(0, 0, 0, 0.2), rgba(0, 0, 0, 0.25), rgba(0, 0, 0, 0.3), rgba(0, 0, 0, 0.35), rgba(0, 0, 0, 0.4),
rgba(0, 0, 0, 0.45), rgba(0, 0, 0, 0.5), rgba(0, 0, 0, 0.55), rgba(0, 0, 0, 0.6), rgba(0, 0, 0, 0.65), rgba(0,
```

```
0, 0, 0.7), rgba(0, 0, 0, 0.75), rgba(0, 0, 0, 0.8), rgba(0, 0, 0, 0.85), rgba(0, 0, 0, 0.9), rgba(0, 0, 0,
0.95), #000)"
        //"linear-gradient(to bottom, transparent, #000)"
        //"linear-gradient(to bottom, transparent 0%, rgba(0, 0, 0, 0.083) 8.33%, rgba(0, 0, 0, 0.167)
16.67%, rgba(0, 0, 0, 0.25) 25%, rgba(0, 0, 0, 0.333) 33.33%, rgba(0, 0, 0, 0.417) 41.67%, rgba(0, 0,
0, 0.5) 50%, rgba(0, 0, 0, 0.583) 58.33%, rgba(0, 0, 0, 0.667) 66.67%, rgba(0, 0, 0, 0.75) 75%, rgba(0,
0, 0, 0.83) 83.33%, rgba(0, 0, 0, 0.91) 91.67% 100%)"
      }
    >
      <Testimonials />
      <BrandStats />
      <img
        src={"/_static/images/software-developer-6521720_1920.jpg"}
        alt={""}
        className={"index-header-image ihi-rot"}
      />
      <Milestones
        title={"Geschichte"}
        milestones={[
          {
            date: "Mai 2022",
            title: "Ideenfindung",
            description: `Die Idee für das Projekt wurde geboren. Es sollte eine Plattform werden, die es
Einsteigern ermöglicht programmieren zu lernen.`,
          },
          {
            date: "6. Juni 2022",
            title: "Erster Commit",
            description: `Der erste Commit wurde auf GitHub gemacht. Das Projekt wurde von Ben
Siebert ins Leben gerufen.`,
          },
          {
            date: "4. Juli 2022",
            title: "Umstieg auf Turborepo",
            description:
              "Das Projekt wurde auf Turborepo umgestellt, um die Entwicklung zu vereinfachen.",
          },
          {
            date: "16. Oktober 2022",
            title: "Admin Panel",
            description:
              "Fertigstellung der ersten Version des Admin Panels.",
          },
          {
            date: "27. November 2022",
            title: "Zertifikate",
            description:
              "Die Idee für das Erlangen von Zertifikaten wurde in die Realität umgesetzt.",
          },
          {
            date: "24. Feburar 2023",
            title: "Regionalwettbewerb",
            description:
              "CodeUp erhielt den ersten Preis beim Regionalwettbewerb Jugend Forscht der IHK in
Münster in der Sparte Arbeitswelt.",
          },
          {
            date: "29. März 2023",
            title: "Landeswettbewerb",
            description:
```

```
            "CodeUp erhielt den ersten Preis beim Landeswettbewerb Jugend Forscht im Bundesland
NRW in der Sparte Arbeitswelt.",
          },
          {
            date: "18. Mai 2023",
            title: "Bundeswettbewerb",
            description:
              "Beim 58. Bundeswettbewerb von Jugend forscht durfte CodeUp in der Sparte Arbeitswelt
antreten.",
          },
          {
            date: "01. Juli 2023",
            title: "StartupTeens",
            description:
              "Bei der Bundesweiten Businessplan Challenge StartupTeens erhielt CodeUp den zweiten
Preis in der Kategorie Education.",
          },
          {
            date: "06. Mai 2024",
            title: "Wiederaufnahme",
            description:
              "Am 06. Mai 2024 ging die Entwicklung von CodeUp nach einer längeren Pause wieder
weiter.",
          },
        ]}
      />
    </Box>
  </>
 );
}
```

## apps/web/src/pages/kids/index.tsx

```
/**
 * apps/web/src/pages/kids/index.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 03.04.2023
 *
 */

import * as React from "react";
import { useLoggedIn } from "@codeup/hooks";
import { useRouter } from "next/router";
import GridLayout from "@codeup/components/dist/global/GridLayout";
import REST from "@codeup/rest/src/index";
import AccountManager from "@codeup/util/dist/AccountManager";
import { FaEye, FaPen, FaPlus, FaTrash } from "react-icons/fa";
import PopupManager from "../../global/PopupManager";
import AuthenticateLoadingIndicator from
"@codeup/components/dist/admin/AuthenticateLoadingIndicator";
import SuperText from "@codeup/components/dist/landingpage/SuperText";
import { Box } from "@chakra-ui/react";
import GridLayoutV2 from "@codeup/components/dist/global/GridLayoutV2";

export default function KidsDashboard() {
  const { userInfo, loggedIn, loaded } = useLoggedIn();
  const router = useRouter();
  const [projects, setProjects] = React.useState<
```

```
  {
    _id: string;
    name: string;
    code: string;
    owner: string;
  }[]
>([]);

React.useEffect(() => {
  if (loaded && !loggedIn) {
    router.push("/#login");
    window.dispatchEvent(new CustomEvent("codeup::open_login"));
  } else {
    if (userInfo) {
      reload();
    }
  }
}, [loggedIn, loaded]);

const reload = () => {
  REST.Kids.getProjects(AccountManager.getActiveToken()).then((res) => {
    if (res.status !== 200) {
      PopupManager.alertAsync({
        title: "Fehler",
        description: "Es ist ein Fehler aufgetreten: " + res.payload.error,
      });
      return;
    }

    setProjects(res.payload.projects);
  });
};

return (
  <>
    {!loggedIn ? (
      <>
        <AuthenticateLoadingIndicator />
      </>
    ) : (
      <>
        <Box pt={4} pb={4} bg={"#000"} h={"fit-content"} minH={"100vh"}>
          <GridLayoutV2
            pageTitle={"CodeUp Kids"}
            entries={projects.map((p) => ({
              title: p.name,
              fields: [
                {
                  text: "",
                  icon: undefined,
                },
              ],
              buttonRows: [
                [
                  {
                    text: "Öffnen",
                    leftIcon: <FaEye />,
                    onClick: () => {
                      router.push(`/kids/${p._id}`);
                    },
```

```
    variant: "brand",
  },
  {
    text: "Umbenennen",
    onClick: async () => {
      const newName = await PopupManager.promptAsync({
        title: "Projekt umbenennen",
        inputType: "INPUT",
        helperText:
          "Gib einen neuen Namen für dein Projekt ein",
      });

      if (!newName) return;

      const res = await REST.Kids.updateProject({
        token: AccountManager.getActiveToken(),
        name: newName,
        id: p._id,
      });

      if (res.status !== 200) {
        PopupManager.alertAsync({
          title: "Fehler",
          description:
            "Es ist ein Fehler aufgetreten: " +
            res.payload.error,
        });
        return;
      }

      reload();

      PopupManager.alertAsync({
        title: "Projekt umbenannt",
        description:
          "Dein Projekt wurde erfolgreich umbenannt.",
      });
    },
    leftIcon: <FaPen />,
    variant: "brand",
  },
],
[
  {
    text: "Löschen",
    onClick: async () => {
      if (
        !(await PopupManager.confirmAsync({
          title: "Projekt löschen",
          question:
            "Bist du dir sicher, dass du das Projekt löschen möchtest?",
        }))
      )
        return;

      const res = await REST.Kids.deleteProject({
        token: AccountManager.getActiveToken(),
        id: p._id,
      });
```

```
            if (res.status !== 200) {
              PopupManager.alertAsync({
                title: "Fehler",
                description:
                  "Es ist ein Fehler aufgetreten: " +
                  res.payload.error,
              });
              return;
            }

            reload();
          },
          leftIcon: <FaTrash />,
          colorScheme: "red",
        },
      ],
    ],
  }))}
  topButtons={[
    {
      text: "Neu",
      leftIcon: <FaPlus />,
      variant: "brand",
      onClick: async () => {
        const name = await PopupManager.promptAsync({
          title: "Neues Projekt",
          inputType: "INPUT",
          helperText: "Gib einen Namen für dein Projekt ein",
        });

        if (!name) return;

        const res = await REST.Kids.createProject({
          name,
          token: AccountManager.getActiveToken(),
        });
        if (res.status !== 200) {
          PopupManager.alertAsync({
            title: "Fehler",
            description:
              "Es ist ein Fehler aufgetreten: " + res.payload.error,
          });
          return;
        }

        reload();
      },
    },
  ]}
  search={{
    show: true,
    fields: true,
  }}
 />
</Box>

<SuperText>CodeUp Kids</SuperText>
<GridLayout
 search={{
   allow: true,
```

```
  searchFields: true,
}}
specialEntries={[
  {
    icon: <FaPlus />,
    onClick: async () => {
      const name = await PopupManager.promptAsync({
        title: "Neues Projekt",
        inputType: "INPUT",
        helperText: "Gib einen Namen für dein Projekt ein",
      });

      if (!name) return;

      const res = await REST.Kids.createProject({
        name,
        token: AccountManager.getActiveToken(),
      });
      if (res.status !== 200) {
        PopupManager.alertAsync({
          title: "Fehler",
          description:
            "Es ist ein Fehler aufgetreten: " + res.payload.error,
        });
        return;
      }

      reload();
    },
  },
]}
entries={projects.map((p) => ({
  title: p.name,
  fields: [],
  buttonRows: [
    {
      buttons: [
        {
          text: "Öffnen",
          onClick: () => {
            router.push(`/kids/${p._id}`);
          },
          icon: <FaEye />,
          colorScheme: "blue",
          disabled: false,
        },
        {
          text: "Umbenennen",
          onClick: async () => {
            const newName = await PopupManager.promptAsync({
              title: "Projekt umbenennen",
              inputType: "INPUT",
              helperText:
                "Gib einen neuen Namen für dein Projekt ein",
            });

            if (!newName) return;

            const res = await REST.Kids.updateProject({
              token: AccountManager.getActiveToken(),
```

```
          name: newName,
          id: p._id,
        });

        if (res.status !== 200) {
          PopupManager.alertAsync({
            title: "Fehler",
            description:
              "Es ist ein Fehler aufgetreten: " +
              res.payload.error,
          });
          return;
        }

        reload();

        PopupManager.alertAsync({
          title: "Projekt umbenannt",
          description:
            "Dein Projekt wurde erfolgreich umbenannt.",
        });
      },
      icon: <FaPen />,
      colorScheme: "green",
      disabled: false,
    },
  ],
},
{
  buttons: [
    {
      text: "Umbenennen",
      onClick: async () => {
        const newName = await PopupManager.promptAsync({
          title: "Projekt umbenennen",
          inputType: "INPUT",
          helperText:
            "Gib einen neuen Namen für dein Projekt ein",
        });

        if (!newName) return;

        const res = await REST.Kids.updateProject({
          token: AccountManager.getActiveToken(),
          name: newName,
          id: p._id,
        });

        if (res.status !== 200) {
          PopupManager.alertAsync({
            title: "Fehler",
            description:
              "Es ist ein Fehler aufgetreten: " +
              res.payload.error,
          });
          return;
        }

        reload();
```

```
                    PopupManager.alertAsync({
                      title: "Projekt umbenannt",
                      description:
                        "Dein Projekt wurde erfolgreich umbenannt.",
                    });
                  },
                  icon: <FaPen />,
                  colorScheme: "green",
                  disabled: false,
                },
              ],
            },
            {
              buttons: [
                {
                  text: "Löschen",
                  onClick: async () => {
                    if (
                      !(await PopupManager.confirmAsync({
                        title: "Projekt löschen",
                        question:
                          "Bist du dir sicher, dass du das Projekt löschen möchtest?",
                      }))
                    )
                      return;

                    const res = await REST.Kids.deleteProject({
                      token: AccountManager.getActiveToken(),
                      id: p._id,
                    });

                    if (res.status !== 200) {
                      PopupManager.alertAsync({
                        title: "Fehler",
                        description:
                          "Es ist ein Fehler aufgetreten: " +
                          res.payload.error,
                      });
                      return;
                    }

                    reload();
                  },
                  icon: <FaTrash />,
                  colorScheme: "red",
                  disabled: false,
                },
              ],
            },
          ],
        }))}
      />
    </>
  )}
 </>
 );
}
```

apps/web/src/pages/kids/playground.tsx

```
/**
```

```
 * apps/web/src/pages/kids/playground.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 02.04.2023
 *
 */

import * as React from "react";
import { Box, Button, Flex } from "@chakra-ui/react";
import BlocklyEditor from "@codeup/incode-blockly/dist/BlocklyEditor";
//import { BlocklyCompiler } from "@codeup/incode-blockly/dist/blockly/BlocklyCompiler";
import { FaCode, FaPlay } from "react-icons/fa";
import * as InCode from "@codeup/incode-compiler/dist";
import PopupManager from "../../global/PopupManager";

export default function Playground() {
  return (
    <>
      <Flex
        w={"100%"}
        minH={"100vh"}
        h={"fit-content"}
        direction={["column", "row"]}
        gap={"2rem"}
        bg={"#000"}
      >
        <Box w={"100%"} h={"100vh"} flex={"100%"}>
          <BlocklyEditor
            initialXml={
              '<xml><block type="start" id="|(^9%DCME)E4UEoWv~G]" x="134" y="70"></block></xml>'
            }
          />
        </Box>
        <Box flex={"5%"}>
          <Button
            onClick={() => {
              const iframe = document.getElementById(
                "codeup-kids_playground_preview"
              ) as HTMLIFrameElement;

              const incode = new InCode.BlocklyCompiler().compile();
              const html = InCode.Compiler.compile(incode);

              iframe.srcdoc =
                "<html><body><script defer async>" +
                html +
                "</script></body></html>";
            }}
            leftIcon={<FaPlay />}
            mt={[0, 8]}
            ml={[8, 0]}
            variant={"brand"}
          >
            Ausführen
          </Button>
          <Button
            variant={"brand"}
            mt={[0, 8]}
```

```
            w={"100%"}
            leftIcon={<FaCode />}
            onClick={async () => {
              const incode = new InCode.BlocklyCompiler().compile();
              const html = InCode.Compiler.compile(incode);

              await PopupManager.alertAsync({
                title: "JavaScript Code",
                description: (
                  <>
                    <pre
                      style={{
                        overflow: "scroll",
                      }}
                    >
                      {html}
                    </pre>
                  </>
                ),
                customSize: "2xl",
              });
            }}
          >
            Code
          </Button>
        </Box>
        <Box flex={"100%"}>
          <iframe
            id={"codeup-kids_playground_preview"}
            width={"100%"}
            height={"100vh"}
            style={{
              border: "none",
              height: "100vh",
            }}
          ></iframe>
        </Box>
      </Flex>
    </>
  );
}
```

## apps/web/src/pages/kids/[id].tsx

```
/**
 * apps/web/src/pages/kids/[id].tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 03.04.2023
 *
 */

import * as React from "react";
import { useLoggedIn } from "@codeup/hooks";
import { useRouter } from "next/router";
import AuthenticateLoadingIndicator from
"@codeup/components/dist/admin/AuthenticateLoadingIndicator";
import REST from "@codeup/rest/src/index";
import AccountManager from "@codeup/util/dist/AccountManager";
```

```
import PopupManager from "../../global/PopupManager";
import { Box, Button, Flex, Textarea } from "@chakra-ui/react";
import BlocklyEditor from "@codeup/incode-blockly/dist/BlocklyEditor";
import { BlocklyCompiler } from "@codeup/util/dist/blockly/blockly/BlocklyCompiler";
import { FaCode, FaPlay, FaSave } from "react-icons/fa";
import Blockly from "blockly";
import * as InCode from "@codeup/incode-compiler";

export default function ProjectViewer() {
  const [project, setProject] = React.useState<{
    _id: string;
    name: string;
    code: string;
    owner: string;
  }>();

  const { loggedIn, loaded, userInfo } = useLoggedIn();

  const router = useRouter();

  React.useEffect(() => {
    if (loaded && !loggedIn) {
      router.push("/#login");
      window.dispatchEvent(new CustomEvent("codeup::open_login"));
      return;
    }

    if (!router.query.id) return;

    REST.Kids.getProject({
      token: AccountManager.getActiveToken(),
      id: router.query.id as string,
    }).then((res) => {
      if (res.status !== 200) {
        PopupManager.alertAsync({
          title: "Fehler",
          description: "Es ist ein Fehler aufgetreten: " + res.payload.error,
        });
        return;
      }

      setProject(res.payload.project);
    });
  }, [loggedIn, loaded]);

  return (
    <>
      {!loaded || !project || !project.code ? (
        <>
          <AuthenticateLoadingIndicator />
        </>
      ) : (
        <>
          <Flex
            w={"100%"}
            minH={"100vh"}
            h={"fit-content"}
            direction={["column", "row"]}
            gap={"2rem"}
            bg={"#000"}
```

```
>
  <Box w={"100%"} h={"100vh"} flex={"100%"}>
    <BlocklyEditor initialXml={project?.code as string} />
  </Box>
  <Box flex={"5%"}>
    <Button
      onClick={() => {
        const iframe = document.getElementById(
          "codeup-kids_preview"
        ) as HTMLIFrameElement;

        const incode = new InCode.BlocklyCompiler().compile();
        const html = InCode.Compiler.compile(incode);

        console.log(incode, html);

        iframe.srcdoc =
          "<html><body><script defer async>" +
          html +
          "</script></body></html>";
      }}
      leftIcon={<FaPlay />}
      mt={[0, 8]}
      ml={[8, 0]}
      variant={"brand"}
    >
      Ausführen
    </Button>
    <Button
      onClick={async () => {
        const res = await REST.Kids.updateProject({
          token: AccountManager.getActiveToken(),
          id: project?._id as string,
          code: Blockly.Xml.workspaceToDom(Blockly.getMainWorkspace())
            .outerHTML,
        });

        if (res.status !== 200) {
          PopupManager.alertAsync({
            title: "Fehler",
            description:
              "Es ist ein Fehler aufgetreten: " + res.payload.error,
          });
          return;
        }

        PopupManager.alertAsync({
          title: "Erfolgreich",
          description: "Das Projekt wurde erfolgreich gespeichert!",
        });
      }}
      leftIcon={<FaSave />}
      mt={[0, 8]}
      ml={[8, 0]}
      variant={"brand"}
    >
      Speichern
    </Button>
    <Button
      variant={"brand"}
```

```
              mt={[0, 8]}
              w={"100%"}
              leftIcon={<FaCode />}
              onClick={async () => {
                const incode = new InCode.BlocklyCompiler().compile();
                const html = InCode.Compiler.compile(incode);

                await PopupManager.alertAsync({
                  title: "JavaScript Code",
                  description: (
                    <>
                      <pre
                        style={{
                          overflow: "scroll",
                        }}
                      >
                        {html}
                      </pre>
                    </>
                  ),
                  customSize: "2xl",
                });
              }}
            >
              Code
            </Button>
          </Box>
          <Box flex={"100%"}>
            <iframe
              id={"codeup-kids_preview"}
              width={"100%"}
              height={"100vh"}
              style={{
                border: "none",
                height: "100vh",
              }}
            ></iframe>
          </Box>
        </Flex>
      </>
    )}
  </>
 );
}
```

## apps/web/src/pages/legal/imprint.tsx

```
/**
 * apps/web/src/pages/legal/imprint.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 17.12.2022
 *
 */

import * as React from "react";
import {
  Button,
  Flex,
```

```
  FormControl,
  FormLabel,
  Heading,
  Input,
  Link,
  Select,
  Stack,
  Text,
  Textarea,
} from "@chakra-ui/react";
import Head from "next/head";
import makeRequest from "@codeup/rest/dist/makeRequest";
import DividerWithText from "@codeup/components/dist/global/DividerWithText";
import { CHeading } from "@codeup/components/dist/global/CHeading";

export default function Imprint() {
  return (
    <>
      <Head>
        <title>Impressum - CodeUp</title>
      </Head>
      <Flex
        align="center"
        justify="center"
        w={"100%"}
        minH={"100vh"}
        bg={"#000"}
        h={"fit-content"}
      >
        <Stack
          spacing={8}
          w="100%"
          maxW="800px"
          p={8}
          bgColor={"black"}
          rounded="lg"
          shadow={["none", "xl"]}
        >
          <CHeading size={"xl"}>Impressum</CHeading>
          <Heading
            as="h2"
            size="lg"
            textAlign={["center", "initial"]}
            color={"brand.500"}
          >
            Angaben gemäß § 5 TMG
          </Heading>
          <Text fontSize={"lg"}>
            Ben Siebert
            <br />
            Im Mühlenwinkel 14
            <br />
            45525 Hattingen
          </Text>
          <Heading
            as="h2"
            size="lg"
            textAlign={["center", "initial"]}
            color={"brand.500"}
          >
```

```
          Kontakt
        </Heading>
        <Text fontSize={"lg"}>
          E-Mail: 
          <Link href="mailto:hello@ben-siebert.de" color={"brand.500"}>
            hello@ben-siebert.de
          </Link>
        </Text>
        <Heading
          as="h2"
          size="lg"
          textAlign={["center", "initial"]}
          color={"brand.500"}
        >
          Redaktionell Verantwortlicher
        </Heading>
        <Text fontSize={"lg"}>Ben Siebert</Text>
      </Stack>
    </Flex>
  </>
 );
}
```

## apps/web/src/pages/legal/privacy.tsx

```
/**
 * apps/web/src/pages/legal/privacy.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 17.12.2022
 *
 */

import * as React from "react";
import {
  Button,
  Flex,
  FormControl,
  FormLabel,
  Heading,
  Input,
  Link,
  Select,
  Stack,
  Text,
  Textarea,
} from "@chakra-ui/react";
import Head from "next/head";
import makeRequest from "@codeup/rest/dist/makeRequest";
import DividerWithText from "@codeup/components/dist/global/DividerWithText";
import { CHeading } from "@codeup/components/dist/global/CHeading";

export default function Imprint() {
  return (
    <>
      <Head>
        <title>Datenschutzerklärung - CodeUp</title>
      </Head>
      <Flex
```

```
  align="center"
  justify="center"
  w={"100%"}
  minH={"100vh"}
  h={"fit-content"}
  bg={"#000"}
  pt={6}
  pb={6}
>
  <Stack
    spacing={8}
    w="100%"
    maxW="800px"
    p={8}
    bgColor={["transparent", "gray.900"]}
    rounded="lg"
    shadow={["none", "xl"]}
  >
    <CHeading size={"2xl"}>Datenschutzerklärung</CHeading>
    <>
      <Heading size="lg" color="brand.500">
        1. Datenschutz auf einen Blick
      </Heading>
      <Heading size="lg" color="brand.500">
        Allgemeine Hinweise
      </Heading>
       
      <Text>
        Die folgenden Hinweise geben einen einfachen Überblick darüber,
        was mit Ihren personenbezogenen Daten passiert, wenn Sie diese
        Website besuchen. Personenbezogene Daten sind alle Daten, mit
        denen Sie persönlich identifiziert werden können. Ausführliche
        Informationen zum Thema Datenschutz entnehmen Sie unserer unter
        diesem Text aufgeführten Datenschutzerklärung.
      </Text>
      <Heading size="lg" color="brand.500">
        Datenerfassung auf dieser Website
      </Heading>
       
      <Heading size="md" color="brand.500">
        Wer ist verantwortlich für die Datenerfassung auf dieser Website?
      </Heading>
       
      <Text>
        Die Datenverarbeitung auf dieser Website erfolgt durch den
        Websitebetreiber. Dessen Kontaktdaten können Sie dem Abschnitt
        „Hinweis zur Verantwortlichen Stelle“ in dieser
        Datenschutzerklärung entnehmen.
      </Text>
       
      <Heading size="md" color="brand.500">
        Wie erfassen wir Ihre Daten?
      </Heading>
       
      <Text>
        Ihre Daten werden zum einen dadurch erhoben, dass Sie uns diese
        mitteilen. Hierbei kann es sich z. B. um Daten handeln, die
        Sie in ein Kontaktformular eingeben.
      </Text>
       
```

<Text>
Andere Daten werden automatisch oder nach Ihrer Einwilligung beim Besuch der Website durch unsere IT-Systeme erfasst. Das sind vor allem technische Daten (z. B. Internetbrowser, Betriebssystem oder Uhrzeit des Seitenaufrufs). Die Erfassung dieser Daten erfolgt automatisch, sobald Sie diese Website betreten.
</Text>
 
<Heading size="md" color="brand.500">
Wofür nutzen wir Ihre Daten?
</Heading>
 
<Text>
Ein Teil der Daten wird erhoben, um eine fehlerfreie Bereitstellung der Website zu gewährleisten. Andere Daten können zur Analyse Ihres Nutzerverhaltens verwendet werden.
</Text>
 
<Heading size="md" color="brand.500">
Welche Rechte haben Sie bezüglich Ihrer Daten?
</Heading>
 
<Text>
Sie haben jederzeit das Recht, unentgeltlich Auskunft über Herkunft, Empfänger und Zweck Ihrer gespeicherten personenbezogenen Daten zu erhalten. Sie haben außerdem ein Recht, die Berichtigung oder Löschung dieser Daten zu verlangen. Wenn Sie eine Einwilligung zur Datenverarbeitung erteilt haben, können Sie diese Einwilligung jederzeit für die Zukunft widerrufen. Außerdem haben Sie das Recht, unter bestimmten Umständen die Einschränkung der Verarbeitung Ihrer personenbezogenen Daten zu verlangen. Des Weiteren steht Ihnen ein Beschwerderecht bei der zuständigen Aufsichtsbehörde zu.
</Text>
 
<Text>
Hierzu sowie zu weiteren Fragen zum Thema Datenschutz können Sie sich jederzeit an uns wenden.
</Text>
<Heading size="lg" color="brand.500">
Analyse-Tools und Tools von Dritt-anbietern
</Heading>
 
<Text>
Beim Besuch dieser Website kann Ihr Surf-Verhalten statistisch ausgewertet werden. Das geschieht vor allem mit sogenannten Analyseprogrammen.
</Text>
 
<Text>
Detaillierte Informationen zu diesen Analyseprogrammen finden Sie in der folgenden Datenschutzerklärung.
</Text>
<Heading size="lg" color="brand.500">
2. Allgemeine Hinweise und Pflicht-informationen
</Heading>
<Heading size="lg" color="brand.500">
Datenschutz
</Heading>
 

<Text>
Die Betreiber dieser Seiten nehmen den Schutz Ihrer persönlichen Daten sehr ernst. Wir behandeln Ihre personenbezogenen Daten vertraulich und entsprechend den gesetzlichen Datenschutzvorschriften sowie dieser Datenschutzerklärung.
</Text>
 
<Text>
Wenn Sie diese Website benutzen, werden verschiedene personenbezogene Daten erhoben. Personenbezogene Daten sind Daten, mit denen Sie persönlich identifiziert werden können. Die vorliegende Datenschutzerklärung erläutert, welche Daten wir erheben und wofür wir sie nutzen. Sie erläutert auch, wie und zu welchem Zweck das geschieht.
</Text>
 
<Text>
Wir weisen darauf hin, dass die Datenübertragung im Internet (z. B. bei der Kommunikation per E-Mail) Sicherheitslücken aufweisen kann. Ein lückenloser Schutz der Daten vor dem Zugriff durch Dritte ist nicht möglich.
</Text>
<Heading size="lg" color="brand.500">
Hinweis zur verantwortlichen Stelle
</Heading>
 
<Text>
Die verantwortliche Stelle für die Datenverarbeitung auf dieser Website ist:
</Text>
 
<Text>
Ben Siebert
<br />
Im Mühlenwinkel 14
<br />
45525 Hattingen
</Text>
<Text>
Telefon: 015254398340
<br />
E-Mail: hello@ben-siebert.de
</Text>
<Text>
Verantwortliche Stelle ist die natürliche oder juristische Person, die allein oder gemeinsam mit anderen über die Zwecke und Mittel der Verarbeitung von personenbezogenen Daten (z. B. Namen, E-Mail-Adressen o. Ä.) entscheidet.
</Text>
<Heading size="lg" color="brand.500">
Speicherdauer
</Heading>
 
<Text>
Soweit innerhalb dieser Datenschutzerklärung keine speziellere Speicherdauer genannt wurde, verbleiben Ihre personenbezogenen Daten bei uns, bis der Zweck für die Datenverarbeitung entfällt. Wenn Sie ein berechtigtes Löschersuchen geltend machen oder eine Einwilligung zur Datenverarbeitung widerrufen, werden Ihre Daten gelöscht, sofern wir keine anderen rechtlich zulässigen Gründe für

die Speicherung Ihrer personenbezogenen Daten haben (z. B. steuer- oder handelsrechtliche Aufbewahrungsfristen); im letztgenannten Fall erfolgt die Löschung nach Fortfall dieser Gründe.
</Text>
<Heading size="lg" color="brand.500">
Allgemeine Hinweise zu den Rechtsgrundlagen der Datenverarbeitung auf dieser Website
</Heading>
 
<Text>
Sofern Sie in die Datenverarbeitung eingewilligt haben, verarbeiten wir Ihre personenbezogenen Daten auf Grundlage von Art. 6 Abs. 1 lit. a DSGVO bzw. Art. 9 Abs. 2 lit. a DSGVO, sofern besondere Datenkategorien nach Art. 9 Abs. 1 DSGVO verarbeitet werden. Im Falle einer ausdrücklichen Einwilligung in die Übertragung personenbezogener Daten in Drittstaaten erfolgt die Datenverarbeitung außerdem auf Grundlage von Art. 49 Abs. 1 lit. a DSGVO. Sofern Sie in die Speicherung von Cookies oder in den Zugriff auf Informationen in Ihr Endgerät (z. B. via Device-Fingerprinting) eingewilligt haben, erfolgt die Datenverarbeitung zusätzlich auf Grundlage von § 25 Abs. 1 TTDSG. Die Einwilligung ist jederzeit widerrufbar. Sind Ihre Daten zur Vertragserfüllung oder zur Durchführung vorvertraglicher Maßnahmen erforderlich, verarbeiten wir Ihre Daten auf Grundlage des Art. 6 Abs. 1 lit. b DSGVO. Des Weiteren verarbeiten wir Ihre Daten, sofern diese zur Erfüllung einer rechtlichen Verpflichtung erforderlich sind auf Grundlage von Art. 6 Abs. 1 lit. c DSGVO. Die Datenverarbeitung kann ferner auf Grundlage unseres berechtigten Interesses nach Art. 6 Abs. 1 lit. f DSGVO erfolgen. Über die jeweils im Einzelfall einschlägigen Rechtsgrundlagen wird in den folgenden Absätzen dieser Datenschutzerklärung informiert.
</Text>
<Heading size="lg" color="brand.500">
Hinweis zur Datenweitergabe in die USA und sonstige Drittstaaten
</Heading>
 
<Text>
Wir verwenden unter anderem Tools von Unternehmen mit Sitz in den USA oder sonstigen datenschutzrechtlich nicht sicheren Drittstaaten. Wenn diese Tools aktiv sind, können Ihre personenbezogene Daten in diese Drittstaaten übertragen und dort verarbeitet werden. Wir weisen darauf hin, dass in diesen Ländern kein mit der EU vergleichbares Datenschutzniveau garantiert werden kann. Beispielsweise sind US-Unternehmen dazu verpflichtet, personenbezogene Daten an Sicherheitsbehörden herauszugeben, ohne dass Sie als Betroffener hiergegen gerichtlich vorgehen könnten. Es kann daher nicht ausgeschlossen werden, dass US-Behörden (z. B. Geheimdienste) Ihre auf US-Servern befindlichen Daten zu Überwachungszwecken verarbeiten, auswerten und dauerhaft speichern. Wir haben auf diese Verarbeitungstätigkeiten keinen Einfluss.
</Text>
<Heading size="lg" color="brand.500">
Widerruf Ihrer Einwilligung zur Datenverarbeitung
</Heading>
 
<Text>
Viele Datenverarbeitungsvorgänge sind nur mit Ihrer ausdrücklichen Einwilligung möglich. Sie können eine bereits erteilte

Einwilligung jederzeit widerrufen. Die Rechtmäßigkeit der bis zum Widerruf erfolgten Datenverarbeitung bleibt vom Widerruf unberührt.
</Text>
<Heading size="lg" color="brand.500">
Widerspruchsrecht gegen die Datenerhebung in besonderen Fällen sowie gegen Direktwerbung (Art. 21 DSGVO)
</Heading>
 
<Text>
WENN DIE DATENVERARBEITUNG AUF GRUNDLAGE VON ART. 6 ABS. 1 LIT. E ODER F DSGVO ERFOLGT, HABEN SIE JEDERZEIT DAS RECHT, AUS GRÜNDEN, DIE SICH AUS IHRER BESONDEREN SITUATION ERGEBEN, GEGEN DIE VERARBEITUNG IHRER PERSONENBEZOGENEN DATEN WIDERSPRUCH EINZULEGEN;
DIES GILT AUCH FÜR EIN AUF DIESE BESTIMMUNGEN GESTÜTZTES PROFILING. DIE JEWEILIGE RECHTSGRUNDLAGE, AUF DENEN EINE VERARBEITUNG BERUHT, ENTNEHMEN SIE DIESER DATENSCHUTZERKLÄRUNG. WENN SIE WIDERSPRUCH EINLEGEN, WERDEN WIR IHRE BETROFFENEN PERSONENBEZOGENEN DATEN NICHT MEHR VERARBEITEN, ES SEI DENN, WIR KÖNNEN ZWINGENDE SCHUTZWÜRDIGE GRÜNDE FÜR DIE VERARBEITUNG NACHWEISEN, DIE IHRE INTERESSEN, RECHTE UND FREIHEITEN ÜBERWIEGEN ODER DIE VERARBEITUNG DIENT DER GELTENDMACHUNG, AUSÜBUNG ODER VERTEIDIGUNG VON RECHTSANSPRÜCHEN (WIDERSPRUCH NACH ART. 21 ABS. 1 DSGVO).
</Text>
 
<Text>
WERDEN IHRE PERSONENBEZOGENEN DATEN VERARBEITET, UM DIREKTWERBUNG
ZU BETREIBEN, SO HABEN SIE DAS RECHT, JEDERZEIT WIDERSPRUCH GEGEN DIE VERARBEITUNG SIE BETREFFENDER PERSONENBEZOGENER DATEN ZUM ZWECKE DERARTIGER WERBUNG EINZULEGEN; DIES GILT AUCH FÜR DAS PROFILING, SOWEIT ES MIT SOLCHER DIREKTWERBUNG IN VERBINDUNG STEHT. WENN SIE WIDERSPRECHEN, WERDEN IHRE PERSONENBEZOGENEN DATEN
ANSCHLIESSEND NICHT MEHR ZUM ZWECKE DER DIREKTWERBUNG VERWENDET (WIDERSPRUCH NACH ART. 21 ABS. 2 DSGVO).
</Text>
<Heading size="lg" color="brand.500">
Beschwerde-recht bei der zuständigen Aufsichts-behörde
</Heading>
 
<Text>
Im Falle von Verstößen gegen die DSGVO steht den Betroffenen ein Beschwerderecht bei einer Aufsichtsbehörde, insbesondere in dem Mitgliedstaat ihres gewöhnlichen Aufenthalts, ihres Arbeitsplatzes oder des Orts des mutmaßlichen Verstoßes zu. Das Beschwerderecht besteht unbeschadet anderweitiger verwaltungsrechtlicher oder gerichtlicher Rechtsbehelfe.
</Text>
<Heading size="lg" color="brand.500">
Recht auf Daten-übertrag-barkeit
</Heading>
 
<Text>
Sie haben das Recht, Daten, die wir auf Grundlage Ihrer Einwilligung oder in Erfüllung eines Vertrags automatisiert verarbeiten, an sich oder an einen Dritten in einem gängigen, maschinenlesbaren Format aushändigen zu lassen. Sofern Sie die

direkte Übertragung der Daten an einen anderen Verantwortlichen verlangen, erfolgt dies nur, soweit es technisch machbar ist.
</Text>
<Heading size="lg" color="brand.500">
Auskunft, Löschung und Berichtigung
</Heading>
 
<Text>
Sie haben im Rahmen der geltenden gesetzlichen Bestimmungen jederzeit das Recht auf unentgeltliche Auskunft über Ihre gespeicherten personenbezogenen Daten, deren Herkunft und Empfänger und den Zweck der Datenverarbeitung und ggf. ein Recht auf Berichtigung oder Löschung dieser Daten. Hierzu sowie zu weiteren Fragen zum Thema personenbezogene Daten können Sie sich jederzeit an uns wenden.
</Text>
<Heading size="lg" color="brand.500">
Recht auf Einschränkung der Verarbeitung
</Heading>
 
<Text>
Sie haben das Recht, die Einschränkung der Verarbeitung Ihrer personenbezogenen Daten zu verlangen. Hierzu können Sie sich jederzeit an uns wenden. Das Recht auf Einschränkung der Verarbeitung besteht in folgenden Fällen:
</Text>
 
<ul>
 
<li>
Wenn Sie die Richtigkeit Ihrer bei uns gespeicherten personenbezogenen Daten bestreiten, benötigen wir in der Regel Zeit, um dies zu überprüfen. Für die Dauer der Prüfung haben Sie das Recht, die Einschränkung der Verarbeitung Ihrer personenbezogenen Daten zu verlangen.
</li>
 
<li>
Wenn die Verarbeitung Ihrer personenbezogenen Daten unrechtmäßig geschah/geschieht, können Sie statt der Löschung die Einschränkung der Datenverarbeitung verlangen.
</li>
 
<li>
Wenn wir Ihre personenbezogenen Daten nicht mehr benötigen, Sie sie jedoch zur Ausübung, Verteidigung oder Geltendmachung von Rechtsansprüchen benötigen, haben Sie das Recht, statt der Löschung die Einschränkung der Verarbeitung Ihrer personenbezogenen Daten zu verlangen.
</li>
 
<li>
Wenn Sie einen Widerspruch nach Art. 21 Abs. 1 DSGVO eingelegt haben, muss eine Abwägung zwischen Ihren und unseren Interessen vorgenommen werden. Solange noch nicht feststeht, wessen Interessen überwiegen, haben Sie das Recht, die Einschränkung der Verarbeitung Ihrer personenbezogenen Daten zu verlangen.
</li>
 
</ul>

 
<Text>
Wenn Sie die Verarbeitung Ihrer personenbezogenen Daten eingeschränkt haben, dürfen diese Daten – von ihrer Speicherung abgesehen – nur mit Ihrer Einwilligung oder zur Geltendmachung, Ausübung oder Verteidigung von Rechtsansprüchen oder zum Schutz der Rechte einer anderen natürlichen oder juristischen Person oder aus Gründen eines wichtigen öffentlichen Interesses der Europäischen Union oder eines Mitgliedstaats verarbeitet werden.
</Text>
<Heading size="lg" color="brand.500">
SSL- bzw. TLS-Verschlüsselung
</Heading>
 
<Text>
Diese Seite nutzt aus Sicherheitsgründen und zum Schutz der Übertragung vertraulicher Inhalte, wie zum Beispiel Bestellungen oder Anfragen, die Sie an uns als Seitenbetreiber senden, eine SSL- bzw. TLS-Verschlüsselung. Eine verschlüsselte Verbindung erkennen Sie daran, dass die Adresszeile des Browsers von „http://“ auf „https://“ wechselt und an dem Schloss-Symbol in Ihrer Browserzeile.
</Text>
 
<Text>
Wenn die SSL- bzw. TLS-Verschlüsselung aktiviert ist, können die Daten, die Sie an uns übermitteln, nicht von Dritten mitgelesen werden.
</Text>
<Heading size="lg" color="brand.500">
3. Datenerfassung auf dieser Website
</Heading>
<Heading size="lg" color="brand.500">
Cookies
</Heading>
 
<Text>
Unsere Internetseiten verwenden so genannte „Cookies“. Cookies sind kleine Datenpakete und richten auf Ihrem Endgerät keinen Schaden an. Sie werden entweder vorübergehend für die Dauer einer Sitzung (Session-Cookies) oder dauerhaft (permanente Cookies) auf Ihrem Endgerät gespeichert. Session-Cookies werden nach Ende Ihres Besuchs automatisch gelöscht. Permanente Cookies bleiben auf Ihrem Endgerät gespeichert, bis Sie diese selbst löschen oder eine automatische Löschung durch Ihren Webbrowser erfolgt.
</Text>
 
<Text>
Teilweise können auch Cookies von Drittunternehmen auf Ihrem Endgerät gespeichert werden, wenn Sie unsere Seite betreten (Third-Party-Cookies). Diese ermöglichen uns oder Ihnen die Nutzung bestimmter Dienstleistungen des Drittunternehmens (z. B. Cookies zur Abwicklung von Zahlungsdienstleistungen).
</Text>
 
<Text>
Cookies haben verschiedene Funktionen. Zahlreiche Cookies sind technisch notwendig, da bestimmte Websitefunktionen ohne diese nicht funktionieren würden (z. B. die Warenkorbfunktion oder die Anzeige von Videos). Andere Cookies dienen dazu, das

Nutzerverhalten auszuwerten oder Werbung anzuzeigen.
</Text>
 
<Text>
Cookies, die zur Durchführung des elektronischen Kommunikationsvorgangs, zur Bereitstellung bestimmter, von Ihnen erwünschter Funktionen (z. B. für die Warenkorbfunktion) oder zur Optimierung der Website (z. B. Cookies zur Messung des Webpublikums) erforderlich sind (notwendige Cookies), werden auf Grundlage von Art. 6 Abs. 1 lit. f DSGVO gespeichert, sofern keine andere Rechtsgrundlage angegeben wird. Der Websitebetreiber hat ein berechtigtes Interesse an der Speicherung von notwendigen Cookies zur technisch fehlerfreien und optimierten Bereitstellung seiner Dienste. Sofern eine Einwilligung zur Speicherung von Cookies und vergleichbaren Wiedererkennungstechnologien abgefragt wurde, erfolgt die Verarbeitung ausschließlich auf Grundlage dieser Einwilligung (Art. 6 Abs. 1 lit. a DSGVO und § 25 Abs. 1 TTDSG); die Einwilligung ist jederzeit widerrufbar.
</Text>
 
<Text>
Sie können Ihren Browser so einstellen, dass Sie über das Setzen von Cookies informiert werden und Cookies nur im Einzelfall erlauben, die Annahme von Cookies für bestimmte Fälle oder generell ausschließen sowie das automatische Löschen der Cookies beim Schließen des Browsers aktivieren. Bei der Deaktivierung von Cookies kann die Funktionalität dieser Website eingeschränkt sein.
</Text>
 
<Text>
Soweit Cookies von Drittunternehmen oder zu Analysezwecken eingesetzt werden, werden wir Sie hierüber im Rahmen dieser Datenschutzerklärung gesondert informieren und ggf. eine Einwilligung abfragen.
</Text>
<Heading size="lg" color="brand.500">
Server-Log-Dateien
</Heading>
 
<Text>
Der Provider der Seiten erhebt und speichert automatisch Informationen in so genannten Server-Log-Dateien, die Ihr Browser automatisch an uns übermittelt. Dies sind:
</Text>
 
<ul>
 
<li>Browsertyp und Browserversion</li>{" "}
<li>verwendetes Betriebssystem</li> 
<li>Referrer URL</li> <li>Hostname des zugreifenden Rechners</li>
 
<li>Uhrzeit der Serveranfrage</li> <li>IP-Adresse</li> 
</ul>
 
<Text>
Eine Zusammenführung dieser Daten mit anderen Datenquellen wird nicht vorgenommen.
</Text>
 
<Text>

Die Erfassung dieser Daten erfolgt auf Grundlage von Art. 6 Abs. 1 lit. f DSGVO. Der Websitebetreiber hat ein berechtigtes Interesse an der technisch fehlerfreien Darstellung und der Optimierung seiner Website – hierzu müssen die Server-Log-Files erfasst werden.
</Text>
<Heading size="lg" color="brand.500">
Kontaktformular
</Heading>
 
<Text>
Wenn Sie uns per Kontaktformular Anfragen zukommen lassen, werden Ihre Angaben aus dem Anfrageformular inklusive der von Ihnen dort angegebenen Kontaktdaten zwecks Bearbeitung der Anfrage und für den Fall von Anschlussfragen bei uns gespeichert. Diese Daten geben wir nicht ohne Ihre Einwilligung weiter.
</Text>
 
<Text>
Die Verarbeitung dieser Daten erfolgt auf Grundlage von Art. 6 Abs. 1 lit. b DSGVO, sofern Ihre Anfrage mit der Erfüllung eines Vertrags zusammenhängt oder zur Durchführung vorvertraglicher Maßnahmen erforderlich ist. In allen übrigen Fällen beruht die Verarbeitung auf unserem berechtigten Interesse an der effektiven Bearbeitung der an uns gerichteten Anfragen (Art. 6 Abs. 1 lit. f DSGVO) oder auf Ihrer Einwilligung (Art. 6 Abs. 1 lit. a DSGVO) sofern diese abgefragt wurde; die Einwilligung ist jederzeit widerrufbar.
</Text>
 
<Text>
Die von Ihnen im Kontaktformular eingegebenen Daten verbleiben bei uns, bis Sie uns zur Löschung auffordern, Ihre Einwilligung zur Speicherung widerrufen oder der Zweck für die Datenspeicherung entfällt (z. B. nach abgeschlossener Bearbeitung Ihrer Anfrage). Zwingende gesetzliche Bestimmungen – insbesondere Aufbewahrungsfristen – bleiben unberührt.
</Text>
<Heading size="lg" color="brand.500">
Anfrage per E-Mail, Telefon oder Telefax
</Heading>
 
<Text>
Wenn Sie uns per E-Mail, Telefon oder Telefax kontaktieren, wird Ihre Anfrage inklusive aller daraus hervorgehenden personenbezogenen Daten (Name, Anfrage) zum Zwecke der Bearbeitung Ihres Anliegens bei uns gespeichert und verarbeitet. Diese Daten geben wir nicht ohne Ihre Einwilligung weiter.
</Text>
 
<Text>
Die Verarbeitung dieser Daten erfolgt auf Grundlage von Art. 6 Abs. 1 lit. b DSGVO, sofern Ihre Anfrage mit der Erfüllung eines Vertrags zusammenhängt oder zur Durchführung vorvertraglicher Maßnahmen erforderlich ist. In allen übrigen Fällen beruht die Verarbeitung auf unserem berechtigten Interesse an der effektiven Bearbeitung der an uns gerichteten Anfragen (Art. 6 Abs. 1 lit. f DSGVO) oder auf Ihrer Einwilligung (Art. 6 Abs. 1 lit. a DSGVO) sofern diese abgefragt wurde; die Einwilligung ist jederzeit widerrufbar.

</Text>
 
<Text>
Die von Ihnen an uns per Kontaktanfragen übersandten Daten verbleiben bei uns, bis Sie uns zur Löschung auffordern, Ihre Einwilligung zur Speicherung widerrufen oder der Zweck für die Datenspeicherung entfällt (z. B. nach abgeschlossener Bearbeitung Ihres Anliegens). Zwingende gesetzliche Bestimmungen – insbesondere gesetzliche Aufbewahrungsfristen – bleiben unberührt.
</Text>
<Heading size="lg" color="brand.500">
Kommentar-funktion auf dieser Website
</Heading>
 
<Text>
Für die Kommentarfunktion auf dieser Seite werden neben Ihrem Kommentar auch Angaben zum Zeitpunkt der Erstellung des Kommentars, Ihre E-Mail-Adresse und, wenn Sie nicht anonym posten, der von Ihnen gewählte Nutzername gespeichert.
</Text>
<Heading size="md" color="brand.500">
Speicherdauer der Kommentare
</Heading>
 
<Text>
Die Kommentare und die damit verbundenen Daten werden gespeichert und verbleiben auf dieser Website, bis der kommentierte Inhalt vollständig gelöscht wurde oder die Kommentare aus rechtlichen Gründen gelöscht werden müssen (z. B. beleidigende Kommentare).
</Text>
<Heading size="md" color="brand.500">
Rechtsgrundlage
</Heading>
 
<Text>
Die Speicherung der Kommentare erfolgt auf Grundlage Ihrer Einwilligung (Art. 6 Abs. 1 lit. a DSGVO). Sie können eine von Ihnen erteilte Einwilligung jederzeit widerrufen. Dazu reicht eine formlose Mitteilung per E-Mail an uns. Die Rechtmäßigkeit der bereits erfolgten Datenverarbeitungsvorgänge bleibt vom Widerruf unberührt.
</Text>
<Heading size="lg" color="brand.500">
4. Soziale Medien
</Heading>
<Heading size="lg" color="brand.500">
Twitter
</Heading>
 
<Text>
Auf dieser Website sind Funktionen des Dienstes Twitter eingebunden. Diese Funktionen werden angeboten durch die Twitter International Company, One Cumberland Place, Fenian Street, Dublin 2, D02 AX07, Irland.
</Text>
 
<Text>
Wenn das Social-Media-Element aktiv ist, wird eine direkte Verbindung zwischen Ihrem Endgerät und dem Twitter-Server

```
  hergestellt. Twitter erhält dadurch Informationen über den Besuch
  dieser Website durch Sie. Durch das Benutzen von Twitter und der
  Funktion „Re-Tweet“ werden die von Ihnen besuchten Websites mit
  Ihrem Twitter-Account verknüpft und anderen Nutzern bekannt
  gegeben. Wir weisen darauf hin, dass wir als Anbieter der Seiten
  keine Kenntnis vom Inhalt der übermittelten Daten sowie deren
  Nutzung durch Twitter erhalten. Weitere Informationen hierzu
  finden Sie in der Datenschutzerklärung von Twitter unter: 
  <a
    href="https://twitter.com/de/privacy"
    target="_blank"
    rel="noopener noreferrer"
  >
    https://twitter.com/de/privacy
  </a>
  .
</Text>
 
<Text>
  Soweit eine Einwilligung (Consent) eingeholt wurde, erfolgt der
  Einsatz des o. g. Dienstes auf Grundlage von Art. 6 Abs. 1
  lit. a DSGVO und § 25 TTDSG. Die Einwilligung ist jederzeit
  widerrufbar. Soweit keine Einwilligung eingeholt wurde, erfolgt
  die Verwendung des Dienstes auf Grundlage unseres berechtigten
  Interesses an einer möglichst umfassenden Sichtbarkeit in den
  Sozialen Medien.
</Text>
 
<Text>
  Die Datenübertragung in die USA wird auf die
  Standardvertragsklauseln der EU-Kommission gestützt. Details
  finden Sie hier: 
  <a
    href="https://gdpr.twitter.com/en/controller-to-controller-transfers.html"
    target="_blank"
    rel="noopener noreferrer"
  >
    https://gdpr.twitter.com/en/controller-to-controller-transfers.html
  </a>
  .
</Text>
 
<Text>
  Ihre Datenschutzeinstellungen bei Twitter können Sie in den
  Konto-Einstellungen unter 
  <a
    href="https://twitter.com/account/settings"
    target="_blank"
    rel="noopener noreferrer"
  >
    https://twitter.com/account/settings
  </a>
    ändern.
</Text>
<Heading size="lg" color="brand.500">
  5. Plugins und Tools
</Heading>
<Heading size="lg" color="brand.500">
  YouTube
</Heading>
```

```
 
<Text>
  Diese Website bindet Videos der Website YouTube ein. Betreiber der
  Website ist die Google Ireland Limited („Google“), Gordon House,
  Barrow Street, Dublin 4, Irland.
</Text>
 
<Text>
  Wenn Sie eine unserer Webseiten besuchen, auf denen YouTube
  eingebunden ist, wird eine Verbindung zu den Servern von YouTube
  hergestellt. Dabei wird dem YouTube-Server mitgeteilt, welche
  unserer Seiten Sie besucht haben.
</Text>
 
<Text>
  Des Weiteren kann YouTube verschiedene Cookies auf Ihrem Endgerät
  speichern oder vergleichbare Technologien zur Wiedererkennung
  verwenden (z. B. Device-Fingerprinting). Auf diese Weise kann
  YouTube Informationen über Besucher dieser Website erhalten. Diese
  Informationen werden u. a. verwendet, um Videostatistiken zu
  erfassen, die Anwenderfreundlichkeit zu verbessern und
  Betrugsversuchen vorzubeugen.
</Text>
 
<Text>
  Wenn Sie in Ihrem YouTube-Account eingeloggt sind, ermöglichen Sie
  YouTube, Ihr Surfverhalten direkt Ihrem persönlichen Profil
  zuzuordnen. Dies können Sie verhindern, indem Sie sich aus Ihrem
  YouTube-Account ausloggen.
</Text>
 
<Text>
  Die Nutzung von YouTube erfolgt im Interesse einer ansprechenden
  Darstellung unserer Online-Angebote. Dies stellt ein berechtigtes
  Interesse im Sinne von Art. 6 Abs. 1 lit. f DSGVO dar. Sofern eine
  entsprechende Einwilligung abgefragt wurde, erfolgt die
  Verarbeitung ausschließlich auf Grundlage von Art. 6 Abs. 1 lit. a
  DSGVO und § 25 Abs. 1 TTDSG, soweit die Einwilligung die
  Speicherung von Cookies oder den Zugriff auf Informationen im
  Endgerät des Nutzers (z. B. Device-Fingerprinting) im Sinne
  des TTDSG umfasst. Die Einwilligung ist jederzeit widerrufbar.
</Text>
 
<Text>
  Weitere Informationen zum Umgang mit Nutzerdaten finden Sie in der
  Datenschutzerklärung von YouTube unter: 
  <a
    href="https://policies.google.com/privacy?hl=de"
    target="_blank"
    rel="noopener noreferrer"
  >
    https://policies.google.com/privacy?hl=de
  </a>
  .
</Text>
<Heading size="lg" color="brand.500">
  Google Fonts (lokales Hosting)
</Heading>
 
<Text>
```

```
            Diese Seite nutzt zur einheitlichen Darstellung von Schriftarten
            so genannte Google Fonts, die von Google bereitgestellt werden.
            Die Google Fonts sind lokal installiert. Eine Verbindung zu
            Servern von Google findet dabei nicht statt.
          </Text>
           
          <Text>
            Weitere Informationen zu Google Fonts finden Sie unter 
            <a
              href="https://developers.google.com/fonts/faq"
              target="_blank"
              rel="noopener noreferrer"
            >
              https://developers.google.com/fonts/faq
            </a>
              und in der Datenschutzerklärung von Google: 
            <a
              href="https://policies.google.com/privacy?hl=de"
              target="_blank"
              rel="noopener noreferrer"
            >
              https://policies.google.com/privacy?hl=de
            </a>
            .
          </Text>
          <Heading size="lg" color="brand.500">
            Font Awesome (lokales Hosting)
          </Heading>
           
          <Text>
            Diese Seite nutzt zur einheitlichen Darstellung von Schriftarten
            Font Awesome. Font Awesome ist lokal installiert. Eine Verbindung
            zu Servern von Fonticons, Inc. findet dabei nicht statt.
          </Text>
           
          <Text>
            Weitere Informationen zu Font Awesome finden Sie in der
            Datenschutzerklärung für Font Awesome unter: 
            <a
              href="https://fontawesome.com/privacy"
              target="_blank"
              rel="noopener noreferrer"
            >
              https://fontawesome.com/privacy
            </a>
            .
          </Text>
          <Text>
            Quelle: 
            <a href="https://www.e-recht24.de">https://www.e-recht24.de</a>
          </Text>
        </>
      </Stack>
    </Flex>
  </>
 );
}
```

apps/web/src/pages/old-landing-v2.tsx

```
/**
```

```
 * apps/web/src/pages/index.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 26.10.2022
 *
 */

import * as React from "react";
import {
  Box,
  Button,
  Divider,
  Flex,
  Grid,
  Heading,
  Image,
  Link as ChakraLink,
  Stack,
  Text,
  useMediaQuery,
} from "@chakra-ui/react";
import Link from "next/link";
import Typed from "react-typed";
import { motion } from "framer-motion";
import { FaBook, FaCode, FaProjectDiagram } from "react-icons/fa";
import VideoPlayer from "@codeup/components/global/VideoPlayer";

export default function OldLandingV2() {
  const sectionCards = [
    {
      title: "Lernen",
      description:
        "Mit unseren interaktiven Kursen kannst du dich in verschiedensten Bereichen der
Programmierung weiterbilden oder neue Programmiersprachen erlernen. Jeder Kurs ist in Lektionen
aufgeteilt und verfügt sowohl über eine Text-Version als auch über ein Video.",
      icon: <FaBook />,
      iconColor: "blue.200",
    },
    {
      title: "Entwickeln",
      description:
        "Mit unserer fortschrittlichen Entwicklungsumgebung kannst du deine Projekte direkt im Browser
entwickeln. Du kannst zwischen verschiedenen Programmiersprachen wählen und deine Projekte mit
anderen teilen.",
      icon: <FaCode />,
      iconColor: "green.200",
    },
    {
      title: "Planen",
      description:
        "Mit unserem Projekt-Management kannst du deine Projekte planen und verwalten.",
      icon: <FaProjectDiagram />,
      iconColor: "yellow.600",
    },
  ];

  return (
    <>
```

```
<Box
  w={"100%"}
  h={"100vh"}
  backgroundImage={"url('/_static/images/codeup_splash_lp.JPG')"}
  backgroundSize={"cover"}
  backgroundPosition={"center"}
  backgroundRepeat={"no-repeat"}
></Box>

<Flex
  w={"100%"}
  minH={"100vh"}
  h="fit-content"
  bg={"gray.900"}
  alignItems="center"
  flexDirection={"column"}
  id="_more"
  pt="2rem"
>
  <motion.div
    initial={{ opacity: 0, scale: 0.5 }}
    whileInView={{ opacity: 1, scale: 1 }}
    transition={{
      delay: 0.5,
      default: {
        duration: 0.3,
        ease: [0, 0.71, 0.2, 1.01],
      },
      scale: {
        type: "spring",
        damping: 5,
        stiffness: 100,
        restDelta: 0.001,
      },
    }}
    viewport={{ once: true }}
  >
    <Heading
      color={"white"}
      fontSize={["2rem", "4rem"]}
      fontFamily={"ARCADE"}
      textAlign={"center"}
      id="landing-title"
    >
      Willkommen bei CodeUp!
    </Heading>
  </motion.div>
  <Grid
    templateColumns={"1fr"}
    maxW={[null, "40%"]}
    gap={"6rem"}
    w={"100%"}
    p={["1rem", "2rem"]}
    mt={["1rem", "2rem"]}
  >
    {sectionCards.map((card, index) => {
      return (
        <>
          <motion.div
            initial={{ opacity: 0, scale: 0.5 }}
```

```
  whileInView={{ opacity: 1, scale: 1 }}
  transition={{
    delay: 0.5 + index * 0.1,
    default: {
      duration: 0.3,
      ease: [0, 0.71, 0.2, 1.01],
    },
    scale: {
      type: "spring",
      damping: 5,
      stiffness: 100,
      restDelta: 0.001,
    },
  }}
  viewport={{ once: true }}
>
  <Box
    bg={"gray.800"}
    w={"100%"}
    h={"100%"}
    p={"1rem"}
    borderRadius={"10px"}
    boxShadow={"0px 0px 10px 0px rgba(0,0,0,0.75)"}
  >
    <Flex gap="2rem" justifyContent="center">
      <Box
        bg={card.iconColor}
        p={"1rem"}
        w={"fit-content"}
        color={"white"}
        fontSize={"2rem"}
        mb={"1rem"}
        borderRadius={"10px"}
        boxShadow={"0px 0px 10px 0px rgba(0,0,0,0.75)"}
      >
        {card.icon}
      </Box>

      <Heading
        color={"white"}
        fontSize={["1rem", "2rem"]}
        fontFamily={"ARCADE"}
        textAlign={"center"}
        mt={"1rem"}
      >
        {card.title}
      </Heading>
    </Flex>
    <Flex
      alignItems={"center"}
      justifyContent={"center"}
      flexDirection={"column"}
    >
      <Text
        color={"white"}
        fontSize={["1.5rem"]}
        textAlign={"center"}
        mt={"1rem"}
      >
        {card.description}
```

```
                </Text>
              </Flex>
            </Box>
          </motion.div>
        </>
      );
    })}
  </Grid>
  <Divider mt="4rem" />
  <motion.div
    initial={{ opacity: 0, scale: 0.5 }}
    whileInView={{ opacity: 1, scale: 1 }}
    transition={{
      delay: 0.5,
      default: {
        duration: 0.3,
        ease: [0, 0.71, 0.2, 1.01],
      },
      scale: {
        type: "spring",
        damping: 5,
        stiffness: 100,
        restDelta: 0.001,
      },
    }}
    viewport={{ once: true }}
  >
    <Heading
      mt={"4rem"}
      mb={"4rem"}
      color={"white"}
      fontSize={["2rem", "4rem"]}
      fontFamily={"ARCADE"}
      textAlign={"center"}
      id="landing-title"
    >
      CodeUp in 100 Sekunden
    </Heading>
  </motion.div>
  <Box
    w={"100%"}
    mt={8}
    mb={8}
    maxW={["100%", "40%"]}
    h={"fit-content"}
    flexDirection={"column"}
    alignItems={"center"}
    justifyContent={"center"}
    p={4}
    bg={"#000000"}
    borderRadius={"md"}
    boxShadow={"2xl"}
  >
    <VideoPlayer url={"https://www.youtube.com/watch?v=iElczHdHUVs"} />
  </Box>
 </Flex>
 </>
 );
}
```

## apps/web/src/pages/old-landing-v3.tsx

```
"use client";

/**
 * apps/web/src/pages/index.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 16.12.2022
 *
 */

import * as React from "react";

import {
  Button,
  ButtonGroup,
  Flex,
  Heading,
  Image,
  Stack,
} from "@chakra-ui/react";
import { motion } from "framer-motion";
import Typed from "react-typed";
import { useLoggedIn } from "@codeup/hooks";
import { useRouter } from "next/router";
import Head from "next/head";
import OverviewSection from "@codeup/components/landingpage/OverviewSection";
import BrandStats from "@codeup/components/landingpage/BrandStats";
import Testimonials from "@codeup/components/landingpage/Testimonials";
import Milestones from "@codeup/components/landingpage/Milestones";

export default function OldLandingV3() {
  const { loggedIn } = useLoggedIn();
  const router = useRouter();

  return (
    <>
      <Head>
        <title>CodeUp - Einfach Programmieren lernen</title>
      </Head>
      <Flex
        flexDirection="column"
        height="100vh"
        justifyContent={"center"}
        alignItems={"center"}
      >
        <Image
          src={"/_static/images/software-developer-6521720_1920.jpg"}
          width={"100%"}
          height={"100vh"}
          objectFit={"cover"}
          position={"absolute"}
          zIndex={-1}
          filter={"blur(10px)"}
          alt={""}
        />

        <Stack textAlign={["center", "left"]} w={"fit-content"} spacing={6}>
```

```
<Heading
  as="h1"
  fontSize={["lg", "2xl"]}
  fontFamily="ARCADE"
  color="white"
  fontStyle="italic"
>
  <Typed
    strings={["// Willkommen bei"]}
    typeSpeed={40}
    showCursor={false}
  />
</Heading>
<Heading
  as="h1"
  fontSize={["4xl", "8xl"]}
  fontFamily="ARCADE"
  color={"brand.500"}
>
  <Typed
    strings={["CodeUp"]}
    typeSpeed={50}
    startDelay={1500}
    showCursor={false}
  />
</Heading>
<Heading
  as="h1"
  fontSize={["lg", "2xl"]}
  fontFamily="ARCADE"
  color="white"
  fontStyle="italic"
>
  <Typed
    strings={["// Dein Weg zum Programmierer"]}
    typeSpeed={40}
    startDelay={2500}
    showCursor={false}
  />
</Heading>
<motion.div
  initial={{ opacity: 0 }}
  animate={{ opacity: 1 }}
  transition={{ delay: 5 }}
>
  <ButtonGroup w={"100%"} justifyContent={["center", "right"]}>
    <Button
      bg={"brand.500"}
      color={"black"}
      _hover={{
        bg: "brand.500",
        color: "black",
      }}
      _focus={{
        bg: "brand.500",
        color: "black",
      }}
      size={"lg"}
      onClick={() => {
        if (loggedIn) {
```

```
                router.push("/dashboard");
              } else {
                window.dispatchEvent(new CustomEvent("codeup::open_login"));
              }
            }}
          >
            {loggedIn ? "Zum Dashboard" : "Jetzt loslegen"}
          </Button>
        </ButtonGroup>
      </motion.div>
    </Stack>
  </Flex>
  <Stack mt={4} bgColor={"black"}>
    <OverviewSection />
    <Flex
      w={"100%"}
      alignItems={"center"}
      bgColor={"transparent"}
      justifyContent={"center"}
    >
      <Testimonials />
    </Flex>
    <Flex w={"100%"} alignItems={"center"} bgColor={"transparent"}>
      <BrandStats />
    </Flex>
    <Flex w={"100%"} alignItems={"center"} bgColor={"transparent"}>
      <Milestones
        title={"Geschichte"}
        milestones={[
          {
            date: "Mai 2022",
            title: "Ideenfindung",
            description: `Die Idee für das Projekt wurde geboren. Es sollte eine Plattform werden, die
es Einsteigern ermöglicht programmieren zu lernen.`,
          },
          {
            date: "6. Juni 2022",
            title: "Erster Commit",
            description: `Der erste Commit wurde auf GitHub gemacht. Das Projekt wurde von Ben
Siebert ins Leben gerufen.`,
          },
          {
            date: "4. Juli 2022",
            title: "Umstieg auf Turborepo",
            description:
              "Das Projekt wurde auf Turborepo umgestellt, um die Entwicklung zu vereinfachen.",
          },
          {
            date: "16. Oktober 2022",
            title: "Admin Panel",
            description:
              "Fertigstellung der ersten Version des Admin Panels.",
          },
          {
            date: "27. November 2022",
            title: "Zertifikate",
            description:
              "Die Idee für das Erlangen von Zertifikaten wurde in die Realität umgesetzt.",
          },
          {
```

```
              date: "24. Feburar 2023",
              title: "Regionalwettbewerb",
              description:
                "CodeUp erhielt den ersten Preis beim Regionalwettbewerb Jugend Forscht der IHK in
Münster in der Sparte Arbeitswelt.",
            },
            {
              date: "29. März 2023",
              title: "Landeswettbewerb",
              description:
                "CodeUp erhielt den ersten Preis beim Landeswettbewerb Jugend Forscht im Bundesland
NRW in der Sparte Arbeitswelt.",
            },
          ]}
        />
      </Flex>
    </Stack>
  </>
 );
}
```

## apps/web/src/pages/old-landing.tsx

```
/**
 * src/pages/old-landing.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 05.06.2022
 *
 */

import {
  Box,
  Button,
  ButtonGroup,
  Flex,
  Heading,
  Link,
  Text,
} from "@chakra-ui/react";
import Image from "next/image";
import { useRouter } from "next/router";
import React from "react";

export default function Home() {
  const allowLoadingAnimation = false;

  return (
    <>
      {allowLoadingAnimation ? (
        <Box
          position={"fixed"}
          top="0"
          left="0"
          width={"100%"}
          height="100vh"
          backgroundColor={"gray.800"}
          zIndex={5}
          display="flex"
```

```
    alignItems="center"
    textAlign={"center"}
    className="hp-loader"
    flexDirection={"column-reverse"}
    pb="2rem"
   >
    <div className="hp-image hp-image-0"></div>
    <Heading fontFamily="ARCADE" color={"white"}>
     Loading CodeUp ...
    </Heading>
   </Box>
  ) : null}
  <Card
   title="Lerne Programmieren"
   callToAction="Jetzt loslegen"
   image="/_static/vectors/home/learn-coding.svg"
   rtl={false}
   key="learn-coding"
  >
   Unsere <b>interaktiven</b> Programmierkurse bringen dir Schritt für
   Schritt bei, wie du eigene Webseiten mittels <b>HTML</b>
   , 
   <b>CSS</b> und <b>JavaScript</b> entwickeln kannst.
   <br />
   Alle Kurse sind für <b>Anfänger</b> konzipiert und erfordern 
   <b>keine Vorkenntnisse</b>. Du lernst von Grund auf, alles was du wissen
   musst, um eine <b>erfolgreiche Online-Präsenz</b>  aufzubauen!
  </Card>
  <Card
   title="Verwirkliche deine Ideen"
   callToAction="Starte jetzt"
   image="/_static/vectors/home/create-projects.svg"
   rtl={true}
   key="create-projects"
  >
   Mit unserer <b>fortschrittlichen</b> Programmierumgebung kannst du jede
   Art von <b>Webanwendung</b> entwickeln. Du kannst sogar 
   <b>mit deinen Freunden</b> an einem Projekt arbeiten und deinen Code mit
   ihnen teilen! Unsere Programmierumgebung unterstützt viele verschiedene
   Programmiersprachen, wie 
   <b>JavaScript</b>, 
   <b>TypeScript</b>, 
   <b>HTML</b>, 
   <b>CSS</b>, 
   <b>Sass/SCSS</b>, sodass du dir selbst die Sprache aussuchen kannst, die
   dir am besten gefällt. Neben den gängigen Sprachen haben wir zusätzlich
   eine eigene Sprache namens <b>InCode</b> entwickelt, die dir
   das Programmieren noch einfacher macht! Du kannst sie 
   <Link
    color={"blue.400"}
    target={"_blank"}
    href={"//incode.codeup.space"}
   >
    hier
   </Link>
    lernen.
  </Card>
  <Card
   title="Plane deine Projekte"
   callToAction="Auf gehts!"
```

```
        rtl={false}
        image="/_static/vectors/home/plan-projects.svg"
      >
        Mit unserem <b>intuitiven</b> Projektmanagement kannst du
        <b>effizient</b> und <b>strukturiert</b> an deinen Projekten arbeiten.
        Du kannst deine Projekte in <b>Teams</b> organisieren, sodass du mit
        deinen Freunden zusammenarbeiten kannst. Du kannst auch <b>Deadlines</b>{" "}
        für deine Projekte festlegen, sodass du immer <b>im Auge behältst</b>,
        was du wann erledigen musst.
      </Card>
      <Card
        title="Lerne von den Besten"
        image="/_static/vectors/home/communicate.svg"
        rtl={true}
        callToAction="Registrieren"
      >
        Unser <b>modernes</b> Forum bietet dir die Möglichkeit, dich mit anderen{" "}
        <b>Programmierern auszutauschen</b> und von ihnen 
        <b>zu lernen</b>. Der Austausch mit anderen Programmierern ist eine der
        besten Möglichkeiten, um deine Fähigkeiten zu verbessern. Du kannst dich
        mit Menschen austauschen, die bereits 
        <b>Erfahrung mit bestimmten Technologien</b> haben oder auf dem 
        <b>gleichen Lernstand</b> wie du sind.
      </Card>
      <Card
        title="Finde Inspiration"
        image="/_static/vectors/home/get-ideas.svg"
        rtl={false}
        callToAction="Starten"
      >
        Unsere <b>riesige</b> Sammlung an <b>Projektideen</b> bietet dir die
        Möglichkeit, immer neue Ideen zu finden, die du umsetzen kannst, um{" "}
        <b>neues Wissen zu erlange</b>. Viele Projektideen haben zusätzlich auch
        noch 
        <b>ausführliche Anleitungen</b> oder <b>Beispiele</b>, sodass du direkt
        loslegen kannst.
      </Card>
    </>
  );
}

function Card(props: {
  rtl: boolean;
  title: string;
  callToAction: string;
  children: React.ReactNode;
  image: string;
}) {
  return (
    <>
      <Flex
        direction={
          props.rtl ? ["column-reverse", "row-reverse"] : ["column", "row"]
        }
        gap="2rem"
        paddingInline={["2rem", "8rem"]}
        mt="8rem"
      >
        <Image
          src={props.image}
```

```
          width={"512"}
          height="512"
          style={{
            flex: "50%",
          }}
        />
        <Box flex="50%">
          <Flex direction="column">
            <Heading fontWeight="bold" fontSize="4rem">
              {props.title}
            </Heading>
            <Text fontSize="1.5rem" mt="2rem">
              {props.children}
            </Text>
            <CallToActionButton
              text={props.callToAction}
              url="/account/register"
            />
          </Flex>
        </Box>
      </Flex>
    </>
  );
}

function CallToActionButton(props: { text: string; url: string }) {
  const router = useRouter();

  return (
    <ButtonGroup mt="2rem">
      <a
        href={props.url}
        onClick={(e) => {
          e.preventDefault();
        }}
      >
        <Button
          colorScheme="messenger"
          size="lg"
          onClick={() => {
            router.push(props.url);
          }}
        >
          {props.text}
        </Button>
      </a>
    </ButtonGroup>
  );
}
```

## apps/web/src/pages/orgs/index.tsx

```
/**
 * apps/web/src/pages/orgs/index.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 05.03.2023
 *
 */
```

```
import * as React from "react";
import { useEffect } from "react";
import { useLoggedIn } from "@codeup/hooks";
import AuthenticateLoadingIndicator from
"@codeup/components/dist/admin/AuthenticateLoadingIndicator";
import REST from "@codeup/rest";
import AccountManager from "@codeup/util/src/AccountManager";
import PopupManager from "../../global/PopupManager";
import { useRouter } from "next/router";
import {
  Box,
  Button,
  ButtonGroup,
  chakra,
  Flex,
  Heading,
  IconButton,
  Input,
  InputGroup,
  InputLeftElement,
  Text,
} from "@chakra-ui/react";
import {
  FaCogs,
  FaDoorOpen,
  FaExternalLinkAlt,
  FaPlus,
  FaSearch,
  FaTrash,
} from "react-icons/fa";
import { CHeading } from "@codeup/components/dist/global/CHeading";

export default function Index() {
  const { loggedIn, loaded, userInfo } = useLoggedIn();
  const [query, setQuery] = React.useState<string>("");
  const router = useRouter();

  const [orgs, setOrgs] = React.useState<
    {
      owner: string;
      name: string;
      members: string[];
      icon: string;
      subscription: string;
      createdAt: string;
      _id: string;
    }[]
  >([]);

  useEffect(() => {
    if (loaded && loggedIn) {
      REST.Orgs.getOrgs(AccountManager.getActiveToken()).then((r) => {
        if (r.status === 200) {
          setOrgs(r.payload.orgs);
        } else {
          PopupManager.alert({
            title: "Fehler",
            description:
              "Organisationen konnten nicht geladen werden: " + r.payload.error,
```

```
        });
      }
    });
  } else if (loaded && !loggedIn) {
    router.push("/#login");
    window.dispatchEvent(new CustomEvent("codeup::open_login"));
  }
}, [loaded]);

if (!loaded)
  return (
    <>
      <AuthenticateLoadingIndicator />
    </>
  );

return (
  <>
    <Flex
      w={"100%"}
      h={"fit-content"}
      alignItems={"center"}
      flexDirection={"column"}
      bg={"#000"}
      pt={4}
      minH={"100vh"}
      pb={4}
    >
      <CHeading>Organisationen</CHeading>
      <Box
        w={["100%", "75%", "50%"]}
        h={"100%"}
        p={4}
        mt={4}
        overflow={"auto"}
      >
        <Flex w={"100%"} direction={"row"} gap={4}>
          <InputGroup w={"100%"}>
            <InputLeftElement>
              <FaSearch />
            </InputLeftElement>
            <Input
              placeholder={"Suche nach Organisationen"}
              onChange={(e) => {
                setQuery(e.target.value.trim());
              }}
            />
          </InputGroup>
          <Button
            variant={"brand"}
            onClick={async () => {
              const name = await PopupManager.promptAsync({
                title: "Neue Organisation",
                helperText: "Gib den Namen der Organisation ein.",
              });

              const res = await REST.Orgs.createOrg({
                token: AccountManager.getActiveToken(),
                name: name,
              });
```

```
      if (res.status === 200) {
        PopupManager.alert({
          title: "Erfolg",
          description: "Die Organisation wurde erfolgreich erstellt.",
        });
        setOrgs([...orgs, res.payload.org]);
      } else {
        PopupManager.alert({
          title: "Fehler",
          description:
            "Die Organisation konnte nicht erstellt werden: " +
            res.payload.error,
        });
      }
    }}
    leftIcon={<FaPlus />}
  >
    Neu
  </Button>
</Flex>
{orgs
  .filter((org) =>
    org.name.toLowerCase().includes(query.toLowerCase())
  )
  .map((org, i) => (
    <Flex
      key={i}
      w={"100%"}
      h={"100%"}
      p={4}
      mt={4}
      overflow={"auto"}
      justifyContent={"space-between"}
      alignItems={"center"}
      flexDirection={["column", "row"]}
      bg={"gray.900"}
      borderRadius={"md"}
      shadow={"xl"}
      onClick={(e) => {
        if (e.target !== e.currentTarget) return;

        router.push("/orgs/" + org.name + "/dashboard");
      }}
      cursor={"pointer"}
    >
      <Flex
        justifyContent={"center"}
        alignItems={"center"}
        flexDirection={"row"}
      >
        <chakra.img
          src={org.icon}
          w={"50px"}
          h={"50px"}
          objectFit={"cover"}
          borderRadius={"50%"}
          mr={4}
        />
        <Flex
```

```
            justifyContent={"center"}
            alignItems={"center"}
            flexDirection={"column"}
          >
            <Heading size={"md"}>{org.name}</Heading>
          </Flex>
        </Flex>
        <Flex
          justifyContent={"center"}
          alignItems={"center"}
          flexDirection={"row"}
          gap={2}
        >
          {org.owner === userInfo?.id ? (
            <>
              <Text color={"red.300"}>Von dir verwaltet</Text>
              <ButtonGroup isAttached>
                <IconButton
                  aria-label={"Verwalten"}
                  icon={<FaCogs />}
                  onClick={() => {
                    router.push("/orgs/" + org.name + "/manage");
                  }}
                  variant={"ghost"}
                  colorScheme={"blue"}
                />
                <IconButton
                  aria-label={"Löschen"}
                  icon={<FaTrash />}
                  onClick={async () => {
                    if (
                      await PopupManager.confirmAsync({
                        title: org.name + " löschen",
                        question:
                          "Bist du dir sicher, dass du diese Organisation löschen möchtest? Dieser Vorgang kann nicht rückgängig gemacht werden. Alle Daten werden unwiederbringlich gelöscht. Alle Mitglieder werden aus der Organisation entfernt. Alle Projekte werden gelöscht.",
                      })
                    ) {
                      if (
                        (await PopupManager.promptAsync({
                          title: "Bestätigung",
                          helperText:
                            "Gib den Namen der Organisation ein, um sie zu löschen.",
                        })) !== org.name
                      )
                        return PopupManager.alert({
                          title: "Fehler",
                          description:
                            "Der Name der Organisation ist nicht korrekt.",
                        });

                      const res = await REST.Orgs.deleteOrg({
                        token: AccountManager.getActiveToken(),
                        name: org.name,
                      });

                      if (res.status === 200) {
                        PopupManager.alert({
                          title: "Erfolg",
```

```
              description:
                "Die Organisation wurde erfolgreich gelöscht.",
            });
            setOrgs(orgs.filter((o) => o._id !== org._id));
          } else {
            PopupManager.alert({
              title: "Fehler",
              description:
                "Die Organisation konnte nicht gelöscht werden: " +
                res.payload.error,
            });
          }
        }
      }}
      variant={"ghost"}
      colorScheme={"red"}
    />
  </ButtonGroup>
 </>
) : (
 <>
  <Button
    leftIcon={<FaDoorOpen />}
    colorScheme={"red"}
    onClick={async () => {
      if (
        await PopupManager.confirmAsync({
          title: "Verlassen",
          question:
            "Bist du dir sicher, dass du die Organisation verlassen möchtest?",
        })
      ) {
        const res = await REST.Orgs.removeOrgMember({
          token: AccountManager.getActiveToken(),
          name: org.name,
          userId: userInfo?.id,
        });

        if (res.status === 200) {
          PopupManager.alert({
            title: "Erfolg",
            description:
              "Du hast die Organisation erfolgreich verlassen.",
          });
          setOrgs(orgs.filter((o) => o._id !== org._id));
        } else {
          PopupManager.alert({
            title: "Fehler",
            description:
              "Die Organisation konnte nicht verlassen werden: " +
              res.payload.error,
          });
        }
      }
    }}
  >
    Verlassen
  </Button>
 </>
)}
```

```
              </Flex>
              <Flex
                justifyContent={"center"}
                alignItems={"center"}
                flexDirection={"column"}
              >
                <Text mr={4}>
                  <b>Mitglieder:</b> {org.members.length}
                </Text>
                <Text mr={4}>
                  <b>Abonnement:</b> {org.subscription}
                </Text>
                <Text mr={4}>
                  <b>Erstellt am:</b>{" "}
                  {new Date(org.createdAt).toLocaleString()}
                </Text>
              </Flex>
            </Flex>
          ))}
        {orgs.filter((org) =>
          org.name.toLowerCase().includes(query.toLowerCase())
        ).length === 0 ? (
          <>
            <Text color={"red.300"} mt={4}>
              Keine Organisationen gefunden
            </Text>
          </>
        ) : null}
      </Box>
    </Flex>
  </>
 );
}
```

## apps/web/src/pages/orgs/[name]/dashboard.tsx

```
/**
 * apps/web/src/pages/orgs/[name]/dashboard.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 18.03.2023
 *
 */

import * as React from "react";
import AuthenticateLoadingIndicator from
"@codeup/components/dist/admin/AuthenticateLoadingIndicator";
import { useEffect } from "react";
import { useRouter } from "next/router";
import { useLoggedIn } from "@codeup/hooks";
import ErrorComponent from "@codeup/components/dist/global/ErrorComponent";
import REST from "@codeup/rest";
import AccountManager from "@codeup/util/dist/AccountManager";
import {
  Accordion,
  AccordionButton,
  AccordionIcon,
  AccordionItem,
  AccordionPanel,
```

```
  Box,
  Button,
  ButtonGroup,
  Divider,
  Flex,
  Grid,
  GridItem,
  Heading,
  HStack,
  IconButton,
  Image,
  Stack,
  StackDivider,
  Tab,
  Table,
  TableContainer,
  TabList,
  TabPanel,
  TabPanels,
  Tabs,
  Tbody,
  Td,
  Text,
  Th,
  Thead,
  Tr,
  VStack,
} from "@chakra-ui/react";
import { FaCogs, FaInfo, FaPlus, FaTrash } from "react-icons/fa";
import PopupManager from "../../../global/PopupManager";
import { CHeading } from "@codeup/components/dist/global/CHeading";

export default function Dashboard() {
  const router = useRouter();
  const { loggedIn, loaded, userInfo } = useLoggedIn();
  const [org, setOrg] = React.useState<{
    owner: string;
    name: string;
    members: string[];
    icon: string;
    subscription: string;
    createdAt: string;
    _id: string;
  }>();
  const [blogs, setBlogs] = React.useState<
    {
      title: string;
      content: string;
      createdAt: string;
      _id: string;
    }[]
  >([]);

  const [error, setError] = React.useState<string>();

  useEffect(() => {
    if (!router || !router.query || !router.query.name) {
      return;
    }
```

```
  if (!loaded) return;

  if (!loggedIn) {
    router.push("/#login");
    window.dispatchEvent(new CustomEvent("codeup::open_login"));
    return;
  }

  REST.Orgs.getOrg({
    token: AccountManager.getActiveToken(),
    name: router.query.name as string,
  }).then(async (data) => {
    if (data.status === 200) {
      setOrg(data.payload.org);
      reloadBlogs();
      reloadTemplates();
      reloadCourses(data.payload.org);
    } else {
      setError(data.payload.error);
    }
  });
}, [router, loggedIn, loaded]);

const [templates, setTemplates] = React.useState<
  {
    name: string;
    projectId: string;
    _id: string;
  }[]
>([]);

const [courses, setCourses] = React.useState<
  {
    _id: string;
    name: string;
    friendlyName: string;
    description: string;
    splashImage: string;
    available: string;
    contentPositions: string[];
    creator: string;
    createdAt: string;
    __v: number;
  }[]
>([]);

const reloadTemplates = async () => {
  const templateRes = await REST.Orgs.getOrgTemplates({
    token: AccountManager.getActiveToken(),
    org: router.query.name as string,
  });

  if (templateRes.status === 200) {
    setTemplates(templateRes.payload.templates);
  }
};

const reloadBlogs = async () => {
  const blogRes = await REST.Orgs.getBlogs({
    token: AccountManager.getActiveToken(),
```

```
    org: router.query.name as string,
  });

  if (blogRes.status === 200) {
    const sortedBlogs = blogRes.payload.blogs.sort((a: any, b: any) => {
      return (
        new Date(b.createdAt).getTime() - new Date(a.createdAt).getTime()
      );
    });
    setBlogs(sortedBlogs);
  }
};

const reloadCourses = async (org: any) => {
  const allCourses = await REST.Course.listCourses({
    token: AccountManager.getActiveToken(),
  });

  if (allCourses.status === 200) {
    const courses = allCourses.payload.courses
      .filter((course: any) => {
        return course.available === "org:" + org._id;
      })
      .map((c: any) => {
        c.contentPositions = c.contentPositions.filter(
          (x: any) => x !== "false"
        );
        return c;
      });
    setCourses(courses);
  }
};

if (error) {
  return (
    <>
      <ErrorComponent description={error} title={"Fehler"} />
    </>
  );
}

return (
  <>
    {!org ? (
      <>
        <AuthenticateLoadingIndicator />
      </>
    ) : (
      <>
        <Flex
          direction={"column"}
          pt={"2rem"}
          alignItems={"center"}
          w={"100%"}
          minH={"100vh"}
          h={"fit-content"}
          bg={"#000"}
        >
          <Box
            w={["100%", "75%"]}
```

```
  backgroundColor={"black"}
  p={4}
  rounded={"xl"}
  boxShadow={"xl"}
>
  {org.owner === userInfo.id ? (
    <>
      <Flex
        w={"100%"}
        justifyContent={"space-between"}
        alignItems={"center"}
      >
        <Heading color={"red.400"} size={"md"}>
          Organisations Administrator
        </Heading>
        <Text></Text>
        <IconButton
          aria-label={"Verwalten"}
          icon={<FaCogs />}
          colorScheme={"red"}
          onClick={() => {
            router.push("/orgs/" + org.name + "/manage");
          }}
        />
      </Flex>
      <Divider mt={2} mb={2} backgroundColor={"red"} />
    </>
  ) : null}
  <Stack>
    <Box textAlign={"center"}>
      <CHeading size="2xl">{org.name}</CHeading>
    </Box>
    <StackDivider />
    <Flex
      w={"100%"}
      justifyContent={"center"}
      alignItems={"center"}
    >
      <Accordion
        maxW={["100%", "20%"]}
        w={["100%", "20%"]}
        allowToggle
      >
        <AccordionItem>
          <h2>
            <AccordionButton>
              <Box as="span" flex={1} textAlign={"left"}>
                Informationen
              </Box>
              <FaInfo />
            </AccordionButton>
          </h2>

          <AccordionPanel pb={4}>
            <Text fontSize={"md"}>
              Mitglieder: {org.members.length}
            </Text>
            <Text fontSize={"md"}>
              Erstellt am:{" "}
              {new Date(org.createdAt).toLocaleString()}
```

```
        </Text>
        <Text fontSize={"md"}>
          Abonnement: {org.subscription}
        </Text>
      </AccordionPanel>
    </AccordionItem>
  </Accordion>
</Flex>
<StackDivider />
<Tabs
  w={"100%"}
  variant={"enclosed-colored"}
  isFitted
  colorScheme={"brand"}
>
  <TabList w={"100%"}>
    <Tab>Mitteilungen</Tab>
    <Tab>Vorlagen</Tab>
    <Tab>Kurse</Tab>
  </TabList>

  <TabPanels>
    <TabPanel>
      <Flex
        w={"100%"}
        justifyContent={"space-between"}
        alignItems={"center"}
        mb={4}
      >
        <Heading>Neuste Meldungen</Heading>
        <i></i>
        {org.owner === userInfo.id ? (
          <Button
            variant={"brand"}
            leftIcon={<FaPlus />}
            onClick={async () => {
              const title = await PopupManager.promptAsync({
                title: "Titel der Meldung",
                helperText:
                  "Gib einen Titel für die Meldung ein",
              });

              if (!title) return;

              const content = await PopupManager.promptAsync({
                title: "Inhalt der Meldung",
                helperText:
                  "Gib einen Inhalt für die Meldung ein",
                inputType: "TEXTAREA",
              });

              if (!content) return;

              const res = await REST.Orgs.createBlog({
                token: AccountManager.getActiveToken(),
                org: org.name,
                title,
                content,
              });
```

```
        if (res.status === 200) {
          reloadBlogs();
        } else {
          PopupManager.alertAsync({
            title: "Fehler",
            description:
              "Die Meldung konnte nicht erstellt werden: " +
              res.payload.error,
          });
        }
      }}
    >
      Neue Meldung
    </Button>
  ) : null}
 </Flex>
 {blogs.length > 0 ? (
  blogs.map((blog) => {
    return (
      <Accordion allowToggle>
        <AccordionItem>
          <h2>
            <AccordionButton>
              <Box as="span" flex={1} textAlign={"left"}>
                {new Date(
                  blog.createdAt
                ).toLocaleString()}
                 - 
                {blog.title}
              </Box>
              <AccordionIcon />
            </AccordionButton>
          </h2>
          <AccordionPanel pb={4}>
            <Text fontSize={"md"}>{blog.content}</Text>
          </AccordionPanel>
        </AccordionItem>
      </Accordion>
    );
  })
 ) : (
  <Text>Keine Meldungen vorhanden</Text>
 )}
</TabPanel>
<TabPanel>
 <Stack>
  {org.owner === userInfo.id ? (
    <>
      <Flex w={"100%"} justifyContent={"flex-end"}>
        <Button
          onClick={async () => {
            const projectsRes =
              await REST.Editor.getProjectsV2(
                AccountManager.getActiveToken()
              );

            if (projectsRes.status === 200) {
              const projects: any[] =
                projectsRes.payload.projects;
```

```
            const choice =
              await PopupManager.selectAsync({
                title: "Projekt auswählen",
                helperText:
                  "Wähle ein Projekt aus, welches als Vorlage verwendet werden soll.",
                choices: projects.map((project) => {
                  return project.name;
                }),
              });

            if (!choice) return;

            const project = projects.find(
              (project) => project.name === choice
            );

            const templateRes =
              await REST.Orgs.addOrgTemplate({
                token: AccountManager.getActiveToken(),
                org: org.name,
                projectId: project._id,
              });

            if (templateRes.status === 200) {
              reloadTemplates();
            } else {
              PopupManager.alert({
                title: "Fehler",
                description:
                  "Projekt konnte nicht als Vorlage hinzugefügt werden: " +
                  templateRes.payload.error,
              });
            }
          } else {
            PopupManager.alert({
              title: "Fehler",
              description:
                "Projekte konnten nicht geladen werden: " +
                projectsRes.payload.error,
            });
          }
        }}
        variant={"brand"}
        leftIcon={<FaPlus />}
      >
        Projekt hinzufügen
      </Button>
    </Flex>

    <StackDivider />
  </>
) : null}
{templates.length === 0 && (
  <>
    <Text
      textAlign={"center"}
      color={"red.300"}
      fontSize={"lg"}
    >
      In dieser Organisation gibt es noch keine
```

```
      veröffentlichten Vorlagen.
    </Text>
  </>
)}

<Grid
  templateColumns={[
    "repeat(1, 1fr)",
    "repeat(2, 1fr)",
    "repeat(3, 1fr)",
    "repeat(4, 1fr)",
    "repeat(5, 1fr)",
  ]}
  gap={6}
  p={4}
  mt={"2rem"}
>
  {templates.map((template) => {
    return (
      <>
        <GridItem
          key={"new"}
          bg={"gray.700"}
          p={"1rem"}
          borderRadius={"0.5rem"}
        >
          <Stack>
            <Heading textAlign={"center"} size={"lg"}>
              {template.name}
            </Heading>
            <StackDivider />
            <StackDivider />
            <Button
              variant={"brand"}
              leftIcon={<FaPlus />}
              onClick={async (e) => {
                if (e.target !== e.currentTarget)
                  return;

                const name =
                  await PopupManager.promptAsync({
                    title: "Neues Projekt",
                    helperText:
                      "Gib einen Namen für das Projekt ein.",
                  });

                if (!name) return;

                const useRes =
                  await REST.Orgs.useOrgTemplate({
                    token:
                      AccountManager.getActiveToken(),
                    org: org.name,
                    projectId: template._id,
                    name,
                  });

                if (useRes.status === 200) {
                  router.push(
                    "/editor/" +
```

```
                                    useRes.payload.project._id
                                  );
                                } else {
                                  PopupManager.alert({
                                    title: "Fehler",
                                    description:
                                      "Projekt konnte nicht erstellt werden: " +
                                      useRes.payload.error,
                                  });
                                }
                              }}
                            >
                              Verwenden
                            </Button>
                            {org.owner === userInfo.id ? (
                              <>
                                <Button
                                  colorScheme={"red"}
                                  leftIcon={<FaTrash />}
                                  onClick={async () => {
                                    if (
                                      await PopupManager.confirmAsync({
                                        title: "Vorlage löschen",
                                        question:
                                          "Bist du dir sicher, dass du diese Vorlage löschen möchtest? Dies
kann nicht rückgängig gemacht werden.",
                                      })
                                    ) {
                                      const r =
                                        await REST.Orgs.removeOrgTemplate(
                                          {
                                            token:
                                              AccountManager.getActiveToken(),
                                            projectId: template._id,
                                            org: org.name,
                                          }
                                        );

                                      if (r.status === 200) {
                                        reloadTemplates();
                                      } else {
                                        PopupManager.alert({
                                          title: "Fehler",
                                          description:
                                            "Vorlage konnte nicht gelöscht werden: " +
                                            r.payload.error,
                                        });
                                      }
                                    }
                                  }}
                                >
                                  Entfernen
                                </Button>
                              </>
                            ) : null}
                          </Stack>
                        </GridItem>
                      </>
                    );
                  })}
```

```
                </Grid>
              </Stack>
            </TabPanel>
            <TabPanel>
              <TableContainer>
                <Table>
                  <Thead>
                    <Tr>
                      <Th></Th>
                      <Th>Name</Th>
                      <Th>Creator</Th>
                      <Th>Beschreibung</Th>
                      <Th>Lektionen</Th>
                    </Tr>
                  </Thead>
                  <Tbody>
                    {courses.map((c) => {
                      return (
                        <>
                          <Tr
                            onClick={() => {
                              router.push("/course/" + c.friendlyName);
                            }}
                            cursor={"pointer"}
                            _hover={{
                              bg: "gray.700",
                            }}
                          >
                            <Td>
                              <Image w={"100px"} src={c.splashImage} />
                            </Td>
                            <Td>{c.friendlyName}</Td>
                            <Td>@{c.creator}</Td>
                            <Td>{c.description}</Td>
                            <Td>{c.contentPositions.length}</Td>
                          </Tr>
                        </>
                      );
                    })}
                  </Tbody>
                </Table>
              </TableContainer>
            </TabPanel>
          </TabPanels>
        </Tabs>
      </Stack>
    </Box>
  </Flex>
 </>
 )}
 </>
 );
}
```

## apps/web/src/pages/orgs/[name]/manage.tsx

```
/**
 * apps/web/src/pages/orgs/[name]/manage.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
```

```
 * License: Project License
 * Created At: 06.03.2023
 *
 */

import * as React from "react";
import { useLoggedIn } from "@codeup/hooks";
import OrgAdminLayout from "@codeup/components/orgs/OrgAdminLayout";
import { useRouter } from "next/router";
import {
  Box,
  Button,
  chakra,
  Divider,
  Flex,
  FormControl,
  Heading,
  HStack,
  IconButton,
  Input,
  InputGroup,
  InputLeftElement,
  Link,
  Stack,
  Table,
  TableCaption,
  TableContainer,
  Tbody,
  Td,
  Text,
  Tfoot,
  Th,
  Thead,
  Tr,
  VStack,
} from "@chakra-ui/react";
import REST from "@codeup/rest";
import AccountManager from "@codeup/util/src/AccountManager";
import { FaPen, FaPlus, FaSearch, FaTrash } from "react-icons/fa";
import PopupManager from "../../../global/PopupManager";

export default function ManageDashboard() {
  const { userInfo, loggedIn, loaded } = useLoggedIn();
  const router = useRouter();
  const [org, setOrg] = React.useState<{
    owner: string;
    name: string;
    members: string[];
    icon: string;
    subscription: string;
    createdAt: string;
    _id: string;
  } | null>();

  const [fetchedMembers, setFetchedMembers] = React.useState<
    {
      id: string;
      username: string;
      firstName: string;
      lastName: string;
```

```
    profilePicture: string;
  }[]
>([]);

const [memberQuery, setMemberQuery] = React.useState<string>("");

const fetchOrg = () => {
  REST.Orgs.getOrg({
    token: AccountManager.getActiveToken(),
    name: router.query.name as string,
  }).then(async (r) => {
    if (r.status === 200) {
      setOrg(r.payload.org);

      let members: typeof fetchedMembers = [];

      for (const member of r.payload.org.members as string[]) {
        const r = await REST.Account.getUserById(member);

        if (r.status === 200) {
          members.push(r.payload.user);
        } else {
          console.error(r);
        }
      }

      setFetchedMembers(members);
    }
  });
};

React.useEffect(() => {
  if (!router || !router.query || !router.query.name) return;
  if (loaded && !loggedIn) {
    router.push("/#login");
    window.dispatchEvent(new CustomEvent("codeup::open_login"));
  } else {
    fetchOrg();
  }
}, [loaded, loggedIn, router]);

return (
  <>
    <OrgAdminLayout orgName={router.query.name as string}>
      <Flex
        w={"100%"}
        h={"fit-content"}
        justifyContent={"center"}
        alignItems={"center"}
        minH={"100vh"}
        bg={"#000"}
      >
        <Box
          w={["100%", "75%", "50%"]}
          shadow={"xl"}
          bgColor={"gray.900"}
          mt={4}
          rounded={"lg"}
          p={6}
        >
```

```
{org && (
  <>
    <Flex
      w={"100%"}
      direction={"row"}
      alignItems={"center"}
      justifyContent={"space-between"}
      gap={6}
    >
      <VStack>
        <chakra.img
          src={org.icon}
          w={"100px"}
          h={"100px"}
          objectFit={"cover"}
          borderRadius={"50%"}
          mr={4}
        />
        <Link
          color={"brand.500"}
          onClick={async () => {
            const fl: FileList = await PopupManager.uploadFileAsync(
              {
                title: "Logo ändern",
                helperText:
                  "Wähle ein neues Logo für die Organisation aus.",
              }
            );

            if (fl.length === 0) return;

            const formData = new FormData();
            formData.append("file", fl[0]);

            const res = await fetch("/media/upload", {
              method: "POST",
              body: formData,
            });

            if (res.status === 200) {
              const data = await res.json();

              const orgRes = await REST.Orgs.updateOrgIcon({
                token: AccountManager.getActiveToken(),
                name: org.name,
                icon: data.data.url,
              });

              if (orgRes.status === 200) {
                fetchOrg();
              } else {
                PopupManager.alertAsync({
                  title: "Fehler",
                  description:
                    "Beim Hochladen des Logos ist ein Fehler aufgetreten: " +
                    orgRes.payload.error,
                });
              }
            } else {
              PopupManager.alertAsync({
```

```
          title: "Fehler",
          description:
            "Beim Hochladen des Logos ist ein Fehler aufgetreten: " +
            res.statusText,
        });
      }
    }}
  >
    Logo ändern
  </Link>
</VStack>
<HStack>
  <Heading size={"xl"}>{org.name}</Heading>
  <IconButton
    aria-label={"Umbennenen"}
    onClick={async () => {
      const name = await PopupManager.promptAsync({
        title: "Umbenennen",
        helperText:
          "Gib den neuen Namen der Organisation ein. Der Name muss eindeutig sein.",
      });

      if (name) {
        const res = await REST.Orgs.renameOrg({
          token: AccountManager.getActiveToken(),
          oldName: org.name,
          newName: name,
        });

        if (res.status === 200) {
          router.push("/orgs/" + name + "/manage");
        } else {
          PopupManager.alertAsync({
            title: "Fehler",
            description:
              "Beim Umbenennen ist ein Fehler aufgetreten: " +
              res.payload.error,
          });
        }
      }
    }}
    icon={<FaPen />}
    variant={"ghost"}
    colorScheme={"blue"}
  />
</HStack>
<Stack>
  <Text>
    <b>Erstellt am:</b> 
    {new Date(org.createdAt).toLocaleString()}
  </Text>
  <Text>
    <b>Abonoment:</b> 
    {org.subscription}
  </Text>
  <Text>
    <b>Mitglieder:</b> 
    {org.members.length}
  </Text>
</Stack>
```

```
</Flex>
<Divider mt={6} />
<Box mt={6}>
  <Heading size={"md"}>Mitglieder</Heading>
  <Flex alignItems={"center"} justifyContent={"center"} gap={4}>
    <InputGroup mt={4} mb={4}>
      <InputLeftElement>
        <FaSearch />
      </InputLeftElement>
      <Input
        placeholder={"Mitglieder suchen..."}
        value={memberQuery}
        onChange={(e) => setMemberQuery(e.target.value)}
      />
    </InputGroup>
    <Button
      variant={"brand"}
      leftIcon={<FaPlus />}
      onClick={async () => {
        const name = await PopupManager.promptAsync({
          title: "Mitglied hinzufügen",
          helperText:
            "Gib den Benutzernamen des Mitglieds ein.",
        });

        if (name) {
          const accRes = await REST.Account.getUserByName(name);

          if (accRes.status === 200) {
            const res = await REST.Orgs.addOrgMember({
              token: AccountManager.getActiveToken(),
              name: org.name,
              userId: accRes.payload.user.id,
            });

            if (res.status === 200) {
              fetchOrg();
            } else {
              PopupManager.alertAsync({
                title: "Fehler",
                description:
                  "Beim Hinzufügen des Mitglieds ist ein Fehler aufgetreten: " +
                  res.payload.error,
              });
            }
          } else {
            PopupManager.alertAsync({
              title: "Fehler",
              description:
                "Beim Hinzufügen des Mitglieds ist ein Fehler aufgetreten: " +
                accRes.payload.error,
            });
          }
        }
      }}
    >
      Hinzufügen
    </Button>
  </Flex>
  <TableContainer mt={2}>
```

```
<Table>
  <Thead>
    <Tr>
      <Th>Profilbild</Th>
      <Th>Benutzername</Th>
      <Th>Name</Th>
      <Th>Aktion</Th>
    </Tr>
  </Thead>
  <Tbody>
    {fetchedMembers
      .filter((m) => {
        return (
          m.username
            .toLowerCase()
            .includes(memberQuery.toLowerCase()) ||
          m.firstName
            .toLowerCase()
            .includes(memberQuery.toLowerCase()) ||
          m.lastName
            .toLowerCase()
            .includes(memberQuery.toLowerCase())
        );
      })
      .map((m) => {
        return (
          <Tr key={m.id}>
            <Td>
              <chakra.img
                src={m.profilePicture}
                w={"50px"}
                h={"50px"}
                objectFit={"cover"}
                borderRadius={"50%"}
              />
            </Td>
            <Td>@{m.username}</Td>
            <Td>{m.firstName + " " + m.lastName}</Td>
            <Td>
              <Button
                colorScheme={"red"}
                leftIcon={<FaTrash />}
                onClick={async () => {
                  if (
                    !(await PopupManager.confirmAsync({
                      title: "Mitglied entfernen",
                      question:
                        "Bist du sicher, dass du " +
                        m.username +
                        " aus der Organisation entfernen möchtest?",
                    }))
                  )
                    return;

                  const res =
                    await REST.Orgs.removeOrgMember({
                      token:
                        AccountManager.getActiveToken(),
                      name: org.name,
                      userId: m.id,
```

```
                  });

                if (res.status === 200) {
                  setFetchedMembers(
                    fetchedMembers.filter(
                      (m0) => m0.id !== m.id
                    )
                  );
                } else {
                  PopupManager.alertAsync({
                    title: "Fehler",
                    description:
                      "Beim Entfernen des Mitglieds ist ein Fehler aufgetreten: " +
                      res.payload.error,
                  });
                }
              }}
            >
              Entfernen
            </Button>
          </Td>
        </Tr>
      );
    })}
  </Tbody>
  <TableCaption>
    {fetchedMembers.filter((m) => {
      return (
        m.username
          .toLowerCase()
          .includes(memberQuery.toLowerCase()) ||
        m.firstName
          .toLowerCase()
          .includes(memberQuery.toLowerCase()) ||
        m.lastName
          .toLowerCase()
          .includes(memberQuery.toLowerCase())
      );
    }).length === 0 ? (
      <Text>Keine Mitglieder gefunden.</Text>
    ) : (
      fetchedMembers.filter((m) => {
        return (
          m.username
            .toLowerCase()
            .includes(memberQuery.toLowerCase()) ||
          m.firstName
            .toLowerCase()
            .includes(memberQuery.toLowerCase()) ||
          m.lastName
            .toLowerCase()
            .includes(memberQuery.toLowerCase())
        );
      }).length + " Mitglieder gefunden."
    )}
  </TableCaption>
 </Table>
</TableContainer>
</Box>
</>
```

```
          )}
        </Box>
      </Flex>
    </OrgAdminLayout>
  </>
 );
}
```

## apps/web/src/pages/payment/success/thank_you.tsx

```
/**
 * apps/web/src/pages/payment/success/thank_you.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 16.06.2023
 *
 */

import * as React from "react";
import { Flex, Heading } from "@chakra-ui/react";
import { FaCheck } from "react-icons/fa";

export default function ThankYou() {
 return (
  <>
   <Flex
    w={"100%"}
    h={"100vh"}
    justifyContent={"center"}
    alignItems={"center"}
    direction={"column"}
    bg={"#000"}
   >
    <Heading fontSize={"16rem"} color={"brand.500"}>
     <FaCheck />
    </Heading>
    <Heading fontSize={["2rem", "4rem", "5rem"]} color={"brand.500"} p={4}>
     Vielen Dank für deinen Kauf
    </Heading>
   </Flex>
  </>
 );
}
```

## apps/web/src/pages/planning/index.tsx

```
/**
 * apps/web/src/pages/planning/index.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 04.12.2022
 *
 */

import * as React from "react";
import { useLoggedIn } from "@codeup/hooks";
import { useEffect } from "react";
import { useRouter } from "next/router";
```

```
import REST from "@codeup/rest";
import {
  Box,
  Button,
  ButtonGroup,
  FormControl,
  FormLabel,
  Heading,
  Input,
  Popover,
  PopoverArrow,
  PopoverBody,
  PopoverCloseButton,
  PopoverContent,
  PopoverHeader,
  PopoverTrigger,
  SimpleGrid,
  Text,
  useToast,
} from "@chakra-ui/react";
import Head from "next/head";
import GridLayout from "@codeup/components/global/GridLayout";
import { FaEye, FaPlus, FaTrash } from "react-icons/fa";
import PopupManager from "../../global/PopupManager";
import GridLayoutV2 from "@codeup/components/dist/global/GridLayoutV2";
import byteSize from "byte-size";

export default function Index() {
  const { userInfo, loggedIn, loaded } = useLoggedIn();
  const router = useRouter();

  const [projects, setProjects] = React.useState<any[]>([]);

  useEffect(() => {
    if (!loggedIn && loaded) {
      router.push("/");
      window.dispatchEvent(new CustomEvent("codeup::open_login"));
    } else if (loaded && loggedIn) {
      reload();
    }
  }, [userInfo, loggedIn, loaded]);

  const toast = useToast();

  const reload = () => {
    REST.ToDo.getV2Projects(localStorage.getItem("token") as string).then(
      (res) => {
        if (res.status === 200) {
          setProjects(res.payload.projects);
        } else {
          toast({
            title: "Fehler",
            description:
              "Beim Laden der Projekte ist ein Fehler aufgetreten: " +
              res.payload.error,
            status: "error",
            duration: 1500,
            isClosable: true,
            position: "bottom-right",
          });
```

```
    }
   }
  );
};

return (
  <>
    <Head>
      <title>Projektplanung - CodeUp</title>
    </Head>
    <Box h={"fit-content"} minH={"100vh"} bg={"#000"}>
      <GridLayoutV2
        pageTitle={"Projekte"}
        search={{
          show: true,
          fields: true,
        }}
        topButtons={[
          {
            text: "Neu",
            leftIcon: <FaPlus />,
            variant: "brand",
            onClick: async () => {
              const name = await PopupManager.promptAsync({
                title: "Neues Projekt",
                helperText: "Gib den Namen des Projektes ein",
              });

              if (!name) return;

              const res = await REST.ToDo.createV2Project({
                token: localStorage.getItem("token") as string,
                name: name,
              });
              if (res.status === 200) {
                reload();
                toast({
                  position: "bottom-right",
                  status: "success",
                  title: "Erfolgreich",
                  description: "Das Projekt wurde erstellt",
                  isClosable: true,
                  duration: 1500,
                });
              } else {
                toast({
                  position: "bottom-right",
                  status: "error",
                  title: "Fehler",
                  description: res.payload.error,
                  isClosable: true,
                  duration: 1500,
                });
              }
            },
          },
        ]}
        entries={projects.map((p) => {
          return {
            title: p.name,
```

```
        fields: [
          {
            text:
              "Gespeicherte Daten: " +
              byteSize(p.saved.length).toString(),
          },
        ],
        buttonRows: [
          [
            {
              text: "Öffnen",
              leftIcon: <FaEye />,
              variant: "brand",
              onClick: () => {
                router.push("/planning/" + p._id);
              },
            },
            {
              text: "Löschen",
              leftIcon: <FaTrash />,
              colorScheme: "red",
              onClick: async () => {
                REST.ToDo.deleteV2Project({
                  token: localStorage.getItem("token") as string,
                  id: p._id,
                }).then((res) => {
                  if (res.status === 200) {
                    toast({
                      position: "bottom-right",
                      status: "success",
                      title: "Erfolgreich",
                      description: "Das Projekt wurde gelöscht",
                      isClosable: true,
                      duration: 1500,
                    });
                    reload();
                  } else {
                    toast({
                      position: "bottom-right",
                      status: "error",
                      title: "Fehler",
                      description: res.payload.error,
                      isClosable: true,
                      duration: 1500,
                    });
                  }
                });
              },
            },
          ],
        ],
      };
    })}
  />
</Box>
<Heading size="2xl" textAlign={"center"} mt={"2rem"}>
  Projekte
</Heading>
<GridLayout
  search={{
```

```
  allow: true,
  searchFields: true,
}}
specialEntries={[
  {
    icon: <FaPlus />,
    onClick: async () => {
      const name = await PopupManager.promptAsync({
        title: "Neues Projekt",
        helperText: "Gib den Namen des Projektes ein",
      });

      if (!name) return;

      const res = await REST.ToDo.createV2Project({
        token: localStorage.getItem("token") as string,
        name: name,
      });
      if (res.status === 200) {
        reload();
        toast({
          position: "bottom-right",
          status: "success",
          title: "Erfolgreich",
          description: "Das Projekt wurde erstellt",
          isClosable: true,
          duration: 1500,
        });
      } else {
        toast({
          position: "bottom-right",
          status: "error",
          title: "Fehler",
          description: res.payload.error,
          isClosable: true,
          duration: 1500,
        });
      }
    },
  },
]}
entries={projects.map((p) => {
  return {
    title: p.name,
    fields: [],
    buttonRows: [
      {
        buttons: [
          {
            text: "Öffnen",
            icon: <FaEye />,
            colorScheme: "blue",
            onClick: () => {
              router.push("/planning/" + p._id);
            },
          },
        ],
      },
      {
        buttons: [
```

```
                {
                  text: "Löschen",
                  icon: <FaTrash />,
                  colorScheme: "red",
                  onClick: async () => {
                    REST.ToDo.deleteV2Project({
                      token: localStorage.getItem("token") as string,
                      id: p._id,
                    }).then((res) => {
                      if (res.status === 200) {
                        toast({
                          position: "bottom-right",
                          status: "success",
                          title: "Erfolgreich",
                          description: "Das Projekt wurde gelöscht",
                          isClosable: true,
                          duration: 1500,
                        });
                        reload();
                      } else {
                        toast({
                          position: "bottom-right",
                          status: "error",
                          title: "Fehler",
                          description: res.payload.error,
                          isClosable: true,
                          duration: 1500,
                        });
                      }
                    });
                  },
                },
              ],
            },
          ],
        };
      })}
    />
   </>
  );
}
```

## apps/web/src/pages/planning/[id]/index.tsx

```
/**
 * apps/web/src/pages/planning/[id]/index.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 04.12.2022
 *
 */

import * as React from "react";
import { useCallback, useEffect } from "react";
import ReactFlow, {
  addEdge,
  applyEdgeChanges,
  applyNodeChanges,
  Background,
```

```
  Connection,
  Controls,
  Edge,
  EdgeChange,
  MarkerType,
  MiniMap,
  Node,
  NodeChange,
  ReactFlowInstance,
} from "reactflow";
import {
  Box,
  Button,
  ButtonGroup,
  Divider,
  Flex,
  Heading,
  Input,
  Select,
  Stack,
  useToast,
} from "@chakra-ui/react";
import { FaPlus, FaTrash } from "react-icons/fa";
import REST from "@codeup/rest";
import { useRouter } from "next/router";
import { useLoggedIn } from "@codeup/hooks";
import Head from "next/head";

export default function Index() {
  const { userInfo, loggedIn, loaded } = useLoggedIn();

  const router = useRouter();

  useEffect(() => {
    if (!loggedIn && loaded) {
      router.push("/");
      window.dispatchEvent(new CustomEvent("codeup::open_login"));
    }
  }, [userInfo, loggedIn, loaded]);

  const [nodes, setNodes] = React.useState<Node[]>([]);
  const [edges, setEdges] = React.useState<Edge[]>([]);

  const onNodesChange = useCallback(
    (changes: NodeChange[]) =>
      setNodes((nds) => applyNodeChanges(changes, nds)),
    [setNodes]
  );
  const onEdgesChange = useCallback(
    (changes: EdgeChange[]) =>
      setEdges((eds) => applyEdgeChanges(changes, eds)),
    [setEdges]
  );
  const onConnect = useCallback(
    (connection: Connection) => setEdges((eds) => addEdge(connection, eds)),
    [setEdges]
  );

  const [currentEdge, setCurrentEdge] = React.useState<Edge | null>(null);
  const [currentNode, setCurrentNode] = React.useState<Node | null>(null);
```

```
const [lastEdgeId, setLastEdgeId] = React.useState<string>("");
const [lastNodeId, setLastNodeId] = React.useState<string>("");

const [rfInstance, setRfInstance] = React.useState<ReactFlowInstance | null>(
  null
);

const toast = useToast();

useEffect(() => {
  setLastEdgeId("__rendered_" + currentEdge?.id ?? "");
}, [currentEdge]);

useEffect(() => {
  setLastNodeId("__rendered_" + currentNode?.id ?? "");
}, [currentNode]);

useEffect(() => {
  if (rfInstance) {
    const saved = rfInstance.toObject();

    REST.ToDo.updateV2Project({
      token: localStorage.getItem("token") as string,
      id: router.query.id as string,
      saved: JSON.stringify(saved),
    });
  }
}, [edges, nodes]);

useEffect(() => {
  const savedNodes: Node[] = [];
  const savedEdges: Edge[] = [];

  REST.ToDo.getV2Project({
    token: localStorage.getItem("token") as string,
    id: router.query.id as string,
  }).then((res) => {
    if (res.status === 200) {
      let p = res.payload.projects[0];
      if (!p.saved || p.saved === "") {
        savedNodes.push({
          id: `1`,
          data: { label: "Hauptknoten" },
          position: {
            x: 100,
            y: 100,
          },
          style: {
            background: "#4422FF",
            color: "white",
          },
          deletable: false,
        });
      } else {
        const saved = JSON.parse(p.saved);
        savedNodes.push(...saved.nodes);
        savedEdges.push(...saved.edges);
      }
```

```
      setNodes(savedNodes);
      setEdges(savedEdges);
    } else {
      toast({
        title: "Fehler",
        description:
          "Beim Laden des Projekts ist ein Fehler aufgetreten: " +
          res.payload.message,
        status: "error",
        duration: 1500,
        isClosable: true,
        position: "bottom-right",
      });
    }
  });
}, [router.query.id]);

return (
  <>
    <Head>
      <title>Projektplanung - CodeUp</title>
    </Head>
    <Flex
      maxH={"100vh"}
      h={"100vh"}
      p={"4rem"}
      paddingBottom={"8rem"}
      paddingTop={"0"}
      alignItems={"center"}
      justifyContent={"center"}
      pt={"2rem"}
      bg={"#000"}
      gap={6}
    >
      <Box h={"100%"} w={"100%"} flex={"80%"}>
        <ReactFlow
          nodes={nodes}
          edges={edges}
          onConnect={onConnect}
          onNodesChange={onNodesChange}
          onEdgesChange={onEdgesChange}
          fitView
          fitViewOptions={{ padding: 0.2 }}
          snapToGrid={true}
          snapGrid={[10, 10]}
          onInit={setRfInstance}
          onEdgeClick={(e, edge) => {
            setCurrentEdge(edge);

            setCurrentNode(null);
          }}
          onNodeClick={(e, node) => {
            setCurrentNode(node);
            setCurrentEdge(null);
          }}
        >
          <Background />
          <Controls />
        </ReactFlow>
      </Box>
```

```
<Box flex={"20%"} maxH={"100%"} h={"100%"} overflow={"auto"}>
  <Stack gap={6}>
    <Heading textAlign="center" size={"xl"}>
      {currentEdge !== null
        ? "Verbindung"
        : currentNode !== null
        ? "Knoten"
        : ""}
    </Heading>
    <ButtonGroup w={"100%"}>
      <Button
        w={"100%"}
        variant={"brand"}
        leftIcon={<FaPlus />}
        onClick={() => {
          const randomNumber = (min: number, max: number) => {
            return Math.floor(Math.random() * (max - min + 1)) + min;
          };

          setNodes((nodes) => [
            ...nodes,
            {
              id: `${nodes.length + 1}`,
              data: { label: "Neuer Knoten" },
              position: {
                x: randomNumber(0, 400),
                y: randomNumber(0, 400),
              },
              style: {
                background: "#4422FF",
                color: "white",
              },
              deletable: false,
            },
          ]);
        }}
      >
        Neuer Knoten
      </Button>
    </ButtonGroup>

    {currentEdge !== null &&
      lastEdgeId === "__rendered_" + currentEdge.id && (
        <Stack gap={5}>
          <Divider />
          <Flex w={"100%"} justifyContent={"space-between"}>
            <Heading size={"lg"}>ID</Heading>
            <Heading size={"lg"} fontWeight={"none"}>
              {currentEdge.id}
            </Heading>
          </Flex>
          <Flex w={"100%"} justifyContent={"space-between"}>
            <Heading size={"lg"} flex={"50%"}>
              Animiert
            </Heading>
            <Select
              flex={"50%"}
              onChange={(e) => {
                setEdges(
                  edges.map((edge) => {
```

```
          if (edge.id === currentEdge?.id) {
            edge.animated = e.target.value === "true";
          }
          return edge;
        })
      );
    }}
    fontSize={"xl"}
  >
    <option value={"true"} selected={currentEdge.animated}>
      Ja
    </option>
    <option value={"false"} selected={!currentEdge.animated}>
      Nein
    </option>
  </Select>
</Flex>
<Flex w={"100%"} justifyContent={"space-between"}>
  <Heading size={"lg"} flex={"50%"}>
    Text
  </Heading>
  <Input
    flex={"50%"}
    defaultValue={currentEdge.label as string}
    onChange={(e) => {
      setEdges(
        edges.map((edge) => {
          if (edge.id === currentEdge?.id) {
            edge.label = e.target.value;
          }
          return edge;
        })
      );
    }}
  />
</Flex>
<Flex w={"100%"} justifyContent={"space-between"}>
  <Heading size={"lg"} flex={"50%"}>
    Typ
  </Heading>
  <Select
    flex={"50%"}
    onChange={(e) => {
      setEdges(
        edges.map((edge) => {
          if (edge.id === currentEdge?.id) {
            edge.type = e.target.value as string;
          }
          return edge;
        })
      );
    }}
    fontSize={"xl"}
  >
    <option
      value={"default"}
      selected={currentEdge.type === "default"}
    >
      Standard
    </option>
```

```
            <option
              value={"smoothstep"}
              selected={currentEdge.type === "smoothstep"}
            >
              Schritt
            </option>
          </Select>
        </Flex>
        <ButtonGroup w={"100%"}>
          <Button
            w={"100%"}
            colorScheme={"red"}
            leftIcon={<FaTrash />}
            onClick={() => {
              setCurrentEdge(null);
              setEdges(
                edges.filter((edge) => edge.id !== currentEdge?.id)
              );
            }}
          >
            Löschen
          </Button>
        </ButtonGroup>
      </Stack>
    )}
  {currentNode !== null &&
    lastNodeId === "__rendered_" + currentNode.id && (
      <Stack gap={5}>
        <Divider />
        <Flex w={"100%"} justifyContent={"space-between"}>
          <Heading size={"lg"}>ID</Heading>
          <Heading size={"lg"} fontWeight={"none"}>
            {currentNode.id}
          </Heading>
        </Flex>
        <Flex w={"100%"} justifyContent={"space-between"}>
          <Heading size={"lg"} flex={"50%"}>
            Text
          </Heading>
          <Input
            fontSize={"xl"}
            flex={"50%"}
            defaultValue={currentNode.data.label as string}
            onChange={(e) => {
              setNodes(
                nodes.map((node) => {
                  if (node.id === currentNode?.id) {
                    node.data.label = e.target.value;
                  }
                  return node;
                })
              );
            }}
          />
        </Flex>
        <Flex w={"100%"} justifyContent={"space-between"}>
          <Heading size={"lg"} flex={"50%"}>
            Typ
          </Heading>
          <Select
```

```
    flex={"50%"}
    onChange={(e) => {
      setNodes(
        nodes.map((node) => {
          if (node.id === currentNode?.id) {
            node.type = e.target.value as string;
          }
          return node;
        })
      );
    }}
    fontSize={"xl"}
  >
    <option
      value={"default"}
      selected={currentNode.type === "default"}
    >
      Standard
    </option>
    <option
      value={"input"}
      selected={currentNode.type === "input"}
    >
      Eingang
    </option>
    <option
      value={"output"}
      selected={currentNode.type === "output"}
    >
      Ausgang
    </option>
  </Select>
</Flex>
<Flex w={"100%"} justifyContent={"space-between"}>
  <Heading size={"lg"} flex={"50%"}>
    Farbe
  </Heading>
  <Input
    flex={"50%"}
    type={"color"}
    defaultValue={
      // @ts-ignore
      currentNode.style.background as string
    }
    onChange={(e) => {
      setNodes(
        nodes.map((node) => {
          if (node.id === currentNode?.id) {
            // @ts-ignore
            node.style.background = e.target.value;
          }
          return node;
        })
      );
    }}
  />
</Flex>

<ButtonGroup w={"100%"}>
  <Button
```

```
                w={"100%"}
                colorScheme={"red"}
                onClick={() => {
                  setCurrentNode(null);
                  setNodes(
                    nodes.filter((node) => node.id !== currentNode?.id)
                  );
                }}
              >
                Löschen
              </Button>
            </ButtonGroup>
          </Stack>
        )}
      </Stack>
    </Box>
  </Flex>
 </>
 );
}
```

## apps/web/src/pages/pricing.tsx

```
/**
 * apps/web/src/pages/pricing.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 18.06.23
 *
 */
import {
  chakra,
  Box,
  Flex,
  Heading,
  Stack,
  Text,
  Button,
  HStack,
  Accordion,
  AccordionItem,
  AccordionButton,
  AccordionIcon,
  AccordionPanel,
} from "@chakra-ui/react";
import { BiCheck, BiCrown, BiX } from "react-icons/bi";
import { useLoggedIn } from "@codeup/hooks";
import React from "react";
import { Card, CardHeader } from "@chakra-ui/card";
import { FaCrown } from "react-icons/fa";
import REST from "@codeup/rest/dist";
import AccountManager from "@codeup/util/dist/AccountManager";
import PopupManager from "../global/PopupManager";
import { CHeading } from "@codeup/components/global/CHeading";

export default function Pricing(props: { ENABLE_PAYMENTS: boolean }) {
  const { userInfo, loggedIn } = useLoggedIn();

  return (
```

```
<>
  <Flex
    w={"100%"}
    minH={"100vh"}
    h={"fit-content"}
    alignItems={"center"}
    direction={"column"}
    p={8}
    bg={"#000"}
  >
    <Box>
      <Heading
        as={"h1"}
        size={"2xl"}
        fontWeight={900}
        letterSpacing={-2}
        fontFamily={"Jetbrains Mono ExtraBold"}
        color={"brand.500"}
      >
        Preisgestaltung
      </Heading>
    </Box>
    <Box w={["100%", "60%"]}>
      <Flex
        direction={["column", "column", "row"]}
        justifyContent={"center"}
        mt={8}
        gap={16}
      >
        <Stack
          bg={"black"}
          p={6}
          rounded={"xl"}
          gap={8}
          w={"100%"}
          shadow={"xl"}
        >
          <CHeading size={"xl"}>Standard</CHeading>
          <Text fontSize={"2xl"}>Gratis</Text>
          {loggedIn ? (
            <>
              <Button colorScheme={"gray"} disabled>
                Bereits aktiv
              </Button>
            </>
          ) : (
            <>
              <Button
                colorScheme={"gray"}
                onClick={() => {
                  window.dispatchEvent(
                    new CustomEvent("codeup::open_login")
                  );
                }}
              >
                Loslegen
              </Button>
            </>
          )}
          <Stack gap={4}>
```

```
<Accordion defaultIndex={[0]} allowMultiple allowToggle>
  <AccordionItem>
    <h2>
      <AccordionButton>
        <Box as="span" flex="1" textAlign="left">
          Kurse und Zertifikate
        </Box>
        <AccordionIcon />
      </AccordionButton>
    </h2>
    <AccordionPanel pb={4}>
      <FeatureList
        features={[
          {
            av: true,
            t: "Zugriff auf alle kostenlosen  Kurse",
          },
          {
            av: true,
            t: "Zugriff auf alle kostenlosen Zertifikate",
          },
        ]}
      />
    </AccordionPanel>
  </AccordionItem>
  <AccordionItem>
    <h2>
      <AccordionButton>
        <Box as="span" flex="1" textAlign="left">
          Forum
        </Box>
        <AccordionIcon />
      </AccordionButton>
    </h2>
    <AccordionPanel pb={4}>
      <FeatureList
        features={[
          {
            av: true,
            t: "Veröffentlichen von Artikeln",
          },
          {
            av: true,
            t: "Hochladen von Medien",
          },
          {
            av: true,
            t: "Zugriff auf Direktnachrichten",
          },
          {
            av: true,
            t: "Anzeigen von Zertifikaten",
          },
          {
            av: false,
            t: "Verändern der Biographie",
          },
          {
            av: false,
            t: "Verändern des Anzeigenamens",
```

```
        },
        {
          av: false,
          t: "Verändern des Webseite und des Ortes",
        },
      ]}
    />
  </AccordionPanel>
</AccordionItem>
<AccordionItem>
  <h2>
    <AccordionButton>
      <Box as="span" flex="1" textAlign="left">
        Editor
      </Box>
      <AccordionIcon />
    </AccordionButton>
  </h2>
  <AccordionPanel pb={4}>
    <FeatureList
      features={[
        {
          av: true,
          t: "Erstellen von drei Projekten",
        },
        {
          av: true,
          t: "5 Seiten pro Projekt",
        },
        {
          av: true,
          t: "1 Serverseitige Funktion pro Projekt",
        },
        {
          av: true,
          t: "Hochladen von ∞ statischen Dateien",
        },
        {
          av: false,
          t: "Export des Projekt",
        },
        {
          av: false,
          t: "Zusammenarbeit mit anderen",
        },
      ]}
    />
  </AccordionPanel>
</AccordionItem>
<AccordionItem>
  <h2>
    <AccordionButton>
      <Box as="span" flex="1" textAlign="left">
        Weitere Funktionen
      </Box>
      <AccordionIcon />
    </AccordionButton>
  </h2>
  <AccordionPanel pb={4}>
    <FeatureList
```

```
            features={[
              {
                av: true,
                t: "Zugriff auf das Discovery",
              },
              {
                av: true,
                t: "Zugriff auf alle Projektideen",
              },
              {
                av: true,
                t: "Erstellen von einer Roadmap",
              },
              {
                av: true,
                t: "Erstellen von zwei Kids Projekten",
              },
              {
                av: true,
                t: "Erstellen von drei Code-Snippets",
              },
              {
                av: true,
                t: "Erstellen von zwei Flows",
              },
              {
                av: true,
                t: "Erstellen von drei ToDo-Listen",
              },
              {
                av: true,
                t: "10 Einträge pro ToDo-Liste",
              },
            ]}
          />
        </AccordionPanel>
      </AccordionItem>
    </Accordion>
  </Stack>
</Stack>
<Stack bg={"black"} p={6} rounded={"xl"} gap={8} w={"100%"}>
  <CHeading size={"xl"}>CodeUp PRO</CHeading>
  <Text fontSize={"2xl"}>
    12 € <chakra.sub>/Monat</chakra.sub>
  </Text>
  {userInfo.subscription === "PRO" ? (
    <>
      <Button variant={"brand"} leftIcon={<FaCrown />} disabled>
        Bereits aktiv
      </Button>
    </>
  ) : (
    <>
      <Button
        variant={"brand"}
        onClick={async () => {
          if (!props.ENABLE_PAYMENTS) {
            await PopupManager.alertAsync({
              title: "Deaktiviert",
              description:
```

```
                "Zum aktuellen Zeitpunkt sind keine Zahlungen möglich. Du kannst alle Funktionen des CodeUp PRO Pakets nutzen.",
              });
              return;
            }

            if (loggedIn) {
              const res = await REST.Payments.createCheckoutLink(
                AccountManager.getActiveToken()
              );

              if (res.status === 200) {
                window.location.href = res.payload.url;
              } else {
                PopupManager.alert({
                  title: "Fehler",
                  description:
                    "Es ist ein Fehler aufgetreten. Bitte versuche es später erneut: " +
                    res.payload.error,
                });
              }
            } else {
              window.dispatchEvent(
                new CustomEvent("codeup::open_login")
              );
            }
          }}
          leftIcon={<FaCrown />}
        >
          Aktivieren
        </Button>
      </>
    )}
    <Stack gap={4}>
      <Accordion defaultIndex={[0]} allowMultiple allowToggle>
        <AccordionItem>
          <h2>
            <AccordionButton>
              <Box as="span" flex="1" textAlign="left">
                Kurse und Zertifikate
              </Box>
              <AccordionIcon />
            </AccordionButton>
          </h2>
          <AccordionPanel pb={4}>
            <FeatureList
              features={[
                {
                  av: true,
                  t: "Zugriff auf alle Kurse",
                },
                {
                  av: true,
                  t: "Zugriff auf alle Zertifikate",
                },
              ]}
            />
          </AccordionPanel>
        </AccordionItem>
        <AccordionItem>
```

```
  <h2>
    <AccordionButton>
      <Box as="span" flex="1" textAlign="left">
        Forum
      </Box>
      <AccordionIcon />
    </AccordionButton>
  </h2>
  <AccordionPanel pb={4}>
    <FeatureList
      features={[
        {
          av: true,
          t: "Veröffentlichen von Artikeln",
        },
        {
          av: true,
          t: "Hochladen von Medien",
        },
        {
          av: true,
          t: "Zugriff auf Direktnachrichten",
        },
        {
          av: true,
          t: "Anzeigen von Zertifikaten",
        },
        {
          av: true,
          t: "Verändern der Biographie",
        },
        {
          av: true,
          t: "Verändern des Anzeigenamens",
        },
        {
          av: true,
          t: "Verändern des Webseite und des Ortes",
        },
      ]}
    />
  </AccordionPanel>
</AccordionItem>
<AccordionItem>
  <h2>
    <AccordionButton>
      <Box as="span" flex="1" textAlign="left">
        Editor
      </Box>
      <AccordionIcon />
    </AccordionButton>
  </h2>
  <AccordionPanel pb={4}>
    <FeatureList
      features={[
        {
          av: true,
          t: "Erstellen von ∞ Projekten",
        },
        {
```

```
          av: true,
          t: "∞ Seiten pro Projekt",
        },
        {
          av: true,
          t: "∞ Serverseitige Funktion",
        },
        {
          av: true,
          t: "Hochladen von ∞ statischen Dateien",
        },
        {
          av: true,
          t: "Export des Projekt",
        },
        {
          av: true,
          t: "Zusammenarbeit mit anderen",
        },
      ]}
    />
  </AccordionPanel>
</AccordionItem>
<AccordionItem>
  <h2>
    <AccordionButton>
      <Box as="span" flex="1" textAlign="left">
        Weitere Funktionen
      </Box>
      <AccordionIcon />
    </AccordionButton>
  </h2>
  <AccordionPanel pb={4}>
    <FeatureList
      features={[
        {
          av: true,
          t: "Zugriff auf das Discovery",
        },
        {
          av: true,
          t: "Zugriff auf alle Projektideen",
        },
        {
          av: true,
          t: "Erstellen von ∞ Roadmap",
        },
        {
          av: true,
          t: "Erstellen von ∞ Kids Projekten",
        },
        {
          av: true,
          t: "Erstellen von ∞ Code-Snippets",
        },
        {
          av: true,
          t: "Erstellen von ∞ Flows",
        },
        {
```

```
                    av: true,
                    t: "Erstellen von ∞ ToDo-Listen",
                  },
                  {
                    av: true,
                    t: "∞ Einträge pro ToDo-Liste",
                  },
                ]}
              />
            </AccordionPanel>
          </AccordionItem>
        </Accordion>
      </Stack>
    </Stack>
  </Flex>
 </Box>
</Flex>
</>
);
}

function FeatureList(props: { features: { av: boolean; t: string }[] }) {
 return (
  <>
   {props.features.map((item) => (
    <HStack fontSize={"xl"}>
     <Text color={item.av ? "brand.500" : "red.400"} fontSize={"4xl"}>
      {item.av ? <BiCheck /> : <BiX />}
     </Text>
     <Text color={"white"}>{item.t}</Text>
    </HStack>
   ))}
  </>
 );
}

export const getServerSideProps = async () => {
 return {
  props: {
   DISABLE_PAYMENTS: process.env.DISABLE_PAYMENTS,
  },
 };
};
```

## apps/web/src/pages/projects/index.tsx

```
/**
 * apps/web/src/pages/projects/index.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2024 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 22.05.2024
 *
 */

import * as React from "react";
import { Box, chakra, Flex, Grid, Image, Text } from "@chakra-ui/react";
import { useUserLoggedInRedirect } from "@codeup/hooks";
import { IProject } from "../../types/IProject";
import REST from "@codeup/rest/dist";
```

```
import AccountManager from "@codeup/util/dist/AccountManager";
import PopupManager from "../../global/PopupManager";
import GridLayoutV2 from "@codeup/components/dist/global/GridLayoutV2";
import {
  FaClock,
  FaCode,
  FaDownload,
  FaHtml5,
  FaInfo,
  FaPlus,
  FaTrash,
} from "react-icons/fa";
import { CHeading } from "@codeup/components/dist/global/CHeading";
import { useRouter } from "next/router";

export default function Projects() {
  useUserLoggedInRedirect({
    allowAnonymous: false,
  });

  const router = useRouter();

  const [projects, setProjects] = React.useState<IProject[]>([]);

  const reload = async () => {
    const res = await REST.Projects.myProjects(AccountManager.getActiveToken());

    if (res.status === 200) {
      setProjects(res.payload.data);
    } else {
      await PopupManager.alertAsync({
        title: "Fehler",
        description: res.payload.error,
      });
    }
  };

  React.useEffect(() => {
    reload();
  }, []);

  const project_types: {
    name: string;
    description: string;
    icon: any;
    identifier: string;
  }[] = [
    {
      name: "Statisch",
      description:
        "Ein statisches Projekt (HTML, CSS, JavaScript, keine Server-Logik)",
      icon: <FaHtml5 />,
      identifier: "static",
    },
  ];

  return (
    <>
      <Box minH={"100vh"} h={"fit-content"} bg={"#000"}>
        <GridLayoutV2
```

```
pageTitle={"Projekte"}
search={{
  show: true,
  fields: true,
}}
topButtons={[
  {
    text: "Neu",
    leftIcon: <FaPlus />,
    onClick: async () => {
      const name = await PopupManager.promptAsync({
        title: "Neues Projekt",
        helperText: "Bitte geben Sie den Namen des Projekts ein.",
      });

      if (!name) return;

      const x = await new Promise(async (resolve) => {
        await PopupManager.alertAsync({
          title: "Projekt-Typ wählen",
          description: (
            <>
              <Text>
                Bitte wähle den Typ des Projektes aus. Du kannst
                diesen später nicht mehr ändern, also wähle weise.
              </Text>
              <Grid
                templateColumns={["repeat(1, 1fr)", "repeat(2, 1fr)"]}
                pt={6}
                gap={4}
              >
                {project_types.map((type) => (
                  <>
                    <Box
                      bg={"#000"}
                      rounded={"xl"}
                      cursor={"pointer"}
                      _hover={{
                        outline: "1px solid #F7DE1F",
                      }}
                      p={4}
                      onClick={async () => {
                        resolve(type.identifier);
                        PopupManager.removeCurrentPopup();
                      }}
                    >
                      <Flex
                        fontSize={"4xl"}
                        w={"100%"}
                        justifyContent={"center"}
                        color={"white"}
                        align={"center"}
                      >
                        {type.icon}
                      </Flex>
                      <Box mt={2} textAlign={"center"}>
                        <CHeading size={"md"}>{type.name}</CHeading>
                      </Box>
                      <Text
                        fontSize={"sm"}
```

```
                    color={"white"}
                    textAlign={"center"}
                    mt={2}
                  >
                    {type.description}
                  </Text>
                </Box>
              </>
            ))}
          </Grid>
        </>
      ),
    });
  });

  if (!x) return;

  const res = await REST.Projects.createProject({
    token: AccountManager.getActiveToken(),
    name,
    projectType: x as string,
    modifiers: ["private"],
  });

  await reload();
 },
 variant: "brand",
},
]}
entries={projects.map((p) => ({
 title: p.name,
 fields: [
  {
   icon: <FaCode />,
   text: "Vorlage: " + p.projectType,
  },
  {
   icon: <FaInfo />,
   text: "ID: " + p._id,
  },
  {
   icon: <FaClock />,
   text: "Erstellt: " + new Date(p.createdAt).toLocaleString(),
  },
 ],
 buttonRows: [
  [
   {
    text: "Öffnen",
    onClick: () => {
     router.push("/projects/" + p._id);
    },
    leftIcon: <FaCode />,
    variant: "brand",
   },
   {
    text: "Löschen",
    onClick: () => {
     alert("Not implemented");
    },
```

```
                leftIcon: <FaTrash />,
                colorScheme: "red",
              },
            ],
            [
              {
                text: "Exportieren",
                onClick: async () => {
                  window.open(
                    "/api/projects/" +
                      p._id +
                      "/download?token=" +
                      AccountManager.getActiveToken()
                  );
                },
                leftIcon: <FaDownload />,
                variant: "brand",
              },
            ],
          ],
        ]))}
      />
    </Box>
  </>
 );
}
```

## apps/web/src/pages/projects/[id]/editor/professional.tsx

```
/**
 * apps/web/src/pages/projects/[id]/editor/professional.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2024 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 09.06.2024
 *
 */
import * as React from "react";
import { useUserLoggedInRedirect } from "@codeup/hooks";
import { useRouter } from "next/router";
import { Box, Flex, Text } from "@chakra-ui/react";
import AppShell from "@codeup/components/dist/global/AppShell";
import { IProject } from "../../../../types/IProject";
import { useEffect } from "react";
import REST from "@codeup/rest/src";
import AccountManager from "@codeup/util/dist/AccountManager";
import PopupManager from "../../../../global/PopupManager";
import EditorProjectLayout from "@codeup/components/editorv2/EditorProjectLayout";
import { DirectoryNode, FileNode, WebContainer } from "@webcontainer/api";
import AuthenticateLoadingIndicator from
"@codeup/components/dist/admin/AuthenticateLoadingIndicator";
import { Stripe } from "stripe";
import Terminal, { ColorMode, TerminalOutput } from "react-terminal-ui";
import EditorFileExplorer from "@codeup/components/editorv2/EditorFileExplorer";
import { Panel, PanelGroup, PanelResizeHandle } from "react-resizable-panels";

export default function Professional() {
  useUserLoggedInRedirect({
    allowAnonymous: false,
  });
```

```
type entry = {
  name: string;
  type: string;
  children: entry[];
  content?: any;
};

const [project, setProject] = React.useState<IProject | null>(null);
const [fileTree, setFileTree] = React.useState<{
  [key: string]: FileNode | DirectoryNode;
}>({});
const [simpleFileTree, setSimpleFileTree] = React.useState<entry | null>(
  null
);

const [webContainer, setWebContainer] = React.useState<WebContainer | null>(
  null
);
const [isBootstrapping, setIsBootstrapping] = React.useState<boolean>(false);
const [terminal, setTerminal] = React.useState<any | null>(null);
const [terminalData, setTerminalData] = React.useState<string[]>([]);

const router = useRouter();

useEffect(() => {
  if (!router || !router.query || !router.query.id) return;

  REST.Projects.project(
    AccountManager.getActiveToken(),
    router.query.id as string
  ).then((res) => {
    if (res.status === 200) {
      setProject(res.payload.project);
      reloadFileTree();
    } else {
      PopupManager.alertAsync({
        title: "Fehler",
        description:
          "Das Projekt konnte nicht empfangen werden: " + res.payload.error,
      });
    }
  });
}, [router, router.query, router.query.id]);

const reloadFileTree = async () => {
  const res = await REST.Projects.GitOps.tree(
    AccountManager.getActiveToken(),
    router.query.id as string,
    "webcontainer"
  );

  if (res.status === 200) {
    setFileTree(res.payload.data);
    setSimpleFileTree({
      name: "root",
      type: "directory",
      children: res.payload.tree,
    });
    //await bootWebContainer(res.payload.data);
```

```
  } else {
    await PopupManager.alertAsync({
      title: "Fehler",
      description: "Das Dateisystem konnte nicht geladen werden",
    });
  }
};

const bootWebContainer = async (wcpaths: any) => {
  if (isBootstrapping) return;
  if (webContainer) return;
  setIsBootstrapping(true);
  const wc = await WebContainer.boot();

  await wc.mount(wcpaths);
  setWebContainer(wc);
  await installDeps(wc);
  setIsBootstrapping(false);
};

const installDeps = async (wc: WebContainer) => {
  if (!wc) return;
  console.log("Installing dependencies");

  const installProcess = await wc.spawn("npm", ["install"], {
    env: {
      NO_COLOR: true,
    },
  });

  installProcess.output.pipeTo(
    new WritableStream({
      write(data) {
        setTerminalData((prev) => [...prev, data]);
        console.log(data);
      },
    })
  );
};

return (
  <>
    <EditorProjectLayout project_id={router.query.id as string}>
      {isBootstrapping && (
        <>
          <AuthenticateLoadingIndicator text={"Starte WebContainer..."} />
        </>
      )}
      {webContainer && !isBootstrapping && <></>}
      {simpleFileTree && (
        <Box p={6}>
          <Flex bg={"gray.900"} rounded={"md"} p={2}>
            <PanelGroup
              autoSaveId={"editor-panel-group"}
              direction={"horizontal"}
              style={{
                gap: "0.75rem",
              }}
            >
              <Panel
```

```
  defaultSize={25}
  minSize={25}
  maxSize={70}
  style={{
    padding: "var(--chakra-space-2)",
    borderRadius: "var(--chakra-radii-md)",
  }}
>
  <EditorFileExplorer
    projectName={project!.name}
    fileTree={simpleFileTree}
    openFile={(path) => {}}
    deleteFile={async (path) => {
      if (
        path.startsWith(".git") ||
        path === "project.codeup.json"
      ) {
        await PopupManager.alertAsync({
          title: "Datei löschen",
          description: `Die Datei ${path} kann nicht gelöscht werden`,
        });
        return;
      }
      if (
        !(await PopupManager.confirmAsync({
          title: "Datei löschen",
          question: `Möchtest du die Datei ${path} wirklich löschen?`,
        }))
      )
        return;

      const res = await REST.Projects.Files.delete(
        AccountManager.getActiveToken(),
        router.query.id as string,
        path
      );

      if (res.status === 200) {
        await reloadFileTree();
      } else {
        await PopupManager.alertAsync({
          title: "Fehler",
          description: res.payload.error,
        });
      }
    }}
    createDirectory={async (path) => {
      const res = await REST.Projects.Files.createFolder(
        AccountManager.getActiveToken(),
        router.query.id as string,
        path
      );

      if (res.status === 200) {
        await reloadFileTree();
      } else {
        await PopupManager.alertAsync({
          title: "Fehler",
          description: res.payload.error,
        });
```

```
      }
    }}
    createFile={async (path) => {
      const res = await REST.Projects.Files.setFileContent(
        AccountManager.getActiveToken(),
        router.query.id as string,
        path,
        ""
      );

      if (res.status === 200) {
        await reloadFileTree();
      } else {
        await PopupManager.alertAsync({
          title: "Fehler",
          description: res.payload.error,
        });
      }
    }}
    renameFile={async (path, newName) => {
      if (
        path.startsWith(".git") ||
        path === "project.codeup.json"
      ) {
        await PopupManager.alertAsync({
          title: "Datei umbenennen",
          description: `Die Datei ${path} kann nicht umbenannt werden`,
        });
        return;
      }

      const res = await REST.Projects.Files.rename(
        AccountManager.getActiveToken(),
        router.query.id as string,
        path,
        newName
      );

      if (res.status === 200) {
        await reloadFileTree();
      } else {
        await PopupManager.alertAsync({
          title: "Fehler",
          description: res.payload.error,
        });
      }
    }}
  />
</Panel>
<PanelResizeHandle
  style={{
    cursor: "col-resize",
    width: "0.5rem",
    backgroundColor: "#2C2E33",
    border: "1px solid gray.800",
    borderRadius: "0.25rem",
  }}
/>
<Panel defaultSize={75} minSize={30} maxSize={100}>
  <Box bg={"#FFAAFF"} rounded={"md"}>
```

```
                Code
              </Box>
            </Panel>
          </PanelGroup>
        </Flex>
      </Box>
    )}
  </EditorProjectLayout>
 </>
 );
}
```

## apps/web/src/pages/projects/[id]/flows/[flow_id].tsx

```
/**
 * apps/web/src/pages/projects/[id]/flows/[flow_id].tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2024 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 01.06.2024
 *
 */

import * as React from "react";
import { useUserLoggedInRedirect } from "@codeup/hooks";
import { IProject } from "../../../../types/IProject";
import { useRouter } from "next/router";
import { useCallback, useEffect, useMemo } from "react";
import REST from "@codeup/rest/src";
import AccountManager from "@codeup/util/dist/AccountManager";
import PopupManager from "../../../../global/PopupManager";
import EditorProjectLayout from "@codeup/components/dist/editorv2/EditorProjectLayout";
import {
  Box,
  Button,
  ButtonGroup,
  Divider,
  Flex,
  Heading,
  Input,
  Select,
  Stack,
  Text,
  useToast,
} from "@chakra-ui/react";
import ReactFlow, {
  addEdge,
  applyEdgeChanges,
  applyNodeChanges,
  Background,
  Connection,
  Controls,
  Edge,
  EdgeChange,
  Node,
  NodeChange,
  ReactFlowInstance,
} from "reactflow";
import { FaPlus, FaTrash } from "react-icons/fa";
import IssueNode from "@codeup/components/projectflows/IssueNode";
```

```
import { VscIssues } from "react-icons/vsc";
import { CHeading } from "@codeup/components/dist/global/CHeading";

export default function ProjectFlowViewer() {
  useUserLoggedInRedirect({
    allowAnonymous: false,
  });

  const nodeTypes = useMemo(
    () => ({
      issue: IssueNode,
    }),
    []
  );

  const [project, setProject] = React.useState<IProject | null>(null);
  const [flow, setFlow] = React.useState<any>();

  const router = useRouter();

  useEffect(() => {
    if (!router || !router.query || !router.query.id) return;

    REST.Projects.project(
      AccountManager.getActiveToken(),
      router.query.id as string
    ).then(async (res) => {
      if (res.status === 200) {
        setProject(res.payload.project);
        reloadFlow();
      } else {
        PopupManager.alertAsync({
          title: "Fehler",
          description:
            "Das Projekt konnte nicht empfangen werden: " + res.payload.error,
        });
      }
    });
  }, [router, router.query, router.query.id]);

  const reloadFlow = async () => {
    const res = await REST.Projects.getFlows(
      AccountManager.getActiveToken(),
      router.query.id as string,
      router.query.flow_id as string
    );

    if (res.status !== 200) {
      await PopupManager.alertAsync({
        title: "Fehler",
        description:
          "Die Flussdiagramme konnten nicht geladen werden: " +
          res.payload.error,
      });
      return;
    }

    let nodesX = [];
    let edgesX = [];
```

```
  let x: any = {};

  if (res.payload.flow.saved && res.payload.flow.saved !== "") {
    x = JSON.parse(res.payload.flow.saved);
  }

  if (!x.nodes) {
    x.nodes = [];
  }

  if (!x.edges) {
    x.edges = [];
  }

  nodesX = x.nodes;
  edgesX = x.edges;

  setNodes(nodesX);
  setEdges(edgesX);

  setFlow(res.payload.flow);
};

const [nodes, setNodes] = React.useState<Node[]>([]);
const [edges, setEdges] = React.useState<Edge[]>([]);

const onNodesChange = useCallback(
  (changes: NodeChange[]) =>
    setNodes((nds) => applyNodeChanges(changes, nds)),
  [setNodes]
);
const onEdgesChange = useCallback(
  (changes: EdgeChange[]) =>
    setEdges((eds) => applyEdgeChanges(changes, eds)),
  [setEdges]
);
const onConnect = useCallback(
  (connection: Connection) => setEdges((eds) => addEdge(connection, eds)),
  [setEdges]
);

const [currentEdge, setCurrentEdge] = React.useState<Edge | null>(null);
const [currentNode, setCurrentNode] = React.useState<Node | null>(null);

const [lastEdgeId, setLastEdgeId] = React.useState<string>("");
const [lastNodeId, setLastNodeId] = React.useState<string>("");

const [rfInstance, setRfInstance] = React.useState<ReactFlowInstance | null>(
  null
);

const toast = useToast();

useEffect(() => {
  setLastEdgeId("__rendered_" + currentEdge?.id ?? "");
}, [currentEdge]);

useEffect(() => {
  setLastNodeId("__rendered_" + currentNode?.id ?? "");
}, [currentNode]);
```

```
useEffect(() => {
  if (rfInstance) {
    const saved = rfInstance.toObject();

    REST.Projects.updateFlow(
      AccountManager.getActiveToken(),
      router.query.id as string,
      router.query.flow_id as string,
      flow.name,
      JSON.stringify(saved)
    );
  }
}, [edges, nodes]);

return (
  <>
    <EditorProjectLayout project_id={router.query.id as string}>
      <Flex
        maxH={"95vh"}
        h={"95vh"}
        p={"4rem"}
        paddingBottom={"8rem"}
        paddingTop={"0"}
        alignItems={"center"}
        justifyContent={"center"}
        pt={"2rem"}
        bg={"#000"}
        gap={6}
      >
        <Box h={"100%"} w={"100%"} flex={"80%"}>
          <ReactFlow
            nodes={nodes}
            edges={edges}
            onConnect={onConnect}
            onNodesChange={onNodesChange}
            onEdgesChange={onEdgesChange}
            fitView
            fitViewOptions={{ padding: 0.2 }}
            snapToGrid={true}
            snapGrid={[10, 10]}
            onInit={setRfInstance}
            onEdgeClick={(e, edge) => {
              setCurrentEdge(edge);

              setCurrentNode(null);
            }}
            onNodeClick={(e, node) => {
              if (node.type !== "issue") {
                setCurrentNode(node);
                setCurrentEdge(null);
              }
            }}
            nodeTypes={nodeTypes}
          >
            <Background />
            <Controls />
          </ReactFlow>
        </Box>
        <Box flex={"20%"} maxH={"100%"} h={"100%"} overflow={"auto"}>
```

```
<Stack gap={6}>
  <Heading textAlign="center" size={"xl"}>
    {currentEdge !== null
      ? "Verbindung"
      : currentNode !== null
      ? "Knoten"
      : ""}
  </Heading>
  <ButtonGroup w={"100%"}>
    <Button
      w={"100%"}
      variant={"brand"}
      leftIcon={<FaPlus />}
      onClick={() => {
        const randomNumber = (min: number, max: number) => {
          return Math.floor(Math.random() * (max - min + 1)) + min;
        };

        setNodes((nodes) => [
          ...nodes,
          {
            id: `${nodes.length + 1}`,
            data: { label: "Neuer Knoten" },
            position: {
              x: randomNumber(0, 400),
              y: randomNumber(0, 400),
            },
            style: {
              background: "#141517",
              color: "#F7DE1F",
              borderRadius: "0.75rem",
            },
            deletable: false,
          },
        ]);
      }}
    >
      Neuer Knoten
    </Button>
  </ButtonGroup>
  <Button
    variant={"brand"}
    leftIcon={<VscIssues />}
    onClick={async () => {
      const issues = (
        await REST.Projects.getIssues(
          AccountManager.getActiveToken(),
          router.query.id as string
        )
      ).payload.issues;

      await PopupManager.alertAsync({
        title: "Problem hinzufügen",
        description: (
          <>
            <Stack gap={4}>
              {issues.map((issue: any) => (
                <>
                  <Box
                    p={4}
```

```
                rounded={"md"}
                bg={"gray.900"}
                cursor={"pointer"}
                onClick={() => {
                  const randomNumber = (
                    min: number,
                    max: number
                  ) => {
                    return (
                      Math.floor(
                        Math.random() * (max - min + 1)
                      ) + min
                    );
                  };

                  setNodes((nodes) => [
                    ...nodes,
                    {
                      id: `${nodes.length + 1}`,
                      data: [project?._id, issue._id],
                      position: {
                        x: randomNumber(0, 400),
                        y: randomNumber(0, 400),
                      },
                      style: {
                        background: "transparent",
                        color: "white",
                      },
                      type: "issue",
                    },
                  ]);

                  PopupManager.removeCurrentPopup();
                }}
              >
                <CHeading size={"lg"}>{issue.title}</CHeading>
                <Text>
                  {issue.status === "open"
                    ? "Offen"
                    : "Geschlossen"}
                </Text>
                <Text>{issue.description}</Text>
              </Box>
            </>
          ))}
        </Stack>
      </>
    ),
  });
 }}
>
  Problem hinzufügen
</Button>

{currentEdge !== null &&
  lastEdgeId === "__rendered_" + currentEdge.id && (
    <Stack gap={5}>
      <Divider />
      <Flex w={"100%"} justifyContent={"space-between"}>
        <Heading size={"lg"}>ID</Heading>
```

```
  <Heading size={"lg"} fontWeight={"none"}>
    {currentEdge.id}
  </Heading>
</Flex>
<Flex w={"100%"} justifyContent={"space-between"}>
  <Heading size={"lg"} flex={"50%"}>
    Animiert
  </Heading>
  <Select
    flex={"50%"}
    onChange={(e) => {
      setEdges(
        edges.map((edge) => {
          if (edge.id === currentEdge?.id) {
            edge.animated = e.target.value === "true";
          }
          return edge;
        })
      );
    }}
    fontSize={"xl"}
  >
    <option value={"true"} selected={currentEdge.animated}>
      Ja
    </option>
    <option
      value={"false"}
      selected={!currentEdge.animated}
    >
      Nein
    </option>
  </Select>
</Flex>
<Flex w={"100%"} justifyContent={"space-between"}>
  <Heading size={"lg"} flex={"50%"}>
    Text
  </Heading>
  <Input
    flex={"50%"}
    defaultValue={currentEdge.label as string}
    onChange={(e) => {
      setEdges(
        edges.map((edge) => {
          if (edge.id === currentEdge?.id) {
            edge.label = e.target.value;
          }
          return edge;
        })
      );
    }}
  />
</Flex>
<Flex w={"100%"} justifyContent={"space-between"}>
  <Heading size={"lg"} flex={"50%"}>
    Typ
  </Heading>
  <Select
    flex={"50%"}
    onChange={(e) => {
      setEdges(
```

```
            edges.map((edge) => {
              if (edge.id === currentEdge?.id) {
                edge.type = e.target.value as string;
              }
              return edge;
            })
          );
        }}
        fontSize={"xl"}
      >
        <option
          value={"default"}
          selected={currentEdge.type === "default"}
        >
          Standard
        </option>
        <option
          value={"smoothstep"}
          selected={currentEdge.type === "smoothstep"}
        >
          Schritt
        </option>
      </Select>
    </Flex>
    <ButtonGroup w={"100%"}>
      <Button
        w={"100%"}
        colorScheme={"red"}
        leftIcon={<FaTrash />}
        onClick={() => {
          setCurrentEdge(null);
          setEdges(
            edges.filter((edge) => edge.id !== currentEdge?.id)
          );
        }}
      >
        Löschen
      </Button>
    </ButtonGroup>
  </Stack>
 )}
{currentNode !== null &&
  lastNodeId === "__rendered_" + currentNode.id && (
    <Stack gap={5}>
      <Divider />
      <Flex w={"100%"} justifyContent={"space-between"}>
        <Heading size={"lg"}>ID</Heading>
        <Heading size={"lg"} fontWeight={"none"}>
          {currentNode.id}
        </Heading>
      </Flex>
      <Flex w={"100%"} justifyContent={"space-between"}>
        <Heading size={"lg"} flex={"50%"}>
          Text
        </Heading>
        <Input
          fontSize={"xl"}
          flex={"50%"}
          defaultValue={currentNode.data.label as string}
          onChange={(e) => {
```

```
      setNodes(
        nodes.map((node) => {
          if (node.id === currentNode?.id) {
            node.data.label = e.target.value;
          }
          return node;
        })
      );
    }}
  />
</Flex>
<Flex w={"100%"} justifyContent={"space-between"}>
  <Heading size={"lg"} flex={"50%"}>
    Typ
  </Heading>
  <Select
    flex={"50%"}
    onChange={(e) => {
      setNodes(
        nodes.map((node) => {
          if (node.id === currentNode?.id) {
            node.type = e.target.value as string;
          }
          return node;
        })
      );
    }}
    fontSize={"xl"}
  >
    <option
      value={"default"}
      selected={currentNode.type === "default"}
    >
      Standard
    </option>
    <option
      value={"input"}
      selected={currentNode.type === "input"}
    >
      Eingang
    </option>
    <option
      value={"output"}
      selected={currentNode.type === "output"}
    >
      Ausgang
    </option>
  </Select>
</Flex>
<Flex w={"100%"} justifyContent={"space-between"}>
  <Heading size={"lg"} flex={"50%"}>
    Farbe
  </Heading>
  <Input
    flex={"50%"}
    type={"color"}
    defaultValue={
      // @ts-ignore
      currentNode.style.background as string
    }
```

```
                onChange={(e) => {
                  setNodes(
                    nodes.map((node) => {
                      if (node.id === currentNode?.id) {
                        // @ts-ignore
                        node.style.background = e.target.value;
                      }
                      return node;
                    })
                  );
                }}
              />
            </Flex>

            <ButtonGroup w={"100%"}>
              <Button
                w={"100%"}
                colorScheme={"red"}
                onClick={() => {
                  setCurrentNode(null);
                  setNodes(
                    nodes.filter((node) => node.id !== currentNode?.id)
                  );
                }}
              >
                Löschen
              </Button>
            </ButtonGroup>
          </Stack>
        )}
      </Stack>
    </Box>
  </Flex>
 </EditorProjectLayout>
 </>
 );
}
```

## apps/web/src/pages/projects/[id]/flows.tsx

```
/**
 * apps/web/src/pages/projects/[id]/flows.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2024 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 01.06.2024
 *
 */

import * as React from "react";
import { useUserLoggedInRedirect } from "@codeup/hooks";
import { IProject } from "../../../types/IProject";
import { IWikiPage } from "../../../types/IWikiPage";
import { useRouter } from "next/router";
import { useEffect } from "react";
import REST from "@codeup/rest/src";
import AccountManager from "@codeup/util/dist/AccountManager";
import PopupManager from "../../../global/PopupManager";
import EditorProjectLayout from "@codeup/components/dist/editorv2/EditorProjectLayout";
import { Box } from "@chakra-ui/react";
```

```
import GridLayoutV2 from "@codeup/components/dist/global/GridLayoutV2";
import { FaClock, FaEye, FaPen, FaPlus, FaTrash } from "react-icons/fa";
import byteSize from "byte-size";
import { FaHardDrive } from "react-icons/fa6";

export default function Flows() {
  useUserLoggedInRedirect({
    allowAnonymous: false,
  });

  const [project, setProject] = React.useState<IProject | null>(null);
  const [flows, setFlows] = React.useState<any[]>([]);

  const router = useRouter();

  useEffect(() => {
    if (!router || !router.query || !router.query.id) return;

    REST.Projects.project(
      AccountManager.getActiveToken(),
      router.query.id as string
    ).then(async (res) => {
      if (res.status === 200) {
        setProject(res.payload.project);
        reloadFlows();
      } else {
        PopupManager.alertAsync({
          title: "Fehler",
          description:
            "Das Projekt konnte nicht empfangen werden: " + res.payload.error,
        });
      }
    });
  }, [router, router.query, router.query.id]);

  const reloadFlows = async () => {
    const res = await REST.Projects.getFlows(
      AccountManager.getActiveToken(),
      router.query.id as string
    );

    if (res.status !== 200) {
      await PopupManager.alertAsync({
        title: "Fehler",
        description:
          "Die Flussdiagramme konnten nicht geladen werden: " +
          res.payload.error,
      });
      return;
    }

    setFlows(res.payload.flows);
  };

  return (
    <>
      <EditorProjectLayout project_id={router.query.id as string}>
        <Box p={2}>
          <GridLayoutV2
            pageTitle={"Flussdiagramme"}
```

```
search={{
  show: true,
  fields: true,
}}
topButtons={[
  {
    text: "Neues Diagramm",
    leftIcon: <FaPlus />,
    variant: "brand",
    onClick: async () => {
      const name = await PopupManager.promptAsync({
        title: "Neues Flussdiagramm",
        helperText:
          "Bitte gib den Namen des neuen Flussdiagramms ein.",
      });

      if (!name) return;

      const res = await REST.Projects.createFlow(
        AccountManager.getActiveToken(),
        router.query.id as string,
        name
      );

      if (res.status !== 200) {
        await PopupManager.alertAsync({
          title: "Fehler",
          description:
            "Das Flussdiagramm konnte nicht erstellt werden: " +
            res.payload.error,
        });
        return;
      }

      await reloadFlows();
    },
  },
]}
entries={flows.map((flow) => {
  return {
    title: flow.name,
    buttonRows: [
      [
        {
          text: "Öffnen",
          leftIcon: <FaEye />,
          variant: "brand",
          onClick: () => {
            router.push(
              "/projects/" + router.query.id + "/flows/" + flow._id
            );
          },
        },
        {
          text: "Löschen",
          leftIcon: <FaTrash />,
          colorScheme: "red",
          onClick: async () => {
            if (
              !(await PopupManager.confirmAsync({
```

```
                title: "Flussdiagramm löschen",
                question:
                  "Willst du dieses Flussdiagramm wirklich löschen? Dieser Vorgang kann nicht
rückgängig gemacht werden!",
              }))
            )
              return;

            const res = await REST.Projects.deleteFlow(
              AccountManager.getActiveToken(),
              router.query.id as string,
              flow._id
            );

            if (res.status !== 200) {
              await PopupManager.alertAsync({
                title: "Fehler",
                description:
                  "Das Flussdiagramm konnte nicht gelöscht werden: " +
                  res.payload.error,
              });
              return;
            }

            await reloadFlows();
          },
        },
      ],
      [
        {
          text: "Umbenennen",
          variant: "brand",
          leftIcon: <FaPen />,
          onClick: async () => {
            const newName = await PopupManager.promptAsync({
              title: "Umbenennen",
              helperText: "Bitte gib den neuen Namen ein.",
            });

            if (!newName) return;
            const res = await REST.Projects.updateFlow(
              AccountManager.getActiveToken(),
              router.query.id as string,
              flow._id,
              newName,
              flow.saved
            );

            if (res.status !== 200) {
              await PopupManager.alertAsync({
                title: "Fehler",
                description:
                  "Das Flussdiagramm konnte nicht umbenannt werden: " +
                  res.payload.error,
              });
              return;
            }

            await reloadFlows();
          },
```

```
                },
              ],
            ],
            fields: [
              {
                text:
                  "Erstellt: " + new Date(flow.createdAt).toLocaleString(),
                icon: <FaClock />,
              },
              {
                text:
                  "Gespeicherte Daten: " + byteSize(flow.saved).toString(),
                icon: <FaHardDrive />,
              },
            ],
          };
        })}
      />
    </Box>
  </EditorProjectLayout>
 </>
 );
}
```

## apps/web/src/pages/projects/[id]/i18n.tsx

```
/**
 * apps/web/src/pages/projects/[id]/i18n.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2024 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 04.06.2024
 *
 */

import * as React from "react";
import { useUserLoggedInRedirect } from "@codeup/hooks";
import { IProject } from "../../../types/IProject";
import { IWikiPage } from "../../../types/IWikiPage";
import { useRouter } from "next/router";
import { useEffect } from "react";
import REST from "@codeup/rest/src";
import AccountManager from "@codeup/util/dist/AccountManager";
import PopupManager from "../../../global/PopupManager";
import EditorProjectLayout from "@codeup/components/dist/editorv2/EditorProjectLayout";
import {
  Box,
  chakra,
  Button,
  Text,
  Grid,
  Stack,
  HStack,
  ButtonGroup,
  IconButton,
  Table,
  Thead,
  Th,
  Tbody,
  Tr,
```

```
  Td,
  Input,
} from "@chakra-ui/react";
import { CHeading } from "@codeup/components/dist/global/CHeading";
import { FaTrash, FaHammer, FaLanguage } from "react-icons/fa";
import { translateOnlineV3 } from "../../../global/i18n/onlineTranslateV3";

export default function I18n() {
  useUserLoggedInRedirect({
    allowAnonymous: false,
  });

  const [project, setProject] = React.useState<IProject | null>(null);
  const [i18n, setI18n] = React.useState<any | null>(null);

  const router = useRouter();

  useEffect(() => {
    if (!router || !router.query || !router.query.id) return;

    REST.Projects.project(
      AccountManager.getActiveToken(),
      router.query.id as string
    ).then(async (res) => {
      if (res.status === 200) {
        setProject(res.payload.project);
        reloadI18n();
      } else {
        PopupManager.alertAsync({
          title: "Fehler",
          description:
            "Das Projekt konnte nicht empfangen werden: " + res.payload.error,
        });
      }
    });
  }, [router, router.query, router.query.id]);

  const reloadI18n = async () => {
    const res = await REST.Projects.getI18nData(
      AccountManager.getActiveToken(),
      router.query.id as string
    );

    if (res.status === 200) {
      setI18n(res.payload.i18n);
    } else {
      PopupManager.alertAsync({
        title: "Fehler",
        description:
          "Die Lokalisierungsdaten konnten nicht empfangen werden: " +
          res.payload.error,
      });
    }
  };

  return (
    <>
      <EditorProjectLayout project_id={router.query.id as string}>
        {i18n && (
          <>
```

```
<Box p={2}>
  <Box textAlign={"center"}>
    <CHeading size={"lg"}>Lokalisierung</CHeading>
    <Text fontSize={"lg"}>
      Hier kannst du verschiedene Sprachen für dein Projekt
      verwalten. Es werden Sprachkürzel wie "de" oder "en"
      verwendet. <br />
      Um die Sprachen in dein Projekt zu integrieren, kannst du die
      JavaScript-Bibliothek{" "}
      <chakra.a
        color={"brand.500"}
        target={"_blank"}
        href={"https://npmjs.com/@codeupspace/i18n"}
      >
        @codeupspace/i18n
      </chakra.a>{" "}
      verwenden.
    </Text>
  </Box>
  <Grid
    templateColumns={["repeat(1, 1fr)", "repeat(2, 1fr)"]}
    gap={6}
    p={4}
    mt={"2rem"}
  >
    <Box rounded={"md"} p={4} bg={"gray.900"} h={"100%"}>
      <CHeading size={"lg"}>Sprachen</CHeading>
      <Text size={"lg"}>
        Deine App unterstützt folgende Sprachen:{" "}
      </Text>
      <Button
        variant={"brand"}
        mt={4}
        onClick={async () => {
          const langCode = await PopupManager.promptAsync({
            title: "Sprache hinzufügen",
            helperText:
              "Bitte gib das Sprachkürzel ein (z.B. 'de')",
          });

          if (!langCode) return;

          const res = await REST.Projects.i18nAddLanguage(
            AccountManager.getActiveToken(),
            router.query.id as string,
            langCode
          );

          if (res.status === 200) {
            reloadI18n();
          } else {
            await PopupManager.alertAsync({
              title: "Fehler",
              description:
                "Die Sprache konnte nicht hinzugefügt werden: " +
                res.payload.error,
            });
          }
        }}
      >
```

```
  Sprache hinzufügen
</Button>
<Stack gap={2} pt={4}>
  {i18n.languages.map((l: string) => {
    return (
      <HStack spacing={2}>
        <Text size={"lg"}>{l}</Text>
        {i18n.defaultLanguage === l && (
          <>
            <Text size={"lg"}> (Standard)</Text>
          </>
        )}
        <ButtonGroup>
          <IconButton
            aria-label={"Entfernen"}
            icon={<FaTrash />}
            colorScheme={"red"}
            variant={"ghost"}
            onClick={async () => {
              if (
                !(await PopupManager.confirmAsync({
                  title: "Sprache entfernen",
                  question:
                    "Möchtest du die Sprache wirklich entfernen?",
                }))
              )
                return;

              const res =
                await REST.Projects.i18nRemoveLanguage(
                  AccountManager.getActiveToken(),
                  router.query.id as string,
                  l
                );

              if (res.status === 200) {
                reloadI18n();
              } else {
                await PopupManager.alertAsync({
                  title: "Fehler",
                  description:
                    "Die Sprache konnte nicht entfernt werden: " +
                    res.payload.error,
                });
              }
            }}
          />
          <IconButton
            aria-label={"Standard machen"}
            icon={<FaHammer />}
            variant={"ghost"}
            colorScheme={"brand"}
            onClick={async () => {
              if (
                !(await PopupManager.confirmAsync({
                  title: "Standard-Sprache",
                  question:
                    "Möchtest du diese Sprache als Standard festlegen?",
                }))
              )
```

```
                return;

              const res =
                await REST.Projects.i18nSetDefaultLanguage(
                  AccountManager.getActiveToken(),
                  router.query.id as string,
                  l
                );

              if (res.status === 200) {
                reloadI18n();
              } else {
                await PopupManager.alertAsync({
                  title: "Fehler",
                  description:
                    "Die Sprache konnte nicht als Standard festgelegt werden: " +
                    res.payload.error,
                });
              }
            }}
            display={
              i18n.defaultLanguage === l ? "none" : undefined
            }
          />
        </ButtonGroup>
      </HStack>
    );
  })}
 </Stack>
</Box>
<Box rounded={"md"} p={4} bg={"gray.900"} h={"100%"}>
 <CHeading size={"lg"}>Übersetzungen</CHeading>
 <Button
  variant={"brand"}
  mt={4}
  onClick={async () => {
    const key = await PopupManager.promptAsync({
      title: "Schlüssel hinzufügen",
      helperText: "Bitte gib den Schlüssel ein",
    });

    if (!key) return;

    for (const l of i18n.languages) {
      const res = await REST.Projects.i18nSetKey(
        AccountManager.getActiveToken(),
        router.query.id as string,
        l,
        key,
        ""
      );
    }

    await reloadI18n();
  }}
 >
  Hinzufügen
 </Button>
 <Table mt={4}>
  <Thead>
```

```
  <Th>Schlüssel</Th>
  <Th>Standard</Th>
</Thead>
<Tbody>
  {i18n.translations[i18n.defaultLanguage] &&
    Object.keys(
      i18n.translations[i18n.defaultLanguage]
    ).map((t: any) => {
      return (
        <>
          <Tr
            _hover={{
              bg: "gray.700",
              cursor: "pointer",
            }}
            onClick={async () => {
              await PopupManager.alertAsync({
                title: "Übersetzung bearbeiten",
                description: (
                  <>
                    <form
                      onSubmit={async (e) => {
                        e.preventDefault();

                        const fd = new FormData(
                          e.target as HTMLFormElement
                        );

                        let res: {
                          [key: string]: string;
                        } = {};

                        for (const l of i18n.languages) {
                          res[l] = fd.get(
                            "t-" + l
                          ) as string;
                        }

                        for (const k of Object.keys(res)) {
                          const r =
                            await REST.Projects.i18nSetKey(
                              AccountManager.getActiveToken(),
                              router.query.id as string,
                              k,
                              t,
                              res[k]
                            );
                        }

                        PopupManager.removeCurrentPopup();

                        await reloadI18n();
                      }}
                    >
                      <Stack spacing={4}>
                        {i18n.languages.map((l: string) => {
                          return (
                            <HStack spacing={2}>
                              <Text size={"lg"}>{l}</Text>
                              <Input
```

```
        size={"lg"}
        name={"t-" + l}
        id={"t-" + l}
        defaultValue={
          i18n.translations[l]
            ? i18n.translations[l][
                t
              ]
            : ""
        }
      />
      <IconButton
        aria-label={
          "Auto-Translate"
        }
        icon={<FaLanguage />}
        onClick={async () => {
          const x =
            await translateOnlineV3(
              {
                text: (
                  document.querySelector(
                    "#t-" +
                      i18n.defaultLanguage
                  ) as HTMLInputElement
                ).value,
                from: i18n.defaultLanguage,
                to: l,
              }
            );

          (
            document.querySelector(
              "#t-" + l
            ) as HTMLInputElement
          ).value = x;
        }}
        variant={"brand"}
        display={
          i18n.defaultLanguage === l
            ? "none"
            : undefined
        }
      />
    </HStack>
  );
})}
<ButtonGroup w={"100%"}>
  <Button
    variant={"brand"}
    w={"100%"}
    type={"submit"}
  >
    Speichern
  </Button>
  <Button
    colorScheme={"red"}
    w={"100%"}
    onClick={async () => {
      if (
```

```
                                    !(await PopupManager.confirmAsync(
                                      {
                                        title: "Löschen",
                                        question:
                                          "Möchtest du den Schlüssel wirklich löschen?",
                                      }
                                    ))
                                  )
                                    return;

                                  for (const k of Object.keys(
                                    i18n.translations
                                  )) {
                                    await REST.Projects.i18nDeleteKey(
                                      AccountManager.getActiveToken(),
                                      router.query.id as string,
                                      k,
                                      t
                                    );
                                  }

                                  PopupManager.removeCurrentPopup();

                                  await reloadI18n();
                                }}
                              >
                                Löschen
                              </Button>
                            </ButtonGroup>
                          </Stack>
                        </form>
                      </>
                    ),
                  });
                }}
              >
                <Td>
                  <Text size={"lg"}>{t}</Text>
                </Td>
                <Td>
                  <Text size={"lg"}>
                    {i18n.translations[i18n.defaultLanguage][t]}
                  </Text>
                </Td>
              </Tr>
            </>
          );
        })}
      </Tbody>
    </Table>
  </Box>
 </Grid>
</Box>
</>
)}
</EditorProjectLayout>
</>
);
}
```

## apps/web/src/pages/projects/[id]/index.tsx

```
/**
 * apps/web/src/pages/projects/[id]/index.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2024 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 30.05.2024
 *
 */

import * as React from "react";
import { useUserLoggedInRedirect } from "@codeup/hooks";
import { useRouter } from "next/router";
import { Box } from "@chakra-ui/react";
import AppShell from "@codeup/components/dist/global/AppShell";
import { IProject } from "../../../types/IProject";
import { useEffect } from "react";
import REST from "@codeup/rest/src";
import AccountManager from "@codeup/util/dist/AccountManager";
import PopupManager from "../../../global/PopupManager";
import EditorProjectLayout from "@codeup/components/editorv2/EditorProjectLayout";

export default function Index() {
  useUserLoggedInRedirect({
    allowAnonymous: false,
  });

  const [project, setProject] = React.useState<IProject | null>(null);

  const router = useRouter();

  useEffect(() => {
    if (!router || !router.query || !router.query.id) return;

    REST.Projects.project(
      AccountManager.getActiveToken(),
      router.query.id as string
    ).then((res) => {
      if (res.status === 200) {
        setProject(res.payload.project);
      } else {
        PopupManager.alertAsync({
          title: "Fehler",
          description:
            "Das Projekt konnte nicht empfangen werden: " + res.payload.error,
        });
      }
    });
  }, [router, router.query, router.query.id]);
  return (
    <>
      <EditorProjectLayout project_id={router.query.id as string}>
        <Box>123</Box>
      </EditorProjectLayout>
    </>
  );
}
```

## apps/web/src/pages/projects/[id]/issues.tsx

```
/**
 * apps/web/src/pages/projects/[id]/issues.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2024 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 31.05.2024
 *
 */

import * as React from "react";
import { useUserLoggedInRedirect } from "@codeup/hooks";
import { IProject } from "../../../types/IProject";
import { useRouter } from "next/router";
import { useEffect } from "react";
import REST from "@codeup/rest/dist";
import AccountManager from "@codeup/util/dist/AccountManager";
import PopupManager from "../../../global/PopupManager";
import EditorProjectLayout from "@codeup/components/dist/editorv2/EditorProjectLayout";
import {
  Box,
  Button,
  FormControl,
  FormLabel,
  Input,
  Select,
  Stack,
  Textarea,
} from "@chakra-ui/react";
import { IIssue } from "../../../types/IIssue";
import GridLayoutV2 from "@codeup/components/dist/global/GridLayoutV2";
import {
  FaClock,
  FaEye,
  FaPen,
  FaPlus,
  FaTimesCircle,
  FaTrash,
} from "react-icons/fa";

export default function Issues() {
  useUserLoggedInRedirect({
    allowAnonymous: false,
  });

  const [project, setProject] = React.useState<IProject | null>(null);

  const router = useRouter();

  useEffect(() => {
    if (!router || !router.query || !router.query.id) return;

    REST.Projects.project(
      AccountManager.getActiveToken(),
      router.query.id as string
    ).then(async (res) => {
      if (res.status === 200) {
        setProject(res.payload.project);

        reloadIssues();
```

```
    } else {
      PopupManager.alertAsync({
        title: "Fehler",
        description:
          "Das Projekt konnte nicht empfangen werden: " + res.payload.error,
      });
    }
  });
}, [router, router.query, router.query.id]);

const [issues, setIssues] = React.useState<IIssue[]>([]);

const reloadIssues = async () => {
  const x = await REST.Projects.getIssues(
    AccountManager.getActiveToken(),
    router.query.id as string
  );

  if (x.status === 200) {
    setIssues(x.payload.issues);
  } else {
    PopupManager.alertAsync({
      title: "Fehler",
      description:
        "Die Probleme konnten nicht empfangen werden: " + x.payload.error,
    });
  }
};
return (
  <>
    <EditorProjectLayout project_id={router.query.id as string}>
      <Box p={2}>
        <GridLayoutV2
          pageTitle={"Probleme"}
          search={{
            show: true,
            fields: true,
          }}
          topButtons={[
            {
              text: "Neues Problem",
              onClick: async () => {
                await PopupManager.alertAsync({
                  title: "Neues Problem",
                  description: (
                    <>
                      <form
                        onSubmit={async (e) => {
                          e.preventDefault();

                          const formD = new FormData(
                            e.target as HTMLFormElement
                          );

                          const title = formD.get("title") as string;
                          const description = formD.get(
                            "description"
                          ) as string;
                          const priority = formD.get("priority") as string;
                          const status = "open";
```

```
        const res = await REST.Projects.createIssue(
          AccountManager.getActiveToken(),
          router.query.id as string,
          title,
          description,
          priority,
          status
        );

        if (res.status === 200) {
          await reloadIssues();
          PopupManager.removeCurrentPopup();
        } else {
          PopupManager.removeCurrentPopup();

          await PopupManager.alertAsync({
            title: "Fehler",
            description:
              "Das Problem konnte nicht erstellt werden: " +
              res.payload.error,
          });
        }
      }}
    >
      <Stack spacing={4}>
        <FormControl isRequired>
          <FormLabel>Titel</FormLabel>
          <Input
            name={"title"}
            placeholder={"Mein Problem"}
          />
        </FormControl>
        <FormControl isRequired>
          <FormLabel>Beschreibung</FormLabel>
          <Textarea
            name={"description"}
            placeholder={
              "Beschreibe das Problem (du kannst Markdown verwenden)"
            }
          />
        </FormControl>
        <FormControl isRequired>
          <FormLabel>Priorität</FormLabel>
          <Select
            name={"priority"}
            placeholder={"Wähle eine Priorität"}
          >
            <option value={"low"}>Niedrig</option>
            <option value={"medium"}>Mittel</option>
            <option value={"high"}>Hoch</option>
          </Select>
        </FormControl>
        <Button variant={"brand"} type={"submit"}>
          Erstellen
        </Button>
      </Stack>
    </form>
  </>
),
```

```
      });
    },
    variant: "brand",
    leftIcon: <FaPlus />,
  },
]}
entries={issues.map((issue) => {
  return {
    title: issue.title,
    buttonRows: [
      [
        {
          text: "Anzeigen",
          leftIcon: <FaEye />,
          variant: "brand",
          onClick: async () => {
            await PopupManager.alertAsync({
              title: issue.title,
              description: (
                <>
                  <Box>
                    <b>Beschreibung:</b>
                    <br />
                    {issue.description}
                  </Box>
                  <Box>
                    <b>Erstellt:</b>
                    <br />
                    {new Date(issue.createdAt).toLocaleString()}
                  </Box>
                  <Box>
                    <b>Status:</b>
                    <br />
                    {issue.status === "open"
                      ? "Offen"
                      : issue.status === "closed"
                      ? "Geschlossen"
                      : "In Bearbeitung"}
                  </Box>
                  <Box>
                    <b>Priorität:</b>
                    <br />
                    {issue.priority === "low"
                      ? "Niedrig"
                      : issue.priority === "medium"
                      ? "Mittel"
                      : "Hoch"}
                  </Box>
                </>
              ),
            });
          },
        },
        {
          text: "Schließen",
          leftIcon: <FaTimesCircle />,
          colorScheme: "red",
          onClick: async () => {
            if (
              !(await PopupManager.confirmAsync({
```

```
          title: "Schließen",
          question:
            "Möchtest du dieses Problem wirklich schließen?",
        }))
      )
        return;

      const res = await REST.Projects.updateIssue(
        AccountManager.getActiveToken(),
        router.query.id as string,
        issue._id,
        issue.title,
        issue.description,
        issue.priority,
        "closed"
      );

      if (res.status === 200) {
        await reloadIssues();
      } else {
        PopupManager.alertAsync({
          title: "Fehler",
          description:
            "Das Problem konnte nicht geschlossen werden: " +
            res.payload.error,
        });
      }
    },
    disabled: issue.status === "closed",
  },
],
[
  {
    text: "Bearbeiten",
    leftIcon: <FaPen />,
    variant: "brand",
    onClick: async () => {
      await PopupManager.alertAsync({
        title: "Bearbeiten",
        description: (
          <>
            <form
              onSubmit={async (e) => {
                e.preventDefault();

                const formD = new FormData(
                  e.target as HTMLFormElement
                );

                const title = formD.get("title") as string;
                const description = formD.get(
                  "description"
                ) as string;
                const priority = formD.get(
                  "priority"
                ) as string;

                const res = await REST.Projects.updateIssue(
                  AccountManager.getActiveToken(),
                  router.query.id as string,
```

```
          issue._id,
          title,
          description,
          priority,
          issue.status
        );

        if (res.status === 200) {
          await reloadIssues();
          PopupManager.removeCurrentPopup();
        } else {
          PopupManager.removeCurrentPopup();

          await PopupManager.alertAsync({
            title: "Fehler",
            description:
              "Das Problem konnte nicht bearbeitet werden: " +
              res.payload.error,
          });
        }
      }}
    >
      <Stack spacing={4}>
        <FormControl isRequired>
          <FormLabel>Titel</FormLabel>
          <Input
            name={"title"}
            placeholder={"Mein Problem"}
            defaultValue={issue.title}
          />
        </FormControl>
        <FormControl isRequired>
          <FormLabel>Beschreibung</FormLabel>
          <Textarea
            name={"description"}
            placeholder={
              "Beschreibe das Problem (du kannst Markdown verwenden)"
            }
            defaultValue={issue.description}
          />
        </FormControl>
        <FormControl isRequired>
          <FormLabel>Priorität</FormLabel>
          <Select
            name={"priority"}
            placeholder={"Wähle eine Priorität"}
            defaultValue={issue.priority}
          >
            <option value={"low"}>Niedrig</option>
            <option value={"medium"}>Mittel</option>
            <option value={"high"}>Hoch</option>
          </Select>
        </FormControl>
        <Button variant={"brand"} type={"submit"}>
          Speichern
        </Button>
      </Stack>
    </form>
  </>
),
```

```
        });
      },
      disabled: issue.status === "closed",
    },
    {
      text: "Löschen",
      leftIcon: <FaTrash />,
      colorScheme: "red",
      onClick: async () => {
        if (
          !(await PopupManager.confirmAsync({
            title: "Löschen",
            question:
              "Möchtest du dieses Problem wirklich löschen?",
          }))
        )
          return;

        const res = await REST.Projects.deleteIssue(
          AccountManager.getActiveToken(),
          router.query.id as string,
          issue._id
        );

        if (res.status === 200) {
          await reloadIssues();
        } else {
          PopupManager.alertAsync({
            title: "Fehler",
            description:
              "Das Problem konnte nicht gelöscht werden: " +
              res.payload.error,
          });
        }
      },
    },
  ],
],
fields: [
  {
    text:
      "Erstellt: " + new Date(issue.createdAt).toLocaleString(),
    icon: <FaClock />,
  },
  {
    text:
      "Status: " +
      (issue.status === "open"
        ? "Offen"
        : issue.status === "closed"
        ? "Geschlossen"
        : "In Bearbeitung"),
  },
  {
    text:
      "Priorität: " +
      (issue.priority === "low"
        ? "Niedrig"
        : issue.priority === "medium"
        ? "Mittel"
```

```
                    : "Hoch"),
                },
              ],
            };
          })}
        />
      </Box>
    </EditorProjectLayout>
  </>
 );
}
```

## apps/web/src/pages/projects/[id]/planning/overview.tsx

```
/**
 * apps/web/src/pages/projects/[id]/planning/overview.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2024 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 02.06.2024
 *
 */

import * as React from "react";
import { useUserLoggedInRedirect } from "@codeup/hooks";
import EditorProjectLayout from "@codeup/components/editorv2/EditorProjectLayout";
import { useRouter } from "next/router";
import { IIssue } from "../../../../types/IIssue";
import REST from "@codeup/rest/src";
import AccountManager from "@codeup/util/dist/AccountManager";
import { Badge, Box, Flex, Grid, Stack, Text } from "@chakra-ui/react";
import { CHeading } from "@codeup/components/dist/global/CHeading";
import PopupManager from "../../../../global/PopupManager";

export default function Overview() {
  useUserLoggedInRedirect({
    allowAnonymous: false,
  });

  const router = useRouter();

  const [issues, setIssues] = React.useState<IIssue[]>([]);
  const [flows, setFlows] = React.useState<any[]>([]);

  React.useEffect(() => {
    reloadData();
  }, []);

  const reloadData = async () => {
    const iRes = await REST.Projects.getIssues(
      AccountManager.getActiveToken(),
      router.query.id as string
    );
    if (iRes.status === 200) {
      setIssues(iRes.payload.issues);
    }
    const fRes = await REST.Projects.getFlows(
      AccountManager.getActiveToken(),
      router.query.id as string
    );
```

```
  if (fRes.status === 200) {
    setFlows(fRes.payload.flows);
  }
};

return (
  <>
    <EditorProjectLayout project_id={router.query.id as string}>
      <Box p={4}>
        <CHeading size={"lg"}>Planungsübersicht</CHeading>
        <Grid
          templateColumns={["repeat(1, 1fr)", "repeat(2, 1fr)"]}
          w={"100%"}
          p={8}
          gap={8}
        >
          <Box bg={"gray.900"} rounded={"md"} p={4}>
            <CHeading size={"md"}>Probleme</CHeading>
            <Stack pt={4}>
              {issues.slice(0, 5).map((issue) => (
                <Flex
                  bg={"gray.800"}
                  rounded={"md"}
                  p={4}
                  alignItems={"center"}
                  justifyContent={"space-between"}
                  cursor={"pointer"}
                  onClick={async () => {
                    await PopupManager.alertAsync({
                      title: issue.title,
                      description: (
                        <>
                          <Box>
                            <b>Beschreibung:</b>
                            <br />
                            {issue.description}
                          </Box>
                          <Box>
                            <b>Erstellt:</b>
                            <br />
                            {new Date(issue.createdAt).toLocaleString()}
                          </Box>
                          <Box>
                            <b>Status:</b>
                            <br />
                            {issue.status === "open"
                              ? "Offen"
                              : issue.status === "closed"
                              ? "Geschlossen"
                              : "In Bearbeitung"}
                          </Box>
                          <Box>
                            <b>Priorität:</b>
                            <br />
                            {issue.priority === "low"
                              ? "Niedrig"
                              : issue.priority === "medium"
                              ? "Mittel"
                              : "Hoch"}
```

```
                    </Box>
                  </>
                ),
              });
            }}
          >
            <CHeading size={"md"}>{issue.title}</CHeading>
            <Text>{new Date(issue.createdAt).toLocaleString()}</Text>
            <Badge
              colorScheme={issue.status === "open" ? "red" : "green"}
            >
              {issue.status}
            </Badge>
            <Badge
              colorScheme={
                issue.priority === "high"
                  ? "red"
                  : issue.priority === "medium"
                  ? "yellow"
                  : "green"
              }
            >
              {issue.priority}
            </Badge>
            <Box></Box>
            <Box></Box>
          </Flex>
        ))}
      </Stack>
    </Box>
    <Box bg={"gray.900"} p={4} rounded={"md"}>
      <CHeading size={"md"}>Flussdiagramme</CHeading>
      <Stack pt={4}>
        {flows.slice(0, 5).map((flow) => (
          <Flex
            bg={"gray.800"}
            rounded={"md"}
            p={4}
            alignItems={"center"}
            justifyContent={"space-between"}
            cursor={"pointer"}
            onClick={async () => {
              router.push(
                "/projects/" + router.query.id + "/flows/" + flow._id
              );
            }}
          >
            <CHeading size={"md"}>{flow.name}</CHeading>
            <Text>{new Date(flow.createdAt).toLocaleString()}</Text>
            <Box></Box>
            <Box></Box>
          </Flex>
        ))}
      </Stack>
    </Box>
  </Grid>
</Box>
</EditorProjectLayout>
</>
);
```

```
}
```

## apps/web/src/pages/projects/[id]/static.tsx

```
/**
 * apps/web/src/pages/projects/[id]/static.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2024 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 31.05.2024
 *
 */

import * as React from "react";
import { useUserLoggedInRedirect } from "@codeup/hooks";
import { IProject } from "../../../types/IProject";
import { useRouter } from "next/router";
import { useEffect } from "react";
import REST from "@codeup/rest/dist";
import AccountManager from "@codeup/util/dist/AccountManager";
import PopupManager from "../../../global/PopupManager";
import EditorProjectLayout from "@codeup/components/dist/editorv2/EditorProjectLayout";
import {
  Box,
  Button,
  Flex,
  Input,
  Table,
  TableContainer,
  Tbody,
  Td,
  Text,
  Th,
  Thead,
  Tr,
} from "@chakra-ui/react";
import { CHeading } from "@codeup/components/dist/global/CHeading";
import byteSize from "byte-size";
import { FaFileUpload, FaTrash } from "react-icons/fa";

export default function Static() {
  useUserLoggedInRedirect({
    allowAnonymous: false,
  });

  const [project, setProject] = React.useState<IProject | null>(null);

  const router = useRouter();

  useEffect(() => {
    if (!router || !router.query || !router.query.id) return;

    REST.Projects.project(
      AccountManager.getActiveToken(),
      router.query.id as string
    ).then(async (res) => {
      if (res.status === 200) {
        setProject(res.payload.project);

        reloadFiles();
```

```
    } else {
      PopupManager.alertAsync({
        title: "Fehler",
        description:
          "Das Projekt konnte nicht empfangen werden: " + res.payload.error,
      });
    }
  });
}, [router, router.query, router.query.id]);

const [files, setFiles] = React.useState<{ name: string; size: number }[]>(
  []
);

const reloadFiles = async () => {
  const x = await REST.Projects.files(
    AccountManager.getActiveToken(),
    router.query.id as string
  );

  if (x.status === 200) {
    setFiles(x.payload.filesArray);
  } else {
    PopupManager.alertAsync({
      title: "Fehler",
      description:
        "Die Dateien konnten nicht empfangen werden: " + x.payload.error,
    });
  }
};

return (
  <>
    <EditorProjectLayout project_id={router.query.id as string}>
      <Box p={2}>
        <Flex justifyContent={"center"}>
          <Box textAlign={"center"} maxW={"500px"}>
            <CHeading size={"lg"}>Statische Dateien</CHeading>
            <Text>
              Lade statische Dateien, wie Bilder hoch, damit du sie in deinem
              Projekt verwenden kannst. Bedenke, dass alle Dateien öffentlich
              zugänglich sind. Du solltest deshalb keine sensiblen Daten
              hochladen. Eine Datei darf bis zu 10MB groß sein.
            </Text>
            <Button
              mt={4}
              variant={"brand"}
              leftIcon={<FaFileUpload />}
              onClick={async () => {
                const fl = await PopupManager.uploadFileAsync({
                  title: "Datei hochladen",
                  helperText:
                    "Wähle eine Datei aus, die du hochladen möchtest",
                });

                if (!fl || fl.length === 0) return;

                const file = fl[0];

                const reader = new FileReader();
```

```
        reader.onload = async () => {
          const content = (reader.result as string).split(
            "; base64,"
          )[1];

          console.log(content);

          const x = await REST.Projects.uploadFile(
            AccountManager.getActiveToken(),
            router.query.id as string,
            file.name,
            content
          );

          if (x.status === 200) {
            await reloadFiles();
          } else {
            PopupManager.alertAsync({
              title: "Fehler",
              description:
                "Die Datei konnte nicht hochgeladen werden: " +
                x.payload.error,
            });
          }
        };

        reader.readAsDataURL(file);
      }}
    >
      Datei hochladen
    </Button>
  </Box>
</Flex>
<TableContainer>
  <Table maxW={"100%"}>
    <Thead>
      <Th>Name</Th>
      <Th>Größe</Th>
      <Th>Aktionen</Th>
    </Thead>
    <Tbody>
      {files.map((file) => (
        <Tr>
          <Td>{file.name}</Td>
          <Td>{byteSize(file.size).toString()}</Td>
          <Td>
            <Button
              leftIcon={<FaTrash />}
              colorScheme={"red"}
              onClick={async () => {
                if (
                  !(await PopupManager.confirmAsync({
                    title: "Löschen",
                    question:
                      "Möchtest du diese Datei wirklich löschen?",
                  }))
                )
                  return;
```

```
                    const x = await REST.Projects.deleteFile(
                      AccountManager.getActiveToken(),
                      router.query.id as string,
                      file.name
                    );

                    if (x.status === 200) {
                      await reloadFiles();
                    } else {
                      PopupManager.alertAsync({
                        title: "Fehler",
                        description:
                          "Die Datei konnte nicht gelöscht werden: " +
                          x.payload.error,
                      });
                    }
                  }}
                >
                  Löschen
                </Button>
                <Button
                  variant={"brand"}
                  ml={2}
                  onClick={() => {
                    PopupManager.alertAsync({
                      title: "URL",
                      description: (
                        <>
                          <Input
                            type={"text"}
                            placeholder={"URL"}
                            value={
                              window.location.origin +
                              "/api/s3/" +
                              project!._id +
                              "/" +
                              file.name
                            }
                          />
                        </>
                      ),
                    });
                  }}
                >
                  URL
                </Button>
              </Td>
            </Tr>
          ))}
        </Tbody>
      </Table>
    </TableContainer>
  </Box>
 </EditorProjectLayout>
 </>
 );
}
```

apps/web/src/pages/projects/[id]/uml/[uml_id].tsx

```
/**
```

```
 * apps/web/src/pages/projects/[id]/uml/[uml_id].tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2024 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 08.06.2024
 *
 */

import * as React from "react";
import { useUserLoggedInRedirect } from "@codeup/hooks";
import { useRouter } from "next/router";
import {
  Box,
  Button,
  Flex,
  HStack,
  IconButton,
  Tab,
  TabList,
  TabPanel,
  TabPanels,
  Tabs,
  Text,
} from "@chakra-ui/react";
import { IProject } from "../../../../types/IProject";
import { useEffect } from "react";
import REST from "@codeup/rest/src";
import AccountManager from "@codeup/util/dist/AccountManager";
import PopupManager from "../../../../global/PopupManager";
import EditorProjectLayout from "@codeup/components/editorv2/EditorProjectLayout";
import { IUMLDiagram } from "../../../../types/IUMLDiagram";
import AuthenticateLoadingIndicator from
"@codeup/components/dist/admin/AuthenticateLoadingIndicator";
import { CHeading } from "@codeup/components/dist/global/CHeading";
import { FaCode, FaEye } from "react-icons/fa";
import UMLEditor from "@codeup/components/editorv2/uml/UMLEditor";
import UMLPreview from "@codeup/components/editorv2/uml/UMLPreview";

export default function UMLViewer() {
  useUserLoggedInRedirect({
    allowAnonymous: false,
  });

  const [project, setProject] = React.useState<IProject | null>(null);
  const [uml, setUml] = React.useState<IUMLDiagram | null>(null);

  const router = useRouter();

  useEffect(() => {
    if (!router || !router.query || !router.query.id) return;

    REST.Projects.project(
      AccountManager.getActiveToken(),
      router.query.id as string
    ).then((res) => {
      if (res.status === 200) {
        setProject(res.payload.project);
      } else {
        PopupManager.alertAsync({
```

```
        title: "Fehler",
        description:
          "Das Projekt konnte nicht empfangen werden: " + res.payload.error,
      });
    }
  });
}, [router, router.query, router.query.id]);

useEffect(() => {
  if (router && router.query && router.query.id && router.query.uml_id) {
    reloadUML();
  }
}, [router, router.query, router.query.uml_id, router.query.id]);

const reloadUML = async () => {
  const res = await REST.Projects.umlDiagrams(
    AccountManager.getActiveToken(),
    router.query.id as string,
    router.query.uml_id as string
  );

  if (res.status === 200) {
    setUml(res.payload.uml);
  } else {
    await PopupManager.alertAsync({
      title: "Fehler",
      description:
        "Das UML-Diagramm konnte nicht empfangen werden: " +
        res.payload.error,
    });
  }
};

return (
  <>
    <EditorProjectLayout project_id={router.query.id as string}>
      {!uml && (
        <>
          <AuthenticateLoadingIndicator text={"Lade UML-Diagramm"} />
        </>
      )}
      {uml && (
        <>
          <Box p={2}>
            <Box textAlign={"center"}>
              <CHeading size={"lg"}>{uml.name}</CHeading>
            </Box>
            <Tabs variant={"solid-rounded"} colorScheme={"brand"} p={4}>
              <TabList w={"fit-content"} gap={4}>
                <Tab>
                  <HStack gap={2} alignItems={"center"}>
                    <FaCode />
                    <Text>Code</Text>
                  </HStack>
                </Tab>
                <Tab>
                  <HStack gap={2} alignItems={"center"}>
                    <FaEye />
                    <Text>Vorschau</Text>
                  </HStack>
```

```
              </Tab>
            </TabList>
            <TabPanels>
              <TabPanel>
                <UMLEditor
                  uml={uml}
                  updateUML={async (u) => {
                    setUml(u);

                    await REST.Projects.updateUmlDiagram(
                      AccountManager.getActiveToken(),
                      router.query.id as string,
                      router.query.uml_id as string,
                      u.data
                    );
                  }}
                />
              </TabPanel>
              <TabPanel>
                <UMLPreview
                  umlData={
                    JSON.parse(uml.data).content || "@startuml\n\n@enduml"
                  }
                />
              </TabPanel>
            </TabPanels>
          </Tabs>
        </Box>
      </>
    )}
  </EditorProjectLayout>
 </>
 );
}
```

## apps/web/src/pages/projects/[id]/uml.tsx

```
/**
 * apps/web/src/pages/projects/[id]/uml.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2024 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 08.06.2024
 *
 */

import * as React from "react";
import { useUserLoggedInRedirect } from "@codeup/hooks";
import { useRouter } from "next/router";
import { Box, Flex, Grid, Text } from "@chakra-ui/react";
import { IProject } from "../../../types/IProject";
import { useEffect } from "react";
import REST from "@codeup/rest/src";
import AccountManager from "@codeup/util/dist/AccountManager";
import PopupManager from "../../../global/PopupManager";
import EditorProjectLayout from "@codeup/components/editorv2/EditorProjectLayout";
import GridLayoutV2 from "@codeup/components/dist/global/GridLayoutV2";
import {
  FaClock,
  FaCode,
```

```
  FaExchangeAlt,
  FaEye,
  FaHtml5,
  FaPlus,
  FaSitemap,
  FaTrash,
} from "react-icons/fa";
import { CHeading } from "@codeup/components/dist/global/CHeading";
import { FaCube, FaHardDrive } from "react-icons/fa6";
import byteSize from "byte-size";

export default function UML() {
  useUserLoggedInRedirect({
    allowAnonymous: false,
  });

  const [project, setProject] = React.useState<IProject | null>(null);
  const [umls, setUmls] = React.useState<any[]>([]);

  const router = useRouter();

  useEffect(() => {
    if (!router || !router.query || !router.query.id) return;

    REST.Projects.project(
      AccountManager.getActiveToken(),
      router.query.id as string
    ).then((res) => {
      if (res.status === 200) {
        setProject(res.payload.project);
        reloadUmls();
      } else {
        PopupManager.alertAsync({
          title: "Fehler",
          description:
            "Das Projekt konnte nicht empfangen werden: " + res.payload.error,
        });
      }
    });
  }, [router, router.query, router.query.id]);

  const reloadUmls = async () => {
    const res = await REST.Projects.umlDiagrams(
      AccountManager.getActiveToken(),
      router.query.id as string
    );

    if (res.status === 200) {
      setUmls(res.payload.umls);
    } else {
      PopupManager.alertAsync({
        title: "Fehler",
        description:
          "Die UML-Diagramme konnten nicht empfangen werden: " +
          res.payload.error,
      });
    }
  };

  const uml_types: {
```

```
  name: string;
  description: string;
  icon: any;
  identifier: string;
}[] = [
  {
    name: "Implementation",
    description:
      "Ein Implementationsdiagramm in dem Klassen und Methoden dargestellt werden.",
    icon: <FaCube />,
    identifier: "impl",
  },
  {
    name: "ER-Diagramm",
    description:
      "Ein ER-Diagramm zur Darstellung von Entitäten und Beziehungen.",
    icon: <FaSitemap />,
    identifier: "er",
  },
  {
    name: "Sequenz",
    description:
      "Ein Sequenzdiagramm zur Darstellung von Abläufen und Methodenaufrufen.",
    icon: <FaExchangeAlt />,
    identifier: "seq",
  },
  {
    name: "Erweitert",
    description:
      "Ein erweitertes UML-Diagramm für Profis. (Nicht empfohlen für Anfänger)",
    icon: <FaCode />,
    identifier: "adv",
  },
];

return (
  <>
    <EditorProjectLayout project_id={router.query.id as string}>
      <Box p={2}>
        <GridLayoutV2
          pageTitle={"UML-Diagramme"}
          search={{
            show: true,
            fields: true,
          }}
          topButtons={[
            {
              text: "Neu",
              variant: "brand",
              onClick: async () => {
                const name = await PopupManager.promptAsync({
                  title: "Neues Diagramm",
                  helperText: "Bitte gib den Namen des neuen Diagramms ein.",
                });

                if (!name) return;

                const x = await new Promise(async (resolve) => {
                  await PopupManager.alertAsync({
                    title: "Neues Diagramm",
```

```
    description: (
      <>
        <Text>Bitte wählen den Typ des Diagramms aus.</Text>
        <Grid
          templateColumns={[
            "repeat(1, 1fr)",
            "repeat(2, 1fr)",
          ]}
          pt={6}
          gap={4}
        >
          {uml_types.map((type) => (
            <>
              <Box
                bg={"#000"}
                rounded={"xl"}
                cursor={"pointer"}
                _hover={{
                  outline: "1px solid #F7DE1F",
                }}
                p={4}
                onClick={async () => {
                  resolve(type.identifier);
                  PopupManager.removeCurrentPopup();
                }}
              >
                <Flex
                  fontSize={"4xl"}
                  w={"100%"}
                  justifyContent={"center"}
                  color={"white"}
                  align={"center"}
                >
                  {type.icon}
                </Flex>
                <Box mt={2} textAlign={"center"}>
                  <CHeading size={"md"}>{type.name}</CHeading>
                </Box>
                <Text
                  fontSize={"sm"}
                  color={"white"}
                  textAlign={"center"}
                  mt={2}
                >
                  {type.description}
                </Text>
              </Box>
            </>
          ))}
        </Grid>
      </>
    ),
  });
});

if (!x) return;

const res = await REST.Projects.createUmlDiagram(
  AccountManager.getActiveToken(),
  router.query.id as string,
```

```
        name,
        JSON.stringify({}),
        x as string
      );

      if (res.status !== 200) {
        await PopupManager.alertAsync({
          title: "Fehler",
          description:
            "Das Diagramm konnte nicht erstellt werden: " +
            res.payload.error,
        });
        return;
      }

      await reloadUmls();
    },
    leftIcon: <FaPlus />,
  },
]}
entries={umls.map((u) => ({
  title: u.name,
  buttonRows: [
    [
      {
        text: "Öffnen",
        onClick: () => {
          router.push(
            "/projects/" +
              router.query.id +
              "/uml/" +
              u._id +
              "?type=" +
              u.type
          );
        },
        leftIcon: <FaEye />,
        variant: "brand",
      },
      {
        text: "Löschen",
        onClick: async () => {
          if (
            !(await PopupManager.confirmAsync({
              title: "Löschen",
              question:
                "Möchtest du das Diagramm wirklich löschen?",
            }))
          )
            return;

          const res = await REST.Projects.deleteUmlDiagram(
            AccountManager.getActiveToken(),
            router.query.id as string,
            u._id
          );

          if (res.status !== 200) {
            await PopupManager.alertAsync({
              title: "Fehler",
```

```
                description:
                  "Das Diagramm konnte nicht gelöscht werden: " +
                  res.payload.error,
              });
              return;
            }

            await reloadUmls();
          },
          leftIcon: <FaTrash />,
          colorScheme: "red",
        },
      ],
    ],
    fields: [
      {
        text:
          "Erstellt am: " + new Date(u.createdAt).toLocaleString(),
        icon: <FaClock />,
      },
      {
        text:
          "Typ: " +
          (uml_types.find((t) => t.identifier === u.type)?.name ??
            "Unbekannt"),
        icon: uml_types.find((t) => t.identifier === u.type)
          ?.icon ?? <FaHtml5 />,
      },
      {
        text:
          "Gespeicherte Daten: " + byteSize(u.data.length).toString(),
        icon: <FaHardDrive />,
      },
    ],
  }))}
 />
 </Box>
 </EditorProjectLayout>
 </>
 );
}
```

## apps/web/src/pages/projects/[id]/wiki/[pageId].tsx

```
/**
 * apps/web/src/pages/projects/[id]/wiki/[pageId].tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2024 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 31.05.2024
 *
 */

import * as React from "react";
import { useUserLoggedInRedirect } from "@codeup/hooks";
import { IProject } from "../../../../types/IProject";
import { IWikiPage } from "../../../../types/IWikiPage";
import { useRouter } from "next/router";
import { useEffect } from "react";
import REST from "@codeup/rest/dist";
```

```
import AccountManager from "@codeup/util/dist/AccountManager";
import PopupManager from "../../../../global/PopupManager";
import EditorProjectLayout from "@codeup/components/editorv2/EditorProjectLayout";
import { Box, Flex, IconButton, Link, Textarea } from "@chakra-ui/react";
import { FaPen, FaSave } from "react-icons/fa";
import Markdown from "markdown-to-jsx";
import { defaultMdOpts } from "../../../../global/md/defaultProps";

export default function PageViewer() {
  useUserLoggedInRedirect({
    allowAnonymous: false,
  });

  const [project, setProject] = React.useState<IProject | null>(null);

  const router = useRouter();

  useEffect(() => {
    if (!router || !router.query || !router.query.id) return;

    REST.Projects.project(
      AccountManager.getActiveToken(),
      router.query.id as string
    ).then(async (res) => {
      if (res.status === 200) {
        setProject(res.payload.project);
        reloadPage();
      } else {
        PopupManager.alertAsync({
          title: "Fehler",
          description:
            "Das Projekt konnte nicht empfangen werden: " + res.payload.error,
        });
      }
    });
  }, [router, router.query, router.query.id]);

  const [page, setPage] = React.useState<IWikiPage | null>(null);

  const reloadPage = async () => {
    const res = await REST.Projects.wikiPages(
      AccountManager.getActiveToken(),
      router.query.id as string,
      router.query.pageId as string
    );

    if (res.status !== 200) {
      PopupManager.alertAsync({
        title: "Fehler",
        description:
          "Die Seite konnte nicht empfangen werden: " + res.payload.error,
      });
      return;
    }

    setPage(res.payload.page);
  };

  const [editMode, setEditMode] = React.useState<boolean>(false);
```

```
if (!project) {
  return <></>;
}

return (
  <>
    <EditorProjectLayout project_id={router.query.id as string}>
      <Flex
        w={"100%"}
        h={"fit-content"}
        minH={"100vh"}
        justifyContent={"center"}
        p={4}
      >
        <Box bg={"black"} rounded={"xl"} w={"100%"} maxW={"800px"} p={4}>
          <Flex w={"100%"} justifyContent={"space-between"}>
            <Link
              color={"brand.500"}
              href={"/projects/" + project._id + "/wiki"}
            >
              Zurück
            </Link>
            <IconButton
              aria-label={"Edit"}
              icon={editMode ? <FaSave /> : <FaPen />}
              colorScheme={"brand"}
              onClick={async () => {
                if (!editMode) {
                  setEditMode(true);
                } else {
                  const newContent = (
                    document.querySelector(
                      "#proj_wiki-page-content-edit"
                    ) as HTMLTextAreaElement
                  ).value;

                  const res = await REST.Projects.updateWikiPage(
                    AccountManager.getActiveToken(),
                    project._id,
                    page!._id,
                    page!.title,
                    newContent
                  );

                  if (res.status !== 200) {
                    PopupManager.alertAsync({
                      title: "Fehler",
                      description:
                        "Die Seite konnte nicht gespeichert werden: " +
                        res.payload.error,
                    });
                    return;
                  }

                  await reloadPage();

                  setEditMode(false);
                }
              }}
              variant={"ghost"}
```

```
            />
          </Flex>
          {!editMode ? (
            <Markdown options={defaultMdOpts}>{page?.content || ""}</Markdown>
          ) : (
            <>
              <Textarea
                minH={"300px"}
                placeholder={"Seiteninhalt eingeben"}
                defaultValue={page?.content || ""}
                id={"proj_wiki-page-content-edit"}
              />
            </>
          )}
        </Box>
      </Flex>
    </EditorProjectLayout>
  </>
 );
}
```

## apps/web/src/pages/projects/[id]/wiki.tsx

```
/**
 * apps/web/src/pages/projects/[id]/wiki.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2024 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 31.05.2024
 *
 */

import * as React from "react";
import { useUserLoggedInRedirect } from "@codeup/hooks";
import { IProject } from "../../../types/IProject";
import { useRouter } from "next/router";
import { useEffect } from "react";
import REST from "@codeup/rest/dist";
import AccountManager from "@codeup/util/dist/AccountManager";
import PopupManager from "../../../global/PopupManager";
import EditorProjectLayout from "@codeup/components/editorv2/EditorProjectLayout";
import { Box } from "@chakra-ui/react";
import { FaClock, FaEye, FaPen, FaPlus, FaTrash } from "react-icons/fa";
import GridLayoutV2 from "@codeup/components/dist/global/GridLayoutV2";
import { IWikiPage } from "../../../types/IWikiPage";

export default function Wiki() {
  useUserLoggedInRedirect({
    allowAnonymous: false,
  });

  const [project, setProject] = React.useState<IProject | null>(null);
  const [pages, setPages] = React.useState<IWikiPage[]>([]);

  const router = useRouter();

  useEffect(() => {
    if (!router || !router.query || !router.query.id) return;

    REST.Projects.project(
```

```
    AccountManager.getActiveToken(),
    router.query.id as string
  ).then(async (res) => {
    if (res.status === 200) {
      setProject(res.payload.project);
      reloadPages();
    } else {
      PopupManager.alertAsync({
        title: "Fehler",
        description:
          "Das Projekt konnte nicht empfangen werden: " + res.payload.error,
      });
    }
  });
}, [router, router.query, router.query.id]);

const reloadPages = async () => {
  const res = await REST.Projects.wikiPages(
    AccountManager.getActiveToken(),
    router.query.id as string
  );

  if (res.status !== 200) {
    await PopupManager.alertAsync({
      title: "Fehler",
      description:
        "Die Seiten konnten nicht geladen werden: " + res.payload.error,
    });
    return;
  }

  setPages(res.payload.pages);
};

return (
  <>
    <EditorProjectLayout project_id={router.query.id as string}>
      <Box p={2}>
        <GridLayoutV2
          pageTitle={"Wiki"}
          search={{
            show: true,
            fields: true,
          }}
          topButtons={[
            {
              text: "Neue Seite",
              leftIcon: <FaPlus />,
              variant: "brand",
              onClick: async () => {
                const name = await PopupManager.promptAsync({
                  title: "Neue Seite",
                  helperText: "Bitte gib den Namen der neuen Seite ein.",
                });

                if (!name) return;

                const res = await REST.Projects.createWikiPage(
                  AccountManager.getActiveToken(),
                  router.query.id as string,
```

```
              name,
              "# " + name
            );

            if (res.status !== 200) {
              await PopupManager.alertAsync({
                title: "Fehler",
                description:
                  "Die Seite konnte nicht erstellt werden: " +
                  res.payload.error,
              });
              return;
            }

            await reloadPages();
          },
        },
      ]}
      entries={pages.map((page) => {
        return {
          title: page.title,
          buttonRows: [
            [
              {
                text: "Öffnen",
                leftIcon: <FaEye />,
                variant: "brand",
                onClick: () => {
                  router.push(
                    "/projects/" + router.query.id + "/wiki/" + page._id
                  );
                },
              },
              {
                text: "Löschen",
                leftIcon: <FaTrash />,
                colorScheme: "red",
                onClick: async () => {
                  if (
                    !(await PopupManager.confirmAsync({
                      title: "Seite löschen",
                      question:
                        "Willst du diese Seite wirklich löschen? Dieser Vorgang kann nicht rückgängig
gemacht werden!",
                    }))
                  )
                    return;

                  const res = await REST.Projects.deleteWikiPage(
                    AccountManager.getActiveToken(),
                    router.query.id as string,
                    page._id
                  );

                  if (res.status !== 200) {
                    await PopupManager.alertAsync({
                      title: "Fehler",
                      description:
                        "Die Seite konnte nicht gelöscht werden: " +
                        res.payload.error,
```

```
                    });
                    return;
                  }

                  await reloadPages();
                },
              },
            ],
            [
              {
                text: "Umbenennen",
                variant: "brand",
                leftIcon: <FaPen />,
                onClick: async () => {
                  const newName = await PopupManager.promptAsync({
                    title: "Umbenennen",
                    helperText: "Bitte gib den neuen Namen ein.",
                  });

                  if (!newName) return;
                  const res = await REST.Projects.updateWikiPage(
                    AccountManager.getActiveToken(),
                    router.query.id as string,
                    page._id,
                    newName,
                    page.content
                  );

                  if (res.status !== 200) {
                    await PopupManager.alertAsync({
                      title: "Fehler",
                      description:
                        "Die Seite konnte nicht umbenannt werden: " +
                        res.payload.error,
                    });
                    return;
                  }

                  await reloadPages();
                },
              },
            ],
          ],
          fields: [
            {
              text:
                "Erstellt: " + new Date(page.createdAt).toLocaleString(),
              icon: <FaClock />,
            },
          ],
        };
      })}
    />
  </Box>
 </EditorProjectLayout>
</>
);
}
```

apps/web/src/pages/redirect.tsx

```
/**
 * apps/web/src/pages/redirect.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 22.08.2022
 *
 */

import * as React from "react";
import { useRouter } from "next/router";
import Head from "next/head";

export default function Redirect() {
  const router = useRouter();

  const url = router.query.url;

  React.useEffect(() => {
    if (url) {
      if (typeof url === "string") {
        window.location.href = url;
      }
    }
  }, [url]);

  return (
    <>
      <Head>
        <title>Weiterleiten...</title>
      </Head>
    </>
  );
}
```

## apps/web/src/pages/roadmap/index.tsx

```
/**
 * apps/web/src/pages/roadmap/index.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 07.04.2023
 *
 */

import * as React from "react";
import { useLoggedIn } from "@codeup/hooks";
import { useRouter } from "next/router";
import REST from "@codeup/rest/src";
import AccountManager from "@codeup/util/dist/AccountManager";
import PopupManager from "../../global/PopupManager";
import AuthenticateLoadingIndicator from
"@codeup/components/dist/admin/AuthenticateLoadingIndicator";
import SuperText from "@codeup/components/dist/landingpage/SuperText";
import GridLayout from "@codeup/components/dist/global/GridLayout";
import {
  FaBookOpen,
  FaCheck,
```

```
  FaCode,
  FaEye,
  FaLightbulb,
  FaPen,
  FaPlus,
  FaProjectDiagram,
  FaTrash,
} from "react-icons/fa";
import { Box } from "@chakra-ui/react";
import GridLayoutV2 from "@codeup/components/dist/global/GridLayoutV2";

export default function Index() {
  const { loggedIn, loaded, userInfo } = useLoggedIn();
  const [roadmaps, setRoadmaps] = React.useState<
    {
      owner: string;
      _id: string;
      name: string;
      projectIdea: string;
      linkedProject: string;
      linkedCourses: string[];
      linkedFlows: string[];
      linkedTodos: string[];
    }[]
  >([]);
  const router = useRouter();

  React.useEffect(() => {
    if (loaded && !loggedIn) {
      router.push("/#login");
      window.dispatchEvent(new CustomEvent("codeup::open_login"));
    } else if (loaded && loggedIn) {
      reload();
    }
  }, [loggedIn, loaded, userInfo]);

  const reload = () => {
    REST.Roadmap.getMyRoads(AccountManager.getActiveToken()).then((res) => {
      if (res.status !== 200) {
        PopupManager.alert({
          title: "Fehler",
          description: "Es ist ein Fehler aufgetreten: " + res.payload.error,
        });
      } else {
        setRoadmaps(res.payload.roadmaps);
      }
    });
  };

  return (
    <>
      {!loaded ? (
        <>
          <AuthenticateLoadingIndicator />
        </>
      ) : (
        <>
          <Box bg={"#000"} minH={"100vh"} h={"fit-content"}>
            <GridLayoutV2
              pageTitle={"Roadmaps"}
```

```
search={{
  show: true,
  fields: true,
}}
topButtons={[
  {
    variant: "brand",
    text: "Neu",
    leftIcon: <FaPlus />,
    onClick: async () => {
      const name = await PopupManager.promptAsync({
        title: "Neue Roadmap",
        helperText: "Gib einen Namen für die Roadmap ein",
        inputType: "INPUT",
      });

      if (!name) return;

      const res = await REST.Roadmap.createRoad({
        token: AccountManager.getActiveToken(),
        name,
      });

      if (res.status !== 200) {
        PopupManager.alert({
          title: "Fehler",
          description:
            "Es ist ein Fehler aufgetreten: " + res.payload.error,
        });
        return;
      }

      reload();
    },
  },
]}
entries={roadmaps.map((r) => {
  return {
    fields: [
      {
        icon: <FaLightbulb />,
        text: "Idee Verlinkt: " + (r.projectIdea ? "Ja" : "Nein"),
      },
      {
        icon: <FaCode />,
        text:
          "Projekt Verlinkt: " +
          (r.linkedProject ? "Ja" : "Nein"),
      },
      {
        icon: <FaBookOpen />,
        text: "Kurse Verlinkt: " + r.linkedCourses.length,
      },
      {
        icon: <FaProjectDiagram />,
        text: "Flows Verlinkt: " + r.linkedFlows.length,
      },
      {
        icon: <FaCheck />,
        text: "Todos Verlinkt: " + r.linkedTodos.length,
```

```
  },
],
title: r.name,
buttonRows: [
  [
    {
      text: "Öffnen",
      leftIcon: <FaEye />,
      disabled: false,
      variant: "brand",
      onClick: () => {
        router.push("/roadmap/" + r._id);
      },
    },
  ],
  [
    {
      text: "Löschen",
      leftIcon: <FaTrash />,
      disabled: false,
      colorScheme: "red",
      onClick: async () => {
        if (
          !(await PopupManager.confirmAsync({
            title: "Löschen",
            question:
              "Willst du die Roadmap wirklich löschen?",
          }))
        ) {
          return;
        }

        const res = await REST.Roadmap.deleteRoad({
          token: AccountManager.getActiveToken(),
          id: r._id,
        });

        if (res.status !== 200) {
          PopupManager.alert({
            title: "Fehler",
            description:
              "Es ist ein Fehler aufgetreten: " +
              res.payload.error,
          });
          return;
        }

        reload();
      },
    },
    {
      text: "Umbenennen",
      leftIcon: <FaPen />,
      variant: "brand",
      disabled: false,
      onClick: async () => {
        const name = await PopupManager.promptAsync({
          title: "Umbenennen",
          helperText:
            "Gib einen neuen Namen für die Roadmap ein",
```

```
                  inputType: "INPUT",
                });

                if (!name) return;

                const res = await REST.Roadmap.updateRoad({
                  token: AccountManager.getActiveToken(),
                  update: {
                    name,
                  },
                  id: r._id,
                });

                if (res.status !== 200) {
                  PopupManager.alert({
                    title: "Fehler",
                    description:
                      "Es ist ein Fehler aufgetreten: " +
                      res.payload.error,
                  });
                  return;
                }

                reload();
              },
            },
          ],
        ],
      };
    })}
  />
 </Box>
 </>
 )}
 </>
 );
}
```

## apps/web/src/pages/roadmap/[id].tsx

```
/**
 * apps/web/src/pages/roadmap/[id].tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 07.04.2023
 *
 */

import * as React from "react";
import { useLoggedIn } from "@codeup/hooks";
import { useRouter } from "next/router";
import REST from "@codeup/rest/dist";
import AccountManager from "@codeup/util/dist/AccountManager";
import AuthenticateLoadingIndicator from
"@codeup/components/dist/admin/AuthenticateLoadingIndicator";
import SuperText from "@codeup/components/landingpage/SuperText";
import {
  Box,
  chakra,
```

```
  Divider,
  Flex,
  FormControl,
  Heading,
  HStack,
  IconButton,
  Image,
  Input,
  InputGroup,
  InputLeftElement,
  Stack,
  StackDivider,
  Text,
  VStack,
} from "@chakra-ui/react";
import ReactFlow, {
  addEdge,
  applyEdgeChanges,
  applyNodeChanges,
  Background,
  Connection,
  Controls,
  Edge,
  EdgeChange,
  Handle,
  MarkerType,
  MiniMap,
  Node,
  NodeChange,
  Position,
  ReactFlowInstance,
} from "reactflow";
import { useCallback, useEffect, useMemo } from "react";
import CourseSelectorNode from "@codeup/components/roadmap/CourseSelectorNode";
import ProjectIdeaSelectorNode from "@codeup/components/roadmap/ProjectIdeaSelectorNode";
import ToDoSelectorNode from "@codeup/components/roadmap/ToDoSelectorNode";
import FlowSelectorNode from "@codeup/components/roadmap/FlowSelectorNode";
import EditorProjectSelectorNode from "@codeup/components/roadmap/EditorProjectSelectorNode";
import { CHeading } from "@codeup/components/dist/global/CHeading";

export default function RoadmapViewer() {
  const nodeTypes = useMemo(
    () => ({
      courseSelector: CourseSelectorNode,
      projectIdeaSelector: ProjectIdeaSelectorNode,
      todoSelector: ToDoSelectorNode,
      flowSelector: FlowSelectorNode,
      editorProjectSelector: EditorProjectSelectorNode,
    }),
    []
  );

  const { loggedIn, loaded, userInfo } = useLoggedIn();
  const [roadmap, setRoadmap] = React.useState<{
    owner: string;
    _id: string;
    name: string;
    projectIdea: string;
    linkedProject: string;
    linkedCourses: string[];
```

```
  linkedFlows: string[];
  linkedTodos: string[];
}>();
const router = useRouter();

React.useEffect(() => {
  if (loaded && !loggedIn) {
    router.push("/#login");
    window.dispatchEvent(new CustomEvent("codeup::open_login"));
  } else if (loaded && loggedIn) {
    reload();
  }
}, [loggedIn, loaded, userInfo]);

const reload = async () => {
  const res = await REST.Roadmap.getRoad({
    token: AccountManager.getActiveToken(),
    id: router.query.id as string,
  });

  if (res.status !== 200) {
    router.push("/roadmap");
    return;
  }

  setRoadmap(res.payload.roadmap);
};

const [nodes, setNodes] = React.useState<Node[]>([]);
const [edges, setEdges] = React.useState<Edge[]>([]);

const onNodesChange = useCallback(
  (changes: NodeChange[]) =>
    setNodes((nds) => applyNodeChanges(changes, nds)),
  [setNodes]
);
const onEdgesChange = useCallback(
  (changes: EdgeChange[]) =>
    setEdges((eds) => applyEdgeChanges(changes, eds)),
  [setEdges]
);
const onConnect = useCallback(
  (connection: Connection) => setEdges((eds) => addEdge(connection, eds)),
  [setEdges]
);

const [currentEdge, setCurrentEdge] = React.useState<Edge | null>(null);
const [currentNode, setCurrentNode] = React.useState<Node | null>(null);

const [lastEdgeId, setLastEdgeId] = React.useState<string>("");
const [lastNodeId, setLastNodeId] = React.useState<string>("");

const [rfInstance, setRfInstance] = React.useState<ReactFlowInstance | null>(
  null
);

useEffect(() => {
  setLastEdgeId("__rendered_" + currentEdge?.id ?? "");
}, [currentEdge]);
```

```
useEffect(() => {
  setLastNodeId("__rendered_" + currentNode?.id ?? "");
}, [currentNode]);

useEffect(() => {
  if (rfInstance) {
  }
}, [edges, nodes]);

useEffect(() => {
  if (rfInstance) {
    rfInstance.addNodes([
      {
        type: "projectIdeaSelector",
        id: "node-1",
        position: { x: 0, y: 0 },
        data: roadmap?.projectIdea,
        draggable: false,
        style: {
          backgroundColor: "black",
          borderRadius: "24px",
        },
        deletable: false,
      },
      {
        type: "courseSelector",
        id: "node-2",
        position: { x: 500, y: 0 },
        data: roadmap?.linkedCourses,
        draggable: false,
        style: {
          backgroundColor: "black",
          borderRadius: "24px",
        },
        deletable: false,
      },
      {
        type: "todoSelector",
        id: "node-3",
        position: { x: 1200, y: -300 },
        data: roadmap?.linkedTodos,
        draggable: false,
        style: {
          backgroundColor: "black",
          borderRadius: "24px",
        },
        deletable: false,
      },
      {
        type: "flowSelector",
        id: "node-4",
        position: { x: 1200, y: 300 },
        data: roadmap!.linkedFlows!,
        draggable: false,
        style: {
          backgroundColor: "black",
          borderRadius: "24px",
        },
        deletable: false,
      },
```

```
  {
    type: "editorProjectSelector",
    id: "node-5",
    position: { x: 1900, y: 0 },
    data: roadmap?.linkedProject,
    draggable: false,
    style: {
      backgroundColor: "black",
      borderRadius: "24px",
    },
    deletable: false,
  },
]);

rfInstance.addEdges([
  {
    id: "edge-1",
    source: "node-1",
    target: "node-2",
    markerEnd: {
      type: MarkerType.ArrowClosed,
      width: 50,
      height: 50,
    },
  },
  {
    id: "edge-2",
    source: "node-2",
    target: "node-3",
    markerEnd: {
      type: MarkerType.ArrowClosed,
      width: 50,
      height: 50,
    },
  },
  {
    id: "edge-3",
    source: "node-2",
    target: "node-4",
    markerEnd: {
      type: MarkerType.ArrowClosed,
      width: 50,
      height: 50,
    },
  },
  {
    id: "edge-4",
    source: "node-3",
    target: "node-5",
    markerEnd: {
      type: MarkerType.ArrowClosed,
      width: 50,
      height: 50,
    },
  },
  {
    id: "edge-5",
    source: "node-4",
    target: "node-5",
    markerEnd: {
```

```
          type: MarkerType.ArrowClosed,
          width: 50,
          height: 50,
        },
      },
    ]);
  }
}, [roadmap, rfInstance]);

return (
  <>
    {!loggedIn ? (
      <>
        <AuthenticateLoadingIndicator />
      </>
    ) : (
      <>
        {roadmap ? (
          <Box pt={4} bg={"#000"} minH={"100vh"} h={"fit-content"}>
            <Box textAlign={"center"}>
              <CHeading>Deine Roadmap: {roadmap.name}</CHeading>
            </Box>
            <Text fontSize={"xl"} textAlign={"center"} mb={6}>
              Von der Idee zum Projekt
            </Text>
            <Box h={"100vh"} w={"100%"}>
              <ReactFlow
                nodeTypes={nodeTypes}
                nodes={nodes}
                edges={edges}
                onConnect={onConnect}
                onNodesChange={onNodesChange}
                onEdgesChange={onEdgesChange}
                fitView
                fitViewOptions={{ padding: 0.2 }}
                snapToGrid={true}
                snapGrid={[10, 10]}
                onInit={setRfInstance}
                onEdgeClick={(e, edge) => {
                  setCurrentEdge(edge);

                  setCurrentNode(null);
                }}
                onNodeClick={(e, node) => {
                  setCurrentNode(node);
                  setCurrentEdge(null);
                }}
              >
                <Background />
                <Controls />
              </ReactFlow>
            </Box>
          </Box>
        ) : (
          <>
            <AuthenticateLoadingIndicator text={"Lade Roadmap..."} />
          </>
        )}
      </>
    )}
```

```
    </>
  );
}
```

## apps/web/src/pages/showcase.tsx

```
/**
 * apps/web/src/pages/showcase.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 09.04.2023
 *
 */

import * as React from "react";
import {
  Box,
  Button,
  ButtonGroup,
  Flex,
  Heading,
  Image,
  Text,
} from "@chakra-ui/react";
import { useLoggedIn } from "@codeup/hooks";
import { useRouter } from "next/router";
import { motion, useScroll } from "framer-motion";

export default function Showcase() {
  const { loggedIn } = useLoggedIn();
  const router = useRouter();
  const { scrollYProgress } = useScroll();

  return (
    <>
      <div
        className="position-absolute mx-auto bottom-0 z-n1 top-0 left-0 width-full events-none"
        style={{
          height: "100%",
          background:
            "linear-gradient(180deg, rgba(85, 48, 152, 0.3) 0%, rgba(83, 48, 152, 0.1) 100%)",
          bottom: 0,
          top: 0,
          left: 0,
          right: 0,
          position: "absolute",
          zIndex: -1,
        }}
      />
      <Box w={"100%"} minH={"100vh"} h={"fit-content"} p={[4, 6]} mt={16}>
        <Flex w={"100%"} justifyContent={"center"}>
          <Box w={["100%", "45%"]}>
            <MonoHeading>Dein Weg zum Programmierer</MonoHeading>
            <Flex justifyContent={"center"} w={"100%"}>
              <Text
                color={"gray.200"}
                textAlign={"center"}
                mt={4}
                fontSize={"2xl"}
```

```
    maxW={["100%", "50%"]}
  >
    CodeUp bietet alle Werkzeuge, die du brauchst, um dich in die
    Welt der Programmierung einzuarbeiten.
  </Text>
</Flex>
<Flex justifyContent={"center"} w={"100%"}>
  <Flex gap={4} direction={["column", "row"]} mt={16}>
    <Button
      bg={"brand.500"}
      color={"black"}
      _hover={{
        bg: "brand.500",
        color: "black",
      }}
      _focus={{
        bg: "brand.500",
        color: "black",
      }}
      size={"lg"}
      onClick={() => {
        if (loggedIn) {
          router.push("/dashboard");
        } else {
          window.dispatchEvent(
            new CustomEvent("codeup::open_login")
          );
        }
      }}
    >
      {loggedIn ? "Zum Dashboard" : "Jetzt loslegen"}
    </Button>
  </Flex>
</Flex>
<Box mt={"20vh"}></Box>
<Flex direction={["column", "row"]}>
  <Image
    src={"/_static/images/showcase/framed_course.png"}
    maxW={"750px"}
  />
  <Box>
    <MonoHeading textAlign={"left"}>Kurse</MonoHeading>
    <Text
      color={"gray.200"}
      textAlign={"left"}
      mt={4}
      fontSize={"2xl"}
      maxW={["100%"]}
    >
      Unsere Kurse sind für alle Altersgruppen geeignet und bieten
      dir viele verschiedene Themen. Jedes Thema ist in eine Lektion
      unterteilt, die du Schritt für Schritt durchlaufen kannst. Am
      Ende jeder Lektion wirst du mit einem Quiz abgefragt, um
      sicherzustellen, dass du das Gelernte verstanden hast.
    </Text>
    <Flex w={"100%"} justifyContent={"flex-end"}>
      <Button
        colorScheme={"messenger"}
        mt={4}
        size={"lg"}
```

```
                onClick={() => {
                  router.push("/course");
                }}
              >
                Kurse ansehen
              </Button>
            </Flex>
          </Box>
        </Flex>
        <Flex direction={"column"} mt={32}>
          <Box>
            <MonoHeading textAlign={"left"}>Zertifikate</MonoHeading>
            <Text
              color={"gray.200"}
              textAlign={"left"}
              mt={4}
              fontSize={"2xl"}
              maxW={["100%"]}
            >
              Zu jedem Kurs kannst du ein Zertifikat erhalten, wenn du alle
              Lektionen erfolgreich durchlaufen hast. Für diese Zertifikate
              musst du einen Test bestehen, damit wir sicherstellen können,
              dass du das Gelernte verstanden hast.
            </Text>
            <Flex w={"100%"} justifyContent={"flex-end"}>
              <Button
                colorScheme={"messenger"}
                mt={4}
                size={"lg"}
                onClick={() => {
                  router.push("/certificates");
                }}
              >
                Zertifikate ansehen
              </Button>
            </Flex>
          </Box>
          <Image
            src={"/_static/images/showcase/framed_certificates.png"}
            w={"100%"}
          />
        </Flex>
      </Box>
    </Flex>
  </Box>
 </>
 );
}

const MonoHeading = (props: {
 children: React.ReactNode;
 [key: string]: any;
}) => {
 return (
  <>
   <Heading
    textAlign={"center"}
    as="h1"
    fontSize={["4xl", "6xl"]}
    fontFamily={"Jetbrains Mono ExtraBold"}
```

```
        {...props}
      >
        {props.children}
      </Heading>
    </>
  );
};
```

## apps/web/src/pages/snippets/my.tsx

```
/**
 * apps/web/src/pages/snippets/my.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 30.12.2022
 *
 */

import * as React from "react";
import { useLoggedIn } from "@codeup/hooks";
import ErrorComponent from "@codeup/components/dist/global/ErrorComponent";
import {
  Box,
  Button,
  ButtonGroup,
  chakra,
  CircularProgress,
  Flex,
  FormControl,
  FormLabel,
  Grid,
  GridItem,
  Heading,
  HStack,
  Input,
  InputGroup,
  InputLeftElement,
  Stack,
  Tag,
  TagCloseButton,
  TagLabel,
  Text,
  useDisclosure,
  useToast,
} from "@chakra-ui/react";
import {
  FaClock,
  FaCode,
  FaCopy,
  FaFile,
  FaPen,
  FaPlus,
  FaSearch,
  FaTrash,
} from "react-icons/fa";
import REST from "@codeup/rest";
import Editor from "@monaco-editor/react";
import CodeUpModal from "@codeup/components/dist/codeup/CodeUpModal";
import GridLayout from "@codeup/components/global/GridLayout";
```

```
import AuthenticateLoadingIndicator from
"@codeup/components/dist/admin/AuthenticateLoadingIndicator";
import { CHeading } from "@codeup/components/dist/global/CHeading";
import GridLayoutV2 from "@codeup/components/global/GridLayoutV2";

export default function MySnippetViewer() {
  const { loggedIn, loaded, userInfo } = useLoggedIn();

  const [snippets, setSnippets] = React.useState<
    {
      _id: string;
      title: string;
      code: string;
      language: string;
      createdAt: string;
      username: string;
    }[]
  >([]);
  const [filteredSnippets, setFilteredSnippets] = React.useState<
    typeof snippets
  >([]);
  const [query, setQuery] = React.useState("");
  const [error, setError] = React.useState(false);
  const { isOpen, onOpen, onClose } = useDisclosure();

  React.useEffect(() => {
    if (loaded && loggedIn) {
      reload();
    }
  }, [loaded, loggedIn, userInfo]);

  React.useEffect(() => {
    let q = query;
    q = q.trim();
    setFilteredSnippets(
      snippets.filter(
        (s: any) =>
          s.title.toLowerCase().includes(q.toLowerCase()) ||
          s.code.toLowerCase().includes(q.toLowerCase()) ||
          s.language.toLowerCase().includes(q.toLowerCase())
      )
    );
  }, [query]);

  const reload = () => {
    REST.Snippets.getSnippets(localStorage.getItem("token") as string).then(
      (res) => {
        if (res.status === 200) {
          setSnippets(res.payload.snippets);
          let q = query;
          q = q.trim();
          setFilteredSnippets(
            res.payload.snippets.filter(
              (s: any) =>
                s.title.toLowerCase().includes(q.toLowerCase()) ||
                s.code.toLowerCase().includes(q.toLowerCase()) ||
                s.language.toLowerCase().includes(q.toLowerCase())
            )
          );
        } else {
```

```
        setError(true);
      }
    }
  );
};
const toast = useToast();

const [currentSnippet, setCurrentSnippet] = React.useState<{
  _id: string;
  title: string;
  code: string;
  language: string;
  createdAt: string;
  username: string;
} | null>(null);
return (
  <>
    {loggedIn && loaded ? (
      <>
        <CodeUpModal
          onOpen={onOpen}
          onClose={onClose}
          isOpen={isOpen}
          title={currentSnippet?.title || "Neues Snippet"}
          noCloseButton
        >
          <Stack spacing={4}>
            <FormControl>
              <FormLabel>Titel</FormLabel>
              <Input
                type={"text"}
                placeholder={"Titel des Snippets"}
                value={currentSnippet?.title}
                onChange={(e) => {
                  if (currentSnippet) {
                    setCurrentSnippet({
                      ...currentSnippet,
                      title: e.target.value as string,
                    });
                  }
                }}
              />
            </FormControl>
            <FormControl>
              <FormLabel>Sprache</FormLabel>
              <Input
                type={"text"}
                placeholder={"Sprache des Snippets"}
                value={currentSnippet?.language}
                onChange={(e) => {
                  if (currentSnippet) {
                    setCurrentSnippet({
                      ...currentSnippet,
                      language: e.target.value as string,
                    });
                  }
                }}
              />
            </FormControl>
            <FormControl data-color-mode={"dark"}>
```

```
  <FormLabel>Code</FormLabel>
  <Editor
    height={"50vh"}
    language={currentSnippet?.language || "javascript"}
    theme={"vs-dark"}
    value={currentSnippet?.code}
    onChange={(value) => {
      if (currentSnippet) {
        setCurrentSnippet({
          ...currentSnippet,
          code: value as string,
        });
      }
    }}
  />
</FormControl>
<ButtonGroup
  display={"flex"}
  justifyContent={"flex-end"}
  w={"100%"}
>
  <Button
    leftIcon={<FaTrash />}
    onClick={() => {
      REST.Snippets.deleteSnippet({
        token: localStorage.getItem("token") as string,
        id: currentSnippet?._id as string,
      }).then((res) => {
        if (res.status === 200) {
          toast({
            title: "Snippet gelöscht",
            description: "Das Snippet wurde erfolgreich gelöscht",
            status: "success",
            duration: 1500,
            isClosable: false,
            position: "bottom-right",
          });
          reload();
          onClose();
        } else {
          toast({
            title: "Fehler",
            description: res.payload.error,
            status: "error",
            duration: 1500,
            isClosable: false,
            position: "bottom-right",
          });
        }
      });
    }}
  >
    Löschen
  </Button>
  <Button
    colorScheme={"red"}
    onClick={() => {
      onClose();
    }}
  >
```

```
        Abbrechen
      </Button>
      <Button
        colorScheme={"green"}
        onClick={() => {
          REST.Snippets.updateSnippet({
            token: localStorage.getItem("token") as string,
            id: currentSnippet?._id as string,
            title: currentSnippet?.title as string,
            code: currentSnippet?.code as string,
            language: currentSnippet?.language as string,
          }).then((res) => {
            if (res.status === 200) {
              reload();
              onClose();
              setCurrentSnippet(null);
            } else {
              toast({
                title: "Fehler",
                description: res.payload.error,
                status: "error",
                duration: 1500,
                isClosable: false,
                position: "bottom-right",
              });
            }
          });
        }}
      >
        Speichern
      </Button>
    </ButtonGroup>
  </Stack>
</CodeUpModal>
<Box bg={"#000"} h={"fit-content"} minH={"100vh"}>
  <GridLayoutV2
    search={{
      show: true,
      fields: true,
    }}
    entries={snippets.map((snippet) => {
      return {
        title: snippet.title,
        fields: [
          {
            icon: <FaCode />,
            text: snippet.language,
          },
          {
            icon: <FaClock />,
            text: new Date(snippet.createdAt).toLocaleString(),
          },
          {
            icon: <FaFile />,
            text: snippet.code.substring(0, 50) + "...",
          },
        ],
        buttonRows: [
          [
            {
```

```
            text: "Bearbeiten",
            onClick: async () => {
              setCurrentSnippet(snippet);
              onOpen();
            },
            variant: "brand",
            leftIcon: <FaPen />,
          },
          {
            leftIcon: <FaCopy />,
            text: "Kopieren",
            colorScheme: "green",
            onClick: async () => {
              await navigator.clipboard.writeText(snippet.code);
              toast({
                title: "Code kopiert",
                description: "Der Code wurde erfolgreich kopiert",
                status: "success",
                duration: 1500,
                isClosable: false,
                position: "bottom-right",
              });
            },
          },
        ],
      ],
    };
  })}
  topButtons={[
    {
      variant: "brand",
      text: "Neu",
      leftIcon: <FaPlus />,
      onClick: () => {
        REST.Snippets.createSnippet({
          title: "Neues Snippet",
          code: 'console.log("Hello World")',
          language: "javascript",
          token: localStorage.getItem("token") as string,
        }).then((res) => {
          if (res.status === 200) {
            reload();
            onOpen();
            setCurrentSnippet(res.payload.snippet);
          } else {
            toast({
              title: "Fehler",
              description: res.payload.error,
              status: "error",
              duration: 1500,
              isClosable: false,
              position: "bottom-right",
            });
          }
        });
      },
    },
  ]}
  pageTitle={"CodeSnippets"}
/>
```

```
</Box>

<GridLayout
  search={{
    allow: true,
    searchFields: true,
  }}
  specialEntries={[
    {
      icon: <FaPlus />,
      onClick: () => {
        REST.Snippets.createSnippet({
          title: "Neues Snippet",
          code: 'console.log("Hello World")',
          language: "javascript",
          token: localStorage.getItem("token") as string,
        }).then((res) => {
          if (res.status === 200) {
            reload();
            onOpen();
            setCurrentSnippet(res.payload.snippet);
          } else {
            toast({
              title: "Fehler",
              description: res.payload.error,
              status: "error",
              duration: 1500,
              isClosable: false,
              position: "bottom-right",
            });
          }
        });
      },
    },
  ]}
  entries={snippets.map((snippet) => {
    return {
      title: snippet.title,
      fields: [
        {
          icon: <FaCode />,
          text: snippet.language,
        },
        {
          icon: <FaClock />,
          text: new Date(snippet.createdAt).toLocaleString(),
        },
        {
          icon: <FaFile />,
          text: snippet.code.substring(0, 50) + "...",
        },
      ],
      buttonRows: [
        {
          buttons: [
            {
              icon: <FaPen />,
              text: "Bearbeiten",
              colorScheme: "blue",
              onClick: async () => {
```

```
                    setCurrentSnippet(snippet);
                    onOpen();
                  },
                },
                {
                  icon: <FaCopy />,
                  text: "Kopieren",
                  colorScheme: "green",
                  onClick: async () => {
                    navigator.clipboard.writeText(snippet.code);
                    toast({
                      title: "Code kopiert",
                      description: "Der Code wurde erfolgreich kopiert",
                      status: "success",
                      duration: 1500,
                      isClosable: false,
                      position: "bottom-right",
                    });
                  },
                },
              ],
            },
          ],
        };
      })}
    />
  </>
) : (
  <>
    {loaded && !loggedIn ? (
      <>
        <ErrorComponent
          description={
            "Du musst eingeloggt sein, um diese Seite zu sehen."
          }
        />
      </>
    ) : (
      <>
        {error ? (
          <>
            <ErrorComponent
              description={
                "Es ist ein Fehler aufgetreten. Bitte versuche es später erneut."
              }
            />
          </>
        ) : (
          <>
            <AuthenticateLoadingIndicator />
          </>
        )}
      </>
    )}
  </>
);
}
```

## apps/web/src/pages/sso/callback.tsx

```
/**
 * apps/web/src/pages/sso/callback.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 22.09.2022
 *
 */

import * as React from "react";
import { useRouter } from "next/router";
import { Flex, Spinner } from "@chakra-ui/react";
import { useEffect } from "react";
import Head from "next/head";

export default function Callback() {
  const router = useRouter();

  useEffect(() => {
    if (window) {
      const urlSearchParams = new URLSearchParams(window.location.search);

      const params = Object.fromEntries(urlSearchParams.entries());

      if (params.token) {
        localStorage.setItem("token", params.token);
        router.push("/");
      } else {
        router.push("/error?error=Kein Token erhalten.");
      }
    }
  }, []);

  return (
    <>
      <Head>
        <title>Laden...</title>
      </Head>
      <Flex
        alignItems={"center"}
        justifyContent={"center"}
        w={"100%"}
        h={"100vh"}
        bg={"#000"}
      >
        <Spinner size={"xl"} />
      </Flex>
    </>
  );
}
```

## apps/web/src/pages/sso/error.tsx

```
/**
 * apps/web/src/pages/sso/error.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
```

```
 * Created At: 22.09.2022
 *
 */

import * as React from "react";
import {
  Alert,
  AlertIcon,
  AlertTitle,
  AlertDescription,
} from "@chakra-ui/react";
import { useRouter } from "next/router";
import Head from "next/head";

export default function Error() {
  const router = useRouter();

  return (
    <>
      <Head>
        <title>Login Fehlgeschlagen - CodeUp</title>
      </Head>
      <Alert status="error">
        <AlertIcon />
        <AlertTitle>Fehler beim Anmelden!</AlertTitle>
        <AlertDescription>{router.query.error}</AlertDescription>
      </Alert>
    </>
  );
}
```

## apps/web/src/pages/test-analytics.tsx

```
/**
 * apps/web/src/pages/test-analytics.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 25.11.2022
 *
 */

import * as React from "react";
import { useEffect } from "react";
import REST from "@codeup/rest";

export default function TestAnalytics() {
  return <></>;
}
```

## apps/web/src/pages/test-datatable.tsx

```
/**
 * apps/web/src/pages/test-datatable.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 08.10.22
 *
 */
```

```
import DataTable from "@codeup/components/extra/DataTable";
import { useEffect, useState } from "react";
import REST from "@codeup/rest";
import { Avatar, Box, Flex } from "@chakra-ui/react";

export default function TestDataTable() {
  const [users, setUsers] = useState([]);

  const reload = async () => {
    const x = await REST.Admin.listAccounts(
      localStorage.getItem("token") as string
    );

    if (x.status === 200) {
      setUsers(x.payload.accounts);
    } else {
      alert("Error: " + x.payload.error);
    }
  };

  useEffect(() => {
    reload();
  }, []);

  const userToRow = (user: any) => {
    return [
      () => {
        alert(user.username);
      },
      <Avatar size={"md"} src={"/api/account/" + user.username + "/picture"} />,
      user.username,
      user.email,
      user.admin ? "Ja" : "Nein",
      user.active ? "Ja" : "Nein",
      user.twoFactor ? "Ja" : "Nein",
      new Date(user.createdAt).toLocaleString(),
    ];
  };

  return (
    <>
      <Flex w={"100%"} justifyContent={"center"} mt={"2rem"}>
        <Box w={"75%"}>
          <DataTable
            columns={[
              "Profilbild",
              "Name",
              "E-Mail",
              "Admin",
              "Aktiv",
              "2FA",
              "Erstellt",
            ]}
            data={users.map(userToRow)}
            filter={(q: string, f: string, s: string) => {
              let x = users.filter((user: any) => {
                return (
                  user.username.toLowerCase().includes(q.toLowerCase()) ||
                  user.email.toLowerCase().includes(q.toLowerCase())
                );
```

```
      });

      switch (f.toLowerCase()) {
        case "alle":
          break;
        case "admins":
          x = x.filter((user: any) => user.admin);
          break;
        case "aktive":
          x = x.filter((user: any) => user.active);
          break;
        case "inaktive":
          x = x.filter((user: any) => !user.active);
          break;
        default:
          break;
      }

      switch (s.toLowerCase()) {
        case "alphabetisch":
          x = x.sort((a: any, b: any) => {
            if (a.username.toLowerCase() < b.username.toLowerCase()) {
              return -1;
            } else if (
              a.username.toLowerCase() > b.username.toLowerCase()
            ) {
              return 1;
            } else {
              return 0;
            }
          });
          break;
        case "neuste zuerst":
          x = x.sort((a: any, b: any) => {
            return (
              new Date(b.createdAt).getTime() -
              new Date(a.createdAt).getTime()
            );
          });
          break;
        case "älteste zuerst":
          x = x.sort((a: any, b: any) => {
            return (
              new Date(a.createdAt).getTime() -
              new Date(b.createdAt).getTime()
            );
          });
          break;
        default:
          break;
      }

      return x.map(userToRow);
    }}
    filterOptions={["Alle", "Admins", "Aktive", "Inaktive"]}
    sortOptions={["Alphabetisch", "Neuste zuerst", "Älteste zuerst"]}
    reload={reload}
   />
  </Box>
</Flex>
```

```
    </>
  );
}
```

## apps/web/src/pages/test-grid-layout.tsx

```
/**
 * apps/web/src/pages/test-grid-layout.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 24.03.2023
 *
 */

import * as React from "react";
import GridLayout from "@codeup/components/global/GridLayout";
import {
  FaClock,
  FaEye,
  FaFileImport,
  FaPen,
  FaPlus,
  FaTrash,
  FaUsers,
} from "react-icons/fa";

export default function Test() {
  return (
    <>
      <GridLayout
        search={{
          allow: true,
          searchFields: true,
        }}
        specialEntries={[
          {
            icon: <FaPlus />,
            onClick: () => {
              alert("PLUS");
            },
          },
          {
            icon: <FaFileImport />,
            onClick: () => {
              alert("IMPORT");
            },
          },
        ]}
        entries={[
          {
            title: "3000",
            fields: [
              {
                icon: <FaClock />,
                text: "19.3.2023, 17:24:38",
              },
            ],
            buttonRows: [
              {
```

```
              buttons: [
                {
                  text: "Löschen",
                  onClick: () => {
                    alert("DELETE");
                  },
                  colorScheme: "red",
                  icon: <FaTrash />,
                },
              ],
            },
            {
              buttons: [
                {
                  text: "Öffnen",
                  onClick: () => {
                    alert("OPEN");
                  },
                  colorScheme: "blue",
                  icon: <FaEye />,
                },
              ],
            },
            {
              buttons: [
                {
                  text: "Umbenennen",
                  onClick: () => {
                    alert("RENAME");
                  },
                  colorScheme: "green",
                  icon: <FaPen />,
                },
                {
                  text: "Mitwirkende",
                  onClick: () => {
                    alert("CONTRIBUTORS");
                  },
                  colorScheme: "yellow",
                  icon: <FaUsers />,
                },
              ],
            },
          ],
        },
      ]}
    />
  </>
 );
}
```

## apps/web/src/pages/test-gridlayout-v2.tsx

```
/**
 * apps/web/src/pages/test-gridlayout-v2.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2024 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 09.05.2024
 *
```

```
 */

import * as React from "react";

import GridLayoutV2 from "@codeup/components/global/GridLayoutV2";
import { FaInfo, FaPlus } from "react-icons/fa";
import { Box } from "@chakra-ui/react";

export default function TestGridlayoutV2() {
  return (
    <Box bg={"#000"} h={"fit-content"} minH={"100vh"}>
      <GridLayoutV2
        pageTitle={"Test GridLayoutV2"}
        search={{
          fields: true,
          show: true,
        }}
        entries={[
          {
            title: "Test4567",
            buttonRows: [
              [
                {
                  text: "Bearbeiten",
                  onClick: () => {
                    alert("Bearbeiten");
                  },
                  variant: "brand",
                },
              ],
            ],
            fields: [
              {
                text: "Test123",
                icon: <FaInfo />,
              },
            ],
          },

          {
            title: "Test789",
            buttonRows: [
              [
                {
                  text: "Bearbeiten",
                  onClick: () => {
                    alert("Bearbeiten");
                  },
                  variant: "brand",
                },
                {
                  text: "Bearbeiten",
                  onClick: () => {
                    alert("Bearbeiten");
                  },
                  variant: "brand",
                },
              ],
              [
                {
```

```
                text: "Bearbeiten",
                onClick: () => {
                  alert("Bearbeiten");
                },
                variant: "brand",
              },
              {
                text: "Bearbeiten",
                onClick: () => {
                  alert("Bearbeiten");
                },
                variant: "brand",
              },
            ],
          ],
          fields: [
            {
              text: "Test9080",
              icon: <FaInfo />,
            },
          ],
        },
      ]}
      topButtons={[
        {
          variant: "brand",
          text: "Neu",
          leftIcon: <FaPlus />,
          onClick: () => {
            alert("Neu");
          },
        },
      ]}
    />
  </Box>
 );
}
```

## apps/web/src/pages/test-img.tsx

```
/**
 * apps/web/src/pages/test-img.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 18.09.2022
 *
 */

import * as React from "react";
import { createAvatar } from "@dicebear/avatars";
import * as style from "@dicebear/avatars-bottts-sprites";
import { useEffect, useState } from "react";

export default function Component() {
  const [data, setData] = useState("");

  useEffect(() => x, []);

  const x = () => {
```

```
    const RANDOM_STRING =
      Math.random().toString(36).substring(2, 15) +
      Math.random().toString(36).substring(2, 15);

    const svg = createAvatar(style, {
      seed: RANDOM_STRING,
      colorful: true,
    });

    setData("" + Buffer.from(svg).toString("base64"));
  };

  return (
    <>
      <img src={data} width={256} height={256} onClick={x} />{" "}
    </>
  );
}
```

## apps/web/src/pages/test-player.tsx

```
/**
 * apps/web/src/pages/test-player.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 16.10.2022
 *
 */

import * as React from "react";
import { Box, Flex } from "@chakra-ui/react";
import VideoPlayer from "@codeup/components/global/VideoPlayer";

export default function TestPlayer() {

  return (
    <>
      <Flex w={"100%"} h={"25%"} alignItems={"center"} justifyContent={"center"}>
        <Box w={"25%"} h={"100%"}>
          <VideoPlayer url={"https://www.youtube.com/watch?v=tKlkDMoHmM8"} />
        </Box>
      </Flex>
    </>
  );
}
```

## apps/web/src/pages/todo/index.tsx

```
/**
 * apps/web/src/pages/todo/index.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 09.07.2022
 *
 */

import * as React from "react";
import { useEffect } from "react";
import redirect from "@codeup/util/src/userworkflow/redirect";
```

```
import { useRouter } from "next/router";
import {
  Box,
  Button,
  ButtonGroup,
  Drawer,
  DrawerBody,
  DrawerCloseButton,
  DrawerContent,
  DrawerFooter,
  DrawerHeader,
  DrawerOverlay,
  Flex,
  FormControl,
  FormLabel,
  Heading,
  IconButton,
  Input,
  Menu,
  MenuButton,
  MenuItem,
  MenuList,
  Popover,
  PopoverArrow,
  PopoverBody,
  PopoverCloseButton,
  PopoverContent,
  PopoverHeader,
  PopoverTrigger,
  SimpleGrid,
  Text,
  useDisclosure,
  useToast,
  VStack,
} from "@chakra-ui/react";
import { FaCheck, FaEye, FaPlus, FaTrash } from "react-icons/fa";
import { FiMoreHorizontal } from "react-icons/fi";
import REST from "@codeup/rest";
import ListViewer from "@codeup/components/todo/ListViewer";
import GridLayout from "@codeup/components/global/GridLayout";
import Head from "next/head";
import PopupManager from "../../global/PopupManager";
import AccountManager from "@codeup/util/dist/AccountManager";

export default function Index() {
  const router = useRouter();
  const toast = useToast();

  const [lists, setLists] = React.useState<
    {
      _id: string;
      name: string;
      tasks: {
        name: string;
        finished: boolean;
        description: string;
        uuid: string;
      }[];
    }[]
  >([]);
```

```
const { isOpen, onOpen, onClose } = useDisclosure();

useEffect(() => {
  redirect({
    allowAnonymous: false,
    redirectTo: router.pathname,
    redirectToLogin: true,
    forceAnonymous: false,
  });
}, [router]);

useEffect(() => {
  reloadLists();
}, []);

async function reloadLists() {
  const res = await REST.ToDo.getMyLists(
    localStorage.getItem("token") as string
  );

  if (res.status === 200) {
    setLists(res.payload.lists);

    if (currentList) {
      setCurrentList(
        res.payload.lists.find((e: any) => e._id === currentList._id)
      );
    }
  } else {
    toast({
      position: "top",
      variant: "solid",
      status: "error",
      title: "Fehler",
      description: res.payload.error,
      isClosable: true,
      duration: 6000,
    });
  }
}

const [currentList, setCurrentList] = React.useState<{
  _id: string;
  name: string;
  tasks: {
    name: string;
    finished: boolean;
    description: string;
    uuid: string;
  }[];
} | null>(null);

return (
  <>
    <Head>
      <title>ToDo - CodeUp</title>
    </Head>
    <Heading size={"xl"} textAlign={"center"} mt={"2rem"}>
      Aufgabenplanung - ToDo
```

```
</Heading>
<GridLayout
  search={{
    allow: true,
    searchFields: false,
  }}
  specialEntries={[
    {
      icon: <FaPlus />,
      onClick: async () => {
        const name = await PopupManager.promptAsync({
          title: "Neue Liste",
          helperText: "Gib den Namen der Liste ein",
        });

        if (!name) return;

        const res = await REST.ToDo.createList({
          token: localStorage.getItem("token") as string,
          name: name,
        });
        if (res.status === 200) {
          reloadLists();
          toast({
            position: "top",
            variant: "solid",
            status: "success",
            title: "Erfolgreich",
            description: "Die Liste wurde erstellt",
            isClosable: true,
            duration: 6000,
          });
        } else {
          toast({
            position: "top",
            variant: "solid",
            status: "error",
            title: "Fehler",
            description: res.payload.error,
            isClosable: true,
            duration: 6000,
          });
        }
      },
    },
  ]}
  entries={lists.map((list) => {
    return {
      title: list.name,
      fields: [
        {
          icon: <FaCheck />,
          text:
            "Erledigt: " +
            list.tasks.filter((task) => task.finished).length +
            "/" +
            list.tasks.length,
        },
      ],
      buttonRows: [
```

```
          {
            buttons: [
              {
                text: "Öffnen",
                icon: <FaEye />,
                colorScheme: "blue",
                onClick: () => {
                  setCurrentList(list);
                  onOpen();
                },
              },
            ],
          },
          {
            buttons: [
              {
                text: "Löschen",
                icon: <FaTrash />,
                colorScheme: "red",
                onClick: async () => {
                  if (
                    await PopupManager.confirmAsync({
                      title: "Liste löschen",
                      question: "Willst du die Liste wirklich löschen?",
                    })
                  ) {
                    const res = await REST.ToDo.deleteList({
                      token: AccountManager.getActiveToken(),
                      listId: list._id,
                    });

                    if (res.status === 200) {
                      reloadLists();
                      return;
                    }

                    PopupManager.alert({
                      title: "Fehler",
                      description:
                        "Die Liste konnte nicht gelöscht werden: " +
                        res.payload.error,
                    });
                  }
                },
              },
            ],
          },
        ],
      };
    })}
  />
  <ListViewer
    isOpen={isOpen}
    onClose={onClose}
    currentList={currentList}
    setCurrentList={setCurrentList}
    reloadLists={reloadLists}
  />
 </>
);
```

```
}
```

## apps/web/src/pages/try.tsx

```
/**
 * apps/web/src/pages/try.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 13.12.2022
 *
 */

import React, { useState } from "react";

// import @chakra-ui/react and @chakra-ui/card
import {
  Flex,
  Box,
  Heading,
  Text,
  ButtonGroup,
  Button,
  chakra,
} from "@chakra-ui/react";
import { Card, CardBody, CardFooter, CardHeader } from "@chakra-ui/card";
import REST from "@codeup/rest";
import { CHeading } from "@codeup/components/global/CHeading";
import PopupManager from "../global/PopupManager";

export default function Try() {
  const [loading, setLoading] = useState(false);

  return (
    <>
      <Flex
        direction="column"
        align="center"
        justify="center"
        w="100%"
        minH="100vh"
        p="2rem"
        bg={"#000"}
      >
        <Box w="100%" maxW="800px">
          <Card
            w={"100%"}
            h={"100%"}
            bgColor={"black"}
            rounded={"xl"}
            boxShadow={"xl"}
            p={5}
          >
            <CardHeader>
              <CHeading size="xl">CodeUp Testen</CHeading>
            </CardHeader>
            <CardBody mt={"2rem"}>
              <Text>
                Teste CodeUp ganz ohne deine Daten zu hinterlassen. Diese
                Funktion ist nicht für die langfristige Nutzung gedacht.
```

```
            </Text>
            <Text mt={2}>
              <chakra.span color={"red.400"} fontWeight={"bold"}>
                ACHTUNG!
              </chakra.span>
               Test-Konten werden regelmäßig gelöscht und können nicht
              wiederhergestellt werden.
            </Text>
          </CardBody>
          <CardFooter mt={"2rem"}>
            <ButtonGroup
              w={"100%"}
              spacing={4}
              justifyContent={"center"}
              onClick={() => {}}
            >
              <Button
                variant={"brand"}
                w={"100%"}
                onClick={async () => {
                  const r = await REST.Account.createTestAccount();

                  if (r.status === 200) {
                    localStorage.setItem("token", r.payload.token);
                    window.location.assign("/account/settings");
                  } else {
                    await PopupManager.alertAsync({
                      title: "Fehler",
                      description:
                        "Es ist ein Fehler aufgetreten: " + r.payload.error,
                    });
                  }
                }}
                isLoading={loading}
              >
                Testen
              </Button>
            </ButtonGroup>
          </CardFooter>
        </Card>
      </Box>
    </Flex>
  </>
 );
}
```

## apps/web/src/pages/user-cert/[id].tsx

```
/**
 * apps/web/src/pages/user-cert/[id].tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 27.11.2022
 *
 */

import * as React from "react";
import { useEffect } from "react";
import { useRouter } from "next/router";
```

```
import {
  Avatar,
  Box,
  chakra,
  CircularProgress,
  Divider,
  Flex,
  Heading,
  HStack,
  Image,
  Stack,
  Text,
} from "@chakra-ui/react";
import { Card, CardBody } from "@chakra-ui/card";
import REST from "@codeup/rest";
import ErrorComponent from "@codeup/components/global/ErrorComponent";
import Head from "next/head";
import { CHeading } from "@codeup/components/dist/global/CHeading";

export default function CertViewer() {
  const router = useRouter();

  const { id } = router.query;

  const [notFound, setNotFound] = React.useState(false);
  const [loading, setLoading] = React.useState(true);

  const [_rawResponse, _setRawResponse] = React.useState<any>(null);

  useEffect(() => {
    if (id) {
      REST.Certificates.getObtainedCertificate(id as string).then((res) => {
        if (res.status === 200) {
          setLoading(false);
          _setRawResponse(res.payload);
        } else {
          setNotFound(true);
          setLoading(false);
        }
      });
    }
  }, [id]);

  return (
    <>
      <Head>
        <title>Zertifikat - CodeUp</title>
      </Head>
      {loading ? (
        <>
          <Flex
            w={"100%"}
            h={"100vh"}
            justifyContent={"center"}
            alignItems={"center"}
            direction={"column"}
            bg={"#000"}
          >
            <CircularProgress
              size={"100px"}
```

```
        isIndeterminate
        color="brand.500"
      />
    </Flex>
  </>
) : (
  <>
    {notFound ? (
      <>
        <ErrorComponent
          description={"Das Zertifikat konnte nicht gefunden werden!"}
        />
      </>
    ) : (
      <>
        <Flex
          w={"100%"}
          minH={"100vh"}
          h={"fit-content"}
          justifyContent={"center"}
          alignItems={"center"}
          direction={"column"}
          bg={"#000"}
        >
          <Card
            w={"100%"}
            maxW={"800px"}
            bgColor={"black"}
            boxShadow={"2xl"}
            mt={"4rem"}
            mb={"4rem"}
            p={"2rem"}
            id={"certificate"}
            rounded={"lg"}
          >
            <CardBody>
              <Image
                src={_rawResponse?.course?.splashImage}
                borderRadius={"lg"}
                alt={_rawResponse?.course?.name}
              />
              <Stack mt={6} spacing={3}>
                <Box textAlign={"center"}>
                  <CHeading size={"2xl"}>
                    {_rawResponse?.course?.name}
                  </CHeading>
                </Box>
                <Divider />
                <HStack pt={"2rem"} pb={"2rem"}>
                  <Flex
                    w={"100%"}
                    alignItems={"center"}
                    justifyContent={"center"}
                    direction={"column"}
                  >
                    <Avatar
                      size={"xl"}
                      src={
                        "/api/account/" +
                        _rawResponse?.obtainedCertificate?.username +
```

```
        "/picture"
      }
      background={"transparent"}
    />
    <Heading
      mt={2}
      fontSize={["xl", "2xl"]}
      textAlign={"center"}
      fontWeight={"bold"}
    >
      {_rawResponse?.obtainedCertificate?.name} <br />
      <Text
        fontSize={["md", "xl"]}
        textAlign={"center"}
        fontWeight={"normal"}
      >
        @{_rawResponse?.obtainedCertificate?.username}
      </Text>
    </Heading>
  </Flex>
  <Flex
    w={"100%"}
    alignItems={"center"}
    justifyContent={"center"}
    direction={"column"}
  >
    <CircularProgress
      size={"100px"}
      color={"brand.500"}
      value={
        _rawResponse?.obtainedCertificate?.percentage
      }
    >
      <Text
        fontSize={["xl", "2xl"]}
        fontWeight={"bold"}
        textAlign={"center"}
      >
        {_rawResponse?.obtainedCertificate?.percentage}%
      </Text>
    </CircularProgress>
  </Flex>
</HStack>
<Divider />
<Stack pt={"2rem"}>
  <CHeading size={"lg"}>Zertifikat</CHeading>
  <Text fontSize={"3xl"} color={"gray.400"}>
    <chakra.span color={"brand.500"}>
      {_rawResponse?.obtainedCertificate?.name}
    </chakra.span>
     (@
    {_rawResponse?.obtainedCertificate?.username}) hat den
    Kurs &quot;
    <chakra.span color={"brand.500"}>
      {_rawResponse?.course?.name}
    </chakra.span>
    &quot; am 
    {new Date(
      _rawResponse?.obtainedCertificate?.date
    ).toLocaleString()}
```

```
                   mit 
                  <chakra.span color={"brand.500"}>
                    {_rawResponse?.obtainedCertificate?.percentage}
                  </chakra.span>
                  % erfolgreich bestanden.
                </Text>
                <Divider />
                <Text fontSize={"xl"} color={"gray.400"}>
                  Dieses Zertifikat wurde von @
                  <chakra.span color={"brand.500"}>
                    {_rawResponse?.course?.creator}
                  </chakra.span>
                   im Namen von 
                  <chakra.span color={"brand.500"}>CodeUp</chakra.span>
                   ausgestellt.
                </Text>
              </Stack>
            </Stack>
            <Flex
              mt={16}
              gap={"1.2rem"}
              direction={"row"}
              alignItems={"center"}
              justifyContent={"center"}
              w={"100%"}
            >
              <img src={"/codeup.svg"} width={64} height={64} />
              <Heading size={"lg"}>CodeUp</Heading>
            </Flex>
          </CardBody>
        </Card>
      </Flex>
    </>
  )}
 </>
 );
}
```

## apps/web/src/pages/_app.tsx

```
import "../styles/globals.scss";
import "../styles/tiptap.css";
import "reactflow/dist/style.css";
import type { AppProps } from "next/app";
import { Box, ChakraProvider, extendTheme } from "@chakra-ui/react";
import React, { useEffect } from "react";
import { DefaultSeo } from "next-seo";
import { useRouter } from "next/router";
import { CommunityNavigationBar } from "@codeup/components/community";
import { PostButton } from "@codeup/components/community/post";
import { Footer, NavigationBar } from "@codeup/components/global";
import ForumNavigationBar from "@codeup/components/forum/ForumNavigationBar";
import CommandPalette from "@codeup/components/global/CommandPalette";
import { StepsStyleConfig as Steps } from "chakra-ui-steps";
import { GoogleAnalytics } from "nextjs-google-analytics";
import REST from "@codeup/rest";
import useAnalytics from "@codeup/hooks/hooks/useAnalytics";
import theme from "../global/theme";
import posthog from "posthog-js";
```

```
import { PostHogProvider } from "posthog-js/react";
import { useLoggedIn } from "@codeup/hooks";

// Check that PostHog is client-side (used to handle Next.js SSR)
if (typeof window !== "undefined") {
  // @ts-ignore
  posthog.init(process.env.NEXT_PUBLIC_POSTHOG_KEY, {
    // @ts-ignore
    api_host: process.env.NEXT_PUBLIC_POSTHOG_HOST || "https://app.posthog.com",
    // Enable debug mode in development
    loaded: (posthog) => {
      if (process.env.NODE_ENV === "development") posthog.debug();
    },
    autocapture: true,
  });
}

function App({ Component, pageProps }: AppProps) {
  const router = useRouter();
  const { loggedIn, userInfo, loaded } = useLoggedIn();

  useEffect(() => {
    if (loaded && loggedIn) {
      posthog?.identify(userInfo.email, {
        name: userInfo.firstName + " " + userInfo.lastName,
        email: userInfo.email,
        username: userInfo.username,
      });
    }
  }, [loaded, loggedIn]);

  useAnalytics();

  useEffect(() => {
    // Track page views
    const handleRouteChange = () => posthog?.capture("$pageview");
    router.events.on("routeChangeComplete", handleRouteChange);

    return () => {
      router.events.off("routeChangeComplete", handleRouteChange);
    };
  }, []);

  return (
    <>
      <DefaultSeo
        description={
          "CodeUp ist eine Platform, um die Grundlagen der Programmierung mit interaktiven
Videokursen zu erlernen."
        }
        additionalMetaTags={[
          {
            name: "keywords",
            content: "seo, keywords",
          },
        ]}
        openGraph={{
          type: "website",
          locale: "de_DE",
          url: "https://codeup.space/",
```

```
          site_name: "CodeUp",
          title: "CodeUp - Programmieren lernen",
          description:
            "CodeUp ist eine Platform, um die Grundlagen der Programmierung mit interaktiven
Videokursen zu erlernen.",
          images: [
            {
              url: "https://codeup.space/codeup-splash-github.jpg",
              alt: "Splash Image of CodeUp",
              type: "image/jpg",
              width: 1200,
              height: 630,
            },
          ],
        }}
        twitter={{
          handle: "@OfficialMCTzOCK",
          cardType: "summary_large_image",
        }}
      />
      <PostHogProvider client={posthog}>
        <ChakraProvider theme={theme}>
          <CommandPalette />
          <Box minH="100vh" minW="100%">
            <div id={"__codeup_other_component_render_space_do_not_use"}></div>
            <NavigationBar
              disablePricing={
                process.env.NEXT_PUBLIC_DISABLE_PAYMENTS === "true"
              }
            />
            <Box
              minH={"96vh"}
              bg={router.pathname.startsWith("/forum") ? "#000" : undefined}
            >
              {router.pathname.startsWith("/community") ? (
                <CommunityNavigationBar />
              ) : null}
              {router.pathname.startsWith("/forum") ? (
                <ForumNavigationBar />
              ) : null}

              <Box id={"__codeup_manual_mount_point"}></Box>

              <Component {...pageProps} />

              {router.pathname.startsWith("/community") ? <PostButton /> : null}
            </Box>
            <Footer />
          </Box>
        </ChakraProvider>
      </PostHogProvider>
    </>
  );
}

export default App;
```

## apps/web/src/pages/_document.tsx

```
/**
 * src/pages/_document.tsx
```

```
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 06.06.2022
 *
 */

import * as React from "react";
import { Head, Html, Main, NextScript } from "next/document";
import { ColorModeScript } from "@chakra-ui/react";
import Script from "next/script";

export default function Document() {
  return (
    <Html lang={"en"}>
      <Head />
      <body>
        <ColorModeScript initialColorMode={"dark"} />
        <Main />
        <NextScript />
      </body>
    </Html>
  );
}
```

## apps/web/src/server/index.ts

```
/**
 * apps/web/src/server/index.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 05.12.2022
 *
 */

import { createServer } from "http";
import { parse } from "url";
import * as next from "next";
import fs from "fs";
import path from "path";
import IDEProjectModel from "@codeup/util/dist/models/IDEProjectModel";
import EditorProjectPageModel from "@codeup/util/dist/models/EditorProjectPageModel";
import mongoose from "mongoose";
import * as ts from "typescript";
import * as sass from "sass";
import * as InCode from "@codeup/incode-compiler/dist";
import { lookup as mimeLookup } from "mime-types";
import formidable from "formidable";
import { randomBytes } from "crypto";
import { createSocketServer } from "./socket";
import EditorProjectModel from "@codeup/util/dist/models/EditorProjectModel";
import EditorProjectFunctionModel from "@codeup/util/dist/models/EditorProjectFunctionModel";
import Docker from "dockerode";
import jwt from "jsonwebtoken";
import EditorProjectKVStorageModel from "@codeup/util/dist/models/EditorProjectKVStorageModel";
import ShortURLModel from "@codeup/util/dist/models/ShortURLModel";
import ErrorModel from "@codeup/util/src/models/ErrorModel";
import { PostHog } from "posthog-node";
```

```
import Stripe from "stripe";
import { isLoggedIn } from "../global/server/isLoggedIn";
import { getS3 } from "../global/getS3";
import ProjectModel from "@codeup/util/dist/models/ProjectModel";
import UserModel from "@codeup/util/dist/models/UserModel";

const app = next.default({
  dev: process.env.NODE_ENV === "development",
});
const handle = app.getRequestHandler();

if (process.env.DISABLE_PAYMENTS === "true") {
  // @ts-ignore
  process.env.DISABLE_PAYMENTS = true;
}
app
  .prepare()
  .then(async () => {
    const port = parseInt(process.env.PORT || "3000", 10);
    if (mongoose.connection.readyState === 0) {
      try {
        await mongoose.connect(process.env.MONGO_URI as string);
      } catch (e) {
        console.error("Could not connect to MongoDB");
        console.error(e);
        process.exit(1);
      }
    }

    const mediaBucket = new mongoose.mongo.GridFSBucket(
      mongoose.connection.db,
      {
        bucketName: "media",
      }
    );

    process.on("uncaughtException", async (e) => {
      await handleError(e);
    });

    const docker = new Docker();

    docker
      .pull("mctzock/codeup-serverless-runtime")
      .then((stream) => {
        docker.modem.followProgress(stream, (err: any, res: any) => {
          console.log(err, res);
        });
      })
      .catch(async (e) => {
        await handleError(e);
      });

    const stripe = new Stripe(process.env.STRIPE_SECRET_KEY as string, {
      apiVersion: "2022-11-15",
    });

    const httpServer = createServer(async (req, res) => {
      try {
        const parsedUrl = parse(req.url!, true);
```

```
res.setHeader("Access-Control-Allow-Origin", "*");
res.setHeader("X-Served-By", "CodeUp");
res.setHeader("Cross-Origin-Opener-Policy", "same-origin");
res.setHeader("Cross-Origin-Embedder-Policy", "require-corp");

const { pathname } = parsedUrl;

const customDomain = req.headers.host!.includes(":")
  ? req.headers.host!.split(":")[0]
  : req.headers.host;

if (customDomain!.split(".").length > 2) {
  const project = await EditorProjectModel.findOne({
    customDomain,
  });

  if (!project) {
    await handle(req, res);
    return;
  }
  if (pathname!.startsWith("/api")) {
    // check if pathname is /api/@codeup/internal/info

    if (pathname!.startsWith("/api/@codeup")) {
      const pathParts = pathname!.split("/").slice(3);
      if (pathParts.length === 2) {
        if (pathParts[0] === "database") {
          if (pathParts[1] === "set") {
            const key = req.headers["x-codeup-database-key"] as string;
            const token = req.headers["x-sf-access"] as string;
            const value = JSON.parse(
              await new Promise((resolve, reject) => {
                let body = "";
                req.on("data", (chunk) => {
                  body += chunk;
                });
                req.on("end", () => {
                  resolve(body);
                });
                req.on("error", (e) => {
                  reject(e);
                });
              })
            ).value as string;

            if (!key || !token || !value) {
              res.statusCode = 400;
              res.write(
                JSON.stringify({
                  key: JSON.stringify(key),
                  token: JSON.stringify(token),
                  value: JSON.stringify(value),
                })
              );
              res.end();
              return;
            }

            try {
```

```
    const decoded = jwt.verify(
      token,
      process.env.JWT_SECRET as string
    ) as { project: string; function: string };

    if (decoded.project !== project._id.toString()) {
      res.statusCode = 403;
      res.end();
      return;
    }

    const existingKVStore =
      await EditorProjectKVStorageModel.findOne({
        project: project._id,
      });

    console.log(
      "[SFDB] Set",
      key,
      "to",
      value,
      "for",
      project._id.toString()
    );

    if (existingKVStore) {
      existingKVStore.storage = {
        ...existingKVStore.storage,
        [key]: value,
      };
      await existingKVStore.save();
    } else {
      await EditorProjectKVStorageModel.create({
        project: project._id,
        storage: {
          [key]: value,
        },
      });
    }

    res.statusCode = 200;
    res.end();
  } catch (e) {
    res.statusCode = 403;
    res.end();
    return;
  }
} else if (pathParts[1] === "get") {
  const key = req.headers["x-codeup-database-key"] as string;
  const token = req.headers["x-sf-access"] as string;

  if (!key || !token) {
    res.statusCode = 400;
    res.end();
    return;
  }

  try {
    const decoded = jwt.verify(
      token,
```

```
            process.env.JWT_SECRET as string
          ) as { project: string; function: string };

          if (decoded.project !== project._id.toString()) {
            res.statusCode = 403;
            res.end();
            return;
          }

          const existingKVStore =
            await EditorProjectKVStorageModel.findOne({
              project: project._id,
            });

          if (existingKVStore) {
            res.statusCode = 200;
            res.end(existingKVStore.storage[key]);
          } else {
            res.statusCode = 404;
            res.end();
          }
        } catch (e) {
          res.statusCode = 403;
          res.end();
          return;
        }
      }
    }
  }

  return;
} else {
  const serverlessFunction =
    await EditorProjectFunctionModel.findOne({
      project: project._id,
      path: pathname!,
    });

  if (!serverlessFunction) {
    res.statusCode = 404;
    res.end("Not Found");
    return;
  }

  const code = serverlessFunction.code;

  let port = Math.floor(Math.random() * 32000) + 1;

  docker
    .createContainer({
      Image: "rce-sf-runtime:codeup",
      Env: [
        "SERVERLESS_METHOD=GET",
        "SERVERLESS_ENDPOINT=/",
        "SERVERLESS_CODE=" + code,
        "SERVERLESS_TOKEN=" +
          jwt.sign(
            {
              project: project._id,
              function: serverlessFunction._id,
```

```
        },
        process.env.JWT_SECRET as string,
        {
          expiresIn: 600,
        }
      ),
    "SERVERLESS_ENDPOINT_URL=http://" + customDomain,
  ],
  HostConfig: {
    PortBindings: {
      "3000/tcp": [
        {
          HostPort: port.toString(),
        },
      ],
    },
  },
  Tty: true,
  OpenStdin: true,
  StdinOnce: true,
  AttachStdin: true,
  AttachStdout: true,
  AttachStderr: true,
  ExposedPorts: {
    "3000/tcp": {},
  },
})
.then(async (c) => {
  await c.start();

  await new Promise((resolve, reject) => {
    setTimeout(() => {
      resolve(true);
    }, 1000);
  });

  const query = Object.keys(parsedUrl.query).map((key) => {
    return key + "=" + parsedUrl.query[key];
  });

  try {
    const res0 = await fetch(
      `http://localhost:${port}?${
        query.length > 0 ? query.join("&") : ""
      }`
    );

    for (const [key, value] of res0.headers.entries()) {
      res.setHeader(key, value);
    }

    res.writeHead(res0.status, {
      ...res.getHeaders(),
    });

    res.end(await res0.text());

    await new Promise((resolve, reject) => {
      setTimeout(() => {
        resolve(true);
```

```
          }, 1000);
        });

        try {
          await c.kill();
        } catch (e) {}
        await c.remove();
      } catch (e) {
        res.statusCode = 500;
        res.end(
          "[SF] Serverless function failed to execute properly. Error details: " +
            (e as any).toString()
        );

        try {
          await c.kill();
        } catch (e) {}
        await c.remove();
      }
    })
    .catch((e) => {
      console.error(e);
      res.statusCode = 500;
      res.setHeader("Content-Type", "application/json");
      res.end(JSON.stringify(e, null, 2));
    });

  return;
 }
}

const page = await EditorProjectPageModel.findOne({
 project: project._id,
 url: pathname,
});

if (!page) {
 await handle(req, res);
 return;
}

res.setHeader("Content-Type", "text/html");

let resourceEmbedCode = "";

for (const resource of page.resources) {
 switch (resource.type.toLowerCase()) {
   case "css":
     resourceEmbedCode += `<style>${resource.content}</style>`;
     break;
   case "scss":
     resourceEmbedCode += `<style>${
       sass.compileString(resource.content, {
         syntax: "scss",
       }).css
     }</style>`;
     break;
   case "sass":
     resourceEmbedCode += `<style>${
       sass.compileString(resource.content, {
```

```
            syntax: "indented",
          }).css
        }</style>`;
        break;
      case "javascript":
        resourceEmbedCode += `<script defer>${resource.content}</script>`;
        break;
      case "typescript":
        resourceEmbedCode += `<script defer>${
          ts.transpileModule(resource.content, {
            compilerOptions: {
              module: ts.ModuleKind.ESNext,
            },
          }).outputText
        }</script>`;
        break;
      default:
        break;
    }
  }

  if (page.type.toLowerCase() === "html") {
    res.statusCode = 200;
    res.write((page.mainContent += resourceEmbedCode));
    res.end();
  } else if (page.type.toLowerCase() === "incode") {
    res.statusCode = 200;
    res.write(
      `<html><head><title>${
        page.url
      }</title></head><body><script defer>${InCode.Compiler.compile(
        page.mainContent
      )}</script></body></html>${resourceEmbedCode}`
    );
    res.end();
  }
} else {
  if (pathname === "/_internal/api/app/stats") {
    if (process.env.NODE_ENV === "production") {
      if (
        fs.existsSync(
          path.join(__dirname, "..", "..", ".codeup_app_stats.json")
        )
      ) {
        res.setHeader("Content-Type", "application/json");
        res.end(
          fs
            .readFileSync(
              path.join(__dirname, "..", "..", ".codeup_app_stats.json")
            )
            .toString()
        );
      } else {
        res.statusCode = 404;
        res.setHeader("Content-Type", "application/json");
        res.end(
          JSON.stringify({
            status: "failed",
            message: "This endpoint is currently not available.",
          })
```

```
      );
    }
  } else {
    res.setHeader("Content-Type", "application/json");
    res.end(
      JSON.stringify({
        status: "failed",
        message:
          "This endpoint is only available in production mode.",
      })
    );
  }

  return;
}

const shortUrl = await ShortURLModel.findOne({
  short: pathname!.startsWith("/") ? pathname!.slice(1) : pathname,
});

if (shortUrl) {
  res.writeHead(302, {
    Location: shortUrl.destination,
  });

  res.end();

  return;
}

if (pathname?.startsWith("/project")) {
  // URL Structure : /project/:customName/:path

  const customName = pathname.split("/")[2];
  let path = pathname.split("/").slice(3).join("/");

  try {
    const project = await IDEProjectModel.findOne({
      publishedCustomName: customName,
    });

    if (!project) {
      throw new Error();
      return;
    }

    if (!project.published) {
      throw new Error();
      return;
    }

    let file = project.files.find(
      (f: { path: string }) =>
        f.path.toLowerCase() === `/${path.toLowerCase()}`
    );

    if (!file) {
      // redirect to index.html
      res.writeHead(302, {
        Location: `/project/${customName}/index.html`,
```

```
  });

  res.end();

  return;
}

let ret = file.content;

if (file.path.toLowerCase().endsWith(".ts")) {
  ret = ts.transpileModule(ret, {
    compilerOptions: {
      module: ts.ModuleKind.ESNext,
    },
  }).outputText;
}

if (
  file.path.toLowerCase().endsWith(".scss") ||
  file.path.toLowerCase().endsWith(".sass")
) {
  ret = sass.compileString(file.content, {
    syntax: file.path.toLowerCase().endsWith(".scss")
      ? "scss"
      : "indented",
  }).css;
}

if (file.path.toLowerCase().endsWith(".ic")) {
  ret = `<html><head><title>${
    file.path
  }</title></head><body><script defer>${InCode.Compiler.compile(
    ret
  )}</script></body></html>`;
}

const mimeTypes: { [key: string]: string } = {
  js: "application/javascript",
  css: "text/css",
  html: "text/html",
  json: "application/json",
  png: "image/png",
  jpg: "image/jpeg",
  jpeg: "image/jpeg",
  gif: "image/gif",
  svg: "image/svg+xml",
  ic: "text/html",
};
const replaceTypes: { [key: string]: string } = {
  "text/x-scss": "text/css",
  "text/x-sass": "text/css",
  "text/x-typescript": "application/javascript",
};

let mimeType = "";

if (mimeTypes[file.path.split(".").pop() as string]) {
  mimeType = mimeTypes[file.path.split(".").pop() as string];
} else {
  mimeType = mimeLookup(file.path) as string;
```

```
    }

    if (!mimeType || !Object.values(mimeTypes).includes(mimeType)) {
      if (mimeType && Object.keys(replaceTypes).includes(mimeType)) {
        mimeType = replaceTypes[mimeType];
      } else {
        mimeType = "text/plain";
      }
    }

    res.setHeader("Content-Type", mimeType);

    res.end(ret);

    return;
  } catch (e) {
    console.log(e);
    await app.render404(req, res, parsedUrl);
    return;
  }
}

if (pathname?.startsWith("/media")) {
  if (pathname?.includes("/upload")) {
    if (req.method === "POST") {
      const form = new formidable.IncomingForm();

      await new Promise((resolve, reject) => {
        form.parse(req, async (err, fields, files) => {
          if (err) {
            await app.render404(req, res, parsedUrl);
            console.log(err);
            return;
          }

          const file = files.file as formidable.File;

          if (!file) {
            await app.render404(req, res, parsedUrl);
            return;
          }

          const content = fs.readFileSync(file.filepath);
          const mimeType = file.mimetype;
          const fileName = file.originalFilename;

          const uploadStream = mediaBucket.openUploadStream(
            randomBytes(128).toString("hex") +
              "." +
              (fileName as string).split(".").pop(),
            {
              contentType: mimeType as string,
              metadata: {
                contentType: mimeType as string,
                fileName: fileName as string,
              },
            }
          );

          uploadStream.write(content);
```

```
        uploadStream.end();

        await new Promise((resolve0) => {
          uploadStream.on("finish", () => {
            resolve0(true);
          });
        });

        res.end(
          JSON.stringify({
            status: "success",
            message: "File uploaded successfully.",
            data: {
              url: `/media/${uploadStream.filename}`,
            },
          })
        );

        resolve(true);
      });
    });
    return;
  } else {
    await app.render404(req, res, parsedUrl);
    return;
  }
} else {
  // URL Structure : /media/:filename

  const filename =
    pathname.split("/")[pathname.split("/").length - 1];

  const file = await mediaBucket.openDownloadStreamByName(filename);

  if (!file) {
    await app.render404(req, res, parsedUrl);
    return;
  }

  const arr = await mediaBucket.find({ filename }).toArray();

  const gfsFile = arr[0] as typeof arr[0] & {
    metadata: { contentType: string };
  };

  res.setHeader("Content-Type", gfsFile.metadata.contentType);

  await new Promise((resolve) => {
    file.on("data", (chunk) => {
      res.write(chunk);
    });
    file.on("end", () => {
      res.end();
      resolve(true);
    });
  });

  return;
}
```

```
}

if (pathname?.startsWith("/editor-static")) {
  // Option 1: /editor-static/page-res/<page_id>/<res_name>
  // Option 2: /editor-static/<project_id>/<res_id>

  const args = pathname
    ?.split("/")
    .splice(2, pathname?.split("/").length);

  if (args.length === 3) {
    const pageId = args[1];
    const resName = args[2];

    const page = await EditorProjectPageModel.findById(pageId);

    if (!page) {
      await app.render404(req, res);
      return;
    }

    const resource = page.resources.find(
      (r: any) => r.name === resName
    );
    if (!resource) {
      await app.render404(req, res);
      return;
    }

    let content = resource.content;

    switch (resource.type.toLowerCase()) {
      case "css":
        res.setHeader("Content-Type", "text/css");
        break;
      case "scss":
        content = sass.compileString(content, {
          syntax: "scss",
        }).css;
        res.setHeader("Content-Type", "text/css");
        break;
      case "sass":
        content = sass.compileString(content, {
          syntax: "indented",
        }).css;
        res.setHeader("Content-Type", "text/css");
        break;
      case "javascript":
        res.setHeader("Content-Type", "text/javascript");
        break;
      case "typescript":
        content = ts.transpileModule(content, {
          compilerOptions: {
            module: ts.ModuleKind.ESNext,
          },
        }).outputText;
        res.setHeader("Content-Type", "text/javascript");
        break;
      case "json":
        res.setHeader("Content-Type", "application/json");
```

```
        break;
      default:
        res.setHeader("Content-Type", "text/plain");
        break;
    }

    res.write(content.toString());
    res.end();
  } else {
    const projectId = args[0];
    const resName = args[1];

    const bucket = new mongoose.mongo.GridFSBucket(
      mongoose.connection.db,
      {
        bucketName: "editorStaticAssets",
      }
    );

    const file = await bucket.find({
      filename: `${projectId}/${resName}`,
    });

    const fileEntry = await file.next();

    if (file) {
      const stream = bucket.openDownloadStream(fileEntry!._id);

      await new Promise((resolve) => {
        stream.on("data", async (chunk) => {
          res.write(chunk);
        });
        stream.on("end", () => {
          res.end();
          resolve(true);
        });
      });
    } else {
      await app.render404(req, res);
      return;
    }
  }
}

if (pathname?.startsWith("/editor-preview")) {
  // /editor-preview/<page_id>

  const args = pathname
    ?.split("/")
    .splice(2, pathname?.split("/").length);

  const pageId = args[0];

  const page = await EditorProjectPageModel.findById(pageId);

  if (!page) {
    await app.render404(req, res);
    return;
  }
```

```
      let resourceEmbedCode = ``;

      for (const resource of page.resources) {
        switch (resource.type.toLowerCase()) {
          case "css":
            resourceEmbedCode += `<style>${resource.content}</style>`;
            break;
          case "scss":
            resourceEmbedCode += `<style>${
              sass.compileString(resource.content, {
                syntax: "scss",
              }).css
            }</style>`;
            break;
          case "sass":
            resourceEmbedCode += `<style>${
              sass.compileString(resource.content, {
                syntax: "indented",
              }).css
            }</style>`;
            break;
          case "javascript":
            resourceEmbedCode += `<script defer>${resource.content}</script>`;
            break;
          case "typescript":
            resourceEmbedCode += `<script defer>${
              ts.transpileModule(resource.content, {
                compilerOptions: {
                  module: ts.ModuleKind.ESNext,
                },
              }).outputText
            }</script>`;
            break;
          default:
            break;
        }
      }

      if (page.type.toLowerCase() === "html") {
        res.statusCode = 200;
        res.write(page.mainContent);
        res.write(resourceEmbedCode);
        res.end();
      } else if (page.type.toLowerCase() === "incode") {
        res.statusCode = 200;
        res.write(
          `<html><head><title>${
            page.url
          }</title>${resourceEmbedCode}</head><body><script defer>${InCode.Compiler.compile(
            page.mainContent
          )}</script></body></html>`
        );
        res.end();
      }
    }

    await handle(req, res, parsedUrl);
  }
} catch (e) {
  await handleError(e);
```

```
        await app.renderError(e as any, req, res, req.url!);
      }
    });

    createSocketServer(httpServer);

    httpServer.listen(port, "0.0.0.0", 10, () => {
      console.log(`> Ready on http://localhost:${port}`);
    });
  })
  .catch(async (e) => {
    handleError(e);
    console.error(e);
    process.exit(1);
  });

const handleError = async (e: any) => {
  const err = {
    name: "Unknown Error",
    message: "No message provided",
    stack: "No stack provided",
  };

  if (e.name) err.name = e.name;
  if (e.message) err.message = e.message;
  if (e.stack) err.stack = e.stack;
  await ErrorModel.create(err);
};
```

## apps/web/src/server/socket/channels/Authenticate.ts

```
/**
 * apps/web/src/server/socket/channels/Authenticate.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 05.01.2023
 *
 */
import { Socket } from "socket.io";
import Channel from "./Channel";
import UserModel from "@codeup/util/dist/models/UserModel";
import { decode, verify } from "jsonwebtoken";
import SocketRegistry from "../SocketRegistry";

export default class AuthenticateChannel extends Channel {
  constructor(socket: Socket, channelName: string) {
    super(socket, channelName);
  }

  public register() {
    this.socket.on(this.channelName, async (token: string) => {
      if (SocketRegistry.loggedIn[this.socket.id]) {
        this.emit({
          authenticated: true,
        });
        return;
      }

      if (verify(token, process.env.JWT_SECRET as string)) {
```

```
        const decoded = decode(token) as any;

        const user = await UserModel.findOne({ username: decoded.username });

        if (user) {
          SocketRegistry.loggedIn[this.socket.id] = {
            id: user._id,
            username: user.username,
            admin: user.role === "admin" ? true : undefined,
            pro: process.env.DISABLE_PAYMENTS
              ? true
              : user.subscription === "PRO",
          };
          this.emit({
            authenticated: true,
          });
        } else {
          this.emit({
            authenticated: false,
          });
        }
      } else {
        this.emit({
          error: "Invalid Token",
          authenticated: false,
        });
      }
    });
  }

  public emit(data: any) {
    this.socket.emit(this.channelName, data);
  }
}
```

## apps/web/src/server/socket/channels/Channel.ts

```
/**
 * apps/web/src/server/socket/channels/Channel.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 05.01.2023
 *
 */
import { Socket } from "socket.io";

export default abstract class Channel {
  constructor(protected socket: Socket, protected channelName: string) {
    this.socket = socket;
    this.channelName = channelName;
  }

  public abstract register(): void;

  public abstract emit(data: any): void;
}
```

## apps/web/src/server/socket/channels/EditorProjectChangeSetting.ts

```
/**
```

```
 * apps/web/src/server/socket/channels/EditorProjectChangeSetting.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 27.01.2023
 *
 */

import { Socket } from "socket.io";
import Channel from "./Channel";
import SocketRegistry from "../SocketRegistry";
import EditorProjectModel from "@codeup/util/dist/models/EditorProjectModel";

export default class EditorProjectChangeSettingChannel extends Channel {
  constructor(socket: Socket, channelName: string) {
    super(socket, channelName);
  }

  public register() {
    this.socket.on(this.channelName, async (data) => {
      try {
        if (SocketRegistry.loggedIn[this.socket.id]) {
          const { id, update } = data;

          const project = await EditorProjectModel.findOne({
            _id: id,
          });

          if (project) {
            if (
              project.owner.equals(
                SocketRegistry.loggedIn[this.socket.id].id
              ) ||
              project.contributors.find(
                (c: any) =>
                  c.user.equals(SocketRegistry.loggedIn[this.socket.id].id) &&
                  c.permissions.includes("WRITE")
              ) ||
              SocketRegistry.loggedIn[this.socket.id].admin
            ) {
              const allowedKeys = [
                "customDomain",
                "name",
                "discovery",
                "discoveryDescription",
              ];

              for (const key of Object.keys(update)) {
                if (
                  key === "discovery" &&
                  project.customDomain.startsWith("disabled:")
                )
                  continue;
                if (allowedKeys.includes(key)) {
                  project[key] = update[key];
                }
                if (
                  key === "customDomain" &&
                  update[key].startsWith("disabled:")
```

```
            ) {
              project.discovery = false;
            }
          }

          project.updated_at = new Date();

          await project.save();

          this.emit({
            success: true,
          });
        } else {
          this.emit({
            error: "Not authorized",
            success: false,
          });
        }
      } else {
        this.emit({
          error: "Project not found",
          success: false,
        });
      }
    } else {
      this.emit({
        error: "Not authenticated",
        success: false,
      });
    }
  } catch (e) {
    this.emit({
      success: false,
      // @ts-ignore
      error: e.message,
    });
  }
 });
}

public emit(data: any) {
  this.socket.emit(this.channelName, data);
}
}
```

## apps/web/src/server/socket/channels/EditorProjectContributorAddChannel.ts

```
/**
 * apps/web/src/server/socket/channels/EditorProjectContributorAddChannel.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 05.01.2023
 *
 */

import { Socket } from "socket.io";
import Channel from "./Channel";
import SocketRegistry from "../SocketRegistry";
import EditorProjectModel from "@codeup/util/dist/models/EditorProjectModel";
```

```
import UserModel from "@codeup/util/dist/models/UserModel";

export default class EditorProjectContributorAddChannel extends Channel {
  constructor(socket: Socket, channelName: string) {
    super(socket, channelName);
  }

  public register() {
    this.socket.on(this.channelName, async (data) => {
      if (SocketRegistry.loggedIn[this.socket.id]) {
        const { id, email } = data;

        const project = await EditorProjectModel.findOne({
          _id: id,
        });

        if (project) {
          if (
            project.owner.equals(SocketRegistry.loggedIn[this.socket.id].id) ||
            project.contributors.find(
              (c: any) =>
                c.user.equals(SocketRegistry.loggedIn[this.socket.id].id) &&
                c.permissions.includes("MANAGE-CONTRIBUTORS")
            ) ||
            SocketRegistry.loggedIn[this.socket.id].admin
          ) {
            if (!SocketRegistry.loggedIn[this.socket.id].pro) {
              this.emit({
                error:
                  "Du musst CodeUp PRO abonnieren, um mit anderen zusammenarbeiten zu können.",
                success: false,
              });
              return;
            }

            const contributor = await UserModel.findOne({
              email,
            });

            if (contributor) {
              if (
                project.contributors.find((c: any) =>
                  c.user.equals(contributor._id)
                ) ||
                project.owner.equals(contributor._id)
              ) {
                this.emit({
                  error: "Contributor already exists",
                });
                return;
              }

              project.contributors.push({
                user: contributor._id,
                permissions: ["READ", "WRITE", "DUPLICATE"],
              });

              project.updated_at = Date.now();

              await project.save();
```

```
            this.emit({
              success: true,
              contributor: {
                email: contributor.email,
                id: contributor._id,
                username: contributor.username,
                firstName: contributor.firstName,
                lastName: contributor.lastName,
                permissions: ["READ", "WRITE", "DUPLICATE"],
              },
            });
          } else {
            this.emit({
              error: "User not found",
              success: false,
            });
          }
        } else {
          this.emit({
            error: "Not authorized",
            success: false,
          });
        }
      } else {
        this.emit({
          error: "Project not found",
          success: false,
        });
      }
    } else {
      this.emit({
        error: "Not authenticated",
        success: false,
      });
    }
  });
 }

 public emit(data: any) {
   this.socket.emit(this.channelName, data);
 }
}
```

## apps/web/src/server/socket/channels/EditorProjectContributorChannel.ts

```
/**
 * apps/web/src/server/socket/channels/EditorProjectContributorChannel.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 05.01.2023
 *
 */

import { Socket } from "socket.io";
import Channel from "./Channel";
import SocketRegistry from "../SocketRegistry";
import EditorProjectModel from "@codeup/util/dist/models/EditorProjectModel";
import UserModel from "@codeup/util/dist/models/UserModel";
```

```
export default class EditorProjectContributorChannel extends Channel {
  constructor(socket: Socket, channelName: string) {
    super(socket, channelName);
  }

  public register() {
    this.socket.on(this.channelName, async (data) => {
      if (SocketRegistry.loggedIn[this.socket.id]) {
        const { id } = data;

        const project = await EditorProjectModel.findOne({
          _id: id,
        });

        if (project) {
          if (
            project.owner.equals(SocketRegistry.loggedIn[this.socket.id].id) ||
            project.contributors.find(
              (c: any) =>
                c.user.equals(SocketRegistry.loggedIn[this.socket.id].id) &&
                c.permissions.includes("MANAGE-CONTRIBUTORS")
            ) ||
            SocketRegistry.loggedIn[this.socket.id].admin
          ) {
            let c: {
              email: string;
              id: string;
              permissions: string[];
              username: string;
              firstName: string;
              lastName: string;
            }[] = [];
            let owner: {
              email: string;
              id: string;
              permissions: string[];
              username: string;
              firstName: string;
              lastName: string;
            };

            for (const contributor of project.contributors) {
              const contributor0 = await UserModel.findById(contributor.user);
              c = [
                ...c,
                {
                  email: contributor0.email,
                  id: contributor.user,
                  permissions: contributor.permissions,
                  username: contributor0.username,
                  firstName: contributor0.firstName,
                  lastName: contributor0.lastName,
                },
              ];
            }

            const owner0 = await UserModel.findById(project.owner);
            owner = {
              email: owner0.email,
```

```
            id: owner0._id,
            permissions: ["OWNER"],
            username: owner0.username,
            firstName: owner0.firstName,
            lastName: owner0.lastName,
          };

          this.emit({
            success: true,
            contributors: c,
            owner,
          });
        } else {
          this.emit({
            error: "Not authorized",
            success: false,
          });
        }
      } else {
        this.emit({
          error: "Project not found",
          success: false,
        });
      }
    } else {
      this.emit({
        error: "Not authenticated",
        success: false,
      });
    }
  });
 }

 public emit(data: any) {
   this.socket.emit(this.channelName, data);
 }
}
```

## apps/web/src/server/socket/channels/EditorProjectContributorDeleteChannel.ts

```
/**
 * apps/web/src/server/socket/channels/EditorProjectContributorDeleteChannel.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 05.01.2023
 *
 */

import { Socket } from "socket.io";
import Channel from "./Channel";
import SocketRegistry from "../SocketRegistry";
import EditorProjectModel from "@codeup/util/dist/models/EditorProjectModel";
import UserModel from "@codeup/util/dist/models/UserModel";

export default class EditorProjectContributorDeleteChannel extends Channel {
 constructor(socket: Socket, channelName: string) {
   super(socket, channelName);
 }
```

```
public register() {
  this.socket.on(this.channelName, async (data) => {
    if (SocketRegistry.loggedIn[this.socket.id]) {
      const { id, email } = data;

      const project = await EditorProjectModel.findOne({
        _id: id,
      });

      if (project) {
        if (
          project.owner.equals(SocketRegistry.loggedIn[this.socket.id].id) ||
          project.contributors.find(
            (c: any) =>
              c.user.equals(SocketRegistry.loggedIn[this.socket.id].id) &&
              c.permissions.includes("MANAGE-CONTRIBUTORS")
          ) ||
          SocketRegistry.loggedIn[this.socket.id].admin
        ) {
          if (!SocketRegistry.loggedIn[this.socket.id].pro) {
            this.emit({
              error:
                "Du musst CodeUp PRO abonnieren, um mit anderen zusammenarbeiten zu können.",
              success: false,
            });
            return;
          }
          const contributor = await UserModel.findOne({
            email,
          });

          if (contributor) {
            if (
              project.contributors.find((c: any) =>
                c.user.equals(contributor._id)
              ) ||
              project.owner.equals(contributor._id)
            ) {
              project.contributors = project.contributors.filter(
                (c: any) => !c.user.equals(contributor._id)
              );

              await project.save();

              this.emit({
                success: true,
              });
            } else {
              this.emit({
                error: "Contributor does not exists",
              });
              return;
            }
          } else {
            this.emit({
              error: "User not found",
              success: false,
            });
          }
```

```
          } else {
            this.emit({
              error: "Not authorized",
              success: false,
            });
          }
        } else {
          this.emit({
            error: "Project not found",
            success: false,
          });
        }
      } else {
        this.emit({
          error: "Not authenticated",
          success: false,
        });
      }
    });
  }

  public emit(data: any) {
    this.socket.emit(this.channelName, data);
  }
}
```

## apps/web/src/server/socket/channels/EditorProjectContributorPermissionChannel.ts

```
/**
 * apps/web/src/server/socket/channels/EditorProjectContributorAddChannel.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 05.01.2023
 *
 */

import { Socket } from "socket.io";
import Channel from "./Channel";
import SocketRegistry from "../SocketRegistry";
import EditorProjectModel from "@codeup/util/dist/models/EditorProjectModel";
import UserModel from "@codeup/util/dist/models/UserModel";

export default class EditorProjectContributorPermissionChannel extends Channel {
  constructor(socket: Socket, channelName: string) {
    super(socket, channelName);
  }

  public register() {
    this.socket.on(this.channelName, async (data) => {
      if (SocketRegistry.loggedIn[this.socket.id]) {
        const { id, email, permissions } = data;

        const project = await EditorProjectModel.findOne({
          _id: id
        });

        if (project) {
          if (
```

```
  project.owner.equals(SocketRegistry.loggedIn[this.socket.id].id) ||
  project.contributors.find(
    (c: any) =>
      c.user.equals(SocketRegistry.loggedIn[this.socket.id].id) &&
      c.permissions.includes("MANAGE-CONTRIBUTORS")
  ) ||
  SocketRegistry.loggedIn[this.socket.id].admin
) {
  if (!SocketRegistry.loggedIn[this.socket.id].pro) {
    this.emit({
      error:
        "Du musst CodeUp PRO abonnieren, um mit anderen zusammenarbeiten zu können.",
      success: false,
    });
    return;
  }
  const contributor = await UserModel.findOne({
    email
  });

  if (contributor) {
    if (
      project.contributors.find((c: any) =>
        c.user.equals(contributor._id)
      ) ||
      project.owner.equals(contributor._id)
    ) {
      let rPerms: string[] = permissions;

      if (!rPerms.includes("READ")) {
        rPerms = [...rPerms, "READ"];
      }

      let contributrs: any[] = [];

      for (const c of project.contributors) {
        if (c.user.equals(contributor._id)) {
          contributrs.push({
            ...c,
            permissions: rPerms
          });
        } else {
          contributrs.push(c);
        }
      }

      project.contributors = contributrs;

      await project.save();

      this.emit({
        success: true,
        permissions: rPerms
      });
    } else {
      this.emit({
        success: false,
        error: "Contributor not found"
      });
    }
```

```
            project.updated_at = Date.now();

            await project.save();

            this.emit({
              success: true,
              contributor: {
                email: contributor.email,
                id: contributor._id,
                username: contributor.username,
                firstName: contributor.firstName,
                lastName: contributor.lastName,
                permissions: ["READ", "WRITE", "DUPLICATE"]
              }
            });
          } else {
            this.emit({
              error: "User not found",
              success: false
            });
          }
        } else {
          this.emit({
            error: "Not authorized",
            success: false
          });
        }
      } else {
        this.emit({
          error: "Project not found",
          success: false
        });
      }
    } else {
      this.emit({
        error: "Not authenticated",
        success: false
      });
    }
  });
 }

 public emit(data: any) {
   this.socket.emit(this.channelName, data);
 }
}
```

## apps/web/src/server/socket/channels/EditorProjectCreate.ts

```
/**
 * apps/web/src/server/socket/channels/EditorProjectCreate.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 05.01.2023
 *
 */

import { Socket } from "socket.io";
```

```
import Channel from "./Channel";
import SocketRegistry from "../SocketRegistry";
import EditorProjectModel from "@codeup/util/dist/models/EditorProjectModel";
import { v4 } from "uuid";

export default class EditorProjectCreateChannel extends Channel {
  constructor(socket: Socket, channelName: string) {
    super(socket, channelName);
  }

  public register() {
    this.socket.on(this.channelName, async (data) => {
      if (SocketRegistry.loggedIn[this.socket.id]) {
        const { name } = data;

        let allowedToCreateProject = true;

        if (!SocketRegistry.loggedIn[this.socket.id].pro) {
          const projects = await EditorProjectModel.find({
            owner: SocketRegistry.loggedIn[this.socket.id].id,
          });
          if (projects.length >= 3) {
            allowedToCreateProject = false;
          }
        }

        if (!allowedToCreateProject) {
          this.emit({
            error:
              "Du musst CodeUp PRO abonnieren, um mehr als 3 Projekte zu erstellen.",
            success: false,
          });
          return;
        }

        const project = await EditorProjectModel.create({
          name,
          owner: SocketRegistry.loggedIn[this.socket.id].id,
          published: false,
          discovery: false,
          customDomain: "disabled:" + v4(),
          discoveryDescription: "",
          created_at: new Date(),
          updated_at: new Date(),
          contributors: [],
        });

        this.emit({
          project,
          success: true,
        });
      } else {
        this.emit({
          error: "Not authenticated",
          success: false,
        });
      }
    });
  }
```

```
  public emit(data: any) {
    this.socket.emit(this.channelName, data);
  }
}
```

## apps/web/src/server/socket/channels/EditorProjectDelete.ts

```
/**
 * apps/web/src/server/socket/channels/EditorProjectDelete.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 05.01.2023
 *
 */

import { Socket } from "socket.io";
import Channel from "./Channel";
import SocketRegistry from "../SocketRegistry";
import EditorProjectModel from "@codeup/util/dist/models/EditorProjectModel";
import EditorProjectPageModel from "@codeup/util/dist/models/EditorProjectPageModel";
import EditorProjectFunctionModel from "@codeup/util/dist/models/EditorProjectFunctionModel";
import OrgProjectTemplateModel from "@codeup/util/dist/models/OrgProjectTemplateModel";

export default class EditorProjectDeleteChannel extends Channel {
  constructor(socket: Socket, channelName: string) {
    super(socket, channelName);
  }

  public register() {
    this.socket.on(this.channelName, async (data) => {
      if (SocketRegistry.loggedIn[this.socket.id]) {
        const { id } = data;

        const project = await EditorProjectModel.findOne({
          _id: id,
        });

        if (project) {
          if (
            project.owner.equals(SocketRegistry.loggedIn[this.socket.id].id) ||
            SocketRegistry.loggedIn[this.socket.id].admin
          ) {
            const pages = await EditorProjectPageModel.find({
              project: project._id,
            });

            for (const page of pages) {
              await page.remove();
            }

            const functions = await EditorProjectFunctionModel.find({
              project: project._id,
            });

            for (const func of functions) {
              await func.remove();
            }

            const templates = await OrgProjectTemplateModel.find({
```

```
            projectId: project._id,
          });

          for (const template of templates) {
            await template.remove();
          }

          await project.remove();
          this.emit({
            success: true,
          });
        } else {
          this.emit({
            error: "Not authorized",
            success: false,
          });
        }
      } else {
        this.emit({
          error: "Project not found",
          success: false,
        });
      }
    } else {
      this.emit({
        error: "Not authenticated",
        success: false,
      });
    }
  });
 }

 public emit(data: any) {
  this.socket.emit(this.channelName, data);
 }
}
```

## apps/web/src/server/socket/channels/EditorProjectFunctionContent.ts

```
/**
 * apps/web/src/server/socket/channels/EditorProjectFunctionContent.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 19.02.23
 *
 */
import { Socket } from "socket.io";
import Channel from "./Channel";
import SocketRegistry from "../SocketRegistry";
import EditorProjectPageModel from "@codeup/util/src/models/EditorProjectPageModel";
import EditorProjectModel from "@codeup/util/src/models/EditorProjectModel";
import EditorProjectFunctionModel from "@codeup/util/dist/models/EditorProjectFunctionModel";

export default class EditorProjectFunctionContentChannel extends Channel {
 constructor(socket: Socket, channelName: string) {
  super(socket, channelName);
 }

 public register() {
```

```
this.socket.on(this.channelName, async (data) => {
  try {
    if (SocketRegistry.loggedIn[this.socket.id]) {
      const { projectId, pageId } = data;

      const project = await EditorProjectModel.findOne({
        _id: projectId,
      });

      if (!project) {
        this.emit({
          error: "Project not found",
          success: false,
        });
        return;
      }

      if (
        !(
          project.owner.equals(
            SocketRegistry.loggedIn[this.socket.id].id
          ) ||
          project.contributors.find((c: any) =>
            c.user.equals(SocketRegistry.loggedIn[this.socket.id].id)
          ) ||
          SocketRegistry.loggedIn[this.socket.id].admin
        )
      ) {
        this.emit({
          error: "Not authorized",
          success: false,
        });
        return;
      }

      const func = await EditorProjectFunctionModel.findOne({
        project: project._id,
        _id: pageId,
      });

      if (!func) {
        this.emit({
          error: "Function does not exists",
          success: false,
        });
        return;
      }

      this.emit({
        success: true,
        function: {
          path: func.path,
          code: func.code,
        },
      });
    } else {
      this.emit({
        error: "Not authenticated",
        success: false,
      });
```

```
      }
    } catch (e) {
      this.emit({
        // @ts-ignore
        error: e.message,
        success: false,
      });
    }
  });
 }

 public emit(data: any) {
  this.socket.emit(this.channelName, data);
 }
}
```

## apps/web/src/server/socket/channels/EditorProjectFunctionContentChange.ts

```
/**
 * apps/web/src/server/socket/channels/EditorProjectFunctionContentChange.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 19.02.23
 *
 */
import { Socket } from "socket.io";
import Channel from "./Channel";
import SocketRegistry from "../SocketRegistry";
import EditorProjectPageModel from "@codeup/util/src/models/EditorProjectPageModel";
import EditorProjectModel from "@codeup/util/src/models/EditorProjectModel";
import EditorProjectFunctionModel from "@codeup/util/dist/models/EditorProjectFunctionModel";

export default class EditorProjectFunctionContentChangeChannel extends Channel {
 constructor(socket: Socket, channelName: string) {
  super(socket, channelName);
 }

 public register() {
  this.socket.on(this.channelName, async (data) => {
    try {
      if (SocketRegistry.loggedIn[this.socket.id]) {
        const { projectId, pageId, code } = data;

        const project = await EditorProjectModel.findOne({
          _id: projectId,
        });

        if (!project) {
          this.emit({
            error: "Project not found",
            success: false,
          });
          return;
        }

        if (
          !(
            project.owner.equals(
              SocketRegistry.loggedIn[this.socket.id].id
```

```
            ) ||
            project.contributors.find(
              (c: any) =>
                c.user.equals(SocketRegistry.loggedIn[this.socket.id].id) &&
                c.permissions.includes("WRITE")
            ) ||
            SocketRegistry.loggedIn[this.socket.id].admin
          )
        ) {
          this.emit({
            error: "Not authorized",
            success: false,
          });
          return;
        }

        const func = await EditorProjectFunctionModel.findOne({
          project: project._id,
          _id: pageId,
        });

        if (!func) {
          this.emit({
            error: "Page does not exists",
            success: false,
          });
          return;
        }

        func.code = code;

        await func.save();

        this.emit({
          success: true,
        });
      } else {
        this.emit({
          error: "Not authenticated",
          success: false,
        });
      }
    } catch (e) {
      this.emit({
        // @ts-ignore
        error: e.message,
        success: false,
      });
    }
  });
 }

 public emit(data: any) {
  this.socket.emit(this.channelName, data);
 }
}
```

## apps/web/src/server/socket/channels/EditorProjectFunctionCreate.ts

```
/**
 * apps/web/src/server/socket/channels/EditorProjectFunctionCreate.ts
```

```
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 19.02.2023
 *
 */

import { Socket } from "socket.io";
import Channel from "./Channel";
import SocketRegistry from "../SocketRegistry";
import EditorProjectPageModel from "@codeup/util/src/models/EditorProjectPageModel";
import EditorProjectModel from "@codeup/util/src/models/EditorProjectModel";
import EditorProjectFunctionModel from "@codeup/util/dist/models/EditorProjectFunctionModel";

export default class EditorProjectFunctionCreateChannel extends Channel {
  constructor(socket: Socket, channelName: string) {
    super(socket, channelName);
  }

  public register() {
    this.socket.on(this.channelName, async (data) => {
      if (SocketRegistry.loggedIn[this.socket.id]) {
        const { id, url } = data;

        const project = await EditorProjectModel.findOne({
          _id: id,
        });

        if (!project) {
          this.emit({
            error: "Project not found",
            success: false,
          });
          return;
        }

        if (
          !(
            project.owner.equals(SocketRegistry.loggedIn[this.socket.id].id) ||
            project.contributors.find(
              (c: any) =>
                c.user.equals(SocketRegistry.loggedIn[this.socket.id].id) &&
                c.permissions.includes("WRITE")
            ) ||
            SocketRegistry.loggedIn[this.socket.id].admin
          )
        ) {
          this.emit({
            error: "Not authorized",
            success: false,
          });
          return;
        }

        const existingFunction = await EditorProjectFunctionModel.findOne({
          project: project._id,
          path: url,
        });
```

```
        if (existingFunction) {
          this.emit({
            error: "Function already exists",
            success: false,
          });
          return;
        }

        let allowedToCreate = true;

        if (!SocketRegistry.loggedIn[this.socket.id].pro) {
          const funcs = await EditorProjectFunctionModel.find({
            project: project._id,
          });
          if (funcs.length >= 1) {
            allowedToCreate = false;
          }
        }

        if (!allowedToCreate) {
          this.emit({
            error:
              "Du musst CodeUp PRO abonnieren, um mehr als 1 Funktion zu erstellen.",
            success: false,
          });
          return;
        }
        const func = await EditorProjectFunctionModel.create({
          project: id,
          path: "/api" + url,
          code: 'res.status(200).json({ message: "Hello World!"});',
        });

        this.emit({
          function: func,
          success: true,
        });
      } else {
        this.emit({
          error: "Not authenticated",
          success: false,
        });
      }
    });
  }

  public emit(data: any) {
    this.socket.emit(this.channelName, data);
  }
}
```

## apps/web/src/server/socket/channels/EditorProjectFunctionDelete.ts

```
/**
 * apps/web/src/server/socket/channels/EditorProjectFunctionDelete.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 19.02.2023
 *
```

```
 */

import { Socket } from "socket.io";
import Channel from "./Channel";
import SocketRegistry from "../SocketRegistry";
import EditorProjectPageModel from "@codeup/util/src/models/EditorProjectPageModel";
import EditorProjectModel from "@codeup/util/src/models/EditorProjectModel";
import EditorProjectFunctionModel from "@codeup/util/dist/models/EditorProjectFunctionModel";

export default class EditorProjectFunctionDeleteChannel extends Channel {
  constructor(socket: Socket, channelName: string) {
    super(socket, channelName);
  }

  public register() {
    this.socket.on(this.channelName, async (data) => {
      if (SocketRegistry.loggedIn[this.socket.id]) {
        const { projectId, pageId } = data;

        const project = await EditorProjectModel.findOne({
          _id: projectId,
        });

        if (!project) {
          this.emit({
            error: "Project not found",
            success: false,
          });
          return;
        }

        if (
          !(
            project.owner.equals(SocketRegistry.loggedIn[this.socket.id].id) ||
            project.contributors.find(
              (c: any) =>
                c.user.equals(SocketRegistry.loggedIn[this.socket.id].id) &&
                c.permissions.includes("DELETE")
            ) ||
            SocketRegistry.loggedIn[this.socket.id].admin
          )
        ) {
          this.emit({
            error: "Not authorized",
            success: false,
          });
          return;
        }

        const existingFunction = await EditorProjectFunctionModel.findOne({
          project: project._id,
          _id: pageId,
        });

        if (!existingFunction) {
          this.emit({
            error: "Page does not exists",
            success: false,
          });
          return;
```

```
        }

        await existingFunction.remove();

        this.emit({
          success: true,
        });
      } else {
        this.emit({
          error: "Not authenticated",
          success: false,
        });
      }
    });
  }

  public emit(data: any) {
    this.socket.emit(this.channelName, data);
  }
}
```

## apps/web/src/server/socket/channels/EditorProjectFunctionRename.ts

```
/**
 * apps/web/src/server/socket/channels/EditorProjectFunctionRename.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 19.02.2023
 *
 */

import { Socket } from "socket.io";
import Channel from "./Channel";
import SocketRegistry from "../SocketRegistry";
import EditorProjectPageModel from "@codeup/util/src/models/EditorProjectPageModel";
import EditorProjectModel from "@codeup/util/src/models/EditorProjectModel";
import EditorProjectFunctionModel from "@codeup/util/dist/models/EditorProjectFunctionModel";

export default class EditorProjectFunctionRenameChannel extends Channel {
  constructor(socket: Socket, channelName: string) {
    super(socket, channelName);
  }

  public register() {
    this.socket.on(this.channelName, async (data) => {
      if (SocketRegistry.loggedIn[this.socket.id]) {
        const { projectId, pageId, newUrl } = data;

        const project = await EditorProjectModel.findOne({
          _id: projectId,
        });

        if (!project) {
          this.emit({
            error: "Project not found",
            success: false,
          });
          return;
        }
```

```
    if (
      !(
        project.owner.equals(SocketRegistry.loggedIn[this.socket.id].id) ||
        project.contributors.find(
          (c: any) =>
            c.user.equals(SocketRegistry.loggedIn[this.socket.id].id) &&
            c.permissions.includes("RENAME")
        ) ||
        SocketRegistry.loggedIn[this.socket.id].admin
      )
    ) {
      this.emit({
        error: "Not authorized",
        success: false,
      });
      return;
    }

    const func = await EditorProjectFunctionModel.findOne({
      project: project._id,
      _id: pageId,
    });

    if (!func) {
      this.emit({
        error: "Page does not exists",
        success: false,
      });
      return;
    }

    const existingFunc = await EditorProjectFunctionModel.findOne({
      project: project._id,
      path: newUrl,
    });

    if (existingFunc) {
      this.emit({
        error: "Page already exists",
        success: false,
      });
      return;
    }

    func.path = newUrl;

    await func.save();

    this.emit({
      success: true,
      function: func,
    });
  } else {
    this.emit({
      error: "Not authenticated",
      success: false,
    });
  }
});
```

```
  }

  public emit(data: any) {
    this.socket.emit(this.channelName, data);
  }
}
```

## apps/web/src/server/socket/channels/EditorProjectFunctions.ts

```
/**
 * apps/web/src/server/socket/channels/EditorProjectFunctions.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 19.02.2023
 *
 */
import Channel from "./Channel";
import { Socket } from "socket.io";
import SocketRegistry from "../SocketRegistry";
import EditorProjectModel from "@codeup/util/src/models/EditorProjectModel";
import EditorProjectPageModel from "@codeup/util/src/models/EditorProjectPageModel";
import EditorProjectFunctionModel from "@codeup/util/dist/models/EditorProjectFunctionModel";

export default class EditorProjectFunctionsChannel extends Channel {
  constructor(socket: Socket, channelName: string) {
    super(socket, channelName);
  }

  public register() {
    this.socket.on(this.channelName, async (data) => {
      if (SocketRegistry.loggedIn[this.socket.id]) {
        const { id } = data;

        const project = await EditorProjectModel.findOne({
          _id: id,
        });

        if (project) {
          if (
            project.owner.equals(SocketRegistry.loggedIn[this.socket.id].id) ||
            project.contributors.find((c: any) =>
              c.user.equals(SocketRegistry.loggedIn[this.socket.id].id)
            ) ||
            SocketRegistry.loggedIn[this.socket.id].admin
          ) {
            const functions = await EditorProjectFunctionModel.find({
              project: project._id,
            });

            let functionsArr = [];

            for (const func of functions) {
              functionsArr.push({
                _id: func._id,
                path: func.path,
                code: func.code,
              });
            }
```

```
            this.emit({
              functions: functionsArr,
              success: true,
            });
          } else {
            this.emit({
              error: "Not authorized",
              success: false,
            });
          }
        } else {
          this.emit({
            error: "Project not found",
            success: false,
          });
        }
      } else {
        this.emit({
          error: "Not authenticated",
          success: false,
        });
      }
    });
  }

  public emit(data: any) {
    this.socket.emit(this.channelName, data);
  }
}
```

## apps/web/src/server/socket/channels/EditorProjectInfo.ts

```
/**
 * apps/web/src/server/socket/channels/EditorProjectInfo.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 27.01.2023
 *
 */

import { Socket } from "socket.io";
import Channel from "./Channel";
import SocketRegistry from "../SocketRegistry";
import EditorProjectModel from "@codeup/util/dist/models/EditorProjectModel";

export default class EditorProjectInfoChannel extends Channel {
  constructor(socket: Socket, channelName: string) {
    super(socket, channelName);
  }

  public register() {
    this.socket.on(this.channelName, async (data) => {
      if (SocketRegistry.loggedIn[this.socket.id]) {
        const { id } = data;

        const project = await EditorProjectModel.findOne({
          _id: id,
        });
```

```
        if (project) {
          if (
            project.owner.equals(SocketRegistry.loggedIn[this.socket.id].id) ||
            project.contributors.find((c: any) =>
              c.user.equals(SocketRegistry.loggedIn[this.socket.id].id)
            ) ||
            SocketRegistry.loggedIn[this.socket.id].admin
          ) {
            this.emit({
              success: true,
              project: project,
            });
          } else {
            this.emit({
              error: "Not authorized",
              success: false,
            });
          }
        } else {
          this.emit({
            error: "Project not found",
            success: false,
          });
        }
      } else {
        this.emit({
          error: "Not authenticated",
          success: false,
        });
      }
    });
  }

  public emit(data: any) {
    this.socket.emit(this.channelName, data);
  }
}
```

## apps/web/src/server/socket/channels/EditorProjectList.ts

```
/**
 * apps/web/src/server/socket/channels/EditorProjectDataChannel.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 05.01.2023
 *
 */

import { Socket } from "socket.io";
import Channel from "./Channel";
import SocketRegistry from "../SocketRegistry";
import EditorProjectModel from "@codeup/util/dist/models/EditorProjectModel";

export default class EditorProjectListChannel extends Channel {
  constructor(socket: Socket, channelName: string) {
    super(socket, channelName);
  }

  public register() {
```

```
    this.socket.on(this.channelName, async (data) => {
      if (SocketRegistry.loggedIn[this.socket.id]) {
        let projects: any[] = [
          ...(await EditorProjectModel.find({
            owner: SocketRegistry.loggedIn[this.socket.id].id,
          })),
          ...(await EditorProjectModel.find({
            contributors: {
              $elemMatch: {
                user: SocketRegistry.loggedIn[this.socket.id].id,
              },
            },
          })),
        ];

        this.emit({
          projects,
          success: true,
        });
      } else {
        this.emit({
          error: "Not authenticated",
          success: false,
        });
      }
    });
  }

  public emit(data: any) {
    this.socket.emit(this.channelName, data);
  }
}
```

## apps/web/src/server/socket/channels/EditorProjectPageContent.ts

```
/**
 * apps/web/src/server/socket/channels/EditorProjectPageContentContent.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 25.01.23
 *
 */
import { Socket } from "socket.io";
import Channel from "./Channel";
import SocketRegistry from "../SocketRegistry";
import EditorProjectPageModel from "@codeup/util/src/models/EditorProjectPageModel";
import EditorProjectModel from "@codeup/util/src/models/EditorProjectModel";

export default class EditorProjectPageContentChannel extends Channel {
  constructor(socket: Socket, channelName: string) {
    super(socket, channelName);
  }

  public register() {
    this.socket.on(this.channelName, async (data) => {
      try {
        if (SocketRegistry.loggedIn[this.socket.id]) {
          const { projectId, pageId } = data;
```

```
    const project = await EditorProjectModel.findOne({
      _id: projectId,
    });

    if (!project) {
      this.emit({
        error: "Project not found",
        success: false,
      });
      return;
    }

    if (
      !(
        project.owner.equals(
          SocketRegistry.loggedIn[this.socket.id].id
        ) ||
        project.contributors.find((c: any) =>
          c.user.equals(SocketRegistry.loggedIn[this.socket.id].id)
        ) ||
        SocketRegistry.loggedIn[this.socket.id].admin
      )
    ) {
      this.emit({
        error: "Not authorized",
        success: false,
      });
      return;
    }

    const page = await EditorProjectPageModel.findOne({
      project: project._id,
      _id: pageId,
    });

    if (!page) {
      this.emit({
        error: "Page does not exists",
        success: false,
      });
      return;
    }

    this.emit({
      success: true,
      page: {
        url: page.url,
        mainContent: page.mainContent,
        type: page.type,
        resources: page.resources,
      },
    });
  } else {
    this.emit({
      error: "Not authenticated",
      success: false,
    });
  }
} catch (e) {
  this.emit({
```

```
          // @ts-ignore
          error: e.message,
          success: false,
        });
      }
    });
  }

  public emit(data: any) {
    this.socket.emit(this.channelName, data);
  }
}
```

## apps/web/src/server/socket/channels/EditorProjectPageContentChange.ts

```
/**
 * apps/web/src/server/socket/channels/EditorProjectPageContentChange.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 25.01.23
 *
 */
import { Socket } from "socket.io";
import Channel from "./Channel";
import SocketRegistry from "../SocketRegistry";
import EditorProjectPageModel from "@codeup/util/src/models/EditorProjectPageModel";
import EditorProjectModel from "@codeup/util/src/models/EditorProjectModel";

export default class EditorProjectPageContentChangeChannel extends Channel {
  constructor(socket: Socket, channelName: string) {
    super(socket, channelName);
  }

  public register() {
    this.socket.on(this.channelName, async (data) => {
      try {
        if (SocketRegistry.loggedIn[this.socket.id]) {
          const { projectId, pageId, content, resource, resourceType } = data;

          const project = await EditorProjectModel.findOne({
            _id: projectId,
          });

          if (!project) {
            this.emit({
              error: "Project not found",
              success: false,
            });
            return;
          }

          if (
            !(
              project.owner.equals(
                SocketRegistry.loggedIn[this.socket.id].id
              ) ||
              project.contributors.find(
                (c: any) =>
                  c.user.equals(SocketRegistry.loggedIn[this.socket.id].id) &&
```

```
        c.permissions.includes("WRITE")
    ) ||
    SocketRegistry.loggedIn[this.socket.id].admin
  )
) {
  this.emit({
    error: "Not authorized",
    success: false,
  });
  return;
}

const page = await EditorProjectPageModel.findOne({
  project: project._id,
  _id: pageId,
});

if (!page) {
  this.emit({
    error: "Page does not exists",
    success: false,
  });
  return;
}

if (resource === "codeup.main_resource") {
  page.mainContent = content;
} else {
  const resourceObject = page.resources.find(
    (r: any) => r.name === resource
  );

  if (!resourceObject) {
    if (resourceType) {
      page.resources.push({
        name: resource,
        type: resourceType,
        content: content,
      });
    } else {
      this.emit({
        error: "Resource not found",
        success: false,
      });
      return;
    }
  } else {
    page.resources = page.resources
      .map((r: any) => {
        if (r.name !== resource) {
          return r;
        } else {
          if (content === "codeup.delete_resource") {
            return undefined;
          }
          return {
            content: content,
            name: r.name,
            type: r.type,
          };
```

```
            }
          })
          .filter((r: any) => r !== null && r !== undefined);
      }
    }

    SocketRegistry.loggedIn[this.socket.id].currentlyEditing = pageId;

    let otherSockets = Object.keys(SocketRegistry.loggedIn).filter(
      (s) => {
        if (!SocketRegistry.loggedIn[s]) return false;
        if (s === this.socket.id) return false;
        if (!SocketRegistry.loggedIn[s].currentlyEditing) return false;
        if (SocketRegistry.loggedIn[s].currentlyEditing !== pageId)
          return false;

        return true;
      }
    );

    let realOtherSockets = otherSockets.map((s) => {
      // @ts-ignore
      return global.io.sockets.sockets.get(s);
    });

    for (const s of realOtherSockets) {
      if (!s) console.log("Socket not found");
      s.emit("editor:projects:page:content", {
        success: true,
        user: SocketRegistry.loggedIn[this.socket.id].username,
        page: {
          url: page.url,
          mainContent: page.mainContent,
          type: page.type,
          resources: page.resources,
        },
      });
    }

    await page.save();

    this.emit({
      success: true,
    });
  } else {
    this.emit({
      error: "Not authenticated",
      success: false,
    });
  }
 } catch (e) {
  this.emit({
    // @ts-ignore
    error: e.message,
    success: false,
  });
 }
});
}
```

```
  public emit(data: any) {
    this.socket.emit(this.channelName, data);
  }
}
```

## apps/web/src/server/socket/channels/EditorProjectPageCreate.ts

```
/**
 * apps/web/src/server/socket/channels/EditorProjectPageCreate.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 05.01.2023
 *
 */

import { Socket } from "socket.io";
import Channel from "./Channel";
import SocketRegistry from "../SocketRegistry";
import EditorProjectPageModel from "@codeup/util/src/models/EditorProjectPageModel";
import EditorProjectModel from "@codeup/util/src/models/EditorProjectModel";

export default class EditorProjectPageCreateChannel extends Channel {
  constructor(socket: Socket, channelName: string) {
    super(socket, channelName);
  }

  public register() {
    this.socket.on(this.channelName, async (data) => {
      if (SocketRegistry.loggedIn[this.socket.id]) {
        const { id, type, url, content } = data;

        if (url.startsWith("/api")) {
          this.emit({
            error: "URL cannot start with api",
          });
          return;
        }

        const project = await EditorProjectModel.findOne({
          _id: id,
        });

        if (!project) {
          this.emit({
            error: "Project not found",
            success: false,
          });
          return;
        }

        if (
          !(
            project.owner.equals(SocketRegistry.loggedIn[this.socket.id].id) ||
            project.contributors.find(
              (c: any) =>
                c.user.equals(SocketRegistry.loggedIn[this.socket.id].id) &&
                c.permissions.includes("WRITE")
            ) ||
            SocketRegistry.loggedIn[this.socket.id].admin
```

```
    )
  ) {
    this.emit({
      error: "Not authorized",
      success: false,
    });
    return;
  }

  const existingPage = await EditorProjectPageModel.findOne({
    project: project._id,
    url,
  });

  if (existingPage) {
    this.emit({
      error: "Page already exists",
      success: false,
    });
    return;
  }

  let allowedToCreate = true;

  if (!SocketRegistry.loggedIn[this.socket.id].pro) {
    const pages = await EditorProjectPageModel.find({
      project: project._id,
    });
    if (pages.length >= 5) {
      allowedToCreate = false;
    }
  }

  if (!allowedToCreate) {
    this.emit({
      error:
        "Du musst CodeUp PRO abonnieren, um mehr als 5 Seiten zu erstellen.",
      success: false,
    });
    return;
  }

  const page = await EditorProjectPageModel.create({
    project: id,
    type,
    url,
    mainContent: content
      ? content
      : type.toLowerCase() === "html"
      ? "<h1>HTML Seite</h1>"
      : 'Gib "Hallo Welt" in der Dialogbox aus',
    resources: [],
  });

  this.emit({
    page,
    success: true,
  });
} else {
  this.emit({
```

```
          error: "Not authenticated",
          success: false,
        });
      }
    });
  }

  public emit(data: any) {
    this.socket.emit(this.channelName, data);
  }
}
```

## apps/web/src/server/socket/channels/EditorProjectPageDelete.ts

```
/**
 * apps/web/src/server/socket/channels/EditorProjectPageDelete.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 05.01.2023
 *
 */

import { Socket } from "socket.io";
import Channel from "./Channel";
import SocketRegistry from "../SocketRegistry";
import EditorProjectPageModel from "@codeup/util/src/models/EditorProjectPageModel";
import EditorProjectModel from "@codeup/util/src/models/EditorProjectModel";

export default class EditorProjectPageDeleteChannel extends Channel {
  constructor(socket: Socket, channelName: string) {
    super(socket, channelName);
  }

  public register() {
    this.socket.on(this.channelName, async (data) => {
      if (SocketRegistry.loggedIn[this.socket.id]) {
        const { projectId, pageId } = data;

        const project = await EditorProjectModel.findOne({
          _id: projectId,
        });

        if (!project) {
          this.emit({
            error: "Project not found",
            success: false,
          });
          return;
        }

        if (
          !(
            project.owner.equals(SocketRegistry.loggedIn[this.socket.id].id) ||
            project.contributors.find(
              (c: any) =>
                c.user.equals(SocketRegistry.loggedIn[this.socket.id].id) &&
                c.permissions.includes("DELETE")
            ) ||
            SocketRegistry.loggedIn[this.socket.id].admin
```

```
        )
      ) {
        this.emit({
          error: "Not authorized",
          success: false,
        });
        return;
      }

      const existingPage = await EditorProjectPageModel.findOne({
        project: project._id,
        _id: pageId,
      });

      if (!existingPage) {
        this.emit({
          error: "Page does not exists",
          success: false,
        });
        return;
      }

      await existingPage.remove();

      this.emit({
        success: true,
      });
    } else {
      this.emit({
        error: "Not authenticated",
        success: false,
      });
    }
  });
 }

 public emit(data: any) {
  this.socket.emit(this.channelName, data);
 }
}
```

## apps/web/src/server/socket/channels/EditorProjectPageRename.ts

```
/**
 * apps/web/src/server/socket/channels/EditorProjectPageRename.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 05.01.2023
 *
 */

import { Socket } from "socket.io";
import Channel from "./Channel";
import SocketRegistry from "../SocketRegistry";
import EditorProjectPageModel from "@codeup/util/src/models/EditorProjectPageModel";
import EditorProjectModel from "@codeup/util/src/models/EditorProjectModel";

export default class EditorProjectPageRenameChannel extends Channel {
 constructor(socket: Socket, channelName: string) {
```

```
  super(socket, channelName);
}

public register() {
  this.socket.on(this.channelName, async (data) => {
    if (SocketRegistry.loggedIn[this.socket.id]) {
      const { projectId, pageId, newUrl } = data;

      if (newUrl.startsWith("/api")) {
        this.emit({
          error: "URL cannot start with api",
        });
        return;
      }

      const project = await EditorProjectModel.findOne({
        _id: projectId,
      });

      if (!project) {
        this.emit({
          error: "Project not found",
          success: false,
        });
        return;
      }

      if (
        !(
          project.owner.equals(SocketRegistry.loggedIn[this.socket.id].id) ||
          project.contributors.find(
            (c: any) =>
              c.user.equals(SocketRegistry.loggedIn[this.socket.id].id) &&
              c.permissions.includes("RENAME")
          ) ||
          SocketRegistry.loggedIn[this.socket.id].admin
        )
      ) {
        this.emit({
          error: "Not authorized",
          success: false,
        });
        return;
      }

      const page = await EditorProjectPageModel.findOne({
        project: project._id,
        _id: pageId,
      });

      if (!page) {
        this.emit({
          error: "Page does not exists",
          success: false,
        });
        return;
      }

      const existingPage = await EditorProjectPageModel.findOne({
        project: project._id,
```

```
          url: newUrl,
        });

        if (existingPage) {
          this.emit({
            error: "Page already exists",
            success: false,
          });
          return;
        }

        page.url = newUrl;

        await page.save();

        this.emit({
          success: true,
          page,
        });
      } else {
        this.emit({
          error: "Not authenticated",
          success: false,
        });
      }
    });
  }

  public emit(data: any) {
    this.socket.emit(this.channelName, data);
  }
}
```

## apps/web/src/server/socket/channels/EditorProjectPages.ts

```
/**
 * apps/web/src/server/socket/channels/EditorProjectPages.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 20.01.2023
 *
 */
import Channel from "./Channel";
import { Socket } from "socket.io";
import SocketRegistry from "../SocketRegistry";
import EditorProjectModel from "@codeup/util/src/models/EditorProjectModel";
import EditorProjectPageModel from "@codeup/util/src/models/EditorProjectPageModel";

export default class EditorProjectPagesChannel extends Channel {
  constructor(socket: Socket, channelName: string) {
    super(socket, channelName);
  }

  public register() {
    this.socket.on(this.channelName, async (data) => {
      if (SocketRegistry.loggedIn[this.socket.id]) {
        const { id } = data;

        const project = await EditorProjectModel.findOne({
```

```
        _id: id,
      });

      if (project) {
        if (
          project.owner.equals(SocketRegistry.loggedIn[this.socket.id].id) ||
          project.contributors.find((c: any) =>
            c.user.equals(SocketRegistry.loggedIn[this.socket.id].id)
          ) ||
          SocketRegistry.loggedIn[this.socket.id].admin
        ) {
          const pages = await EditorProjectPageModel.find({
            project: project._id,
          });

          let pagesArr = [];

          for (const page of pages) {
            pagesArr.push({
              _id: page._id,
              url: page.url,
              type: page.type,
            });
          }

          this.emit({
            pages: pagesArr,
            success: true,
          });
        } else {
          this.emit({
            error: "Not authorized",
            success: false,
          });
        }
      } else {
        this.emit({
          error: "Project not found",
          success: false,
        });
      }
    } else {
      this.emit({
        error: "Not authenticated",
        success: false,
      });
    }
  });
 }

 public emit(data: any) {
  this.socket.emit(this.channelName, data);
 }
}
```

## apps/web/src/server/socket/channels/EditorProjectRename.ts

```
/**
 * apps/web/src/server/socket/channels/EditorProjectRename.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
```

```
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 05.01.2023
 *
 */

import { Socket } from "socket.io";
import Channel from "./Channel";
import SocketRegistry from "../SocketRegistry";
import EditorProjectModel from "@codeup/util/dist/models/EditorProjectModel";

export default class EditorProjectRenameChannel extends Channel {
  constructor(socket: Socket, channelName: string) {
    super(socket, channelName);
  }

  public register() {
    this.socket.on(this.channelName, async (data) => {
      if (SocketRegistry.loggedIn[this.socket.id]) {
        const { id, name } = data;

        const project = await EditorProjectModel.findOne({
          _id: id,
        });

        if (project) {
          if (
            project.owner.equals(SocketRegistry.loggedIn[this.socket.id].id) ||
            SocketRegistry.loggedIn[this.socket.id].admin
          ) {
            project.name = name;
            project.updated_at = new Date();

            await project.save();

            this.emit({
              success: true,
            });
          } else {
            this.emit({
              error: "Not authorized",
              success: false,
            });
          }
        } else {
          this.emit({
            error: "Project not found",
            success: false,
          });
        }
      } else {
        this.emit({
          error: "Not authenticated",
          success: false,
        });
      }
    });
  }

  public emit(data: any) {
```

```
    this.socket.emit(this.channelName, data);
  }
}
```

## apps/web/src/server/socket/channels/EditorProjectStaticAssetDeleteChannel.ts

```
/**
 * apps/web/src/server/socket/channels/EditorProjectStaticAssetDeleteChannel.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 28.01.23
 *
 */
import { Socket } from "socket.io";
import Channel from "./Channel";
import SocketRegistry from "../SocketRegistry";
import EditorProjectModel from "@codeup/util/dist/models/EditorProjectModel";
import mongoose from "mongoose";
import { v4 } from "uuid";
import { ObjectID } from "bson";

export default class EditorProjectStaticAssetDeleteChannel extends Channel {
  constructor(socket: Socket, channelName: string) {
    super(socket, channelName);
  }

  public register() {
    this.socket.on(this.channelName, async (data: any) => {
      if (SocketRegistry.loggedIn[this.socket.id]) {
        const { projectId, assetName } = data;

        const project = await EditorProjectModel.findOne({
          _id: projectId,
        });

        if (project) {
          if (
            project.owner.equals(SocketRegistry.loggedIn[this.socket.id].id) ||
            SocketRegistry.loggedIn[this.socket.id].admin
          ) {
            const bucket = new mongoose.mongo.GridFSBucket(
              mongoose.connection.db,
              {
                bucketName: "editorStaticAssets",
              }
            );

            const file = await bucket.find({
              filename: `${project._id.toString()}/${assetName}`,
            });

            const fileEntry = await file.next();

            if (file) {
              await bucket.delete(new ObjectID(fileEntry!._id.toString()));

              this.emit({
                success: true,
```

```
            });
          } else {
            this.emit({
              error: "File not found",
              success: false,
            });
          }
        } else {
          this.emit({
            error: "Not authorized",
            success: false,
          });
        }
      } else {
        this.emit({
          error: "Project not found",
          success: false,
        });
      }
    } else {
      this.emit({
        error: "Not authenticated",
        success: false,
      });
    }
  });
 }

 public emit(data: any) {
   this.socket.emit(this.channelName, data);
 }
}
```

## apps/web/src/server/socket/channels/EditorProjectStaticAssetsChannel.ts

```
/**
 * apps/web/src/server/socket/channels/EditorProjectStaticAssetsChannel.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 28.01.23
 *
 */
import { Socket } from "socket.io";
import Channel from "./Channel";
import SocketRegistry from "../SocketRegistry";
import EditorProjectModel from "@codeup/util/dist/models/EditorProjectModel";
import mongoose, { mongo } from "mongoose";

export default class EditorProjectStaticAssetsChannel extends Channel {
  constructor(socket: Socket, channelName: string) {
    super(socket, channelName);
  }

  public register() {
    this.socket.on(this.channelName, async (data) => {
      try {
        if (SocketRegistry.loggedIn[this.socket.id]) {
          const { id } = data;
```

```
        const project = await EditorProjectModel.findOne({
          _id: id,
        });

        if (project) {
          if (
            project.owner.equals(
              SocketRegistry.loggedIn[this.socket.id].id
            ) ||
            SocketRegistry.loggedIn[this.socket.id].admin
          ) {
            const bucket = new mongoose.mongo.GridFSBucket(
              mongoose.connection.db,
              {
                bucketName: "editor-static-assets",
              }
            );

            const files = await mongoose.connection.db
              .collection("editorStaticAssets.files")
              .find({
                metadata: {
                  projectId: id,
                },
              })
              .toArray();

            this.emit({
              success: true,
              files: files,
            });
          } else {
            this.emit({
              error: "Not authorized",
              success: false,
            });
          }
        } else {
          this.emit({
            error: "Project not found",
            success: false,
          });
        }
      } else {
        this.emit({
          error: "Not authenticated",
          success: false,
        });
      }
    } catch (e) {
      this.emit({
        success: false,
        // @ts-ignore
        error: e.message,
      });
    }
  });
}

public emit(data: any) {
```

```
    this.socket.emit(this.channelName, data);
  }
}
```

## apps/web/src/server/socket/channels/EditorProjectStaticAssetUploadChannel.ts

```
/**
 * apps/web/src/server/socket/channels/EditorProjectStaticAssetUploadChannel.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 28.01.23
 *
 */
import { Socket } from "socket.io";
import Channel from "./Channel";
import SocketRegistry from "../SocketRegistry";
import EditorProjectModel from "@codeup/util/dist/models/EditorProjectModel";
import mongoose from "mongoose";
import { v4 } from "uuid";
import EditorProjectFunctionModel from "@codeup/util/dist/models/EditorProjectFunctionModel";

export default class EditorProjectStaticAssetUploadChannel extends Channel {
  constructor(socket: Socket, channelName: string) {
    super(socket, channelName);
  }

  public register() {
    this.socket.on(
      this.channelName,
      async (data: any, callback: (status: any) => void) => {
        if (SocketRegistry.loggedIn[this.socket.id]) {
          let { id, fileContent, fileName } = data;

          fileName = fileName.replace(/[^a-z0-9.]/gi, "_").toLowerCase();

          const project = await EditorProjectModel.findOne({
            _id: id,
          });

          if (project) {
            if (
              project.owner.equals(
                SocketRegistry.loggedIn[this.socket.id].id
              ) ||
              SocketRegistry.loggedIn[this.socket.id].admin
            ) {
              const bucket = new mongoose.mongo.GridFSBucket(
                mongoose.connection.db,
                {
                  bucketName: "editorStaticAssets",
                }
              );

              const uploadStream = bucket.openUploadStream(
                `${project._id.toString()}/${fileName}`,
                {
                  metadata: {
                    projectId: id,
```

```
              },
            }
          );

          uploadStream.write(fileContent, async () => {
            uploadStream.end();

            project.updated_at = new Date();
            await project.save();

            this.emit({
              success: true,
              fileUrl:
                "/editor-static-assets/" +
                `${project._id.toString()}/${fileName}`,
            });
          });
        } else {
          this.emit({
            error: "Not authorized",
            success: false,
          });
        }
      } else {
        this.emit({
          error: "Project not found",
          success: false,
        });
      }
    } else {
      this.emit({
        error: "Not authenticated",
        success: false,
      });
    }
  }
  );
 }

 public emit(data: any) {
  this.socket.emit(this.channelName, data);
 }
}
```

## apps/web/src/server/socket/index.ts

```
/**
 * apps/web/src/server/socket/index.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 05.01.2023
 *
 */
import { Server as HTTPServer } from "http";
import { Server, Socket } from "socket.io";
import AuthenticateChannel from "./channels/Authenticate";
import EditorProjectListChannel from "./channels/EditorProjectList";
import SocketRegistry from "./SocketRegistry";
import EditorProjectCreateChannel from "./channels/EditorProjectCreate";
```

```
import EditorProjectDeleteChannel from "./channels/EditorProjectDelete";
import EditorProjectRenameChannel from "./channels/EditorProjectRename";
import EditorProjectContributorChannel from "./channels/EditorProjectContributorChannel";
import EditorProjectContributorAddChannel from "./channels/EditorProjectContributorAddChannel";
import EditorProjectContributorDeleteChannel from
"./channels/EditorProjectContributorDeleteChannel";
import EditorProjectContributorPermissionChannel from
"./channels/EditorProjectContributorPermissionChannel";
import EditorProjectPagesChannel from "./channels/EditorProjectPages";
import EditorProjectPageCreateChannel from "./channels/EditorProjectPageCreate";
import EditorProjectPageDeleteChannel from "./channels/EditorProjectPageDelete";
import EditorProjectPageRenameChannel from "./channels/EditorProjectPageRename";
import EditorProjectPageContentChannel from "./channels/EditorProjectPageContent";
import EditorProjectPageContentChangeChannel from
"./channels/EditorProjectPageContentChange";
import EditorProjectInfoChannel from "./channels/EditorProjectInfo";
import EditorProjectChangeSettingChannel from "./channels/EditorProjectChangeSetting";
import EditorProjectStaticAssetUploadChannel from
"./channels/EditorProjectStaticAssetUploadChannel";
import EditorProjectStaticAssetsChannel from "./channels/EditorProjectStaticAssetsChannel";
import EditorProjectStaticAssetDeleteChannel from
"./channels/EditorProjectStaticAssetDeleteChannel";
import EditorProjectFunctionsChannel from "./channels/EditorProjectFunctions";
import EditorProjectFunctionCreateChannel from "./channels/EditorProjectFunctionCreate";
import EditorProjectFunctionDeleteChannel from "./channels/EditorProjectFunctionDelete";
import EditorProjectFunctionRenameChannel from "./channels/EditorProjectFunctionRename";
import EditorProjectFunctionContentChannel from "./channels/EditorProjectFunctionContent";
import EditorProjectFunctionContent from "./channels/EditorProjectFunctionContent";
import EditorProjectFunctionContentChangeChannel from
"./channels/EditorProjectFunctionContentChange";

export function createSocketServer(httpServer: HTTPServer) {
  const io = new Server(httpServer, {
    cors: {
      origin: "*",
      methods: ["GET", "POST"],
    },
    maxHttpBufferSize: 1e8,
  });

  // @ts-ignore
  global.io = io;

  io.on("connection", (socket: Socket) => {
    // system channels

    const authenticateChannel = new AuthenticateChannel(socket, "authenticate");

    // system channels end

    // editor project

    const editorProjectListChannel = new EditorProjectListChannel(
      socket,
      "editor:projects:list"
    );

    const editorProjectCreateChannel = new EditorProjectCreateChannel(
      socket,
      "editor:projects:create"
```

```
);

const editorProjectDeleteChannel = new EditorProjectDeleteChannel(
  socket,
  "editor:projects:delete"
);

const editorProjectRenameChannel = new EditorProjectRenameChannel(
  socket,
  "editor:projects:rename"
);
const editorProjectInfoChannel = new EditorProjectInfoChannel(
  socket,
  "editor:projects:info"
);

// editor project end

// editor project contributors

const editorProjectContributorChannel = new EditorProjectContributorChannel(
  socket,
  "editor:projects:contributors"
);

const editorProjectContributorAddChannel =
  new EditorProjectContributorAddChannel(
    socket,
    "editor:projects:contributors:add"
  );

const editorProjectContributorDeleteChannel =
  new EditorProjectContributorDeleteChannel(
    socket,
    "editor:projects:contributors:delete"
  );

const editorProjectContributorPermissionChannel =
  new EditorProjectContributorPermissionChannel(
    socket,
    "editor:projects:contributors:permission"
  );

// editor project contributors end

// editor pages

const editorProjectPagesChannel = new EditorProjectPagesChannel(
  socket,
  "editor:projects:pages"
);
const editorProjectPageCreateChannel = new EditorProjectPageCreateChannel(
  socket,
  "editor:projects:pages:create"
);
const editorProjectPageDeleteChannel = new EditorProjectPageDeleteChannel(
  socket,
  "editor:projects:pages:delete"
);
const editorProjectPageRenameChannel = new EditorProjectPageRenameChannel(
```

```
  socket,
  "editor:projects:pages:rename"
);
const editorProjectPageContentChannel = new EditorProjectPageContentChannel(
  socket,
  "editor:projects:page:content"
);
const editorProjectPageContentChangeChannel =
  new EditorProjectPageContentChangeChannel(
    socket,
    "editor:projects:page:content:set"
  );

// editor pages end

// project settings

const editorProjectChangeSettingChannel =
  new EditorProjectChangeSettingChannel(socket, "editor:projects:settings");

// project settings end

// static assets

const editorProjectStaticAssetsUploadChannel =
  new EditorProjectStaticAssetUploadChannel(
    socket,
    "editor:projects:static:assets:upload"
  );

const editorProjectStaticAssetsChannel =
  new EditorProjectStaticAssetsChannel(
    socket,
    "editor:projects:static:assets"
  );

const editorProjectStaticAssetsDeleteChannel =
  new EditorProjectStaticAssetDeleteChannel(
    socket,
    "editor:projects:static:assets:delete"
  );

// static assets end

// serverless functions
const editorProjectFunctionsChannel = new EditorProjectFunctionsChannel(
  socket,
  "editor:projects:functions"
);

const editorProjectFunctionCreateChannel =
  new EditorProjectFunctionCreateChannel(
    socket,
    "editor:projects:functions:create"
  );

const editorProjectFunctionDeleteChannel =
  new EditorProjectFunctionDeleteChannel(
    socket,
    "editor:projects:functions:delete"
```

```
      );

    const editorProjectFunctionRenameChannel =
      new EditorProjectFunctionRenameChannel(
        socket,
        "editor:projects:functions:rename"
      );

    const editorProjectFunctionContentChannel =
      new EditorProjectFunctionContentChannel(
        socket,
        "editor:projects:functions:content"
      );

    const editorProjectFunctionContentChangeChannel =
      new EditorProjectFunctionContentChangeChannel(
        socket,
        "editor:projects:functions:content:set"
      );

    // serverless functions end

    authenticateChannel.register();

    editorProjectListChannel.register();
    editorProjectCreateChannel.register();
    editorProjectDeleteChannel.register();
    editorProjectRenameChannel.register();
    editorProjectInfoChannel.register();
    editorProjectContributorChannel.register();
    editorProjectContributorAddChannel.register();
    editorProjectContributorDeleteChannel.register();
    editorProjectContributorPermissionChannel.register();
    editorProjectPagesChannel.register();
    editorProjectPageCreateChannel.register();
    editorProjectPageDeleteChannel.register();
    editorProjectPageRenameChannel.register();
    editorProjectPageContentChannel.register();
    editorProjectPageContentChangeChannel.register();
    editorProjectChangeSettingChannel.register();
    editorProjectStaticAssetsUploadChannel.register();
    editorProjectStaticAssetsChannel.register();
    editorProjectStaticAssetsDeleteChannel.register();
    editorProjectFunctionsChannel.register();
    editorProjectFunctionCreateChannel.register();
    editorProjectFunctionDeleteChannel.register();
    editorProjectFunctionRenameChannel.register();
    editorProjectFunctionContentChannel.register();
    editorProjectFunctionContentChangeChannel.register();

    socket.on("disconnect", () => {
      delete SocketRegistry.loggedIn[socket.id];
    });
  });
}
```

## apps/web/src/server/socket/SocketRegistry.ts

```
/**
 * apps/web/src/server/socket/SocketRegistry.ts
 *
```

```
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 05.01.2023
 *
 */

export default class SocketRegistry {
  public static loggedIn: {
    [key: string]: {
      id: string;
      username: string;
      currentlyEditing?: string;
      admin?: boolean;
      pro?: boolean;
    };
  } = {};
}
```

## apps/web/src/special-components/codeblock.module.css

```
.wrapper {
  border: 1px solid gray;
  overflow: auto;
  padding: 0.5em;
}

.lineNo {
  display: table-cell;
  text-align: right;
  padding-right: 1em;
  user-select: none;
  opacity: 0.5;
}

.lineContent {
  display: table-cell;
}
```

## apps/web/src/special-components/CodeBlock.tsx

```
import {
  Box,
  Button,
  IconButton,
  useClipboard,
  useColorMode,
} from "@chakra-ui/react";
import Highlight, { defaultProps } from "prism-react-renderer";
import darkTheme from "prism-react-renderer/themes/dracula";
import { FaCheck, FaCopy } from "react-icons/fa";
import styles from "./codeblock.module.css";

const CodeBlock = ({
  children,
  className,
  allowCopy,
}: {
  children: any;
  className: any;
  allowCopy?: boolean;
}) => {
```

```
  if (allowCopy === undefined) {
    allowCopy = true;
  }

  const { hasCopied, onCopy } = useClipboard(children);

  const language = className.replace(/language-/, "");

  return (
    <Box pos="relative" marginTop="0.5rem" fontSize="1.2rem">
      {allowCopy ? (
        <IconButton
          colorScheme="messenger"
          pos="absolute"
          right="2"
          top="2"
          size="lg"
          onClick={onCopy}
          aria-label="Copy to clipboard"
          icon={hasCopied ? <FaCheck /> : <FaCopy />}
        />
      ) : null}

      <Highlight
        {...defaultProps}
        theme={darkTheme}
        code={children + "\n"}
        language={language}
      >
        {({ className, style, tokens, getLineProps, getTokenProps }) => (
          <pre
            className={`${className} ${styles.wrapper}`}
            style={{ ...style }}
          >
            {tokens.map((line, i) => (
              <div key={i} {...getLineProps({ line, key: i })}>
                <span className={styles.lineNo}>{i + 1}</span>

                <span className={styles.lineContent}>
                  {line.map((token, key) => (
                    <span key={key} {...getTokenProps({ token, key })} />
                  ))}
                </span>
              </div>
            ))}
          </pre>
        )}
      </Highlight>
    </Box>
  );
};

export default CodeBlock;
```

## apps/web/src/special-components/MDRenderer.tsx

```
/**
 * apps/web/src/special-components/MDRenderer.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
```

```
 * License: Project License
 * Created At: 05.11.2022
 *
 */

import * as React from "react";
import {
  Heading,
  Link,
  ListItem,
  OrderedList,
  Text,
  UnorderedList,
} from "@chakra-ui/react";
import CodeBlock from "./CodeBlock";
import dynamic from "next/dynamic";
import ReactMarkdown from "react-markdown";
export default function MDRenderer(props: { content: string }) {
  return (
    <>
      <ReactMarkdown
        components={{
          p: ({ node, ...props }) => <Text fontSize={"xl"} {...props} />,
          h1: ({ node, ...props }) => <Heading fontSize={"4xl"} {...props} />,
          h2: ({ node, ...props }) => <Heading fontSize={"2xl"} {...props} />,
          h3: ({ node, ...props }) => <Heading fontSize={"xl"} {...props} />,
          h4: ({ node, ...props }) => <Heading fontSize={"lg"} {...props} />,
          h5: ({ node, ...props }) => <Heading fontSize={"md"} {...props} />,
          h6: ({ node, ...props }) => <Heading fontSize={"sm"} {...props} />,
          ul: ({ node, ...props }) => <UnorderedList {...props} />,
          li: ({ node, ...props }) => <ListItem {...props} />,
          ol: ({ node, ...props }) => <OrderedList {...props} />,
          a: ({ node, ...props }) => (
            <Link color={"blue.500"} {...props} target={"_blank"} />
          ),
          code: ({ node, inline, className, children, ...props }) => {
            const match = /language-(\w+)/.exec(className || "");
            return !inline && match ? (
              <CodeBlock
                className={className}
                allowCopy={true}
                // eslint-disable-next-line react/no-children-prop
                children={children}
                {...props}
              />
            ) : (
              <code className={className} {...props}>
                {children}
              </code>
            );
          },
        }}
      >
        {props.content}
      </ReactMarkdown>
    </>
  );
}
```

## apps/web/src/styles/globals.scss

```
@import url("https://rsms.me/inter/inter.css");
@import url("https://fonts.googleapis.com/css?family=Seymour One");
@import url("https://cdnjs.cloudflare.com/ajax/libs/grapesjs/0.20.1/css/grapes.min.css");

@font-face {
  font-family: ARCADE;
  src: url(ARCADE_R.TTF);
}

@font-face {
  font-family: "Jetbrains Mono ExtraBold";
  src: url("JetBrainsMono-ExtraBold.woff2") format("woff2");
}

/* Global Styles */

html,
body {
  padding: 0;
  margin: 0;
  font-family: -apple-system, BlinkMacSystemFont, Segoe UI, Roboto, Oxygen,
  Ubuntu, Cantarell, Fira Sans, Droid Sans, Helvetica Neue, sans-serif;
  scrollbar-width: thin;
  scroll-behavior: smooth;
  overflow-x: hidden;
}

* {
  scrollbar-width: thin;
}

a {
  color: inherit;
  text-decoration: none;
}

* {
  box-sizing: border-box;
}

/* Home Page Animation */

.hp-loader {
  animation: hp-loader 0.1s ease;
  animation-fill-mode: forwards;
  animation-delay: 4s;
  display: block;
}

.hp-loader {
  .hp-image {
    position: absolute;
    top: 50%;
    left: 50%;
    transform: translate(-50%, -50%);

    background-image: url("/codeup.png");
    background-size: cover;
    background-position: center;
    background-repeat: no-repeat;
```

```
    width: 512px;
    height: 512px;
  }

  .hp-image-0 {
    animation: imgPolyPath 2s infinite ease;
    animation-fill-mode: forwards;
  }
}

@keyframes imgPolyPath {
  0% {
    clip-path: polygon(50% 0%, 100% 50%, 50% 100%, 0% 50%);
  }
  10% {
    clip-path: polygon(0 0, 100% 50%, 50% 100%, 0% 50%);
  }

  20% {
    clip-path: polygon(0 0, 100% 50%, 50% 100%, 0 100%);
  }

  30% {
    clip-path: polygon(0 0, 100% 50%, 100% 100%, 0 100%);
  }

  40% {
    clip-path: polygon(0 0, 100% 0, 100% 100%, 0 100%);
  }

  50% {
    clip-path: polygon(0 0, 50% 0, 100% 100%, 0 100%);
  }

  60% {
    clip-path: polygon(0 50%, 50% 0, 100% 100%, 0 100%);
  }

  70% {
    clip-path: polygon(0 50%, 50% 0, 100% 100%, 50% 100%);
  }

  80% {
    clip-path: polygon(0 50%, 50% 0, 100% 50%, 50% 100%);
  }

  100% {
    clip-path: polygon(50% 0%, 100% 50%, 50% 100%, 0% 50%);
  }
}

@keyframes hp-loader {
  99% {
    opacity: 1;
  }
  100% {
    opacity: 0;
    display: none;
    z-index: -4;
  }
```

```
}

/* Printing Quiz results */

@media print {
  body,
  html,
  #root {
    height: 100%;
  }
}

/* EasyMDE Path */

.EasyMDEContainer {
  width: 100%;
  height: 100%;

  .editor-toolbar {
    .active {
      background: transparent !important;
    }

    button:hover {
      background: transparent !important;
    }

    .side-by-side, .fullscreen, .guide {
      display: none;
    }

    .separator {
      display: none;
    }
  }

  .CodeMirror {
    height: 100%;
    width: 100%;
    background: #1A1B1E;
    color: #fff;
  }

  .editor-preview {
    background: #1A1B1E;
    color: #fff;
  }
}

.index-header-image {
  max-width: 90vw;
  overflow: hidden;
  object-fit: cover;
  position: absolute;
  z-index: -1;
  filter: blur(5px);
  border-radius: 0.75rem;
  width: 100%;
  height: 90vh;
  border: 2px solid #F7DE1F;
```

```
  transform: perspective(1000px) rotateY(0deg) rotateX(20deg) rotateZ(15deg) scale(1.1);
  animation: ihiA 30s infinite;
  animation-timing-function: linear;
}

.ihi-rot {
  transform: perspective(1000px) rotateY(0deg) rotateX(20deg) rotateZ(-15deg) scale(1.1);
  animation: ihiRA 30s infinite;
  animation-timing-function: linear;
}

@keyframes ihiA {
  0% {
    transform: perspective(1000px) rotateY(0deg) rotateX(20deg) rotateZ(20deg) scale(1.1);
  }
  50% {
    transform: perspective(1000px) rotateY(0deg) rotateX(20deg) rotateZ(20deg) scale(1.05);
  }
  100% {
    transform: perspective(1000px) rotateY(0deg) rotateX(20deg) rotateZ(20deg) scale(1.1);
  }
}

@keyframes ihiRA {
  0% {
    transform: perspective(1000px) rotateY(0deg) rotateX(20deg) rotateZ(-15deg) scale(1.1);
  }
  50% {
    transform: perspective(1000px) rotateY(0deg) rotateX(20deg) rotateZ(-15deg) scale(1.05);
  }
  100% {
    transform: perspective(1000px) rotateY(0deg) rotateX(20deg) rotateZ(-15deg) scale(1.1);
  }
}
```

## apps/web/src/styles/tiptap.css

```
/** Original: https://tiptap.dev/examples/formatting **/
/* Basic editor styles */
.ProseMirror {
    margin-top: 1rem;
}

.ProseMirror > * + * {
    margin-top: 0.75em;
}

.ProseMirror ul, .ProseMirror ol {
    padding: 0 1rem;
}

.ProseMirror h1, .ProseMirror h2, .ProseMirror h3, .ProseMirror h4, .ProseMirror h5, .ProseMirror h6 {
    line-height: 1.1;
}

.ProseMirror h1 {
    font-size: 2.5rem;
    margin-bottom: 0.5rem;
}

.ProseMirror h2 {
```

```
    font-size: 2rem;
    margin-bottom: 0.5rem;
}

.ProseMirror h3 {
    font-size: 1.75rem;
    margin-bottom: 0.5rem;
}

.ProseMirror code {
    background-color: rgba(97, 97, 97, 0.1);
    color: #616161;
}

.ProseMirror pre {
    background: #0d0d0d;
    color: #fff;
    font-family: 'JetBrainsMono', monospace;
    padding: 0.75rem 1rem;
    border-radius: 0.5rem;
}

.ProseMirror pre code {
    color: inherit;
    padding: 0;
    background: none;
    font-size: 0.8rem;
}

.ProseMirror mark {
    background-color: #faf594;
}

.ProseMirror img {
    max-width: 100%;
    height: auto;
}

.ProseMirror blockquote {
    padding-left: 1rem;
    border-left: 2px solid rgba(13, 13, 13, 0.1);
}

.ProseMirror hr {
    border: none;
    border-top: 2px solid rgba(13, 13, 13, 0.1);
    margin: 2rem 0;
}

.post-link {
    color: #5a7be8;
    text-decoration: underline;
}

#post-display h1 {
    font-size: 2.5rem;
    margin-bottom: 0.5rem;
}

#post-display h2 {
```

```
    font-size: 2rem;
    margin-bottom: 0.5rem;
}

#post-display h3 {
    font-size: 1.75rem;
    margin-bottom: 0.5rem;
}
```

## apps/web/src/types/IAskPost.ts

```
/**
 * apps/web/src/types/IAskPost.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 24.06.23
 *
 */

export interface IAskPost {
  _id: string;
  content: string;
  username: string;
  user: string;
  createdAt: string;
  embeds: any[];
  parent: string;
}
```

## apps/web/src/types/IAskProfile.ts

```
/**
 * apps/web/src/types/IAskProfile.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 26.06.23
 *
 */

export interface IAskProfile {
  user: string;
  username: string;
  banner: string;
  website: string;
  bio: string;
  badges: string[];
  certificates: string[];
  projects: string[];
}
```

## apps/web/src/types/IIssue.ts

```
/**
 * apps/web/src/types/IIssue.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2024 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 31.05.2024
```

```
 *
 */

export interface IIssue {
  _id: string;
  createdAt: string;
  project: string;
  title: string;
  description: string;
  status: string;
  priority: string;
}
```

## apps/web/src/types/IProject.ts

```
/**
 * apps/web/src/types/IProject.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2024 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 22.05.2024
 *
 */

export interface IProject {
  _id: string;
  createdAt: string;
  users: string[];
  name: string;
  projectType: "static";
  modifiers: string[];
}
```

## apps/web/src/types/IUMLDiagram.ts

```
/**
 * apps/web/src/types/IUMLDiagram.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2024 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 08.06.2024
 *
 */

export interface IUMLDiagram {
  _id: string;
  createdAt: string;
  project: string;
  name: string;
  type: string;
  data: string;
}
```

## apps/web/src/types/IWikiPage.ts

```
/**
 * apps/web/src/types/IWikiPage.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2024 Ben Siebert. All rights reserved.
```

```
 * License: Project License
 * Created At: 31.05.2024
 *
 */

export interface IWikiPage {
  _id: string;
  createdAt: string;
  project: string;
  title: string;
  content: string;
}
```

## apps/web/tsconfig.json

```
{
  "extends": "tsconfig/nextjs.json",
  "include": [
    "next-env.d.ts",
    "**/*.ts",
    "**/*.tsx"
  ],
  "exclude": [
    "node_modules"
  ],
  "compilerOptions": {
    "jsx": "preserve"
  }
}
```

## apps/web/tsconfig.server.json

```
{
  "extends": "./tsconfig.json",
  "compilerOptions": {
    "module": "commonjs",
    "outDir": "server-build",
    "target": "es2017",
    "isolatedModules": false,
    "noEmit": false
  },
  "include": ["src/server/**/*.ts"],
  "ts-node": {
    "transpileOnly": true,
    "module": "commonjs"
  }
}
```

## captain-definition

```
{
  "schemaVersion": 2,
  "dockerfilePath": "./Dockerfile.Web"
}
```

## captain-definition-codeup

```
{
  "schemaVersion": 2,
  "dockerfileLines": [
    "FROM node:18",
    "RUN mkdir -p /usr/src/app",
    "WORKDIR /usr/src/app",
    "RUN npm i -g turbo",
```

```
    "COPY ./ /usr/src/app/",
    "RUN yarn && turbo run build --filter=web",
    "EXPOSE 3000",
    "CMD [ \"yarn\", \"--cwd\", \"./apps/web\", \"ts-start-prod\" ]"
  ]
}
```

## CODE_OF_CONDUCT.md

# Contributor Covenant Code of Conduct

## Our Pledge

We as members, contributors, and leaders pledge to make participation in our community a harassment-free experience for everyone, regardless of age, body size, visible or invisible disability, ethnicity, sex characteristics, gender identity and expression, level of experience, education, socio-economic status, nationality, personal appearance, race, religion, or sexual identity and orientation.

We pledge to act and interact in ways that contribute to an open, welcoming, diverse, inclusive, and healthy community.

## Our Standards

Examples of behavior that contributes to a positive environment for our community include:

* Demonstrating empathy and kindness toward other people
* Being respectful of differing opinions, viewpoints, and experiences
* Giving and gracefully accepting constructive feedback
* Accepting responsibility and apologizing to those affected by our mistakes, and learning from the experience
* Focusing on what is best not just for us as individuals, but for the overall community

Examples of unacceptable behavior include:

* The use of sexualized language or imagery, and sexual attention or advances of any kind
* Trolling, insulting or derogatory comments, and personal or political attacks
* Public or private harassment
* Publishing others' private information, such as a physical or email address, without their explicit permission
* Other conduct which could reasonably be considered inappropriate in a professional setting

## Enforcement Responsibilities

Community leaders are responsible for clarifying and enforcing our standards of acceptable behavior and will take appropriate and fair corrective action in response to any behavior that they deem inappropriate, threatening, offensive, or harmful.

Community leaders have the right and responsibility to remove, edit, or reject comments, commits, code, wiki edits, issues, and other contributions that are not aligned to this Code of Conduct, and will communicate reasons for moderation decisions when appropriate.

## Scope

This Code of Conduct applies within all community spaces, and also applies when an individual is officially representing the community in public spaces.
Examples of representing our community include using an official e-mail address, posting via an official social media account, or acting as an appointed representative at an online or offline event.

## Enforcement

Instances of abusive, harassing, or otherwise unacceptable behavior may be reported to the community leaders responsible for enforcement at https://codeup.space/codeup/contact.
All complaints will be reviewed and investigated promptly and fairly.

All community leaders are obligated to respect the privacy and security of the reporter of any incident.

## Enforcement Guidelines

Community leaders will follow these Community Impact Guidelines in determining the consequences for any action they deem in violation of this Code of Conduct:

### 1. Correction

**Community Impact**: Use of inappropriate language or other behavior deemed unprofessional or unwelcome in the community.

**Consequence**: A private, written warning from community leaders, providing clarity around the nature of the violation and an explanation of why the behavior was inappropriate. A public apology may be requested.

### 2. Warning

**Community Impact**: A violation through a single incident or series of actions.

**Consequence**: A warning with consequences for continued behavior. No interaction with the people involved, including unsolicited interaction with those enforcing the Code of Conduct, for a specified period of time. This includes avoiding interactions in community spaces as well as external channels like social media. Violating these terms may lead to a temporary or permanent ban.

### 3. Temporary Ban

**Community Impact**: A serious violation of community standards, including sustained inappropriate behavior.

**Consequence**: A temporary ban from any sort of interaction or public communication with the community for a specified period of time. No public or private interaction with the people involved, including unsolicited interaction with those enforcing the Code of Conduct, is allowed during this period. Violating these terms may lead to a permanent ban.

### 4. Permanent Ban

**Community Impact**: Demonstrating a pattern of violation of community standards, including sustained inappropriate behavior,  harassment of an individual, or aggression toward or disparagement of classes of individuals.

**Consequence**: A permanent ban from any sort of public interaction within

the community.

## Attribution

This Code of Conduct is adapted from the [Contributor Covenant][homepage], version 2.0, available at https://www.contributor-covenant.org/version/2/0/code_of_conduct.html.

Community Impact Guidelines were inspired by [Mozilla's code of conduct enforcement ladder](https://github.com/mozilla/diversity).

[homepage]: https://www.contributor-covenant.org

For answers to common questions about this code of conduct, see the FAQ at https://www.contributor-covenant.org/faq. Translations are available at https://www.contributor-covenant.org/translations.

## docker-compose.yml

```
version: "3"
services:
  web:
    build:
      context: .
      dockerfile: Dockerfile
    network_mode: host
    ports:
      - "8899:8899"
    env_file: ".env.web"
    user: "root"
    privileged: true
    restart: unless-stopped
    volumes:
      - /var/run/docker.sock:/var/run/docker.sock
      - /tmp:/tmp
      - /etc/codeup/git:/etc/codeup/git
```

## Dockerfile

```
FROM node:18
RUN mkdir -p /usr/src/app
WORKDIR /usr/src/app
RUN npm i -g turbo
COPY ./ /usr/src/app/
RUN yarn && turbo run build --filter=@codeup/web
EXPOSE 3000
CMD yarn --cwd apps/web ts-start-prod
```

## images.docker/codeup-java-rce/Dockerfile

```
FROM eclipse-temurin:21
LABEL authors="Ben Siebert"

WORKDIR /usr/src/app
```

## images.docker/codeup-tsx-rce/Dockerfile

```
FROM node:latest
LABEL authors="Ben Siebert"
RUN npm i -g typescript tsx
```

## package.json

```
{
```

```
  "name": "codeup",
  "version": "2.0.0",
  "private": true,
  "workspaces": [
    "apps/*",
    "packages/*"
  ],
  "scripts": {
    "build": "turbo run build",
    "dev": "turbo run dev --parallel",
    "lint": "turbo run lint",
    "format": "prettier --write \"**/*.{ts,tsx,md}\""
  },
  "devDependencies": {
    "eslint-config-custom": "*",
    "prettier": "latest",
    "turbo": "latest"
  },
  "engines": {
    "npm": ">=7.0.0",
    "node": ">=14.0.0"
  },
  "dependencies": {
    "byte-size": "^8.1.0",
    "configcat-js-ssr": "^5.5.0",
    "docx": "^8.5.0"
  },
  "packageManager": "yarn@1.22.19"
}
```

## packages/components/admin/AdminContentsLayout.tsx

```
/**
 * packages/components/admin/AdminContentsLayout.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 30.10.2022
 *
 */

import * as React from "react";
import { ReactNode, useEffect } from "react";
import {
  Box,
  Button,
  Divider,
  Flex,
  Grid,
  Heading,
  Icon,
  Input,
  Link,
  Text,
  useColorModeValue,
} from "@chakra-ui/react";

import {
  FaBookOpen,
  FaCartPlus,
```

```
  FaFileArchive,
  FaFilePdf,
  FaFileVideo,
  FaFolderPlus,
  FaPenFancy,
  FaPlus,
  FaSearchPlus,
  FaUserCircle,
  FaVideo,
} from "react-icons/fa";
import { useRouter } from "next/router";
import { BsFillHouseDoorFill } from "react-icons/bs";
import REST from "@codeup/rest";
import redirect from "@codeup/util/dist/userworkflow/redirect";

export default function AdminContentsLayout(props: { children: ReactNode }) {
  const router = useRouter();

  useEffect(() => {
    router.push("/creator/studio");
  }, []);

  const sections: { name: string; url: string; icon: ReactNode }[] = [
    {
      name: "Alle Kurse",
      url: "/admin/contents/courses",
      icon: <FaBookOpen />,
    },
    {
      name: "Neuer Kurs",
      url: "/admin/contents/courses/new",
      icon: <FaPlus />,
    },
    {
      name: "Neuer Inhalt",
      url: "/admin/contents/new",
      icon: <FaPlus />,
    },
  ];

  return (
    <>
      <Flex direction={["column", "row"]} minW={"100%"} minH={"100vh"}>
        <Flex
          direction={"column"}
          flex={"15%"}
          minH={"100%"}
          bgColor={useColorModeValue("gray.400", "gray.900")}
          p={"1.2rem"}
        >
          <Heading textAlign={"center"}>Inhalte</Heading>
          <Divider mt={"1.2rem"} />
          {sections.map((s) => (
            <>
              <Button
                m={"1.2rem"}
                leftIcon={s.icon as any}
                onClick={() => router.push(s.url)}
                variant={router.pathname === s.url ? "solid" : "outline"}
              >
```

```
              {s.name}
            </Button>
          </>
        ))}
      </Flex>
      <Flex direction={"column"} flex={"85%"} minH={"100%"}>
        {props.children}
      </Flex>
    </Flex>
  </>
 );
}
```

## packages/components/admin/AdminLayout.tsx

```
/**
 * packages/components/admin/AdminLayout.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 07.10.22
 *
 */

import * as React from "react";
import { useLoggedIn } from "@codeup/hooks";
import { ReactNode, useEffect, useState } from "react";
import { useRouter } from "next/router";
import {
  Box,
  Button,
  CircularProgress,
  Flex,
  Grid,
  Heading,
  IconButton,
  Progress,
  Spinner,
  Tooltip,
} from "@chakra-ui/react";
import {
  FaBookOpen,
  FaCertificate,
  FaChartBar,
  FaCheckSquare,
  FaChevronLeft,
  FaChevronRight,
  FaChild,
  FaCode,
  FaComment,
  FaComments,
  FaFileCode,
  FaLightbulb,
  FaLink,
  FaNewspaper,
  FaPeopleArrows,
  FaPeopleCarry,
  FaProjectDiagram,
  FaRocket,
  FaSitemap,
```

```
  FaTrophy,
  FaUser,
  FaUsers,
} from "react-icons/fa";
import AuthenticateLoadingIndicator from "./AuthenticateLoadingIndicator";
import ErrorComponent from "../global/ErrorComponent";
import { SiWritedotas } from "react-icons/si";
import Sidebar from "../global/Sidebar";
import { AiFillDashboard } from "react-icons/ai";
import { BsPeople, BsCheck2Square } from "react-icons/bs";
import { BiCodeBlock, BiError } from "react-icons/bi";
import AppShell from "../global/AppShell";

export default function AdminLayout(props: { children: ReactNode }) {
  const { loggedIn, userInfo, loaded: userLoaded } = useLoggedIn();

  const [error, setError] = useState("");

  const router = useRouter();

  useEffect(() => {
    if (!userLoaded) return;

    if (!loggedIn) {
      setError("Du bist nicht dazu berechtigt diese Seite zu sehen!");
      return;
    }

    if (userInfo?.role !== "admin") {
      setError("Du bist nicht dazu berechtigt diese Seite zu sehen!");
    }
  }, [loggedIn, userInfo, userLoaded]);

  const [collapsed, setCollapsed] = useState(false);

  useEffect(() => {
    if (localStorage) {
      if (
        localStorage.getItem("admin.dashboard.sidebar.collapsed") === "true"
      ) {
        setCollapsed(true);
      } else {
        setCollapsed(false);
      }
    }
  }, []);

  return (
    <>
      {error !== "" ? (
        <>
          <ErrorComponent
            description={error}
            title={"Authentifizierungsfehler"}
          />
        </>
      ) : null}
      {!userLoaded && error === "" ? (
        <>
          <AuthenticateLoadingIndicator />
```

```
  </>
) : null}
{userLoaded && error === "" ? (
  <>
    <AppShell
      title={"Admin"}
      sub_pages={[
        {
          text: "Dashboard",
          icon: <AiFillDashboard />,
          href: "/admin",
        },
        {
          text: "Benutzer",
          icon: <FaUser />,
          href: "/admin/users",
        },
        {
          text: "Profile",
          icon: <FaUsers />,
          href: "/admin/profiles",
        },
        {
          text: "Analytics",
          icon: <FaChartBar />,
          href: "/admin/analytics",
        },
        {
          text: "Nachrichten",
          icon: <FaComment />,
          href: "/admin/contacts",
        },
        {
          text: "Zertifikate",
          icon: <FaCertificate />,
          href: "/admin/certificates",
        },
        {
          text: "Projekt Ideen",
          icon: <FaLightbulb />,
          href: "/admin/project-ideas",
        },
        {
          text: "Blog",
          icon: <SiWritedotas />,
          href: "/admin/blog",
        },
        {
          text: "Meinungen",
          icon: <BsPeople />,
          href: "/admin/testimonials",
        },
        {
          text: "Organisationen",
          icon: <FaSitemap />,
          href: "/admin/orgs",
        },
        {
          text: "Newsletter",
          icon: <FaNewspaper />,
```

```
      href: "/admin/newsletter",
    },
    {
      text: "Discovery",
      icon: <BiCodeBlock />,
      href: "/admin/discovery",
    },
    {
      text: "Flows",
      icon: <FaProjectDiagram />,
      href: "/admin/flows",
    },
    {
      text: "Kids",
      icon: <FaChild />,
      href: "/admin/kids",
    },
    {
      text: "Roadmaps",
      icon: <FaRocket />,
      href: "/admin/roads",
    },
    {
      text: "ToDo",
      icon: <BsCheck2Square />,
      href: "/admin/todos",
    },
    {
      text: "Code Snippets",
      icon: <FaFileCode />,
      href: "/admin/snippets",
    },
    {
      text: "Shorts",
      icon: <FaLink />,
      href: "/admin/shorts",
    },
    {
      text: "IDE",
      icon: <FaCode />,
      href: "/admin/ide",
    },
    {
      text: "Editor",
      icon: <FaCode />,
      href: "/admin/editor",
    },
    {
      text: "Fehler",
      icon: <BiError />,
      href: "/admin/errors",
    },
    {
      text: "Challenges",
      icon: <FaTrophy />,
      href: "/admin/challenges",
    },
  ]}
>
  {props.children}
```

```
          </AppShell>
        </>
      ) : null}
    </>
  );
}
```

## packages/components/admin/AdminProfileViewer.tsx

```
/**
 * packages/components/admin/AdminProfileViewer.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 08.10.22
 *
 */

import * as React from "react";
import CodeUpModal from "../codeup/CodeUpModal";
import {
  Alert,
  AlertDescription,
  AlertIcon,
  AlertTitle,
  Avatar,
  Box,
  Button,
  ButtonGroup,
  Center,
  CircularProgress,
  Flex,
  FormControl,
  FormLabel,
  Grid,
  Heading,
  HStack,
  Icon,
  IconButton,
  Input,
  InputGroup,
  InputLeftElement,
  Menu,
  MenuButton,
  MenuItem,
  MenuList,
  Progress,
  Select,
  Spinner,
  Table,
  TableContainer,
  Tbody,
  Td,
  Text,
  Textarea,
  Th,
  Thead,
  Tr,
  useToast,
} from "@chakra-ui/react";
```

```
import {
  FaCheck,
  FaCheckSquare,
  FaCross,
  FaEdit,
  FaTrash,
  FaWindowClose,
  FaUser,
  FaLock,
  FaEnvelope,
  FaPen,
  FaUserShield,
  FaMapMarkerAlt,
  FaGlobe,
  FaSave,
  FaTimes,
  FaCheckCircle,
  FaTimesCircle,
} from "react-icons/fa";
import { useEffect } from "react";
import REST from "@codeup/rest";
import { useLoggedIn, useUserInfo } from "@codeup/hooks";

export default function AdminProfileViewer(props: {
  onOpen: () => void;
  onClose: () => void;
  isOpen: boolean;
  profile: {
    username: string;
    displayName: string;
    verified: boolean;
    bio: string;
    website: string;
    location: string;
    createdAt: string;
  };
  profiles: any[];
  setProfiles: (profiles: any[]) => void;
  setCurrentProfile: (profile: any) => void;
}) {
  const [editMode, setEditMode] = React.useState(false);

  const { userInfo, loggedIn, loaded } = useLoggedIn();

  const toast = useToast();

  if (!props.profile) {
    return <></>;
  }

  return (
    <>
      <CodeUpModal
        onOpen={props.onOpen}
        onClose={() => {
          setEditMode(false);
          props.onClose();
        }}
        isOpen={props.isOpen}
        title={"Profilverwaltung"}
```

```
>
  <form
    id={"upd_forum_profile"}
    onSubmit={async (e) => {
      e.preventDefault();

      const data = new FormData(e.target as HTMLFormElement);

      const displayName = data.get("displayName") as string;
      const bio = data.get("bio") as string;
      const website = data.get("website") as string;
      const location = data.get("location") as string;
      const verified = data.get("verified") as string;

      const r = await REST.Admin.updateForumProfile({
        token: localStorage.getItem("token") as string,
        username: props.profile.username,
        update: {
          displayName:
            displayName !== "" &&
            displayName !== props.profile.displayName
              ? displayName
              : props.profile.displayName,
          bio:
            bio !== "" && bio !== props.profile.bio
              ? bio
              : props.profile.bio,
          website:
            website !== "" && website !== props.profile.website
              ? website
              : props.profile.website,
          location:
            location !== "" && location !== props.profile.location
              ? location
              : props.profile.location,
          // @ts-ignore
          verified:
            // @ts-ignore
            verified !== "" && verified !== props.profile.verified
              ? verified
              : props.profile.verified,
        },
      });

      if (r.status === 200) {
        toast({
          title: "Profil aktualisiert",
          description: "Das Profil wurde erfolgreich aktualisiert",
          status: "success",
          duration: 5000,
          isClosable: true,
          position: "top",
        });

        const profile = r.payload.profile;

        props.setProfiles(
          props.profiles.map((p) => {
            if (p.username === profile.username) {
              return profile;
```

```
          } else {
            return p;
          }
        })
      );

      props.setCurrentProfile(profile);

      setEditMode(false);
    }
  }}
>
  <Grid templateColumns="repeat(2, 1fr)" gap={6}>
    <Box>
      {!editMode ? (
        <>
          <Heading size="md">{props.profile.displayName}</Heading>
          <Avatar
            size={"xl"}
            src={"/api/account/" + props.profile.username + "/picture"}
            mt={4}
          />
          <Text mt={4}>
            <b>Benutzername:</b> @{props.profile.username}
          </Text>
          <Text mt={4}>
            <b>Erstellt am:</b>{" "}
            {new Date(props.profile.createdAt).toLocaleString()}
          </Text>
          <Text mt={4} alignItems={"center"} justifyContent={"center"}>
            <b>Verifiziert:</b> 
            {props.profile.verified ? (
              <Icon as={FaCheckCircle} color={"brand.500"} />
            ) : (
              <Icon as={FaTimesCircle} color={"red.500"} />
            )}
          </Text>
          <Text mt={4}>
            <b>Biografie:</b> {props.profile.bio}
          </Text>
          <Text mt={4}>
            <b>Website:</b> {props.profile.website}
          </Text>
          <Text mt={4}>
            <b>Ort:</b> {props.profile.location}
          </Text>
        </>
      ) : (
        <>
          <FormControl id="displayName">
            <FormLabel>Anzeigename</FormLabel>
            <InputGroup>
              <InputLeftElement pointerEvents="none">
                <FaUser />
              </InputLeftElement>
              <Input
                type="text"
                defaultValue={props.profile.displayName}
                name={"displayName"}
              />
```

```
    </InputGroup>
  </FormControl>
  <FormControl mt={4}>
    <FormLabel>Benutzername</FormLabel>
    <InputGroup>
      <InputLeftElement pointerEvents="none">
        <FaUser />
      </InputLeftElement>
      <Input
        type="text"
        defaultValue={props.profile.username}
        disabled
      />
    </InputGroup>
  </FormControl>
  <FormControl id="bio" mt={4}>
    <FormLabel>Biografie</FormLabel>
    <Textarea defaultValue={props.profile.bio} name={"bio"} />
  </FormControl>
  <FormControl id={"website"} mt={4}>
    <FormLabel>Website</FormLabel>
    <InputGroup>
      <InputLeftElement pointerEvents="none">
        <FaGlobe />
      </InputLeftElement>
      <Input
        type="text"
        defaultValue={props.profile.website}
        name={"website"}
      />
    </InputGroup>
  </FormControl>
  <FormControl id="location" mt={4}>
    <FormLabel>Ort</FormLabel>
    <InputGroup>
      <InputLeftElement pointerEvents="none">
        <FaMapMarkerAlt />
      </InputLeftElement>
      <Input
        type="text"
        defaultValue={props.profile.location}
        name={"location"}
      />
    </InputGroup>
  </FormControl>
  <FormControl id="verified" mt={4}>
    <FormLabel>Verifiziert</FormLabel>
    <Select name={"verified"}>
      <option value={"true"} selected={props.profile.verified}>
        Ja
      </option>
      <option
        value={"false"}
        selected={!props.profile.verified}
      >
        Nein
      </option>
    </Select>
  </FormControl>
</>
```

```
 )}
</Box>
<Box>
 <Heading size="md">Aktionen</Heading>
 <ButtonGroup mt={4} spacing={4}>
  <Button
   leftIcon={<Icon as={FaEdit} />}
   variant={"brand"}
   onClick={() => {
    setEditMode(true);
   }}
   disabled={editMode}
  >
   Bearbeiten
  </Button>
  <Button
   leftIcon={<Icon as={FaTrash} />}
   colorScheme={"red"}
   onClick={async () => {
    const r = await REST.Admin.deleteForumProfile({
     token: localStorage.getItem("token") as string,
     username: props.profile.username,
    });

    if (r.status === 200) {
     props.setProfiles(
      props.profiles.filter(
       // @ts-ignore
       (profile) =>
        profile.username !== props.profile.username
      )
     );

     props.onClose();
    } else {
     toast({
      title: "Fehler",
      description:
       "Beim Löschen des Profils ist ein Fehler aufgetreten.",
      status: "error",
      duration: 5000,
      isClosable: true,
      position: "top",
     });
    }
   }}
  >
   Löschen
  </Button>
 </ButtonGroup>
 {editMode ? (
  <>
   <Heading size={"md"} mt={4}>
    Bearbeitung
   </Heading>
   <ButtonGroup mt={4} spacing={4}>
    <Button
     leftIcon={<Icon as={FaSave} />}
     variant={"brand"}
     type={"submit"}
```

```
                >
                  Speichern
                </Button>
                <Button
                  leftIcon={<Icon as={FaTimes} />}
                  colorScheme={"red"}
                  onClick={() => {
                    setEditMode(false);
                  }}
                >
                  Abbrechen
                </Button>
              </ButtonGroup>
            </>
          ) : null}
        </Box>
      </Grid>
    </form>
  </CodeUpModal>
 </>
 );
}
```

## packages/components/admin/AdminQuizMaker.tsx

```
/**
 * packages/components/admin/AdminQuizMaker.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 04.11.2022
 *
 */

import * as React from "react";
import { useState } from "react";
import {
  Box,
  Button,
  Heading,
  IconButton,
  Input,
  Table,
  TableContainer,
  Tbody,
  Td,
  Th,
  Thead,
  Tr,
} from "@chakra-ui/react";
import { FaTrash } from "react-icons/fa";

export default function AdminQuizMaker(props: {
  quizQuestions: {
    question: string;
    answers: {
      answer: string;
      correct: boolean;
    }[];
  }[];
```

```
  setQuizQuestions: React.Dispatch<
    React.SetStateAction<
      {
        question: string;
        answers: {
          answer: string;
          correct: boolean;
        }[];
      }[]
    >
  >;
}) {
  const [newQuestion, setNewQuestion] = useState<string>("");

  const quizQuestions = props.quizQuestions;
  const setQuizQuestions = props.setQuizQuestions;

  return (
    <>
      <Heading size={"md"} mt={"2rem"}>
        Quiz
      </Heading>
      <TableContainer mt={"2rem"}>
        <Table variant="simple">
          <Thead>
            <Tr>
              <Th>Frage</Th>
              <Th>Antworten</Th>
              <Th>Aktion</Th>
            </Tr>
          </Thead>
          <Tbody>
            {quizQuestions.map((question, index) => (
              <Tr>
                <Td>
                  {question.question}
                  <IconButton
                    aria-label={"Löschen"}
                    icon={<FaTrash />}
                    ml={"1rem"}
                    colorScheme={"red"}
                    variant={"ghost"}
                    onClick={() => {
                      setQuizQuestions(
                        quizQuestions.filter(
                          (q) => q.question !== question.question
                        )
                      );
                    }}
                  />
                </Td>
                <Td>
                  {question.answers.map((answer, i) => (
                    <Box>
                      <Input
                        mt={2}
                        type={"text"}
                        defaultValue={answer.answer}
                        onChange={(e) => {
                          setQuizQuestions(
```

```
              quizQuestions.map((q) => {
                if (q.question === question.question) {
                  q.answers[i].answer = e.target.value as string;
                }
                return q;
              })
            );
          }}
        />
      </Box>
    ))}
  </Td>
  <Td>
    {question.answers.map((answer, j) => (
      <Box>
        <Button
          mt={2}
          colorScheme={answer.correct ? "green" : "red"}
          onClick={() => {
            setQuizQuestions(
              quizQuestions.map((q) => {
                if (q.question === question.question) {
                  q.answers[j].correct = !q.answers[j].correct;
                }
                return q;
              })
            );
          }}
        >
          {answer.correct ? "Richtig" : "Falsch"}
        </Button>
      </Box>
    ))}
  </Td>
 </Tr>
))}
<Tr>
 <Td>
  <Input
   type={"text"}
   placeholder={"Frage"}
   id={"new-question-input"}
   onChange={(e) => {
    setNewQuestion(e.target.value as string);
   }}
  />
 </Td>
 <Td>
  <Button
   variant={"brand"}
   onClick={() => {
    setQuizQuestions([
     ...quizQuestions,
     {
      question: newQuestion,
      answers: [
       {
        answer: "",
        correct: false,
       },
```

```
                    {
                      answer: "",
                      correct: false,
                    },
                    {
                      answer: "",
                      correct: false,
                    },
                    {
                      answer: "",
                      correct: false,
                    },
                  ],
                },
              ]);

              setNewQuestion("");

              (
                document.getElementById(
                  "new-question-input"
                ) as HTMLInputElement
              ).value = "";
            }}
          >
            Erstellen
          </Button>
        </Td>
        <Td></Td>
      </Tr>
    </Tbody>
  </Table>
 </TableContainer>
 </>
 );
}
```

## packages/components/admin/AdminUserContact.tsx

```
/**
 * packages/components/admin/AdminUserContact.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 13.04.2023
 *
 */

import * as React from "react";
import CodeUpModal from "../codeup/CodeUpModal";
import REST from "@codeup/rest/dist";
import {
  Button,
  FormControl,
  FormLabel,
  Textarea,
  useToast,
} from "@chakra-ui/react";

export default function AdminUserContact(props: {
```

```
  onOpen: () => void;
  onClose: () => void;
  isOpen: boolean;
  title: string;
  username: string;
}) {
  const toast = useToast();

  return (
    <CodeUpModal
      onOpen={props.onOpen}
      onClose={props.onClose}
      isOpen={props.isOpen}
      title={props.title}
      customSize={"md"}
    >
      <form
        onSubmit={async (e) => {
          e.preventDefault();

          const form = e.target as HTMLFormElement;

          const data = new FormData(form);

          const message = data.get("message") as string;

          const res = await REST.Admin.contactUser({
            token: localStorage.getItem("token") as string,
            username: props.username,
            message,
          });

          if (res.status === 200) {
            toast({
              title: "Erfolgreich",
              description: "Nachricht gesendet",
              status: "success",
              duration: 1500,
              isClosable: true,
              position: "bottom-right",
            });
            props.onClose();
          } else {
            toast({
              title: "Fehler",
              description:
                "Es ist ein Fehler aufgetreten: " + res.payload.error,
              status: "error",
              duration: 1500,
              isClosable: true,
              position: "bottom-right",
            });
          }
        }}
      >
        <FormControl>
          <FormLabel>Nachricht</FormLabel>
          <Textarea placeholder={"Nachricht"} name={"message"} />
        </FormControl>
        <Button variant={"brand"} mt={4} type={"submit"} w={"100%"}>
```

```
          Senden
        </Button>
      </form>
    </CodeUpModal>
  );
}
```

## packages/components/admin/AdminUserCreator.tsx

```
/**
 * packages/components/admin/AdminUserCreator.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 08.10.22
 *
 */

import * as React from "react";
import CodeUpModal from "../codeup/CodeUpModal";
import {
  Button,
  FormControl,
  FormLabel,
  Input,
  Stack,
  Flex,
  Box,
  useToast,
  Select,
  Checkbox,
  InputGroup,
  InputLeftElement,
} from "@chakra-ui/react";
import REST from "@codeup/rest";
import { FaLock, FaMailBulk, FaUser, FaUserTag } from "react-icons/fa";

export default function AdminUserCreator(props: {
  onOpen: () => void;
  onClose: () => void;
  isOpen: boolean;
  setUsers: (users: any) => void;
  users: any[];
}) {
  const toast = useToast();

  return (
    <>
      <CodeUpModal
        onOpen={props.onOpen}
        onClose={props.onClose}
        isOpen={props.isOpen}
        title={"Benutzer Erstellen"}
      >
        <form
          onSubmit={async (e) => {
            e.preventDefault();

            const form = e.target as HTMLFormElement;
```

```
    // @ts-ignore
    const name = form.username.value;
    // @ts-ignore
    const firstName = form.firstName.value;
    // @ts-ignore
    const lastName = form.lastName.value;
    // @ts-ignore
    const email = form.email.value;
    // @ts-ignore
    const password = form.password.value;
    // @ts-ignore
    const role = form.role.value;
    // @ts-ignore
    const active = form.active.checked;

    const a = await REST.Admin.createAccount({
      username: name,
      email: email,
      password: password,
      role: role,
      active: active,
      firstName: firstName,
      lastName: lastName,
      token: localStorage.getItem("token") as string,
    });

    if (a.status === 200) {
      props.setUsers([...props.users, a.payload.user]);
      props.onClose();
    } else {
      toast({
        title: "Fehler",
        description: a.payload.error,
        status: "error",
        duration: 5000,
        isClosable: true,
      });
    }
  }}
>
  <Stack spacing={3}>
    <FormControl id="username">
      <FormLabel>Benutzername</FormLabel>
      <InputGroup>
        <InputLeftElement>
          <FaUser />
        </InputLeftElement>
        <Input type="text" name={"username"} />
      </InputGroup>
    </FormControl>
    <FormControl id={"email"}>
      <FormLabel>E-Mail</FormLabel>
      <InputGroup>
        <InputLeftElement>
          <FaMailBulk />
        </InputLeftElement>
        <Input type="email" name={"mail"} />
      </InputGroup>
    </FormControl>
    <FormControl id={"firstName"}>
```

```
            <FormLabel>Vorname</FormLabel>
            <InputGroup>
              <InputLeftElement>
                <FaUser />
              </InputLeftElement>
              <Input type="text" name={"firstName"} />
            </InputGroup>
          </FormControl>
          <FormControl id={"lastName"}>
            <FormLabel>Nachname</FormLabel>
            <InputGroup>
              <InputLeftElement>
                <FaUser />
              </InputLeftElement>
              <Input type="text" name={"lastName"} />
            </InputGroup>
          </FormControl>
          <FormControl id="password">
            <FormLabel>Passwort</FormLabel>
            <InputGroup>
              <InputLeftElement>
                <FaLock />
              </InputLeftElement>
              <Input type="password" name={"password"} />
            </InputGroup>
          </FormControl>
          <FormControl id="role">
            <FormLabel>Rolle</FormLabel>
            <InputGroup>
              <InputLeftElement>
                <FaUserTag />
              </InputLeftElement>
              <Input type="text" name={"role"} defaultValue={"user"} />
            </InputGroup>
          </FormControl>
          <FormControl id="active">
            <FormLabel>Aktiv</FormLabel>
            <Checkbox name={"active"} defaultChecked colorScheme={"brand"}>
              Aktiv
            </Checkbox>
          </FormControl>

          <Flex>
            <Button variant={"brand"} type="submit" w={"100%"}>
              Erstellen
            </Button>
          </Flex>
        </Stack>
      </form>
    </CodeUpModal>
  </>
 );
}
```

## packages/components/admin/AdminUserViewer.tsx

```
/**
 * packages/components/admin/AdminUserViewer.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
```

```
 * License: Project License
 * Created At: 07.10.22
 *
 */

import * as React from "react";
import CodeUpModal from "../codeup/CodeUpModal";
import {
  Alert,
  AlertDescription,
  AlertIcon,
  AlertTitle,
  Avatar,
  Box,
  Button,
  ButtonGroup,
  Center,
  CircularProgress,
  Flex,
  FormControl,
  FormLabel,
  Grid,
  Heading,
  HStack,
  Icon,
  IconButton,
  Input,
  InputGroup,
  InputLeftElement,
  Menu,
  MenuButton,
  MenuItem,
  MenuList,
  Progress,
  Spinner,
  Stack,
  Table,
  TableContainer,
  Tbody,
  Td,
  Text,
  Th,
  Thead,
  Tr,
  useDisclosure,
} from "@chakra-ui/react";
import {
  FaCheck,
  FaCheckSquare,
  FaCross,
  FaEdit,
  FaTrash,
  FaWindowClose,
  FaUser,
  FaLock,
  FaEnvelope,
  FaPen,
  FaUserShield,
  FaUserTag,
  FaChartLine,
```

```
} from "react-icons/fa";
import { useEffect } from "react";
import REST from "@codeup/rest";
import { useLoggedIn, useUserInfo } from "@codeup/hooks";
import PopupManager from "@codeup/web/src/global/PopupManager";
import AccountManager from "@codeup/util/src/AccountManager";
import { useRouter } from "next/router";
import { FiMail } from "react-icons/fi";
import AdminUserContact from "./AdminUserContact";
import { CHeading } from "../global/CHeading";

export default function AdminUserViewer(props: {
  user?: {
    id: string;
    username: string;
    email: string;
    password: string;
    firstName: string;
    lastName: string;
    twoFactorEnabled: boolean;
    twoFactorCode: string;
    active: boolean;
    activationToken: string;
    deletionToken: string;
    resetToken: string;
    emailChangeCode: string;
    newEmail: string;
    profilePicture: string;
    createdAt: string;
    sso: string;
    role: string;
    __v: number;
  };
  onOpen: () => void;
  onClose: () => void;
  isOpen: boolean;
  users: any[];
  setUsers: (users: any[]) => void;
  setCurrentUser: (user: any) => void;
}) {
  const [editMode, setEditMode] = React.useState(false);

  const { userInfo, loggedIn, loaded } = useLoggedIn();

  const [analyticsData, setAnalyticsData] = React.useState<{
    first: {
      view: string;
      page: string;
      browser: string;
      os: string;
    };
    last: {
      view: string;
      page: string;
      browser: string;
      os: string;
    };
    totalViews: number;
    averageViewsPerDay: number;
  }>();
```

```
const { isOpen, onOpen, onClose } = useDisclosure();

React.useEffect(() => {
  if (props.user) {
    REST.Admin.getUserAnalytics({
      token: AccountManager.getActiveToken(),
      // @ts-ignore
      userId: props.user._id,
    }).then((res) => {
      setAnalyticsData(res.payload as any);
    });
  }
}, [props.user]);

const router = useRouter();

return (
  <>
    <CodeUpModal
      onOpen={props.onOpen}
      onClose={() => {
        setEditMode(false);
        props.onClose();
      }}
      isOpen={props.isOpen}
      title={"Benutzerverwaltung"}
    >
      <Box>
        <Flex alignItems={"center"}>
          <Avatar
            size={"xl"}
            src={"/api/account/" + props.user?.username + "/picture"}
          />
          <Box ml={4}>
            {!editMode ? (
              <>
                <CHeading size={"md"}>
                  {props.user?.firstName} {props.user?.lastName}
                </CHeading>
                <Text>@{props.user?.username}</Text>
                <Text>{props.user?.email}</Text>
              </>
            ) : (
              <>
                <form
                  onSubmit={async (e) => {
                    e.preventDefault();

                    const data = new FormData(e.target as HTMLFormElement);

                    const firstName = data.get("firstName");
                    const lastName = data.get("lastName");
                    const email = data.get("email");
                    const password = data.get("password");
                    const role = data.get("role");

                    const update: { [key: string]: string } = {};

                    if (
```

```
  firstName &&
  firstName !== "" &&
  firstName !== props.user?.firstName
) {
  update["firstName"] = firstName as string;
}

if (
  lastName &&
  lastName !== "" &&
  lastName !== props.user?.lastName
) {
  update["lastName"] = lastName as string;
}

if (
  email &&
  email !== "" &&
  email !== props.user?.email
) {
  update["email"] = email as string;
}

if (
  password &&
  password !== "" &&
  password !== props.user?.password
) {
  update["password"] = password as string;
}

if (role && role !== "" && role !== props.user?.role) {
  update["role"] = role as string;
}

const r = await REST.Admin.updateAccount({
  token: localStorage.getItem("token") as string,
  username: props.user?.username as string,
  update: update,
});

const userU = r.payload.user;

const newUsers = props.users.map((user) => {
  if (user.username === props.user?.username) {
    return userU;
  } else {
    return user;
  }
});

props.setUsers(newUsers);

props.setCurrentUser(userU);

setEditMode(false);

if (userU.username === userInfo.username) {
  localStorage.removeItem("token");
  return;
```

```
    }
  }}
>
  <Stack spacing={2}>
    <FormControl>
      <FormLabel>Benutzername</FormLabel>
      <InputGroup mb={3}>
        <InputLeftElement pointerEvents="none">
          <Icon as={FaUser} />
        </InputLeftElement>
        <Input
          disabled
          placeholder="Benutzername"
          defaultValue={props.user?.username}
        />
      </InputGroup>
    </FormControl>
    <FormControl>
      <FormLabel>Vorname</FormLabel>
      <InputGroup mb={3}>
        <InputLeftElement pointerEvents="none">
          <Icon as={FaPen} />
        </InputLeftElement>
        <Input
          placeholder="Vorname"
          defaultValue={props.user?.firstName}
          name={"firstName"}
        />
      </InputGroup>
    </FormControl>
    <FormControl>
      <FormLabel>Nachname</FormLabel>
      <InputGroup mb={3}>
        <InputLeftElement pointerEvents="none">
          <Icon as={FaPen} />
        </InputLeftElement>
        <Input
          placeholder="Nachname"
          defaultValue={props.user?.lastName}
          name={"lastName"}
        />
      </InputGroup>
    </FormControl>
    <FormControl>
      <FormLabel>E-Mail</FormLabel>
      <InputGroup mb={3}>
        <InputLeftElement pointerEvents="none">
          <Icon as={FaEnvelope} />
        </InputLeftElement>
        <Input
          placeholder="E-Mail"
          defaultValue={props.user?.email}
          type={"email"}
          name={"email"}
        />
      </InputGroup>
    </FormControl>
    <FormControl>
      <FormLabel>Passwort</FormLabel>
      <InputGroup mb={3}>
```

```
            <InputLeftElement pointerEvents="none">
              <Icon as={FaLock} />
            </InputLeftElement>
            <Input
              placeholder="Passwort"
              defaultValue={""}
              name={"password"}
            />
          </InputGroup>
        </FormControl>
        <FormControl>
          <FormLabel>Rolle</FormLabel>
          <InputGroup mb={3}>
            <InputLeftElement pointerEvents="none">
              <Icon as={FaUserTag} />
            </InputLeftElement>
            <Input
              placeholder="Rolle"
              defaultValue={props.user?.role}
              name={"role"}
            />
          </InputGroup>
        </FormControl>
        <ButtonGroup w={"100%"} justifyContent={"end"}>
          <Button
            onClick={() => {
              setEditMode(false);
            }}
            w={"100%"}
          >
            Abbrechen
          </Button>
          <Button variant={"brand"} type={"submit"} w={"100%"}>
            Speichern
          </Button>
        </ButtonGroup>
      </Stack>
    </form>
  </>
 )}
</Box>
{!editMode ? (
  <Flex ml={"auto"} direction={["column", "row"]} gap={2}>
    <IconButton
      aria-label={"message"}
      icon={<FaEnvelope />}
      variant={"brand"}
      onClick={onOpen}
    />
    <IconButton
      aria-label={"analytics"}
      variant={"brand"}
      icon={<FaChartLine />}
      onClick={async () => {
        // @ts-ignore
        router.push("/admin/analytics/" + props.user._id);
      }}
    />
    {props.user?.active ? (
      <>
```

```
<IconButton
  colorScheme={"red"}
  aria-label={"delete"}
  icon={<FaTrash />}
  onClick={async () => {
    if (
      await PopupManager.confirmAsync({
        title: "Benutzer löschen",
        question:
          "Willst du den Benutzer wirklich unwiederruflich löschen?",
      })
    ) {
      const r = await REST.Admin.deleteAccount({
        token: localStorage.getItem("token") as string,
        username: props.user?.username as string,
      });

      if (r.payload.success) {
        const newUsers = props.users.filter(
          (user) => user.username !== props.user?.username
        );

        props.setUsers(newUsers);
        props.setCurrentUser(null);

        props.onClose();
      } else {
        PopupManager.alert({
          title: "Fehler",
          description: r.payload.error,
        });
      }
    }
  }}
/>
<IconButton
  aria-label={"deactivate"}
  icon={<FaWindowClose />}
  colorScheme={"red"}
  onClick={async () => {
    const update: { [key: string]: any } = {
      active: false,
    };

    const r = await REST.Admin.updateAccount({
      token: localStorage.getItem("token") as string,
      username: props.user?.username as string,
      update: update,
    });

    const userU = r.payload.user;

    const newUsers = props.users.map((user) => {
      if (user.username === props.user?.username) {
        return userU;
      } else {
        return user;
      }
    });
```

```
        props.setUsers(newUsers);

        props.setCurrentUser(userU);

        setEditMode(false);
      }}
    />
  </>
) : (
  <>
    <IconButton
      aria-label={"activate"}
      icon={<FaCheckSquare />}
      colorScheme={"green"}
      onClick={async () => {
        const update: { [key: string]: any } = {
          active: true,
        };

        const r = await REST.Admin.updateAccount({
          token: localStorage.getItem("token") as string,
          username: props.user?.username as string,
          update: update,
        });

        const userU = r.payload.user;

        const newUsers = props.users.map((user) => {
          if (user.username === props.user?.username) {
            return userU;
          } else {
            return user;
          }
        });

        props.setUsers(newUsers);

        props.setCurrentUser(userU);

        setEditMode(false);
      }}
    />
  </>
)}
<IconButton
  aria-label={"login as user"}
  icon={<FaLock />}
  variant={"brand"}
  onClick={async () => {
    const jwtRes = await REST.Admin.getUserJWT({
      token: localStorage.getItem("token") as string,
      username: props.user?.username as string,
    });

    if (jwtRes.status === 200) {
      await AccountManager.setActiveToken(jwtRes.payload.jwt);
    } else {
      PopupManager.alert({
        title: "Fehler",
        description: jwtRes.payload.error,
```

```
            });
          }
        }}
      />
      <IconButton
        aria-label={"edit"}
        icon={<FaEdit />}
        variant={"brand"}
        onClick={() => setEditMode(true)}
      />
    </Flex>
  ) : null}
</Flex>
<Box mt={4}>
  <CHeading size={"md"}>Allgemein</CHeading>
  <TableContainer>
    <Table variant={"simple"}>
      <Thead>
        <Tr>
          <Th>Benutzername</Th>
          <Th>E-Mail</Th>
          <Th>Rolle</Th>
          <Th>2FA</Th>
          <Th>Aktiv</Th>
        </Tr>
      </Thead>
      <Tbody>
        <Tr>
          <Td>{props.user?.username}</Td>
          <Td>{props.user?.email}</Td>
          <Td>{props.user?.role}</Td>
          <Td>{props.user?.twoFactorEnabled ? "Ja" : "Nein"}</Td>
          <Td>{props.user?.active ? "Ja" : "Nein"}</Td>
        </Tr>
      </Tbody>
    </Table>
  </TableContainer>
</Box>
<Box mt={4}>
  <CHeading size={"md"}>Sonstiges</CHeading>
  <Table variant={"simple"}>
    <Thead>
      <Tr>
        <Th>SSO</Th>
        <Th>Registrierungsdatum</Th>
        <Th>ID</Th>
      </Tr>
    </Thead>
    <Tbody>
      <Tr>
        <Td>
          {props.user?.sso === "no-sso"
            ? "Echter Account"
            : props.user?.sso}
        </Td>
        <Td>
          {new Date(props.user?.createdAt as string).toLocaleString()}
        </Td>
        <Td>
          {
```

```
              // @ts-ignore
              props.user?._id
            }
          </Td>
        </Tr>
      </Tbody>
    </Table>
  </Box>
  <Box mt={4}>
    <CHeading size={"md"}>Analytics</CHeading>
    <Table variant={"simple"}>
      <Thead>
        <Tr>
          <Th>Erster Login</Th>
          <Th>Letzter Login</Th>
          <Th>Besuche</Th>
          <Th>&#216;Besuche/Tag</Th>
        </Tr>
      </Thead>
      <Tbody>
        <Tr>
          <Td>
            {new Date(
              analyticsData?.first.view as string
            ).toLocaleString()}
            <br />
            <b>Seite: </b>
            {analyticsData?.first.page}
            <br />
            <b>Browser: </b>
            {analyticsData?.first.browser}
            <br />
            <b>OS: </b>
            {analyticsData?.first.os}
          </Td>
          <Td>
            {new Date(
              analyticsData?.last.view as string
            ).toLocaleString()}
            <br />
            <b>Seite: </b>
            {analyticsData?.last.page}
            <br />
            <b>Browser: </b>
            {analyticsData?.last.browser}
            <br />
            <b>OS: </b>
            {analyticsData?.last.os}
          </Td>
          <Td>{analyticsData?.totalViews}</Td>
          <Td>
            {Math.round(analyticsData?.averageViewsPerDay as number)}
          </Td>
        </Tr>
      </Tbody>
    </Table>
  </Box>
 </Box>
</CodeUpModal>
<AdminUserContact
```

```
        onOpen={onOpen}
        isOpen={isOpen}
        onClose={onClose}
        title={"Nachricht an " + (props.user ? props.user.username : "")}
        username={props.user ? props.user.username : ""}
      />
    </>
  );
}
```

## packages/components/admin/AuthenticateLoadingIndicator.tsx

```
/**
 * packages/components/admin/AuthenticateLoadingIndicator.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 07.10.22
 *
 */

import * as React from "react";
import { Flex, CircularProgress, Heading } from "@chakra-ui/react";

export default function AuthenticateLoadingIndicator(props: {
  text?: string;
  inline?: boolean;
}) {
  return (
    <>
      <Flex
        w={"100%"}
        h={!props.inline ? "100vh" : "fit-content"}
        justifyContent={"center"}
        alignItems={"center"}
        direction={"column"}
        bg={"#000"}
      >
        <CircularProgress size={"80px"} isIndeterminate color="brand.500" />
        <Heading size={"lg"} mt={4}>
          {props.text ? props.text : "Authentifiziere..."}
        </Heading>
      </Flex>
    </>
  );
}
```

## packages/components/ask/AskLayout.tsx

```
/**
 * packages/components/ask/AskLayout.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 24.06.23
 *
 */
import { Flex, IconButton, Stack, useDisclosure } from "@chakra-ui/react";
import { FaHome, FaPlus, FaSearch } from "react-icons/fa";
import PopupManager from "@codeup/web/src/global/PopupManager";
```

```
import REST from "@codeup/rest/dist";
import AccountManager from "@codeup/util/dist/AccountManager";
import { AiFillMessage } from "react-icons/ai";
import React from "react";
import { useLoggedIn } from "@codeup/hooks";
import { useRouter } from "next/router";
import FTSPopup from "./FTSPopup";

export default function AskLayout(props: { children: React.ReactNode }) {
  const { loggedIn, userInfo } = useLoggedIn();
  const router = useRouter();

  const {
    isOpen: isFTSOpen,
    onOpen: onFTSOpen,
    onClose: onFTSClose,
  } = useDisclosure();

  return (
    <>
      <Flex
        w={"100%"}
        h={"fit-content"}
        minH={"100vh"}
        direction={["column", "column", "row"]}
        paddingInline={["0", "0", "0", "20%"]}
        paddingBlock={"2rem"}
        bg={"#000"}
      >
        <Flex
          flex={["100%", "100%", "100%", "10%"]}
          w={"100%"}
          roundedTopLeft={"xl"}
          roundedBottomLeft={"xl"}
          backgroundColor={"black"}
        >
          <Stack w={"100%"}>
            <IconButton
              aria-label={""}
              variant="ghost"
              size={"lg"}
              fontSize={"2xl"}
              p={"2rem"}
              icon={<FaHome />}
              rounded={"none"}
              roundedTopLeft={"xl"}
              onClick={() => {
                router.push("/ask");
              }}
            />
            <IconButton
              aria-label={""}
              variant="ghost"
              size={"lg"}
              fontSize={"2xl"}
              p={"2rem"}
              icon={<FaPlus />}
              rounded={"none"}
              onClick={async () => {
                if (!loggedIn) {
```

```
              window.dispatchEvent(new CustomEvent("codeup::open_login"));
              return;
            }

            const content = await PopupManager.promptAsync({
              title: "Neuer Post",
              helperText: "Was möchtest du posten?",
              inputType: "TEXTAREA",
            });

            if (!content) return;

            const res = await REST.Ask.createPost(
              content,
              AccountManager.getActiveToken()
            );

            PopupManager.alertAsync({
              title: "Erfolgreich",
              description: "Dein Post wurde erfolgreich erstellt.",
            });
          }}
        />
        <IconButton
          aria-label={""}
          variant="ghost"
          size={"lg"}
          fontSize={"2xl"}
          p={"2rem"}
          icon={<FaSearch />}
          rounded={"none"}
          onClick={() => {
            onFTSOpen();
          }}
        />
        <IconButton
          aria-label={""}
          variant="ghost"
          size={"lg"}
          fontSize={"2xl"}
          p={"2rem"}
          icon={<AiFillMessage />}
          rounded={"none"}
          onClick={async () => {
            if (!loggedIn) {
              window.dispatchEvent(new CustomEvent("codeup::open_login"));
              return;
            }
            router.push("/ask/direct");
          }}
        />
      </Stack>
    </Flex>
    {props.children}
  </Flex>
  <FTSPopup isOpen={isFTSOpen} onOpen={onFTSOpen} onClose={onFTSClose} />
 </>
 );
}
```

## packages/components/ask/AskProfileCertificateDisplay.tsx

```
/**
 * packages/components/ask/AskProfileCertificateDisplay.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 02.07.2023
 *
 */

import * as React from "react";
import REST from "@codeup/rest/dist";
import AccountManager from "@codeup/util/dist/AccountManager";
import { Flex, Image, Link, SimpleGrid, Stack, Text } from "@chakra-ui/react";
import { FaTimes } from "react-icons/fa";

export default function AskProfileCertificateDisplay(props: {
  certificates: string[];
}) {
  const [certificates, setCertificates] = React.useState<any[]>([]);

  React.useEffect(() => {
    (async () => {
      let certs: any[] = [];
      for (const c of props.certificates) {
        const res = await REST.Certificates.getObtainedCertificate(c);
        if (res.status === 200) {
          certs.push({
            cert: res.payload.obtainedCertificate,
            course: res.payload.course,
          });
        }
      }
      setCertificates(certs);
      console.log(certs);
    })();
  }, [props.certificates]);

  return (
    <>
      {certificates.length === 0 && (
        <>
          <Flex
            alignItems={"center"}
            gap={4}
            color={"red.400"}
            fontWeight={"bold"}
          >
            <FaTimes />
            <Text>Keine Zertifikate angegeben</Text>
          </Flex>
        </>
      )}
      {certificates.length > 0 && (
        <>
          <SimpleGrid columns={[1, 1, 2, 3]} spacing={4}>
            {certificates.map((c) => {
              return (
                <Link
```

```
              key={c.cert.id}
              isExternal={true}
              href={"/user-cert/" + c.cert._id}
            >
              <Flex alignItems={"center"} gap={4} fontWeight={"bold"}>
                <Image src={c.course.splashImage} maxW={"128px"} />
                <Stack>
                  <Text>{c.course.name}</Text>
                  <Text
                    color={
                      c.cert.percentage >= 80
                        ? "green.400"
                        : c.cert.percentage >= 50
                        ? "yellow.400"
                        : "red.400"
                    }
                  >
                    {c.cert.percentage} %
                  </Text>
                </Stack>
              </Flex>
            </Link>
          );
        })}
      </SimpleGrid>
    </>
  )}
 </>
 );
}
```

## packages/components/ask/AskProfileProjectDisplay.tsx

```
/**
 * packages/components/ask/AskProfileProjectDisplay.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 02.07.2023
 *
 */

import * as React from "react";
import REST from "@codeup/rest/dist";
import { Flex, Image, Link, SimpleGrid, Stack, Text } from "@chakra-ui/react";
import { FaTimes } from "react-icons/fa";

export default function AskProfileProjectDisplay(props: {
  projects: string[];
}) {
  const [projects, setProjects] = React.useState<any[]>([]);

  React.useEffect(() => {
    (async () => {
      const pro: any[] = [];
      for (const p of props.projects) {
        const res = await REST.Editor.getDomainById(p);
        if (res.status === 200) {
          pro.push(res.payload.project);
        }
```

```
      }
      setProjects(pro);
    })();
  }, [props.projects]);

  return (
    <>
      {projects.length === 0 && (
        <>
          <Flex
            alignItems={"center"}
            gap={4}
            color={"red.400"}
            fontWeight={"bold"}
          >
            <FaTimes />
            <Text>Keine Projekte angegeben</Text>
          </Flex>
        </>
      )}
      {projects.length > 0 && (
        <>
          <SimpleGrid columns={[3, 3, 4, 5]} spacing={4}>
            {projects.map((p) => {
              return (
                <Link key={p._id} isExternal={true} href={"http://" + p.domain}>
                  <Flex alignItems={"center"} gap={4} fontWeight={"bold"}>
                    <Stack>
                      <Text>{p.name}</Text>
                    </Stack>
                  </Flex>
                </Link>
              );
            })}
          </SimpleGrid>
        </>
      )}
    </>
  );
}
```

## packages/components/ask/EditProfileModal.tsx

```
/**
 * packages/components/ask/EditProfileModal.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 27.06.23
 *
 */
import { IAskProfile } from "@codeup/web/src/types/IAskProfile";
import React from "react";
import CodeUpModal from "../codeup/CodeUpModal";
import {
  Button,
  Checkbox,
  CheckboxGroup,
  FormControl,
  FormLabel,
```

```
  Image,
  Input,
  InputGroup,
  InputLeftElement,
  Stack,
  Textarea,
} from "@chakra-ui/react";
import { FaGlobe, FaLink, FaSave } from "react-icons/fa";
import PopupManager from "@codeup/web/src/global/PopupManager";
import REST from "@codeup/rest/dist";
import AccountManager from "@codeup/util/dist/AccountManager";
import { useRouter } from "next/router";

export default function EditProfileModal(props: {
  onOpen: () => void;
  isOpen: boolean;
  onClose: () => void;
  profile: IAskProfile;
}) {
  const router = useRouter();
  const [banner, setBanner] = React.useState<string>(props.profile.banner);
  const [certificates, setCertificates] = React.useState<any[]>([]);
  const [activeCerts, setActiveCerts] = React.useState<string[]>([]);
  const [projects, setProjects] = React.useState<any[]>([]);
  const [activeProjcts, setActiveProjects] = React.useState<string[]>([]);

  React.useEffect(() => {
    if (props.isOpen) {
      setBanner(props.profile.banner);
      setActiveCerts(props.profile.certificates);
      setActiveProjects(props.profile.projects);
      REST.Certificates.getObtainedCertificates(
        AccountManager.getActiveToken()
      ).then((res) => {
        if (res.status === 200) {
          setCertificates(res.payload.certs);
        }
      });
      REST.Editor.getProjectsV2(AccountManager.getActiveToken()).then((res) => {
        if (res.status === 200) {
          setProjects(
            res.payload.projects.filter(
              (p: any) => !p.customDomain.startsWith("disabled:")
            )
          );
        }
      });
    }
  }, [props.isOpen]);

  return (
    <>
      <CodeUpModal
        onOpen={props.onOpen}
        onClose={props.onClose}
        isOpen={props.isOpen}
        title={"Profil bearbeiten"}
        customSize={["full", "2xl"]}
      >
        <form
```

```
  onSubmit={async (e) => {
    e.preventDefault();

    const data = new FormData(e.target as HTMLFormElement);
    let website = data.get("website") as string;
    let bio = data.get("bio") as string;
    let bannerUrl = banner;

    let update: {
      [key: string]: any;
    } = {
      certificates: activeCerts,
      projects: activeProjcts,
    };

    if (bannerUrl !== props.profile.banner) {
      update["banner"] = bannerUrl;
    }

    if (website.length > 0) {
      update["website"] = website;
    } else {
      update["website"] = "codeup.remove()";
    }

    if (bio.length > 0) {
      update["bio"] = bio;
    } else {
      update["bio"] = "codeup.remove()";
    }

    const res = await REST.Ask.updateProfile(
      props.profile.username,
      AccountManager.getActiveToken(),
      update
    );

    if (res.status === 200) {
      router.reload();
    } else {
      await PopupManager.alertAsync({
        title: "Fehler",
        description: "Dein Profil konnte nicht aktualisiert werden.",
      });
    }
  }}
>
  <Stack gap={4}>
    <FormControl>
      <FormLabel>Webseite</FormLabel>
      <InputGroup>
        <InputLeftElement>
          <FaLink />
        </InputLeftElement>
        <Input
          type={"url"}
          name={"website"}
          placeholder={"https://my.codeup.space"}
          defaultValue={props.profile.website}
        />
```

```
  </InputGroup>
</FormControl>
<FormControl>
  <FormLabel>Biography</FormLabel>
  <Textarea
    name={"bio"}
    placeholder={"Erzähle etwas über dich"}
    maxLength={250}
    defaultValue={props.profile.bio}
  />
</FormControl>
<FormControl>
  <FormLabel>Banner</FormLabel>
  <Image
    src={banner}
    w={"100%"}
    h={[312, 624]}
    maxHeight={[250, 450]}
    maxW={"100vw"}
    objectFit={"cover"}
    borderRadius={"12px"}
    borderTopRadius={0}
    alt={"Banner"}
    shadow={"lg"}
    cursor={"pointer"}
    onClick={async () => {
      const imageFL = await PopupManager.uploadFileAsync({
        title: "Banner hochladen",
        helperText: "Lade ein neues Banner hoch",
      });

      if (!imageFL || !imageFL.item(0)) return;

      const form = document.createElement("form");

      const input = document.createElement("input");
      input.type = "file";
      input.name = "file";
      input.files = imageFL;

      form.appendChild(input);

      const formData = new FormData(form);

      const r = await fetch("/media/upload", {
        method: "POST",
        body: formData,
      });

      if (r.status === 200) {
        const data = await r.json();
        setBanner(data.data.url);
      } else {
        await PopupManager.alertAsync({
          title: "Fehler",
          description:
            "Beim Upload ist ein Fehler aufgetreten. Bitte versuche es erneut.",
        });
      }
    }}
```

```
            />
          </FormControl>
          <FormControl>
            <FormLabel>Zertifikate</FormLabel>
            <CheckboxGroup
              colorScheme={"brand"}
              value={activeCerts}
              onChange={(e) => {
                setActiveCerts(e as string[]);
              }}
            >
              {certificates.map((cert) => {
                return (
                  <>
                    <Checkbox mr={4} value={cert._id}>
                      {cert.course}
                    </Checkbox>
                  </>
                );
              })}
            </CheckboxGroup>
          </FormControl>
          <FormControl>
            <FormLabel>Projekte</FormLabel>
            <CheckboxGroup
              colorScheme={"brand"}
              value={activeProjcts}
              onChange={(e) => {
                setActiveProjects(e as string[]);
              }}
            >
              {projects.map((proj) => {
                return (
                  <>
                    <Checkbox mr={4} value={proj._id}>
                      {proj.name}
                    </Checkbox>
                  </>
                );
              })}
            </CheckboxGroup>
          </FormControl>
          <Button leftIcon={<FaSave />} variant={"brand"} type={"submit"}>
            Speichern
          </Button>
        </Stack>
      </form>
    </CodeUpModal>
  </>
 );
}
```

## packages/components/ask/FTSPopup.tsx

```
/**
 * packages/components/ask/FTSPopup.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 24.06.23
```

```
 *
 */

import React from "react";
import CodeUpModal from "../codeup/CodeUpModal";
import { IAskPost } from "@codeup/web/src/types/IAskPost";
import REST from "@codeup/rest/dist";
import {
  Input,
  InputGroup,
  InputLeftElement,
  Spinner,
  Stack,
} from "@chakra-ui/react";
import { FaSearch } from "react-icons/fa";
import PostCardV3 from "./PostCardV3";
import { useLoggedIn } from "@codeup/hooks";
import InfiniteScroll from "react-infinite-scroll-component";
import { useRouter } from "next/router";

export default function FTSPopup(props: {
  isOpen: boolean;
  onOpen: () => void;
  onClose: () => void;
}) {
  const [query, setQuery] = React.useState<string>("");
  const [posts, setPosts] = React.useState<IAskPost[]>([]);
  const [page, setPage] = React.useState<number>(0);
  const [maxPages, setMaxPages] = React.useState<number>(0);

  const updateResults = () => {
    REST.Ask.searchPosts(query, 1).then((res) => {
      setPosts(res.payload.posts);
      setPage(1);
      setMaxPages(res.payload.pages);
    });
  };

  const loadNext = () => {
    REST.Ask.searchPosts(query, page + 1).then((res) => {
      setPosts([...posts, ...res.payload.posts]);
      setPage(page + 1);
      setMaxPages(res.payload.pages);
    });
  };

  React.useEffect(updateResults, [query]);

  const { loggedIn, userInfo } = useLoggedIn();
  const router = useRouter();

  return (
    <>
      <CodeUpModal
        onOpen={props.onOpen}
        onClose={props.onClose}
        isOpen={props.isOpen}
        title={"Suche"}
        customSize={["full", "3xl"]}
      >
```

```
        <InputGroup>
          <InputLeftElement>
            <FaSearch />
          </InputLeftElement>
          <Input
            placeholder={"Suche"}
            value={query}
            onChange={(e) => {
              setQuery(e.target.value);
            }}
          />
        </InputGroup>
        <Stack
          w={"100%"}
          rounded={"xl"}
          maxH={"100%"}
          overflow={"auto"}
          id={"scrollableDiv"}
          mt={2}
        >
          <InfiniteScroll
            next={loadNext}
            hasMore={page < maxPages && posts.length > 0}
            loader={<Spinner />}
            dataLength={posts.length}
            scrollableTarget="scrollableDiv"
          >
            {posts.map((p) => {
              return (
                <>
                  <PostCardV3
                    post={p}
                    loggedIn={loggedIn}
                    owner={
                      p.username === userInfo.username ||
                      userInfo.role === "admin"
                    }
                    setPosts={setPosts}
                    onClick={() => {
                      router.push("/ask/t/" + p._id);
                    }}
                  />
                </>
              );
            })}
          </InfiniteScroll>
        </Stack>
      </CodeUpModal>
    </>
  );
}
```

## packages/components/ask/PostCardV3.tsx

```
/**
 * packages/components/ask/PostCardV3.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 24.06.23
```

```
 *
 */
import { IAskPost } from "@codeup/web/src/types/IAskPost";
import React from "react";
import {
  Avatar,
  Box,
  ButtonGroup,
  Flex,
  Heading,
  HStack,
  IconButton,
  Stack,
  Text,
} from "@chakra-ui/react";
import { AiFillMessage } from "react-icons/ai";
import { FaTrash, FaUser } from "react-icons/fa";
import PopupManager from "@codeup/web/src/global/PopupManager";
import REST from "@codeup/rest/dist";
import AccountManager from "@codeup/util/dist/AccountManager";
import { FiDownload, FiFile } from "react-icons/fi";
import { useRouter } from "next/router";

export default function PostCardV3(props: {
  post: IAskPost;
  onClick?: () => void;
  loggedIn?: boolean;
  owner?: boolean;
  setPosts: React.Dispatch<React.SetStateAction<IAskPost[]>>;
}) {
  const router = useRouter();

  return (
    <>
      <Box bg={"black"} w={"100%"} h={"fit-content"} minH={"25%"}>
        <HStack
          gap={4}
          h={"fit-content"}
          _hover={{
            cursor: "pointer",
            backgroundColor: "gray.900",
          }}
          p={8}
          onClick={props.onClick}
        >
          <Flex
            alignItems={"center"}
            justifyContent={"center"}
            h={"100%"}
            flex={"30%"}
          >
            <Stack>
              <Avatar
                size={"lg"}
                src={"/api/account/" + props.post.username + "/picture"}
              />
              <Heading size={"md"}>@{props.post.username}</Heading>
              <Text>{new Date(props.post.createdAt).toLocaleString()}</Text>
            </Stack>
          </Flex>
```

```
<Box h={"100%"} flex={"100%"}>
  <Text fontSize={"xl"}>{props.post.content}</Text>
  <Stack
    w={"100%"}
    p={4}
    justifyContent={"center"}
    alignItems={"center"}
  >
    {
      // @ts-ignore
      props.post.embeds.map((e) => {
        if (e.type === "link") {
          return (
            <>
              <Box mt={2} p={4} bg={"gray.800"} borderRadius={"md"}>
                <HStack gap={4}>
                  <Stack>
                    {e.site_name && <Text>{e.site_name}</Text>}
                    {e.title && (
                      <Heading
                        size={"md"}
                        color={"blue.300"}
                        cursor={"pointer"}
                        as={"a"}
                        href={e.url}
                        target={"_blank"}
                      >
                        {e.title}
                      </Heading>
                    )}
                    {e.description && (
                      <Text mt={2}>{e.description}</Text>
                    )}
                  </Stack>
                  {e.image && (
                    <img
                      src={e.image}
                      width={"128"}
                      height={"128"}
                      alt={e.image || ""}
                    />
                  )}
                </HStack>
              </Box>
            </>
          );
        } else if (e.type === "image") {
          return (
            <>
              <Box mt={2} p={4} bg={"gray.800"} borderRadius={"md"}>
                <img
                  src={e.url}
                  alt={e.url || ""}
                  style={{
                    maxWidth: "600px",
                    maxHeight: "400px",
                  }}
                />
              </Box>
            </>
```

```
          );
        } else if (e.type === "video") {
          return (
            <>
              <Box mt={2} p={4} bg={"gray.800"} borderRadius={"md"}>
                <video src={e.url} controls />
              </Box>
            </>
          );
        } else if (e.type === "file") {
          return (
            <>
              <Flex
                alignItems={"center"}
                justifyContent={"space-between"}
                w={"100%"}
                p={4}
                rounded={"xl"}
                bg={"gray.800"}
                shadow={"xl"}
                gap={4}
              >
                <Flex alignItems={"center"} gap={"1rem"}>
                  <FiFile />
                  <Text>{e.url.split("/").pop()}</Text>
                </Flex>
                <IconButton
                  as={"a"}
                  href={e.url}
                  target={"_blank"}
                  rel={"noreferrer"}
                  colorScheme={"blue"}
                  icon={<FiDownload />}
                  aria-label={"Download"}
                />
              </Flex>
            </>
          );
        }
      })
    }
  </Stack>
 </Box>
</HStack>
<Flex
 direction={"row"}
 w={"100%"}
 gap={2}
 h={"20%"}
 justifyContent={"flex-end"}
>
 <IconButton
  variant={"ghost"}
  size={"lg"}
  aria-label={"Kommentieren"}
  icon={<AiFillMessage />}
  disabled={props.post.parent !== null}
  onClick={async () => {
   if (!props.loggedIn) {
    window.dispatchEvent(new CustomEvent("codeup::open_login"));
```

```
            return;
          }

          const content = await PopupManager.promptAsync({
            title: "Kommentieren",
            helperText: "Was möchtest du kommentieren?",
            inputType: "TEXTAREA",
          });

          if (!content) return;

          const res = await REST.Ask.createPost(
            content,
            AccountManager.getActiveToken(),
            props.post._id
          );

          PopupManager.alert({
            title: "Kommentar erstellt",
            description: "Dein Kommentar wurde erfolgreich erstellt.",
          });
        }}
      />
      {props.owner && (
        <IconButton
          variant={"ghost"}
          aria-label={"Löschen"}
          size={"lg"}
          icon={<FaTrash />}
          onClick={async () => {
            if (
              !(await PopupManager.confirmAsync({
                title: "Post löschen",
                question: "Möchtest du diesen Post wirklich löschen?",
              }))
            )
              return;

            const res = await REST.Ask.deletePost(
              props.post._id,
              AccountManager.getActiveToken()
            );

            if (res.status === 200) {
              await PopupManager.alertAsync({
                title: "Post gelöscht",
                description:
                  "Der Post wurde erfolgreich gelöscht. Bitte beachte, dass es einige Minuten dauern
kann, bis er nicht mehr sichtbar ist.",
              });
            } else {
              await PopupManager.alertAsync({
                title: "Fehler",
                description: "Der Post konnte nicht gelöscht werden.",
              });
            }

            props.setPosts((prev) => {
              return prev.filter((p) => p._id !== props.post._id);
            });
```

```
            }}
          />
        )}
        <IconButton
          variant={"ghost"}
          aria-label={"Profil aufrufen"}
          size={"lg"}
          icon={<FaUser />}
          onClick={async () => {
            router.push("/ask/u/@" + props.post.username);
          }}
        />
      </Flex>
    </Box>
  </>
 );
}
```

## packages/components/codeup/CodeUpModal.tsx

```
/**
 * packages/components/codeup/CodeUpModal.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 18.09.2022
 *
 */

import * as React from "react";
import {
  Button,
  Flex,
  Heading,
  Modal,
  ModalBody,
  ModalCloseButton,
  ModalContent,
  ModalFooter,
  ModalHeader,
  ModalOverlay,
} from "@chakra-ui/react";
import { CHeading } from "../global/CHeading";

export default function CodeUpModal(props: {
  onOpen: () => void;
  onClose: () => void;
  isOpen: boolean;
  noCloseButton?: boolean;
  children: React.ReactNode;
  title?: string;
  blackBackground?: boolean;
  customSize?: string | string[];
}) {
  return (
    <>
      <Modal
        isOpen={props.isOpen}
        onClose={props.onClose}
        size={props.customSize ? props.customSize : ["full", "2xl"]}
```

```
        isCentered
        scrollBehavior={"inside"}
      >
        <ModalOverlay
          bg={props.blackBackground ? "black" : "blackAlpha.300"}
          backdropFilter={props.blackBackground ? "blur(0px)" : "blur(10px)"}
        />
        <ModalContent backgroundColor={"#000"}>
          <ModalHeader>
            {props.title ? (
              <CHeading size={"lg"}>{props.title}</CHeading>
            ) : null}
            {!props.noCloseButton ? <ModalCloseButton /> : null}
          </ModalHeader>
          <ModalBody>{props.children}</ModalBody>
          <ModalFooter>
            <Flex
              gap={"1.2rem"}
              direction={"row"}
              alignItems={"center"}
              justifyContent={"left"}
              w={"100%"}
            >
              <img src={"/codeup.svg"} width={64} height={64} />
              <Heading size={"lg"}>CodeUp</Heading>
            </Flex>
            {!props.noCloseButton ? (
              <Button variant={"brand"} onClick={props.onClose}>
                Schließen
              </Button>
            ) : null}
          </ModalFooter>
        </ModalContent>
      </Modal>
    </>
  );
}
```

## packages/components/community/CommunityNavigationBar.tsx

```
/**
 * src/components/community/CommunityNavigationBar.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 21.06.2022
 *
 */

import * as React from "react";
import {
  Box,
  Button,
  Divider,
  Drawer,
  DrawerBody,
  DrawerCloseButton,
  DrawerContent,
  DrawerHeader,
  DrawerOverlay,
```

```
  Flex,
  Heading,
  IconButton,
  Link as ChakraLink,
  useDisclosure,
  useMediaQuery,
} from "@chakra-ui/react";
import { GiHamburgerMenu } from "react-icons/gi";
import { useLoggedIn } from "@codeup/hooks";
import Link from "next/link";

export default function CommunityNavigationBar() {
  const [isMobile] = useMediaQuery("(max-width: 768px)");
  const { isOpen, onOpen, onClose } = useDisclosure();

  const { loggedIn, userInfo } = useLoggedIn();
  const Menu = () => {
    return (
      <>
        <Link href={"/community"}>
          <Button variant={"ghost"} size={"lg"} onClick={onClose}>
            Forum
          </Button>
        </Link>
        <Link href={"/community/profile/" + userInfo.username}>
          <Button variant={"ghost"} size={"lg"} onClick={onClose}>
            Mein Profil
          </Button>
        </Link>
        <Link href={"/community/search"}>
          <Button variant={"ghost"} size={"lg"} onClick={onClose}>
            Suchen
          </Button>
        </Link>
      </>
    );
  };

  return (
    <>
      <Box w={"100%"} bg={"gray.900"} padding={"1.2rem"}>
        <Flex direction={"row"} alignItems={"center"} justifyContent={"center"}>
          <Heading flex={"5%"} textAlign={["center", "left"]}>
            <ChakraLink href={"/community"}>
              <Link href={"/community"}>Forum (veraltet)</Link>
            </ChakraLink>
          </Heading>
          <Flex justifyContent={"right"} flex={"100%"} alignItems={"right"}>
            {!isMobile ? (
              <Menu />
            ) : (
              <>
                <IconButton
                  onClick={onOpen}
                  aria-label={"Menu"}
                  icon={<GiHamburgerMenu />}
                  flex={"flex-end"}
                />
                <Drawer
                  isOpen={isOpen}
```

```
              placement="right"
              onClose={onClose}
              size={"full"}
            >
              <DrawerOverlay />
              <DrawerContent bg={"black"}>
                <DrawerCloseButton />
                <DrawerHeader>CodeUp | Forum (veraltet)</DrawerHeader>

                <DrawerBody>
                  <Flex w={"100%"} direction={"column"}>
                    <Menu />
                  </Flex>
                </DrawerBody>
              </DrawerContent>
            </Drawer>
          </>
        )}
      </Flex>
    </Flex>
  </Box>
  <Divider />
 </>
 );
}
```

## packages/components/community/index.ts

```
/**
 * packages/components/community/index.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 03.07.2022
 *
 */

import * as React from "react";
export { default as CommunityNavigationBar } from "./CommunityNavigationBar";
```

## packages/components/community/post/index.ts

```
/**
 * packages/components/community/post/index.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 03.07.2022
 *
 */

import * as React from "react";
import PostButton from "./PostButton";
import PostCard from "./PostCard";
import PostMediaUploader from "./PostMediaUploader";
import { StarterKit } from "./StarterKit";
import TipTap from "./TipTap";

export { PostButton, PostCard, PostMediaUploader, StarterKit, TipTap };
```

## packages/components/community/post/PostButton.tsx

```
/**
 * src/components/community/post/PostButton.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 19.06.2022
 *
 */

import * as React from "react";
import { useEffect, useState } from "react";
import {
  IconButton,
  Modal,
  ModalBody,
  ModalCloseButton,
  ModalContent,
  ModalHeader,
  ModalOverlay,
  useDisclosure,
} from "@chakra-ui/react";
import { FaPlus } from "react-icons/fa";
import TipTap from "./TipTap";
import { useLoggedIn } from "@codeup/hooks";

export default function PostButton() {
  const { isOpen, onOpen, onClose } = useDisclosure();
  const [value, setValue] = useState("");

  const [showPostButton, setShowPostButton] = useState(false);

  const { loggedIn, userInfo } = useLoggedIn();

  useEffect(() => {
    setShowPostButton(loggedIn);
  }, [loggedIn]);

  return (
    <>
      <Modal isOpen={isOpen} onClose={onClose} size={["full", "4xl"]}>
        <ModalOverlay />
        <ModalContent>
          <ModalHeader>Post</ModalHeader>
          <ModalCloseButton />
          <ModalBody padding={"2rem"}>
            <TipTap />
          </ModalBody>
        </ModalContent>
      </Modal>

      <div
        style={{
          position: "fixed",
          right: "5%",
          bottom: "5%",
          display: showPostButton ? "block" : "none",
        }}
      >
```

```
        <IconButton
          aria-label={"Post"}
          icon={<FaPlus />}
          size={"lg"}
          colorScheme={"whatsapp"}
          onClick={onOpen}
        />
      </div>
    </>
  );
}
```

## packages/components/community/post/PostCard.tsx

```
/**
 * src/components/community/post/PostCard.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 19.06.2022
 *
 */

import * as React from "react";
import { useEffect, useState } from "react";
import {
  Avatar,
  Box,
  Button,
  Divider,
  Flex,
  Heading,
  IconButton,
  Menu,
  MenuButton,
  MenuItem,
  MenuList,
  Text,
  useToast,
} from "@chakra-ui/react";
import { FaComment, FaHeart, FaRegHeart } from "react-icons/fa";
import { useLoggedIn } from "@codeup/hooks";
import TipTap from "./TipTap";
import { useRouter } from "next/router";
import { GiHamburgerMenu } from "react-icons/gi";
import REST from "@codeup/rest";

export default function PostCard(props: { id: string }) {
  const { loggedIn, userInfo } = useLoggedIn();
  const toast = useToast();
  const router = useRouter();
  const [showEditor, setShowEditor] = useState(false);

  const [post, setPost] = useState<{
    _id: string;
    user: string;
    username: string;
    content: string;
    createdAt: Date;
    likes: string[];
```

```
  comments: string[];
  parent: string | null;
}>({
  _id: "",
  user: "",
  username: "",
  content: "",
  createdAt: new Date(),
  likes: [],
  comments: [],
  parent: null,
});

const [authorData, setAuthorData] = useState<{
  _id: string;
  user: string;
  username: string;
  profilePicture: string;
  profileBanner: string;
  displayName: string;
  verified: boolean;
  specialStatus: string;
  bio: string;
  location: string;
  birthday: string;
  website: string;
  followers: string[];
}>({
  _id: "",
  user: "",
  username: "",
  profilePicture: "",
  profileBanner: "",
  displayName: "",
  verified: false,
  specialStatus: "",
  bio: "",
  location: "",
  birthday: "",
  website: "",
  followers: [],
});

useEffect(() => {
  if (post.username !== "") {
    REST.Community.getProfile(post.username).then((res) => {
      if (res.payload.success) {
        setAuthorData(res.payload.profile);
      } else {
        toast({
          title: "Fehler",
          description: res.payload.error,
          status: "error",
          duration: 9000,
          position: "top",
        });
      }
    });
  }
}, [post, toast]);
```

```
useEffect(() => {
  if (props.id) {
    REST.Community.getPost(props.id).then((res) => {
      if (res.payload.success) {
        setPost(res.payload.post);
      } else {
        toast({
          title: "Fehler",
          description: res.payload.error,
          status: "error",
          duration: 9000,
          position: "top",
        });
      }
    });
  }
}, [router, props.id, toast]);

return (
  <>
    <Flex direction={"column"} mt={"2rem"}>
      <Flex direction={"row"}>
        <Avatar
          src={"/api/community/profile/" + post.username + "/avatar"}
          size={"lg"}
          onClick={() => {
            location.href = "/community/profile/" + post.username;
          }}
          cursor={"pointer"}
        />
        <Flex direction={"column"} ml={"1rem"} flex={"100%"}>
          <Heading as={"h1"} fontSize={["3xl", "4xl"]}>
            {authorData.displayName}
          </Heading>
          <Text color={"gray.500"}>@{authorData.username}</Text>
          <Text
            color={"gray.500"}
            _hover={{
              textDecoration: "none",
            }}
          >
            {new Date(post.createdAt).getFullYear()}/
            {new Date(post.createdAt).getMonth() + 1}/
            {new Date(post.createdAt).getDate()} 
            {new Date(post.createdAt).getHours()}:
            {new Date(post.createdAt).getMinutes()}
          </Text>
        </Flex>
        <Box flex={"100%"} textAlign={"right"}>
          <Menu>
            <MenuButton as={IconButton} icon={<GiHamburgerMenu />} />
            <MenuList>
              {userInfo.username === post.username ? (
                <MenuItem
                  onClick={async () => {
                    const r = await REST.Community.deletePost({
                      postId: post._id,
                      token: localStorage.getItem("token") as string,
                    });
```

```
              if (r.payload.success) {
                toast({
                  title: "Erfolg",
                  description: "Der Beitrag wurde gelöscht",
                  status: "success",
                  duration: 6000,
                  position: "top",
                  onCloseComplete: () => {
                    if (!post.parent) {
                      router.push("/community");
                    }
                  },
                });
              } else {
                toast({
                  title: "Fehler",
                  description: r.payload.error,
                  status: "error",
                  duration: 6000,
                  position: "top",
                });
              }
            }}
          >
            Löschen
          </MenuItem>
        ) : null}
        <MenuItem
          onClick={() => {
            navigator.clipboard
              .writeText(
                "https://" +
                  window.location.host +
                  "/community/post/" +
                  post._id
              )
              .then(() => {
                toast({
                  title: "Erfolg",
                  description:
                    "Der Link wurde in die Zwischenablage kopiert.",
                  status: "success",
                  duration: 3000,
                  isClosable: true,
                  position: "top",
                });
              });
          }}
        >
          Teilen
        </MenuItem>
      </MenuList>
    </Menu>
  </Box>
 </Flex>
</Flex>
<Flex direction={"column"} id={"post-display"}>
 <div
  style={{ marginTop: ".5rem", marginBottom: ".5rem" }}
```

```
    dangerouslySetInnerHTML={{ __html: post.content }}
  />
  <Divider />
  <Flex direction={"row"} justifyContent={"space-between"}>
    <Flex
      direction={"row"}
      w={"100%"}
      justifyContent={"center"}
      mt={".7rem"}
      mb={".7rem"}
    >
      <Button
        aria-label={"Likes"}
        rightIcon={
          post.likes.includes(userInfo.id) ? <FaHeart /> : <FaRegHeart />
        }
        flex="100%"
        onClick={async () => {
          const r = await REST.Community.likePost({
            token: localStorage.getItem("token") as string,
            postId: post._id,
          });

          if (r.payload.success) {
            setPost(r.payload.post);
          } else {
            toast({
              title: "Fehler",
              description: r.payload.error,
              status: "error",
              duration: 9000,
              position: "top",
            });
          }
        }}
        variant={"ghost"}
        size={"lg"}
      >
        {post.likes.length}
      </Button>
      {!post.parent ? (
        <Button
          aria-label={"Kommentare"}
          rightIcon={<FaComment />}
          flex="100%"
          onClick={() => {
            setShowEditor(!showEditor);
          }}
          variant={"ghost"}
          size={"lg"}
        >
          {post.comments.length}
        </Button>
      ) : null}
    </Flex>
  </Flex>
</Flex>
<Divider />
{showEditor && !post.parent ? (
  <Box
```

```
          backgroundColor={"gray.900"}
          p={"1.2rem"}
          mt={".5rem"}
          rounded={"xl"}
        >
          <TipTap parentPost={post._id} setShowEditor={setShowEditor} />
        </Box>
      ) : null}
    </>
  );
}
```

## packages/components/community/post/PostMediaUploader.tsx

```
/**
 * src/components/community/post/PostMediaUploader.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 19.06.2022
 *
 */

import * as React from "react";
import {
  Button,
  FormControl,
  FormErrorMessage,
  FormLabel,
  Modal,
  ModalBody,
  ModalCloseButton,
  ModalContent,
  ModalFooter,
  ModalHeader,
  ModalOverlay,
  Spinner,
  useToast,
} from "@chakra-ui/react";

export default function PostMediaUploader(props: {
  isOpen: boolean;
  onOpen: () => void;
  onClose: () => void;
  editor: any;
}) {
  const [fileInvalid, setFileInvalid] = React.useState(false);
  const [loading, setLoading] = React.useState(false);
  const toast = useToast();

  return (
    <Modal onClose={props.onClose} isOpen={props.isOpen} isCentered size={"xl"}>
      <ModalOverlay />
      <ModalContent>
        <ModalHeader>Bild hochladen</ModalHeader>
        <ModalCloseButton disabled={loading} />
        <ModalBody>
          {loading ? (
            <Spinner />
          ) : (
```

```
    <FormControl isInvalid={fileInvalid}>
      <FormLabel>Bild</FormLabel>
      <input
        type="file"
        accept="image/svg+xml"
        onChange={(e) => {
          setFileInvalid(false);
          // @ts-ignore
          if (e.target.files.length > 0) {
            // @ts-ignore
            const file = e.target.files[0];

            if (file.type.startsWith("image/")) {
              if (file.size > 8 * 1024 * 1024) {
                setFileInvalid(true);
                return;
              } else {
                setLoading(true);
                const formData = new FormData();
                formData.append("uplFile", file);
                fetch(
                  location.protocol.includes("s")
                    ? "https://upload.craftions.net/api/upload"
                    : "http://direct.craftions.net:9097/api/upload",
                  {
                    method: "POST",
                    body: formData,
                  }
                ).then((res) => {
                  console.log(res);
                  res.text().then((data) => {
                    const url = "https://upload.craftions.net" + data;

                    props.editor
                      .chain()
                      .focus()
                      .setImage({ src: url })
                      .run();
                    props.onClose();
                  });
                });
              }
            } else {
              setFileInvalid(true);
            }
          }
        }}
      />
      <FormErrorMessage>
        Die hochgeladene Datei ist kein Bild oder ist größer als 8MB
      </FormErrorMessage>
    </FormControl>
  )}
 </ModalBody>
 <ModalFooter>
  <Button onClick={props.onClose} disabled={loading}>
    Abbrechen
  </Button>
 </ModalFooter>
</ModalContent>
```

```
    </Modal>
  );
}
```

## packages/components/community/post/StarterKit.ts

```
/**
 * src/components/community/post/StarterKit.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 19.06.2022
 *
 */

import { Extension } from "@tiptap/core";
import Blockquote, { BlockquoteOptions } from "@tiptap/extension-blockquote";
import Bold, { BoldOptions } from "@tiptap/extension-bold";
import BulletList, { BulletListOptions } from "@tiptap/extension-bullet-list";
import Code, { CodeOptions } from "@tiptap/extension-code";
import CodeBlock, { CodeBlockOptions } from "@tiptap/extension-code-block";
import Document from "@tiptap/extension-document";
import Dropcursor, { DropcursorOptions } from "@tiptap/extension-dropcursor";
import Gapcursor from "@tiptap/extension-gapcursor";
import HardBreak, { HardBreakOptions } from "@tiptap/extension-hard-break";
import Heading, { HeadingOptions } from "@tiptap/extension-heading";
import History, { HistoryOptions } from "@tiptap/extension-history";
import HorizontalRule, {
  HorizontalRuleOptions,
} from "@tiptap/extension-horizontal-rule";
import Italic, { ItalicOptions } from "@tiptap/extension-italic";
import ListItem, { ListItemOptions } from "@tiptap/extension-list-item";
import OrderedList, {
  OrderedListOptions,
} from "@tiptap/extension-ordered-list";
import Paragraph, { ParagraphOptions } from "@tiptap/extension-paragraph";
import Strike, { StrikeOptions } from "@tiptap/extension-strike";
import Text from "@tiptap/extension-text";

export interface StarterKitOptions {
  blockquote: Partial<BlockquoteOptions> | false;
  bold: Partial<BoldOptions> | false;
  bulletList: Partial<BulletListOptions> | false;
  code: Partial<CodeOptions> | false;
  codeBlock: Partial<CodeBlockOptions> | false;
  document: false;
  dropcursor: Partial<DropcursorOptions> | false;
  gapcursor: false;
  hardBreak: Partial<HardBreakOptions> | false;
  heading: Partial<HeadingOptions> | false;
  history: Partial<HistoryOptions> | false;
  horizontalRule: Partial<HorizontalRuleOptions> | false;
  italic: Partial<ItalicOptions> | false;
  listItem: Partial<ListItemOptions> | false;
  orderedList: Partial<OrderedListOptions> | false;
  paragraph: Partial<ParagraphOptions> | false;
  strike: Partial<StrikeOptions> | false;
  text: false;
}
```

```
export const StarterKit = Extension.create<StarterKitOptions>({
  name: "starterKit",

  addExtensions() {
    const extensions = [];

    if (this.options.blockquote !== false) {
      extensions.push(Blockquote.configure(this.options?.blockquote));
    }

    if (this.options.bold !== false) {
      extensions.push(Bold.configure(this.options?.bold));
    }

    if (this.options.bulletList !== false) {
      extensions.push(BulletList.configure(this.options?.bulletList));
    }

    if (this.options.code !== false) {
      extensions.push(Code.configure(this.options?.code));
    }

    if (this.options.codeBlock !== false) {
      extensions.push(CodeBlock.configure(this.options?.codeBlock));
    }

    if (this.options.document !== false) {
      extensions.push(Document.configure(this.options?.document));
    }

    if (this.options.dropcursor !== false) {
      extensions.push(Dropcursor.configure(this.options?.dropcursor));
    }

    if (this.options.gapcursor !== false) {
      extensions.push(Gapcursor.configure(this.options?.gapcursor));
    }

    if (this.options.hardBreak !== false) {
      extensions.push(HardBreak.configure(this.options?.hardBreak));
    }

    if (this.options.heading !== false) {
      extensions.push(Heading.configure(this.options?.heading));
    }

    if (this.options.history !== false) {
      extensions.push(History.configure(this.options?.history));
    }

    if (this.options.horizontalRule !== false) {
      extensions.push(HorizontalRule.configure(this.options?.horizontalRule));
    }

    if (this.options.italic !== false) {
      extensions.push(Italic.configure(this.options?.italic));
    }

    if (this.options.listItem !== false) {
      extensions.push(ListItem.configure(this.options?.listItem));
```

```
    }

    if (this.options.orderedList !== false) {
      extensions.push(OrderedList.configure(this.options?.orderedList));
    }

    if (this.options.paragraph !== false) {
      extensions.push(Paragraph.configure(this.options?.paragraph));
    }

    if (this.options.strike !== false) {
      extensions.push(Strike.configure(this.options?.strike));
    }

    if (this.options.text !== false) {
      extensions.push(Text.configure(this.options?.text));
    }

    return extensions;
  },
});
```

## packages/components/community/post/TipTap.tsx

```
/**
 * src/components/community/post/TipTap.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 19.06.2022
 *
 */

import * as React from "react";
import { EditorContent, useEditor } from "@tiptap/react";
import { StarterKit } from "./StarterKit";
import {
  Box,
  Button,
  Flex,
  Menu,
  MenuButton,
  MenuItem,
  MenuList,
  useDisclosure,
  useToast,
} from "@chakra-ui/react";
import { TextAlign } from "@tiptap/extension-text-align";
import { Highlight } from "@tiptap/extension-highlight";
import Code from "@tiptap/extension-code";
import CodeBlock from "@tiptap/extension-code-block";
import { GoChevronDown } from "react-icons/go";
import { Image } from "@tiptap/extension-image";
import PostMediaUploader from "./PostMediaUploader";
import { FontFamily } from "@tiptap/extension-font-family";
import { Link } from "@tiptap/extension-link";
import { TextStyle } from "@tiptap/extension-text-style";
import { Color } from "@tiptap/extension-color";
import { useRouter } from "next/router";
import { Dispatch, SetStateAction } from "react";
```

```
import REST from "@codeup/rest";

const MenuBar = ({ editor, onOpen }: { editor: any; onOpen: () => void }) => {
  if (!editor) {
    return null;
  }

  return (
    <Flex
      direction={"row"}
      mb={"2rem"}
      gap={"2rem"}
      display={["block", "block"]}
      justifyContent={"center"}
      alignItems={"center"}
      textAlign={"center"}
    >
      <Menu>
        <MenuButton
          marginInline=".7rem"
          marginBlock=".7rem"
          as={Button}
          rightIcon={<GoChevronDown />}
        >
          Größe
        </MenuButton>
        <MenuList>
          <MenuItem
            onClick={() =>
              editor.chain().focus().toggleHeading({ level: 1 }).run()
            }
            style={{ flex: "1 1 auto" }}
            className={
              editor.isActive("heading", { level: 1 }) ? "is-active" : ""
            }
          >
            Riesig
          </MenuItem>
          <MenuItem
            onClick={() =>
              editor.chain().focus().toggleHeading({ level: 2 }).run()
            }
            style={{ flex: "1 1 auto" }}
            className={
              editor.isActive("heading", { level: 2 }) ? "is-active" : ""
            }
          >
            Groß
          </MenuItem>
          <MenuItem
            onClick={() =>
              editor.chain().focus().toggleHeading({ level: 3 }).run()
            }
            style={{ flex: "1 1 auto" }}
            className={
              editor.isActive("heading", { level: 3 }) ? "is-active" : ""
            }
          >
            Mittel
          </MenuItem>
```

```
    <MenuItem
      onClick={() => editor.chain().focus().setParagraph().run()}
      style={{ flex: "1 1 auto" }}
      className={editor.isActive("paragraph") ? "is-active" : ""}
    >
      Normal
    </MenuItem>
  </MenuList>
</Menu>
<Menu>
  <MenuButton
    marginInline=".7rem"
    marginBlock=".7rem"
    as={Button}
    rightIcon={<GoChevronDown />}
  >
    Dekoration
  </MenuButton>
  <MenuList>
    <MenuItem
      onClick={() => editor.chain().focus().toggleBold().run()}
      style={{ flex: "1 1 auto" }}
      className={editor.isActive("bold") ? "is-active" : ""}
    >
      Fett
    </MenuItem>
    <MenuItem
      onClick={() => editor.chain().focus().toggleItalic().run()}
      style={{ flex: "1 1 auto" }}
      className={editor.isActive("italic") ? "is-active" : ""}
    >
      Kursiv
    </MenuItem>
    <MenuItem
      onClick={() => editor.chain().focus().toggleStrike().run()}
      style={{ flex: "1 1 auto" }}
      className={editor.isActive("strikeThrough") ? "is-active" : ""}
    >
      Durchgestrichen
    </MenuItem>
  </MenuList>
</Menu>
<Menu>
  <MenuButton
    marginInline=".7rem"
    marginBlock=".7rem"
    as={Button}
    rightIcon={<GoChevronDown />}
  >
    Markieren
  </MenuButton>
  <MenuList>
    <MenuItem
      onClick={() => editor.chain().focus().toggleHighlight().run()}
      style={{ flex: "1 1 auto" }}
      className={editor.isActive("highlight") ? "is-active" : ""}
    >
      Gelb
    </MenuItem>
    <MenuItem
```

```
      onClick={() =>
        editor.chain().focus().toggleHighlight({ color: "red" }).run()
      }
      style={{ flex: "1 1 auto" }}
      className={
        editor.isActive("highlight", { color: "red" }) ? "is-active" : ""
      }
    >
      Rot
    </MenuItem>
    <MenuItem
      onClick={() =>
        editor.chain().focus().toggleHighlight({ color: "green" }).run()
      }
      style={{ flex: "1 1 auto" }}
      className={
        editor.isActive("highlight", { color: "green" })
          ? "is-active"
          : ""
      }
    >
      Grün
    </MenuItem>
    <MenuItem
      onClick={() =>
        editor.chain().focus().toggleHighlight({ color: "blue" }).run()
      }
      style={{ flex: "1 1 auto" }}
      className={
        editor.isActive("highlight", { color: "blue" }) ? "is-active" : ""
      }
    >
      Blau
    </MenuItem>
  </MenuList>
</Menu>
<Menu>
  <MenuButton
    marginInline=".7rem"
    marginBlock=".7rem"
    as={Button}
    rightIcon={<GoChevronDown />}
  >
    Ausrichtung
  </MenuButton>
  <MenuList>
    <MenuItem
      onClick={() => editor.chain().focus().setTextAlign("left").run()}
      style={{ flex: "1 1 auto" }}
      className={
        editor.isActive({ textAlign: "left" }) ? "is-active" : ""
      }
    >
      Links
    </MenuItem>
    <MenuItem
      onClick={() => editor.chain().focus().setTextAlign("center").run()}
      style={{ flex: "1 1 auto" }}
      className={
        editor.isActive({ textAlign: "center" }) ? "is-active" : ""
```

```
      }
    >
      Mitte
    </MenuItem>
    <MenuItem
      onClick={() => editor.chain().focus().setTextAlign("right").run()}
      style={{ flex: "1 1 auto" }}
      className={
        editor.isActive({ textAlign: "right" }) ? "is-active" : ""
      }
    >
      Rechts
    </MenuItem>
  </MenuList>
</Menu>
<Menu>
  <MenuButton
    marginInline=".7rem"
    marginBlock=".7rem"
    as={Button}
    rightIcon={<GoChevronDown />}
  >
    Liste
  </MenuButton>
  <MenuList>
    <MenuItem
      onClick={() => editor.chain().focus().toggleBulletList().run()}
      style={{ flex: "1 1 auto" }}
      className={
        editor.isActive({ textAlign: "left" }) ? "is-active" : ""
      }
    >
      Ungeordnet
    </MenuItem>
    <MenuItem
      onClick={() => editor.chain().focus().toggleOrderedList().run()}
      style={{ flex: "1 1 auto" }}
      className={
        editor.isActive({ textAlign: "center" }) ? "is-active" : ""
      }
    >
      Geordnet
    </MenuItem>
  </MenuList>
</Menu>
<Menu>
  <MenuButton
    marginInline=".7rem"
    marginBlock=".7rem"
    as={Button}
    rightIcon={<GoChevronDown />}
  >
    Code
  </MenuButton>
  <MenuList>
    <MenuItem
      onClick={() => editor.chain().focus().toggleCode().run()}
      style={{ flex: "1 1 auto" }}
      className={
        editor.isActive({ textAlign: "left" }) ? "is-active" : ""
```

```
      }
    >
      Code
    </MenuItem>
    <MenuItem
      onClick={() => editor.chain().focus().toggleCodeBlock().run()}
      style={{ flex: "1 1 auto" }}
      className={
        editor.isActive({ textAlign: "center" }) ? "is-active" : ""
      }
    >
      Code Block
    </MenuItem>
  </MenuList>
</Menu>
<Menu>
  <MenuButton
    marginInline=".7rem"
    marginBlock=".7rem"
    as={Button}
    onClick={() => {
      onOpen();
    }}
  >
    Bild
  </MenuButton>
</Menu>
<Menu>
  <MenuButton
    marginInline=".7rem"
    marginBlock=".7rem"
    as={Button}
    rightIcon={<GoChevronDown />}
  >
    Schriftart
  </MenuButton>
  <MenuList>
    <MenuItem
      onClick={() => editor.chain().focus().unsetFontFamily().run()}
      style={{ flex: "1 1 auto" }}
    >
      Standart
    </MenuItem>
    <MenuItem
      onClick={() =>
        editor
          .chain()
          .focus()
          .setFontFamily("Comic Sans MS, Comic Sans")
          .run()
      }
      style={{ flex: "1 1 auto" }}
    >
      Comic Sans
    </MenuItem>
    <MenuItem
      onClick={() =>
        editor.chain().focus().setFontFamily("sans-serif").run()
      }
      style={{ flex: "1 1 auto" }}
```

```
          >
            Sans Serif
          </MenuItem>
          <MenuItem
            onClick={() =>
              editor.chain().focus().setFontFamily("monospace").run()
            }
            style={{ flex: "1 1 auto" }}
          >
            Monospace
          </MenuItem>
        </MenuList>
      </Menu>
      <Menu>
        <MenuButton
          marginInline=".7rem"
          marginBlock=".7rem"
          as={Button}
          onClick={() => {
            (
              document.getElementById("post-color-picker") as HTMLInputElement
            ).click();
          }}
        >
          Farbe
        </MenuButton>
        <input
          type="color"
          onInput={(event) =>
            // @ts-ignore
            editor.chain().focus().setColor(event.target.value).run()
          }
          value={editor.getAttributes("textStyle").color}
          id={"post-color-picker"}
          style={{ display: "none" }}
        />
      </Menu>
    </Flex>
  );
};

const TipTap = (props: {
  parentPost?: string;
  setShowEditor?: Dispatch<SetStateAction<boolean>>;
}) => {
  const editor = useEditor({
    extensions: [
      StarterKit,
      TextAlign.configure({
        types: ["heading", "paragraph"],
      }),
      Highlight.configure({
        multicolor: true,
      }),
      Code,
      CodeBlock,
      Image.configure({
        allowBase64: true,
      }),
      FontFamily,
```

```
    TextStyle,
    Link.configure({
      autolink: true,
      protocols: ["http", "https", "mailto"],
      linkOnPaste: true,
      HTMLAttributes: {
        class: "post-link",
      },
    }),
    Color,
  ],
  content: "<p>Hallo Welt!</p>",
});

const { isOpen, onOpen, onClose } = useDisclosure();
const router = useRouter();
const toast = useToast();

return (
  <>
    <PostMediaUploader
      isOpen={isOpen}
      onOpen={onOpen}
      onClose={onClose}
      editor={editor}
    />
    <MenuBar editor={editor} onOpen={onOpen} />
    <EditorContent editor={editor} />

    <Box w={"100%"} textAlign={"right"}>
      <Button
        mt={"2rem"}
        colorScheme={"whatsapp"}
        onClick={async () => {
          const html = editor?.getHTML();

          const res = await REST.Community.createPost({
            token: localStorage.getItem("token") as string,
            content: html as string,
            parent: props.parentPost || null,
          });

          if (res.payload.success) {
            if (props.parentPost) {
              toast({
                title: "Kommentar erstellt!",
                description:
                  "Dein Kommentar wurde erstellt mund ist bald sichtbar.",
                status: "success",
                duration: 6000,
                isClosable: false,
                position: "top",
              });
              if (props.setShowEditor) {
                props.setShowEditor(false);
              }
            } else {
              window.location.href = "/community/post/" + res.payload.post;
            }
          } else {
```

```
            toast({
              title: "Fehler",
              description: res.payload.error,
              status: "error",
              duration: 9000,
            });
          }
        }}
      >
        Posten
      </Button>
    </Box>
  </>
 );
};

export default TipTap;
```

## packages/components/community/profile/index.ts

```
/**
 * packages/components/community/profile/index.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 03.07.2022
 *
 */

import * as React from "react";
export { default as ProfileAvatar } from "./ProfileAvatar";
export { default as ProfileAvatarModifier } from "./ProfileAvatarModifier";
export { default as ProfileBadges } from "./ProfileBadges";
export { default as ProfileBanner } from "./ProfileBanner";
export { default as ProfileBannerModifier } from "./ProfileBannerModifier";
export { default as ProfileDataDisplay } from "./ProfileDataDisplay";
export { default as ProfileMenuButton } from "./ProfileMenuButton";
export { default as ProfileNotFound } from "./ProfileNotFound";
```

## packages/components/community/profile/ProfileAvatar.tsx

```
/**
 * src/components/community/profile/ProfileAvatar.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 16.06.2022
 *
 */

import * as React from "react";
import { Avatar, Flex } from "@chakra-ui/react";

export default function ProfileAvatar(props: { url: string }) {
  return (
    <>
      <Flex justifyContent={["center", "flex-start"]}>
        <Avatar
          src={props.url}
          w={128}
```

```
          h={128}
          mt={"-3rem"}
          marginInlineStart={["0", "1rem"]}
          borderColor={"#FFF"}
          borderWidth={"4px"}
        />
      </Flex>
    </>
  );
}
```

## packages/components/community/profile/ProfileAvatarModifier.tsx

```
/**
 * src/components/community/profile/ProfileAvatarModifier.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 18.06.2022
 *
 */

import * as React from "react";
import {
  Button,
  FormControl,
  FormErrorMessage,
  FormLabel,
  Modal,
  ModalBody,
  ModalCloseButton,
  ModalContent,
  ModalFooter,
  ModalHeader,
  ModalOverlay,
  Spinner,
  useToast,
} from "@chakra-ui/react";
import REST from "@codeup/rest";

export default function ProfileAvatarModifier(props: {
  isOpen: boolean;
  onOpen: () => void;
  onClose: () => void;
}) {
  const [fileInvalid, setFileInvalid] = React.useState(false);
  const [loading, setLoading] = React.useState(false);

  const toast = useToast();

  return (
    <>
      <Modal
        onClose={props.onClose}
        isOpen={props.isOpen}
        isCentered
        size={"xl"}
      >
        <ModalOverlay />
        <ModalContent>
```

```
<ModalHeader>Avatar hochladen</ModalHeader>
<ModalCloseButton disabled={loading} />
<ModalBody>
  {loading ? (
    <Spinner />
  ) : (
    <FormControl isInvalid={fileInvalid}>
      <FormLabel>Avatar</FormLabel>
      <input
        type="file"
        accept="image/svg+xml"
        onChange={(e) => {
          setFileInvalid(false);
          // @ts-ignore
          if (e.target.files.length > 0) {
            // @ts-ignore
            const file = e.target.files[0];

            if (file.type.startsWith("image/")) {
              if (file.size > 8 * 1024 * 1024) {
                setFileInvalid(true);
                return;
              } else {
                setLoading(true);
                const formData = new FormData();
                formData.append("uplFile", file);
                fetch(
                  location.protocol.includes("s")
                    ? "https://upload.craftions.net/api/upload"
                    : "http://direct.craftions.net:9097/api/upload",
                  {
                    method: "POST",
                    body: formData,
                  }
                ).then((res) => {
                  console.log(res);
                  res.text().then((data) => {
                    const url = "https://upload.craftions.net" + data;

                    REST.Community.updateProfile({
                      token: localStorage.getItem("token") as string,
                      update: {
                        profilePicture: url,
                      },
                    }).then((res) => {
                      if (res.status === 200) {
                        toast({
                          title: "Profil aktualisiert",
                          description:
                            "Dein Profil wurde erfolgreich aktualisiert.",
                          status: "success",
                          position: "top",
                          duration: 3000,
                          onCloseComplete: () => {
                            window.location.reload();
                          },
                        });
                      } else {
                        toast({
                          title: "Fehler",
```

```
                                description:
                                  "Dein Profil konnte nicht aktualisiert werden.",
                                status: "error",
                              });
                            }
                          });
                        });
                      });
                    }
                  } else {
                    setFileInvalid(true);
                  }
                }
              }}
            />
            <FormErrorMessage>
              Die hochgeladene Datei ist kein Bild oder ist größer als 8MB
            </FormErrorMessage>
          </FormControl>
        )}
      </ModalBody>
      <ModalFooter>
        <Button onClick={props.onClose} disabled={loading}>
          Abbrechen
        </Button>
      </ModalFooter>
    </ModalContent>
  </Modal>
 </>
 );
}
```

## packages/components/community/profile/ProfileBadges.tsx

```
/**
 * src/components/community/profile/ProfileBadges.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 16.06.2022
 *
 */

import * as React from "react";
import { Badge, Flex, Tooltip } from "@chakra-ui/react";

export default function ProfileBadges(props: {
  verified: boolean;
  specialStatus: string;
}) {
  return (
    <>
      <Flex
        direction={"row"}
        mt={".4rem"}
        marginInline={".4rem"}
        gap={"0"}
        justifyContent={"center"}
        alignItems={"center"}
      >
```

```
        w
        {props.verified ? (
          <Tooltip label={"Official Account"} hasArrow>
            <Badge
              ml={"1"}
              p={".2rem"}
              colorScheme="green"
              display={"block ruby"}
            >
              Offiziell
            </Badge>
          </Tooltip>
        ) : null}
        {props.specialStatus.length > 0 ? (
          <Tooltip label={props.specialStatus} hasArrow>
            <Badge
              ml={"1"}
              p={".2rem"}
              colorScheme="yellow"
              display={"block ruby"}
            >
              Speziell
            </Badge>
          </Tooltip>
        ) : null}
      </Flex>
    </>
  );
}
```

## packages/components/community/profile/ProfileBanner.tsx

```
/**
 * src/components/community/profile/ProfileBanner.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 16.06.2022
 *
 */

import * as React from "react";
import { Image } from "@chakra-ui/react";

export default function ProfileBanner(props: { url: string }) {
  return (
    <>
      <Image
        src={props.url}
        w={"100%"}
        h={[312, 624]}
        maxHeight={[250, 450]}
        maxW={"100vw"}
        style={{ objectFit: "none" }}
        borderRadius={"12px"}
        borderTopRadius={0}
        alt={"Banner"}
        shadow={"lg"}
      />
    </>
```

```
  );
}
```

## packages/components/community/profile/ProfileBannerModifier.tsx

```
/**
 * src/components/community/profile/ProfileAvatarModifier.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 18.06.2022
 *
 */

import * as React from "react";
import {
  Button,
  FormControl,
  FormErrorMessage,
  FormLabel,
  Modal,
  ModalBody,
  ModalCloseButton,
  ModalContent,
  ModalFooter,
  ModalHeader,
  ModalOverlay,
  Spinner,
  useToast,
} from "@chakra-ui/react";
import REST from "@codeup/rest";

export default function ProfileBannerModifier(props: {
  isOpen: boolean;
  onOpen: () => void;
  onClose: () => void;
}) {
  const [fileInvalid, setFileInvalid] = React.useState(false);
  const [loading, setLoading] = React.useState(false);

  const toast = useToast();

  return (
    <>
      <Modal
        onClose={props.onClose}
        isOpen={props.isOpen}
        isCentered
        size={"xl"}
      >
        <ModalOverlay />
        <ModalContent>
          <ModalHeader>Banner hochladen</ModalHeader>
          <ModalCloseButton disabled={loading} />
          <ModalBody>
            {loading ? (
              <Spinner />
            ) : (
              <FormControl isInvalid={fileInvalid}>
                <FormLabel>Profil Banner</FormLabel>
```

```
<input
  type="file"
  accept="image/svg+xml"
  onChange={(e) => {
    setFileInvalid(false);
    // @ts-ignore
    if (e.target.files.length > 0) {
      // @ts-ignore
      const file = e.target.files[0];

      if (file.type.startsWith("image/")) {
        if (file.size > 8 * 1024 * 1024) {
          setFileInvalid(true);
          return;
        } else {
          setLoading(true);
          const formData = new FormData();
          formData.append("uplFile", file);
          fetch(
            location.protocol.includes("s")
              ? "https://upload.craftions.net/api/upload"
              : "http://direct.craftions.net:9097/api/upload",
            {
              method: "POST",
              body: formData,
            }
          ).then((res) => {
            console.log(res);
            res.text().then((data) => {
              const url = "https://upload.craftions.net" + data;

              REST.Community.updateProfile({
                token: localStorage.getItem("token") as string,
                update: {
                  profileBanner: url,
                },
              }).then((res) => {
                if (res.status === 200) {
                  toast({
                    title: "Profil aktualisiert",
                    description:
                      "Dein Profil wurde erfolgreich aktualisiert.",
                    status: "success",
                    position: "top",
                    duration: 3000,
                    onCloseComplete: () => {
                      window.location.reload();
                    },
                  });
                } else {
                  toast({
                    title: "Fehler",
                    description:
                      "Dein Profil konnte nicht aktualisiert werden.",
                    status: "error",
                  });
                }
              });
            });
          });
```

```
                  }
                } else {
                  setFileInvalid(true);
                }
              }
            }}
          />
          <FormErrorMessage>
            Die hochgeladene Datei ist kein Bild oder ist größer als 8MB
          </FormErrorMessage>
        </FormControl>
      )}
    </ModalBody>
    <ModalFooter>
      <Button onClick={props.onClose} disabled={loading}>
        Abbrechen
      </Button>
    </ModalFooter>
  </ModalContent>
</Modal>
</>
);
}
```

## packages/components/community/profile/ProfileDataDisplay.tsx

```
/**
 * src/components/community/profile/ProfileDataDisplay.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 18.06.2022
 *
 */

import * as React from "react";
import { useEffect, useState } from "react";
import {
  Avatar,
  Breadcrumb,
  BreadcrumbItem,
  BreadcrumbLink,
  Divider,
  Flex,
  Heading,
  Modal,
  ModalBody,
  ModalCloseButton,
  ModalContent,
  ModalHeader,
  ModalOverlay,
  Text,
  useDisclosure,
} from "@chakra-ui/react";
import REST from "@codeup/rest";

export default function ProfileDataDisplay(props: {
  profile: {
    username: string;
    displayName: string;
```

```
  };
}) {
  const [followersCount, setFollowersCount] = useState(0);
  const {
    isOpen: isFollowerOpen,
    onOpen: onFollowerOpen,
    onClose: onFollowerClose,
  } = useDisclosure();

  useEffect(() => {
    REST.Community.getProfileFollowers({
      name: props.profile.username,
      count: true,
    }).then((res) => {
      if (res.status === 200) {
        setFollowersCount(res.payload.count);
      }
    });
  }, [props.profile]);

  const [followers, setFollowers] = useState([]);

  useEffect(() => {
    if (props.profile.username.length > 0) {
      REST.Community.getProfileFollowers({
        name: props.profile.username,
      }).then((res) => {
        let f: any[] = [];

        res.payload.followers.forEach(async (follower: string) => {
          const r = await REST.Community.getProfile(follower);

          f.push({
            username: r.payload.profile.username,
            displayName: r.payload.profile.displayName,
            avatar: r.payload.profile.profilePicture,
            bio: r.payload.profile.bio,
          });
        });
        setFollowers(f as never[]);
      });
    }
  }, [props.profile]);

  return (
    <>
      <Breadcrumb>
        <BreadcrumbItem>
          <BreadcrumbLink onClick={onFollowerOpen}>
            {followersCount} Follower
          </BreadcrumbLink>
        </BreadcrumbItem>
      </Breadcrumb>
      <Modal
        isOpen={isFollowerOpen}
        onClose={onFollowerClose}
        isCentered
        size={["full", "lg"]}
      >
        <ModalOverlay />
```

```
        <ModalContent>
          <ModalHeader>
            {props.profile.displayName}&amp;apos;s&amp;nbsp;Followers
          </ModalHeader>
          <ModalCloseButton />
          <ModalBody>
            <Flex direction={"column"} justifyContent={"center"}>
              {followers.map((follower: any) => {
                return (
                  <>
                    <Flex
                      direction={"row"}
                      mb={"1.2rem"}
                      cursor={"pointer"}
                      onClick={() => {
                        window.location.href =
                          "/community/profile/" + follower.username;
                      }}
                    >
                      <Avatar
                        src={follower.avatar}
                        size={"lg"}
                        borderColor={"#FFF"}
                        borderWidth={"2px"}
                      />
                      <Flex direction={"column"} ml={"1.2rem"}>
                        <Heading as={"h1"} fontSize={["2xl", "3xl"]}>
                          {follower.displayName}
                        </Heading>
                        <Text color={"gray.500"}>@{follower.username}</Text>
                      </Flex>
                    </Flex>
                    <Divider />
                  </>
                );
              })}
            </Flex>
          </ModalBody>
        </ModalContent>
      </Modal>
    </>
  );
}
```

## packages/components/community/profile/ProfileMenuButton.tsx

```
/**
 * src/components/community/profile/ProfileMenuButtonj.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 18.06.2022
 *
 */

import * as React from "react";
import { useEffect, useState } from "react";
import {
  Box,
  Button,
```

```
  IconButton,
  Menu,
  MenuButton,
  MenuItem,
  MenuList,
} from "@chakra-ui/react";
import { GiHamburgerMenu } from "react-icons/gi";
import { useLoggedIn } from "@codeup/hooks";
import { FaMinus, FaPlus } from "react-icons/fa";
import REST from "@codeup/rest";

export default function ProfileMenuButton(props: {
  profile: {
    website: string;
    username: string;
  };
  editProfile: {
    onEditOpen: () => void;
    onEditClose: () => void;
    isEditOpen: boolean;
  };
}) {
  const [isOwner, setIsOwner] = useState(false);

  const { loggedIn, userInfo } = useLoggedIn();

  useEffect(() => {
    if (userInfo.username === props.profile.username) {
      setIsOwner(true);
    }
  }, [userInfo, props.profile.username]);

  const [isFollowing, setFollowing] = useState(false);

  useEffect(() => {
    if (userInfo.username.length > 0 && props.profile.username.length > 0) {
      REST.Community.isFollowing(
        props.profile.username,
        userInfo.username
      ).then((res) => {
        if (res.status === 200) {
          setFollowing(res.payload.isFollowing);
        } else {
          setFollowing(false);
        }
      });
    }
  }, [userInfo, props.profile.username]);

  return (
    <Box mt={"1.2rem"} textAlign={"center"}>
      <Button
        marginRight={"2rem"}
        colorScheme={isFollowing ? "red" : "blue"}
        leftIcon={isFollowing ? <FaMinus /> : <FaPlus />}
        onClick={async () => {
          const res = await REST.Community.toggleFollow({
            token: localStorage.getItem("token") as string,
            profile: props.profile.username,
          });
```

```
          if (res.status === 200) {
            setFollowing(res.payload.following);
          }
        }}
      >
        {isFollowing ? "Unfollow" : "Follow"}
      </Button>
      <Menu>
        <MenuButton
          as={IconButton}
          icon={<GiHamburgerMenu />}
          colorScheme={"gray"}
        >
          Aktionen
        </MenuButton>
        <MenuList>
          {props.profile.website.length > 0 ? (
            <MenuItem
              onClick={() => {
                window.open(props.profile.website, "_blank");
              }}
            >
              Website
            </MenuItem>
          ) : null}
          {isOwner ? (
            <MenuItem onClick={props.editProfile.onEditOpen}>Edit</MenuItem>
          ) : null}
          <MenuItem
            onClick={() => {
              window.location.href =
                "/community/profile/" + props.profile.username + "/report";
            }}
          >
            Melden
          </MenuItem>
        </MenuList>
      </Menu>
    </Box>
  );
}
```

## packages/components/community/profile/ProfileNotFound.tsx

```
/**
 * src/components/community/profile/ProfileNotFound.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 16.06.2022
 *
 */

import * as React from "react";
import { Flex, Heading, Text } from "@chakra-ui/react";
import { BsQuestionSquareFill } from "react-icons/bs";

export default function ProfileNotFound() {
  return (
```

```
    <>
      <Flex
        alignItems={"center"}
        justifyContent={"center"}
        minW={"100%"}
        minH={"99vh"}
      >
        <Heading
          color={"green.300"}
          fontSize={"10rem"}
          textAlign={"center"}
          display={"flex"}
          justifyContent={"center"}
          alignItems={"center"}
          flexDirection={"column"}
          as={"div"}
        >
          <BsQuestionSquareFill />
          <Heading color={"white"} mt={"1.2rem"}>
            Nicht gefunden
          </Heading>
          <Text
            color={"white"}
            fontSize={"1.2rem"}
            textDecoration={"none"}
            fontWeight={"normal"}
            mt={"1.2rem"}
          >
            Das gesuchte Profil konnte nicht gefunden werden!
          </Text>
        </Heading>
      </Flex>
    </>
  );
}
```

## packages/components/course/CourseNavigationBar.tsx

```
/**
 * src/components/community/CommunityNavigationBar.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 21.06.2022
 *
 */

import * as React from "react";
import {
  Box,
  Button,
  Divider,
  Drawer,
  DrawerBody,
  DrawerCloseButton,
  DrawerContent,
  DrawerHeader,
  DrawerOverlay,
  Flex,
  Heading,
```

```
  IconButton,
  Link as ChakraLink,
  useDisclosure,
  useMediaQuery,
} from "@chakra-ui/react";
import { GiHamburgerMenu } from "react-icons/gi";
import { useLoggedIn } from "@codeup/hooks";
import Link from "next/link";

export default function CourseNavigationBar() {
  const [isMobile] = useMediaQuery("(max-width: 768px)");
  const { isOpen, onOpen, onClose } = useDisclosure();

  const { loggedIn, userInfo } = useLoggedIn();
  const Menu = () => {
    return (
      <>
        <Link href={"/courses"}>
          <Button variant={"ghost"} size={"lg"} onClick={onClose}>
            Alle Kurse
          </Button>
        </Link>
        <Link href={"/course/my-courses"}>
          <Button variant={"ghost"} size={"lg"} onClick={onClose}>
            Meine Kurse
          </Button>
        </Link>
      </>
    );
  };

  return (
    <>
      <Box w={"100%"} bg={"gray.900"} padding={"1.2rem"}>
        <Flex direction={"row"} alignItems={"center"} justifyContent={"center"}>
          <Heading flex={"5%"} textAlign={["center", "left"]}>
            <ChakraLink href={"/community"}>
              <Link href={"/community"}>Kurse</Link>
            </ChakraLink>
          </Heading>
          <Flex justifyContent={"right"} flex={"100%"} alignItems={"right"}>
            {!isMobile ? (
              <Menu />
            ) : (
              <>
                <IconButton
                  onClick={onOpen}
                  aria-label={"Menu"}
                  icon={<GiHamburgerMenu />}
                  flex={"flex-end"}
                />
                <Drawer
                  isOpen={isOpen}
                  placement="right"
                  onClose={onClose}
                  size={"full"}
                >
                  <DrawerOverlay />
                  <DrawerContent bg={"black"}>
                    <DrawerCloseButton />
```

```
                <DrawerHeader>CodeUp | Kurse</DrawerHeader>

                <DrawerBody>
                  <Flex w={"100%"} direction={"column"}>
                    <Menu />
                  </Flex>
                </DrawerBody>
              </DrawerContent>
            </Drawer>
          </>
        )}
      </Flex>
    </Flex>
  </Box>
  <Divider />
 </>
 );
}
```

## packages/components/course/CourseSections.tsx

```
/**
 * packages/components/course/CourseSections.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 18.09.2022
 *
 */

import * as React from "react";
import { Button, Flex, useDisclosure } from "@chakra-ui/react";
import { FaFile } from "react-icons/fa";
import { useRouter } from "next/router";
import CodeUpModal from "../codeup/CodeUpModal";
import { useEffect } from "react";

export default function CourseSections(props: {
  course: string;
  sections: {
    title: string;
    url: string;
  }[];
}) {
  const router = useRouter();

  const { isOpen, onOpen, onClose } = useDisclosure();

  const [buttonVisible, setButtonVisible] = React.useState(true);

  useEffect(() => {
    window.addEventListener(
      "codeup::courses:open_sections?" +
        props.course.toLowerCase().replace(" ", "_"),
      onOpen
    );

    return () => {
      window.removeEventListener(
        "codeup::courses:open_sections?" +
```

```
        props.course.toLowerCase().replace(" ", "_"),
      onOpen
    );
  };
});

useEffect(() => {
  const handler = () => {
    setButtonVisible(!buttonVisible);
  };

  window.addEventListener(
    "codeup::courses:toggle_s_button?" +
      props.course.toLowerCase().replace(" ", "_"),
    handler
  );

  return () => {
    window.removeEventListener(
      "codeup::courses:toggle_s_button?" +
        props.course.toLowerCase().replace(" ", "_"),
      handler
    );
  };
});

return (
  <>
    <Flex
      bottom={["1.2rem", "initial"]}
      right={["initial", "1.2rem"]}
      top={["initial", "10rem"]}
      position={"fixed"}
      width={"100%"}
      alignItems={"center"}
      justifyContent={["center", "flex-end"]}
      zIndex={5}
    >
      <Button
        aria-label={"section-view"}
        leftIcon={<FaFile />}
        colorScheme={"blue"}
        onClick={onOpen}
        size={"lg"}
        display={buttonVisible ? "initial" : "none"}
        id={"open-sections-" + props.course.toLowerCase().replace(" ", "_")}
      >
        Lektionen
      </Button>
    </Flex>
    <CodeUpModal
      onOpen={onOpen}
      onClose={onClose}
      isOpen={isOpen}
      title={props.course}
      noCloseButton={false}
    >
      <Flex direction={"column"} gap={"1.2rem"}>
        {props.sections.map((section, i) => {
          return (
```

```
            <Button
              key={i}
              rounded={"none"}
              justifyContent={"left"}
              onClick={() => {
                router.push(section.url);
                onClose();
              }}
            >
              {i + 1}. {section.title}
            </Button>
          );
        })}
      </Flex>
    </CodeUpModal>
  </>
 );
}
```

## packages/components/course/CourseTitle.tsx

```
/**
 * src/components/course/CourseTitle.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 30.06.2022
 *
 */

import * as React from "react";
import { Heading, Text } from "@chakra-ui/react";

export default function CourseTitle(props: {
  course: string;
  section: number;
  sectionTitle: string;
}) {
  return (
    <>
      <Text textAlign={"center"} color={"gray.400"}>
        {props.course}
      </Text>
      <Heading textAlign={"center"}>
        Lektion {props.section}: 
        {props.sectionTitle}
      </Heading>
    </>
  );
}
```

## packages/components/course/HTMLCourseSections.tsx

```
/**
 * packages/components/course/HTMLCourseSections.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 18.09.2022
 *
```

```
 */

import * as React from "react";
import CourseSections from "./CourseSections";

export default function HTMLCourseSections() {
  return (
    <CourseSections
      course={"HTML Basics"}
      sections={[
        {
          title: "Grundlagen - Einführung in den Kurs",
          url: "/course/html-basics/basics",
        },
        {
          title: "Texte - Arten von Texten und Texte formatieren",
          url: "/course/html-basics/text",
        },
        {
          title: "Links - Links und interne Links",
          url: "/course/html-basics/links",
        },
        {
          title: "Bilder - Bilder einfügen, beschreiben und skalieren.",
          url: "/course/html-basics/image",
        },
        {
          title: "Andere Medien - Videos, Audios und Webseiten einbinden.",
          url: "/course/html-basics/other-media",
        },
        {
          title: "Listen - Arten von Listen und Einträge erstellen.",
          url: "/course/html-basics/lists",
        },
        {
          title: "Formulare - Komponenten und Formulare erstellen.",
          url: "/course/html-basics/forms",
        },
        {
          title: "Tabellen - Tabellen, Spalten, Zeilen, etc.",
          url: "/course/html-basics/tables",
        },
      ]}
    />
  );
}
```

## packages/components/course/index.ts

```
/**
 * packages/components/course/index.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 03.07.2022
 *
 */

import { default as CourseNavigationBar } from "./CourseNavigationBar";
import { default as CourseTitle } from "./CourseTitle";
```

```
import { default as ModuleCard } from "./ModuleCard";
import { default as ModuleCardRow } from "./ModuleCardRow";
import { default as Quiz } from "./Quiz";

export { CourseTitle, ModuleCard, CourseNavigationBar, ModuleCardRow, Quiz };
```

## packages/components/course/ModuleCard.tsx

```
/**
 * src/components/course/ModuleCard.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 01.07.2022
 *
 */

import * as React from "react";
import { Box, Heading } from "@chakra-ui/react";
import { useHover } from "@codeup/hooks";

export default function ModuleCard(props: {
  title: string;
  color: string;
  locked: boolean;
  url: string;
  id: string;
}) {
  const [hoverRef, isHovered] = useHover();

  return (
    <>
      <a
        href={!props.locked ? props.url : undefined}
        style={{ flex: "100%" }}
        target={"_blank"}
        rel={"noreferrer"}
      >
        <Box
          backgroundColor={props.locked ? "black" : props.color}
          borderWidth={"4px"}
          padding={"2rem"}
          rounded={"xl"}
          shadow={isHovered && !props.locked ? "2xl" : "none"}
          cursor={!props.locked ? "pointer" : "not-allowed"}
          id={props.id}
          // @ts-ignore
          ref={hoverRef}
          transition={"120ms ease"}
        >
          <Heading textAlign={"center"} size={"md"}>
            {props.locked ? (
              <>
                &#128274;
                <br />
              </>
            ) : null}
            {props.title}
          </Heading>
        </Box>
```

```
      </a>
    </>
  );
}
```

## packages/components/course/ModuleCardRow.tsx

```
/**
 * src/components/course/ModuleCardRow.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 02.07.2022
 *
 */

import * as React from "react";
import { Flex } from "@chakra-ui/react";
import { ReactElement } from "react";

export default function ModuleCardRow(props: { children: any }) {
  return (
    <Flex direction={"row"} gap={"3rem"} key={"0"}>
      {props.children}
    </Flex>
  );
}
```

## packages/components/course/Quiz.tsx

```
/**
 * src/components/course/Quiz.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 17.09.2022
 *
 */

import * as React from "react";

import {
  Accordion,
  AccordionButton,
  AccordionIcon,
  AccordionItem,
  AccordionPanel,
  Badge,
  Box,
  Button,
  ButtonGroup,
  CircularProgress,
  CircularProgressLabel,
  Divider,
  Flex,
  Heading,
  IconButton,
  Modal,
  ModalBody,
  ModalContent,
```

```
  ModalFooter,
  ModalHeader,
  ModalOverlay,
  Progress,
  Text,
  useDisclosure,
} from "@chakra-ui/react";
import { motion } from "framer-motion";
import { FaTwitter } from "react-icons/fa";
import CodeUpModal from "../codeup/CodeUpModal";
import { CHeading } from "../global/CHeading";

export default function Quiz(props: {
  title: string;
  questions: {
    question: string | React.ReactNode;
    difficulty?: "easy" | "medium" | "hard";
    answers: {
      answer: string | React.ReactNode;
      correct: boolean;
    }[];
  }[];
  successCB?: () => void;
}) {
  const { isOpen, onOpen, onClose } = useDisclosure();

  const [currentQuestion, setCurrentQuestion] = React.useState(0);
  const [correctAnswers, setCorrectAnswers] = React.useState(0);
  const [finished, setFinished] = React.useState(false);
  const [correctAnswersList, setCorrectAnswersList] = React.useState<number[]>(
    []
  );
  const [finishedValue, setFinishedValue] = React.useState(0);

  React.useEffect(() => {
    if (
      finished &&
      finishedValue < 100 &&
      finishedValue <
        Math.round((correctAnswers * 100) / props.questions.length)
    ) {
      setFinishedValue(finishedValue + 1);
    }
  }, [finished, finishedValue]);

  return (
    <>
      <CodeUpModal
        onOpen={onOpen}
        onClose={onClose}
        isOpen={isOpen}
        blackBackground={true}
        title={
          props.title +
          " (" +
          (finished
            ? "Ergebnis"
            : "Frage " +
              (currentQuestion + 1) +
              " von " +
```

```
        props.questions.length) +
      ")"
    }
    noCloseButton={true}
  >
    {finished ? (
      <>
        <Flex
          alignItems="center"
          justifyContent="center"
          direction="column"
        >
          <CircularProgress
            value={finishedValue}
            color={
              finishedValue >= 50 && finishedValue < 75
                ? "orange.400"
                : finishedValue >= 75
                ? "green.400"
                : "red.400"
            }
            thickness={"10px"}
            mt={"1.2rem"}
            size="120px"
          >
            <CircularProgressLabel
              color={
                finishedValue >= 50 && finishedValue < 75
                  ? "orange.400"
                  : finishedValue >= 75
                  ? "green.400"
                  : "red.400"
              }
            >
              {finishedValue}%
            </CircularProgressLabel>
          </CircularProgress>

          <Text color={"white"} mt="2rem" mb={"3rem"}>
            Du hast {correctAnswers} von {props.questions.length} Fragen
            richtig beantwortet.
          </Text>

          <CHeading size={"lg"}>Richtige Antworten</CHeading>
          <Accordion mt="1.2rem" allowToggle>
            {props.questions.map((q, i) => {
              return (
                <AccordionItem key={i}>
                  <AccordionButton>
                    <Box
                      flex="1"
                      textAlign="left"
                      color={
                        correctAnswersList.includes(i)
                          ? "green.400"
                          : "red.400"
                      }
                    >
                      &gt; {q.question}
                    </Box>
```

```
            <AccordionIcon />
          </AccordionButton>
          <AccordionPanel pb={4}>
            {q.answers.map((a, i) => {
              return (
                <Text
                  color={a.correct ? "green.400" : "gray.400"}
                  key={i}
                  textDecoration={
                    !a.correct ? "line-through" : "none"
                  }
                >
                  {a.answer}
                  {correctAnswersList.includes(i) ? (
                    <Badge
                      ml="1rem"
                      colorScheme={a.correct ? "green" : "red"}
                    >
                      Ausgewählt
                    </Badge>
                  ) : null}
                </Text>
              );
            })}
          </AccordionPanel>
        </AccordionItem>
      );
    })}
  </Accordion>

  <ButtonGroup mt="3rem" isAttached>
    <Button
      variant={"brand"}
      onClick={() => {
        let correctAnswers = correctAnswersList.length;
        let totalQuestions = props.questions.length;

        setCurrentQuestion(0);
        setCorrectAnswers(0);
        setFinishedValue(0);
        setCorrectAnswersList([]);
        setFinished(false);
        onClose();

        if (props.successCB && correctAnswers === totalQuestions) {
          props.successCB();
        }
      }}
    >
      Schließen
    </Button>
  </ButtonGroup>
</Flex>

<ButtonGroup
  isAttached
  position={"absolute"}
  top={"2rem"}
  right={"2rem"}
>
```

```
            <IconButton
              icon={<FaTwitter />}
              aria-label="Share on Twitter"
              variant={"brand"}
              onClick={async () => {
                window.open(
                  `https://twitter.com/intent/tweet?text=Ich%20habe%20${correctAnswers}%20von%20${
                    props.questions.length
                  }%20Fragen%20im%20%23CodeUp%20Quiz%20%22${
                    props.title

}%22%20richtig%20beantwortet.%20Wie%20viel%20schaffst%20du%3F%20${encodeURICompone
nt(
                    window.location.href
                  )}`,
                  "_blank"
                );
              }}
            />
          </ButtonGroup>
        </>
      ) : null}
      {!finished ? (
        <>
          <motion.div
            animate={{
              opacity: 1,
              x: 0,
            }}
            initial={{
              opacity: 0,
              x: -100,
            }}
            // @ts-ignore
            transition={{
              duration: 0.3,
            }}
            key={currentQuestion}
            exit={{
              x: -40,
            }}
          >
            <Progress
              size="sm"
              value={(currentQuestion / props.questions.length) * 100}
              colorScheme={"brand"}
            />

            <Text fontSize={"1.5rem"} mt={"1rem"}>
              {props.questions[currentQuestion].question}
              <Badge
                ml="1rem"
                variant="outline"
                display={"none"}
                colorScheme={
                  props.questions[currentQuestion].difficulty === "easy"
                    ? "green"
                    : props.questions[currentQuestion].difficulty === "medium"
                    ? "orange"
                    : "red"
```

```
          }
        >
          {props.questions[currentQuestion].difficulty === "easy"
            ? "Einfach"
            : props.questions[currentQuestion].difficulty === "medium"
            ? "Mittel"
            : "Schwer"}
        </Badge>
      </Text>
      <Divider mt={"1rem"} />
      <Flex direction={"column"} gap={"1rem"} mt={"1rem"}>
        {props.questions[currentQuestion].answers.map((a, i) => (
          <Button
            key={i}
            onClick={() => {
              if (a.correct) {
                setCorrectAnswers(correctAnswers + 1);
                setCorrectAnswersList([
                  ...correctAnswersList,
                  currentQuestion,
                ]);
              }
              if (currentQuestion + 1 < props.questions.length) {
                setCurrentQuestion(currentQuestion + 1);
              } else {
                setFinished(true);
              }
            }}
          >
            {a.answer}
          </Button>
        ))}
      </Flex>
    </motion.div>
  </>
 ) : null}
</CodeUpModal>

<Box mt="1rem">
  <Heading size={"lg"}>Quiz</Heading>
  <Text fontSize="1.5rem" mt="1.2rem">
    Schau, wie gut du diese Lektion verstanden hast! Dieses Quiz besteht
    aus 
    {props.questions.length} Fragen.
  </Text>
  <Button
    mt="1.2rem"
    variant={"brand"}
    size="lg"
    onClick={() => {
      setFinished(false);
      setCurrentQuestion(0);
      setCorrectAnswers(0);
      setFinishedValue(0);
      setCorrectAnswersList([]);
      onOpen();
    }}
  >
    {finished ? "Erneut starten" : "Quiz starten"}
  </Button>
```

```
      </Box>
    </>
  );
}
```

## packages/components/creator/studio/CourseEditor.tsx

```
/**
 * packages/components/creator/studio/CourseEditor.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 18.11.2022
 *
 */

import * as React from "react";
import CodeUpModal from "../../codeup/CodeUpModal";
import {
  Badge,
  Box,
  Button,
  ButtonGroup,
  Flex,
  Heading,
  Image,
  Input,
  InputGroup,
  InputLeftAddon,
  Select,
  Textarea,
  useToast,
} from "@chakra-ui/react";
import { FaCheckCircle, FaPen, FaTimesCircle, FaTrash } from "react-icons/fa";
import REST from "@codeup/rest";
import { useLoggedIn } from "@codeup/hooks";
import AccountManager from "@codeup/util/dist/AccountManager";

export default function CourseEditor(props: {
  course: {
    __v: number;
    _id: string;
    available: string;
    contentPositions: string[];
    createdAt: string;
    creator: string;
    description: string;
    friendlyName: string;
    name: string;
    splashImage: string;
  } | null;
  onOpen: () => void;
  onClose: () => void;
  isOpen: boolean;
  reload: () => void;
}) {
  const toast = useToast();

  const { userInfo } = useLoggedIn();
```

```
const [orgs, setOrgs] = React.useState<
  {
    owner: string;
    name: string;
    members: string[];
    icon: string;
    subscription: string;
    createdAt: string;
    _id: string;
  }[]
>([]);

React.useEffect(() => {
  REST.Orgs.getOrgs(AccountManager.getActiveToken()).then((r) => {
    setOrgs(r.payload.orgs);
  });
}, [userInfo]);

return (
  <>
    <CodeUpModal
      onOpen={props.onOpen}
      onClose={props.onClose}
      isOpen={props.isOpen}
    >
      {props.course !== null && (
        <form
          onSubmit={(e) => {
            e.preventDefault();

            const data = new FormData(e.target as HTMLFormElement);

            const name = data.get("name") as string;
            const friendlyName = data.get("friendlyName") as string;
            const available = data.get("available") as any;
            const description = data.get("description") as string;
            const splashImage = data.get("splashImage") as File;

            const FR = new FileReader();

            FR.readAsDataURL(splashImage);

            FR.onload = async () => {
              if (props.course) {
                const res = await REST.Creator.updateCourse({
                  token: localStorage.getItem("token") as string,
                  id: props.course ? props.course._id : "0x00",
                  update: {
                    name: name ? name : props.course.name,
                    friendlyName: friendlyName
                      ? friendlyName
                      : props.course.friendlyName,
                    available: available,
                    description: description
                      ? description
                      : props.course.description,
                    splashImage:
                      FR.result === "data:application/octet-stream; base64," ||
                      FR.result === "data:"
                        ? props.course.splashImage
```

```
              : FR.result,
          },
        });

        if (res.status === 200) {
          toast({
            title: "Kurs aktualisiert",
            description: "Der Kurs wurde erfolgreich aktualisiert",
            status: "success",
            duration: 1500,
            isClosable: true,
          });

          props.reload();
          props.onClose();
        } else {
          toast({
            title: "Fehler",
            description:
              "Aktualisierung fehlgeschlagen: " + res.payload.error,
            status: "error",
            duration: 1500,
            isClosable: true,
          });
        }
      }
    };
  }}
>
  <Image
    src={props.course.splashImage}
    alt={props.course.name}
    shadow={"xl"}
    w={"100%"}
    h={"150px"}
    cursor={"pointer"}
    _hover={{
      filter: "brightness(0.5)",
    }}
    transition={"filter 0.25s ease"}
    onClick={() => {
      document.getElementById(`edit_course-splashImage`)?.click();
    }}
    id={"edit_course-splashImage_Image"}
  />
  <input
    type={"file"}
    accept="image/svg+xml"
    name={"splashImage"}
    id={"edit_course-splashImage"}
    style={{
      display: "none",
    }}
    onChange={(e) => {
      const file = e.target.files?.item(0);
      if (file) {
        const reader = new FileReader();
        reader.readAsDataURL(file);
        reader.onload = () => {
          if (reader.result) {
```

```
          (
            document.getElementById(
              `edit_course-splashImage_Image`
            ) as HTMLElement
          ).setAttribute("src", reader.result as string);
        }
      };
    }
  }}
/>
<Input
  placeholder={"Name"}
  name={"name"}
  defaultValue={props.course.name}
  mt={"2rem"}
/>
<InputGroup mt={"2rem"}>
  <InputLeftAddon>/course/</InputLeftAddon>
  <Input
    placeholder={"URL Freundlicher Name"}
    name={"friendlyName"}
    defaultValue={props.course.friendlyName}
  />
</InputGroup>
<Select
  name={"available"}
  mt={"2rem"}
  defaultValue={props.course.available}
>
  <optgroup label={"Ganze Platform"}>
    {["editor", "admin"].includes(userInfo?.role) ? (
      <>
        <option value={"public"}>Öffentlich</option>
      </>
    ) : null}
    <option value={"private"}>Privat</option>
  </optgroup>
  <optgroup label={"Deine Organisationen"}>
    {orgs
      .filter((o) => {
        return o.owner === userInfo?.id;
      })
      .map((org) => {
        return (
          <option value={"org:" + org._id}>
            {org.name} ({org.members.length} Mitglieder)
          </option>
        );
      })}
  </optgroup>
</Select>
<Textarea
  placeholder={"Beschreibung"}
  mt={"2rem"}
  name={"description"}
  defaultValue={props.course.description}
/>
<ButtonGroup mt={"2rem"} w={"100%"}>
  <Button
    variant={"brand"}
```

```
              leftIcon={<FaCheckCircle />}
              w={"100%"}
              type={"submit"}
            >
              Speichern
            </Button>
          </ButtonGroup>
        </form>
      )}
    </CodeUpModal>
  </>
 );
}
```

## packages/components/creator/studio/CreatorStudioLayout.tsx

```
/**
 * packages/components/creator/studio/CreatorStudioLayout.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 18.11.2022
 *
 */

import * as React from "react";
import { useLoggedIn } from "@codeup/hooks";
import { ReactNode, useEffect, useState } from "react";
import ErrorComponent from "../../global/ErrorComponent";
import AuthenticateLoadingIndicator from "../../admin/AuthenticateLoadingIndicator";
import { useRouter } from "next/router";
import { Box } from "@chakra-ui/react";

export default function CreatorStudioLayout(props: { children: ReactNode }) {
  const { loggedIn, userInfo, loaded: userLoaded } = useLoggedIn();

  const [error, setError] = useState("");

  const router = useRouter();

  useEffect(() => {
    if (!userLoaded) return;

    if (!loggedIn) {
      setError("Du bist nicht dazu berechtigt diese Seite zu sehen!");
      return;
    }

    /* -- REMOVE CREATOR LOCK -- */
    /*
    if (!["admin", "editor"].includes(userInfo?.role)) {
      setError("Du bist nicht dazu berechtigt diese Seite zu sehen!");
    }
    */
  }, [loggedIn, userInfo, userLoaded]);

  return (
    <>
      {error !== "" ? (
        <>
```

```
          <ErrorComponent
            description={error}
            title={"Authentifizierungsfehler"}
          />
        </>
      ) : null}
      {!userLoaded && error === "" ? (
        <>
          <AuthenticateLoadingIndicator />
        </>
      ) : null}
      {userLoaded && error === "" ? (
        <Box bg={"#000"} h={"fit-content"} minH={"100vh"}>
          {props.children}
        </Box>
      ) : null}
    </>
  );
}
```

## packages/components/editor/CollaboratorManager.tsx

```
/**
 * packages/components/editor/CollaboratorManager.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 05.01.2023
 *
 */

import * as React from "react";
import {
  Button,
  Modal,
  ModalBody,
  ModalCloseButton,
  ModalContent,
  ModalFooter,
  ModalHeader,
  ModalOverlay,
} from "@chakra-ui/react";
import { Socket } from "socket.io-client";
import CollaboratorManagerBody from "./CollaboratorManagerBody";

export default function CollaboratorManager(props: {
  isOpen: boolean;
  onOpen: () => void;
  onClose: () => void;
  projectId: string | undefined;
  socket: Socket;
}) {
  return (
    <>
      <Modal
        isOpen={props.isOpen}
        onClose={props.onClose}
        size={["full", "5xl"]}
        scrollBehavior={"inside"}
      >
```

```
        <ModalOverlay />
        <ModalContent>
          <ModalHeader>Mitwirkende verwalten</ModalHeader>
          <ModalCloseButton />
          <ModalBody>
            <CollaboratorManagerBody
              projectId={props.projectId}
              socket={props.socket}
            />
          </ModalBody>

          <ModalFooter>
            <Button variant={"brand"} mr={3} onClick={props.onClose}>
              Schließen
            </Button>
          </ModalFooter>
        </ModalContent>
      </Modal>
    </>
  );
}
```

## packages/components/editor/CollaboratorManagerBody.tsx

```
/**
 * packages/components/editor/CollaboratorManagerBody.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 21.01.2023
 *
 */

import * as React from "react";
import {
  Avatar,
  Box,
  Button,
  ButtonGroup,
  Checkbox,
  Divider,
  Flex,
  FormControl,
  FormLabel,
  IconButton,
  Input,
  InputGroup,
  InputLeftAddon,
  Popover,
  PopoverArrow,
  PopoverBody,
  PopoverCloseButton,
  PopoverContent,
  PopoverHeader,
  PopoverTrigger,
  SimpleGrid,
  Stack,
  Tag,
  Text,
  Tooltip,
```

```
  useToast,
} from "@chakra-ui/react";
import { FaCogs, FaPlus, FaTrash } from "react-icons/fa";
import { HiMail } from "react-icons/hi";
import { Socket } from "socket.io-client";
import { useLoggedIn } from "@codeup/hooks";
import { useEffect, useState } from "react";

export default function CollaboratorManagerBody(props: {
  projectId: string | undefined;
  socket: Socket;
}) {
  const toast = useToast();

  const { userInfo } = useLoggedIn();

  const [contributors, setContributors] = useState<{
    owner: {
      email: string;
      id: string;
      permissions: string[];
      username: string;
      firstName: string;
      lastName: string;
    };
    contributors: {
      email: string;
      id: string;
      permissions: string[];
      username: string;
      firstName: string;
      lastName: string;
    }[];
  } | null>(null);

  useEffect(() => {
    if (props.projectId && props.projectId !== "0") {
      props.socket.on("editor:projects:contributors", (data: any) => {
        console.log(data);
        if (data.success) {
          setContributors({
            owner: data.owner,
            contributors: data.contributors,
          });
        } else {
          toast({
            title: "Error",
            description: data.error,
            status: "error",
            duration: 1500,
            isClosable: true,
            position: "bottom-right",
          });
        }
      });

      props.socket.emit("editor:projects:contributors", {
        id: props.projectId,
      });
    }
```

```
}, [props.projectId]);

return (
  <>
    {props.projectId &&
    props.projectId !== "0" &&
    contributors &&
    contributors.owner &&
    contributors.contributors ? (
      <>
        <Flex justifyContent={"right"} mr={[0, "2rem"]}>
          <Popover>
            <PopoverTrigger>
              <Button variant={"brand"} leftIcon={<FaPlus />}>
                Hinzufügen
              </Button>
            </PopoverTrigger>
            <PopoverContent bgColor={"black"}>
              <PopoverArrow bgColor={"black"} />
              <PopoverCloseButton />
              <PopoverHeader fontSize={["xl", "2xl"]}>
                Mitwirkenden hinzufügen
              </PopoverHeader>
              <PopoverBody fontSize={["lg", "xl"]}>
                <form
                  onSubmit={async (e) => {
                    e.preventDefault();

                    // @ts-ignore
                    const email = e.target.elements.email_i.value;

                    props.socket.emit("editor:projects:contributors:add", {
                      id: props.projectId,
                      email: email,
                    });

                    props.socket.on(
                      "editor:projects:contributors:add",
                      (data: any) => {
                        console.log(data);
                        if (data.success) {
                          let c0n: any = contributors;

                          c0n.contributors.push(data.contributor);

                          setContributors({
                            owner: contributors.owner,
                            contributors: [...c0n.contributors],
                          });
                        } else {
                          toast({
                            title: "Error",
                            description: data.error,
                            status: "error",
                            duration: 1500,
                            isClosable: true,
                            position: "bottom-right",
                          });
                        }
                      }
```

```
          );
        }}
      >
        <Stack>
          <FormControl>
            <FormLabel>E-Mail</FormLabel>
            <InputGroup>
              <InputLeftAddon>
                <HiMail />
              </InputLeftAddon>
              <Input
                id={"email_i"}
                name="email_i"
                placeholder={"E-Mail"}
              />
            </InputGroup>
          </FormControl>
          <Button type="submit" colorScheme={"whatsapp"}>
            Hinzufügen
          </Button>
        </Stack>
      </form>
    </PopoverBody>
  </PopoverContent>
 </Popover>
</Flex>
<Flex m={[0, "2rem"]} direction={"column"} fontSize={["xl", "2xl"]}>
 <Flex direction={"row"} mb={"1.2rem"}>
  <Box flex={"10%"}></Box>
  <Box flex={"20%"} fontWeight={"bold"}>
   Name
  </Box>
  <Box flex={"50%"}></Box>
  <Flex
   flex={"20%"}
   alignItems={"center"}
   justifyContent={"center"}
   fontWeight={"bold"}
  >
   Berechtigungen
  </Flex>
 </Flex>
 <Divider />
 <Flex direction={"row"} mt={"1.2rem"} mb={"1.2rem"}>
  <Box flex={"10%"}>
   <Avatar
    src={
     "/api/account/" + contributors.owner.username + "/picture"
    }
    size={"md"}
   />
  </Box>
  <Box flex={"20%"}>
   {contributors.owner.firstName} 
   {contributors.owner.lastName}
   <br />
   <Text color={"gray.400"} fontSize={["lg", "xl"]}>
    @{contributors.owner.username}
   </Text>
  </Box>
```

```
            <Box flex={"50%"}></Box>
            <Flex flex={"20%"} alignItems={"center"} justifyContent={"right"}>
              <Tooltip
                label={
                  "Diesem Benutzer gehört das Projekt, er hat alle Berechtigungen, inklusive der
Verwaltung von Mitwirkenden"
                }
                hasArrow={true}
              >
                <Tag size={"lg"} variant={"solid"} colorScheme={"red"}>
                  Besitzer
                </Tag>
              </Tooltip>
            </Flex>
          </Flex>
          <Divider />
          {contributors.contributors.map((contributor) => {
            return (
              <>
                <Flex direction={"row"} mt={"1.2rem"} mb={"1.2rem"}>
                  <Box flex={"10%"}>
                    <Avatar
                      src={
                        "/api/account/" + contributor.username + "/picture"
                      }
                      size={"md"}
                    />
                  </Box>
                  <Box flex={"20%"}>
                    {contributor.firstName} 
                    {contributor.lastName}
                    <br />
                    <Text color={"gray.400"} fontSize={["lg", "xl"]}>
                      @{contributor.username}
                    </Text>
                  </Box>
                  <Box flex={"10%"}></Box>
                  <Flex
                    flex={"60%"}
                    justifyContent={"right"}
                    direction={["column", "row"]}
                    gap={["2rem", 0]}
                  >
                    <Box flex={"80%"}>
                      <SimpleGrid minChildWidth={130} w={"100%"} spacing={3}>
                        {contributor.permissions.includes("READ") ? (
                          <Tag
                            size={"lg"}
                            variant={"solid"}
                            colorScheme={"teal"}
                          >
                            Lesen
                          </Tag>
                        ) : null}
                        {contributor.permissions.includes("WRITE") ? (
                          <Tag
                            size={"lg"}
                            variant={"solid"}
                            colorScheme={"teal"}
                          >
```

```
        Schreiben
      </Tag>
    ) : null}
    {contributor.permissions.includes("DELETE") ? (
      <Tag
        size={"lg"}
        variant={"solid"}
        colorScheme={"teal"}
      >
        Löschen
      </Tag>
    ) : null}
    {contributor.permissions.includes("RENAME") ? (
      <Tag
        size={"lg"}
        variant={"solid"}
        colorScheme={"teal"}
      >
        Umbenennen
      </Tag>
    ) : null}
    {contributor.permissions.includes(
      "MANAGE-CONTRIBUTORS"
    ) ? (
      <Tag
        size={"lg"}
        variant={"solid"}
        colorScheme={"teal"}
      >
        Mitwirkende verwalten
      </Tag>
    ) : null}
  </SimpleGrid>
</Box>
<Flex
  flex={"20%"}
  justifyContent={"center"}
  alignItems={"center"}
>
  <ButtonGroup>
    <Popover>
      <PopoverTrigger>
        <IconButton aria-label={"Mitwirkenden entfernen"}>
          <FaTrash />
        </IconButton>
      </PopoverTrigger>
      <PopoverContent bgColor={"gray.800"}>
        <PopoverArrow bgColor={"gray.800"} />
        <PopoverCloseButton />
        <PopoverHeader>
          Mitwirkenden entfernen
        </PopoverHeader>
        <PopoverBody>
          <Stack>
            <Text fontSize={"lg"}>
              Willst du den mitwirkenden 
              {contributor.firstName} 
              {contributor.lastName} (
              {contributor.username}) wirklich entfernen?
            </Text>
```

```
        <Button
          variant={"solid"}
          colorScheme={"red"}
          onClick={async () => {
            props.socket.emit(
              "editor:projects:contributors:delete",
              {
                id: props.projectId,
                email: contributor.email,
              }
            );

            props.socket.on(
              "editor:projects:contributors:delete",
              (data: any) => {
                if (data.success) {
                  let c0n: any = contributors;

                  c0n = c0n.contributors.filter(
                    (c: any) =>
                      c.email !== contributor.email
                  );

                  setContributors({
                    owner: contributors.owner,
                    contributors: [...c0n],
                  });
                } else {
                  toast({
                    title: "Fehler",
                    description: data.error,
                    position: "top",
                    status: "error",
                    variant: "solid",
                    isClosable: true,
                  });
                }
              }
            );
          }}
        >
          Entfernen
        </Button>
      </Stack>
    </PopoverBody>
  </PopoverContent>
</Popover>

{userInfo.id !== contributor.id ? (
  <>
    <Popover>
      <PopoverTrigger>
        <IconButton
          aria-label={"Berechtigungen verwalten"}
        >
          <FaCogs />
        </IconButton>
      </PopoverTrigger>
      <PopoverContent bgColor={"gray.800"}>
        <PopoverArrow bgColor={"gray.800"} />
```

```
<PopoverCloseButton />
<PopoverHeader>
  Berechtigungen ändern
</PopoverHeader>
<PopoverBody>
  <Stack>
    <Checkbox
      defaultChecked={contributor.permissions.includes(
        "READ"
      )}
      disabled={true}
      colorScheme={"brand"}
    >
      Lesen
    </Checkbox>
    <Checkbox
      defaultChecked={contributor.permissions.includes(
        "WRITE"
      )}
      id={
        "cb-upd-perms-" +
        contributor.id +
        "-write"
      }
      colorScheme={"brand"}
    >
      Schreiben
    </Checkbox>
    <Checkbox
      defaultChecked={contributor.permissions.includes(
        "DELETE"
      )}
      id={
        "cb-upd-perms-" +
        contributor.id +
        "-delete"
      }
      colorScheme={"brand"}
    >
      Löschen
    </Checkbox>
    <Checkbox
      defaultChecked={contributor.permissions.includes(
        "RENAME"
      )}
      id={
        "cb-upd-perms-" +
        contributor.id +
        "-rename"
      }
      colorScheme={"brand"}
    >
      Umbenennen
    </Checkbox>
    <Checkbox
      defaultChecked={contributor.permissions.includes(
        "MANAGE-CONTRIBUTORS"
      )}
      id={
        "cb-upd-perms-" +
```

```
    contributor.id +
    "-manage_contributors"
  }
  colorScheme={"brand"}
>
  Mitwirkende verwalten
</Checkbox>
<Button
  variant={"brand"}
  onClick={async () => {
    let perms: string[] = [
      "READ",
      (
        document.getElementById(
          "cb-upd-perms-" +
            contributor.id +
            "-write"
        ) as HTMLInputElement
      ).checked
        ? "WRITE"
        : "null",
      (
        document.getElementById(
          "cb-upd-perms-" +
            contributor.id +
            "-delete"
        ) as HTMLInputElement
      ).checked
        ? "DELETE"
        : "null",
      (
        document.getElementById(
          "cb-upd-perms-" +
            contributor.id +
            "-rename"
        ) as HTMLInputElement
      ).checked
        ? "RENAME"
        : "null",
      (
        document.getElementById(
          "cb-upd-perms-" +
            contributor.id +
            "-manage_contributors"
        ) as HTMLInputElement
      ).checked
        ? "MANAGE-CONTRIBUTORS"
        : "null",
    ].filter((e) => e !== "null");

    props.socket.emit(
      "editor:projects:contributors:permission",
      {
        id: props.projectId,
        email: contributor.email,
        permissions: perms,
      }
    );

    props.socket.on(
```

```
                                "editor:projects:contributors:permission",
                                (data: any) => {
                                  if (data.success) {
                                    props.socket.emit(
                                      "editor:projects:contributors",
                                      {
                                        id: props.projectId,
                                      }
                                    );
                                  } else {
                                    toast({
                                      title: "Fehler",
                                      description: data.error,
                                      position: "top",
                                      status: "error",
                                      variant: "solid",
                                      isClosable: true,
                                    });
                                  }
                                }
                              );
                            }}
                          >
                            Speichern
                          </Button>
                        </Stack>
                      </PopoverBody>
                    </PopoverContent>
                  </Popover>
                </>
              ) : null}
            </ButtonGroup>
          </Flex>
        </Flex>
      </Flex>
    </>
  );
})}
</Flex>
</>
) : null}
</>
);
}
```

## packages/components/editor/EditorPageSidebar.tsx

```
/**
 * packages/components/editor/EditorPageSidebar.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 25.01.23
 *
 */
import * as React from "react";
import {
  Box,
  Flex,
  IconButton,
```

```
  Tooltip,
  useDisclosure,
} from "@chakra-ui/react";
import {
  FaCode,
  FaEye,
  FaFile,
  FaFileAlt,
  FaFileCode,
  FaHome,
  FaJs,
  FaLaptopCode,
  FaResolving,
} from "react-icons/fa";
import { useRouter } from "next/router";
import { Socket } from "socket.io-client";
import PageResourceManager from "./PageResourceManager";
import { PreviewModal, SnippetViewer } from "../ide";
import EditorPreviewModal from "./EditorPreviewModal";

export default function EditorPageSidebar(props: {
  page: {
    url: string;
    mainContent: string;
    type: string;
    resources: {
      name: string;
      type: string;
      content: string;
    }[];
  };
  socket: Socket;
  projectId: string;
  setPage: (newPage: any) => void;
  setCurrentResource: (xx: string) => void;
}) {
  const router = useRouter();

  const {
    onOpen: onRMOpen,
    isOpen: isRMOpen,
    onClose: onRMClose,
  } = useDisclosure();

  const {
    onOpen: onSVOpen,
    isOpen: isSVOpen,
    onClose: onSVClose,
  } = useDisclosure();

  const {
    onOpen: onPROpen,
    isOpen: isPROpen,
    onClose: onPRClose,
  } = useDisclosure();

  return (
    <>
      <Box p={"2rem"} w={"100%"} h={"100%"} rounded={"xl"}>
        <Box backgroundColor={"black"} h={"100%"} boxShadow={"2xl"}>
```

```
<Flex direction={"column"} h={"100%"}>
  <Tooltip label={"Code"} placement={"right"} hasArrow>
    <IconButton
      aria-label={""}
      variant="ghost"
      size={"lg"}
      fontSize={"2xl"}
      p={"2rem"}
      icon={<FaCode />}
      rounded={"none"}
      onClick={() => {
        props.setCurrentResource("codeup.main_resource");
        // @ts-ignore
        window.__editor_current_resource = "codeup.main_resource";
      }}
    />
  </Tooltip>
  <Tooltip label={"Resourcen"} placement={"right"} hasArrow>
    <IconButton
      aria-label={""}
      variant="ghost"
      size={"lg"}
      fontSize={"2xl"}
      p={"2rem"}
      icon={<FaFileAlt />}
      rounded={"none"}
      onClick={onRMOpen}
    />
  </Tooltip>
  <Tooltip label={"Snippets"} placement={"right"} hasArrow>
    <IconButton
      aria-label={""}
      variant="ghost"
      size={"lg"}
      fontSize={"2xl"}
      p={"2rem"}
      icon={<FaLaptopCode />}
      rounded={"none"}
      onClick={onSVOpen}
    />
  </Tooltip>

  <Tooltip label={"Vorschau"} placement={"right"} hasArrow>
    <IconButton
      aria-label={""}
      variant="ghost"
      size={"lg"}
      fontSize={"2xl"}
      p={"2rem"}
      icon={<FaEye />}
      rounded={"none"}
      onClick={onPROpen}
    />
  </Tooltip>
  <Tooltip label={"Übersicht"} placement={"right"} hasArrow>
    <IconButton
      aria-label={""}
      variant="ghost"
      size={"lg"}
      fontSize={"2xl"}
```

```
              p={"2rem"}
              icon={<FaHome />}
              rounded={"none"}
              onClick={() => {
                router.push("/editor/" + props.projectId);
              }}
            />
          </Tooltip>
        </Flex>
      </Box>
    </Box>
    <PageResourceManager
      socket={props.socket}
      onOpen={onRMOpen}
      onClose={onRMClose}
      isOpen={isRMOpen}
      page={props.page}
      setPage={props.setPage}
      setCurrentResource={props.setCurrentResource}
    />
    <SnippetViewer isOpen={isSVOpen} onClose={onSVClose} />
    <EditorPreviewModal
      url={"/editor-preview/" + router.query.page_id}
      isOpen={isPROpen}
      onOpen={onPROpen}
      onClose={onPRClose}
    />
   </>
  );
}
```

## packages/components/editor/EditorPagesSidebar.tsx

```
/**
 * packages/components/editor/EditorPagesSidebar.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 20.01.2023
 *
 */

import * as React from "react";
import { ReactNode } from "react";
import { useRouter } from "next/router";
import {
  Button,
  Flex,
  Input,
  InputGroup,
  InputLeftElement,
} from "@chakra-ui/react";
import {
  FaChartArea,
  FaCogs,
  FaDatabase,
  FaExternalLinkSquareAlt,
  FaFileExport,
  FaGit,
  FaHome,
```

```
  FaRegFile,
  FaRegImages,
  FaRocket,
  FaSearch,
  FaServer,
  FaUsers,
} from "react-icons/fa";
import { BiCodeBlock } from "react-icons/bi";
import Sidebar from "../global/Sidebar";

export default function EditorPagesSidebar() {
  const router = useRouter();

  const [projectId, setProjectId] = React.useState<string | null>(null);

  React.useEffect(() => {
    if (router.query.project_id) {
      setProjectId(router.query.project_id as string);
    }
  }, [router.query.project_id]);

  const subPages: {
    title: string;
    link: string;
    icon: ReactNode;
    available: boolean;
  }[] = [
    {
      title: "Seiten",
      link: "/editor/" + projectId,
      icon: <FaRegFile />,
      available: true,
    },
    {
      title: "Serverless",
      link: "/editor/" + projectId + "/serverless",
      icon: <FaServer />,
      available: true,
    },
    {
      title: "Mitwirkende",
      link: "/editor/" + projectId + "/contributors",
      icon: <FaUsers />,
      available: true,
    },
    {
      title: "Deployment",
      link: "/editor/" + projectId + "/deployment",
      icon: <FaRocket />,
      available: true,
    },
    {
      title: "Discovery",
      link: "/editor/" + projectId + "/discovery",
      icon: <BiCodeBlock />,
      available: true,
    },
    {
      title: "Statische Dateien",
      link: "/editor/" + projectId + "/staticassets",
```

```
      icon: <FaRegImages />,
      available: true,
    },
    {
      title: "Export",
      link: "/editor/" + projectId + "/export",
      icon: <FaFileExport />,
      available: true,
    },
    {
      title: "Home",
      link: "/editor",
      icon: <FaHome />,
      available: true,
    },
  ];

  const [query, setQuery] = React.useState<string>("");

  return (
    <>
      <Sidebar
        entries={subPages.map((e) => {
          return {
            title: e.title,
            onClick: () => {
              router.push(e.link);
            },
            icon: e.icon,
            disabled: !e.available,
          };
        })}
        title={"Projekt Editor"}
      />
    </>
  );
}
```

## packages/components/editor/EditorPreviewModal.tsx

```
/**
 * packages/components/editor/EditorPreviewModal.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 27.01.23
 *
 */

import * as React from "react";
import {
  Button,
  Modal,
  ModalBody,
  ModalCloseButton,
  ModalContent,
  ModalFooter,
  ModalHeader,
  ModalOverlay,
} from "@chakra-ui/react";
```

```
import { FaExternalLinkAlt } from "react-icons/fa";

export default function EditorPreviewModal(props: {
  url: string;
  isOpen: boolean;
  onOpen: () => void;
  onClose: () => void;
}) {
  const onClose = () => {
    props.onClose();
  };

  return (
    <>
      <Modal isOpen={props.isOpen} onClose={onClose} size={"full"}>
        <ModalOverlay />
        <ModalContent>
          <ModalHeader>Vorschau</ModalHeader>
          <ModalCloseButton />
          <ModalBody>
            <iframe
              width={"100%"}
              style={{ border: "none", height: "80vh" }}
              id={"ifr-prev"}
              src={props.url}
            ></iframe>
          </ModalBody>

          <ModalFooter>
            <Button
              rightIcon={<FaExternalLinkAlt />}
              variant="outline"
              mr={3}
              onClick={() => {
                window.open(props.url, "_blank");
              }}
            >
              In neuem Tab öffnen
            </Button>

            <Button
              variant="outline"
              mr={3}
              onClick={() => {
                const iframe = document.getElementById(
                  "ifr-prev"
                ) as HTMLIFrameElement;

                if (iframe.requestFullscreen) {
                  iframe.requestFullscreen();
                  // @ts-ignore
                } else if (iframe.webkitRequestFullscreen) {
                  // @ts-ignore
                  iframe.webkitRequestFullscreen();
                  // @ts-ignore
                } else if (iframe.mozRequestFullScreen) {
                  // @ts-ignore
                  iframe.mozRequestFullScreen();
                  // @ts-ignore
                } else if (iframe.msRequestFullscreen) {
```

```
                // @ts-ignore
                iframe.msRequestFullscreen();
              }
            }}
          >
            Vollbild
          </Button>
        </ModalFooter>
      </ModalContent>
    </Modal>
  </>
 );
}
```

## packages/components/editor/EditorProjectLayout.tsx

```
/**
 * packages/components/editor/EditorProjectLayout.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 21.01.2023
 *
 */

import * as React from "react";
import { useLoggedIn } from "@codeup/hooks";
import { Box, CircularProgress, Flex, Heading } from "@chakra-ui/react";
import EditorPagesSidebar from "./EditorPagesSidebar";

export default function EditorProjectLayout(props: {
  children: React.ReactNode;
}) {
  const { loggedIn, loaded, userInfo } = useLoggedIn();

  return (
    <>
      {loggedIn ? (
        <>
          <Flex w={"100%"} minH={"100vh"} direction={"row"} bg={"#000"}>
            <Box flex={"10%"}>
              <EditorPagesSidebar />
            </Box>
            <Box flex={"80%"} bgColor={"#000"}>
              {props.children}
            </Box>
          </Flex>
        </>
      ) : (
        <>
          <Flex
            w={"100%"}
            h={"100vh"}
            justifyContent={"center"}
            alignItems={"center"}
            direction={"column"}
          >
            <CircularProgress size={"80px"} isIndeterminate color="green.300" />
            <Heading size={"lg"} mt={4}>
              Anmelden ...
```

```
          </Heading>
        </Flex>
      </>
    )}
  </>
 );
}
```

## packages/components/editor/PageResourceManager.tsx

```
/**
 * packages/components/editor/PageResourceManager.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 26.01.23
 *
 */

import * as React from "react";
import { Socket } from "socket.io-client";
import {
  Button,
  ButtonGroup,
  Grid,
  GridItem,
  Heading,
  Modal,
  ModalBody,
  ModalCloseButton,
  ModalContent,
  ModalFooter,
  ModalHeader,
  ModalOverlay,
  Stack,
  Text,
  chakra,
  Badge,
} from "@chakra-ui/react";
import { FaLink, FaPen, FaPlus, FaTrash } from "react-icons/fa";
import { useRouter } from "next/router";
import REST from "@codeup/rest/dist";
import PopupManager from "@codeup/web/src/global/PopupManager";

export default function PageResourceManager(props: {
  page: any;
  setPage: (newPage: any) => void;
  socket: Socket;
  onOpen: () => void;
  onClose: () => void;
  isOpen: boolean;
  setCurrentResource: (xx: string) => void;
}) {
  const router = useRouter();

  return (
    <>
      <Modal
        isOpen={props.isOpen}
        onClose={props.onClose}
```

```
  size={["full", "4xl"]}
>
  <ModalOverlay>
    <ModalContent>
      <ModalHeader>Resourcen verwalten</ModalHeader>
      <ModalCloseButton />
      <ModalBody>
        <Text mb={4}>
          <chakra.span color={"brand.500"} fontWeight={"bold"}>
            Information
          </chakra.span>
          : Jede Resource ist mit der eindeutigen URL öffentlich
          erreichbar. Speichere keine sensiblen Daten in einer Resource.
        </Text>
        <Grid templateColumns={"repeat(2, 1fr)"} gap={6}>
          <GridItem
            key={"new"}
            bg={"gray.900"}
            p={"1rem"}
            borderRadius={"0.5rem"}
            _hover={{
              cursor: "pointer",
              borderColor: "brand.500",
              borderWidth: "1px",
              borderStyle: "solid",
            }}
            onClick={async () => {
              const name = (
                await PopupManager.promptAsync({
                  title: "Name",
                  helperText:
                    "Gib einen eindeutigen Namen für die Resource ein",
                })
              ).replace(/_/g, "-");

              const type = await PopupManager.selectAsync({
                title: "Typ auswählen",
                helperText: "Wähl den Typ der Resource aus.",
                choices: [
                  "CSS",
                  "SCSS",
                  "Sass",
                  "JavaScript",
                  "TypeScript",
                  "JSON",
                  "Text (Beliebig)",
                ],
              });

              if (name && type) {
                props.socket.emit("editor:projects:page:content:set", {
                  projectId: router.query.project_id as string,
                  pageId: router.query.page_id as string,
                  resource: name,
                  resourceType: type,
                  content: "Code einfügen",
                });

                props.socket.emit("editor:projects:page:content", {
                  projectId: router.query.project_id as string,
```

```
        pageId: router.query.page_id as string,
      });

      props.setPage({
        ...props.page,
        resources: [
          ...props.page.resources,
          {
            name,
            type,
            content: "Code einfügen",
          },
        ],
      });
    }
  }}
>
  <Heading
    fontSize={"8xl"}
    textAlign={"center"}
    color={"brand.500"}
    alignItems={"center"}
    justifyContent={"center"}
    display={"flex"}
    h={"100%"}
  >
    <FaPlus />
  </Heading>
</GridItem>
{props.page.resources.map((r: any) => (
  <>
    <GridItem
      key={r.name}
      bg={"gray.800"}
      p={"1rem"}
      borderRadius={"0.5rem"}
    >
      <Heading size={"lg"}>{r.name}</Heading>
      <Badge>{r.type}</Badge>
      <Stack mt={"2rem"}>
        <Button
          colorScheme={"red"}
          leftIcon={<FaTrash />}
          onClick={async () => {
            if (
              await PopupManager.confirmAsync({
                title: "Löschen",
                question:
                  "Willst du die Resource wirklich unwiederruflich löschen?",
              })
            ) {
              props.socket.emit(
                "editor:projects:page:content:set",
                {
                  projectId: router.query.project_id as string,
                  pageId: router.query.page_id as string,
                  resource: r.name,
                  content: "codeup.delete_resource",
                }
              );
```

```
                props.setPage({
                  ...props.page,
                  resources: props.page.resources.filter(
                    (rx: any) => rx.name !== r.name
                  ),
                });
              }
            }}
          >
            Löschen
          </Button>
          <Button
            colorScheme={"blue"}
            leftIcon={<FaPen />}
            onClick={() => {
              // @ts-ignore
              window.__editor_current_resource = r.name;
              props.setCurrentResource(r.name);
              props.onClose();
            }}
          >
            Bearbeiten
          </Button>
          <Button
            colorScheme={"green"}
            leftIcon={<FaLink />}
            onClick={() => {
              const url =
                window.location.protocol +
                "//" +
                window.location.host +
                "/editor-static/page-res/" +
                router.query.page_id +
                "/" +
                r.name;

              navigator.clipboard.writeText(url);
              PopupManager.alert({
                title: "URL Kopiert",
                description: "URL: " + url,
                callback: () => {},
              });
            }}
          >
            Statische URL kopieren
          </Button>
        </Stack>
      </GridItem>
    </>
  ))}
 </Grid>
</ModalBody>
<ModalFooter>
 <ButtonGroup>
  <Button
   colorScheme={"red"}
   variant={"outline"}
   onClick={props.onClose}
  >
   Schließen
```

```
              </Button>
            </ButtonGroup>
          </ModalFooter>
        </ModalContent>
      </ModalOverlay>
    </Modal>
  </>
 );
}
```

## packages/components/editorv2/EditorFileExplorer.tsx

```
/**
 * packages/components/editorv2/EditorFileExplorer.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2024 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 10.06.2024
 *
 */

import * as React from "react";
import {
  Box,
  Breadcrumb,
  BreadcrumbItem,
  Flex,
  Grid,
  HStack,
  IconButton,
  ListIcon,
  ListItem,
  MenuItem,
  MenuList,
  Stack,
  Text,
  UnorderedList,
} from "@chakra-ui/react";
import { CHeading } from "../global/CHeading";
import {
  FaFile,
  FaFolder,
  FaFolderPlus,
  FaInfoCircle,
  FaPen,
  FaTrash,
} from "react-icons/fa";
import EditorFileExplorerIcon from "./EditorFileExplorerIcon";
import { ContextMenu } from "chakra-ui-contextmenu";
import PopupManager from "@codeup/web/src/global/PopupManager";
import byteSize from "byte-size";
import { useEffect } from "react";

type entry = {
  name: string;
  type: string;
  children: entry[];
  content?: any;
};
```

```
export default function EditorFileExplorer(props: {
  projectName: string;
  fileTree: entry;
  deleteFile: (path: string) => void;
  createFile: (path: string) => void;
  createDirectory: (path: string) => void;
  openFile: (path: string) => void;
  renameFile: (path: string, newName: string) => void;
}) {
  const [currentPath, setCurrentPath] = React.useState<string[]>([]);
  const [parentEntry, setParentEntry] = React.useState<entry | null>(null);
  const [currentEntry, setCurrentEntry] = React.useState<entry | null>(
    props.fileTree
  );

  useEffect(() => {
    if (currentPath.length > 0) {
      setCurrentEntry(findEntryByPath(props.fileTree, currentPath));
    } else {
      setCurrentEntry(props.fileTree);
      setCurrentPath([]);
    }
  }, [props.fileTree]);

  if (!props.fileTree) return <></>;

  return (
    <Stack gap={2}>
      <HStack justifyContent={"space-between"} gap={2}>
        <Text fontWeight={700} color={"brand.500"} fontSize={"lg"}>
          Explorer
        </Text>
        <HStack gap={2}>
          <IconButton
            aria-label={"Create File"}
            icon={<FaFile />}
            onClick={async () => {
              const name = await PopupManager.promptAsync({
                title: "Neue Datei",
                helperText: "Bitte gib den Namen der neuen Datei ein.",
              });

              if (!name) return;

              props.createFile(currentPath.concat(name).join("/"));
            }}
          />
          <IconButton
            aria-label={"Create File"}
            icon={<FaFolderPlus />}
            onClick={async () => {
              const name = await PopupManager.promptAsync({
                title: "Neuer Ordner",
                helperText: "Bitte gib den Namen des neuen Ordners ein.",
              });

              if (!name) return;

              props.createDirectory(currentPath.concat(name).join("/"));
            }}
```

```
    />
  </HStack>
</HStack>
<Breadcrumb>
  <BreadcrumbItem>
    <Text
      color={"brand.500"}
      cursor={"pointer"}
      onClick={() => {
        setCurrentPath([]);
        setCurrentEntry(props.fileTree);
      }}
    >
      {props.projectName}
    </Text>
  </BreadcrumbItem>
  {currentPath.map((path, index) => (
    <BreadcrumbItem
      key={index}
      isCurrentPage={index === currentPath.length - 1}
    >
      <Text
        color={"brand.500"}
        cursor={"pointer"}
        onClick={() => {
          setCurrentPath(currentPath.slice(0, index + 1));
          setCurrentEntry(
            findEntryByPath(
              props.fileTree,
              currentPath.slice(0, index + 1)
            )
          );
        }}
      >
        {path}
      </Text>
    </BreadcrumbItem>
  ))}
</Breadcrumb>
<Grid
  templateColumns={"repeat(auto-fit, minmax(150px, 1fr))"}
  autoColumns={"minmax(0,1fr)"}
  gap={6}
>
  {currentEntry &&
    currentEntry.children
      .sort((a, b) => {
        if (a.type === "directory" && b.type === "file") {
          return -1;
        } else if (a.type === "file" && b.type === "directory") {
          return 1;
        } else {
          return 0;
        }
      })
      .map((entry, index) => {
        return (
          <ContextMenu
            renderMenu={() => (
              <>
```

```
                <MenuList bg={"#000"}>
                  <MenuItem
                    icon={<FaInfoCircle />}
                    onClick={async () => {
                      await PopupManager.alertAsync({
                        title: entry.name,
                        description: (
                          <>
                            <UnorderedList spacing={2}>
                              <ListItem>Name: {entry.name}</ListItem>
                              <ListItem>Typ: {entry.type}</ListItem>
                              {entry.content && (
                                <ListItem>
                                  Größe:{" "}
                                  {byteSize(
                                    entry.content.length
                                  ).toString()}
                                </ListItem>
                              )}
                            </UnorderedList>
                          </>
                        ),
                      });
                    }}
                  >
                    Informationen
                  </MenuItem>
                  <MenuItem
                    icon={<FaTrash />}
                    color={"red.500"}
                    onClick={() => {
                      props.deleteFile(
                        currentPath.concat(entry.name).join("/")
                      );
                    }}
                  >
                    Löschen
                  </MenuItem>
                  <MenuItem
                    icon={<FaPen />}
                    color={"brand.500"}
                    onClick={async () => {
                      const newName = await PopupManager.promptAsync({
                        title: "Neuer Name",
                        helperText:
                          "Bitte geben Sie den neuen Namen ein. Tipp: Um eine Datei zu verschieben, gib den neuen relativen Pfad ein (z.B. ../public/index.css)",
                      });

                      if (!newName) return;

                      props.renameFile(
                        currentPath.concat(entry.name).join("/"),
                        newName
                      );
                    }}
                  >
                    Umbenennen
                  </MenuItem>
                </MenuList>
```

```
                </>
              )}
            >
              {(ref) => (
                <Stack
                  p={2}
                  gap={2}
                  cursor={"pointer"}
                  rounded={"md"}
                  bg={"gray.800"}
                  w={"100%"}
                  ref={ref as any}
                  onClick={() => {
                    if (entry.type === "file") {
                      props.openFile(entry.name);
                    } else {
                      setCurrentPath([...currentPath, entry.name]);
                      setParentEntry(currentEntry);
                      setCurrentEntry(entry);
                    }
                  }}
                >
                  <Flex
                    fontSize={"2xl"}
                    w={"100%"}
                    justifyContent={"center"}
                  >
                    <EditorFileExplorerIcon entry={entry as any} />
                  </Flex>
                  <Text textAlign={"center"}>{entry.name}</Text>
                </Stack>
              )}
            </ContextMenu>
          );
        })}
      </Grid>
    </Stack>
  );
}

const findEntryByPath = (entry: entry, path: string[]): entry | null => {
  if (path.length === 0) return entry;
  const next = entry.children.find((c) => c.name === path[0]);
  if (!next) return null;
  return findEntryByPath(next, path.slice(1));
};
```

## packages/components/editorv2/EditorFileExplorerIcon.tsx

```
/**
 * packages/components/editorv2/EditorFileExplorerIcon.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2024 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 10.06.2024
 *
 */

import * as React from "react";
import { FaFile, FaFolder, FaGit, FaJsSquare } from "react-icons/fa";
```

```
import {
  FaCss3,
  FaDocker,
  FaFileCircleQuestion,
  FaHtml5,
  FaNodeJs,
} from "react-icons/fa6";
import {
  BsFiletypeCss,
  BsFiletypeHtml,
  BsFiletypeJava,
  BsFiletypeJs,
  BsFiletypeJson,
  BsFiletypeJsx,
  BsFiletypeMd,
  BsFiletypeMdx,
  BsFiletypeSass,
  BsFiletypeScss,
  BsFiletypeSh,
  BsFiletypeSql,
  BsFiletypeSvg,
  BsFiletypeTsx,
} from "react-icons/bs";
import { SiTypescript } from "react-icons/si";

export default function EditorFileExplorerIcon(props: {
  entry: {
    type: string;
    name: string;
    content: string;
    children: any[];
  };
}) {
  const iconByType: {
    [key: string]: JSX.Element;
  } = {
    directory: <FaFolder />,
    file: <FaFile />,
  };

  const iconByExtension: {
    [key: string]: JSX.Element;
  } = {
    html: <BsFiletypeHtml />,
    js: <BsFiletypeJs />,
    css: <BsFiletypeCss />,
    jsx: <BsFiletypeJsx />,
    tsx: <BsFiletypeTsx />,
    md: <BsFiletypeMd />,
    mdx: <BsFiletypeMdx />,
    java: <BsFiletypeJava />,
    json: <BsFiletypeJson />,
    sass: <BsFiletypeSass />,
    scss: <BsFiletypeScss />,
    sh: <BsFiletypeSh />,
    sql: <BsFiletypeSql />,
    svg: <BsFiletypeSvg />,
  };

  const iconByName: {
```

```
    [key: string]: JSX.Element;
  } = {
    "package.json": <FaNodeJs />,
    "tsconfig.json": <SiTypescript />,
    Dockerfile: <FaDocker />,
    ".git": <FaGit />,
    "project.codeup.json": <img src={"/codeup.svg"} width={26} height={26} />,
  };

  if (!props.entry) return <></>;

  return (
    <>
      {Object.keys(iconByName).includes(props.entry.name) ? (
        iconByName[props.entry.name]
      ) : Object.keys(iconByExtension).includes(
          props.entry.name.split(".").pop() as string
        ) ? (
        iconByExtension[props.entry.name.split(".").pop() as string]
      ) : Object.keys(iconByType).includes(props.entry.type) ? (
        iconByType[props.entry.type]
      ) : (
        <FaFileCircleQuestion />
      )}
    </>
  );
}
```

## packages/components/editorv2/EditorProjectLayout.tsx

```
/**
 * packages/components/editorv2/EditorProjectLayout.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2024 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 30.05.2024
 *
 */

import * as React from "react";
import { IProject } from "@codeup/web/src/types/IProject";
import AppShell from "../global/AppShell";
import { useEffect } from "react";
import { useUserLoggedInRedirect } from "@codeup/hooks";
import {
  FaBookOpen,
  FaChartLine,
  FaCode,
  FaCogs,
  FaDatabase,
  FaFile,
  FaHome,
} from "react-icons/fa";
import { GoProject } from "react-icons/go";
import { VscIssues } from "react-icons/vsc";
import {
  FaChartSimple,
  FaCodeBranch,
  FaDiagramNext,
  FaDiagramSuccessor,
```

```
  FaFileLines,
  FaLanguage,
  FaRotate,
} from "react-icons/fa6";

export default function EditorProjectLayout(props: {
  children: React.ReactNode;
  project_id: string;
}) {
  useUserLoggedInRedirect({
    allowAnonymous: false,
  });

  return (
    <>
      <AppShell
        title={"Projekt"}
        sub_pages={[
          {
            text: "Home",
            icon: <FaHome />,
            href: "/projects/" + props.project_id,
          },
          {
            text: "Probleme",
            icon: <VscIssues />,
            href: "/projects/" + props.project_id + "/issues",
          },
          {
            text: "Planungsübersicht",
            icon: <GoProject />,
            href: "/projects/" + props.project_id + "/planning/overview",
          },
          {
            text: "Flussdiagramme",
            icon: <FaDiagramSuccessor />,
            href: "/projects/" + props.project_id + "/flows",
          },
          {
            text: "Versionsverwaltung",
            icon: <FaCodeBranch />,
            href: "/projects/" + props.project_id + "/vsc",
          },
          {
            text: "Einstellungen",
            icon: <FaCogs />,
            href: "/projects/" + props.project_id + "/settings",
          },
          {
            text: "Statische Ressourcen",
            icon: <FaFile />,
            href: "/projects/" + props.project_id + "/static",
          },
          {
            text: "Wiki",
            icon: <FaBookOpen />,
            href: "/projects/" + props.project_id + "/wiki",
          },
          {
            text: "Datenbanken",
```

```
            icon: <FaDatabase />,
            href: "/projects/" + props.project_id + "/dbs",
          },
          {
            text: "Statistiken",
            icon: <FaChartLine />,
            href: "/projects/" + props.project_id + "/stats",
          },
          {
            text: "Server-Logs",
            icon: <FaFileLines />,
            href: "/projects/" + props.project_id + "/srvlogs",
          },
          {
            text: "Profi-Editor",
            icon: <FaCode />,
            href: "/projects/" + props.project_id + "/editor/professional",
          },
          {
            text: "Einfacher Editor",
            icon: <FaCode />,
            href: "/projects/" + props.project_id + "/editor/simple",
          },
          {
            text: "UML-Diagramme",
            icon: <FaChartSimple />,
            href: "/projects/" + props.project_id + "/uml",
          },
          {
            text: "Lokalisierung",
            icon: <FaLanguage />,
            href: "/projects/" + props.project_id + "/i18n",
          },
          {
            text: "CI/CD",
            icon: <FaRotate />,
            href: "/projects/" + props.project_id + "/ci-cd",
          },
        ]}
      >
        {props.children}
      </AppShell>
    </>
  );
}
```

## packages/components/editorv2/uml/UMLEditor.tsx

```
/**
 * packages/components/editorv2/uml/UMLEditor.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2024 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 08.06.2024
 *
 */

import * as React from "react";
import { IUMLDiagram } from "@codeup/web/src/types/IUMLDiagram";
import AuthenticateLoadingIndicator from "../../admin/AuthenticateLoadingIndicator";
```

```
import { Button, Flex, Stack, Text } from "@chakra-ui/react";
import Editor, { useMonaco } from "@monaco-editor/react";
import { useEffect, useRef, useState } from "react";
import { editor } from "monaco-editor";
import IStandaloneCodeEditor = editor.IStandaloneCodeEditor;
import { uml_language_config, uml_language } from "@codeup/util/src/monaco/uml";
import { FaPlus } from "react-icons/fa";
import PopupManager from "@codeup/web/src/global/PopupManager";
import UMLImplEditor from "./UMLImplEditor";
import UMLSeqEditor from "./UMLSeqEditor";
import UMLEREditor from "./UMLEREditor";

export default function UMLEditor(props: {
  uml: IUMLDiagram;
  updateUML: (uml: IUMLDiagram) => void;
}) {
  if (!props.uml) {
    return (
      <>
        <AuthenticateLoadingIndicator text={"Lade UML Diagramm..."} />
      </>
    );
  }

  if (props.uml.type === "adv") {
    const monaco = useMonaco();
    const [monacoTheme, setMonacoTheme] = useState("vs-dark");
    const [monacoLanguage, setMonacoLanguage] = useState("html");

    const editorRef = useRef<IStandaloneCodeEditor>();

    React.useEffect(() => {
      if (monaco && monaco.editor) {
        monaco.editor.defineTheme("codeup-dark", {
          base: "vs-dark",
          inherit: true,
          rules: [
            {
              token: "keyword",
              foreground: "#F7DE1F",
            },
            {
              token: "string",
              foreground: "#FF6400",
            },
          ],
          colors: {
            "editor.background": "#000000",
          },
        });

        setMonacoTheme("codeup-dark");

        monaco.languages.register({
          id: "plantuml",
        });

        monaco.languages.setMonarchTokensProvider("plantuml", uml_language);
        monaco.languages.setLanguageConfiguration(
          "plantuml",
```

```
        uml_language_config
      );
    }
  }, [monaco]);
  return (
    <>
      <Editor
        defaultLanguage={"plantuml"}
        defaultValue={
          JSON.parse(props.uml.data).content || "@startuml\n\n@enduml"
        }
        onMount={(e) => {
          editorRef.current = e;
        }}
        onChange={(e) => {
          props.updateUML({
            ...props.uml,
            data: JSON.stringify({
              content: e,
            }),
          });
        }}
        theme={monacoTheme}
        options={{
          fontSize: 22,
          wrappingIndent: "indent",
          insertSpaces: true,
        }}
        height={"90vh"}
      />
    </>
  );
}

if (props.uml.type === "impl") {
  return (
    <>
      <UMLImplEditor uml={props.uml} updateUML={props.updateUML} />
    </>
  );
}

if (props.uml.type === "seq") {
  return (
    <>
      <UMLSeqEditor uml={props.uml} updateUML={props.updateUML} />
    </>
  );
}

if (props.uml.type === "er") {
  return (
    <>
      <UMLEREditor uml={props.uml} updateUML={props.updateUML} />
    </>
  );
}

return (
  <>
```

```
      <Text>Dieser Diagrammtyp wird aktuell nicht unterstützt!</Text>
    </>
  );
}
```

## packages/components/editorv2/uml/UMLEREditor.tsx

```
/**
 * packages/components/editorv2/uml/UMLEREditor.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2024 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 09.06.2024
 *
 */

import * as React from "react";
import { IUMLDiagram } from "@codeup/web/src/types/IUMLDiagram";
import { useEffect } from "react";
import {
  Box,
  Button,
  Checkbox,
  Flex,
  Grid,
  HStack,
  IconButton,
  Input,
  Stack,
} from "@chakra-ui/react";
import { FaPen, FaPlus, FaSave, FaTrash } from "react-icons/fa";
import PopupManager from "@codeup/web/src/global/PopupManager";
import { CHeading } from "../../global/CHeading";
import DividerWithTextV2 from "../../global/DividerWithTextV2";

export default function UMLEREditor(props: {
  uml: IUMLDiagram;
  updateUML: (uml: IUMLDiagram) => void;
}) {
  const [entities, setEntities] = React.useState<
    {
      name: string;
      fields: { name: string; key: boolean }[];
    }[]
  >([]);
  const [relations, setRelations] = React.useState<
    {
      name: string;
      from: string;
      to: string;
      fromRelation: string; // 1, N, M
      toRelation: string; // 1, N, M
      fields: { name: string; key: boolean }[];
    }[]
  >([]);

  const updateUML = async () => {
    let content = "@startchen\n";

    for (const e of entities) {
```

```
    content += `entity ${e.name} {\n`;
    for (const f of e.fields) {
      content += `${f.name} ${f.key ? " <<key>>" : ""}\n`;
    }
  }

  for (const r of relations) {
    content += `relationship ${r.name} {\n`;
    for (const f of r.fields) {
      content += `${f.name} ${f.key ? " <<key>>" : ""}\n`;
    }
    content += "}\n";
  }

  for (const r of relations) {
    content += `${r.name} -${r.fromRelation}- ${r.from}\n`;
    content += `${r.name} -${r.toRelation}- ${r.to}\n`;
  }

  content += "@endchen";

  props.updateUML({
    ...props.uml,
    data: JSON.stringify({ entities, relations, content }),
  });
};

useEffect(() => {
  if (props.uml.data) {
    const p = JSON.parse(props.uml.data);
    setEntities(p.entities || []);
    setRelations(p.relations || []);
  }
}, [props.uml.data]);

return (
  <>
    <Flex gap={4} direction={"column"}>
      <Stack maxW={"200px"} gap={2}>
        <Button
          variant={"brand"}
          onClick={async () => {
            await updateUML();
          }}
          w={"100%"}
          leftIcon={<FaSave />}
        >
          Speichern
        </Button>
        <Button
          variant={"brand"}
          leftIcon={<FaPlus />}
          w={"100%"}
          onClick={async () => {
            const name = await PopupManager.promptAsync({
              title: "Entität hinzufügen",
              helperText: "Bitte gib den Namen der Entität ein.",
            });

            if (!name) return;
```

```
    if (entities.find((c) => c.name === name)) return;

    setEntities([
      ...entities,
      {
        name,
        fields: [],
      },
    ]);
  }}
>
  Entität hinzufügen
</Button>
<Button
  variant={"brand"}
  leftIcon={<FaPlus />}
  w={"100%"}
  onClick={async () => {
    const name = await PopupManager.promptAsync({
      title: "Relation hinzufügen",
      helperText: "Bitte gib den Namen der Relation ein.",
    });

    if (!name) return;

    const from = await PopupManager.selectAsync({
      title: "Von Entität",
      choices: entities.map((e) => e.name),
      helperText:
        "Bitte wähle die Entität aus, von der die Relation ausgeht.",
    });

    if (!from) return;

    const to = await PopupManager.selectAsync({
      title: "Zu Entität",
      choices: entities.map((e) => e.name),
      helperText:
        "Bitte wähle die Entität aus, zu der die Relation geht.",
    });

    if (!to) return;

    if (relations.find((c) => c.from === from && c.to === to)) return;

    setRelations([
      ...relations,
      {
        name,
        from,
        to,
        fromRelation: "1",
        toRelation: "1",
        fields: [],
      },
    ]);
  }}
>
  Relation hinzufügen
</Button>
```

```
</Stack>

<Grid
  templateColumns={[
    "repeat(1, 1fr)",
    "repeat(2, 1fr)",
    "repeat(3, 1fr)",
  ]}
  gap={4}
>
  {entities.map((e, i) => (
    <>
      <Box bg={"gray.900"} rounded={"md"} p={4}>
        <CHeading size={"md"}>{e.name} (Entität)</CHeading>
        <Flex
          gap={4}
          alignItems={"center"}
          mt={4}
          mb={4}
          direction={["column", "column", "row"]}
        >
          <Button
            variant={"brand"}
            leftIcon={<FaPen />}
            onClick={async () => {
              const name = await PopupManager.promptAsync({
                title: "Entität umbenennen",
                helperText:
                  "Bitte gib den neuen Namen der Entität ein.",
              });

              if (!name) return;

              const newEntities = [...entities];
              newEntities[i].name = name;
              setEntities(newEntities);
            }}
          >
            Umbenennen
          </Button>
          <Button
            colorScheme={"red"}
            leftIcon={<FaTrash />}
            onClick={async () => {
              if (
                !(await PopupManager.confirmAsync({
                  title: "Entität löschen",
                  question:
                    "Möchtest du diese Entität wirklich löschen?",
                }))
              )
                return;
              const newEntities = [...entities];
              newEntities.splice(i, 1);
              setEntities(newEntities);
            }}
          >
            Löschen
          </Button>
        </Flex>
```

```
      <DividerWithTextV2 text={"Attribute"} />
      <Button
        leftIcon={<FaPlus />}
        variant={"brand"}
        onClick={async () => {
          const name = await PopupManager.promptAsync({
            title: "Attribut hinzufügen",
            helperText: "Bitte gib den Namen des Attributs ein.",
          });
          if (!name) return;
          const newEntities = [...entities];
          newEntities[i].fields.push({ name, key: false });
          setEntities(newEntities);
        }}
      >
        Attribut hinzufügen
      </Button>
      <Stack mt={4} mb={4} gap={4}>
        {e.fields.map((f, j) => (
          <HStack gap={2}>
            <Input
              value={f.name}
              onChange={(e) => {
                const newEntities = [...entities];
                newEntities[i].fields[j].name = e.target.value;
                setEntities(newEntities);
              }}
              placeholder={"Attributname"}
            />
            <Checkbox
              colorScheme={"brand"}
              checked={f.key}
              defaultChecked={f.key}
              onChange={(e) => {
                const newEntities = [...entities];
                newEntities[i].fields[j].key = e.target.checked;
                setEntities(newEntities);
              }}
            >
              Schlüssel
            </Checkbox>
            <IconButton
              aria-label={"Löschen"}
              icon={<FaTrash />}
              colorScheme={"red"}
              onClick={() => {
                const newEntities = [...entities];
                newEntities[i].fields.splice(j, 1);
                setEntities(newEntities);
              }}
            />
          </HStack>
        ))}
      </Stack>
    </Box>
  </>
))}
{relations.map((r, i) => (
  <>
    <Box bg={"gray.900"} rounded={"md"} p={4}>
```

```
<CHeading size={"md"}>
  {r.from} [{r.name}] {r.to} (Relation)
</CHeading>
<Flex
  gap={4}
  alignItems={"center"}
  mt={4}
  mb={4}
  direction={["column", "column", "row"]}
>
  <Button
    variant={"brand"}
    leftIcon={<FaPen />}
    onClick={async () => {
      const name = await PopupManager.promptAsync({
        title: "Relation umbenennen",
        helperText:
          "Bitte gib den neuen Namen der Relation ein.",
      });

      if (!name) return;

      const newRelations = [...relations];
      newRelations[i].from = name;
      setRelations(newRelations);
    }}
  >
    Umbenennen
  </Button>
  <Button
    colorScheme={"red"}
    leftIcon={<FaTrash />}
    onClick={async () => {
      if (
        !(await PopupManager.confirmAsync({
          title: "Relation löschen",
          question:
            "Möchtest du diese Relation wirklich löschen?",
        }))
      )
        return;
      const newRelations = [...relations];
      newRelations.splice(i, 1);
      setRelations(newRelations);
    }}
  >
    Löschen
  </Button>
</Flex>
<DividerWithTextV2 text={"Relationsart"} />
<HStack gap={4}>
  <Input
    value={r.fromRelation}
    onChange={(e) => {
      const newRelations = [...relations];
      newRelations[i].fromRelation = e.target.value;
      setRelations(newRelations);
    }}
    placeholder={"Von Relation"}
  />
```

```
  <Input
    value={r.toRelation}
    onChange={(e) => {
      const newRelations = [...relations];
      newRelations[i].toRelation = e.target.value;
      setRelations(newRelations);
    }}
    placeholder={"Zu Relation"}
  />
</HStack>
<DividerWithTextV2 text={"Attribute"} />
<Button
  leftIcon={<FaPlus />}
  variant={"brand"}
  onClick={async () => {
    const name = await PopupManager.promptAsync({
      title: "Attribut hinzufügen",
      helperText: "Bitte gib den Namen des Attributs ein.",
    });
    if (!name) return;
    const newRelations = [...relations];
    newRelations[i].fields.push({ name, key: false });
    setRelations(newRelations);
  }}
>
  Attribut hinzufügen
</Button>
<Stack mt={4} mb={4} gap={4}>
  {r.fields.map((f, j) => (
    <HStack gap={2}>
      <Input
        value={f.name}
        onChange={(e) => {
          const newRelations = [...relations];
          newRelations[i].fields[j].name = e.target.value;
          setRelations(newRelations);
        }}
        placeholder={"Attributname"}
      />
      <Checkbox
        colorScheme={"brand"}
        checked={f.key}
        defaultChecked={f.key}
        onChange={(e) => {
          const newRelations = [...relations];
          newRelations[i].fields[j].key = e.target.checked;
          setRelations(newRelations);
        }}
      >
        Schlüssel
      </Checkbox>
      <IconButton
        aria-label={"Löschen"}
        icon={<FaTrash />}
        colorScheme={"red"}
        onClick={() => {
          const newRelations = [...relations];
          newRelations[i].fields.splice(j, 1);
          setRelations(newRelations);
        }}
```

```
                  />
                </HStack>
              ))}
            </Stack>
          </Box>
        </>
      ))}
    </Grid>
  </Flex>
 </>
 );
}
```

## packages/components/editorv2/uml/UMLImplEditor.tsx

```
/**
 * packages/components/editorv2/uml/UMLImplEditor.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2024 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 08.06.2024
 *
 */

import * as React from "react";
import { IUMLDiagram } from "@codeup/web/src/types/IUMLDiagram";
import { useEffect, useState } from "react";
import {
  Accordion,
  AccordionButton,
  AccordionIcon,
  AccordionItem,
  AccordionPanel,
  Box,
  Button,
  Divider,
  Flex,
  Grid,
  HStack,
  IconButton,
  Input,
  Select,
  Stack,
  Text,
} from "@chakra-ui/react";
import { FaPen, FaPlus, FaSave, FaTrash } from "react-icons/fa";
import PopupManager from "@codeup/web/src/global/PopupManager";
import { CHeading } from "../../global/CHeading";
import DividerWithText from "../../global/DividerWithText";
import DividerWithTextV2 from "../../global/DividerWithTextV2";

export default function UMLImplEditor(props: {
  uml: IUMLDiagram;
  updateUML: (uml: IUMLDiagram) => void;
}) {
  const [classes, setClasses] = useState<
    {
      name: string;
      extends: string;
      type: "class" | "interface" | "abstract class";
```

```
    fields: {
      name: string;
      type: string;
      visibility: "public" | "private" | "protected";
    }[];
    methods: {
      name: string;
      returnType: string;
      visibility: "public" | "private" | "protected";
      args: {
        name: string;
        type: string;
      }[];
    }[];
  }[]
>([]);

useEffect(() => {
  if (props.uml.data) {
    setClasses(JSON.parse(props.uml.data).classes || []);
  }
}, [props.uml.data]);

const updateUML = async () => {
  let content = "@startuml\n";
  content += "title " + props.uml.name + "\n";

  let refList = "";

  for (const c of classes) {
    content += `${c.type} ${c.name}`;
    if (c.extends) {
      content += `<${c.extends}>`;
      refList += `${c.name} --|> ${c.extends}\n`;
    }
    content += " {\n";
    for (const f of c.fields) {
      if (classes.map((c) => c.name).includes(f.type)) {
        refList += `${c.name} --> ${f.type} : ${
          f.visibility === "private"
            ? "-"
            : f.visibility === "public"
            ? "+"
            : "#"
        } ${f.name}: ${f.type}\n`;
      } else {
        content += `${
          f.visibility === "private"
            ? "-"
            : f.visibility === "public"
            ? "+"
            : "#"
        } ${f.name}: ${f.type}\n`;
      }
    }

    for (const m of c.methods) {
      content += `${
        m.visibility === "private"
          ? "-"
```

```
          : m.visibility === "public"
          ? "+"
          : "#"
      } ${m.name}(${m.args.map((a) => `${a.name}: ${a.type}`).join(", ")}): ${
        m.returnType
      }\n`;
    }

    content += "}\n";
  }

  content += refList;

  content += "@enduml";

  props.updateUML({
    ...props.uml,
    data: JSON.stringify({ classes, content }),
  });
};

return (
  <>
    <Flex gap={4} direction={"column"}>
      <Stack maxW={"200px"} gap={2}>
        <Button
          variant={"brand"}
          onClick={async () => {
            await updateUML();
          }}
          w={"100%"}
          leftIcon={<FaSave />}
        >
          Speichern
        </Button>
        <Button
          variant={"brand"}
          leftIcon={<FaPlus />}
          w={"100%"}
          onClick={async () => {
            const name = await PopupManager.promptAsync({
              title: "Klasse hinzufügen",
              helperText: "Bitte gib den Namen der Klasse ein.",
            });

            if (!name) return;
            if (classes.find((c) => c.name === name)) return;

            setClasses([
              ...classes,
              {
                name: name,
                extends: "",
                type: "class",
                fields: [],
                methods: [],
              },
            ]);
          }}
        >
```

```
    Klasse hinzufügen
  </Button>
</Stack>
<Grid
  templateColumns={[
    "repeat(1, 1fr)",
    "repeat(2, 1fr)",
    "repeat(3, 1fr)",
  ]}
  gap={4}
>
  {classes.map((c, i) => (
    <>
      <Box bg={"gray.900"} rounded={"md"} p={4}>
        <CHeading size={"md"}>{c.name}</CHeading>
        <Flex
          gap={4}
          alignItems={"center"}
          mt={4}
          mb={4}
          direction={["column", "column", "row"]}
        >
          <Select
            value={c.type}
            onChange={(e) => {
              const newClasses = [...classes];
              newClasses[i].type = e.target.value as any;
              setClasses(newClasses);
            }}
            w={"fit-content"}
          >
            <option value={"class"}>Klasse</option>
            <option value={"interface"}>Interface</option>
            <option value={"abstract class"}>Abstrakte Klasse</option>
          </Select>

          <Button
            variant={"brand"}
            leftIcon={<FaPen />}
            onClick={async () => {
              const name = await PopupManager.promptAsync({
                title: "Klasse umbenennen",
                helperText: "Bitte gib den neuen Namen der Klasse ein.",
              });

              if (!name) return;

              const newClasses = [...classes];
              newClasses[i].name = name;
              setClasses(newClasses);
            }}
          >
            Umbenennen
          </Button>
          <Button
            colorScheme={"red"}
            leftIcon={<FaTrash />}
            onClick={async () => {
              if (
                !(await PopupManager.confirmAsync({
```

```
          title: "Klasse löschen",
          question:
            "Möchtest du diese Klasse wirklich löschen?",
        }))
      )
        return;
      const newClasses = [...classes];
      newClasses.splice(i, 1);
      setClasses(newClasses);
    }}
  >
    Löschen
  </Button>
</Flex>
<Input
  value={c.extends}
  onChange={(e) => {
    const newClasses = [...classes];
    newClasses[i].extends = e.target.value;
    setClasses(newClasses);
  }}
  placeholder={"Erweitert von"}
  mb={4}
/>
<CHeading size={"sm"}>Attribute</CHeading>
<Flex gap={4} alignItems={"center"} mt={4} mb={4}>
  <Button
    variant={"brand"}
    leftIcon={<FaPlus />}
    onClick={async () => {
      const name = await PopupManager.promptAsync({
        title: "Attribut hinzufügen",
        helperText: "Bitte gib den Namen des Attributs ein.",
      });

      if (!name) return;

      const newClasses = [...classes];
      newClasses[i].fields.push({
        name: name,
        type: "String",
        visibility: "public",
      });
      setClasses(newClasses);
    }}
  >
    Attribut hinzufügen
  </Button>
  <Button
    variant={"brand"}
    leftIcon={<FaPlus />}
    onClick={async () => {
      const name = await PopupManager.promptAsync({
        title: "Bezugsobjekt hinzufügen",
        helperText:
          "Bitte gib den Namen des Bezugsobjekts ein.",
      });

      if (!name) return;
```

```
      const c = await PopupManager.selectAsync({
        title: "Klasse auswählen",
        helperText:
          "Wähle die Klasse aus, die referenziert werden soll.",
        choices: classes.map((c) => c.name),
      });

      if (!c) return;

      const newClasses = [...classes];
      newClasses[i].fields.push({
        name: name,
        type: c,
        visibility: "public",
      });
      setClasses(newClasses);
    }}
  >
    Bezugsobjekt hinzufügen
  </Button>
</Flex>
<Stack mt={4} mb={4} gap={4}>
  {c.fields.map((f) => (
    <HStack>
      <Input
        value={f.name}
        onChange={(e) => {
          const newClasses = [...classes];
          newClasses[i].fields[
            newClasses[i].fields.indexOf(f)
          ].name = e.target.value;
          setClasses(newClasses);
        }}
        placeholder={"Name"}
      />
      <Input
        value={f.type}
        onChange={(e) => {
          const newClasses = [...classes];
          newClasses[i].fields[
            newClasses[i].fields.indexOf(f)
          ].type = e.target.value;
          setClasses(newClasses);
        }}
        placeholder={"String"}
      />
      <Select
        value={f.visibility}
        onChange={(e) => {
          const newClasses = [...classes];
          newClasses[i].fields[
            newClasses[i].fields.indexOf(f)
          ].visibility = e.target.value as any;
          setClasses(newClasses);
        }}
      >
        <option value={"public"}>public</option>
        <option value={"private"}>private</option>
        <option value={"protected"}>protected</option>
      </Select>
```

```
      <IconButton
        aria-label={"Löschen"}
        icon={<FaTrash />}
        colorScheme={"red"}
        onClick={() => {
          const newClasses = [...classes];
          newClasses[i].fields.splice(
            newClasses[i].fields.indexOf(f),
            1
          );
          setClasses(newClasses);
        }}
      />
    </HStack>
  ))}
</Stack>
<CHeading size={"sm"}>Methoden</CHeading>
<Flex gap={4} alignItems={"center"} mt={4} mb={4}>
  <Button
    variant={"brand"}
    leftIcon={<FaPlus />}
    onClick={async () => {
      const name = await PopupManager.promptAsync({
        title: "Methode hinzufügen",
        helperText: "Bitte gib den Namen der Methode ein.",
      });

      if (!name) return;

      const newClasses = [...classes];

      newClasses[i].methods.push({
        name: name,
        returnType: "void",
        visibility: "public",
        args: [],
      });

      setClasses(newClasses);
    }}
  >
    Methode hinzufügen
  </Button>
</Flex>
<Stack mt={4} mb={4} gap={4}>
  {c.methods.map((m) => (
    <>
      <DividerWithTextV2 text={m.name + "()"} />
      <HStack>
        <Input
          value={m.name}
          onChange={(e) => {
            const newClasses = [...classes];
            newClasses[i].methods[
              newClasses[i].methods.indexOf(m)
            ].name = e.target.value;
            setClasses(newClasses);
          }}
          placeholder={"Name"}
        />
```

```
<Input
  value={m.returnType}
  onChange={(e) => {
    const newClasses = [...classes];
    newClasses[i].methods[
      newClasses[i].methods.indexOf(m)
    ].returnType = e.target.value;
    setClasses(newClasses);
  }}
  placeholder={"void"}
/>
<Select
  value={m.visibility}
  onChange={(e) => {
    const newClasses = [...classes];
    newClasses[i].methods[
      newClasses[i].methods.indexOf(m)
    ].visibility = e.target.value as any;
    setClasses(newClasses);
  }}
>
  <option value={"public"}>public</option>
  <option value={"private"}>private</option>
  <option value={"protected"}>protected</option>
</Select>
<IconButton
  aria-label={"Löschen"}
  icon={<FaTrash />}
  colorScheme={"red"}
  onClick={() => {
    const newClasses = [...classes];
    newClasses[i].methods.splice(
      newClasses[i].methods.indexOf(m),
      1
    );
    setClasses(newClasses);
  }}
/>
<IconButton
  aria-label={"Argument hinzufügen"}
  icon={<FaPlus />}
  variant={"brand"}
  onClick={async () => {
    const name = await PopupManager.promptAsync({
      title: "Argument hinzufügen",
      helperText:
        "Bitte gib den Namen des Arguments ein.",
    });

    if (!name) return;

    const newClasses = [...classes];
    newClasses[i].methods[
      newClasses[i].methods.indexOf(m)
    ].args.push({
      name: name,
      type: "String",
    });
    setClasses(newClasses);
  }}
```

```
      />
    </HStack>
    <DividerWithTextV2
      text={"Argumente"}
      color={"gray.300"}
    />
    <Stack mt={4} mb={4} gap={4}>
      {m.args.map((a) => (
        <HStack>
          <Input
            value={a.name}
            onChange={(e) => {
              const newClasses = [...classes];
              newClasses[i].methods[
                newClasses[i].methods.indexOf(m)
              ].args[
                newClasses[i].methods[
                  newClasses[i].methods.indexOf(m)
                ].args.indexOf(a)
              ].name = e.target.value;
              setClasses(newClasses);
            }}
            placeholder={"Name"}
          />
          <Input
            value={a.type}
            onChange={(e) => {
              const newClasses = [...classes];
              newClasses[i].methods[
                newClasses[i].methods.indexOf(m)
              ].args[
                newClasses[i].methods[
                  newClasses[i].methods.indexOf(m)
                ].args.indexOf(a)
              ].type = e.target.value;
              setClasses(newClasses);
            }}
            placeholder={"String"}
          />
          <IconButton
            aria-label={"Löschen"}
            icon={<FaTrash />}
            colorScheme={"red"}
            onClick={() => {
              const newClasses = [...classes];
              newClasses[i].methods[
                newClasses[i].methods.indexOf(m)
              ].args.splice(
                newClasses[i].methods[
                  newClasses[i].methods.indexOf(m)
                ].args.indexOf(a),
                1
              );
              setClasses(newClasses);
            }}
          />
        </HStack>
      ))}
    </Stack>
  </>
```

```
              ))}
            </Stack>
          </Box>
        </>
      ))}
    </Grid>
  </Flex>
 </>
 );
}
```

## packages/components/editorv2/uml/UMLPreview.tsx

```
/**
 * packages/components/editorv2/uml/UMLPreview.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2024 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 08.06.2024
 *
 */

import * as React from "react";
import AuthenticateLoadingIndicator from "../../admin/AuthenticateLoadingIndicator";
import { DiagramModule } from "@brainstack/diagram";
import {
  Button,
  Flex,
  FormControl,
  FormLabel,
  Image,
  Select,
} from "@chakra-ui/react";
import Networking from "@codeup/util/dist/Networking";

export default function UMLPreview(props: { umlData: string }) {
  DiagramModule.setConfig({
    baseUrl: process.env.NEXT_PUBLIC_PLANTUML_ENDPOINT,
  });

  if (!props.umlData)
    return (
      <>
        <AuthenticateLoadingIndicator text={"Lade Vorschau..."} />
      </>
    );

  const [url, setUrl] = React.useState<string>("");
  const [theme, setTheme] = React.useState<string>("toy");

  React.useEffect(() => {
    setUrl(DiagramModule.generate_svg(props.umlData + "\n!theme " + theme));
  }, [props.umlData, theme]);

  return (
    <>
      <FormControl>
        <FormLabel>Theme</FormLabel>
        <Select value={theme} onChange={(e) => setTheme(e.target.value)}>
          <option value="black-knight">Black-Knight</option>
```

```
          <option value="spacelab">Spacelab</option>
          <option value="blueprint">Blueprint</option>
          <option value="toy">Toy</option>
          <option value="vibrant">Vibrant</option>
        </Select>
      </FormControl>
      <FormControl>
        <FormLabel>Herunterladen</FormLabel>
        <Flex gap={2} direction={["column", "column", "row"]}>
          {["png", "svg", "txt", "uml"].map((type) => (
            <>
              <Button
                variant={"brand"}
                onClick={() => {
                  const c = props.umlData + "\n!theme " + theme;
                  switch (type) {
                    case "png":
                      window.open(DiagramModule.generate_png(c));
                      break;
                    case "svg":
                      window.open(DiagramModule.generate_svg(c));
                      break;
                    case "txt":
                      window.open(DiagramModule.generate_txt(c));
                      break;
                    case "uml":
                      Networking.download("data.uml", c, "text/plain");
                      break;
                  }
                }}
              >
                .{type}
              </Button>
            </>
          ))}
        </Flex>
      </FormControl>
      <Image
        src={url}
        width={"100%"}
        maxW={"600px"}
        bg={"#FFF"}
        mt={4}
        rounded={"xl"}
      />
    </>
  );
}
```

## packages/components/editorv2/uml/UMLSeqEditor.tsx

```
/**
 * packages/components/editorv2/uml/UMLSeqEditor.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2024 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 09.06.2024
 *
 */
```

```
import * as React from "react";
import { IUMLDiagram } from "@codeup/web/src/types/IUMLDiagram";
import { useEffect } from "react";
import {
  Box,
  Button,
  Grid,
  HStack,
  IconButton,
  Input,
  Stack,
} from "@chakra-ui/react";
import { CHeading } from "../../global/CHeading";
import { FaPlus, FaSave, FaTrash } from "react-icons/fa";

export default function UMLSeqEditor(props: {
  uml: IUMLDiagram;
  updateUML: (uml: IUMLDiagram) => void;
}) {
  const [seq, setSeq] = React.useState<
    {
      from: string;
      to: string;
      message: string;
    }[]
  >([]);

  useEffect(() => {
    if (props.uml.data) {
      setSeq(JSON.parse(props.uml.data).seq || []);
    }
  }, [props.uml.data]);

  const updateUML = async () => {
    let content = "@startuml\n";
    content += "title " + props.uml.name + "\n";

    for (const s of seq) {
      content += `${s.from} -> ${s.to}: ${s.message}\n`;
    }

    content += "@enduml";

    props.updateUML({
      ...props.uml,
      data: JSON.stringify({ seq, content }),
    });
  };

  return (
    <>
      <Grid
        templateColumns={["repeat(1, 1fr)", "repeat(2, 1fr)", "repeat(3, 1fr)"]}
        gap={4}
      >
        <Box bg={"gray.900"} rounded={"md"} p={4}>
          <CHeading size={"lg"}>Sequenz</CHeading>

          <Stack maxW={"200px"} gap={2}>
            <Button
```

```
    variant={"brand"}
    mt={4}
    leftIcon={<FaSave />}
    onClick={async () => {
      await updateUML();
    }}
  >
    Speichern
  </Button>
  <Button
    variant={"brand"}
    mt={4}
    leftIcon={<FaPlus />}
    onClick={() =>
      setSeq([...seq, { from: "", to: "", message: "" }])
    }
  >
    Schritt Hinzufügen
  </Button>
</Stack>

<Stack gap={4} mt={4}>
  {seq.map((s, i) => (
    <>
      <HStack>
        <Input
          placeholder={"Von"}
          value={s.from}
          onChange={(e) => {
            seq[i].from = e.target.value;
            setSeq([...seq]);
          }}
        />
        <Input
          placeholder={"Nach"}
          value={s.to}
          onChange={(e) => {
            seq[i].to = e.target.value;
            setSeq([...seq]);
          }}
        />
        <Input
          placeholder={"Nachricht"}
          value={s.message}
          onChange={(e) => {
            seq[i].message = e.target.value;
            setSeq([...seq]);
          }}
        />
        <IconButton
          aria-label={"Löschen"}
          colorScheme={"red"}
          icon={<FaTrash />}
          onClick={() => {
            seq.splice(i, 1);
            setSeq([...seq]);
          }}
        />
      </HStack>
    </>
```

```
          ))}
        </Stack>
      </Box>
    </Grid>
  </>
 );
}
```

## packages/components/emails/DeleteUser.tsx

```
/**
 * packages/components/emails/DeleteUser.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 26.04.23
 *
 */

import {
  Body,
  Button,
  Column,
  Container,
  Head,
  Heading,
  Hr,
  Html,
  Img,
  Link,
  Preview,
  Row,
  Section,
  Tailwind,
  Text,
} from "@react-email/components";
import * as React from "react";

const baseUrl = "https://codeup.space";

export default function DeleteUserEmail(props: { firstName: string }) {
  return (
    <Html>
      <Head />
      <Preview>Dein Konto wurde erfolgreich gelöscht!</Preview>
      <Tailwind>
        <Body className="bg-white my-auto mx-auto font-sans">
          <Container className="border border-solid border-[#eaeaea] rounded my-[40px] mx-auto p-
[20px] w-[465px]">
            <Section className="mt-[32px]">
              <Img
                src={`${baseUrl}/codeup.svg`}
                alt="CodeUp"
                width={60}
                height={57}
                className="my-0 mx-auto max-w-full max-h-full"
              />
            </Section>
            <Heading className="text-black text-[24px] font-normal text-center p-0 my-[30px] mx-0">
              Löschung <strong>erfolgreich</strong>
```

```
          </Heading>
          <Text className="text-black text-[14px] leading-[24px]">
            Hallo, {props.firstName}!
          </Text>
          <Text className="text-black text-[14px] leading-[24px]">
            Dein Konto wurde erfolgreich gelöscht.
          </Text>
          <Text className="text-black text-[14px] leading-[24px]">
            Liebe Grüße aus Hattingen,
            <br />
            Ben von CodeUp
          </Text>
        </Container>
      </Body>
    </Tailwind>
  </Html>
 );
}
```

## packages/components/emails/Login.tsx

```
/**
 * packages/components/emails/Login.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 26.04.23
 *
 */

import {
  Body,
  Button,
  Column,
  Container,
  Head,
  Heading,
  Hr,
  Html,
  Img,
  Link,
  Preview,
  Row,
  Section,
  Tailwind,
  Text,
} from "@react-email/components";
import * as React from "react";

const baseUrl = "https://codeup.space";

export default function LoginEmail(props: { firstName: string }) {
  return (
    <Html>
      <Head />
      <Preview>Du hast dich erfolgreich bei CodeUp angemeldet!</Preview>
      <Tailwind>
        <Body className="bg-white my-auto mx-auto font-sans">
          <Container className="border border-solid border-[#eaeaea] rounded my-[40px] mx-auto p-
[20px] w-[465px]">
```

```
          <Section className="mt-[32px]">
            <Img
              src={`${baseUrl}/codeup.svg`}
              alt="CodeUp"
              width={60}
              height={57}
              className="my-0 mx-auto max-w-full max-h-full"
            />
          </Section>
          <Heading className="text-black text-[24px] font-normal text-center p-0 my-[30px] mx-0">
            Anmeldung <strong>erfolgreich</strong>
          </Heading>
          <Text className="text-black text-[14px] leading-[24px]">
            Hallo, {props.firstName}!
          </Text>
          <Text className="text-black text-[14px] leading-[24px]">
            Du hast dich erfolgreich bei CodeUp angemeldet. Solltest du das
            nicht gewesen sein, melde dich bitte bei uns unter&amp;nbsp;
            <Link
              className="text-[#0070f3] hover:underline"
              href="mailto:hello@ben-siebert.de"
            >
              hello@ben-siebert.de
            </Link>
          </Text>
          <Text className="text-black text-[14px] leading-[24px]">
            Liebe Grüße aus Hattingen,
            <br />
            Ben von CodeUp
          </Text>
        </Container>
      </Body>
    </Tailwind>
  </Html>
 );
}
```

## packages/components/emails/Newsletter.tsx

```
/**
 * packages/components/emails/Newsletter.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 26.04.23
 *
 */

import {
  Body,
  Button,
  Column,
  Container,
  Head,
  Heading,
  Hr,
  Html,
  Img,
  Link,
  Preview,
```

```
  Row,
  Section,
  Tailwind,
  Text,
} from "@react-email/components";
import * as React from "react";

const baseUrl = "https://codeup.space";

export default function NewsletterEmail(props: { firstName: string; content: string }) {
  return (
    <Html>
      <Head />
      <Preview>Neuer Newsletter von CodeUp</Preview>
      <Tailwind>
        <Body className="bg-white my-auto mx-auto font-sans">
          <Container className="border border-solid border-[#eaeaea] rounded my-[40px] mx-auto p-
[20px] w-[465px]">
            <Section className="mt-[32px]">
              <Img
                src={`${baseUrl}/codeup.svg`}
                alt="CodeUp"
                width={60}
                height={57}
                className="my-0 mx-auto max-w-full max-h-full"
              />
            </Section>
            <Heading className="text-black text-[24px] font-normal text-center p-0 my-[30px] mx-0">
              <strong>Newsletter</strong>
            </Heading>
            <Text className="text-black text-[14px] leading-[24px]">
              Hallo, {props.firstName}!
            </Text>
            <Text className="text-black text-[14px] leading-[24px]">
              {props.content}
            </Text>
            <Text className="text-black text-[14px] leading-[24px]">
              Liebe Grüße aus Hattingen,
              <br />
              Ben von CodeUp
            </Text>
          </Container>
        </Body>
      </Tailwind>
    </Html>
  );
}
```

## packages/components/emails/PrivateMessage.tsx

```
/**
 * packages/components/emails/PrivateMessage.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 26.04.23
 *
 */

import {
```

```
  Body,
  Button,
  Column,
  Container,
  Head,
  Heading,
  Hr,
  Html,
  Img,
  Link,
  Preview,
  Row,
  Section,
  Tailwind,
  Text,
} from "@react-email/components";
import * as React from "react";

const baseUrl = "https://codeup.space";

export default function PrivateMessageEmail(props: { firstName: string; content: string }) {
  return (
    <Html>
      <Head />
      <Preview>Nachricht von CodeUp</Preview>
      <Tailwind>
        <Body className="bg-white my-auto mx-auto font-sans">
          <Container className="border border-solid border-[#eaeaea] rounded my-[40px] mx-auto p-
[20px] w-[465px]">
            <Section className="mt-[32px]">
              <Img
                src={`${baseUrl}/codeup.svg`}
                alt="CodeUp"
                width={60}
                height={57}
                className="my-0 mx-auto max-w-full max-h-full"
              />
            </Section>
            <Heading className="text-black text-[24px] font-normal text-center p-0 my-[30px] mx-0">
              Nachricht von <strong>CodeUp</strong>
            </Heading>
            <Text className="text-black text-[14px] leading-[24px]">
              Hallo, {props.firstName}!
            </Text>
            <Text className="text-black text-[14px] leading-[24px]">
              {props.content}
            </Text>
            <Text className="text-black text-[14px] leading-[24px]">
              Liebe Grüße aus Hattingen,
              <br />
              Ben von CodeUp
            </Text>
          </Container>
        </Body>
      </Tailwind>
    </Html>
  );
}
```

## packages/components/emails/Register.tsx

```
/**
 * packages/components/emails/Register.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 26.04.23
 *
 */

import {
  Body,
  Button,
  Column,
  Container,
  Head,
  Heading,
  Hr,
  Html,
  Img,
  Link,
  Preview,
  Row,
  Section,
  Tailwind,
  Text,
} from "@react-email/components";
import * as React from "react";

const baseUrl = "https://codeup.space";

export default function RegisterEmail(props: { firstName: string; link: string }) {
  return (
    <Html>
      <Head />
      <Preview>Willkommen bei CodeUp!</Preview>
      <Tailwind>
        <Body className="bg-white my-auto mx-auto font-sans">
          <Container className="border border-solid border-[#eaeaea] rounded my-[40px] mx-auto p-
[20px] w-[465px]">
            <Section className="mt-[32px]">
              <Img
                src={`${baseUrl}/codeup.svg`}
                alt="CodeUp"
                width={60}
                height={57}
                className="my-0 mx-auto max-w-full max-h-full"
              />
            </Section>
            <Heading className="text-black text-[24px] font-normal text-center p-0 my-[30px] mx-0">
              Registrierung <strong>abschließen</strong>
            </Heading>
            <Text className="text-black text-[14px] leading-[24px]">
              Hallo, {props.firstName}!
            </Text>
            <Text className="text-black text-[14px] leading-[24px]">
              Vielen Dank für deine Registrierung bei CodeUp. Wir freuen uns,
              dich bei uns begrüßen zu dürfen.
            </Text>
            <Text className="text-black text-[14px] leading-[24px]">
```

```
            Klicke auf den Knopf unten, um deine Registrierung abzuschließen.
          </Text>
          <Section className="text-center mt-[32px] mb-[32px]">
            <Button
              pX={20}
              pY={12}
              className="bg-[#000000] rounded text-white text-[12px] font-semibold no-underline text-
center"
              href={props.link}
            >
              Abschließen
            </Button>
          </Section>

          <Text className="text-black text-[14px] leading-[24px]">
            Sollte der Knopf nicht funktionieren, kopiere bitte folgenden Link
            in deinen Browser:
            <br />
            <Link
              className="text-[#0070f3] hover:underline"
              href={props.link}
            >
              {props.link}
            </Link>
          </Text>

          <Text className="text-black text-[14px] leading-[24px]">
            Liebe Grüße aus Hattingen,
            <br />
            Ben von CodeUp
          </Text>
        </Container>
      </Body>
    </Tailwind>
  </Html>
 );
}
```

## packages/components/emails/Reset.tsx

```
/**
 * packages/components/emails/Reset.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 26.04.23
 *
 */

import {
  Body,
  Button,
  Column,
  Container,
  Head,
  Heading,
  Hr,
  Html,
  Img,
  Link,
```

```
  Preview,
  Row,
  Section,
  Tailwind,
  Text,
} from "@react-email/components";
import * as React from "react";

const baseUrl = "https://codeup.space";

export default function ResetEmail(props: { firstName: string; link: string }) {
  return (
    <Html>
      <Head />
      <Preview>Passwort zurücksetzen</Preview>
      <Tailwind>
        <Body className="bg-white my-auto mx-auto font-sans">
          <Container className="border border-solid border-[#eaeaea] rounded my-[40px] mx-auto p-
[20px] w-[465px]">
            <Section className="mt-[32px]">
              <Img
                src={`${baseUrl}/codeup.svg`}
                alt="CodeUp"
                width={60}
                height={57}
                className="my-0 mx-auto max-w-full max-h-full"
              />
            </Section>
            <Heading className="text-black text-[24px] font-normal text-center p-0 my-[30px] mx-0">
              Passwort <strong>zurücksetzen</strong>
            </Heading>
            <Text className="text-black text-[14px] leading-[24px]">
              Hallo, {props.firstName}!
            </Text>
            <Text className="text-black text-[14px] leading-[24px]">
              Klicke auf den Knopf unten, um dein Passwort zurückzusetzen.
            </Text>
            <Section className="text-center mt-[32px] mb-[32px]">
              <Button
                pX={20}
                pY={12}
                className="bg-[#000000] rounded text-white text-[12px] font-semibold no-underline text-
center"
                href={props.link}
              >
                Abschließen
              </Button>
            </Section>

            <Text className="text-black text-[14px] leading-[24px]">
              Sollte der Knopf nicht funktionieren, kopiere bitte folgenden Link
              in deinen Browser:
              <br />
              <Link
                className="text-[#0070f3] hover:underline"
                href={props.link}
              >
                {props.link}
              </Link>
            </Text>
```

```
            <Text className="text-black text-[14px] leading-[24px]">
              Liebe Grüße aus Hattingen,
              <br />
              Ben von CodeUp
            </Text>
          </Container>
        </Body>
      </Tailwind>
    </Html>
  );
}
```

## packages/components/emails/UpdateEmail.tsx

```
/**
 * packages/components/emails/UpdateEmail.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 26.04.23
 *
 */

import {
  Body,
  Button,
  Column,
  Container,
  Head,
  Heading,
  Hr,
  Html,
  Img,
  Link,
  Preview,
  Row,
  Section,
  Tailwind,
  Text,
} from "@react-email/components";
import * as React from "react";

const baseUrl = "https://codeup.space";

export default function UpdateEmail(props: { firstName: string; link: string }) {
  return (
    <Html>
      <Head />
      <Preview>Bestätige das Ändern deiner E-Mail Adresse</Preview>
      <Tailwind>
        <Body className="bg-white my-auto mx-auto font-sans">
          <Container className="border border-solid border-[#eaeaea] rounded my-[40px] mx-auto p-
[20px] w-[465px]">
            <Section className="mt-[32px]">
              <Img
                src={`${baseUrl}/codeup.svg`}
                alt="CodeUp"
                width={60}
                height={57}
```

```
            className="my-0 mx-auto max-w-full max-h-full"
          />
        </Section>
        <Heading className="text-black text-[24px] font-normal text-center p-0 my-[30px] mx-0">
          E-Mail Adresse <strong>ändern</strong>
        </Heading>
        <Text className="text-black text-[14px] leading-[24px]">
          Hallo, {props.firstName}!
        </Text>
        <Text className="text-black text-[14px] leading-[24px]">
          Klicke auf den folgenden Knopf, um das Ändern deiner E-Mail
          Adresse zu bestätigen.
        </Text>
        <Section className="text-center mt-[32px] mb-[32px]">
          <Button
            pX={20}
            pY={12}
            className="bg-[#000000] rounded text-white text-[12px] font-semibold no-underline text-
center"
            href={props.link}
          >
            Abschließen
          </Button>
        </Section>

        <Text className="text-black text-[14px] leading-[24px]">
          Sollte der Knopf nicht funktionieren, kopiere bitte folgenden Link
          in deinen Browser:
          <br />
          <Link
            className="text-[#0070f3] hover:underline"
            href={props.link}
          >
            {props.link}
          </Link>
        </Text>
        <Text className="text-black text-[14px] leading-[24px]">
          Liebe Grüße aus Hattingen,
          <br />
          Ben von CodeUp
        </Text>
      </Container>
    </Body>
  </Tailwind>
 </Html>
 );
}
```

## packages/components/explore/CourseCard.tsx

```
/**
 * src/components/explore/CourseCard.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 21.06.2022
 *
 */

import * as React from "react";
```

```
import { Box, Button, Flex, Heading, Image, Text } from "@chakra-ui/react";
import { Course } from "@codeup/util/src/courses/Course";
import { useRouter } from "next/router";

export default function CourseCard(props: { course: Course }) {
  const router = useRouter();

  return (
    <>
      <Box shadow={"xl"} rounded={"xl"} bg={"gray.900"}>
        <Flex direction={["column", "row"]}>
          <Image
            src={props.course.splashImage}
            alt={props.course.displayName}
            w={"full"}
            h={64}
            flex={"45%"}
          />
          <Box flex={"65%"} p={"2rem"}>
            <Flex direction={"row"}>
              <Flex direction={"column"} flex={"100%"}>
                <Heading>{props.course.displayName}</Heading>
                <Text color={"gray.500"}>{props.course.description}</Text>
                <Flex mt={"1.2rem"}>
                  <Button
                    colorScheme={props.course.colorScheme}
                    onClick={() => {
                      router.push("/course/" + props.course.name);
                    }}
                  >
                    Kurs öffnen
                  </Button>
                </Flex>
              </Flex>
              <Box flex={[0, "100%"]}></Box>
            </Flex>
          </Box>
        </Flex>
      </Box>
    </>
  );
}
```

## packages/components/extra/DataTable.tsx

```
/**
 * packages/components/extra/DataTable.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 08.10.22
 *
 */
import React from "react";

import { useEffect, useState } from "react";
import {
  FaCheck,
  FaEdit,
  FaEllipsisH,
```

```
  FaEllipsisV,
  FaEye,
  FaFilter,
  FaHome,
  FaPlus,
  FaSort,
  FaTrash,
  FaUser,
} from "react-icons/fa";
import {
  Alert,
  AlertDescription,
  AlertIcon,
  AlertTitle,
  Avatar,
  Box,
  Button,
  ButtonGroup,
  Center,
  CircularProgress,
  Flex,
  Grid,
  Heading,
  HStack,
  Icon,
  IconButton,
  Input,
  InputGroup,
  InputLeftElement,
  Menu,
  MenuButton,
  MenuItem,
  MenuList,
  Progress,
  Spinner,
  Table,
  TableCaption,
  TableContainer,
  Tbody,
  Td,
  Text,
  Tfoot,
  Th,
  Thead,
  Tr,
  useDisclosure,
  useToast,
} from "@chakra-ui/react";
import { FiRotateCw, FiSearch } from "react-icons/fi";

export default function DataTable(props: {
  reload: () => void;
  columns: string[];
  data: any[][];
  filterOptions: string[];
  sortOptions: string[];
  filter: (query: string, filter: string, sort: string) => any[][];
}) {
  const [query, setQuery] = useState("");
```

```
const [filter, setFilter] = useState(props.filterOptions[0]);

const [sort, setSort] = useState(props.sortOptions[0]);

const [loading, setLoading] = useState(false);

return (
  <>
    <Box w={"100%"} mb={"2rem"}>
      <Flex
        w={"100%"}
        direction={"row"}
        alignItems={"center"}
        justifyContent={"space-between"}
        gap={2}
      >
        <InputGroup>
          <InputLeftElement pointerEvents="none">
            <Icon as={FiSearch} />
          </InputLeftElement>
          <Input
            type="text"
            placeholder="Suche..."
            value={query}
            onChange={(e) => setQuery(e.target.value)}
          />
        </InputGroup>
        <Menu>
          <IconButton
            aria-label={"refresh"}
            icon={<FiRotateCw />}
            onClick={async () => {
              setLoading(true);
              await props.reload();
              setLoading(false);
            }}
          />
          <MenuButton
            as={IconButton}
            icon={<FaFilter />}
            aria-label={"Filter"}
          />
          <MenuList>
            {props.filterOptions.map((option) => (
              <MenuItem
                key={option}
                onClick={() => {
                  setFilter(option);
                }}
                icon={filter === option ? <FaCheck /> : <></>}
              >
                {option}
              </MenuItem>
            ))}
          </MenuList>
        </Menu>
        <Menu>
          <MenuButton
            as={IconButton}
            icon={<FaSort />}
```

```
        aria-label={"Sortieren"}
      />
      <MenuList>
        {props.sortOptions.map((option) => (
          <MenuItem
            key={option}
            onClick={() => {
              setSort(option);
            }}
            icon={sort === option ? <FaCheck /> : <></>}
          >
            {option}
          </MenuItem>
        ))}
      </MenuList>
    </Menu>
  </Flex>
</Box>
<TableContainer>
  <Table variant="simple">
    <Thead>
      <Tr>
        {props.columns.map((c) => {
          return (
            <>
              <Th>{c}</Th>
            </>
          );
        })}
      </Tr>
    </Thead>
    <Tbody>
      {!loading ? (
        <>
          {props.filter(query, filter, sort).map((d) => {
            return (
              <>
                <Tr
                  onClick={(e: any) => {
                    d[0](e);
                  }}
                  _hover={{
                    backgroundColor: "gray.600",
                  }}
                  cursor={"pointer"}
                >
                  {d.slice(1).map((c) => {
                    return (
                      <>
                        <Td>{c}</Td>
                      </>
                    );
                  })}
                </Tr>
              </>
            );
          })}
        </>
      ) : (
        <>
```

```
              <Flex gap={"2rem"} alignItems={"center"}>
                <Spinner size={"lg"} />
                Lade...
              </Flex>
            </>
          )}
        </Tbody>
        <TableCaption>
          {props.filter(query, filter, sort).length} Einträge gefunden
        </TableCaption>
      </Table>
    </TableContainer>
  </>
 );
}
```

## packages/components/forum/ChatCardV2.tsx

```
/**
 * packages/components/forum/ChatCardV2.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 24.09.2022
 *
 */

import * as React from "react";
import {
  Avatar,
  Button,
  chakra,
  Flex,
  Heading,
  HStack,
  IconButton,
  Text,
  useToast,
} from "@chakra-ui/react";
import { FaCheckCircle, FaTrash } from "react-icons/fa";
import REST from "@codeup/rest";
import { useRouter } from "next/router";

export default function ChatCardV2(props: {
  chat: {
    id: string;
    username: string;
    displayName: string;
    verified: boolean;
  };
  setActiveChat: (i: string) => void;
  reloadChats: () => void;
}) {
  const toast = useToast();

  const router = useRouter();

  return (
    <>
      <Flex
```

```
        direction={"row"}
        alignItems={"center"}
        gap={"1rem"}
        p={8}
        key={props.chat.id}
        cursor={"pointer"}
        onClick={() => {
          // check if mobile
          if (window.innerWidth < 768) {
            router.push("/forum/messages/" + props.chat.id);
          } else {
            router.push("/forum/messages/?chat=" + props.chat.id);
          }
        }}
      >
        <Avatar
          size={"md"}
          src={"/api/account/" + props.chat.username + "/picture"}
        />
        <Flex direction={"column"} flex={"100%"}>
          <HStack>
            <Heading size={"md"}>{props.chat.displayName}</Heading>
            {props.chat.verified ? (
              <chakra.span color={"brand.500"} fontSize={"2xl"}>
                <FaCheckCircle />
              </chakra.span>
            ) : null}
          </HStack>
          <Text color={"gray"}>@{props.chat.username}</Text>
        </Flex>
        <Flex flex={"100%"} alignItems={"center"} justifyContent={"flex-end"}>
          <IconButton
            aria-label={"delete_chat"}
            icon={<FaTrash />}
            onClick={() => {
              REST.Forum.deleteChatV2({
                token: localStorage.getItem("token") as string,
                id: props.chat.id,
              }).then((res) => {
                if (res.status === 200) {
                  props.reloadChats();
                  props.setActiveChat("");
                } else {
                  toast({
                    title: "Fehler",
                    description: "Der Chat konnte nicht gelöscht werden.",
                    status: "error",
                    duration: 5000,
                    isClosable: true,
                    position: "top",
                  });
                }
              });
            }}
          />
        </Flex>
      </Flex>
    </>
  );
}
```

## packages/components/forum/ChatMessageV2.tsx

```
/**
 * packages/components/forum/ChatMessageV2.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 24.09.2022
 *
 */

import * as React from "react";

export default function ChatMessageV2(props: {
  content: string;
  fromSelf: boolean;
  createdAt: string;
  displayName: string;
}) {
  return <></>;
}
```

## packages/components/forum/ChatViewV2.tsx

```
/**
 * packages/components/forum/ChatView.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 24.09.2022
 *
 */

import * as React from "react";
import { useEffect, useState } from "react";
import REST from "@codeup/rest";
import {
  Avatar,
  Box,
  Button,
  Flex,
  IconButton,
  Input,
  Menu,
  MenuButton,
  MenuItem,
  MenuList,
  Text,
  useToast,
} from "@chakra-ui/react";
import { FiDownload, FiFile, FiMessageSquare, FiSend } from "react-icons/fi";
import { useLoggedIn } from "@codeup/hooks";
import { FaFile, FaImage, FaPlus, FaVideo } from "react-icons/fa";

export default function ChatViewV2(props: {
  chatId: string;
  setActiveChat: (i: string) => void;
}) {
  const [messages, setMessages] = useState<
```

```
  {
    username: string;
    content: string;
    createdAt: string;
    _id: string;
  }[]
>([]);

const toast = useToast();

const reloadMessages = () => {
  REST.Forum.getChatMessagesV2({
    token: localStorage.getItem("token") as string,
    id: props.chatId,
  }).then((r) => {
    if (r.status === 200) {
      setMessages(r.payload.messages);
    } else {
      toast({
        title: "Fehler",
        description: "Beim Laden der Nachrichten ist ein Fehler aufgetreten.",
        status: "error",
        duration: 5000,
        isClosable: true,
        position: "top",
      });
      props.setActiveChat("");
    }
  });
};

useEffect(() => {
  const s = setInterval(() => {
    reloadMessages();
  }, 1000);

  return () => {
    clearInterval(s);
  };
}, [props.chatId]);

useEffect(reloadMessages, [props.chatId]);

const { userInfo, loggedIn, loaded: loadedUserInfo } = useLoggedIn();

function upload(options: {
  prefix: string;
  mimeType: string;
  fileNameAsSuffix: boolean;
}) {
  const form = document.createElement("form");
  form.method = "POST";
  form.action = "/media/upload";
  const a = document.createElement("input");
  a.type = "file";
  a.accept = options.mimeType;
  a.name = "file";

  form.appendChild(a);
```

```
  a.click();

  a.onchange = async () => {
    const formData = new FormData(form);
    const r = await fetch("/media/upload", {
      method: "POST",
      body: formData,
    });

    if (r.status === 200) {
      const json = await r.json();

      const url = await json.data.url;

      await REST.Forum.sendMessageV2({
        token: localStorage.getItem("token") as string,
        id: props.chatId,
        content: `${options.prefix}: ${url} ${
          options.fileNameAsSuffix ? a.files?.[0].name : ""
        }`,
      });
    } else {
      toast({
        title: "Fehler",
        description: "Beim Hochladen ist ein Fehler aufgetreten.",
        status: "error",
        duration: 1500,
        isClosable: true,
        position: "top",
      });
    }
  };
}

return (
  <>
    <Flex
      direction="column"
      flex={"100%"}
      overflowX="hidden"
      h={"90vh"}
      p={8}
      position={"relative"}
    >
      <Box
        maxH={"80vh"}
        minH={"80vh"}
        overflow={"auto"}
        w={"100%"}
        id={"chat_msgs"}
      >
        {messages
          .sort((a, b) => {
            return new Date(a.createdAt).getTime() <
              new Date(b.createdAt).getTime()
              ? 1
              : -1;
          })
          .map((m) => {
            return (
```

```
<Flex
  w={"100%"}
  alignItems={"center"}
  justifyContent={
    userInfo.username === m.username ? "flex-end" : "flex-start"
  }
  pr={"2rem"}
  mt={8}
  mb={8}
>
  <Box maxW={["100%", "50%"]}>
    <Flex
      w={"100%"}
      alignItems={"center"}
      justifyContent={
        userInfo.username === m.username
          ? "flex-end"
          : "flex-start"
      }
      paddingBlock={4}
      gap={"1rem"}
    >
      <Flex
        w={"100%"}
        alignItems={"center"}
        justifyContent={
          userInfo.username === m.username
            ? "flex-end"
            : "flex-start"
        }
        paddingBlock={4}
        gap={"1rem"}
        color={"gray"}
      >
        {new Date(m.createdAt).toLocaleString()}
      </Flex>
      <Avatar
        src={"/api/account/" + m.username + "/picture"}
        size={"md"}
      />
    </Flex>
    <Box
      bg={
        userInfo.username === m.username
          ? "brand.600"
          : "gray.500"
      }
      p={4}
      color={
        userInfo.username === m.username ? "black" : "white"
      }
      fontSize={"xl"}
      rounded={"xl"}
      shadow={"xl"}
    >
      {m.content.startsWith("image: ") ? (
        <>
          <img src={m.content.replace("image: ", "")} />
        </>
      ) : m.content.startsWith("video: ") ? (
```

```
              <>
                <video
                  src={m.content.replace("video: ", "")}
                  controls
                />
              </>
            ) : m.content.startsWith("file: ") ? (
              <>
                <Flex
                  alignItems={"center"}
                  justifyContent={"space-between"}
                  w={"100%"}
                  p={4}
                  rounded={"xl"}
                  bg={"gray.800"}
                  shadow={"xl"}
                  gap={4}
                >
                  <Flex alignItems={"center"} gap={"1rem"}>
                    <FiFile />
                    <Text>{m.content.split(" ")[2]}</Text>
                  </Flex>
                  <IconButton
                    as={"a"}
                    href={m.content.split(" ")[1]}
                    target={"_blank"}
                    rel={"noreferrer"}
                    colorScheme={"blue"}
                    icon={<FiDownload />}
                    aria-label={"Download"}
                  />
                </Flex>
              </>
            ) : (
              m.content
            )}
          </Box>
        </Box>
      </Flex>
    );
  })}
</Box>
<Box position={"absolute"} bottom={"2rem"} left={"5%"} w={"90%"}>
  <form
    onSubmit={async (e) => {
      e.preventDefault();

      const form = e.target as HTMLFormElement;

      const message = form.message.value;

      if (message.trim() !== "") {
        const r = await REST.Forum.sendMessageV2({
          token: localStorage.getItem("token") as string,
          id: props.chatId,
          content: message,
        });

        if (r.status === 200) {
          form.message.value = "";
```

```
        setMessages(r.payload.messages);
      } else {
        toast({
          title: "Fehler",
          description:
            "Beim Senden der Nachricht ist ein Fehler aufgetreten.",
          status: "error",
          duration: 5000,
          isClosable: true,
          position: "top",
        });
      }
    }
  }}
>
  <Flex
    w={"100%"}
    direction={"row"}
    alignItems={"center"}
    gap={"1rem"}
  >
    <Menu>
      <MenuButton as={IconButton} icon={<FaPlus />} />
      <MenuList>
        <MenuItem
          icon={<FaImage />}
          onClick={async () => {
            upload({
              fileNameAsSuffix: false,
              mimeType: "image/*",
              prefix: "image",
            });
          }}
        >
          Bild
        </MenuItem>
        <MenuItem
          icon={<FaVideo />}
          onClick={async () => {
            upload({
              fileNameAsSuffix: false,
              mimeType: "video/*",
              prefix: "video",
            });
          }}
        >
          Video
        </MenuItem>
        <MenuItem
          icon={<FaFile />}
          onClick={async () => {
            upload({
              fileNameAsSuffix: true,
              mimeType: "*",
              prefix: "file",
            });
          }}
        >
          Datei
```

```
              </MenuItem>
            </MenuList>
          </Menu>
          <Input type={"text"} placeholder={"Nachricht"} name={"message"} />
          <Button leftIcon={<FiSend />} type={"submit"} variant={"brand"}>
            Senden
          </Button>
        </Flex>
      </form>
    </Box>
  </Flex>
 </>
 );
}
```

## packages/components/forum/DeferredPostCardV2.tsx

```
/**
 * packages/components/forum/DeferedPostCardV2.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 24.09.2022
 *
 */

import * as React from "react";
import { useEffect, useState } from "react";
import { useLoggedIn } from "@codeup/hooks";
import REST from "@codeup/rest";
import PostCardV2 from "./PostCardV2";

export default function DeferredPostCardV2(props: {
  post: {
    parent: string | null;
    _id: string;
    user: string;
    username: string;
    content: string;
    likes: string[];
    createdAt: string;
    __v: 0;
  };
  debug?: boolean;
  reloadPosts?: () => void;
}) {
  const [profile, setProfile] = useState<any>(null);

  const { userInfo } = useLoggedIn();

  useEffect(() => {
    REST.Forum.getProfileV2(props.post.username).then((res) => {
      if (res.status === 200) {
        setProfile(res.payload.profile);
      } else {
        setProfile(null);
      }
    });
  }, []);
```

```
  return (
    <>
      {props.post && profile && userInfo && props.reloadPosts ? (
        <PostCardV2
          post={props.post}
          profile={profile}
          postOwner={
            userInfo.username === props.post.username ||
            userInfo.role === "admin"
          }
          reloadPosts={props.reloadPosts}
          userId={userInfo.id}
        />
      ) : null}
    </>
  );
}
```

## packages/components/forum/ForumNavigationBar.tsx

```
/**
 * src/components/community/CommunityNavigationBar.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 21.06.2022
 *
 */

import * as React from "react";
import {
  Box,
  Button,
  Divider,
  Drawer,
  DrawerBody,
  DrawerCloseButton,
  DrawerContent,
  DrawerHeader,
  DrawerOverlay,
  Flex,
  Heading,
  IconButton,
  Link as ChakraLink,
  useDisclosure,
  useMediaQuery,
} from "@chakra-ui/react";
import { GiHamburgerMenu } from "react-icons/gi";
import { useLoggedIn } from "@codeup/hooks";
import Link from "next/link";
import { useEffect, useState } from "react";
import REST from "@codeup/rest";
import { useRouter } from "next/router";
import { CHeading } from "../global/CHeading";

export default function ForumNavigationBar() {
  const [isMobile] = useMediaQuery("(max-width: 768px)");
  const { isOpen, onOpen, onClose } = useDisclosure();

  const router = useRouter();
```

```
const { loggedIn, userInfo, loaded } = useLoggedIn();

const [hasProfile, setHasProfile] = useState(false);

useEffect(() => {
  if (userInfo && userInfo.username) {
    REST.Forum.getProfileV2(userInfo.username).then((res) => {
      if (res.status === 200) {
        setHasProfile(true);
      }
    });
  }
}, [loaded, userInfo]);

const Menu = () => {
  return (
    <>
      <Button
        variant={"ghost"}
        size={"lg"}
        onClick={() => {
          router.push("/forum");
          onClose();
        }}
      >
        Home
      </Button>
      {loggedIn && hasProfile ? (
        <>
          <Button
            variant={"ghost"}
            size={"lg"}
            onClick={() => {
              router.push("/forum/" + userInfo.username);
              onClose();
            }}
          >
            Mein Profil
          </Button>
        </>
      ) : null}
      {loggedIn && hasProfile ? (
        <>
          <Button
            variant={"ghost"}
            size={"lg"}
            onClick={() => {
              router.push("/forum/messages");
              onClose();
            }}
          >
            Nachrichten
          </Button>
        </>
      ) : null}
    </>
  );
};
```

```
  return (
    <>
      <Box w={"100%"} bg={"gray.900"} padding={"1.2rem"}>
        <Flex direction={"row"} alignItems={"center"} justifyContent={"center"}>
          <CHeading size={"xl"}>
            <ChakraLink
              onClick={() => {
                router.push("/forum");
              }}
            >
              Forum
            </ChakraLink>
          </CHeading>
          <Flex justifyContent={"right"} flex={"100%"} alignItems={"right"}>
            {!isMobile ? (
              <Menu />
            ) : (
              <>
                <IconButton
                  onClick={onOpen}
                  aria-label={"Menu"}
                  icon={<GiHamburgerMenu />}
                  flex={"flex-end"}
                />
                <Drawer
                  isOpen={isOpen}
                  placement="right"
                  onClose={onClose}
                  size={"full"}
                >
                  <DrawerOverlay />
                  <DrawerContent bg={"black"}>
                    <DrawerCloseButton />
                    <DrawerHeader>CodeUp | Forum</DrawerHeader>

                    <DrawerBody>
                      <Flex w={"100%"} direction={"column"}>
                        <Menu />
                      </Flex>
                    </DrawerBody>
                  </DrawerContent>
                </Drawer>
              </>
            )}
          </Flex>
        </Flex>
      </Box>
      <Divider />
    </>
  );
}
```

## packages/components/forum/PostCardV2.tsx

```
/**
 * packages/components/forum/PostCardV2.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 24.09.2022
```

```
 *
 */

import * as React from "react";
import {
  Avatar,
  Badge,
  Box,
  Button,
  ButtonGroup,
  chakra,
  Flex,
  Heading,
  IconButton,
  Link,
  Popover,
  PopoverArrow,
  PopoverBody,
  PopoverContent,
  PopoverFooter,
  PopoverHeader,
  PopoverTrigger,
  Stack,
  Text,
  Textarea,
  useToast,
} from "@chakra-ui/react";
import {
  FaCheckCircle,
  FaClock,
  FaComment,
  FaHeart,
  FaRegHeart,
  FaTrash,
} from "react-icons/fa";
import REST from "@codeup/rest";
import { useEffect, useId } from "react";
import { useRouter } from "next/router";
import { FiDownload, FiFile } from "react-icons/fi";

export default function PostCardV2(props: {
  post: {
    _id: string;
    createdAt: string;
    content: string;
    likes: string[];
    parent: string | null;
  };
  profile: {
    username: string;
    verified: boolean;
    displayName?: string;
  };
  postOwner: boolean;
  reloadPosts: () => void;
  userId: string;
}) {
  const toast = useToast();

  const post_ta_id = useId();
```

```
const [isFullScreen, setIsFullScreen] = React.useState(false);

useEffect(() => {
  if (window.location.pathname.includes("/post/")) {
    setIsFullScreen(true);
  }
}, []);

const router = useRouter();

return (
  <Box
    key={props.post._id}
    mt={4}
    mb={4}
    position={"relative"}
    cursor={isFullScreen ? "default" : "pointer"}
    _hover={{
      bg: isFullScreen ? "black" : "gray.800",
    }}
    rounded={"xl"}
    p={2}
    onClick={
      isFullScreen
        ? undefined
        : () => router.push(`/forum/post/${props.post._id}`)
    }
  >
    <Link
      onClick={() => {
        router.push("/forum/" + props.profile.username);
      }}
    >
      <Flex direction={"row"} gap={"1rem"} alignItems={"flex-start"} mb={4}>
        <Avatar
          size={"md"}
          src={`/api/account/${props.profile.username}/picture`}
        />
        {props.profile.displayName ? (
          <>
            <Flex direction={"column"}>
              <Heading size={"md"}>{props.profile.displayName}</Heading>
              <Text color={"gray"}>@{props.profile.username}</Text>
            </Flex>
          </>
        ) : null}
        {!props.profile.displayName ? (
          <Heading size={"md"}>{props.profile.username}</Heading>
        ) : null}
        {props.profile.verified ? (
          <chakra.span
            color={"brand.500"}
            fontSize={props.profile.displayName ? "2xl" : "lg"}
          >
            <FaCheckCircle />
          </chakra.span>
        ) : null}
      </Flex>
    </Link>
```

```
<Flex alignItems={"center"} gap={".6rem"}>
  <FaClock />
  {new Date(props.post.createdAt).toLocaleString()}
</Flex>
{props.post.parent !== null ? (
  <Badge colorScheme={"pink"} mt={2}>
    Antwort
  </Badge>
) : null}

<Text mt={2}>{props.post.content}</Text>

{
  // @ts-ignore
  props.post.embeds.map((e) => {
    if (e.type === "link") {
      return (
        <>
          <Box mt={2} p={4} bg={"gray.800"} borderRadius={"md"}>
            <Stack>
              {e.site_name && <Text>{e.site_name}</Text>}
              {e.title && (
                <Heading
                  size={"md"}
                  color={"brand.500"}
                  cursor={"pointer"}
                  as={"a"}
                  href={e.url}
                  target={"_blank"}
                >
                  {e.title}
                </Heading>
              )}
              {e.description && <Text mt={2}>{e.description}</Text>}
              {e.image && <img src={e.image} alt={e.image || ""} />}
            </Stack>
          </Box>
        </>
      );
    } else if (e.type === "image") {
      return (
        <>
          <Box mt={2} p={4} bg={"gray.800"} borderRadius={"md"}>
            <img src={e.url} alt={e.url || ""} />
          </Box>
        </>
      );
    } else if (e.type === "video") {
      return (
        <>
          <Box mt={2} p={4} bg={"gray.800"} borderRadius={"md"}>
            <video src={e.url} controls />
          </Box>
        </>
      );
    } else if (e.type === "file") {
      return (
        <>
          <Flex
            alignItems={"center"}
```

```
            justifyContent={"space-between"}
            w={"100%"}
            p={4}
            rounded={"xl"}
            bg={"gray.800"}
            shadow={"xl"}
            gap={4}
          >
            <Flex alignItems={"center"} gap={"1rem"}>
              <FiFile />
              <Text>{e.url.split("/").pop()}</Text>
            </Flex>
            <IconButton
              as={"a"}
              href={e.url}
              target={"_blank"}
              rel={"noreferrer"}
              colorScheme={"blue"}
              icon={<FiDownload />}
              aria-label={"Download"}
            />
          </Flex>
        </>
      );
    }
  })
}

<ButtonGroup isAttached w={"100%"} mt={4}>
  <Button
    disabled={!isFullScreen}
    leftIcon={
      props.post.likes.includes(props.userId) ? (
        <FaHeart />
      ) : (
        <FaRegHeart />
      )
    }
    variant={"ghost"}
    flex={"100%"}
    onClick={async () => {
      const res = await REST.Forum.likePostV2({
        token: localStorage.getItem("token") as string,
        id: props.post._id,
      });

      if (res.status === 200) {
        props.reloadPosts();
      } else {
        toast({
          title: "Fehler",
          description:
            "Beim Liken des Beitrags ist ein Fehler aufgetreten.",
          status: "error",
          duration: 5000,
          isClosable: true,
          position: "top",
        });
      }
    }}
```

```
  alignItems={"center"}
>{`${props.post.likes.length} ${
  props.post.likes.length === 1 ? "Like" : "Likes"
}
`}</Button>

{props.post.parent === null ? (
  <Popover>
    <PopoverTrigger>
      <Button
        variant={"ghost"}
        flex={"100%"}
        leftIcon={<FaComment />}
        disabled={!isFullScreen}
      >
        Antworten
      </Button>
    </PopoverTrigger>
    <PopoverContent bg={"black"} textAlign={"left"}>
      <PopoverArrow bg={"black"} />
      <PopoverHeader>Antworten</PopoverHeader>
      <PopoverBody>
        <Textarea id={post_ta_id} placeholder={"Inhalt"} />
      </PopoverBody>
      <PopoverFooter>
        <Button
          variant={"brand"}
          onClick={async () => {
            const content = document.getElementById(
              post_ta_id
            ) as HTMLInputElement;

            const r = await REST.Forum.createPostV2({
              token: localStorage.getItem("token") as string,
              content: content.value,
              parent: props.post._id,
            });

            if (r.status === 200) {
              props.reloadPosts();
              content.value = "";
            } else {
              toast({
                title: "Fehler",
                description:
                  "Der Beitrag konnte nicht veröffentlicht werden",
                status: "error",
                duration: 5000,
                isClosable: true,
                position: "top",
              });
            }
          }}
        >
          Veröffentlichen
        </Button>
      </PopoverFooter>
    </PopoverContent>
  </Popover>
) : null}
```

```
      </ButtonGroup>
      {props.postOwner ? (
        <Box position={"absolute"} top={0} right={0}>
          <IconButton
            aria-label={"delete_post"}
            size={"md"}
            icon={<FaTrash />}
            onClick={async () => {
              const res = await REST.Forum.deletePostV2({
                token: localStorage.getItem("token") as string,
                id: props.post._id,
              });

              if (res.status === 200) {
                toast({
                  title: "Post gelöscht",
                  description: "Der Post wurde erfolgreich gelöscht!",
                  status: "success",
                  duration: 5000,
                  isClosable: true,
                });
                props.reloadPosts();
              } else {
                toast({
                  title: "Fehler",
                  description:
                    "Beim Löschen des Beitrags ist ein Fehler aufgetreten.",
                  status: "error",
                  duration: 5000,
                  isClosable: true,
                  position: "top",
                });
              }
            }}
          />
        </Box>
      ) : null}
    </Box>
  );
}
```

## packages/components/global/AppShell.tsx

```
/**
 * packages/components/global/AppShell.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2024 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 19.05.2024
 *
 */

import * as React from "react";
import { useRouter } from "next/router";
import {
  Box,
  Button,
  Drawer,
  DrawerBody,
  DrawerCloseButton,
```

```
  DrawerContent,
  DrawerHeader,
  DrawerOverlay,
  Flex,
  IconButton,
  Input,
  Stack,
  useDisclosure,
} from "@chakra-ui/react";
import { FaBars } from "react-icons/fa";
import { CHeading } from "./CHeading";

export default function AppShell(props: {
  children: React.ReactNode;
  title: string;
  sub_pages: {
    text: string;
    icon: any;
    href: string;
  }[];
}) {
  const router = useRouter();
  const { isOpen, onOpen, onClose } = useDisclosure();

  const [query, setQuery] = React.useState("");

  return (
    <>
      <Box
        w={"100%"}
        overflow={"hidden"}
        minH={"100vh"}
        h={"fit-content"}
        bg={"#000"}
      >
        <Box
          position={"relative"}
          top={2}
          left={2}
          w={"100%"}
          bg={"#000"}
          display={isOpen ? "none" : "initial"}
        >
          <IconButton
            aria-label={"Open Menu"}
            icon={<FaBars />}
            onClick={onOpen}
          />
        </Box>
        {props.children}
      </Box>
      <Drawer isOpen={isOpen} onClose={onClose} placement={"left"}>
        <DrawerOverlay />
        <DrawerContent bg={"#000"} color={"#fff"}>
          <DrawerCloseButton />
          <DrawerHeader>
            <CHeading size={"lg"}>{props.title}</CHeading>
          </DrawerHeader>
          <DrawerBody>
            <Stack gap={2}>
```

```
            <Input
              placeholder={"Suchen..."}
              onChange={(e) => setQuery(e.target.value)}
            />
            {props.sub_pages
              .filter((page) =>
                page.text.toLowerCase().includes(query.toLowerCase())
              )
              .sort((a, b) => a.text.localeCompare(b.text))
              .map((page) => (
                <>
                  <Button
                    variant={router.asPath === page.href ? "brand" : "ghost"}
                    leftIcon={page.icon}
                    textAlign={"left"}
                    alignItems={"center"}
                    justifyContent={"flex-start"}
                    size={"lg"}
                    onClick={() => router.push(page.href)}
                  >
                    {page.text}
                  </Button>
                </>
              ))}
          </Stack>
        </DrawerBody>
      </DrawerContent>
    </Drawer>
  </>
 );
}
```

## packages/components/global/CHeading.tsx

```
/**
 * packages/components/global/CHeading.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2024 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 09.05.2024
 *
 */

import * as React from "react";

import { Heading as ChakraHeading } from "@chakra-ui/react";

// ...rest
export function CHeading(props: {
  children: React.ReactNode | string;
  size?: string;
}) {
  return (
    <ChakraHeading
      as={"h1"}
      size={(props.size as any) || "2xl"}
      fontWeight={900}
      letterSpacing={-2}
      fontFamily={"Jetbrains Mono ExtraBold"}
      color={"brand.500"}
```

```
    >
      {props.children}
    </ChakraHeading>
  );
}
```

## packages/components/global/CommandPalette.tsx

```
/**
 * packages/components/global/CommandPalette.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 29.12.2022
 *
 */

import * as React from "react";
import {
  Box,
  Flex,
  Heading,
  Input,
  InputGroup,
  InputLeftElement,
  InputRightElement,
  ListItem,
  Text,
  UnorderedList,
  useToast,
} from "@chakra-ui/react";
import {
  FaBookOpen,
  FaCode,
  FaFolder,
  FaGlobe,
  FaLightbulb,
  FaPlus,
  FaSearch,
  FaTachometerAlt,
  FaUser,
  FaUsers,
} from "react-icons/fa";
import { TiDelete } from "react-icons/ti";
import REST from "@codeup/rest/dist";
import { useRouter } from "next/router";
import { FiMessageSquare } from "react-icons/fi";
import { useLoggedIn, useUserInfo } from "@codeup/hooks";

export default function CommandPalette() {
  const [open, setOpen] = React.useState(false);

  const inputRef = React.useRef() as React.MutableRefObject<HTMLInputElement>;
  const [value, setValue] = React.useState("");
  const { loggedIn, loaded, userInfo } = useLoggedIn();

  const [results, setResults] = React.useState<
    {
      title: string;
      subtitle?: string;
```

```
    icon: any;
    onClick: () => void;
  }[]
>([]);

const router = useRouter();
const toast = useToast();

const [currentSelection, setCurrentSelection] = React.useState(0);

React.useEffect(() => {
  window.addEventListener("codeup::open_search", (e) => {
    e.preventDefault();
    setOpen(true);
  });

  const listener = (e: KeyboardEvent) => {
    // CTRL + K or CMD + K or CTRL + SHIFT + P or CMD + SHIFT + P
    if (
      (e.ctrlKey || e.metaKey) &&
      (e.key === "k" || e.key === "p") &&
      !open &&
      inputRef.current
    ) {
      e.preventDefault();
      setOpen(true);
      inputRef.current.focus();
    } else if (e.key === "Escape" && open && inputRef.current) {
      e.preventDefault();
      hide();
    }
  };

  window.addEventListener("keydown", listener);

  return () => {
    document.removeEventListener("keydown", listener);
  };
});

React.useEffect(() => {
  if (value.length < 3 || !loggedIn) {
    return;
  }

  setCurrentSelection(0);

  if (value.startsWith("@")) {
    setResults([
      {
        title: "Zu Benutzer @" + value.substring(1) + " springen",
        icon: <FaUser />,
        onClick: () => {
          router.push("/forum/" + value.substring(1));
          hide();
        },
      },
      {
        title: "Direktnachricht an @" + value.substring(1) + " senden",
        icon: <FiMessageSquare />,
```

```
      onClick: async () => {
        const directMessage = await REST.Forum.createChatV2({
          token: localStorage.getItem("token") as string,
          user: value.substring(1),
        });

        router.push(
          "/forum/messages/?chat=" + directMessage.payload.chat._id
        );
        hide();
      },
    },
  ]);
} else if (value.startsWith("!")) {
  let page = value.substring(1).split("/")[0];
  let sub_page = value.substring(1).split("/")[1];

  switch (page.toLowerCase()) {
    case "forum":
      if (!sub_page) {
        setResults([
          {
            title: "Zum Forum springen",
            icon: <FaUsers />,
            subtitle: "!forum",
            onClick: () => {
              router.push("/forum");
              hide();
            },
          },
          {
            title: "Zu deinem Profil springen",
            subtitle: "!forum/profil",
            icon: <FaUser />,
            onClick: () => {
              router.push("/forum/" + userInfo?.username);
              hide();
            },
          },
          {
            title: "Zu den Nachrichten springen",
            icon: <FiMessageSquare />,
            subtitle: "!forum/nachrichten",
            onClick: () => {
              router.push("/forum/messages");
              hide();
            },
          },
        ]);
      } else {
        let x = sub_page.toLowerCase().trim();
        if (x === "nachrichten") {
          setResults([
            {
              title: "Zu den Nachrichten springen",
              icon: <FiMessageSquare />,
              onClick: () => {
                router.push("/forum/messages");
                hide();
              },
```

```
        },
      ]);
    } else if (x === "mein-profil" || x === "profil") {
      setResults([
        {
          title: "Zu deinem Profil springen",
          icon: <FaUser />,
          onClick: () => {
            router.push("/forum/" + userInfo?.username);
            hide();
          },
        },
      ]);
    } else {
      if (x.startsWith("@")) {
        x = x.substring(1);
      }

      setResults([
        {
          title: "Zu Benutzer @" + x + " springen",
          icon: <FaUser />,
          onClick: () => {
            router.push("/forum/" + x);
            hide();
          },
        },
        {
          title:
            "Direktnachricht an @" + value.substring(1) + " senden",
          icon: <FiMessageSquare />,
          onClick: async () => {
            const directMessage = await REST.Forum.createChatV2({
              token: localStorage.getItem("token") as string,
              user: value.substring(1),
            });

            router.push(
              "/forum/messages/?chat=" + directMessage.payload.chat._id
            );
            hide();
          },
        },
      ]);
    }
  }
  break;
case "editor":
  if (!sub_page) {
    setResults([
      {
        title: "Zum Editor springen",
        subtitle: "!editor",
        icon: <FaCode />,
        onClick: () => {
          router.push("/ide");
          hide();
        },
      },
      {
```

```
      title: "Zu deinen Projekten springen",
      subtitle: "!editor/projekte",
      icon: <FaFolder />,
      onClick: () => {
        router.push("/ide?component=open");
        hide();
      },
    },
    {
      title: "Neues Projekt erstellen",
      subtitle: "!editor/neu",
      icon: <FaPlus />,
      onClick: () => {
        router.push("/ide?component=create");
        hide();
      },
    },
  ]);
} else {
  let x = sub_page.toLowerCase().trim();
  if (x === "projekte") {
    setResults([
      {
        title: "Zu deinen Projekten springen",
        subtitle: "!editor/projekte",
        icon: <FaFolder />,
        onClick: () => {
          router.push("/ide?component=open");
          hide();
        },
      },
    ]);
  } else if (x.split(" ")[0] === "neu") {
    if (x.split(" ").length > 1) {
      const name = x.split(" ").slice(1).join(" ");

      setResults([
        {
          title: "Neues Projekt mit Namen '" + name + "' erstellen",
          subtitle: "!editor/neu " + name,
          icon: <FaPlus />,
          onClick: () => {
            REST.IDE.createProject({
              token: localStorage.getItem("token") as string,
              name: name,
            }).then((res) => {
              if (res.status !== 200) {
                toast({
                  title: "Fehler",
                  description: res.payload.error,
                  status: "error",
                  duration: 1500,
                  isClosable: true,
                  position: "bottom-right",
                });
              } else {
                router.push(
                  "/ide/project/" +
                    res.payload.project._id +
                    "#editor-wrapper-flex"
```

```
            );
            hide();
          }
        });
      },
    },
  ]);
 } else {
  setResults([
    {
      title: "Neues Projekt erstellen",
      subtitle: "!editor/neu",
      icon: <FaPlus />,
      onClick: () => {
        router.push("/ide?component=create");
        hide();
      },
    },
  ]);
 }
} else {
  REST.IDE.getProjects({
    token: localStorage.getItem("token") as string,
    minimal: true,
    type: "ALL",
    withStats: false,
  }).then((res) => {
    if (res.status === 200) {
      let projects = res.payload.projects.filter((project: any) =>
        project.name.toLowerCase().includes(x)
      );

      setResults(
        projects.map((project: any) => {
          return {
            title: project.name,
            icon: <FaFolder />,
            onClick: () => {
              router.push(
                "/ide/project/" +
                  project.id +
                  "#editor-wrapper-flex"
              );
              hide();
            },
          };
        })
      );
    } else {
      setResults([
        {
          title: "Zum Editor springen",
          subtitle: "!editor",
          icon: <FaCode />,
          onClick: () => {
            router.push("/ide");
            hide();
          },
        },
      ]);
```

```
        }
      });
    }
  }
  break;
case "projekte":
  setResults([
    {
      title: "Zu den Projekten springen",
      subtitle: "!projekte",
      icon: <FaFolder />,
      onClick: () => {
        router.push("/discovery");
        hide();
      },
    },
  ]);
  break;
case "dashboard":
  setResults([
    {
      title: "Zum Dashboard springen",
      subtitle: "!dashboard",
      icon: <FaTachometerAlt />,
      onClick: () => {
        router.push("/dashboard");
        hide();
      },
    },
  ]);
  break;
case "ideen":
  setResults([
    {
      title: "Zu den Ideen springen",
      subtitle: "!ideen",
      icon: <FaLightbulb />,
      onClick: () => {
        router.push("/ideas");
        hide();
      },
    },
  ]);
  break;
case "impressum":
  setResults([
    {
      title: "Zum Impressum springen",
      subtitle: "!impressum",
      icon: <FaBookOpen />,
      onClick: () => {
        router.push("/legal/imprint");
        hide();
      },
    },
  ]);
  break;
case "datenschutz":
  setResults([
    {
```

```
            title: "Zur Datenschutzerklärung springen",
            subtitle: "!datenschutz",
            icon: <FaBookOpen />,
            onClick: () => {
              router.push("/legal/privacy");
              hide();
            },
          },
        ]);
        break;
      default:
        const fullPage = page + (sub_page ? "/" + sub_page : "");

        setResults([
          {
            title: "Zu Seite /" + fullPage + " springen",
            icon: <FaGlobe />,
            onClick: () => {
              router.push("/" + fullPage);
              hide();
            },
          },
        ]);
        break;
    }
  } else {
    setResults([
      {
        title: "Zu Seite /" + value + " springen",
        icon: <FaGlobe />,
        onClick: () => {
          router.push("/" + value);
          hide();
        },
      },
    ]);
  }
}, [value]);

const hide = () => {
  if (inputRef.current) {
    inputRef.current.value = "";
  }
  setOpen(false);
  setValue("");
};

return (
  <>
    <Box
      position="fixed"
      top="0"
      left="0"
      width="100vw"
      height="100vh"
      backgroundColor="rgba(0, 0, 0, 0.5)"
      zIndex="999"
      display={open ? "block" : "none"}
    >
      <Box
```

```
  position="absolute"
  top="50%"
  left="50%"
  transform="translate(-50%, -50%)"
  width="50%"
  height="50%"
  borderRadius="10px"
  rounded={"xl"}
  backgroundColor={"gray.800"}
  p={"2rem"}
>
  <InputGroup>
    <InputLeftElement>
      <FaSearch />
    </InputLeftElement>
    <Input
      placeholder={"Suchen oder Befehl eingeben"}
      id={"cmd-p-i"}
      ref={inputRef}
      defaultValue={value}
      onChange={() => {
        setValue(inputRef.current.value);
      }}
      onKeyDown={(e) => {
        if (e.key === "ArrowDown") {
          e.preventDefault();
          if (currentSelection + 1 === results.length) {
            setCurrentSelection(0);
          } else {
            setCurrentSelection((c) => c + 1);
          }
        } else if (e.key === "ArrowUp") {
          e.preventDefault();
          if (currentSelection - 1 < 0) {
            setCurrentSelection(results.length - 1);
          } else {
            setCurrentSelection((c) => c - 1);
          }
        } else if (e.key === "Enter") {
          e.preventDefault();
          results[currentSelection].onClick();
        }
      }}
    />
    <InputRightElement>
      <Heading
        size={"xl"}
        cursor={"pointer"}
        onClick={() => {
          setValue("");
          inputRef.current.value = "";
        }}
      >
        <TiDelete />
      </Heading>
    </InputRightElement>
  </InputGroup>
  <Box
    display={"flex"}
    flexDirection={"column"}
```

```
  alignItems={"flex-start"}
  justifyContent={"flex-start"}
  width={"100%"}
  height={"100%"}
  overflow={"auto"}
>
  {value.length >= 3 && loggedIn ? (
    <>
      {results.map((r, i) => (
        <Box
          key={i}
          display={"flex"}
          flexDirection={"row"}
          alignItems={"center"}
          justifyContent={"flex-start"}
          width={"100%"}
          height={"auto"}
          cursor={"pointer"}
          mt={4}
          backgroundColor={
            currentSelection === i ? "gray.700" : "transparent"
          }
          onClick={() => {
            r.onClick();
          }}
          _hover={{
            backgroundColor: "gray.700",
          }}
          p={"0.5rem"}
        >
          {r.icon}
          <Box
            display={"flex"}
            flexDirection={"row"}
            alignItems={"center"}
            justifyContent={"flex-start"}
            width={"100%"}
            height={"auto"}
          >
            <Box
              display={"flex"}
              flexDirection={"row"}
              alignItems={"center"}
              justifyContent={"flex-start"}
              width={"auto"}
              height={"auto"}
              mr={"1rem"}
            >
              <Box
                display={"flex"}
                flexDirection={"column"}
                alignItems={"flex-start"}
                justifyContent={"flex-start"}
                width={"auto"}
                height={"auto"}
                ml={"0.5rem"}
              >
                <Text fontSize={"xl"}>{r.title}</Text>
                <Text fontSize={"md"} color={"gray.200"}>
                  {r.subtitle}
```

```
            </Text>
          </Box>
        </Box>
      </Box>
    </Box>
  ))}
 </>
) : (
 <>
  <Flex
   w={"100%"}
   h={"fit-content"}
   minH={"100%"}
   alignItems={"center"}
   justifyContent={"center"}
   flexDirection={"column"}
  >
   <Box>
    {loggedIn ? (
     <>
      <Heading size={"lg"}>
       Suchen oder Befehl eingeben
      </Heading>
      <Text mt={8} fontSize={"lg"} fontFamily={"monospace"}>
       Beispiele:
      </Text>
      <UnorderedList fontSize={"lg"} fontFamily={"monospace"}>
       <ListItem>
        <code>@ben.sbrt</code> - Optionen für Benutzer
        anzeigen
       </ListItem>
       <ListItem>
        <code>!forum</code> - Forum öffnen + Optionen
        anzeigen
       </ListItem>
       <ListItem>
        <code>!editor</code> - Editor öffnen + Optionen
        anzeigen
       </ListItem>
       <ListItem>
        <code>!editor/neu &amp;lt;name&amp;gt;</code> - Erstellt ein
        neues Projekt mit dem Namen &amp;lt;name&amp;gt;
       </ListItem>
       <ListItem>
        <code>!editor/&amp;lt;name&amp;gt;</code> - Öffnet das
        Projekt mit dem Namen &amp;lt;name&amp;gt;
       </ListItem>
      </UnorderedList>
     </>
    ) : (
     <>
      <Heading size={"lg"}>Schnellzugriff</Heading>
      <Text mt={8} fontSize={"lg"} fontFamily={"monospace"}>
       Bitte melde dich an, um den Schnellzugriff zu nutzen.
      </Text>
     </>
    )}
   </Box>

   <Text
```

```
                mt={8}
                fontSize={"lg"}
                fontFamily={"monospace"}
                color={"blue.300"}
                cursor={"pointer"}
                onClick={() => {
                  setOpen(false);
                }}
              >
                Schließen
              </Text>
            </Flex>
          </>
        )}
      </Box>
    </Box>
  </Box>
 </>
 );
}
```

## packages/components/global/DividerWithText.tsx

```
/**
 * packages/components/global/DividerWithText.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 03.10.22
 *
 */

import { Divider, Flex, Text } from "@chakra-ui/react";
import * as React from "react";

export default function DividerWithText(props: {
  text?: string;
  color?: string;
}) {
  return (
    <>
      <Flex align="center">
        <Divider />
        <Text padding="2" color={props.color ? props.color : "white"}>
          {props.text ? props.text.toUpperCase() : "ODER"}
        </Text>
        <Divider />
      </Flex>
    </>
  );
}
```

## packages/components/global/DividerWithTextV2.tsx

```
/**
 * packages/components/global/DividerWithTextV2.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2024 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 09.06.2024
```

```
 *
 */

import { Divider, Flex, Text } from "@chakra-ui/react";
import * as React from "react";

export default function DividerWithTextV2(props: {
  text?: string;
  color?: string;
}) {
  return (
    <>
      <Flex align="center">
        <Divider />
        <Text
          padding="2"
          color={props.color ? props.color : "brand.500"}
          fontWeight={"bold"}
        >
          {props.text ? props.text : ""}
        </Text>
        <Divider />
      </Flex>
    </>
  );
}
```

## packages/components/global/ErrorComponent.tsx

```
/**
 * packages/components/global/ErrorComponent.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 07.10.22
 *
 */

import * as React from "react";
import {
  Box,
  Button,
  Flex,
  Heading,
  Image,
  Stack,
  VStack,
} from "@chakra-ui/react";
import { useRouter } from "next/router";

export default function ErrorComponent(props: {
  title?: string;
  description: string;
  image?: string;
}) {
  const router = useRouter();
  return (
    <>
      <Flex
        w={"100%"}
```

```
        gap="2rem"
        direction={["column", "row", "row"]}
        p={8}
        pt={"4rem"}
        pb={"4rem"}
        h={"fit-content"}
        minH={"100vh"}
        bg={"#000"}
        alignItems={"center"}
        justifyContent={"center"}
      >
        <Flex gap={4} direction={"column"}>
          <Heading
            as={"h1"}
            fontSize={["4xl", "5xl", "6xl"]}
            fontWeight={900}
            letterSpacing={-2}
            fontFamily={"Jetbrains Mono ExtraBold"}
            color={"brand.500"}
            w={"100%"}
          >
            {props.title || "Fehler"}
          </Heading>
          <Heading size={"lg"} fontFamily={"Jetbrains Mono ExtraBold"}>
            {props.description}
          </Heading>
          <Button
            variant={"brand"}
            size={"lg"}
            w={"fit-content"}
            onClick={() => {
              router.push("/");
            }}
          >
            Home
          </Button>
        </Flex>
        <Image
          src={props.image ?? "/_static/vectors/error/warning.svg"}
          width={"512"}
          height={"512"}
          alt={""}
        />
      </Flex>
    </>
  );
}
```

## packages/components/global/Footer.tsx

```
/**
 * src/components/Footer.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 06.06.2022
 *
 */

import * as React from "react";
```

```
import {
  Box,
  Button,
  chakra,
  Divider,
  Flex,
  Heading,
  HStack,
  IconButton,
  Image,
  Link,
  LinkProps,
  Stack,
  Text,
  VStack,
} from "@chakra-ui/react";
import {
  FaGithub,
  FaInstagram,
  FaLinkedin,
  FaLinkedinIn,
  FaTwitter,
} from "react-icons/fa";
import Newsletter from "./Newsletter";

const Footer = () => {
  return (
    <Box w={"100%"} backgroundColor={"black"}>
      <Box p={{ base: 5, md: 8 }} maxW="7xl" marginInline="auto">
        <Stack
          spacing={{ base: 8, md: 0 }}
          justifyContent="space-between"
          direction={{ base: "column", md: "row" }}
          gap={24}
        >
          <Box maxW="300px">
            <Heading
              as="h1"
              fontSize={"4xl"}
              fontFamily="ARCADE"
              color={"brand.500"}
            >
              CODEUP
            </Heading>
            <Text mt={2} color="white" fontSize="md">
              Made with ❤ and ☕ by Ben Siebert
            </Text>
          </Box>
          <Newsletter />
          <HStack
            spacing={8}
            display={"flex"}
            justifyContent={{ sm: "space-between", md: "normal" }}
            direction={{ base: "column", md: "row" }}
          >
            <VStack spacing={4} alignItems="flex-start">
              <Text fontSize="md" fontWeight="bold">
                Über
              </Text>
              <VStack spacing={2} alignItems="flex-start" color="white">
```

```
        <CustomLink href={"/codeup/about"}>Über CodeUp</CustomLink>
        <CustomLink href={"/codeup/contact"}>Kontakt</CustomLink>
        <CustomLink href={"/codeup/blog"}>Blog</CustomLink>
        <CustomLink href={"/legal/imprint"}>Impressum</CustomLink>
        <CustomLink href={"/legal/privacy"}>Datenschutz</CustomLink>
      </VStack>
    </VStack>
    <VStack spacing={4} alignItems="flex-start">
      <Text fontSize="md" fontWeight="bold">
        Weitere Angebote
      </Text>
      <VStack spacing={2} alignItems="flex-start" color="white">
        <CustomLink href={"/forum"}>Forum</CustomLink>
        <CustomLink href={"/editor"}>Editor</CustomLink>
        <CustomLink href={"/todo"}>ToDos</CustomLink>
        <CustomLink href={"/planning"}>Flowcharts</CustomLink>
        <CustomLink href={"/certificates"}>Zertifikate</CustomLink>
      </VStack>
    </VStack>
    <VStack spacing={4} alignItems="flex-start">
      <Text
        fontSize="md"
        fontWeight="bold"
        color={"transparent"}
        userSelect={"none"}
      >
        Weitere Angebote
      </Text>
      <VStack spacing={2} alignItems="flex-start" color="white">
        <CustomLink href={"/course"}>Kurse</CustomLink>
        <CustomLink href={"/discovery"}>Discovery</CustomLink>
        <CustomLink href={"/creator/studio"}>Creator Panel</CustomLink>
        <CustomLink href={"/dashboard"}>Dashboard</CustomLink>
        <CustomLink href={"/snippets/my"}>Code Snippets</CustomLink>
      </VStack>
    </VStack>
  </HStack>
</Stack>

<Divider my={4} />

<Stack
  direction={{ base: "column", md: "row" }}
  spacing={3}
  justifyContent="space-between"
>
  <Text fontSize="md">
    Copyright &copy; {new Date().getFullYear()} 
    <Link
      href="https://ben-siebert.de"
      _hover={{ textDecoration: "underline" }}
      isExternal
    >
      Ben Siebert
    </Link>
  </Text>
  <Stack spacing={2} direction={{ base: "column", md: "row" }} gap={4}>
    <Text fontSize="md">Ausgezeichnet durch:</Text>
    <Link
      isExternal
```

```
            href={
              "https://www.jugend-forscht.de/projektdatenbank/codeup-einfacher-einstieg-in-die-
programmierung.html"
            }
          >
            <Image
              src={"/_static/images/logos/jugend-forscht.svg"}
              alt={"Jugend Forscht"}
              width={"150px"}
            />
          </Link>
          <Link
            isExternal
            href={
              "https://www.startupteens.de/challenge-2023/voting/education/codeup/"
            }
          >
            <Image
              src={"/_static/images/logos/startupteens.png"}
              alt={"STARTUPTEENS"}
              width={"150px"}
            />
          </Link>
        </Stack>
        <Stack spacing={2} direction={{ base: "column", md: "row" }}>
          <IconButton
            aria-label={"Twitter"}
            as={Link}
            target={"_blank"}
            href={"https://twitter.com/codeupspace"}
            icon={<FaTwitter />}
          />
          <IconButton
            aria-label={"Instagram"}
            as={Link}
            target={"_blank"}
            href={"https://instagram.com/codeupspace"}
            icon={<FaInstagram />}
          />
          <IconButton
            aria-label={"LinkedIn"}
            as={Link}
            target={"_blank"}
            href={"https://www.linkedin.com/in/ben-siebert-111595278/"}
            icon={<FaLinkedinIn />}
          />
        </Stack>
      </Stack>
    </Box>
  </Box>
 );
};

const CustomLink = ({ children, ...props }: LinkProps) => {
 return (
  <Link
    href="#"
    fontSize="sm"
    _hover={{ textDecoration: "underline" }}
    {...props}
```

```
    >
      {children}
    </Link>
  );
};

export default Footer;
```

## packages/components/global/GridLayout.tsx

```
/**
 * packages/components/global/GridLayout.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 24.03.2023
 *
 */

import * as React from "react";
import { ReactNode } from "react";
import {
  Box,
  Button,
  ButtonGroup,
  Flex,
  FormControl,
  Grid,
  GridItem,
  Heading,
  Input,
  InputGroup,
  InputLeftElement,
  Stack,
  Text,
} from "@chakra-ui/react";
import PopupManager from "@codeup/web/src/global/PopupManager";
import {
  FaClock,
  FaEye,
  FaPen,
  FaPlus,
  FaSearch,
  FaTrash,
  FaUsers,
} from "react-icons/fa";

export default function GridLayout(props: {
  search?: {
    allow: boolean;
    searchFields: boolean;
  };
  specialEntries?: {
    icon: ReactNode;
    onClick: () => void;
  }[];
  entries?: {
    title: string;
    fields: {
      icon?: ReactNode;
```

```
    text: string;
  }[];
  buttonRows: {
    buttons: {
      text: string;
      onClick: () => void;
      colorScheme: string;
      icon?: ReactNode;
      disabled?: boolean;
    }[];
  }[];
}[];
}) {
  const [query, setQuery] = React.useState<string>("");

  return (
    <>
      {props.search?.allow && (
        <Flex justifyContent={"center"} mt={8}>
          <FormControl w={["75%", "25%"]}>
            <InputGroup>
              <InputLeftElement>
                <FaSearch />
              </InputLeftElement>
              <Input
                placeholder={"Suche"}
                value={query}
                onChange={(e) => {
                  setQuery(e.target.value);
                }}
              />
            </InputGroup>
          </FormControl>
        </Flex>
      )}
      <Grid
        templateColumns={["repeat(1, 1fr)", "repeat(2, 1fr)", "repeat(3, 1fr)"]}
        gap={6}
        p={4}
        mt={"2rem"}
      >
        {props.specialEntries?.map((e, i) => {
          return (
            <>
              <GridItem
                key={"new"}
                bg={"gray.900"}
                p={"1rem"}
                borderRadius={"0.5rem"}
                _hover={{
                  cursor: "pointer",
                  borderColor: "blue.300",
                  borderWidth: "1px",
                  borderStyle: "solid",
                }}
                onClick={e.onClick}
              >
                <Heading
                  fontSize={"8xl"}
                  textAlign={"center"}
```

```
          color={"brand.500"}
          alignItems={"center"}
          justifyContent={"center"}
          display={"flex"}
          h={"100%"}
        >
          {e.icon}
        </Heading>
      </GridItem>
    </>
  );
})}
{props.entries
  ?.filter((e) => {
    if (query === "") {
      return true;
    } else {
      if (e.title.toLowerCase().includes(query.toLowerCase())) {
        return true;
      }
      if (props.search?.searchFields === false) {
        return false;
      }
      let found = false;
      e.fields.forEach((f) => {
        if (f.text.toLowerCase().includes(query.toLowerCase())) {
          found = true;
        }
      });
      return found;
    }
  })
  .map((e, i) => {
    return (
      <GridItem
        key={i}
        bg={"gray.900"}
        p={"1rem"}
        borderRadius={"0.5rem"}
      >
        <Stack spacing={4}>
          <Heading fontSize={"2xl"}>{e.title}</Heading>
          {e.fields.map((f, j) => (
            <>
              <Text
                fontSize={"xl"}
                display={"flex"}
                alignItems={"center"}
                gap={3}
              >
                <Box>{f.icon}</Box>
                {f.text}
              </Text>
            </>
          ))}

          {e.buttonRows.map((b, j) => {
            return (
              <ButtonGroup>
                {b.buttons.map((c, k) => {
```

```
                    return (
                      <Button
                        colorScheme={c.colorScheme}
                        leftIcon={<>{c.icon}</>}
                        onClick={c.onClick}
                        w={"100%"}
                        disabled={c.disabled}
                      >
                        {c.text}
                      </Button>
                    );
                  })}
                </ButtonGroup>
              );
            })}
          </Stack>
        </GridItem>
      );
    })}
  </Grid>
 </>
 );
}
```

## packages/components/global/GridLayoutV2.tsx

```
/**
 * packages/components/global/GridLayoutV2.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2024 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 09.05.2024
 *
 */

import * as React from "react";
import {
  Box,
  Button,
  ButtonGroup,
  Flex,
  FormControl,
  Grid,
  HStack,
  Image,
  Input,
  InputGroup,
  InputLeftElement,
  Stack,
  Text,
} from "@chakra-ui/react";
import { FaSearch } from "react-icons/fa";
import { CHeading } from "./CHeading";

interface IButton {
  colorScheme?: string;
  variant?: string;
  text: string;
  leftIcon?: any;
  onClick: () => void;
```

```
  disabled?: boolean;
}

interface IField {
  icon?: any;
  text: string;
}

interface IProps {
  pageTitle: string;
  noTopBar?: boolean;
  search: {
    show: boolean;
    fields: boolean;
  };
  topButtons: IButton[];
  entries: {
    image?: string;
    title: string;
    buttonRows: IButton[][];
    fields: IField[];
  }[];
}

export default function GridLayoutV2(props: IProps) {
  const [query, setQuery] = React.useState("");
  return (
    <>
      <Flex
        justifyContent={"space-between"}
        p={4}
        direction={["column", "row"]}
        gap={2}
        display={props.noTopBar ? "none" : "flex"}
      >
        <Box></Box>
        <CHeading>{props.pageTitle}</CHeading>
        <Flex
          gap={2}
          direction={["column", "row", "row"]}
          display={props.topButtons.length === 0 ? "none" : "flex"}
        >
          {props.topButtons.map((b) => {
            return (
              <>
                <Button
                  variant={b.variant}
                  colorScheme={b.colorScheme}
                  leftIcon={b.leftIcon}
                  onClick={b.onClick}
                  disabled={b.disabled}
                >
                  {b.text}
                </Button>
              </>
            );
          })}
        </Flex>
        {props.topButtons.length === 0 ? <Box></Box> : null}
      </Flex>
```

```
{props.search.show && (
  <Flex justifyContent={"center"} mt={4} p={4}>
    <FormControl maxW={["100%", "100%", "25%"]}>
      <InputGroup>
        <InputLeftElement>
          <FaSearch />
        </InputLeftElement>
        <Input
          placeholder={"Suche"}
          value={query}
          onChange={(e) => {
            setQuery(e.target.value);
          }}
        />
      </InputGroup>
    </FormControl>
  </Flex>
)}
<Flex justifyContent={"center"}>
  <Grid
    templateColumns={[
      "repeat(1, 1fr)",
      "repeat(2, 1fr)",
      "repeat(3, 1fr)",
    ]}
    gap={6}
    p={4}
    mt={"2rem"}
    maxW={["100%", "100%", "50%"]}
    w={"100%"}
  >
    {props.entries
      .filter((e) => {
        if (query === "") {
          return true;
        }
        if (props.search.fields) {
          for (const f of e.fields) {
            if (f.text.toLowerCase().includes(query.toLowerCase())) {
              return true;
            }
          }
        }
        return e.title.toLowerCase().includes(query.toLowerCase());
      })
      .map((e) => {
        return (
          <>
            <Box bg={"black"} rounded={"xl"} p={4} w={"100%"}>
              {e.image && (
                <Image
                  src={e.image}
                  w={"100%"}
                  objectFit={"contain"}
                  objectPosition={"center"}
                  borderRadius={"lg"}
                  h={"150px"}
                />
              )}
              <CHeading size={"lg"}>{e.title}</CHeading>
```

```
                <Stack
                  gap={3}
                  mt={e.fields.length > 0 ? 4 : 0}
                  mb={e.fields.length > 0 ? 4 : 0}
                >
                  {e.fields.map((f) => {
                    return (
                      <>
                        <HStack alignItems={"center"} gap={4}>
                          {f.icon}
                          <Text fontSize={"lg"}>{f.text}</Text>
                        </HStack>
                      </>
                    );
                  })}
                </Stack>
                <Stack gap={2}>
                  {e.buttonRows.map((br) => {
                    return (
                      <>
                        <Flex
                          w={"100%"}
                          direction={["column", "row"]}
                          gap={2}
                          justifyContent={"space-between"}
                        >
                          {br.map((b) => {
                            return (
                              <>
                                <Button
                                  w={["100%", "100%", "45%"]}
                                  variant={b.variant}
                                  colorScheme={b.colorScheme}
                                  onClick={b.onClick}
                                  leftIcon={b.leftIcon}
                                  disabled={b.disabled}
                                >
                                  {b.text}
                                </Button>
                              </>
                            );
                          })}
                        </Flex>
                      </>
                    );
                  })}
                </Stack>
              </Box>
            </>
          );
        })}
      </Grid>
    </Flex>
  </>
 );
}
```

## packages/components/global/index.ts

```
/**
 * packages/components/global/index.ts
```

```
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 03.07.2022
 *
 */

export { default as Footer } from "./Footer";
export { default as NavigationBar } from "./NavigationBar";
export { default as UserAuthAction } from "./UserAuthAction";
export { default as UserAuthRegister } from "./UserAuthRegister";
```

## packages/components/global/MenuButtons.tsx

```
/**
 * packages/components/global/MenuButtons.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 18.09.2022
 *
 */

import * as React from "react";
import {
  Avatar,
  AvatarBadge,
  Badge,
  Box,
  Button,
  ButtonGroup,
  Center,
  chakra,
  Heading,
  HStack,
  IconButton,
  Menu,
  MenuButton,
  MenuDivider,
  MenuItem,
  MenuList,
  Stack,
  StackDivider,
  Text,
  Tooltip,
} from "@chakra-ui/react";
import { useLoggedIn } from "@codeup/hooks";
import Link from "next/link";
import { FiAlertTriangle, FiPlus, FiRepeat, FiShuffle } from "react-icons/fi";
import { useEffect } from "react";
import REST from "@codeup/rest";
import {
  FaCheck,
  FaCogs,
  FaCrown,
  FaPlus,
  FaSitemap,
  FaTrash,
  FaUsers,
```

```
} from "react-icons/fa";
import PopupManager from "@codeup/web/src/global/PopupManager";
import { HiSwitchHorizontal } from "react-icons/hi";
import AccountManager from "@codeup/util/src/AccountManager";
import { useRouter } from "next/router";

export default function MenuButtons(props: {
  onClose: () => void;
  onUserAuthOpen: () => void;
  onLogoutOpen: () => void;
}) {
  const { loggedIn, userInfo } = useLoggedIn();
  const router = useRouter();

  return (
    <>
      {!loggedIn ? (
        <>
          <Button
            onClick={() => {
              props.onClose();
              props.onUserAuthOpen();
            }}
            variant={"brand"}
          >
            Anmelden
          </Button>
        </>
      ) : null}
      {loggedIn ? (
        <>
          <HStack gap={[6, 2]}>
            <Menu>
              <MenuButton
                as={Button}
                rounded={"full"}
                variant={"link"}
                cursor={"pointer"}
                minW={0}
              >
                <Avatar
                  src={"/api/account/" + userInfo.username + "/picture"}
                  size={"md"}
                />
              </MenuButton>
              <MenuList bg={"#000"} zIndex={10}>
                <br />
                <Center>
                  <Avatar
                    src={"/api/account/" + userInfo.username + "/picture"}
                    size={"xl"}
                  />
                </Center>
                <br />
                <Center>
                  <Heading size={"lg"}>
                    {userInfo.firstName} {userInfo.lastName}
                  </Heading>
                </Center>
                <Center display={"flex"} alignItems={"center"} gap={2}>
```

```
  <Text color={"gray"}>@{userInfo.username}</Text>
  {userInfo.subscription === "PRO" ||
  userInfo.role === "admin" ? (
    <Badge colorScheme={"brand"}>PRO</Badge>
  ) : null}
</Center>
<MenuDivider />
{userInfo.subscription === "FREE" && userInfo.role !== "admin" && (
  <MenuItem
    onClick={async () => {
      const res = await REST.Payments.createCheckoutLink(
        AccountManager.getActiveToken()
      );

      if (res.status === 200) {
        window.location.href = res.payload.url;
      } else {
        PopupManager.alert({
          title: "Fehler",
          description:
            "Es ist ein Fehler aufgetreten. Bitte versuche es später erneut: " +
            res.payload.error,
        });
      }
    }}
    color={"orange.400"}
    fontWeight={"bold"}
    icon={<FaCrown />}
  >
    Upgrade auf PRO
  </MenuItem>
)}
<MenuItem
  icon={<FaPlus />}
  onClick={() => {
    router.push("/creator/studio");
  }}
  color={"blue.300"}
  fontWeight={"bold"}
>
  Creator
</MenuItem>
<MenuDivider />
<MenuItem
  icon={<FaUsers />}
  onClick={() => {
    const UserCard = (props: {
      username: string;
      firstName: string;
      lastName: string;
      pro: boolean;
    }) => {
      return (
        <>
          <HStack
            display={"flex"}
            justifyContent={"space-between"}
          >
            <Box>
              <HStack display={"flex"}>
```

```
    <Avatar
      src={
        "/api/account/" +
        props.username +
        "/picture"
      }
      size={"lg"}
    />
    <Stack>
      <Heading size={"lg"}>
        {props.firstName} {props.lastName}
      </Heading>
      <HStack>
        <Text color={"gray"}>
          @{props.username}
        </Text>
        {props.pro && (
          <Badge colorScheme={"orange"} size={"lg"}>
            PRO
          </Badge>
        )}
      </HStack>
    </Stack>
  </HStack>
</Box>
{userInfo.username !== props.username ? (
  <ButtonGroup isAttached>
    <IconButton
      aria-label={"switch"}
      icon={<HiSwitchHorizontal />}
      onClick={async () => {
        const oldToken = localStorage.getItem(
          "token"
        ) as string;
        const ac =
          AccountManager.getAccounts().find(
            (a) => a.username === props.username
          );
        const newToken = ac!.token;

        AccountManager.removeAccount(ac!);
        await AccountManager.addAccount(oldToken);

        localStorage.setItem("token", newToken);

        PopupManager.removeCurrentPopup();
      }}
    />
    <IconButton
      aria-label={"delete"}
      icon={<FaTrash />}
      onClick={async () => {
        const ac =
          AccountManager.getAccounts().find(
            (a) => a.username === props.username
          );
        AccountManager.removeAccount(ac!);
        PopupManager.removeCurrentPopup();
      }}
    />
```

```
            </ButtonGroup>
          ) : (
            <>
              <chakra.span color={"green.400"}>
                <FaCheck />
              </chakra.span>
            </>
          )}
        </HStack>
      </>
    );
  };

  PopupManager.alert({
    title: "Account wechseln",
    description: (
      <>
        <Stack w={"100%"}>
          <UserCard
            firstName={userInfo.firstName}
            lastName={userInfo.lastName}
            username={userInfo.username}
            pro={userInfo.subscription === "PRO"}
          />
          {AccountManager.getAccounts().map((account) => (
            <>
              <UserCard
                firstName={account.firstName}
                lastName={account.lastName}
                username={account.username}
                pro={userInfo.subscription === "PRO"}
              />
              <StackDivider />
            </>
          ))}
          <StackDivider />
          <Button
            variant={"brand"}
            leftIcon={<FaPlus />}
            onClick={async () => {
              await AccountManager.addAccount(
                localStorage.getItem("token") as string
              );
              localStorage.removeItem("token");

              PopupManager.removeCurrentPopup();

              window.dispatchEvent(
                new CustomEvent("codeup::open_login")
              );
            }}
          >
            Hinzufügen
          </Button>
        </Stack>
      </>
    ),
  });
 }}
>
```

```
              Account wechseln
            </MenuItem>
            <Link href={"/orgs"}>
              <MenuItem icon={<FaSitemap />}>Organisationen</MenuItem>
            </Link>
            <MenuDivider />
            <Link href={"/dashboard"}>
              <MenuItem>Dashboard</MenuItem>
            </Link>
            <Link href={"/account/settings"}>
              <MenuItem>Einstellungen</MenuItem>
            </Link>
            <MenuItem
              color={"red.300"}
              onClick={() => {
                props.onLogoutOpen();
              }}
            >
              Abmelden
            </MenuItem>
          </MenuList>
        </Menu>
        {userInfo.role === "admin" ? (
          <Tooltip
            label={`
            Du bist als Admin angemeldet.
            Du kannst alle Funktionen nutzen und auf die Admin-Seite zugreifen.
            Nutze deine Berechtigungen nicht aus und verhalte dich fair.
            `}
          >
            <Avatar
              size={"md"}
              backgroundColor={"red.200"}
              icon={<FaCogs />}
              cursor={"pointer"}
              onClick={() => {
                router.push("/admin");
              }}
            />
          </Tooltip>
        ) : null}
      </HStack>
    </>
  ) : null}
 </>
);

return (
 <>
  {!loggedIn ? (
    <>
      <Button
        variant={"solid"}
        onClick={() => {
          props.onClose();
          props.onUserAuthOpen();
        }}
        colorScheme={"whatsapp"}
      >
        Anmelden
```

```
          </Button>
        </>
      ) : (
        <>
          <Link href={"/dashboard"}>
            <Button
              variant={"solid"}
              colorScheme={"blue"}
              onClick={props.onClose}
            >
              Dashboard
            </Button>
          </Link>
          <Button
            variant={"solid"}
            colorScheme={"red"}
            onClick={() => {
              props.onClose();
              props.onLogoutOpen();
            }}
          >
            Abmelden
          </Button>
        </>
      )}
    </>
  );
}
```

## packages/components/global/NavigationBar.tsx

```
/**
 * src/components/NavigationBar.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 06.06.2022
 *
 */

import * as React from "react";
import { useEffect, useState } from "react";
import {
  Badge,
  Box,
  Divider,
  Drawer,
  DrawerBody,
  DrawerCloseButton,
  DrawerContent,
  DrawerHeader,
  DrawerOverlay,
  Flex,
  Grid,
  GridItem,
  Heading,
  IconButton,
  Input,
  InputGroup,
  InputLeftElement,
```

```
  Link as ChakraLink,
  Text,
  Tooltip,
  useDisclosure,
  useMediaQuery,
} from "@chakra-ui/react";
import { GiHamburgerMenu } from "react-icons/gi";
import Link from "next/link";
import {
  FaBook,
  FaChild,
  FaComment,
  FaFileCode,
  FaProjectDiagram,
  FaRocket,
  FaSearch,
  FaTrophy,
} from "react-icons/fa";
import { useRouter } from "next/router";
import { BiCodeAlt, BiCodeBlock } from "react-icons/bi";
import { TbCertificate } from "react-icons/tb";
import { HiLightBulb } from "react-icons/hi";
import { BsCheck2Square } from "react-icons/bs";
import InCodeIcon from "../icons/InCodeIcon";
import UserAuthAction from "./UserAuthAction";
import NavigationBarMenu from "./NavigationBarMenu";
import MenuButtons from "./MenuButtons";
import CodeUpModal from "../codeup/CodeUpModal";
import PopupManager from "@codeup/web/src/global/PopupManager";
import SuperText from "../landingpage/SuperText";
import { useLoggedIn } from "@codeup/hooks";
import { CHeading } from "./CHeading";

export default function NavigationBar(props: { disablePricing?: boolean }) {
  const whatEverRef = React.useRef();

  const logout = async () => {
    if (
      await PopupManager.confirmAsync({
        title: "Abmelden",
        question: "Willst du dich wirklich abmelden?",
      })
    ) {
      localStorage.removeItem("token");
      router.push("/");
    }
  };

  const [isMobile] = useMediaQuery("(max-width: 768px)");
  const { isOpen, onOpen, onClose } = useDisclosure();
  const { userInfo, loggedIn, loaded } = useLoggedIn();

  const router = useRouter();

  const {
    isOpen: isMoreMenuOpen,
    onOpen: onMoreMenuOpen,
    onClose: onMoreMenuClose,
  } = useDisclosure();
```

```
const {
  isOpen: isUserAuthOpen,
  onOpen: onUserAuthOpen,
  onClose: onUserAuthClose,
} = useDisclosure();

useEffect(() => {
  if (router.asPath.includes("#login")) {
    onUserAuthOpen();
  }
}, []);

const ExpandedMenuOption = (props: {
  name: string;
  description: string;
  icon: any;
  url: string;
}) => {
  return (
    <>
      <Box
        padding={".8rem"}
        borderRadius={"6px"}
        h={"100%"}
        w={"100%"}
        zIndex={100}
        _hover={{
          backgroundColor: "whiteAlpha.100",
        }}
        _active={{
          backgroundColor: "whiteAlpha.200",
        }}
        cursor={"pointer"}
        transition={"200ms"}
        onClick={async () => {
          await onMoreMenuClose();
          await onClose();
          await router.push(props.url);
        }}
      >
        <Flex direction={"row"} w={"100%"} alignItems={"center"} h={"100%"}>
          <Box flex={"10%"}>
            <Heading color={"white"}>{props.icon}</Heading>
          </Box>
          <Box flex={"10%"} minW={"20px"}></Box>
          <Box flex={"80%"}>
            <Flex w={"100%"} direction={"column"}>
              <Heading size={"lg"}>{props.name}</Heading>
              <Text color={"gray.300"}>{props.description}</Text>
            </Flex>
          </Box>
        </Flex>
      </Box>
    </>
  );
};

const moreOptions: {
  name: string;
  text: string;
```

```
  icon: any;
  href: string;
  category: string;
}[] = [
  {
    name: "Forum",
    icon: <FaComment />,
    text: "Unterhalte dich mit anderen Entwicklern",
    href: "/forum",
    category: "communicate",
  },
  {
    name: "Editor",
    icon: <BiCodeAlt />,
    href: "/editor",
    text: "Verwirkliche deine Projektideen",
    category: "develop",
  },
  {
    name: "CodeUp Kids",
    icon: <FaChild />,
    href: "/kids",
    text: "Blockbasierte Programmierung mit CodeUp Kids",
    category: "develop",
  },
  {
    name: "Discovery",
    icon: <BiCodeBlock />,
    text: "Sieh dir Projekte an, die von anderen Entwicklern veröffentlicht wurden.",
    href: "/discovery",
    category: "communicate",
  },
  {
    name: "InCode",
    icon: <InCodeIcon />,
    text: "Originale Dokumentation der InCode Programmiersprache",
    href: "/redirect?url=https://incode.codeup.space/docs/intro",
    category: "other",
  },
  {
    name: "Kurse",
    icon: <FaBook />,
    text: "Lerne neue Programmiersprachen und Technologien",
    href: "/course",
    category: "learn",
  },
  {
    name: "Zertifikate",
    icon: <TbCertificate />,
    text: "Erhalte Zertifikate für absolvierte Kurse",
    href: "/certificates",
    category: "learn",
  },
  {
    name: "Projektideen",
    icon: <HiLightBulb />,
    text: "Finde Inspirationen für neue Projekte",
    href: "/ideas",
    category: "develop",
  },
```

```
  {
    name: "Todo",
    icon: <BsCheck2Square />,
    text: "Plane Projekte mit einfachen To-Do Listen",
    href: "/todo",
    category: "plan",
  },
  {
    name: "Flows",
    icon: <FaProjectDiagram />,
    text: "Plane Projekte mit komplexen Flussdiagrammen",
    href: "/planning",
    category: "plan",
  },
  {
    name: "Code Snippets",
    icon: <FaFileCode />,
    text: "Speicher Code Snippets für spätere Verwendung",
    href: "/snippets/my",
    category: "develop",
  },
  {
    name: "Roadmap",
    icon: <FaRocket />,
    href: "/roadmap",
    text: "Behalte den Überblick über deinen Entwicklungsprozess",
    category: "other",
  },
  {
    name: "IDE (veraltet)",
    icon: <BiCodeAlt />,
    href: "/ide",
    text: "Verwirkliche deine Projektideen",
    category: "other",
  },
  {
    name: "Challenges",
    category: "learn",
    href: "/challenges",
    text: "Fordere dein Können heraus",
    icon: <FaTrophy />,
  },
];

const [filteredMoreOptions, setFilteredMoreOptions] =
  useState<typeof moreOptions>(moreOptions);

return (
  <>
    <UserAuthAction
      onOpen={onUserAuthOpen}
      onClose={onUserAuthClose}
      isOpen={isUserAuthOpen}
    />
    <Box w={"100%"} bg={"transparent"} padding={"1.2rem"}>
      <Flex direction={"row"} alignItems={"center"} justifyContent={"center"}>
        <Heading flex={"flex-start"} textAlign={["center", "left"]}>
          <Link href={"/"}>
            <ChakraLink
              href={"/"}
```

```
      display={"flex"}
      flexDirection="row"
      alignItems={"center"}
      gap="1.2rem"
    >
      <img src="/codeup.svg" width="48" />
      <Text
        textTransform={"uppercase"}
        fontWeight={"extrabold"}
        ml={4}
      >
        CodeUp
      </Text>
    </ChakraLink>
  </Link>
</Heading>
<Flex
  justifyContent={isMobile ? "right" : "center"}
  flex={"100%"}
  alignItems={"center"}
>
  {!isMobile ? (
    <NavigationBarMenu
      onClose={onClose}
      onOpen={onOpen}
      onMoreMenuOpen={onMoreMenuOpen}
      onLogoutOpen={logout}
      disablePricing={props.disablePricing}
    />
  ) : (
    <>
      <IconButton
        onClick={onOpen}
        aria-label={"Menu"}
        icon={<GiHamburgerMenu />}
        flex={"flex-end"}
      />
      <Drawer
        isOpen={isOpen}
        placement="right"
        onClose={onClose}
        size={"full"}
      >
        <DrawerOverlay />
        <DrawerContent bg={"black"}>
          <DrawerCloseButton />
          <DrawerHeader>CodeUp</DrawerHeader>

          <DrawerBody>
            <Flex w={"100%"} direction={"column"}>
              <NavigationBarMenu
                onClose={onClose}
                onOpen={onOpen}
                onMoreMenuOpen={onMoreMenuOpen}
                onLogoutOpen={logout}
                disablePricing={props.disablePricing}
              />
            </Flex>
            <Flex
              mt={"2rem"}
```

```
                w={"100%"}
                alignItems={"center"}
                justifyContent={"center"}
              >
                <MenuButtons
                  onClose={onClose}
                  onUserAuthOpen={onUserAuthOpen}
                  onLogoutOpen={logout}
                />
              </Flex>
            </DrawerBody>
          </DrawerContent>
        </Drawer>
      </>
    )}
  </Flex>
  {!isMobile ? (
    <Flex
      justifyContent={["center", "flex-end"]}
      flex={"10%"}
      alignItems={"center"}
      gap={"1.2rem"}
      mt={["2rem", 0]}
    >
      <MenuButtons
        onClose={onClose}
        onUserAuthOpen={onUserAuthOpen}
        onLogoutOpen={logout}
      />
    </Flex>
  ) : null}
 </Flex>
</Box>
<CodeUpModal
 onOpen={onMoreMenuOpen}
 onClose={onMoreMenuClose}
 isOpen={isMoreMenuOpen}
 title={"Angebote"}
 customSize={["full"]}
>
 <Box mb={8} backgroundColor={"black"} shadow={"2xl"}>
  <InputGroup>
   <InputLeftElement>
    <FaSearch />
   </InputLeftElement>
   <Input
    placeholder={"Suchbegriff eingeben"}
    onChange={(event) => {
     const searchTerm = event.target.value.toLowerCase() as string;

     setFilteredMoreOptions(
      moreOptions.filter((option) => {
       return (
        option.name.toLowerCase().includes(searchTerm) ||
        option.text.toLowerCase().includes(searchTerm)
       );
      })
     );
    }}
   />
```

```
  </InputGroup>
</Box>
<Grid templateColumns={["repeat(1, 1fr)", "repeat(2, 1fr)"]} gap={6}>
  <GridItem
    borderStyle={"dashed"}
    borderRadius={"6px"}
    borderColor={"gray.700"}
    borderWidth={"3px"}
    p={2}
    pt={0}
    display={
      filteredMoreOptions.filter(
        (option) => option.category === "learn"
      ).length > 0
        ? "block"
        : "none"
    }
  >
    <Box textAlign={"center"} p={2}>
      <CHeading size={"xl"}>Lernen</CHeading>
    </Box>
    <Grid
      templateColumns={["repeat(1, 1fr)", "repeat(2, 1fr)"]}
      gap={6}
    >
      {filteredMoreOptions
        .filter((option) => option.category === "learn")
        .map((option) => (
          <>
            <GridItem>
              <ExpandedMenuOption
                name={option.name}
                description={option.text}
                icon={option.icon}
                url={option.href}
              />
            </GridItem>
          </>
        ))}
    </Grid>
  </GridItem>
  <GridItem
    borderStyle={"dashed"}
    borderRadius={"6px"}
    borderColor={"gray.700"}
    borderWidth={"3px"}
    p={2}
    pt={0}
    display={
      filteredMoreOptions.filter(
        (option) => option.category === "develop"
      ).length > 0
        ? "block"
        : "none"
    }
  >
    <Box textAlign={"center"} p={2}>
      <CHeading size={"xl"}>Entwickeln</CHeading>
    </Box>
    <Grid
```

```
    templateColumns={["repeat(1, 1fr)", "repeat(2, 1fr)"]}
    gap={6}
  >
    {filteredMoreOptions
      .filter((option) => option.category === "develop")
      .map((option) => (
        <>
          <GridItem>
            <ExpandedMenuOption
              name={option.name}
              description={option.text}
              icon={option.icon}
              url={option.href}
            />
          </GridItem>
        </>
      ))}
  </Grid>
</GridItem>
<GridItem
  borderStyle={"dashed"}
  borderRadius={"6px"}
  borderColor={"gray.700"}
  borderWidth={"3px"}
  p={2}
  pt={0}
  display={
    filteredMoreOptions.filter(
      (option) => option.category === "communicate"
    ).length > 0
      ? "block"
      : "none"
  }
>
  <Box textAlign={"center"} p={2}>
    <CHeading size={"xl"}>Austauschen</CHeading>
  </Box>
  <Grid
    templateColumns={["repeat(1, 1fr)", "repeat(2, 1fr)"]}
    gap={6}
  >
    {filteredMoreOptions
      .filter((option) => option.category === "communicate")
      .map((option) => (
        <>
          <GridItem>
            <ExpandedMenuOption
              name={option.name}
              description={option.text}
              icon={option.icon}
              url={option.href}
            />
          </GridItem>
        </>
      ))}
  </Grid>
</GridItem>
<GridItem
  borderStyle={"dashed"}
  borderRadius={"6px"}
```

```
  borderColor={"gray.700"}
  borderWidth={"3px"}
  p={2}
  pt={0}
  display={
    filteredMoreOptions.filter((option) => option.category === "plan")
      .length > 0
      ? "block"
      : "none"
  }
>
  <Box textAlign={"center"} p={2}>
    <CHeading size={"xl"}>Planen</CHeading>
  </Box>
  <Grid
    templateColumns={["repeat(1, 1fr)", "repeat(2, 1fr)"]}
    gap={6}
  >
    {filteredMoreOptions
      .filter((option) => option.category === "plan")
      .map((option) => (
        <>
          <GridItem>
            <ExpandedMenuOption
              name={option.name}
              description={option.text}
              icon={option.icon}
              url={option.href}
            />
          </GridItem>
        </>
      ))}
  </Grid>
</GridItem>
<GridItem
  borderStyle={"dashed"}
  borderRadius={"6px"}
  borderColor={"gray.700"}
  borderWidth={"3px"}
  p={2}
  pt={0}
  display={
    filteredMoreOptions.filter(
      (option) => option.category === "other"
    ).length > 0
      ? "block"
      : "none"
  }
>
  <Box textAlign={"center"} p={2}>
    <CHeading size={"xl"}>Anderes</CHeading>
  </Box>
  <Grid
    templateColumns={["repeat(1, 1fr)", "repeat(2, 1fr)"]}
    gap={6}
  >
    {filteredMoreOptions
      .filter((option) => option.category === "other")
      .map((option) => (
        <>
```

```
                <GridItem>
                  <ExpandedMenuOption
                    name={option.name}
                    description={option.text}
                    icon={option.icon}
                    url={option.href}
                  />
                </GridItem>
              </>
            ))}
          </Grid>
        </GridItem>
      </Grid>
    </CodeUpModal>
  </>
 );
}
```

## packages/components/global/NavigationBarMenu.tsx

```
/**
 * packages/components/global/NavigationBarMenu.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 18.09.2022
 *
 */

import * as React from "react";
import {
  Button,
  Divider,
  Flex,
  Input,
  InputGroup,
  InputLeftElement,
  InputRightElement,
  Popover,
  PopoverArrow,
  PopoverBody,
  PopoverContent,
  PopoverTrigger,
  Stack,
  StackDivider,
  Text,
} from "@chakra-ui/react";
import { AiFillCaretDown } from "react-icons/ai";
import Link from "next/link";
import { useLoggedIn } from "@codeup/hooks";
import { FaCaretDown, FaDollarSign, FaSearch } from "react-icons/fa";
import DividerWithText from "./DividerWithText";
import { useRouter } from "next/router";

export default function NavigationBarMenu(props: {
  onClose: () => void;
  onOpen: () => void;
  onMoreMenuOpen: () => void;
  onLogoutOpen: () => void;
  disablePricing?: boolean;
```

```
}) {
  const { loggedIn } = useLoggedIn();
  const router = useRouter();

  return (
    <>
      <Button
        variant={"ghost"}
        size={"lg"}
        rightIcon={<AiFillCaretDown />}
        onClick={props.onMoreMenuOpen}
      >
        Angebote
      </Button>
      <Button
        variant={"ghost"}
        size={"lg"}
        onClick={() => {
          router.push("/pricing");
        }}
        display={props.disablePricing ? "none" : "initial"}
      >
        Preise
      </Button>
      <Popover>
        <PopoverTrigger>
          <Button variant={"ghost"} size={"lg"} rightIcon={<FaCaretDown />}>
            CodeUp
          </Button>
        </PopoverTrigger>
        <PopoverContent bg={"black"}>
          <PopoverArrow bg={"black"} />
          <PopoverBody>
            <Stack textAlign={"center"}>
              <Link href={"/codeup/about"} onClick={props.onClose}>
                <Text
                  fontSize={"xl"}
                  cursor={"pointer"}
                  _hover={{
                    bg: "gray.700",
                  }}
                  rounded={"lg"}
                >
                  Über Uns
                </Text>
              </Link>
              <Link href={"/codeup/contact"} onClick={props.onClose}>
                <Text
                  fontSize={"xl"}
                  cursor={"pointer"}
                  _hover={{
                    bg: "gray.700",
                  }}
                  rounded={"lg"}
                >
                  Kontakt
                </Text>
              </Link>
              <Link href={"/blog"} onClick={props.onClose}>
                <Text
```

```
            fontSize={"xl"}
            cursor={"pointer"}
            _hover={{
              bg: "gray.700",
            }}
            rounded={"lg"}
          >
            Blog
          </Text>
        </Link>
        <Link href={"/creator"} onClick={props.onClose}>
          <Text
            fontSize={"xl"}
            cursor={"pointer"}
            _hover={{
              bg: "gray.700",
            }}
            rounded={"lg"}
          >
            Creator
          </Text>
        </Link>
        <DividerWithText text={"Rechtliches"} color={"gray.400"} />
        <Link href={"/legal/imprint"} onClick={props.onClose}>
          <Text
            fontSize={"xl"}
            cursor={"pointer"}
            _hover={{
              bg: "gray.700",
            }}
            rounded={"lg"}
          >
            Impressum
          </Text>
        </Link>
        <Link href={"/legal/privacy"} onClick={props.onClose}>
          <Text
            fontSize={"xl"}
            cursor={"pointer"}
            _hover={{
              bg: "gray.700",
            }}
            rounded={"lg"}
          >
            Datenschutzerklärung
          </Text>
        </Link>
      </Stack>
    </PopoverBody>
  </PopoverContent>
</Popover>
<Button
  variant={"ghost"}
  size={"lg"}
  onClick={() => {
    window.dispatchEvent(new CustomEvent("codeup::open_search"));
  }}
  display={["none", "initial"]}
>
  Suche
```

```
      </Button>
    </>
  );
}
```

## packages/components/global/Newsletter.tsx

```
/**
 * packages/components/global/Newsletter.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 13.04.2023
 *
 */

import * as React from "react";
import { useLoggedIn } from "@codeup/hooks";
import REST from "@codeup/rest/dist";
import AccountManager from "@codeup/util/dist/AccountManager";
import { subscribed } from "@codeup/rest/dist/api/Newsletter";
import { Button, ButtonGroup, Heading, Stack, Text } from "@chakra-ui/react";
import { FaNewspaper } from "react-icons/fa";
import PopupManager from "@codeup/web/src/global/PopupManager";

export default function Newsletter() {
  const { userInfo, loggedIn, loaded } = useLoggedIn();

  const [subscribed, setSubscribed] = React.useState(false);

  React.useEffect(() => {
    if (loggedIn && loaded) {
      REST.Newsletter.subscribed(AccountManager.getActiveToken()).then(
        (res) => {
          if (res.status === 200) {
            setSubscribed(res.payload.newsletter);
          }
        }
      );
    }
  }, [loggedIn, loaded]);

  if (!loaded) {
    return <></>;
  }

  return (
    <>
      <Stack gap={2}>
        <Heading
          fontWeight={"extrabold"}
          display={"flex"}
          flexDirection={"row"}
          gap={2}
          alignItems={"center"}
        >
          <FaNewspaper />
          Newsletter
        </Heading>
        <Text color={"gray.300"} fontSize={"md"}>
```

```
  Erhalte regelmäßig Updates zu CodeUp und den neusten Projekten.
</Text>
{loggedIn ? (
  <>
    {subscribed ? (
      <>
        <Button
          colorScheme={"red"}
          onClick={async () => {
            const res = await REST.Newsletter.unsubscribe(
              AccountManager.getActiveToken()
            );

            if (res.status === 200) {
              setSubscribed(false);
              await PopupManager.alertAsync({
                title: "Abgemeldet",
                description:
                  "Du wurdest erfolgreich vom Newsletter abgemeldet.",
              });
            } else {
              await PopupManager.alertAsync({
                title: "Fehler",
                description:
                  "Beim abmelden ist ein Fehler aufgetreten: " +
                  res.payload.error,
              });
            }
          }}
        >
          Abmelden
        </Button>
      </>
    ) : (
      <>
        <Button
          colorScheme={"green"}
          onClick={async () => {
            const res = await REST.Newsletter.subscribe(
              AccountManager.getActiveToken()
            );

            if (res.status === 200) {
              setSubscribed(true);
              await PopupManager.alertAsync({
                title: "Angemeldet",
                description:
                  "Du wurdest erfolgreich beim Newsletter angemeldet.",
              });
            } else {
              await PopupManager.alertAsync({
                title: "Fehler",
                description:
                  "Beim anmelden ist ein Fehler aufgetreten: " +
                  res.payload.error,
              });
            }
          }}
        >
          Anmelden
```

```
              </Button>
            </>
          )}
        </>
      ) : (
        <>
          <Button
            bg={"brand.500"}
            color={"black"}
            _hover={{
              bg: "brand.500",
              color: "black",
            }}
            _focus={{
              bg: "brand.500",
              color: "black",
            }}
            size={"lg"}
            onClick={() => {
              window.dispatchEvent(new CustomEvent("codeup::open_login"));
            }}
          >
            Zum Abonieren anmelden
          </Button>
        </>
      )}
    </Stack>
  </>
 );
}
```

## packages/components/global/Sidebar.tsx

```
/**
 * packages/components/global/Sidebar.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 01.04.2023
 *
 */

import * as React from "react";
import { ReactNode } from "react";
import {
  Button,
  Flex,
  Input,
  InputGroup,
  InputLeftElement,
} from "@chakra-ui/react";
import { FaSearch } from "react-icons/fa";

export default function Sidebar(props: {
  entries: {
    title: string;
    icon: ReactNode;
    onClick: () => void;
    disabled?: boolean;
  }[];
```

```
  title: string;
}) {
  const [query, setQuery] = React.useState<string>("");

  return (
    <>
      <Flex
        w={"100%"}
        h={"100%"}
        minH={"100vh"}
        bgColor={"black"}
        direction={"column"}
        pt={"2rem"}
        alignItems={"center"}
      >
        <InputGroup w={"90%"} mb={4}>
          <InputLeftElement>
            <FaSearch />
          </InputLeftElement>
          <Input
            placeholder={"Seite suchen..."}
            value={query}
            onChange={(e) => setQuery(e.target.value)}
          />
        </InputGroup>
        {props.entries
          .filter((p) => p.title.toLowerCase().includes(query.toLowerCase()))
          .sort((a, b) => {
            if (a.title < b.title) {
              return -1;
            } else if (a.title > b.title) {
              return 1;
            } else {
              return 0;
            }
          })
          .map((entry, i) => {
            return (
              <>
                <Button
                  rounded={"none"}
                  colorScheme={"gray"}
                  variant={"ghost"}
                  w={"100%"}
                  justifyContent={"flex-start"}
                  size={"lg"}
                  leftIcon={entry.icon as any}
                  fontSize={"2xl"}
                  onClick={entry.onClick}
                  disabled={
                    entry.disabled === undefined ? false : entry.disabled
                  }
                >
                  {entry.title}
                </Button>
              </>
            );
          })}
      </Flex>
    </>
```

```
  );
}
```

## packages/components/global/UserAuthAction.tsx

```
/**
 * packages/components/global/UserAuthAction.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 18.09.2022
 *
 */

import * as React from "react";
import {
  Box,
  Button,
  ButtonGroup,
  Divider,
  Flex,
  Heading,
  Input,
  Modal,
  ModalBody,
  ModalCloseButton,
  ModalContent,
  ModalFooter,
  ModalHeader,
  ModalOverlay,
  Stack,
  Tab,
  TabList,
  TabPanel,
  TabPanels,
  Tabs,
  Text,
} from "@chakra-ui/react";
import UserAuthRegister from "./UserAuthRegister";
import UserAuthLogin from "./UserAuthLogin";
import UserAuthReset from "./UserAuthReset";
import CodeUpModal from "../codeup/CodeUpModal";
import UserAuthSSO from "./UserAuthSSO";
import { useEffect } from "react";
import DividerWithText from "./DividerWithText";

export default function UserAuthAction(props: {
  onOpen: () => void;
  onClose: () => void;
  isOpen: boolean;
}) {
  useEffect(() => {
    window.addEventListener("codeup::open_login", props.onOpen);

    return () => {
      window.removeEventListener("codeup::open_login", props.onOpen);
    };
  });

  return (
```

```
    <>
      <CodeUpModal
        onOpen={props.onOpen}
        onClose={props.onClose}
        isOpen={props.isOpen}
        title={"Anmelden"}
      >
        <Tabs isFitted variant={"line"} colorScheme={"brand"} flex={"50%"}>
          <TabList>
            <Tab>Registrieren</Tab>
            <Tab>Anmelden</Tab>
            <Tab>Passwort Zurücksetzen</Tab>
          </TabList>
          <TabPanels>
            <TabPanel>
              <UserAuthRegister />
            </TabPanel>
            <TabPanel>
              <UserAuthLogin onClose={props.onClose} />
            </TabPanel>
            <TabPanel>
              <UserAuthReset />
            </TabPanel>
          </TabPanels>
          <DividerWithText />
          <UserAuthSSO />
        </Tabs>
      </CodeUpModal>
    </>
  );
}

/*
 OLD CODE
 --------
<Tabs isFitted variant={"line"} colorScheme={"red"} flex={"50%"}>
          <TabList>
            <Tab>SSO</Tab>
            <Tab>Registrieren</Tab>
            <Tab>Anmelden</Tab>
            <Tab>Passwort Zurücksetzen</Tab>
          </TabList>
          <TabPanels>
            <TabPanel>
              <UserAuthSSO />
            </TabPanel>
            <TabPanel>
              <UserAuthRegister />
            </TabPanel>
            <TabPanel>
              <UserAuthLogin onClose={props.onClose} />
            </TabPanel>
            <TabPanel>
              <UserAuthReset />
            </TabPanel>
          </TabPanels>
        </Tabs>
 */
```

packages/components/global/UserAuthLogin.tsx

```
/**
 * packages/components/global/UserAuthLogin.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 18.09.2022
 *
 */

import * as React from "react";
import {
  Alert,
  AlertDescription,
  AlertIcon,
  Button,
  Flex,
  FormControl,
  FormLabel,
  Heading,
  HStack,
  Input,
  InputGroup,
  InputRightElement,
  PinInput,
  PinInputField,
  Text,
} from "@chakra-ui/react";
import { AiFillEye, AiFillEyeInvisible } from "react-icons/ai";
import REST from "@codeup/rest";
import { IResponse } from "@codeup/rest/src/types/IResponse";
import AccountManager, { Account } from "@codeup/util/src/AccountManager";

export default function UserAuthLogin(props: { onClose: () => void }) {
  const [email, setEmail] = React.useState("");
  const [password, setPassword] = React.useState("");

  const [passwordVisible, setPasswordVisible] = React.useState(false);

  const [btnDisabled, setBtnDisabled] = React.useState(false);

  const [alertVisible, setAlertVisible] = React.useState(false);
  const [alertMessage, setAlertMessage] = React.useState("");
  const [alertType, setAlertType] = React.useState<
    "error" | "info" | "success" | "warning"
  >("error");

  const [_2fa, set2fa] = React.useState(false);

  return (
    <>
      {alertVisible ? (
        <Alert status={alertType} variant={"top-accent"} mb={"1.2rem"}>
          <AlertIcon />
          <AlertDescription>{alertMessage}</AlertDescription>
        </Alert>
      ) : null}
      {!_2fa ? (
        <>
          <form
```

```
  onSubmit={async (e) => {
    e.preventDefault();
    setBtnDisabled(true);
    setAlertVisible(false);

    let res: IResponse;

    if (email.match(/^[\w-\.]+@([\w-]+\.)+[\w-]{2,4}$/g)) {
      res = await REST.Account.login({
        email,
        password,
      });
    } else {
      res = await REST.Account.loginWithUsername({
        username: email,
        password,
      });
    }

    if (res.status !== 200) {
      setAlertType("error");
      setAlertMessage(res.payload.error);
      setAlertVisible(true);
      setBtnDisabled(false);
    } else {
      if (res.payload.token) {
        localStorage.setItem("token", res.payload.token);
        if (
          AccountManager.getAccounts().find(
            (a) => a.username === email || a.email === email
          ) !== undefined
        ) {
          AccountManager.removeAccount(
            AccountManager.getAccounts().find(
              (a) => a.username === email || a.email === email
            ) as Account
          );
        }

        props.onClose();
      } else {
        set2fa(true);
      }
    }
  }}
>
  <Flex direction={"column"} gap={"2rem"}>
    <FormControl>
      <FormLabel htmlFor={"email"}>
        E-Mail oder Benutzername
      </FormLabel>
      <Input
        id={"email"}
        placeholder={"E-Mail oder Benutzername"}
        onChange={(e) => {
          setEmail(e.target.value);
        }}
      />
    </FormControl>
    <FormControl>
```

```
          <FormLabel htmlFor={"password"}>Passwort</FormLabel>
          <InputGroup>
            <Input
              id={"password"}
              placeholder={"Passwort"}
              type={passwordVisible ? "text" : "password"}
              onChange={(e) => {
                setPassword(e.target.value);
              }}
            />
            <InputRightElement
              cursor={"pointer"}
              onClick={() => {
                setPasswordVisible(!passwordVisible);
              }}
            >
              {passwordVisible ? <AiFillEyeInvisible /> : <AiFillEye />}
            </InputRightElement>
          </InputGroup>
        </FormControl>
        <Button
          variant={"brand"}
          type={"submit"}
          isLoading={btnDisabled}
          disabled={btnDisabled}
        >
          Login
        </Button>
      </Flex>
    </form>
  </>
) : (
  <>
    <Flex direction={"column"} gap={"2rem"}>
      <Flex
        alignItems={"center"}
        justifyContent={"center"}
        direction={"column"}
      >
        <Heading size={"xl"} textAlign={"center"}>
          2 Faktor Authentifizierung
        </Heading>
        <Text textAlign={"center"} mt={".4rem"}>
          Bitte gib den Code aus deiner Authentifizierungs-App ein.
        </Text>
      </Flex>
      <HStack alignItems={"center"} justifyContent={"center"}>
        <PinInput
          size="lg"
          onComplete={async (code) => {
            let res: IResponse | null;

            if (email.match(/^[\w-\.]+@([\w-]+\.)+[\w-]{2,4}$/g)) {
              res = await REST.Account.login({
                email,
                password,
                code,
              });
            } else {
              res = await REST.Account.loginWithUsername({
```

```
                username: email,
                password,
                code,
              });
            }

            if (res.status !== 200) {
              setAlertType("error");
              setAlertMessage(res.payload.error);
              setAlertVisible(true);
            } else {
              localStorage.setItem("token", res.payload.token);
              if (
                AccountManager.getAccounts().find(
                  (a) => a.username === email || a.email === email
                ) !== undefined
              ) {
                AccountManager.removeAccount(
                  AccountManager.getAccounts().find(
                    (a) => a.username === email || a.email === email
                  ) as Account
                );
              }
              props.onClose();
            }
          }}
        >
          <PinInputField />
          <PinInputField />
          <PinInputField />
          <PinInputField />
          <PinInputField />
          <PinInputField />
        </PinInput>
      </HStack>
    </Flex>
  </>
 )}
</>
 );
}
```

## packages/components/global/UserAuthRegister.tsx

```
/**
 * packages/components/global/UserAuthRegister.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 18.09.2022
 *
 */

import * as React from "react";
import {
  Alert,
  AlertDescription,
  AlertIcon,
  Button,
  Flex,
```

```
  FormControl,
  FormErrorMessage,
  FormLabel,
  Input,
  InputGroup,
  InputRightElement,
  Link as ChakraLink,
  Text,
} from "@chakra-ui/react";
import REST from "@codeup/rest";
import { AiFillEye, AiFillEyeInvisible } from "react-icons/ai";
import checkPassword from "@codeup/util/src/checkPassword";
import Link from "next/link";

export default function UserAuthRegister() {
  const [usernameInvalid, setUsernameInvalid] = React.useState(false);
  const [firstNameInvalid, setFirstNameInvalid] = React.useState(false);
  const [lastNameInvalid, setLastNameInvalid] = React.useState(false);
  const [emailInvalid, setEmailInvalid] = React.useState(false);
  const [passwordInvalid, setPasswordInvalid] = React.useState(false);
  const [passwordConfirmInvalid, setPasswordConfirmInvalid] =
    React.useState(false);

  const [passwordVisible, setPasswordVisible] = React.useState(false);
  const [passwordConfirmVisible, setPasswordConfirmVisible] =
    React.useState(false);

  const [username, setUsername] = React.useState("");
  const [firstName, setFirstName] = React.useState("");
  const [lastName, setLastName] = React.useState("");
  const [email, setEmail] = React.useState("");
  const [password, setPassword] = React.useState("");
  const [passwordConfirm, setPasswordConfirm] = React.useState("");

  const [alertVisible, setAlertVisible] = React.useState(false);
  const [alertMessage, setAlertMessage] = React.useState("");
  const [alertType, setAlertType] = React.useState<
    "error" | "info" | "success" | "warning"
  >("error");

  const [loading, setLoading] = React.useState(false);

  return (
    <>
      {alertVisible ? (
        <Alert status={alertType} variant={"top-accent"} mb={"1.2rem"}>
          <AlertIcon />
          <AlertDescription>{alertMessage}</AlertDescription>
        </Alert>
      ) : null}
      <Flex direction={"column"} gap={"2rem"}>
        <form
          onSubmit={async (e) => {
            e.preventDefault();

            if (!checkPassword(password)) {
              setPasswordInvalid(true);
              return;
            }
```

```
        if (password !== passwordConfirm) {
          setPasswordConfirmInvalid(true);
          return;
        }

        if (
          !email.match(
            /^[a-zA-Z0-9.!#$%&'*+/=?^_`{|}~-]+@[a-zA-Z0-9](?:[a-zA-Z0-9-]{0,61}[a-zA-Z0-9])?(?:\.[a-
zA-Z0-9](?:[a-zA-Z0-9-]{0,61}[a-zA-Z0-9])?)*$/
          )
        ) {
          setEmailInvalid(true);
          return;
        }

        if (!(firstName.length > 0)) {
          setFirstNameInvalid(true);
          return;
        }

        if (!(lastName.length > 0)) {
          setLastNameInvalid(true);
          return;
        }

        setLoading(true);
        const body = {
          email: email,
          password: password,
          firstName: firstName,
          lastName: lastName,
          username: username,
        };

        const res = await REST.Account.register({
          email,
          username,
          password,
          firstName,
          lastName,
        });

        if (res.status === 200) {
          setAlertType("success");
          setAlertMessage(res.payload.message);
        } else {
          setAlertType("error");
          setAlertMessage(res.payload.error);
        }
        setAlertVisible(true);

        setLoading(false);
      }}
    >
      <Flex direction={"column"} gap={"2rem"}>
        <FormControl isInvalid={usernameInvalid}>
          <FormLabel htmlFor={"username"}>Benutzername</FormLabel>
          <Input
            id={"username"}
            placeholder={"Benutzername"}
```

```
    onChange={async (e) => {
      const res = await REST.Account.checkUsername(e.target.value);

      if (res.status === 200) {
        setUsernameInvalid(false);
        setUsername(e.target.value);
      } else {
        setUsernameInvalid(true);
      }
    }}
  />
  <FormErrorMessage>
    Dieser Benutzername ist nicht verfügbar.
  </FormErrorMessage>
</FormControl>
<Flex direction={"row"} gap={"2rem"}>
  <FormControl isInvalid={firstNameInvalid}>
    <FormLabel htmlFor={"firstName"}>Vorname</FormLabel>
    <Input
      id={"firstName"}
      placeholder={"Vorname"}
      onChange={(e) => {
        if (e.target.value.length > 0) {
          setFirstNameInvalid(false);
          setFirstName(e.target.value);
        } else {
          setFirstNameInvalid(true);
        }
      }}
    />
    <FormErrorMessage>
      Bitte gib deinen Vorname ein.
    </FormErrorMessage>
  </FormControl>
  <FormControl isInvalid={lastNameInvalid}>
    <FormLabel htmlFor={"lastName"}>Nachname</FormLabel>
    <Input
      id={"lastName"}
      placeholder={"Nachname"}
      onChange={(e) => {
        if (e.target.value.length > 0) {
          setLastNameInvalid(false);
          setLastName(e.target.value);
        } else {
          setLastNameInvalid(true);
        }
      }}
    />
    <FormErrorMessage>
      Bitte gib deinen Nachname ein.
    </FormErrorMessage>
  </FormControl>
</Flex>
<FormControl isInvalid={emailInvalid}>
  <FormLabel htmlFor={"email"}>E-Mail</FormLabel>
  <Input
    id={"email"}
    placeholder={"E-Mail"}
    onChange={(e) => {
      if (
```

```
                e.target.value.match(
                  /^(([^<>()\[\]\\.,;:\s@"]+(\.[^<>()\[\]\\.,;:\s@"]+)*)|(".+"))@((\[[0-9]{1,3}\.[0-9]{1,3}\.[0-
9]{1,3}\.[0-9]{1,3}\])|(([a-zA-Z\-0-9]+\.)+[a-zA-Z]{2,}))$/
                )
              ) {
                setEmailInvalid(false);
                setEmail(e.target.value);
              } else {
                setEmailInvalid(true);
              }
            }}
          />
          <FormErrorMessage>
            Bitte verwende eine echte E-Mail.
          </FormErrorMessage>
        </FormControl>
        <FormControl isInvalid={passwordInvalid}>
          <FormLabel htmlFor={"password"}>Passwort</FormLabel>
          <InputGroup>
            <Input
              id={"password"}
              placeholder={"Passwort"}
              type={passwordVisible ? "text" : "password"}
              onChange={(e) => {
                if (
                  // @ts-ignore
                  checkPassword(e.target.value)
                ) {
                  setPasswordInvalid(false);
                  setPassword(e.target.value);
                } else {
                  setPasswordInvalid(true);
                }
              }}
            />
            <InputRightElement
              cursor={"pointer"}
              onClick={() => {
                setPasswordVisible(!passwordVisible);
              }}
            >
              {passwordVisible ? <AiFillEyeInvisible /> : <AiFillEye />}
            </InputRightElement>
          </InputGroup>
          <FormErrorMessage>
            Bitte gib ein Passwort ein, welches aus mindestens 8 Buchstaben,
            einen Großbuchstaben, einen Kleinbuchstaben eine Ziffer und ein
            Sonderzeichen besteht.
          </FormErrorMessage>
        </FormControl>
        <FormControl isInvalid={passwordConfirmInvalid}>
          <FormLabel htmlFor={"passwordConfirm"}>
            Passwort bestätigen
          </FormLabel>
          <InputGroup>
            <Input
              id={"passwordConfirm"}
              placeholder={"Passwort bestätigen"}
              type={passwordConfirmVisible ? "text" : "password"}
              onChange={(e) => {
```

```
                if (e.target.value === password) {
                  setPasswordConfirmInvalid(false);
                  setPasswordConfirm(e.target.value);
                } else {
                  setPasswordConfirmInvalid(true);
                }
              }}
            />
            <InputRightElement
              cursor={"pointer"}
              onClick={() => {
                setPasswordConfirmVisible(!passwordConfirmVisible);
              }}
            >
              {passwordConfirmVisible ? (
                <AiFillEyeInvisible />
              ) : (
                <AiFillEye />
              )}
            </InputRightElement>
          </InputGroup>
          <FormErrorMessage>
            Die Passwörter stimmen nicht überein.
          </FormErrorMessage>
        </FormControl>
        <Button variant={"brand"} type={"submit"} isLoading={loading}>
          Registrieren
        </Button>
      </Flex>
      <Text textAlign={"center"} mt={"1rem"}>
        Wenn auf &amp;apos;Registrieren&amp;apos; klickst, stimmt du der&amp;nbsp;
        <ChakraLink
          href={"/legal/privacy"}
          color={"brand.500"}
          target={"_blank"}
        >
          Datenschutzerklärung
        </ChakraLink>
        &amp;nbsp;zu
      </Text>
    </form>
  </Flex>
 </>
 );
}
```

## packages/components/global/UserAuthReset.tsx

```
/**
 * packages/components/global/UserAuthReset.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 18.09.2022
 *
 */

import * as React from "react";
import {
  Alert,
```

```
  AlertDescription,
  AlertIcon,
  Box,
  Button,
  Flex,
  FormControl,
  FormErrorMessage,
  FormLabel,
  Heading,
  Input,
  Text,
  useColorModeValue,
} from "@chakra-ui/react";
import REST from "@codeup/rest";

export default function UserAuthReset() {
  const [alertVisible, setAlertVisible] = React.useState(false);
  const [alertMessage, setAlertMessage] = React.useState("");
  const [alertType, setAlertType] = React.useState<
    "error" | "info" | "success" | "warning"
  >("error");

  const [emailInvalid, setEmailInvalid] = React.useState(false);
  const [emailError, setEmailError] = React.useState("");

  const [loading, setLoading] = React.useState(false);

  const [forceDisable, setForceDisable] = React.useState(false);

  return (
    <>
      {alertVisible ? (
        <Alert status={alertType} variant={"top-accent"} mb={"1.2rem"}>
          <AlertIcon />
          <AlertDescription>{alertMessage}</AlertDescription>
        </Alert>
      ) : null}
      <Flex direction={"column"} gap={"2rem"}>
        <Flex
          alignItems={"center"}
          justifyContent={"center"}
          direction={"column"}
        >
          <Text textAlign={"center"} mt={".4rem"}>
            Wenn du dein Passwort vergessen hast, kannst du unten deine E-Mail
            Adresse eingeben und wir schicken dir einen Link zum Zurücksetzen.
          </Text>
        </Flex>
        <form
          onSubmit={async (e) => {
            e.preventDefault();
            setLoading(true);
            const res = await REST.Account.reset(
              // @ts-ignore
              e.target.elements.email.value as string
            );

            setLoading(false);
            if (res.status === 200) {
              setAlertVisible(true);
```

```
            setAlertMessage(res.payload.message);
            setAlertType("success");
            setForceDisable(true);
          } else {
            setAlertVisible(true);
            setAlertMessage(res.payload.error);
            setAlertType("error");
          }
        }}
      >
        <FormControl isInvalid={emailInvalid}>
          <FormLabel htmlFor={"email"}>E-Mail</FormLabel>
          <Input id={"email"} placeholder={"E-Mail"} onChange={(e) => {}} />
          <FormErrorMessage>{emailError}</FormErrorMessage>
        </FormControl>
        <Button
          variant={"brand"}
          type={"submit"}
          isLoading={loading}
          disabled={forceDisable || loading}
          w={"100%"}
          mt={"2rem"}
        >
          Link erhalten
        </Button>
      </form>
    </Flex>
  </>
 );
}
```

## packages/components/global/UserAuthSSO.tsx

```
/**
 * packages/components/global/UserAuthSSO.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 22.09.2022
 *
 */

import * as React from "react";
import {
  Button,
  ButtonGroup,
  Grid,
  Heading,
  Stack,
  Text,
} from "@chakra-ui/react";
import { FaDiscord, FaGithub, FaGitlab, FaMicrosoft } from "react-icons/fa";
import { useRouter } from "next/router";

export default function UserAuthSSO() {
  const router = useRouter();

  return (
    <>
      <Stack spacing={4}>
```

```
        <Button
          colorScheme={"gray"}
          onClick={() => {
            router.push("/api/sso/request?type=github");
          }}
          leftIcon={<FaGithub />}
          size={"lg"}
        >
          GitHub
        </Button>
        <Button
          colorScheme={"gray"}
          onClick={() => {
            router.push("/api/sso/request?type=gitlab");
          }}
          leftIcon={<FaGitlab />}
          size={"lg"}
        >
          GitLab
        </Button>
        <Button
          colorScheme={"gray"}
          onClick={() => {
            router.push("/api/sso/request?type=discord");
          }}
          leftIcon={<FaDiscord />}
          size={"lg"}
        >
          Discord
        </Button>
      </Stack>
    </>
  );
}
```

## packages/components/global/VideoPlayer.tsx

```
/**
 * packages/components/global/VideoPlayer.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 16.10.2022
 *
 */

import * as React from "react";
import { useEffect, useRef, useState } from "react";
import Plyr, { usePlyr } from "plyr-react";
import "plyr-react/plyr.css";

export default function VideoPlayer(props: { url: string }) {
  return (
    <>
      <Plyr
        source={{
          type: "video",
          title: "Intro",
          sources: [
            {
```

```
            src: props.url,
            provider: "youtube",
            size: 720,
          },
        ],
        poster: "",
        previewThumbnails: {
          src: "",
        },
        tracks: [],
      }}
    />
  </>
 );
}
```

## packages/components/icons/InCodeIcon.tsx

```
/**
 * packages/components/icons/InCodeIcon.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 22.08.2022
 *
 */

import * as React from "react";

export default function InCodeIcon() {
  return (
    <svg
      width="100%"
      height="100%"
      viewBox="0 0 512 512"
      version="1.1"
      xmlns="http://www.w3.org/2000/svg"
      xmlnsXlink="http://www.w3.org/1999/xlink"
      xmlSpace="preserve"
      style={{
        fillRule: "evenodd",
        clipRule: "evenodd",
        strokeLinejoin: "round",
        strokeMiterlimit: 2,
      }}
    >
      <rect x={0} y={0} width={512} height={512} style={{ fill: "white" }} />
      <g transform="matrix(2.66987,0,0,2.66987,-711.089,-713.981)">
        <text
          x="273.509px"
          y="421.836px"
          style={{
            fontFamily:
              '"ArialRoundedMTBold", "Arial Rounded MT Bold", sans-serif',
            fontSize: "163.533px",
          }}
        >
          IC
        </text>
      </g>
```

```
    </svg>
  );
}
```

## packages/components/ide/ContributorManager.tsx

```
/**
 * packages/components/ide/ContributorManager.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 24.07.2022
 *
 */

import * as React from "react";
import { useEffect, useState } from "react";
import REST from "@codeup/rest";
import {
  Box,
  Button,
  ButtonGroup,
  Checkbox,
  Divider,
  Flex,
  FormControl,
  FormLabel,
  IconButton,
  Input,
  InputGroup,
  InputLeftAddon,
  Modal,
  ModalBody,
  ModalCloseButton,
  ModalContent,
  ModalFooter,
  ModalHeader,
  ModalOverlay,
  Popover,
  PopoverArrow,
  PopoverBody,
  PopoverCloseButton,
  PopoverContent,
  PopoverHeader,
  PopoverTrigger,
  SimpleGrid,
  Stack,
  Tag,
  Text,
  Tooltip,
  useToast,
} from "@chakra-ui/react";
import { FaCogs, FaPlus, FaTrash } from "react-icons/fa";
import { HiMail } from "react-icons/hi";
import { useLoggedIn } from "@codeup/hooks";

export default function ContributorManager(props: {
  isOpen: boolean;
  onOpen: () => void;
  onClose: () => void;
```

```
  projectId: string | undefined;
}) {
  const toast = useToast();

  const { loggedIn, userInfo } = useLoggedIn();

  const [contributors, setContributors] = useState<{
    owner: {
      email: string;
      id: string;
      permissions: string[];
      username: string;
      firstName: string;
      lastName: string;
    };
    contributors: {
      email: string;
      id: string;
      permissions: string[];
      username: string;
      firstName: string;
      lastName: string;
    }[];
  } | null>(null);

  useEffect(() => {
    if (props.projectId && props.projectId !== "0") {
      REST.IDE.getContributors({
        token: localStorage.getItem("token") as string,
        projectId: props.projectId,
      }).then((res) => {
        setContributors({
          owner: res.payload.owner,
          contributors: res.payload.contributors,
        });
      });
    }
  }, [props.projectId]);

  return (
    <>
      <Modal
        isOpen={props.isOpen}
        onClose={props.onClose}
        size={["full", "5xl"]}
        scrollBehavior={"inside"}
      >
        <ModalOverlay />
        <ModalContent>
          <ModalHeader>Mitwirkende verwalten</ModalHeader>
          <ModalCloseButton />
          <ModalBody>
            {props.projectId &&
            props.projectId !== "0" &&
            contributors &&
            contributors.owner &&
            contributors.contributors ? (
              <>
                <Flex justifyContent={"right"} mr={[0, "2rem"]}>
                  <Popover>
```

```
<PopoverTrigger>
  <Button variant={"brand"} leftIcon={<FaPlus />}>
    Hinzufügen
  </Button>
</PopoverTrigger>
<PopoverContent bgColor={"black"}>
  <PopoverArrow bgColor={"black"} />
  <PopoverCloseButton />
  <PopoverHeader fontSize={["xl", "2xl"]}>
    Mitwirkenden hinzufügen
  </PopoverHeader>
  <PopoverBody fontSize={["lg", "xl"]}>
    <form
      onSubmit={async (e) => {
        e.preventDefault();

        const res = await REST.IDE.addContributor({
          token: localStorage.getItem("token") as string,
          projectId: props.projectId as string,
          // @ts-ignore
          email: e.target.elements.email_i.value,
        });
        if (res.status === 200) {
          let c0n: any = contributors;

          c0n.contributors.push(res.payload.contributor);

          setContributors({
            owner: contributors.owner,
            contributors: [...c0n.contributors],
          });
          // @ts-ignore
          e.target.elements.email_i.value = "";
        } else {
          toast({
            title: "Fehler",
            description: res.payload.error,
            position: "top",
            status: "error",
            variant: "solid",
            isClosable: true,
          });
        }
      }}
    >
      <Stack>
        <FormControl>
          <FormLabel>E-Mail</FormLabel>
          <InputGroup>
            <InputLeftAddon>
              <HiMail />
            </InputLeftAddon>
            <Input
              id={"email_i"}
              name="email_i"
              placeholder={"E-Mail"}
            />
          </InputGroup>
        </FormControl>
        <Button type="submit" variant={"brand"}>
```

```
                    Hinzufügen
                  </Button>
                </Stack>
              </form>
            </PopoverBody>
          </PopoverContent>
        </Popover>
      </Flex>
      <Flex
        m={[0, "2rem"]}
        direction={"column"}
        fontSize={["xl", "2xl"]}
      >
        <Flex direction={"row"} mb={"1.2rem"}>
          <Box flex={"20%"} fontWeight={"bold"}>
            Name
          </Box>
          <Box flex={"60%"}></Box>
          <Flex
            flex={"20%"}
            alignItems={"center"}
            justifyContent={"center"}
            fontWeight={"bold"}
          >
            Berechtigungen
          </Flex>
        </Flex>
        <Divider />
        <Flex direction={"row"} mt={"1.2rem"} mb={"1.2rem"}>
          <Box flex={"20%"}>
            {contributors.owner.firstName} 
            {contributors.owner.lastName}
            <br />
            <Text color={"gray.500"} fontSize={["lg", "xl"]}>
              @{contributors.owner.username}
            </Text>
          </Box>
          <Box flex={"60%"}></Box>
          <Flex
            flex={"20%"}
            alignItems={"center"}
            justifyContent={"right"}
          >
            <Tooltip
              label={
                "Diesem Benutzer gehört das Projekt, er hat alle Berechtigungen, inklusive der
Verwaltung von Mitwirkenden"
              }
              hasArrow={true}
            >
              <Tag size={"lg"} variant={"solid"} colorScheme={"red"}>
                Besitzer
              </Tag>
            </Tooltip>
          </Flex>
        </Flex>
        <Divider />
        {contributors.contributors.map((contributor) => {
          return (
            <>
```

```
<Flex direction={"row"} mt={"1.2rem"} mb={"1.2rem"}>
  <Box flex={"20%"}>
    {contributor.firstName} 
    {contributor.lastName}
    <br />
    <Text color={"gray.500"} fontSize={["lg", "xl"]}>
      @{contributor.username}
    </Text>
  </Box>
  <Box flex={"20%"}></Box>
  <Flex
    flex={"60%"}
    justifyContent={"right"}
    direction={["column", "row"]}
    gap={["2rem", 0]}
  >
    <Box flex={"80%"}>
      <SimpleGrid
        minChildWidth={130}
        w={"100%"}
        spacing={3}
      >
        {contributor.permissions.includes("READ") ? (
          <Tag
            size={"lg"}
            variant={"solid"}
            colorScheme={"teal"}
          >
            Lesen
          </Tag>
        ) : null}
        {contributor.permissions.includes("WRITE") ? (
          <Tag
            size={"lg"}
            variant={"solid"}
            colorScheme={"teal"}
          >
            Schreiben
          </Tag>
        ) : null}
        {contributor.permissions.includes("DELETE") ? (
          <Tag
            size={"lg"}
            variant={"solid"}
            colorScheme={"teal"}
          >
            Löschen
          </Tag>
        ) : null}
        {contributor.permissions.includes(
          "DUPLICATE"
        ) ? (
          <Tag
            size={"lg"}
            variant={"solid"}
            colorScheme={"teal"}
          >
            Klonen
          </Tag>
        ) : null}
```

```
    {contributor.permissions.includes("RENAME") ? (
      <Tag
        size={"lg"}
        variant={"solid"}
        colorScheme={"teal"}
      >
        Umbenennen
      </Tag>
    ) : null}
    {contributor.permissions.includes("EXPORT") ? (
      <Tag
        size={"lg"}
        variant={"solid"}
        colorScheme={"teal"}
      >
        Exportieren
      </Tag>
    ) : null}
    {contributor.permissions.includes(
      "MANAGE-CONTRIBUTORS"
    ) ? (
      <Tag
        size={"lg"}
        variant={"solid"}
        colorScheme={"teal"}
      >
        Mitwirkende verwalten
      </Tag>
    ) : null}
  </SimpleGrid>
</Box>
<Flex
  flex={"20%"}
  justifyContent={"center"}
  alignItems={"center"}
>
  <ButtonGroup>
    <Popover>
      <PopoverTrigger>
        <IconButton
          aria-label={"Mitwirkenden entfernen"}
        >
          <FaTrash />
        </IconButton>
      </PopoverTrigger>
      <PopoverContent bgColor={"gray.800"}>
        <PopoverArrow bgColor={"gray.800"} />
        <PopoverCloseButton />
        <PopoverHeader>
          Mitwirkenden entfernen
        </PopoverHeader>
        <PopoverBody>
          <Stack>
            <Text fontSize={"lg"}>
              Willst du den mitwirkenden 
              {contributor.firstName} 
              {contributor.lastName} (
              {contributor.username}) wirklich
              entfernen?
            </Text>
```

```
        <Button
          variant={"solid"}
          colorScheme={"red"}
          onClick={async () => {
            const res =
              await REST.IDE.removeContributor({
                token: localStorage.getItem(
                  "token"
                ) as string,
                projectId:
                  props.projectId as string,
                email: contributor.email,
              });

            if (res.status === 200) {
              let c0n: any = contributors;

              c0n = c0n.contributors.filter(
                (c: any) =>
                  c.email !== contributor.email
              );

              setContributors({
                owner: contributors.owner,
                contributors: [...c0n],
              });
            } else {
              toast({
                title: "Fehler",
                description: res.payload.error,
                position: "top",
                status: "error",
                variant: "solid",
                isClosable: true,
              });
            }
          }}
        >
          Entfernen
        </Button>
      </Stack>
    </PopoverBody>
  </PopoverContent>
</Popover>

{userInfo.id !== contributor.id ? (
  <>
    <Popover>
      <PopoverTrigger>
        <IconButton
          aria-label={
            "Berechtigungen verwalten"
          }
        >
          <FaCogs />
        </IconButton>
      </PopoverTrigger>
      <PopoverContent bgColor={"gray.800"}>
        <PopoverArrow bgColor={"gray.800"} />
        <PopoverCloseButton />
```

```
<PopoverHeader>
  Berechtigungen ändern
</PopoverHeader>
<PopoverBody>
  <Stack>
    <Checkbox
      defaultChecked={contributor.permissions.includes(
        "READ"
      )}
      disabled={true}
      colorScheme={"brand"}
    >
      Lesen
    </Checkbox>
    <Checkbox
      defaultChecked={contributor.permissions.includes(
        "WRITE"
      )}
      id={
        "cb-upd-perms-" +
        contributor.id +
        "-write"
      }
      colorScheme={"brand"}
    >
      Schreiben
    </Checkbox>
    <Checkbox
      defaultChecked={contributor.permissions.includes(
        "DELETE"
      )}
      id={
        "cb-upd-perms-" +
        contributor.id +
        "-delete"
      }
      colorScheme={"brand"}
    >
      Löschen
    </Checkbox>
    <Checkbox
      defaultChecked={contributor.permissions.includes(
        "DUPLICATE"
      )}
      id={
        "cb-upd-perms-" +
        contributor.id +
        "-duplicate"
      }
      colorScheme={"brand"}
    >
      Klonen
    </Checkbox>
    <Checkbox
      defaultChecked={contributor.permissions.includes(
        "RENAME"
      )}
      id={
        "cb-upd-perms-" +
        contributor.id +
```

```
    "-rename"
  }
  colorScheme={"brand"}
>
  Umbenennen
</Checkbox>
<Checkbox
  defaultChecked={contributor.permissions.includes(
    "EXPORT"
  )}
  id={
    "cb-upd-perms-" +
    contributor.id +
    "-export"
  }
  colorScheme={"brand"}
>
  Exportieren
</Checkbox>
<Checkbox
  defaultChecked={contributor.permissions.includes(
    "MANAGE-CONTRIBUTORS"
  )}
  id={
    "cb-upd-perms-" +
    contributor.id +
    "-manage_contributors"
  }
  colorScheme={"brand"}
>
  Mitwirkende verwalten
</Checkbox>
<Button
  variant={"brand"}
  onClick={async () => {
    const res =
      await REST.IDE.overwriteContributorPermissions(
        {
          token:
            localStorage.getItem(
              "token"
            ) as string,
          projectId:
            props.projectId as string,
          contributorEmail:
            contributor.email,
          permissions: [
            "READ",
            (
              document.getElementById(
                "cb-upd-perms-" +
                  contributor.id +
                  "-write"
              ) as HTMLInputElement
            ).checked
              ? "WRITE"
              : null,
            (
              document.getElementById(
                "cb-upd-perms-" +
```

```
            contributor.id +
            "-delete"
          ) as HTMLInputElement
        ).checked
          ? "DELETE"
          : null,
        (
          document.getElementById(
            "cb-upd-perms-" +
              contributor.id +
              "-duplicate"
          ) as HTMLInputElement
        ).checked
          ? "DUPLICATE"
          : null,
        (
          document.getElementById(
            "cb-upd-perms-" +
              contributor.id +
              "-rename"
          ) as HTMLInputElement
        ).checked
          ? "RENAME"
          : null,
        (
          document.getElementById(
            "cb-upd-perms-" +
              contributor.id +
              "-export"
          ) as HTMLInputElement
        ).checked
          ? "EXPORT"
          : null,
        (
          document.getElementById(
            "cb-upd-perms-" +
              contributor.id +
              "-manage_contributors"
          ) as HTMLInputElement
        ).checked
          ? "MANAGE-CONTRIBUTORS"
          : null,
      ],
    }
  );
if (res.status === 200) {
  let c0n: any = contributors;

  c0n.contributors.find(
    (c: any) =>
      c.id === contributor.id
  ).permissions =
    res.payload.contributor.permissions;

  setContributors({
    owner: contributors.owner,
    contributors: [
      ...c0n.contributors,
    ],
  });
```

```
                                    } else {
                                      toast({
                                        title: "Fehler",
                                        description:
                                          res.payload.error,
                                        position: "top",
                                        status: "error",
                                        variant: "solid",
                                        isClosable: true,
                                      });
                                    }
                                  }}
                                >
                                  Speichern
                                </Button>
                              </Stack>
                            </PopoverBody>
                          </PopoverContent>
                        </Popover>
                      </>
                    ) : null}
                  </ButtonGroup>
                </Flex>
              </Flex>
            </Flex>
          </>
        );
      })}
    </Flex>
  </>
) : null}
</ModalBody>

<ModalFooter>
  <Button variant="brand" mr={3} onClick={props.onClose}>
    Schließen
  </Button>
</ModalFooter>
</ModalContent>
</Modal>
</>
);
}
```

## packages/components/ide/CreateProjectMenu.tsx

```
/**
 * packages/components/ide/CreateProjectMenu.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 22.08.2022
 *
 */

import * as React from "react";
import {
  Box,
  Button,
  Divider,
```

```
  Flex,
  FormControl,
  FormLabel,
  Heading,
  IconButton,
  Input,
  InputGroup,
  InputLeftAddon,
  Link,
  Modal,
  ModalBody,
  ModalContent,
  ModalFooter,
  ModalHeader,
  ModalOverlay,
  Progress,
  Stack,
  Text,
  Tooltip,
  useDisclosure,
  useToast,
} from "@chakra-ui/react";
import { useId, useState } from "react";
import * as generate from "project-name-generator";
import { FaFileImport, FaPlus } from "react-icons/fa";
import REST from "@codeup/rest";
import { HiTemplate } from "react-icons/hi";
import { BiRename } from "react-icons/bi";
import { FiRotateCcw } from "react-icons/fi";
import TemplatePicker from "./TemplatePicker";
import { useRouter } from "next/router";
import { CHeading } from "../global/CHeading";

export default function CreateProjectMenu() {
  const toast = useToast();
  const nameId = useId();

  const [loading, setLoading] = useState<boolean>(false);

  const [section, setSection] = useState("menu");

  function Card(props: {
    icon: any;
    title: string;
    section?: string;
    onClick?: () => void;
  }) {
    return (
      <>
        <Box
          flex={"33%"}
          backgroundColor={"gray.700"}
          shadow={"xl"}
          rounded={"xl"}
          h={"100%"}
          cursor={"pointer"}
          onClick={() => {
            if (props.section) {
              setSection(props.section);
            }
```

```
        if (props.onClick) {
          props.onClick();
        }
      }}
    >
      <Flex direction={"row"} h={"100%"} w={"100%"}>
        <Flex
          flex={"30%"}
          fontSize={"6xl"}
          alignItems={"center"}
          justifyContent={"center"}
          h={"100%"}
          w={"100%"}
          color={"brand.500"}
        >
          {props.icon}
        </Flex>
        <Flex
          flex={"70%"}
          pl={".2rem"}
          justifyContent={"center"}
          h={"100%"}
          w={"100%"}
          direction={"column"}
        >
          <CHeading size={"xl"}>{props.title}</CHeading>
        </Flex>
      </Flex>
    </Box>
  </>
 );
}

const importFileInputId = useId();

const [importProgress, setImportProgress] = useState(0);
const {
  isOpen: isImportOpen,
  onOpen: onImportOpen,
  onClose: onImportClose,
} = useDisclosure();

const [importProgressMax, setImportProgressMax] = useState(0);
const [importProgressCurrent, setImportProgressCurrent] = useState("");

const [importFinished, setImportFinished] = useState(false);

const [importedProjectId, setImportedProjectId] = useState("");

const router = useRouter();

const [suggestedProjectName, setSuggestedProjectName] = useState(
  generate.default().dashed
);

return (
  <>
    <Flex alignItems={"center"} justifyContent={"center"} mt={"1.2rem"}>
      <Box
        w={["100%", "85%"]}
```

```
          minW={["100%", "85%"]}
          backgroundColor={"gray.900"}
          p={"1.2rem"}
          rounded={"xl"}
        >
          <Box textAlign={"center"}>
            <CHeading size={"xl"}>Projekt erstellen</CHeading>
          </Box>
          <Divider mt={"1.2rem"} />
          {section === "menu" ? (
            <>
              <Box mt={"1.2rem"}>
                <Flex
                  direction={["column", "row"]}
                  gap={["1.2rem", "2rem"]}
                  h={["30vh", "15vh"]}
                >
                  <Card
                    icon={<FaPlus />}
                    title={"Leeres Projekt"}
                    section={"empty"}
                  />
                  <Card
                    icon={<FaFileImport />}
                    title={"Importieren"}
                    onClick={async () => {
                      const element = document.getElementById(
                        importFileInputId
                      ) as HTMLInputElement;

                      element.click();

                      element.onchange = async (e) => {
                        const target = e.target as HTMLInputElement;

                        if (target && target.files) {
                          if (target.files.length > 0) {
                            const file = target.files.item(0);

                            if (file) {
                              try {
                                const data = JSON.parse(await file.text());
                                if (
                                  !(
                                    data.codeUp &&
                                    data.codeUp.version &&
                                    data.codeUp.version === "1.0"
                                  )
                                ) {
                                  toast({
                                    status: "error",
                                    position: "top",
                                    variant: "solid",
                                    isClosable: true,
                                    duration: 3000,
                                    title: "Fehler",
                                    description:
                                      "Die Datei-Version (codeUp.version) ist nicht 1.0. Die Datei kann nicht
gelesen werden.",
                                  });
```

```
  return;
}

if (!(data.files && data.name)) {
  toast({
    status: "error",
    position: "top",
    variant: "solid",
    isClosable: true,
    duration: 3000,
    title: "Fehler",
    description:
      "Die Datei beinhaltet keine Projekt Daten.",
  });
  return;
}

onImportOpen();

const cRes = await REST.IDE.createProject({
  token: localStorage.getItem(
    "token"
  ) as string,
  name:
    data.name +
    " - Impo. " +
    new Date().toLocaleTimeString(),
});

await REST.IDE.deleteFile({
  token: localStorage.getItem(
    "token"
  ) as string,
  projectId: cRes.payload.project._id,
  path: "/README.MD",
});

setImportProgressMax(data.files.length);
setImportProgress(0);

for (const [
  index,
  file,
] of data.files.entries()) {
  setImportProgressCurrent(file.path);

  await REST.IDE.saveFile({
    token: localStorage.getItem(
      "token"
    ) as string,
    projectId: cRes.payload.project._id,
    path: file.path,
    content: file.content,
  });

  setImportProgress(index + 1);
}

setImportProgress(0);
setImportProgressCurrent("");
```

```
                  setImportProgressMax(1);
                  setImportedProjectId(cRes.payload.project._id);

                  setImportFinished(true);
                } catch (e: any) {
                  toast({
                    status: "error",
                    position: "top",
                    variant: "solid",
                    isClosable: true,
                    duration: 3000,
                    title: "Fehler",
                    description:
                      "Die Datei konnte nicht gelesen werden: " +
                      e.message,
                  });
                }
              }
            }
          }
        };
      }}
    />
    <Card
      icon={<HiTemplate />}
      title={"Vorlage"}
      section={"templates"}
    />
  </Flex>
 </Box>
</>
) : null}
{section === "empty" ? (
 <>
  <form
   onSubmit={async (e) => {
    e.preventDefault();

    setLoading(true);

    const res = await REST.IDE.createProject({
     token: localStorage.getItem("token") as string,
     // @ts-ignore
     name: e.target.elements[nameId].value,
    });

    if (res.status === 200) {
     toast({
      title: "Erfolgreich",
      description: "Das Projekt wurde erstellt",
      status: "success",
      variant: "solid",
      position: "top",
      isClosable: true,
     });
     setLoading(false);
     router.push("/ide?component=open");
    } else {
     setLoading(false);
     toast({
```

```
        title: "Fehler",
        description: res.payload.error,
        status: "error",
        variant: "solid",
        position: "top",
        isClosable: true,
      });
    }
  }}
  id={"c_form"}
>
  <FormControl mt={"2rem"} isRequired>
    <FormLabel>Projekt Name</FormLabel>
    <InputGroup>
      <InputLeftAddon>
        <BiRename />
      </InputLeftAddon>
      <Input
        disabled={loading}
        type={"text"}
        id={nameId}
        placeholder={"Projekt Name"}
      />
    </InputGroup>
  </FormControl>
  <Text mt={"1.2rem"}>
    Wie wäre es mit 
    <Link
      color={"brand.500"}
      onClick={() => {
        (
          document.getElementById(nameId) as HTMLInputElement
        ).value = suggestedProjectName;
        (
          document.getElementById("c_for_sb") as HTMLButtonElement
        ).click();
      }}
    >
      {suggestedProjectName}
    </Link>
    <Tooltip label={"Neuer Vorschlag"} hasArrow>
      <IconButton
        ml={".6rem"}
        aria-label={"regenerate"}
        icon={<FiRotateCcw />}
        variant={"ghost"}
        onClick={() => {
          setSuggestedProjectName(generate.default().dashed);
        }}
      />
    </Tooltip>
  </Text>
  <FormControl mt={"1.2rem"}>
    <Button
      isLoading={loading}
      id="c_for_sb"
      type={"submit"}
      variant={"brand"}
    >
      Erstellen
```

```
            </Button>
          </FormControl>
        </form>
      </>
    ) : null}
    {section === "templates" ? (
      <>
        <TemplatePicker />
      </>
    ) : null}
  </Box>
</Flex>
<input
  type={"file"}
  id={importFileInputId}
  style={{ visibility: "hidden" }}
  accept={".cupp,.codeup"}
/>

<Modal isOpen={isImportOpen} onClose={onImportClose} isCentered>
  <ModalOverlay />
  <ModalContent>
    <ModalHeader>
      {importFinished ? "Import abgeschlossen" : "Importieren"}
    </ModalHeader>
    <ModalBody>
      {importFinished ? (
        <>
          <Text color={"gray.400"}>
            Der Import wurde abgeschlossen! Du kannst das Projekt nun
            bearbeiten.
          </Text>
          <Stack direction={"column"} mt={"2rem"} gap={".4rem"}>
            <Button
              variant={"solid"}
              colorScheme={"blue"}
              onClick={() => {
                router.push(
                  "/ide/project/" +
                    importedProjectId +
                    "#editor-wrapper-flex"
                );
              }}
            >
              Bearbeiten
            </Button>
            <Button
              variant={"outline"}
              onClick={() => {
                setImportedProjectId("0");
                onImportClose();
              }}
            >
              Später
            </Button>
          </Stack>
        </>
      ) : (
        <>
          <Progress
```

```
                size={"lg"}
                max={importProgressMax}
                value={importProgress}
                w={"100%"}
                rounded={"xl"}
                hasStripe
              />
              <Text color={"gray.500"}>
                Importiere Datei <code>{importProgressCurrent}</code>
              </Text>
            </>
          )}
        </ModalBody>
        <ModalFooter></ModalFooter>
      </ModalContent>
    </Modal>
  </>
 );
}
```

## packages/components/ide/FileManager.tsx

```
/**
 * packages/components/ide/FileManager.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 02.08.2022
 *
 */

import * as React from "react";
import { useEffect, useState } from "react";
import REST from "@codeup/rest";
import pathsToStructure from "@codeup/util/src/pathsToStructure";
import {
  Accordion,
  AccordionButton,
  AccordionIcon,
  AccordionItem,
  AccordionPanel,
  Box,
  Button,
  ButtonGroup,
  Drawer,
  DrawerBody,
  DrawerCloseButton,
  DrawerContent,
  DrawerFooter,
  DrawerHeader,
  DrawerOverlay,
  Flex,
  FormControl,
  FormLabel,
  IconButton,
  Input,
  Link,
  Popover,
  PopoverArrow,
  PopoverBody,
```

```
  PopoverCloseButton,
  PopoverContent,
  PopoverHeader,
  PopoverTrigger,
  Select,
  Stack,
  Text,
  Tooltip,
  useToast,
} from "@chakra-ui/react";
import { FiDownload, FiFilePlus, FiTrash2, FiUpload } from "react-icons/fi";
import {
  SiCss3,
  SiHtml5,
  SiJavascript,
  SiMarkdown,
  SiSass,
  SiTypescript,
} from "react-icons/si";
import { FaFile, FaFolder, FaHome } from "react-icons/fa";
import { BiRename } from "react-icons/bi";
import { lookup } from "mime-types";
import { useHover, useLoggedIn } from "@codeup/hooks";
import InCodeIcon from "../icons/InCodeIcon";

export default function FileManager(props: {
  onFileMenuClose: () => void;
  isFileMenuOpen: boolean;
  permissions: string[];
  project: {
    _id: string;
    owner: string;
    name: string;
    contributors: {
      user: string;
      permissions: string[];
    }[];
    files: {
      path: string;
      content: string;
    }[];
  } | null;
  setProject: (project: any) => void;
  currentFile: {
    path: string;
    content: string;
  } | null;
  setCurrentFile: (newFile: any) => void;
  setSettingsOpen: (bool: boolean) => void;
  editorRef: any;
  monaco: any;
}) {
  const toast = useToast();

  const LANGUAGES: {
    [key: string]: {
      icon: any;
      language: string;
    };
  } = {
```

```
  md: {
    language: "markdown",
    icon: <SiMarkdown />,
  },
  js: {
    language: "javascript",
    icon: <SiJavascript />,
  },
  html: {
    language: "html",
    icon: <SiHtml5 />,
  },
  css: {
    language: "css",
    icon: <SiCss3 />,
  },
  ts: {
    language: "typescript",
    icon: <SiTypescript />,
  },
  scss: {
    language: "scss",
    icon: <SiSass />,
  },
  sass: {
    language: "sass",
    icon: <SiSass />,
  },
  ic: {
    language: "codeup-incode",
    icon: <InCodeIcon />,
  },
  _default: {
    language: "plain",
    icon: <FaFile />,
  },
};

const [structure, setStructure] = useState<any>([]);

useEffect(() => {
  if (props.project) {
    let paths = [];
    let contents: { [path: string]: string } = {};
    for (const file of props.project.files) {
      paths.push(file.path);
      contents[file.path] = file.content;
    }

    const x = pathsToStructure(paths, contents);
    setStructure(x);
  }
}, [props.project]);

const { loggedIn, userInfo } = useLoggedIn();

return (
  <>
    <Drawer
      isOpen={props.isFileMenuOpen}
```

```
  placement="left"
  onClose={props.onFileMenuClose}
  size={["full", "lg"]}
>
  <DrawerOverlay />
  <DrawerContent bgColor={"gray.900"}>
    <DrawerCloseButton />
    <DrawerHeader>Dateien</DrawerHeader>

    <DrawerBody>
      <Flex direction={"column"} w="100%">
        <Flex direction="row" w="100%">
          <Box flex="90%"></Box>
          <Flex flex="10%" direction={"row"}>
            {props.permissions.includes("WRITE") ||
            props.permissions.includes("OWNER") ? (
              <>
                <ButtonGroup>
                  <Popover>
                    <PopoverTrigger>
                      <IconButton
                        aria-label="Neue Datei"
                        size="lg"
                        fontSize={"2xl"}
                        icon={<FiFilePlus />}
                        variant="ghost"
                      />
                    </PopoverTrigger>
                    <PopoverContent bgColor="black">
                      <PopoverArrow bgColor="black" />
                      <PopoverCloseButton />
                      <PopoverHeader>Neue Datei</PopoverHeader>
                      <PopoverBody>
                        <form
                          onSubmit={async (e) => {
                            e.preventDefault();
                            let path =
                              // @ts-ignore
                              e.target.elements.newfilepath.value.toLowerCase();

                            let extension =
                              // @ts-ignore
                              e.target.elements.nf_extension
                                .selectedOptions[0].value;

                            if (!path.startsWith("/")) {
                              path = "/" + path;
                            }

                            path = path.replace(/\/\/g, "/");

                            path = path += extension;

                            if (
                              props.project?.files.find(
                                (f) => f.path === path
                              )
                            ) {
                              toast({
                                status: "error",
```

```
        variant: "solid",
        position: "top",
        title: "Fehler",
        description:
          "Diese Datei existiert bereits.",
        isClosable: true,
        duration: 4000,
      });
      return;
    }

    const res = await REST.IDE.saveFile({
      token: localStorage.getItem(
        "token"
      ) as string,
      path: path,
      content: "Happy Coding!",
      projectId: props.project?._id as string,
    });

    if (res.status === 200) {
      let f = props.project?.files;
      f?.push({
        path,
        content: "",
      });
      if (props.project && f) {
        props.setProject({
          _id: props.project?._id,
          contributors:
            props.project?.contributors,
          owner: props.project?.owner,
          name: props.project?.name,
          files: f,
        });
      }
    } else {
      toast({
        status: "error",
        variant: "solid",
        position: "top",
        title: "Fehler",
        description: res.payload.error,
        isClosable: true,
        duration: 4000,
      });
    }
  }}
>
  <FormControl isRequired>
    <FormLabel>Pfad</FormLabel>
    <Input
      type="text"
      name="newfilepath"
      id="newfilepath"
      placeholder="Pfad"
    />
  </FormControl>
  <FormControl isRequired mt={".6rem"}>
    <FormLabel>Dateiendung</FormLabel>
```

```
            <Select name="nf_extension" id="nf_extension">
              <FileExtensionOptions />
            </Select>
          </FormControl>
          <Button type="submit" mt=".6rem">
            Erstellen
          </Button>
        </form>
      </PopoverBody>
    </PopoverContent>
  </Popover>
</ButtonGroup>
<Tooltip label={"Datei hochladen"} hasArrow>
  <IconButton
    aria-label="Datei hochladen"
    size="lg"
    fontSize={"2xl"}
    icon={<FiUpload />}
    variant="ghost"
    onClick={() => {
      const f = document.createElement("input");
      f.type = "file";
      f.style.display = "none";

      f.onchange = async () => {
        if (f.files && f.files.length > 0) {
          const file = f.files.item(0);
          if (file) {
            if (file.type.startsWith("text/")) {
              const content = await file.text();

              await REST.IDE.saveFile({
                token: localStorage.getItem(
                  "token"
                ) as string,
                path: "/" + file.name,
                content: content,
                projectId: props.project?._id as string,
              });

              const x = await REST.IDE.getProject({
                token: localStorage.getItem(
                  "token"
                ) as string,
                projectId: props.project?._id as string,
              });

              props.setProject(x.payload.project);
            } else {
              const FR = new FileReader();

              FR.readAsDataURL(file);

              FR.onload = async () => {
                await REST.IDE.saveFile({
                  token: localStorage.getItem(
                    "token"
                  ) as string,
                  path: "/" + file.name,
                  content: FR.result as string,
```

```
                projectId: props.project?._id as string,
              });

              const x = await REST.IDE.getProject({
                token: localStorage.getItem(
                  "token"
                ) as string,
                projectId: props.project?._id as string,
              });
              props.setProject(x.payload.project);
            };
            return;
          }
        }
      }
    };

    document.body.appendChild(f);
    f.click();
    document.body.removeChild(f);
  }}
 />
 </Tooltip>
 </>
 ) : null}
 </Flex>
</Flex>
<Flex direction="column" maxH="90vh" overflow="auto">
 <Accordion
  allowMultiple
  allowToggle
  borderColor={"gray.900"}
  defaultIndex={[0]}
 >
  <AccordionItem borderWidth={0}>
   <h2>
    <AccordionButton>
     <Box flex="1" textAlign="left">
      <Flex direction="row" alignItems="center">
       <Box flex="5%" fontSize="xl">
        <FaHome />
       </Box>
       <Box flex="2%"></Box>
       <Box flex="83%">
        /@{userInfo.username}/{props.project?.name}
       </Box>
      </Flex>
     </Box>
     <AccordionIcon />
    </AccordionButton>
   </h2>
   <AccordionPanel>
    {getFileComponentTree(
     structure,
     LANGUAGES,
     props.permissions,
     props.currentFile,
     props.setCurrentFile,
     props.project,
     props.setProject,
```

```
                    props.onFileMenuClose,
                    props.editorRef,
                    props.monaco,
                    props.setSettingsOpen
                  )}
                </AccordionPanel>
              </AccordionItem>
            </Accordion>
            {(props.project?.files?.length as number) < 1 ? (
              <>
                Dieses Projekt beinhaltet keine Dateien. Erstelle die erste
                Datei über die Schaltfläche oben rechts.
              </>
            ) : null}
          </Flex>
        </Flex>
      </DrawerBody>

      <DrawerFooter>
        <Button variant="outline" mr={3} onClick={props.onFileMenuClose}>
          Schließen
        </Button>
      </DrawerFooter>
    </DrawerContent>
  </Drawer>
 </>
 );
}

function getFileComponentTree(
 structure: any,
 LANGUAGES: any,
 permissions: string[],
 currentFile: any,
 setCurrentFile: (cf: any) => void,
 project: any,
 setProject: (cf: any) => void,
 onFileMenuClose: () => void,
 editorRef: any,
 monaco: any,
 setSettingsOpen: (b: boolean) => void
) {
 if (structure && structure[0] && structure[0].children) {
  const r = [<></>];

  structure[0].children.map((x: any) => {
   if (x.children.length > 0) {
    r.push(
     <>
      <Accordion allowMultiple allowToggle>
       <AccordionItem borderWidth={"0"} borderColor={"gray.900"}>
        <h2>
         <AccordionButton>
          <Box flex={"1"} textAlign={"left"}>
           <Flex
            direction="row"
            alignItems="center"
            justifyContent={"left"}
           >
            <Box flex="5%" fontSize="xl">
```

```
                    <FaFolder />
                  </Box>
                  <Box flex="2%"></Box>
                  <Box flex="83%">{x.name}</Box>
                </Flex>
              </Box>
              <AccordionIcon />
            </AccordionButton>
          </h2>
          <AccordionPanel pb={4}>
            {x.children.map((y: any) => {
              return getFileComponentTree(
                y,
                LANGUAGES,
                permissions,
                currentFile,
                setCurrentFile,
                project,
                setProject,
                onFileMenuClose,
                editorRef,
                monaco,
                setSettingsOpen
              );
            })}
          </AccordionPanel>
        </AccordionItem>
      </Accordion>
    </>
  );
 } else {
  r.push(
    <FileEntry
      structure={x}
      LANGUAGES={LANGUAGES}
      permissions={permissions}
      currentFile={currentFile}
      setCurrentFile={setCurrentFile}
      project={project}
      setProject={setProject}
      onFileMenuClose={onFileMenuClose}
      editorRef={editorRef}
      monaco={monaco}
      setSettingsOpen={setSettingsOpen}
    />
  );
 }
});

 return r;
} else {
 if (structure.name && structure.children.length < 1) {
  return (
    <FileEntry
      structure={structure}
      LANGUAGES={LANGUAGES}
      permissions={permissions}
      currentFile={currentFile}
      setCurrentFile={setCurrentFile}
      project={project}
```

```
          setProject={setProject}
          onFileMenuClose={onFileMenuClose}
          editorRef={editorRef}
          monaco={monaco}
          setSettingsOpen={setSettingsOpen}
        />
      );
    } else {
      return (
        <>
          <Accordion allowMultiple allowToggle>
            <AccordionItem borderWidth={"0"} borderColor={"gray.900"}>
              <h2>
                <AccordionButton>
                  <Box flex={"1"} textAlign={"left"}>
                    <Flex direction="row" alignItems="center">
                      <Box flex="5%" fontSize="xl">
                        <FaFolder />
                      </Box>
                      <Box flex="2%"></Box>
                      <Box flex="83%">{structure.name}</Box>
                    </Flex>
                  </Box>
                  <AccordionIcon />
                </AccordionButton>
              </h2>
              <AccordionPanel pb={4}>
                {structure.children && structure.children.map
                  ? structure.children.map((y: any) => {
                      return getFileComponentTree(
                        y,
                        LANGUAGES,
                        permissions,
                        currentFile,
                        setCurrentFile,
                        project,
                        setProject,
                        onFileMenuClose,
                        editorRef,
                        monaco,
                        setSettingsOpen
                      );
                    })
                  : null}
              </AccordionPanel>
            </AccordionItem>
          </Accordion>
        </>
      );
    }
  }
}

function FileEntry(props: {
  structure: {
    name: string;
    children?: any[];
    fullPath: string;
    contents: string;
  };
```

```
  LANGUAGES: any;
  permissions: string[];
  currentFile: any;
  setCurrentFile: (cf: any) => void;
  project: any;
  setProject: (cf: any) => void;
  editorRef: any;
  monaco: any;
  onFileMenuClose: () => void;
  setSettingsOpen: (b: boolean) => void;
}) {
  const [hoverRef, isHovering] = useHover();
  const toast = useToast();

  return (
    <>
      <Flex
        direction="row"
        alignItems="center"
        // @ts-ignore
        ref={hoverRef}
      >
        <Box flex="5%" fontSize="xl">
          {props.LANGUAGES[props.structure.name.split(".").pop() as string]
            ? props.LANGUAGES[props.structure.name.split(".").pop() as string]
                .icon
            : props.LANGUAGES["_default"].icon}
        </Box>
        <Box flex="2%"></Box>
        <Box
          flex="83%"
          onClick={() => {
            props.setSettingsOpen(false);

            props.setCurrentFile({
              path: props.structure.fullPath,
              content: props.structure.contents,
            });
            // @ts-ignore
            props.editorRef?.current?.setValue(props.structure.contents);

            props.onFileMenuClose();

            // @ts-ignore
            props.monaco.editor.setModelLanguage(
              // @ts-ignore
              props.editorRef?.current?.getModel(),
              props.LANGUAGES[
                props.structure.fullPath.split(".").pop() as string
              ]
                ? props.LANGUAGES[
                    props.structure.fullPath.split(".").pop() as string
                  ].language
                : props.LANGUAGES["_default"].language
            );
          }}
        >
          <Stack>
            <Link w={"100%"}>{props.structure.name}</Link>
          </Stack>
```

```
</Box>
<Box flex="10%" visibility={isHovering ? "visible" : "hidden"}>
  <ButtonGroup>
    {props.permissions.includes("WRITE") ||
    props.permissions.includes("OWNER") ? (
      <>
        <Popover>
          <PopoverTrigger>
            <IconButton
              aria-label="Löschen"
              size="lg"
              fontSize={"xl"}
              icon={<FiTrash2 />}
              variant="ghost"
            />
          </PopoverTrigger>
          <PopoverContent bgColor={"black"}>
            <PopoverArrow bgColor={"black"} />
            <PopoverCloseButton />
            <PopoverHeader>Datei Löschen</PopoverHeader>
            <PopoverBody>
              <Stack>
                <Text>
                  Bist du dir sicher, dass du die Datei 
                  <code>{props.structure.fullPath}</code> 
                  <b>UNWIEDERFLICH</b> löschen möchtest?
                </Text>
                <Button
                  variant="solid"
                  colorScheme="red"
                  mt=".4rem"
                  onClick={async () => {
                    if (props.project.files.length === 1) {
                      toast({
                        title: "Fehler",
                        description:
                          "Du kannst die letzte Datei nicht löschen.",
                        status: "error",
                        duration: 1500,
                        isClosable: true,
                      });
                      return;
                    }

                    if (
                      props.currentFile?.path ===
                      props.structure.fullPath
                    ) {
                      props.setCurrentFile(null);
                    }

                    const res = await REST.IDE.deleteFile({
                      token: localStorage.getItem("token") as string,
                      projectId: props.project?._id as string,
                      path: props.structure.fullPath,
                    });

                    const rPro = await REST.IDE.getProject({
                      token: localStorage.getItem("token") as string,
                      projectId: props.project?._id as string,
```

```
            });

            props.setProject(rPro.payload.project);
          }}
        >
          Löschen
        </Button>
      </Stack>
    </PopoverBody>
  </PopoverContent>
</Popover>
<Popover>
  <PopoverTrigger>
    <IconButton
      aria-label="Umbenennen"
      size="lg"
      fontSize={"xl"}
      icon={<BiRename />}
      variant="ghost"
      title={"Umbenennen/Verschieben"}
    />
  </PopoverTrigger>
  <PopoverContent bgColor={"black"}>
    <PopoverArrow bgColor={"black"} />
    <PopoverCloseButton />
    <PopoverHeader>Datei umbenennen</PopoverHeader>
    <PopoverBody>
      <form
        onSubmit={async (e) => {
          e.preventDefault();

          let path =
            // @ts-ignore
            e.target.elements.renamenewpath.value;

          let extension =
            // @ts-ignore
            e.target.elements.rf_extension.selectedOptions[0]
              .value;

          if (!path.startsWith("/")) {
            path = "/" + path;
          }

          path = path.replace(/\/\/g, "/");

          path = path += extension;

          const rDel = await REST.IDE.deleteFile({
            token: localStorage.getItem("token") as string,
            path: props.structure.fullPath,
            projectId: props.project?._id as string,
          });

          const rSav = await REST.IDE.saveFile({
            token: localStorage.getItem("token") as string,
            path,
            content:
              props.structure.contents.length > 0
                ? props.structure.contents
```

```
                    : "Pfad: " + props.structure.fullPath,
                  projectId: props.project?._id as string,
                });

                const rPro = await REST.IDE.getProject({
                  token: localStorage.getItem("token") as string,
                  projectId: props.project?._id as string,
                });

                props.setProject(rPro.payload.project);
              }}
            >
              <Stack>
                <FormControl>
                  <FormLabel>Neuer Pfad</FormLabel>
                  <Input
                    type={"text"}
                    defaultValue={
                      props.structure.fullPath
                        ?.split(".")[0]
                        .split("/")[1]
                    }
                    id={"renamenewpath"}
                    name={"renamenewpath"}
                    placeholder={"Pfad"}
                  />
                </FormControl>
                <FormControl isRequired mt={".6rem"}>
                  <FormLabel>Dateiendung</FormLabel>
                  <Select name={"rf_extension"} id={"rf_extension"}>
                    <FileExtensionOptions />
                  </Select>
                </FormControl>
                <Button mt={".6rem"} type={"submit"}>
                  Umbenennen
                </Button>
              </Stack>
            </form>
          </PopoverBody>
        </PopoverContent>
      </Popover>
    </>
  ) : null}
  <Tooltip label={"Herunterladen"} hasArrow>
    <IconButton
      aria-label="Datei herunterladen"
      size="lg"
      fontSize={"2xl"}
      icon={<FiDownload />}
      variant="ghost"
      onClick={() => {
        if (
          props.structure.contents.match(
            /^\s*data:([a-z]+\/[a-z]+(;[a-z\-]+\=[a-z\-]+)?)?(;base64)?,[a-z0-9\!\$\&\'\,\(\)\*\+\,\;\=\-
\.\_\~\:\@\/\?\%\s]*\s*$/i
          )
        ) {
          const element = document.createElement("a");

          element.setAttribute("href", props.structure.contents);
```

```
                element.setAttribute(
                  "download",
                  props.structure.fullPath.split("/")[
                    props.structure.fullPath.split("/").length - 1
                  ]
                );

                element.style.display = "none";
                document.body.appendChild(element);

                element.click();

                document.body.removeChild(element);
                return;
              }

              const element = document.createElement("a");

              element.setAttribute(
                "href",
                "data:" +
                  lookup(props.structure.fullPath) +
                  ";charset=utf-8," +
                  encodeURIComponent(props.structure.contents)
              );
              element.setAttribute(
                "download",
                props.structure.fullPath.split("/")[
                  props.structure.fullPath.split("/").length - 1
                ]
              );

              element.style.display = "none";
              document.body.appendChild(element);

              element.click();

              document.body.removeChild(element);
            }}
          />
        </Tooltip>
      </ButtonGroup>
    </Box>
  </Flex>
 </>
 );
}

function FileExtensionOptions() {
 return (
  <>
   <option disabled={true}>All-In-One</option>
   <option value={".ic"}>InCode</option>
   <option disabled={true}>Struktur</option>
   <option value={".html"}>HTML</option>
   <option disabled={true}>Aussehen</option>
   <option value={".css"}>CSS</option>
   <option value={".scss"}>SCSS</option>
   <option value={".sass"}>Sass</option>
   <option disabled={true}>Logik</option>
```

```
      <option value={".js"}>JavaScript</option>
      <option value={".ts"}>TypeScript</option>
      <option disabled={true}>Anderes</option>
      <option value={".txt"}>Text</option>
      <option value={".json"}>JSON</option>
      <option value={".xml"}>XML</option>
    </>
  );
}
```

## packages/components/ide/index.ts

```
/**
 * packages/components/ide/index.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 24.07.2022
 *
 */

export { default as ContributorManager } from "./ContributorManager";
export { default as PreviewModal } from "./PreviewModal";
export { default as KeyboardShortcuts } from "./KeyboardShortcuts";
export { default as SnippetViewer } from "./SnippetViewer";
```

## packages/components/ide/KeyboardShortcuts.tsx

```
/**
 * packages/components/ide/KeyboardShortcuts.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 25.07.2022
 *
 */

import {
  Modal,
  ModalOverlay,
  ModalContent,
  ModalHeader,
  ModalFooter,
  ModalBody,
  ModalCloseButton,
  Button,
  Kbd,
  Table,
  Thead,
  Tbody,
  Tfoot,
  Tr,
  Th,
  Td,
  TableCaption,
  TableContainer,
} from "@chakra-ui/react";
import * as React from "react";

export default function KeyboardShortcuts(props: {
```

```
  isOpen: boolean;
  onOpen: () => void;
  onClose: () => void;
}) {
  return (
    <>
      <Modal
        onClose={props.onClose}
        isOpen={props.isOpen}
        isCentered
        size={["full", "xl"]}
        scrollBehavior={"inside"}
      >
        <ModalOverlay />
        <ModalContent>
          <ModalHeader>Tastenkürzel</ModalHeader>
          <ModalCloseButton />
          <ModalBody>
            <TableContainer>
              <Table variant="simple">
                <Thead>
                  <Tr>
                    <Th>Tastenkürzel</Th>
                    <Th>Aktion</Th>
                  </Tr>
                </Thead>
                <Tbody>
                  <Tr>
                    <Td>
                      <Kbd>STRG</Kbd> + <Kbd>S</Kbd>
                    </Td>
                    <Td>Speichern</Td>
                  </Tr>
                  <Tr>
                    <Td>
                      <Kbd>STRG</Kbd> + <Kbd>O</Kbd>
                    </Td>
                    <Td>Datei Öffnen</Td>
                  </Tr>
                  <Tr>
                    <Td>
                      <Kbd>STRG</Kbd> + <Kbd>C</Kbd>
                    </Td>
                    <Td>Kopieren</Td>
                  </Tr>
                  <Tr>
                    <Td>
                      <Kbd>STRG</Kbd> + <Kbd>V</Kbd>
                    </Td>
                    <Td>Einfügen</Td>
                  </Tr>
                  <Tr>
                    <Td>
                      <Kbd>STRG</Kbd> + <Kbd>Z</Kbd>
                    </Td>
                    <Td>Ausschneiden</Td>
                  </Tr>
                  <Tr>
                    <Td>
                      <Kbd>STRG</Kbd> + <Kbd>F</Kbd>
```

```
                </Td>
                <Td>Suchen</Td>
              </Tr>
              <Tr>
                <Td>
                  <Kbd>STRG</Kbd> + <Kbd>H</Kbd>
                </Td>
                <Td>Ersetzen</Td>
              </Tr>
              <Tr>
                <Td>
                  <Kbd>STRG</Kbd> + <Kbd>ALT</Kbd> + <Kbd>F</Kbd>
                </Td>
                <Td>Formatieren</Td>
              </Tr>
              <Tr>
                <Td>
                  <Kbd>STRG</Kbd> + <Kbd>G</Kbd>
                </Td>
                <Td>Zu Zeile springen</Td>
              </Tr>
              <Tr>
                <Td>
                  <Kbd>STRG</Kbd> + <Kbd>Z</Kbd>
                </Td>
                <Td>Rückgängig</Td>
              </Tr>
              <Tr>
                <Td>
                  <Kbd>STRG</Kbd> + <Kbd>Y</Kbd>
                </Td>
                <Td>Wiederherstellen</Td>
              </Tr>
            </Tbody>
            <Tfoot>
              <Tr>
                <Th>Tastenkürzel</Th>
                <Th>Aktion</Th>
              </Tr>
            </Tfoot>
          </Table>
        </TableContainer>
      </ModalBody>
      <ModalFooter>
        <Button onClick={props.onClose} variant={"outline"}>
          Schließen
        </Button>
      </ModalFooter>
    </ModalContent>
  </Modal>
 </>
 );
}
```

## packages/components/ide/OpenProjectMenu.tsx

```
/**
 * packages/components/ide/OpenProjectMenu.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
```

```
 * License: Project License
 * Created At: 22.08.2022
 *
 */

import * as React from "react";
import { useEffect, useId, useState } from "react";
import {
  Box,
  Button,
  Checkbox,
  Divider,
  Flex,
  FormControl,
  Heading,
  IconButton,
  Input,
  InputGroup,
  InputLeftAddon,
  InputRightAddon,
  Link,
  Menu,
  MenuButton,
  MenuItem,
  MenuList,
  Spinner,
  Text,
  useDisclosure,
  useToast,
} from "@chakra-ui/react";
import REST from "@codeup/rest";
import { useLoggedIn } from "@codeup/hooks";
import {
  FaCloudUploadAlt,
  FaFileExport,
  FaSearch,
  FaSortNumericDownAlt,
  FaSortNumericUpAlt,
  FaThumbtack,
  FaTrash,
  FaUsersCog,
} from "react-icons/fa";
import { BiDuplicate } from "react-icons/bi";
import * as generate from "project-name-generator";
import byteSize from "byte-size";
import { FiMoreHorizontal } from "react-icons/fi";
import { BsFillPencilFill, BsFillTrashFill } from "react-icons/bs";
import { ContributorManager } from "./index";
import TemplatePublisher from "./TemplatePublisher";
import { default as NextLink } from "next/link";
import Networking from "@codeup/util/src/Networking";
import { CHeading } from "../global/CHeading";

export default function OpenProjectMenu() {
  const [projectList, setProjectList] = useState<
    {
      name: string;
      id: string;
      permissions: string[];
      hasTemplate: boolean;
```

```
    updated_at: string;
    pinned: string[];
    stats: {
      total_files: number;
      total_contributors: number;
      total_size: number;
    };
  }[]
>([]);
const [filteredProjectList, setFilteredProjectList] = useState<
  {
    name: string;
    id: string;
    permissions: string[];
    hasTemplate: boolean;
    updated_at: string;
    pinned: string[];
    stats: {
      total_files: number;
      total_contributors: number;
      total_size: number;
    };
  }[]
>([]);
const [query, setQuery] = useState<string>("");

const [loading, setLoading] = useState<boolean>(true);
const toast = useToast();

useEffect(() => {
  REST.IDE.getProjects({
    token: localStorage.getItem("token") as string,
    type: "ALL",
    withStats: true,
  }).then(async (res) => {
    if (res.status === 200) {
      let pList = [];

      for (const project of res.payload.projects) {
        const r = await REST.IDE.getProjectPermissions({
          token: localStorage.getItem("token") as string,
          projectId: project._id,
        });

        const hasTemplate = await REST.IDE.hasTemplate({
          token: localStorage.getItem("token") as string,
          projectId: project._id,
        });

        pList.push({
          id: project._id,
          name: project.name,
          permissions: r.payload.permissions,
          hasTemplate: hasTemplate.payload.hasTemplate,
          updated_at: project.updated_at,
          pinned: project.pinned,
          stats: project.stats,
        });
      }
```

```
      setProjectList(pList);
      setFilteredProjectList(pList);
      setLoading(false);
    } else {
      toast({
        title: "Fehler",
        description: res.payload.error,
        status: "error",
        variant: "solid",
        position: "top",
        isClosable: true,
      });
    }
  });
}, []);

useEffect(() => {
  setFilteredProjectList(projectList.filter((p) => p.name.includes(query)));
}, [query]);

const {
  isOpen: isContributorOpen,
  onOpen: onContributorOpen,
  onClose: onContributorClose,
} = useDisclosure();

const sBoxId = useId();

const [projectId, setProjectId] = useState("0");

const { loggedIn, userInfo } = useLoggedIn();

const {
  isOpen: isPublisherOpen,
  onOpen: onPublisherOpen,
  onClose: onPublisherClose,
} = useDisclosure();

const [publisherTemplateId, setPublisherTemplateId] = useState("0");

const [selectedProjects, setSelectedProjects] = useState<string[]>([]);

const [sorting, setSorting] = useState<string>("latest-first");

useEffect(() => {
  console.log(selectedProjects);
}, [selectedProjects]);

useEffect(() => {
  if (sorting === "latest-first") {
    setFilteredProjectList(
      filteredProjectList.sort((a, b) => {
        return (
          Number(new Date(b.updated_at)) - Number(new Date(a.updated_at))
        );
      })
    );
  } else if (sorting === "oldest-first") {
    setFilteredProjectList(
      filteredProjectList.sort((a, b) => {
```

```
        return (
          Number(new Date(a.updated_at)) - Number(new Date(b.updated_at))
        );
      })
    );
  }
}, [sorting]);

return (
  <>
    <Flex alignItems={"center"} justifyContent={"center"} mt={"1.2rem"}>
      <Box
        w={["100%", "65%"]}
        minW={["100%", "65%"]}
        backgroundColor={"gray.900"}
        p={"1.2rem"}
        rounded={"xl"}
      >
        <Box textAlign={"center"}>
          <CHeading size={"xl"}>Projekt öffnen</CHeading>
        </Box>
        {loading ? (
          <>
            <Flex justifyContent={"center"} mt={"1.2rem"}>
              <Spinner size={"xl"} />
            </Flex>
          </>
        ) : null}
        {!loading && projectList.length === 0 ? (
          <>
            <Flex justifyContent={"center"} mt={"1.2rem"}>
              <Box>
                Du hast noch keine Projekte erstellt. 
                <Link color="brand.500" href={"?component=create"}>
                  <NextLink href={"?component=create"}>
                    Erstelle dein erstes Projekt
                  </NextLink>
                </Link>
                .
              </Box>
            </Flex>
          </>
        ) : null}
        {projectList.length > 0 ? (
          <>
            <FormControl mt={"2rem"}>
              <InputGroup>
                <InputLeftAddon>
                  <FaSearch />
                </InputLeftAddon>
                <Input
                  type={"text"}
                  placeholder={"Suche..."}
                  id={sBoxId}
                  defaultValue={query}
                  onChange={(e) => {
                    // @ts-ignore
                    setQuery(e.target.value);
                  }}
                />
```

```
    <InputRightAddon>
      <IconButton
        aria-label={"sort"}
        icon={
          sorting === "latest-first" ? (
            <FaSortNumericDownAlt />
          ) : (
            <FaSortNumericUpAlt />
          )
        }
        onClick={() => {
          if (sorting === "latest-first") {
            setSorting("oldest-first");
          } else {
            setSorting("latest-first");
          }
        }}
        backgroundColor={"transparent"}
        _hover={{
          backgroundColor: "transparent",
        }}
        _active={{
          backgroundColor: "transparent",
        }}
      />
    </InputRightAddon>
  </InputGroup>
</FormControl>
{filteredProjectList.length > 0 ? (
  <>
    <Flex direction={"column"} mt={"2rem"}>
      <Box pt={"0.75rem"} pb={"0.75rem"} fontSize={"2xl"}>
        <Flex direction={"row"}>
          <Box flex={"10%"}></Box>
          <Box
            flex={"20%"}
            textAlign={"center"}
            fontWeight={"bold"}
          >
            Name
          </Box>
          <Box
            flex={[0, "50%"]}
            textAlign={"center"}
            fontWeight={"bold"}
            display={["none", "initial"]}
          >
            Statistik
          </Box>
          <Box
            flex={"20%"}
            textAlign={"center"}
            fontWeight={"bold"}
          >
            Aktion
          </Box>
        </Flex>
      </Box>
      {selectedProjects.length > 0 ? (
        <>
```

```
<Box pt={".75rem"} pb={".75rem"} fontSize={"2xl"}>
  <Flex direction={"row"}>
    <Box flex={"5%"}></Box>
    <Box flex={"20%"} textAlign={"left"}>
      {selectedProjects.length} Projekte
    </Box>
    <Box flex={[0, "50%"]}></Box>
    <Box flex={"20%"}>
      <Flex
        w={"100%"}
        h={"100%"}
        alignItems={"center"}
        justifyContent={"center"}
      >
        <Button
          variant={"ghost"}
          leftIcon={<FaTrash />}
          colorScheme={"red"}
          onClick={async () => {
            let npl = projectList;

            for (const projectId of selectedProjects) {
              const res = await REST.IDE.deleteProject({
                token: localStorage.getItem(
                  "token"
                ) as string,
                projectId: projectId,
              });

              if (res.status === 200) {
                npl = npl.filter(
                  (p) => p.id !== projectId
                );
                (
                  document.getElementById(
                    sBoxId
                  ) as HTMLInputElement
                ).value = "";
                toast({
                  title: "Erfolg",
                  description: `Das Projekt ${projectId} wurde erfolgreich gelöscht`,
                  status: "success",
                  variant: "solid",
                  position: "top",
                  isClosable: true,
                });
              } else {
                toast({
                  title: "Fehler",
                  description: res.payload.error,
                  status: "error",
                  variant: "solid",
                  position: "top",
                  isClosable: true,
                });
              }
            }
            setSelectedProjects([]);
            setProjectList(npl);
            setFilteredProjectList(npl);
```

```
    setQuery("");
  }}
>
  Löschen
</Button>
<Button
  variant={"ghost"}
  leftIcon={<FaFileExport />}
  colorScheme={"orange"}
  onClick={async () => {
    for (const projectId of selectedProjects) {
      const fullProject = (
        await REST.IDE.getProject({
          token: localStorage.getItem(
            "token"
          ) as string,
          projectId: projectId as string,
        })
      ).payload.project;

      const exportObject: any = {};

      exportObject.codeUp = {
        version: "1.0",
        href: location.href,
        exportedAt: new Date().toLocaleString(),
        userInfo: {
          username: userInfo.username,
          id: userInfo.id,
        },
      };

      exportObject.name = fullProject.name;

      exportObject.files = fullProject.files;

      Networking.download(
        fullProject.name + ".cupp",
        JSON.stringify(exportObject, null, 2),
        "text/plain"
      );
    }
  }}
>
  Exportieren
</Button>
<Button
  variant={"ghost"}
  leftIcon={<BiDuplicate />}
  colorScheme={"blue"}
  onClick={async () => {
    let npl = projectList;

    for (const projectId of selectedProjects) {
      const newName = generate.default().dashed;

      const res =
        await REST.IDE.duplicateProject({
          token: localStorage.getItem(
            "token"
```

```
                    ) as string,
                    projectId: projectId,
                    name: newName,
                  });

                  if (res.status === 200) {
                    npl = [
                      ...npl,
                      {
                        id: res.payload.project._id,
                        name: newName,
                        permissions: ["OWNER"],
                        hasTemplate: false,
                        updated_at:
                          res.payload.project.updated_at,
                        pinned: res.payload.project.pinned,
                        stats: {
                          total_files: 1,
                          total_contributors: 1,
                          total_size: 0,
                        },
                      },
                    ];
                    (
                      document.getElementById(
                        sBoxId
                      ) as HTMLInputElement
                    ).value = "";
                    toast({
                      title: "Erfolg",
                      description:
                        "Das Projekt wurde erfolgreich dupliziert",
                      status: "success",
                      variant: "solid",
                      position: "top",
                      isClosable: true,
                    });
                  } else {
                    toast({
                      title: "Fehler",
                      description: res.payload.error,
                      status: "error",
                      variant: "solid",
                      position: "top",
                      isClosable: true,
                    });
                  }
                }
                setProjectList(npl);
                setFilteredProjectList(npl);
              }}
            >
              Klonen
            </Button>
          </Flex>
        </Box>
      </Flex>
    </Box>
  </>
) : null}
```

```
{filteredProjectList
  .sort((a, b) => {
    let f = a;
    let s = b;

    if (sorting !== "latest-first") {
      f = b;
      s = a;
    }

    return (
      Number(new Date(s.updated_at)) -
      Number(new Date(f.updated_at))
    );
  })
  .sort((a, b) => {
    if (
      a.pinned.includes(userInfo.id) &&
      !b.pinned.includes(userInfo.id)
    ) {
      return -1;
    } else if (
      !a.pinned.includes(userInfo.id) &&
      b.pinned.includes(userInfo.id)
    ) {
      return 1;
    }
    return 0;
  })
  .map((project) => {
    return (
      <>
        <Box
          pt={"0.75rem"}
          pb={"0.75rem"}
          fontSize={"2xl"}
          bgColor={
            selectedProjects.includes(project.id)
              ? "gray.800"
              : "gray.900"
          }
        >
          <Flex direction={"row"}>
            <Box flex={"10%"}>
              <Flex
                justifyContent={"center"}
                alignItems={"center"}
                h={"100%"}
              >
                <Checkbox
                  size={"lg"}
                  onChange={(e) => {
                    console.log(e.target.checked);
                    if (e.target.checked) {
                      setSelectedProjects([
                        project.id,
                        ...selectedProjects,
                      ]);
                    } else {
                      let x = selectedProjects;
```

```
          x = x.filter((y) => y !== project.id);
          setSelectedProjects(x);
        }
      }}
    />
  </Flex>
</Box>
<Box flex={"20%"} textAlign={"center"}>
  <Flex direction={"column"}>
    <Link
      href={
        "/ide/project/" +
        project.id +
        "#editor-wrapper-flex"
      }
      flex={"100%"}
    >
      <NextLink
        href={
          "/ide/project/" +
          project.id +
          "#editor-wrapper-flex"
        }
      >
        <Flex
          alignItems={"center"}
          justifyContent={"center"}
          gap={".6rem"}
        >
          <Text
            fontSize={"lg"}
            color={"blue.600"}
          >
            {project.pinned.includes(
              userInfo.id
            ) ? (
              <FaThumbtack />
            ) : null}
          </Text>
          {project.name}
        </Flex>
      </NextLink>
    </Link>
    <Text color={"gray.500"} fontSize={"lg"}>
      {new Date(
        project.updated_at
      ).toLocaleDateString()}
       
      {new Date(
        project.updated_at
      ).toLocaleTimeString()}
    </Text>
  </Flex>
</Box>
<Box
  flex={[0, "50%"]}
  display={["none", "initial"]}
  textAlign={"center"}
>
  <Flex direction={"column"} gap={".4rem"}>
```

```
    <Text fontSize={"md"}>
      {project.stats.total_files} 
      {project.stats.total_files === 1
        ? "Datei"
        : "Dateien"}
    </Text>
    <Text fontSize={"md"}>
      {project.stats.total_contributors} 
      {project.stats.total_contributors === 1
        ? "Mitwirkender"
        : "Mitwirkende"}
    </Text>
    <Text fontSize={"md"}>
      <>
        Projektgröße: 
        {byteSize(
          project.stats.total_size
        ).toString()}
      </>
    </Text>
  </Flex>
</Box>
<Box
  flex={"20%"}
  alignItems={"center"}
  justifyContent={"center"}
  textAlign={"center"}
>
  <Menu>
    <MenuButton>
      <IconButton
        variant="outline"
        aria-label={"menu"}
      >
        <FiMoreHorizontal />
      </IconButton>
    </MenuButton>
    <MenuList fontSize={"xl"}>
      {project.permissions.includes("OWNER") ||
      project.permissions.includes("DELETE") ? (
        <MenuItem
          onClick={async () => {
            const res =
              await REST.IDE.deleteProject({
                token: localStorage.getItem(
                  "token"
                ) as string,
                projectId: project.id,
              });

            if (res.status === 200) {
              let npl = projectList.filter(
                (p) => p.id !== project.id
              );
              setProjectList(npl);
              setFilteredProjectList(npl);
              setQuery("");
              (
                document.getElementById(
                  sBoxId
```

```
          ) as HTMLInputElement
        ).value = "";
        toast({
          title: "Erfolg",
          description:
            "Das Projekt wurde erfolgreich gelöscht",
          status: "success",
          variant: "solid",
          position: "top",
          isClosable: true,
        });
      } else {
        toast({
          title: "Fehler",
          description: res.payload.error,
          status: "error",
          variant: "solid",
          position: "top",
          isClosable: true,
        });
      }
    }}
  >
    <Flex
      direction={"row"}
      alignItems={"center"}
      gap={".6rem"}
      color={"red.400"}
    >
      <BsFillTrashFill /> Löschen
    </Flex>
  </MenuItem>
) : null}
{project.permissions.includes("OWNER") ||
project.permissions.includes("RENAME") ? (
  <MenuItem
    onClick={async (e) => {
      const newName = prompt(
        "Gib den neuen Namen ein"
      );

      if (!newName) {
        return;
      }

      const res =
        await REST.IDE.renameProject({
          token: localStorage.getItem(
            "token"
          ) as string,
          projectId: project.id,
          newName,
        });

      if (res.status === 200) {
        let npl = projectList.map((p) => {
          if (p.id === project.id) {
            return {
              id: p.id,
              name: newName,
```

```
              permissions: p.permissions,
              hasTemplate: p.hasTemplate,
              updated_at: p.updated_at,
              pinned: p.pinned,
              stats: p.stats,
            };
          } else {
            return p;
          }
        });
        setProjectList(npl);
        setFilteredProjectList(npl);
        setQuery("");
        (
          document.getElementById(
            sBoxId
          ) as HTMLInputElement
        ).value = "";
        toast({
          title: "Erfolg",
          description:
            "Das Projekt wurde erfolgreich umbenannt",
          status: "success",
          variant: "solid",
          position: "top",
          isClosable: true,
        });
      } else {
        toast({
          title: "Fehler",
          description: res.payload.error,
          status: "error",
          variant: "solid",
          position: "top",
          isClosable: true,
        });
      }
    }}
  >
    <Flex
      direction={"row"}
      alignItems={"center"}
      gap={".6rem"}
      color={"green.400"}
    >
      <BsFillPencilFill /> Umbenennen
    </Flex>
  </MenuItem>
) : null}
{project.permissions.includes("OWNER") ||
project.permissions.includes(
  "DUPLICATE"
) ? (
  <MenuItem
    onClick={async () => {
      const newName =
        generate.default().dashed;

      const res =
        await REST.IDE.duplicateProject({
```

```
    token: localStorage.getItem(
      "token"
    ) as string,
    projectId: project.id,
    name: newName,
  });

 if (res.status === 200) {
  setProjectList([
    ...projectList,
    {
      id: res.payload.project._id,
      name: newName,
      permissions: ["OWNER"],
      hasTemplate: false,
      updated_at:
        res.payload.project
          .updated_at,
      pinned:
        res.payload.project.pinned,
      stats: {
        total_files: 1,
        total_contributors: 1,
        total_size: 0,
      },
    },
  ]);

  setFilteredProjectList([
    ...projectList,
    {
      id: res.payload.project._id,
      name: newName,
      permissions: ["OWNER"],
      hasTemplate: false,
      updated_at:
        res.payload.project
          .updated_at,
      pinned:
        res.payload.project.pinned,
      stats: {
        total_files: 1,
        total_contributors: 1,
        total_size: 0,
      },
    },
  ]);
  setQuery("");
  (
    document.getElementById(
      sBoxId
    ) as HTMLInputElement
  ).value = "";
  toast({
    title: "Erfolg",
    description:
      "Das Projekt wurde erfolgreich dupliziert",
    status: "success",
    variant: "solid",
    position: "top",
```

```
          isClosable: true,
        });
      } else {
        toast({
          title: "Fehler",
          description: res.payload.error,
          status: "error",
          variant: "solid",
          position: "top",
          isClosable: true,
        });
      }
    }}
  >
    <Flex
      direction={"row"}
      alignItems={"center"}
      gap={".6rem"}
      color={"blue.400"}
    >
      <BiDuplicate /> Klonen
    </Flex>
  </MenuItem>
) : null}
{project.permissions.includes("OWNER") ||
project.permissions.includes(
  "MANAGE-CONTRIBUTORS"
) ? (
  <MenuItem
    onClick={async () => {
      setProjectId(project.id);
      onContributorOpen();
    }}
  >
    <Flex
      direction={"row"}
      alignItems={"center"}
      gap={".6rem"}
      color={"pink.400"}
    >
      <FaUsersCog /> Mitwirkende verwalten
    </Flex>
  </MenuItem>
) : null}
{project.permissions.includes("OWNER") ||
project.permissions.includes("EXPORT") ? (
  <MenuItem
    onClick={async () => {
      const fullProject = (
        await REST.IDE.getProject({
          token: localStorage.getItem(
            "token"
          ) as string,
          projectId: project.id as string,
        })
      ).payload.project;

      const exportObject: any = {};

      exportObject.codeUp = {
```

```
        version: "1.0",
        href: location.href,
        exportedAt:
          new Date().toLocaleString(),
        userInfo: {
          username: userInfo.username,
          id: userInfo.id,
        },
      };

      exportObject.name =
        fullProject.name;

      exportObject.files =
        fullProject.files;

      Networking.download(
        fullProject.name + ".cupp",
        JSON.stringify(
          exportObject,
          null,
          2
        ),
        "text/plain"
      );
    }}
  >
    <Flex
      direction={"row"}
      alignItems={"center"}
      gap={".6rem"}
      color={"yellow.400"}
    >
      <FaFileExport /> Exportieren
    </Flex>
  </MenuItem>
) : null}
{project.permissions.includes("OWNER") &&
!project.hasTemplate ? (
  <MenuItem
    onClick={async () => {
      setPublisherTemplateId(project.id);
      onPublisherOpen();
    }}
  >
    <Flex
      direction={"row"}
      alignItems={"center"}
      gap={".6rem"}
      color={"purple.400"}
    >
      <FaCloudUploadAlt /> Vorlage
      Veröffentlichen
    </Flex>
  </MenuItem>
) : null}
{project.permissions.includes("OWNER") &&
project.hasTemplate ? (
  <MenuItem
    onClick={async () => {
```

```
      const res =
        await REST.IDE.removeTemplate({
          token: localStorage.getItem(
            "token"
          ) as string,
          projectId: project.id,
        });

      let y = projectList;

      (
        y.find(
          (p) => p.id === project.id
        ) as any
      ).hasTemplate = false;

      setProjectList(y);

      setQuery(project.name);

      setTimeout(() => {
        setQuery(query);
      }, 100);

      toast({
        status: "success",
        position: "top",
        variant: "solid",
        isClosable: true,
        duration: 3000,
        title: "Erfolgreich",
        description:
          "Die Vorlage wurde erfolgreich gelöscht!",
      });
    }}
  >
    <Flex
      direction={"row"}
      alignItems={"center"}
      gap={".6rem"}
      color={"purple.400"}
    >
      <FaTrash /> Vorlage Löschen
    </Flex>
  </MenuItem>
) : null}
<MenuItem
  onClick={async () => {
    let l = projectList;
    const beforePinned =
      project.pinned.includes(
        userInfo.id
      );

    await REST.IDE.toggleProjectPin({
      token: localStorage.getItem(
        "token"
      ) as string,
      projectId: project.id,
    });
```

```
                                l = l.map((x) => {
                                  if (x.id !== project.id) {
                                    return x;
                                  } else {
                                    if (beforePinned) {
                                      x.pinned = x.pinned.filter(
                                        (y) => y !== userInfo.id
                                      );
                                    } else {
                                      x.pinned.push(userInfo.id);
                                    }
                                    return x;
                                  }
                                });

                                setProjectList(l);
                                setQuery("loading...");
                                setTimeout(() => {
                                  setQuery("");
                                }, 100);
                              }}
                            >
                              <Flex
                                direction={"row"}
                                alignItems={"center"}
                                gap={".6rem"}
                                color={"blue.600"}
                              >
                                <FaThumbtack />
                                 
                                {project.pinned.includes(userInfo.id)
                                  ? "Abheften"
                                  : "Anheften"}
                              </Flex>
                            </MenuItem>
                          </MenuList>
                        </Menu>
                      </Box>
                    </Flex>
                  </Box>
                  <Divider />
                </>
              );
            })}
          </Flex>
        </>
      ) : (
        <>
          <Box mt={"2rem"} textAlign={"center"}>
            Es wurden keine Projekte gefunden, die der auf die Suche
            &amp;quot;{query}&amp;quot; zutreffen
          </Box>
        </>
      )}
    </>
  ) : null}
 </Box>
</Flex>
<ContributorManager
```

```
        isOpen={isContributorOpen}
        onOpen={onContributorOpen}
        onClose={onContributorClose}
        projectId={projectId}
      />
      <TemplatePublisher
        isOpen={isPublisherOpen}
        onOpen={onPublisherOpen}
        onClose={onPublisherClose}
        projectId={publisherTemplateId}
        setProjectList={setProjectList}
        projectList={projectList}
      />
    </>
  );
}
```

## packages/components/ide/PreviewModal.tsx

```
/**
 * packages/components/ide/PreviewModal.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 24.07.2022
 *
 */

import * as React from "react";
import {
  Button,
  Modal,
  ModalBody,
  ModalCloseButton,
  ModalContent,
  ModalFooter,
  ModalHeader,
  ModalOverlay,
} from "@chakra-ui/react";
import { FaExternalLinkAlt } from "react-icons/fa";

export default function PreviewModal(props: {
  project: string | undefined;
  path: string | undefined;
  isOpen: boolean;
  onOpen: () => void;
  onClose: () => void;
}) {
  const onClose = () => {
    props.onClose();
  };

  return (
    <>
      {props.project && props.path ? (
        <Modal isOpen={props.isOpen} onClose={onClose} size={"full"}>
          <ModalOverlay />
          <ModalContent>
            <ModalHeader>Vorschau</ModalHeader>
            <ModalCloseButton />
```

```
      <ModalBody>
        <iframe
          width={"100%"}
          style={{ border: "none", height: "80vh" }}
          id={"ifr-prev"}
          src={
            "/api/ide/projects/" + props.project + "/preview" + props.path
          }
        ></iframe>
      </ModalBody>

      <ModalFooter>
        <Button
          rightIcon={<FaExternalLinkAlt />}
          variant="outline"
          mr={3}
          onClick={() => {
            window.open(
              "/api/ide/projects/" +
                props.project +
                "/preview" +
                props.path,
              "_blank"
            );
          }}
        >
          In neuem Tab öffnen
        </Button>

        <Button
          variant="outline"
          mr={3}
          onClick={() => {
            const iframe = document.getElementById(
              "ifr-prev"
            ) as HTMLIFrameElement;

            if (iframe.requestFullscreen) {
              iframe.requestFullscreen();
              // @ts-ignore
            } else if (iframe.webkitRequestFullscreen) {
              // @ts-ignore
              iframe.webkitRequestFullscreen();
              // @ts-ignore
            } else if (iframe.mozRequestFullScreen) {
              // @ts-ignore
              iframe.mozRequestFullScreen();
              // @ts-ignore
            } else if (iframe.msRequestFullscreen) {
              // @ts-ignore
              iframe.msRequestFullscreen();
            }
          }}
        >
          Vollbild
        </Button>
      </ModalFooter>
    </ModalContent>
  </Modal>
) : null}
```

```
    </>
  );
}
```

## packages/components/ide/PublishManager.tsx

```
/**
 * packages/components/ide/PublishManager.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 06.12.2022
 *
 */

import * as React from "react";
import { useEffect, useState } from "react";
import REST from "@codeup/rest";
import {
  Box,
  Button,
  ButtonGroup,
  Checkbox,
  Divider,
  Flex,
  FormControl,
  FormLabel,
  Heading,
  IconButton,
  Input,
  InputGroup,
  InputLeftAddon,
  InputLeftElement,
  Modal,
  ModalBody,
  ModalCloseButton,
  ModalContent,
  ModalFooter,
  ModalHeader,
  ModalOverlay,
  Popover,
  PopoverArrow,
  PopoverBody,
  PopoverCloseButton,
  PopoverContent,
  PopoverHeader,
  PopoverTrigger,
  SimpleGrid,
  Spinner,
  Stack,
  Tag,
  Text,
  Textarea,
  Tooltip,
  useToast,
} from "@chakra-ui/react";
import { FaCogs, FaPlus, FaTrash } from "react-icons/fa";
import { HiMail } from "react-icons/hi";

export default function PublishManager(props: {
```

```
  projectId?: string;
  isOpen: boolean;
  onOpen: () => void;
  onClose: () => void;
}) {
  const [project, setProject] = useState<{
    _id: string;
    owner: string;
    name: string;
    contributors: {
      user: string;
      permissions: string[];
    }[];
    files: {
      path: string;
      content: string;
    }[];
    published: boolean;
    publishedCustomName: string;
    discovery: boolean;
    discoveryDescription: string;
  } | null>(null);

  const toast = useToast();

  useEffect(() => {
    if (props.projectId && props.projectId !== "0") {
      REST.IDE.getProject({
        token: localStorage.getItem("token") as string,
        projectId: props.projectId,
      }).then((res) => {
        if (res.status === 200) {
          setProject(res.payload.project);
        } else {
          toast({
            title: "Fehler",
            description: "Das Projekt konnte nicht geladen werden.",
            status: "error",
            duration: 1500,
            isClosable: true,
            position: "bottom-left",
          });
        }
      });
    }
  }, [props.projectId]);

  return (
    <>
      <Modal
        isOpen={props.isOpen}
        onClose={props.onClose}
        size={["full", "5xl"]}
        scrollBehavior={"inside"}
      >
        <ModalOverlay />
        <ModalContent>
          <ModalHeader>Veröffentlichung verwalten</ModalHeader>
          <ModalCloseButton />
          <ModalBody>
```

```
{project && !project.published ? (
  <>
    <Text>
      Du kannst dein Projekt veröffentlichen, um es anderen
      Benutzern zugänglich zu machen. Du kannst auch andere Benutzer
      hinzufügen, die an deinem Projekt mitarbeiten können.
    </Text>
    <Divider my={4} />
    <form
      onSubmit={(e) => {
        e.preventDefault();

        const form = new FormData(e.target as HTMLFormElement);

        const customName = form.get("customName") as string;

        REST.IDE.publishProject({
          token: localStorage.getItem("token") as string,
          projectId: project._id,
          customName,
        }).then((res) => {
          if (res.status === 200) {
            toast({
              title: "Erfolg",
              description: "Das Projekt wurde veröffentlicht.",
              status: "success",
              duration: 1500,
              isClosable: true,
              position: "bottom-left",
            });
            setProject({
              ...project,
              published: true,
            });
          } else {
            if (res.status === 700) {
              toast({
                title: "Fehler",
                description: "Dieser Name ist bereits vergeben.",
                status: "error",
                duration: 1500,
                isClosable: true,
                position: "bottom-left",
              });
            } else {
              toast({
                title: "Fehler",
                description:
                  "Das Projekt konnte nicht veröffentlicht werden.",
                status: "error",
                duration: 1500,
                isClosable: true,
                position: "bottom-left",
              });
            }
          }
        });
      }}
    >
      <SimpleGrid columns={[1, 2]} spacing={4}>
```

```
        <FormControl isRequired>
          <FormLabel>Projektname</FormLabel>
          <InputGroup>
            <InputLeftAddon>
              https://codeup.space/project/
            </InputLeftAddon>
            <Input
              name={"customName"}
              value={project.publishedCustomName}
              onChange={(e) => {
                setProject({
                  ...project,
                  publishedCustomName: e.target.value,
                });
              }}
            />
          </InputGroup>
        </FormControl>
        <FormControl>
          <FormLabel>Veröffentlichen</FormLabel>
          <Button colorScheme={"green"} type={"submit"}>
            Veröffentlichen
          </Button>
        </FormControl>
      </SimpleGrid>
    </form>
  </>
) : (
  <>
    <Text>
      Dein Projekt ist veröffentlicht. Du kannst es jederzeit
      verstecken, indem du auf &amp;quot;Verstecken&amp;quot; klickst.
      Außerdem kannst du die URL des Projektes ändern.
    </Text>
    <Divider my={4} />
    <form
      onSubmit={(e) => {
        e.preventDefault();

        const form = new FormData(e.target as HTMLFormElement);

        const customName = form.get("customName") as string;

        REST.IDE.publishProject({
          token: localStorage.getItem("token") as string,
          projectId: (project as any)._id as string,
          customName,
        }).then((res) => {
          if (res.status === 200) {
            toast({
              title: "Erfolg",
              description: "Das Projekt wurde veröffentlicht.",
              status: "success",
              duration: 1500,
              isClosable: true,
              position: "bottom-left",
            });
            // @ts-ignore
            setProject({
              ...project,
```

```
          published: true,
        });
      } else {
        if (res.status === 700) {
          toast({
            title: "Fehler",
            description: "Dieser Name ist bereits vergeben.",
            status: "error",
            duration: 1500,
            isClosable: true,
            position: "bottom-left",
          });
        } else {
          toast({
            title: "Fehler",
            description:
              "Das Projekt konnte nicht veröffentlicht werden.",
            status: "error",
            duration: 1500,
            isClosable: true,
            position: "bottom-left",
          });
        }
      }
    });
  }}
>
  <SimpleGrid columns={[1, 2]} spacing={4}>
    <FormControl isRequired>
      <FormLabel>Projektname</FormLabel>
      <InputGroup>
        <InputLeftAddon>
          https://codeup.space/project/
        </InputLeftAddon>
        <Input
          name={"customName"}
          value={project?.publishedCustomName}
          onChange={(e) => {
            // @ts-ignore
            setProject({
              ...project,
              publishedCustomName: e.target.value,
            });
          }}
        />
      </InputGroup>
    </FormControl>
    <FormControl>
      <FormLabel>Veröffentlichen</FormLabel>
      <Button colorScheme={"green"} type={"submit"}>
        Übernehmen
      </Button>
    </FormControl>
    <FormControl>
      <FormLabel>Teilen</FormLabel>
      <Button
        colorScheme={"blue"}
        onClick={() => {
          if (window.isSecureContext) {
            navigator.clipboard.writeText(
```

```
          `https://codeup.space/project/${project?.publishedCustomName}`
        );

        toast({
          title: "Erfolg",
          description:
            "Die URL wurde in die Zwischenablage kopiert.",
          status: "success",
          duration: 1500,
          isClosable: true,
          position: "bottom-left",
        });
      } else {
        toast({
          title: "Fehler",
          description:
            "Dieser Browser unterstützt das Teilen nicht.",
          status: "error",
          duration: 1500,
          isClosable: true,
          position: "bottom-left",
        });
      }
    }}
  >
    URL kopieren
  </Button>
</FormControl>

<FormControl>
  <FormLabel>Verstecken</FormLabel>
  <Button
    colorScheme={"red"}
    onClick={() => {
      REST.IDE.unpublishProject({
        token: localStorage.getItem("token") as string,
        // @ts-ignore
        projectId: project?._id,
      }).then((res) => {
        if (res.status === 200) {
          toast({
            title: "Erfolg",
            description: "Das Projekt wurde versteckt.",
            status: "success",
            duration: 1500,
            isClosable: true,
            position: "bottom-left",
          });
          // @ts-ignore
          setProject({
            ...project,
            published: false,
          });
        } else {
          toast({
            title: "Fehler",
            description:
              "Das Projekt konnte nicht versteckt werden.",
            status: "error",
            duration: 1500,
```

```
              isClosable: true,
              position: "bottom-left",
            });
          }
        });
      }}
    >
      Verstecken
    </Button>
  </FormControl>
 </SimpleGrid>
</form>
<Divider my={4} />
<Heading size={"md"}>Entdeckungszentrale</Heading>
<Text>
 Du kannst dein Projekt in der Entdeckungszentrale
 veröffentlichen, damit andere es finden können.
</Text>
<SimpleGrid columns={[1, 2]} spacing={4} mt={4}>
 {!project?.discovery ? (
  <>
   <form
    onSubmit={async (e) => {
     e.preventDefault();

     const data = new FormData(
      e.target as HTMLFormElement
     );

     const description = data.get("description") as string;

     if (description.length < 50) {
      toast({
       title: "Fehler",
       description:
        "Die Beschreibung ist zu kurz. Sie muss mindestens 50 Zeichen lang sein.",
       status: "error",
       duration: 1500,
       isClosable: true,
       position: "bottom-left",
      });
      return;
     }

     const r = await REST.IDE.enableDiscovery({
      token: localStorage.getItem("token") as string,
      projectId: (project as any)._id as string,
      description,
     });

     if (r.status === 200) {
      toast({
       title: "Erfolg",
       description:
        "Das Projekt wurde in der Entdeckungszentrale veröffentlicht.",
       status: "success",
       duration: 1500,
       isClosable: true,
       position: "bottom-left",
      });
```

```
            setProject({
              ...project,
              discovery: true,
              discoveryDescription: description,
            } as any);
          } else {
            toast({
              title: "Fehler",
              description:
                "Das Projekt konnte nicht in der Entdeckungszentrale veröffentlicht werden.",
              status: "error",
              duration: 1500,
              isClosable: true,
              position: "bottom-left",
            });
          }
        }}
      >
        <Box>
          <FormControl isRequired mb={2}>
            <Textarea
              name={"description"}
              placeholder={"Beschreibung"}
            />
          </FormControl>
          <Button colorScheme={"green"} type={"submit"}>
            Veröffentlichen
          </Button>
        </Box>
      </form>
    </>
  ) : (
    <>
      <Button
        colorScheme={"red"}
        onClick={async () => {
          const r = await REST.IDE.disableDiscovery({
            token: localStorage.getItem("token") as string,
            projectId: (project as any)._id as string,
          });

          if (r.status === 200) {
            toast({
              title: "Erfolg",
              description:
                "Das Projekt wurde aus der Entdeckungszentrale entfernt.",
              status: "success",
              duration: 1500,
              isClosable: true,
              position: "bottom-left",
            });

            setProject({
              ...project,
              discovery: false,
              discoveryDescription: "",
            });
          } else {
            toast({
```

```
                        title: "Fehler",
                        description:
                          "Das Projekt konnte nicht aus der Entdeckungszentrale entfernt werden.",
                        status: "error",
                        duration: 1500,
                        isClosable: true,
                        position: "bottom-left",
                      });
                    }
                  }}
                >
                  Verstecken
                </Button>
              </>
            )}
          </SimpleGrid>
        </>
      )}
    </ModalBody>

    <ModalFooter>
      <Button colorScheme="blue" mr={3} onClick={props.onClose}>
        Schließen
      </Button>
    </ModalFooter>
  </ModalContent>
</Modal>
</>
);
}
```

## packages/components/ide/SnippetViewer.tsx

```
/**
 * packages/components/ide/SnippetViewer.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 30.12.2022
 *
 */

import * as React from "react";
import {
  Button,
  ButtonGroup,
  chakra,
  Drawer,
  DrawerBody,
  DrawerCloseButton,
  DrawerContent,
  DrawerFooter,
  DrawerHeader,
  DrawerOverlay,
  Flex,
  Grid,
  GridItem,
  Heading,
  Input,
  InputGroup,
```

```
  InputLeftElement,
  Text,
  useToast,
} from "@chakra-ui/react";
import REST from "@codeup/rest";
import { FaCopy, FaSearch } from "react-icons/fa";
import { CHeading } from "../global/CHeading";

export default function SnippetViewer(props: {
  isOpen: boolean;
  onClose: () => void;
}) {
  const [snippets, setSnippets] = React.useState<
    {
      _id: string;
      title: string;
      code: string;
      language: string;
      createdAt: string;
    }[]
  >([]);
  const [error, setError] = React.useState(false);

  React.useEffect(() => {
    REST.Snippets.getSnippets(localStorage.getItem("token") as string).then(
      (res) => {
        if (res.status === 200) {
          setSnippets(res.payload.snippets);
        } else {
          setError(true);
        }
      }
    );
  }, []);

  const [filteredSnippets, setFilteredSnippets] = React.useState<
    typeof snippets
  >([]);

  const [query, setQuery] = React.useState("");

  const toast = useToast();

  return (
    <>
      <Drawer
        isOpen={props.isOpen}
        placement="left"
        onClose={props.onClose}
        size={["full", "lg"]}
      >
        <DrawerOverlay />
        <DrawerContent bgColor={"gray.900"}>
          <DrawerCloseButton />
          <DrawerHeader>Code Snippets</DrawerHeader>

          <DrawerBody>
            <Flex
              w={"100%"}
              alignItems={"center"}
```

```
  justifyContent={"center"}
  mb={4}
>
  <InputGroup w={"100%"}>
    <InputLeftElement>
      <FaSearch />
    </InputLeftElement>
    <Input
      placeholder={"Suche"}
      onChange={(e) => {
        setQuery(e.currentTarget.value);
      }}
    />
  </InputGroup>
</Flex>
<Grid templateColumns={"repeat(1, 1fr)"} gap={6}>
  {snippets
    .filter((s) => {
      return (
        s.title.toLowerCase().includes(query.toLowerCase()) ||
        s.code.toLowerCase().includes(query.toLowerCase()) ||
        s.language.toLowerCase().includes(query.toLowerCase())
      );
    })
    .map((snippet) => (
      <>
        <GridItem
          key={snippet._id}
          bg={"gray.800"}
          p={"1rem"}
          borderRadius={"0.5rem"}
        >
          <Flex alignItems={"center"}>
            <CHeading size={"lg"}>{snippet.title}</CHeading>
            <Text ml={"auto"}>
              {new Date(snippet.createdAt).toLocaleString()}
            </Text>
          </Flex>
          <Text mb={"2rem"}>
            <chakra.span color={"brand.500"}>
              {snippet.language}
            </chakra.span>
          </Text>
          <p>
            <pre>
              <code>{snippet.code}</code>
            </pre>
          </p>
          <ButtonGroup mt={"2rem"}>
            <Button
              variant={"brand"}
              leftIcon={<FaCopy />}
              onClick={() => {
                try {
                  navigator.clipboard.writeText(snippet.code);
                  toast({
                    title: "Code kopiert",
                    description:
                      "Der Code wurde erfolgreich kopiert",
                    status: "success",
```

```
                            duration: 1500,
                            isClosable: false,
                            position: "bottom-right",
                          });
                        } catch (e) {
                          toast({
                            title: "Fehler",
                            description:
                              "Der Code konnte nicht kopiert werden. Eventuell wird dein Browser nicht
unterstützt.",
                            status: "error",
                            duration: 1500,
                            isClosable: false,
                            position: "bottom-right",
                          });
                        }
                      }}
                    >
                      Kopieren
                    </Button>
                  </ButtonGroup>
                </GridItem>
              </>
            ))}
          </Grid>
        </DrawerBody>

        <DrawerFooter>
          <Button variant="outline" mr={3} onClick={props.onClose}>
            Schließen
          </Button>
        </DrawerFooter>
      </DrawerContent>
    </Drawer>
  </>
 );
}
```

## packages/components/ide/TemplateImporter.tsx

```
/**
 * packages/components/ide/TemplateImporter.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 24.07.2022
 *
 */

import * as React from "react";
import { useEffect, useState } from "react";
import {
  Button,
  Modal,
  ModalBody,
  ModalContent,
  ModalFooter,
  ModalHeader,
  ModalOverlay,
  Stack,
```

```
  Text,
  useDisclosure,
  Progress,
} from "@chakra-ui/react";
import { useRouter } from "next/router";
import REST from "@codeup/rest";
import * as generate from "project-name-generator";

export default function TemplateImporter(props: { templateId: string }) {
  const [importProgress, setImportProgress] = useState(0);
  const {
    isOpen: isImportOpen,
    onOpen: onImportOpen,
    onClose: onImportClose,
  } = useDisclosure();

  const [importProgressMax, setImportProgressMax] = useState(0);
  const [importProgressCurrent, setImportProgressCurrent] = useState("");

  const [importFinished, setImportFinished] = useState(false);

  const [importedProjectId, setImportedProjectId] = useState("");

  const router = useRouter();

  let f = false;

  useEffect(() => {
    if (props.templateId && props.templateId !== "0" && !isImportOpen && !f) {
      f = true;
      onImportOpen();
      REST.IDE.useTemplate({
        token: localStorage.getItem("token") as string,
        templateId: props.templateId,
      }).then((useTemplateRes) => {
        REST.IDE.createProject({
          token: localStorage.getItem("token") as string,
          name: generate.default().dashed,
        }).then(async (createProjectRes) => {
          await REST.IDE.deleteFile({
            token: localStorage.getItem("token") as string,
            path: "/README.MD",
            projectId: createProjectRes.payload.project._id,
          });

          setImportProgressMax(useTemplateRes.payload.files.length);
          setImportProgress(0);

          for (const [index, file] of useTemplateRes.payload.files.entries()) {
            setImportProgressCurrent(file.path);

            await REST.IDE.saveFile({
              token: localStorage.getItem("token") as string,
              projectId: createProjectRes.payload.project._id,
              path: file.path,
              content: file.content,
            });

            setImportProgress(index + 1);
          }
```

```
        setImportProgress(0);
        setImportProgressCurrent("");
        setImportProgressMax(1);
        setImportedProjectId(createProjectRes.payload.project._id);

        setImportFinished(true);
      });
    });
  }
}, [props.templateId]);

return (
  <>
    {props.templateId && props.templateId !== "0" ? (
      <Modal isOpen={isImportOpen} onClose={onImportClose} isCentered>
        <ModalOverlay />
        <ModalContent>
          <ModalHeader>
            {importFinished ? "Import abgeschlossen" : "Importieren"}
          </ModalHeader>
          <ModalBody>
            {importFinished ? (
              <>
                <Text color={"gray.400"}>
                  Der Import wurde abgeschlossen! Du kannst das Projekt nun
                  bearbeiten.
                </Text>
                <Stack direction={"column"} mt={"2rem"} gap={".4rem"}>
                  <Button
                    variant={"solid"}
                    colorScheme={"blue"}
                    onClick={() => {
                      router.push(
                        "/ide/project/" +
                          importedProjectId +
                          "#editor-wrapper-flex"
                      );
                    }}
                  >
                    Bearbeiten
                  </Button>
                  <Button
                    variant={"outline"}
                    onClick={() => {
                      setImportedProjectId("0");
                      onImportClose();
                      router.push("?component=open");
                    }}
                  >
                    Später
                  </Button>
                </Stack>
              </>
            ) : (
              <>
                <Progress
                  size={"lg"}
                  max={importProgressMax}
                  value={importProgress}
```

```
                w={"100%"}
                rounded={"xl"}
                hasStripe
              />
              <Text color={"gray.500"}>
                Importiere Datei <code>{importProgressCurrent}</code>
              </Text>
            </>
          )}
        </ModalBody>
        <ModalFooter></ModalFooter>
      </ModalContent>
    </Modal>
  ) : null}
 </>
 );
}
```

## packages/components/ide/TemplatePicker.tsx

```
/**
 * packages/components/ide/TemplatePicker.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 24.07.2022
 *
 */

import * as React from "react";
import { useEffect, useState } from "react";
import REST from "@codeup/rest";
import {
 Box,
 Button,
 Flex,
 Heading,
 Input,
 Link,
 Popover,
 PopoverArrow,
 PopoverBody,
 PopoverContent,
 PopoverHeader,
 PopoverTrigger,
 SimpleGrid,
 Spinner,
 Stack,
 Text,
} from "@chakra-ui/react";
import { FaPlus, FaStar } from "react-icons/fa";
import { useLoggedIn } from "@codeup/hooks";
import TemplateImporter from "./TemplateImporter";
import { useRouter } from "next/router";

export default function TemplatePicker() {
 const [templates, setTemplates] = useState<
  {
   _id: string;
   owner: string;
```

```
    authorDisplayName: string;
    displayName: string;
    project: string;
    description: string;
    uses: number;
    ratings: {
      user: string;
      rating: number;
      _id: string;
    }[];
    __v: number;
  }[]
>([]);

const [loaded, setLoaded] = useState(false);

useEffect(() => {
  REST.IDE.getAllTemplates().then((res) => {
    setTemplates(res.payload.templates);
    setLoaded(true);
  });
}, []);

const [term, setTerm] = useState("");

const [importing, setImporting] = useState(false);
const [importTemplate, setImportTemplate] = useState("0");

const { loggedIn, userInfo } = useLoggedIn();

const router = useRouter();

return (
  <>
    {importing ? (
      <TemplateImporter templateId={importTemplate} />
    ) : (
      <>
        {!loaded ? (
          <>
            <Flex
              w={"100%"}
              alignItems={"center"}
              justifyContent={"center"}
              mt={"1.2rem"}
            >
              <Spinner size={"xl"} />
            </Flex>
          </>
        ) : null}

        {loaded && templates.length === 0 ? (
          <>
            <Text textAlign={"center"} mt={"1.2rem"}>
              Es wurden noch keine Vorlagen veröffentlicht!
            </Text>
          </>
        ) : null}
        {templates.length !== 0 ? (
          <>
```

```
<Input
  type={"text"}
  onChange={(e) => {
    setTerm(e.target.value.toLowerCase().trim());
  }}
  placeholder={"Suchen..."}
  mt={"1.2rem"}
/>
<SimpleGrid
  minChildWidth={200}
  w={"100%"}
  mt={"2rem"}
  gap={"1.2rem"}
>
  {templates.map((t) => {
    if (
      t.displayName.toLowerCase().includes(term) ||
      t.description.toLowerCase().includes(term)
    ) {
      let rating = 0;

      for (const ra of t.ratings) {
        rating += ra.rating;
      }

      rating = rating / t.ratings.length;

      if (isNaN(rating)) {
        rating = 0;
      }

      async function vote(voting: number) {
        await REST.IDE.rateTemplate({
          token: localStorage.getItem("token") as string,
          rating: voting,
          templateId: t._id,
        });

        REST.IDE.getAllTemplates().then((res) => {
          setTemplates(res.payload.templates);
        });
      }

      return (
        <>
          <Box
            rounded={"xl"}
            shadow={"xl"}
            w={"100%"}
            padding={"1.2rem"}
            bgColor={"gray.800"}
          >
            <Stack>
              <Heading size={"lg"} textAlign={"center"}>
                {t.displayName}
              </Heading>
              <Flex
                alignItems={"center"}
                justifyContent={"center"}
              >
```

```
  {rating}
  &amp;nbsp;&amp;nbsp;/&amp;nbsp;&amp;nbsp;5&amp;nbsp;
  <FaStar color={"orange"} />
</Flex>
<Flex
  alignItems={"center"}
  justifyContent={"center"}
>
  von&amp;nbsp;
  <Link
    color={"blue.300"}
    onClick={() => {
      router.push("/forum/" + t.authorDisplayName);
    }}
  >
    {t.authorDisplayName}
  </Link>
</Flex>
<Flex
  alignItems={"center"}
  justifyContent={"center"}
>
  {t.uses} mal verwendet
</Flex>
<Text>{t.description}</Text>
<Button
  colorScheme={"blue"}
  leftIcon={<FaPlus />}
  onClick={async () => {
    setImportTemplate(t._id);
    setImporting(true);
  }}
>
  Verwenden
</Button>
<Popover>
  <PopoverTrigger>
    <Button
      disabled={
        t.ratings.find(
          (r) => r.user === userInfo.id
        ) !== undefined
      }
      leftIcon={<FaStar />}
    >
      Bewerten
    </Button>
  </PopoverTrigger>
  <PopoverContent bgColor={"black"}>
    <PopoverArrow bgColor={"black"} />
    <PopoverHeader>Vorlage Bewerten</PopoverHeader>
    <PopoverBody>
      <Button
        m=".4rem"
        colorScheme="red"
        leftIcon={<FaStar />}
        onClick={() => {
          vote(1);
        }}
      >
```

```
                  1
                </Button>
                <Button
                  m=".4rem"
                  colorScheme="orange"
                  leftIcon={<FaStar />}
                  onClick={() => {
                    vote(2);
                  }}
                >
                  2
                </Button>
                <Button
                  m=".4rem"
                  colorScheme="yellow"
                  leftIcon={<FaStar />}
                  onClick={() => {
                    vote(3);
                  }}
                >
                  3
                </Button>
                <Button
                  m=".4rem"
                  colorScheme={"blue"}
                  leftIcon={<FaStar />}
                  onClick={() => {
                    vote(4);
                  }}
                >
                  4
                </Button>
                <Button
                  m=".4rem"
                  colorScheme={"green"}
                  leftIcon={<FaStar />}
                  onClick={() => {
                    vote(5);
                  }}
                >
                  5
                </Button>
              </PopoverBody>
            </PopoverContent>
          </Popover>
        </Stack>
      </Box>
    </>
  );
 } else {
  return null;
 }
})}
</SimpleGrid>
</>
) : null}
</>
)}
</>
);
```

```
}
```

## packages/components/ide/TemplatePublisher.tsx

```
/**
 * packages/components/ide/TemplatePublisher.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 24.07.2022
 *
 */

import * as React from "react";
import {
  Button,
  chakra,
  FormControl,
  FormLabel,
  Input,
  Modal,
  ModalBody,
  ModalCloseButton,
  ModalContent,
  ModalFooter,
  ModalHeader,
  ModalOverlay,
  Text,
  Textarea,
  useToast,
} from "@chakra-ui/react";
import REST from "@codeup/rest";
import { useLoggedIn } from "@codeup/hooks";

export default function TemplatePublisher(props: {
  isOpen: boolean;
  onOpen: () => void;
  onClose: () => void;
  projectId: string;
  setProjectList: (list: any) => void;
  projectList: {
    name: string;
    id: string;
    permissions: string[];
    hasTemplate: boolean;
  }[];
}) {
  const toast = useToast();

  const { loggedIn, userInfo } = useLoggedIn();

  return (
    <>
      <Modal isOpen={props.isOpen} onClose={props.onClose}>
        <ModalOverlay />
        <ModalContent>
          <form
            onSubmit={async (e) => {
              e.preventDefault();
```

```
const target = e.target as HTMLFormElement;

const displayName = (
  target.elements.namedItem("displayName") as HTMLInputElement
).value;
const description = (
  target.elements.namedItem("description") as HTMLTextAreaElement
).value;

if (displayName.length < 4) {
  toast({
    status: "error",
    position: "top",
    isClosable: true,
    duration: 3000,
    variant: "solid",
    title: "Fehler",
    description:
      "Der Anzeigename muss mindestens 4 Buchstaben lang sein.",
  });
  return;
}

if (description.length < 50) {
  toast({
    status: "error",
    position: "top",
    isClosable: true,
    duration: 3000,
    variant: "solid",
    title: "Fehler",
    description:
      "Die Beschreibung muss mindestens 50 Buchstaben lang sein.",
  });
  return;
}

const res = await REST.IDE.publishTemplate({
  token: localStorage.getItem("token") as string,
  projectId: props.projectId,
  displayName,
  description,
});

let y = props.projectList;

if (y.find((p) => p.id === props.projectId) !== undefined) {
  (
    y.find((p) => p.id === props.projectId) as {
      hasTemplate: boolean | undefined;
    }
  ).hasTemplate = true;
}

props.setProjectList(y);

props.onClose();

toast({
  status: "success",
```

```
            position: "top",
            isClosable: true,
            duration: 3000,
            variant: "solid",
            title: "Erfolgreich",
            description: "Die Vorlage wurde veröffentlicht",
          });
        }}
      >
        <ModalHeader>Vorlage Veröffentlichen</ModalHeader>
        <ModalCloseButton />
        <ModalBody>
          <FormControl>
            <FormLabel>Anzeigename</FormLabel>
            <Input
              type={"text"}
              placeholder={"Anzeigename"}
              id={"displayName"}
              name={"displayName"}
            />
          </FormControl>
          <FormControl mt={"2rem"}>
            <FormLabel>Beschreibung</FormLabel>
            <Textarea
              placeholder={"Beschreibung"}
              id={"description"}
              name={"description"}
            />
          </FormControl>
          <Text mt={"2rem"}>
            <chakra.b color={"red.400"}>Disclaimer:</chakra.b>
            <br />
            Wenn du eine Vorlage veröffentlichst, kann jeder Benutzer den
            kompletten Code deines Projektes einsehen. Bearbeiten können
            weiterhin nur die von dir hinzugefügten Mitwirkenden.
          </Text>
        </ModalBody>

        <ModalFooter>
          <Button variant={"outline"} mr={3} onClick={props.onClose}>
            Abbrechen
          </Button>
          <Button variant="solid" colorScheme={"blue"} type={"submit"}>
            Veröffentlichen
          </Button>
        </ModalFooter>
      </form>
    </ModalContent>
  </Modal>
 </>
 );
}
```

## packages/components/landingpage/BrandStats.tsx

```
/**
 * packages/components/landingpage/Stats.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
```

```
 * Created At: 12.03.2023
 * Original: https://templateskart.com
 */

import * as React from "react";
import {
  Container,
  Text,
  SimpleGrid,
  Box,
  Center,
  Spinner,
} from "@chakra-ui/react";
import SuperText from "./SuperText";
import { ReactNode, useEffect } from "react";

interface StatData {
  id: number;
  label: string;
  score: string | ReactNode;
}

const BrandStats = () => {
  const [statData, setStatData] = React.useState<StatData[]>([]);

  useEffect(() => {
    fetch("/api/stats").then((res) => {
      res.json().then((data) => {
        setStatData([
          {
            id: 1,
            label: "Benutzer",
            score: data.users,
          },
          {
            id: 2,
            label: "Projekte",
            score: data.projects,
          },
          {
            id: 3,
            label: "Lektionen",
            score: data.lessons,
          },
          {
            id: 4,
            label: "Organisationen",
            score: data.orgs,
          },
        ]);
      });
    });
  }, []);

  return (
    <Container maxW="7xl" p={{ base: 5, md: 10 }}>
      <Center>
        <Box textAlign="center">
          <SuperText>Entwickler vertrauen CodeUp</SuperText>
        </Box>
```

```
      </Center>
      <SimpleGrid
        columns={{ base: 1, sm: 2, md: 4 }}
        spacing={{ base: 2, sm: 5 }}
        mt={12}
        mb={4}
      >
        {statData.map((data) => (
          <Box key={data.id} p={{ base: 2, sm: 5 }} textAlign="center">
            <Text fontWeight="extrabold" fontSize="xx-large">
              {data.score}
            </Text>
            <Text fontSize="sm">{data.label}</Text>
          </Box>
        ))}
        {statData.length === 0 ? (
          <Center>
            <Spinner />
          </Center>
        ) : null}
      </SimpleGrid>
    </Container>
  );
};

export default BrandStats;
```

## packages/components/landingpage/Milestones.tsx

```
/**
 * packages/components/landingpage/Milestones.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 12.03.2023
 * Original: https://templateskart.com
 */

import React from "react";
import {
  Box,
  chakra,
  Container,
  Flex,
  HStack,
  Stack,
  Text,
  useBreakpointValue,
  useColorModeValue,
  VStack,
} from "@chakra-ui/react";
import SuperText from "./SuperText";
import { CHeading } from "../global/CHeading";

const Milestones = (props: {
  title: string;
  milestones: {
    date: string;
    title: string;
    description: string;
```

```
    [key: string]: any;
  }[];
}) => {
  const isMobile = useBreakpointValue({ base: true, md: false });
  const isDesktop = useBreakpointValue({ base: false, md: true });

  const ms: {
    id: number;
    date: string;
    title: string;
    description: string;
    [key: string]: any;
  }[] = props.milestones.map((m) => {
    return {
      id: props.milestones.indexOf(m),
      ...m,
    };
  });

  return (
    <Container maxWidth="7xl" p={{ base: 2, sm: 10 }}>
      <Box mb={8} textAlign={"center"}>
        <CHeading size={"xl"}>{props.title}</CHeading>
      </Box>
      {ms.map((milestone) => (
        <Flex key={milestone.id} mb="10px">
          {isDesktop && milestone.id % 2 === 0 && (
            <>
              <EmptyCard />
              <LineWithDot icon={milestone.lwdIcon} />
              <Card {...milestone} />
            </>
          )}

          {/* Mobile view */}
          {isMobile && (
            <>
              <LineWithDot icon={milestone.lwdIcon} />
              <Card {...milestone} />
            </>
          )}

          {/* Desktop view(right card) */}
          {isDesktop && milestone.id % 2 !== 0 && (
            <>
              <Card {...milestone} />

              <LineWithDot icon={milestone.lwdIcon} />
              <EmptyCard />
            </>
          )}
        </Flex>
      ))}
    </Container>
  );
};

interface CardProps {
  id: number;
  title: string;
```

```
  description: string;
  date: string;
  [key: string]: any;
}

const Card = ({ id, title, description, date, ...rest }: CardProps) => {
  const isEvenId = id % 2 == 0;
  let borderWidthValue = isEvenId ? "15px 15px 15px 0" : "15px 0 15px 15px";
  let leftValue = isEvenId ? "-15px" : "unset";
  let rightValue = isEvenId ? "unset" : "-15px";

  const isMobile = useBreakpointValue({ base: true, md: false });
  if (isMobile) {
    leftValue = "-15px";
    rightValue = "unset";
    borderWidthValue = "15px 15px 15px 0";
  }

  return (
    <HStack
      flex={1}
      p={{ base: 3, sm: 6 }}
      bg={"gray.800"}
      spacing={5}
      rounded="lg"
      alignItems="center"
      pos="relative"
      _before={{
        content: `""`,
        w: "0",
        h: "0",
        borderColor: `transparent #1A1B1E transparent`,
        borderStyle: "solid",
        borderWidth: borderWidthValue,
        position: "absolute",
        left: leftValue,
        right: rightValue,
        display: "block",
      }}
    >
      <Box
        // add all other props from rest
        {...rest}
      >
        <Text fontSize="lg" color={"brand.500"}>
          {date}
        </Text>

        <Stack spacing={2} mb={3} textAlign="left">
          <CHeading size={"lg"}>{title}</CHeading>
          <Text fontSize="md">{description}</Text>
        </Stack>
      </Box>
    </HStack>
  );
};

const LineWithDot = (props: { icon?: any }) => {
  return (
    <Flex
```

```
      pos="relative"
      alignItems="center"
      mr={{ base: "40px", md: "40px" }}
      ml={{ base: "0", md: "40px" }}
    >
      <chakra.span
        position="absolute"
        left="50%"
        height="calc(100% + 10px)"
        border="1px solid"
        borderColor={useColorModeValue("gray.200", "gray.700")}
        top="0px"
      ></chakra.span>
      <Box pos="relative" p="15px">
        <Flex
          pos="absolute"
          top="0"
          left="0"
          bottom="0"
          right="0"
          width="100%"
          height="100%"
          backgroundSize="cover"
          backgroundRepeat="no-repeat"
          backgroundPosition="center center"
          bg={"brand.600"}
          borderRadius="100px"
          backgroundImage="none"
          opacity={1}
          color={"black"}
          alignItems={"center"}
          justifyContent={"center"}
        >
          {props.icon ? props.icon : undefined}
        </Flex>
      </Box>
    </Flex>
  );
};

const EmptyCard = () => {
  return (
    <Box
      flex={{ base: 0, md: 1 }}
      p={{ base: 0, md: 6 }}
      bg="transparent"
    ></Box>
  );
};

export default Milestones;
```

## packages/components/landingpage/OverviewSection.tsx

```
/**
 * packages/components/landingpage/HowItWorks.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 12.03.2023
```

```
 * Original: https://templateskart.com
 */

import * as React from "react";
import {
  chakra,
  Container,
  Stack,
  HStack,
  VStack,
  Flex,
  Text,
  Image,
  Box,
} from "@chakra-ui/react";
import SuperText from "./SuperText";

const overviewList = [
  {
    id: 1,
    label: "Lernen",
    subLabel:
      "Schau dir interaktive Videokurse an und lerne so neue Programmiersprachen.",
  },
  {
    id: 2,
    label: "Planen",
    subLabel: "Erstelle dir einen Plan, um deine Ziele zu erreichen.",
  },
  {
    id: 3,
    label: "Verwirklichen",
    subLabel: "Erstelle Projekte und lerne so neue Technologien.",
  },
];

const OverviewSection = () => {
  return (
    <Container maxW="6xl" py={10}>
      <SuperText>Wie funktioniert CodeUp?</SuperText>
      <Stack
        direction={{ base: "column", md: "row" }}
        spacing={{ base: 0, md: 3 }}
        justifyContent="center"
        alignItems="center"
      >
        <VStack
          spacing={4}
          alignItems="flex-start"
          mb={{ base: 5, md: 0 }}
          maxW="md"
        >
          {overviewList.map((data) => (
            <Box key={data.id}>
              <HStack spacing={2}>
                <Flex
                  fontWeight="bold"
                  boxShadow="md"
                  bg={"brand.500"}
                  color={"black"}
```

```
                rounded="full"
                justifyContent="center"
                alignItems="center"
                w={10}
                h={10}
              >
                {data.id}
              </Flex>
              <Text fontSize="xl">{data.label}</Text>
            </HStack>
            <Text fontSize="md" color="gray.200" ml={12}>
              {data.subLabel}
            </Text>
          </Box>
        ))}
      </VStack>
      <Image
        boxSize={{ base: "auto", md: "lg" }}
        objectFit="contain"
        src="/_static/vectors/home/develop-1.svg"
        rounded="lg"
      />
    </Stack>
  </Container>
 );
};

export default OverviewSection;
```

## packages/components/landingpage/SuperText.tsx

```
/**
 * packages/components/landingpage/SuperText.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 12.03.2023
 *
 */

import * as React from "react";
import { Box, Text } from "@chakra-ui/react";
import { CHeading } from "../global/CHeading";

export default function SuperText(props: {
  children: React.ReactNode;
  fontzSize?: string;
}) {
  return (
    <>
      <Box textAlign={"center"} pb={""}>
        <CHeading size={props.fontzSize || "xl"}>{props.children}</CHeading>
      </Box>
    </>
  );
  return (
    <>
      <Text
        fontWeight="extrabold"
        fontSize={props.fontzSize ? props.fontzSize : "4xl"}
```

```
        mb={2}
        textAlign={"center"}
      >
        <Box as="span" display="inline-block" position="relative">
          {props.children}
          <Box
            as="span"
            display="block"
            position="absolute"
            bg={"blue.600"}
            w={"100%"}
            h={"1px"}
          />
        </Box>
      </Text>
    </>
  );
}
```

## packages/components/landingpage/Testimonials.tsx

```
/**
 * packages/components/landingpage/Testimonials.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 12.03.2023
 * Original: https://templateskart.com
 */

import {
  Avatar,
  Box,
  chakra,
  Container,
  Flex,
  SimpleGrid,
  Stack,
  Text,
  useColorModeValue,
} from "@chakra-ui/react";
import React from "react";
import SuperText from "./SuperText";
import REST from "@codeup/rest/dist";

interface TestimonialAttributes {
  name: string;
  position: string;
  company: string;
  content: string;
  image: string;
}

const testimonials0: TestimonialAttributes[] = [
  {
    name: "Markus Lüersmann",
    position: "Head of Engineering",
    company: "Hengst Filtration",
    image: "/_static/customers/markus.jpeg",
```

```
    content: `Ben hat eine einzigartige Plattform und Lernprogramm erschaffen, dass jedem den
Einstieg in die Programmierwelt auf einfache Weise ermöglicht. Die Jury beeindruckte besonders, wie
er seine bemerkenswerte Kenntnisse mit Leidenschaft anderen interessierten Programmierern
weitergeben möchte.`,
  },
  {
    name: "Ben Siebert",
    position: "Entwickler",
    company: "CodeUp",
    image: "https://avatars.githubusercontent.com/u/53553315",
    content: `CodeUp ist eine tolle Plattform, um neue Technologien zu lernen.
    Ich freue mich sehr, dass CodeUp mitlerweile schon so viele Nutzer hat und hoffe, dass es vielen
    Menschen hilft, programmieren zu lernen.`,
  },
  {
    name: "Felix Schlesinger",
    position: "Entwickler",
    image: "/_static/customers/felix.jpg",
    company: "Ratzfatz Foto",
    content: `
    Man kennt viele IDEs, welche für den Informatikunterricht eingesetzt werden.
    CodeUp ist eine Möglichkeit für Schüler einen besseren Einstieg in die Programierung zu finden
und für Lehrkräfte, um ihren Unterricht von Grund auf neu und interessanter zu gestalten.
    Wir setzen CodeUp im Unterrricht ein und sind einfach nur begeistert.`,
  },
];

const Testimonials = () => {
  const [testimonials, setTestimonials] = React.useState<
    TestimonialAttributes[]
  >([]);

  const [loading, setLoading] = React.useState<boolean>(true);

  React.useEffect(() => {
    REST.Landingpage.getTestimonials().then((res) => {
      if (res.status === 200) {
        setTestimonials(res.payload as any);
        setLoading(false);
      } else {
        setTestimonials(testimonials0);
        setLoading(false);
      }
    });
  }, []);

  return (
    <Container maxW="5xl" py={10} px={6}>
      <Flex justify="center" mb={8}>
        <SuperText>Was sagen unsere Benutzer?</SuperText>
      </Flex>
      {loading ? (
        <Text>Meinungen werden geladen...</Text>
      ) : (
        <SimpleGrid
          columns={{ base: 1, md: 2 }}
          placeItems="center"
          spacing={1}
          mt={12}
          mb={4}
```

```
>
 {testimonials.map((obj, index) => (
  <>
   <Stack
    key={index}
    direction={{ base: "column", sm: "row" }}
    spacing={2}
    mb={5}
    justify="center"
   >
    <Stack
     maxW="330px"
     minW={"300px"}
     boxShadow="lg"
     rounded="md"
     p={6}
     pos="relative"
     bg={"gray.900"}
     _after={{
      content: `""`,
      w: "0",
      h: "0",
      borderColor: `transparent ${"#141517"} transparent`,
      borderStyle: "solid",
      borderWidth: "10px 0 10px 10px",
      position: "absolute",
      top: { base: "unset", sm: "45%" },
      right: { base: "unset", sm: "-10px" },
      left: { base: "48%", sm: "unset" },
      bottom: { base: "-15px", sm: "unset" },
      transform: { base: "rotate(90deg)", sm: "unset" },
      display: "block",
     }}
    >
     <Text fontWeight="medium" fontSize="sm">
      {obj.content}
     </Text>
    </Stack>
    <Stack
     direction="column"
     spacing={2}
     p={2}
     justify="flex-end"
     alignItems="center"
    >
     <Avatar
      size="xl"
      showBorder={true}
      borderColor="brand.500"
      name="avatar"
      src={obj.image}
     />
     <Box textAlign="center">
      <Text fontWeight="bold" fontSize="lg">
       {obj.name}
      </Text>
      <Text fontWeight="medium" fontSize="xs" color="white">
       {obj.position}, {obj.company}
      </Text>
     </Box>
```

```
              </Stack>
            </Stack>
          </>
        ))}
      </SimpleGrid>
    )}
  </Container>
 );
};

export default Testimonials;
```

## packages/components/landingpagev2/BrandStats.tsx

```
/**
 * packages/components/landingpage/Stats.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 12.03.2023
 * Original: https://templateskart.com
 */

import * as React from "react";
import {
  Container,
  Text,
  SimpleGrid,
  Box,
  Center,
  Spinner,
  Heading,
} from "@chakra-ui/react";
import { ReactNode, useEffect } from "react";

interface StatData {
  id: number;
  label: string;
  score: string | ReactNode;
}

const BrandStats = () => {
  const [statData, setStatData] = React.useState<StatData[]>([]);

  useEffect(() => {
    fetch("/api/stats").then((res) => {
      res.json().then((data) => {
        setStatData([
          {
            id: 1,
            label: "Benutzer",
            score: data.users,
          },
          {
            id: 2,
            label: "Projekte",
            score: data.projects,
          },
          {
            id: 3,
```

```
          label: "Lektionen",
          score: data.lessons,
        },
        {
          id: 4,
          label: "Organisationen",
          score: data.orgs,
        },
      ]);
    });
  });
}, []);

return (
  <Container maxW="7xl" p={{ base: 5, md: 10 }}>
    <Center>
      <Box textAlign="center">
        <Heading
          color={"brand.500"}
          textAlign={"center"}
          fontFamily={"Jetbrains Mono ExtraBold"}
          letterSpacing={-2}
          fontSize={["2xl", "4xl", "5xl"]}
        >
          Entwickler vertrauen CodeUp
        </Heading>
      </Box>
    </Center>
    <SimpleGrid
      columns={{ base: 1, sm: 2, md: 4 }}
      spacing={{ base: 2, sm: 5 }}
      mt={12}
      mb={4}
    >
      {statData.map((data) => (
        <>
          <Box
            key={data.id}
            bg={"gray.900"}
            p={5}
            rounded={"md"}
            textAlign={"center"}
            boxShadow={"lg"}
          >
            <Text
              fontWeight="extrabold"
              fontSize="xx-large"
              color={"brand.500"}
              fontFamily={"Jetbrains Mono ExtraBold"}
            >
              {data.score}
            </Text>
            <Text fontSize="sm">{data.label}</Text>
          </Box>
        </>
      ))}
      {statData.length === 0 ? (
        <Center>
          <Spinner />
        </Center>
```

```
        ) : null}
      </SimpleGrid>
    </Container>
  );
};

export default BrandStats;
```

## packages/components/landingpagev2/Milestones.tsx

```
/**
 * packages/components/landingpage/Milestones.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 12.03.2023
 * Original: https://templateskart.com
 */

import React from "react";
import {
  Box,
  chakra,
  Container,
  Flex,
  Heading,
  HStack,
  Text,
  useBreakpointValue,
  useColorModeValue,
  VStack,
} from "@chakra-ui/react";

const Milestones = (props: {
  title: string;
  milestones: {
    date: string;
    title: string;
    description: string;
    [key: string]: any;
  }[];
}) => {
  const isMobile = useBreakpointValue({ base: true, md: false });
  const isDesktop = useBreakpointValue({ base: false, md: true });

  const ms: {
    id: number;
    date: string;
    title: string;
    description: string;
    [key: string]: any;
  }[] = props.milestones.map((m) => {
    return {
      id: props.milestones.indexOf(m),
      ...m,
    };
  });

  return (
    <Container maxWidth="7xl" p={{ base: 2, sm: 10 }}>
```

```
      <Box mb={8}>
        <Heading
          color={"brand.500"}
          textAlign={"center"}
          fontFamily={"Jetbrains Mono ExtraBold"}
          letterSpacing={-2}
          fontSize={["2xl", "4xl", "5xl"]}
        >
          {props.title}
        </Heading>
      </Box>
      {ms.map((milestone) => (
        <Flex key={milestone.id} mb="10px">
          {isDesktop && milestone.id % 2 === 0 && (
            <>
              <EmptyCard />
              <LineWithDot />
              <Card {...milestone} />
            </>
          )}

          {/* Mobile view */}
          {isMobile && (
            <>
              <LineWithDot />
              <Card {...milestone} />
            </>
          )}

          {/* Desktop view(right card) */}
          {isDesktop && milestone.id % 2 !== 0 && (
            <>
              <Card {...milestone} />

              <LineWithDot />
              <EmptyCard />
            </>
          )}
        </Flex>
      ))}
    </Container>
  );
};

interface CardProps {
  id: number;
  title: string;
  description: string;
  date: string;
  [key: string]: any;
}

const Card = ({ id, title, description, date, ...rest }: CardProps) => {
  const isEvenId = id % 2 == 0;
  let borderWidthValue = isEvenId ? "15px 15px 15px 0" : "15px 0 15px 15px";
  let leftValue = isEvenId ? "-15px" : "unset";
  let rightValue = isEvenId ? "unset" : "-15px";

  const isMobile = useBreakpointValue({ base: true, md: false });
  if (isMobile) {
```

```
    leftValue = "-15px";
    rightValue = "unset";
    borderWidthValue = "15px 15px 15px 0";
  }

  return (
    <HStack
      flex={1}
      p={{ base: 3, sm: 6 }}
      bg={"gray.800"}
      spacing={5}
      rounded="lg"
      alignItems="center"
      pos="relative"
      _before={{
        content: `""`,
        w: "0",
        h: "0",
        borderColor: `transparent #1A1B1E transparent`,
        borderStyle: "solid",
        borderWidth: borderWidthValue,
        position: "absolute",
        left: leftValue,
        right: rightValue,
        display: "block",
      }}
    >
      <Box
        // add all other props from rest
        {...rest}
      >
        <Text fontSize="lg" color={"gray.200"}>
          {date}
        </Text>

        <VStack spacing={2} mb={3} textAlign="left">
          <Heading
            fontSize="2xl"
            lineHeight={1.2}
            w="100%"
            fontFamily={"Jetbrains Mono ExtraBold"}
            color={"brand.500"}
          >
            {title}
          </Heading>
          <Text fontSize="md">{description}</Text>
        </VStack>
      </Box>
    </HStack>
  );
};

const LineWithDot = () => {
  return (
    <Flex
      pos="relative"
      alignItems="center"
      mr={{ base: "40px", md: "40px" }}
      ml={{ base: "0", md: "40px" }}
    >
```

```
      <chakra.span
        position="absolute"
        left="50%"
        height="calc(100% + 10px)"
        border="1px solid"
        borderColor={useColorModeValue("gray.200", "gray.700")}
        top="0px"
      ></chakra.span>
      <Box pos="relative" p="10px">
        <Box
          pos="absolute"
          top="0"
          left="0"
          bottom="0"
          right="0"
          width="100%"
          height="100%"
          backgroundSize="cover"
          backgroundRepeat="no-repeat"
          backgroundPosition="center center"
          bg={"brand.600"}
          borderRadius="100px"
          backgroundImage="none"
          opacity={1}
        ></Box>
      </Box>
    </Flex>
  );
};

const EmptyCard = () => {
  return (
    <Box
      flex={{ base: 0, md: 1 }}
      p={{ base: 0, md: 6 }}
      bg="transparent"
    ></Box>
  );
};

export default Milestones;
```

## packages/components/landingpagev2/Testimonials.tsx

```
/**
 * packages/components/landingpagev2/Testimonials.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2024 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 08.05.2024
 *
 */

import * as React from "react";
import {
  Avatar,
  Box,
  Container,
  Heading,
  SimpleGrid,
```

```
  Stack,
  Text,
} from "@chakra-ui/react";
import REST from "@codeup/rest/dist";

interface TestimonialAttributes {
  name: string;
  position: string;
  company: string;
  content: string;
  image: string;
}

const testimonials0: TestimonialAttributes[] = [
  {
    name: "Markus Lüersmann",
    position: "Head of Engineering",
    company: "Hengst Filtration",
    image: "/_static/customers/markus.jpeg",
    content: `Ben hat eine einzigartige Plattform und Lernprogramm erschaffen, dass jedem den
Einstieg in die Programmierwelt auf einfache Weise ermöglicht. Die Jury beeindruckte besonders, wie
er seine bemerkenswerte Kenntnisse mit Leidenschaft anderen interessierten Programmierern
weitergeben möchte.`,
  },
  {
    name: "Ben Siebert",
    position: "Entwickler",
    company: "CodeUp",
    image: "https://avatars.githubusercontent.com/u/53553315",
    content: `CodeUp ist eine tolle Plattform, um neue Technologien zu lernen.
    Ich freue mich sehr, dass CodeUp mitlerweile schon so viele Nutzer hat und hoffe, dass es vielen
    Menschen hilft, programmieren zu lernen.`,
  },
  {
    name: "Felix Schlesinger",
    position: "Entwickler",
    image: "/_static/customers/felix.jpg",
    company: "Ratzfatz Foto",
    content: `
    Man kennt viele IDEs, welche für den Informatikunterricht eingesetzt werden.
    CodeUp ist eine Möglichkeit für Schüler einen besseren Einstieg in die Programierung zu finden
und für Lehrkräfte, um ihren Unterricht von Grund auf neu und interessanter zu gestalten.
    Wir setzen CodeUp im Unterrricht ein und sind einfach nur begeistert.`,
  },
];

export default function Testimonials() {
  const [testimonials, setTestimonials] = React.useState<
    TestimonialAttributes[]
  >([]);

  const [loading, setLoading] = React.useState<boolean>(true);

  React.useEffect(() => {
    REST.Landingpage.getTestimonials().then((res) => {
      if (res.status === 200) {
        setTestimonials(res.payload as any);
        setLoading(false);
      } else {
        setTestimonials(testimonials0);
```

```
      setLoading(false);
    }
  });
}, []);
return (
  <>
    <Box p={6} mt={6} mb={6} roundedTop={"xl"}>
      <Heading
        color={"brand.500"}
        textAlign={"center"}
        fontFamily={"Jetbrains Mono ExtraBold"}
        letterSpacing={-2}
        fontSize={["2xl", "4xl", "5xl"]}
      >
        Was Unsere Kunden Sagen
      </Heading>

      <Container maxW="8xl" py={10} px={6}>
        {loading ? (
          <Text>Meinungen werden geladen...</Text>
        ) : (
          <SimpleGrid
            columns={{ base: 1, md: 2 }}
            placeItems="center"
            spacing={1}
            mt={12}
            mb={4}
          >
            {testimonials.map((obj, index) => (
              <>
                <Stack
                  key={index}
                  direction={{ base: "column", sm: "row" }}
                  spacing={2}
                  mb={5}
                  justify="center"
                >
                  <Stack
                    maxW="330px"
                    minW={"300px"}
                    boxShadow="lg"
                    rounded="md"
                    p={6}
                    pos="relative"
                    bg={"black"}
                    _after={{
                      content: `""`,
                      w: "0",
                      h: "0",
                      borderColor: `transparent ${"#141517"} transparent`,
                      borderStyle: "solid",
                      borderWidth: "10px 0 10px 10px",
                      position: "absolute",
                      top: { base: "unset", sm: "45%" },
                      right: { base: "unset", sm: "-10px" },
                      left: { base: "48%", sm: "unset" },
                      bottom: { base: "-15px", sm: "unset" },
                      transform: { base: "rotate(90deg)", sm: "unset" },
                      display: "block",
                    }}
```

```
                >
                  <Text fontWeight="medium" fontSize="md">
                    {obj.content}
                  </Text>
                </Stack>
                <Stack
                  direction="column"
                  spacing={2}
                  p={2}
                  justify="flex-end"
                  alignItems="center"
                >
                  <Avatar
                    size="xl"
                    showBorder={true}
                    borderColor="brand.500"
                    name="avatar"
                    src={obj.image}
                  />
                  <Box textAlign="center">
                    <Text
                      fontWeight="bold"
                      fontSize="lg"
                      color={"brand.500"}
                    >
                      {obj.name}
                    </Text>
                    <Text fontWeight="medium" fontSize="xs" color="white">
                      {obj.position}, {obj.company}
                    </Text>
                  </Box>
                </Stack>
              </Stack>
            </>
          ))}
        </SimpleGrid>
      )}
    </Container>
  </Box>
 </>
 );
}
```

## packages/components/orgs/OrgAdminLayout.tsx

```
/**
 * packages/components/orgs/OrgAdminLayout.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 07.03.23
 *
 */
import React, { ReactNode, useState } from "react";
import { useLoggedIn } from "@codeup/hooks";
import { useRouter } from "next/router";
import REST from "@codeup/rest";
import AccountManager from "@codeup/util/dist/AccountManager";
import AuthenticateLoadingIndicator from "../admin/AuthenticateLoadingIndicator";
```

```
export default function OrgAdminLayout(props: {
  children: ReactNode;
  orgName: string;
}) {
  const { userInfo, loggedIn, loaded } = useLoggedIn();
  const router = useRouter();
  const [loading, setLoading] = useState(true);

  React.useEffect(() => {
    if (!router || !router.query || !router.query.name) return;
    if (loaded && !loggedIn) {
      router.push("/#login");
      window.dispatchEvent(new CustomEvent("codeup::open_login"));
    } else {
      REST.Orgs.getOrg({
        token: AccountManager.getActiveToken(),
        name: router.query.name as string,
      }).then((r) => {
        if (r.status === 200) {
          setLoading(false);
        } else {
          router.push("/error?error=Organisation konnte nicht gefunden werden");
        }
      });
    }
  }, [loaded, loggedIn, router]);

  return (
    <>{loading ? <AuthenticateLoadingIndicator /> : <>{props.children}</>}</>
  );
}
```

## packages/components/package.json

```
{
  "name": "@codeup/components",
  "version": "0.0.0",
  "license": "GPL-3.0-only",
  "scripts": {
    "lint": "eslint *.ts*",
    "build": "tsc"
  },
  "devDependencies": {
    "eslint": "^7.32.0",
    "eslint-config-custom": "*",
    "tsconfig": "*",
    "typescript": "^4.5.2"
  },
  "dependencies": {
    "@brainstack/diagram": "^1.0.16",
    "@chakra-ui/react": "^2.1.2",
    "@choc-ui/chakra-autocomplete": "^5.1.4",
    "@codeup/hooks": "*",
    "@codeup/rest": "*",
    "@codeup/util": "*",
    "@emotion/react": "^11",
    "@emotion/styled": "^11",
    "@monaco-editor/react": "4.4.5",
    "@react-email/components": "0.0.6",
    "@types/react": "^18.0.21",
    "@types/react-dom": "^17.0.11",
```

```
    "monaco-editor": "0.33.0",
    "plyr-react": "^5.1.0",
    "prism-react-renderer": "^1.3.5",
    "react": "^18.2.0",
    "react-icons": "^5.2.1",
    "react-infinite-scroll-component": "^6.1.0",
    "react-player": "^2.11.0",
    "reactflow": "^11.3.0",
    "socket.io-client": "^4.5.4"
  }
}
```

## packages/components/projectflows/IssueNode.tsx

```
/**
 * packages/components/projectflows/IssueNode.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2024 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 01.06.2024
 *
 */

import * as React from "react";
import { useCallback, useEffect } from "react";
import { useRouter } from "next/router";
import {
  Box,
  Button,
  FormControl,
  Stack,
  Text,
  Textarea,
} from "@chakra-ui/react";
import { CHeading } from "../global/CHeading";
import { FaSave } from "react-icons/fa";
import REST from "@codeup/rest/dist";
import AccountManager from "@codeup/util/dist/AccountManager";
import PopupManager from "@codeup/web/src/global/PopupManager";
import { Handle, Position } from "reactflow";
import { IIssue } from "@codeup/web/src/types/IIssue";

export default function IssueNode({
  data,
  isConnectable,
}: {
  data: string;
  isConnectable: boolean;
}) {
  const onChange = useCallback((evt: any) => {}, []);

  const router = useRouter();

  const [issue, setIssue] = React.useState<IIssue | null>(null);

  const reloadIssue = async () => {
    if (data !== undefined && data.length > 0) {
      const projectID = data[0];
      const issueID = data[1];
```

```
      const res = await REST.Projects.getIssues(
        AccountManager.getActiveToken(),
        projectID,
        issueID
      );

      if (res.status === 200) {
        setIssue(res.payload.issue);
      }
    }
  };

  useEffect(() => {
    reloadIssue();
  }, [data]);

  return (
    <>
      <Box p={6} w={"100%"} bg={"gray.900"} rounded={"xl"}>
        <Handle
          type={"target"}
          position={Position.Left}
          isConnectable={isConnectable}
        />
        <Box textAlign={"center"}>
          <CHeading size={"xl"}>Problem</CHeading>
        </Box>
        <Box>
          <Text>{issue?.description || "Lade Problem..."}</Text>
        </Box>
        <Handle
          type={"source"}
          position={Position.Right}
          isConnectable={isConnectable}
        />
      </Box>
    </>
  );
}
```

## packages/components/roadmap/CourseSelectorNode.tsx

```
/**
 * packages/components/roadmap/CourseSelectorNode.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 07.04.2023
 *
 */

import * as React from "react";
import { useCallback } from "react";
import REST from "@codeup/rest/dist";
import AccountManager from "@codeup/util/dist/AccountManager";
import {
  Box,
  Divider,
  Flex,
  FormControl,
```

```
  Heading,
  IconButton,
  Image,
  Stack,
  StackDivider,
  Text,
} from "@chakra-ui/react";
import SuperText from "../landingpage/SuperText";
import PopupManager from "@codeup/web/src/global/PopupManager";
import { FaTrash } from "react-icons/fa";
import { useRouter } from "next/router";
import { Handle, Position } from "reactflow";
import {
  AutoComplete,
  AutoCompleteInput,
  AutoCompleteItem,
  AutoCompleteList,
} from "@choc-ui/chakra-autocomplete";
import AuthenticateLoadingIndicator from "../admin/AuthenticateLoadingIndicator";
import { CHeading } from "../global/CHeading";

export default function CourseSelectorNode({
  data,
  isConnectable,
}: {
  data: string[];
  isConnectable: boolean;
}) {
  const onChange = useCallback((evt: any) => {}, []);

  const [loaded, setLoaded] = React.useState<boolean>(false);
  const [courses, setCourses] = React.useState<any[]>([]);
  const [availableCourses, setAvailableCourses] = React.useState<any[]>([]);

  React.useEffect(() => {
    (async () => {
      let courses: any = [];
      let availableCourses: any = [];
      for (const course of data) {
        const res = await REST.Course.getCourse({
          token: AccountManager.getActiveToken(),
          course: course,
        });

        if (res.status === 200) {
          courses.push(res.payload.course);
        }
      }

      setCourses(courses);

      const res = await REST.Course.listCourses({
        token: AccountManager.getActiveToken(),
      });

      if (res.status === 200) {
        availableCourses = res.payload.courses;
      }

      setAvailableCourses(availableCourses);
```

```
    setLoaded(true);
  })();
}, []);

const router = useRouter();

return (
  <>
    <Box p={6} w={"100%"} bg={"gray.900"} rounded={"xl"}>
      <Handle type={"target"} position={Position.Left} />
      <Box textAlign={"center"}>
        <CHeading size={"xl"}>Kurse</CHeading>
      </Box>
      {loaded ? (
        <>
          <FormControl mb={2} mt={6} w={"100%"}>
            <AutoComplete
              suggestWhenEmpty
              openOnFocus
              emptyState={"Keine Kurse gefunden"}
              onChange={async (v: string) => {
                if (v === "") return;

                (
                  document.querySelector(
                    "#course_ac_input"
                  ) as HTMLInputElement
                ).value = "";

                const course = availableCourses.find((c) => c.name === v);

                if (!course) return;

                const res = await REST.Roadmap.updateRoad({
                  token: AccountManager.getActiveToken(),
                  id: router.query.id as string,
                  update: {
                    linkedCourses: [...courses, course].map((c) => c._id),
                  },
                });

                if (res.status !== 200) {
                  PopupManager.alert({
                    title: "Fehler",
                    description:
                      "Es ist ein Fehler aufgetreten: " + res.payload.error,
                  });
                  return;
                }

                setCourses([...courses, course]);
              }}
            >
              <AutoCompleteInput
                variant={"filled"}
                placeholder={"Kurs hinzufügen"}
                id={"course_ac_input"}
                w={"100%"}
              />
              <AutoCompleteList backgroundColor={"gray.900"} w={"100%"} p={4}>
```

```
      {availableCourses
        .filter((a) => {
          return !courses.find((c) => c._id === a._id);
        })
        .map((a) => {
          return (
            <>
              <AutoCompleteItem
                value={a.name}
                key={"course_" + a._id}
                align={"center"}
              >
                <Image w={20} src={a.splashImage} mr={4} />
                <Stack>
                  <Heading size={"md"}>{a.name}</Heading>
                  <Text>von @{a.creator}</Text>
                </Stack>
              </AutoCompleteItem>
            </>
          );
        })}
    </AutoCompleteList>
  </AutoComplete>
</FormControl>
<Stack>
  <StackDivider />
  <Divider />
  {courses.map((c) => {
    return (
      <>
        <Flex
          w={"100%"}
          alignItems={"center"}
          gap={5}
          p={2}
          _hover={{
            cursor: "pointer",
            backgroundColor: "gray.700",
          }}
          onClick={(e) => {
            if (e.target === e.currentTarget) {
              router.push("/course/" + c._id);
            }
          }}
        >
          <Image w={20} src={c.splashImage} mr={4} flex={"20%"} />
          <Stack flex={"70%"}>
            <Heading size={"md"}>{c.name}</Heading>
            <Text>von @{c.creator}</Text>
          </Stack>
          <IconButton
            flex={"10%"}
            aria-label={"Kurs entfernen"}
            icon={<FaTrash />}
            colorScheme={"red"}
            variant={"ghost"}
            onClick={async () => {
              const res = await REST.Roadmap.updateRoad({
                token: AccountManager.getActiveToken(),
                id: router.query.id as string,
```

```
                update: {
                  linkedCourses: courses
                    .filter((c2) => c2._id !== c._id)
                    .map((c) => c._id),
                },
              });

              if (res.status !== 200) {
                PopupManager.alert({
                  title: "Fehler",
                  description:
                    "Es ist ein Fehler aufgetreten: " +
                    res.payload.error,
                });
                return;
              }

              setCourses(courses.filter((c2) => c2._id !== c._id));
            }}
          />
        </Flex>
        <StackDivider />
        <Divider />
      </>
    );
  })}
</Stack>
</>
) : (
  <AuthenticateLoadingIndicator text={"Lade Kurse..."} inline={true} />
)}
<Handle type={"source"} position={Position.Right} />
</Box>
</>
);
}
```

## packages/components/roadmap/EditorProjectSelectorNode.tsx

```
/**
 * packages/components/roadmap/EditorProjectSelectorNode.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 07.04.2023
 *
 */

import * as React from "react";
import { useCallback, useEffect } from "react";
import REST from "@codeup/rest/dist";
import AccountManager from "@codeup/util/dist/AccountManager";
import {
  Box,
  Button,
  Divider,
  Flex,
  FormControl,
  Heading,
  IconButton,
```

```
  Image,
  Stack,
  StackDivider,
  Text,
  Textarea,
  useDisclosure,
} from "@chakra-ui/react";
import SuperText from "../landingpage/SuperText";
import PopupManager from "@codeup/web/src/global/PopupManager";
import { FaSave, FaTrash } from "react-icons/fa";
import { useRouter } from "next/router";
import { Handle, Position } from "reactflow";
import {
  AutoComplete,
  AutoCompleteInput,
  AutoCompleteItem,
  AutoCompleteList,
} from "@choc-ui/chakra-autocomplete";
import AuthenticateLoadingIndicator from "../admin/AuthenticateLoadingIndicator";
import { CHeading } from "../global/CHeading";

export default function EditorProjectSelectorNode({
  data,
  isConnectable,
}: {
  data: string;
  isConnectable: boolean;
}) {
  const onChange = useCallback((evt: any) => {}, []);
  const [loaded, setLoaded] = React.useState<boolean>(false);

  const [availableProjects, setAvailableProjects] = React.useState<
    {
      _id: string;
      name: string;
    }[]
  >([]);

  const [project, setProject] = React.useState<{
    _id: string;
    name: string;
  }>();

  const router = useRouter();

  useEffect(() => {
    reload();
  }, []);

  const reload = async () => {
    const res = await REST.Editor.getProjectsV2(
      AccountManager.getActiveToken()
    );

    setLoaded(true);

    if (res.status === 200) {
      setAvailableProjects(res.payload.projects);

      if (data === undefined || data.length === 0) return;
```

```
    setProject(res.payload.projects.find((p: any) => p._id === data));
  }
};

return (
  <>
    <Box
      p={6}
      w={"100%"}
      borderWidth={"2px"}
      borderStyle={"solid"}
      rounded={"xl"}
      borderColor={"brand.500"}
      bg={"gray.900"}
    >
      <Handle
        type={"target"}
        position={Position.Left}
        isConnectable={isConnectable}
      />
      <Box textAlign={"center"}>
        <CHeading size={"xl"}>Projekt</CHeading>
      </Box>
      {loaded ? (
        <>
          <FormControl mb={2} mt={6} w={"100%"}>
            <AutoComplete
              suggestWhenEmpty
              openOnFocus
              emptyState={"Keine Projekte gefunden"}
              onChange={async (v: string) => {
                if (v === "") return;

                (
                  document.querySelector(
                    "#course_ac_input"
                  ) as HTMLInputElement
                ).value = "";

                const project = availableProjects.find(
                  (p: any) => p.name === v
                );

                if (!project) return;

                const res = await REST.Roadmap.updateRoad({
                  token: AccountManager.getActiveToken(),
                  id: router.query.id as string,
                  update: {
                    linkedProject: project._id,
                  },
                });

                if (res.status !== 200) {
                  PopupManager.alert({
                    title: "Fehler",
                    description:
                      "Es ist ein Fehler aufgetreten: " + res.payload.error,
                  });
```

```
        return;
      }

      setProject(project);
      setAvailableProjects(
        availableProjects.filter((p: any) => p._id !== project._id)
      );
    }}
  >
    <AutoCompleteInput
      variant={"filled"}
      placeholder={"Projekt verbinden"}
      id={"todo_ac_input"}
      w={"100%"}
    />
    <AutoCompleteList backgroundColor={"gray.900"} w={"100%"} p={4}>
      {availableProjects
        .filter((a) => {
          if (!project) return true;

          return a._id !== project!._id;
        })
        .map((a) => {
          return (
            <>
              <AutoCompleteItem
                value={a.name}
                key={"project_" + a._id}
                align={"center"}
              />
            </>
          );
        })}
    </AutoCompleteList>
  </AutoComplete>
</FormControl>
{project && (
  <Stack>
    <Flex
      w={"100%"}
      alignItems={"center"}
      gap={5}
      p={2}
      _hover={{
        cursor: "pointer",
        backgroundColor: "gray.700",
      }}
      onClick={(e) => {
        if (
          !["button", "svg", "path"].includes(
            (e.target as HTMLElement).tagName!.toLowerCase()
          )
        ) {
          router.push("/editor/" + project._id);
        }
      }}
    >
      <Stack flex={"70%"}>
        <Heading size={"md"}>{project.name}</Heading>
      </Stack>
```

```
                <IconButton
                  flex={"10%"}
                  aria-label={"Projekt entfernen"}
                  icon={<FaTrash />}
                  colorScheme={"red"}
                  variant={"ghost"}
                  onClick={async () => {
                    if (
                      !(await PopupManager.confirmAsync({
                        title: "Projekt entfernen",
                        question: "Möchtest du das Projekt entfernen?",
                      }))
                    ) {
                      return;
                    }

                    const res = await REST.Roadmap.updateRoad({
                      token: AccountManager.getActiveToken(),
                      id: router.query.id as string,
                      update: {
                        linkedProject: "",
                      },
                    });

                    if (res.status !== 200) {
                      PopupManager.alert({
                        title: "Fehler",
                        description:
                          "Es ist ein Fehler aufgetreten: " +
                          res.payload.error,
                      });
                      return;
                    }

                    setProject(undefined);
                    setAvailableProjects([
                      ...availableProjects,
                      project as any,
                    ]);
                  }}
                />
              </Flex>
            </Stack>
          )}
        </>
      ) : (
        <>
          <AuthenticateLoadingIndicator text={"Lade Projekte..."} inline />
        </>
      )}
    </Box>
  </>
 );
}
```

## packages/components/roadmap/FlowSelectorNode.tsx

```
/**
 * packages/components/roadmap/FlowSelectorNode.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
```

```
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 07.04.2023
 *
 */

import * as React from "react";
import { useCallback, useEffect } from "react";
import REST from "@codeup/rest/dist";
import AccountManager from "@codeup/util/dist/AccountManager";
import {
  Box,
  Button,
  Divider,
  Flex,
  FormControl,
  Heading,
  IconButton,
  Image,
  Stack,
  StackDivider,
  Text,
  Textarea,
  useDisclosure,
} from "@chakra-ui/react";
import SuperText from "../landingpage/SuperText";
import PopupManager from "@codeup/web/src/global/PopupManager";
import { FaPlus, FaSave, FaTrash } from "react-icons/fa";
import { useRouter } from "next/router";
import { Handle, Position } from "reactflow";
import {
  AutoComplete,
  AutoCompleteInput,
  AutoCompleteItem,
  AutoCompleteList,
} from "@choc-ui/chakra-autocomplete";
import AuthenticateLoadingIndicator from "../admin/AuthenticateLoadingIndicator";
import ListViewer from "../todo/ListViewer";
import { CHeading } from "../global/CHeading";

export default function FlowSelectorNode({
  data,
  isConnectable,
}: {
  data: string[];
  isConnectable: boolean;
}) {
  const onChange = useCallback((evt: any) => {}, []);

  const [loaded, setLoaded] = React.useState<boolean>(false);

  const [usedLists, setUsedLists] = React.useState<string[]>(data);

  const [availableLists, setAvailableLists] = React.useState<any[]>([]);

  const [lists, setLists] = React.useState<any[]>([]);

  const router = useRouter();

  useEffect(() => {
```

```
  reload();
}, []);

const reload = async () => {
  const res = await REST.ToDo.getV2Projects(AccountManager.getActiveToken());

  if (res.status === 200) {
    setAvailableLists(res.payload.projects);
  }

  let l1: any[] = [];
  for (const l of res.payload.projects) {
    if (usedLists.includes(l._id)) {
      l1.push(l);
    }
  }

  setLists(l1);
  setLoaded(true);
};

return (
  <>
    <Box p={6} w={"100%"} bg={"gray.900"} rounded={"xl"}>
      <Handle
        type={"target"}
        position={Position.Left}
        isConnectable={isConnectable}
      />
      <Box textAlign={"center"}>
        <CHeading size={"xl"}>Flows</CHeading>
      </Box>
      {loaded ? (
        <>
          <Stack mt={6}>
            <Button
              variant={"brand"}
              leftIcon={<FaPlus />}
              onClick={async () => {
                const name = await PopupManager.promptAsync({
                  title: "Neuer Flow",
                  inputType: "INPUT",
                  helperText: "Gib einen Namen für den neuen Flow ein",
                });

                if (!name) return;

                const res = await REST.ToDo.createV2Project({
                  token: AccountManager.getActiveToken(),
                  name: name,
                });

                if (res.status !== 200) {
                  PopupManager.alert({
                    title: "Fehler",
                    description:
                      "Es ist ein Fehler aufgetreten: " + res.payload.error,
                  });
                  return;
                }
```

```
      reload();
    }}
  >
    Neuer Flow
  </Button>
</Stack>
<FormControl mb={2} mt={6} w={"100%"}>
  <AutoComplete
    suggestWhenEmpty
    openOnFocus
    emptyState={"Keine Flows gefunden"}
    onChange={async (v: string) => {
      if (v === "") return;

      (
        document.querySelector(
          "#course_ac_input"
        ) as HTMLInputElement
      ).value = "";

      const list = availableLists.find((c) => c.name === v);

      if (!list) return;

      const res = await REST.Roadmap.updateRoad({
        token: AccountManager.getActiveToken(),
        id: router.query.id as string,
        update: {
          linkedFlows: [...usedLists, list._id],
        },
      });

      if (res.status !== 200) {
        PopupManager.alert({
          title: "Fehler",
          description:
            "Es ist ein Fehler aufgetreten: " + res.payload.error,
        });
        return;
      }

      setLists(
        lists.concat({
          ...list,
        })
      );
    }}
  >
    <AutoCompleteInput
      variant={"filled"}
      placeholder={"Flow hinzufügen"}
      id={"course_ac_input"}
      w={"100%"}
    />
    <AutoCompleteList backgroundColor={"gray.900"} w={"100%"} p={4}>
      {availableLists
        .filter((a) => {
          return !lists.find((l) => l._id === a._id);
        })
```

```
        .map((a) => {
          return (
            <>
              <AutoCompleteItem
                value={a.name}
                key={"flow_" + a._id}
                align={"center"}
              />
            </>
          );
        })}
    </AutoCompleteList>
  </AutoComplete>
</FormControl>
<Stack>
  <StackDivider />
  <Divider />
  {lists.map((l) => {
    return (
      <>
        <Flex
          w={"100%"}
          alignItems={"center"}
          gap={5}
          p={2}
          _hover={{
            cursor: "pointer",
            backgroundColor: "gray.700",
          }}
          onClick={(e) => {
            if (
              !["button", "svg", "path"].includes(
                (e.target as HTMLElement).tagName!.toLowerCase()
              )
            ) {
              router.push("/planning/" + l._id);
            }
          }}
        >
          <Stack flex={"70%"}>
            <Heading size={"md"}>{l.name}</Heading>
          </Stack>
          <IconButton
            flex={"10%"}
            aria-label={"Flow entfernen"}
            icon={<FaTrash />}
            colorScheme={"red"}
            variant={"ghost"}
            onClick={async () => {
              if (
                !(await PopupManager.confirmAsync({
                  title: "Flow entfernen",
                  question:
                    "Möchtest du den Flow wirklich entfernen?",
                }))
              ) {
                return;
              }

              const res = await REST.Roadmap.updateRoad({
```

```
                      token: AccountManager.getActiveToken(),
                      id: router.query.id as string,
                      update: {
                        linkedFlows: data.filter((c) => c !== l._id),
                      },
                    });

                    if (res.status !== 200) {
                      PopupManager.alert({
                        title: "Fehler",
                        description:
                          "Es ist ein Fehler aufgetreten: " +
                          res.payload.error,
                      });
                      return;
                    }

                    setLists(lists.filter((c) => c._id !== l._id));
                  }}
                />
              </Flex>
              <StackDivider />
              <Divider />
            </>
          );
        })}
      </Stack>
    </>
  ) : (
    <AuthenticateLoadingIndicator text={"Lade Flows..."} inline={true} />
  )}
  <Handle
    type={"source"}
    position={Position.Right}
    isConnectable={isConnectable}
  />
 </Box>
</>
);
}
```

## packages/components/roadmap/ProjectIdeaSelectorNode.tsx

```
/**
 * packages/components/roadmap/ProjectIdeaSelectorNode.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 07.04.2023
 *
 */

import * as React from "react";
import { useCallback } from "react";
import REST from "@codeup/rest/dist";
import AccountManager from "@codeup/util/dist/AccountManager";
import {
  Box,
  Button,
  Divider,
```

```
  Flex,
  FormControl,
  Heading,
  IconButton,
  Image,
  Stack,
  StackDivider,
  Text,
  Textarea,
} from "@chakra-ui/react";
import SuperText from "../landingpage/SuperText";
import PopupManager from "@codeup/web/src/global/PopupManager";
import { FaSave, FaTrash } from "react-icons/fa";
import { useRouter } from "next/router";
import { Handle, Position } from "reactflow";
import {
  AutoComplete,
  AutoCompleteInput,
  AutoCompleteItem,
  AutoCompleteList,
} from "@choc-ui/chakra-autocomplete";
import { CHeading } from "../global/CHeading";

export default function ProjectIdeaSelectorNode({
  data,
  isConnectable,
}: {
  data: string;
  isConnectable: boolean;
}) {
  const onChange = useCallback((evt: any) => {}, []);

  const router = useRouter();

  return (
    <>
      <Box p={6} w={"100%"} bg={"gray.900"} rounded={"xl"}>
        <Box textAlign={"center"}>
          <CHeading size={"xl"}>Idee</CHeading>
        </Box>
        <FormControl mb={2} mt={6} w={"100%"}>
          <Stack>
            <Textarea
              resize={"none"}
              placeholder={"Projekt Idee eingeben"}
              defaultValue={data}
              id={"project_idea_ta"}
            />
            <Button
              variant={"brand"}
              leftIcon={<FaSave />}
              onClick={async () => {
                const value = (
                  document.querySelector(
                    "#project_idea_ta"
                  ) as HTMLTextAreaElement
                ).value;

                const res = await REST.Roadmap.updateRoad({
                  token: AccountManager.getActiveToken(),
```

```
                id: router.query.id as string,
                update: {
                  projectIdea: value,
                },
              });

              if (res.status !== 200) {
                PopupManager.alert({
                  title: "Fehler",
                  description:
                    "Es ist ein Fehler aufgetreten: " + res.payload.error,
                });
              }
            }}
          >
            Speichern
          </Button>
        </Stack>
      </FormControl>
      <Handle
        type={"source"}
        position={Position.Right}
        isConnectable={isConnectable}
      />
    </Box>
  </>
 );
}
```

## packages/components/roadmap/ToDoSelectorNode.tsx

```
/**
 * packages/components/roadmap/ToDoSelectorNode.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 07.04.2023
 *
 */

import * as React from "react";
import { useCallback, useEffect } from "react";
import REST from "@codeup/rest/dist";
import AccountManager from "@codeup/util/dist/AccountManager";
import {
  Box,
  Button,
  Divider,
  Flex,
  FormControl,
  Heading,
  IconButton,
  Image,
  Stack,
  StackDivider,
  Text,
  Textarea,
  useDisclosure,
} from "@chakra-ui/react";
import SuperText from "../landingpage/SuperText";
```

```
import PopupManager from "@codeup/web/src/global/PopupManager";
import { FaPlus, FaSave, FaTrash } from "react-icons/fa";
import { useRouter } from "next/router";
import { Handle, Position } from "reactflow";
import {
  AutoComplete,
  AutoCompleteInput,
  AutoCompleteItem,
  AutoCompleteList,
} from "@choc-ui/chakra-autocomplete";
import AuthenticateLoadingIndicator from "../admin/AuthenticateLoadingIndicator";
import ListViewer from "../todo/ListViewer";
import { CHeading } from "../global/CHeading";

export default function ToDoSelectorNode({
  data,
  isConnectable,
}: {
  data: string[];
  isConnectable: boolean;
}) {
  const onChange = useCallback((evt: any) => {}, []);

  const [loaded, setLoaded] = React.useState<boolean>(false);

  const [usedLists, setUsedLists] = React.useState<string[]>(data);
  const [currentList, setCurrentList] = React.useState<{
    _id: string;
    name: string;
    tasks: {
      name: string;
      finished: boolean;
      description: string;
      uuid: string;
    }[];
  } | null>(null);

  const [availableLists, setAvailableLists] = React.useState<
    {
      _id: string;
      name: string;
      tasks: {
        name: string;
        finished: boolean;
        description: string;
        uuid: string;
      }[];
    }[]
  >([]);

  const [lists, setLists] = React.useState<
    {
      _id: string;
      name: string;
      tasks: {
        name: string;
        finished: boolean;
        description: string;
        uuid: string;
      }[];
```

```
  }[]
>([]);

const router = useRouter();

useEffect(() => {
  reload();
}, []);

const reload = async () => {
  const res = await REST.ToDo.getMyLists(AccountManager.getActiveToken());

  if (res.status === 200) {
    setAvailableLists(res.payload.lists);
  }

  let l1: any[] = [];
  for (const l of res.payload.lists) {
    if (usedLists.includes(l._id)) {
      l1.push(l);
    }
  }

  setLists(l1);
  setLoaded(true);
};

const { isOpen, onClose, onOpen } = useDisclosure();

return (
  <>
    <ListViewer
      isOpen={isOpen}
      onClose={onClose}
      currentList={currentList}
      setCurrentList={setCurrentList}
      reloadLists={reload}
    />
    <Box p={6} w={"100%"} bg={"gray.900"} rounded={"xl"}>
      <Handle
        type={"target"}
        position={Position.Left}
        isConnectable={isConnectable}
      />
      <Box textAlign={"center"}>
        <CHeading>ToDo-Listen</CHeading>
      </Box>
      {loaded ? (
        <>
          <Stack mt={6}>
            <Button
              variant={"brand"}
              leftIcon={<FaPlus />}
              onClick={async () => {
                const name = await PopupManager.promptAsync({
                  title: "Neue Liste",
                  inputType: "INPUT",
                  helperText: "Gib einen Namen für die neue Liste ein",
                });
```

```
      if (!name) return;

      const res = await REST.ToDo.createList({
        token: AccountManager.getActiveToken(),
        name: name,
      });

      if (res.status !== 200) {
        PopupManager.alert({
          title: "Fehler",
          description:
            "Es ist ein Fehler aufgetreten: " + res.payload.error,
        });
        return;
      }

      reload();
    }}
  >
    Neue Liste
  </Button>
</Stack>
<FormControl mb={2} mt={6} w={"100%"}>
  <AutoComplete
    suggestWhenEmpty
    openOnFocus
    emptyState={"Keine Listen gefunden"}
    onChange={async (v: string) => {
      if (v === "") return;

      (
        document.querySelector(
          "#course_ac_input"
        ) as HTMLInputElement
      ).value = "";

      const list = availableLists.find((c) => c.name === v);

      if (!list) return;

      const res = await REST.Roadmap.updateRoad({
        token: AccountManager.getActiveToken(),
        id: router.query.id as string,
        update: {
          linkedTodos: [...usedLists, list._id],
        },
      });

      if (res.status !== 200) {
        PopupManager.alert({
          title: "Fehler",
          description:
            "Es ist ein Fehler aufgetreten: " + res.payload.error,
        });
        return;
      }

      setLists(
        lists.concat({
          _id: list._id,
```

```
          name: list.name,
          tasks: list.tasks,
        })
      );
    }}
  >
    <AutoCompleteInput
      variant={"filled"}
      placeholder={"Liste hinzufügen"}
      id={"todo_ac_input"}
      w={"100%"}
    />
    <AutoCompleteList backgroundColor={"gray.800"} w={"100%"} p={4}>
      {availableLists
        .filter((a) => {
          return !lists.find((l) => l._id === a._id);
        })
        .map((a) => {
          return (
            <>
              <AutoCompleteItem
                value={a.name}
                key={"todo_" + a._id}
                align={"center"}
              />
            </>
          );
        })}
    </AutoCompleteList>
  </AutoComplete>
</FormControl>
<Stack>
  <StackDivider />
  <Divider />
  {lists.map((l) => {
    return (
      <>
        <Flex
          w={"100%"}
          alignItems={"center"}
          gap={5}
          p={2}
          _hover={{
            cursor: "pointer",
            backgroundColor: "gray.700",
          }}
          onClick={(e) => {
            if (
              !["button", "svg", "path"].includes(
                (e.target as HTMLElement).tagName!.toLowerCase()
              )
            ) {
              setCurrentList(l);
              onOpen();
            }
          }}
        >
          <Stack flex={"70%"}>
            <Heading size={"md"}>{l.name}</Heading>
          </Stack>
```

```
                  <IconButton
                    flex={"10%"}
                    aria-label={"Liste entfernen"}
                    icon={<FaTrash />}
                    colorScheme={"red"}
                    variant={"ghost"}
                    onClick={async () => {
                      if (
                        !(await PopupManager.confirmAsync({
                          title: "Liste entfernen",
                          question:
                            "Möchtest du die Liste wirklich entfernen?",
                        }))
                      ) {
                        return;
                      }

                      const res = await REST.Roadmap.updateRoad({
                        token: AccountManager.getActiveToken(),
                        id: router.query.id as string,
                        update: {
                          linkedTodos: data.filter((c) => c !== l._id),
                        },
                      });

                      if (res.status !== 200) {
                        PopupManager.alert({
                          title: "Fehler",
                          description:
                            "Es ist ein Fehler aufgetreten: " +
                            res.payload.error,
                        });
                        return;
                      }

                      setLists(lists.filter((c) => c._id !== l._id));
                    }}
                  />
                </Flex>
                <StackDivider />
                <Divider />
              </>
            );
          })}
        </Stack>
      </>
    ) : (
      <AuthenticateLoadingIndicator text={"Lade Listen..."} inline={true} />
    )}
    <Handle
      type={"source"}
      position={Position.Right}
      isConnectable={isConnectable}
    />
  </Box>
 </>
 );
}
```

packages/components/todo/ListItem.tsx

```
/**
 * packages/components/todo/ListItem.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 05.08.2022
 *
 */

import * as React from "react";
import {
  Box,
  ButtonGroup,
  Checkbox,
  Flex,
  IconButton,
  Stack,
  Text,
  Tooltip,
  useToast,
} from "@chakra-ui/react";
import { FaPen, FaTrash } from "react-icons/fa";
import { BsCheckCircleFill, BsCircle } from "react-icons/bs";
import REST from "@codeup/rest";
import AccountManager from "@codeup/util/dist/AccountManager";
import PopupManager from "@codeup/web/src/global/PopupManager";
import { BiRename } from "react-icons/bi";

export default function ListItem(props: {
  name: string;
  uuid: string;
  description: string;
  finished: boolean;
  listId: string;
  currentList: any;
  setCurrentList: (cf: any) => void;
  reloadLists: () => void;
}) {
  const toast = useToast();

  const [name, setName] = React.useState(props.name);
  const [description, setDescription] = React.useState(props.description);

  return (
    <>
      <Flex w={"100%"} alignItems={"center"}>
        <Box flex={"10%"}>
          <IconButton
            aria-label={"check-uncheck"}
            icon={
              props.finished ? (
                <BsCheckCircleFill color={"pink"} />
              ) : (
                <BsCircle color={"pink"} />
              )
            }
            variant={"ghost"}
            onClick={async () => {
              const res = await REST.ToDo.toggleItemFinish({
```

```
        token: localStorage.getItem("token") as string,
        listId: props.listId,
        taskUUID: props.uuid,
      });

      props.reloadLists();

      props.setCurrentList({
        ...props.currentList,
        tasks: props.currentList.tasks.map((task: any) => {
          if (task.uuid === props.uuid) {
            return {
              ...task,
              finished: !task.finished,
            };
          } else {
            return task;
          }
        }),
      });
    }}
  />
</Box>
<Box flex={"80%"}>
  <Stack>
    <Text fontSize={"lg"}>{name}</Text>
    <Text>{description}</Text>
  </Stack>
</Box>
<Box flex={"10%"}>
  <ButtonGroup>
    <IconButton
      aria-label={"edit-item"}
      variant="ghost"
      color={"blue.300"}
      icon={<BiRename />}
      onClick={async () => {
        const newName = await PopupManager.promptAsync({
          title: "Gib einen neuen Namen ein",
          helperText: "Der neue Name der Aufgabe",
        });

        if (!newName) return;

        const res = await REST.ToDo.updateItem({
          token: AccountManager.getActiveToken(),
          listId: props.listId,
          taskUUID: props.uuid,
          update: {
            name: newName,
          },
        });

        if (res.status === 200) {
          props.reloadLists();

          setName(newName);
        } else {
          toast({
            position: "top",
```

```
        variant: "solid",
        status: "error",
        title: "Fehler",
        description:
          "Beim Ändern des Namens ist ein Fehler aufgetreten.",
      });
    }
  }}
/>
<IconButton
  aria-label={"edit-item"}
  variant="ghost"
  color={"blue.300"}
  icon={<FaPen />}
  onClick={async () => {
    const newDesc = await PopupManager.promptAsync({
      title: "Gib eine neue Beschreibung ein",
      helperText: "Der Inhalt der Aufgabe",
      inputType: "TEXTAREA",
    });

    if (!newDesc) return;

    const res = await REST.ToDo.updateItem({
      token: AccountManager.getActiveToken(),
      listId: props.listId,
      taskUUID: props.uuid,
      update: {
        description: newDesc,
      },
    });

    if (res.status === 200) {
      props.reloadLists();

      setDescription(newDesc);
    } else {
      toast({
        position: "top",
        variant: "solid",
        status: "error",
        title: "Fehler",
        description:
          "Beim Ändern der Beschreibung ist ein Fehler aufgetreten.",
      });
    }
  }}
/>
<IconButton
  color={"red.300"}
  variant="ghost"
  aria-label={"delete-item"}
  icon={<FaTrash />}
  onClick={async () => {
    const res = await REST.ToDo.deleteTask({
      token: localStorage.getItem("token") as string,
      listId: props.currentList?._id as string,
      taskUUID: props.uuid,
    });
```

```
                if (res.status === 200) {
                  props.reloadLists();
                  let x = props.currentList;
                  (x.tasks as any) = x?.tasks.filter(
                    (y: any) => y.uuid !== props.uuid
                  );
                  props.setCurrentList(x);

                  toast({
                    position: "top",
                    variant: "solid",
                    status: "success",
                    title: "Erfolgreich",
                    description: "Die Aufgabe wurde gelöscht",
                    isClosable: true,
                    duration: 6000,
                  });
                } else {
                  toast({
                    position: "top",
                    variant: "solid",
                    status: "error",
                    title: "Fehler",
                    description: res.payload.error,
                    isClosable: true,
                    duration: 6000,
                  });
                }
              }}
            />
          </ButtonGroup>
        </Box>
      </Flex>
    </>
  );
}
```

## packages/components/todo/ListViewer.tsx

```
/**
 * packages/components/todo/ListView.tsx
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 05.08.2022
 *
 */

import * as React from "react";
import {
  Drawer,
  DrawerBody,
  DrawerFooter,
  DrawerHeader,
  DrawerOverlay,
  DrawerContent,
  DrawerCloseButton,
  useToast,
  Button,
  Flex,
```

```
  IconButton,
  Tooltip,
  Box,
  Heading,
  Accordion,
  AccordionItem,
  AccordionButton,
  AccordionIcon,
  AccordionPanel,
  Popover,
  PopoverContent,
  PopoverHeader,
  PopoverBody,
  PopoverCloseButton,
  PopoverArrow,
  PopoverTrigger,
  Input,
  FormControl,
  FormLabel,
  Text,
  ButtonGroup,
} from "@chakra-ui/react";
import { FaPlus } from "react-icons/fa";
import ListItem from "./ListItem";
import REST from "@codeup/rest";
import PopupManager from "@codeup/web/src/global/PopupManager";

export default function ListViewer(props: {
  isOpen: boolean;
  onClose: () => void;
  currentList: {
    _id: string;
    name: string;
    tasks: {
      name: string;
      finished: boolean;
      description: string;
      uuid: string;
    }[];
  } | null;
  setCurrentList: (cf: any) => void;
  reloadLists: () => void;
}) {
  const toast = useToast();

  if (!props.currentList) {
    return <></>;
  }

  return (
    <>
      <Drawer
        isOpen={props.isOpen}
        placement="right"
        size={["full", "lg"]}
        onClose={props.onClose}
      >
        <DrawerOverlay />
        <DrawerContent backgroundColor={"gray.900"}>
          <DrawerCloseButton />
```

```
<DrawerHeader>{props.currentList.name}</DrawerHeader>

<DrawerBody>
  <Flex w={"100%"} p={"2rem"} direction={"column"} gap={"2rem"}>
    <Flex w={"100%"} alignItems={"center"}>
      <Box flex={"90%"}>
        <Heading fontSize={"3xl"}>Aufgaben</Heading>
      </Box>

      <IconButton
        colorScheme="green"
        aria-label={"new-item"}
        icon={<FaPlus />}
        onClick={async () => {
          const name = await PopupManager.promptAsync({
            title: "Neue Aufgabe",
            helperText: "Gib die neue Aufgabe ein",
          });

          if (!name) return;
          const description = await PopupManager.promptAsync({
            title: "Neue Aufgabe",
            helperText: "Gib eine Beschreibung der Liste ein",
            inputType: "TEXTAREA",
          });

          if (!description) return;

          const res = await REST.ToDo.createTask({
            token: localStorage.getItem("token") as string,
            listId: props.currentList?._id as string,
            task: name,
          });

          if (res.status === 200) {
            const res2 = await REST.ToDo.updateItem({
              token: localStorage.getItem("token") as string,
              listId: props.currentList?._id as string,
              taskUUID: res.payload.task,
              update: {
                description,
              },
            });

            props.reloadLists();
            let x = props.currentList;
            x?.tasks.push({
              name,
              description,
              finished: false,
              uuid: res.payload.task,
            });
            props.setCurrentList(x);

            toast({
              position: "top",
              variant: "solid",
              status: "success",
              title: "Erfolgreich",
              description: "Die Aufgabe wurde erstellt",
```

```
                isClosable: true,
                duration: 6000,
              });
            } else {
              toast({
                position: "top",
                variant: "solid",
                status: "error",
                title: "Fehler",
                description: res.payload.error,
                isClosable: true,
                duration: 6000,
              });
            }
          }}
        />
      </Flex>

      {props.currentList.tasks.map((task) => {
        return (
          <ListItem
            listId={props.currentList?._id || ""}
            name={task.name}
            uuid={task.uuid}
            finished={task.finished}
            description={task.description}
            currentList={props.currentList}
            setCurrentList={props.setCurrentList}
            reloadLists={props.reloadLists}
          />
        );
      })}
      {props.currentList.tasks.length === 0 ? (
        <Text textAlign={"center"} color={"red.300"} fontSize={"lg"}>
          Keine Aufgaben vorhanden
        </Text>
      ) : (
        <></>
      )}
    </Flex>
  </DrawerBody>
 </DrawerContent>
</Drawer>
</>
);
}
```

## packages/components/tsconfig.json

```
{
  "extends": "tsconfig/react-library.json",
  "exclude": ["dist", "build", "node_modules"],
  "compilerOptions": {
    "lib": ["DOM"],
    "jsx": "react",
    "rootDir": ".",
    "outDir": "dist"
  }
}
```

## packages/eslint-config-custom/index.js

```
module.exports = {
  extends: ["next", "prettier"],
  rules: {
    "@next/next/no-html-link-for-pages": "off",
    "react/jsx-key": "off",
  },
};
```

## packages/eslint-config-custom/package.json

```
{
  "name": "eslint-config-custom",
  "version": "0.0.0",
  "main": "index.js",
  "license": "MIT",
  "dependencies": {
    "eslint-config-next": "^12.0.8",
    "eslint-config-prettier": "^8.3.0",
    "eslint-plugin-react": "7.28.0"
  },
  "publishConfig": {
    "access": "public"
  }
}
```

## packages/hooks/hooks/useAnalytics.ts

```
/**
 * @codeup/hooks/hooks/useAnalytics.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 25.11.2022
 *
 */
import { useEffect, useState } from "react";
import REST from "@codeup/rest/dist";
import { useRouter } from "next/router";

export default function useAnalytics() {
  const router = useRouter();

  let lastPath = "";

  useEffect(() => {
    if (lastPath !== router.pathname) {
      REST.Analytics.postAnalytics({
        token: localStorage.getItem("token") as string,
      });
      lastPath = router.pathname;
    }
  }, [router.pathname]);
}
```

## packages/hooks/hooks/useCourseProgress.ts

```
/**
 * src/hooks/useCourseProgress.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
```

```
 * License: Project License
 * Created At: 28.06.2022
 *
 */
import { useEffect, useState } from "react";

export function useCourseProgress(course: string): {
  course: string;
  started: boolean;
  currentSection: string;
  finished: boolean;
  loaded: boolean;
} {
  const [progress, setProgress] = useState<{
    course: string;
    started: boolean;
    currentSection: string;
    finished: boolean;
    loaded: boolean;
  }>({
    course: "",
    started: false,
    currentSection: "0",
    finished: false,
    loaded: false,
  });

  useEffect(() => {
    fetch("/api/course/started", {
      method: "POST",
      headers: {
        "Content-Type": "application/json",
      },
      body: JSON.stringify({
        token: localStorage.getItem("token"),
        course: course,
      }),
    }).then((res) => {
      res.json().then((data) => {
        console.log(data);
        if (res.status === 200) {
          setProgress({
            course: course,
            started: data.progress.started,
            currentSection: data.progress.currentSection,
            finished: data.progress.finished,
            loaded: true,
          });
        } else {
          setProgress({
            course: course,
            started: false,
            currentSection: "0",
            finished: false,
            loaded: true,
          });
        }
      });
    });
  }, [course]);
```

```
  return progress;
}
```

## packages/hooks/hooks/useCourseRedirect.ts

```
/**
 * src/hooks/useCourseRedirect.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 30.06.2022
 *
 */

import { useRouter } from "next/router";
import { useEffect, useState } from "react";
import { default as redirect } from "@codeup/util/dist/userworkflow/redirect";
import { useCourseProgress } from "./useCourseProgress";

export default function useCourseRedirect(course: string) {
  const router = useRouter();
  const [section, setSection] = useState<number>(0);

  useEffect(() => {
    redirect({
      forceAnonymous: false,
      redirectToLogin: true,
      redirectTo: router.pathname,
      allowAnonymous: false,
    });

    setSection(parseInt(router.query.section as string));
  }, [router]);

  const progress = useCourseProgress("html-basics");

  useEffect(() => {
    if (progress.loaded) {
      if (parseInt(progress.currentSection) !== section) {
        if (section !== parseInt(progress.currentSection)) {
          router.push(
            "/course/" + course + "/section/" + progress.currentSection
          );
        }
      }
    }
  }, [progress, section]);
}
```

## packages/hooks/hooks/useHasProfile.ts

```
/**
 * src/hooks/hasProfile.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 12.06.2022
 *
 */
```

```
import { useEffect, useState } from "react";
import REST from "@codeup/rest/dist";
import { useLoggedIn } from "./useLoggedIn";

export default function useHasProfile(): {
  hasProfile: boolean;
  tried: boolean;
} {
  const [hasProfile, setHasProfile] = useState({
    hasProfile: false,
    tried: false,
  });

  const { loggedIn, userInfo } = useLoggedIn();

  useEffect(() => {
    if (loggedIn) {
      REST.Community.existsProfile(userInfo.id).then((res) => {
        if (res.status === 200) {
          setHasProfile({
            hasProfile: true,
            tried: true,
          });
        } else {
          setHasProfile({
            hasProfile: false,
            tried: true,
          });
        }
      });
    }
  }, [userInfo, loggedIn]);

  return hasProfile;
}
```

## packages/hooks/hooks/useHover.ts

```
/**
 * src/hooks/useHover.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 02.07.2022
 *
 */
import { useEffect, useRef, useState } from "react";

export default function useHover() {
  const [value, setValue] = useState(false);

  const ref = useRef(null);

  const handleMouseOver = () => setValue(true);
  const handleMouseOut = () => setValue(false);

  useEffect(
    () => {
      const node = ref.current;
      if (node) {
```

```
      (node as HTMLElement).addEventListener("mouseover", handleMouseOver);
      (node as HTMLElement).addEventListener("mouseout", handleMouseOut);

      return () => {
        (node as HTMLElement).removeEventListener(
          "mouseover",
          handleMouseOver
        );
        (node as HTMLElement).removeEventListener("mouseout", handleMouseOut);
      };
    }
  },
  [ref.current] // Recall only if ref changes
);

  return [ref, value];
}
```

## packages/hooks/hooks/useKeyPress.ts

```
// DISCLAIMER START

/* Original: https://usehooks.com/useKeyPress/ */
/* The original code has been modified */

// DISCLAIMER END

/**
 * @codeup/hooks/hooks/useKeyPress.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 25.07.2022
 *
 */

import { useEffect, useState } from "react";

export default function useKeyPress(
  targetKey: string,
  ctrlKey: boolean = false
): boolean {
  const [keyPressed, setKeyPressed] = useState(false);

  function downHandler(ev: KeyboardEvent): void {
    let ctrlCon = ctrlKey ? ev.ctrlKey : true;
    if (ev.key === targetKey && ctrlCon) {
      ev.preventDefault();
      setKeyPressed(true);
    }
  }

  const upHandler = (ev: KeyboardEvent): void => {
    if (ev.key === targetKey) {
      ev.preventDefault();
      setKeyPressed(false);
    }
  };

  useEffect(() => {
```

```
    window.addEventListener("keydown", downHandler);
    window.addEventListener("keyup", upHandler);

    return () => {
      window.removeEventListener("keydown", downHandler);
      window.removeEventListener("keyup", upHandler);
    };
  }, []);
  return keyPressed;
}
```

## packages/hooks/hooks/useLoggedIn.ts

```
/**
 * src/hooks/useLoggedIn.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 18.09.2022
 *
 */

import REST from "@codeup/rest/dist";
import { useEffect, useState } from "react";

export function useLoggedIn(): {
  loggedIn: boolean;
  loaded: boolean;
  userInfo: {
    id: string;
    email: string;
    firstName: string;
    lastName: string;
    _2fa: boolean;
    username: string;
    sso?: string | boolean;
    role: string;
    subscription?: string;
  };
} {
  const [loggedIn, setLoggedIn] = useState(false);
  const [loaded, setLoaded] = useState(false);
  const [userInfo, setUserInfo] = useState({
    id: "",
    email: "",
    firstName: "",
    lastName: "",
    _2fa: false,
    username: "",
    role: "user",
  });

  useEffect(() => {
    let token: string | null = null;
    const interval = setInterval(() => {
      const newToken = localStorage.getItem("token");
      if (newToken !== null) {
        if (newToken !== token) {
          REST.Account.verify(newToken).then(async (res) => {
            if (res.status === 200) {
```

```
            if (res.payload.token) {
              localStorage.setItem("token", res.payload.token);
              token = res.payload.token;

              const r = await REST.Account.verify(token as string);

              if (r.status === 200) {
                setLoggedIn(true);
                setUserInfo(r.payload.data);
              } else {
                setLoggedIn(false);
              }
              return;
            }

            token = newToken;
            setLoggedIn(true);
            setUserInfo(res.payload.data);
          } else {
            token = null;
            setLoggedIn(false);
            localStorage.removeItem("token");
          }
          setLoaded(true);
        });
      }
    } else {
      token = null;
      setLoggedIn(false);
      setLoaded(true);
    }
  }, 100);
  return () => clearInterval(interval);
 }, []);

 return {
  loggedIn,
  loaded,
  userInfo,
 };
}
```

## packages/hooks/hooks/useUserInfo.ts

```
/**
 * src/hooks/useUserInfo.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 06.06.2022
 *
 */

import { decode } from "jsonwebtoken";
import { useEffect, useState } from "react";

export default function useUserInfo(): {
 id: string;
 email: string;
 firstName: string;
```

```
  lastName: string;
  _2fa: boolean;
  username: string;
} {
  const [userInfo, setUserInfo] = useState({
    id: "",
    email: "",
    firstName: "",
    lastName: "",
    _2fa: false,
    username: "",
  });

  useEffect(() => {
    if (localStorage.getItem("token") !== null) {
      const token = localStorage.getItem("token") as string;
      try {
        const decoded = decode(token);
        setUserInfo(decoded as any);
      } catch (e) {
        localStorage.removeItem("token");
        document.cookie = "jwt=;path=/";
      }
    }
  }, []);

  return userInfo;
}
```

## packages/hooks/hooks/useUserLoggedIn.ts

```
/**
 * src/hooks/useToken.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 05.06.2022
 *
 */
import { useEffect, useState } from "react";
import useUserInfo from "./useUserInfo";

export default function useUserLoggedIn() {
  const [userLoggedIn, setUserLoggedIn] = useState(false);

  const userInfo = useUserInfo();

  useEffect(() => {
    setUserLoggedIn(userInfo.id !== "");
  }, [userInfo]);

  return userLoggedIn;
}
```

## packages/hooks/hooks/useUserLoggedInRedirect.ts

```
/**
 * src/hooks/useUserLoggedInRedirect.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
```

```
 * License: Project License
 * Created At: 05.06.2022
 *
 */
import { useEffect } from "react";
import useUserInfo from "./useUserInfo";

export default function useUserLoggedInRedirect(options: {
  allowAnonymous?: boolean;
  redirectTo?: string;
  redirectToLogin?: boolean;
}) {
  const userInfo = useUserInfo();

  useEffect(() => {
    if (userInfo.id !== "") {
      return;
    }

    if (options.redirectToLogin) {
      window.location.href =
        "/account/login?redirect=" + encodeURIComponent(window.location.href);
    }
  }, [userInfo]);
}
```

## packages/hooks/index.ts

```
/**
 * @codeup/hooks/index.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 03.07.2022
 *
 */

import { useCourseProgress } from "./hooks/useCourseProgress";
import useCourseRedirect from "./hooks/useCourseRedirect";
import useHasProfile from "./hooks/useHasProfile";
import useHover from "./hooks/useHover";
import useUserInfo from "./hooks/useUserInfo";
import useUserLoggedIn from "./hooks/useUserLoggedIn";
import useUserLoggedInRedirect from "./hooks/useUserLoggedInRedirect";
import useKeyPress from "./hooks/useKeyPress";
import { useLoggedIn } from "./hooks/useLoggedIn";

export {
  useCourseRedirect,
  useCourseProgress,
  useHasProfile,
  useHover,
  useUserInfo,
  useUserLoggedInRedirect,
  useUserLoggedIn,
  useKeyPress,
  useLoggedIn,
};
```

## packages/hooks/package.json

```
{
  "name": "@codeup/hooks",
  "version": "1.0.0",
  "description": "Custom Hooks used by CodeUp",
  "main": "index.ts",
  "author": "Ben Siebert",
  "license": "GPL-3.0-only",
  "private": false,
  "devDependencies": {
    "@chakra-ui/react": "^2.1.2",
    "@codeup/rest": "*",
    "@codeup/util": "*",
    "@emotion/react": "^11",
    "@emotion/styled": "^11",
    "@monaco-editor/react": "^4.4.5",
    "@types/react": "^17.0.37",
    "@types/react-dom": "^17.0.11",
    "eslint": "^7.32.0",
    "eslint-config-custom": "*",
    "tsconfig": "*",
    "typescript": "^4.5.2",
    "react": "^18.2.0"
  }
}
```

## packages/hooks/tsconfig.json

```
{
  "extends": "tsconfig/react-library.json",
  "exclude": ["dist", "build", "node_modules"],
  "compilerOptions": {
    "lib": ["DOM"],
    "jsx": "react",
  }
}
```

## packages/i18n/package.json

```
{
  "name": "@codeupspace/i18n",
  "version": "1.0.0",
  "description": "I18n library for CodeUp Projects",
  "main": "dist/index.js",
  "scripts": {
    "build": "tsc"
  },
  "author": {
    "email": "hello@ben-siebert.de",
    "name": "Ben Siebert",
    "url": "https://ben-siebert.com"
  },
  "license": "MIT",
  "dependencies": {
    "typescript": "^5.4.5"
  },
  "publishConfig": {
    "access": "public"
  }
}
```

## packages/i18n/src/index.ts

```
/**
 * packages/i18n/src/index.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2024 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 05.06.2024
 *
 */

export class CodeUpI18n {
  private readonly projectId: string;
  private language: string;

  constructor(projectId: string, language: string) {
    this.projectId = projectId;
    this.language = language;
  }

  setLanguage(language: string) {
    this.language = language;
  }

  async getTranslation(key: string, language?: string) {
    const res = await fetch(
      `http://127.0.0.1/api/projects/${
        this.projectId
      }/i18n/getTranslation?key=${key}&language=${language || this.language}`
    );
    const j = await res.json();
    return j.translation;
  }

  async translateOnlineV3(text: string, to: string, from?: string) {
    const body = {
      text,
      to,
      from: from || "de",
    };

    const res = await fetch(
      "https://translate-cache.ben-siebert.com/translate",
      {
        method: "POST",
        body: JSON.stringify(body),
        headers: {
          "Content-Type": "application/json",
          Accept: "application/json",
        },
      }
    );

    const j = await res.json();

    return j.text;
  }
}
```

## packages/i18n/tsconfig.json

```
{
```

```
  "compilerOptions": {
    "target": "ES6",
    "module": "CommonJS",
    "esModuleInterop": true,
    "forceConsistentCasingInFileNames": true,
    "strict": true,
    "skipLibCheck": true,
    "outDir": "./dist",
    "rootDir": "./src",
    "baseUrl": "./src",
    "declaration": true
  }
}
```

## packages/incode-ast-generator/package.json

```
{
  "name": "@codeup/incode-ast-generator",
  "version": "1.0.0",
  "description": "Integration of InCode's Abstract Syntax Tree (AST) Generator",
  "main": "dist/index.js",
  "repository": "https://github.com/MCTzOCK/codeup",
  "author": "Ben Siebert",
  "license": "GPL-3.0-only",
  "private": false,
  "scripts": {
    "build": "tsc"
  }
}
```

## packages/incode-ast-generator/src/AbstractSyntaxTreeGenerator.ts

```
/**
 * packages/incode-ast-generator/AbstractSyntaxTreeGenerator.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 13.08.2022
 *
 */

// THIS CODE SNIPPET WAS ORIGINALLY CREATED BY BEN SIEBERT FOR THE INCODE
PROGRAMMING LANGUAGE
//
//
// Original:
https://github.com/InCodeDevs/InCode/blob/63f73726d0a02a96a023e0ec81aef3504ffe4f1d/packages
/compiler/src/module/AbstractSyntaxTreeGenerator.ts#L5
// Copyright © Ben Siebert

import { AbstractSyntaxTree } from "./types/AbstractSyntaxTree";
import { PreCompiler } from "./util/PreCompiler";

export class AbstractSyntaxTreeGenerator {
  public static generate(source: string): AbstractSyntaxTree[] {
    source = PreCompiler.preCompile(source);
    const lines = source.split("\n");

    if (lines.length === 0) {
      return [];
    }
```

```
const words: string[][] = this.splitWords(source);

let result: AbstractSyntaxTree[];

let temp: AbstractSyntaxTree[] = [];

words.forEach((wordList) => {
  if (wordList.length === 0) {
    return;
  }

  const ast: AbstractSyntaxTree = {
    intents: 0,
    command: "",
    args: [],
    children: [],
  };

  const firstWord = wordList[0];

  ast.intents = firstWord.split("    ").length - 1;
  ast.command = firstWord.trim();
  ast.args = wordList.slice(1);

  temp.push(ast);
});

temp.forEach((ast) => {
  if (ast.intents === 0) {
    temp.push(ast);
    return;
  }
  if (temp[temp.length - 1].intents === ast.intents - 1) {
    temp[temp.length - 1].children.push(ast);
    return;
  } else {
    let x =
      temp[temp.length - 1].children[
        temp[temp.length - 1].children.length - 1
      ];
    let y = x;
    while (
      x !== null &&
      x.intents !== ast.intents - 1 &&
      x.intents !== ast.intents
    ) {
      x = y.children[y.children.length - 1];
      y = x;
    }
    if (x.intents === ast.intents - 1) {
      x.children.push(ast);
    } else {
      y.children.push(ast);
    }
  }
});

temp = temp.filter((t) => t.intents === 0);
```

```
    temp = Array.from(new Set(temp.map((a) => a))).map((a0) => {
      return temp.find((a) => a === a0);
    }) as AbstractSyntaxTree[];

    result = temp;

    return result;
  }

  private static splitWords(source: string): string[][] {
    const matrix: string[][] = [];

    const lines = source.split("\n");
    lines.forEach((line) => {
      if (line.match(/(["'])(?:(?=(\\?))\2.)*?\1/g)) {
        const quotes = line.match(/(["'])(?:(?=(\\?))\2.)*?\1/g);
        (quotes as string[]).forEach((quote) => {
          line = line.replace(quote, quote.split(" ").join("\0"));
        });
      }
      matrix.push(line.split(" "));
    });

    return matrix;
  }
}
```

## packages/incode-ast-generator/src/index.ts

```
/**
 * packages/incode-ast-generator/index.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 13.08.2022
 *
 */

export { AbstractSyntaxTreeGenerator } from "./AbstractSyntaxTreeGenerator";
export { AbstractSyntaxTree } from "./types/AbstractSyntaxTree";
export { PreCompiler } from "./util/PreCompiler";
```

## packages/incode-ast-generator/src/types/AbstractSyntaxTree.ts

```
/**
 * packages/incode-ast-generator/types/AbstractSyntaxTree.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 13.08.2022
 */

// THIS CODE SNIPPET WAS ORIGINALLY CREATED BY BEN SIEBERT FOR THE INCODE
PROGRAMMING LANGUAGE
//
//
// Original:
https://github.com/InCodeDevs/InCode/blob/63f73726d0a02a96a023e0ec81aef3504ffe4f1d/packages
/compiler/src/types/AbstractSyntaxTree.ts#L6
// Copyright © Ben Siebert
```

```
export interface AbstractSyntaxTree {
  command: string;
  intents: number;
  children: AbstractSyntaxTree[];
  args: string[];
}
```

## packages/incode-ast-generator/src/util/PreCompiler.ts

```
/**
 * packages/incode-ast-generator/util/PreCompiler.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 13.08.2022
 *
 */

// THIS CODE SNIPPET WAS ORIGINALLY CREATED BY BEN SIEBERT FOR THE INCODE
PROGRAMMING LANGUAGE
//
//
// Original:
https://github.com/InCodeDevs/InCode/blob/63f73726d0a02a96a023e0ec81aef3504ffe4f1d/packages
/compiler/src/util/PreCompiler.ts#L6
// Copyright © Ben Siebert

export class PreCompiler {
  public static preCompile(source: string): string {
    source = source.replace(/ {2}/g, "");
    source = source.replace(/\r/g, "");
    return source;
  }
}
```

## packages/incode-ast-generator/tsconfig.json

```
{
  "extends": "tsconfig/base.json",
  "exclude": ["dist", "build", "node_modules"],
  "compilerOptions": {
    "isolatedModules": false,
    "lib": ["DOM"],
    "rootDir": "src",
    "outDir": "dist"
  }
}
```

## packages/incode-blockly/package.json

```
{
  "name": "@codeup/incode-blockly",
  "version": "1.0.0",
  "author": "Ben Siebert",
  "license": "GPL-3.0-only",
  "private": false,
  "scripts": {
    "build": "tsc"
  },
  "dependencies": {
    "blockly": "^9.3.1",
```

```
    "react": "18.2.0"
  }
}
```

## packages/incode-blockly/src/blockly/BlocklyCompiler.ts

```
/**
 * @author Ben Siebert <ben@mctzock.de>
 * @copyright (c) 2018-2021 Ben Siebert. All rights reserved.
 */

import * as Blockly from "blockly";

export class BlocklyCompiler {
  private compiledBlocks: any[] = [];

  /**
   * Compiles the current blockly workspace
   * @return The compiled code
   */
  compile(): string {
    let code = "";
    Blockly.getMainWorkspace()
      .getAllBlocks(true)
      .forEach((block) => {
        if (!this.compiledBlocks.includes(block)) {
          code += this.compileBlock(block);
        }
      });

    // remove empty lines
    code = code.replace(/^\s*[\r ]*\n/gm, "");

    return code;
  }

  /**
   * Compiles a single block given by the compile method
   * @param block The block to be compiled
   */
  private compileBlock(block: Blockly.Block): string {
    this.compiledBlocks.push(block);

    let code = "";

    let b = block.getInputTargetBlock("STATEMENT");

    if (b) {
      code += this.getBlockTabs(block, true) + this.getBlockText(block) + "\n";
      do {
        this.compiledBlocks.push(b);
        code += this.getBlockTabs(b, true) + this.compileBlock(b) + "\n";
        /* eslint-disable  no-cond-assign */
      } while ((b = b.getNextBlock()));
    } else {
      code += this.getBlockTabs(block, true) + this.getBlockText(block) + "\n";
    }

    return code;
  }
```

```
  /**
   * Calculates the TABS (\t) for a given block
   * @param block The block which should be calculated
   * @param log Log the TABS (\t) to the console DEFAULT = false. DEBUGGING ONLY
   * @param b1 Ignore this boolean, idiot. Just kidding, basically I created it for debugging and it now
has no sense anymore
   */
  private getBlockTabs(block: Blockly.Block, log = false, b1 = true): string {
    let tabs = "";

    let currentBlock = block;

    if (block.getParent() != null && b1) {
      let neededTabs = -2;
      neededTabs =
        neededTabs +
        this.getBlockTabs(block.getParent() as Blockly.Block, log, true)
          .length -
        1;
      if ((block.getParent() as Blockly.Block).getInputTargetBlock("STATEMENT"))
        neededTabs++;
      for (let i = 0; i < neededTabs; i++) {
        tabs += "    ";
      }
    }

    while (currentBlock.getParent() != null) {
      let b = (currentBlock.getParent() as Blockly.Block).getInputTargetBlock(
        "STATEMENT"
      );
      if (
        this.getBlockText(currentBlock.getParent() as Blockly.Block).includes(
          "Methode"
        )
      )
        tabs += "    ";

      if (b) {
        do {
          if (b === block) tabs += "    ";
          /* eslint-disable  no-cond-assign */
        } while ((b = b.getNextBlock()));
      }
      currentBlock = currentBlock.getParent() as Blockly.Block;
    }

    return tabs;
  }

  /**
   * Actually compiles a block
   * @param block The block to be compiled
   */
  private getBlockText(block: Blockly.Block): string {
    let c = "";
    block.inputList.forEach((i) => {
      i.fieldRow.forEach((r) => {
        if (r.getValue() != "Start") {
          c += r.getValue() + " ";
        }
```

```
      });
    });
    return c;
  }
}
```

## packages/incode-blockly/src/blockly/blocks/AddToElementBlock.ts

```
/**
 * @author The InCode Devs
 * @copyright 2018-2021 The InCode Developers <https://github.com/InCodeDevs>
 */
import * as Blockly from "blockly";

export class AddToElementBlock {
  /**
   * Registers the "AddToElement" block to the blockly registry
   */
  public static registerBlock() {
    Blockly.defineBlocksWithJsonArray([
      {
        type: "add_to_element",
        message0: "Füge %1 zu %2 hinzu",
        args0: [
          {
            type: "field_input",
            name: "CHILD",
            text: "x",
          },
          {
            type: "field_input",
            name: "PARENT",
            text: "y",
          },
        ],
        previousStatement: null,
        nextStatement: null,
        colour: 120,
        tooltip: "",
        helpUrl: "",
      },
    ]);
  }
}
```

## packages/incode-blockly/src/blockly/blocks/AddToScreenBlock.ts

```
/**
 * @author The InCode Devs
 * @copyright 2018-2021 The InCode Developers <https://github.com/InCodeDevs>
 */
import * as Blockly from "blockly";

export class AddToScreenBlock {
  /**
   * Registers the "AddToScreen" block to the blockly registry
   */
  public static registerBlock() {
    Blockly.defineBlocksWithJsonArray([
      {
        type: "add_to_screen",
        message0: "Füge %1 zum Bildschirm hinzu",
```

```
        args0: [
          {
            type: "field_input",
            name: "VARIABLE_NAME",
            text: "x",
          },
        ],
        previousStatement: null,
        nextStatement: null,
        colour: 120,
        tooltip: "",
        helpUrl: "",
      },
    ]);
  }
}
```

## packages/incode-blockly/src/blockly/blocks/AskBlock.ts

```
/**
 * @author The InCode Devs
 * @copyright 2018-2021 The InCode Developers <https://github.com/InCodeDevs>
 */
import * as Blockly from "blockly";

export class AskBlock {
  /**
   * Registers the "Ask" block to the blockly registry
   */
  public static registerBlock() {
    Blockly.defineBlocksWithJsonArray([
      {
        type: "ask",
        message0: 'Frage "%1" und speicher die Antwort in %2',
        args0: [
          {
            type: "field_input",
            name: "QUESTION",
            text: '"Frage"',
          },
          {
            type: "field_input",
            name: "VARIABLE_NAME",
            text: "x",
          },
        ],
        previousStatement: null,
        nextStatement: null,
        colour: 120,
        tooltip: "",
        helpUrl: "",
      },
    ]);
  }
}
```

## packages/incode-blockly/src/blockly/blocks/CallMethodBlock.ts

```
/**
 * @author The InCode Devs
 * @copyright 2018-2021 The InCode Developers <https://github.com/InCodeDevs>
 */
```

```
import * as Blockly from "blockly";

export class CallMethodBlock {
  /**
   * Registers the "CallMethod" block to the blockly registry
   */
  public static registerBlock() {
    Blockly.defineBlocksWithJsonArray([
      {
        type: "call_method",
        message0: "Rufe die Methode %1 auf",
        args0: [
          {
            type: "field_input",
            name: "METHOD_NAME",
            text: "methodenName",
          },
        ],
        previousStatement: null,
        nextStatement: null,
        colour: 290,
        tooltip: "",
        helpUrl: "",
      },
    ]);
  }
}
```

## packages/incode-blockly/src/blockly/blocks/CreateMethodBlock.ts

```
/**
 * @author The InCode Devs
 * @copyright 2018-2021 The InCode Developers <https://github.com/InCodeDevs>
 */
import * as Blockly from "blockly";

export class CreateMethodBlock {
  /**
   * Registers the "CreateMethod" block to the blockly registry
   */
  public static registerBlock() {
    Blockly.defineBlocksWithJsonArray([
      {
        type: "create_method",
        message0: "Erstelle %1 als Methode",
        args0: [
          {
            type: "field_input",
            name: "METHOD_NAME",
            text: "methodenName",
          },
        ],
        nextStatement: null,
        colour: 290,
        tooltip: "",
        helpUrl: "",
      },
    ]);
  }
}
```

## packages/incode-blockly/src/blockly/blocks/CreateVarBlock.ts

```
/**
 * @author The InCode Devs
 * @copyright 2018-2021 The InCode Developers <https://github.com/InCodeDevs>
 */
import * as Blockly from "blockly";

export class CreateVarBlock {
  /**
   * Registers the "CreateVar" block to the blockly registry
   */
  public static registerBlock() {
    Blockly.defineBlocksWithJsonArray([
      {
        type: "var_create",
        message0: "Erstelle %1",
        args0: [
          {
            type: "field_input",
            name: "VARIABLE_NAME",
            text: "x",
          },
        ],
        previousStatement: null,
        nextStatement: null,
        colour: 30,
        tooltip: "",
        helpUrl: "",
      },
    ]);
  }
}
```

## packages/incode-blockly/src/blockly/blocks/CreateVarTypeBlock.ts

```
/**
 * @author The InCode Devs
 * @copyright 2018-2021 The InCode Developers <https://github.com/InCodeDevs>
 */
import * as Blockly from "blockly";

export class CreateVarTypeBlock {
  /**
   * Registers the "CreateVarType" block to the blockly registry
   */
  public static registerBlock() {
    Blockly.defineBlocksWithJsonArray([
      {
        type: "var_create_type",
        message0: "Erstelle %1 als %2",
        args0: [
          {
            type: "field_input",
            name: "VARIABLE_NAME",
            text: "x",
          },
          {
            type: "field_dropdown",
            name: "VARIABLE_TYPE",
            options: [
              ["Knopf", "Knopf"],
```

```
            ["Absatz", "Absatz"],
            ["Text", "Text"],
            ["Bild", "Bild"],
            ["Überschrift", "Überschrift"],
            ["Eingabefeld", "Eingabefeld"],
            ["Tabelle", "Tabelle"],
            ["Zeile", "Zeile"],
            ["Spalte", "Spalte"],
            ["Zeilenumbruch", "Zeilenumbruch"],
            ["Video", "Video"],
            ["Ton", "Ton"],
            ["Fenster", "Fenster"],
          ],
        },
      ],
      previousStatement: null,
      nextStatement: null,
      colour: 30,
      tooltip: "",
      helpUrl: "",
    },
  ]);
 }
}
```

## packages/incode-blockly/src/blockly/blocks/ElseBlock.ts

```
/**
 * @author The InCode Devs
 * @copyright 2018-2021 The InCode Developers <https://github.com/InCodeDevs>
 */

import * as Blockly from "blockly";

export class ElseBlock {
  /**
   * Registers the "Else" block to the blockly registry
   */
  public static registerBlock() {
    Blockly.defineBlocksWithJsonArray([
      {
        type: "else",
        message0: "Sonst %1",
        args0: [
          {
            type: "input_statement",
            name: "STATEMENT",
          },
        ],
        previousStatement: null,
        nextStatement: null,
        colour: "#5577EE",
        tooltip: "",
        helpUrl: "",
      },
    ]);
  }
}
```

## packages/incode-blockly/src/blockly/blocks/ElseIfNumberBlock.ts

```
/**
```

```
 * @author The InCode Devs
 * @copyright 2018-2021 The InCode Developers <https://github.com/InCodeDevs>
 */

import * as Blockly from "blockly";

export class ElseIfNumberBlock {
  /**
   * Registers the "ElseIfNumber" block to the blockly registry
   */
  public static registerBlock() {
    Blockly.defineBlocksWithJsonArray([
      {
        type: "else_if_number",
        message0: "Sonst wenn %1 %2 %3 ist %4",
        args0: [
          {
            type: "field_input",
            name: "VARIABLE_NAME",
            text: "x",
          },
          {
            type: "field_dropdown",
            name: "OPERATOR",
            options: [
              ["Gleich", "gleich"],
              ["Kleiner", "kleiner"],
              ["Größer", "größer"],
              ["Kleiner oder gleich", "kleiner-gleich"],
              ["Größer oder gleich", "größer-gleich"],
            ],
          },
          {
            type: "field_number",
            name: "NUMBER",
            value: 0,
          },
          {
            type: "input_statement",
            name: "STATEMENT",
          },
        ],
        previousStatement: null,
        nextStatement: null,
        colour: "#5577EE",
        tooltip: "",
        helpUrl: "",
      },
    ]);
  }
}
```

## packages/incode-blockly/src/blockly/blocks/ElseIfTextBlock.ts

```
/**
 * @author The InCode Devs
 * @copyright 2018-2021 The InCode Developers <https://github.com/InCodeDevs>
 */

import * as Blockly from "blockly";
```

```
export class ElseIfTextBlock {
  /**
   * Registers the "ElseIfText" block to the blockly registry
   */
  public static registerBlock() {
    Blockly.defineBlocksWithJsonArray([
      {
        type: "else_if_text",
        message0: "Sonst wenn %1 %2 %3 ist %4",
        args0: [
          {
            type: "field_input",
            name: "VARIABLE_NAME",
            text: "x",
          },
          {
            type: "field_dropdown",
            name: "OPERATOR",
            options: [
              ["Gleich", "gleich"],
              ["Kleiner", "kleiner"],
              ["Größer", "größer"],
              ["Kleiner oder gleich", "kleiner-gleich"],
              ["Größer oder gleich", "größer-gleich"],
            ],
          },
          {
            type: "field_input",
            name: "VALUE",
            text: "Text",
          },
          {
            type: "input_statement",
            name: "STATEMENT",
          },
        ],
        previousStatement: null,
        nextStatement: null,
        colour: "#5577EE",
        tooltip: "",
        helpUrl: "",
      },
    ]);
  }
}
```

## packages/incode-blockly/src/blockly/blocks/FunctionBlock.ts

```
/**
 * packages/incode-blockly/src/blockly/blocks/FunctionBlock.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 04.04.2023
 *
 */
import * as Blockly from "blockly";

export class FunctionBlock {
  /**
```

```
   * Registers the "Function" block to the blockly registry
   */
  public static registerBlock() {
    Blockly.defineBlocksWithJsonArray([
      {
        type: "function",
        message0: "Funktion %1 %2",
        args0: [
          {
            type: "field_input",
            name: "MESSAGE",
            text: "Wert",
          },
          {
            type: "input_statement",
            name: "STATEMENT",
          },
        ],
        colour: 120,
        tooltip: "",
        helpUrl: "",
      },
    ]);
  }
}
```

## packages/incode-blockly/src/blockly/blocks/IfEventBlock.ts

```
/**
 * @author The InCode Devs
 * @copyright 2018-2021 The InCode Developers <https://github.com/InCodeDevs>
 */

import * as Blockly from "blockly";

export class IfEventBlock {
  /**
   * Registers the "IfEvent" block to the blockly registry
   */
  public static registerBlock() {
    Blockly.defineBlocksWithJsonArray([
      {
        type: "if_event",
        message0: "Wenn %1 %2 wird rufe %3 auf",
        args0: [
          {
            type: "field_input",
            name: "VARIABLE_NAME",
            text: "x",
          },
          {
            type: "field_dropdown",
            name: "EVENT",
            options: [
              ["Gedrück", "gedrückt"],
              ["Berührt", "berührt"],
              ["Nicht Berührt", "nicht-berührt"],
            ],
          },
          {
            type: "field_input",
```

```
            name: "METHOD:NAME",
            text: "methodenName",
          },
        ],
        previousStatement: null,
        nextStatement: null,
        colour: "#5577EE",
        tooltip: "",
        helpUrl: "",
      },
    ]);
  }
}
```

## packages/incode-blockly/src/blockly/blocks/IfNumberBlock.ts

```
/**
 * @author The InCode Devs
 * @copyright 2018-2021 The InCode Developers <https://github.com/InCodeDevs>
 */

import * as Blockly from "blockly";

export class IfNumberBlock {
  /**
   * Registers the "IfNumber" block to the blockly registry
   */
  public static registerBlock() {
    Blockly.defineBlocksWithJsonArray([
      {
        type: "if_number",
        message0: "Wenn %1 %2 %3 ist %4",
        args0: [
          {
            type: "field_input",
            name: "VARIABLE_NAME",
            text: "x",
          },
          {
            type: "field_dropdown",
            name: "OPERATOR",
            options: [
              ["Gleich", "gleich"],
              ["Kleiner", "kleiner"],
              ["Größer", "größer"],
              ["Kleiner oder gleich", "kleiner-gleich"],
              ["Größer oder gleich", "größer-gleich"],
            ],
          },
          {
            type: "field_number",
            name: "NUMBER",
            value: 0,
          },
          {
            type: "input_statement",
            name: "STATEMENT",
          },
        ],
        previousStatement: null,
        nextStatement: null,
```

```
        colour: "#5577EE",
        tooltip: "",
        helpUrl: "",
      },
    ]);
  }
}
```

## packages/incode-blockly/src/blockly/blocks/IfTextBlock.ts

```
/**
 * @author The InCode Devs
 * @copyright 2018-2021 The InCode Developers <https://github.com/InCodeDevs>
 */

import * as Blockly from "blockly";

export class IfTextBlock {
  /**
   * Registers the "IfText" block to the blockly registry
   */
  public static registerBlock() {
    Blockly.defineBlocksWithJsonArray([
      {
        type: "if_text",
        message0: 'Wenn %1 %2 "%3" ist %4',
        args0: [
          {
            type: "field_input",
            name: "VARIABLE_NAME",
            text: "x",
          },
          {
            type: "field_dropdown",
            name: "OPERATOR",
            options: [
              ["Gleich", "gleich"],
              ["Kleiner", "kleiner"],
              ["Größer", "größer"],
              ["Kleiner oder gleich", "kleiner-gleich"],
              ["Größer oder gleich", "größer-gleich"],
            ],
          },
          {
            type: "field_input",
            name: "VALUE",
            text: '"Wert"',
          },
          {
            type: "input_statement",
            name: "STATEMENT",
          },
        ],
        previousStatement: null,
        nextStatement: null,
        colour: "#5577EE",
        tooltip: "",
        helpUrl: "",
      },
    ]);
  }
```

```
}
```

## packages/incode-blockly/src/blockly/blocks/LogBlock.ts

```
/**
 * @author The InCode Devs
 * @copyright 2018-2021 The InCode Developers <https://github.com/InCodeDevs>
 */

import * as Blockly from "blockly";

export class LogBlock {
  /**
   * Registers the "Log" block to the blockly registry
   */
  public static registerBlock() {
    Blockly.defineBlocksWithJsonArray([
      {
        type: "log",
        message0: 'Gib "%1" in der %2 aus',
        args0: [
          {
            type: "field_input",
            name: "MESSAGE",
            text: "Wert",
          },
          {
            type: "field_dropdown",
            name: "OUTPUT",
            options: [
              ["Konsole", "Konsole"],
              ["Dialogbox", "Dialogbox"],
            ],
          },
        ],
        previousStatement: null,
        nextStatement: null,
        colour: 120,
        tooltip: "",
        helpUrl: "",
      },
    ]);
  }
}
```

## packages/incode-blockly/src/blockly/blocks/RepeatWhile.ts

```
/**
 * @author The InCode Devs
 * @copyright 2018-2021 The InCode Developers <https://github.com/InCodeDevs>
 */
import * as Blockly from "blockly";

export class RepeatWhile {
  /**
   * Registers the "RepeatWhile" block to the blockly registry
   */
  public static registerBlock() {
    Blockly.defineBlocksWithJsonArray([
      {
        type: "repeat_while",
        message0: "Wiederhole solange %1 %2 %3 ist %4",
```

```
        args0: [
          {
            type: "field_input",
            name: "VARIABLE_NAME",
            text: "x",
          },
          {
            type: "field_dropdown",
            name: "OPERATOR",
            options: [
              ["Gleich", "gleich"],
              ["Nicht gleich", "nicht-gleich"],
            ],
          },
          {
            type: "field_input",
            name: "VALUE",
            text: "Text",
          },
          {
            type: "input_statement",
            name: "STATEMENT",
          },
        ],
        previousStatement: null,
        nextStatement: null,
        colour: 210,
        tooltip: "",
        helpUrl: "",
      },
    ]);
  }
}
```

## packages/incode-blockly/src/blockly/blocks/RepeatWhileNumber.ts

```
/**
 * @author The InCode Devs
 * @copyright 2018-2021 The InCode Developers <https://github.com/InCodeDevs>
 */

import * as Blockly from "blockly";

export class RepeatWhileNumber {
  /**
   * Registers the "RepeatWhileNumber" block to the blockly registry
   */
  public static registerBlock() {
    Blockly.defineBlocksWithJsonArray([
      {
        type: "repeat_while_number",
        message0: "Wiederhole solange %1 %2 %3 ist %4",
        args0: [
          {
            type: "field_input",
            name: "VARIABLE_NAME",
            text: "x",
          },
          {
            type: "field_dropdown",
            name: "OPERATOR",
```

```
            options: [
              ["Gleich", "gleich"],
              ["Kleiner", "kleiner"],
              ["Größer", "größer"],
              ["Kleiner oder gleich", "kleiner-gleich"],
              ["Größer oder gleich", "größer-gleich"],
            ],
          },
          {
            type: "field_number",
            name: "NAME",
            value: 0,
          },
          {
            type: "input_statement",
            name: "STATEMENT",
          },
        ],
        previousStatement: null,
        nextStatement: null,
        colour: 210,
        tooltip: "",
        helpUrl: "",
      },
    ]);
  }
}
```

## packages/incode-blockly/src/blockly/blocks/RepeatXTimesBlock.ts

```
/**
 * @author The InCode Devs
 * @copyright 2018-2021 The InCode Developers <https://github.com/InCodeDevs>
 */
import * as Blockly from "blockly";

export class RepeatXTimesBlock {
  /**
   * Registers the "RepeatXTimes" block to the blockly registry
   */
  public static registerBlock() {
    Blockly.defineBlocksWithJsonArray([
      {
        type: "repeat_x_times",
        message0: "Wiederhole %1 mal %2",
        args0: [
          {
            type: "field_number",
            name: "TIMES",
            value: 0,
          },
          {
            type: "input_statement",
            name: "STATEMENT",
          },
        ],
        previousStatement: null,
        nextStatement: null,
        colour: 210,
        tooltip: "",
        helpUrl: "",
```

```
      },
    ]);
  }
}
```

## packages/incode-blockly/src/blockly/blocks/SetVarBorderStyle.ts

```
/**
 * @author The InCode Devs
 * @copyright 2018-2021 The InCode Developers <https://github.com/InCodeDevs>
 */
import * as Blockly from "blockly";

export class SetVarBorderStyle {
  /**
   * Registers the "SetVarBorderStyle" block to the blockly registry
   */
  public static registerBlock() {
    Blockly.defineBlocksWithJsonArray([
      {
        type: "var_set_border_style",
        message0: "Setze den %1 von %2 auf %3",
        args0: [
          {
            type: "field_dropdown",
            name: "TYPE",
            options: [
              ["Umrandungsstil", "Umrandungsstil"],
              ["Umrandungsstil-Rechts", "Umrandungsstil-Rechts"],
              ["Umrandungsstil-Links", "Umrandungsstil-Links"],
              ["Umrandungsstil-Oben", "Umrandungsstil-Oben"],
              ["Umrandungsstil-Unten", "Umrandungsstil-Unten"],
            ],
          },
          {
            type: "field_input",
            name: "VARIABLE_NAME",
            text: "x",
          },
          {
            type: "field_dropdown",
            name: "COLOR",
            options: [
              ["Nichts", "Nichts"],
              ["Versteckt", "Versteckt"],
              ["Gepunktet", "Gepunktet"],
              ["Gestrichelt", "Gestrichelt"],
              ["Solide", "Solide"],
              ["Doppelt", "Doppelt"],
              ["Gerillt", "Gerillt"],
              ["3d", "3d"],
              ["Eingesetzt", "Eingesetzt"],
              ["Draufgelegt", "Draufgelegt"],
            ],
          },
        ],
        previousStatement: null,
        nextStatement: null,
        colour: 30,
        tooltip: "",
        helpUrl: "",
```

```
      },
    ]);
  }
}
```

## packages/incode-blockly/src/blockly/blocks/SetVarColorBlock.ts

```
/**
 * @author The InCode Devs
 * @copyright 2018-2021 The InCode Developers <https://github.com/InCodeDevs>
 */
import * as Blockly from "blockly";

export class SetVarColorBlock {
  /**
   * Registers the "SetVarColor" block to the blockly registry
   */
  public static registerBlock() {
    Blockly.defineBlocksWithJsonArray([
      {
        type: "var_set_color",
        message0: "Setze die %1 von %2 auf %3",
        args0: [
          {
            type: "field_dropdown",
            name: "TYPE",
            options: [
              ["Farbe", "Farbe"],
              ["Hintergrundfarbe", "Hintergrundfarbe"],
              ["Umrandungsfarbe", "Umrandungsfarbe"],
              ["Umrandungsfarbe-Oben", "Umrandungsfarbe-Oben"],
              ["Umrandungsfarbe-Unten", "Umrandungsfarbe-Unten"],
              ["Umrandungsfarbe-Rechts", "Umrandungsfarbe-Rechts"],
              ["Umrandungsfarbe-Links", "Umrandungsfarbe-Links"],
            ],
          },
          {
            type: "field_input",
            name: "VARIABLE_NAME",
            text: "x",
          },
          {
            type: "field_dropdown",
            name: "COLOR",
            options: [
              ["Schwarz", "Schwarz"],
              ["Weiß", "Weiß"],
              ["Blau", "Blau"],
              ["Grün", "Grün"],
              ["Gelb", "Gelb"],
              ["Grau", "Grau"],
            ],
          },
        ],
        previousStatement: null,
        nextStatement: null,
        colour: 30,
        tooltip: "",
        helpUrl: "",
      },
    ]);
```

```
  }
}
```

## packages/incode-blockly/src/blockly/blocks/SetVarDecorationPropsBlock.ts

```
/**
 * @author The InCode Devs
 * @copyright 2018-2021 The InCode Developers <https://github.com/InCodeDevs>
 */
import * as Blockly from "blockly";

export class SetVarDecorationPropsBlock {
  /**
   * Registers the "SetVarDecorProps" block to the blockly registry
   */
  public static registerBlock() {
    Blockly.defineBlocksWithJsonArray([
      {
        type: "var_set_decor_props",
        message0: "Setze %1 von %2 auf %3",
        args0: [
          {
            type: "field_dropdown",
            name: "TYPE",
            options: [
              ["die Textdekoration", "die Textdekoration"],
              ["den Schriftstil", "den Schriftstil"],
            ],
          },
          {
            type: "field_input",
            name: "VARIABLE_NAME",
            text: "x",
          },
          {
            type: "field_dropdown",
            name: "COLOR",
            options: [
              ["Unterstrichen", "Unterstrichen"],
              ["Überstrichen", "Überstrichen"],
              ["Durchgestrichen", "Durchgestrichen"],
              ["Blink", "Blink"],
              ["Nichts", "Nichts"],
              ["Unterstrichen-Überstrichen", "Unterstrichen-Überstrichen"],
            ],
          },
        ],
        previousStatement: null,
        nextStatement: null,
        colour: 30,
        tooltip: "",
        helpUrl: "",
      },
    ]);
  }
}
```

## packages/incode-blockly/src/blockly/blocks/SetVarFontWeight.ts

```
/**
 * @author The InCode Devs
 * @copyright 2018-2021 The InCode Developers <https://github.com/InCodeDevs>
```

```
 */
import * as Blockly from "blockly";

export class SetVarFontWeight {
  /**
   * Registers the "SetVarFontWeight" block to the blockly registry
   */
  public static registerBlock() {
    Blockly.defineBlocksWithJsonArray([
      {
        type: "var_set_font_weight",
        message0: "Setze die Schriftbreite von %1 auf %2",
        args0: [
          {
            type: "field_input",
            name: "VARIABLE_NAME",
            text: "x",
          },
          {
            type: "field_dropdown",
            name: "COLOR",
            options: [
              ["Nichts", "Nichts"],
              ["Fett", "Fett"],
              ["Normal", "Normal"],
            ],
          },
        ],
        previousStatement: null,
        nextStatement: null,
        colour: 30,
        tooltip: "",
        helpUrl: "",
      },
    ]);
  }
}
```

## packages/incode-blockly/src/blockly/blocks/SetVarPositionBlock.ts

```
/**
 * @author The InCode Devs
 * @copyright 2018-2021 The InCode Developers <https://github.com/InCodeDevs>
 */
import * as Blockly from "blockly";

export class SetVarPositionBlock {
  /**
   * Registers the "SetVarPosition" block to the blockly registry
   */
  public static registerBlock() {
    Blockly.defineBlocksWithJsonArray([
      {
        type: "var_set_pos",
        message0: "Setze die Position von %1 auf %2",
        args0: [
          {
            type: "field_input",
            name: "VARIABLE_NAME",
            text: "x",
          },
```

```
        {
          type: "field_dropdown",
          name: "COLOR",
          options: [
            ["Absolut", "Absolut"],
            ["Fest", "Fest"],
            ["Relativ", "Relativ"],
          ],
        },
      ],
      previousStatement: null,
      nextStatement: null,
      colour: 30,
      tooltip: "",
      helpUrl: "",
    },
  ]);
 }
}
```

## packages/incode-blockly/src/blockly/blocks/SetVarPropertyBlock.ts

```
/**
 * @author The InCode Devs
 * @copyright 2018-2021 The InCode Developers <https://github.com/InCodeDevs>
 */

import * as Blockly from "blockly";

export class SetVarPropertyBlock {
  /**
   * Registers the "SetVarText" block to the blockly registry
   */
  public static registerBlock() {
    Blockly.defineBlocksWithJsonArray([
      {
        type: "var_set_text",
        message0: "Setze %1 von %2 auf %3",
        args0: [
          {
            type: "field_dropdown",
            name: "PROPERTY_NAME",
            options: [
              ["den Text", "den Text"],
              ["den Wert", "den Wert"],
              ["die Schriftart", "die Schriftart"],
              ["den Abstand-Oben", "den Abstand-Oben"],
              ["den Abstand-Unten", "den Abstand-Unten"],
              ["den Abstand-Links", "den Abstand-Links"],
              ["den Abstand-Rechts", "den Abstand-Rechts"],
              ["den Abstand", "den Abstand"],
              ["den Innerer-Abstand-Oben", "den Innerer-Abstand-Oben"],
              ["den Innerer-Abstand-Unten", "den Innerer-Abstand-Unten"],
              ["den Innerer-Abstand-Links", "den Innerer-Abstand-Links"],
              ["den Innerer-Abstand-Rechts", "den Innerer-Abstand-Rechts"],
              ["den Innerer-Abstand", "den Innerer-Abstand"],
              ["die Id", "die Id"],
              ["die Schriftgröße", "die Schriftgröße"],
              ["die Breite", "die Breite"],
              ["die Höhe", "die Höhe"],
              ["die Rahmenbreite", "die Rahmenbreite"],
```

```
            ["die Rahmenhöhe", "die Rahmenhöhe"],
            ["die Quelle", "die Quelle"],
            ["die Umrandungsbreite-Oben", "die Umrandungsbreite-Oben"],
            ["die Umrandungsbreite-Unten", "die Umrandungsbreite-Unten"],
            ["die Umrandungsbreite-Links", "die Umrandungsbreite-Links"],
            ["die Umrandungsbreite-Rechts", "die Umrandungsbreite-Rechts"],
            ["die Umrandungsbreite", "die Umrandungsbreite"],
            ["den Umrandungsradius", "den Umrandungsradius"],
          ],
        },
        {
          type: "field_input",
          name: "VARIABLE_NAME",
          text: "x",
        },
        {
          type: "field_input",
          name: "VALUE",
          text: "'Wert'",
        },
      ],
      previousStatement: null,
      nextStatement: null,
      colour: 30,
      tooltip: "",
      helpUrl: "",
    },
  ]);
 }
}
```

## packages/incode-blockly/src/blockly/blocks/SetVarTextAlign.ts

```
/**
 * @author The InCode Devs
 * @copyright 2018-2021 The InCode Developers <https://github.com/InCodeDevs>
 */
import * as Blockly from "blockly";

export class SetVarTextAlign {
  /**
   * Registers the "SetVarTextAlign" block to the blockly registry
   */
  public static registerBlock() {
    Blockly.defineBlocksWithJsonArray([
      {
        type: "var_set_text_align",
        message0: "Setze die Textausrichtung von %1 auf %2",
        args0: [
          {
            type: "field_input",
            name: "VARIABLE_NAME",
            text: "x",
          },
          {
            type: "field_dropdown",
            name: "COLOR",
            options: [
              ["Links", "Links"],
              ["Rechts", "Rechts"],
              ["Mitte", "Mitte"],
```

```
          ],
        },
      ],
      previousStatement: null,
      nextStatement: null,
      colour: 30,
      tooltip: "",
      helpUrl: "",
    },
  ]);
 }
}
```

## packages/incode-blockly/src/blockly/blocks/StartBlock.ts

```
/**
 * @author The InCode Devs
 * @copyright 2018-2021 The InCode Developers <https://github.com/InCodeDevs>
 */

import * as Blockly from "blockly";

export class StartBlock {
  /**
   * Registers the "Start" block to the blockly registry
   * Attention: This block is ignored when compiling
   */
  public static registerBlock() {
    Blockly.defineBlocksWithJsonArray([
      {
        type: "start",
        message0: "Start",
        nextStatement: null,
        colour: 210,
      },
    ]);
  }
}
```

## packages/incode-blockly/src/blockly/Blocks.ts

```
/**
 * @author Ben Siebert <ben@mctzock.de>
 * @copyright (c) 2018-2021 Ben Siebert. All rights reserved.
 */

import { AddToElementBlock } from "./blocks/AddToElementBlock";
import { AddToScreenBlock } from "./blocks/AddToScreenBlock";
import { AskBlock } from "./blocks/AskBlock";
import { CallMethodBlock } from "./blocks/CallMethodBlock";
import { CreateMethodBlock } from "./blocks/CreateMethodBlock";
import { CreateVarBlock } from "./blocks/CreateVarBlock";
import { CreateVarTypeBlock } from "./blocks/CreateVarTypeBlock";
import { ElseBlock } from "./blocks/ElseBlock";
import { ElseIfNumberBlock } from "./blocks/ElseIfNumberBlock";
import { IfEventBlock } from "./blocks/IfEventBlock";
import { IfNumberBlock } from "./blocks/IfNumberBlock";
import { ElseIfTextBlock } from "./blocks/ElseIfTextBlock";
import { IfTextBlock } from "./blocks/IfTextBlock";
import { LogBlock } from "./blocks/LogBlock";
import { RepeatWhile } from "./blocks/RepeatWhile";
import { RepeatWhileNumber } from "./blocks/RepeatWhileNumber";
```

```
import { RepeatXTimesBlock } from "./blocks/RepeatXTimesBlock";
import { SetVarBorderStyle } from "./blocks/SetVarBorderStyle";
import { SetVarColorBlock } from "./blocks/SetVarColorBlock";
import { SetVarDecorationPropsBlock } from "./blocks/SetVarDecorationPropsBlock";
import { SetVarFontWeight } from "./blocks/SetVarFontWeight";
import { SetVarPositionBlock } from "./blocks/SetVarPositionBlock";
import { SetVarPropertyBlock } from "./blocks/SetVarPropertyBlock";
import { SetVarTextAlign } from "./blocks/SetVarTextAlign";
import { StartBlock } from "./blocks/StartBlock";
import { FunctionBlock } from "./blocks/FunctionBlock";

export default class Blocks {
  public static register() {
    FunctionBlock.registerBlock();
    AddToElementBlock.registerBlock();
    AddToScreenBlock.registerBlock();
    AskBlock.registerBlock();
    CallMethodBlock.registerBlock();
    CreateMethodBlock.registerBlock();
    CreateVarBlock.registerBlock();
    CreateVarTypeBlock.registerBlock();
    ElseBlock.registerBlock();
    ElseIfNumberBlock.registerBlock();
    ElseIfTextBlock.registerBlock();
    IfEventBlock.registerBlock();
    IfNumberBlock.registerBlock();
    IfTextBlock.registerBlock();
    LogBlock.registerBlock();
    RepeatWhile.registerBlock();
    RepeatWhileNumber.registerBlock();
    RepeatXTimesBlock.registerBlock();
    SetVarBorderStyle.registerBlock();
    SetVarColorBlock.registerBlock();
    SetVarDecorationPropsBlock.registerBlock();
    SetVarFontWeight.registerBlock();
    SetVarPositionBlock.registerBlock();
    SetVarPropertyBlock.registerBlock();
    SetVarTextAlign.registerBlock();
    StartBlock.registerBlock();
  }
}
```

## packages/incode-blockly/src/blockly/theme/BlocklyThemeDark.ts

```
/**
 * Original by Google: https://github.com/google/blockly-sampes/blob/master/plugins/theme-
dark/src/index.js
 */

/**
 * @license
 * Copyright 2021 Google LLC
 * SPDX-License-Identifier: Apache-2.0
 */

/**
 * @fileoverview Dark theme.
 */

import * as Blockly from "blockly/core";
```

```
// Temporarily required to ensure there's no conflict with
// Blockly.Themes.Dark
Blockly.registry.unregister("theme", "dark");
Blockly.registry.unregister("theme", "ui_dark");

/**
 * Dark theme.
 */
export default Blockly.Theme.defineTheme("ui_dark", {
  componentStyles: {
    workspaceBackgroundColour: "#1e1e1e",
    toolboxBackgroundColour: "blackBackground",
    toolboxForegroundColour: "#fff",
    flyoutBackgroundColour: "#252526",
    flyoutForegroundColour: "#ccc",
    flyoutOpacity: 1,
    scrollbarColour: "#797979",
    insertionMarkerColour: "#fff",
    insertionMarkerOpacity: 0.3,
    scrollbarOpacity: 0.4,
    cursorColour: "#d0d0d0",
    // @ts-ignore
    blackBackground: "#333",
  },
});
```

## packages/incode-blockly/src/BlocklyEditor.tsx

```
/**
 * @author Ben Siebert <ben@mctzock.de>
 * @copyright (c) 2018-2021 Ben Siebert. All rights reserved.
 */

import * as React from "react";
import { useEffect } from "react";
import Blockly from "blockly";
import * as DarkTheme from "./blockly/theme/BlocklyThemeDark";
import Blocks from "./blockly/Blocks";
import * as DE from "blockly/msg/de";

export interface BlocklyProps {
  initialXml: string;
  readonly?: boolean;
}

export default function BlocklyEditor(props: BlocklyProps) {
  useEffect(() => {
    // @ts-ignore
    if (window.injectedBlockly) return;
    // @ts-ignore
    window.injectedBlockly = true;

    Blocks.register();
    Blockly.setLocale(DE);
    const ws = Blockly.inject("blockly", {
      toolbox: defaultToolbox,
      renderer: "zelos",
      theme: DarkTheme.default,
      zoom: {
        controls: true,
      },
```

```
    });
    Blockly.Xml.domToWorkspace(Blockly.Xml.textToDom(props.initialXml), ws);
  }, []);

  return (
    <>
      <div
        id="blockly"
        style={{ width: "100%", height: "100vh", margin: "0" }}
      />
    </>
  );
}

const defaultToolbox = `
<xml xmlns="https://developers.google.com/blockly/xml" id="toolbox" style="display: none">
   <category colour="210" name="Programm">
      <block type="start" />
   </category>
   <category colour="120" name="Anzeige">
      <block type="log" />
      <block type="ask" />
      <block type="add_to_screen" />
      <block type="add_to_element" />
   </category>
   <category colour="290" name="Funktionen">
      <block type="create_method" />
      <block type="call_method" />
   </category>
   <category colour="30" name="Variabeln">
      <block type="var_create" />
      <block type="var_create_type" />
      <block type="var_set_text" />
      <block type="var_set_color" />
      <block type="var_set_decor_props" />
      <block type="var_set_text_align" />
      <block type="var_set_pos" />
      <block type="var_set_border_style" />
      <block type="var_set_font_weight" />
   </category>
   <category colour="210" name="Schleifen">
      <block type="repeat_x_times" />
      <block type="repeat_while" />
      <block type="repeat_while_number" />
   </category>
   <category colour="#5577EE" name="Abfragen">
      <block type="if_text" />
      <block type="if_number" />
      <block type="if_event" />
      <block type="else_if_text" />
      <block type="else_if_number" />
      <block type="else" />
   </category>
</xml>`;
```

## packages/incode-blockly/tsconfig.json

```
{
  "extends": "tsconfig/react-library.json",
  "exclude": ["dist", "build", "node_modules"],
  "compilerOptions": {
```

```
    "lib": ["DOM"],
    "jsx": "react",
    "rootDir": "src",
    "outDir": "dist"
  }
}
```

## packages/incode-compiler/jest.config.js

```
module.exports = {
  transform: { "^.+\\.ts?$": "ts-jest" },
  testEnvironment: "node",
  moduleFileExtensions: ["ts", "tsx", "js", "jsx", "json", "node"],
};
```

## packages/incode-compiler/package.json

```
{
  "name": "@codeup/incode-compiler",
  "version": "1.0.0",
  "description": "Integration of InCode's Language Compiler",
  "main": "dist/index.js",
  "repository": "https://github.com/MCTzOCK/codeup",
  "author": "Ben Siebert",
  "license": "GPL-3.0-only",
  "private": false,
  "scripts": {
    "test": "jest",
    "build": "tsc"
  },
  "dependencies": {
    "@codeup/incode-ast-generator": "*",
    "@types/jest": "^28.1.7",
    "@types/js-beautify": "^1.13.3",
    "@types/uglify-js": "^3.16.0",
    "blockly": "^9.3.1",
    "jest": "^28.1.3",
    "js-beautify": "^1.14.5",
    "ts-jest": "^28.0.8",
    "uglify-js": "^3.16.3"
  }
}
```

## packages/incode-compiler/src/Blockly.ts

```
/**
 * packages/incode-compiler/src/Blockly.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 04.04.2023
 *
 */

import * as Blockly from "blockly";

export class BlocklyCompiler {
  private compiledBlocks: any[] = [];

  /**
   * Compiles the current blockly workspace
```

```
   * @return The compiled code
   */
  compile(): string {
    const blocks = Blockly.getMainWorkspace().getTopBlocks(true);

    let code = "";

    for (const topBlock of blocks) {
      let baseIntents = 0;
      if (topBlock.type !== "start") {
        baseIntents++;
        code += this.getBlockText(topBlock) + "\n";
      }

      const allChildren = topBlock
        .getDescendants(true)
        .filter((b) => {
          if (b.getSurroundParent()) return false;
          return true;
        })
        .filter((b) => b !== topBlock);

      const processBlock = (block: Blockly.Block, addInts: number = 0) => {
        code +=
          "\t".repeat(baseIntents + addInts) + this.getBlockText(block) + "\n";
        const children = block.getDescendants(true).filter((b) => {
          return (b.getSurroundParent() as Blockly.Block) === block;
        });

        if (children.length > 0) {
          for (const c of children) {
            processBlock(c, addInts + 1);
          }
        }
      };

      for (const b of allChildren) {
        processBlock(b);
      }
    }
    return code;
  }

  /**
   * Actually compiles a block
   * @param block The block to be compiled
   */
  private getBlockText(block: Blockly.Block): string {
    let c = "";
    block.inputList.forEach((i) => {
      i.fieldRow.forEach((r) => {
        if (r.getValue() != "Start") {
          c += r.getValue() + " ";
        }
      });
    });
    return c;
  }
}
```

## packages/incode-compiler/src/index.ts

```
/**
 * @author Ben Siebert <ben@mctzock.de>
 * @copyright (c) 2018-2021 Ben Siebert. All rights reserved.
 */

export { AbstractSyntaxTree } from "./types/AbstractSyntaxTree";
export { CodeGenerator } from "./module/CodeGenerator";
export { AbstractSyntaxTreeGenerator } from "./module/AbstractSyntaxTreeGenerator";
export { AliasManager } from "./module/AliasManager";
export { Compiler } from "./module/Compiler";
export { CommandExecutor } from "./module/commands/CommandExecutor";
export { BlocklyCompiler } from "./Blockly";
```

## packages/incode-compiler/src/module/AbstractSyntaxTreeGenerator.ts

```
/**
 * @author Ben Siebert <ben@mctzock.de>
 * @copyright (c) 2018-2021 Ben Siebert. All rights reserved.
 */
import { AbstractSyntaxTree } from "../types/AbstractSyntaxTree";
import { PreCompiler } from "../util/PreCompiler";

export class AbstractSyntaxTreeGenerator {
  public static generate(source: string): AbstractSyntaxTree[] {
    source = PreCompiler.preCompile(source);
    const lines = source.split("\n");

    if (lines.length === 0) {
      return [];
    }

    const words: string[][] = this.splitWords(source);

    let result: AbstractSyntaxTree[];

    let temp: AbstractSyntaxTree[] = [];

    words.forEach((wordList) => {
      wordList = wordList.filter((w) => w !== "");

      if (
        wordList.length === 0 ||
        (wordList.length === 1 && wordList[0] === "")
      ) {
        return;
      }

      const ast: AbstractSyntaxTree = {
        intents: 0,
        command: "",
        args: [],
        children: [],
      };

      const firstWord = wordList[0];

      ast.intents =
        firstWord.split("\t").length - 2 < 0
          ? 0
          : firstWord.split("\t").length - 1;
```

```
    ast.command =
      firstWord.split("\t").length === 1
        ? firstWord
        : firstWord.split("\t")[ast.intents];
    ast.args = wordList.slice(1);

    ast.args = ast.args.filter((arg) => arg !== "");

    console.log(wordList, ast);

    temp.push(ast);
  });

  temp.forEach((ast) => {
    if (ast.intents === 0) {
      temp.push(ast);
      return;
    }
    if (temp[temp.length - 1].intents === ast.intents - 1) {
      temp[temp.length - 1].children.push(ast);
      return;
    } else {
      let x =
        temp[temp.length - 1].children[
          temp[temp.length - 1].children.length - 1
        ];
      let y = x;
      while (x.intents !== ast.intents - 1 && x.intents !== ast.intents) {
        x = y.children[y.children.length - 1];
        y = x;
      }
      if (x.intents === ast.intents - 1) {
        x.children.push(ast);
      } else {
        y.children.push(ast);
      }
    }
  });

  temp = temp.filter((t) => t.intents === 0);

  temp = Array.from(new Set(temp.map((a) => a))).map((a0) => {
    return temp.find((a) => a === a0);
  }) as AbstractSyntaxTree[];

  result = temp;

  return result;
}

private static splitWords(source: string): string[][] {
  const matrix: string[][] = [];

  const lines = source.split("\n");
  lines.forEach((line) => {
    if (line.match(/(["'])(?:(?=(\\?))\2.)*?\1/g)) {
      const quotes = line.match(/(["'])(?:(?=(\\?))\2.)*?\1/g);
      (quotes as string[]).forEach((quote) => {
        line = line.replace(quote, quote.split(" ").join("\0"));
      });
```

```
      }
      matrix.push(line.split(" "));
    });

    return matrix;
  }
}
```

## packages/incode-compiler/src/module/AliasManager.ts

```
/**
 * @author Ben Siebert <ben@mctzock.de>
 * @copyright (c) 2018-2021 Ben Siebert. All rights reserved.
 */

export class AliasManager {
  public static readonly CMD_ALIASES = [
    ["@", "📜"],
    ["//", "#", "📖"],
    ["create", "erstelle", "definiere", "define", "➕"],
    ["call", "execute", "rufe"],
    ["add", "füge"],
    ["remove", "entferne"],
    ["repeat", "wiederhole", "🔁"],
    ["print", "gib"],
    ["ask", "frage", "❓"],
    ["if", "wenn", "falls"],
    ["else", "sonst"],
    ["wait", "warte", "⏳"],
    ["set", "setze"],
  ];

  public static readonly SPECIAL_ALIASES = [["value", "wert"]];

  public static readonly TYPE_ALIASES = [
    ["button", "knopf"],
    ["p", "paragraph", "absatz"],
    ["span", "text"],
    ["img", "image", "bild"],
    ["h1", "heading1", "überschrift1"],
    ["h2", "heading2", "überschrift2"],
    ["h3", "heading3", "überschrift3"],
    ["h4", "heading4", "überschrift4"],
    ["h5", "heading5", "überschrift5"],
    ["h6", "heading6", "überschrift6"],
    ["table", "tabelle"],
    ["tr", "row", "zeile"],
    ["tb", "column", "spalte"],
    ["br", "linebreak", "zeilenumbruch"],
    ["video"],
    ["audio", "ton"],
    ["iframe", "frame", "fenster"],
    ["div", "container"],
    ["li", "listitem", "listenpunkt"],
    ["ol", "ordered-list", "geordnete-liste"],
    ["ul", "unordered-list", "ungeordnete-liste"],
    ["a", "link"],
    ["function", "method", "methode", "funktion"],
    ["document.body", "screen", "bildschirm"],
    ["console.log", "console", "konsole"],
```

```
  ["alert", "dialogbox"],
  ["key", "taste"],
];

public static readonly OPERATOR_ALIASES = [
  ["<", "kleiner", "smaller"],
  [">", "größer"],
  ["===", "gleich", "equals"],
  ["!==", "ungleich", "unequal"],
  [">=", "größergleich", "greaterequal"],
  ["<=", "kelinergleich", "smallerequal"],
];

public static readonly EVENT_ALIASES = [
  ["click", "pressed", "gedrückt"],
  ["mouseover", "hovered", "berührt"],
  ["mouseleave", "not-hovered", "nicht-berührt"],
];

public static readonly COLOR_PROPERTY_ALIASES = [
  ["style.color", "color", "farbe"],
  ["style.backgroundColor", "backgroundcolor", "hintergrundfarbe"],
  ["blue", "blau"],
  ["lime", "green", "grün"],
  ["red", "rot"],
  ["orange"],
  ["yellow", "gelb"],
  ["white", "weiß"],
  ["black", "schwarz"],
  ["grey", "grau"],
  ["purple", "lila"],
  ["pink", "pink"],
  ["brown", "braun"],
  ["cyan", "cyan"],
  ["magenta", "magenta"],
  ["maroon", "maroon"],
  ["navy", "navy"],
  ["olive", "olive"],
  ["silver", "silber"],
  ["teal", "teal"],
  ["transparent", "transparent"],
];

public static getCommandAliases(command: string): string[] {
  return (
    AliasManager.CMD_ALIASES.find((alias) =>
      alias.includes(command.toLowerCase())
    ) || []
  );
}

public static getTypeAliases(type: string): string[] {
  return (
    AliasManager.TYPE_ALIASES.find((alias) =>
      alias.includes(type.toLowerCase())
    ) || []
  );
}

public static getOperatorAliases(operator: string): string[] {
```

```
    return (
      AliasManager.OPERATOR_ALIASES.find((alias) =>
        alias.includes(operator.toLowerCase())
      ) || []
    );
  }

  public static getEventAliases(event: string): string[] {
    return (
      AliasManager.EVENT_ALIASES.find((alias) =>
        alias.includes(event.toLowerCase())
      ) || []
    );
  }

  public static getColorPropertyAliases(colorProperty: string): string[] {
    return (
      AliasManager.COLOR_PROPERTY_ALIASES.find((alias) =>
        alias.includes(colorProperty.toLowerCase())
      ) || []
    );
  }

  public static getSpecialAliases(special: string): string[] {
    return (
      AliasManager.SPECIAL_ALIASES.find((alias) =>
        alias.includes(special.toLowerCase())
      ) || []
    );
  }
}
```

## packages/incode-compiler/src/module/CodeGenerator.ts

```
/**
 * @author Ben Siebert <ben@mctzock.de>
 * @copyright (c) 2018-2021 Ben Siebert. All rights reserved.
 */
import { AbstractSyntaxTree } from "../types/AbstractSyntaxTree";

export class CodeGenerator {
  public static generate(ast: AbstractSyntaxTree[]): string {
    let code = "";

    ast.forEach((node) => {
      code += this.generateNode(node);
    });

    return code;
  }

  private static generateNode(node: AbstractSyntaxTree): string {
    let code = node.command + " " + node.args.join(" ").replace("\0", " ");

    for (let i = 0; i < node.intents; i++) {
      code = "\t" + code;
    }

    let addedLine = false;

    if (node.children.length > 0) {
```

```
      code += "\n";
      code += node.children.map((child) => this.generateNode(child)).join("");
      addedLine = true;
    }

    if (!addedLine) {
      code += "\n";
    }

    return code;
  }
}
```

## packages/incode-compiler/src/module/commands/AddCommand.ts

```
/**
 * @author Ben Siebert <ben@mctzock.de>
 * @copyright (c) 2018-2021 Ben Siebert. All rights reserved.
 */
import { AliasManager } from "../AliasManager";
import { InCodeCommand } from "./InCodeCommand";
import { Compiler } from "../Compiler";
import { Error } from "../Error";

export class AddCommand extends InCodeCommand {
  public execute(args: string[]): string {
    if (args.length < 4) {
      return Error.ERROR_MISSING_PARAMETER;
    } else {
      const element = args[0];
      let parent = args[args.length - 2];
      if (AliasManager.getTypeAliases(parent).length > 0) {
        parent = AliasManager.getTypeAliases(parent)[0];
      }

      return `// Diese Zeile fügt das Element ${element} zu dem Element ${parent}
hinzu.\n${parent}.appendChild(${element});`;
    }
  }
}
```

## packages/incode-compiler/src/module/commands/AskCommand.ts

```
/**
 * @author Ben Siebert <ben@mctzock.de>
 * @copyright (c) 2018-2021 Ben Siebert. All rights reserved.
 */

import { InCodeCommand } from "./InCodeCommand";
import { Error } from "../Error";

export class AskCommand extends InCodeCommand {
  public execute(args: string[]): string {
    if (args.length === 0) {
      return Error.ERROR_MISSING_PARAMETER;
    } else {
      args[0] = args[0].replace(/\u0000/g, " ");
      if (args.length === 1) {
        return `// Diese Zeile fragt den Benutzer "${args[0]}"\nprompt(${args[0]});`;
      } else {
        return `// Diese Zeile fragt den Benutzer "${
          args[0]
```

```
        }" und speichert die Antwort in ${args[args.length - 1]}\nlet ${
          args[args.length - 1]
        } = prompt(${args[0]});`;
      }
    }
  }
}
```

## packages/incode-compiler/src/module/commands/CallCommand.ts

```
/**
 * @author Ben Siebert <ben@mctzock.de>
 * @copyright (c) 2018-2021 Ben Siebert. All rights reserved.
 */
import { InCodeCommand } from "./InCodeCommand";
import { Error } from "../Error";

export class CallCommand extends InCodeCommand {
  public execute(args: string[]): string {
    if (args.length < 1) {
      return Error.ERROR_MISSING_PARAMETER;
    } else {
      return `// Diese Zeile ruft die Funktion ${args[0]} auf\nwindow.incode.${args[0]}();`;
    }
  }
}
```

## packages/incode-compiler/src/module/commands/CommandExecutor.ts

```
/**
 * @author Ben Siebert <ben@mctzock.de>
 * @copyright (c) 2018-2021 Ben Siebert. All rights reserved.
 */

import { AliasManager } from "../AliasManager";
import { JSCommand } from "./JSCommand";
import { CreateCommand } from "./CreateCommand";
import { CallCommand } from "./CallCommand";
import { AddCommand } from "./AddCommand";
import { RepeatCommand } from "./RepeatCommand";
import { PrintCommand } from "./PrintCommand";
import { AskCommand } from "./AskCommand";
import { AbstractSyntaxTree } from "../../types/AbstractSyntaxTree";
import { AbstractSyntaxTreeGenerator } from "../AbstractSyntaxTreeGenerator";
import { InCodeCommand } from "./InCodeCommand";
import { IfCommand } from "./IfCommand";
import { ElseCommand } from "./ElseCommand";
import { WaitCommand } from "./WaitCommand";
import { SetCommand } from "./SetCommand";
import { CommentCommand } from "./CommentCommand";
import { RemoveCommand } from "./RemoveCommand";

export class CommandExecutor {
  public static readonly COMMANDS: {
    [key: string]: InCodeCommand;
  } = {
    "@": new JSCommand(),
    "//": new CommentCommand(),
    create: new CreateCommand(),
    call: new CallCommand(),
    add: new AddCommand(),
    remove: new RemoveCommand(),
```

```
    repeat: new RepeatCommand(),
    print: new PrintCommand(),
    ask: new AskCommand(),
    if: new IfCommand(),
    else: new ElseCommand(),
    wait: new WaitCommand(),
    set: new SetCommand(),
  };

  public static executeCommand(
    abstractSyntaxTree: AbstractSyntaxTree | string
  ): string {
    let ast: AbstractSyntaxTree;

    if (typeof abstractSyntaxTree === "string") {
      ast = AbstractSyntaxTreeGenerator.generate(abstractSyntaxTree)[0];
    } else {
      ast = abstractSyntaxTree;
    }

    let code = "\n";

    if (AliasManager.getCommandAliases(ast.command.toLowerCase()).length > 0) {
      if (
        this.COMMANDS[
          AliasManager.getCommandAliases(ast.command.toLowerCase())[0]
        ]
      ) {
        code = this.COMMANDS[
          AliasManager.getCommandAliases(ast.command.toLowerCase())[0]
        ].execute(ast.args);
      }
    } else {
      if (ast.command !== "") {
        code += `// "${ast.command} ${ast.args.join(" ")}" -> Der Befehl ${
          ast.command
        } konnte nicht gefunden werden.`;
      }
    }

    if (ast.children.length > 0) {
      code += " {\n";
      ast.children.forEach((child) => {
        for (let i = 0; i < child.intents; i++) {
          code += "  ";
        }
        code += CommandExecutor.executeCommand(child);
      });
      code += "}\n";
    }

    code += "\n";

    return code;
  }
}
```

## packages/incode-compiler/src/module/commands/CommentCommand.ts

```
/**
 * @author Ben Siebert <ben@mctzock.de>
```

```
 * @copyright (c) 2018-2021 Ben Siebert. All rights reserved.
 */
import { InCodeCommand } from "./InCodeCommand";

export class CommentCommand extends InCodeCommand {
  execute(args: string[]): string {
    return "// " + args.join(" ");
  }
}
```

## packages/incode-compiler/src/module/commands/CreateCommand.ts

```
/**
 * @author Ben Siebert <ben@mctzock.de>
 * @copyright (c) 2018-2021 Ben Siebert. All rights reserved.
 */
import { AliasManager } from "../AliasManager";
import { InCodeCommand } from "./InCodeCommand";
import { Error } from "../Error";

export class CreateCommand extends InCodeCommand {
  public execute(args: string[]): string {
    if (args.length < 1) {
      return Error.ERROR_MISSING_PARAMETER;
    }
    if (args.length === 1) {
      return `// Diese Zeile erstellt die Variable ${args[0]}\nlet ${args[0]};`;
    } else {
      if (AliasManager.getTypeAliases(args[2]).length > 0) {
        if (AliasManager.getTypeAliases(args[2])[0] === "function") {
          return `// Diese Zeile erstellt die Funktion ${args[0]}\nwindow.incode.${args[0]} = () =>`;
        }
        return `// Diese Zeile erstellt die Variable ${args[0]} mit dem Typen ${
          args[2]
        }\nlet ${args[0]} = document.createElement('${
          AliasManager.getTypeAliases(args[2])[0]
        }');`;
      } else {
        return `// Diese Zeile erstellt die Variable ${args[0]} mit dem Typen div, da der eigentliche Typ
nicht gefunden wurde\nlet ${args[0]} = document.createElement("div"); // "${args[2]}" -> Not Found`;
      }
    }
  }
}
```

## packages/incode-compiler/src/module/commands/ElseCommand.ts

```
/**
 * @author Ben Siebert <ben@mctzock.de>
 * @copyright (c) 2018-2021 Ben Siebert. All rights reserved.
 */
import { InCodeCommand } from "./InCodeCommand";
import { CommandExecutor } from "./CommandExecutor";

export class ElseCommand extends InCodeCommand {
  public execute(args: string[]): string {
    if (args.length > 0) {
      return `else ${CommandExecutor.executeCommand(args.join(" "))}`;
    } else {
      return "else";
    }
  }
```

```
}
```

## packages/incode-compiler/src/module/commands/IfCommand.ts

```
/**
 * @author Ben Siebert <ben@mctzock.de>
 * @copyright (c) 2018-2021 Ben Siebert. All rights reserved.
 */
import { AliasManager } from "../AliasManager";
import { InCodeCommand } from "./InCodeCommand";
import { CommandExecutor } from "./CommandExecutor";

export class IfCommand extends InCodeCommand {
  public execute(args: string[]): string {
    // [4] x gleich 2 ist
    // [5] x is equal to 2
    // [4] x gedrückt wird <command>
    // [4] x is pressed <command>
    // [6] die Taste x gedrückt wird <command>
    // [6] the key x is pressed <command>

    if (args.length < 4) {
      return "// This line contained an if command, but it was missing arguments.";
    } else {
      if (
        AliasManager.getOperatorAliases(args[1]).length > 0 ||
        AliasManager.getOperatorAliases(args[2]).length > 0
      ) {
        // normal if --- then
        let operator;
        let operatorPosition;
        let first;
        let second;

        if (AliasManager.getOperatorAliases(args[1]).length > 0) {
          operator = AliasManager.getOperatorAliases(args[1])[0];
          operatorPosition = 1;
        } else {
          operator = AliasManager.getOperatorAliases(args[2])[0];
          operatorPosition = 2;
        }

        if (operatorPosition === 1) {
          first = args[0];
          second = args[2].replace(/\u0000/g, " ");
        } else {
          first = args[0];
          second = args[1].replace(/\u0000/g, " ");
        }
        return `if (${first} ${operator} ${second})`;
      } else if (
        AliasManager.getEventAliases(args[1]).length > 0 ||
        AliasManager.getEventAliases(args[2]).length > 0
      ) {
        // event listener
        let first = args[0];
        let event;
        let eventPosition;
        let command;
        command = args.slice(3).join(" ");
```

```
        if (AliasManager.getEventAliases(args[1]).length > 0) {
          event = AliasManager.getEventAliases(args[1])[0];
          eventPosition = 1;
        } else {
          event = AliasManager.getEventAliases(args[2])[0];
          eventPosition = 2;
        }

        return `${first}.addEventListener("${event}", () => {
  ${CommandExecutor.executeCommand(command)}
});`;
      } else if (
        AliasManager.getTypeAliases(args[1]).length > 0 &&
        AliasManager.getTypeAliases(args[1])[0] === "key"
      ) {
        const command = args.slice(5).join(" ");

        return `document.addEventListener("keydown", (e) => {
  if (e.key === "${args[2]}") {
    ${CommandExecutor.executeCommand(command)}
  }
});`;
      }
    }
    return "";
  }
}
```

## packages/incode-compiler/src/module/commands/InCodeCommand.ts

```
/**
 * @author Ben Siebert <ben@mctzock.de>
 * @copyright (c) 2018-2021 Ben Siebert. All rights reserved.
 */

export abstract class InCodeCommand {
  abstract execute(args: string[]): string;
}
```

## packages/incode-compiler/src/module/commands/JSCommand.ts

```
/**
 * @author Ben Siebert <ben@mctzock.de>
 * @copyright (c) 2018-2021 Ben Siebert. All rights reserved.
 */
import { InCodeCommand } from "./InCodeCommand";

export class JSCommand extends InCodeCommand {
  public static readonly COMMAND_NAME = "@";

  public execute(args: string[]): string {
    let c = args.join(" ");
    c = c.replace(/\u0000/g, " ");
    return c;
  }
}
```

## packages/incode-compiler/src/module/commands/PrintCommand.ts

```
/**
 * @author Ben Siebert <ben@mctzock.de>
 * @copyright (c) 2018-2021 Ben Siebert. All rights reserved.
```

```
 */
import { AliasManager } from "../AliasManager";
import { InCodeCommand } from "./InCodeCommand";
import { Error } from "../Error";

export class PrintCommand extends InCodeCommand {
  public execute(args: string[]): string {
    if (args.length < 4) {
      return Error.ERROR_MISSING_PARAMETER;
    } else {
      if (AliasManager.getTypeAliases(args[3]).length > 0) {
        return `// Diese Zeile gibt den angegebenen Text in der ${
          args[3]
        } aus\n${AliasManager.getTypeAliases(args[3])[0]}(${args[0].replace(
          /\u0000/g,
          " "
        )});`;
      } else {
        return Error.ERROR_UNKNOWN_TYPE;
      }
    }
  }
}
```

## packages/incode-compiler/src/module/commands/RemoveCommand.ts

```
/**
 * @author Ben Siebert <ben@mctzock.de>
 * @copyright (c) 2018-2021 Ben Siebert. All rights reserved.
 */
import { AliasManager } from "../AliasManager";
import { InCodeCommand } from "./InCodeCommand";
import { Error } from "../Error";

export class RemoveCommand extends InCodeCommand {
  public execute(args: string[]): string {
    if (args.length < 3) {
      return Error.ERROR_MISSING_PARAMETER;
    } else {
      const element = args[0];
      let parent = args[args.length - 1];
      if (AliasManager.getTypeAliases(parent).length > 0) {
        parent = AliasManager.getTypeAliases(parent)[0];
      }
      return `// Diese Zeile entfernt das Element ${element} von dem Element
${parent}\n${parent}.removeChild(${element});`;
    }
  }
}
```

## packages/incode-compiler/src/module/commands/RepeatCommand.ts

```
/**
 * @author Ben Siebert <ben@mctzock.de>
 * @copyright (c) 2018-2021 Ben Siebert. All rights reserved.
 */
import { AliasManager } from "../AliasManager";
import { InCodeCommand } from "./InCodeCommand";
import { Error } from "../Error";

export class RepeatCommand extends InCodeCommand {
  public execute(args: string[]): string {
```

```
    if (args.length < 2) {
      return Error.ERROR_MISSING_PARAMETER;
    } else {
      if (args.length === 2) {
        return `// Diese Zeile wiederholt die darunter stehenden Instruktionen ${args[0]} mal\nfor (let i =
0; i < ${args[0]}; i++)`;
      } else if (args.length === 6) {
        if (AliasManager.getOperatorAliases(args[2]).length > 0) {
          return `// Diese Zeile wiederholt die darunter stehenden Instruktionen solange die Variable ${
            args[1]
          } ${args[2]} der Variable ${args[4]} ist\nwhile(${args[1]} ${
            AliasManager.getOperatorAliases(args[2])[0]
          } ${args[4]})`;
        } else {
          return Error.ERROR_UNKNOWN_OPERATOR;
        }
      } else {
        return Error.ERROR_MISSING_PARAMETER;
      }
    }
  }
}
```

## packages/incode-compiler/src/module/commands/SetCommand.ts

```
/**
 * @author Ben Siebert <ben@mctzock.de>
 * @copyright (c) 2018-2021 Ben Siebert. All rights reserved.
 */
import { InCodeCommand } from "./InCodeCommand";
import { AliasManager } from "../AliasManager";
import { Error } from "../Error";

export class SetCommand extends InCodeCommand {
  public static readonly colors = [
    {
      name: "blue",
      aliases: ["blue", "blau"],
    },
    {
      name: "lime",
      aliases: ["green", "grün"],
    },
    {
      name: "yellow",
      aliases: ["yellow", "gelb"],
    },
    {
      name: "red",
      aliases: ["red", "rot"],
    },
    {
      name: "white",
      aliases: ["white", "weiss"],
    },
    {
      name: "black",
      aliases: ["black", "schwarz"],
    },
  ];
```

```
public static readonly data = [
  {
    name: "color",
    aliases: ["farbe", "color"],
    type: "style",
    append: "",
    values: SetCommand.colors,
  },
  {
    name: "backgroundColor",
    aliases: ["hintergrundfarbe", "backgroundcolor"],
    type: "style",
    append: "",
    values: SetCommand.colors,
  },
  {
    name: "innerText",
    aliases: ["text"],
    type: "attribute",
    append: "",
    values: [],
  },
  {
    name: "textDecoration",
    aliases: ["textdekoration", "textdecoration"],
    type: "style",
    append: "",
    values: [
      {
        name: "underline",
        aliases: ["underline", "unterstrichen"],
      },
      {
        name: "overline",
        aliases: ["overline", "überstrichen"],
      },
      {
        name: "line-through",
        aliases: ["line-through", "durchgestrichen"],
      },
      {
        name: "underline overline",
        aliases: ["underline-overline", "unterstrichen-überstrichen"],
      },
    ],
  },
  {
    name: "textAlign",
    aliases: ["textausrichtung", "textalign"],
    type: "style",
    append: "",
    values: [
      {
        name: "left",
        aliases: ["left", "links"],
      },
      {
        name: "center",
        aliases: ["center", "zentriert"],
      },
```

```
    {
      name: "right",
      aliases: ["right", "rechts"],
    },
  ],
},
{
  name: "fontWeight",
  aliases: ["fontweight", "schriftbreite"],
  type: "style",
  append: "",
  values: [
    {
      name: "normal",
      aliases: ["normal", "normal"],
    },
    {
      name: "bold",
      aliases: ["bold", "fett"],
    },
    {
      name: "bolder",
      aliases: ["bolder", "schwächer"],
    },
    {
      name: "lighter",
      aliases: ["lighter", "leichter"],
    },
  ],
},
{
  name: "fontFamily",
  aliases: ["fontfamily", "schriftart", "font"],
  type: "style",
  append: "",
  values: [],
},
{
  name: "margingTop",
  aliases: ["margintop", "abstand-oben"],
  type: "style",
  append: "px",
  values: [],
},
{
  name: "marginRight",
  aliases: ["marginright", "abstand-rechts"],
  type: "style",
  append: "px",
  values: [],
},
{
  name: "marginBottom",
  aliases: ["marginbottom", "abstand-unten"],
  type: "style",
  append: "px",
  values: [],
},
{
  name: "marginLeft",
```

```
  aliases: ["marginleft", "abstand-links"],
  type: "style",
  append: "px",
  values: [],
},
{
  name: "margin",
  aliases: ["margin", "abstand"],
  type: "style",
  append: "px",
  values: [],
},
{
  name: "paddingTop",
  aliases: ["paddingtop", "innerer-abstand-oben"],
  type: "style",
  append: "px",
  values: [],
},
{
  name: "paddingRight",
  aliases: ["paddingright", "innerer-abstand-rechts"],
  type: "style",
  append: "px",
  values: [],
},
{
  name: "paddingBottom",
  aliases: ["paddingbottom", "innerer-abstand-unten"],
  type: "style",
  append: "px",
  values: [],
},
{
  name: "paddingLeft",
  aliases: ["paddingleft", "innerer-abstand-links"],
  type: "style",
  append: "px",
  values: [],
},
{
  name: "padding",
  aliases: ["padding", "innerer-abstand"],
  type: "style",
  append: "px",
  values: [],
},
{
  name: "fontSize",
  aliases: ["fontsize", "schriftgröße"],
  type: "style",
  append: "px",
  values: [],
},
{
  name: "width",
  aliases: ["width", "breite"],
  type: "style",
  append: "px",
  values: [],
```

```
},
{
  name: "height",
  aliases: ["height", "höhe"],
  type: "style",
  append: "px",
  values: [],
},
{
  name: "src",
  aliases: ["src", "quelle"],
  type: "attribute",
  append: "",
  values: [],
},
{
  name: "position",
  aliases: ["position"],
  type: "style",
  append: "",
  values: [
    {
      name: "absolute",
      aliases: ["absolute", "absolut"],
    },
    {
      name: "relative",
      aliases: ["relative", "relativ"],
    },
    {
      name: "fixed",
      aliases: ["fixed", "fest"],
    },
  ],
},
{
  name: "borderTopWidth",
  aliases: ["bordertopwidth", "umrandungsbreite-oben"],
  type: "style",
  append: "px",
  values: [],
},
{
  name: "borderRightWidth",
  aliases: ["borderrightwidth", "umrandungsbreite-rechts"],
  type: "style",
  append: "px",
  values: [],
},
{
  name: "borderBottomWidth",
  aliases: ["borderbottomwidth", "umrandungsbreite-unten"],
  type: "style",
  append: "px",
  values: [],
},
{
  name: "borderLeftWidth",
  aliases: ["borderleftwidth", "umrandungsbreite-links"],
  type: "style",
```

```
  append: "px",
  values: [],
},
{
  name: "borderWidth",
  aliases: ["borderwidth", "umrandungsbreite"],
  type: "style",
  append: "px",
  values: [],
},
{
  name: "borderTopStyle",
  aliases: ["bordertopstyle", "umrandungsstil-oben"],
  type: "style",
  append: "",
  values: [
    {
      name: "none",
      aliases: ["none", "keine"],
    },
    {
      name: "solid",
      aliases: ["solid", "durchgezogen", "solide"],
    },
    {
      name: "dashed",
      aliases: ["dashed", "gestrichen"],
    },
    {
      name: "dotted",
      aliases: ["dotted", "gepunktet"],
    },
    {
      name: "double",
      aliases: ["double", "doppelt"],
    },
  ],
},
{
  name: "borderRightStyle",
  aliases: ["borderrightstyle", "umrandungsstil-rechts"],
  type: "style",
  append: "",
  values: [
    {
      name: "none",
      aliases: ["none", "keine"],
    },
    {
      name: "solid",
      aliases: ["solid", "durchgezogen", "solide"],
    },
    {
      name: "dashed",
      aliases: ["dashed", "gestrichen"],
    },
    {
      name: "dotted",
      aliases: ["dotted", "gepunktet"],
    },
```

```
    {
      name: "double",
      aliases: ["double", "doppelt"],
    },
  ],
},
{
  name: "borderBottomStyle",
  aliases: ["borderbottomstyle", "umrandungsstil-unten"],
  type: "style",
  append: "",
  values: [
    {
      name: "none",
      aliases: ["none", "keine"],
    },
    {
      name: "solid",
      aliases: ["solid", "durchgezogen", "solide"],
    },
    {
      name: "dashed",
      aliases: ["dashed", "gestrichen"],
    },
    {
      name: "dotted",
      aliases: ["dotted", "gepunktet"],
    },
    {
      name: "double",
      aliases: ["double", "doppelt"],
    },
  ],
},
{
  name: "borderLeftStyle",
  aliases: ["borderleftstyle", "umrandungsstil-links"],
  type: "style",
  append: "",
  values: [
    {
      name: "none",
      aliases: ["none", "keine"],
    },
    {
      name: "solid",
      aliases: ["solid", "durchgezogen", "solide"],
    },
    {
      name: "dashed",
      aliases: ["dashed", "gestrichen"],
    },
    {
      name: "dotted",
      aliases: ["dotted", "gepunktet"],
    },
    {
      name: "double",
      aliases: ["double", "doppelt"],
    },
```

```
  ],
},
{
  name: "borderStyle",
  aliases: ["borderstyle", "umrandungsstil"],
  type: "style",
  append: "",
  values: [
    {
      name: "none",
      aliases: ["none", "keine"],
    },
    {
      name: "solid",
      aliases: ["solid", "durchgezogen", "solide"],
    },
    {
      name: "dashed",
      aliases: ["dashed", "gestrichen"],
    },
    {
      name: "dotted",
      aliases: ["dotted", "gepunktet"],
    },
    {
      name: "double",
      aliases: ["double", "doppelt"],
    },
  ],
},
{
  name: "borderRadius",
  aliases: ["borderradius", "umrandungsradius"],
  type: "style",
  append: "px",
  values: [],
},
{
  name: "borderTopLeftRadius",
  aliases: ["bordertopleftradius", "umrandungsradius-oben-links"],
  type: "style",
  append: "px",
  values: [],
},
{
  name: "borderTopRightRadius",
  aliases: ["bordertoprightradius", "umrandungsradius-oben-rechts"],
  type: "style",
  append: "px",
  values: [],
},
{
  name: "borderBottomLeftRadius",
  aliases: ["borderbottomleftradius", "umrandungsradius-unten-links"],
  type: "style",
  append: "px",
  values: [],
},
{
  name: "borderBottomRightRadius",
```

```
  aliases: ["borderbottomrightradius", "umrandungsradius-unten-rechts"],
  type: "style",
  append: "px",
  values: [],
},
{
  name: "borderTopLeftRadius",
  aliases: ["bordertopleftradius", "umrandungsradius-oben-links"],
  type: "style",
  append: "px",
  values: [],
},
{
  name: "borderTopRightRadius",
  aliases: ["bordertoprightradius", "umrandungsradius-oben-rechts"],
  type: "style",
  append: "px",
  values: [],
},
{
  name: "borderBottomLeftRadius",
  aliases: ["borderbottomleftradius", "umrandungsradius-unten-links"],
  type: "style",
  append: "px",
  values: [],
},
{
  name: "borderBottomRightRadius",
  aliases: ["borderbottomrightradius", "umrandungsradius-unten-rechts"],
  type: "style",
  append: "px",
  values: [],
},
{
  name: "borderTopColor",
  aliases: ["bordertopcolor", "umrandungsfarbe-oben"],
  type: "style",
  append: "",
  values: SetCommand.colors,
},
{
  name: "borderRightColor",
  aliases: ["borderrightcolor", "umrandungsfarbe-rechts"],
  type: "style",
  append: "",
  values: SetCommand.colors,
},
{
  name: "borderBottomColor",
  aliases: ["borderbottomcolor", "umrandungsfarbe-unten"],
  type: "style",
  append: "",
  values: SetCommand.colors,
},
{
  name: "borderLeftColor",
  aliases: ["borderleftcolor", "umrandungsfarbe-links"],
  type: "style",
  append: "",
  values: SetCommand.colors,
```

```
  },
  {
    name: "borderColor",
    aliases: ["bordercolor", "umrandungsfarbe"],
    type: "style",
    append: "",
    values: SetCommand.colors,
  },
  {
    name: "display",
    aliases: ["display", "sichtbarkeit"],
    type: "style",
    append: "",
    values: [
      {
        name: "none",
        aliases: ["invisible", "unsichtbar"],
      },
      {
        name: "block",
        aliases: ["block", "sichtbar"],
      },
    ],
  },
];

public execute(args: string[]): string {
  if (args.length < 3) {
    return Error.ERROR_MISSING_PARAMETER;
  } else {
    const property = args[1];
    const value = args[5].replace(/\u0000/g, " ");
    const variable = args[3];

    let obj: {
      name: string;
      aliases: string[];
      type: string;
      append: string;
      values: { name: string; aliases: string[] }[];
    } = {
      name: "_incode.default",
      aliases: [],
      type: "",
      append: "",
      values: [],
    };

    for (let i = 0; i < SetCommand.data.length; i++) {
      const entry = SetCommand.data[i];
      if (entry.aliases.includes(property.toLowerCase())) {
        obj = entry as {
          name: string;
          aliases: string[];
          type: string;
          append: string;
          values: { name: string; aliases: string[] }[];
        };
        break;
      }
```

```
      }

      if (obj.name !== "_incode.default") {
        let v = value;
        for (let i = 0; i < obj.values.length; i++) {
          if (obj.values[i].aliases.includes(v.toLowerCase())) {
            v = obj.values[i].name;
            break;
          }
        }
        if (obj.type === "style") {
          return `// Diese Zeile setzt eine Eigenschaft (style) der Variable
${variable}\n${variable}.style.${obj.name} = "${v}${obj.append}";`;
        } else if (obj.type === "attribute") {
          return `// Diese Zeile setzt eine Eigenschaft der Variable ${variable}\n${variable}.${obj.name} =
${v}${obj.append};`;
        }
      } else {
        if (
          AliasManager.getSpecialAliases(property).length > 0 &&
          AliasManager.getSpecialAliases(property)[0] === "value"
        ) {
          return `// Diese Zeile setzt den Wert der Variable ${variable}\n${variable} = ${value};`;
        } else {
          return Error.ERROR_UNKNOWN_TYPE;
        }
      }

      return `// Diese Zeile setzt eine Eigenschaft der Variable ${variable}\n${variable}.${property} =
${value};`;
    }
  }
}
```

## packages/incode-compiler/src/module/commands/WaitCommand.ts

```
/**
 * @author Ben Siebert <ben@mctzock.de>
 * @copyright (c) 2018-2021 Ben Siebert. All rights reserved.
 */
import { InCodeCommand } from "./InCodeCommand";
import { Error } from "../Error";

export class WaitCommand extends InCodeCommand {
  execute(args: string[]): string {
    if (args.length < 1) {
      return Error.ERROR_MISSING_PARAMETER;
    } else {
      if (!parseFloat(args[0])) {
        return Error.ERROR_NOT_A_NUMBER;
      } else {
        return `// Diese Zeile wartet ${args[0]}000 Millisekunden\nawait new Promise(done =>
setTimeout(() => done(), ${args[0]}000))`;
      }
    }
  }
}
```

## packages/incode-compiler/src/module/Compiler.ts

```
/**
 * @author Ben Siebert <ben@mctzock.de>
```

```
 * @copyright (c) 2018-2021 Ben Siebert. All rights reserved.
 */
import { AbstractSyntaxTree } from "../types/AbstractSyntaxTree";
import { AbstractSyntaxTreeGenerator } from "./AbstractSyntaxTreeGenerator";
import { CommandExecutor } from "./commands/CommandExecutor";
import { js_beautify as beautify } from "js-beautify";
import { PreCompiler } from "../util/PreCompiler";

export class Compiler {
  public static compile(source: string | AbstractSyntaxTree[]): string {
    if (typeof source === "string") {
      source = PreCompiler.preCompile(source);
    }
    let ast: AbstractSyntaxTree[] = [];
    if (typeof source === "string") {
      ast = AbstractSyntaxTreeGenerator.generate(source);
    }

    let code = `/**
 * @generator InCode
 * @version 2.x
 */\nwindow.incode = {};\n(async () => {\n\n`;

    ast.forEach((node) => {
      code += "\n" + CommandExecutor.executeCommand(node) + "\n";
    });
    code += "\n})();";

    code = code.replace(/\n\n/g, "");

    return beautify(code, {
      indent_char: " ",
      indent_size: 2,
    });
  }
}
```

## packages/incode-compiler/src/module/Error.ts

```
/**
 * @author Ben Siebert <ben@mctzock.de>
 * @copyright (c) 2018-2021 Ben Siebert. All rights reserved.
 */

export class Error {
  public static readonly ERROR_MISSING_PARAMETER =
    "// FEHLER: Diese Zeile enthielt zu wenige Parameter.";
  public static readonly ERROR_UNKNOWN_TYPE =
    "// FEHLER: Diese Zeile enthielt einen unbekannten Typ.";
  public static readonly ERROR_UNKNOWN_OPERATOR =
    "// FEHLER: Diese Zeile enthielt einen unbekannten Operator.";
  public static readonly ERROR_NOT_A_NUMBER =
    "// FEHLER: Diese Zeile enthielt keine Zahl.";
}
```

## packages/incode-compiler/src/module/__tests__/AbstractSyntaxTreeGenerator.test.ts

```
/**
 * @author Ben Siebert <ben@mctzock.de>
 * @copyright (c) 2018-2021 Ben Siebert. All rights reserved.
```

```
 */

import { AbstractSyntaxTreeGenerator } from "../AbstractSyntaxTreeGenerator";

test("Generate AST", () => {
  const code =
    'Gib "Hallo Welt" in der Dialogbox aus\nErstelle x als Methode\n\tGib "1" in der Konsole aus\n\tGib
"2" in der Konsole aus\nGib "Hello World" in der Konsole aus';
  expect(
    JSON.stringify(AbstractSyntaxTreeGenerator.generate(code), null, 4)
  ).toBe(
    JSON.stringify(
      [
        {
          intents: 0,
          command: "Gib",
          args: ["'Hallo\u0000Welt'", "in", "der", "Dialogbox", "aus"],
          children: [],
        },
        {
          intents: 0,
          command: "Erstelle",
          args: ["x", "als", "Methode"],
          children: [
            {
              intents: 1,
              command: "Gib",
              args: ["'1'", "in", "der", "Konsole", "aus"],
              children: [],
            },
            {
              intents: 1,
              command: "Gib",
              args: ["'2'", "in", "der", "Konsole", "aus"],
              children: [],
            },
          ],
        },
        {
          intents: 0,
          command: "Gib",
          args: ["'Hello\u0000World'", "in", "der", "Konsole", "aus"],
          children: [],
        },
      ],
      null,
      4
    )
  );
});
```

## packages/incode-compiler/src/module/__tests__/CodeGenerator.test.ts

```
/**
 * @author Ben Siebert <ben@mctzock.de>
 * @copyright (c) 2018-2021 Ben Siebert. All rights reserved.
 */

import { CodeGenerator } from "../CodeGenerator";

test("Generate Code", () => {
```

```
  const code = JSON.stringify(
    'Gib "Hallo Welt" in der Dialogbox aus\nErstelle x als Methode\n\tGib "1" in der Konsole aus\n\tGib
"2" in der Konsole aus\nGib "Hello World" in der Konsole aus\n'
  );
  expect(
    JSON.stringify(
      CodeGenerator.generate([
        {
          intents: 0,
          command: "Gib",
          args: ["'Hallo\u0000Welt'", "in", "der", "Dialogbox", "aus"],
          children: [],
        },
        {
          intents: 0,
          command: "Erstelle",
          args: ["x", "als", "Methode"],
          children: [
            {
              intents: 1,
              command: "Gib",
              args: ["'1'", "in", "der", "Konsole", "aus"],
              children: [],
            },
            {
              intents: 1,
              command: "Gib",
              args: ["'2'", "in", "der", "Konsole", "aus"],
              children: [],
            },
          ],
        },
        {
          intents: 0,
          command: "Gib",
          args: ["'Hello\u0000World'", "in", "der", "Konsole", "aus"],
          children: [],
        },
      ])
    )
  ).toBe(code);
});
```

## packages/incode-compiler/src/types/AbstractSyntaxTree.ts

```
/**
 * @author Ben Siebert <ben@mctzock.de>
 * @copyright (c) 2018-2021 Ben Siebert. All rights reserved.
 */

export interface AbstractSyntaxTree {
  command: string;
  intents: number;
  children: AbstractSyntaxTree[];
  args: string[];
}
```

## packages/incode-compiler/src/types/JSONObject.ts

```
/**
 * @author Ben Siebert <ben@mctzock.de>
 * @copyright (c) 2018-2021 Ben Siebert. All rights reserved.
```

```
 */

export interface JSONObject {
  [key: string]: any;
}
```

## packages/incode-compiler/src/util/PreCompiler.ts

```
/**
 * @author Ben Siebert <ben@mctzock.de>
 * @copyright (c) 2018-2021 Ben Siebert. All rights reserved.
 */

export class PreCompiler {
  public static preCompile(source: string): string {
    source = source.replace(/ {2}/g, "");
    source = source.replace(/\r/g, "");
    return source;
  }
}
```

## packages/incode-compiler/src/util/__tests__/PreCompiler.test.ts

```
/**
 * @author Ben Siebert <ben@mctzock.de>
 * @copyright (c) 2018-2021 Ben Siebert. All rights reserved.
 */

import { PreCompiler } from "../PreCompiler";

test("Remove Whitespaces", () => {
  expect(PreCompiler.preCompile("W  hitespaces")).toBe("Whitespaces");
});

test("Remove Escape Characters", () => {
  expect(PreCompiler.preCompile("\r")).toBe("");
});
```

## packages/incode-compiler/tsconfig.json

```
{
  "extends": "tsconfig/base.json",
  "exclude": ["dist", "build", "node_modules"],
  "compilerOptions": {
    "isolatedModules": false,
    "lib": ["DOM"],
    "rootDir": "src",
    "outDir": "dist"
  }
}
```

## packages/rest/package.json

```
{
  "name": "@codeup/rest",
  "version": "1.0.0",
  "main": "src/index.ts",
  "author": "Ben Siebert",
  "license": "GPL-3.0-only",
  "private": false,
  "scripts": {
    "build": "tsc"
  }
}
```

## packages/rest/src/api/Account.ts

```
/**
 * packages/rest/src/api/Account.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 20.07.2022
 *
 */
import { IResponse } from "../types/IResponse";
import makeRequest from "../makeRequest";

export async function register(options: {
  username: string;
  email: string;
  firstName: string;
  lastName: string;
  password: string;
}): Promise<IResponse> {
  return await makeRequest({
    path: "/api/account/register",
    body: {
      username: options.username,
      email: options.email,
      firstName: options.firstName,
      lastName: options.lastName,
      password: options.password,
    },
    method: "POST",
  });
}

export async function login(options: {
  email: string;
  password: string;
  code?: string;
}): Promise<IResponse> {
  return await makeRequest({
    path: "/api/account/login",
    body: {
      email: options.email,
      password: options.password,
      code: options.code ? options.code : null,
    },
    method: "POST",
  });
}

export async function loginWithUsername(options: {
  username: string;
  password: string;
  code?: string;
}): Promise<IResponse> {
  return await makeRequest({
    path: "/api/account/login",
    body: {
      username: options.username,
      password: options.password,
```

```
      code: options.code ? options.code : null,
    },
    method: "POST",
  });
}

export async function checkUsername(name: string): Promise<IResponse> {
  return await makeRequest({
    path: "/api/account/check-username/" + name,
    method: "GET",
  });
}

export async function checkResetToken(token: string): Promise<IResponse> {
  return await makeRequest({
    method: "GET",
    path: "/api/account/check-reset-token/" + token,
  });
}

export async function deleteAccount(token: string): Promise<IResponse> {
  return await makeRequest({
    method: "POST",
    path: "/api/account/delete",
    token: token,
  });
}

export async function reset(email: string): Promise<IResponse> {
  return await makeRequest({
    path: "/api/account/reset",
    method: "POST",
    body: {
      email: email,
    },
  });
}

export async function resetConfirm(options: {
  resetToken: string;
  password: string;
}): Promise<IResponse> {
  return await makeRequest({
    method: "POST",
    path: "/api/account/reset-confirm",
    body: {
      token: options.resetToken,
      password: options.password,
    },
  });
}

export async function update(options: {
  keyName: string;
  keyValue: string;
  token: string;
}): Promise<IResponse> {
  return await makeRequest({
    path: "/api/account/update",
    method: "POST",
```

```
    token: options.token,
    body: {
      keyName: options.keyName,
      keyValue: options.keyValue,
    },
  });
}

export async function verify(token: string): Promise<IResponse> {
  return await makeRequest({
    path: "/api/account/verify",
    method: "GET",
    token: token,
  });
}

export async function createTestAccount(): Promise<IResponse> {
  return await makeRequest({
    path: "/api/account/try",
    method: "POST",
  });
}

export async function update2FA(options: {
  token: string;
  code?: string;
  enabled: boolean;
}): Promise<IResponse> {
  return await makeRequest({
    path: "/api/account/update-2fa",
    method: "POST",
    token: options.token,
    body: {
      code: options.code,
      enable: options.enabled ? true : undefined,
      disable: options.enabled ? undefined : true,
    },
  });
}

export async function getUserById(id: string): Promise<IResponse> {
  return await makeRequest({
    path: "/api/users/" + id,
    method: "GET",
  });
}

export async function getUserByName(name: string): Promise<IResponse> {
  return await makeRequest({
    path: "/api/user/" + name,
    method: "GET",
  });
}
```

## packages/rest/src/api/Admin.ts

```
/**
 * packages/rest/src/api/Admin.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
```

```
 * License: Project License
 * Created At: 07.10.22
 *
 */
import { IResponse } from "../types/IResponse";
import makeRequest from "../makeRequest";

export async function listAccounts(token: string): Promise<IResponse> {
  return makeRequest({
    method: "GET",
    token,
    path: "/api/admin/account/list",
  });
}

export async function updateAccount(options: {
  token: string;
  username: string;
  update: { [key: string]: string };
}): Promise<IResponse> {
  return makeRequest({
    method: "POST",
    token: options.token,
    path: "/api/admin/account/update",
    body: {
      username: options.username,
      update: options.update,
    },
  });
}

export async function deleteAccount(options: {
  token: string;
  username: string;
}): Promise<IResponse> {
  return makeRequest({
    method: "POST",
    token: options.token,
    path: "/api/admin/account/delete",
    body: {
      username: options.username,
    },
  });
}

export async function createAccount(options: {
  token: string;
  username: string;
  email: string;
  firstName: string;
  lastName: string;
  active: boolean;
  role: string;
  password: string;
}): Promise<IResponse> {
  return makeRequest({
    method: "POST",
    token: options.token,
    path: "/api/admin/account/create",
    body: {
```

```
      username: options.username,
      email: options.email,
      firstName: options.firstName,
      lastName: options.lastName,
      active: options.active,
      role: options.role,
      password: options.password,
    },
  });
}

export async function listForumProfiles(token: string): Promise<IResponse> {
  return makeRequest({
    method: "GET",
    token,
    path: "/api/admin/forum_profiles/list",
  });
}

export async function deleteForumProfile(options: {
  token: string;
  username: string;
}): Promise<IResponse> {
  return makeRequest({
    method: "POST",
    token: options.token,
    path: "/api/admin/forum_profiles/delete",
    body: {
      username: options.username,
    },
  });
}

export async function updateForumProfile(options: {
  token: string;
  username: string;
  update: { [key: string]: string };
}): Promise<IResponse> {
  return makeRequest({
    method: "POST",
    token: options.token,
    path: "/api/admin/forum_profiles/update",
    body: {
      username: options.username,
      update: options.update,
    },
  });
}

export async function listCourses(
  token: string,
  names_only: boolean = false
): Promise<IResponse> {
  return makeRequest({
    method: "GET",
    token,
    path: `/api/admin/course/list${names_only ? "?list=true" : ""}`,
  });
}
```

```
export async function createCourse(options: {
  token: string;
  name: string;
  friendlyName: string;
  description: string;
  splashImage: string;
}): Promise<IResponse> {
  return makeRequest({
    method: "POST",
    token: options.token,
    path: "/api/admin/course/create",
    body: {
      name: options.name,
      friendlyName: options.friendlyName,
      description: options.description,
      splashImage: options.splashImage,
    },
  });
}

export async function deleteCourse(options: {
  token: string;
  friendlyName: string;
}): Promise<IResponse> {
  return makeRequest({
    method: "POST",
    token: options.token,
    path: "/api/admin/course/delete",
    body: {
      friendlyName: options.friendlyName,
    },
  });
}

export async function updateCourse(options: {
  token: string;
  id: string;
  update: { [key: string]: string };
}): Promise<IResponse> {
  return makeRequest({
    method: "POST",
    token: options.token,
    path: "/api/admin/course/edit",
    body: {
      id: options.id,
      update: options.update,
    },
  });
}

export async function createVideo(options: {
  token: string;
  title: string;
  friendlyName: string;
  youtubeURL: string;
  course: string;
  transcript: string;
  quiz: any;
}): Promise<IResponse> {
  return makeRequest({
```

```
    method: "POST",
    token: options.token,
    path: "/api/admin/video/create",
    body: {
      title: options.title,
      friendlyName: options.friendlyName,
      youtubeURL: options.youtubeURL,
      course: options.course,
      transcript: options.transcript,
      quiz: options.quiz,
    },
  });
}

export async function listVideos(token: string): Promise<IResponse> {
  return makeRequest({
    method: "GET",
    token,
    path: "/api/admin/video/list",
  });
}

export async function getVideo(options: {
  token: string;
  vid: string;
}): Promise<IResponse> {
  return makeRequest({
    method: "GET",
    token: options.token,
    path: `/api/admin/video/youtube/${options.vid}`,
  });
}

export async function deleteVideo(options: {
  token: string;
  vid: string;
}): Promise<IResponse> {
  return makeRequest({
    method: "DELETE",
    token: options.token,
    path: `/api/admin/video/youtube/${options.vid}`,
  });
}

export async function updateVideo(options: {
  token: string;
  vid: string;
  update: { [key: string]: string };
}): Promise<IResponse> {
  return makeRequest({
    method: "POST",
    token: options.token,
    path: `/api/admin/video/youtube/${options.vid}`,
    body: {
      update: options.update,
    },
  });
}

/** **/
```

```
export async function createText(options: {
  token: string;
  title: string;
  friendlyName: string;
  course: string;
  content: string;
  quiz: any;
}): Promise<IResponse> {
  return makeRequest({
    method: "POST",
    token: options.token,
    path: "/api/admin/text/create",
    body: {
      title: options.title,
      friendlyName: options.friendlyName,
      course: options.course,
      content: options.content,
      quiz: options.quiz,
    },
  });
}

export async function listTexts(token: string): Promise<IResponse> {
  return makeRequest({
    method: "GET",
    token,
    path: "/api/admin/text/list",
  });
}

export async function getText(options: {
  token: string;
  tid: string;
}): Promise<IResponse> {
  return makeRequest({
    method: "GET",
    token: options.token,
    path: `/api/admin/text/${options.tid}`,
  });
}

export async function deleteText(options: {
  token: string;
  tid: string;
}): Promise<IResponse> {
  return makeRequest({
    method: "DELETE",
    token: options.token,
    path: `/api/admin/text/${options.tid}`,
  });
}

export async function updateText(options: {
  token: string;
  tid: string;
  update: { [key: string]: string };
}): Promise<IResponse> {
  return makeRequest({
    method: "POST",
```

```
    token: options.token,
    path: `/api/admin/text/${options.tid}`,
    body: {
      update: options.update,
    },
  });
}

export async function createContent(options: {
  token: string;
  displayName: string;
  course: string;
  url: string;
  type: "VIDEO" | "TEXT";
  textContent: string;
  quiz: {
    question: string;
    answers: {
      answer: string;
      correct: boolean;
    }[];
  }[];
  videoUrl?: string;
}): Promise<IResponse> {
  return makeRequest({
    method: "POST",
    token: options.token,
    path: "/api/admin/contents",
    body: {
      displayName: options.displayName,
      course: options.course,
      url: options.url,
      type: options.type,
      textContent: options.textContent,
      quiz: options.quiz,
      videoUrl: options.videoUrl,
    },
  });
}

export async function getContents(options: {
  token: string;
  cid?: string;
  search?: string;
  course?: string;
  type?: string;
}): Promise<IResponse> {
  // not the best way to remove the token from the options object
  // so that it doesn't get added to the query string
  // but it works for now.
  let token = options.token;
  // @ts-ignore
  delete options.token;
  return makeRequest({
    method: "GET",
    token: token,
    path: "/api/admin/contents?" + new URLSearchParams(options).toString(),
  });
}
```

```
export async function deleteContent(options: {
  token: string;
  cid: string;
}): Promise<IResponse> {
  return makeRequest({
    method: "DELETE",
    token: options.token,
    path: `/api/admin/contents`,
    body: {
      id: options.cid,
    },
  });
}

export async function getContactMessages(token: string): Promise<IResponse> {
  return makeRequest({
    method: "GET",
    token,
    path: "/api/admin/contact/messages",
  });
}

export async function getContactMessageCount(
  token: string
): Promise<IResponse> {
  return makeRequest({
    method: "GET",
    token,
    path: "/api/admin/contact/messages?count=true",
  });
}

export async function deleteContactMessage(options: {
  token: string;
  id: string;
}): Promise<IResponse> {
  return makeRequest({
    method: "DELETE",
    token: options.token,
    path: "/api/admin/contact/messages?id=" + options.id,
  });
}

export async function contactUser(options: {
  token: string;
  username: string;
  message: string;
}): Promise<IResponse> {
  return makeRequest({
    method: "POST",
    token: options.token,
    path: "/api/admin/contact/user",
    body: {
      username: options.username,
      message: options.message,
    },
  });
}

export async function getAllObtainedCertificates(
```

```
  token: string
): Promise<IResponse> {
  return makeRequest({
    method: "GET",
    token,
    path: "/api/admin/certificates/list",
  });
}

export async function deleteObtainedCertificate(options: {
  token: string;
  id: string;
}): Promise<IResponse> {
  return makeRequest({
    method: "DELETE",
    token: options.token,
    path: "/api/admin/certificates/delete?id=" + options.id,
  });
}

export async function getProjectIdeas(token: string): Promise<IResponse> {
  return makeRequest({
    method: "GET",
    token,
    path: "/api/admin/project-ideas",
  });
}

export async function deleteProjectIdea(options: {
  token: string;
  id: string;
}): Promise<IResponse> {
  return makeRequest({
    method: "DELETE",
    token: options.token,
    path: "/api/admin/project-ideas?id=" + options.id,
  });
}

export async function createProjectIdea(options: {
  token: string;
  title: string;
  description: string;
  exampleDeployedURL?: string;
  exampleSourceURL?: string;
  level: string;
}): Promise<IResponse> {
  return makeRequest({
    method: "POST",
    token: options.token,
    path: "/api/admin/project-ideas",
    body: {
      title: options.title,
      description: options.description,
      exampleDeployedURL: options.exampleDeployedURL,
      exampleSourceURL: options.exampleSourceURL,
      level: options.level,
    },
  });
}
```

```
export async function getBlogs(options: { token: string }): Promise<IResponse> {
  return makeRequest({
    method: "GET",
    token: options.token,
    path: "/api/admin/blog",
  });
}

export async function createBlog(options: {
  token: string;
  title: string;
  content: string;
  tags: string[];
}): Promise<IResponse> {
  return makeRequest({
    method: "POST",
    token: options.token,
    path: "/api/admin/blog",
    body: {
      title: options.title,
      content: options.content,
      tags: options.tags,
    },
  });
}

export async function deleteBlog(options: {
  token: string;
  id: string;
}): Promise<IResponse> {
  return makeRequest({
    method: "DELETE",
    token: options.token,
    path: "/api/admin/blog?id=" + options.id,
  });
}

export async function updateBlog(options: {
  token: string;
  id: string;
  update: {
    [key: string]: any;
  };
}): Promise<IResponse> {
  return makeRequest({
    method: "PUT",
    token: options.token,
    path: "/api/admin/blog",
    body: {
      id: options.id,
      update: options.update,
    },
  });
}

export async function getUserJWT(options: {
  token: string;
  username: string;
}): Promise<IResponse> {
```

```
  return makeRequest({
    method: "POST",
    token: options.token,
    path: "/api/admin/account/jwt",
    body: {
      username: options.username,
    },
  });
}

export async function getOverview(token: string): Promise<IResponse> {
  return makeRequest({
    method: "GET",
    token,
    path: "/api/admin/overview",
  });
}

export async function getUserAnalytics(options: {
  token: string;
  userId: string;
  detailed?: boolean;
}): Promise<IResponse> {
  return makeRequest({
    method: "GET",
    token: options.token,
    path: `/api/admin/analytics/user-summary?userId=${
      options.userId
    }&detailed=${options.detailed ? "true" : "false"}`,
  });
}

export async function getTestimonials(token: string): Promise<IResponse> {
  return makeRequest({
    method: "GET",
    token,
    path: "/api/admin/testimonials",
  });
}

export async function createTestimonial(options: {
  token: string;
  name: string;
  position: string;
  company: string;
  image: string;
  content: string;
}): Promise<IResponse> {
  return makeRequest({
    method: "POST",
    token: options.token,
    path: "/api/admin/testimonials",
    body: {
      name: options.name,
      position: options.position,
      company: options.company,
      image: options.image,
      content: options.content,
    },
  });
```

```
}

export async function deleteTestimonial(options: {
  token: string;
  id: string;
}): Promise<IResponse> {
  return makeRequest({
    method: "DELETE",
    token: options.token,
    path: "/api/admin/testimonials?id=" + options.id,
  });
}

export async function updateTestimonial(options: {
  token: string;
  id: string;
  update: {
    [key: string]: any;
  };
}): Promise<IResponse> {
  return makeRequest({
    method: "PUT",
    token: options.token,
    path: "/api/admin/testimonials?id=" + options.id,
    body: {
      update: options.update,
    },
  });
}

export async function getOrgs(token: string): Promise<IResponse> {
  return makeRequest({
    method: "GET",
    token,
    path: "/api/admin/orgs",
  });
}

export async function getNewsletterMembers(options: {
  token: string;
  count?: boolean;
}): Promise<IResponse> {
  return makeRequest({
    method: "GET",
    token: options.token,
    path: `/api/admin/newsletter/members${options.count ? "?count=true" : ""}`,
  });
}

export async function sendNewsletter(options: {
  token: string;
  title: string;
  content: string;
}): Promise<IResponse> {
  return makeRequest({
    method: "POST",
    token: options.token,
    path: "/api/admin/newsletter",
    body: {
      title: options.title,
```

```
      content: options.content,
    },
  });
}

export async function removeFromDiscovery(options: {
  token: string;
  id: string;
}): Promise<IResponse> {
  return makeRequest({
    method: "DELETE",
    token: options.token,
    path: `/api/admin/discovery/delete?id=${options.id}`,
  });
}

export async function getFlows(token: string): Promise<IResponse> {
  return makeRequest({
    method: "GET",
    token,
    path: "/api/admin/flows",
  });
}

export async function getKidsProjects(token: string): Promise<IResponse> {
  return makeRequest({
    method: "GET",
    token,
    path: "/api/admin/kids",
  });
}

export async function getRoads(token: string): Promise<IResponse> {
  return makeRequest({
    method: "GET",
    token,
    path: "/api/admin/roads",
  });
}

export async function getTodos(token: string): Promise<IResponse> {
  return makeRequest({
    method: "GET",
    token,
    path: "/api/admin/todos",
  });
}

export async function getSnippets(token: string): Promise<IResponse> {
  return makeRequest({
    method: "GET",
    token,
    path: "/api/admin/snippets",
  });
}

export async function getShorts(token: string): Promise<IResponse> {
  return makeRequest({
    method: "GET",
    token,
```

```
    path: "/api/admin/shorturls",
  });
}

export async function createShort(options: {
  token: string;
  dest: string;
  short: string;
}): Promise<IResponse> {
  return makeRequest({
    method: "POST",
    token: options.token,
    path: "/api/admin/shorturls",
    body: {
      short: options.short,
      destination: options.dest,
    },
  });
}

export async function deleteShort(options: {
  token: string;
  short: string;
}): Promise<IResponse> {
  return makeRequest({
    method: "DELETE",
    token: options.token,
    path: "/api/admin/shorturls?short=" + options.short,
  });
}

export async function getIDEProjects(token: string): Promise<IResponse> {
  return makeRequest({
    method: "GET",
    token,
    path: "/api/admin/ide",
  });
}

export async function getEditorProjects(token: string): Promise<IResponse> {
  return makeRequest({
    method: "GET",
    token,
    path: "/api/admin/editor",
  });
}

export async function getErrors(token: string): Promise<IResponse> {
  return makeRequest({
    method: "GET",
    token,
    path: "/api/admin/errors",
  });
}

export async function createChallenge(options: {
  token: string;
  title: string;
  task: string;
  difficulty: number;
```

```
  expectedOutput: string;
}): Promise<IResponse> {
  return makeRequest({
    method: "POST",
    token: options.token,
    path: "/api/admin/challenges",
    body: {
      title: options.title,
      task: options.task,
      difficulty: options.difficulty,
      expectedOutput: options.expectedOutput,
    },
  });
}

export async function deleteChallenge(options: {
  token: string;
  id: string;
}): Promise<IResponse> {
  return makeRequest({
    method: "DELETE",
    token: options.token,
    path: "/api/admin/challenges?id=" + options.id,
  });
}

export async function updateChallenge(options: {
  token: string;
  id: string;
  title: string;
  task: string;
  difficulty: number;
  expectedOutput: string;
}): Promise<IResponse> {
  return makeRequest({
    method: "PUT",
    token: options.token,
    path: "/api/admin/challenges",
    body: {
      id: options.id,
      title: options.title,
      task: options.task,
      difficulty: options.difficulty,
      expectedOutput: options.expectedOutput,
    },
  });
}
```

## packages/rest/src/api/Analytics.ts

```
/**
 * packages/rest/src/api/Analytics.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 25.11.2022
 *
 */
import { IResponse } from "../types/IResponse";
import makeRequest from "../makeRequest";
```

```
export async function postAnalytics(options: {
  token?: string;
}): Promise<IResponse> {
  return await makeRequest({
    method: "POST",
    path: "/api/analytics",
    token: options.token,
  });
}

export async function getSummary(
  token: string,
  userId?: string
): Promise<IResponse> {
  return await makeRequest({
    method: "GET",
    path: `/api/admin/analytics/summary${userId ? `?userId=${userId}` : ""}`,
    token,
  });
}

export async function getViews(options: {
  token: string;
  periodStart?: string;
  periodEnd?: string;
}): Promise<IResponse> {
  return await makeRequest({
    method: "GET",
    path:
      "/api/admin/analytics/views" +
      (options.periodStart && options.periodEnd
        ? `?periodStart=${options.periodStart}&periodEnd=${options.periodEnd}`
        : ""),
    token: options.token,
  });
}

export async function deleteAnalytics(options: {
  token: string;
  periodStart: string;
  periodEnd: string;
}): Promise<IResponse> {
  return await makeRequest({
    method: "POST",
    path:
      "/api/admin/analytics/delete" +
      `?periodStart=${options.periodStart}&periodEnd=${options.periodEnd}`,
    token: options.token,
  });
}
```

## packages/rest/src/api/Ask.ts

```
/**
 * packages/rest/src/api/Ask.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 24.06.23
```

```
 *
 */

import { IResponse } from "../types/IResponse";
import makeRequest from "../makeRequest";

export async function deletePost(
  id: string,
  token: string
): Promise<IResponse> {
  return await makeRequest({
    method: "DELETE",
    token: token,
    path: "/api/ask/posts?id=" + id,
  });
}

export async function createPost(
  content: string,
  token: string,
  parent?: string
): Promise<IResponse> {
  return await makeRequest({
    method: "POST",
    path: "/api/ask/posts",
    body: {
      content: content,
      parent: parent,
    },
    token: token,
  });
}

export async function getPost(id: string): Promise<IResponse> {
  return await makeRequest({
    method: "GET",
    path: "/api/ask/posts?id=" + id,
  });
}

export async function getPostsByUser(
  username: string,
  page?: number
): Promise<IResponse> {
  return await makeRequest({
    method: "GET",
    path: "/api/ask/posts?user=" + username + "&page=" + page,
  });
}

export async function getPostsByParent(
  id: string,
  page?: number
): Promise<IResponse> {
  return await makeRequest({
    method: "GET",
    path: "/api/ask/posts?parent=" + id + "&page=" + page,
  });
}
```

```
export async function searchPosts(
  query: string,
  page?: number
): Promise<IResponse> {
  return await makeRequest({
    method: "GET",
    path: "/api/ask/posts?query=" + query + "&page=" + page,
  });
}

export async function getPosts(page?: number): Promise<IResponse> {
  return await makeRequest({
    method: "GET",
    path: "/api/ask/posts?page=" + page,
  });
}

export async function getProfile(username: string): Promise<IResponse> {
  return await makeRequest({
    method: "GET",
    path: "/api/ask/profiles/" + username,
  });
}

export async function updateProfile(
  username: string,
  token: string,
  update: {
    bio?: string;
    banner?: string;
    website?: string;
    projects?: string[];
    certificates?: string[];
  }
): Promise<IResponse> {
  return await makeRequest({
    method: "POST",
    path: "/api/ask/profiles/" + username,
    body: { update },
    token: token,
  });
}
```

## packages/rest/src/api/Blog.ts

```
/**
 * packages/rest/src/api/Blog.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 27.12.2022
 *
 */

import { IResponse } from "../types/IResponse";
import makeRequest from "../makeRequest";

export async function getBlogs(): Promise<IResponse> {
  return await makeRequest({
    method: "GET",
```

```
    path: "/api/blog",
  });
}

export async function getBlog(id: string): Promise<IResponse> {
  return await makeRequest({
    method: "GET",
    path: `/api/blog/${id}`,
  });
}
```

## packages/rest/src/api/Certificates.ts

```
/**
 * packages/rest/src/api/Certificates.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 27.11.2022
 *
 */
import { IResponse } from "../types/IResponse";
import makeRequest from "../makeRequest";

export async function getCertificate(options: {
  token: string;
  course: string;
  onlyLookup: boolean;
}): Promise<IResponse> {
  return makeRequest({
    method: "GET",
    token: options.token,
    path: `/api/course/${options.course}/certificate?onlyLookup=${options.onlyLookup}`,
  });
}

export async function submitCertificate(options: {
  token: string;
  course: string;
  answers: any[];
}): Promise<IResponse> {
  return makeRequest({
    method: "POST",
    path: `/api/course/${options.course}/certificate`,
    token: options.token,
    body: {
      answers: options.answers,
    },
  });
}

export async function getObtainedCertificate(id: string): Promise<IResponse> {
  return makeRequest({
    method: "GET",
    path: `/api/cert?id=${id}`,
  });
}

export async function getObtainedCertificates(
  token: string
```

```
): Promise<IResponse> {
  return makeRequest({
    method: "GET",
    path: `/api/certificates/my`,
    token: token,
  });
}
```

## packages/rest/src/api/CodeChallenges.ts

```
/**
 * packages/rest/src/api/CodeChallenges.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2024 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 18.05.2024
 *
 */

import makeRequest from "../makeRequest";
import { IResponse } from "../types/IResponse";

export async function getChallenges(
  token: string,
  id?: string
): Promise<IResponse> {
  return await makeRequest({
    method: "GET",
    path: "/api/challenges" + (id ? `?id=${id}` : ""),
    token,
  });
}

export async function getCompletedChallenges(
  token: string
): Promise<IResponse> {
  return await makeRequest({
    method: "GET",
    path: "/api/challenges/completed",
    token,
  });
}

export async function finalCheck(
  token: string,
  challengeId: string,
  code: string,
  language: string
): Promise<IResponse> {
  return await makeRequest({
    method: "POST",
    path: "/api/challenges/check",
    token,
    body: {
      challengeId,
      code,
      language,
    },
  });
}
```

## packages/rest/src/api/Community.ts

```
/**
 * packages/rest/src/api/Community.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 21.07.2022
 *
 */
import { IResponse } from "../types/IResponse";
import makeRequest from "../makeRequest";

export async function search(term: string): Promise<IResponse> {
  return await makeRequest({
    path: "/api/community/search?term=" + term,
    method: "GET",
  });
}

export async function getFeed(options: {
  from: number;
  to: number;
  idsOnly?: boolean;
  noReplies?: boolean;
}): Promise<IResponse> {
  return await makeRequest({
    path: `/api/community/feed?from=${options.from}&to=${options.to}&idsOnly=${
      options.idsOnly || false
    }&noReplies=${options.noReplies || false}`,
    method: "GET",
  });
}

export async function existsProfile(id: string): Promise<IResponse> {
  return await makeRequest({
    path: "/api/community/has-profile/" + id,
    method: "GET",
  });
}

export async function createPost(options: {
  token: string;
  content: string;
  parent: string | null | undefined;
}): Promise<IResponse> {
  return await makeRequest({
    path: "/api/community/post/create",
    method: "POST",
    token: options.token,
    body: {
      content: options.content,
      parent: options.parent ? options.parent : null,
    },
  });
}

export async function getPost(id: string): Promise<IResponse> {
  return await makeRequest({
```

```
    method: "GET",
    path: "/api/community/post/" + id,
  });
}

export async function likePost(options: {
  token: string;
  postId: string;
}): Promise<IResponse> {
  return await makeRequest({
    path: "/api/community/post/" + options.postId + "/like",
    method: "POST",
    token: options.token,
  });
}

export async function deletePost(options: {
  token: string;
  postId: string;
}): Promise<IResponse> {
  return await makeRequest({
    path: "/api/community/post/" + options.postId + "/delete",
    method: "POST",
    token: options.token,
  });
}

export async function createProfile(options: {
  token: string;
  displayName: string;
  biography: string;
}): Promise<IResponse> {
  return await makeRequest({
    path: "/api/community/profile/create",
    token: options.token,
    body: {
      displayName: options.displayName,
      bio: options.biography,
    },
    method: "POST",
  });
}

export async function getProfile(
  name: string,
  small?: boolean
): Promise<IResponse> {
  let path = "/api/community/profile/" + name;

  if (small) {
    path += "?small=true";
  }

  return await makeRequest({
    path,
    method: "GET",
  });
}

export async function updateProfile(options: {
```

```
  token: string;
  update: { [key: string]: string };
}): Promise<IResponse> {
  return await makeRequest({
    path: "/api/community/profile/update",
    method: "POST",
    body: {
      update: options.update,
    },
    token: options.token,
  });
}

export async function isFollowing(
  profile: string,
  otherProfile: string
): Promise<IResponse> {
  return await makeRequest({
    path: "/api/community/profile/" + profile + "/is-following/" + otherProfile,
    method: "GET",
  });
}

export async function getProfileFollowers(options: {
  name: string;
  count?: boolean;
  from?: number;
  to?: number;
}): Promise<IResponse> {
  let path = "/api/community/profile/" + options.name + "/followers?";

  if (options.count) {
    path += "count=true&";
  }

  if (options.from) {
    path += "from=" + options.from + "&";
  }

  if (options.to) {
    path += "to=" + options.to + "&";
  }

  return await makeRequest({
    path,
    method: "GET",
  });
}

export async function getProfilePosts(options: {
  name: string;
  count?: boolean;
  from?: number;
  to?: number;
}): Promise<IResponse> {
  let path = "/api/community/profile/" + options.name + "/posts?";

  if (options.count) {
    path += "count=true&";
  }
```

```
  if (options.from) {
    path += "from=" + options.from + "&";
  }

  if (options.to) {
    path += "to=" + options.to + "&";
  }

  return await makeRequest({
    path,
    method: "GET",
  });
}

export async function toggleFollow(options: {
  token: string;
  profile: string;
}): Promise<IResponse> {
  return await makeRequest({
    path: "/api/community/profile/" + options.profile + "/toggle-follow",
    method: "POST",
    token: options.token,
  });
}
```

## packages/rest/src/api/Course.ts

```
/**
 * packages/rest/src/api/Course.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 05.11.2022
 *
 */
import { IResponse } from "../types/IResponse";
import makeRequest from "../makeRequest";

export async function getSection(options: {
  course: string;
  section: string;
  token?: string;
}): Promise<IResponse> {
  return await makeRequest({
    path: "/api/course/" + options.course + "/" + options.section,
    method: "GET",
    token: options.token ? options.token : undefined,
  });
}

export async function getCourse(options: {
  course: string;
  token?: string;
}): Promise<IResponse> {
  return await makeRequest({
    path: "/api/course/" + options.course,
    method: "GET",
    token: options.token ? options.token : undefined,
  });
```

```
}

export async function listCourses(options: {
  token?: string;
}): Promise<IResponse> {
  return await makeRequest({
    path: "/api/course/list-courses",
    method: "GET",
    token: options.token ? options.token : undefined,
  });
}

export async function getEnrollments(options: {
  token: string;
}): Promise<IResponse> {
  return await makeRequest({
    path: "/api/course/enrollments",
    method: "GET",
    token: options.token,
  });
}

export async function getEnrollment(options: {
  token: string;
  course: string;
}): Promise<IResponse> {
  return await makeRequest({
    path: "/api/course/" + options.course + "/enrollment",
    method: "GET",
    token: options.token,
  });
}

export async function changeEnrollment(options: {
  token: string;
  course: string;
}): Promise<IResponse> {
  return await makeRequest({
    path: "/api/course/" + options.course + "/enrollment",
    method: "POST",
    token: options.token,
  });
}

export async function getCourseProgress(options: {
  token: string;
  course: string;
}): Promise<IResponse> {
  return await makeRequest({
    path: "/api/course/" + options.course + "/progress",
    method: "GET",
    token: options.token,
  });
}

export async function updateCourseProgress(options: {
  token: string;
  course: string;
  section: string;
}): Promise<IResponse> {
```

```
  return await makeRequest({
    path: "/api/course/" + options.course + "/progress",
    method: "POST",
    token: options.token,
    body: {
      section: options.section,
    },
  });
}
```

## packages/rest/src/api/Creator.ts

```
/**
 * packages/rest/src/api/Creator.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 18.11.2022
 *
 */
import { IResponse } from "../types/IResponse";
import makeRequest from "../makeRequest";

export async function getCourses(token: string): Promise<IResponse> {
  return makeRequest({
    path: "/api/creator/course",
    method: "GET",
    token,
  });
}

export async function createCourse(options: {
  name: string;
  friendlyName: string;
  description: string;
  splashImage: string;
  token: string;
}): Promise<IResponse> {
  return makeRequest({
    path: "/api/creator/course",
    method: "POST",
    token: options.token,
    body: {
      name: options.name,
      friendlyName: options.friendlyName,
      description: options.description,
      splashImage: options.splashImage,
    },
  });
}

export async function deleteCourse(options: {
  id: string;
  token: string;
}): Promise<IResponse> {
  return makeRequest({
    path: "/api/creator/course",
    method: "DELETE",
    token: options.token,
    body: {
```

```
      id: options.id,
    },
  });
}

export async function updateCourse(options: {
  id: string;
  token: string;
  update: {
    [key: string]: string | any;
  };
}): Promise<IResponse> {
  return makeRequest({
    path: "/api/creator/course",
    method: "PATCH",
    token: options.token,
    body: {
      id: options.id,
      update: options.update,
    },
  });
}
export async function createContent(options: {
  token: string;
  displayName: string;
  course: string;
  url: string;
  type: "VIDEO" | "TEXT";
  textContent: string;
  quiz: {
    question: string;
    answers: {
      answer: string;
      correct: boolean;
    }[];
  }[];
  videoUrl?: string;
}): Promise<IResponse> {
  return makeRequest({
    method: "POST",
    token: options.token,
    path: "/api/creator/content",
    body: {
      displayName: options.displayName,
      course: options.course,
      url: options.url,
      type: options.type,
      textContent: options.textContent,
      quiz: options.quiz,
      videoUrl: options.videoUrl,
    },
  });
}

export async function getContents(options: {
  token: string;
  courseId?: string;
  contentId?: string;
}): Promise<IResponse> {
  const urlSearchParams = new URLSearchParams();
```

```
  if (options.courseId) {
    urlSearchParams.append("course", options.courseId);
  }

  if (options.contentId) {
    urlSearchParams.append("contentId", options.contentId);
  }

  return makeRequest({
    method: "GET",
    token: options.token,
    path:
      "/api/creator/content" +
      (urlSearchParams.toString() ? "?" + urlSearchParams.toString() : ""),
  });
}

export async function deleteContent(options: {
  token: string;
  cid: string;
}): Promise<IResponse> {
  return makeRequest({
    method: "DELETE",
    token: options.token,
    path: `/api/creator/content`,
    body: {
      id: options.cid,
    },
  });
}

export async function updateContent(options: {
  token: string;
  cid: string;
  update: {
    [key: string]: string | any;
  };
}): Promise<IResponse> {
  return makeRequest({
    method: "PATCH",
    token: options.token,
    path: `/api/creator/content`,
    body: {
      id: options.cid,
      update: options.update,
    },
  });
}

export async function getTopCreators(): Promise<IResponse> {
  return makeRequest({
    path: "/api/creator/top",
    method: "GET",
  });
}

export async function getCertificate(options: {
  token: string;
  courseId: string;
```

```
}): Promise<IResponse> {
  return makeRequest({
    path: `/api/creator/certificate?courseId=${options.courseId}`,
    method: "GET",
    token: options.token,
  });
}

export async function createCertificate(options: {
  token: string;
  courseId: string;
}): Promise<IResponse> {
  return makeRequest({
    path: `/api/creator/certificate`,
    method: "POST",
    token: options.token,
    body: {
      create: true,
      courseId: options.courseId,
    },
  });
}

export async function updateCertificate(options: {
  token: string;
  courseId: string;
  update: {
    [key: string]: string | any;
  };
}): Promise<IResponse> {
  return makeRequest({
    path: `/api/creator/certificate`,
    method: "POST",
    token: options.token,
    body: {
      update: options.update,
      courseId: options.courseId,
    },
  });
}
```

## packages/rest/src/api/Discovery.ts

```
/**
 * packages/rest/src/api/Discovery.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 23.12.2022
 *
 */

import { IResponse } from "../types/IResponse";
import makeRequest from "../makeRequest";

export async function getDiscoveryProjects(): Promise<IResponse> {
  return await makeRequest({
    path: "/api/discovery",
    method: "GET",
  });
```

```
}

export async function starProject(options: {
  token: string;
  projectId: string;
}): Promise<IResponse> {
  return await makeRequest({
    path: "/api/discovery/star",
    method: "POST",
    token: options.token,
    body: {
      projectId: options.projectId,
    },
  });
}

export async function commentProject(options: {
  token: string;
  projectId: string;
  comment: string;
}): Promise<IResponse> {
  return await makeRequest({
    path: "/api/discovery/comment",
    method: "POST",
    token: options.token,
    body: {
      projectId: options.projectId,
      comment: options.comment,
    },
  });
}
```

## packages/rest/src/api/Editor.ts

```
/**
 * packages/rest/src/api/Editor.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 18.03.2023
 *
 */

import { IResponse } from "../types/IResponse";
import makeRequest from "../makeRequest";

export async function getProjects(token: string): Promise<IResponse> {
  return await makeRequest({
    token: token,
    path: "/api/editor/projects",
    method: "GET",
  });
}

export async function getDomainById(id: string): Promise<IResponse> {
  return await makeRequest({
    path: `/api/editor/published-domain?id=${id}`,
    method: "GET",
  });
}
```

## packages/rest/src/api/Forum.ts

```
/**
 * packages/rest/src/api/Forum.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 23.09.2022
 *
 */

import { IResponse } from "../types/IResponse";
import makeRequest from "../makeRequest";

export async function createProfile(options: {
  token: string;
  username: string;
}): Promise<IResponse> {
  return await makeRequest({
    path: "/api/forum/" + options.username + "/profile",
    method: "POST",
    token: options.token,
  });
}

export async function getProfile(username: string): Promise<IResponse> {
  return await makeRequest({
    path: "/api/forum/" + username + "/profile",
    method: "GET",
  });
}

export async function modifyProfile(options: {
  update: { [key: string]: any };
  token: string;
  username: string;
}): Promise<IResponse> {
  return await makeRequest({
    path: "/api/forum/" + options.username + "/profile",
    method: "POST",
    token: options.token,
    body: options.update,
  });
}

export async function deleteProfile(options: {
  token: string;
  username: string;
}): Promise<IResponse> {
  return await makeRequest({
    path: "/api/forum/" + options.username + "/profile",
    method: "DELETE",
    token: options.token,
  });
}

export async function createPost(options: {
  token: string;
  content: string;
```

```
  parent?: string;
}): Promise<IResponse> {
  return await makeRequest({
    path: "/api/forum/post",
    method: "POST",
    token: options.token,
    body: {
      content: options.content,
      parent: options.parent,
    },
  });
}

export async function getPost(id: string): Promise<IResponse> {
  return await makeRequest({
    path: "/api/forum/post/" + id,
    method: "GET",
  });
}

export async function getProfilePosts(options: {
  username: string;
}): Promise<IResponse> {
  return await makeRequest({
    path: "/api/forum/" + options.username + "/posts",
    method: "GET",
  });
}

export async function deletePost(options: {
  token: string;
  id: string;
}): Promise<IResponse> {
  return await makeRequest({
    path: "/api/forum/post/" + options.id + "/delete",
    method: "POST",
    token: options.token,
  });
}

export async function getProfiles(): Promise<IResponse> {
  return await makeRequest({
    path: "/api/forum/profiles",
    method: "GET",
  });
}

export async function likePost(options: {
  token: string;
  id: string;
}): Promise<IResponse> {
  return await makeRequest({
    path: "/api/forum/post/" + options.id + "/like",
    method: "POST",
    token: options.token,
  });
}

export async function getComments(postId: string): Promise<IResponse> {
  return await makeRequest({
```

```
    path: "/api/forum/post/" + postId + "/comments",
    method: "GET",
  });
}

export async function getAllPosts(): Promise<IResponse> {
  return await makeRequest({
    path: "/api/forum/post/all",
    method: "GET",
  });
}

export async function createChat(options: {
  token: string;
  user: string;
}): Promise<IResponse> {
  return await makeRequest({
    path: "/api/forum/chats",
    method: "POST",
    token: options.token,
    body: {
      user: options.user,
    },
  });
}

export async function getChatList(options: {
  token: string;
}): Promise<IResponse> {
  return await makeRequest({
    path: "/api/forum/chats",
    method: "GET",
    token: options.token,
  });
}

export async function deleteChat(options: {
  token: string;
  id: string;
}): Promise<IResponse> {
  return await makeRequest({
    path: "/api/forum/chats",
    method: "DELETE",
    token: options.token,
    body: {
      chat: options.id,
    },
  });
}

export async function getChatMessages(options: {
  token: string;
  id: string;
}): Promise<IResponse> {
  return await makeRequest({
    path: "/api/forum/chats/" + options.id + "/messages",
    method: "GET",
    token: options.token,
  });
}
```

```
export async function sendMessage(options: {
  token: string;
  id: string;
  content: string;
}): Promise<IResponse> {
  return await makeRequest({
    path: "/api/forum/chats/" + options.id + "/messages",
    method: "POST",
    token: options.token,
    body: {
      content: options.content,
    },
  });
}

export async function getEmbeds(links: string[]): Promise<IResponse> {
  return await makeRequest({
    path: "/api/forum/embeds",
    method: "POST",
    body: {
      links: links,
    },
  });
}
```

## packages/rest/src/api/IDE.ts

```
/**
 * packages/rest/src/api/IDE.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 20.07.2022
 *
 */
import { IResponse } from "../types/IResponse";
import makeRequest from "../makeRequest";

export async function getProjectPermissions(options: {
  token: string;
  projectId: string;
}): Promise<IResponse> {
  return await makeRequest({
    path: "/api/ide/projects/" + options.projectId + "/permissions",
    token: options.token,
    method: "GET",
  });
}

export async function saveFile(options: {
  token: string;
  projectId: string;
  path: string;
  content: string;
}): Promise<IResponse> {
  return await makeRequest({
    path: "/api/ide/projects/" + options.projectId + "/file/save",
    method: "POST",
    token: options.token,
```

```
    body: {
      path: options.path,
      content: options.content,
    },
  });
}

export async function deleteFile(options: {
  token: string;
  projectId: string;
  path: string;
}): Promise<IResponse> {
  return await makeRequest({
    path: "/api/ide/projects/" + options.projectId + "/file/delete",
    method: "POST",
    token: options.token,
    body: {
      path: options.path,
    },
  });
}

export async function getProject(options: {
  projectId: string;
  token: string;
}): Promise<IResponse> {
  return await makeRequest({
    path: "/api/ide/projects/" + options.projectId,
    method: "GET",
    token: options.token,
  });
}

export async function addContributor(options: {
  projectId: string;
  email: string;
  token: string;
}): Promise<IResponse> {
  return await makeRequest({
    path: "/api/ide/projects/contributors/add",
    method: "POST",
    token: options.token,
    body: {
      project: options.projectId,
      contributor: options.email,
    },
  });
}

export async function removeContributor(options: {
  projectId: string;
  email: string;
  token: string;
}): Promise<IResponse> {
  return await makeRequest({
    path: "/api/ide/projects/contributors/remove",
    method: "POST",
    token: options.token,
    body: {
      project: options.projectId,
```

```
      contributor: options.email,
    },
  });
}

export async function getContributors(options: {
  token: string;
  projectId: string;
}): Promise<IResponse> {
  return await makeRequest({
    path: "/api/ide/projects/contributors/list",
    method: "POST",
    token: options.token,
    body: {
      project: options.projectId,
    },
  });
}

export async function overwriteContributorPermissions(options: {
  token: string;
  projectId: string;
  contributorEmail: string;
  permissions: [
    "READ",
    "WRITE" | null,
    "DELETE" | null,
    "DUPLICATE" | null,
    "RENAME" | null,
    "EXPORT" | null,
    "MANAGE-CONTRIBUTORS" | null
  ];
}): Promise<IResponse> {
  return await makeRequest({
    path: "/api/ide/projects/contributors/permissions",
    method: "POST",
    token: options.token,
    body: {
      project: options.projectId,
      contributor: options.contributorEmail,
      permissions: options.permissions,
    },
  });
}

export async function renameProject(options: {
  token: string;
  projectId: string;
  newName: string;
}): Promise<IResponse> {
  return await makeRequest({
    path: "/api/ide/projects/update/name",
    method: "POST",
    token: options.token,
    body: {
      id: options.projectId,
      name: options.newName,
    },
  });
}
```

```
export async function createProject(options: {
  token: string;
  name: string;
}): Promise<IResponse> {
  return await makeRequest({
    path: "/api/ide/projects/create",
    method: "POST",
    token: options.token,
    body: {
      name: options.name,
    },
  });
}

export async function deleteProject(options: {
  token: string;
  projectId: string;
}): Promise<IResponse> {
  return await makeRequest({
    path: "/api/ide/projects/delete",
    method: "POST",
    token: options.token,
    body: {
      id: options.projectId,
    },
  });
}

export async function duplicateProject(options: {
  token: string;
  projectId: string;
  name: string;
}): Promise<IResponse> {
  return await makeRequest({
    path: "/api/ide/projects/duplicate",
    method: "POST",
    token: options.token,
    body: {
      id: options.projectId,
      name: options.name,
    },
  });
}

export async function getProjects(options: {
  token: string;
  type: "OWNER" | "CONTRIBUTOR" | "ALL";
  minimal?: boolean;
  withStats?: boolean;
}): Promise<IResponse> {
  return await makeRequest({
    path: "/api/ide/projects/list",
    method: "POST",
    token: options.token,
    body: {
      type: options.type,
      minimal: options.minimal,
      withStats: options.withStats,
    },
```

```
  });
}

export async function publishTemplate(options: {
  token: string;
  projectId: string;
  displayName: string;
  description: string;
}): Promise<IResponse> {
  return await makeRequest({
    path: "/api/ide/projects/" + options.projectId + "/template/publish",
    method: "POST",
    token: options.token,
    body: {
      displayName: options.displayName,
      description: options.description,
    },
  });
}

export async function removeTemplate(options: {
  token: string;
  projectId: string;
}): Promise<IResponse> {
  return await makeRequest({
    path: "/api/ide/projects/" + options.projectId + "/template/remove",
    method: "POST",
    token: options.token,
  });
}

export async function updateTemplate(options: {
  token: string;
  projectId: string;
  update: {
    displayName?: string;
    description?: string;
  };
}): Promise<IResponse> {
  return await makeRequest({
    path: "/api/ide/projects/" + options.projectId + "/template/update",
    token: options.token,
    method: "POST",
    body: {
      update: options.update,
    },
  });
}

export async function getTemplate(templateId: string): Promise<IResponse> {
  return await makeRequest({
    path: "/api/ide/templates/" + templateId,
    method: "GET",
  });
}

export async function rateTemplate(options: {
  token: string;
  templateId: string;
  rating: number;
```

```
}): Promise<IResponse> {
  return await makeRequest({
    path: "/api/ide/templates/" + options.templateId + "/rating",
    method: "POST",
    token: options.token,
    body: {
      rating: options.rating,
    },
  });
}

export async function useTemplate(options: {
  token: string;
  templateId: string;
}): Promise<IResponse> {
  return await makeRequest({
    path: "/api/ide/templates/" + options.templateId + "/use",
    method: "POST",
    token: options.token,
  });
}

export async function hasTemplate(options: {
  token: string;
  projectId: string;
}): Promise<IResponse> {
  return await makeRequest({
    path: "/api/ide/projects/" + options.projectId + "/template/has",
    method: "GET",
    token: options.token,
  });
}

export async function getAllTemplates(): Promise<IResponse> {
  return await makeRequest({
    path: "/api/ide/templates/list",
    method: "GET",
  });
}

export async function toggleProjectPin(options: {
  token: string;
  projectId: string;
}): Promise<IResponse> {
  return await makeRequest({
    path: "/api/ide/projects/" + options.projectId + "/toggle-pin",
    token: options.token,
    method: "POST",
  });
}

export async function publishProject(options: {
  token: string;
  projectId: string;
  customName: string;
}): Promise<IResponse> {
  return await makeRequest({
    path: "/api/ide/projects/" + options.projectId + "/publish",
    token: options.token,
    method: "POST",
```

```
    body: {
      customName: options.customName,
      publishState: "public",
    },
  });
}

export async function unpublishProject(options: {
  token: string;
  projectId: string;
}): Promise<IResponse> {
  return await makeRequest({
    path: "/api/ide/projects/" + options.projectId + "/publish",
    token: options.token,
    method: "POST",
    body: {
      publishState: "private",
    },
  });
}

export async function enableDiscovery(options: {
  token: string;
  projectId: string;
  description: string;
}): Promise<IResponse> {
  return await makeRequest({
    path: "/api/ide/projects/" + options.projectId + "/discovery",
    token: options.token,
    method: "POST",
    body: {
      enableDiscovery: true,
      discoveryDescription: options.description,
    },
  });
}

export async function disableDiscovery(options: {
  token: string;
  projectId: string;
}): Promise<IResponse> {
  return await makeRequest({
    path: "/api/ide/projects/" + options.projectId + "/discovery",
    token: options.token,
    method: "POST",
    body: {
      disableDiscovery: true,
    },
  });
}
```

## packages/rest/src/api/Kids.ts

```
/**
 * packages/rest/src/api/Kids.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 02.04.2023
 *
```

```
 */

import { IResponse } from "../types/IResponse";
import makeRequest from "../makeRequest";

export async function getProjects(token: string): Promise<IResponse> {
  return await makeRequest({
    method: "GET",
    path: "/api/kids/projects",
    token,
  });
}

export async function createProject(options: {
  token: string;
  name: string;
  code?: string;
}): Promise<IResponse> {
  return await makeRequest({
    method: "POST",
    path: "/api/kids/projects",
    token: options.token,
    body: {
      name: options.name,
      code: options.code,
    },
  });
}

export async function deleteProject(options: {
  token: string;
  id: string;
}): Promise<IResponse> {
  return await makeRequest({
    method: "DELETE",
    path: `/api/kids/projects`,
    token: options.token,
    body: {
      id: options.id,
    },
  });
}

export async function updateProject(options: {
  token: string;
  id: string;
  name?: string;
  code?: string;
}): Promise<IResponse> {
  return await makeRequest({
    method: "PUT",
    path: `/api/kids/projects`,
    token: options.token,
    body: {
      id: options.id,
      update: {
        name: options.name,
        code: options.code,
      },
    },
```

```
  });
}

export async function getProject(options: {
  token: string;
  id: string;
}): Promise<IResponse> {
  return await makeRequest({
    method: "GET",
    path: `/api/kids/projects?id=${options.id}`,
    token: options.token,
  });
}
```

## packages/rest/src/api/Landingpage.ts

```
/**
 * packages/rest/src/api/Landingpage.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 12.04.2023
 *
 */

import { IResponse } from "../types/IResponse";
import makeRequest from "../makeRequest";

export async function getTestimonials(): Promise<IResponse> {
  return await makeRequest({
    path: "/api/testimonials",
    method: "GET",
  });
}
```

## packages/rest/src/api/Newsletter.ts

```
/**
 * packages/rest/src/api/Newsletter.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 13.04.2023
 *
 */

import { IResponse } from "../types/IResponse";
import makeRequest from "../makeRequest";

export function subscribe(token: string): Promise<IResponse> {
  return makeRequest({
    method: "GET",
    path: "/api/newsletter/subscribe",
    token,
  });
}

export function unsubscribe(token: string): Promise<IResponse> {
  return makeRequest({
    method: "GET",
```

```
    path: "/api/newsletter/unsubscribe",
    token,
  });
}

export function subscribed(token: string): Promise<IResponse> {
  return makeRequest({
    method: "GET",
    path: "/api/newsletter/subscribed",
    token,
  });
}
```

## packages/rest/src/api/Orgs.ts

```
/**
 * packages/rest/src/api/Orgs.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 04.03.23
 *
 */
import makeRequest from "../makeRequest";
import { IResponse } from "../types/IResponse";

export async function createOrg(options: {
  token: string;
  name: string;
  icon?: string;
}): Promise<IResponse> {
  return await makeRequest({
    token: options.token,
    path: "/api/orgs/create",
    method: "POST",
    body: {
      name: options.name,
      icon: options.icon,
    },
  });
}

export async function getOrgs(token: string): Promise<IResponse> {
  return await makeRequest({
    token: token,
    path: "/api/orgs/my",
    method: "GET",
  });
}

export async function deleteOrg(options: {
  token: string;
  name: string;
}): Promise<IResponse> {
  return await makeRequest({
    token: options.token,
    path: "/api/orgs/delete",
    method: "POST",
    body: {
      name: options.name,
```

```
    },
  });
}

export async function getOrg(options: {
  token: string;
  name: string;
}): Promise<IResponse> {
  return await makeRequest({
    token: options.token,
    path: `/api/orgs/${options.name}`,
    method: "GET",
  });
}

export async function renameOrg(options: {
  token: string;
  oldName: string;
  newName: string;
}): Promise<IResponse> {
  return await makeRequest({
    token: options.token,
    path: "/api/orgs/" + options.oldName + "/update",
    method: "POST",
    body: {
      update: {
        name: options.newName,
      },
    },
  });
}

export async function updateOrgIcon(options: {
  token: string;
  name: string;
  icon: string;
}): Promise<IResponse> {
  return await makeRequest({
    token: options.token,
    path: "/api/orgs/" + options.name + "/update",
    method: "POST",
    body: {
      update: {
        icon: options.icon,
      },
    },
  });
}

export async function updateOrg(options: {
  token: string;
  name: string;
  update: any;
}): Promise<IResponse> {
  return await makeRequest({
    token: options.token,
    path: "/api/orgs/" + options.name + "/update",
    method: "POST",
    body: {
      update: options.update,
```

```
    },
  });
}

export async function addOrgMember(options: {
  token: string;
  name: string;
  userId: string;
}): Promise<IResponse> {
  return await makeRequest({
    token: options.token,
    path: "/api/orgs/" + options.name + "/members",
    method: "POST",
    body: {
      action: "ADD",
      userId: options.userId,
    },
  });
}

export async function removeOrgMember(options: {
  token: string;
  name: string;
  userId: string;
}): Promise<IResponse> {
  return await makeRequest({
    token: options.token,
    path: "/api/orgs/" + options.name + "/members",
    method: "POST",
    body: {
      action: "REMOVE",
      userId: options.userId,
    },
  });
}

export async function addOrgTemplate(options: {
  token: string;
  projectId: string;
  org: string;
}): Promise<IResponse> {
  return await makeRequest({
    token: options.token,
    path: "/api/orgs/" + options.org + "/templates",
    method: "POST",
    body: {
      action: "ADD",
      projectId: options.projectId,
    },
  });
}

export async function removeOrgTemplate(options: {
  token: string;
  projectId: string;
  org: string;
}): Promise<IResponse> {
  return await makeRequest({
    token: options.token,
    path: "/api/orgs/" + options.org + "/templates",
```

```
    method: "POST",
    body: {
      action: "REMOVE",
      projectId: options.projectId,
    },
  });
}

export async function getOrgTemplates(options: {
  token: string;
  org: string;
}): Promise<IResponse> {
  return await makeRequest({
    token: options.token,
    path: "/api/orgs/" + options.org + "/templates",
    method: "GET",
  });
}

export async function useOrgTemplate(options: {
  token: string;
  org: string;
  projectId: string;
  name: string;
}): Promise<IResponse> {
  return await makeRequest({
    token: options.token,
    path:
      "/api/orgs/" +
      options.org +
      "/use-template?templateId=" +
      options.projectId +
      "&newProjectName=" +
      options.name,
    method: "GET",
  });
}

export async function createBlog(options: {
  token: string;
  org: string;
  title: string;
  content: string;
}): Promise<IResponse> {
  return await makeRequest({
    token: options.token,
    path: "/api/orgs/" + options.org + "/blog",
    method: "POST",
    body: {
      title: options.title,
      content: options.content,
    },
  });
}

export async function getBlogs(options: {
  token: string;
  org: string;
}): Promise<IResponse> {
  return await makeRequest({
```

```
    token: options.token,
    path: "/api/orgs/" + options.org + "/blog",
    method: "GET",
  });
}
```

## packages/rest/src/api/Payments.ts

```
/**
 * packages/rest/src/api/Payments.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 16.06.2023
 *
 */
import { IResponse } from "../types/IResponse";
import makeRequest from "../makeRequest";

export async function createCheckoutLink(jwt: string): Promise<IResponse> {
  return await makeRequest({
    method: "POST",
    path: "/api/payments/createCheckoutSession",
    token: jwt,
  });
}

export async function cancelSubscription(jwt: string): Promise<IResponse> {
  return await makeRequest({
    method: "POST",
    path: "/api/payments/cancelSubscription",
    token: jwt,
  });
}
```

## packages/rest/src/api/ProjectIdeas.ts

```
/**
 * packages/rest/src/api/ProjectIdeas.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 03.12.2022
 *
 */
import { IResponse } from "../types/IResponse";
import makeRequest from "../makeRequest";

export async function getProjectIdeas(): Promise<IResponse> {
  return await makeRequest({
    path: "/api/project-ideas",
    method: "GET",
  });
}
```

## packages/rest/src/api/Projects.ts

```
/**
 * packages/rest/src/api/Projects.ts
 *
```

```
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2024 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 21.05.2024
 *
 */

import makeRequest from "../makeRequest";
import { IResponse } from "../types/IResponse";

export async function createProject(options: {
  token: string;
  name: string;
  projectType: string;
  modifiers: string[];
}): Promise<IResponse> {
  return await makeRequest({
    method: "POST",
    path: "/api/projects/create",
    token: options.token,
    body: {
      name: options.name,
      projectType: options.projectType,
      modifiers: options.modifiers,
    },
  });
}

export async function myProjects(token: string): Promise<IResponse> {
  return await makeRequest({
    method: "GET",
    path: "/api/projects/my",
    token,
  });
}

export async function project(token: string, id: string): Promise<IResponse> {
  return await makeRequest({
    method: "GET",
    path: `/api/projects/${id}`,
    token,
  });
}

export async function files(token: string, id: string): Promise<IResponse> {
  return await makeRequest({
    method: "GET",
    path: `/api/projects/${id}/static/files`,
    token,
  });
}

export async function deleteFile(
  token: string,
  id: string,
  fileName: string
): Promise<IResponse> {
  return await makeRequest({
    method: "DELETE",
    path: `/api/projects/${id}/static/files`,
```

```
    token,
    body: {
      file: fileName,
    },
  });
}

export async function uploadFile(
  token: string,
  id: string,
  name: string,
  content: string
): Promise<IResponse> {
  return await makeRequest({
    method: "POST",
    path: `/api/projects/${id}/static/upload`,
    token,
    body: {
      name,
      content,
    },
  });
}

export async function wikiPages(
  token: string,
  id: string,
  page_id?: string
): Promise<IResponse> {
  return await makeRequest({
    method: "GET",
    path:
      `/api/projects/${id}/wiki/pages` + (page_id ? `?pageId=${page_id}` : ""),
    token,
  });
}

export async function createWikiPage(
  token: string,
  id: string,
  title: string,
  content: string
): Promise<IResponse> {
  return await makeRequest({
    method: "POST",
    path: `/api/projects/${id}/wiki/pages`,
    token,
    body: {
      title,
      content,
    },
  });
}

export async function updateWikiPage(
  token: string,
  id: string,
  page_id: string,
  title: string,
  content: string
```

```
): Promise<IResponse> {
  return await makeRequest({
    method: "POST",
    path: `/api/projects/${id}/wiki/pages`,
    token,
    body: {
      title,
      content,
      page_id,
    },
  });
}

export async function deleteWikiPage(
  token: string,
  id: string,
  page_id: string
): Promise<IResponse> {
  return await makeRequest({
    method: "DELETE",
    path: `/api/projects/${id}/wiki/pages`,
    token,
    body: {
      page_id,
    },
  });
}

export async function getIssues(
  token: string,
  id: string,
  issueId?: string
): Promise<IResponse> {
  return await makeRequest({
    method: "GET",
    path: `/api/projects/${id}/issues` + (issueId ? `?issueId=${issueId}` : ""),
    token,
  });
}

export async function createIssue(
  token: string,
  id: string,
  title: string,
  description: string,
  priority: string,
  status: string
): Promise<IResponse> {
  return await makeRequest({
    method: "POST",
    path: `/api/projects/${id}/issues`,
    token,
    body: {
      title,
      description,
      priority,
      status,
    },
  });
}
```

```
export async function updateIssue(
  token: string,
  id: string,
  issueId: string,
  title: string,
  description: string,
  priority: string,
  status: string
): Promise<IResponse> {
  return await makeRequest({
    method: "PUT",
    path: `/api/projects/${id}/issues?issueId=${issueId}`,
    token,
    body: {
      title,
      description,
      priority,
      status,
    },
  });
}

export async function deleteIssue(
  token: string,
  id: string,
  issueId: string
): Promise<IResponse> {
  return await makeRequest({
    method: "DELETE",
    path: `/api/projects/${id}/issues?issueId=${issueId}`,
    token,
  });
}

export async function getFlows(
  token: string,
  id: string,
  flowId?: string
): Promise<IResponse> {
  return await makeRequest({
    method: "GET",
    path: `/api/projects/${id}/flows` + (flowId ? `?flowId=${flowId}` : ""),
    token,
  });
}

export async function createFlow(
  token: string,
  id: string,
  name: string
): Promise<IResponse> {
  return await makeRequest({
    method: "POST",
    path: `/api/projects/${id}/flows`,
    token,
    body: {
      name,
    },
  });
```

```
}

export async function updateFlow(
  token: string,
  id: string,
  flowId: string,
  name: string,
  saved: string
): Promise<IResponse> {
  return await makeRequest({
    method: "PUT",
    path: `/api/projects/${id}/flows?flowId=${flowId}`,
    token,
    body: {
      name,
      saved,
    },
  });
}

export async function deleteFlow(
  token: string,
  id: string,
  flowId: string
): Promise<IResponse> {
  return await makeRequest({
    method: "DELETE",
    path: `/api/projects/${id}/flows?flowId=${flowId}`,
    token,
  });
}

export async function getI18nData(
  token: string,
  id: string
): Promise<IResponse> {
  return await makeRequest({
    method: "GET",
    path: `/api/projects/${id}/i18n`,
    token,
  });
}

export async function i18nAddLanguage(
  token: string,
  id: string,
  language: string
): Promise<IResponse> {
  return await makeRequest({
    method: "POST",
    path: `/api/projects/${id}/i18n`,
    token,
    body: {
      action: "addLanguage",
      language,
    },
  });
}

export async function i18nRemoveLanguage(
```

```
  token: string,
  id: string,
  language: string
): Promise<IResponse> {
  return await makeRequest({
    method: "POST",
    path: `/api/projects/${id}/i18n`,
    token,
    body: {
      action: "removeLanguage",
      language,
    },
  });
}

export async function i18nSetDefaultLanguage(
  token: string,
  id: string,
  language: string
): Promise<IResponse> {
  return await makeRequest({
    method: "POST",
    path: `/api/projects/${id}/i18n`,
    token,
    body: {
      action: "setDefaultLanguage",
      language,
    },
  });
}

export async function i18nSetKey(
  token: string,
  id: string,
  language: string,
  key: string,
  value: string
): Promise<IResponse> {
  return await makeRequest({
    method: "POST",
    path: `/api/projects/${id}/i18n`,
    token,
    body: {
      action: "setKey",
      key,
      value,
      language,
    },
  });
}

export async function i18nDeleteKey(
  token: string,
  id: string,
  language: string,
  key: string
): Promise<IResponse> {
  return await makeRequest({
    method: "POST",
    path: `/api/projects/${id}/i18n`,
```

```
    token,
    body: {
      action: "deleteKey",
      key,
      language,
    },
  });
}

export async function umlDiagrams(
  token: string,
  id: string,
  umlId?: string
): Promise<IResponse> {
  return await makeRequest({
    method: "GET",
    path: `/api/projects/${id}/uml` + (umlId ? `?umlId=${umlId}` : ""),
    token,
  });
}

export async function createUmlDiagram(
  token: string,
  id: string,
  name: string,
  data: string,
  type: string
): Promise<IResponse> {
  return await makeRequest({
    method: "POST",
    path: `/api/projects/${id}/uml`,
    token,
    body: {
      name,
      data,
      type,
    },
  });
}

export async function updateUmlDiagram(
  token: string,
  id: string,
  umlId: string,
  data: string
): Promise<IResponse> {
  return await makeRequest({
    method: "POST",
    path: `/api/projects/${id}/uml?umlId=${umlId}`,
    token,
    body: {
      data,
    },
  });
}

export async function deleteUmlDiagram(
  token: string,
  id: string,
  umlId: string
```

```
): Promise<IResponse> {
  return await makeRequest({
    method: "DELETE",
    path: `/api/projects/${id}/uml?umlId=${umlId}`,
    token,
  });
}

export const GitOps = {
  tree: async (
    token: string,
    id: string,
    withContent?: string
  ): Promise<IResponse> => {
    return await makeRequest({
      method: "GET",
      path: `/api/projects/${id}/gitops/tree${
        withContent ? `?withContent=${withContent}` : ""
      }`,
      token,
    });
  },
  branch: async (token: string, id: string): Promise<IResponse> => {
    return await makeRequest({
      method: "GET",
      path: `/api/projects/${id}/gitops/branch`,
      token,
    });
  },
  setBranch: async (
    token: string,
    id: string,
    branch: string
  ): Promise<IResponse> => {
    return await makeRequest({
      method: "POST",
      path: `/api/projects/${id}/gitops/branch`,
      token,
      body: {
        branch,
      },
    });
  },
  commit: async (
    token: string,
    id: string,
    message: string
  ): Promise<IResponse> => {
    return await makeRequest({
      method: "POST",
      path: `/api/projects/${id}/gitops/commit`,
      token,
      body: {
        message,
      },
    });
  },
  commits: async (token: string, id: string): Promise<IResponse> => {
    return await makeRequest({
      method: "GET",
```

```
      path: `/api/projects/${id}/gitops/commits`,
      token,
    });
  },
};

export const Files = {
  get: async (token: string, id: string, path: string): Promise<IResponse> => {
    return await makeRequest({
      method: "GET",
      path: `/api/projects/${id}/files/${path}`,
      token,
    });
  },
  setFileContent: async (
    token: string,
    id: string,
    path: string,
    content: string
  ): Promise<IResponse> => {
    return await makeRequest({
      method: "POST",
      path: `/api/projects/${id}/files/${path}`,
      token,
      body: {
        content,
      },
    });
  },
  delete: async (
    token: string,
    id: string,
    path: string
  ): Promise<IResponse> => {
    return await makeRequest({
      method: "DELETE",
      path: `/api/projects/${id}/files/${path}`,
      token,
    });
  },
  createFolder: async (
    token: string,
    id: string,
    path: string
  ): Promise<IResponse> => {
    return await makeRequest({
      method: "POST",
      path: `/api/projects/${id}/files/${path}`,
      token,
      body: {
        folder: true,
      },
    });
  },
  rename: async (
    token: string,
    id: string,
    oldPath: string,
    newPath: string
  ): Promise<IResponse> => {
```

```
    return await makeRequest({
      method: "PUT",
      path: `/api/projects/${id}/files/${oldPath}`,
      token,
      body: {
        newName: newPath,
      },
    });
  },
};
```

## packages/rest/src/api/RCE.ts

```
/**
 * packages/rest/src/api/RCE.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2024 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 18.05.2024
 *
 */

import { IResponse } from "../types/IResponse";
import makeRequest from "../makeRequest";

export async function rceJavaScript(
  code: string,
  token: string
): Promise<IResponse> {
  return await makeRequest({
    path: "/api/rce/javascript",
    method: "POST",
    token: token,
    body: {
      code: code,
    },
    returnsRaw: true,
  });
}

export async function rceTypeScript(
  code: string,
  token: string
): Promise<IResponse> {
  return await makeRequest({
    path: "/api/rce/typescript",
    method: "POST",
    token: token,
    body: {
      code: code,
    },
    returnsRaw: true,
  });
}

export async function rcePython(
  code: string,
  token: string
): Promise<IResponse> {
  return await makeRequest({
```

```
    path: "/api/rce/python",
    method: "POST",
    token: token,
    body: {
      code: code,
    },
    returnsRaw: true,
  });
}

export async function rceJava(code: string, token: string): Promise<IResponse> {
  return await makeRequest({
    path: "/api/rce/java",
    method: "POST",
    token: token,
    body: {
      code: code,
    },
    returnsRaw: true,
  });
}

export async function rceGo(code: string, token: string): Promise<IResponse> {
  return await makeRequest({
    path: "/api/rce/go",
    method: "POST",
    token: token,
    body: {
      code: code,
    },
    returnsRaw: true,
  });
}
```

## packages/rest/src/api/Roadmap.ts

```
/**
 * packages/rest/src/api/Roadmap.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 07.04.2023
 *
 */

import { IResponse } from "../types/IResponse";
import makeRequest from "../makeRequest";

export async function getMyRoads(token: string): Promise<IResponse> {
  return await makeRequest({
    method: "GET",
    path: "/api/roadmap/my",
    token: token,
  });
}

export async function createRoad(options: {
  token: string;
  name: string;
}): Promise<IResponse> {
```

```
  return await makeRequest({
    method: "POST",
    path: "/api/roadmap/create",
    token: options.token,
    body: {
      name: options.name,
    },
  });
}

export async function deleteRoad(options: {
  token: string;
  id: string;
}): Promise<IResponse> {
  return await makeRequest({
    method: "DELETE",
    path: `/api/roadmap/${options.id}/delete`,
    token: options.token,
  });
}

export async function updateRoad(options: {
  token: string;
  id: string;
  update: {
    projectIdea?: string;
    linkedProject?: string;
    linkedCourses?: string[];
    linkedTodos?: string[];
    linkedFlows?: string[];
    name?: string;
  };
}): Promise<IResponse> {
  return await makeRequest({
    method: "POST",
    path: `/api/roadmap/${options.id}/update`,
    token: options.token,
    body: {
      update: options.update,
    },
  });
}

export async function getRoad(options: {
  token: string;
  id: string;
}): Promise<IResponse> {
  return await makeRequest({
    method: "GET",
    path: `/api/roadmap/${options.id}`,
    token: options.token,
  });
}
```

## packages/rest/src/api/Snippets.ts

```
/**
 * packages/rest/src/api/Snippets.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
```

```
 * License: Project License
 * Created At: 30.12.2022
 *
 */

import { IResponse } from "../types/IResponse";
import makeRequest from "../makeRequest";

export async function getSnippets(token: string): Promise<IResponse> {
  return await makeRequest({
    method: "GET",
    path: "/api/snippets",
    token,
  });
}

export async function getSnippet(options: {
  token: string;
  id: string;
}): Promise<IResponse> {
  return await makeRequest({
    method: "GET",
    path: "/api/snippets?id=" + options.id,
    token: options.token,
  });
}

export async function createSnippet(options: {
  token: string;
  title: string;
  code: string;
  language: string;
}): Promise<IResponse> {
  return await makeRequest({
    method: "POST",
    path: "/api/snippets",
    token: options.token,
    body: {
      title: options.title,
      code: options.code,
      language: options.language,
    },
  });
}

export async function deleteSnippet(options: {
  token: string;
  id: string;
}): Promise<IResponse> {
  return await makeRequest({
    method: "DELETE",
    path: "/api/snippets?id=" + options.id,
    token: options.token,
  });
}

export async function updateSnippet(options: {
  token: string;
  id: string;
  title: string;
```

```
  code: string;
  language: string;
}): Promise<IResponse> {
  return await makeRequest({
    method: "PUT",
    path: "/api/snippets?id=" + options.id,
    token: options.token,
    body: {
      title: options.title,
      code: options.code,
      language: options.language,
    },
  });
}
```

## packages/rest/src/api/ToDo.ts

```
/**
 * packages/rest/src/api/ToDo.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 20.07.2022
 *
 */
import { IResponse } from "../types/IResponse";
import makeRequest from "../makeRequest";

export async function getMyLists(token: string): Promise<IResponse> {
  return await makeRequest({
    path: "/api/todo/my-lists",
    token: token,
    method: "GET",
  });
}

export async function createList(options: {
  token: string;
  name: string;
}): Promise<IResponse> {
  return await makeRequest({
    path: "/api/todo/lists/create",
    method: "POST",
    token: options.token,
    body: {
      name: options.name,
    },
  });
}

export async function deleteList(options: {
  token: string;
  listId: string;
}): Promise<IResponse> {
  return await makeRequest({
    path: "/api/todo/lists/delete",
    method: "POST",
    token: options.token,
    body: {
      list: options.listId,
```

```
    },
  });
}

export async function getList(options: {
  token: string;
  listId: string;
}): Promise<IResponse> {
  return await makeRequest({
    path: "/api/todo/lists/" + options.listId,
    token: options.token,
    method: "GET",
  });
}

export async function createTask(options: {
  token: string;
  listId: string;
  task: string;
}): Promise<IResponse> {
  return await makeRequest({
    path: "/api/todo/lists/" + options.listId + "/tasks/create",
    token: options.token,
    method: "POST",
    body: {
      task: options.task,
    },
  });
}

export async function deleteTask(options: {
  token: string;
  listId: string;
  taskUUID: string;
}): Promise<IResponse> {
  return await makeRequest({
    path: "/api/todo/lists/" + options.listId + "/tasks/delete",
    token: options.token,
    method: "POST",
    body: {
      task: options.taskUUID,
    },
  });
}

export async function toggleItemFinish(options: {
  token: string;
  listId: string;
  taskUUID: string;
}): Promise<IResponse> {
  return await makeRequest({
    path: "/api/todo/lists/" + options.listId + "/tasks/toggle-finish",
    token: options.token,
    method: "POST",
    body: {
      task: options.taskUUID,
    },
  });
}
```

```
export async function updateItem(options: {
  token: string;
  listId: string;
  taskUUID: string;
  update: { [key: string]: any };
}): Promise<IResponse> {
  return await makeRequest({
    path: "/api/todo/lists/" + options.listId + "/tasks/update",
    token: options.token,
    method: "POST",
    body: {
      task: options.taskUUID,
      update: options.update,
    },
  });
}

export async function getV2Projects(token: string): Promise<IResponse> {
  return await makeRequest({
    path: "/api/todo/projects",
    token: token,
    method: "GET",
  });
}

export async function getV2Project(options: {
  token: string;
  id: string;
}): Promise<IResponse> {
  return await makeRequest({
    path: "/api/todo/projects?id=" + options.id,
    method: "GET",
    token: options.token,
  });
}

export async function deleteV2Project(options: {
  token: string;
  id: string;
}): Promise<IResponse> {
  return await makeRequest({
    path: "/api/todo/projects?id=" + options.id,
    method: "DELETE",
    token: options.token,
  });
}

export async function createV2Project(options: {
  token: string;
  name: string;
}): Promise<IResponse> {
  return await makeRequest({
    path: "/api/todo/projects?name=" + options.name,
    method: "POST",
    token: options.token,
  });
}

export async function updateV2Project(options: {
  token: string;
```

```
  id: string;
  saved: any;
}): Promise<IResponse> {
  return await makeRequest({
    path: "/api/todo/projects?id=" + options.id,
    method: "POST",
    token: options.token,
    body: {
      saved: options.saved,
    },
  });
}
```

## packages/rest/src/index.ts

```
/**
 * packages/rest/src/index.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 20.07.2022
 *
 */
import {
  checkResetToken,
  checkUsername,
  createTestAccount,
  deleteAccount,
  getUserById,
  getUserByName,
  login,
  loginWithUsername,
  register,
  reset,
  resetConfirm,
  update,
  update2FA,
  verify,
} from "./api/Account";
import makeRequest from "./makeRequest";
import {
  createList,
  createTask,
  createV2Project,
  deleteList,
  deleteTask,
  deleteV2Project,
  getList,
  getMyLists,
  getV2Project,
  getV2Projects,
  toggleItemFinish,
  updateItem,
  updateV2Project,
} from "./api/ToDo";
import {
  addContributor,
  createProject,
  deleteFile,
  deleteProject,
```

```
  disableDiscovery,
  duplicateProject,
  enableDiscovery,
  getAllTemplates,
  getContributors,
  getProject,
  getProjectPermissions,
  getProjects,
  hasTemplate,
  overwriteContributorPermissions,
  publishProject,
  publishTemplate,
  rateTemplate,
  removeContributor,
  removeTemplate,
  renameProject,
  saveFile,
  toggleProjectPin,
  unpublishProject,
  updateTemplate,
  useTemplate,
} from "./api/IDE";
import {
  createPost,
  createProfile,
  deletePost,
  existsProfile,
  getFeed,
  getPost,
  getProfile,
  getProfileFollowers,
  getProfilePosts,
  isFollowing,
  likePost,
  search,
  toggleFollow,
  updateProfile,
} from "./api/Community";
import {
  createChat as createChatV2,
  createPost as createPostV2,
  createProfile as createProfileV2,
  deleteChat as deleteChatV2,
  deletePost as deletePostV2,
  deleteProfile as deleteProfileV2,
  getAllPosts as getAllPostsV2,
  getChatList as getChatListV2,
  getChatMessages as getChatMessagesV2,
  getComments as getCommentsV2,
  getEmbeds,
  getPost as getPostV2,
  getProfile as getProfileV2,
  getProfilePosts as getProfilePostsV2,
  getProfiles as getProfilesV2,
  likePost as likePostV2,
  modifyProfile as modifyProfileV2,
  sendMessage as sendMessageV2,
} from "./api/Forum";
import {
  contactUser,
```

createAccount,
createBlog,
createContent,
createCourse,
createProjectIdea,
createTestimonial,
createText,
createVideo,
deleteAccount as deleteAccountA,
deleteBlog,
deleteContactMessage,
deleteContent,
deleteCourse,
deleteForumProfile,
deleteObtainedCertificate,
deleteProjectIdea,
deleteTestimonial,
deleteText,
deleteVideo,
getAllObtainedCertificates,
getBlogs,
getContactMessageCount,
getContactMessages,
getContents,
getOverview,
getProjectIdeas as getProjectIdeasAdmin,
getText,
getUserAnalytics,
getUserJWT,
getVideo,
listAccounts,
listCourses,
listForumProfiles,
listTexts,
listVideos,
updateAccount,
updateBlog,
updateCourse,
updateForumProfile,
updateTestimonial,
updateText,
updateVideo,
getOrgs as getOrgsAdmin,
getNewsletterMembers,
sendNewsletter,
removeFromDiscovery,
getFlows,
getKidsProjects as getKidsProjectsAdmin,
getRoads,
getTodos,
getSnippets as getSnippetsAdmin,
getShorts,
createShort,
deleteShort,
getIDEProjects,
getEditorProjects,
getErrors,
createChallenge,
deleteChallenge,
updateChallenge,

```
} from "./api/Admin";

import {
  changeEnrollment,
  getCourse,
  getCourseProgress,
  getEnrollment,
  getEnrollments,
  getSection,
  listCourses as listCoursesU,
  updateCourseProgress,
} from "./api/Course";

import {
  createCertificate,
  createContent as createContentCreator,
  createCourse as createCourseCreator,
  deleteContent as deleteContentCreator,
  deleteCourse as deleteCourseCreator,
  getCertificate,
  getContents as getContentsCreator,
  getCourses as getCoursesCreator,
  getTopCreators,
  updateCertificate,
  updateContent as updateContentCreator,
  updateCourse as updateCourseCreator,
} from "./api/Creator";
import {
  deleteAnalytics,
  getSummary,
  getViews,
  postAnalytics,
} from "./api/Analytics";
import {
  getCertificate as getCertificateC,
  getObtainedCertificate,
  getObtainedCertificates,
  submitCertificate,
} from "./api/Certificates";
import { getProjectIdeas } from "./api/ProjectIdeas";
import {
  commentProject,
  getDiscoveryProjects,
  starProject,
} from "./api/Discovery";
import { getBlog, getBlogs as getBlogsB } from "./api/Blog";
import {
  createSnippet,
  deleteSnippet,
  getSnippet,
  getSnippets,
  updateSnippet,
} from "./api/Snippets";
import {
  addOrgMember,
  addOrgTemplate,
  createOrg,
  deleteOrg,
  getOrg,
  getOrgs,
```

```
  getOrgTemplates,
  removeOrgMember,
  removeOrgTemplate,
  renameOrg,
  updateOrg,
  updateOrgIcon,
  useOrgTemplate,
  getBlogs as getBlogsOrg,
  createBlog as createBlogOrg,
} from "./api/Orgs";
import { getDomainById, getProjects as getProjectsV2 } from "./api/Editor";
import {
  getProjects as getKidsProjects,
  deleteProject as deleteKidsProject,
  createProject as createKidsProject,
  updateProject as updateKidsProject,
  getProject as getKidsProject,
} from "./api/Kids";
import {
  createRoad,
  deleteRoad,
  getMyRoads,
  getRoad,
  updateRoad,
} from "./api/Roadmap";
import { getTestimonials } from "./api/Landingpage";
import { subscribe, subscribed, unsubscribe } from "./api/Newsletter";
import { cancelSubscription, createCheckoutLink } from "./api/Payments";
import {
  deletePost as deletePostA,
  createPost as createPostA,
  getPost as getPostA,
  getPostsByParent,
  searchPosts,
  getPosts,
  getProfile as getProfileA,
  getPostsByUser,
  updateProfile as updateProfileA,
} from "./api/Ask";
import {
  rceGo,
  rceJava,
  rceJavaScript,
  rcePython,
  rceTypeScript,
} from "./api/RCE";
import {
  finalCheck,
  getChallenges,
  getCompletedChallenges,
} from "./api/CodeChallenges";
import {
  myProjects,
  createProject as createProjectP,
  project,
  files,
  deleteFile as deleteFileP,
  uploadFile as uploadFileP,
  uploadFile,
  wikiPages,
```

```
  createWikiPage,
  updateWikiPage,
  deleteWikiPage,
  deleteIssue,
  updateIssue,
  createIssue,
  getIssues,
  deleteFlow,
  updateFlow,
  createFlow,
  getFlows as getFlowsP,
  getI18nData,
  i18nAddLanguage,
  i18nRemoveLanguage,
  i18nSetDefaultLanguage,
  i18nSetKey,
  i18nDeleteKey,
  deleteUmlDiagram,
  updateUmlDiagram,
  createUmlDiagram,
  umlDiagrams,
  GitOps,
  Files,
} from "./api/Projects";

const REST = {
  Ask: {
    deletePost: deletePostA,
    createPost: createPostA,
    getPost: getPostA,
    getPostsByParent,
    searchPosts,
    getPosts,
    getProfile: getProfileA,
    getPostsByUser,
    updateProfile: updateProfileA,
  },
  Challenges: {
    getChallenges,
    getCompletedChallenges,
    finalCheck,
  },
  Admin: {
    getErrors,
    listAccounts,
    updateAccount,
    deleteAccount: deleteAccountA,
    createAccount,
    listForumProfiles,
    deleteForumProfile,
    updateForumProfile,
    createCourse,
    listCourses,
    deleteCourse,
    updateCourse,
    createVideo,
    listVideos,
    deleteVideo,
    getVideo,
    updateVideo,
```

```
  createText,
  listTexts,
  deleteText,
  getText,
  updateText,
  createContent,
  getContents,
  deleteContent,
  getContactMessages,
  getContactMessageCount,
  deleteContactMessage,
  contactUser,
  getAllObtainedCertificates,
  deleteObtainedCertificate,
  deleteProjectIdea,
  createProjectIdea,
  getProjectIdeasAdmin,
  getBlogs,
  createBlog,
  updateBlog,
  deleteBlog,
  getUserJWT,
  getOverview,
  getUserAnalytics,
  getTestimonials,
  createTestimonial,
  deleteTestimonial,
  updateTestimonial,
  getOrgs: getOrgsAdmin,
  getNewsletterMembers,
  sendNewsletter,
  removeFromDiscovery,
  getFlows,
  getKidsProjects: getKidsProjectsAdmin,
  getRoads,
  getTodos,
  getSnippets: getSnippetsAdmin,
  getShorts,
  createShort,
  deleteShort,
  getIDEProjects,
  getEditorProjects,
  createChallenge,
  deleteChallenge,
  updateChallenge,
},
Blog: {
  getBlog,
  getBlogsB,
},
Landingpage: {
  getTestimonials,
},
Account: {
  register,
  login,
  loginWithUsername,
  checkUsername,
  checkResetToken,
  deleteAccount,
```

```
  reset,
  resetConfirm,
  update,
  update2FA,
  verify,
  createTestAccount,
  getUserById,
  getUserByName,
},
ToDo: {
  getMyLists,
  createList,
  deleteList,
  getList,
  createTask,
  deleteTask,
  toggleItemFinish,
  updateItem,
  createV2Project,
  getV2Project,
  getV2Projects,
  deleteV2Project,
  updateV2Project,
},
IDE: {
  getProjectPermissions,
  saveFile,
  deleteFile,
  getProject,
  addContributor,
  removeContributor,
  getContributors,
  overwriteContributorPermissions,
  renameProject,
  createProject,
  deleteProject,
  duplicateProject,
  getProjects,
  publishTemplate,
  removeTemplate,
  updateTemplate,
  rateTemplate,
  useTemplate,
  hasTemplate,
  getAllTemplates,
  toggleProjectPin,
  publishProject,
  unpublishProject,
  enableDiscovery,
  disableDiscovery,
},
Projects: {
  myProjects,
  createProject: createProjectP,
  project,
  files,
  deleteFile: deleteFileP,
  uploadFile: uploadFileP,
  wikiPages,
  createWikiPage,
```

```
  updateWikiPage,
  deleteWikiPage,
  deleteIssue,
  updateIssue,
  createIssue,
  getIssues,
  deleteFlow,
  updateFlow,
  createFlow,
  getFlows: getFlowsP,
  getI18nData,
  i18nAddLanguage,
  i18nRemoveLanguage,
  i18nSetDefaultLanguage,
  i18nSetKey,
  i18nDeleteKey,
  deleteUmlDiagram,
  updateUmlDiagram,
  createUmlDiagram,
  umlDiagrams,
  GitOps,
  Files,
},
Community: {
  search,
  getFeed,
  existsProfile,
  createPost,
  getPost,
  likePost,
  deletePost,
  createProfile,
  getProfile,
  updateProfile,
  isFollowing,
  getProfileFollowers,
  getProfilePosts,
  toggleFollow,
},
Util: {
  makeRequest: makeRequest,
},
Discovery: {
  getDiscoveryProjects,
  starProject,
  commentProject,
},
Forum: {
  createProfileV2: createProfileV2,
  deleteProfileV2: deleteProfileV2,
  getProfileV2: getProfileV2,
  modifyProfileV2: modifyProfileV2,
  createPostV2: createPostV2,
  getPostV2: getPostV2,
  deletePostV2: deletePostV2,
  getProfilePostsV2: getProfilePostsV2,
  getProfilesV2: getProfilesV2,
  likePostV2: likePostV2,
  getCommentsV2: getCommentsV2,
  getAllPostsV2: getAllPostsV2,
```

```
  createChatV2: createChatV2,
  getChatListV2: getChatListV2,
  deleteChatV2: deleteChatV2,
  sendMessageV2: sendMessageV2,
  getChatMessagesV2: getChatMessagesV2,
  getEmbeds,
},
Course: {
  getSection,
  getCourse,
  listCourses: listCoursesU,
  getEnrollments,
  getEnrollment,
  changeEnrollment,
  getCourseProgress,
  updateCourseProgress,
},
Creator: {
  getCourses: getCoursesCreator,
  createCourse: createCourseCreator,
  deleteCourse: deleteCourseCreator,
  updateCourse: updateCourseCreator,
  createContent: createContentCreator,
  deleteContent: deleteContentCreator,
  getContents: getContentsCreator,
  updateContent: updateContentCreator,
  getTopCreators: getTopCreators,
  getCertificate: getCertificate,
  createCertificate: createCertificate,
  updateCertificate: updateCertificate,
},
Analytics: {
  postAnalytics,
  getSummary,
  getViews,
  deleteAnalytics,
},
Certificates: {
  getCertificate: getCertificateC,
  submitCertificate,
  getObtainedCertificate,
  getObtainedCertificates,
},
Ideas: {
  getProjectIdeas,
},
Snippets: {
  getSnippets,
  getSnippet,
  createSnippet,
  deleteSnippet,
  updateSnippet,
},
Orgs: {
  getOrgs,
  deleteOrg,
  createOrg,
  getOrg,
  updateOrg,
  updateOrgIcon,
```

```
    renameOrg,
    removeOrgMember,
    addOrgMember,
    removeOrgTemplate,
    addOrgTemplate,
    getOrgTemplates,
    useOrgTemplate,
    createBlog: createBlogOrg,
    getBlogs: getBlogsOrg,
  },
  Editor: {
    getProjectsV2: getProjectsV2,
    getDomainById,
  },
  Kids: {
    getProjects: getKidsProjects,
    deleteProject: deleteKidsProject,
    createProject: createKidsProject,
    updateProject: updateKidsProject,
    getProject: getKidsProject,
  },
  Roadmap: {
    getMyRoads,
    createRoad,
    deleteRoad,
    updateRoad,
    getRoad,
  },
  Newsletter: {
    subscribe,
    unsubscribe,
    subscribed,
  },
  Payments: {
    createCheckoutLink,
    cancelSubscription,
  },
  Rce: {
    rceJavaScript,
    rceTypeScript,
    rcePython,
    rceGo,
    rceJava,
  },
};

export default REST;
```

## packages/rest/src/makeRequest.ts

```
/**
 * packages/rest/src/makeRequest.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 20.07.2022
 *
 */
import { IResponse } from "./types/IResponse";
```

```
export default async function makeRequest(options: {
  path: string;
  body?: object;
  token?: string;
  headers?: object;
  returnsRaw?: boolean;
  method:
    | "GET"
    | "POST"
    | "DELETE"
    | "PUT"
    | "HEAD"
    | "PATCH"
    | "OPTIONS"
    | "CONNECT"
    | "TRACE";
}): Promise<IResponse> {
  const headers: { [key: string]: string } = {
    "Content-Type": "application/json",
    "X-Executed-By": "@codeup/rest:makeRequest",
  };

  if (options.token) {
    headers["X-JWT"] = options.token;
  }

  if (options.headers) {
    Object.keys(options.headers).forEach((customHeader) => {
      // @ts-ignore
      headers[customHeader] = options.headers[customHeader];
    });
  }

  const res = await fetch(options.path, {
    method: options.method,
    headers,
    body: options.body ? JSON.stringify(options.body) : null,
  });

  let payload = null;

  if (!options.returnsRaw) {
    payload = await res.json();
  } else {
    payload = await res.text();
  }

  return {
    status: res.status,
    payload: payload,
  };
}
```

## packages/rest/src/types/IResponse.ts

```
/**
 * packages/rest/src/types/IResponse.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
```

```
 * Created At: 20.07.2022
 *
 */

export interface IResponse {
  status: number;
  payload: {
    [key: string]: any;
  };
}
```

## packages/rest/tsconfig.json

```
{
  "extends": "tsconfig/react-library.json",
  "exclude": ["dist", "build", "node_modules"],
  "compilerOptions": {
    "lib": ["DOM"],
    "jsx": "react",
    "rootDir": "src",
    "outDir": "dist"
  }
}
```

## packages/serverless-runtime/Dockerfile

```
FROM node:18
WORKDIR /codeup-serverless-function
COPY package.json .
COPY index.js .
RUN npm install
EXPOSE 3000
CMD ["node", "index.js"]
```

## packages/serverless-runtime/index.js

```
const express = require("express");

const app = express();

const METHOD = process.env.SERVERLESS_METHOD;
const ENDPOINT = process.env.SERVERLESS_ENDPOINT;
const CODE = process.env.SERVERLESS_CODE;
const ACCESS_TOKEN = process.env.SERVERLESS_TOKEN;
const ENDPOINT_URL = process.env.SERVERLESS_ENDPOINT_URL;

const codeup = {
  database: {
    get: async (key) => {
      const res = await fetch(ENDPOINT_URL + "/api/@codeup/database/get", {
        method: "GET",
        headers: {
          "X-SF-Access": ACCESS_TOKEN,
          "x-codeup-database-key": key,
        },
      });

      const text = await res.text();

      console.log(text);

      if (res.status === 200) {
        return text;
```

```
      } else {
        return null;
      }
    },
    set: async (key, value) => {
      const res = await fetch(ENDPOINT_URL + "/api/@codeup/database/set", {
        method: "POST",
        headers: {
          "X-SF-Access": ACCESS_TOKEN,
          "x-codeup-database-key": key,
        },
        body: JSON.stringify({
          value,
        }),
      });

      console.log(await res.text());

      return res.status === 200;
    },
  },
};

if (METHOD === "POST") {
  app.post(ENDPOINT, async (req, res) => {
    eval(`(async () => {${CODE}})()`);
  });
} else {
  app.get(ENDPOINT, async (req, res) => {
    eval(`(async () => {${CODE}})()`);
  });
}

app.listen(3000);
```

## packages/serverless-runtime/package.json

```
{
  "name": "@codeup/serverless-runtime",
  "verion": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "build": "docker build -t mctzock/codeup-serverless-runtime ."
  },
  "keywords": [],
  "author": "Ben Siebert",
  "license": "GPL-3.0-only",
  "dependencies": {
    "express": "^4.18.2"
  }
}
```

## packages/tsconfig/base.json

```
{
  "$schema": "https://json.schemastore.org/tsconfig",
  "display": "Default",
  "compilerOptions": {
    "composite": false,
    "declaration": true,
    "declarationMap": true,
```

```
    "esModuleInterop": true,
    "forceConsistentCasingInFileNames": true,
    "inlineSources": false,
    "isolatedModules": true,
    "moduleResolution": "node",
    "noUnusedLocals": false,
    "noUnusedParameters": false,
    "preserveWatchOutput": true,
    "skipLibCheck": true,
    "strict": true
  },
  "exclude": ["node_modules"]
}
```

## packages/tsconfig/nextjs.json

```
{
  "$schema": "https://json.schemastore.org/tsconfig",
  "display": "Next.js",
  "extends": "./base.json",
  "compilerOptions": {
    "target": "ES6",
    "lib": ["dom", "dom.iterable", "esnext"],
    "allowJs": true,
    "skipLibCheck": true,
    "strict": true,
    "forceConsistentCasingInFileNames": true,
    "noEmit": true,
    "incremental": true,
    "esModuleInterop": true,
    "module": "esnext",
    "resolveJsonModule": true,
    "isolatedModules": true,
    "jsx": "preserve"
  },
  "include": ["src", "next-env.d.ts"],
  "exclude": ["node_modules"]
}
```

## packages/tsconfig/package.json

```
{
  "name": "tsconfig",
  "version": "0.0.0",
  "private": true,
  "main": "index.js",
  "files": [
    "base.json",
    "nextjs.json",
    "react-library.json"
  ]
}
```

## packages/tsconfig/react-library.json

```
{
  "$schema": "https://json.schemastore.org/tsconfig",
  "display": "React Library",
  "extends": "./base.json",
  "compilerOptions": {
    "lib": ["ES2015"],
    "module": "ESNext",
```

```
    "target": "ES6",
    "jsx": "react-jsx"
  }
}
```

## packages/tsconfig/README.md

```
# `tsconfig`

These are base shared `tsconfig.json`s from which all other `tsconfig.json`'s inherit from.
```

## packages/util/package.json

```
{
  "name": "@codeup/util",
  "version": "1.0.0",
  "main": "src",
  "author": "Ben Siebert",
  "license": "GPL-3.0-only",
  "private": false,
  "scripts": {
    "build": "tsc"
  },
  "dependencies": {
    "@chakra-ui/react": "^2.1.2",
    "@codeup/rest": "*",
    "@emotion/react": "^11",
    "@emotion/styled": "^11",
    "@monaco-editor/react": "^4.4.5",
    "monaco-editor": "0.33.0",
    "mongoose": "^6.3.5"
  }
}
```

## packages/util/src/AccountManager.ts

```
/**
 * packages/util/src/AccountManager.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 02.03.23
 *
 */
import REST from "@codeup/rest/dist";
import PopupManager from "@codeup/web/src/global/PopupManager";

export type Account = {
  username: string;
  token: string;
  email: string;
  firstName: string;
  lastName: string;
};

export default class AccountManager {
  public static readonly LS_KEY = "codeup.accounts";

  public static getAccounts(): Account[] {
    try {
      if (
```

```
      localStorage.getItem(AccountManager.LS_KEY) === null ||
      localStorage.getItem(AccountManager.LS_KEY) === undefined
    ) {
      localStorage.setItem(AccountManager.LS_KEY, JSON.stringify([]));
    }

    const accs = JSON.parse(
      localStorage.getItem(AccountManager.LS_KEY) as string
    );

    if (accs === null || accs === undefined) {
      return [];
    }

    return accs;
  } catch (e) {
    return [];
  }
}

public static async addAccount(token: string): Promise<boolean> {
  const accs = AccountManager.getAccounts();

  const res = await REST.Account.verify(token);

  if (res.status !== 200) {
    return false;
  }

  if (res.payload.newToken) {
    return await this.addAccount(res.payload.newToken);
  }

  accs.push({
    token: token,
    username: res.payload.data.username,
    email: res.payload.data.email,
    firstName: res.payload.data.firstName,
    lastName: res.payload.data.lastName,
  });

  try {
    localStorage.setItem(AccountManager.LS_KEY, JSON.stringify(accs));

    return true;
  } catch (e) {
    return false;
  }
}

public static removeAccount(acc: Account): boolean {
  const accs = AccountManager.getAccounts().filter((a) => {
    return a.username !== acc.username;
  });

  localStorage.setItem(AccountManager.LS_KEY, JSON.stringify(accs));

  return true;
}
```

```
  public static getActiveToken(): string {
    return localStorage.getItem("token") as string;
  }

  public static async setActiveToken(token: string): Promise<void> {
    await AccountManager.addAccount(localStorage.getItem("token") as string);

    localStorage.setItem("token", token);
  }

  public static overrideActiveToken(token: string): void {
    localStorage.setItem("token", token);
  }
}
```

## packages/util/src/blockly/blockly/BlocklyCompiler.ts

```
/**
 * packages/util/src/blockly/blockly/BlocklyCompiler.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 02.04.2023
 */ import * as Blockly from "blockly";

export class BlocklyCompiler {
  private compiledBlocks: any[] = [];

  /**
   * Compiles the current blockly workspace
   * @return The compiled code
   */
  compile(): string {
    let code = "";
    Blockly.getMainWorkspace()
      .getAllBlocks(true)
      .forEach((block) => {
        if (!this.compiledBlocks.includes(block)) {
          code += this.compileBlock(block);
        }
      });
    return code;
  }

  /**
   * Compiles a single block given by the compile method
   * @param block The block to be compiled
   */
  private compileBlock(block: Blockly.Block): string {
    this.compiledBlocks.push(block);

    let code = "";

    let b = block.getInputTargetBlock("STATEMENT");

    if (b) {
      code += this.getBlockTabs(block, true) + this.getBlockText(block) + "\n";
      do {
        this.compiledBlocks.push(b);
        code += this.getBlockTabs(b, true) + this.compileBlock(b) + "\n";
```

```
      /* eslint-disable  no-cond-assign */
    } while ((b = b.getNextBlock()));
  } else {
    code += this.getBlockTabs(block, true) + this.getBlockText(block) + "\n";
  }

  return code;
}

/**
 * Calculates the TABS (\t) for a given block
 * @param block The block which should be calculated
 * @param log Log the TABS (\t) to the console DEFAULT = false. DEBUGGING ONLY
 * @param b1 Ignore this boolean, idiot. Just kidding, basically I created it for debugging and it now
has no sense anymore
 */
private getBlockTabs(block: Blockly.Block, log = false, b1 = true): string {
  let tabs = "";

  let currentBlock = block;

  if (block.getParent() != null && b1) {
    let neededTabs = -1;
    neededTabs =
      neededTabs +
      this.getBlockTabs(block.getParent() as Blockly.Block, log, true).length;
    if ((block.getParent() as Blockly.Block).getInputTargetBlock("STATEMENT"))
      neededTabs++;
    for (let i = 0; i < neededTabs; i++) {
      tabs += "\t";
    }
  }

  while (currentBlock.getParent() != null) {
    let b = (currentBlock.getParent() as Blockly.Block).getInputTargetBlock(
      "STATEMENT"
    );
    if (
      this.getBlockText(currentBlock.getParent() as Blockly.Block).includes(
        "Methode"
      )
    )
      tabs += "\t";

    if (b) {
      do {
        if (b === block) tabs += "\t";
        /* eslint-disable  no-cond-assign */
      } while ((b = b.getNextBlock()));
    }
    currentBlock = currentBlock.getParent() as Blockly.Block;
  }

  return tabs;
}

/**
 * Actually compiles a block
 * @param block The block to be compiled
 */
```

```
  private getBlockText(block: Blockly.Block): string {
    let c = "";
    block.inputList.forEach((i) => {
      i.fieldRow.forEach((r) => {
        c += r.getValue() + " ";
      });
    });
    return c;
  }
}
```

## packages/util/src/blockly/blockly/Blocks.ts

```
/**
 * @author Ben Siebert <ben@mctzock.de>
 * @copyright (c) 2018-2021 Ben Siebert. All rights reserved.
 */

import Blockly from "blockly";

export default class Blocks {
  public static register() {
    Blockly.defineBlocksWithJsonArray([
      {
        type: "html",
        message0: "<html>",
        nextStatement: null,
        colour: 210,
      },
      {
        type: "html_close",
        message0: "</html>",
        previousStatement: null,
        colour: 210,
      },
      {
        type: "head",
        message0: "<head>",
        colour: 210,
        nextStatement: null,
        previousStatement: null,
      },
      {
        type: "head_close",
        message0: "</head>",
        colour: 210,
        nextStatement: null,
        previousStatement: null,
      },
      {
        type: "body",
        message0: "<body>",
        colour: 210,
        nextStatement: null,
        previousStatement: null,
      },
      {
        type: "body_close",
        message0: "</body>",
        colour: 210,
        nextStatement: null,
```

```
  previousStatement: null,
},
{
  type: "title",
  message0: "<title> %1 </title>",
  colour: 180,
  nextStatement: null,
  previousStatement: null,
  inputsInline: true,
  args0: [
    {
      type: "field_input",
      name: "TITLE",
    },
  ],
},
{
  type: "meta",
  message0: "<meta %1 = %2 />",
  colour: 180,
  nextStatement: null,
  previousStatement: null,
  inputsInline: true,
  args0: [
    {
      type: "field_dropdown",
      name: "NAME",
      options: [
        ["name", "name"],
        ["content", "content"],
        ["charset", "charset"],
      ],
    },
    {
      type: "field_input",
      name: "TITLE",
    },
  ],
},
{
  type: "h1",
  message0: "<h1> %1 </h1>",
  colour: 120,
  nextStatement: null,
  previousStatement: null,
  inputsInline: true,
  args0: [
    {
      type: "field_input",
      name: "TITLE",
    },
  ],
},
{
  type: "h2",
  message0: "<h2> %1 </h2>",
  colour: 120,
  nextStatement: null,
  previousStatement: null,
  inputsInline: true,
```

```
  args0: [
    {
      type: "field_input",
      name: "TITLE",
    },
  ],
},
{
  type: "h3",
  message0: "<h3> %1 </h3>",
  colour: 120,
  nextStatement: null,
  previousStatement: null,
  inputsInline: true,
  args0: [
    {
      type: "field_input",
      name: "TITLE",
    },
  ],
},
{
  type: "h4",
  message0: "<h4> %1 </h4>",
  colour: 120,
  nextStatement: null,
  previousStatement: null,
  inputsInline: true,
  args0: [
    {
      type: "field_input",
      name: "TITLE",
    },
  ],
},
{
  type: "h5",
  message0: "<h5> %1 </h5>",
  colour: 120,
  nextStatement: null,
  previousStatement: null,
  inputsInline: true,
  args0: [
    {
      type: "field_input",
      name: "TITLE",
    },
  ],
},
{
  type: "h6",
  message0: "<h6> %1 </h6>",
  colour: 120,
  nextStatement: null,
  previousStatement: null,
  inputsInline: true,
  args0: [
    {
      type: "field_input",
      name: "TITLE",
```

```
    },
  ],
},
{
  type: "p",
  message0: "<p> %1 </p>",
  colour: 120,
  nextStatement: null,
  previousStatement: null,
  inputsInline: true,
  args0: [
    {
      type: "field_input",
      name: "TITLE",
    },
  ],
},
{
  type: "span",
  message0: "<span> %1 </span>",
  colour: 120,
  nextStatement: null,
  previousStatement: null,
  inputsInline: true,
  args0: [
    {
      type: "field_input",
      name: "TITLE",
    },
  ],
},
{
  type: "a",
  message0: "<a href='%1' target='%2'>",
  colour: 120,
  nextStatement: null,
  previousStatement: null,
  inputsInline: true,
  args0: [
    {
      type: "field_input",
      name: "HREF",
    },
    {
      type: "field_dropdown",
      name: "TARGET",
      options: [
        ["_blank", "_blank"],
        ["_self", "_self"],
        ["_parent", "_parent"],
        ["_top", "_top"],
      ],
    },
  ],
},
{
  type: "a_close",
  message0: "</a>",
  colour: 120,
  nextStatement: null,
```

```
  previousStatement: null,
},
{
  type: "b",
  message0: "<b>",
  colour: 120,
  nextStatement: null,
  previousStatement: null,
  inputsInline: true,
},
{
  type: "b_close",
  message0: "</b>",
  colour: 120,
  nextStatement: null,
  previousStatement: null,
},
{
  type: "em",
  message0: "<em>",
  colour: 120,
  nextStatement: null,
  previousStatement: null,
  inputsInline: true,
},
{
  type: "em_close",
  message0: "</em>",
  colour: 120,
  nextStatement: null,
  previousStatement: null,
},
{
  type: "i",
  message0: "<i>",
  colour: 120,
  nextStatement: null,
  previousStatement: null,
  inputsInline: true,
},
{
  type: "i_close",
  message0: "</i>",
  colour: 120,
  nextStatement: null,
  previousStatement: null,
},
{
  type: "blockquote",
  message0: "<blockquote>",
  colour: 120,
  nextStatement: null,
  previousStatement: null,
  inputsInline: true,
},
{
  type: "blockquote_close",
  message0: "</blockquote>",
  colour: 120,
  nextStatement: null,
```

```
  previousStatement: null,
},
{
  type: "br",
  message0: "<br />",
  colour: 120,
  nextStatement: null,
  previousStatement: null,
  inputsInline: true,
},
{
  type: "custom_text",
  message0: "%1",
  colour: 120,
  nextStatement: null,
  previousStatement: null,
  args0: [
    {
      type: "field_input",
      name: "TEXT",
      text: "Text",
    },
  ],
},
{
  type: "div",
  message0: "<div>",
  colour: 240,
  nextStatement: null,
  previousStatement: null,
},
{
  type: "div_close",
  message0: "</div>",
  colour: 240,
  nextStatement: null,
  previousStatement: null,
},
{
  type: "hr",
  message0: "<hr />",
  colour: 240,
  nextStatement: null,
  previousStatement: null,
  inputsInline: true,
},
{
  type: "image",
  message0: "<img src='%1' width='%2px' height='%3px' />",
  colour: 140,
  nextStatement: null,
  previousStatement: null,
  inputsInline: true,
  args0: [
    {
      type: "field_input",
      name: "SOURCE",
    },
    {
      type: "field_number",
```

```
      name: "WIDTH",
    },
    {
      type: "field_number",
      name: "HEIGHT",
    },
  ],
},
{
  type: "video",
  message0: "<video src='%1' width='%2px' height='%3px' controls />",
  colour: 140,
  nextStatement: null,
  previousStatement: null,
  inputsInline: true,
  args0: [
    {
      type: "field_input",
      name: "SOURCE",
    },
    {
      type: "field_number",
      name: "WIDTH",
    },
    {
      type: "field_number",
      name: "HEIGHT",
    },
  ],
},
{
  type: "ul",
  message0: "<ul>",
  colour: 170,
  nextStatement: null,
  previousStatement: null,
  inputsInline: true,
},
{
  type: "ul_close",
  message0: "</ul>",
  colour: 170,
  nextStatement: null,
  previousStatement: null,
  inputsInline: true,
},
{
  type: "ol",
  message0: "<ol>",
  colour: 170,
  nextStatement: null,
  previousStatement: null,
  inputsInline: true,
},
{
  type: "ol_close",
  message0: "</ol>",
  colour: 170,
  nextStatement: null,
  previousStatement: null,
```

```
  inputsInline: true,
},
{
  type: "li",
  message0: "<li>",
  colour: 170,
  nextStatement: null,
  previousStatement: null,
  inputsInline: true,
},
{
  type: "li_close",
  message0: "</li>",
  colour: 170,
  nextStatement: null,
  previousStatement: null,
  inputsInline: true,
},

{
  type: "table",
  message0: "<table>",
  colour: 0,
  nextStatement: null,
  previousStatement: null,
  inputsInline: true,
},
{
  type: "table_close",
  message0: "</table>",
  colour: 0,
  nextStatement: null,
  previousStatement: null,
  inputsInline: true,
},

{
  type: "thead",
  message0: "<thead>",
  colour: 0,
  nextStatement: null,
  previousStatement: null,
  inputsInline: true,
},
{
  type: "thead_close",
  message0: "</thead>",
  colour: 0,
  nextStatement: null,
  previousStatement: null,
  inputsInline: true,
},

{
  type: "tbody",
  message0: "<tbody>",
  colour: 0,
  nextStatement: null,
  previousStatement: null,
  inputsInline: true,
```

```
},
{
  type: "tbody_close",
  message0: "</tbody>",
  colour: 0,
  nextStatement: null,
  previousStatement: null,
  inputsInline: true,
},
{
  type: "tfoot",
  message0: "<tfoot>",
  colour: 0,
  nextStatement: null,
  previousStatement: null,
  inputsInline: true,
},
{
  type: "tfoot_close",
  message0: "</tfoot>",
  colour: 0,
  nextStatement: null,
  previousStatement: null,
  inputsInline: true,
},
{
  type: "td",
  message0: "<td>",
  colour: 0,
  nextStatement: null,
  previousStatement: null,
  inputsInline: true,
},
{
  type: "td_close",
  message0: "</td>",
  colour: 0,
  nextStatement: null,
  previousStatement: null,
  inputsInline: true,
},
{
  type: "th",
  message0: "<th>",
  colour: 0,
  nextStatement: null,
  previousStatement: null,
  inputsInline: true,
},
{
  type: "th_close",
  message0: "</th>",
  colour: 0,
  nextStatement: null,
  previousStatement: null,
  inputsInline: true,
},
{
  type: "tr",
  message0: "<tr>",
```

```
        colour: 0,
        nextStatement: null,
        previousStatement: null,
        inputsInline: true,
      },
      {
        type: "tr_close",
        message0: "</tr>",
        colour: 0,
        nextStatement: null,
        previousStatement: null,
        inputsInline: true,
      },
    ]);
  }
}
```

## packages/util/src/blockly/blockly/theme/BlocklyThemeDark.ts

```
/**
 * Original by Google: https://github.com/google/blockly-sampes/blob/master/plugins/theme-dark/src/index.js
 */

/**
 * @license
 * Copyright 2021 Google LLC
 * SPDX-License-Identifier: Apache-2.0
 */

/**
 * @fileoverview Dark theme.
 */

import * as Blockly from "blockly/core";

// Temporarily required to ensure there's no conflict with
// Blockly.Themes.Dark
Blockly.registry.unregister("theme", "dark");

/**
 * Dark theme.
 */
export default Blockly.Theme.defineTheme("dark", {
  componentStyles: {
    workspaceBackgroundColour: "#1e1e1e",
    toolboxBackgroundColour: "blackBackground",
    toolboxForegroundColour: "#fff",
    flyoutBackgroundColour: "#252526",
    flyoutForegroundColour: "#ccc",
    flyoutOpacity: 1,
    scrollbarColour: "#797979",
    insertionMarkerColour: "#fff",
    insertionMarkerOpacity: 0.3,
    scrollbarOpacity: 0.4,
    cursorColour: "#d0d0d0",
    // @ts-ignore
    blackBackground: "#333",
  },
});
```

## packages/util/src/blockly/BlocklyEditor.tsx

```
/**
 * @author Ben Siebert <ben@mctzock.de>
 * @copyright (c) 2018-2021 Ben Siebert. All rights reserved.
 */

import * as React from "react";
import { useEffect } from "react";
import Blockly from "blockly";
import * as DarkTheme from "./blockly/theme/BlocklyThemeDark";
import Blocks from "./blockly/Blocks";
import * as DE from "blockly/msg/de";
import { defaultToolbox } from "./toolbox";

export interface BlocklyProps {
  initialXml: string;
  readonly?: boolean;
}

export default function BlocklyEditor(props: BlocklyProps) {
  useEffect(() => {
    // @ts-ignore
    if (window.injectedBlockly) return;
    // @ts-ignore
    window.injectedBlockly = true;

    Blocks.register();
    Blockly.setLocale(DE);
    const ws = Blockly.inject("blockly", {
      toolbox: defaultToolbox,
      renderer: "zelos",
      theme: DarkTheme.default,
      scrollbars: true,
      zoom: {
        controls: true,
        startScale: 0.8,
      },
    });
    Blockly.Xml.domToWorkspace(Blockly.Xml.textToDom(props.initialXml), ws);
    console.log(Blockly.getMainWorkspace());
  }, []);

  return (
    <>
      <div
        id="blockly"
        style={{ width: "100%", height: "100vh", margin: "0" }}
      />
    </>
  );
}
```

## packages/util/src/blockly/toolbox.ts

```
/**
 * packages/util/src/blockly/toolbox.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 02.04.2023
```

```
 *
 */

export const defaultToolbox = `
<xml xmlns="https://developers.google.com/blockly/xml" id="toolbox" style="display: none">
  <category colour="210" name="Global">
    <block type="html" />
    <block type="html_close" />
    <block type="head" />
    <block type="head_close" />
    <block type="body" />
    <block type="body_close" />
  </category>
  <category colour="180" name="Meta">
    <block type="title" />
    <block type="meta" />
  </category>
  <category colour="120" name="Text">
    <block type="custom_text" />
    <block type="h1" />
    <block type="h2" />
    <block type="h3" />
    <block type="h4" />
    <block type="h5" />
    <block type="h6" />
    <block type="p" />
    <block type="span" />
    <block type="a" />
    <block type="a_close" />
    <block type="b" />
    <block type="b_close" />
    <block type="blockquote" />
    <block type="blockquote_close" />
    <block type="br" />
  </category>
  <category colour="170" name="Listen">
    <block type="ul" />
    <block type="ul_close" />
    <block type="ol" />
    <block type="ol_close" />
    <block type="li" />
    <block type="li_close" />
  </category>
  <category colour="240" name="Layout">
    <block type="div" />
    <block type="div_close" />
    <block type="hr" />
  </category>
  <category colour="0" name="Tabellen">
    <block type="table" />
    <block type="table_close" />
    <block type="thead" />
    <block type="thead_close" />
    <block type="tbody" />
    <block type="tbody_close" />
    <block type="tfoot" />
    <block type="tfoot_close" />
    <block type="td" />
    <block type="td_close" />
    <block type="th" />
```

```
      <block type="th_close" />
      <block type="tr" />
      <block type="tr_close" />
    </category>
    <category colour="140" name="Medien">
      <block type="image" />
      <block type="video" />
    </category>
</xml>`;
```

## packages/util/src/checkPassword.ts

```
/**
 * packages/util/src/checkPassword.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 13.07.2022
 *
 */

export default function checkPassword(pass: string): boolean {
  const regex = new RegExp(
    "^(?=.*[a-z])(?=.*[A-Z])(?=.*[0-9])(?=.*[!@#$%^&*])(?=.{8,})"
  );

  return regex.test(pass);
}
```

## packages/util/src/courses/Course.ts

```
/**
 * src/util/courses/Course.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 21.06.2022
 *
 */

export interface Course {
  name: string;
  displayName: string;
  description: string;
  splashImage: string;
  level: "Beginner" | "Intermediate" | "Advanced";
  colorScheme:
    | "gray"
    | "red"
    | "orange"
    | "yellow"
    | "green"
    | "teal"
    | "blue"
    | "cyan"
    | "purple"
    | "pink"
    | "linkedin"
    | "facebook"
    | "messenger"
```

```
    | "whatsapp"
    | "twitter"
    | "telegram";
}
```

## packages/util/src/courses/CourseDB.ts

```
/**
 * src/util/courses/CourseDB.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 21.06.2022
 *
 */
import { Course } from "./Course";

export default class CourseDB {
  public static readonly courses: Course[] = [
    {
      name: "html-basics",
      displayName: "HTML Basics",
      description:
        "In diesem Kurs lernst du die Grundlagen von HTML, was HTML ist, wofür es verwendet wird
und wie du es in deinen eigenen Projekten verwenden kannst.",
      level: "Beginner",
      splashImage: "/_static/images/courses/splashes/html-basics.svg",
      colorScheme: "orange",
    },
    {
      name: "javascript-basics",
      displayName: "JavaScript Basics",
      description:
        "In diesem Kurs lernst du die Grundlagen von JavaScript, was JavaScript ist, wofür es verwendet
wird und wie du es in deinen eigenen Projekten verwenden kannst.",
      level: "Beginner",
      splashImage: "/_static/images/courses/splashes/javascript-basics.svg",
      colorScheme: "yellow",
    },
  ];
}
```

## packages/util/src/generateJWT.ts

```
/**
 * src/util/generateJWT.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 06.06.2022
 *
 */
import * as jwt from "jsonwebtoken";

export default function generateJWT(user: any) {
  return jwt.sign(
    {
      id: user._id,
      email: user.email,
      firstName: user.firstName,
```

```
      lastName: user.lastName,
      _2fa: user.twoFactorEnabled,
      username: user.username,
      sso: user.sso || false,
      role: user.role || "user",
      subscription: user.subscription || "FREE",
    },
    process.env.JWT_SECRET as string,
    {
      expiresIn: "30d",
    }
  );
}
```

## packages/util/src/generateMail.ts

```
/**
 * src/util/generateMail.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 18.06.2022
 *
 */

export default function generateMail(firstName: string, content: string) {
  return {
    text: `Moin ${firstName}!,\n\n
${content}\n\n
Beste Grüße aus Hattingen,\n
Ben von CodeUp`,
    html: `
<!DOCTYPE html>
<html lang="de">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <meta http-equiv="X-UA-Compatible" content="ie=edge" />
    <meta name="color-scheme" content="dark light" />
    <meta name="supported-color-schemes" content="dark light" />
  </head>
  <body>
    <div style="
          width: 100vw;
          min-height: 100vh;
          height: fit-content; margin: 0 auto;
          ">
      <div style="
          background-color: #000000;
          padding: 20px;
          width: 100vw;
          min-height: 100vh;
          height: fit-content;
          ">

        <div style="width: 100%; text-align: center; display: flex; flex-direction: column; gap: 1rem; align-
items: center; justify-content: center;">
          <img src="https://codeup.space/codeup-splash-github.jpg" alt="Ben Siebert" style="width:
100%; min-width: 100%;">
        </div>
```

```
        <div style="width: 100%; text-align: center;">
          <h1 style="font-size: 2.5rem; font-weight: bold; color: #fff;">Moin ${firstName}!</h1>
        </div>

        <div style="width: 100%; text-align: center;">
          <p style="font-size: 1.5rem; color: #fff;">${content}</p>
          <p style="font-size: 1.5rem; color: #fff;">
            Beste Grüße aus Hattingen,<br>
            Ben von <a href="https://codeup.space" style="color: deeppink">CodeUp</a>
          </p>
          <p style="font-size: 1.5rem; color: #fff;">
            <a href="https://codeup.space" style="color: deeppink">codeup.space</a> | <a
href="https://ben-siebert.de" style="color: deeppink">ben-siebert.de</a>
          </p>
        </div>

      </div>
    </div>
  </body>
</html>
    `,
  };
}
```

## packages/util/src/grapejs/plugin-flexbox.ts

```
/**
 * packages/util/src/grapejs-plugin/flexbox.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 19.11.2022
 *
 * Original: https://github.com/artf/grapesjs-blocks-
flexbox/blob/c56b4f6d9d0395407251f349f8c2b5cf7c45512e/src/blocks.js
 *
 */
import { FaColumns } from "react-icons/fa";

export default (
  editor: any,
  opt: any = {
    flexboxBlock: {},
    stylePrefix: "",
    labelRow: "Row",
    labelColumn: "Column",
  }
) => {
  const bm = editor.BlockManager;
  const blocks = opt.blocks || [];
  const stylePrefix = opt.stylePrefix;
  const toAdd = (name: any) => blocks.indexOf(name) >= 0;
  const nameClm1 = "flexbox";
  const clsRow = `${stylePrefix}row`;
  const clsCell = `${stylePrefix}cell`;
  const labelRow = opt.labelRow;
  const labelCell = opt.labelColumn;
  const attrsToString = (attrs: any) => {
    const result = [];
```

```
  for (let key in attrs) {
    let value = attrs[key];
    const toParse = value instanceof Array || value instanceof Object;
    value = toParse ? JSON.stringify(value) : value;
    result.push(`${key}=${toParse ? `'${value}'` : `"${value}"`}`);
  }

  return result.length ? ` ${result.join(" ")}` : "";
};

// Flex elements do not react on width style change therefore I use
// 'flex-basis' as keyWidth for the resizer on columns
const keyWidth = "flex-basis";
const step = 0.2;
const minDim = 1;
const currentUnit = 1;
const resizerBtm = {
  tl: 0,
  tc: 0,
  tr: 0,
  cl: 0,
  cr: 0,
  bl: 0,
  br: 0,
  minDim,
};
const resizerRight = {
  ...resizerBtm,
  cr: 1,
  bc: 0,
  keyWidth,
  currentUnit,
  minDim,
  step,
};
const rowAttr = {
  class: clsRow,
  "data-gjs-droppable": `.${clsCell}`,
  "data-gjs-resizable": resizerBtm,
  "data-gjs-custom-name": labelRow,
};

const colAttr = {
  class: clsCell,
  "data-gjs-draggable": `.${clsRow}`,
  "data-gjs-resizable": resizerRight,
  "data-gjs-custom-name": labelCell,
  "data-gjs-droppable": "*",
  // X Flex
  "data-gjs-unstylable": ["width"],
  "data-gjs-stylable-require": ["flex-basis"],
};

const privateCls = [`.${clsRow}`, `.${clsCell}`];
editor.on(
  "selector:add",
  (selector: any) =>
    privateCls.indexOf(selector.getFullName()) >= 0 &&
    selector.set("private", 1)
```

```
);

const label = "Flexbox";
const category = "Layout";
const attrsRow = attrsToString(rowAttr);
const attrsCell = attrsToString(colAttr);
const styleRow = `
  .${clsRow} {
    display: flex;
    justify-content: flex-start;
    align-items: stretch;
    padding: 10px;
  }
  `;
const styleClm = `
  .${clsCell} {
    min-height: 75px;
    flex: 100%;
  }`;

bm.add("flexbox-row", {
  label: "Row",
  category,
  media: reactIconToSVG(FaColumns),
  content: `
      <div ${attrsRow}>
      </div>
      <style>
        ${styleRow}
      </style>
      `,
});

bm.add("flexbox-column", {
  label: "Column",
  category,
  media: reactIconToSVG(FaColumns),
  content: `
      <div ${attrsCell}>
      </div>
      <style>
        ${styleClm}
      </style>
      `,
});

bm.add("flexbox-1c", {
  label: "1 Column",
  category,
  media: reactIconToSVG(FaColumns),
  content: `
      <div ${attrsRow}>
        <div ${attrsCell}></div>
      </div>
      <style>
        ${styleRow}
        ${styleClm}
      </style>
      `,
});
```

```
  bm.add("flexbox-2c", {
    label: "2 Columns",
    category,
    media: reactIconToSVG(FaColumns),
    content: `
        <div ${attrsRow}>
          <div ${attrsCell}></div>
          <div ${attrsCell}></div>
        </div>
        <style>
          ${styleRow}
          ${styleClm}
        </style>
        `,
  });

  bm.add("flexbox-3c", {
    label: "3 Columns",
    category,
    media: reactIconToSVG(FaColumns),
    content: `
        <div ${attrsRow}>
          <div ${attrsCell}></div>
          <div ${attrsCell}></div>
          <div ${attrsCell}></div>
        </div>
        <style>
          ${styleRow}
          ${styleClm}
        </style>
        `,
  });
};

function reactIconToSVG(icon: any) {
  return `<svg viewBox="0 0 512 512"><path d="${
    icon().props.children[0].props.d
  }" /></svg>`;
}
```

## packages/util/src/models/AnalyticsModel.ts

```
/**
 * packages/util/src/models/AnalyticsModel.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 25.11.2022
 *
 */

import mongoose from "mongoose";

const Analytics = new mongoose.Schema({
  loggedIn: {
    type: Boolean,
    required: true,
  },
  userId: {
```

```
    type: mongoose.Schema.Types.ObjectId,
    ref: "User",
    default: null,
    required: false,
  },
  path: {
    type: String,
    required: true,
  },
  createdAt: {
    type: Date,
    default: Date.now,
    required: true,
    immutable: true,
  },
  browser: {
    type: String,
    required: true,
  },
  os: {
    type: String,
    required: true,
  },
});

export default mongoose.models?.Analytics ||
  mongoose.model("Analytics", Analytics);
```

## packages/util/src/models/AskPostModel.ts

```
/**
 * packages/util/src/models/AskPostModel.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 23.06.23
 *
 */

import mongoose from "mongoose";

const AskPostModel = new mongoose.Schema({
  user: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "User",
    required: true,
  },
  username: {
    type: String,
    required: true,
  },
  content: {
    type: String,
    required: true,
  },
  embeds: {
    type: Array,
    default: [],
  },
  createdAt: {
```

```
    type: Date,
    default: Date.now,
    immutable: true,
  },
  parent: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "AskPost",
    default: null,
  },
});

export default mongoose.models?.AskPost ||
  mongoose.model("AskPost", AskPostModel);
```

## packages/util/src/models/AskProfileModel.ts

```
/**
 * src/util/models/AskProfileModel.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 26.06.2023
 *
 */

import mongoose from "mongoose";

const AskProfileModel = new mongoose.Schema({
  user: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "User",
    required: true,
  },
  username: {
    type: String,
    required: true,
  },
  banner: {
    type: String,
    required: true,
    default: "/_static/images/software-developer-6521720_1920.jpg",
  },
  website: {
    type: String,
    required: false,
  },
  bio: {
    type: String,
    required: false,
  },
  badges: {
    type: [String],
    required: false,
  },
  certificates: {
    type: [mongoose.Schema.Types.ObjectId],
    ref: "Certificate",
    required: false,
  },
  projects: {
```

```
    type: [mongoose.Schema.Types.ObjectId],
    ref: "Project",
    required: false,
  },
});

export default mongoose.models?.AskProfile ||
  mongoose.model("AskProfile", AskProfileModel);
```

## packages/util/src/models/BlogModel.ts

```
/**
 * packages/util/src/models/BlogModel.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 27.12.2022
 *
 */
import mongoose from "mongoose";

const Blog = new mongoose.Schema({
  title: {
    type: String,
    required: true,
  },
  content: {
    type: String,
    required: true,
  },
  authorUsername: {
    type: String,
    required: true,
  },
  published_at: {
    type: Date,
    required: true,
  },
  tags: {
    type: [String],
    required: true,
  },
});

export default mongoose.models?.Blog || mongoose.model("Blog", Blog);
```

## packages/util/src/models/Certificate.ts

```
/**
 * packages/util/src/models/Certificate.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 26.11.2022
 *
 */
import mongoose from "mongoose";

const Certificate = new mongoose.Schema({
  course: {
```

```
    type: mongoose.Schema.Types.ObjectId,
    required: true,
    ref: "Course",
  },
  minPercentage: {
    type: Number,
    required: true,
  },
  steps: {
    type: ["Mixed"],
  },
});

export default mongoose.models?.Certificate ||
  mongoose.model("Certificate", Certificate);

/*

====================
Schema as JSON File
====================


{
  "course": "Course-ID",
  "minPercentage": 50,
  "steps": [
    {
      "step_type": "MULTIPLE_CHOICE | TEXT | CODE_COMPARE | CODE_CHECK",
      "step_title": "Step Title",
      "multiple_choice": {
        "question": "Question",
        "answers": [
          {
            "answer": "Answer #N",
            "correct": false
          }
        ]
      },
      "text_parts": [
        {
          "type": "TEXT",
          "value": "<button>"
        },
        {
          "type": "INPUT",
          "value": "klick mich"
        }
      ],
      "code_compare": {
        "code_a": {
          "correct": false,
          "screenshot": "https://screen.shot/.png"
        },
        "code_b": {
          "correct": false,
          "screenshot": "https://screen.shot/.png"
        },
        "output": "https://screen.shot/.png"
      },
```

```
      "code_check": {
        "code_a": {
          "correct": false,
          "screenshot": "https://screen.shot/.png"
        },
        "code_b": {
          "correct": false,
          "screenshot": "https://screen.shot/.png"
        }
      }
    }
  ]
}


 */
```

## packages/util/src/models/CodeChallengeModel.ts

```
/**
 * packages/util/src/models/CodeChallengeModel.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2024 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 18.05.2024
 *
 */

import mongoose from "mongoose";

const CodeChallengeModel = new mongoose.Schema({
  createdAt: {
    type: Date,
    required: true,
    default: Date.now,
  },
  difficulty: {
    type: Number,
    required: true,
  },
  title: {
    type: String,
    required: true,
  },
  task: {
    type: String,
    required: true,
  },
  expectedOutput: {
    type: String,
    required: true,
  },
});

export default mongoose.models?.CodeChallenge ||
  mongoose.model("CodeChallenge", CodeChallengeModel);
```

## packages/util/src/models/CodeSnippet.ts

```
/**
 * packages/util/src/models/CodeSnippet.ts
```

```
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 30.12.2022
 *
 */

import mongoose from "mongoose";

const Snippet = new mongoose.Schema({
  username: {
    type: String,
    required: true,
  },
  code: {
    type: String,
    required: true,
  },
  createdAt: {
    type: Date,
    default: Date.now,
    immutable: true,
    required: true,
  },
  language: {
    type: String,
    required: true,
  },
  title: {
    type: String,
    required: true,
  },
});

export default mongoose.models?.Snippet || mongoose.model("Snippet", Snippet);
```

## packages/util/src/models/CodeUpKidsProjectModel.ts

```
/**
 * packages/util/src/models/CodeUpKidsProjectModel.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 02.04.2023
 *
 */

import mongoose from "mongoose";

const CodeUpKidsProject = new mongoose.Schema({
  owner: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "User",
    required: true,
  },
  name: {
    type: String,
    required: true,
  },
```

```
  code: {
    type: String,
    required: true,
  },
});

export default mongoose.models?.CodeUpKidsProject ||
  mongoose.model("CodeUpKidsProject", CodeUpKidsProject);
```

## packages/util/src/models/CommunityPostModel.ts

```
/**
 * src/util/models/CommunityPostModel.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 19.06.2022
 *
 */

import mongoose from "mongoose";

const CommunityPost = new mongoose.Schema({
  user: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "User",
    required: true,
  },
  username: {
    type: String,
    required: true,
  },
  content: {
    type: String,
    required: true,
    minlength: 1,
  },
  createdAt: {
    type: Date,
    default: Date.now,
    required: true,
    immutable: true,
  },
  likes: {
    type: [mongoose.Schema.Types.ObjectId],
    default: [],
    required: true,
  },
  comments: {
    type: [mongoose.Schema.Types.ObjectId],
    default: [],
    required: true,
  },
  parent: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "CommunityPost",
    default: null,
  },
});
```

```
export default mongoose.models?.CommunityPost ||
  mongoose.model("CommunityPost", CommunityPost);
```

## packages/util/src/models/CommunityProfileModel.ts

```
/**
 * src/util/models/UserModel.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 05.06.2022
 *
 */

import mongoose from "mongoose";

const CommunityProfile = new mongoose.Schema({
  user: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "User",
    required: true,
  },
  username: {
    type: String,
    required: true,
  },
  profilePicture: {
    type: String,
    default: "https://static.content.ben-siebert.de/api/public/dl/lSFt9Tkg",
    required: true,
  },
  profileBanner: {
    type: String,
    default: "https://static.content.ben-siebert.de/api/public/dl/92eo0fdj",
    required: true,
  },
  displayName: {
    type: String,
    required: true,
  },
  verified: {
    type: Boolean,
    default: false,
  },
  specialStatus: {
    type: String,
    default: "",
  },
  bio: {
    type: String,
    default: "",
  },
  website: {
    type: String,
    default: "",
  },
  location: {
    type: String,
    default: "",
  },
```

```
  birthday: {
    type: String,
    default: "2007-03-20",
  },
  createdAt: {
    type: Date,
    default: Date.now,
    immutable: true,
  },
  followers: {
    type: [
      {
        type: String,
      },
    ],
    default: [],
    required: true,
  },
});

export default mongoose.models?.CommunityProfile ||
  mongoose.model("CommunityProfile", CommunityProfile);
```

## packages/util/src/models/CompletedUserChallenges.ts

```
/**
 * packages/util/src/models/CompletedUserChallenges.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2024 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 18.05.2024
 *
 */

import mongoose from "mongoose";

const CompletedUserChallenges = new mongoose.Schema({
  createdAt: {
    type: Date,
    required: true,
    default: Date.now,
  },
  user: {
    type: mongoose.Schema.Types.ObjectId,
    required: true,
    ref: "User",
  },
  challenges: {
    type: [mongoose.Schema.Types.ObjectId],
    required: true,
    ref: "Challenge",
  },
});

export default mongoose.models?.CompletedUserChallenges ||
  mongoose.model("CompletedUserChallenges", CompletedUserChallenges);
```

## packages/util/src/models/ContactModel.ts

```
/**
 * packages/util/src/models/ContactModel.ts
```

```
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 26.11.2022
 *
 */

import mongoose from "mongoose";

const Contact = new mongoose.Schema({
  username: {
    type: String,
    required: true,
  },
  message: {
    type: String,
    required: true,
  },
  createdAt: {
    type: Date,
    default: Date.now,
    immutable: true,
    required: true,
  },
  category: {
    type: String,
    required: true,
    default: "sonstiges",
  },
  title: {
    type: String,
    required: true,
  },
});

export default mongoose.models?.Contact || mongoose.model("Contact", Contact);
```

## packages/util/src/models/ContentModel.ts

```
/**
 * packages/util/src/models/ContentModel.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 06.11.2022
 *
 */

import mongoose from "mongoose";

const Content = new mongoose.Schema({
  displayName: {
    type: String,
    required: true,
  },
  course: {
    type: String,
    required: true,
  },
```

```
  url: {
    type: String,
    required: true,
  },
  createdAt: {
    type: Date,
    default: Date.now,
    immutable: true,
    required: true,
  },
  type: {
    type: String,
    required: true,
  },
  contentType: {
    type: String,
    required: true,
    default: "textContent",
  },
  textContent: {
    type: String,
    required: false,
  },
  quiz: {
    type: [
      {
        question: {
          type: String,
          required: true,
        },
        answers: {
          type: [
            {
              answer: {
                type: String,
                required: true,
              },
              correct: {
                type: Boolean,
                required: true,
              },
            },
          ],
        },
      },
    ],
  },
  videoUrl: {
    type: String,
    required: false,
  },
  creator: {
    type: String,
    required: true,
  },
});

export default mongoose.models?.Content || mongoose.model("Content", Content);
```

## packages/util/src/models/CourseEnrollment.ts

```
/**
 * packages/util/src/models/CourseEnrollment.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2024 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 16.05.2024
 *
 */

import mongoose from "mongoose";

const CourseEnrollmentModel = new mongoose.Schema({
  createdAt: {
    type: Date,
    required: true,
    default: Date.now,
  },
  user: {
    type: mongoose.Schema.Types.ObjectId,
    required: true,
    ref: "User",
  },
  course: {
    type: mongoose.Schema.Types.ObjectId,
    required: true,
    ref: "Course",
  },
});

export default mongoose.models?.CourseEnrollment ||
  mongoose.model("CourseEnrollment", CourseEnrollmentModel);
```

## packages/util/src/models/CourseModel.ts

```
/**
 * packages/util/src/models/CourseModel.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 30.10.2022
 *
 */
import mongoose from "mongoose";

const Course = new mongoose.Schema({
  name: {
    type: String,
    required: true,
  },
  friendlyName: {
    type: String,
    required: true,
    unique: true,
  },
  description: {
    type: String,
    required: true,
  },
  splashImage: {
```

```
    type: String,
    required: true,
  },
  createdAt: {
    type: Date,
    default: Date.now,
    immutable: true,
  },
  // PUBLIC (editor and admin only) | PRIVATE | ORG:ORG_ID (org owner only)
  available: {
    type: String,
    default: false,
    required: true,
  },
  contentPositions: {
    type: [String],
    required: true,
    default: false,
  },
  creator: {
    type: String,
    required: true,
  },
});

export default mongoose.models?.Course || mongoose.model("Course", Course);
```

## packages/util/src/models/CourseProgressModel.ts

```
/**
 * src/util/models/CourseProgressModel.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 02.07.2022
 *
 */

import mongoose from "mongoose";

const CourseProgress = new mongoose.Schema({
  user: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "User",
    required: true,
  },
  course: {
    type: String,
    required: true,
  },
  finishedModules: {
    type: [String],
    required: true,
    default: [],
  },
  finishedCourse: {
    type: Boolean,
    required: true,
    default: false,
  },
```

```
});

export default mongoose.models?.CourseProgress ||
  mongoose.model("CourseProgress", CourseProgress);
```

## packages/util/src/models/CourseSectionProgressModel.ts

```
/**
 * packages/util/src/models/CourseSectionProgressModel.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2024 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 16.05.2024
 *
 */

import mongoose from "mongoose";

const CourseSectionProgressModel = new mongoose.Schema({
  createdAt: {
    type: Date,
    required: true,
    default: Date.now,
  },
  user: {
    type: mongoose.Schema.Types.ObjectId,
    required: true,
    ref: "User",
  },
  course: {
    type: mongoose.Schema.Types.ObjectId,
    required: true,
    ref: "Course",
  },
  section: {
    type: mongoose.Schema.Types.ObjectId,
    required: true,
    ref: "Content",
  },
});

export default mongoose.models?.CourseSectionProgress ||
  mongoose.model("CourseSectionProgress", CourseSectionProgressModel);
```

## packages/util/src/models/CustomerChoiceModel.ts

```
/**
 * packages/util/src/models/CustomerChoiceModel.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 12.04.2023
 *
 */

import mongoose from "mongoose";

const CustomerChoice = new mongoose.Schema({
  name: {
    type: String,
```

```
    required: true,
  },
  position: {
    type: String,
    required: true,
  },
  company: {
    type: String,
    required: true,
  },
  content: {
    type: String,
    required: true,
  },
  image: {
    type: String,
    required: true,
  },
});

export default mongoose.models?.CustomerChoice ||
  mongoose.model("CustomerChoice", CustomerChoice);
```

## packages/util/src/models/EditorProjectFunctionModel.ts

```
/**
 * packages/util/src/models/EditorProjectPageModel.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 20.01.2023
 *
 */

import mongoose from "mongoose";

const EditorProjectFunction = new mongoose.Schema({
  project: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "EditorProject",
    required: true,
  },
  path: {
    type: String,
    required: true,
  },
  code: {
    type: String,
    required: true,
    default: "",
  }
});

export default mongoose.models?.EditorProjectFunction ||
  mongoose.model("EditorProjectFunction", EditorProjectFunction);
```

## packages/util/src/models/EditorProjectKVStorageModel.ts

```
/**
 * packages/util/src/models/EditorProjectKVStorageModel.ts
 *
```

```
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 19.02.2023
 *
 */

import mongoose from "mongoose";

const EditorProjectKVStorage = new mongoose.Schema({
  project: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "EditorProject",
    required: true
  },
  storage: { type: Object }
});

export default mongoose.models?.EditorProjectKVStorage ||
mongoose.model("EditorProjectKVStorage", EditorProjectKVStorage);
```

## packages/util/src/models/EditorProjectModel.ts

```
/**
 * packages/util/src/models/EditorProjectModel.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 05.01.2023
 *
 */

import mongoose from "mongoose";

const EditorProject = new mongoose.Schema({
  owner: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "User",
    required: true,
  },
  name: {
    type: String,
    required: true,
  },
  contributors: {
    type: [
      {
        user: {
          type: mongoose.Schema.Types.ObjectId,
          ref: "User",
          required: true,
        },
        permissions: {
          type: [String],
          required: true,
          /*
           * Permissions
           * READ; WRITE; DELETE; DUPLICATE; MANAGE-CONTRIBUTORS; RENAME; EXPORT
           */
          default: ["READ", "WRITE", "DUPLICATE"],
```

```
        },
      },
    ],
    required: true,
    default: [],
  },
  updated_at: {
    type: Date,
    default: Date.now,
  },
  created_at: {
    type: Date,
    default: Date.now,
    immutable: true,
  },
  discovery: {
    type: Boolean,
    default: false,
  },
  discoveryDescription: {
    type: String,
    default: "",
  },
  customDomain: {
    type: String,
    default: "",
    unique: true,
  },
  discoveryLikes: {
    type: [mongoose.Schema.Types.ObjectId],
    default: [],
    required: true,
  },
  discoveryComments: {
    type: [
      {
        username: {
          type: String,
          required: true,
        },
        comment: {
          type: String,
          required: true,
        },
        created_at: {
          type: Date,
          default: Date.now,
        },
      },
    ],
    default: [],
    required: true,
  },
});

export default mongoose.models?.EditorProject ||
  mongoose.model("EditorProject", EditorProject);
```

## packages/util/src/models/EditorProjectPageModel.ts

```
/**
```

```
 * packages/util/src/models/EditorProjectPageModel.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 20.01.2023
 *
 */

import mongoose from "mongoose";

const EditorProjectPage = new mongoose.Schema({
  project: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "EditorProject",
    required: true,
  },
  type: {
    type: String,
    required: true,
    // Types: HTML; InCode
    default: "html",
  },
  mainContent: {
    type: String,
    required: true,
    default: "",
  },
  resources: {
    type: Array,
    required: true,
    default: [],
  },
  url: {
    type: String,
    default: "",
    required: true,
  },
});

export default mongoose.models?.EditorProjectPage ||
  mongoose.model("EditorProjectPage", EditorProjectPage);
```

## packages/util/src/models/ErrorModel.ts

```
/**
 * packages/util/src/models/ErrorModel.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 26.04.2023
 *
 */

import mongoose from "mongoose";

const ErrorModel = new mongoose.Schema({
  name: {
    type: String,
    required: true,
```

```
  },
  message: {
    type: String,
    required: true,
  },
  stack: {
    type: String,
    required: true,
  },
  createdAt: {
    type: Date,
    default: Date.now,
    required: true,
    immutable: true,
  },
});

export default mongoose.models?.ErrorModel ||
  mongoose.model("ErrorModel", ErrorModel);
```

## packages/util/src/models/ForumChatModel.ts

```
/**
 * packages/util/src/models/ForumProfileModel.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 23.09.2022
 *
 */

import mongoose from "mongoose";

const ForumChat = new mongoose.Schema({
  users: {
    type: [String],
    required: true,
  },
  messages: {
    type: [
      {
        username: {
          type: String,
          required: true,
        },
        content: {
          type: String,
          required: true,
        },
        createdAt: {
          type: Date,
          default: Date.now,
          immutable: true,
        },
      },
    ],
  },
  createdAt: {
    type: Date,
    default: Date.now,
```

```
    immutable: true,
  },
});

export default mongoose.models?.ForumChat ||
  mongoose.model("ForumChat", ForumChat);
```

## packages/util/src/models/ForumPostModel.ts

```
/**
 * packages/util/src/models/ForumProfileModel.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 23.09.2022
 *
 */
import mongoose from "mongoose";

const ForumPost = new mongoose.Schema({
  user: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "User",
    required: true,
  },
  username: {
    type: String,
    required: true,
  },
  content: {
    type: String,
    required: true,
  },
  createdAt: {
    type: Date,
    default: Date.now,
    immutable: true,
  },
  likes: {
    type: [mongoose.Schema.Types.ObjectId],
    ref: "User",
    default: [],
    required: true,
  },
  parent: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "ForumPost",
    default: null,
  },
  embeds: {
    type: Array,
    default: [],
  },
});

export default mongoose.models?.ForumPost ||
  mongoose.model("ForumPost", ForumPost);
```

## packages/util/src/models/ForumProfileModel.ts

```
/**
```

```
 * packages/util/src/models/ForumProfileModel.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 23.09.2022
 *
 */

import mongoose from "mongoose";

const ForumProfile = new mongoose.Schema({
  user: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "User",
    required: true,
  },
  username: {
    type: String,
    required: true,
  },
  displayName: {
    type: String,
    required: true,
  },
  verified: {
    type: Boolean,
    default: false,
  },
  bio: {
    type: String,
    default: "",
  },
  website: {
    type: String,
    default: "",
  },
  location: {
    type: String,
    default: "",
  },
  createdAt: {
    type: Date,
    default: Date.now,
    immutable: true,
  },
  badges: {
    type: [String],
    default: [],
  },
  certificates: {
    type: [String],
    default: [],
  },
});

export default mongoose.models?.ForumProfile ||
  mongoose.model("ForumProfile", ForumProfile);
```

## packages/util/src/models/IDEProjectModel.ts

```
/**
 * packages/util/src/models/IDEProjectModel.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 06.07.2022
 *
 */

import mongoose from "mongoose";

const IDEProject = new mongoose.Schema({
  owner: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "User",
    required: true,
  },
  name: {
    type: String,
    required: true,
  },
  contributors: {
    type: [
      {
        user: {
          type: mongoose.Schema.Types.ObjectId,
          ref: "User",
          required: true,
        },
        permissions: {
          type: [String],
          required: true,
          /*
           * Permissions
           * READ; WRITE; DELETE; DUPLICATE; MANAGE-CONTRIBUTORS; RENAME; EXPORT
           */
          default: ["READ", "WRITE", "DUPLICATE"],
        },
      },
    ],
    required: true,
    default: [],
  },
  files: {
    type: [
      {
        path: {
          type: String,
          default: "/README.MD",
        },
        content: {
          type: String,
          default: "# Readme\nWillkommen im CodeUp Editor!",
        },
      },
    ],
    required: true,
    default: [{}],
  },
```

```
  updated_at: {
    type: Date,
    default: Date.now,
  },
  created_at: {
    type: Date,
    default: Date.now,
    immutable: true,
  },
  pinned: {
    type: [mongoose.Schema.Types.ObjectId],
    default: [],
  },
  published: {
    type: Boolean,
    default: false,
  },
  publishedCustomName: {
    type: String,
    default: "",
  },
  discovery: {
    type: Boolean,
    default: false
  },
  discoveryDescription: {
    type: String,
    default: ""
  },
});

export default mongoose.models?.IDEProject ||
  mongoose.model("IDEProject", IDEProject);
```

## packages/util/src/models/IDETemplateModel.ts

```
/**
 * packages/util/src/models/IDETemplateModel.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 24.07.2022
 *
 */
import mongoose from "mongoose";

const IDETemplate = new mongoose.Schema({
  owner: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "User",
    required: true,
  },
  authorDisplayName: {
    type: String,
    required: true,
  },
  displayName: {
    type: String,
    required: true,
  },
```

```
  project: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "IDEProject",
    required: true,
  },
  description: {
    type: String,
    required: true,
  },
  ratings: {
    type: [
      {
        user: {
          type: mongoose.Schema.Types.ObjectId,
          ref: "User",
          required: true,
        },
        rating: {
          type: Number,
          required: true,
        },
      },
    ],
    required: true,
  },
  uses: {
    type: Number,
    default: 0,
  },
});

export default mongoose.models?.IDETemplate ||
  mongoose.model("IDETemplate", IDETemplate);
```

## packages/util/src/models/NewsletterModel.ts

```
/**
 * packages/util/src/models/NewsletterModel.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 13.04.2023
 *
 */
import mongoose from "mongoose";

const Newsletter = new mongoose.Schema({
  user: {
    type: mongoose.Schema.Types.ObjectId,
    required: true,
    ref: "User",
  },
});

export default mongoose.models?.Newsletter ||
  mongoose.model("Newsletter", Newsletter);
```

## packages/util/src/models/ObtainedCertificateModel.ts

```
/**
 * packages/util/src/models/Certificate.ts
```

```
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 26.11.2022
 *
 */
import mongoose from "mongoose";

const ObtainedCertificate = new mongoose.Schema({
  course: {
    type: mongoose.Schema.Types.ObjectId,
    required: true,
    ref: "Course",
  },
  username: {
    type: String,
    required: true,
  },
  name: {
    type: String,
    required: true,
  },
  date: {
    type: Date,
    default: Date.now,
  },
  percentage: {
    type: Number,
    required: true,
  },
  screenshot: {
    type: String,
    required: false,
  },
});

export default mongoose.models?.ObtainedCertificate ||
  mongoose.model("ObtainedCertificate", ObtainedCertificate);
```

## packages/util/src/models/OrgBlogModel.ts

```
/**
 * packages/util/src/models/OrgBlogModel.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 19.03.2023
 *
 */
import mongoose from "mongoose";

const OrgBlogModel = new mongoose.Schema({
  org: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "Org",
    required: true,
  },
  createdAt: {
    type: Date,
```

```
    default: Date.now,
  },
  title: {
    type: String,
    required: true,
  },
  content: {
    type: String,
    required: true,
  },
});

export default mongoose.models?.OrgBlog ||
  mongoose.model("OrgBlog", OrgBlogModel);
```

## packages/util/src/models/OrgModel.ts

```
/**
 * packages/util/src/models/OrgModel.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 03.03.23
 *
 */
import mongoose from "mongoose";

const OrgModel = new mongoose.Schema({
  owner: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "User",
    required: true,
  },
  name: {
    type: String,
    required: true,
    unique: true,
  },
  members: {
    type: Array,
    required: true,
    default: [],
  },
  icon: {
    type: String,
    required: false,
    default: "/codeup.png",
  },
  subscription: {
    type: String,
    required: true,
    default: "FREE",
  },
  createdAt: {
    type: Date,
    required: true,
    default: Date.now,
    immutable: true,
  }
});
```

```
export default mongoose.models?.Org || mongoose.model("Org", OrgModel);
```

## packages/util/src/models/OrgProjectTemplateModel.ts

```
/**
 * packages/util/src/models/OrgProjectTemplateModel.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 18.03.2023
 *
 */
import mongoose from "mongoose";

const OrgTemplateModel = new mongoose.Schema({
  name: {
    type: String,
    required: true,
  },
  org: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "Org",
    required: true,
  },
  projectId: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "EditorProject",
    required: true,
  },
});

export default mongoose.models?.OrgTemplate ||
  mongoose.model("OrgTemplate", OrgTemplateModel);
```

## packages/util/src/models/ProjectFlowModel.ts

```
/**
 * packages/util/src/models/ProjectFlowModel.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2024 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 01.06.2024
 *
 */

import mongoose from "mongoose";

const ProjectFlowModel = new mongoose.Schema({
  createdAt: {
    type: Date,
    required: true,
    default: Date.now,
  },
  name: {
    type: String,
    required: true,
  },
  project: {
    type: mongoose.Schema.Types.ObjectId,
```

```
    required: true,
    ref: "Project",
  },
  saved: {
    type: "Mixed",
    required: true,
  },
});

export default mongoose.models?.ProjectFlow ||
  mongoose.model("ProjectFlow", ProjectFlowModel);
```

## packages/util/src/models/ProjectI18nModel.ts

```
/**
 * packages/util/src/models/ProjectI18nModel.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2024 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 02.06.2024
 *
 */

import mongoose from "mongoose";

const ProjectI18nModel = new mongoose.Schema({
  createdAt: {
    type: Date,
    required: true,
    default: Date.now,
  },
  project: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "Project",
    required: true,
  },
  defaultLanguage: {
    type: String,
    required: true,
  },
  languages: {
    type: [String],
    required: true,
  },
  translations: {
    type: Object,
    required: true,
  },
});

export default mongoose.models?.ProjectI18n ||
  mongoose.model("ProjectI18n", ProjectI18nModel);
```

## packages/util/src/models/ProjectIdeaModel.ts

```
/**
 * packages/util/src/models/ProjectIdeaModel.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
```

```
 * Created At: 03.12.2022
 *
 */

import mongoose from "mongoose";

const ProjectIdea = new mongoose.Schema({
  title: {
    type: String,
    required: true,
  },
  description: {
    type: String,
    required: true,
  },
  exampleDeployedURL: {
    type: String,
    required: false,
  },
  exampleSourceURL: {
    type: String,
    required: false,
  },
  level: {
    type: String,
    required: true,
  },
});

export default mongoose.models?.ProjectIdea ||
  mongoose.model("ProjectIdea", ProjectIdea);
```

## packages/util/src/models/ProjectIssueModel.ts

```
/**
 * packages/util/src/models/ProjectIssueModel.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2024 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 31.05.2024
 *
 */

import mongoose from "mongoose";

const ProjectIssueModel = new mongoose.Schema({
  createdAt: {
    type: Date,
    required: true,
    default: Date.now,
  },
  project: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "Project",
    required: true,
  },
  title: {
    type: String,
    required: true,
  },
```

```
  description: {
    type: String,
    required: true,
  },
  status: {
    type: String,
    required: true,
    default: "open",
  },
  priority: {
    type: String,
    required: true,
    default: "normal",
  },
});

export default mongoose.models?.ProjectIssue ||
  mongoose.model("ProjectIssue", ProjectIssueModel);
```

## packages/util/src/models/ProjectModel.ts

```
/**
 * packages/util/src/models/ProjectModel.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2024 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 20.05.2024
 *
 */

import mongoose from "mongoose";

const ProjectModel = new mongoose.Schema({
  createdAt: {
    type: Date,
    required: true,
    default: Date.now,
  },
  users: {
    type: [mongoose.Schema.Types.ObjectId],
    required: true,
    ref: "User",
  },
  name: {
    type: String,
    required: true,
  },
  projectType: {
    type: String,
    required: true,
    default: "static",
  },
  modifiers: {
    type: [String],
    required: true,
    default: [],
  },
});

export default mongoose.models?.Project ||
```

```
  mongoose.model("Project", ProjectModel);
```

## packages/util/src/models/ProjectUMLModel.ts

```
/**
 * packages/util/src/models/ProjectUMLModel.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2024 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 08.06.2024
 *
 */

import mongoose from "mongoose";

const ProjectUMLModel = new mongoose.Schema({
  createdAt: {
    type: Date,
    required: true,
    default: Date.now,
  },
  name: {
    type: String,
    required: true,
  },
  project: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "Project",
    required: true,
  },
  data: {
    type: String,
    required: true,
  },
  type: {
    type: String,
    required: true,
  },
});

export default mongoose.models?.ProjectUML ||
  mongoose.model("ProjectUML", ProjectUMLModel);
```

## packages/util/src/models/ProjectWikiPageModel.ts

```
/**
 * packages/util/src/models/ProjectWikiPageModel.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2024 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 31.05.2024
 *
 */

import mongoose from "mongoose";

const ProjectWikiPageModel = new mongoose.Schema({
  createdAt: {
    type: Date,
    required: true,
```

```
    default: Date.now,
  },
  project: {
    type: mongoose.Schema.Types.ObjectId,
    required: true,
    ref: "Project",
  },
  title: {
    type: String,
    required: true,
  },
  content: {
    type: String,
    required: true,
  },
});

export default mongoose.models?.ProjectWikiPage ||
  mongoose.model("ProjectWikiPage", ProjectWikiPageModel);
```

## packages/util/src/models/RoadmapModel.ts

```
/**
 * packages/util/src/models/RoadmapModel.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 07.04.2023
 *
 */

import mongoose from "mongoose";

const Roadmap = new mongoose.Schema({
  name: {
    type: String,
    required: true,
  },
  owner: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "User",
    required: true,
  },
  projectIdea: {
    type: String,
    ref: "ProjectIdea",
  },
  linkedCourses: {
    type: [mongoose.Schema.Types.ObjectId],
    ref: "Course",
  },
  linkedTodos: {
    type: [mongoose.Schema.Types.ObjectId],
    ref: "Todo",
  },
  linkedFlows: {
    type: [mongoose.Schema.Types.ObjectId],
    ref: "Flow",
  },
  linkedProject: {
```

```
    type: mongoose.Schema.Types.ObjectId,
    ref: "Project",
  },
});

export default mongoose.models?.Roadmap || mongoose.model("Roadmap", Roadmap);
```

## packages/util/src/models/ShortURLModel.ts

```
/**
 * packages/util/src/models/ShortURLModel.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2023 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 14.04.2023
 *
 */
import mongoose from "mongoose";

const ShortURLModel = new mongoose.Schema({
  createdAt: {
    type: Date,
    default: Date.now,
    immutable: true,
  },
  destination: {
    type: String,
    required: true,
  },
  short: {
    type: String,
    required: true,
    unique: true,
  },
});

export default mongoose.models?.ShortURL ||
  mongoose.model("ShortURL", ShortURLModel);
```

## packages/util/src/models/TextContentModel.ts

```
/**
 * packages/util/src/models/TextContentModel.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 30.10.2022
 *
 */
import mongoose from "mongoose";

const TextContent = new mongoose.Schema({
  name: {
    type: String,
    required: true,
  },
  course: {
    type: String,
    required: true,
  },
```

```
  friendlyName: {
    type: String,
    required: true,
    unique: true,
  },
  content: {
    type: String,
    required: true,
  },
  createdAt: {
    type: Date,
    default: Date.now,
    immutable: true,
  },
  quiz: {
    type: [
      {
        question: {
          type: String,
          required: true,
        },
        answers: {
          type: [
            {
              answer: {
                type: String,
                required: true,
              },
              correct: {
                type: Boolean,
                required: true,
              },
            },
          ],
        },
      },
    ],
    required: true,
  },
});

export default mongoose.models?.TextContent ||
  mongoose.model("TextContent", TextContent);
```

## packages/util/src/models/TodoListModel.ts

```
/**
 * packages/util/src/models/TodoListModel.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 09.07.2022
 *
 */
import mongoose from "mongoose";

const TodoSchema = new mongoose.Schema({
  owner: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "User",
```

```
    required: true,
  },
  name: {
    type: String,
    required: true,
  },
  tasks: {
    type: [
      {
        finished: {
          type: Boolean,
          required: true,
          default: false,
        },
        name: {
          type: String,
          required: true,
        },
        description: {
          type: String,
          required: false,
        },
        uuid: {
          type: String,
          required: true,
        },
      },
    ],
    required: true,
    default: [],
  },
});

export default mongoose.models?.Todo || mongoose.model("Todo", TodoSchema);
```

## packages/util/src/models/ToDoProjectModel.ts

```
/**
 * packages/util/src/models/ToDoProjectModel.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 04.12.2022
 *
 */

import mongoose from "mongoose";

const ToDoProject = new mongoose.Schema({
  owner: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "User",
    required: true,
  },
  name: {
    type: String,
    required: true,
  },
  saved: {
    type: "Mixed",
```

```
    required: true,
  },
});

export default mongoose.models?.ToDoProject ||
  mongoose.model("ToDoProject", ToDoProject);
```

## packages/util/src/models/UserModel.ts

```
/**
 * src/util/models/UserModel.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 05.06.2022
 *
 */

import mongoose from "mongoose";

const UserSchema = new mongoose.Schema({
  username: {
    type: String,
    required: true,
    unique: true,
    maxlength: 30,
    minlength: 4,
  },
  email: {
    type: String,
    required: true,
    unique: true,
  },
  password: {
    type: String,
    required: true,
  },
  firstName: {
    type: String,
    required: true,
  },
  lastName: {
    type: String,
    required: true,
  },
  createdAt: {
    type: Date,
    default: Date.now,
    required: true,
    immutable: true,
  },
  twoFactorEnabled: {
    type: Boolean,
    default: false,
    required: true,
  },
  twoFactorCode: {
    type: String,
    default: "",
  },
```

```
  active: {
    type: Boolean,
    default: false,
    required: true,
  },
  activationToken: {
    type: String,
    default: "",
  },
  deletionToken: {
    type: String,
    default: "",
  },
  resetToken: {
    type: String,
    default: "",
  },
  emailChangeCode: {
    type: String,
    default: "",
  },
  newEmail: {
    type: String,
    default: "",
  },
  profilePicture: {
    type: String,
    default: "",
  },
  sso: {
    type: String,
    default: "no-sso",
  },
  role: {
    type: String,
    default: "user",
  },
  twoFactorSecret: {
    type: String,
    default: "",
  },
  subscription: {
    type: String,
    default: "FREE",
  },
  stripeSubscriptionId: {
    type: "String",
    default: "",
  },
  stripeCustomerId: {
    type: "String",
    default: "",
  }
});

export default mongoose.models?.User || mongoose.model("User", UserSchema);
```

## packages/util/src/models/VideoModel.ts

```
/**
 * packages/util/src/models/VideoModel.ts
```

```
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 30.10.2022
 *
 */
import mongoose from "mongoose";

const Video = new mongoose.Schema({
  name: {
    type: String,
    required: true,
  },
  course: {
    type: String,
    required: true,
  },
  friendlyName: {
    type: String,
    required: true,
    unique: true,
  },
  transcript: {
    type: String,
    required: true,
  },
  youtube: {
    type: String,
    required: true,
    unique: true,
  },
  createdAt: {
    type: Date,
    default: Date.now,
    immutable: true,
  },
  quiz: {
    type: [
      {
        question: {
          type: String,
          required: true,
        },
        answers: {
          type: [
            {
              answer: {
                type: String,
                required: true,
              },
              correct: {
                type: Boolean,
                required: true,
              },
            },
          ],
        },
      },
    ],
```

```
    required: true,
  },
});

export default mongoose.models?.Video || mongoose.model("Video", Video);
```

## packages/util/src/monaco/uml.ts

```
/**
 * packages/util/src/monaco/uml.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2024 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 08.06.2024
 *
 */

export const uml_language: any = {
  keywords: [
    "start",
    "stop",
    "end",
    "participant",
    "actor",
    "usecase",
    "class",
    "interface",
    "abstract",
    "enum",
    "package",
    "node",
    "database",
    "cloud",
    "artifact",
    "folder",
    "frame",
    "rectangle",
    "agent",
    "boundary",
    "control",
    "entity",
    "collections",
    "queue",
    "hexagon",
    "archimate",
    "diamond",
    "component",
    "use",
    "namespace",
    "state",
    "object",
    "as",
    "title",
    "alt",
    "else",
    "opt",
    "loop",
    "par",
    "break",
    "critical",
```

```
    "group",
    "note",
    "rnote",
    "hnote",
    "legend",
    "skinparam",
    "left",
    "right",
    "top",
    "bottom",
    "of",
    "on",
    "link",
    "over",
    "endnote",
    "noteend",
    "footer",
    "header",
    "center",
    "rotate",
    "color",
    "background",
    "line",
    "over",
    "include",
    "!define",
    "!undef",
    "!include",
    "!include_many",
    "!include_once",
    "!unlinked",
    "hide",
    "show",
    "skin",
    "return",
    "is",
    "also",
    "label",
    "activate",
    "deactivate",
    "destroy",
    "create",
    "autonumber",
    "fork",
    "again",
    "split",
    "merge",
    "detach",
    "newpage",
    "box",
    "linetype",
    "linestyle",
    "linecolor",
  ],
  operators: ["=", ":", "(", ")", "[", "]", "{", "}", "<", ">"],
  symbols: /[=:\(\)\[\]\{\}<>]/,

  tokenizer: {
    root: [
      // Comments
```

```
      [/\/'.*$/, "comment"],
      [/\/'[^\/]*'\/$/, "comment"],
      [/'[^']*$/, "comment"],
      [/'[^']*'/, "comment"],

      // Keywords
      [

/\b(?:start|stop|end|participant|actor|usecase|class|interface|abstract|enum|package|node|database|cloud|artifact|folder|frame|rectangle|agent|boundary|control|entity|collections|queue|hexagon|archimate|diamond|component|use|namespace|state|object|as|title|alt|else|opt|loop|par|break|critical|group|note|rnote|hnote|legend|skinparam|left|right|top|bottom|of|on|link|over|end\s*note|note\s*end|footer|header|center|rotate|color|background|line|over|include|!define|!undef|!include|!include_many|!include_once|!unlinked|hide|show|skin|return|is|also|label|activate|deactivate|destroy|create|autonumber|else|opt|loop|alt|par|break|critical|group|end|fork|again|split|merge|detach|alt|legend|newpage|box|end|title|participant|actor|boundary|control|entity|database|collections|queue|delay|destroy|footer|header|center|rotate|color|background|linetype|linestyle|linecolor|line:|legend|note|hnote|rnote|line|note|legend|box|end|end|note|note\s*end)\b/,
        "keyword",
      ],

      // Preprocessor
      [/(?:[@]startuml|[@]enduml|!startsub|!endsub)/, "keyword.preprocessor"],

      // Strings
      [/"[^"]*"/, "string"],

      // Delimiters and operators
      [/[{}[\]()<>:=]/, "delimiter"],

      // Numbers
      [/\b\d+\b/, "number"],

      // Identifiers
      [/\b[a-zA-Z_][a-zA-Z0-9_]*\b/, "identifier"],
    ],
  },
};

export const uml_language_config: any = {
  comments: {
    lineComment: "'",
    blockComment: ["'/", "/'"],
  },
  brackets: [
    ["{", "}"],
    ["[", "]"],
    ["(", ")"],
  ],
  autoClosingPairs: [
    { open: "{", close: "}" },
    { open: "[", close: "]" },
    { open: "(", close: ")" },
    { open: "'", close: "'" },
  ],
  surroundingPairs: [
    { open: "{", close: "}" },
    { open: "[", close: "]" },
    { open: "(", close: ")" },
    { open: "'", close: "'" },
```

```
  ],
};
```

## packages/util/src/mongo.ts

```
/**
 * @author Ben Siebert <ben@mctzock.de>
 * @copyright (c) 2018-2021 Ben Siebert. All rights reserved.
 */

import mongoose from "mongoose";

const MONGO_URI = process.env.MONGO_URI;

if (!MONGO_URI) {
  throw new Error("Please define the MONGO_URI variable in your environment");
}

// @ts-ignore
let cached = global.mongoose;

if (!cached) {
  // @ts-ignore
  cached = global.mongoose = { conn: null, promise: null };
}

async function mongoConnect() {
  if (cached.conn) {
    return cached.conn;
  }

  if (!cached.promise) {
    cached.promise = mongoose.connect(MONGO_URI as string).then((mongoose) => {
      return mongoose;
    });
  }
  cached.conn = await cached.promise;
  return cached.conn;
}

export default mongoConnect;
```

## packages/util/src/Networking.ts

```
/**
 * packages/util/src/Networking.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 24.07.2022
 *
 */

export default class Networking {
  public static download(
    filename: string,
    text: string,
    mimeType = "text/plain"
  ) {
    const element = document.createElement("a");
    element.setAttribute(
```

```
      "href",
      "data:" + mimeType + ";charset=utf-8," + encodeURIComponent(text)
    );
    element.setAttribute("download", filename);

    element.style.display = "none";
    document.body.appendChild(element);

    element.click();

    document.body.removeChild(element);
  }

  public static downloadCustom(href: string, filename: string) {
    const element = document.createElement("a");
    element.setAttribute("href", href);
    element.setAttribute("download", filename);

    element.style.display = "none";
    document.body.appendChild(element);

    element.click();

    document.body.removeChild(element);
  }
}
```

## packages/util/src/pathsToStructure.ts

```
/**
 * packages/util/src/pathsToStructure.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 04.08.2022
 *
 */

export default function pathsToStructure(
  paths: string[],
  contents: { [path: string]: string }
) {
  let result: any = [];
  let level = { result };

  for (const path of paths) {
    path.split("/").reduce((r: any, name: any, i: any, a: any) => {
      if (!r[name]) {
        r[name] = { result: [] };
        r.result.push({
          name,
          children: r[name].result,
          fullPath: path,
          contents: contents[path],
        });
      }
      return r[name];
    }, level);
  }
```

```
  result[0].name = "/root";

  return result;
}
```

## packages/util/src/sendMail.ts

```
/**
 * src/util/sendMail.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 18.06.2022
 *
 */
import * as nodemailer from "nodemailer";
import generateMail from "./generateMail";

export default async function sendMail(
  to: string,
  subject: string,
  content: string,
  firstName: string
) {
  if (process.env.SMTP_DISABLE) {
    console.log("SMTP is disabled");
    return;
  }

  let transport = nodemailer.createTransport({
    // @ts-ignore
    host: process.env.SMTP_HOST,
    port: process.env.SMTP_PORT,
    secure: process.env.SMTP_SECURE === "true",
    auth: {
      user: process.env.SMTP_USER,
      pass: process.env.SMTP_PASSWORD,
    },
    tls: {
      rejectUnauthorized: false,
    },
  });

  const tmp = generateMail(firstName, content);

  await transport.sendMail({
    from: "Ben von CodeUp <" + process.env.SMTP_USER + ">",
    to: to,
    replyTo: process.env.SMTP_USER,
    subject: subject,
    text: tmp.text,
    html: tmp.html,
  });
}

export async function sendMailV2(options: {
  to: string;
  subject: string;
  html: string;
  text: string;
```

```
}) {
  if (process.env.SMTP_DISABLE) {
    console.log("SMTP is disabled");
    return;
  }

  let transport = nodemailer.createTransport({
    // @ts-ignore
    host: process.env.SMTP_HOST,
    port: process.env.SMTP_PORT,
    secure: process.env.SMTP_SECURE === "true",
    auth: {
      user: process.env.SMTP_USER,
      pass: process.env.SMTP_PASSWORD,
    },
    from: process.env.SMTP_FROM,
    tls: {
      rejectUnauthorized: false,
    },
  });

  await transport.sendMail({
    from: "Ben von CodeUp <" + process.env.SMTP_FROM + ">",
    to: options.to,
    replyTo: process.env.SMTP_USER,
    subject: options.subject,
    text: options.text,
    html: options.html,
  });
}
```

## packages/util/src/types/CourseProgress.ts

```
/**
 * src/util/types/CourseProgress.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 02.07.2022
 *
 */

export interface CourseProgress {
  user: string;
  course: string;
  finishedModules: string[];
  finishedCourse: boolean;
}
```

## packages/util/src/updateForumBadges.ts

```
/**
 * packages/util/src/updateForumBadges.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 31.12.2022
 *
 */
import ForumPostModel from "./models/ForumPostModel";
```

```
import ForumProfileModel from "./models/ForumProfileModel";

export default async function updateForumBadges(username: any) {
  const postsOfUser = await ForumPostModel.find({ username: username });
  const profile = await ForumProfileModel.findOne({ username: username });

  const badges: {
    [key: string]: {
      name: string;
      posts: number;
    };
  } = {
    newbie: {
      name: "Newbie",
      posts: 10,
    },
    beginner: {
      name: "Beginner",
      posts: 25,
    },
    expert: {
      name: "Expert",
      posts: 50,
    },
    master: {
      name: "Master",
      posts: 100,
    },
  };

  if (profile) {
    for (const badge in badges) {
      if (postsOfUser.length >= badges[badge].posts) {
        if (!profile.badges.includes(badges[badge].name.toLowerCase())) {
          profile.badges.push(badges[badge].name.toLowerCase());
        }
      }
    }
    await profile.save();
  }
}
```

## packages/util/src/userworkflow/redirect.ts

```
/**
 * src/util/userworkflow/redirect.ts
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 06.06.2022
 *
 */

import { decode } from "jsonwebtoken";

export default function redirect(options: {
  redirectToLogin?: boolean;
  redirectTo?: string;
  allowAnonymous?: boolean;
  forceAnonymous?: boolean;
```

```
}) {
  if (localStorage.getItem("token") !== null) {
    const token = localStorage.getItem("token") as string;
    try {
      const decoded = decode(token);
      if (decoded === null) {
        throw new Error("Token could not be decoded");
      }
      if (options.forceAnonymous) {
        window.location.href = options.redirectTo as string | "/dashboard";
      }
    } catch (e) {
      localStorage.removeItem("token");
      document.cookie = "jwt=;path=/";
      if (!options.allowAnonymous) {
        if (options.redirectToLogin) {
          window.location.href = `/#login`;
        }
      }
    }
  } else {
    if (!options.allowAnonymous) {
      if (options.redirectToLogin) {
        window.location.href = `/#login`;
      }
    }
  }
}
```

## packages/util/tsconfig.json

```
{
  "extends": "tsconfig/react-library.json",
  "exclude": ["dist", "build", "node_modules"],
  "compilerOptions": {
    "module": "CommonJS",
    "moduleResolution": "node",
    "isolatedModules": true,
    "lib": ["DOM"],
    "jsx": "react",
    "rootDir": "src",
    "outDir": "dist"
  }
}
```

## pullImages.sh

```
# Serverless Runtimes
docker pull node:latest
docker pull python:bullseye
docker pull golang:latest
docker pull mctzock/codeup-serverless-runtime:latest
docker pull mctzock/codeup-tsx-rce:latest
docker pull mctzock/codeup-java-rce:latest

docker image tag node:latest rce-node:codeup
docker image tag python:bullseye rce-python:codeup
docker image tag golang:latest rce-golang:codeup
docker image tag mctzock/codeup-serverless-runtime:latest rce-sf-runtime:codeup
docker image tag mctzock/codeup-tsx-rce:latest rce-tsx:codeup
docker image tag mctzock/codeup-java-rce:latest rce-java:codeup
```

## qodana.yaml

```
version: "1.0"
linter: jetbrains/qodana-js:2022.1-eap
bootstrap: yarn install
```

## README.md

```
# CodeUp Reborn

## Lokal ausführen

### Anforderungen

1. MongoDB Server: [Download](https://mongodb.com)
1. NodeJS >16: [Download](https://nodejs.org)
1. NPM: Ist in NodeJS enthalten.
1. Yarn: `npm i -g yarn`
1. Turborepo: `npm i -g turbo`
1. Git: [Download](https://git-scm.org)

### Schritt 1

CodeUp Repository von GitHub klonen und ins Verzeichnis wechseln.

```bash
  git clone https://github.com/MCTzOCK/CodeUp && cd CodeUp
```

### Schritt 2

Bibliotheken installieren

```bash
  yarn install
```

### Schritt 3

Bennene die Datei `apps/web/.env.example` zu `apps/web/.env` um.
Öffne anschließend die Datei mit deinem bevorzugten Texteditor
und passe die Umgebungsvariablen an.

### Schritt 4

Wenn du CodeUp im Entwickler-Modus (mit Hot-Module-Reloading) starten möchtest, dann führe folgenden Befehl aus:

```bash
  turbo run dev --filter=web
```

Wenn du CodeUp im Produktivmodus starten willst, musst du zunächst
das "Production Next.JS Build" von CodeUp erzeugen:

```bash
  turbo run build --filter=web
```

Anschließend kannst du CodeUp mit folgendem Befehl starten:

```bash
```
```

```
yarn --cwd apps/web ts-start-prod
```

## Lizenz

CodeUp ist unter der [GNU GENERAL PUBLIC LICENSE v3.0](https://choosealicense.com/licenses/gpl-3.0/) lizensiert.

## rest-api.md

# REST API

The following document tells you how to use the REST API of CodeUp.
The {base-url} marker represents "https://codeup.space" in production
and "http://localhost:3000" in development.

## Accounts

### Register

`POST /api/account/register`

Post Data (body)

| Property | Type | Description | Required |
|---|---|---|---|
| email | `string` | The E-Mail of the new user | true |
| firstName | `string` | The first name | true |
| lastName | `string` | The last name | true |
| password | `string` | The password | true |
| username | `string` | The username of the user | true |

#### Responses

- Status: 400; error: "Username must be between 4 and 20 characters long and can only contain letters, numbers, and the underscore"
  - Invalid username
- Status: 400; error: "Invalid fields"
  - Some required fields are not submitted or the types do not match.
- Status: 500, error: any
  - Any Server related error, most likely a MongoDB error that the `username` or `email` is not valid.
- Status: 200, success: true, message: "User created - Please check your email for an activation link"
  - Everything worked just fine

### Login

`POST /api/account/login`

Post Data (body)

| Property | Type | Description | Required |
|---|---|---|---|
| email | `string` | The E-Mail of the user | true |
| password | `string` | The password | true |
| code | `string` | The 2FA Code | false |

#### Responses

- Status: 400; error: "Missing required fields"
  - Some required fields are not submitted or the types do not match.

- Status: 400; error: "User not found"
  - The account does not exist
- Status: 400; error: "Wrong password"
  - The password does not match to the accounts' password.
- Status: 400; error: "Wrong code"
  - The 2FA Code is not valid
- Status: 200; error: null; _2fa: true
  - The user received an e-mail containing the 2FA Code
- Status: 200, success: true, token: JWT
  - Everything worked just fine and you got an access token

### Reset

`POST /api/account/reset`

Post Data (body)

| Property | Type | Description | Required |
|---|---|---|---|
| email | `string` | The E-Mail of the user | true |

#### Responses

- Status: 400; error: "Missing parameters"
  - Some required fields are not submitted or the types do not match.
- Status: 404; error: "User not found"
  - The account does not exist
- Status: 400; error: "Reset E-Mail already sent"
  - The E-Mail to reset the password was already sent
- Status: 200, success: true, message: "Reset E-Mail sent"
  - Everything worked just fine and the user got an E-Mail

### Reset-Confirm

`POST /api/account/reset-confirm`

Post Data (body)

| Property | Type | Description | Required |
|---|---|---|---|
| token | `string` | The Reset Token | true |
| password | `string` | The new Password | true |

#### Responses

- Status: 400; error: "Missing required fields"
  - Some required fields are not submitted or the types do not match.
- Status: 400; error: "Invalid Token"
  - The Reset Token is not valid
- Status: 400; error: "Password must be at least 8 characters long and contain at least one uppercase letter, one
  lowercase letter, one number and one special character"
  - The password is not safe enough
- Status: 200, success: true, message: "Password changed successfully"
  - Everything worked just fine

### Update

`POST /api/account/update`

Post Data (body)

| Property | Type | Description | Required |
|---|---|---|---|
| token | `string` | A JWT Access Token of the user | true |
| keyName | `firstName`, `lastName`, `email`, `2fa` | The key to be modified | true |
| keyValue | `string` | The new value | true |

#### Responses

- Status: 400; error: "Missing parameters"
  - Some required fields are not submitted or the types do not match.
- Status: 404; error: "User not found"
  - The account does not exist
- Status: 400; error: "Invalid key name"
  - Invalid `keyName` value
- Status: 400; error: "Invalid email"
  - Invalid email in `keyValue`
- Status: 400; error: "Email change code already sent"
  - The E-Mail last changed to was not confirmed yet.
- Status: 400; error: "Email already in use"
  - The E-Mail provided is already used by another account
- Status: 401; error: "Invalid token"
  - The provided `token` is not valid.
- Status: 200, success: true, message: "Please check your email for a verification code"
  - Everything worked just fine and the user got an E-Mail
- Status: 200, success: true, newToken: JWT
  - Everything worked just fine and a new JWT was generated

### Check-Reset-Token

`GET /api/account/check-reset-token`

Post Data (query -> ?key=value&key2=value2...)

| Property | Type | Description | Required |
|---|---|---|---|
| token | `string` | The Reset Token | true |

#### Responses

- Status: 400; error: "Missing token"
  - The Reset Token was not provided
- Status: 400; error: "Invalid Token"
  - The Reset Token is not valid
- Status: 200; valid: true
  - The Reset Token is valid

### Action

`GET /api/account/action`

***CAN NOT BE USED PROGRAMMATICALLY***

## SECURITY.md

# Sicherheitsrichtlinie

## Unterstützte Versionen

Die EINZIGE unterstützte Version ist zu JEDER Zeit die aktuelle Version des `develop`-Zweigs. Jeder andere Versionszweig (`vX.Y.Z`) dient nur als Referenz.

## Sicherheitslücke Melden

Um eine Sicherheitslücke zu melden, kann das Kontakt-Formular unter https://codeup.space/codeup/contact verwendet werden. Damit die Kontaktanfrage als gemeldete Sicherheitslücke gilt, muss die Kategorie "Fehler melden" ausgewählt sein.

## sloc.js

```
/**
 * @author Ben Siebert <ben@mctzock.de>
 * @copyright (c) 2018-2022 Ben Siebert. All rights reserved.
 */

const old_console_log = console.log;

if (process.argv.includes("--codeup-stats-only-lines-raw")) {
  console.log = (...args) => {};
  console.warn = (...args) => {};
  console.error = (...args) => {};
}

const fs = require("fs");
const path = require("path");
const byteSize = require("byte-size");

let paths = getAllPaths(".", []);

const disallowed = [
  ".turbo",
  ".git",
  ".idea",
  ".ico",
  ".jpg",
  ".png",
  ".docusaurus",
  ".ttf",
  ".afdesign",
  "LICENSE",
  ".lock",
  "package-lock.json",
  ".log",
  "node_modules",
  ".next",
  "dist",
  "build",
  ".codeup_app_stats.json",
  "privacy.tsx",
  "imprint.tsx",
  ".react-email",
];

// filter disallowed files
paths = paths.filter((p) => {
  let allowed = true;
  disallowed.forEach((d) => {
    if (p.toLowerCase().includes(d.toLowerCase())) {
      allowed = false;
    }
```

```
  });
  return allowed;
});

let lines = 0;
let size = 0;

let filesWithLines = [];

for (let i = 0; i < paths.length; i++) {
  let fLines = getFileLines(paths[i]);
  filesWithLines.push({
    path: paths[i],
    lines: fLines,
  });
  lines += fLines;
}

if (process.argv.includes("-v")) {
  console.log(JSON.stringify(paths, null, 2));
}

console.warn("\u001b[31mProject Statistics\u001b[0m");
console.table({
  "Total Lines": lines,
  "Total Files": paths.length,
  "Total Size": byteSize(size).value + " " + byteSize(size).unit,
  "Total Characters": size,
});

console.log("\n");

console.warn("\u001b[31mBigest Files\u001b[0m");
filesWithLines.sort((a, b) => b.lines - a.lines);
filesWithLines.slice(0, 10).forEach((f, i) => {
  console.log(i + 1 + ". " + f.path + ": " + f.lines);
});

if (process.argv.includes("--list-all")) {
  filesWithLines.slice(10, filesWithLines.length).forEach((f, i) => {
    console.log(i + 11 + ". " + f.path + ": " + f.lines);
  });
}

if (process.argv.includes("--codeup-stats-only-lines-raw")) {
  old_console_log(lines);
}

function getAllPaths(dirPath, arrayOfFiles) {
  files = fs.readdirSync(dirPath);

  arrayOfFiles = arrayOfFiles || [];

  files.forEach(function (file) {
    if (fs.statSync(dirPath + "/" + file).isDirectory()) {
      arrayOfFiles = getAllPaths(dirPath + "/" + file, arrayOfFiles);
    } else {
      arrayOfFiles.push(path.join(__dirname, dirPath, "/", file));
    }
  });
```

```
  return arrayOfFiles;
}

function getFileLines(p) {
  let file = fs.readFileSync(p, "utf8");
  size += file.toString().length;
  return file.toString().split("\n").length;
}
```

## toDocx.js

```
/**
 * /toLatex.js
 *
 * Author: Ben Siebert <hello@ben-siebert.de>
 * Copyright: Copyright (c) 2018-2024 Ben Siebert. All rights reserved.
 * License: Project License
 * Created At: 12.06.2024
 *
 */

const fs = require("fs");
const path = require("path");
const byteSize = require("byte-size");
const docx = require("docx");

const ignore = [
  ".turbo",
  ".git",
  ".idea",
  ".ico",
  ".jpg",
  ".jpeg",
  ".png",
  ".docusaurus",
  ".ttf",
  ".zip",
  ".afdesign",
  "LICENSE",
  ".lock",
  "package-lock.json",
  ".log",
  "node_modules",
  ".next",
  "dist",
  "build",
  ".codeup_app_stats.json",
  ".react-email",
  ".env",
  ".env.web",
  "pdf",
  "docs",
  "svg",
  "codeup.tex",
  ".woff2",
  "toMarkdown.js",
];

let paths = getAllPaths(".", []);
```

```
paths = paths.filter((p) => {
  let allowed = true;
  ignore.forEach((d) => {
    if (p.toLowerCase().includes(d.toLowerCase())) {
      allowed = false;
    }
  });
  return allowed;
});

const sections = [
  {
    children: [
      new docx.Paragraph({
        children: [
          new docx.TextRun({
            text: "Inhaltsverzeichnis",
            font: "Arial",
          }),
        ],
        heading: docx.HeadingLevel.TITLE,
      }),
      new docx.TableOfContents("Inhaltsverzeichnis", {
        hyperlink: true,
      }),
    ],
  },
];

for (let file of paths) {
  file = file.replace(path.join(__dirname, "/"), "");
  file = file.replace(/\\/g, "/");
  console.log(`Reading ${file}`);
  let content = fs.readFileSync(file, "utf8");
  /*content = removeEmojis(content);
  content = content.replace("[", "\\verb|[|");
  content = content.replace("]", "\\verb|]|");*/

  let trLines = content.split("\n").map((line) => {
    return new docx.Paragraph({
      children: [
        new docx.TextRun({
          text: line,
          font: "Arial",
        }),
      ],
    });
  });

  const section = {
    properties: {
      type: docx.SectionType.CONTINUOUS,
    },
    children: [
      new docx.Paragraph({
        children: [
          new docx.TextRun({
            text: file,
            font: "Arial",
          }),
```

```
        ],
        heading: docx.HeadingLevel.HEADING_2,
      }),
      ...trLines,
    ],
  };

  sections.push(section);
}

const doc = new docx.Document({
  features: {
    updateFields: true,
  },
  creator: "Ben Siebert",
  description: "CodeUp Quellcode",
  title: "CodeUp Quellcode",
  sections,
});

docx.Packer.toBuffer(doc).then((buffer) => {
  fs.writeFileSync(path.join(__dirname, "codeup.docx"), buffer);
});
console.log("Done!");

function getAllPaths(dirPath, arrayOfFiles) {
  files = fs.readdirSync(dirPath);

  arrayOfFiles = arrayOfFiles || [];

  files.forEach(function (file) {
    if (fs.statSync(dirPath + "/" + file).isDirectory()) {
      arrayOfFiles = getAllPaths(dirPath + "/" + file, arrayOfFiles);
    } else {
      arrayOfFiles.push(path.join(__dirname, dirPath, "/", file));
    }
  });

  return arrayOfFiles;
}
```

## turbo.json

```
{
  "tasks": {
    "build": {
      "dependsOn": ["^build"],
      "outputs": ["dist/**", ".next/**"]
    },
    "lint": {
      "outputs": []
    },
    "dev": {
      "cache": false
    }
  }
}
```